PostHog handbook

Art, brand, and merch requests

`contents/handbook/brand/art-requests.md`

> 🎨 **Need artwork or merch?** Please request it using the request templates. Do not request art or merch over Slack or email.

All artwork and merch requests are handled by , , and on the team.

They can help you with things like:

- Custom visuals for paid ad campaigns
- Blog and social media artwork
- New themed hedgehogs
- Custom CTAs and banners
- Branded merch
- Animated UI elements

They get a lot of work requests, so they use two separate project boards to organize work – one for merch and one for other art projects. This reflects that merch projects often have much longer timelines and need to be handled differently.

Whenever you want to request a new merch design or other artwork, you should use the relevant design request templates in the posthog.com repo – one template for merch, one for other art requests. Each template automatically assigns work to the correct project board.

Art board automations

The Art & Brand Planning board uses GitHub Actions to keep work moving:

- **Reminders** — A daily job (9 AM UTC) posts one-time comments on issues that have been stuck in...
  - **Feedback/Review** for 10+ days: asks if any feedback is needed to move the task forward.
  - **No Status** for 7+ days: asks someone to pick it up or assign it to a column.
- **Status changes** — When an issue's Status is changed on the board:
  - **Moved to "Done"** → the issue is automatically closed (as completed).
  - **Moved to "Assigned: Daniel", "Assigned: Lottie", or "Assigned: Heidi"** → other default assignees are removed so only the assigned person is on the issue. Internal requests (from the design team) keep all assignees.
  - These changes do not impact the "Assigned: Cleo" column, as Cleo has a different workload.
- Workflows run under the **Art Board Bot** GitHub App and live in `.github/workflows/` (`art-board-reminder.yml`, `art-board-reminders.yml`, `art-board-status-change.yml`).

To establish a clear connection between the task and the working file, designers will create a frame containing a link to the task. They should then add a link to that frame within the task for easy reference.

Lottie and Daniel usually ask for two weeks minimum notice, but can often work faster on things if needed. If your request is genuinely urgent, please share your request issue in #team-marketing channel and mention Lottie, Daniel, and/or Cory.

Hedgehog library

For team members we keep all our currently approved hedgehogs in this Figma file. This enables us to look through the library of approved hogs, and to export them at required sizes without relying on the design team.

Here's how:

1. Open the Figma file. You can manually browse, or use `Cmd + F` to search based on keywords such as 'happy', 'sad', or 'will smith'.
2. Select the hog you want. If needed, adjust the size using the 'Frame' menu in the top of the right-hand sidebar.
3. At the bottom of the right-hand sidebar, select the file type you need in the 'Export' menu, choose `@2x`, then select 'Export [filename]' to download the image.

If you can't find a suitable hog, you can request one from the design team.

> Non-team members can find some of the most-used hogs to download on our press page.

Designing posthog.com

`contents/handbook/brand/designing-posthog-website.md`

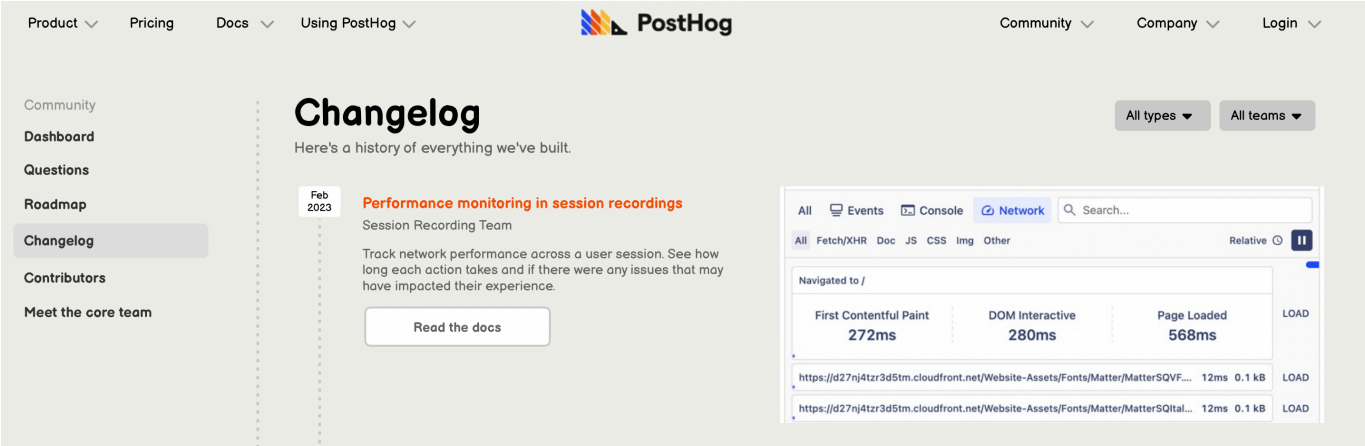
The is responsible for everything you see on posthog.com. We treat our website & docs as a product, which means we're constantly iterating on it and improving it.

Because our website has a well-defined aesthetic, we often skip the hifi design process and jump straight from wireframes into code. Having a designer who can code means we can reach the desired level of polish without *always* having to produce hifi designs, thus leading to huge time savings.

Step 1: Wireframes [Balsamiq]

We often produce hi-fidelity wireframes because this allows us to closely envision a design which in turn helps us skip the hi-fi Figma process.

Note: Balsamiq uses its own Comic Sans-style font. Don't get hung up on this!



Step 2: Hi-fi designs [Figma]

Designs are scattered across a variety of unorganized Figma files, but here's some of the most recent iteration.

If there are multiple iterations of a single page, we typically work left to right.

Any mocks in pages that appear to be faded out are considered *old* and *out of date* and can be ignored, as there is a better replacement nearby. (We sometimes want to keep them around for easy reference (and to leave a comment trail), but they're easily identifiable because their artboards are set to 50% opacity.)

Even with this loosely-documented process, things move quickly and we don't always follow this process. If you're looking for something in particular, it's worth pinging in the #team-brand channel.

We're also working on creating a singular place for product screenshots, which are exported in light and dark mode using html.to.design.

Email comms

contents/handbook/brand/email-comms.md

Our email communications can be broadly divided into broadcasts (one-off emails to specific lists, like a newsletter), campaigns (repeatable workflows which users move through dynamically), and API triggered emails (self-explanatory).

This page doesn't deal with our Product for Engineers newsletter, which is sent through Substack and managed by the Content & Docs team.

Email broadcasts

We regularly send three types of email broadcasts.

1. Changelog, a product announcement email sent bi-weekly via Customer.io.
2. PostHog for Startups, an email to users of our startup program. Sent monthly, via Customer.io.
3. Launch emails, which are 'random acts of marketing', but fairly consistent with every product team shipping alpha, beta and GA features each quarter.

Occasionally we send other ad-hoc email broadcasts for specific activities such as outages, reminders, announcements, or deprecations.

Changelog

The changelog email is part of the new release process and is used for product announcements.

Every month, we use Customer.io to share a broadcast which summarizes the highlights from the weekly changelog over the last month. We use our discretion to choose which updates to highlight, usually showcasing three or four of the most impactful changes. We usually reserve the top spot for making users aware of new beta features. A test is shared with the team before we send to users.

We tag these emails as `Product updates` in Customer.io, so users can manage their subscriptions. In order to maintain high deliverability, we target users in the `Recently Engaged (4 months)` segment which includes everyone who has logged in the last quarter.

PostHog for Startups

Each month, we send an email to users in our PostHog for Startup program. A test is shared with the team before we send to users. This email is targeted to users in the following segments, all at once: `PostHog for Startups (Old)`, `Users in the YC program, old and new`, `Old startup teams (Backfill only)`, and `PostHog for Startups and YC (new)`.

The email is usually comprised of three sections, which inform users of new guides which are relevant to startup use-cases, new betas which are available for them to try, and a spotlight written about a new org in the program. We end by asking for feedback.

We categorize these emails as `Actually useful marketing emails` in Customer.io, so users can unsubscribe if they wish. This email usually comes directly from Joe.

Launch emails

Most product and feature launch emails come from the Product Marketer who sent them -- but sometimes campaigns trigger from others, such as billing@posthog.com.

The exceptions and other solutions are:

- Sending emails from hey@posthog.com -- this is what we usually do for BIG sends, because it would overwhelm the sender's inbox with 'out-of-office' auto-replies.
- Sending emails from beta-feedback@posthog.com -- this is a Google group tied to the automation in #posthog-feedback by a Slack bot. Any responses to this address get posted in that channel and anyone can reply to them using the info in there.
- Sending emails from a specific person, but setting the reply-to address as one of the above. This is not common, but it's there if you want to use it.

> We specifically do not want emails we think people will reply to going into hey@posthog.com because it is sporadically monitored at best, and hard to collaborate through.

When we ask users to share feedback through email, it should either link to beta-feedback@posthog.com, the support modal, or to ourselves personally. Never hey@posthog.com.

Doing this lets us filter out the noise for everyone else while still giving good visibility on meaningful feedback internally. If a user sends you feedback, you should share that with the relevant product team or in https://posthog.slack.com/archives/C011L071P8U

> When adding yourself as a send-from address in Customer.io, be sure to edit the display name to '[your name] **from PostHog**'.

Other broadcasts

Any ad-hoc customer email broadcasts are owned by the , and are usually sent via Customer.io. These can include product updates, outage alerts, or other PostHog news if needed.

These emails are usually tagged as `Service updates` in Customer.io when they include important account or product information. These emails are given a dedicated unsubscribe option in the footer, making it clear that we do not recommend users unsubscribe to these emails.

Important service updates are the *only* type of email we may send to unsubscribed users, and only if we feel it is warranted to do so.

> `Service updates` emails are often part of an engineering incident. We handle comms for those too.

Whenever we need to send an email broadcast like this we begin by creating an issue in the Meta repo, unless it involves discussion of personal information - in which case it is discussed in Company Internal. This enables us to summarize information and seek approval from teams while also keeping our work open source, and without requiring everyone to log in to Customer.io. Issues are closed when an email is sent.

If you'd like to work with Marketing on an email activity, please begin by opening an issue in the `meta` repo.

Email campaigns

We maintain many email campaigns to help users get the most out of the product. The most developed and documented of these are our four onboarding campaigns.

Onboarding emails

Generally, when we talk about onboarding emails we refer specifically to the flow for PostHog Cloud sign-ups, but there are also other flows in use for other occasions.

PostHog Cloud onboarding emails

The latest revision is Onboarding 8. You can read about old revisions on the blog.

The onboarding flow regularly changes as we test new ideas. Any changes to it are, as with all other email campaigns, documented in the Meta repo.

We aim for all content in this flow to be relevant and helpful to users, without being salesy. All emails come directly from Joe and he triages replies on a daily basis, answering or redirecting as needed. The campaign is triggered when a user signs up for the first time and has a goal of users achieving `billing product activated` within 7 days of opening any email in the flow.

We tag all these email flows as `onboarding` in Customer.io and categorize them as `Welcome emails` so that users can easily manage their preferences.

Self-hosted and open source onboarding emails

We sunset our paid self-hosted product a long time ago, but some users still try to use the legacy version. For this reason we run a dedicated self-hosted onboarding campaign which includes three emails sent over a course of six weeks. These emails come from the `hey@posthog.com` email address.

The goal of this flow is to set expectations for what the self-hosted experience is like and to encourage users to move to the PostHog Cloud product for a better experience.

Our open source onboarding email is essentially identical to the self-hosted onboarding flow, but excludes information about the sunsetting of the self-hosted product.

Beta onboarding emails

When a user opts in to a beta via the feature preview menu we enter them into an email flow designed to help us collect feedback from users.

This flow currently comprises a single, personal email from either Joe or the team lead working on the beta feature. This email is sent one week after the user joins the beta and features tailored content based on which beta the user joined.

When responses come in, Joe generally triages replies and directs feedback to the relevant team, as well as rewarding users with merch as thanks for their feedback.

> **Launching a beta?** It helps to let the Brand team know in the team Slack. The team can then add your beta to the beta onboarding flow, and plan ahead for marketing announcements as needed.

Onboarding - new hires

This is an internal email flow for new hires, which triggers whenever a new user signs up with a PostHog email address. We currently exclude most old-time hires from this flow, to avoid blocking their inboxes.

This campaign runs for a new hire's first 30 days and sends them 7 emails with information to help them get set up at PostHog.

There's no way to unsubscribe from these emails, but if you're triggering them with test accounts then let the Brand team know and they can exclude you from the campaign.

Other email campaigns

We run a series of other small campaigns with smaller volumes. These include:

- **The replay recommender** is a campaign which encourages users who have ingested a large number of unwatched replays to watch some of the recordings.
- **Teams upsells & cancellations** are two separate campaigns. The first triggers when a team invites their sixth and ninth team member, suggesting the Teams add-on to boost collaboration. The second triggers when the add-on is disabled, comes from Zach, and requests feedback.
- **G2 review Requester** is described in Testimonials & G2
- **Startup & YC updates** is a series of campaigns for the startups and YC programs. These broadly notify users when they join the program, and use 50%, 75% and 100% of their available credit.

API triggered emails

We maintain a series of API triggered emails by working with the . These are found in Customer.io's transactional tool and broadly encompass billing and security updates, such as an upcoming bill or a change to 2FA settings. These emails are triggered by API in order to keep them highly relevant and with high deliverability.

Transactional emails feature Liquid code to help personalize their content. All transactional emails should contain Liquid in the main body content to clearly indicate to the user which project or organization the email is regarding, with suitable fallbacks. For example:

```
We turned the free allowance for {{ trigger.product_name | default: "a product"
}} on {% if trigger.team_name %}{{ trigger.team_name }}{% else %}your account{%
endif %} off and on again, giving you another month of free usage.
```

In-app comms

contents/handbook/brand/in-app.md

> These are instructions for internal in-app comms tools at PostHog. To do in-app comms of your own, check out surveys.

Occasionally, we use in-app messages to tell users about certain things. We recognize that in-app messages can be intrusive and we want to avoid spamming our users with too many of them, too frequently. For that reason, we're judicious about the way in which we use them.

We currently don't have a separate system for tracking in-app messages, so Brand currently owns the channel and is responsible for ensuring that messages aren't used excessively.

Types of in-app message

Currently, there are three ways in which we can send in-app messages.

- **Notification bar:** A message displayed across the top of the page, activated using the Notification Bar app.
- **In-app prompt:** A customizable pop-up which can be targeted to certain URLs and made to appear in the center of the page, or anchored in the corner. Activated using the `prompt` feature flag. More info.
- **Notifications:** A notification which is pushed into the navigation bar, as a number on the bell icon. Activated using the `changelog-notification` feature flag.

How we use in-app messages

We use each of the three channels above for different purposes, guided by the needs of a message and the level of intrustion.

- The notification bar is only used for messages which must be urgently communicated to all users, such as messages about service disruption.
- In-app prompts can be used for a wide variety of purposes, including promotion of new features. However, users will only see one in-app prompt per day and should be targeted to appear only on relevant pages and to relevant users. In-app prompts shouldn't be used to message all users at once, or to direct users to another part of the app.
- Notifications can be used for a wide variety of purposes and are minimally intrusive. We regularly use notifications to promote new features via the PostHog changelog.

Creating new in-app prompts

In-app prompts are intrusive to users, but can be used for a wide variety of reasons. Therefore, if you create one we ask that you...

- Add the `Marketing` tag to the feature flag used to power your in-app prompt

This will enable others in the team to more easily keep track of what in-app messages are being shown, and what their content is. As a reminder:

- Users will only see one in-app prompt per day, at most
- In-app prompts should be used only on relevant pages and towards targeted cohorts

If you have any questions, please ask in #ask-posthog-anything on Slack.

Brand overview

contents/handbook/brand/overview.md

The focuses on design and the PostHog website. This includes writing the copy for pages outside of the docs and handbook. This is considered the source of truth for how we talk about our products.

We work closely with the and .

Partners

contents/handbook/brand/partners.md

We're frequently contacted about revenue-sharing partnerships, or individuals and agencies that want to be listed as official partners. Also, technology integrations!

If someone contacts you about partnering with PostHog, refer them to our partnerships page and ask them to complete the survey there. This will directly alert relevant teams internally.

We recommend users who need implementation help explore our existing resources, purchase time with the onboarding team or contact us. If we can help, we will!

Helpful resources for users

Users who contact us about wanting support from a partner often want particular types of help. We've curated some resources below which we can give them so they can self-serve where possible.

Migration help

If a customer contacts us about migrating data into PostHog we should first refer them to the Sales & CS Team, who will triage them. We also have guides to help teams migrate data on their own.

- Migrating from Amplitude to PostHog
- Migrating from Mixpanel to PostHog
- Migrating from Heap to PostHog
- Migrating from LaunchDarkly to PostHog
- Migrating from Statsig to PostHog
- Migrating from a self-hosted deployment to PostHog Cloud
- Syncing other platforms to our data warehouse

Implementation help

Sometimes teams want help or advice on their event taxonomy, or creating specific insights. Users who look like they have the potential to pay >$20k should generally be referred to the Sales & CS team, otherwise they should go through the regular support flow. We also have a wide variety of dashboard templates and tutorials to help teams get started.

If the user is very new then we usually strongly advise enabling auto-capture and creating an AARRR dashboard as a first step.

Self-hosted help

We no longer provide support for self-hosted deployments. If users contact us for help with self-hosted deployments then we refer them to our legacy docs and strongly recommend they migrate to PostHog Cloud.

Getting more help (from someone else)

If users need more help than we can reasonably provide, they may ask for external support or partners. **We do not have any official partners** and users should know that any suggestions we may make are not vetted or accreddited in any way.

That said, some users have found success working with the following external partners:

- Taleno.Digital (US)
- Mentat Analytics (US)
- Marketing Engineers (NL)

Sometimes teams are able to find success by posting on platforms such as Upwork.

Design philosophy

contents/handbook/brand/philosophy.md

> Looking for our brand style guide? Look no further.

Different ~~by~~ *with* design

Design at PostHog works differently than most companies. We fundamentally believe that we can differentiate ourselves with design – by thinking outside the box and pushing boundaries. This means we're not structured like a typical design org.

How our customers interact with product analytics (and other tools) has largely remained unchanged since these tools were created over the past couple decades. There's nothing *wrong* with how they work currently, but people were also very happy with riding horses until cars were widely adopted.

Does that mean we're going to change how everything works? Not necessarily. It just means we have the freedom to try different things and see what sticks.

Our philosophy started with our website

Our first design hire was our graphic designer, Lottie. It's not every day you see a graphic designer in the first 5 hires! This was the result of a belief by our founders that having a bold, yet relatable brand would be a differentiator.

After a year of constant iteration on our website and docs over 2021-2022, we landed in a place where our website is now a reference for many other startups who are looking to do something innovative with their websites.

We're now extending this thinking into our product.

Press

contents/handbook/brand/press.md

Press enquiries

Any press-related enquiries should be directed to press@posthog.com - this includes any emails you receive personally. Only Joe, James, Tim or Charles should be talking to the press on PostHog's behalf.

With the exception of occasional major press releases (see below), PR is purely a reactive activity at PostHog. We do not invest in proactive PR yet, as we believe other channels are a higher priority.

Managing press releases

From time to time, we may have significant company news that we want to release via the press, in addition to our usual channels. This is usually for significant company milestones such as funding rounds.

We have a simple process to ensure that any press releases go smoothly.

First steps

- [ ] Write up objectives and comms strategy - what is the purpose of the press release? What key message(s) are we trying to get across?
- [ ] Set an approximate target date

Two weeks before release

- [ ] Confirm key messages and write first draft press release
- [ ] Finalize target date
- [ ] Pitch and secure a media exclusive - our investors can help with this
- [ ] Secure approval for any third party involvement, e.g. quotes we want to use

We currently prefer working with a single media partner on an exclusive basis, as we believe a single, high-quality story is more impactful than taking a broad approach, given our current early stage.

One week before release

- [ ] Finalize press release and share with exclusive media partner
- [ ] Any media prep if interviews have been scheduled

On the day of release

- [ ] *Wait for the media partner's story to go live first!* Check it carefully and ask for any errors to be amended before proceeding with the below...

- [ ] Push out the press release via BusinessWire
- [ ] Submit via YC's social media request from
- [ ] James to post on his personal LinkedIn (and tag all relevant people)
- [ ] Post in our PostHog Users Slack
- [ ] Post in YC Slack
- [ ] Write post on our blog about the news
- [ ] Post on PostHog Twitter (and tag all relevant people)
- [ ] Share links to all of the above to the PostHog team so they can share
- [ ] Update any relevant online company profiles; Crunchbase, Pitchbook, Glassdoor.

Press release template

Include media and quotes from James, Tim or influential people.

```
# Headline

News

## About PostHog

PostHog is an open source developer platform. PostHog enables software teams to understand user behavior – auto-capturing events, performing product analytics and dashboarding, enabling video replays, and rolling out new features behind feature flags, all based on their single open source platform. The product’s open source approach enables companies to self-host, removing the need to send data externally.

Founded in 2020 by James Hawkins and Tim Glaser, PostHog was a member of Y Combinator’s Winter 2020 batch, and has subsequent raised $12m in funding from GV, Y Combinator and notable angel investors including Jason Warner (CTO, GitHub), Solomon Hykes (Founder, Docker).

## About Y Combinator Continuity Fund

YC Continuity is an investment fund dedicated to supporting founders as they scale their companies. Our primary goal is to support YC alumni companies by investing in their subsequent funding rounds, though we occasionally invest in non-YC companies as well.

Like YC’s early-stage partners, the entire YC Continuity team has strong operating experience. We work to create opportunities for founders to continue their personal growth and scale their companies successfully.
```

```
We also run the YC Growth Program, which brings together founder-CEOs who are
leading rapidly growing companies.
```

Product design process

contents/handbook/brand/process.md

No product design within small teams

We encourage engineers to act like feature owners, carrying a project from ideation to completion. We maintain a design system in Storybook, so engineers can build high-quality features independently, as much as possible.

Because engineers choose their sprint tasks near the beginning of a sprint (and product doesn't plan tasks *for* engineers in advance), our process doesn't allow for us to have a product manager and a designer to work closely together before a task gets selected by an engineer.

In our process of short, 2-week sprints with no pre-planning, design would become a blocker to an engineer quickly iterating on a feature. Thus, engineers don't get support from product designers. Product designers should deliver high quality components. The product teams should have people in them that can ship good-enough quality interfaces using those components. If that's not true, we should hire or move people around.

Learn more about how we decide this in our guide to working with product designers, for engineers.

Requesting artwork and brand materials.

Need some custom artwork? Read the art and branding request guidelines.

Portfolio

You can find PostHog's design portfolio on Dribbble... or just have a look around!

Startups & Y Combinator

`contents/handbook/brand/startups.md`

> **Want to apply for our startups program?** Sign-up here, or apply on Bookface if you're in Y Combinator.

We run two special programs for early-stage teams. The primary place for discussing both programs is the #project-startups-and-yc channel in Slack.

Feature	Startups	Y Combinator
Eligibility	<2 years old, <$5M raised, not acquired	Must be in YC, <$25m raised
Credit	$50,000 for 12 months	$50k per year, whilst eligible
Can use credit for add-ons?	⚠️ Yes, but cannot use credit for BAA in Boost add-on	✅ Yes, and can use credit for BAA in Boost add-on
Founder merch	Welcome pack (max 1)	Different welcome pack (max 4)
Community	—	Tim's Whatsapp, priority support
Apply via...	Startup page	Secret YC page

PostHog for Startups

Any company that is <2 years old and has raised less than $5M in funding is eligible to apply and claim the following:

- $50,000 in PostHog credits (valid for 12 months)
- One unique welcome pack for founders
- Partner benefits with Speakeasy, Incident.io, and Chroma
- A monthly newsletter for founders

> ❗ Credits **cannot** be used toward a BAA under the Boost plan.

> ⭐ **Small open source projects** without corporate backing and less than $200k annual revenue can contact support to have the 12-month credit expiry waived.

All applications are **automatically approved**, then **manually reviewed** for eligibility.

We track all PostHog for Startups applications in this Zapier table and this Zap.

PostHog for Y Combinator

This program is similar to our startup program but has some key differences for YC teams. Teams can be in any YC batch, with any amount of funding raised, and can claim the following:

- 50,000 *per year* – *they only need to register once and it will renew automatically while they're eligible* (<25m raised)
- If they previously registered for the old deal and it expired, they need to re-register
- Up to 4 unique founder merch packs (different from the startup program)
- Access to HogPatch for the duration of their time in the batch
- Partner benefits with Speakeasy, Incident.io and Chroma

You can find the copy for the latest deal on Bookface in this doc. To post updates, you need to ask James or Tim to do it.

> This deal is not available to YC alumni, who started another company - if they're eligible, they can apply for PostHog for Startups instead

> ✅ Credits **can** be used to claim a BAA under the Boost plan.

YC teams must apply via our secret YC page, where we ask for a screenshot from Bookface to prove their eligibility.

We track all PostHog for YC applications in this Zapier table.

What happens after companies apply?

1. **Application** A company signs up to PostHog, adds billing details, and applies via the startup form.
2. **Credit** If they meet the basic criteria, we automatically apply the correct amount of Stripe credit.
3. **Welcome + merch** Shortly after, they receive an automated email) from , in which we
    - Confirm their acceptance, welcome them and explain perks
    - Provide unique code(s) to claim founder kit(s) from the merch store (orders are fulfilled by Micromerch, merch questions can go in the `#merch` Slack channel)
4. **Milestones** When teams reach 50%, 75%, or 100% of their credit usage — or when credits expire — they receive milestone emails. These come from Customer.io and are managed by .
5. **Post-credit** Once credit is fully used or expired, teams are moved to a standard paid plan automatically. We automatically email users to let them know and offer a one-time $500 credit bonus to help soften the transition.

Reviewing applications

Applications are automatically enriched with Clearbit and Clay, then approved. We then manually review all emails to ensure eligibility. If there's a mismatch (e.g. on founding date or funding raised), we'll email the founder. If we don't hear back in a week or confirm ineligibility, we remove the credits.

Merch

All merch is fulfilled through the PostHog store by Micromerch.

- Founders receive unique codes via email to claim merch.
- They can request up to 4 merch packs for co-founders during signup.
- If a team has more than 4 founders, they can submit a support ticket to request more.
- Additional merch is occasionally granted at our discretion.

Issues? Reach out in #merch Slack. Founders can also email merch@posthog.com.

Monthly Newsletter

We send a short, founder-focused newsletter once per month to all program participants. This is handled as a Customer.io broadcast using a prebuilt template.

Credit usage

Credits can be used for almost all PostHog products and add-ons, including platform packages.

- **Startups**: ❌ Cannot use credits toward a BAA due to legal risk.
- **YC teams**: ✅ Can use credits for a BAA under the Boost plan.

Credits are valid are not transferable, and don't carry over or convert to cash. They are valid for 12 months and that timer begins at application. Once expired or fully used, teams are moved to standard billing.

Partners

We currently partner with:

- **Incident.io** — $1,500 off a teams plan
- **Speakeasy** — 50% off for 6 months
- **Chroma** - $5,000 of credit

Discount codes are sent in the welcome email after signup.

If users run into issues with redemption, we can help liaise — though all offers are ultimately at partner discretion.

Contacts:

- Incident.io: Zain Mobarik
- Speakeasy: Nolan Di Mare Sullivan
- Chroma: Philip Thomas

We previously offered DigitalOcean credits ($25k) and a Mintlify partnership, but these were retired in Q2 2025.

Program extensions

We don’t usually extend credits — the 12-month window is intended to be firm and fair. However, we’re open to requests in exceptional cases.

Founders must clearly explain why they couldn’t use the credit in time and provide evidence of recent progress or changes. Requests are reviewed manually by the Customer Success team.

Reporting

We have a dashboard for this.

Troubleshooting

If the Slack invite isn't sent or you discover founders did not receive it, you can manually invite users to the #posthog-founders-club channel. Make sure to select that they are "An external organization" when prompted right after adding their email address. A Slack admin will need to approve them before they're fully added to the channel.

If they did not receive an automated coupon to order the YC Kit from the merch store, you can generate a new coupon code manually in the Shopify admin view. The easiest way to do that is to duplicate an existing coupon, regenerate the coupon code, and save it. You'll have to repeat the process for every founder.

Credentials for Zapier, Shoify, etc. are available in the shared 1Password account.

Style guide

contents/handbook/brand/style-guide.md

This guide explains how the PostHog brand should appear across:

- the website
- the product UI
- documentation
- blog content
- marketing
- external assets

Brand != marketing

It's the sum of how people experience PostHog – from the homepage to documentation to support conversations.

Our goal is simple: **earn the trust of developers.**

Developers tend to distrust marketing. They prefer tools that feel human, honest, and thoughtfully built, not overly polished corporate brands.

Because of this, PostHog deliberately avoids sounding or looking like typical B2B SaaS companies.

Brand is everyone's job

Every interaction contributes to the brand.

Two ideas shape how we build and present PostHog.

Yes and...

We expand ideas instead of shutting them down. This mindset encourages creativity and experimentation.

It'd be better if we built it ourselves

The best products are built by people who care deeply about what they create.

In practice: everyone who ships something contributes to the brand.

Brand personality

PostHog should feel	Avoid being
Opinionated	Corporate
Human	Generic
Slightly weird	Overly polished
Thoughtful	Forced or cheesy
Direct	
Honest	

Developers connect with products that feel authentic, not brands trying to sound impressive.

Voice and tone

Write the way you would explain something to a smart friend.

- Be clear and simple
- Avoid jargon
- Avoid buzzwords
- Avoid fluff
- Be conversational
- Be honest

Humor is welcome, but it should never feel forced.

The dating profile test

Most SaaS companies write like they're submitting a résumé. Safe. Formal. Generic.

PostHog writes more like a dating profile: authentic, memorable, slightly weird, showing personality.

Users don't connect with product specs. They connect with people and ideas.

Design philosophy

PostHog design focuses on thoughtfulness and clarity, not looking luxurious.

Taste > polish

Design should feel crafted, not corporate. A thoughtful design builds trust.

Care about the details

Small details communicate quality. Examples: typography, spacing, illustration, layout, visual balance.

Users may not consciously notice them, but they still feel them.

Be intentional

Every design element should serve a purpose. Design should help:

- explain something
- guide attention
- improve readability
- add personality

Avoid decoration without meaning.

Visual identity

The PostHog visual style is intentionally distinctive. It should feel: handcrafted, playful, slightly weird, thoughtful, recognizable.

If something could exist on any other SaaS website, it probably isn't PostHog enough.

Core visual elements

Hedgehogs

Hedgehogs are a core brand element (though not every asset needs one).

Guidelines:

- Bold monoline
- No highlights
- Two tone shadow (one at 100% opacity to be used sparingly, one at 24% for creating depth, shadows always to be `#000000` black)
- Playful personality

The hedgehog is a creative vehicle for translating our personality into something expressive and alive.

Illustration

Illustrations help explain ideas and add personality. They should:

- Support the content
- Tell a story
- Feel hand-crafted

- Remain simple

Avoid:

- Stock illustrations
- Trendy SaaS artwork
- Overly complex drawings
- Boring or obvious designs

Simple is usually better.

Typography

Primary fonts:

- **IBM Plex Sans** – main typography
- **Squeak** – expressive marketing headlines
- **Loud Noises** – used for quotes in hedgehog artwork

Typography should prioritize readability, hierarchy, and clarity.

Color

Color should guide attention rather than dominate the page.

General approach:

- Solid backgrounds
- Limited palette
- Illustrations more colorful than the layout

Avoid:

- Gradients everywhere
- Too many colors
- Visual noise

PostHog vs. typical SaaS

	Average SaaS	**PostHog**
Headline	Buzzwords	Clear and direct
Visual style	Gradients and abstract shapes	Custom illustrations
Tone	Formal	Conversational

	Average SaaS	PostHog
Design goal	Look professional	Look intentional
Brand	Generic	Distinctive

Common mistakes

Generic SaaS design – Gradients, blobs, and stock illustrations make designs forgettable.

Decoration over clarity – Design should support the content, not distract from it.

Forced humor – Humor should feel natural.

Overly polished designs – Perfect designs can feel corporate.

Copying competitors – PostHog aims to be distinctive, not trendy.

Design checklist

Before publishing something, ask:

- Does this feel like PostHog?
- If you remove the logo, will it still feel on brand?
- Is the message clear?
- Is the design intentional?
- Does it feel human?
- Would someone enjoy looking at this?

If the answer to most of these is yes, you're probably on the right track.

The goal isn't to look expensive. The goal is to make people think: "Someone clearly cared about making this."

Testimonials and G2

contents/handbook/brand/testimonials.md

Social reviews on G2

We collect reviews from users on G2, both to act as social proof and to collect feedback on our product. After a process of trialling incentives, messaging and processes throughout 2022 we have established that:

- Reviews are best sought from users who have had a meaningful experience with the product (see below)
- Direct gift card incentives work better than other incentives, including charitable donations
- Batching reviews into monthly sends is imprecise and a non-trivial amount of work

As such, we have automated our review request process using Customer.io.

The automation currently invites users to leave an honest review in exchange for a $25 gift card, if they match the following criteria.

- User has completed the `insight analyzed` event at least 3 times in the last 30 days OR
- User has completed the `recording analyzed` event at least 3 times in the last 30 days OR
- User has completed the `feature flag created` event at least 1 time in the last 30 days OR
- User has completed the `experiment launched` event at least 1 time in the last 30 days AND
- User has a valid email address and is in the `Valid Email Address` segment AND
- User has not previously been asked to review PostHog and is not in the `Historic G2 Requests` segment

This process is handled in Customer.io using the `G2 Review Requests` segment and the `G2 Review Requester` campaign workflow. Users are only asked to review PostHog once, with a 2-day delay after the targeting confirms a match. This is important so we can avoid bombarding users with emails and do not nag users for reviews after the initial request.

More information about the G2 review process is available in the initial G2 automation RFC.

New reviews are automatically collected for team members in the internal #posthogfeedback Slack channel.

Testimonials

We speak to our users regularly and are often fortunate enough that they say nice things about our product or our way of working. Other times users talk about us in public, such as on social media or on review platforms and forums.

Not all of the feedback we receive can be used publicly. We don't assume that comments from product feedback calls can be used without explicit approval, for example, though approved customer stories, public reviews and social media comments certainly can.

If feedback can be used publicly then we collect it here, so that we can use it elsewhere to enhance our website or docs.

Adding company-wide tools and vendors

contents/handbook/company/adding-tools.md

What is it?

In the software section of our spending money page we say:

> There needs to be a very significant upside to introducing a new piece of software to outweigh its cost.

This is our mechanism for making decisions where we need to assess the cost of introducing a new piece of software.

It is inspired by this post on "fad resilience" from Slack. We want to be able to introduce new tools and services, without introducing overlapping tools and unnecessary complexity.

What makes us fad resilient is that you are free (and encouraged) to try new things. But by introducing new things, you become responsible for rolling them out. And for replacing anything they make obsolete.

What is it *not*?

This doesn't apply to making "cheap decisions". A cheap decision is one that can be easily completed or reversed, or one that only affects your work not other people's. For those types of decisions you should continue to follow the guidance in the the software section of our spending money page. This is about the adoption of new company-wide tools, or implementation of vendors that are going to be used in the PostHog product.

How does it work?

If you find yourself saying something like:

- "we should use Notion, not Google Docs"
- or "(Haskell|Rust|Chicken) would be a better programming language for us"

Then you need to do the following:

1. Try the tool in a low-risk context

Use the tool in a context where it is easily replaced and does not involve sensitive data. If you have doubts about what information can be shared at this stage, check with #legal first. Similar to a spike.

The goal is to:

- Check whether the tool works as well as you expect

- Understand the consequences of introducing it
- Learn what data the tool may have access to
- Give others a way to see it in action

2. Open an pull request in Company Internal

At the same time open an issue describing why we should adopt the tool. Anyone proposing a new vendor should think about the impact on the *whole* company, not just their team or use case.

You should carefully be thinking about and your proposal should consider the types of things described below:

What to think about?

Problem and motivation

- Why should we introduce this tool now? What problem does it solve?
- How large is the benefit vs. the status quo? Is this solving a real issue or just something interesting to try?
- What existing tools or processes would it replace?
- Could this be solved using an existing tool or by building something directly in PostHog?

Trial/Proof of concept

- Have you tested the tool in a small, reversible context (spike or sandbox)?
- Can it be evaluated without sending real data?
- What did you learn from the trial?

Data exposure and privacy

- What type of data would be sent to the tool/vendor and does the benefit justify that risk?

  From least to most sensitive:

  - General data – publicly accessible information
  - Business data – internal PostHog data without customer data
  - Customer data – customer PII (name, email, address, IP addresses, etc.)
  - Customer's customers' data – end-user PII

Also consider:

- Where will the data be stored or processed? (significant preference toward EU/US as these jurisdictions are lower risk, well vetted, and have robust privacy frameworks)
- Can we avoid sending customer or end-user PII?
- Can data be aggregated, redacted or irreversibly anonymized before leaving our systems?

Vendor due diligence

- Where is the company that provides the proposed tool headquartered and where do they operate?
- Who are their customers? How long have they been around?
- Do they demonstrate a credible security posture (SOC2, GDPR, HIPAA, etc.)?
- Are they a well-established tool in the industry, or something experimental and less well known?

Alternatives and competition

- Why this tool instead of competitors?
- What other credible options exist and how do they compare (cost, security, reputation, risk)?
- What are other companies in our space using? Checking their list of subprocessors is a good place to start.

Internal impact

- Have other engineers been consulted about technical impact or prior experience?
- Are relevant teams supportive of introducing this tool?
- Has security/infra reviewed the vendor and given their thoughts on their security posture?
- Has legal chimed in and given their thoughts on risks? Is this tool going to qualify as a subprocessor such that customers will need to be aware we're sending them data?
- Would using this tool impact how sales pitches the PostHog product to potential customers?
- Does this change anything about how we need to communicate with existing customers (marketing/support)?

Customer defensibility

- If a customer asked why we use this tool/vendor and send data to them, could we clearly and transparently justify the decision?
- Would customers that fit our ideal customer profile view this as a standard and responsible choice?
- How would enterprise-level customers react to this decision if we were selling PostHog to them?

These are guidelines, not a rigid checklist. The goal is for everyone to be thinking about the overall impact of introducing a new tool, and to allow for a holistic review of the risks against the benefits.

Many proposals will not make it past this stage – that's good. We don't want a stack that changes constantly, but we also don't want one that never improves.

After a decision is made: Review process

Once a decision has been made to adopt a tool/vendor, the person proposing the tool is responsible for coordinating the next steps.

1. Finalize business terms

Work with the vendor to negotiate the commercial and business terms, such as:

- Cost

- Number of licenses or usage terms and limits
- Contract length
- Any implementation or onboarding details

Once the business terms are mostly settled, the vendor's documents will need to go through legal review before anything is signed.

Typically, these includes:

- **Master Services Agreement** – the primary contract governing the relationship.
- **Data Processing Agreement** – required if the vendor processes personal data.
- **Security/compliance documentation** – e.g. SOC 2, ISO certifications, or similar.

As soon as it looks like we intend to move forward with the vendor, post in #legal, and give a heads-up that:

- A decision has been made to use the vendor.
- Business terms are being negotiated.
- Contract documents will be shared for review shortly.
- As much context you can provide as possible to aid in the review.

As soon as documents are available for review, send the documents to #legal (in an editable format such as .docx).

2. Plan time for legal review

Legal review usually takes a few business days depending on bandwidth, priorities, and existing obligations, and negotiations may take longer depending on the use case, the vendor's contract terms and how quickly they review and negotiate proposed changes.

Plan accordingly and involve legal early. If you have a deadline for implementing the tool or there is another reason the standard timeline above needs to be expedited, please make sure to let legal know ahead of time.

3. Additional requirements for Subprocessors

If a vendor qualifies as a **subprocessor**, the review process will usually be more involved.

Generally speaking, a subprocessor is a vendor or tool that is going to be used to processes customer or end-user data as fundamental part of the PostHog product or infrastructure. For example, infrastructure providers (like cloud hosting) or services that process production data are clearly subprocessors.

Many internal tools used for productivity or operations (for example, documentation and productivity tools) are not *necessarily* subprocessors.

As a rule of thumb, any vendor that needs to have access to customer end-user data in order for a part of the PostHog product to function should raise alarm bells, but if you are unsure whether a tool/vendor qualifies as a subprocessor, always check with #legal early.

For new subprocessors:

- Customers must receive 14 days' notice after the agreement is finalized before the vendor can be used in production.
- Because of this requirement, and because the legal and compliance documents for a subprocessor are generally going to be reviewed with careful detail, implementations involving new subprocessors will likely take additional time.

4. Using the tool

Once:

- Legal review is complete.
- Contract documents are finalized and signed.
- Any required subprocessor notice period has passed.

...the tool can be used in production.

Logos, brand, hedgehogs

contents/handbook/company/brand-assets.md

> **Looking for brand voice, design philosophy, and visual identity guidelines?** Check out our Style guide.

> **Want to use our hedgehogs for your community event or article?** We have a huge library of them you can use. Can't see what you need? Let us know! Please don't use AI art though. We're quite particular about our illustrations and AI just doesn't get it right.

Logo and brand usage for third-parties

We're really happy people want to build on top of PostHog, but we want to keep it clear when something is made by us or made by someone else. If you've built a third-party app on top of PostHog or want to partner with us in some way, here is some high-level guidance for you to bear in mind.

- We're *generally OK* with people using the PostHog name to describe compatibility. For example, you can say your product "works with PostHog," is "built for PostHog" or "built on PostHog".
- We're *not OK* with people using the PostHog name to make it look like your project is made by, endorsed by, or is officially partnered with PostHog if it isn't. So for example, while "Desktop Studio for PostHog" would be fine, "Official PostHog Desktop Studio" or "PostHog Desktop Studio" would not be.
- You can use our logo or brand assets only in unmodified form, and not as the main branding for your own project. However, you may not use our hedgehog mascot or other illustrative brand assets in any commercial or marketing materials without explicit permission, as this can imply endoresement and confuse people. You cannot make it seem like your product is an official PostHog product, or that we've endorsed your product or partnered with you if we haven't. Please make sure that logo, brand asset and name usage are consistent with the rules we've laid out in this page.

We don't like doing it, but if we spot some name, brand asset or logo usage that are inconsistent with our guidelines or brand, we will reach out to try to get that sorted out, so please try to be thoughtful about branding and try to be consistent with the guidelines we've set out here. If you have questions, please reach out to us at marketing@posthog.com for clarification.

Logo

If you're looking for the PostHog logo, you came to the right place. Please keep the logo intact. SVG is always preferred as it will infinitely scale with no quality loss.

(Images shown below have transparent backgrounds but appear here with a solid background color.)

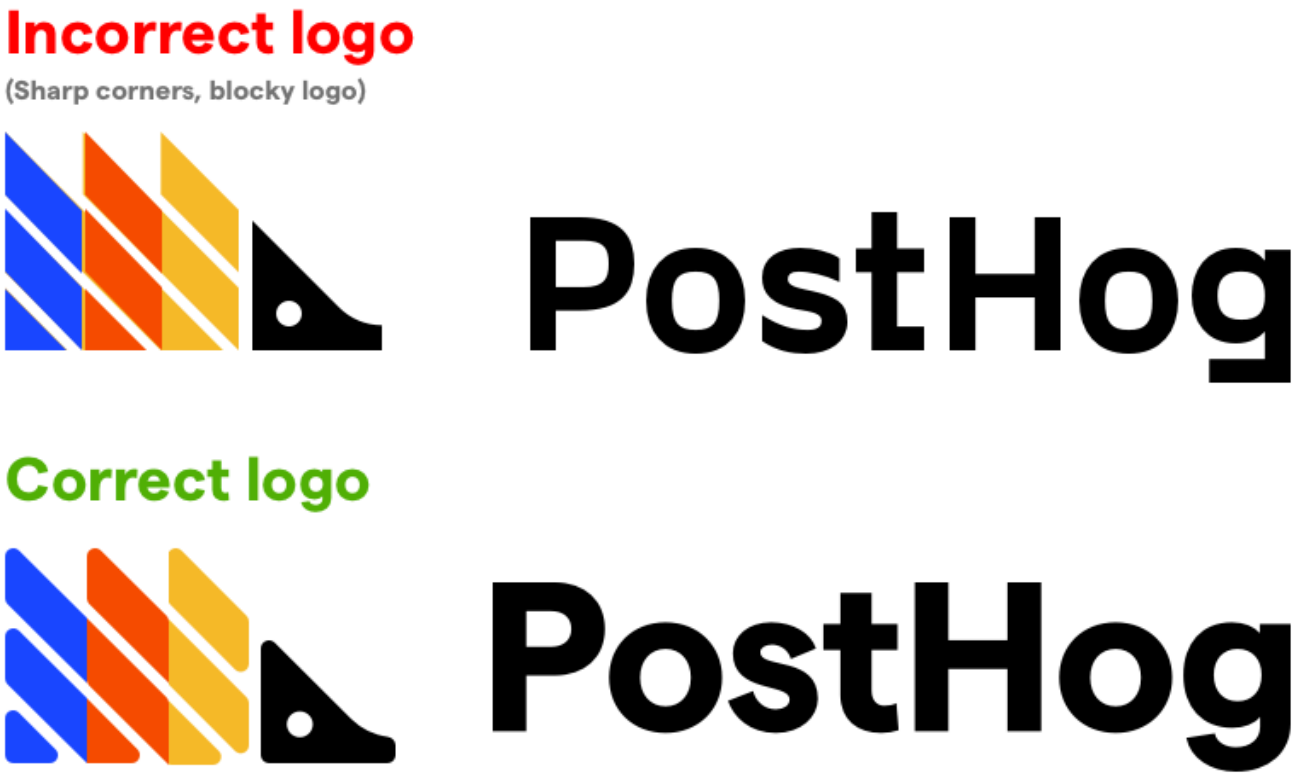
Preview	Name	Vector	PNG	PNG w/ padding*
	Standard logo	SVG	PNG | PNG @2x	PNG | PNG @2x
	Dark logo	SVG	PNG | PNG @2x	PNG | PNG @2x
	Light logo	SVG	PNG | PNG @2x	PNG | PNG @2x
	Logomark	SVG	PNG | PNG @2x	PNG | PNG @2x
	Logo (stacked)	SVG	PNG | PNG @2x	PNG | PNG @2x

*PNGs with padding are useful when uploading the logo to a third-party service where there is limited control over padding/margin around the logo.

When using the logo on a dark background, use the white-only version of the logo. *Never* modify the colors in the logomark (like changing the hedgehog's face color to white when using on a dark background).

The @2x version of PNGs are designed for hi-dpi (or "Retina") screens. When using the logo in third party services that support uploading multiple versions (standard and hi-dpi), please be sure to include the @2x logo as it will appear crisper on newer devices, tablets and high resolution mobile devices.

> **Important:** We updated our logo in 2021. (Note the square font and sharp edges on the logomark in the old version.) Please be sure to use the *correct* version. 👇



If you have any questions or need clarification about which version to use, ask Cory, or reach out in our community page and we'll be happy to help.

Typography

We use Displaay's typeface called *Matter SQ*. (SQ = square dots.)

On the website, we use this for all text. In-product, we only use for titles and buttons.

Building for web

On posthog.com, we use the variable font version. This allows us to specify our own font weights, which we do for paragraph text.

Context: *Matter Regular*'s weight is `430` and the next step up is *Matter Medium* at `570`, so we use our own weight of `475` for paragraph text.

Developing locally

Fonts are hosted outside of our posthog.com GitHub repo (due to licensing reasons). To protect the font files, they are restricted to loading on posthog.com and are not currently used for local development. Contributors will see the system default font load in place of Matter.

Workaround for local development

Restricted to PostHog employees, it's possible to reference the font locally to see an exact replication of what will be published on posthog.com.

global.css contains some commented out code which can be used, in conjunction with the variable webfont files (restricted to PostHog organization members). Here's how to use them:

1. Download the webfont files from the zip above
2. Extract the files and place them in `/public/fonts`
3. In `global.css`, comment out the `src` for both fonts with production (Cloudfront) URLs and uncomment the relative URLs.
4. Optionally use `.gitignore` to keep the files locally without inadvertently checking them in

Note: When submitting a PR, be sure to revert changes made to `global.css`

Designing on desktop

We use 4 cuts of Displaay's *Matter SQ* typeface (SQ stands for square dots):

1. *Bold* (titles and section headers)
2. *Semibold* (paragraphs accompanying headers and paragraph links)
3. *Regular* & *Regular Italic* (paragraph text)

Note that *Regular* and *Regular Italic* are lighter than the font-weight we use on the web, so paragraph text in Figma mockups will look noticeably thinner than how it appears on posthog.com.

When designing ads or other content with non-paragraph text, use *Semibold* instead of *Regular*.

We have a handful of licenses for desktop use of Matter. Contact Cory if you need the desktop fonts (OTFs).

Name	Weight	Size	Letter spacing	Line height
h1	Bold	64px	-1%	100%
h2	Bold	48px	-1%	120%
h3	Bold	30px	-2%	140%
h4	Bold	24px	-2%	
h5	Semibold	20px	-2%	
h6	Semibold	16px	0	
Paragraphs accompanying large headers	Semibold	20px	-1%	125%
p	Regular	17px		175%

Name	Weight	Size	Letter spacing	Line height
p (small)	Regular	15px		150%

Other fonts

We use two other fonts for special purposes. Please adhere to their usage guidelines listed below.

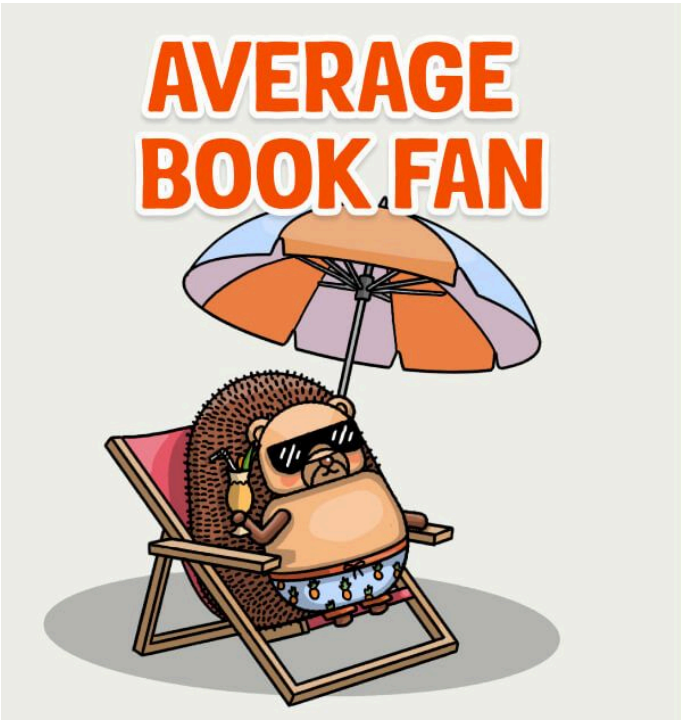
Squeak

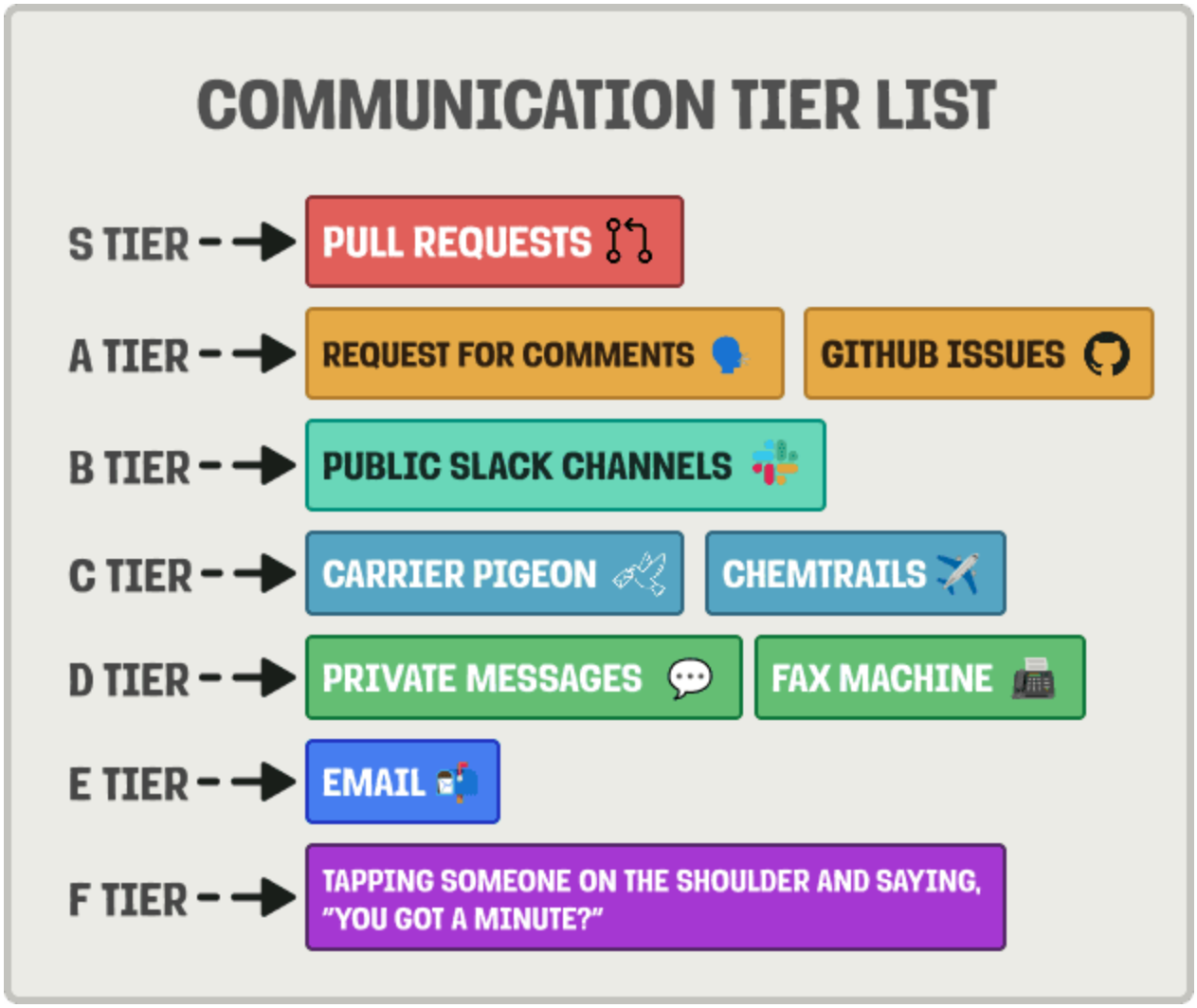
Squeak is used in informal settings, generally accompanied by hedgehog artwork.

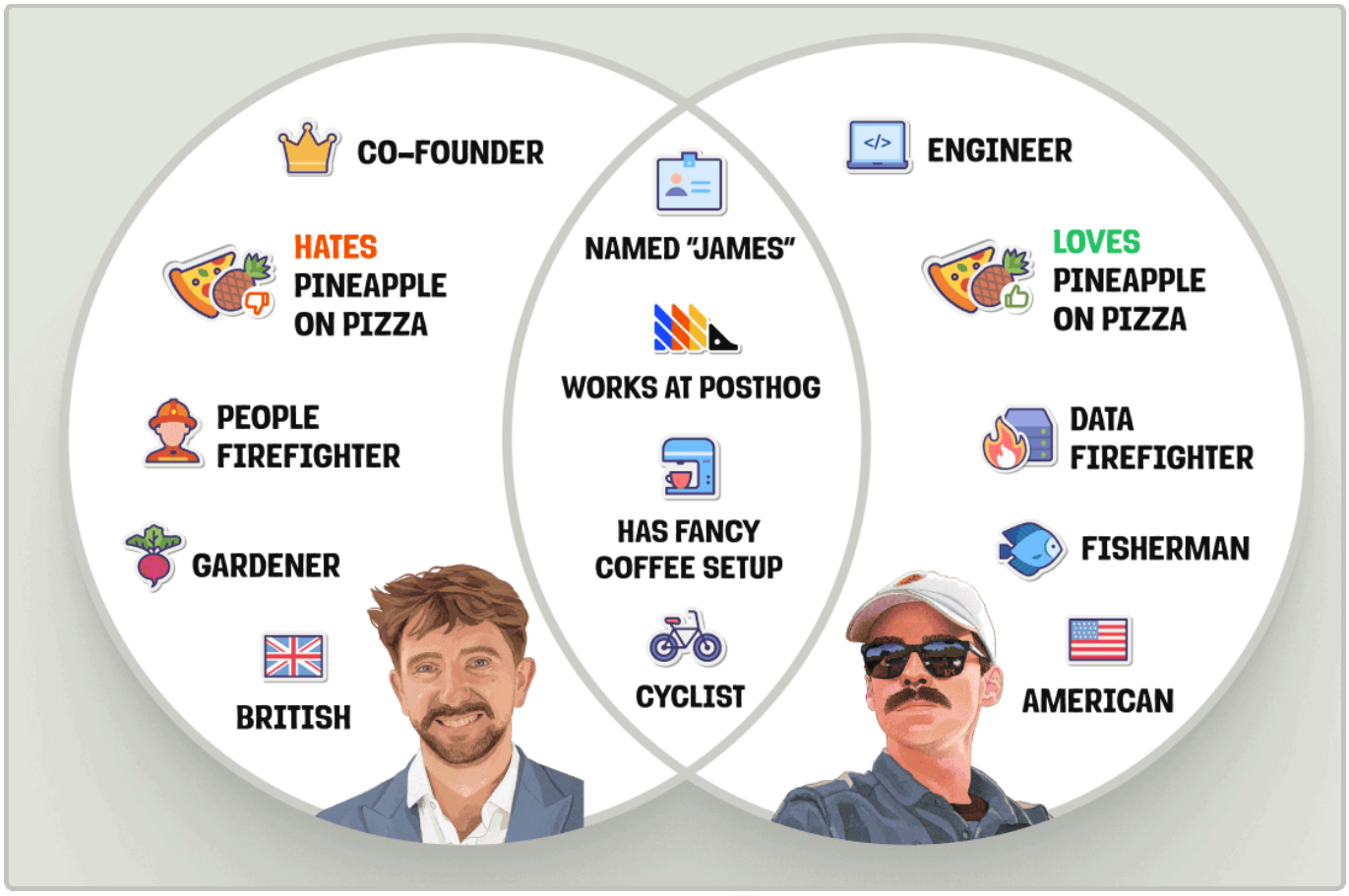
Usage guidelines

- When used for headlines or at larger sizes, use the `Bold` variant
- Only for small (description) text, use the `Normal` variant in regular casing. Never use for more than a couple lines of text in a row.
- Always use uppercase letters
- Letter spacing: -2%
- Line height: 100% (generally)

Examples




COMMUNICATION TIER LIST
S TIER
PULL REQUESTS
A TIER
REQUEST FOR COMMENTS
GITHUB ISSUES
B TIER
PUBLIC SLACK CHANNELS
C TIER
CARRIER PIGEON
CHEMTRAILS
D TIER
PRIVATE MESSAGES
FAX MACHINE
E TIER
EMAIL
F TIER
TAPPING SOMEONE ON THE SHOULDER AND SAYING, "YOU GOT A MINUTE?"



Loud Noises

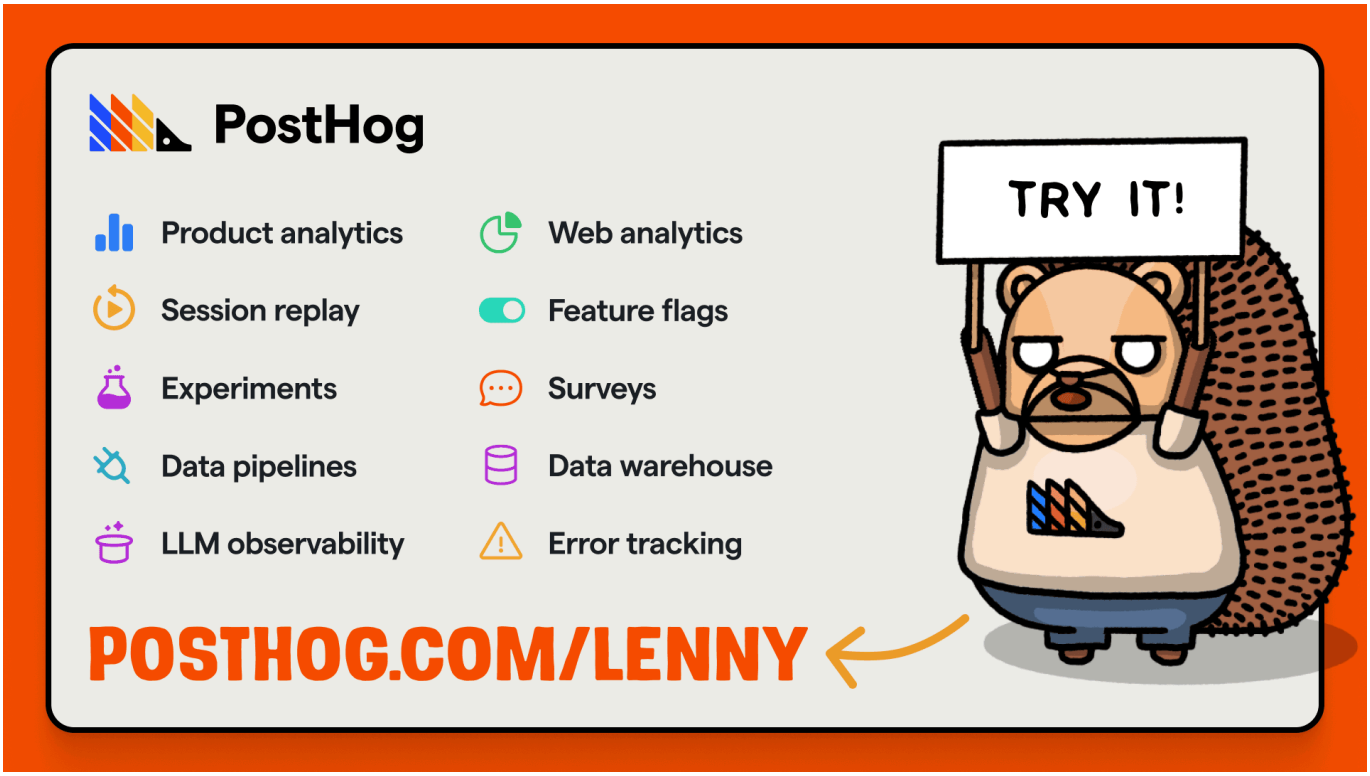
Loud Noises is used for quotes in hedgehog artwork.

Usage guidelines

- Only use for quotes in hedgehog artwork or where hedgehogs are otherwise communicating something
- Only use uppercase

Example

Loud Noises is used in the sign the hedgehog is holding:



If you have questions about which font to use, please ask in #team-website - don't just do what feels right to you!

Colors

We have two color schemes (light and dark mode), but primarily use light mode.

We use the same set of colors, and only swap out a couple hues depending on the color scheme.

Colors denoted with an asterisk (*) are the same between palettes.

Name	Light mode	Dark mode
Text color (at 90% opacity)	■ #151515	■ #EEEFE9
Background color	■ #EEEFE9	■ #151515
Accent	■ #E5E7E0	■ #2C2C2C
Dashed divider line	■ #D0D1C9	■ #4B4B4B
Red*	■ #F54E00	

Name	Light mode	Dark mode
Yellow	#DC9300	#F1A82C
Blue*	#1D4AFF	
Gray*	#BFBFBC	
Links	Use Red	

Use `opacity` over more colors

When possible, use opacity to modify colors. This allows us to use fewer colors in our palette, which is light years easier when working with two color schemes.

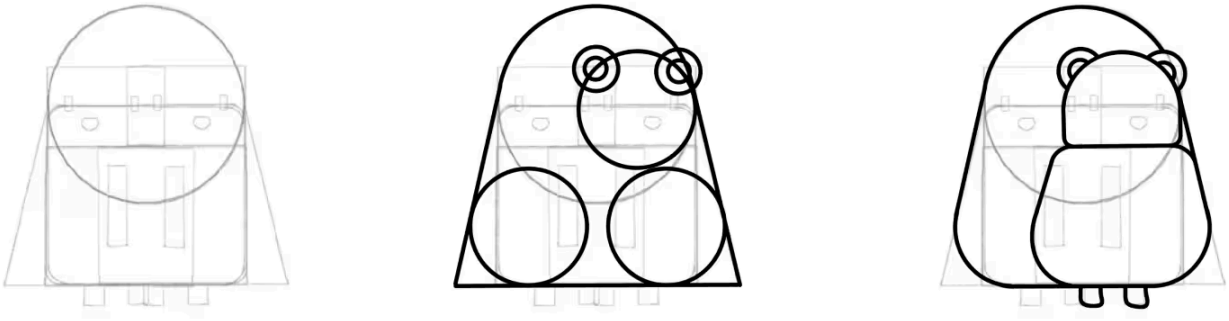
Paragraph text	`rgba($value, 90%)`
Links	`rgba($value, 95%)` (and semibold)
Links:hover	`rgba($value, 100%)` (and semibold)

Presentations

We use Pitch for polished presentations (like when giving a talk). Read more about this in our communication guidelines.

Illustration guide

Our hedgehog mascot is called Max and we're quite particular about how he (or any of his hoggy pals) are illustrated. We're exploring AI tools for internal use, but currently ask that you don't use AI tools to create your own hedgehog art. Instead, you can follow the guidelines below, or create a new art request.



If Max is drawn in color he should always have a beige body with brown spines, arms, and legs. His arms should only bend once in the middle and he doesn't have fingers unless swearing or pointing. His feet are stubby by design and his snout lines should be visible unless obscured by a mask or beard. His expression comes mainly from his eyebrows.

He should be outlined with a strong, black monoline with consistent thickness. He should always face left, right, or straight-on but shouldn't be drawn with a side profile or from behind as he's self-conscious.

> A more detailed version of this guide is available on Figma for team members.

Hedgehog library

For team members we keep all our currently approved hedgehogs in this Figma file. This enables us to look through the library of approved hogs, and to export them at required sizes without relying on the design team.

Here's how:

1. Open the Figma file. You can manually browse, or use `Cmd + F` to search based on keywords such as 'happy', 'sad', or 'will smith'.
2. Select the hog you want. If needed, adjust the size using the 'Frame' menu in the top of the right-hand sidebar.
3. At the bottom of the right-hand sidebar, select the file type you need in the 'Export' menu, choose `@2x`, then select 'Export [filename]' to download the image.

If you can't find a suitable hog, you can request one from the design team.

> Non-team members can find some of the most-used hogs to download on our press page.

Communication

contents/handbook/company/communication.md

Introduction

With team members across many countries, it's important for us to practice clear communication in ways that help us stay connected and work more efficiently.

To accomplish this, we use **asynchronous communication as a starting point** and stay as open and transparent as we can by communicating on GitHub through public issues and pull requests, as well as in our PostHog User and internal Slack.

Our communication values

1. **Assume positive intent.** Always coming from a position of positivity and grace.
2. **Form an opinion.** We live in different locations and often have very different perspectives. We want to know your thoughts, opinions, and feelings on things.
3. **Feedback is essential.** Help everyone up their game in a direct but constructive way.

Golden rules

1. Use **asynchronous communication** when possible: pull requests (preferred) or issues. Announcements happen on the appropriate Slack channels and people should be able to do their work without getting interrupted by chat.
2. Discussion in GitHub issues or pull requests is preferred over everything else. If you need a response urgently, you can Slack someone with a link to your comment on an issue or pull request, asking them to respond there. However, be aware that they still may not see it straight away (and that's OK in our book). That said, casual conversations in Slack are completely normal — it's our main space for day-to-day communication.
3. You are not expected to be available all the time. There is **no** expectation to respond to messages outside of your planned working hours.
4. It is 100% OK to ask as many questions as you have - please ask in public channels! If someone sends you a handbook link, that means they are proud that we have the answer documented - they don't mean that you should have found that yourself or that this is the complete answer. If the answer to a question isn't documented yet please immediately make a pull request to add it to the handbook in a place you have looked for it.
5. When someone asks for something, reply back with a deadline or by noting that you already did it. Answers like: 'will do', 'OK', or 'it is on my todo list' are not helpful. If it is a small task for you but will unblock someone else, consider spending a few minutes to do the task so the other person can move forward.
6. By default, avoid creating private groups for internal discussions.

Public by default

We make things public by default because transparency is core to our culture. The kinds of information we share falls into one of three buckets:

- *Public* - most things, including our product, roadmap, handbook and strategy.
- *Shared internally* - almost everything else, such as financial performance, security, fundraising and recruitment.
- *Private internally* - personal team information, i.e. compensation, disciplinary issues.

Information that is not publicly shared is in areas with complex signals that can impact our ability to sell, raise money or are inappropriate to share more widely for personal privacy reasons.

We have two repos to centralize and document private internal communication. These are the source of truth for any internal information, and anything that should be written down (as established in these guidelines) should live in these repos or (better) in this Handbook, not on Slack. This will make it easier when having to search for older stuff, sharing context between public and internal repos, and for newcomers to have all information they might need readily available.

Company Internal

Repository can be found in https://github.com/PostHog/company-internal

Documents any company-wide information that can't be shared publicly within People, Ops, Legal, Finance or Strategy.

Examples of information that should go here:

- ✅ Hiring plans and discussions *before* we post a job ad
- ✅ People discussions, e.g. benefits, pensions, share options, org structure
- ✅ Onboarding/offboarding checklists
- ✅ Non-engineering team sprint planning (as these will often be a mix of public and private tasks and we don't want to restrict people)
- ✅ Sensitive discussions about future positioning, customer strategy, fundraising, board meetings
- ✅ [Sometimes] Discussions about replacing or adding tools, services, and systems that we use

For company-related issues that *can* be discussed publicly, these should go in the `meta` repo which can be found in https://github.com/PostHog/meta/

Examples of information that should NOT go here:

- ❌ Any information that should be public (see guidelines on public by default), this should go in the public repositories (`posthog`, `posthog.com`, `meta`, ...). Things like:
  - Some marketing campaigns where it doesn't matter if our competitors see it; retros after campaigns
  - Offsite planning and retros

    - Discussions about future positioning and strategy that will end up in the Handbook anyway
    - Discussions about tools where there isn't a security risk and it interfaces with our customers (e.g. marketing, customer support)
    - Generally anything that will end up in the Handbook anyway, including culture and values discussions
- ❌ Bug reports, security issues, or any other engineering-related discussions. These should go in the Product Internal repo.
- ❌ Billing issues, product or growth discussions. These should go in the Product Internal repo.

Product Internal

Repository can be found in https://github.com/PostHog/product-internal

Contains internal information related to the PostHog product. Documents any non-public information (as established in these guidelines) that specifically relates to engineering, product, growth or design.

This repository was introduced to aid maintenance and day-to-day usage of internal repositories. Having these discussions together with the company-wide information proved unwieldy. More context on this decision.

> Please be sure to read the README of the repo for guidelines on how to file specific issues.

Examples of information that should go here:

- ✅ Vulnerabilities (security bugs) reports
- ✅ Bug reports where most of the context of the report depends on customer's PII. *Some bug reports require screenshots, recordings, or some other information that contains PII and as such can't be public.*
- ✅ Post-mortems on outages, or other issues affecting a large portion of customers. The results of these should usually be made public though.
- ✅ Documentation of internal infrastructure, where if it was public knowledge could provide valuable information to an attacker.
- ✅ Experiment (A/B testing) results.
- ✅ Product or growth strategy discussions (unless they should be public).
- ✅ Interview exercises or questions for engineering, product, growth or design tasks that should not be public.
- ✅ Documentation of engineering or product requirements documents that can't be public (these should be quite rare).
- ✅ Billing or pricing-related discussions that is not yet public.

Examples of information that should NOT go here:

- ❌ Any information that should be public (see guidelines on public by default), this should go in the public repositories (`posthog`, `posthog.com`, `meta`, ...).
- ❌ Any internal information that does not fall under the scope of purely engineering, product, growth or design. This should go in the Company Internal repo if private or `meta` if public.

- ❌ Bug reports that don't contain any PII or where the PII only contains supporting information. In this case, file the bug under the relevant public repo and add a protected link to the additional information (e.g. a private Slack link, or a link to this repo).

Written communication

GitHub

Everything starts with a pull request

It's best practice to start a discussion where possible with a Pull Request (PR) instead of an issue. A PR is associated with a specific change that is proposed and transparent for everyone to review and openly discuss. The nature of PRs facilitate discussions around a proposed solution to a problem that is actionable. A PR is actionable, while an issue will inevitably lead to a longer period before the problem is addressed.

Always open a PR for things you are suggesting and/or proposing. Whether something is not working right or we are iterating on new internal process, it is worth opening a pull request with the minimal viable change instead of opening an issue encouraging open feedback on the problem without proposing any specific change directly. Remember, a PR also invites discussion, but it's specific to the proposed change, which facilitates focused decisions.

By default, pull requests are **non-confidential**. However, for things that are not public please open a confidential issue with suggestions to specific changes that you are proposing. When possible, consider not including sensitive information so the wider community can contribute.

Not every solution will solve the problem at hand. Keep discussions focused by *defining the problem first* and *explaining your rationale* behind the Minimal Viable Change (MVC) proposed in the PR. Have a bias for action and don't aim for consensus - some improvement is better than none.

Issues

GitHub Issues are useful when there isn't a specific code or document change that is being proposed or needed. For example, you may want to start an issue for tracking progress or for project management purposes that do not pertain to code commits. This can be particularly useful when tracking team tasks and creating issue boards.

However, it is still important to maintain focus when opening issues by defining a single specific topic of discussion as well as defining the desired outcome that would result in the resolution of the issue. The point is to not keep issues open-ended and to prevent issues from going stale due to lack of resolution. For example, a team member may open an issue to track the progress of a blog post with associated to-do items that need to be completed by a certain date (e.g. first draft, peer review, publish). Once the specific items are completed, the issue can successfully be closed.

> **Note:** If you're new to using GitHub, check out this handy primer - it's specific to how we use GitHub at PostHog. You'll learn the key concepts and how to manage notifications. It's important, as this is where the bulk of our company-wide communication happens. (Think of GitHub notifications as a replacement for your work email.)

Keeping on top of reviews, issues and notifications

Keeping track of everything that's happening in GitHub can be daunting, but it's important to make sure your team receives reviews and feedback on a timely manner.

To keep on top of this, we suggest going through issues where you've been mentioned regularly. Some tricks which can help are:

- (Highly recommended) Turn on Github slack notifications
  - This will send you a slack notification when someone mentions you in a PR or issue. You can also get periodic reminders for PRs that you've been requested to review.
- (Highly recommended) Join the `#github-rfcs` channel on Slack. This is where we post all the RFCs.
- Turning on GitHub email notifications and use filters to focus on your teams activity.
- Using the GitHub notifier extension.

Tip for easy searching through everything

To search all code, PRs and issues ever written at PostHog you can search everything in the PostHog organization on Github. To do that can go to github.com/posthog and search in the top left corner.

For extra convenience, you can also add this search as a 'search engine' in Chrome. That way you can type in `ph <tab>` and instantly find anything. To do that, follow these steps:

1. Hit `command + ,` in your browser
2. Type `search`, find "manage search engines"
3. Click "add" next to "other search engines"
4. For "Search engine" type in `github posthog organization`
5. For "keyword" type in `ph`
6. For "url" copy in `https://github.com/search?q=org%3Aposthog+%s&type=issues`

You can now type `ph + tab` into your browser and search issues directly

Slack

Slack is used for more informal communication, or where it doesn't make sense to create an issue or pull request. Use your judgment to determine the appropriate channel, and whether you should be chatting publicly (default) or privately.

Also keep in mind that, as an open source platform, PostHog has contributors who don't have access to Slack. Having too much context in a private location can be detrimental to those who are trying to understand the rationale for a certain decision.

> Slack canvasses are useful for storing information like schedules, bookmarks, personal to-do lists, scratch notes etc. However, things like quarterly goals, runbooks, sprint plans, FAQs etc. should live in the Handbook, Docs, or in a GitHub RFC or Issue by default. If you find yourself documenting something useful in Slack, it's much better to put it in GitHub instead and link to it from Slack so that PostHog AI can include it in future search results. Slack canvasses are terrible for searchability!

> Slack recap is a great way to learn from others by adding channels like `#ask-max` and `#today-i-learned` to the recap. You can also use it to keep tabs on teams you may not directly work on, but still want to know what's being discussed.

> Slackbot is a handy AI agent that can search across the PostHog workspace in Slack to help answer your questions. If you're looking for information or trying to find a past conversation, Slackbot is a great place to start.

Slack etiquette

Slack is used differently in different organizations. Here are some guidelines for how we use Slack at PostHog:

1. Keep `#general` open for company-wide announcements.
2. `@channel` or `@here` mentions should be reserved for urgent or time-sensitive posts that require immediate attention by everyone in the channel. (Examples: changing a meeting invite URL just before a meeting, or soliciting urgent help for a service disruption, where you're not sure who is immediately available)
3. Make use of threads when responding to a post. This allows informal discussion to take place without notifications being sent to everyone in the channel on every reply.
4. When possible, summarize multiple thoughts into a single message instead of sending multiple messages sequentially.
5. You don't need to tell people if you're away from your computer, especially on no-meeting days. There's no general expectation people are available to reply to messages in real time, including in Slack.
6. Keep your Slack profile up to date with the right information, including the appropriate name eg with surname or surname initial if you share a name with a colleauge.

Channel naming conventions so people don't get confused:

- `#team-[team-name]` - small team channels, only as listed on the teams page
- `#project-[project-name]` - one-off initiatives that may involve people across multiple teams, but don't fit neatly into a team channel

- `#posthog-[customer-name]` - shared channels with *customers* only (if you want to create a shared channel with an external partner, use `#[partner-name]-posthog` instead)
- `#alerts-[team-name]` - useful to create a separate channel for your team to send alerts into, so your main channel doesn't get noisy
- `#support-[product-name]` - similarly, useful to feed support requests in if helpful without adding clutter
- `#offsite-[team]-[month]-[year]-[where]` - For planning and coordination of team offsite events
- `#onboarding-[who]-[team]-[month]-[year]-[where]` - For coordinating and supporting new team member onboarding
- `#hiring-[team-name]` - For recruiting discussions, candidate feedback, and hiring coordination for a specific team
- `#superday-[first-name]-[role]` - For candidate interview coordination and feedback during intensive interview days

On the very rare occasions you need to create a private channel for some reason - most commonly hiring-related - then it's probably worth sticking `#private-xxxxx` in front so people don't accidentally add external parties who shouldn't be in there.

Google Docs and Slides

Never use a Google Doc / Slides for something non-confidential that has to end up on the website or this handbook. Work on these edits via commits to a pull request. Then link to the pull request or diff to present the change to people. This prevents a duplication of effort and/or an out of date handbook.

We mainly use Google Docs to capture internal information like meeting notes or to share company updates and metrics. We always make the doc accessible so you can comment and ask questions.

Please avoid using presentations for internal use. They are a poor substitute for a discussion on an issue. They lack the depth, and don't add enough context to enable asynchronous work.

When giving a talk which requires a presentation, use Pitch to build your slides. (It offers more control over design than Google Slides.) They also have a desktop app. We don't (yet) have templates configured, but you can draw from existing slides in other presentations - just copy/paste into your own presentation and modify accordingly. If you'd like assistance with slide design (or using Pitch), talk to Cory.

James (H) and Cory are admins on the Pitch account. Because Pitch charges per seat, we remove users who only need periodic access but can easily re-add when needed.

Email

1. Internal email should be avoided in nearly all cases. Use GitHub for feature / product discussion, use Slack if you cannot use GitHub, and use Google Docs for anything else.
2. The only uses we have for internal email are:
    - Obtaining approvals for legal things

- Sending some types of more official company documents (e.g. job offers, payroll forms)
- Communicating with external partners

Writing style

1. We use American English as the standard written language in our public-facing comms, including this handbook. This extends to date formats (September 4, 2021) and defaulting pricing to the US Dollar ($42).
2. Do not use acronyms when you can avoid them. Acronyms have the effect of excluding people from the conversation if they are not familiar with a particular term.
3. Common terms can be abbreviated without periods unless absolutely necessary, as it's more friendly to read on a screen. (Ex: *USA* instead of *U.S.A.*, or *vs* over *vs.*)
4. We use the Oxford comma.
5. Do not create links like "here" or "click here". All links should have relevant anchor text that describes what they link to. Using meaningful links is important to both search engine crawlers (SEO) and people with accessibility issues.
6. We use sentence case for titles.
7. When writing numbers in the thousands to the billions, it's acceptable to abbreviate them (like 10M or 100B - capital letter, no space). If you write out the full number, use commas (like 15,000,000).

Requests for comment (RFCs)

We use RFCs to communicate and gather feedback on a decision. RFCs are useful because they help us stay transparent, and the process of writing them forces you to clearly articulate your thoughts in a structured way.

Here are the steps for an RFC:

1. Identify a problem and a decision to be made
2. Create an RFC as a pull request using one of the RFC templates.
    - Using a template isn't a requirement, though it is a helpful and recommended starting place if you haven't written many RFCs here before. You can also get inspiration from other RFCs, as many have different sections and styles depending on the type of thing being discussed.
3. Share the RFC:
    - Assign people whom this RFC will impact, or who may have good opinions on the topic, as reviews to the pull request.
    - Post in the relevant Slack channel (normally the teams slack channel, or `#tell-posthog-anything` if it's a bigger cross-team RFC)
    - The aim is not to get one person to mark the RFC as approved, but to get the right people involved and commenting. It's a request for *comments* not a request for *approval*. That means you might need to chase them to make sure it happens - tag the people, present at all hands, use offsites or other sync time - **it's your responsibility to nudge the relevant people for their input**

4. If an RFC is cross-team and is causing a large amount of disagreement, it might be worth having a sync meeting to reach a decision
5. Once a decision is made, include the decision in the pull request, merge it in and share this in the relevant channel and `#github-rfcs` again.

When does it work best to write an RFC?

Writing an RFC may be helpful when any of the following is true:

- You want to clarify something for yourself or it affects just one team
- It is a relatively non controversial change/idea that doesn't require much extra context. In other words, it doesn't create problems for another team.
- It will be a large amount of work (more than 2-3 weeks of people's time)
- It's introducing a new technology
- It's a major new feature, change to the product, or change to the company
- It will have a major customer impact

When does a meeting / another approach work better than an RFC?

An RFC is likely to be unhelpful as a first step in other circumstances. Specifically, when you want to ship or suggest a change to something that significantly affects teams outside your own. In this instance, we've seen that RFCS can lead to 10 to 25+ comments, which feels antagonistic (teams having to explain all the context around their strategy down to why this decision is something they perhaps disagree with), and creates a lot of work. A single call in this instance is likely much faster than lots of frustrated people in 1/1s talking about it *and* the energy/time needed to respond to everything in a long thread.

However please write notes on such a call - to ensure everyone *is* on the same page. This could then be copy pasted into an RFC for transparency's sake / future reference.

Top tips for RFCs

- RFCs can be very short and are often better than making decisions by Slack threads.
- You don't need to have a long decision-making time - 2 days is fine for smaller changes if you receive the relevant input and are confident in your decision.
- You don't need to reach full agreement to decide, particularly if the decision is reversible. Instead, it should be when the decision maker has considered the feedback and is confident in their decision.
- Double-check whether your input will be useful or add noise. Or wait for the people closest to first discuss the problem. At PostHog, we don't make decisions by committee - instead we have great people divide and conquer. This particularly applies to the controversial areas such as pricing.
- As the decision maker, you should use your judgment as to which comments you want to respond fully to. It's fine to politely decline a question if you think it's not required for the decision being made.
- If you're introducing new technologies, you'll likely want to tag someone from Team Infrastructure.

- You don't need to wait until the date you've said to make the decision if you've already consulted with the key people.
- Make it easy for others to give feedback, e.g. if you only need input from someone on the Infra team about adding websockets then say that, rather than leaving it for them to work out
- Write your RFC with the busy reader in mind. For example, if there is a lot of technical context to give, write a summary that people can read through quickly to get a high level overview of the proposed changes, then go deeper below, or in appendices.
- It's fine to nudge people on slack if they are being slow to give feedback

Internal meetings

PostHog uses Google Meet for video communications. For large meetings, use `CMD + minus key` to zoom out and see everyone - you'll usually need to do this in All Hands.

Use video calls if you find yourself going back and forth in an issue/via email or over chat. Sometimes it is still more valuable to have a 40+ message conversation via chat as it improves transparency, is easy to refer back to, and is friendlier to newcomers getting up to speed.

1. Most scheduled meetings should have a Google Doc linked or a relevant GitHub issue. This contains an agenda, including any preparation materials.
2. Please click 'Guests can modify event' so people can update the time in the calendar instead of having to reach out via other channels. You can configure this to be checked by default under Event Settings.
3. Try to have your video on at all times because it's much more engaging for participants. Having pets, children, significant others, friends, and family visible during video chats is encouraged - please introduce them!
4. As a remote company we are always striving to have the highest fidelity, collaborative conversations. Use of a headset with a microphone, is strongly recommended - use your company card if you need.
5. Always advise participants to mute their mics if there is unnecessary background noise to ensure the speaker is able to be heard by all attendees.
6. You should take notes of the points and to-dos during the meeting. Being able to structure conclusions and follow-up actions in real time makes a video call more effective than an in-person meeting. If it is important enough to schedule a meeting, it is important enough to have taken notes.
7. We start on time and do not wait for people. People are expected to join no later than the scheduled minute of the meeting, and we don't spend time bringing latecomers up to speed.
8. It can feel rude in video calls to interrupt people. This is because the latency causes you to talk over the speaker for longer than during an in-person meeting. You should not be discouraged by this, as the questions and context provided by interruptions are valuable.
9. We end on the scheduled time. Again, it might feel rude to end a meeting, but you're actually allowing all attendees to be on time for their next meeting.

10. It is unusual to smoke or vape in an open office, and the same goes for video calls - please don't do this out of respect for others on the call.

For external meetings, the above is also helpful.

Indicating availability

1. Put your planned away time including holidays, vacation, travel time, and other leave in your own calendar.
2. Set your working hours in your Google Calendar - you can do this under *Settings* > *Working Hours*. This is helpful as we work across different timezones.

Google Calendar

We recommend you set your Google Calendar access permissions to 'Make available for PostHog - See all event details'. Consider marking the following appointments as 'Private':

1. Personal appointments
2. Particularly confidential & sensitive meetings with third-parties outside of PostHog
3. 1-1 performance or evaluation meetings
4. Meetings on organizational changes

Calendly

We use Calendly for scheduling external meetings, such as demos or product feedback calls. If you need an account, ask Simon in `#sales` to invite you to the PostHog team account.

Communication Methods

PostHog employees are frequent targets of scams and phishing. Expect all communication to occur over Slack. Phone calls, SMS, and WhatsApp are **never** used for initiating requests, approvals, or asking for sensitive info. With few exceptions, email is never used for this either.

If someone contacts you outside of Slack, treat it as **untrusted until verified**. Message them on Slack to confirm, and only continue the conversation over Slack. Other communication methods are susceptible to phishing, but our Slack instance is locked down and generally well protected from phishing and impersonation.

Best practices

James, Tim, and other execs will never ask for wire transfers, gift cards, MFA codes, or access changes over email/SMS/WhatsApp/phone. Treat such requests as phishing and report them to `#phishing-attempts`.

By email: Only trust `@posthog.com` senders. Verify via the company directory. Be cautious of look-alike domains (e.g., `posthog.co` vs `posthog.com`), unexpected attachments, and “urgent” requests.

Phone/SMS/WhatsApp: **Never** used for initiating requests, approvals, or asking for sensitive info.

If something feels off, it probably is. When in doubt, slow down and verify in Slack.

Culture

contents/handbook/company/culture.md

So, what's it like working at PostHog?

We're 100% remote

Our team is 100% remote, and distributed across more than 20 countries.

Being all remote has a bunch of advantages:

- We can hire amazing people from a global talent pool.
- It encourages thoughtful, and intentional, written communication
- It creates space for lots of uninterrupted work.
- We judge performance based on real outcomes, not hours spent in an office.

In addition to all the equipment you'll need, we provide a budget to help you find coworking space, or to cover coffee shop expenses. Everyone also has a $1,500 quarterly travel budget for ad-hoc meetups.

We're extremely welcoming

This is so important to us that it has its own dedicated page.

We're extremely transparent

As the builders of an open-source product, we believe it is only right that we be as transparent as possible as a company.

This isn't just a meaningless corporate statement. Most of our communication happens publicly on GitHub, our roadmap is open for anyone to see, and our open-source handbook explains everything from how we hire and pay team members to how we email investors!

Almost everything we do is open for anyone else to edit. This includes things like the contents of this very Handbook. Anyone can give direct feedback on work they think could be improved, which helps increase our responsiveness to the community.

We're committed to much more than just public code.

We write everything down

We're an all-remote company that allows people to work from almost anywhere in the world. With team members across many countries, it's important for us to practice clear communication in ways that help us stay connected and work more efficiently.

- It creates clear and deep thought.
- We have an open core business model. This helps the community understand our decision-making.
- It is usually clearer than a conversation, so everyone can row in the same direction.
- It is very leveraged as we grow a large community and look to hire people around the world.

To accomplish this, we use asynchronous communication as a starting point and stay as open and transparent as we can by communicating through public issues, pull requests, and (minimally) Slack.

Putting things in writing helps us clarify our own ideas, as well as allow others to provide better feedback. It has been key to our development and growth.

We give direct feedback early and often

Everyone should help everyone else raise their game. After completing difficult work, fatigue tends to set in. It is challenging to maintain objective views of the quality of your own work when you are fatigued. It's easier for outsiders with fresh eyes and energy to raise the level of others around them.

We are direct about the quality of work. That doesn't always mean work needs to be completely polished, as it depends on the speed and impact of a task. Being great at giving and receiving feedback is a key part of our culture.

We bias for action

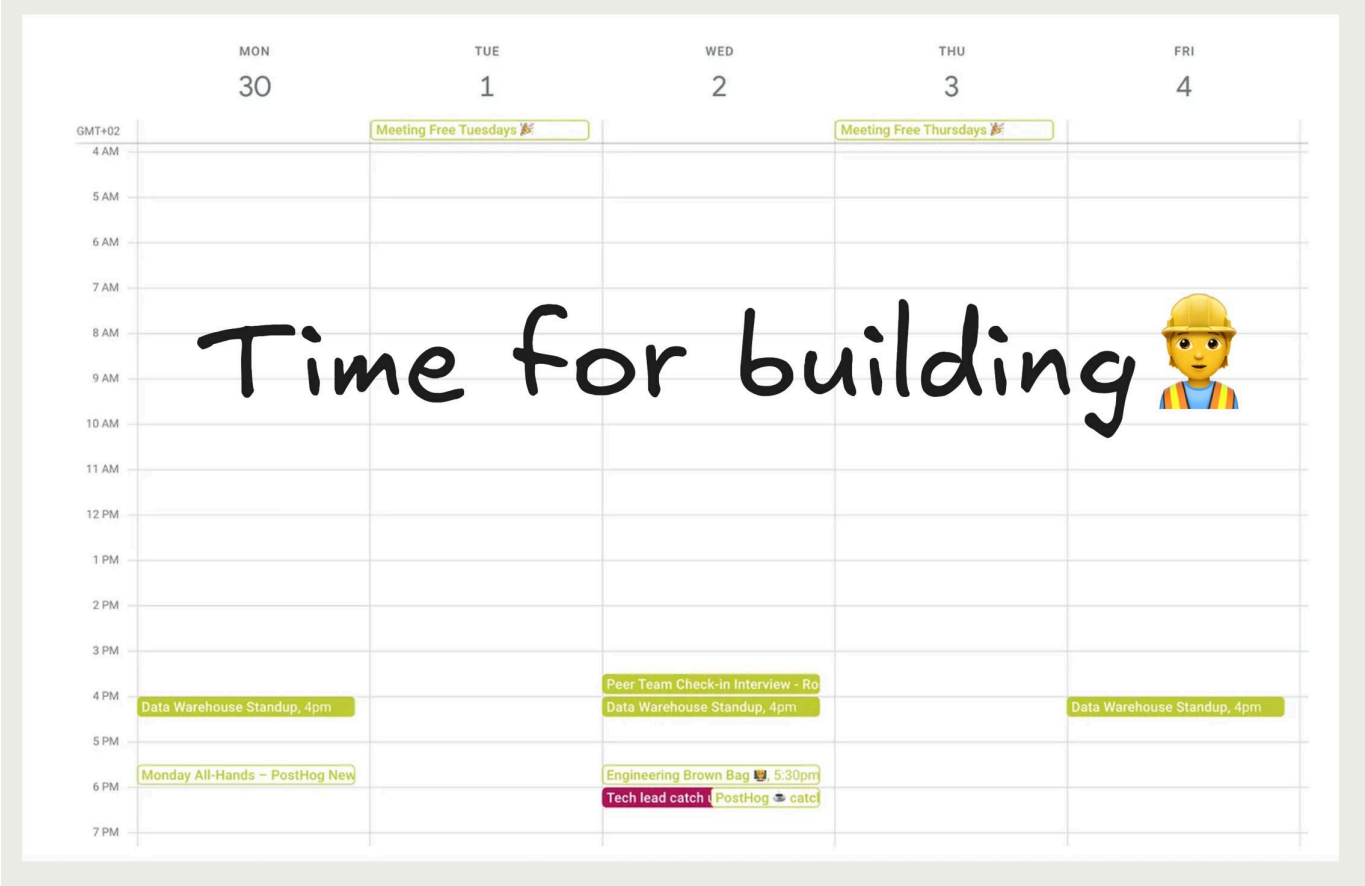
If given a choice, go live. If you can't go live, reduce the task size so you can.

- We are small, and can only win based on speed and agility.
- Going live forces a level of completion, on which you can build.

Default to *not* asking for permission to do something if you are acting in the best interests of PostHog. It is ok to ask for more context though.

We're on the maker's schedule

We're big believers in the importance of the maker's schedule. If we have meetings at all, we'll cluster them around any stand-ups, so our day doesn't get split up.



On Tuesdays and Thursdays, we don't have internal meetings at all. Occasionally an external meeting will slip in on those days, such as interviews, but we try to keep those to an absolute minimum.

We're structured for speed and autonomy

Hiring high-performing and self-sufficient team members means we don't need the typical corporate processes that are designed to slow teams down. Instead, we're organized into small teams, which prioritize speed by delegating decision-making autonomy as much as possible.

Our management approach is super simple - small teams report to their team leader, and each of the team leaders reports to one of our four execs. We don't want to create a fancy hierarchy of titles, as we believe this can lead, consciously or not, to people feeling less empowered to make changes and step on toes, especially if they are not in a 'senior' role.

It's up to you how to get things done. If you want to make a change, feel free to just create the pull request. If you want to discuss something more widely for a bigger piece of work, it might make sense to use an RFC for a change inside your team.

If your RFC could significantly impact other teams as well, it usually works best to book a call with them as well as it usually saves time – "fewer meetings" doesn't mean "no meetings", just that they should be meaningful and intentional, not routine.

Read How you can help to understand how you can contribute to this culture.

Do more weird

contents/handbook/company/do-more-weird.md

For some (all?) at PostHog, do more weird isn't just a benign corporate value - it's a way of life. A craft, to be honed. This page will help you navigate do more weird at PostHog so that you too may do... weird things.

What do more weird is

PostHog's competition for attention is not with other boring B2B SaaS companies - it's with the internet as a whole. Memes. TikTok. HackerNews. We bring a consumer mindset to a B2B context.

Things to bear in mind:

- It's a numbers game. Expect 95% ideas to go nowhere. And if you try them, only some will really take off. That's ok!
- We're trying to drive overall awareness of PostHog as a brand with the people we care about, aka *product engineers at high growth companies*. We are not trying to get people to sign up to PostHog.
- Be genuinely entertaining - create something that you would enthusiastically share with friends.

Weird ideas are fragile, so the most important feedback you can give if you see a weird idea is 'do I think this is a good idea' not 'can we do it'.

What do more weird isn't

Weirdness isn't *purely* vibes based - it has to be good and vaguely relevant to our users.

Some of the pitfalls we've learned to avoid:

- *Corporate try hard* - aka 'how do you do fellow kids' energy. We know it when we see it. If you wouldn't genuinely enjoy and enthusiastically recommend it to your friends, our audience won't either.
- *"Marketing/Website team do some work"* - it's fine if you have an idea that you want someone else to execute, but others teams are busy (and have a bunch of weird ideas of their own).
- *In jokes* - the thing has to be weird/entertaining/funny to people who don't work at PostHog. Describe the thing to a friend/partner/stranger - do they chuckle?
- *Spending money for the sake of it* - we are willing to spend money on weird, but just spending money and then doing nothing doesn't work. There needs to be follow through.

Weirdness is relative

Sometimes the thing just isn't *that* weird, but that may still be ok depending on the context. For example, transparent pricing is weird in the context of how we bill customers, but it wouldn't make sense to do something truly weird like bartering grain for PostHog credit or something.

On the other hand, the bar for weirdness in a marketing campaign is extremely high, because the world is full of marketing teams trying to do the same thing. A wry smile in response to the idea will not cut it.

Got a weird idea?

Depending on the idea, you have a couple of options:

- Just do it. Usually best for things you can ship yourself - you're the driver after all.
- Post it in `#do-more-weird` and see if others want to get involved.

Making bigger weird happen

Sometimes weird ideas take a lot of money and/or people's time. We have a monthly do more weird marketing budget that we can put towards such things. Lottie and Charles, together with the Council of Weird, meet monthly to pull from the frozen locker of weird things to see what we want to invest in next.

Setting quarterly goals

contents/handbook/company/goal-setting.md

We plan objectives every quarter. The set the direction and overall objectives for PostHog, and then small teams set their own objectives that feed into these. Longer-term planning that the Blitzscale team does is covered separately in the annual planning guide.

How quarterly planning works

1. ~3 weeks before the end of each quarter, the Blitzscale team meets to come up with larger goals for the company, which sometimes (but not always) trickle down to individual teams.
2. ~2 weeks before the end of the quarter, team leads should schedule planning meetings to go through these - these will be run by the team lead and include the relevant Blitzscale team member, following the template below. Each small team can change or propose alternate objectives, goals, and/or key results (we are not prescriptive about the exact terms used here - use these as a starting point).
3. After the planning meeting, the team lead creates a PR on their small team page with the new goals. Make sure you tag the relevant member of the Blitzscale team for review at a minimum.
4. Goal PRs need to be merged before the next quarter starts. We usually then run through the objectives in the first all hands of the next quarter.

In terms of accountability, will notify all the small teams and make sure that the quarterly meetings happen (and that each small team has a PR), but he will not schedule the meetings for you.

> If you prep properly (see below), planning meetings should take 1 hour max. The meeting is not the end of the process - you may still have some back and forth on the PR, but the meeting should give the team lead enough info to write a good PR.

Planning template

Teams should fill in the previous quarter and HOGS sections async in the doc before the meeting starts. The meeting itself should be 20% reviewing the past, and 80% talking about goals for next quarter. Don't fall into the trap of spending most of your time reviewing and then rushing the goals right at the end.

```
## Last quarter objectives reflection (5 mins - do as a team)

[Paste in previous quarter objectives from the team page]

For each objective, write up a reflection - usually the person who was the lead
        on the objective should do this, but some might be shared.
```

```
What else did we get done? List items from the changelog or PRs merged if they
        were significant items that deviated from the original goals (changing
        goals mid-quarter is okay!)

You may also want to write some overall thoughts about how the quarter generally
        went.

## HOGS (10 minutes - all the content should be here **before** the meeting
        starts)

This should be done BEFORE the meeting by everyone, independently and in this
        doc before the meeting starts. Spend 10 minutes in the meeting silently
        reading through everyone's HOGS during the meeting. Add any common
        themes or otherwise important items to the section below as you read,
        then only discuss the themes (there generally isn't time to discuss
        every line of the HOGS).

Add entries under each question with your initials/name like: "- (your name): My
        comment"


- Hope
  - What are you most excited about this quarter?
  - What exploration do you want to do?
  - For Error Tracking/Session
        Replay/Experiments/Conversations/Experiments/Conversations/Product
        Analytics/LLM Analytics:
    We want to make PostHog proactive. How do we programmatically emit signals -
        i.e. research prompts for an agent with code access & PostHog MCP - that
        will result in quality concrete code changes?
- Obstruction
  - Is there anything embarrassing about your product?
  - What's stopping you from shipping 2x what you're shipping now?
  - Can your product be used by agents (Claude Code/Cursor/etc.) via our MPC
        server? What MCP tools or skills are missing right now?
- Growth
  - What single thing would move the needle the most this quarter?
    - _If it's more people, please expand on how many and exactly what type of
        hire you'd be looking for._
  - What are users asking for? Which of these are we ignoring?
    - _Check Vitally for feature requests (if applicable for your team/product)_
  - How will users interact with your product in 1 year's time and what can we
        do now to get this ready?
- Sneak attack
  - Say a competitor beats your team's product, what would that product do
        differently?
  - What are we not talking about enough?


## Themes (20 minutes - do as a team)
```

```
What themes can we distill from the above HOGS list? What are categories of
        things we should consider working on? What are other things we might
        want to consider?


## New goals (15 minutes - do as a team)

This is an example - feel free to adapt as you need. Generally it is a good idea
        to have at least one person's name against each thing for accountability
        even if multiple people work on it - shared goals usually results in
        less getting shipped.


Objective 1: PostHog in the EU
Motivation: Unblock 1,000s of customers [link to data] who need to keep data in
        the EU but are not capable of self hosting.
What we'll ship:
  - This thing (Name)
  - Another thing (Name)
  - Maybe another thing (Name)
```

If you aren't on a product team, replace 'product' with the equivalent 'thing' on your team - e.g. if you do recruitment, your 'product' is how we do recruitment at PostHog and your users are job applicants.

Publishing and viewing goals

When a team has set their quarterly goals it is the responsibility of the team lead to document the goals on their team page, publicly. This enables teams to see what each other are working on, helps us hold teams accountable to their goals, and creates a shared sense of urgency and direction.

You can easily see what goals teams have set on the WIP page, which pulls all goals from the respective team pages.

Teams can choose to document their goals publicly in a number of formats, but below is a useful template for getting started. Individuals may also choose to create planning issues to track their work in greater detail.

```
<details>
<summary>Write the goal here (owner)</summary>

- **Rationale:** Why is this important?
- **What we'll ship:** Keep this brief
- **We'll know we're successful when:** Metric or outcome
```

```
</details>
```

Good goal setting

- As few objectives as possible
- Motivation - explains why the objective is set
- Things we'll ship that show if we're en route to achieving an objective
- Objectives are simple - it's really clear if you are/aren't hitting them
- Objectives are ambitious - they move the needle for PostHog
- Hitting an overarching objective is more important than shipping specific things
- Things we'll ship are leading indicators and can be achieved quickly
- If you set specific targets, they should be specific and measurable if possible
- Setting anti-goals can be helpful to clarify what you are *not* working on

Bear the following in mind:

- Use metrics only if they help you. Goals should be primarily output-based - the actual things that we will do and build.
- Don't fall into an existential crisis every time we do this exercise - while objectives are important, they're easy to change, so iterate if you need to mid-quarter
- All objectives are bad - they have many compromises, are fallible, easy to game, or may be affected by external factors, so use the least bad ones
- Use counter metrics where needed (X happens, but Y shouldn't happen)
- Don't have a lot of things to ship if you can't capture everything in one - just pick the most important one or two
- Don't set arbitrary targets that a team cannot achieve
- Consistently hitting ambitious objectives over the long term is an important factor in the pay review process, but if you miss extremely tough objectives and still achieve great things en route, that's also fine

FAQ

What if I don't have time to do work towards my objectives because of customer support/urgent board reporting/something else?

Picking up the occasional thing that isn't technically going to help your goal is ok. This is because we're small and may not set 100% perfect goals. As ever, prioritize as you see fit. However, spending a bunch of time on a pet project is not - this means the planning process has failed.

If my team repeatedly miss objectives, what happens?

Objectives should be ambitious but achievable - you should be able to hit them by challenging yourself, but not to the point of burnout. If your team is consistently missing objectives, they are too hard or possibly the wrong objectives for PostHog/your team.

A grown-up company

contents/handbook/company/grown-ups.md

We're very proud to have a genuinely welcoming environment where PostHog treats you, and we treat each other, like grown ups.

We're an international bunch of weirdos, but one thing us weirdos have in common is that everyone is kind, courteous, and professional towards each other - and that's something we're really proud of. And we all ship, of course!

Things we do to create a welcoming environment

We have tried many different tactics over the years, and these are the things we have found *actually* make a difference.

- We are all remote. At the time of writing, we employ people in 23 countries across 3 continents.
- Asynchronous and transparent communication - so people can get the context they need to work effectively no matter what their schedule.
- We offer near complete flexibility over working hours. You can do the school run, or schedule that dentist appointment you've been avoiding!
- An anti-meeting culture. Ever had someone schedule a meeting over family time because they couldn't find a slot in your schedule? That doesn't happen here.
- Generous parental leave (inc. up to 6 months maternity at full pay) so those raising families can do so while still working for us. We also extend our bereavement leave to cover pregnancy loss.
- Transparent pay - so people get paid in line with their ability and experience, not their negotiation skills!
- Proactive pay process - we review *everyone's* pay 3 times per year, so we don't reward people who loudly argue over those who quietly perform.
- Generous pay - so people can afford to work here and finance doesn't prevent this. We take the 50th percentile for any given role then add 20%.
- We write our policies up publicly, so we're accountable to the world, instead of hiding them. Hence this handbook! And anyone can edit it.
- A culture of transparent feedback that is constantly reinforced - this discourages gossip or playing politics, which is corrosive.
- Training budget, especially for those in roles where we don't have lots of existing experience as a company, to help people develop.
- Health insurance for those from countries that do not provide this freely.
- We pay people for SuperDays because we think this is the right thing to do and it enables those who could not otherwise take a day off work to participate in our recruitment process.
- Unlimited vacation policy with a mandatory minimum time off - so you can fit work around your life.

- We discourage heavy drinking at company events - you'll need to join another company if you're a bro, alas.
- Political issues can be more important to people than work, and they are frequently divisive and distract from our purpose. We therefore don't stand for any political cause and don't tolerate or allow political discussions in Slack, GitHub, or any other PostHog-owned or sponsored tools or events.
- We don't care about college degrees. We care about what you've achieved.
- We expect people to act kindly and inclusively towards each other. We take a very strict stance on people behaving inappropriately towards others.

Things we don't do that other companies might

We care about doing what works for PostHog's culture, rather than worrying too much about what other companies are doing or how they judge this. These are some of the things that we don't do as a result.

- We don't track the metrics for how many people are underrepresented at PostHog or in our application process because we don't want to optimize for these numbers when we know we offer a welcoming place to work. We used to do this and realized it wasn't helping us.
- Unconscious bias training. Our applicant data shows that under-represented groups are no more or less advantaged by our hiring process, and the effectiveness of such training is debatable.
- Making PostHog's culture the responsibility of the people or an 'HR' team - culture starts with the founders and executive team. Otherwise you end up with policies and actual behavior starting to diverge.
- Advertise on external job boards. We used to do this (and get 1000s more applications), but found we virtually never hired anyone who didn't apply directly to posthog.com or through referrals.
- We don't avoid doing business with certain customers except under very strictly defined exceptional circumstances. We allow the government to determine what is acceptable instead of getting into discussions about who we should deal with for each of our 100,000+ customers.

Are you a potential candidate reading this? Excited to join a grown up company? Get in touch!

Kudos

`contents/handbook/company/kudos.md`

Kudos

> Definition of 'kudos'
>
> (kjuːdɒs IPA Pronunciation Guide, US kuːdoʊz IPA Pronunciation Guide)
>
> *Kudos is admiration or recognition that someone or something gets as a result of a particular action or achievement.*

As an all-remote team, we need to put extra effort into celebrating each others' achievements, as not being in the same physical location can often make good work less visible.

We use Monday All-Hands as an opportunity to acknowledge cool things that people have done in the previous week. Can be *anything* - shipping a new feature, a great piece of content, fixing an issue, or just generally doing something nice for someone else.

How to give kudos

You can use `/kudos @person for [reason]` to give someone kudos in Slack whenever you want. The kudos gift won't be visible in the chat for anyone else, but the gifted person will probably enjoy seeing themselves in All-Hands on Monday.

To list all kudos from the last week, use `/kudos show 7`. This shows the previous 7 days of submissions.

Alternatively, you can just write directly into the All-Hands doc, though this relies on you remembering things that happened the previous week.

Lore

contents/handbook/company/lore.md

Lore of PostHog / inside jokes

A beginner's guide to some of our custom Slack emojis and various anecdotes you'll see and hear about.

- Yakko always had bad internet when demoing. *Always*.
- wore a skin tight green all-body suit for months to improve his Zoom background game without us realizing.
- has the same pose in 90% of PostHog photos. It's a reference to a meme.
- where X is a team member. Used in times of extremely impressive performance, unless used sarcastically.
- Mr Blobby. We once changed how we ingest session recording data, to use S3 blob storage. We called it Mr Blobby. Mr Blobby is a creepy '90s TV character from the UK. This project was nightmarishly hard, which is why this character was fitting.
- will make you eat gelato at every offsite.
- Sometimes people screenshot each other's faces and Zoom screens and use them as their backgrounds. Usually when an all-hands is too dry.
- wore a suit to his performance review. He is the only person in history to wear a suit to anything PostHog-related. Unsure if he was making a point, we later abandoned the practice of performance reviews regardless.
- We took lots of buses at an offsite in Portugal. The roads were incredibly twisty, the driver was in a bad mood, drove too quickly, and people threw up. It was bad.
- / A reference to Marie Kondo's book on tidying your house, generally used to describe things that are particularly good or bad from a user's perspective
- / We once made isgoogleanalyticsillegal.com when there were privacy rulings about Google Analytics. We put it on Hacker News, got the top of the front page, and it was our biggest *ever* day of signups at the time. The website was supposed to be tongue in cheek, but the internet took it seriously. The person in the emoji is Ursula von der Leyen, who introduced the GDPR legislation.
- IPO promises. There is a list of these that is brought out at certain moments. You may see.

- will train you on Post-it notes if you go to an offsite with him. Success of a good Post-it note posting is in the lift away from the surface – the most important thing is to peel off the Post-it note, as opposed to pulling.
- Three finger rule - another Marius invention, if someone holds up three fingers while you're talking, it means you aren't being concise enough. We don't actually use this much as it's predictably awkward and distracting, so ruins any meeting it could have otherwise helped.
- When we hit 10,000 GitHub stars, read every username on a live stream that took over six hours.
- We like to nail things. It's not uncommon to see a GitHub issue titled "Nail [feature name]". Sometimes we'll even assign an absurd version number like "3000". (The codename for the next generation UI of the PostHog app is referred to as PostHog 3000, and other projects have also adopted this naming convention as well.)
- once decided to go off piste in a new starters intro of all hands and asked the question “Do you moisturize?”
- has also gone viral multiple times for his tweets about "hopping on a quick call" and that is entirely what he is known for now.
- Everyone says "thanks dylan" to because he once had a really good all-hands demo.

Managers and management

contents/handbook/company/management.md

A manager at PostHog has a short list of responsibilities:

1. Setting the right context for your direct reports to do their jobs
2. Making sure your direct reports are happy and productive
3. Acting as the hiring manager for new roles in your team
4. Creating good plans for new person onboarding and small team offsites
5. Collaborating with execs on team performance concerns that need early intervention

That's it.

A manager at PostHog is *not* responsible for:

1. Deciding compensation - we have a compensation calculator and the process is managed by the exec team
2. Setting tasks for your direct reports - that is not how small teams work
3. Providing a career progression plan for your team
4. Figuring out team structure - today that is all handled by the exec team
5. "Approving," whether that's projects, expenses, days off, or accounts - people should have admin access by default to most things
6. Dealing with HR issues - you should escalate these to Fraser or Charles
7. Anything legal-related, e.g. someone wants to quit or thinks they did something illegal - route this to the exec team
8. Deciding to hire or fire people - the exec team do this

This guidance applies to all teams, irrespective of whether you manage an engineering or non-engineering team.

Part-time managers

Because of the relatively short list of tasks that managers have, management at PostHog is a part-time job. That means nearly everyone still spends the majority of their time on practising what they do best. For most managers, this isn't actually management!

As an engineer, you want the opinion of someone who can actually code. As a designer, you really want your manager to have an eye for design. As an operator, you want to be managed by someone who has scaled a business. That's why it's important for managers to keep practising their craft.

However, management tasks do come *first*, as giving context to your team tends to have a multiplying effect vs. getting one more PR out. After that though, it's back to work.

Management is intentionally spread thin at PostHog. This is a forcing function for making sure that teams and ICs continue to have high levels of autonomy. Bored managers are micromanagers. By working across several teams, people like #team-blitzscale and product managers are forced to only give their attention where it's truly needed, and give space & autonomy everywhere else.

> You'll sometimes hear us use the term "team lead". A team lead is the leader of a small team. By default they also manage the individuals that are part of their team, though very occasionally they don't, such as when a new small team has just been created.

How do I set context?

At PostHog, we hire highly experienced people for 99% of roles. That means managers won't need to spend time telling their direct reports what to do.

However, for those people to make the best decisions, they need context. The things a manager can do to set context include:

- Creating a roadmap that the team can work towards
- Helping the team level-up their understanding of your target customer and the problem space you are working in (eg by encouraging them to talk to users, and doing so yourself)
- Helping someone figure out who else to talk to within PostHog
- Enabling or encouraging the team to measure their impact
- Improving the process in which a team works (things like standups, reviews etc.)
- Organizing a team offsite or other meetup to work in person

Pitfalls to avoid

The biggest difference between PostHog and other places is that in the end it is up to the *individual* to make the decisions. All you can do as a manager is set context. From there, you'll have to trust that we've made the right hiring decisions and that the individual is able to execute on that. If they can't, we have a generous severance policy.

Decisions aren't just about buying a piece of software or choosing a color for a button. It's also about what to work on, what to invest time in, or where to take entire parts of our product.

As a manager, it's tempting to see yourself as the sole owner of all the information, and give it out sparingly. People will come to you often with questions (because they don't have the context) and when they do you'll get more validation that holding all the context yourself makes you an Important Person. What managers should aim for at PostHog is to make themselves obsolete. Share as much context as possible, in written form and in a public channel. That way everyone will be able to do their best work.

Ways to burn yourself out:

- Become the sole point of communication between your team and others. Instead, connect the right people together directly.

- Take sole responsibility for writing up the detailed plan for your team. Instead, set the vision/roadmap, then encourage your team to contribute objectives too.
- Move from IC to manager and just add the management on top of your existing work. Instead, you should cut your IC work down slightly to make room.
- Be the only person on your team who talks to customers. Instead, encourage everyone to do this - this starts at onboarding!

How do I make sure my direct reports are happy and productive?

First, make sure you are setting the right context. Next, the most useful thing you can do here is to schedule regular 1-1s. Typically we find that you should have higher frequency 1-1s with your reports when they join PostHog and reduced frequency over time as they settle in. There are some types that we've found useful:

- **When they start** schedule a longer 1-1 to get to know each other and set expectations on each other
- **During their probation period** have weekly 1-1s as a regular check in (this is an important time to be giving clear feedback about how they are doing)
- **After their probation** have bi-weekly or monthly 1-1s to discuss how they are doing outside of the regular day-to-day context

The key thing here is to be pragmatic - 1-1s should feel useful and not like a waste of time. Everyone should see it as their own responsibility to raise important feedback or issues as they happen and not wait unnecessarily for a scheduled meeting.

Talking about long-term career goals every now and again is also important but easy to let slip when things get busy. If you can help people achieve long term goals while at the same time hitting PostHog's short term needs - whether at PostHog or not - you'll get people's best work!

We have a set of handy templates to use - feel free to adapt these for each team member. These are not to be followed strictly if you don't want to - this is to just save you having to create something from scratch.

Performance

We care about having a consistent, transparent, and fair way to handle recurring performance issues. We don’t want this to be a source of stress for you - it’s not your core responsibility as a team lead, and we want you to feel supported. The People & Ops team will prompt you to consider performance within your team at key moments to make this easy and straightforward, but you should proactively give feedback and raise concerns with your exec as they arise.

- We expect you to regularly give proactive, actionable feedback to everyone on your team - it’s the most direct way to help troubleshoot issues upstream. This is particularly important at the 30-day, 60-day and 80-day check-ins after a new starter joins.

- The team lead will be asked to consider the following questions that are aligned with our values: i. Is the person a driver or a passenger? ii. Does this person get things done proactively? iii. Are they optimistic by default?
- We expect you to actively raise performance issues with your exec.
    - Once you do, your exec will take the lead on the process. You'll likely deliver feedback directly to the employee, but your exec will support and coach you through those conversations.
    - Your exec will look after the process and make any decisions required.
    - If it ever comes to someone leaving, your exec will work with the People team to handle it carefully, sensitively and fairly.

The keeper test

As PostHog grows, it's increasingly important that all team leads help us keep the bar for performance high - we can't centralize this with the founders. To help us scale this, each team lead will be asked to do a keeper test on their team members throughout the year, this will be sent in an automated form, by Deel, through Slack. The format is as follows:

1. Ask the team lead 'if X was leaving for a similar role at another company, would you try to keep them?' - the answers should be derived from our values, similar to the questions above.
2. Dig in where the answer is 'no' - what would it take for this to be a 'yes'? Is this just temporary, or is there a deeper issue to resolve?
3. Make sure the manager is sharing all of this feedback with their team to help them improve.

That form will be shared with the relevant team Blitzscale member, so they can help where necessary.

> Side note: anyone can ask their manager 'how hard would you work to change my mind if I were thinking of leaving?'. It's a great way to solicit valuable feedback!

Weave

We use a tool called Weave to collect stats for engineers. Engineers can log in to see their numbers and those of other engineers.

We understand that all the work an engineer does can't be properly represented in a tool that just looks at PR output. Data in Weave is *not* the decision-maker for whether someone is succeeding in their role at PostHog. It can be, however, a part of the conversation.

We use Weave to:

- Look for outliers in the company in terms of output (both high and low - sometimes unexpected people are rising to the top!)
- Watch for issues with overall team productivity to identify possible blockers
- Start conversations with team leads

We don't use Weave to:

- Make a decision to let someone go - if and when this happens, it only follows detailed discussion with and recommendation from the team lead
- Monitor your PRs - our management layer is stretched way too thin to micromanage this
- Creep on your use of AI - we don't care how or if you use AI to get a job done, as long as it gets done
- Make a call on how valuable someone is - some people with low PR output are very valuable to the company, and we're 100% aware that things like heavy support load can impact output

We have compared statistics in Weave against other (imperfect!) metrics that can be used to gauge productivity, such as number of commits, number of PRs, total github activity, etc, and see similar patterns amongst them. Weave gives us more detail and a nice UI for evaluating output across all the engineers we have, which we don't have any other good interface for. In addition, it gives engineers access to the same information we have about them, so using it increases transparency.

What does being a hiring manager entail?

Two things:

- You will conduct the technical interview by default. You'll also kick off the SuperDay with candidates, and be their main point of contact in Slack. Please help us keep hiring moving by giving feedback quickly!
- If you think your team needs someone, make a new hire request. The exec and people team are generally on top of hiring for all teams, but this is a good approach if you think something has been missed. You'll also be asked to do one of these anyway if we're hiring for a new type of role.

See the `#technical-interviewers` channel for more info here.

What you can expect as a manager

Management roles at PostHog are often (but not always) temporary. That's because as the company changes, our needs for different people in different roles will change as well. Because all of our managers are also strong ICs (individual contributors), sometimes putting someone back into an IC role makes sense if that's what's best for the company. This has happened many times with people at PostHog, some who have gone back and forth between being a manager and not being a manager multiple times (hi !).

As such, management roles are paid on the same pay scale as other ICs. Becoming a manager does not mean you get a pay raise, and going from a manager role back to an IC role does not mean you get a pay decrease.

Management is a skill of its own, and it's not any more important than any other skills that make someone a great IC. It's possible that you may be a manager for a short time, but it becomes clear that your strengths lie primarily in the other skills that are involved with being an IC. In this case we might move you back to a pure IC position, where your skills can really shine, and move someone else from your team or from around the company into the manager / team lead role.

Additionally, managers who are excelling with their teams may have limited interaction with their own manager. This is because, as discussed above, management is intentionally spread thin. If you feel like your manager is mostly ignoring you, this isn't necessarily a bad thing and usually means you and your team are doing a fine job!

Recommended reading

These have been recommended by multiple managers on the team:

- The Making of a Manager: What to Do When Everyone Looks to You by Julie Zhuo (great for first time managers)
- High Growth Handbook by Elad Gil (covers a lot of ground beyond management)

Engineering-specific:

- The Manager's Path by Camille Fournier
- An Elegant Puzzle: Systems of Engineering Management by Will Larson

Merch store

contents/handbook/company/merch-store.md

We have a merch store where our community can purchase high quality PostHog-branded merch. The People & Ops team is responsible for managing merch inventory, fulfillment etc. even though multiple people contribute, and Kendal is the point person.

We use Micromerch to manufacture and fulfill our merch. Anyone can suggest a product for us to sell or give away.

The Brand team ultimately decide on what items we wish to sell or give away (including how many and sizes), and Lottie provide assets to produce and order these items in to stock.

We generally try to launch new products in line with the typical fashion cycle (spring/summer and fall/winter). However, this doesn't mean we can't do fun side quests! If you are looking to do an off-cycle merch run, just make sure you keep Kendal in the loop so the admin side goes smoothly.

How to reorder merch

All of our permanent merch items are reordered via Micromerch. To do this you need to:

1. Request a restock quote for the item(s) in the Slack channel and enter the quantity you need
2. Approve the estimate that will be sent from Micromerch
3. Pay the invoice via Brex once it comes in (usually in 1-2 days after estimate approval)

It's really important that we do not allow stock levels to run low as restocking items can take a couple of weeks, so the Ops team will regularly check inventory levels. However if you happen to see anything looking amiss, or you know you want to place a big order for a customer that may affect our stock levels a lot, just let Kendal know ahead of time!

Adding new items

Micromerch is integrated with our Shopify store, so all orders are made and processed through there. To add new products to Shopify, follow these instructions..

Shipping

Shipping is also done through Micromerch (in partnership with Shiphero) - they can ship to over 200 territories worldwide:

- When orders come in from our Shopify store they will automatically be shipped to the people who order them via Shiphero
- If you want to ship merch for an event or as part of a giveaway, do this from the Shopify dashboard.

Merch giveaways

Customers

Create a discount code in Shopify admin. You don't need to be invited to Shopify, instead the login details are stored in 1Password.

When creating the discount, select "amount off products" then choose if it is a percentage off or a fixed amount - usually we do fixed amounts of $30, $50, or $100 depending on the purpose. The you can choose "specific collections" and choose "All Products".

> Limit the use to one use only (*not* one use per customer), otherwise it's unlimited free stuff for them, unlimited high cost for us!

For feedback or general rewards we typically give users $30, which is enough for a t-shirt. For code contributions we tend to do $50, which is enough for a bigger selection of things. We don't put expiration dates on the codes, typically.

If you need any help just send a message to the #merch channel and somebody will be happy to help. Merch codes can also be generated directly from within Zendesk.

If you want to send physical merch to a customer instead of a merch code, this can be done in Shopify by creating an order, selecting the chosen merch and applying a discount for the whole price of the item (don't forget to do this step otherwise it'll try and charge the customer!)

PostHog team

If you want more, here's how to get it!

As always, we expect you to use this with restraint and with your own good judgement. The merch store should not become your sole source of clothing for your wardrobe, nor where you go any time a friend has a birthday. But sure, go ahead and buy your mom (or yourself) a hat or a hoodie!

YC Deal

You can find instructions for this on the dedicated YC Deal page.

Troubleshooting customer orders

Sometimes customers get in touch with us because their order hasn't arrived. There are a couple of things you can do:

1. Check the Shopify. This will show you the status of the order, if something looks amiss, please mention this in the #merch channel immediately so Kendal can look into it.

> **Note:** There have been some issues with fulfilling orders to Brazil due to the country's customs policies.

If for some reason their second order attempt doesn't make it through, refund their money and apologetically let them know that unfortunately our supplier is having issues shipping to their address. It's better to stop the back-and-forth at that point, rather than having a frustrated customer placing multiple orders that don't work. We aren't an e-commerce business, so ensuring a flawless merch store experience for a handful of edge case orders is not a priority!

If the customer was given a merch code to thank them for submitting a PR, you can offer to make a donation on their behalf for the equivalent amount to a company of their choice on Open Collective instead.

A primer on using GitHub at PostHog

contents/handbook/company/new-to-github.md

> If you're new to GitHub, it can be a little confusing. (Heck, I've been using GitHub for years and it's *still* confusing.) It doesn't have the best search and notifications can get out of hand — and in general, it can be really intimidating to join a company that uses a tool you've never used before as its primary means of communication.
>
> I wrote this guide to help explain how we work, and how to stay on top of the volume of information that flows through our team's organization on GitHub.
>
> —
>
> P.S. Have questions? Feel free to file an issue on GitHub - I explain how to do this later in the article!

Key concepts

At its core, GitHub essentially hosts code that helps keep everyone in sync. Each team member can download this code, make changes, and upload their changes back into GitHub.

Code is stored in a "repository" (or "repo" for short) - it's like a folder for code. (As of writing this, PostHog has 401 repos - like the code for posthog.com and even a repo for internal company discussions that doesn't actually contain any code.) This is because each repo comes with a handful of collaboration tools. Here's a list of the key concepts on GitHub:

1. Discussions
2. Issues
3. Projects
4. Pull requests
5. Actions

You can take any task linearly from start to finish using this set of tools, though you don't have to use them all. (For example, PostHog doesn't really use Discussions, and Projects are only used by certain teams.) But if you wanted to use the whole suite, here's how it would work:

1. If you decide you want to change something in the product or website, you could start a *discussion* about it. This is like a casual forum-style conversation. (Again, we don't use these.
2. A discussion can be converted to an *issue*, which is a formalized proposal of the discussion.) People can reply to these posts with feedback.
3. In my workflow, this is a good time to add the issue to a *project*, because it's something you want to track through to completion. Project boards are a great way to stack-rank tasks (issues), because you can order them in a way that makes sense based upon the project and see everything in one

place. This helps keep a team in sync.
4. A *pull request* (also known as a *PR*) references the code that's changed to solve an *issue*. It's a way to summarize the changes in code and explain them so others can review them.
5. *Actions* usually occur after you commit code. It makes sure things are working as expected (and that whoever wrote the code didn't break anything). (Don't worry about these for now.)

You can use any of these features on their own, or use them together. Primarily, PostHog uses issues, pull requests, and actions. If you're not super familiar with GitHub, just focus on issues and pull requests, as that's where the bulk of the interesting work happens.

Note: The PostHog handbook covers GitHub issues and pull requests, and suggests everything should start with a pull request because it represents one of our values, "Why not now?".

Notifications

The best way to stay up-to-date with what happens on GitHub is by subscribing to (following) the areas that are most relevant to what you do. This sends updates to your GitHub notifications.

By default, you'll receive email notifications for everything you subscribe to. There are a few ways this happens:

1. Creating an issue or pull request
2. Commenting on an issue or pull request
3. "Watching" a repository

As I'm not a huge fan of email, I prefer to visit a centralized place for my GitHub notifications, although many engineers prefer email notifications. Personally, I don't like GitHub's /notifications page, as it feels cumbersome (slow) to read through updates. Here are two much better ways to consume GitHub notifications (entirely my opinion):

1. GitHub's iPad app - provides an email-like interface that feels a lot more natural to reading notifications than github.com/notifications. (If only GitHub had this UI on the web...)
2. octobox.io - uses the same email-like interface, but in a browser

I have octobox.io set to my homepage in Chrome, so anytime I want to see my notifications, I just click the Home button and I have one-click access to my work "inbox".

Install the GitHub app in Slack

A great way to get realtime updates about what's happening in GitHub is to install the GitHub Slack app and subscribe to repos. After linking with your Slack account, type `/github subscribe posthog/posthog.com` (org/repo-name) in Slack, for example, to get updates when things happen in the posthog.com repo.

Finding issues or pull requests

Given the volume of issues and PRs, search will be your best friend. Unfortunately GitHub's global search leaves something to be desired, so usually the easiest way to find something is to visit a repo, then clicking either Issues or Pull requests (depending on what you're looking for) and searching from there. Type a few keywords, and if you know who authored the issue or PR, apply an author search. (You'll see GitHub pre-populate search syntax (eg: `is:open is:issue author:corywatilo`), similar to how Gmail's search works.

Filing an issue

Issue is the primary method of getting a message in front of the team. Think of it like creating a ticket in a typical project management system. (We prefer issues over Slack messages because it's public and can sync with the rest of our code workflow. You can use Slack if you'd like to bump an issue to a group of people, but link to the issue (or PR) as GitHub acts as our source of truth.)

Issue templates

Some repos have issue templates set up to make issue creation faster. However, if the issue you're going to create doesn't fit into one of these templates, don't worry about these! Just create a new blank issue.

Referencing another issue

This isn't mandatory, but if your issue is related to other (previous) issues, it's worth cross-linking so others have full context. To cross-link in an issue or PR, type a # and either part of an issue's/PR's name or number and GitHub will populate a list of items that match.

You can find an issue's or PR's number in the URL.

Writing Markdown

It can take some getting used to if you've never written Markdown syntax. Fortunately GitHub makes it easy by providing WYSIWYG buttons. When you press a button like `B`, `I`, or `U`, GitHub will insert the Markdown code required to format your text accordingly.

Tips for faster writing

- You can use keyboard shortcuts like you would in a word processor.
- Quickly insert a link by copying it to your clipboard, selecting the word or phrase you'd like to link, then using `Cmd + V`. GitHub will automatically convert the text into a link.
- Create a checklist by typing `- [ ] Your text`. You (and others) can check things off of this list after the issue/PR is created.
- Paste an image from your clipboard directly into an issue/PR. It's much faster than attaching from your computer. For example, if you're screenshotting something on a Mac, use `Cmd + Shift + Ctrl + 4` to select part of your screen, then `Cmd + V` into an issue. GitHub will upload the image

automatically and add the Markdown embed code for you. Voila!

Creating a pull request

If you see something minor on posthog.com (in Handbook or Docs) that needs to be updated, you can easily propose the change by creating a pull request *without* having to run the full codebase on your computer. (This is a great way to contribute if you're in a less-technical role.) To make a small change, find the *Edit this page* link within the Handbook or Docs which will take you to GitHub where you'll see the source file. From there, click the pencil icon. (Our Handbook and Docs use the same Markdown format as GitHub's issue and PR editor, so this should look familiar!)

When you're done making your changes, be sure to preview what the changes look like (to make sure formatting is accurate). At the bottom of the page, you'll see a section called *Commit changes*. Here's how to use it:

- Briefly describe the change you made in the top line
- Optionally add a more detailed description
- Choose "Create a new branch..." and optionally give it a name (but not required)

Clicking *Propose changes* will create a pull request!

"Closing keywords"

If you're changing code to address an open issue, you can tell GitHub to automatically close the issue when the PR is merged by using a closing keyword. For example, in your PR description, you can write "Closes #123" (where #123 is an issue number).

Requesting a review

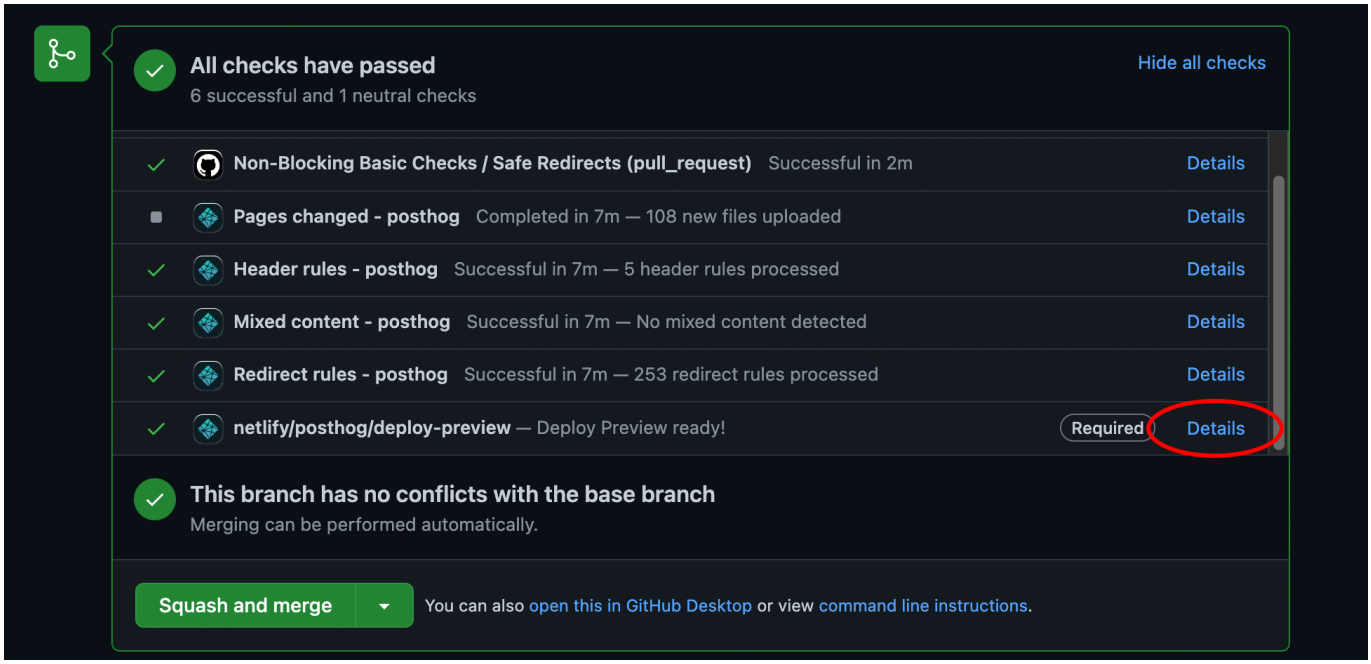
Now that your PR is created, you can request a review (best practice) from someone relevant so they can make sure everything looks good and that they agree the change is ready to go live. They'll be notified of your request. (By the way, you can filter to reviews that others request from you by going to your notifications, then choosing the Review requested filter.)

Previewing changes

If you're making changes to posthog.com, you'll be able to see your changes on a "preview" version of the website. It takes 10-20 minutes for this preview to be ready.

(Remember when I said we also use GitHub Actions? It basically runs some automated tests to make sure everything is spelled correctly and that nothing else broke.)

Near the bottom of a pull request page, you'll see a box like this:



(Note: This box only appears if you're a member of the PostHog GitHub org - it's not available to the public.)

You can click the *Visit Preview* link in the Vercel bot comment to see the preview.

Merging changes

Once a team member approves your pull request, you (or they) can publish the changes by clicking the *Squash and merge* button. It will take another 10-20 minutes for your changes to appear on the site, but they'll go live automatically. At that point, you can send a link to your friends and family and tell them you're a coder now!

Next steps

This was a primer on using GitHub for communication at PostHog. If you're interested in making more substantial changes to the website, you can follow our instructions on how to develop the website. It can take a little work to get your computer set up to run the site from your computer, so don't hesitate to reach out for help if you get stuck – or don't even know where to begin. That's what we're here for!

Offsites

contents/handbook/company/offsites.md

While we're async by default, there's a very real upside to being in the same room - we've consistently found that a lot of our best ideas come from actually building things together in real life.

We understand organizing travel can be a challenge when you have personal/family commitments to manage, so we try to take a balanced approach to meetups:

- Once a year: all-company offsite
- Once a year: Small Team offsite (app and platform teams do this as a single combined offsite)
- Occasionally: in-person onboarding
- Whenever you like: in-person meetups

All-company offsites

> Curious about our all-company offsites? Check out these links:
>
> - **2021** We shot this video at our Portugal offsite
> - **2023** What we built at our sun-kissed Aruba hackathon
> - **2024** What we built at our windswept Mykonos hackathon

Once a year, the entire company will get together somewhere in the world for a week. Usually we'll all fly on Sunday, have an opening dinner, spend the week doing a mix of hard work, strategy, culture and fun activities and we then all fly back home on Friday. Our past offsites have been in Italy, Portugal and Iceland. We try to ensure that everyone has their own bedroom.

These are organized by the Ops & People team, and we budget up to $3,000 per person in total for these.

Typical agenda:

- A couple of structured social events
- Team dinners
- 24hr Hackathon
- All-hands strategic sessions and workshops
- All-hands culture exercises
- A small amount of downtime so people can explore

Small team offsites

We want to try to encourage small teams to get together once each year. These are more focused on work and on creating strong bonds within teams. Ideally they are spaced appropriately through the year in relation to the all-company offsite.

Planning a small team offsite? Kendal's got you covered. Here's how it works:

- The team lead should message Kendal with their proposed dates and location.
- Kendal will then go away and find suitable accommodation for the group.
- Once that's sorted, she'll:
    - Create a Slack channel for the offsite (use `#offsite-[team]-[month]-[year]-[where]`) and add everyone who's going.
    - Set up the offsite budget in Brex.
    - Update the team's Canvas with:
        - Accommodation details and a handy map
        - A flight tracker
        - A rough itinerary (Kendal will include the mandatory bits, like 360° feedback/Readme sessions, team leads can fill in the rest)
    - Look up dining options for a couple of group dinners (we like to keep a few nights free for "choose your own adventure" dining).
    - Suggest and book a whole group activity for everyone to enjoy together. If there's anything ad hoc you'd like Kendal to take point on, just let her know, she's happy to help! Each team member is still responsible for booking their own flights.

Some guidelines:

- Quarterly planning is a great focal point for team offsites – it's worth scheduling your meetup for the week of planning.
- Outside of your small team, you should only invite people who actually need to attend to make the offsite a success - if it would be 'nice to have' them attend, they shouldn't be going.
- It can be useful to combine offsites but beware if you add too many people everything gets harder to arrange. There's no hard and fast rule here but the more people that are attending the more concrete you need to be on who is organising and why people are attending. It will be more work and you should be purposeful about it.
- If the number of attendees is >10 actively consider if Kendal should be attending so there is a person attending whose only focus is making sure it goes well and everyone else can focus on the work of the offsite.
- Specify offsite start and end times down to the hour, for clarity and efficient use of everyone's time.
- These offsites don't happen very often and involve a lot of travel, so make sure you make the most out of it by having an agenda and an idea of what you want to achieve *before* the start of the trip. Also, it's a good idea to have an expectation-setting session (can be async in a Figjam) to ensure everyone is on the same page about what the outcome/output of the offsite should be.
- Choose a location that minimises layovers for attendees.
- Make it very clear who is participating in each session. Sessions / activities require full participation from attendees, especially for the likes of a hackathon given it runs over multiple days. Ideally one person should be responsible for the agenda and run a kick-off at the start of the hackathon.

- You should do a 360 degree feedback session. It can feel uncomfortable doing these, but almost everyone who's done one at PostHog has come out feeling better and with a whole host of things they can improve. These are best in person.
    - This can work better over a shared cooked meal or takeaway in the accommodation rather than a noisy restaurant, particularly for people who might be anxious about the format or the feedback.

Ideas for the agenda:

- 360 degree feedback session (mandatory!)
- A spoken README session early in the week to share "Who am I/How I work best"
- Planning session – what does the team want to achieve in the next month/quarter/year?
- Look at the team page - what needs to be updated?
- Dogfooding session – set PostHog up in a toy project from scratch, looking for pain points
- Hackathon - try to leave 2 days for this, and most importantly avoid sessions interrupting hacking
- Even some regular work on ongoing challenging projects - this is the best time for exchanging knowledge!

> Don't run a hackathon during an onboarding offsite. Other offsites normally do have a hackathon. Participation should be *very* strongly encouraged but not mandatory - if not everyone is taking part make sure that working spaces are available to accommodate the different styles of work. It is super important that people taking part are fully available and focused on participating. Given the offsite is an opportunity to work together there should be no teams of one. This extends beyond the formation of teams and into the hackathon itself in cases where there is team switching.

Here's a real-world example: Product Analytics team's Munich offsite agenda (internal Slack link). Feel free to take inspiration – though your team's needs and wants might be quite different!

The budget for these trips is up to $2,000 per person in total. We ask team members to use their best judgement for these and try to be thrifty where possible - these should be enjoyable, but not feel like a holiday. Generally it's easier to hit budget if you have people travel in on a Monday and out on a Friday - they don't need to be as long as a whole team offsite.

You should assign someone on the small team to be responsible for planning the offsite (doesn't have to be the lead), and they will be supported by the Ops & People team to ensure a successful experience.

> On occasion during busy hiring peak time, we do recommend any team member involved in the interview process to dedicate at least one hour block per day during the offsites to accommodate candidate interviews so that this does not delay the hiring process on your team while you're away. Please coordinate directly with #team-talent if you have additional questions.

Hedge House

PostHog runs two Hedge Houses in the UK - a small one in Cambridge and a larger one in London. They are actual houses (yes, with a few bedrooms attached!) designed for small teams to run their offsites, host in-person onboardings, or come together for larger internal events like hackathons. Anyone at PostHog is welcome to use them as much as they like. We'd recommend using the Hedge House for small team offsites if you are in Europe as it removes a lot of the friction of finding somewhere new, and they're genuinely great places to get work done at a very high standard.

Cambridge

Message to check availability or make a booking at the Cambridge Hedge House.

London

Our light-filled, studious office is a reliable homebase between Farringdon and Barbican. It's entirely ours, open 24/7 and the perfect place to stay if you're visiting from abroad. Use the Hedge House London slack tool to see the full address, book a room and/or desk, plus see who else will be there during the week you visit. This means you can easily self-serve, but ask with any questions.

> If you're planning an offsite or onboarding in San Francisco, Hogpatch is the perfect spot to focus, talk to users and get product feedback.

London hotel recommendations

For offsites and onboardings in London, below is a list of hotels recommended in our #London Slack channel by folks who have stayed at these hotels.:

- Marrable's Farringdon hotel
- The Zetter Clerkenwell
- Hotel Indigo Clerkenwell
- Yotel London City
- Z Hotel City
- citizenM Tower of London hotel
- Clayton Hotel - City of London
- Hampton by Hilton - London Old Street
- hub by Premier Inn London Clerkenwell hotel
- Bob W Tower Hill Studios
- Ruby Stella hotel

If hotel prices are above £200 per night, it is worth quickly looking for alternatives as ~£170 per night should be achievable midweek in London. If prices are high, you should optimise travel for total cost (flights & accom) so if you can get cheaper flights or hotel by moving dates +/- 1 day, then look into these options.

Border Control

Quite often you will be required to travel to places where some kind of visa is required even if just a visitor visa like an ESTA. When entering places like the US, for work purposes, border control agents may ask the purpose of your trip. In these instances it's best to avoid using PostHog terms like "onboarding" as this can be confusing. It's much better to more generally describe the purpose of your trip. In nearly all circumstances this will be to hang out with your colleagues and to take part in team building exercises. It's usually good to emphasize that you'll be on a short trip and that the company is paying for everything. You should be prepared with the exact addresses of where you are staying and the details of your flight out of the country.

A successful strategy is usually to start off with a high-level purpose of your trip which is usually something like "hanging out with colleauges" or "I am here for a business meeting with colleauges", it is also usually advisable to only respond with a minimal amount, saying only what is necessary. If the agent asks for more details it's usually good to go into a bit more detail about the company structure "I work for a US tech company and I am based in [Insert your country] where I work remotely. I am here to do some in-person meetings with my colleagues for the next few days and I fly back on [insert date]". Sometimes the border patrol agent will ask more about the business, it's fine to give these details and be as honest about that as you would anybody else. If further details are required of the content of the trip, you can again give some context of how we like to lean into the benefits of in-person working and since most of your colleagues are based in the US, you are travelling their for a few days to meet in-person and will be returning home afterwards.

For all company offsites, it's best to describe this as a company gathering where you will be hanging out with colleagues for the week. Generally, it is best to avoid using the phrase "training" as this can also be confusing.

Travel insurance

Many of our company offsites involve team members traveling abroad, and although we hope that these trips are uneventful and safe for all, in the event of an accident or medical emergency, we carry travel insurance through as well as general & auto liability policies through our partner Embroker.

In the event of an emergency, please cover any related expenses (ideally on your company card) and keep receipts, and then reach out to Kendal as soon as possible. We will assist with making a claim based on our policy binders.

Flight delays

If your travel plans are affected due to a flight delay or an airline-induced missed connection and you are forced to stay somewhere unplanned overnight, push the airline to cover the cost of your accommodations (including meals). It's not uncommon for them to initially tell you they no longer offer free hotel rooms for delays that were caused by the airline, but with a little bit of polite coaxing, they will likely give in.

Partners / family joining offsites

Sometimes at PostHog you will be asked to travel to places you've never been before and it could be a good opportunity to travel with your partner / family. At PostHog, we do infrequent in-person work that we want to maximise this time in person and your focus to be on PostHog, without distraction. This is why we don't allow partners or family to join you for the dates that the offsite / onboarding takes place. If timing allows it, you are able to tack holiday onto either side and for your partner/family to join you for those dates. However, for the dates of the offsite, you should be staying alone and focusing on your time with your teammates.

How to plan an offsite in 8 weeks - a checklist

Below is a rough timeline for planning your next offsite, as well as links to templates and resources that you can repurpose and customize as needed. Here's a spreadsheet template you can use with your team to democratically vote for the meetup location, and in other tabs, include travel information (in case someone's flight gets delayed/cancelled), schedule, project ideas, team activities, etc. To use any of the templates, create a copy to your own drive and edit as you see fit.

8 weeks out

- [ ] Choose dates
    - Try to avoid public holidays and be mindful of proximity to other offsites to minimize travel fatigue
    - School holidays are more difficult for people with children
    - Be mindful of the season of your chosen location, as this will dictate what activities you can plan
    - It is worth getting people to hold dates as early as possible, even before you've selected a location
- [ ] Choose location
    - Consider choosing a location that is relatively easy for most people to attend without having to travel outrageous distances or deal with difficult visa processes.
    - Transportation to the offsite is usually one of the larger budget line items, so do some research on the cost of flights from team locations before finalizing a location.
    - Consider the cost of transportation to/from the airport, and around town, for when your team arrives at the offsite location.
    - Be mindful that people who take long flights won't appreciate a 2hr journey from the airport to accommodation!
- [ ] Announce to team
    - You can have fun with this and build excitement by progressively dropping hints and having folks try to guess the location
- [ ] Create an offsite Slack channel and invite team
    - This is super useful for making announcements and keeping the team updated throughout planning

7 weeks out

- [ ] Secure accommodations
    - The ideal location will depend on the size of your team
    - We recommend booking a large Airbnb for teams under 10 people as this provides for a more casual atmosphere and can help control costs if you opt to have the team cook meals together. You can also book an Airbnb and supplement with a nearby hotel if the Airbnb doesn't have enough rooms.
    - For larger teams, consider a centrally located hotel that has a bookable space with configurable furniture for different activities, an onsite restaurant or bar to simplify meals and provide a location for free social time, as well as amenities like a gym for those who like to stay active while they travel
- [ ] Start flight booking process
    - To simplify this process, we give all team members access to a company card, and we ask people to book their own flights
    - We strongly recommend this approach as centralizing flight booking can be a huge headache for offsite lead, and this allows team members to accurately enter their personal information including airline frequent flier and trusted traveller numbers
    - Encourage folks to buy flights early and with the option to refund if they are unable to attend to save on costs. Use your judgement here - 20% or so is reasonable for the flexibility, but double the price is not really worth it.
    - In the event that a new team member will be attending an offsite, but has not started yet, please contact the Ops team to help coordinate. In these cases, the process is:
        1. Preemptively create the new team member a Google account
        2. Issue them a Brex card to their work email with a sufficiently high temporary balance to cover travel costs
        3. Add them as a guest to any planning Slack channels and/or share any necessary itinerary information such as arrival dates/times and airports
        4. Have the new team member book travel as usual
- [ ] Draft rough schedule
    - Building the schedule as a separate Google Calendar that can eventually be shared with the team will allow you to flexibly move sessions around as you finalize the itinerary
- [ ] Send info gathering form
    - Use this form to collect important information like flight details, clothing sizes for merch, dietary restriction, and preferences for things like sharing rooms
- [ ] Brainstorm merch/meeting room decorations (generally just for company-wide offsites)
    - Having some branded merch to commemorate the offsite upon arrival is a great way to welcome people and get them excited for their time together (generally this is for whole team offsites only for budgetary reasons)
    - If you are staying in a hotel, decorating a meeting room makes it feel much more personal and less corporate
    - In the past, we've done shirts, hats, scarves, notebooks, stickers, pens, and water bottles -- feel free to get creative with it

6 weeks out

- [ ] Draft budget
    - Using the tentative schedule, you can begin to estimate a rough budget to guide planning. Here are some benchmarks to use, however these will vary a lot based on location, size of team, and how cost-constrained you are:
        - Accommodations = $200/night/person
        - Intercontinental airfare = $1000/person; intracontinental airfare = $500/person
        - Ground transportation = $50/day/person
        - Food & drinks = $50/day/person
        - Contingency = 10% of total budget
- [ ] Secure transportation
    - Book any group transportation like coaches or a rental car ahead of time
- [ ] Finalize team outings and book as needed
- [ ] Flights booking deadline
- [ ] Info gathering form deadline

5 weeks out

- [ ] Assign rooms
    - Using preferences from the info gathering form regarding sharing rooms, assign accordingly
- [ ] Secure visas where necessary
- [ ] Review and finalize merch proofs
- [ ] Finalize itinerary
- [ ] Finalize budget
- [ ] Assign session leads & secretaries
    - Decide in advance who is going to record the notes from each session - it is *really* easy for post-offsite followup to fall through the cracks
    - With the schedule finalized, assign folks as session leads to design session plans

4 weeks out

- [ ] Reserve restaurants for communal meals
- [ ] Book any remaining activities
- [ ] Order merchandise
    - Important to give yourself lots of lead time here, in case of production or shipping delays
    - Depending on how restrictive customs are for your destination, consider shipping merch to teammates and checking them as additional bags on flights to the offsite

3 weeks out

- [ ] Draft session plans/presentations due
- [ ] Draft offsite guide package
- [ ] Design and print superlatives/awards

- One activity we've found quite popular is giving out superlative awards to the team during the closing ceremonies

2 Weeks Out

- [ ] Review and finalize session plans/presentations
  - We recommend having the offsite lead connect with session leads to review their plans and offer feedback before finalizing them - you want to make the most out of your sync time together
- [ ] Block your calendar and send a reminder in Slack for other team members do the same, to avoid interviews and other meetings being booked during key sessions or at times incompatible with the new timezone. The offsite calendar event needs to be marked as "busy" to prevent others from booking over it, which means changing the default for all-day events.
- [ ] Designate someone to bring or organise some of the essential supplies you expect to need for the week. At a minimum have post-it notes (don't skimp on cheap ones that fall off the wall), sharpies, a HDMI cable and / or a Chrome Cast.

1 Week Out

- [ ] Final plan review
  - The offsite lead should do a final, thorough review of the full plan and finalize any outstanding details - visually walk through the entire schedule and see if anything is missing
- [ ] Unveil the final offsite guide to the team

1 day before

- [ ] Add your new timezone as a secondary timezone in Google Calendar and check the 'Ask to update my primary timezone to current location' option. **Don't** update your primary timezone to the new one, as this causes issues for interview scheduling.
- [ ] We recommend that the offsite organizer consider arriving a day early to prep for the team's arrival
- [ ] Shop for any miscellaneous supplies and groceries (onsite)
- [ ] Print and organize a few paper copies of the itinerary (onsite)
- [ ] Create "Careless Whispers" envelopes (onsite)
  - This passive activity involves creating an envelope for every member of the team, posting them in a central location, and encouraging folks to write small notes to each other. These can be anything from retreat memories to compliments, and make a really nice memento to remember the offsite
  - Consider also making and posting envelopes for folks who cannot attend

1 week after

- [ ] Collect post-mortem feedback from the team
  - We generally do this as an open GitHub issue, but you can also create a Google form to facilitate this

All company offsite hackathon

The hackathon is always a highlight of the offsite. We tend to run them like this:

Session 1: ideation dinner The day before the start of the hackathon we do a casual 'ideation' dinner where we encourage people to chat about ideas

Session 2: hackathon kick-off The hackathon kick off is 1.5 hours at the end of the day. Ideally we do this in a conference room with beers and wine.

- Everyone writes down their ideas on multiple post-it notes in about 10 minutes.
- People come up to the front one-by-one, and they get 30 seconds to pitch all their ideas.
- Everyone gets three votes to put on whichever ideas they like most. Just do this as a tally. You can't vote on your own idea
- We dismiss all ideas with zero votes, and sort all the other ideas from top to bottom based on the number of votes.
- Everyone writes their name on the other side of the sticky bit of a small piece of post-it note, then add their name to the idea they want to work on. Every group should have at least two people in it, and ideally 3.
- Once groups are formed, everyone can go off and ideate or hack.

Session 3: presentations

This should be the last work related session of the offsite. Again ideally in a conference center with beer and wine provided.

Each group gets 5 minutes to demo and present their idea.

2 weeks after

- [ ] Compile post-mortem feedback and share with team
- [ ] Host post-mortem meeting to discuss any outstanding issues (optional)

Feature flags service outage

`contents/handbook/company/post-mortems/2025-09-29-flags-is-down.md`

Internal post-mortem: https://github.com/PostHog/incidents-analysis/pull/120

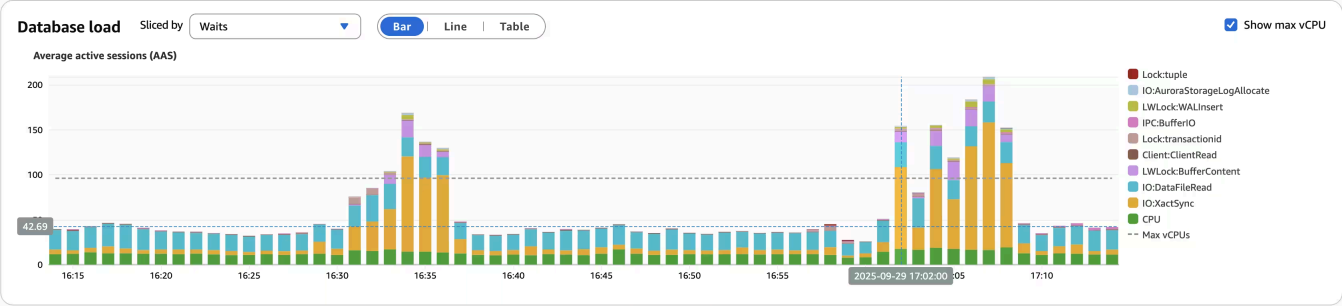
On September 29, 2025, the PostHog Feature Flags service experienced an outage lasting 1 hour and 48 minutes, from 16:58 to 18:46 UTC. During this period, approximately 78% of flag evaluation requests in the US region failed with HTTP 504 errors.

Summary

A database connection timeout reduction from 1 second to 300 milliseconds coincided with elevated load on our writer database from person ingestion. This combination triggered cascading failures in our connection retry logic, resulting in a service-wide outage. Recovery was significantly delayed by hardcoded configuration values and procedural failures in our incident response.

Timeline

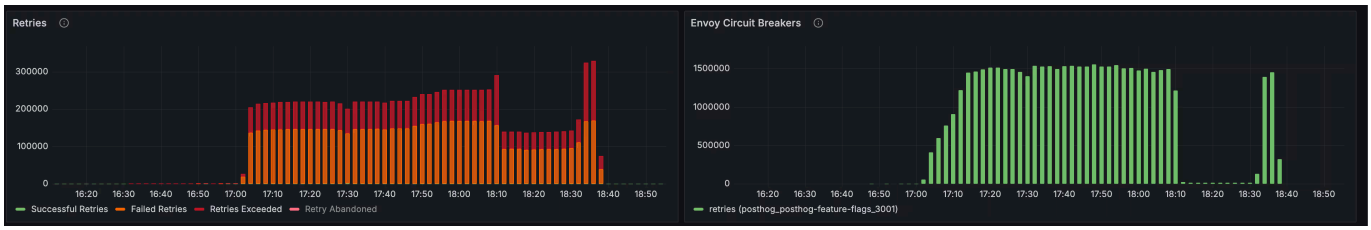
- **16:58 UTC** – Writer database begins experiencing connection saturation from person ingestion post-deployment workload



- **17:02 UTC** – Unrelated deployment reduces database connection timeout from 1s to 300ms
- **17:05 UTC** – Initial pods begin failing database connections and entering crash loops

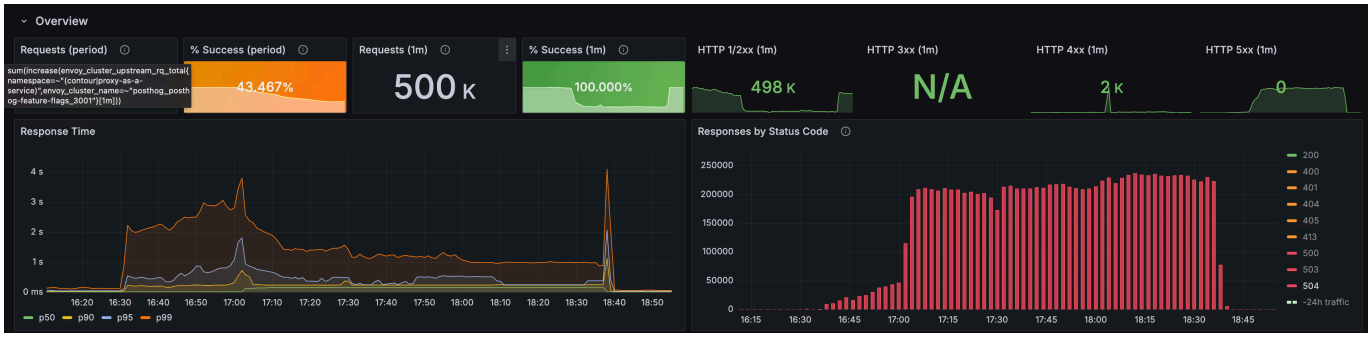
```
{"timestamp":"2025-09-29T16:54:16.154136Z","level":"ERROR","fields":
{"message":"Failed to create database pools","error":"Database error: pool timed
out while waiting for an open
connection"},"target":"feature_flags::server","threadId":"ThreadId(1)"}
thread 'main' panicked at feature-flags/src/main.rs:119:5:
internal error: entered unreachable code: Server exited unexpectedly
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

- **17:12 UTC** – Retry amplification begins overwhelming the writer database



- **17:25 UTC** – Incident declared, rollback attempted
- **17:40 UTC** – Rollback fails due to ArgoCD configuration issues
- **18:15 UTC** – Manual configuration changes deployed
- **18:46 UTC** – Service fully restored

Full service degradation timeline



Root cause analysis

The outage resulted from three compounding factors:

1. **Configuration change timing**: A connection timeout reduction deployed during a period of database stress created conditions where pods could not establish connections within the new timeout window.

2. **Retry amplification**: Our retry logic lacked circuit breakers and exponential backoff, causing failed connection attempts to multiply rapidly. This transformed a manageable database load issue into complete service unavailability.

3. **Health check configuration**: Kubernetes continued routing traffic to pods in crash loops for up to 45 minutes due to improperly configured liveness and readiness probes.

The incident duration was extended by operational failures: timeout values were hardcoded in the application rather than externalized as configuration, requiring a full deployment cycle to modify. Additionally, our standard ArgoCD rollback procedure failed due to misconfigured permissions.

Impact

- 78% of feature flag evaluation requests failed in the US region
- All flag types were affected, including read-only flags that did not require writer database access
- Customers experienced HTTP 504 errors regardless of their specific flag configurations

Remediation

Immediate actions (completed)

- Database connection timeouts moved to runtime configuration
- Timeout values increased to accommodate peak load scenarios

Short-term improvements (Follow along here)

- **Read/write path separation**: Implementing distinct connection pools and failure domains for read-only operations versus write operations. Read-only flag evaluations will continue functioning during writer database issues.
- **Circuit breaker implementation**: Adding circuit breakers with exponential backoff to prevent retry amplification during connection failures.
- **Health check optimization**: Configuring aggressive liveness and readiness probes to remove failing pods from rotation within seconds rather than minutes.
- **Rollback procedure documentation**: Creating detailed runbooks for ArgoCD rollbacks with proper permission configurations and validation steps.

Long-term improvements (Q4 2025 – Q1 2026)

- Development of specialized tooling for rapid pod termination during incidents
- Comprehensive load testing to validate connection pool behavior under contention
- Quarterly incident response drills to ensure operational readiness

Lessons learned

This incident highlighted critical gaps in our defensive architecture and operational procedures. The coupling of read and write operations created unnecessary failure domains, while our retry logic lacked basic protective mechanisms against amplification. Most significantly, our incident response was hampered by inflexible configuration management and untested rollback procedures.

The architectural improvements underway will provide proper isolation between different operational modes of the feature flags service. This separation, combined with improved circuit breaking and configuration management, will prevent similar cascading failures in the future.

Surveys SDK bug

contents/handbook/company/post-mortems/2025-10-03-surveys-sdk-bug.md

On October 3, 2025, a backwards compatibility issue in the PostHog Surveys SDK (version 1.270.0) caused widespread JavaScript exceptions for customers using SDK versions older than 1.257.1. The issue lasted 5 hours and 26 minutes, affecting 305 teams and disrupting both survey functionality and error tracking metrics.

Summary

A backwards compatibility break in SDK version 1.270.0 introduced a dependency on the `isDisabled` function from the `PostHogPersistence` class, which was only added in version 1.257.1 (July 2025). The issue manifested when the asynchronously-loaded survey extension attempted to call this function on older SDK versions where it didn't exist, causing JavaScript exceptions in customer applications. The incident was initially detected through customer support tickets rather than automated monitoring, leading to a 4+ hour detection delay and extended customer impact.

Timeline

All times in UTC.

- **10:45** – SDK version 1.270.0 deployed to production with backwards compatibility issue
- **14:59** – Support engineer notices two similar customer reports in tickets
- **15:25** – Issue confirmed by surveys team. Begin reverting suspected PRs: Backend PR and SDK PR
- **16:00** – Error Tracking team independently notices spike in exception throughput
- **16:11** – Reverts deployed, issue mitigated. SDK version 1.270.1 released
- **16:25** – Formal incident declared retroactively
- **16:45** – Customer communications sent to affected teams
- **22:28** – Incident closed, post-mortem phase begins

Total impact duration: 5 hours 26 minutes (10:45 – 16:11 UTC) **Detection delay:** 4 hours 14 minutes

Root cause analysis

The culprit PR introduced the backwards compatibility issue.

The technical problem

The PR modified the surveys SDK to use `posthog.persistence` instead of accessing `localStorage` directly – a reasonable architectural improvement. To ensure backwards compatibility, the code needed to check whether `posthog.persistence` was available before attempting to use it.

The implementation used the `isDisabled` function from the `PostHogPersistence` class, adding a utility in `survey-utils.ts` to verify persistence availability. However, this function was only introduced in a PR merged on July 11 and first made available in SDK version 1.257.1.

Why it failed

When PR #2355 was merged, both the main SDK code (`posthog-surveys.ts`) and the extension code (`extensions/surveys.tsx`) relied on the `isDisabled` function.

For the main SDK bundle, this worked correctly – customers on older versions never loaded the new code containing the reference to `isDisabled`.

However, the survey extension creates an asymmetric loading scenario:

1. The customer's application loads the SDK at whatever version they have installed (potentially months or years old)
2. The survey extension is loaded asynchronously from our CDN and **always downloads the latest version**

This created a version mismatch where:

- The old SDK (< 1.257.1) didn't have the `isDisabled` function
- The new extension (1.270.0) expected it to exist
- JavaScript threw `TypeError: isDisabled is not a function` exceptions

Why it wasn't caught

1. **No version compatibility testing**: We lack automated tests that verify new extension code works with older SDK versions
2. **Code review gaps**: We don't have a process to flag when new APIs are added to main SDK files that might be called by extensions
3. **No static analysis**: No linter rules prevent extensions from calling functions that may not exist in older SDK versions
4. **Detection gaps**: No monitoring alerted us to the spike in customer-side exceptions – we learned about it from support tickets

Impact

Severity: Major (High Impact, Service Degradation)

Affected customers: 305 teams running SDK versions < 1.257.1

User-facing impact:

- All Survey functionality completely broken (surveys failed to load or display)
- Increased error tracking bill for affected customers due to exception volume

- JavaScript exceptions thrown in customer applications, potentially affecting their own application functionality and exception-tracking bills in other platforms

Duration: 5 hours 26 minutes of active impact

Error tracking billing impact:

- Initially 305 teams saw increased exception volumes
- Successfully filtered out the most common exception pattern via this PR, reducing billable impact to 90 teams
- The remaining 215 teams' exceptions were successfully excluded from their bills
- The 90 teams still affected experienced other related exception patterns that couldn't be automatically filtered

Remediation

We reverted the problematic changes and released SDK version 1.270.1, which restored compatibility with all SDK versions.

Immediate actions

- Reverted the backwards-incompatible changes and released version 1.270.1
- Issued credits / refunds to all customers affected by increased error tracking bills
- Sent communications to all 305 affected teams explaining the issue and resolution

Action item: Start incidents earlier. We should declare incidents as soon as we confirm an issue (around 14:59), not almost two hours after mitigation. This enables proper coordination and customer communication. Owner: @lucasheriques

Short-term improvements

- **API compatibility layer**: Modify the `isDisabled` function check in `posthog-persistence.ts` to be nullable and provide a safe fallback when undefined. This will prevent similar issues when extensions call potentially-unavailable functions. PR

- **Ignore PostHog SDK exceptions:** By default we should not capture exceptions known to be caused by the PostHog SDK. Offering a config option to toggle this parameter would allow teams (especially our own) to configure this setting. [PR]

- **Monitor suspicious exceptions:** The error tracking team should monitor the number of exceptions that look likely to be coming from our own SDK. Adding alerting to this metric would allow us to detect anomalies sooner. [PR]

Long-term improvements (Target: Q4 2025 – Q1 2026)

- **Version compatibility testing**: Add automated tests that run the latest extension code against the last N minor. Owner: @lucasheriques
- **Static analysis tooling**: Implement a linter rule or TypeScript checking to flag when extension code calls SDK functions that aren't marked as "stable API" or don't have proper fallback handling. This should run in CI and block PRs. Owner: @lucasheriques
- **Client-side error monitoring**: Set up monitoring and alerting for exception spikes in customer applications that use our SDK. This would have detected the issue within minutes instead of hours. Owner: @lucasheriques

Lessons learned

What went well

- **Quick mitigation once identified**: From confirmation (15:25) to mitigation (16:11) was only 46 minutes
- **Cross-team collaboration**: Support, Error Tracking, and Surveys teams all contributed to identifying and resolving the issue
- **Effective customer remediation**: Successfully filtered out most exceptions to reduce billing impact from 305 to 90 teams

What went poorly

- **Detection delay**: 4+ hours from deployment to detection is unacceptable for an issue affecting 305 customers. We relied on customer reports rather than proactive monitoring
- **Backwards compatibility blindspot**: Our development and review process had no safeguards against this class of bug
- **Incident declaration timing**: We declared the incident after it was resolved, missing the opportunity for coordinated response and timely customer communication
- **Testing gaps**: No integration tests covering the extension/SDK version compatibility scenario

Key takeaways

This incident revealed a critical architectural weakness in how our asynchronously-loaded extensions interact with versioned SDK code. The assumption that extensions can safely call any SDK function breaks down when we have customers on old SDK versions but always serve them the latest extension code.

We also had this similar issue in another incident here.

The 4+ hour detection delay highlights gaps in our observability for client-side errors. We lack visibility into exceptions occurring in customer applications using our SDK.

The improvements outlined above will address both the immediate technical issue and the systemic gaps in testing, monitoring, and deployment practices that allowed this to reach production and persist for over 5 hours.

Feature flags recurring outages

contents/handbook/company/post-mortems/2025-10-21-feature-flags-recurring-outages.md

Between October 21 and October 30, 2025, the PostHog Feature Flags service experienced four separate incidents, exposing systemic architectural weaknesses that required comprehensive remediation. This post-mortem documents all four incidents and our path to stability.

Summary

Over a 10-day period in October 2025, the feature flags service experienced four separate incidents totaling over 14 hours of cumulative major impact (errors or severe latency). While each incident had different surface-level symptoms, three of the four incidents shared the same root cause: improper CPU resource sizing. Our nodes were too small relative to pod resource requests, causing Kubernetes to pack too many pods per node and saturate CPU capacity. This CPU saturation led to connection pool exhaustion, excessive parallelism (too many concurrent operations), and ultimately cascading failures. The fourth incident was a rate limiting misconfiguration unrelated to resource sizing.

Incidents:

- **October 21** (103 minutes): Redis overload from excessive parallelism and connection pool exhaustion
- **October 24** (72 minutes): Rate limiting misconfiguration causing 429 errors for ~97% of requests
- **October 28** (123 minutes): Connection pool exhaustion and excessive parallelism (same root cause as October 21)
- **October 29-30** (7 hours 9 minutes): CPU-bound latency from node CPU pressure exceeding 90%

Incident timeline

October 21, 2025 – Redis overload

Duration: 21:45 to 23:28 UTC (103 minutes) **Impact:** ~38% of evaluation requests returning errors in US datacenter

A deployment intended to reduce timeout errors (PR #39821) incorrectly addressed symptoms rather than root causes. While rolled back within 2 minutes, it triggered excessive parallelism and connection pool exhaustion, which manifested as massive data transfer from Postgres to Redis and a surge in concurrent connections that overwhelmed our cache layer. Redis memory exhaustion followed, leading to prolonged service degradation.

What "excessive parallelism" means: Under CPU pressure, degraded requests triggered Envoy retries between the load balancer and service. Each retry spawned new concurrent requests, and each request performed multiple concurrent Redis reads. A single degraded request could fan out to dozens of

concurrent Redis operations. Combined with cache misses (on cache miss, we synchronously loaded full flag and team state from Postgres and wrote it into Redis), this created bursty write storms that overwhelmed Redis.

Connection pool mechanics: Each pod maintains its own Postgres connection pool. Creating a pool involves TLS handshakes, authentication, and initial connection establishment—operations that are computationally expensive, especially when pods are CPU-bound. Under CPU pressure exceeding 90%, new pods struggled to initialize these pools within the 20-second startup timeout, leading to crash loops and reduced healthy pod capacity.

Critical issue: The Redis overload from the flags service also impacted the main PostHog application, demonstrating dangerous coupling through shared infrastructure. The flags service can operate without Redis but falls back to heavier database queries, making responses slower.

Root causes:

- **Primary root cause: CPU resource undersizing** – Nodes were too small relative to pod resource requests, causing Kubernetes to pack too many pods per node. This led to CPU saturation exceeding 90%, which caused excessive parallelism and connection pool exhaustion
- Symptom-focused fix that didn't address underlying CPU sizing issues
- Unbounded cache population logic with no rate limiting (on cache miss, synchronous full state load from Postgres to Redis)
- Envoy retries → more concurrent /flags requests → more pool acquisitions + Redis reads → overload
- Shared Redis instance between flags service and main application (critical infrastructure coupling)
- **Missing CPU alerting**: No alerts existed for CPU pressure, preventing early detection
- Lack of monitoring for Postgres-to-Redis transfer patterns

Timeline:

- **21:45 UTC** – Deploy timeout handling change
- **21:47 UTC** – Automated monitoring detects increased error rates
- **21:49 UTC** – Immediate rollback initiated and completed
- **21:50 UTC** – Error rates remain elevated despite rollback
- **22:30 UTC** – Redis metrics show memory exhaustion on ElastiCache
- **22:35 UTC** – Postgres connection spike observed, overwhelming connection pool
- **22:45 UTC** – Discovery: Massive data transfer from Postgres to Redis in progress
- **22:50 UTC** – Root cause identified: Excessive parallelism triggering cache population overload
- **22:50 UTC** – Status page updated with incident details
- **23:00 UTC** – Begin throttling connections and Redis writes
- **23:28 UTC** – Service fully recovered

October 24, 2025 – Rate limiting misconfiguration

Duration: 18:00 to 19:12 UTC (72 minutes) **Impact:** ~97% of evaluation requests returning 429 (rate limit) errors worldwide

Deployed IP-based rate limiting (PR #40074) as a protective measure following Tuesday's incident. The tower-governor library (our Rust rate limiting middleware) saw all traffic as coming from a single IP (our load balancer) rather than actual client IPs, immediately triggering rate limits for all legitimate traffic.

Root causes:

- Rate limiting implementation didn't account for load balancer architecture
- Library's secure defaults (not trusting X-Forwarded-For headers) were inappropriate for our trusted infrastructure. We terminate TLS at the load balancer and route to the service over a private network, so trusting X-Forwarded-For from our own load balancer would have been safe; the default of ignoring it was wrong for our setup
- No alerting configured for 429 errors, requiring customer reports for detection (62-minute detection delay)
- We validated rate limiting only in direct-to-service tests, not behind our production load balancers

Timeline:

- **18:00 UTC** – Deploy IP-based rate limiting to /flags endpoint
- **18:01 UTC** – Rate limiter begins returning 429 errors for most requests
- **18:02 UTC** – All traffic appears as single IP to rate limiter
- **18:10 UTC** – Initial customer reports of widespread failures
- **18:30 UTC** – More customer reports escalate urgency
- **18:45 UTC** – Engineering begins investigation into customer reports
- **19:00 UTC** – Team identifies 429 errors in logs
- **19:02 UTC** – Root cause identified: rate limiter sees load balancer IP only
- **19:05 UTC** – Decision to disable rate limiting immediately
- **19:12 UTC** – Rate limiting disabled, service fully recovered
- **Note:** Status page was not updated during this incident due to the rapid resolution timeline post-detection (detection to resolution in ~12 minutes)

October 28, 2025 – Connection pool exhaustion and excessive parallelism

Duration: 19:28 to 21:31 UTC (123 minutes) **Impact:** ~34% of evaluation requests failing in US datacenter

A routine deployment with no changes directly related to the flags service triggered a rollout of feature flag pods in the US region. New pods couldn't connect to Postgres within the 20-second startup timeout, entering crash loops due to excessive parallelism and connection pool exhaustion—the same root cause as October 21. Under CPU pressure, pods couldn't initialize Postgres connection pools (TLS handshakes, authentication, connection establishment) within the timeout. Simultaneously, a massive spike in Redis

writes caused key evictions, effectively making the cache unavailable. While the flags service can operate without Redis (falling back to heavier database queries), with both cache unavailable and database under pressure, a significant portion of US traffic failed.

Critical issue: The Redis overload from the flags service also impacted the main PostHog application, highlighting dangerous infrastructure coupling. Unrelated deployments shouldn't trigger feature flags rollouts.

Root causes:

- **Primary root cause: CPU resource undersizing** – Same root cause as October 21: nodes too small relative to pod requests, causing too many pods per node and CPU saturation
- Unrelated deployment triggered feature flags pod rollout
- New pods failing to connect to Postgres within 20s timeout under CPU pressure (connection pool initialization too slow)
- Pods entering crash loops, reducing available capacity
- Redis write storm during deployment causing key evictions (cache miss → synchronous full state load from Postgres to Redis)
- Shared Redis instance between flags service and main application (critical infrastructure coupling)
- Startup timeout too aggressive for production conditions under CPU pressure
- **Missing CPU alerting**: No alerts existed for CPU pressure, preventing early detection

Timeline:

- **19:12 UTC** – Routine deployment triggers feature flags pod rollout in US (no /flags code changes)
- **19:15 UTC** – New US pods begin failing to connect to Postgres within 20s timeout
- **19:18 UTC** – Pods enter crash loops, reducing available capacity in US
- **19:20 UTC** – Massive spike in Redis writes begins in US region
- **19:23 UTC** – On-call receives high error count alert, initiates incident
- **19:23 UTC** – Status page updated with incident details
- **19:25 UTC** – Redis key evictions spike, cache becomes effectively unavailable
- **19:26 UTC** – Main PostHog app begins experiencing issues due to shared Redis overload
- **19:28 UTC** – Service degradation begins, ~34% of US requests failing
- **19:35 UTC** – Team identifies dual failure: pod crashes + Redis overload
- **19:45 UTC** – Decision to halt rollout and scale US pods to zero
- **20:00 UTC** – US pods scaled to zero, waiting for Redis to stabilize
- **20:30 UTC** – Redis begins recovering from write storm
- **20:53 UTC** – Partial recovery as stable US pods brought back online
- **21:15 UTC** – Gradual pod scaling continues in US
- **21:31 UTC** – Full service restored, US region fully operational

Note: We initially attempted the same remediation approach from October 21 before implementing other solutions to decrease parallelism.

October 29-30, 2025 – CPU-bound latency

Duration: 22:30 UTC on October 29 to 05:39 UTC on October 30 (7 hours 9 minutes) **Impact:** Slow queries and degraded performance due to node CPU pressure

Query performance was impacted for over 7 hours. While queries were slow to both Redis and Postgres, metrics for both dependencies confirmed they were healthy. The slow queries were due to CPU pressure on the nodes, which exceeded 90%. This impacted connections and slowed response times for the service to several times the usual.

Root causes:

- CPU pressure on nodes exceeding 90% (nodes too small relative to pod requests, causing too many pods per node)
- Pod resource requests not properly sized, causing unhealthy distribution of pods per node
- **Critical gap: CPU alerting was completely missing** – No alerts existed for CPU pressure, which allowed the issue to persist undetected for over 7 hours
- Insufficient observability around CPU-bound failure modes

Timeline:

- **22:30 UTC (Oct 29)** – Incident reported, increased error rates and latency detected
- **22:30 UTC (Oct 29)** – Status page updated with incident details
- **00:03 UTC (Oct 30)** – Rolled back hardware changes, errors mostly subsided but latencies persist
- **05:39 UTC** – Incident resolved, query timings returned to normal

Resolution: After identifying connectivity issues due to resource exhaustion on feature flags nodes, we applied changes that resolved this resource exhaustion. Increasing pod resource requests for the flag service resulted in a healthier distribution of pods per node, which caused per-node CPU usage to go down and the service to return to a healthy state.

Root cause analysis

While each incident had specific triggers, three of the four incidents shared the same fundamental root cause:

1. **CPU resource undersizing (primary root cause)**: Our nodes were too small relative to pod resource requests, causing Kubernetes to pack too many pods per node and saturate CPU capacity (exceeding 90%). This CPU saturation was the root cause of October 21, 28, and 29-30 incidents:
    - **October 21 & 28**: CPU saturation caused excessive parallelism (Envoy retries → concurrent requests → concurrent Redis reads) and connection pool exhaustion (pods couldn't initialize Postgres pools under CPU pressure), which manifested as Redis overload and database connection failures

    - **October 29-30**: CPU saturation directly caused slow queries and degraded performance, even though Redis and Postgres metrics showed healthy dependencies
    - Proper CPU right-sizing (fewer pods per node, better-resourced pods) resolved the underlying issues in all three incidents

2. **Connection pool management complexity**: Each pod maintains its own Postgres connection pool. Creating a pool involves TLS handshakes, authentication, and connection establishment—operations that are computationally expensive, especially when pods are CPU-bound. This complexity, combined with CPU saturation, exacerbated connection pool exhaustion issues.

3. **Shared Redis is a critical single point of failure**: Redis overload from the flags service impacted the main PostHog application, demonstrating dangerous coupling through shared infrastructure. Isolation is critical despite implementation complexity.

4. **Critical monitoring gap: CPU alerting was missing**: CPU alerting was completely absent throughout these incidents, preventing early detection of CPU saturation that was the root cause of three outages. This was a fundamental gap in our monitoring strategy that allowed CPU pressure to escalate unnoticed.

5. **Unbounded retries**: Unbounded retries in Envoy (between load balancer and endpoint) amplified failures (now fixed with retry limits)

6. **Rate limiting misconfiguration (October 24 only)**: The October 24 incident was unrelated to CPU sizing—it was caused by rate limiting configuration that didn't account for load balancer architecture

Impact

- **Total major impact:** Over 14 hours across four incidents (errors or severe latency)
- **Error rates:** Ranging from 34% to 97% of requests during incidents
- **Service degradation:** All flag types affected, including read-only evaluations
- **Cross-service impact:** Redis overload from flags service affected main PostHog application
- **Customer impact:** HTTP 429, 504 errors and degraded performance regardless of flag configurations
- **Recurring issues:** Connection pool exhaustion and excessive parallelism occurred twice (October 21 and 28), indicating insufficient initial remediation

Remediation

Immediate actions (completed)

- **Configuration externalization**: Database connection timeouts and other critical settings moved to runtime configuration
- **Timeout adjustments**: Values increased to accommodate peak load scenarios

- **Rate limiting fixes**: Fixed rate-limiting configuration that caused October 24 incident (configured tower-governor to trust X-Forwarded-For from our load balancer)
- **Retry limits**: Implemented retry limits in Envoy (between load balancer and endpoint) to prevent unbounded retry amplification
- **CPU and infrastructure right-sizing (critical fix)**: Increased pod resource requests and adjusted Kubernetes fleet size to reduce pods per node. This was the primary remediation for three of the four incidents (October 21, 28, and 29-30), addressing the root cause of excessive parallelism, connection pool exhaustion, and CPU-bound latency. Running smaller fleets with better-resourced pods rather than larger fleets with CPU-bound pods.
- **CPU alerting**: Added per-node and per-pod CPU alerts with thresholds at 80% sustained for 5 minutes, paging on-call
- **Observability improvements**: Added monitoring for previously invisible failure modes

Short-term improvements (Tracked in GitHub Issue #40885)

In progress (next 2 weeks):

- **Strike team formation**: Engineers from flags, ingestion, and infrastructure teams conducting comprehensive review of application and infrastructure to identify remaining bottlenecks
- **Redis isolation**: Investigating decoupling flags Redis instance from application Redis instance to prevent cross-service impact

To complete before re-enabling ArgoCD sync:

- Evaluate current state of synced flags deployment and ensure durability against future outages
- Update flags service charts config to match values currently in ArgoCD
- Define deployment strategy for short-term (considering deployment vs rollout to avoid 503s)
- Define and implement Redis strategy
- Establish feature flags team as hard code-owners for flags-related code

Medium-term improvements

Incident response and monitoring:

- Build high-level dashboard of important flag metrics with runbook links
- Implement rollout/annotation controls to disable staged rollouts and enable "force-merge" for rolling changes
- Update feature flag runbooks with dashboard links and deeper investigation paths
- Add missing alerts against existing service/infrastructure level metrics
- Update readiness checks to validate dependencies that degraded under load (e.g., ping database instead of mirroring liveness checks)

Architectural improvements:

- **Rate limiting for cache operations**: Prevent Redis overwhelm from cache population
- **Connection pool monitoring**: Automatic throttling when pools approach exhaustion

- **Connection limiting**: Prevent unbounded concurrent connections

Long-term improvements

- **Load testing framework**: Production-scale testing to catch load-dependent issues before deployment
- **Progressive rollout infrastructure**: Gradual deployments to limit blast radius
- **Deployment strategy evolution**: Re-evaluate rollout vs deployment approaches with programmatic controls
- **Comprehensive monitoring**: Document Postgres-to-Redis data flow patterns and create runbooks for data transfer storm scenarios

Lessons learned

What went well

- **Rapid detection** – Monitoring caught issues within 2 minutes in most cases
- **Quick initial response** – Rollbacks executed immediately when possible
- **Systematic investigation** – Teams methodically identified overload patterns
- **Cross-team collaboration** – Flags, infrastructure, and ingestion teams worked together effectively

What didn't go well

- **Symptom-focused fixes** – Multiple PRs addressed symptoms rather than root causes
- **Unbounded operations** – No limits on retries, cache population, or connection creation
- **Rollback insufficiency** – Data transfers and resource exhaustion persisted after code reverted
- **Complex failure modes** – Interactions between database, cache, and application layers not well understood
- **Shared infrastructure** – Flags service overloads impacted main application
- **Customer comms** – While we generally did a good job of making public-facing status pages during each one of these incidents, one notable gap was that we never made an externally-facing status page update for the rate-limiting incident on October 24th.
- **Diagnosis delays** – Took significant time to connect symptoms to root causes
- **Configuration rigidity** – Hardcoded values prevented rapid remediation
- **Missing CPU alerting** – CPU alerting was completely absent, allowing CPU pressure to escalate undetected for hours

Key takeaways

1. **CPU right-sizing is fundamental** – The biggest takeaway: nodes were too small relative to pod resource requests, causing Kubernetes to pack too many pods per node and saturate CPU capacity. This CPU saturation led to excessive parallelism (Envoy retries → concurrent requests → concurrent Redis reads), connection pool exhaustion (pods couldn't initialize Postgres pools under

CPU pressure), and slow queries. Right-sizing (fewer pods per node, better-resourced pods) addressed the underlying issues that caused October 21, 28, and 29-30 incidents. This must be a primary consideration for any service deployment.

2. **Connection pool management architecture matters** – Each pod maintains its own Postgres connection pool. Creating a pool involves TLS handshakes, authentication, and connection establishment—operations that are computationally expensive, especially when pods are CPU-bound. This complexity, combined with CPU saturation, exacerbated connection pool exhaustion. Better approach: reduce concurrency and run smaller fleets with better-resourced pods rather than larger fleets with CPU-bound pods.

3. **Shared Redis is a critical single point of failure** – When flags service overloads Redis, it takes down the main app too. This was evident in October 21 and 28 incidents where Redis overload from flags service impacted the main PostHog application. Isolation is critical despite implementation complexity.

4. **CPU alerting was completely missing** – CPU alerting was absent throughout these incidents, preventing early detection of CPU saturation that was the root cause of three outages. This was a fundamental gap in our monitoring strategy. CPU metrics must be monitored and alertable from day one.

5. **Monitor data flow patterns** – Postgres-to-Redis transfer spikes should trigger alerts. Watch for unusual data movement.

6. **Test under load** – Overload patterns only appeared under production traffic. Load testing is non-negotiable.

7. **Progressive rollouts save lives** – Gradual deployments limit blast radius and enable rapid detection. We're implementing rollout/annotation controls to disable staged rollouts and enable "force-merge" for rolling changes.

8. **Configuration must be flexible** – Critical settings must be adjustable without full deployment cycles.

9. **Unbounded retries amplify failures** – Retries without bounds in Envoy (between load balancer and endpoint) can cascade failures. We've implemented retry limits to prevent this.

Moving forward

These four incidents highlighted critical gaps in our defensive architecture and operational procedures. The compounding failures demonstrated that our service needed fundamental improvements, not just quick fixes. The primary root cause—CPU resource undersizing (nodes too small relative to pod requests, causing too many pods per node)—manifested differently across three incidents (October 21, 28, and 29-30), requiring us to recognize that excessive parallelism, connection pool exhaustion, and slow queries were all symptoms of the same underlying issue. The recurrence of these symptoms between October 21

and 28 showed that we needed to address the root cause (CPU sizing) rather than the symptoms. We initially attempted the same remediation approach from October 21 before implementing CPU right-sizing, which resolved the underlying issues.

We've implemented immediate remediations and are executing a comprehensive review of the entire service architecture. Our strike team is systematically identifying and addressing remaining bottlenecks. Once we complete the short-term improvements tracked in GitHub Issue #40885, we'll have confidence that the service is durable against future outages.

The architectural improvements underway—including Redis isolation, connection pool management, and comprehensive monitoring—will prevent similar cascading failures in the future. We're committed to ensuring the feature flags service meets the reliability standards our customers expect.

Persons database migration

contents/handbook/company/post-mortems/2025-11-15-persons-db-migration.md

Between November 11 and November 15, 2025 we hit a Postgres limit that required us to migrate our Persons database for US Cloud. This led to ingestion delays which had a knock-on effect for products relying on person data, including feature flags and experiments.

This post-mortem document examines the root cause of the issue, steps taken, and our future plans derived from the lessons learned.

Incident summary

From November 11, 4:02 PM UTC to November 14, 3:02 PM UTC the performance of PostHog's ingestion processing pipeline became severely degraded, resulting in processing delays of events of up to 2 days for all customers in the US region.

The root cause of the performance degradation was that our Postgres database responsible for storing our Person information reached a previously unseen limits in Postgres related to the JSONb field we use to store person properties. This led to a state where writes to the database kept waiting for OIDs in the TOAST table to become available, which could take multiple seconds per update, slowing down the ingestion pipeline to the point where there were no standard scaling options available. See Root Cause Analysis below for more technical details.

The root cause was not identified until November 12th 10:17pm UTC, a day and a half after the issue arose. We enlisted help from engineers on the AWS RDS team and external consultants to identify the cause. Diagnosis proved difficult even with specialist support, but we eventually found out we were left with only one option: migrate to a new partitioned table.

By November 14, 2025, 15:02 UTC, ingestion was healthy again and we shifted focus to the accumulated backlog. During this recovery phase, we hit a secondary issue with AWS MSK (Kafka): local disk usage reached 85% because tiered storage keeps the most recent 4 hours of data on disk before offloading to S3. The backlog catchup created an unusually dense last-4-hours window, driving up local disk usage. We temporarily paused ingestion, reduced the topic's local retention window, confirmed disk headroom, and then resumed ingestion.

After that, catchup progressed smoothly. We moved our backfill into Dagster to gain better visibility and stability for long-running backfill jobs, knowing remediation would take at least the weekend. By the morning of November 15, all events since the start of the incident had been processed, all systems were fully operational, and we began a background backfill of older Person data purely for housekeeping. No data was lost.

By November 15, 6:20 AM UTC we had worked through the backlog of events and fully recovered.

Incident impact

Scope

- This issue only impacted US Cloud
- Time window of degraded ingestion performance: Approximately November 11, 16:02 UTC – November 14, 15:02 UTC
- Time to fully process backlog and complete recovery: Until November 15, 06:20 UTC

Customers experienced ingestion delays ranging from 10 minutes to up to 2 days. During this period recent events sent to PostHog did not appear, leading to the following potential per-product impact:

- **Analytics:** Charts and queries which included recent time ranges would be inaccurate due to data from the incident period not being present. Customers were still able to analyze historical data.
- **Feature flags and Experiments:** The flag evaluation service continued to operate, however flags that relied on person properties would have had delays in those properties being used to evaluate feature flags.
- **Error tracking and Session replay:** Ingestion of errors and replays remained healthy. Filtering and segmentation based on Person updates was affected similarly to Analytics.
- **CDP & Workflows:** Destinations and Workflows reliant on Persons were affected similarly to Feature Flags. Downstream actions were delayed and should have been automatically corrected as the backlog was processed.

Data integrity

No data was lost.

Once the backlog was cleared, all reporting tools indicate accurate values were processed for the delayed period.

Timeline

A full timeline of updates is available on the PostHog status page.

Root cause analysis

Primary issue: Postgres TOAST OID exhaustion

PostgreSQL stores large column values (>2KB compressed) in a separate, out-of-line table, called the TOAST table. Posthog's Persons table has a `properties` column that frequently exceeds this 2KB threshold, resulting in there being many TOASTed values associated with the Persons table.

Each value that is moved "out of line" is assigned a unique OID (Object identifier) from a finite 32-bit space (~4 billion values), that the main table uses to track it in the table's associated TOAST table.

From postgresql wiki:

> The OIDs used for this purpose are generated from a global counter that wraps around every 4 billion values, so that from time to time an already-used value will be generated again. Postgres detects that, and tries again with the next OID.

When the space of used OIDs approaches the limit, there will be longer and longer sequential runs of used OIDs. This results in the database engine having to do an incredible amount of reads (checking every used OID it is given by the counter to see if it's free or not) to make a single INSERT or UPDATE for a TOAST'ed row.

It is important to note that before the table hits the hard limit of 4 billion OIDs, write performance for TOAST'ed rows will be severely degraded, because the space of available OIDs is so sparse. If there is just a single free OID left, the database engine would, on average, have to read through billions of used OIDs and check to see if they are free, before it finds the free OID to complete the write. This OID exhaustion increased the amount of disk reads we were doing per write query from 10kb to 15MB, increasing latency for those queries by 100x and grinding the ingestion of events to a halt.

Secondary issue: AWS MSK disk pressure during catch-up

During backlog processing, we hit a separate but related operational issue:

- Our AWS MSK cluster uses tiered storage, keeping the most recent 4 hours of data on local disk before offloading to S3.
- As we processed the backlog, the amount of data in the "last 4 hours" window became unusually large.
- This pushed local disk utilization up to ~85%, triggering alerts.

To mitigate this we paused ingestion, reduced the local retention configuration for the relevant topic, and resumed ingestion once disk usage returned to a safe level.

Appendix: OID exhaustion diagrams

Stage 1: Healthy database state

OID Space (each block = used OID, each dash = free OID) [----------] 0-1M OIDs [-----XX----------XX----------] 1M-2M OIDs [---------XX----------XX----------] 2M-3M OIDs [----------] 3M-4M OIDs ↑ Next OID Counter (finds free OID immediately)

Stage 2: OID exhaustion

OID Space (each block = used OID, each dash = free OID) [XXXXXXXXXXXXXXX--XXXXXXXXX--XXXXXXXXXXXXXX--XXXXXXX] 0-1M OIDs [XXXXXXXXXXXXXXXXXXXXXXXX----XXXXXXXXXXXXXXXXXXXXXX--XX] 1M-2M OIDs [XXXXXXXXXX--

XXXXXXXXXXXXXXXXXXXXXXXXXXX--XXXXXXXXXXXXX] 2M-3M OIDs
[XXXXXXXXXXXXXXXXXXX----XXXXXXXXXXXXXXXXXXXXXXXXXX--X] 3M-4M OIDs ↑ Next OID Counter (must skip over many used OIDs, each skip requiring disk reads)

Why was this hard to detect?

PostgreSQL's OID exhaustion behavior is rare and not commonly encountered even at large scale. Additionally, standard dashboards (CPU, memory, IOPS, lock contention) did not immediately point at OID exhaustion.

Diagnosis of the issue was also frustrated by a lack of dedicated observability on:

- TOAST table size / OID usage
- Disk read amplification per write on specific tables

Eventual diagnosis was only possible due to the dedicated effort of our engineers working in tandem with external experts and AWS engineers to connect:

- Massive read amplification
- Specific pattern of TOAST usage
- A nearly exhausted global OID space

Remediation

Immediate actions (completed)

1. **Root cause discovery and isolation** We Identified TOAST OID exhaustion as the root cause by engaging internal teams, external consultants, and AWS engineers to analyze:

- Query plans
- Disk read amplification
- TOAST and OID behavior for the persons table

2. **Migration to a new partitioned Persons table**

- Created a new partitioned table architecture for Persons with a fresh TOAST table and OID space.
- Implemented database triggers to keep old and new tables in sync for live writes.
- Ran a backfill script to copy existing Person data from the old table to the new table.
- Modified ingestion and application logic so that:
- All new writes go to the new table.
- Reads go to the new table first, with fallback to the old table during the migration window.

3. **Careful deployment of application changes** Given the risk of introducing new issues during an incident, we chose a manually controlled deploy to production for the web app rather than a fully automated rollout. Multiple engineers worked in shifts throughout the weekend and made

changes across:

- Feature flags API
- Error tracking ingestion
- Django web application

4. **Scaling ingestion to clear backlog** Once ingestion performance was restored on the new Persons table, we scaled ingestion workers to process the accumulated backlog while monitoring:

- Lag per partition
- Throughput
- Resource utilization

5. **MSK disk pressure mitigation**

- Paused ingestion temporarily when MSK local disk hit ~85% usage.
- Reduced local retention for the affected Kafka topic so less data needed to be stored on disk before offloading to S3.
- Resumed ingestion after confirming sufficient headroom.

6. **Dagster-based backfill** We moved the backfill process into Dagster to provide:

- Better monitoring and visibility
- More robust handling of long-running backfill jobs
- Used Dagster to complete the remaining backfill and housekeeping tasks over the weekend.

7. **Final cleanup and confirmation** Communicated final resolution and announced a small upcoming maintenance window to consolidate on the new tables. We verified that:

- All event backlogs had been processed.
- Services were reading correctly from the new Persons table (with safe fallback while the old table still existed).

Planned actions (planned or in-progress)

1. **Deeper Postgres engine monitoring** We plan to add metrics and alerts around:

- Heavy disk read amplification per write
- TOAST table statistics
- Other engine limits that may become relevant at large scale

2. **Improved runbooks for engine-level limits** We plan to document symptoms and diagnostics for similar Postgres engine-level issues. This will include clear decision trees for when to migrate vs repair-in-place.

3. **Improved and new runbooks for customer comms** We have begun creating new customer communication runbooks which clarify how and when to communicate with customers about the issue and provide a clear escalation path and redundancies.
4. **Exploring other data stores** We've been exploring using other data stores for the persons database and will continue to evaluate those.

Lessons learned

What went well?

- **Data integrity remained intact** We preserved all incoming events and persons data. While delayed, data was not lost.
- **Coordinated multi-team response** Engineers across ingestion, infrastructure, and application teams, plus external consultants and AWS engineers, collaborated effectively to diagnose a rare engine-level problem.
- **Safe migration under load** We successfully migrated to a new Persons table using triggers and backfill while the system remained live, minimizing additional downtime.
- **Transparent customer communication** We provided regular engineer-led status updates and committed to a public post-mortem.

What could have gone better?

- **Diagnosis took too long** It took roughly a day and a half to conclusively identify OID exhaustion as the root cause. We had no dedicated monitoring for TOAST growth, OID usage, or disk read amplification per write.
- **Single critical dependency on Persons** Many core features (analytics, flags, replay filters, CDP) rely heavily on timely updates to the persons table. When that became unhealthy, a wide surface area of the product was affected.
- **Backfill visibility and tooling** Our initial backfill approach lacked the visibility and robustness needed for a prolonged, large-scale migration. We had to move this logic into Dagster during the incident.
- **MSK disk pressure during catch-up** While secondary to the main cause, disk pressure on MSK during catch-up highlighted that our tiered storage configuration and alerting were not tuned for large backlog scenarios.
- **Lack of communication redundancies** All of the team members who are normally responsible for customer communications were unavailable for the duration of this incident and we had to scramble to identify fallbacks.

Moving forward

This incident surfaced a rare but serious interaction between our data model and a low-level PostgreSQL engine limit. It also highlighted how central the Persons data model is to the rest of PostHog: when the persons table slowed down, a wide range of features---from analytics and feature flags to replay filtering and CDP---were indirectly impacted.

We've taken immediate steps to recover by migrating to a new partitioned Persons table, stabilizing ingestion, and clearing the backlog of events. We are now focused on:

- Completing reconsolidation on the new tables
- Hardening observability and alerting around TOAST/OID behavior and disk read amplification
- Improving our backfill tooling and Kafka tiered storage safeguards
- Proactively designing and operating Person-like tables in a way that avoids similar limits in the future

We're committed to continuing to invest in the resilience of our ingestion and Persons infrastructure so that incidents like this become less likely, easier to detect early, and faster to remediate when they do occur.

Shai-Hulud supply chain attack

contents/handbook/company/post-mortems/2025-11-26-shai-hulud-attack.md

At 4:11 AM UTC on November 24th, a number of our SDKs and other packages were compromised, with a malicious self-replicating worm – Shai-Hulud 2.0. New versions were published to npm, which contained a preinstall script that:

1. Scanned the environment the install script was running in for credentials of any kind using Trufflehog, an open-source security tool that searches codebases, Git histories, and other data sources for secrets.
2. Exfiltrated those credentials by creating a new public repository on GitHub and pushing the credentials to it.
3. Used any npm credentials found to publish malicious packages to npm, propagating the breach.

By 9:30 AM UTC, we had identified the malicious packages, deleted them, and revoked the tokens used to publish them. We also began the process of rolling all potentially compromised credentials pre-emptively, although we had not at the time established how our own npm credentials had been compromised (we have now, details below).

The attack only affected our Javascript SDKs published in npm. The most relevant compromised packages and versions were:

- `posthog-node` 4.18.1, 5.13.3 and 5.11.3
- `posthog-js` 1.297.3
- `posthog-react-native` 4.11.1
- `posthog-docusaurus` 2.0.6
- `posthog-react-native-session-replay@1.2.2`
- `@posthog/agent@1.24.1`
- `@posthog/ai@7.1.2`
- `@posthog/cli@0.5.15`

What should you do?

Our recommendations are to:

1. Look for the malicious files locally, in your home folder, or your document roots:

```
find . -name "setup_bun.js" \
  -o -name "bun_environment.js" \
  -o -name "cloud.json" \
  -o -name "contents.json" \
```

```
  -o -name "environment.json" \
  -o -name "truffleSecrets.json"
```

2. Check npm logs for suspicious entries:

```
grep -R "shai" ~/.npm/_logs
grep -R "preinstall" ~/.npm/_logs
```

3. Delete any cached dependencies:

```
rm -rf node_modules
npm cache clean --force
pnpm cache delete
```

Pin any dependencies to a known-good version (in our case, all the **latest published versions**, which have been published after we identified the attack, are known-good), and then reinstall your dependencies.

We also suggest you make use of the `minimumReleaseAge` setting present both in `yarn` and `pnpm`. By setting this to a high enough value (like 3 days), you can make sure you won't be hit by these vulnerabilities before researchers, package managers, and library maintainers have the chance to wipe the malicious packages.

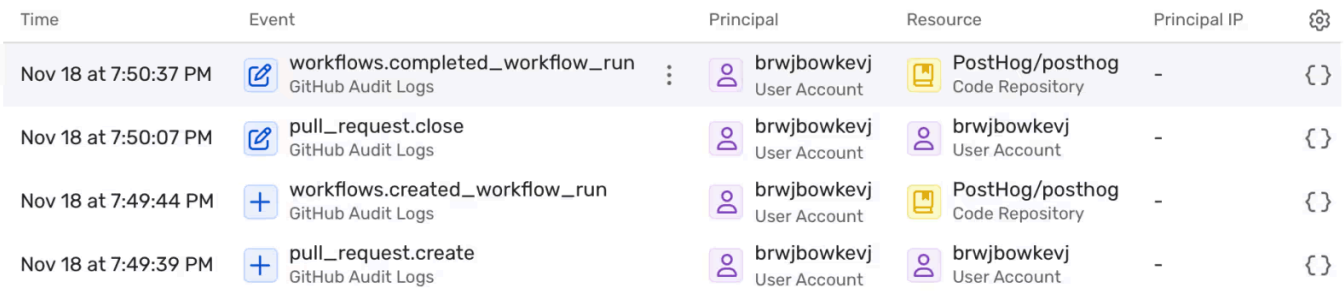
How did it happen?

PostHog's own package publishing credentials were not compromised by the worm described above. We were targeted directly, as were a number of other major vendors, to act as a "patient zero" for this attack.

The first step the attacker took was to steal the Github Personal Access Token of one of our bots, and then use that to steal the rest of the Github secrets available in our CI runners, which included this npm token. These steps were done days before the attack on the 24th of November.

At 5:40PM on November 18th, now-deleted user `brwjbowkevj` opened a pull request against our `posthog` repository, including this commit. This PR changed the code of a script executed by a workflow we were running against external contributions, modifying it to send the secrets available during that script's execution to a webhook controlled by the attacker. These secrets included the Github Personal Access Token of one of our bots, which had broad repo write permissions across our organization. The PR itself was deleted along with the fork it came from when the user was deleted, but the commit was not.

The PR was opened, the workflow run, and the PR closed within the space of 1 minute (screenshots include timestamps in UTC+2, the author's timezone):



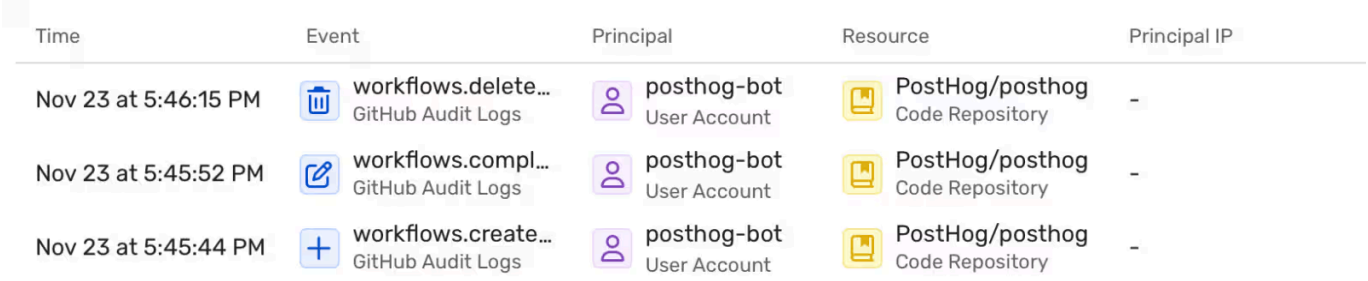
Time	Event	Principal	Resource	Principal IP
Nov 18 at 7:50:37 PM	workflows.completed_workflow_run GitHub Audit Logs	brwjbowkevj User Account	PostHog/posthog Code Repository	-
Nov 18 at 7:50:07 PM	pull_request.close GitHub Audit Logs	brwjbowkevj User Account	brwjbowkevj User Account	-
Nov 18 at 7:49:44 PM	workflows.created_workflow_run GitHub Audit Logs	brwjbowkevj User Account	PostHog/posthog Code Repository	-
Nov 18 at 7:49:39 PM	pull_request.create GitHub Audit Logs	brwjbowkevj User Account	brwjbowkevj User Account	-

At 3:28 PM UTC on November 23rd, the attacker used these credentials to delete a workflow run. We believe this was a test, to see if the stolen credentials were still valid (it was successful).

At 3:43 PM, the attacker used these credentials again, to create another commit masquerading, by chance, as the report's author (we believe this was a randomly chosen branch on which the author happened to be the last legitimate contributor given that the author does not possess any special privileges on his GitHub account).

This commit was pushed directly as a detached commit, not as part of a pull request or similar. In it, the attacker modified an arbitrary `Lint PR` workflow directly to exfiltrate all of our Github secrets. Unlike the previous PR attack, which could only modify the script called from the workflow, and as such could only exfiltrate our bot PAT, this commit had full write access to our repository given the ultra-permissive PAT which meant they could run arbitrary code on the scope of our Github Actions runners.

With that done, the attacker was able to run their modified workflow, and did so at 3:45 PM UTC:



Time	Event	Principal	Resource	Principal IP
Nov 23 at 5:46:15 PM	workflows.delete... GitHub Audit Logs	posthog-bot User Account	PostHog/posthog Code Repository	-
Nov 23 at 5:45:52 PM	workflows.compl... GitHub Audit Logs	posthog-bot User Account	PostHog/posthog Code Repository	-
Nov 23 at 5:45:44 PM	workflows.create... GitHub Audit Logs	posthog-bot User Account	PostHog/posthog Code Repository	-

The principal associated with these workflow actions is `posthog-bot`, our Github bot user, whose PAT was stolen in the initial PR. We were only able to identify this specific commit as the pivot after the fact using Github audit logs, due to the attackers deletion of the workflow run following its completion.

At this point, the attacker had our npm publishing token, and 12 hours later, at 4:11 AM UTC the following morning, published the malicious packages to npm, starting the worm.

As noted, PostHog was not the only vendor used as an initial vector for this broader attack. We expect other vendors will be able to identify similar attack patterns in their own audit logs.

Why did it happen?

PostHog is proudly open-source, and that means a lot of our repositories frequently receive external contributions (thank you).

For external contributions, we want to automatically assign reviewers depending on which parts of our codebase the contribution changed. GitHub's `CODEOWNERS` file is typically used for this, but we want the review to be a "soft" requirement, rather than blocking the PR for internal contributors who might be working on code they don't own.

We had a workflow, `auto-assign-reviewers.yaml`, which was supposed to do this, but it never really worked for external contributions since it required manual approval defeating the purpose of automatically tagging the right people without manual interference.

One of our engineers figured out this was because it triggered `on: pull_request` which means external contributions (which come from forks, rather than branches in the repo like internal contributions) would not have the workflow automatically run. The fix for this was changing the trigger to be `on: pull_request_target`, which runs the workflow *as it's defined in the PR target repo/branch*, and is therefore considered safe to auto-run.

Our engineer opened a PR to make this change, and also make some fixes to the script, including checking out the current branch, rather than the PR base branch, so that the diffing would work properly. This change seemed safe, as our understanding of `on: pull_request_target` was, roughly, "ok, this runs the code as it is in master/the target repo".

This was a dangerous misconception, for a few reasons:

- `on: pull_request_target` only ensures the *workflow* is being run as defined in the PR target, not the code being run – that's controlled by the checkout step.
- This particular workflow executed code from within the repo – a script called `assign-reviewers.js`, which was initially developed for internal (and crucially, trusted) auto-assignment, but was now being used for external assignment too.
- The workflow was modified to manually checkout the git commit of the PR head, rather than the PR base, so that the diffing would work correctly for external contributions, but this meant that the code being run was controlled by the PR author.

These pieces together meant it was possible for a pull request which modified `assign-reviewers.js` to run arbitrary code, within the context of a trusted CI run, and therefore steal our bot token.

Why did this workflow change get merged? Honestly, security is unintuitive.

1. The engineer making the change thought `pull_request_target` ensured that the version of `assign-reviewers.js` being executed, a script stored in `.github/scripts` in the repository, would be the one on master, rather than the one in the PR.

2. The engineer reviewing the PR thought the same.
3. None of us noticed the security hole in the month and a half between the PR being merged and the attack (the PR making this change was merged on the 11th of September). This workflow change was even flagged by one of our static analysis tools before merge, but we explicitly dismissed the alert because we mistakenly thought our usage was safe.

Workflow rules, triggers and execution contexts are hard to reason about – so hard to reason about that Github is actively making changes to make them simpler and closer to our understanding above. Although, in our case, these changes would not have protected us against the initial attack.

Notably, we identified copycat attacks on the following day attempting to leverage the same vulnerability, and while we prevented those, we had to take frustratingly manual and uncertain measures to do so. The changes Github is making to the behaviour of `pull_request_target` would have prevented those copycats automatically for us.

How are we preventing it from happening again?

This is the largest and most impactful security incident we've ever had. We feel terrible about it, and we're doing everything we can to prevent something like this from happening again.

I won't enumerate all the process and posture changes we're implementing here, beyond saying:

- We've significantly tightened our package release workflows (moving to the trusted publisher model).
- Increased the scrutiny any PR modifying a workflow file gets (requiring a specific review from someone on our security team).
- Switched to pnpm 10 (to disable `preinstall`/`postinstall` scripts and use `minimumReleaseAge`).
- Re-worked our Github secrets management to make our response to incidents like this faster and more robust.

PostHog is, in many of our engineers minds, first and foremost a data company. We've grown a lot in the last few years, and for that time, our focus has always been on data security – ensuring the data you send us is safe, that our cloud environments are secure, and that we never expose personal information. This kind of attack, being leveraged as an initial vector for an ecosystem-wide worm, simply wasn't something we'd prepared for.

At a higher level, we've started to take broad security a lot more seriously, even prior to this incident. In July, we hired Tom P, who's been fully dedicated to improving our overall security posture. Both our incident response and the analysis in this post-mortem simply wouldn't have been possible without the tools and practices he's put in place, and while there's a huge amount still to do, we feel good about the progress we're making. We have to do better here, and we feel confident we will.

Replay SDK fetch wrapper incident

contents/handbook/company/post-mortems/2026-01-17-replay-sdk-fetch-wrapper-incident.md

Date: January 14-19, 2026

Severity: Critical

Status: Resolved

Summary

A customer reported a critical issue in the PostHog SDK's Fetch API wrapper that rendered their site unusable. The issue had started a week earlier and was caused by the wrapper failing to pass the RequestInit object through to window.fetch, resulting in a TypeError for requests with a ReadableStream body.

Two SDK releases were made in an attempt to fix this bug. However, both introduced different but related regressions affecting other customers. Because the changes were in the lazy-loaded Replay extension rather than the core SDK, the issues also impacted customers who had not updated their SDK during this period.

After these regressions were reported, a bugfix release (1.327.0) was prepared. When this failed to resolve the issues, a follow-up release (1.328.0) rolled back all fetch wrapper changes.

Due to a recently introduced manual step in the process for publishing SDKs to the PostHog CDN, neither the bugfix nor the rollback was actually deployed. As a result, even customers pinned to version 1.328.0 continued to receive the broken lazy-loaded script from the CDN. The issue persisted for an additional two days until the missing deployment was identified and completed.

The original customer issue remains unresolved, but the customer has been provided with a temporary workaround.

Impact

- At least 6 customers reported issues with the PostHog SDK after the changes were released, not including the customer that reported issues with the SDK initially
- At least 4 customers reported critical issues with their production systems
- Customers were forced to remove the PostHog SDK entirely to restore functionality if they did not identify that only network header/body capture needed to be disabled
- The issue persisted across 5 SDK versions (1.323.0 – 1.328.0)
- Even customers who use a fixed SDK version and did not update the SDK during this incident were affected

Timeline

Time (UTC)	Event
Jan 14, 10:18 AM	A customer reached out to inform us that they had removed the PostHog SDK from their site as it was causing fetch requests to fail with a TypeError. They had first become aware of this issue a week prior and this was a high-severity issue for them that rendered their site unusable.
Jan 15, 5:33 PM	A new version (1.323.0) of the PostHog SDK was released with an attempted fix for the TypeError issue.
Jan 16, 5:44 AM	The customer informed us that the fix in version 1.323.0 was not effective.
Jan 16, 12:52 PM	A new version (1.325.0) of the PostHog SDK was released with an amended fix for the TypeError issue.
Jan 17, 3:31 AM	Another customer reached out to inform us that their site was down due to issues with the integrity of fetch requests and that disabling PostHog immediately caused the issues to resolve.
Jan 17, 6:43 AM	A GitHub issue was submitted describing an issue with the fetch wrapper in the PostHog SDK causing mismatched FormData boundaries.
Jan 17, 8:43 AM	A new version (1.327.0) of the PostHog SDK was released with a fix for the mismatched FormData boundaries.
Jan 17, 7:38 PM	More customer reports of critical issues with the fetch wrapper in the PostHog SDK surfaced and the decision was made to roll back all recent changes.
Jan 17, 8:13 PM	A new version (1.328.0) of the PostHog SDK was released, undoing all recent changes to the fetch wrapper, restoring it to the last known working version.
Jan 19, 4:04 PM	We became aware that the SDK version bump had not been merged into the primary PostHog repository, meaning that even for customers who had pinned their SDK version to 1.328.0, the faulty lazy-loaded script was still being served by our CDN and was still causing issues.

Root cause

The initial issue was caused by the SDK fetch wrapper being too simplistic and not passing on request options that are sometimes required – specifically any request with a body of type `ReadableStream` must include the request option `duplex: half` or `duplex: full` on all modern browsers. Even if the customer site *does* provide this option, the fetch wrapper does not pass it down to the original `window.fetch` method, resulting in a `TypeError`.

The fixes that were introduced to attempt to address this caused another issue. The updated fetch wrapper was creating a new `Request` object and passing both this object *and* the request options to downstream wrappers and `window.fetch`. This was causing the request body to be consumed multiple times which is not a problem for most requests but which results in mismatching boundaries for `FormData` requests. When the FormData boundaries do not match, the request is typically rejected by the server.

A fix for this issue was released (1.327.0) but due to the missing manual approval step, this fix was never actually deployed to the CDN. Customers continued to report problems and, believing the fix to be ineffective, the decision was made to roll back all changes to the fetch wrapper rather than attempt to diagnose further.

The decision to roll back was delayed due to a lack of understanding of the scope of the issue – ultimately we had to rely on reports from customers to get the full picture. This also contributed to the incident not being handled within the usual incident response process.

The incident was prolonged because a recently introduced process requiring manual intervention to publish SDK releases to the PostHog CDN was not followed. There was no verification step to confirm that releases were successfully deployed to the CDN. As a result, the bugfix (1.327.0) and rollback (1.328.0) releases were never actually served to customers.

Resolution

All recent changes to the fetch wrapper were reverted, including on the CDN, restoring it to the simple, original implementation:

```
const req = new Request(url, init)
return originalFetch(req)
```

The original `TypeError` issue remains.

Insights

1. **Fetch wrapper changes are high-risk:** The fetch API is fundamental to web applications; changes require extensive testing across diverse use cases, which our current test suite does not fully cover.
2. **Use feature flags for high-risk changes:** Feature flags would have enabled rapid rollback without requiring a new release.
3. **Issues with the SDK can be hard to detect:** Unlike issues with our backend systems, we do not get alerts when the SDK fails and so we relied entirely on customer feedback.

4. **There should never be a mismatch between the latest SDK version deployed to NPM and the latest version being served up by our CDN:** This is a sign that something has gone wrong in the release process.

5. **Multiple fix attempts signal deeper issues:** When fixes don't resolve the problem, step back to understand the root cause rather than iterating on partial solutions.

6. **Changes to lazy-loaded SDK extensions affect all customers:** We only test changes to the SDK extensions with the latest version of the core SDK but then we release it to customers running (much) older versions of the core SDK, without testing.

Action items

We are committed to:

- Adding comprehensive integration tests for fetch wrapper with FormData, ReadableStream, and chained wrappers
    - https://github.com/PostHog/posthog-js/pull/2935
    - https://github.com/PostHog/posthog-js/pull/2936
- Establishing a more robust testing strategy for the PostHog SDK – we should test lazy loaded extensions with *all* past versions of the core SDK or implement a way to pin a version of the extensions
- Implementing monitoring and alerting for SDK errors – consider tracking SDK exception rates, failed network requests, or other client-side signals in PostHog itself to detect issues proactively rather than relying on customer reports
- Documenting and communicating SDK release process changes to the team
- Adding an automated verification step to confirm releases are successfully deployed to the CDN
- Implementing an automated notification polling on-call engineers to manually approve new SDK releases
- Implement an easier mechanism for customers to opt in to lazy loading without opting in to auto-updating

Feature flags cache degradation

contents/handbook/company/post-mortems/2026-02-06-feature-flags-cache-degradation.md

Between February 2-6, 2026, PostHog's feature flags cache workers experienced escalating memory pressure, resulting in degraded cache update reliability. The issue was stabilized on February 6 at 22:34 UTC.

Summary

When a feature flag is updated, PostHog kicks off two Celery tasks: one to update the cache used by the `/flags` evaluation endpoint, and another to update flag definitions fetched by SDKs using local evaluation. Both tasks run on the same pool of Celery workers.

These workers experienced escalating out-of-memory (OOM) kills over a 4-day period, causing both caches to fall behind. Teams that updated flag rollout conditions or targeting rules would see those changes reflected in the PostHog UI but not propagated to the `/flags` endpoint or SDKs using local evaluation until the cache backlog cleared.

The root cause was an internal test automation system that had accumulated excessive test data over several months, creating cache update tasks that exceeded worker memory limits.

Timeline

All times in UTC.

- **Feb 2-5** – Intermittent OOM kills observed on feature-flags Celery workers; initially appeared low-severity
- **Feb 6 20:31** – Incident declared as OOMs escalate and 116k task backlog discovered
- **Feb 6 21:34** – Root cause identified
- **Feb 6 22:34** – **Stabilized**: OOMs reduced to 0-2 per 5 minutes, backlog clearing
- **Feb 7** – Internal test automation updated to use isolated environment
- **Feb 8** – Stale test data cleaned up

Root cause analysis

Accumulated test data

An internal test automation system had been running against production for several months. Due to a bug in test cleanup logic, failed test runs left behind test data that accumulated over time. This created an internal account with far more data than any typical customer workload.

No batching in cache updates

The cache update task loads all data into memory at once — flag definitions, cohorts, serialized representations, and the final JSON payload. For typical workloads this is fine, but the accumulated test data created tasks that exceeded the 8GB worker memory limit on a single execution. Each task for this account required holding all the data in memory simultaneously, causing immediate OOM kills regardless of worker age or prior memory state.

Impact

- **Stale flag evaluations**: Both the `/flags` endpoint and SDKs using local evaluation could serve stale flag definitions when cache updates were delayed
- **No data loss**: Flag definitions remained intact in the database; only cache freshness was affected
- **No downtime**: Both the `/flags` endpoint and local evaluation continued responding to requests, but could return outdated results
- **Duration**: Degraded reliability over ~4 days. Majority stabilized on Feb 6 with increased memory limits; intermittent issues continued until stale test data was cleaned up on Feb 8

Detection

The incident was detected through monitoring showing OOM kills escalating on the feature-flags Celery workers. The 116k task backlog was discovered during investigation.

OOMs were observable in the days prior, but the root cause wasn't investigated deeply at first because the numbers were low and seemed intermittent. Initial mitigation attempts focused on isolating the task to a dedicated queue and optimizing memory usage, but these didn't address the underlying issue.

It wasn't until a colleague noticed a team with an abnormal amount of data that the root cause was identified.

Recovery

1. Increased memory limits for workers
2. Enabled worker recycling (`max_tasks_per_child=100`) to give workers more headroom
3. Reduced worker load by pausing non-critical tasks
4. Purged backlogged tasks from the queue
5. Cleaned up stale test data (the actual fix)

Remediation

Completed

- Cleaned up stale test data from internal account

- Enabled worker recycling to provide more memory headroom
- Added dashboard panels for worker health monitoring and queue backlog visibility
- Fixed test cleanup bug to register resources before assertions
- Updated internal test automation to use an isolated environment

In progress

Follow-up	Priority
Better alerts, metrics, and visualizations for celery queue backlogs	High
Add metrics for anomalous workloads	Medium
Task deduplication for cache updates	Medium
Improve memory usage of cache update task	High
Merge flag caches into a single cache build	Medium

Lessons learned

What went well

- Once we started investigating, we updated the status page and made regular updates
- Once the root cause was identified, stabilization was achieved within ~90 minutes
- The immediate fix (`max_tasks_per_child`) combined with increased memory limits was simple and effective
- No customer data was lost; the issue only affected cache freshness
- The long-term fix (cleaning up stale test data) brought us back to normal operations

What went poorly

- The gradual escalation over several days wasn't investigated deeply until the backlog became severe
- OOM metrics can be misleading—pods in crash loops don't generate OOMs during backoff periods, creating a false sense of improvement
- Internal test automation running against production accumulated data invisibly over months

Key takeaways

1. **Unbounded data loading is risky**: Operations that load all data into memory work fine for typical workloads but can fail catastrophically for outliers. Consider batching or streaming for tasks that scale with customer data.
2. **Correlate OOMs with pod health**: A drop in OOM kills might mean workers are healthy, or it might mean they're stuck in crash loops and not processing anything. Always check pod status alongside OOM metrics.

3. **Isolate test environments**: Even with cleanup logic, test automation against production will eventually accumulate artifacts. Use isolated environments for integration testing.
4. **Monitor queue backlogs**: We had visibility into OOMs but not the growing task backlog. Better queue monitoring would have surfaced the issue sooner.

Logs data loss

contents/handbook/company/post-mortems/2026-02-20-posthog-us-logs-data-loss.md

On February 19th, PostHog's Logs product experienced a major incident, which caused the loss of data collected more than 3 days ago in our US region. This data loss only impacted the Logs product, all other PostHog data is intact.

Summary

As with most queryable data in PostHog, we store data for Logs in a ClickHouse cluster. When we started building Logs, we decided to use a new, dedicated cluster, rather than building it on top of our main ClickHouse cluster, which is shared across most other PostHog products. This had a few advantages, allowing us to:

- iterate faster without cross-team or organisational risk
- use later database versions without time consuming backwards compatibility testing
- optimise the cluster for Logs specific access patterns
- isolate our other product from the impact of bugs or load from logs, a very high-data-volume system

This new cluster uses S3 disks in ClickHouse, with data parts being automatically uploaded to S3 after 24 hours – this is what enables us to handle the significant data volume required for Logs (in PostHog, we alone produce about 500MB/s of logs from across our systems, or about 1PB/month uncompressed).

A bug in ClickHouse caused it to unexpectedly attempt to delete almost all of the data parts in S3. The Logs database is replicated, with two replicas, however very early on in the project we had enabled "Zero Copy Replication" in the Logs cluster nodes. This is an experimental feature that ClickHouse **does not recommend** in production, for exactly this reason: a bug that should have caused a single replica to be deleted instead deleted the data everywhere.

Timeline

All times in UTC.

- **Feb 19: 10:54**: A routine mutation was run to materialize an index
- **Feb 19: 11:02**: The mutation finished, this triggered a bug in ClickHouse's zero-copy replication which caused one of the replicas to erroneously believe all of the data parts in the database were no longer referenced
- **Feb 19: 11:02-19:40**: During this time the replicas were busy diligently deleting all of the data stored in S3 for the entire database. As data is only moved to S3 after 24 hours, and the vast majority of our queries are for recent data, no automated alarms were triggered as the volume of query errors was relatively low

- **Feb 19: 19:40**: One of the nodes in the cluster crashes and restarts, it fails to start up due to the large volume of missing data it can't find. Engineers investigate and after some checks discover that the vast majority of S3-backed data is missing
- **Feb 19: 21:45**: It is determined that the data is most likely unrecoverable – disaster recovery procedures start
- **Feb 19: 22:15**: Decision is made to cut over to a new table and restore data from Kafka, where we have 3 days of retention
- **Feb 19: 23:00**: We have switched over to the new table (without zero-copy replication) and caught up on recent messages.
- **Feb 20: 10:05**: Data backfill from Kafka history begins
- **Feb 20: 12:21**: Data backfill completes

Root cause analysis

Zero Copy replication bug

The decision to use zero-copy replication was taken extremely early in the Logs product development when it was an experimental internal-only tool.

Once Logs was released to external users this decision should have been revisited, but wasn't. Due to experiencing no issues at all during several months of internal usage, settings that had been set at the beginning were largely unvisited and unchanged.

Zero-copy replication has been largely unmaintained for the last 4 years, and still contains critical bugs, including the one we hit here. Because Zero Copy replication uses a shared storage medium (S3) for multiple replicas, when the logic on one node failed and issued delete commands for the underlying S3 objects, those files were removed for the entire cluster immediately. There was no redundancy layer between the database application logic and the storage layer.

Lack of detection

We lacked specific monitoring for the integrity of "cold" data stored in S3. Our alerts are optimized for ingestion lag, query latency, and error rates on active queries. Since users rarely query logs older than 24 hours, and the deletion process happened silently in the background without throwing application-level errors, the system remained "green" on our dashboards until the node restart forced a consistency check.

Lessons learned

What went well

- **Service Isolation:** Despite the severity of this incident, all other products and features were completely unaffected. Our decision to isolate the logs product massively reduced the blast radius of this incident.

- **Kafka Retention Strategy:** Configuring Kafka with 3 days of retention saved us from total data loss for recent activity.

What went poorly

- **Configuration Lifecycle Management:** Experimental configurations (Zero Copy Replication) intended for MVP/Alpha stages were allowed to persist into production
- **Silent Failure:** The system deleted petabytes of data over an 8-hour window without a single alarm firing. We were blind to the deletion of historical data because we only monitor the health of *incoming* data and *hot* data.
- **Backup Strategy:** Relying solely on the database replication for data durability (when using shared storage) created a single point of failure. We did not have S3 Versioning enabled on the bucket, which would have allowed us to "undelete" the files removed by the application.

Key takeaways

1. **Immediate Configuration Audit:** Disable Zero Copy Replication on all clusters immediately. Conduct a full audit of the Logs ClickHouse configuration and ensure no experimental features are used in production.
2. **Implement S3 Object Protection:** Enable S3 Versioning on the underlying storage buckets. This ensures that even if the database application issues a destructive command due to a bug, the underlying data objects can be recovered.
3. Before a product is made Generally Available, we spot check configurations and our data integrity strategies to find and correct for potential single points of failure

Public post-mortems

contents/handbook/company/post-mortems/index.md

> For PostHog employees, see the post-mortem guidance for how and when to write a post-mortem.

This page contains public post-mortems for significant incidents at PostHog. We publish these because we believe transparency builds trust, and because we think the wider engineering community benefits from shared lessons.

For security-specific incidents, see our security advisories. For real-time status updates, check our status page.

Our approach to post-mortems

We write post-mortems to understand what happened, not to assign blame. Every incident is an opportunity to improve our systems and processes. Our post-mortems typically cover:

- A clear timeline of what happened
- Root cause analysis
- Impact assessment
- What went well and what went poorly
- Concrete remediation steps

Not every post-mortem is made public. Minor incidents that partially affect services are documented internally. We publish a public post-mortem when an incident results in permanent impact on user data (such as data loss), directly disrupts customers' own services (such as SDK bugs breaking customer sites) or result in extended unavailability of PostHog services for customers (e.g. if dashboards would not load for multiple hours).

For internal guidance on how we handle incidents, see handling an incident.

Public post-mortems

- Logs data loss – February 20, 2026
- Feature flags cache degradation – February 6, 2026
- Replay SDK fetch wrapper incident – January 17, 2026
- Shai-Hulud supply chain attack – November 26, 2025
- Persons database migration – November 15, 2025
- Feature flags recurring outages – October 21, 2025
- Surveys SDK bug – October 3, 2025
- Feature flags service outage – September 29, 2025

Security advisories

contents/handbook/company/security-advisories.md

This page contains security advisories and Common Vulnerabilities and Exposures (CVEs) related to PostHog. We maintain this page to ensure transparency and help our users stay informed about any security issues that may impact them. In the event that a security incident leads to a confirmed exposure or requires action from users we will always contact users proactively.

For coverage of other, non-security incidents, please check our status page.

Our approach to security advisories

At PostHog, we take security seriously. Not as a checkbox, but with hardware security keys and healthy paranoia. We have a robust security program that includes:

- Regular security audits, architecture reviews, and penetration testing
- Automated code and infrastructure as code (IaC) linting
- Responsible disclosure program
- Proactive vulnerability monitoring
- Transparent communication with our community

For more information about our security practices, see our main security page.

Reporting security issues

Security vulnerabilities and other security related findings can be reported via our vulnerability disclosure program or by emailing security-reports@posthog.com. Valid findings will be rewarded with PostHog swag.

Updating this page

PRs to this page which update advisories or CVEs should only occur as part of an incident and should follow all our usual processes for an incident. If you need to issue an advisory or CVE and an incident is *not* declared, you should declare one.

Declaring an incident will ensure that there is good internal visibility and that members of relevant teams, including our Support team, are aware. Once an advisory is posted to this page, you should also update other teams by posting in the #tell-posthog-anything Slack channel.

Security best practices

Security is everyone's responsibility, so we encourage all our users and staff to follow some basic best practices within their own organizations.

- **Use PostHog Cloud** - We sunset K8s deployments long ago and our OSS version isn't suitable for use at scale. Use PostHog Cloud to ensure you benefit from the latest security updates.
- **Use strong authentication** - Always enable multi-factor authentication, strong passwords, and SSO where available. PostHog supports all of these.
- **Monitor access** - Regularly review who has access to your PostHog data and follow the principle of least privilege by only giving access to things people *actually* need.

We will always proactively reach out to affected users in the event of an advisory requiring attention or action. However, if you'd like to stay updated about future incidents or advisories, please subscribe to our status page. If you want to drink updates from the firehose, you can also follow our GitHub repos for real-time updates about everything we do, as we're committed to working in the open wherever possible.

Current advisories

No active advisories

Currently, there are no active security advisories or CVEs. All is well.

Past advisories

▶ August 15, 2025 / PSA-2025-00001

Advisory template

```
  <p><strong>Date:</strong> August 15, 2025<br />
  <strong>Advisory:</strong> PSA-2025-XXXXX<br />
  <strong>Severity:</strong> Low / Medium / Critical<br />
  <strong>Status:</strong>Reported / Fixed / Resolved</p>

  <h4>Description</h4>
  <p>Brief description of the vulnerability and its potential impact.</p>

  <h4>Affected users</h4>
  <ul>
    <li>Confirm if the advisory is limited to specific products.</li>
    <li>Confirm if the advisory is limited to either US or EU customers, or
both</li>
  </ul>
```

```
  <h4>Resolution</h4>
  <p>Where possible, add a link to a PR. Be clear on any next steps.</p>

  <h4>What we learned</h4>
    <p>If there's a lesson we took to prevent this happening again, document it
briefly.</p>

  <h4>Timeline</h4>
  <ul>
    <li><strong>Vulnerable code shipped:</strong> January 10, 2024, 00:00
UTC</li>
    <li><strong>Discovered:</strong> January 10, 2024, 00:00 UTC</li>
    <li><strong>Reported:</strong> January 10, 2024, 00:00 UTC</li>
    <li><strong>Fixed:</strong> January 10, 2024, 00:00 UTC</li>
    <li><strong>Disclosed:</strong> January 10, 2024, 00:00 UTC</li>
  </ul>
```

Security & Privacy

contents/handbook/company/security.md

It is critical that everyone in the PostHog team follows these guidelines. We take people not following these rules very seriously - it can put the entire company and all of our users at risk if you do not.

Overview

We maintain a robust security program that follows best practice in order to meet the needs of our PostHog Cloud customers, making PostHog the ideal solution for customers who have GDPR, SOC 2, or CCPA obligations themselves. PostHog Cloud customers own the data they send to us for processing. We collect and analyze data about the use of PostHog Cloud by our customers, but that data does not include the user data that customers send to us to process on their behalf.

This page covers SOC 2, GDPR, and CCPA compliance.

For information about security advisories and CVEs, see our advisories & CVEs page.

Multi-factor authentication

All team members are required to enable multi-factor authentication (MFA) on their accounts. **Passkeys are the preferred method** for securing all accounts — they are phishing-resistant, easy to use, and supported by most major services including Google Workspace, GitHub, 1Password, and macOS.

Please set up passkeys for Google Workspace and GitHub at the very least. If you are new, please do this within your first week so you don't get locked out.

It is recommended to have most passkeys saved in 1Password itself, which will allow you to use them from your phone.

YubiKeys for infrastructure accounts

YubiKeys are required for certain infrastructure-specific accounts as determined by the security team. If your role requires access to these accounts you will be told by the team - if in doubt ask in #team-security.

We recommend purchasing:

- One YubiKey 5C Nano for use with the work computer (can be left plugged in most of the time)
- One YubiKey 5C NFC (or YubiKey 5Ci if you have an older iPhone model) for use with mobile devices, and as backup

Setting up your YubiKeys

1. **Register your YubiKeys** with each required service. The security team will let you know which accounts need YubiKey authentication.
2. **Always register both keys** with every service so the second acts as a backup if you lose one.
3. **Disable OTP mode** — avoid spamming OTPs if you accidentally touch your YubiKey by installing the YubiKey Manager or by running `brew install ykman && ykman config usb --disable OTP`

SOC 2

import SOC2 from './_snippets/soc2.mdx'

These policies are also relevant for GDPR (see below).

GDPR

For the purposes of GDPR, customers use PostHog in one of two ways:

- PostHog Cloud
- Self-hosting a hobbyist PostHog instance

If a customer is using PostHog Cloud, then PostHog is acting as **Data Processor** and the customer is the **Data Controller**. We have some GDPR obligations to the customer's end users here.

If a customer is self-hosting PostHog then they are both the **Data Processor** *and* the **Data Controller** because they are responsible for their PostHog instance. We do not have access to any of their user data, so we do not have specific GDPR obligations to the customer's end users here.

PostHog's obligations as a Data Processor

We have reviewed our architecture, data flows and agreements to ensure that our platform is GDPR compliant. PostHog Cloud does not directly interact with our customers' end users, nor does the platform automatically collect personal data. However, our customers might collect and send personal data to PostHog for processing.

PostHog does not require personally identifiable information or personal data to perform product analytics, and we provide extensive controls for customers wishing to minimize personal data collection from their end users. We provide separate guidance for our customers on how to use PostHog in a GDPR-compliant way in our Docs.

Technical and Organizational Measures ('TOMs')

- We maintain an extensive security policies to ensure we are managing data responsibly - see above.

- We enter into Data Processing Agreements ('DPAs') with PostHog Cloud customers when requested - you can generate a DPA here. We maintain a register of all DPAs we have entered into.
- Customers can choose whether to host data on our AWS servers in the EU (eu-central-1 in Germany) or the US (us-east-1 in Virginia). If data transfer is required from the United Kingdom, EU, or EEA to our US based AWS environment, we rely on EU Standard Contractual Clauses (SCCs).
- We are registered with the Information Commissioner's Office in the United Kingdom as Hiberly Ltd., which is the legal name for our UK entity.
- A list of sub-Processors is maintained as part of our DPA - we keep this to a strict minimum.
- Our Data Processing Register is available for viewing by any interested party upon request.

Charles Cook (VP Operations) is our assigned Data Protection Officer and is responsible for overseeing compliance. Customers can email privacy@posthog.com for any questions relating to GDPR or privacy more generally.

CCPA

Under the California Consumer Privacy Act (CCPA), PostHog as a Service Provider to PostHog Cloud customers only. This is similar to the Processor definition under GDPR. We include a CCPA Addendum in our Privacy Policy.

We give all PostHog customers the tools to easily comply with their end users' requests under CCPA, including deletion of their data. We provide separate guidance for our customers on how to use PostHog in a CCPA-compliant way in our Docs.

We receive data collected by our customers from end-users and allow them to understand usage metrics of their products. We don't access customer end-user data unless instructed by a customer, and customer data is never sold to third parties. We do not have access to data collected by our customers who are using a self-hosted version of PostHog from end-users at all, unless they give us access to their instance.

Pen tests

We conduct these annually, most recently in May 2025 - you can find the report in our Trust Center.

Responsible disclosure

Security vulnerabilities and other security related findings can be reported via our vulnerability disclosure program or by emailing security-reports@posthog.com. Valid findings will be rewarded with PostHog swag.

For information about current and past security advisories and CVEs, see our advisories & CVEs page.

Reporting phishing

If you receive a phishing email/text/whatsapp, it's useful to report it to the security team so that they can make other employees aware. Take a screenshot and post it in `#phishing-attempts`. You may be asked to forward the email to security-internal@posthog.com for further inspection.

Secure communication (aka preventing social engineering)

We follow several best practices to combat social engineering attacks. See Communication Methods for more information.

Impersonating users

To provide a great customer experience, PostHog employees may occasionally need to access customer data or log in as a user (i.e. *impersonate* them). We allow this access when it's necessary to deliver our service, following these guidelines:

1. **Only impersonate when there's a clear, demonstrable benefit for the customer.**
   For example, to investigate an incident, resolve a support issue, or review a customer's setup to give recommendations on how to use PostHog more successfully.

2. **Do not make any changes to a customer's setup without explicit consent.** Exceptions to this are cases where we are reacting to incidents or bad configurations that are negatively impacting PostHog services in order to protect ourselves *and* the customer.

3. **Ask for permission whenever possible.**
   While this isn't always feasible, such as during an active incident, it's best practice to inform the customer before accessing their account. When a customer raises a support ticket, we take this as consent to be able to impersonate their account and investigate based on the contents of the ticket. Customers will not be actively asked for permission by our support engineers when they are investigating a ticket, and the customer should inform us in the ticket if they explicitly do not wish for our support engineers to access their account.

4. **Use good judgment.**
   If you're unsure whether impersonation is justified, or if a customer might object, either seek their consent or find another way to get the information (for example, by checking our internal PostHog instance).

Small teams

contents/handbook/company/small-teams.md

PostHog is structured for speed, autonomy and innovation.

Many traditional organizations have big, separate functions. You have a product team, an engineering team, customer support, and so on. This slows things down when you scale because there are more layers of communication and complex approval chains. This stifles innovation because you have to get your boss to talk to someone else's boss to get work done. It also means that people can't really see the impact of their work.

PostHog started off as a completely flat company with one big goal: to increase the number of successful products in the world.

As we are getting bigger, we anticipate that it will get harder for people to see the direct impact of their work, which reduces the sense of ownership.

We have therefore introduced small teams. These are designed to each operate like a startup. We maintain our full org chart in our ops platform.

How it works

- The overall goal for a small team is to own an area of the product/company and be as close to its own startup as possible, with only a handful of centralized processes.
- A small team should *strictly* be between 2-6 people.
- A small team has a team lead responsible for its performance - whoever is most appropriate depending on what the team is working on. This does *not* mean the most experienced person on the team.
- A small team must have a customer (internal or external).
- There may be certain functions where at our current stage we don't need a small team yet.
- Each small team runs its own retrospective + sprint every week. This must be done transparently.
- A small team has the final call in which of its features get into production, with no need for external QA/control - within our existing release schedule.
- A small team will, at some stage, be able to create its own pricing.
- A small team is responsible for talking to users, documenting what they build, and ensuring their features are highlighted in releases.

What does owning an area of the product mean?

The product small team is responsible for everything related to their area, particularly:

1. Usage
2. Quality

3. Revenue

What actions should the small teams be doing for their area?

Each quarter:

1. Create good quarterly goals

During the quarter:

1. Maintain a prioritized roadmap to help them achieve their objectives
2. Speak to customers
3. Monitor relevant metrics including those covering Usage, Quality and Revenue
4. Triage and fix related bugs
5. Assist the support hero in answering related questions
6. Collaborate with other small teams such as marketing
7. Become power users of their area of PostHog and use PostHog in their processes

What is the role of the team lead?

Overall, the team lead is responsible for ensuring the above happens. They should focus on enabling the team to solve these tasks together rather than trying to do it all themselves.

Team leads do not necessarily = managers. Read more about how we think about management.

Once a new team lead is appointed, or a small team is created, team leaders take on additional responsibilities, along with a checklist of actions. To kick off the process, run `/org-change` in Slack and select the relevant change type – it'll create a tracked issue in company-internal with the right checklist.

Team leads also take on a range of broader responsibilities that revolve around releasing new features and communicating with other teams. Some helpful guidelines on what team leads should be taking responsibility for are listed below.

Setting up support processes

Setting up support processes is a team lead responsibility, but if you need any assistance just contact the Support team directly.

Team leads are responsible for creating Slack channels for their support function and ensuring integration with Zendesk, so that the team can be alerted to support issues. Once the support process is set up, team leads are responsible for ensuring a sustainable and fair support rotation and setting up SLA and support hero notifications.

To kick off any org change, run `/org-change` in Slack.

Launching new products and features

It's the responsibility of the team lead to keep Marketing and Billing teams informed about product progress, so that product marketers can coordinate launches and the Billing team can implement pricing.

For a complete walkthrough of the product lifecycle (from initial setup through GA), see releasing new products and features and use the new product RFC template.

Some guidelines on how to do this are below, but if in doubt team leads should always aim to overcommunicate with Marketing and Billing teams.

Adding ideas to the roadmap

- [ ] As soon as you start seriously planning a new product, add it to the in-app feature preview roadmap as a `concept`.
- [ ] Inform the marketing teams a new roadmap item is available via the #team-marketing channel

Launching a new beta

- [ ] As soon as user opt-in is available, move your roadmap item from `concept` to `beta`
- [ ] Ensure your opt-in beta has a feedback link and docs link
- [ ] Inform the marketing teams a new beta is available via the #team-marketing channel

Launching a new product

Typically, you must give at least 2-3 weeks notice of a product launch and you should reach out directly to marketing team leads if this is not possible.

- [ ] Create a new launch plan issue
- [ ] Continue to communicate timelines / updates in the Slack channel created

Leading quarterly goal setting

Team leads are responsible for organizing quarterly goal setting within their team, leading the goal-setting session, and documenting the goals on their team page.

How do small teams and product managers work together?

With our engineering-led culture, the engineers on the small team are normally responsible for their area of the product.

We have a small number of product managers who support the product small teams in achieving their goals. This includes helping with prioritization, creating/updating dashboards, competitors analysis, speaking to customers etc. However, having product managers doesn't mean that the engineers can abdicate these responsibilities. The engineers should be the experts of the product they are building and their customers.

Additionally, the product managers should pay particular attention to cross-team alignment.

How do small teams and designers work together?

Similar to product, designers support small teams. Read our guide for engineers on how to work with design.

Managing larger cross-team projects

Each project should be owned by an individual within a single small team. However, some projects affect multiple other teams and require their support. For example, the performance work owned by Karl in product analytics requires support from the pipeline and infrastructure team.

For these projects, we recommend the individual owning it write a "Status update" every 2 weeks on slack and add a link to this update in the "Updates on bigger projects that affect multiple teams" section of the all hands doc. These status updates might include: what's been done since the last update, any blockers, and what are the next steps.

Small teams intros

Every small team should have an agreed charter which should include:

- Mission
- Long term goals
- Description of what the team does
- Target customer
- Who is on the team
- Key metrics

These should all be visible in the Handbook, updated when changes are made & confirmed ahead of each quarter so everyone is on the same page.

List of small teams

See the list of all small teams.

Forming new small teams

We have a defined process for proposing changes to teams, or creating a new team.

Once a decision is made, the following happens:

- [ ] Ops team updates the Org Chart in Deel.

- [ ] The team lead runs `/org-change` in Slack to kick off the tracking issue. Ops will be notified and picks up execution from there.
- [ ] Exec informs everyone else in the company in the next all hands session.

The small teams template contains a list of tasks for the Ops team and the team lead.

These include standard tasks, such as creating Slack groups and a team page to ensure the team can communicate efficiently.

FAQ

Who do small teams report to? How does this work with management?

The team lead has the final say in a given small team's decision-making - they decide what to build / work on.

Each person's line manager is their role's functional leader (if possible). For example, engineers, no matter which small team they're in, will report to an engineer.

It's important to note that management at PostHog is very minimalistic – it's critical that managers don't set tasks for those in small teams.

Think of the small team as the company you work for, and your line manager as your coach.

Can someone be in multiple small teams?

Only if they're in some kind of supportive role. For example, product managers and designers can be attached to more than one team, but product engineers should never be in more than one team because this acts against proper ownership.

Who is in a small team?

No more than 6 people, but that's the only rule. It could be any group of people working together.

Will this lead to inconsistent design?

Eventually, yes. Other companies have a UX team that build components for everyone to use. Since we currently use Ant Design, we don't need this just yet.

Can I still step on toes?

Yes. In fact, it's actively encouraged.

We still expect people to have an understanding of the entire company and what various people are working on. In engineering, we still expect you to understand how the entire system works, even if you're only working on infrastructure. You can only do your job well if you understand how it fits in with other parts of the system.

You're actively encouraged to raise pull requests or propose changes to stuff that doesn't have anything to do with your small team.

Can people change teams?

We try to keep moves infrequent and when needed. We anticipate moving people roughly every 3-9 months. We'd rather hire new people than create gaps by shifting people around.

There are two scenarios that will trigger a move:

- The small team may realize they no longer need someone, or that they could really do with someone currently in another small team internally.
- An individual team member may wish to move in order to develop their skills or experience.

It is **very important** to raise any desire for a team change with your relevant [teams/blitzscale] (Blitzscale team) member early. Any changes are at their discretion, as their job is to ensure that our small teams continue to function and that any moves fit into our current hiring plans. They will also have the best context about which teams you may be a good fit for, based on your skillset but also each team's needs. Please don't go talking to other teams directly first, as it makes it harder to manage everyone's expectations.

Aren't most small teams way too small?

In general, no – it's surprisingly great how much just 2-6 people can get done.

If more mature product areas cannot cope with the workload, small teams will clarify where we need to hire too. In fact, it'll make sure we keep the scrappy fun side of working here as we get bigger. A team doesn't *have* to be six people.

How does hiring in the small team work?

The small team is responsible for creating roles for those that they need.

We have a centralized team that will then help you hire.

James and Tim used to interview every candidate because it's a standard startup failure for founders to get too removed from hiring. We've relaxed this so that someone Team Blitzscale always interviews candidates, normally whichever team member sponsors the team the candidate will be joining.

Regardless of the team, we aim to retain a high bar for new hires. In the words of James Greenhill: "If it's not a hell yes, it's a hell no." See how we hire for more on this.

How do we create new teams, or make changes to existing teams?

See how we make team changes for a more detailed breakdown of the process.

Does a small team have a budget?

Spend money when it makes sense to do so. See our general policy on spending money.

How do you keep the product together as a company?

James and Tim are ultimately responsible for us having (i) no gaps in product (ii) eliminating duplicate work (iii) making sure all small teams are working on something rational. This is how we manage the product.

How do you stop duplicate work?

James and Tim have the ultimate responsibility to make sure we don't build the same thing in two different teams, or that we don't accidentally compete with each other internally.

By keeping communication asynchronous and transparent, this is made much easier to do than is typical at other organizations.

Can a small team "own" another small team?

Not for now, no. Perhaps when we're much larger this is something to think about.

Sprints

`contents/handbook/company/sprints.md`

PostHog works based on Sprints. These are when a Small Team meets to discuss how the last Sprint went, and what the plan is for the next one.

Sprints are shared transparently inside the company, for every team – including the Executive Team. This means people can coordinate work without having to do meetings.

- There should be a GitHub issue for the sprint up in advance and everyone should add their notes to it before the meeting starts.
- Each *individual* should come with specific written suggestions for what they'll work on over the next sprint. Note: if you're in an engineering role, product won't dictate to you what to build – it is up to you to drive this.
- The *team leader* for a small team is responsible for making sure the sprint takes place regularly.
- Any important points discussed should be written down to clarify any decisions and to help those who didn't attend.
- Teams generally meet either once a week or every two weeks.
- Everyone in a small team should attend their small team's sprint as far as possible.
- Anyone can attend a specific small team's sprints. However, all attendees should have a specific reason to be there.
- Anyone can comment on the sprint issue before or after the sprint.

Team changes

contents/handbook/company/team-changes.md

import { CalloutBox } from 'components/Docs/CalloutBox'

There are three key principles here:

1. Anyone can propose a change by creating an issue suggesting the change.
2. Decisions should be made quickly – i.e. less than a week.
3. Team Blitzscale ultimately own the decision to make a change or not.

Complete consensus isn't necessary, but there should always be time for people to share feedback, and alternative solutions, before a decision is made.

We should never run lengthy consultations, or individual meetings with all those affected by a proposed team change, but a group meeting to make a final call can be useful provided you follow the process below first.

How to propose a team change

Follow this process whether you're proposing creating a new team, splitting up an existing team, or even closing down a team.

1. Create a team change proposal issue

You can use the team change proposal template in company internal to do so. A good proposal should:

- Tag all those **directly** affected by the change, and the Blitzscale Team member directly responsible for this area of the business.
- Include context about why you're suggesting the change and the goals you think this change will help us achieve.
- Be as concise as possible. This isn't an RFC, our goal is to make a quick decision.

2. Share your proposal widely

Please share the issue in the relevant team Slack channels, the #team-blitzscale Slack channel, and any relevant public channels, requesting feedback.

It's generally best to post once and then forward that message to other relevant channels to keep things tidy.

Include the deadline for the decision in your message and tag the directly affected people.

Our goal is to make the best possible decision as fast as possible. When giving feedback, consider the following:

- Are there considerations the proposer isn't aware of that could impact the decision we make? Please share them and suggest solutions. Often these relate to feature ownership.
- Is there a better or alternative solution? Disagreeing with a proposal is fine, but it's always best to propose a solution than to just disagree without an alternative. If you think no change is necessary, explain why.
- Be direct and clear about how strongly you feel. If you're strongly against a change, explain why and make that clear. Likewise, if you're unsure about a change, but don't feel strongly, articulate that. Consensus is not our goal and decisions being blocked by people who are less invested in the outcome will slow us down and lead to worse decision-making.

3. Share the final decision in Slack

The final decision should always be made by the relevant member(s) of the Blitzscale Team in a timely fashion.

Once made, they should share their decision in #tell-posthog-anything and the relevant team channels, alongside a short summary of why we're making that change.

4. Execute the change

Once the decision is shared, the team lead kicks off execution by running `/org-change` in Slack and selecting the relevant change type. This creates a tracked issue with the right checklist, assigned to those involved.

FAQ

What if I want to move teams?

This process exists purely for making larger changes to existing teams, or forming new ones, that impact multiple people. If you are personally looking to change team, see the small teams handbook page.

What happens after a decision is made?

This is covered on the small teams handbook page.

Overview

`contents/handbook/content/index.md`

The Content team has two core goals:

1. Increase awareness of PostHog, especially among people in our ideal customer profile
2. Help developers and PostHog users be more successful through great content, videos, and docs

We do this by:

- Building a reputation for world-class content
- Constantly working to improve our documentation
- Identifying where content can help users be more successful
- Holding a high bar for quality in everything we do
- Never being satisfied with how high that bar is
- Being weird, opinionated, and unafraid of being wrong
- Reacting quickly to opportunities whenever they arise
- Not being precious about our work and priorities
- Having a team of talented, technically literate writers (and Andy)
- Not relying on freelancers or guest contributors for content
- Avoiding tedious enterprise marketing nonsense (PDFs, gated content, webinars, etc.)

Content is the main pillar of our marketing strategy. Our strategy is to go *deeper* and create better content as we grow. We don't rely on AI. We don't take content in exchange for links. We don't have arbitrary volume goals.

Our latest goals can be found on the page. You can share ad-hoc ideas in our #content-ideas Slack channel.

Who is our audience?

It should be the same as who we are building for. Specifically:

- **Product engineers:** Software engineers who want to improve their product skills, understand users, and build successful new products.
- **Founders:** Technical and non-technical founders seeking advice on how to run a successful startup.
- **PostHog users:** Existing PostHog users who want to develop their skills and learn how to get the most out of PostHog.
- **B2B SaaS companies:** Our ideal customers are B2B SaaS companies who need reliable user data and a simplified data stack. Most of our output is tailored toward B2B use cases.

What kind of content do we produce?

1. **Opinionated advice:** Articles where we offer a strong point of view on a topic that impacts our audience. Examples include The Product-Market Fit Game, Burning money on paid ads for a dev tool, and How to design your company for speed

2. **High intent SEO comparisons:** Articles for people actively considering PostHog, or searching for a product like ours. Examples include comparisons between PostHog and competing products, guides on the best alternatives to popular tools, and guides to most popular tools in our segments.

3. **Helpful evergreen guides:** Articles on topics of interest to our users and potential users. They generally target popular search terms. Examples include How to measure product-market fit, The AARRR pirate funnel explained, and 8 annoying A/B testing mistakes every engineer should know.

4. **Engineering tutorials:** Guides on how to do specific things in PostHog. These can be for existing PostHog users, or aimed at potential users who are trying to solve a specific problem. Some, like How to set up Python A/B testing are SEO focused. Others focus on specific PostHog user pain points.

5. **Newsletters:** Our newsletter, Product for Engineers, is both a distribution channel and its own content category. Issues often curate or summarize our existing content, or that of others, into an easy-to-digest, snackable format.

How we work

We work autonomously. You don't need permission or approval to write something or make a change. You're the driver.

It is often helpful to share ideas in our #content-ideas Slack channel or as a GitHub issue. The GitHub issue template provides a structure to help you think through your idea.

You can ask for feedback whenever, but it's often better when it's:

1. Clear what feedback you're looking for. Ask if specific points or examples work, how you could word something better, where do you get bored, etc.
2. You have something solid to give feedback on. Again, pull requests are better than issues.

For specific details about writing, see our style guide. When you're ready to get writing reviewed, create a pull request for your Markdown file(s) (`*.md` or `*.mdx`) in the posthog.com repo. See developing the website for more.

Once you've gone through the pull request checklist and got an approval from the relevant person on the content team, you're ready to merge (aka publish).

Content distribution

So you've written a great piece of content. Now what? Here are various ways to spread the word:

1. SEO

If we can capture search traffic, we should try to do it. Start by identifying the keywords most relevant to your article, and aim for a mix of short-tail terms (broad, high-volume) and long-tail ones (specific, lower-volume but higher intent). Use them naturally throughout the piece – especially in headings, intros, and anchor text – and make sure your target keyword appears in the meta title and meta description.

Good SEO doesn't just help your content rank in search, it also improves your chances of being cited by LLMs (aka AEO).

Follow our SEO best practices guide for more on structure, formatting, and linking.

2. LinkedIn

Share a post using either your own account or the company account, but note that the company account will have dramatically less reach than your personal one. To post using the company account, use Buffer (ask to add you to it if you don't have access).

See our LinkedIn posting advice for more.

3. Twitter / X

Again, use Buffer to post from the company account.

Tips for writing a good post:

- Write a brief summary of the post while sharing as much content as possible in it. Good example.
- Attach an image to the post (don't rely on the link's social graph preview). Again, you can use **Add entry** in the changelog to create a nice image.
- Do your best not to sound "corporate" or serious. Authenticity is appreciated on Twitter. Have fun with it!

4. Share internally

Internal teams, especially sales, CS, and the relevant product team, can often make use of the content you write if they know about it. They can share it with customers and use the ideas and examples in their conversations.

It's worth sharing in their Slack channels directly as they don't see everything we publish. Asking them to smash the like button, subscribe, and share with their friends and family is a good tactic too.

5. Paid ads + newsletters

You can promote your post by buying sponsored slots in newsletters. has a list of newsletters and booked slots we can use to promote content. See sponsorships for more.

If you want to run a paid ad campaign on Reddit, Google, or Twitter, see the paid ads page.

It's a good idea to create an issue highlighting what you'd like to achieve in your campaign. Here's an example.

LinkedIn

contents/handbook/content/linkedin.md

Yes, I realize LinkedIn has a bad reputation, but it's a popular channel with our ICP, important for recruiting, and our posts often do well there. We're posting more there, so here's what we've learned about doing it well so far.

Advice on LinkedInposting

- The hook is everything. Get people to click "show more." What works:
    - **Money** - "We spend $X and here are the best things we learned"
    - **Dilemma** - "What would you choose X or Y?"
    - **Provocative statements** - "Collaboration sucks"
    - **Transformation stories, before and after** - "This product helped us go from X to Y"
    - **Data reveals** - "PostHog has seen an ~8x increase in traffic from ChatGPT in the last year"
    - **Resource lists** - "Here are the 10 best posts on X"
    - **Takeaways** - "After weeks of researching X, I've published Y deepdive. Here are Z interesting things I learned a long the way."
    - **Work** - "I wrote a 2000-word long article on how AI impacts performance of software and systems."
- Use lists and numbers.
- Be specific with numbers. Say "$8,500" not "around 8k." This feels more credible, like you aren't making it up.
- Ask yourself if there's a story or anecdote you can use to make this real.
- Be useful. Write the posts you want to read.
- To drive clicks to links, either either say "link in comments" and add it to the post ~6 hours later or include an image in your post. The algorithm hates direct posts to links.
- A great graphic goes a lot way. "Zero-click" content like ByteByteGo gets thousands of likes with basically just a graphic. Information does better than memes.
- Add a question at the end to get comments. People want to respond. Comments boost posts in the algorithm, often more than shares do.
- Commenting on popular posts works, comments can get 30k+ impressions.
- Posting time doesn't matter to going viral.

- Posting daily beats 1-2x/week “perfect” posts. A lot won’t hit, but this will more than pay off for the ones that do.
- If you are posting a changelog update, you can create nice images when clicking **Add entry** in the changelog. It's under the **Social sharing** header.

> Thank you to for many of these tips.

LinkedIn posters and their newsletters

A primary way we use LinkedIn is to promote our newsletter, so here are some examples of people doing the same:

- Gergely Orosz - The Pragmatic Engineer
- Tom Orbach - Marketing Ideas
- Lenny Rachitsky - Lenny’s Newsletter
- Alex Xu - ByteByteGo
- Aakash Gupta - Product Growth
- Ben Lang - Next Play
- Paweł Huryn - The Product Compass
- Luca Rossi - Refactoring
- Jordan Cutler - High Growth Engineer
- Alexandre Zajac - Hungry Minds
- Gregor Ojstersek - Engineering Leadership
- Neo Kim - System Design Newsletter
- Ashish Pratap Singh - Algomaster

Writing metadata

contents/handbook/content/metadata.md

Every piece of writing we do has metadata included in it.

URLs and content folders

The URL is defined by the folder it's placed in and the `filename.md` of the markdown file - e.g. a post in the `founders` folder with the filename `i-love-hedgehogs.md` would have the URL `/founders/i-love-hedgehogs`.

Folders also decide *where* on the website articles appear.

The main folders are:

- `/contents/docs` - Docs.
- `/contents/blog` - A catch-all posts section. Company announcements, technical deep dives, SEO-focused comparisons, and more.
- `/contents/founders` - Posts written for founders.
- `/contents/product-engineers` - Posts written for product engineers.
- `/contents/newsletter` - Newsletters republished from Product for Engineers.
- `/contents/tutorials` - Tutorials
- `/contents/customers` - Customer stories
- `/contents/spotlight` - Startup spotlight
- `/contents/handbook` - The PostHog company handbook

Important: Some articles can rightfully belong in both the founder hub *and* the product engineers hub. In this case, choose the most appropriate hub folder and then add the `crosspost:` field to your frontmatter so it appears in both. So, add `crosspost: product-engineers` to post a founder's hub article in product engineers as well, and vice versa. You can also add tags from either hub like normal.

Frontmatter

This is the default frontmatter for most posts:

```
---
title: "Your headline here"
date: YYYY-MM-DD
author: ["your-name"]
featuredImage: IMAGE URL
featuredImageType: full
tags:
```

```
  - Content tag here
  - Content tag here
  - Content tag here
  - Content tag here
  - Content tag here
---
```

The frontmatter for tutorials is similar, but they don't require a featured image:

```
---
title: "Your headline here"
date: YYYY-MM-DD
author: ["your-name"]
tags:
  - Content tag here
  - Content tag here
---
```

> **Note:** Each handle in the `author` field must match a handle in the `authors.json` file. If you're a first-time author, add yourself to `authors.json` in the authors data file using this format:
>
> ```
> {
>   "handle": "your-handle", // This is what you'll use in the author field
>   "name": "Your name",
>   "role": "Your role at PostHog",
>   "link_type": "linkedin", // Or "twitter", "github", etc.
>   "link_url": "https://www.linkedin.com/in/yourprofile",
>   "profile_id": 12345 // Your community profile ID from
>         posthog.com/community/profiles/[ID]
> }
> ```

Tags

Below is a complete list of tags, organized by section.

You can use tags from the Founder's hub in product engineer posts, and vice versa, if you're crossposting the article.

Founder's hub

- Being a founder

- Culture
- Fundraising
- Growth
- Marketing
- Ops & finance
- People
- Product
- Revenue
- Sales & CS

Product engineer's hub

- Experiments
- Feature management
- Growth engineering
- Product analytics
- User research
- Engineering

Blog

- CEO diaries
- PostHog news
- Inside PostHog
- Using PostHog
- Comparisons
- General

Guides & tutorials

- product os
- product analytics
- session replay
- feature flags
- experimentation (labeled A/B testing on website)
- surveys
- cdp
- LLM analytics

Note, there are other tags we've used in the past here, but they're largely optional.

Creating new tags

Creating a new tag is as simple as adding the text to a post – it also means typos can generate new tag pages, so please be observant.

It's best to avoid a proliferation of tags, so please raise an issue before creating a new one.

Newsletter ads

`contents/handbook/content/newsletter-ads.md`

We promote our newsletter across a variety of different channels. This page covers the paid options.

Budget

Budget plans can be viewed in this spreadsheet.

Uploading emails to Substack

Annoyingly, Substack doesn't have an API, which means that we manually need to upload emails we capture via our website, InstantForm ads, and most other paid marketing channels. This means that we have to manually upload these emails to Substack. does this once a week.

The emails are to PostHog via an event called `newsletter_subscribed`, which are sent to the #newsletter-sub-alerts in Slack.

If you wish to avoid doing this, an alternative is to send users directly to our Substack so they can sign up directly there. The drawback is that we're unable to send conversion events to 3rd parties (e.g. Meta, Reddit), so their algorithms won't know if their targeting is working (and we cannot send them the emails of people who signed up, because privacy).

You can still track how campaigns on Substack by using this loophole found:

1. Sign up for Substack with another email (lior+refer@substack.com or something, can create multiple).
2. Subscribe to Product for Engineers.
3. Go to https://newsletter.posthog.com/leaderboard and get your ref code/link. You can add the `?r=1tb4kk` bit to any newsletter link and it will track who signs up using it.
4. See results here.

Meta Ads

In Q2+Q3 2025, we're testing Meta ads as a way to increase newsletter subscribers. You can view our ads in Ads Manager.

For access to the Ads Manager, please contact or .

We do not have Meta's pixel installed, as we do not allow any third-party cookies on our site. For tracking conversions, we use Meta's Conversions API via the PostHog destination.

> 🧯 **Important**: We must be extremely careful not to include any personally identifiable information. We should only include the `fbclid` parameter and the `client_user_agent`. Avoid sending personal identifiable information to Meta such as name or email.

Our ad creative can be accessed in Figma.

This issue has some information on learnings from previous ad campaigns

Instant Form ads

Meta has something called InstantForm ads, which enable users to sign up to our newsletter directly in the FB+IG apps without needing to open our website. Facebook then sends us these emails via Zapier.

Paid placements in other newsletters

> We're not running paid placements anymore due to the high cost per conversion. More details in Slack

As mentioned above, Substack's attribution sucks. Historically, we instead created a custom link for each campaign using Dub.co and calculate cost per click to measure success. However, we should now be able to track signups using leaderboard + referral code workaround mentioned above.

We generally prefer to use a pay-per-sub model, which perform better and are easier to track. This issue outlines our current partnerships with other newsletter as of June 2025.

We look for newsletters that focus on software development and engineering. We don't care about list size or reach as much as we care about clickthrough rate (you can ask for their average CTR). Some we like working with and sponsoring include:

- Pointer
- Bytes, React newsletter (same publisher for both)
- Quastor
- Tech Lead Digest, Programming Digest (same publisher for both)
- Software Lead Weekly
- Architecture Notes
- React Status, Frontend Focus, Node Weekly (same publisher)
- The .NET Weekly
- hackernewsletter
- Unzip
- Internal Tech Emails
- This Week in React

Smaller newsletters that we also have supported:

- Level Up
- Console

- FOSS Weekly (same publisher as Console)
- Fullstack Bulletin
- freek.dev

Newsletter sponsorship content

Titles that work well include:

- Product for Engineers: A newsletter helping flex your product muscles
- Product for Engineers: The first newsletter dedicated to product engineers

The main copy is some variation of:

> Product for Engineers is PostHog's newsletter dedicated to helping engineers improve their product skills. Learn how to talk to users, build new features users love, and find product market fit. Subscribe for free to get curated advice on building great products, lessons (and mistakes) from building PostHog, and deep dives into the strategies of top startups.

We have also found that linking to an article directly converts better than just a generic "subscribe to our newsletter" link.

If you need images, there is a collection of many sizes of them in Figma.

LinkedIn and Reddit Ads

We tried to run LinkedIn and Reddit ads for the newsletter but both were unsuccessful. Here's what we found:

- LinkedIn is too expensive. Cost per link clicks were north of $5, about 10x-20x more expensive than meta ads
- Reddit ads had CPCs similar to meta ads, but converted at a significantly lower rate (about 20x worse), meaning that cost per sign up was between 50–90.

Tips for new writers

contents/handbook/content/newsletter-tips.md

PostHog has unusually high editorial standards (especially for a B2B SaaS company). When I first started writing here, I struggled quite a bit. The feedback was good, but it was a lot, and for a while I couldn't tell if I was actually getting better or just spinning my wheels.

This handbook page is all the stuff I would tell myself from back then if I could. I wrote it for any future writers who join our editorial team. (It might also be useful for anyone trying to understand our unique developer content marketing and style more deeply.) It won't make the learning curve disappear, but hopefully makes it easier!

Before you write anything

Remember that you are new. Learning how to do any creative skill in a specific style takes time. Even the most talented writer in the world would make mistakes adopting PostHog's voice. As the wise old saying goes, "sucking at something is the first step towards being sorta good at something."

Don't compare your ramp-up speed to peers or people who've been here longer. Comparison is almost meaningless because everyone here has wildly different backgrounds It's why you were hired; it makes us better as a team. You have your own strengths and experiences, so make use of them.

While things are still relatively chill early on, read and absorb as much PostHog content as you can: blogs, newsletters, the handbook. Fix typos or update things as you go. Small PRs are genuinely appreciated and noticed.

> **Bonus tip:** Keep a daily work journal. The random thoughts, questions, and observations you have as you're onboarding will make for great material later on. Even this guide is based on my notes from onboarding.

Timeline expectations for your first newsletter

A lot of people underestimate how much work it takes to write a genuinely good newsletter.

Experienced writers (i.e., people who've been doing this at PostHog for years) take about **2 weeks end-to-end** to ship a great newsletter. That includes outlining, drafting, and 2–3 rounds of review to get to the finished piece (plus time juggling other projects in between).

As someone new, expect it to take longer. My first newsletter took me about 4–6 weeks:

- 1 week purely on the outline
- 5-8 rounds of feedback (shorter toward the end) spread over 2-3 weeks

- The last stretch was tiny edits and infographics over 1 week
- You will get sick of writing it. That's normal, too.

At the end, I didn't personally feel like my first newsletter was amazing, even though I was told it was very solid. Like I said earlier, remember it takes time to adapt to a new style for creative work.

To keep from going insane, have 1–2 smaller shippable projects running alongside the newsletter. In my first month, I tunneled on just the newsletter because it felt like The One Thing I had to do to prove to myself I could do this. In hindsight, putting all my confidence eggs in one basket added a lot of unnecessary pressure and honestly dampened my creativity. A blog refresh, smaller SEO post, along with handbook edits in parallel gives you breathing room. The early wins and visibility on the team are a real bonus, too.

The #1 guiding principle when writing here

Make it second nature to constantly ask yourself: *"What do I want the reader's reaction to this piece to be?"*

For example: *"I want to challenge their assumptions and make them feel surprised."*

Charles' Collaboration sucks article is a great example. It was right on the edge of clickbaity, enough for someone to comment "I was expecting to be annoyed but then I read it and was like, okay."

That's the goal. This one question should drive your title, your headings, your tone, your pacing — everything, really. A "How to do X" title can almost always be turned into something that makes a person *feel* something.

This matters most for newsletters, but it applies to everything we write. Our distinguishing factor is that we always have an opinion, a flair, a point of view. Without that, we'd become so bland, so fast.

Coming up with ideas

Ideas come from anyone and anywhere. A lot of times, conversations that happen in all-hands or Slack can be the inspiration for a blog. Basically, whatever you think might be interesting is game – you were hired as a developer who likes writing, so you have the advantage of already somewhat knowing our ICP's interests.

Even when you are new, don't let content ideas live inside your head. Turn them into a GitHub issue as soon as possible. Before you commit to writing something (including your first newsletter), have 2–3 issues with some preliminary research already done so you can make a more informed choice about what to actually write.

Not all ideas are good. Many ideas will die, fizzle out, or get picked up again later.

How to write for our readers

Writing nonfiction is a user-centered design problem. You have to start with: *who is this for?*

Our newsletter has three main reader groups:

- **Product engineers** — developers who also own product work at startups
- **Technical founders** — CEOs of early startups with technical or product backgrounds
- **Software engineers** — not always our ICP, but developers who are curious about broadening their knowledge of product and startup culture (think: a SWE at a big tech company who wonders what else is out there)

Every article should naturally appeal to at least one of these groups — ideally all three, but that's not always possible.

Once you know your reader, make sure the intro and headline appeal to them immediately. The hook should answer: *why should an engineer care?*

Don't narrow the audience too much, but don't be so broad that you capture nobody. And note that the most compelling hook for your audience might not be the most interesting one for *you* to write and that's okay.

For example, when I was writing my first newsletter, 10x job posts for 10x engineers, I first kept gravitating toward hooks like "recruiting is so hard" or "we've all read boring job ads." Those were fun and interesting to write, but none of them were actually that targeted for any of the three audiences.

I realized that the best audience was technical founders who want to hire great talent early on, so I ended up opening with "your company is only as good as your people". It's a line that's been said a million times and I personally found a little dull. But it was highly effective for technical founders.

(Think of choosing a hook like choosing a character in a fighting game: sometimes your second-highest DPS character is the best pick because they do physical damage, and the enemy has a lot of magic resistance.)

You can also angle for one group first but weave in relevance for others. For example, the 10x job posts piece was aimed at founders, but by telling them what to look for when hiring great engineers, it was also implicitly signaling to the product engineers and SWEs reading it what traits they should aspire to have themselves while job searching and interviewing.

A few smaller principles worth keeping in mind:

- Avoid double intros. Pick one good hook and then get right into the content.
- Never say "it depends". If you're leaning that way, it usually means you need to go straight to examples
- Paint the problem, then get to "how to deal with it" as early as possible. Developers want the useful part fast.

- The line between funny and cringe is very easy to cross.
- Don't lean on big enterprise company examples unless it genuinely fits the piece. It usually won't.
- Snark and strong opinions are good. Pure negativity is not.

How to actually write a newsletter

The biggest lesson I learned was that writing a newsletter is 80% research, 20% writing.

What sets PostHog content apart is that we actually do the work. We don't just say what we think, we actually go and find out.

In other words, we gather evidence usually in the form of (1) real-world examples or (2) first-hand experience to establish credibility.

Real-world examples are observed data from other companies, blogs, and people — and this is what you'll lean on most as a writer. Research examples first. They don't just support your outline; they *are* the foundation of it. You might have a strong opinion and feel certain it's right, but you don't actually know until you go find out.

For example, for the 10x job posts newsletter, the "data" I used to develop my opinion were literally job posts from other companies.

You should include real examples even during your outline phase because without them, you don't actually have an informed opinion to build on yet.

Where to find good examples:

- exa.ai — an AI search engine that's much better than ChatGPT for surfacing real examples (ChatGPT has a tendency to give you plausible-sounding examples that are just made up)
- First Round Review
- HN Algolia search — the comments can be just as useful as the posts themselves
- Slack search for what PostHog people have actually said about the topic
- GitHub — past PRs and issues at PostHog

Avoid using other blog posts as your primary source material. Basic digital literacy. "I saw it on the internet" or "I made that shit up" is not a valid source.

First-hand experience is more like "things we've learned at PostHog." Charles can write a piece that's mostly just his perspective because he has the experience and credibility as an exec — he'll almost always open with a personal anecdote or something from PostHog's history to establish that.

As a writer, you probably don't have that kind of experience to lean on yet, but you can do this too by framing things as "what we've learned at PostHog". I do this by reading all past PostHog content on a topic before I start writing, and then pulling out the real internal perspectives and examples. (Conveniently, you can save those to put in as internal links later!)

When your writing doesn't feel good

A useful gut-check: *would someone who knows a lot about this topic share this with someone else?* Worth noting that this person might not be the same as your target reader. For the 10x job posts piece aimed at technical founders, the person who'd actually share it is more like a seasoned recruiter or someone on a talent team.

If the answer is no, here's a quick diagnostic list:

- **The topic feels overdone and unoriginal.** That's okay, originality isn't really the goal. Your goal is to write something that genuinely moves *this* audience at *this* time.
- **We've written about something too similar to this in the past.** Also fine. Nobody remembers something we published two years ago. We commonly refresh old blogs into newsletters because the structure and style naturally evolves it into something different
- **It's too clickbait-y or fluffy.** You probably need to go one level deeper. If it feels fluffy, it means you're not even convinced by your own argument. Find more concrete examples to back up your opinion.
- **It's getting dry and boring.** It probably needs a stronger take. Talk to people with real experience on the subject. For example, when I was writing the "WTF does a PM do?" piece, I asked PMs directly what they thought were the most important parts of the role, and what made for a bad PM. That made it so much better.
- **The content is interesting but isn't flowing well.** Rewrite it, then rewrite it again. When I was stuck on the intro for "WTF does a product manager do?", I wrote three completely different versions, then wrote out a pros/cons list of what I liked and disliked about each before deciding.
- **It's too deep and detailed.** Try zooming out. Ask yourself "why would anyone actually read this?" Most of our newsletter topics hover at a certain level of detail. For example, we've written "An engineer's guide to talking to users," but we haven't written "An engineer's guide to dealing with difficult customers on quick calls." If the topic feels too niche, reconsider the scope (You'll probably also struggle to find enough examples for it.)
- **I can't figure out what's wrong, I just know it is.** Ask for help — especially in your first few pieces. I asked Ian to write an entire section for my newsletter because I just needed to move on at a certain point.
- **I tried all of the above and it still just feels forced.** The topic might just not be working. Depending on how deep you are, it might be worth pivoting rather than pushing through. You might just save it for a later issue.

When you hit writer's block

Writer's block is real and it will happen. It usually isn't actually about the writing — for me it's almost always anxiety or self-doubt in disguise. Things that helped me:

- Change your environment. Go work at a cafe or outside for a bit. Especially since we work fully remote, getting out of the house makes a huge difference.
- Switch to a different project. Something more tactical and less creative helps reset things and give you a little boost.
- Brain dump — stream of consciousness, no editing, just write it. Even if you don't use the output later, it's like a nice warm-up exercise.
- If your stuckness is rooted in self-doubt, try all of the classic self-care and self-compassion tips. Journaling, exercise, or whatever works for you. Staring at the doc rarely fixes anything for me, personally.

Things click eventually, but might just take longer than you'd expect. As always, don't hesitate to ask for help and feedback from the rest of the editorial team along the way. We want you to succeed!

–

Newsletter

contents/handbook/content/newsletter.md

Our newsletter is called Product for Engineers. It's owned by Andy.

Sent and managed via Substack, we put together an issue planning content for each installment of the newsletter. One person writes it and Andy edits and publishes it.

The newsletter is long-form, original copy, often based on blog posts we already wrote. It focuses on product and business lessons and information for engineers.

We run ads to drive subscriptions for this newsletter.

Art from previous newsletters is in Figma and diagrams are in FigJam.

How to write a good newsletter

These aren't rules, just things that have worked well in the past. They provide some guidance on writing a successful newsletter.

Topic

- **Write about ideas, practices, and experiences unique to PostHog.** Challenge conventional wisdom (ChatGPT is good for discovering what "conventional" is). For example, "Product management is broken. Engineers can fix it" goes against standard practice and details the way product management works at PostHog.
- **Help our audience directly.** Our audience is engineers, founders, aspiring founders. Helping them get a job or launch a startup works well. Buying software, less so.
- **Let the examples guide you.** It's ideal to have strong examples in mind before you start. These can be from PostHog (like How we got our first 1,000 users) or from similar companies (like Doist, Gitlab, and Zapier in Habits of effective remote teams). It's easy to say things, examples prove them.

Title

- **The title is the frame for the entire piece.** It is worth spending more effort on upfront. Come up with multiple options and get feedback on them if you can.
- **Be bold and direct.** Address common questions. Focus on a specific role (engineers, founders). In retrospect, "Using your own product is a superpower" is too boring and generic. Less is better: Gmail on mobile truncates titles at 35-40 characters

- **Get readers curious to learn more.** Highlight a gap between where readers are and where they want to be. Hint at exclusive or non-obvious information.
- **Some title formats that have worked well:**
  - Non-obvious lessons / advice [about topic]
  - Mistakes to avoid [doing a thing]
  - WTF is (thing) and what you should you care?
  - How to think like (person)
  - The magic of (thing)
  - What we learned about (blah) when doing (blah)
  - What nobody tells you about (thing)
  - X things we've learned about (thing)

Intro

- **Why trust us to write about this?** We can write about hiring because 900 people applied in the last two months. We can write about A/B testing because Lior has run hundreds of them. Build credibility.
- **Use a counterintuitive take as a hook.** If you're writing about something we do differently than others, the intro is a great place to start. For example: "When Tim and I first started PostHog in 2020, I was adamant we would never hire a product manager."
- **Clarify what the reader will get out of it.** A playbook, framework, lessons learned, pitfalls to avoid. Better yet, what's the benefit to them? More sales, a job, product-market fit?

Structure

- **Headings, lists, and numbers are your friend.** These help readers know where they are and create a sense of progress. 2, 3, 5, 7 are all a good number of points to aim for. 4 and 6 are awkward.
- **Use takeaways.** Help readers implement the ideas themselves. This makes posts more actionable. Non-obvious behaviors that will kill your startup does a good job of this.
- **Use pattern breakers.** Walls of text are hard to read. Make graphics in Excalidraw. Use hedgehogs. Add screenshots and quote blocks. Get more visually skilled people to help you if you need. Use these at the beginning and/or end of sections.
- **Go deeper.** Longer newsletters let us fully explore a concept. How we choose technologies ended up being ~1750 words and Product management is broken. Engineers can fix it was ~1900.

Style & tone

- **Think about rhythm:** Two long paragraphs back-to-back is tiring. Use bullet points to break things up where needed, and mix short, clear sentences with longer ones, so the pace doesn't become monotonous.
- **Break up very hard to read sentences:** Use a tool like Hemingway to identify sections that are very hard to read. Some long sentences aren't bad, but lots of them consecutively will drain the reader's attention. Aim for readability grade of 8 or less.
- **Use footnotes tactically:** They're useful for adding context that's useful, but not important enough to bog down your core narrative. If something is hard to explain and slowing things down, consider using a footnote. They're also a fun way to add jokes, rants, easter eggs, and references.
- **Be opinionated:** Sitting on the fence isn't interesting. It's ok for people to disagree with you, so avoid too much hedging.
- **Use graphics and charts:** These are great ways for explaining complex ideas and make for great social content. Create bad version and ask Cory to help you make it better.
- **Be fun and lighthearted:** We're writing about building software, not internet safety. Throw in jokes and memes occasionally. Again, footnotes and captions can be useful here.
- **But use memes sparingly:** Too many memes can become overwhelming and a distraction. One per article is probably enough – two if they're really good, or the article is on long / serious side.
- **Address the reader directly:** Say this "this will help you" rather than "this will help your company" or "this will help people". You're talking to one person, not a collective.

Publishing details

- Having a good post preview image is important. Either create one using hedgehogs from the Hoggies file in Figma or open an art request to have Lottie make one for you (give her at least 1 week to do so). This needs to be 1200x630 px for the `posthog.com` OG image and 1456x1048 px for the Substack preview image.
- We publish the newsletter on Substack and then add it to `posthog.com/newsletter` via GitHub.
- Make sure links in the newsletter point to `posthog.com` and include UTMs like `?utm_source=posthog-newsletter&utm_medium=post&utm_campaign=enter_name_here`.

Style guide

contents/handbook/content/posthog-style-guide.md

This style guide explains our guidelines for contributions to PostHog's documentation, tutorials, and blog.

Be sure to familiarize yourself with our library of MDX components that are supported in Markdown to make your article more scannable and engaging.

General principles

Assume almost nothing

As you gain mastery of a product or feature, some things become second nature, but remember they weren't always so obvious. Call these out, and provide links to relevant docs or websites.

Make it easy for your reader to implement their feature or solve their issue, whether they are an expert or just starting out with PostHog.

Get to the point

If you're explaining something, don't wait three paragraphs to do so. Start with the explanation and expand later. Almost all articles can be improved by shortening (or removing) the intro.

Don't be boring.

Make it easy to read

Most readers will scan a page before committing to reading it. They're looking for signs it'll answer their question(s) and quality.

Use clear headings, diagrams and tables to demonstrate thoroughness.

Avoid hedging

We are opinionated at PostHog. That means avoiding hedging like saying "it's complicated" or "it depends." This is frustrating for the reader and doesn't add value. Instead:

1. Have an opinion.
2. Provide an example.
3. Do the research until you can do 1. or 2.

Style rules

Use American English

PostHog is a global company. Our team and our customers are distributed around the world. For consistency, we use American English spelling, grammar, date, and time formatting.

Use sentence case for titles

Write "Documentation style guide", not "Documentation Style Guide" and "PostHog has product analytics and session replay apps", not "PostHog has Product Analytics and Session Replay apps".

But...

Capitalize product names and proper nouns as appropriate

When using a product's name, capitalize it as a proper noun, like: "PostHog's second product was Session Replay." When referring to the general industry term while *not* referencing a product name, you'd use it lowercase, like: "how many companies now offer product analytics."

Capitalize acronyms and define where needed

Write "URLs", not "urls".

Many acronyms, like that one, will be familiar to developers. When in doubt, link the first use of an acronym to a definition, or provide one.

Use the Oxford comma

Write "bananas, apples, and oranges", not "bananas, apples and oranges".

Why does this matter? Consider the old joke:

"There are two hard problems in computer science: naming things, cache invalidation, and off by one errors."

That doesn't work without the Oxford.

Use "enable", not "allow"

Allow is another way of saying permit.

> **Example:** Your partner allows you to stay up late and play video games.

Enable means providing the means or opportunity.

> **Example:** PostHog enables you to understand user behavior.

In most cases, PostHog *enables* users to do things.

Add extra line breaks between long bullet points

Sections with long bullet point items are hard to read without extra line breaks (when looking at Markdown). For example, this passage:

import Tab from "components/Tab" import ListNoNextLine from "./_snippets/list-no-nextline.md" import ListNoNextMarkdown from "./_snippets/list-no-nextline-md.md"

<Tab.Group tabs={['Markdown', 'Preview']}> <Tab.List> Markdown Preview </Tab.List> <Tab.Panels> <Tab.Panel> </Tab.Panel> <Tab.Panel> </Tab.Panel> </Tab.Panels> </Tab.Group>

Is harder to read than this passage:

import ListWithNextLine from "./_snippets/list-with-nextline.md" import ListWithNextMarkdown from "./_snippets/list-with-nextline-md.md"

<Tab.Group tabs={['Markdown', 'Preview']}> <Tab.List> Markdown Preview </Tab.List> <Tab.Panels> <Tab.Panel> </Tab.Panel> <Tab.Panel> </Tab.Panel> </Tab.Panels> </Tab.Group>

Both render as the same list, but one is easier to read in Markdown. This isn't necessary for shorter bullet-point lists.

Use straight apostrophes and quote marks

Many writing tools, such as Google Docs, Notion and Word, add curly quotes and apostrophes. Please avoid using these. They can normally be turned off in the settings.

"Open source" vs "open-source"

Both can be correct depending on usage.

Open source should be hyphenated when it appears before a noun.

> **Example:** "The open-source community is awesome"

But should be written without a hyphen in other contexts.

> **Example:** "PostHog loves being open source."

Use British-style en dashes

While we default to American English in most things, we prefer using the British-style en dash (–) with a space either side rather than the longer em dash with no spaces (—) used in American English.

> **Example:** "Don’t up vote your own content, and don’t ask other people to – post it and pray."

Please don't use a hyphen instead of en dash. On Macs, holding down `Option` and the hyphen key will give you an en dash.

> **A short public service announcement from :**
>
> As an editor, readability / aesthetics are more important to me than following grammar and style rules to the letter. British-style en dashes are a case in point.
>
> Don't get me started on using hyphens instead (like - this) – that's just wrong. Here's that last sentence with an em dash instead... "Don't get me started on using hyphens instead (like - this)—that's just wrong". Doesn't that em dash look cramped and nasty?
>
> Honestly, though, I don't care that much, but I will find and replace every em dash and orphaned hyphen on the website. It's fine. It's not a big deal. I'm cool about it.

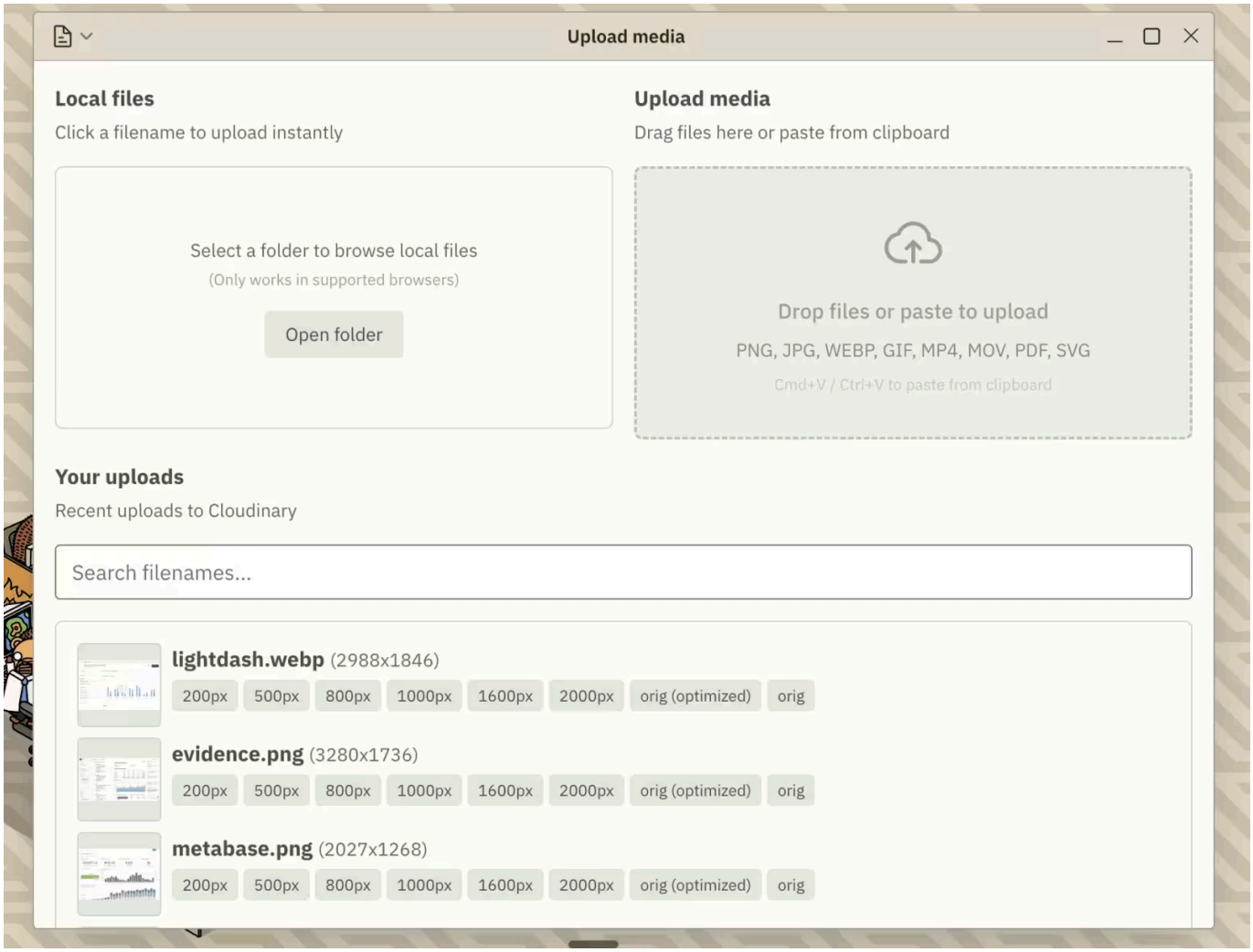
Adding media

Images, gifs, and short videos

Most media for your article should be uploaded to Cloudinary (under 20 MB).

You can do this from posthog.com by signing in, clicking on your avatar in the top right, then clicking **Upload media** in the dropdown menu (available to moderators only).

Our uploader supports images, gifs, mp4 and mov, PDFs, and SVGs.



Copy the link and paste it where you want the image or movie to appear in your file. A max of `1600px` is usually good, as this is double the typical display width of an article. Using an image twice the size of the display resolution will make screenshots look crisp on hi-DPI/Retina screens.

Use the `orig (optimized)` size when adding a `featuredImage` to an article in Markdown frontmatter, as Cloudinary's resize strategy isn't supported by our Markdown parser.

See more details in the uploading assets with Cloudinary handbook page.

There are MDX components available for embedding images or gifs (`<ProductScreenshot />`) and videos (`<ProductVideo />`).

Videos

Short videos (like screen recordings) should be uploaded to Cloudinary.

There are two other places we host videos:

- YouTube - videos that are intended for wide distribution

- Wistia - hosted videos used for embedding on PostHog.com (like our product demos) – like in product presentations, or for large videos (for blog posts or tutorials) that aren't beneficial to have on social media

YouTube embeds

When embedding YouTube videos, use YouTube's iframe embed code with the "Enabled privacy-enhanced mode" box ticked. This ensures Google doesn't drop a cookie on our website. You'll know it's enabled if the code includes "https://www.youtube-nocookie.com" in the URL. Also add the `allowfullscreen` attribute to the iframe so users have the option to watch the video in fullscreen (useful for reading code snippets).

Wistia

or can upload videos to Wistia. It's best to also have a thumbnail image which can be uploaded to Wistia as well. Videos can be embedded on the site using our Wistia component.

Best practices for images and videos

- In most cases, PNGs are the ideal file format. Images are optimized for the web and converted to `webp` automatically. That said, don't upload 4K resolution images. Be sensible.
- *Do not* upload animated GIFs. They're large and lossy. Instead, record short clips as MP4s using Screen Studio and add them to your markdown file as you would any normal image.
- If your article needs custom artwork, please file a request. See Art and branding requests for instructions.

Technical and docs writing

See our docs style guide.

SEO best practices

`contents/handbook/content/seo-guide.md`

General principles

1. Start with search intent

Don't obsess over exact-match keywords, ask: What is the searcher really trying to accomplish? For example, a user searching "difference between retention rate and churn" will likely also benefit from actionable insight on improving customer retention, not just definitions.

We craft our content to address those underlying needs. Cover the main topic thoroughly, include related sub-questions and themes, and anticipate next steps a reader might take.

Here's what that might look like for "Retention rate vs churn"

- **Quick answer first:** Retention rate shows who stayed, churn shows who left. If churn is 20%, retention is 80%.
- **Context next:** Add formulas, a simple numeric example, and a short paragraph on why retention matters for growth.
- **Related questions:**
  - What's a "good" churn rate?
  - How do you reduce churn?
  - When should you focus on retention vs. acquisition?
- **Next steps:** Link to guides on retention strategies, cohort analysis, and churn reduction.

2. Make it easy to digest

When answering a question, lead with the answer first, then expand with supporting details. This helps impatient readers and aligns with how AI tools select responses.

We keep our structure simple and scannable:

- **Use a clear heading hierarchy** (`H1`, `H2`, `H3`) so readers and crawlers can follow your logic.
  - Use one `H1` per page to avoid confusion about the page's main topic.
- **Write short paragraphs** and use bullets or numbered steps for lists.
- **Use plain language** – drop the jargon and explain terms plainly so anyone (and any language model) can understand what you're talking about.
- **Facilitate easy navigation** – use anchor links or a mini table of contents for long articles.
- **Use visuals** – charts, tables, and screenshots make key points faster to absorb.
- **Structured extracts** – add TL;DRs, "Key takeaways" boxes, or pull-quotes to highlight what's most important at a glance.

3. Headlines matter

Our headlines are the front door to our content – they're what convince someone (or an AI) to pick us. They should stand out in search results, be enticing enough to click, and still make it clear what the page is about.

We should be bold, creative, and opinionated – but not so clever that we lose relevance. If every result has a nearly identical headline, we win by being different, but if we get too abstract or too witty, we risk missing the actual query intent and dropping out of search entirely.

> **Quick rule of thumb:** If it sounds like every other search result, sharpen it. If it sounds clever but hides what the article's about, clarify it.

4. Demonstrate expertise and authority

The internet has never been so full of words. The friction for content creation has dropped to nearly zero (thanks, ChatGPT), which means the bar for quality has shot up. The only way to win attention is to raise the bar: create content that actually teaches, clarifies, and adds something new.

Establish yourself (and PostHog) as a subject matter expert. We do this by:

- **Backing our claims with data.** Include relevant statistics, research findings, or mention credible studies. Citing reputable sources or adding footnotes for facts can also build trust (and AI models tend to favor answers with a cited source).
- **Including expert insights.** If possible, add quotations or insights from experts (it can be an internal one). An authoritative quote or a first-hand insight provides uniqueness and value that generic content lacks.
- **Offering a unique point of view or proprietary data.** Bring something new to the table – something only we can. Share internal data* or a novel insight from your personal experience. Google's algorithms now consider "information gain," which measures the uniqueness of information your content adds beyond what's already out there.
- Be thoughtful about what you share – protect sensitive data and respect user privacy – but don't shy away from leveraging the knowledge only we have. Our unique perspective is our moat.

5. Be conversational

Our tone is friendly, focused, and human – especially now that voice search and AI chat engines are shaping how people consume information. Content that sounds natural and answers questions simply is more likely to show up in featured snippets, *People Also Ask* boxes, and AI overviews.

That said, conversational doesn't mean rambling. Stay on topic and be clear and direct. Think of how you'd explain the topic if speaking to a colleague – friendly but focused.

A more dialog-like tone can also help capture featured snippets or *People Also Ask* boxes, as the content directly addresses how users phrase questions.

> **Bad Q&A example**
>
> - **Heading**: Strategies for reducing customer attrition
> - **Body copy**: Customer attrition is a key challenge for many businesses and must be addressed with a comprehensive set of initiatives. Companies should consider improving their product offering, implementing proactive customer success programs, and monitoring engagement metrics over time.

> **Good Q&A example**
>
> - **Heading**: How do we reduce churn?
> - **Body copy**: Start by identifying where customers are dropping off – look at cancellation reasons, churn cohorts, and feedback surveys. Then tackle the biggest issues first, like onboarding problems or missing features. Even small fixes (e.g. a clearer onboarding flow) can reduce churn quickly. Follow-up questions we could answer: What's a "good" churn rate for SaaS? What metrics should we track to spot churn early? How do we measure if our retention efforts are working? How can we build a feedback survey?

6. Don't put all our eggs in one keyword basket

Good SEO articles always target more than one search term. While you may start with a core query (or prompt) in mind, remember there are always multiple ways to search for the same information. Sometimes it's better to target a similar but lower volume search term than the big obvious one.

For example, the parent search term "user persona" (27,000/mo) has numerous derivations:

- Define user persona (8,100)
- Create user persona (3,600)
- Persona modelling (720)
- Benefits of personas (50)
- User persona examples (5,400)
- Examples of user persona (260)
- How to create personas (2,900)
- What is a user persona (260)
- User persona template (27,000)

We target clusters of intent, not just one keyword. Long-tail variations are often easier wins and build topical relevance. Over time, our page can rank for multiple terms and even capture the broad head term as authority grows.

7. Write for our ICP

The more specific we make our content, the more likely it is to resonate – and perform. This matters more than ever with AI-driven search and tools like ChatGPT's Deep Research, which don't just answer the initial query but often fan out into follow-up questions and related recommendations.

For example, a generic "Best session replay tools" list might compete with thousands of others. But "Best open source session replay tools for startups" positions us as the exact match for a highly qualified search.

When we write, we should ask ourselves:

- **Who exactly is this for?** What unique context, goals, or constraints does our ICP have?
- **What contextual qualifiers would they use?** (e.g. "for nonprofits," "for remote teams," "for Europe in 2025") and weave them naturally into the copy.
- **What's next?** Anticipate the next three questions they'd ask after reading and answer them in the same piece. This keeps us the source that AI models (and readers) turn to as the conversation deepens.

8. Updates work / are important

Publishing a great article is not the end of the story. SEO is an ongoing process, and one of the best ways to maintain or boost rankings is to keep content up-to-date.

How often this should happen is very subjective, but the more traffic a page gets the more often it should be updated. When updating, don't just change a few words or the date; search engines are smart about detecting meaningful updates versus superficial ones. Add genuinely valuable content: new stats, a new tip, clearer structure, recent developments, etc. And if your last update was a while ago, consider adding an "Updated on [Date]" notice to show readers (and Google) that the page is maintained.

Likewise, updating and improving a page that isn't ranking is often the best way to get it to rank successfully. Just because something didn't rank at the first attempt, doesn't mean it never will.

9. Internal linking isn't optional

Internal linking is a vital part of successful SEO. It helps Google find our content and understand how pages relate to each other. It can also help prevent internal conflicts (where Google is unsure which article to list for a term), by signalling to Google what specific term we think a page should rank for.

Here are some best practices for internal linking:

- **Link early, where it makes sense.** Google tends to value links placed higher up on the page more than ones buried at the bottom. So, when you mention a concept that you have a deeper article on, link it to that first mention if appropriate.

- **Use descriptive, varied anchor text.** The anchor text (the clickable text of a link) should give a hint about the destination page's content. Instead of saying "click here" or linking the same generic phrase every time, use keywords or descriptive phrases that fit naturally in your sentence.
- **Link relevant pages only.** Ensure your internal links are contextually relevant. Don't force a link where it doesn't belong; Google can tell if links are unnatural. The goal is to guide readers to related content they'd find useful, which in turn guides search engines.
- **Don't overdo it.** A handful of well-placed internal links (3–5) is usually enough. You don't need to link every other sentence. Too many links can dilute their value and be distracting for readers.
- **Maintain your links.** Periodically, use tools or audits such as `#alerts-broken-website-internal-links` to check for broken internal links (if you reorganize pages or change URLs, update any old links). Broken links hurt user experience and can waste crawl budget.

10. Optimize for LLMs

We're no longer just writing for Google – we're writing for the answer engines too. ChatGPT, Perplexity, Claude, and Google's AI Overviews are pulling from our content to build answers. To win those spots, we need to make our pages easy to retrieve, easy to quote, and obviously authoritative.

The goal is to make our content the easiest, clearest, most trustworthy answer in the room, for both humans and machines. How we do that:

- **Clarity over cleverness:** Say the thing plainly. LLMs work best with clear, declarative sentences.
- **Structure for retrieval:** Use clean headings, bulleted lists, and short paragraphs so answers can be extracted in chunks. Each section should stand alone if it's pulled out of context.
- **Front-load the answer:** Start with the takeaway, then explain. (Think: "TL;DR first, nuance after.")
- **Semantic redundancy:** Repeat key terms and phrases naturally – it helps LLMs reinforce relevance without guessing.
- **Authority signals:** Cite sources, include data, and highlight expert input. Models tend to favor content that "looks" trustworthy. Author bios, sources, and first-hand insights boost trust.
- **Chunk quality:** Keep sections focused. A 200-word section that completely answers one question is more reusable than a 1,000-word wall of text.
- **Stay fresh and correct:** Outdated or wrong info can keep us out of results (or worse, get us quoted incorrectly). Include timestamps, years, and up-to-date references ("as of 2025").
- **Favor Q&A format:** Perplexity loves conversational answers and listicles (think "Top 5 tools for X").
- **Consistency matters:** Keep facts about PostHog accurate and aligned across different pieces of content.
- **Watch competitors:** Monitor what SGE cites and improve on those answers to outrank them.

11. Steelman competitors

Many other companies "straw man" their competitors. They claim their competitors are worse than reality, focus on differences that don't matter, and make hyperbolic claims about how much better they are. We don't do this.

When writing about competitors, be honest about their capabilities. Assume they are reading and will dunk on you for being dishonest. PostHog may not have all the features competitors have today, that's okay. Our reputation and trust with readers is more important than whatever "marketing win" being dishonest gives us.

It's also okay to make mistakes here. Competitors change faster than we can keep up. Whenever we find a mistake, we fix it as soon as we realize. We also happily accept updates from competitors if they make our post more accurate.

Additional tips

Good metadata is like a handshake – it's the first impression users (and AI tools) get before they ever see the page. Well-crafted titles and descriptions can improve click-through rates and help AI engines understand context.

Quick metadata checklist:

- [ ] Meta title includes the primary keyword and stays under ~60 characters
- [ ] Meta description is under 160 characters and compelling (can include primary or secondary keywords)
- [ ] Each page has unique metadata (no duplicates)
- [ ] Preview in a SERP simulator before publishing to check for truncation
- [ ] Add dates, numbers, or benefit-driven language where relevant to make metadata feel fresh and worth clicking.

Useful SEO tools

We use and recommend all the following tools to all writers.

Ahrefs

Ahrefs is an all-in-one tool. It's useful for:

- **Rank tracking**: We use the built-in rank tracking to keep an eye on our visibility in Google for terms we're targeting with content. It updates ranking every 7 days. We only track desktop rankings in the United States atm.
- **Competitor analysis**: Arguably the most useful feature. Use the Site Explorer feature to analyze traffic and keyword patterns for competing websites.
- **Keyword research**: There are better keyword research tools, but the Ahrefs Keyword Explorer is still a useful way to find and analyze keyword and article opportunities.

- **Site audits**: We use Ahref's Site Audit tool to identify website issues – 404s, broken internal links, etc. A scan runs once a week. Andy looks after this.
- **Backlink analysis**: Allows us to see who is linking to our website and competitors. We don't use this extensively atm, but it's useful every once in a while.

Keywords Everywhere

Keywords Everywhere is a very useful Chrome extension that adds keyword research context to Google searches and other popular SEO tools. It's a great way to do quick bits of keyword research and find related terms.

It's only ~$15 annually.

Google Search Console

While the data is somewhat sampled, Search Console is a useful tool for analyzing the top-level numbers, or specific pages. Especially useful for seeing exactly which search terms are driving traffic to a particular page – sometimes the results will surprise you.

Mangools Google SERP Simulator

A free tool that lets you test how your headline will look in Google search results. This is useful for seeing:

1. Whether Google will clip the headline because it's too long – Google has a 600px width limit on headlines.
2. Comparing your headline to other results – ideally we want headlines that stand out / are more enticing than other results

AlsoAsked

A useful little tool with a decent free tier – 3 searches per day. It generates "people also asked" questions based on search terms.

It's useful for deciding what subheadings to include in articles, though exact matches aren't really necessary.

YouTube

contents/handbook/content/youtube.md

> See also how we do video at PostHog.

We experimented with YouTube from November 2022 to July 2023, but have paused creation and publishing for now. We may try again in the future.

Although videos were driving X00s of views each (some hit X000s), and we received some positive feedback, we didn't see an increase in signups, traffic, or mentions from the videos. For example, the video on why and how we use GitHub as our CMS got 3,000 views in 1 one week, but made no noticeable impact on signups.

We also were starting to run out of obvious tutorial and SEO blog content to turn into videos. Basically, we ran out of low-hanging fruit. New videos would have taken increasing amounts of time.

Learnings

- The less PostHog-related videos did better across all three types.
- Title and thumbnail matter more than video content.
- YouTube growth compounds heavily, but it requires multiple years and 100s of videos to reach the scale we'd need for a meaningful impact.
- Lighting is what makes the most difference in video quality.
- The PostHog demo is the most important and popular video we have. It should be updated at some point.
- OBS works well to manage the recording of videos (screen, audio, webcam).
- Only 2.5% of video traffic came from `posthog.com`. 60% came from YouTube features like search, channel pages, or recommendations.

Types of videos we made

1. Tutorials like How to bootstrap feature flags
2. SEO-ish content like The best GA4 alternatives for apps and websites
3. "Essay" videos like The modern data stack sucks

YouTube comments

YouTube comments are posted to Slack using Make. It's a tool similar to Zapier, except Zapier doesn't support YouTube comments.

For access to Make, ask anyone in the . They're all admins and so they can add you.

Thumbnails

Thumbnails can be accessed in Figma

Common churn reasons

contents/handbook/cs-and-onboarding/churn-reasons.md

There are a number of recurring themes on why a customer might churn. Below is a non-comprehensive list of reasons we've encountered and some ideas around how we could mitigate churn risk in each scenario where applicable.

Situations where we have more control

Champion is gone

When a champion leaves, depending on the size of the company and more importantly, the number of active users in a given account, this can have a major impact if PostHog was principally used primarily by our champion. The best way to combat this risk is to increase PostHog's footprint across the organization and increase the number of users adopting PostHog within the org. This may mean increasing the number of teams using PostHog at a given company and the number of products the company adopts. The more people using PostHog, the less risk it poses if our champion leaves.

We can create value by engaging with different team members, building relationships with more than one champion at a given company, and help deliver value across different teams to decrease the risk of a single champion having a significant impact once they're gone.

Champion was not key decision maker

Sometimes we connect and build a great relationship with a champion who truly loves PostHog but they are not a key decision maker in their org. This can be great in terms of building a relationship and getting updates but tough in terms of ensuring there is influence to how PostHog is adopted at their company. In situations like this, it would be good to try and leverage contacts we do have to build relationships with key decision makers at the org. Note that a decision maker does not necessarily mean they are in a leadership or management role. It just means they have the capacity to make decisions and influence PostHog's adoption within the org.

Customer replaces PostHog (whether internally or with a competitor)

Customers may churn for a number of reasons, some examples are:

- They needed to build feature parity internally
- We lack a critical feature they need
- Leadership loves using a competitor
- They're very price conscious and got a sweetheart deal

Whatever the reason may be, the best way to combat churn in this situation is to increase the number of products a customer is using and the overall value the customer gets from having all their data live in one app. It doesn't necessarily prevent this from happening completely, but it does help decrease the risk.

Ideally we try to resolve the situation when customers are considering alternatives, and advocate for changes we think could help with decreasing churn risks, but in situations where that isn't possible, having customers use PostHog's other product offerings might help customers only churn for a specific product usage and not as a customer as a whole.

Customer experience has been poor

If you notice there has been a pattern where a customer has really struggled to get help or quick responses in the past, or if they've communicated this openly in your discussions, it is vital we turn this impression around by staying on top of things for the customer moving forward.

When there are opportunities to help a customer, it is recommended we provide the solution where possible, explain to the customer what we did so they have a clear understanding of the solution provided and how they can solve this themselves.

In situations where it requires us to advocate for feature requests, follow up on bug fixes, or stay on top of something for the customer, it is incredibly helpful to be proactive and frequently circle back to the customer to keep them up to date when possible rather than wait for them to follow up again.

By staying on top of things on behalf of the customer and proactively communicate, it will help develop a sense of trust, especially when customers have had a poor experience, in particular a lack of communication and follow up.

Customer hasn't been able to extract value out of PostHog

If a customer has communicated this with you, offer to work with their team to set them up for success. Make yourself available to understand their team needs, offer to set up regular meetings if they're open to it, and help them get the specific stats that would move the needle for them.

It is also possible their team may not understand how PostHog could be helpful. Offer workshops, training calls, and other things to give them concrete examples of how PostHog can help them accomplish their goals.

Lack of features or speed of delivery for specific needs

If the customer is an ICP fit, and there is risk of churn due to lacking critical features or speed in which we deliver certain results, it might be worth looping in the relevant engineering team and product manager to discuss what our options are for each specific situation that arises from this. Sometimes the key PM will want to jump on a call with the customer to learn about their specific needs.

Openly communicate in the relevant teams channel that this is a churn risk if this feature is something we can't ship. Use this opportunity as a way to help our PMs get direct feedback. We never want to lose a customer because we failed to deliver a key feature they need but these kinds of discussions are helpful to our team to learn what matters to our customers and helps us figure out if we can prioritize them or not.

Lack of trust for using PostHog as source of truth

We've heard the feedback that sometimes customers can't rely on PostHog as a source of truth because the data we collect is at odds with data they see elsewhere. This is a great opportunity to dive deeper on understanding what kind of stats they're seeing, what could be wrong with their implementation, and how we can possibly correct this so they have more trust in their PostHog data.

If a customer is relying on a different source of truth and possibly moving PostHog data to another external source, it can pose as a risk long term that they're not as tied to using PostHog, so fixing this so customers can rely on their PostHog data is important even if it doesn't pose an immediate threat.

Privacy, compliance, or data governance reasons

Some customers require strict privacy, compliance, or data governance controls. In some situations, it might be out of our control in terms of providing a solution that will work e.g. some customers can't store specific data with 3rd party services and must keep all data on prem. It's important we clarify all data controls customers do have with PostHog so they can make as concise of an informed decision as possible regarding where and how PostHog can be used. PostHog is anonymous by default and even among some of our products, such as Session Replay, we mask certain data to protect privacy. Some customers may not be aware of this and assume they can't use certain products. Helping them understand what privacy controls is available will help them be more confident in adopting certain PostHog products in this situation.

Even if we don't control local laws, industry rulings, etc., we can help our customers better understanding how to optimize their data collection, mask information, add privacy controls, or follow key compliance practices such as cookieless tracking or GDPR.

As much as we can, we should help customers better understand what they can and can't do with regards to privacy when using PostHog, and what data deletion methods are available.

Situations where we have less control

Customer has been acquired

This doesn't necessarily pose as an immediate risk or assume the customer will churn but we've seen many times where a company gets acquired and eventually moves off for a number of reasons. It would be good to learn what risks exists when you've learned that one of your book of business has just been acquired.

Customer ceases operations (for any number of reasons)

This unfortunately is completely out of our control. If a company ceases operations for any number of reasons, there's not much we can do here.

Lack of ICP fit

This is a more recent development and it can be a tough situation. In situations like this, it would be good to understand where we underserved the customer and why it was difficult or wasn't a good fit given they don't match our ICP, and help relay feedback to our team.

Learn from churn

`contents/handbook/cs-and-onboarding/customer-churn-retros.md`

Churn retros

When a human-managed account churns from PostHog, we share learnings in #customer-churn-retros. The goal is simple: learn from what happened so we can prevent it next time.

Who does this

The CSM or AE who managed the account writes the retro. Post it as soon as possible after the churn (or even when the risk is first surfaced as a possible churn) - while the details are fresh.

What to include

Keep it concise. We're looking for signal, not noise.

Basic info

- **Customer name:**
- **ARR at churn:** $X,XXX
- **Tenure:** X months/years
- **ICP fit (1-10):** X/10
    - Scoring guide:
        - 1-3: Poor fit (wrong industry, too small, misaligned use case)
        - 4-6: Marginal fit (some alignment but missing key characteristics)
        - 7-8: Good fit (matches most ICP criteria)
        - 9-10: Perfect fit (textbook ICP customer)
- **Primary reason for churn:** One sentence

What we did well

Bullet points. Be specific about what actually worked:

- Things we tried that had positive impact
- Successful interventions or saves along the way
- Strong relationship moments or engagement wins
- Features or support that resonated

What we could do better

This is the important part. Be honest:

- Warning signs we missed or ignored
- Outreach we didn't do or mistimed
- Technical issues we didn't catch early enough
- Relationship gaps or communication failures
- Contract/commercial missteps

Don't sugarcoat it. If we screwed up, say so.

Product learnings

What did this churn teach us about the product?

- Feature gaps that mattered
- Integration or performance issues
- Competitive losses (what did they switch to and why?)
- Pricing or packaging problems
- UX friction that drove them away

Tag relevant product teams if needed.

Process learnings

What do we need to change in how we work?

- Health score signals we should've caught
- Playbook gaps or broken processes
- Tools or data we needed but didn't have
- Handoff failures (sales → CS, onboarding → CS, etc.)
- Communication cadence issues

Example retro

Customer name: HogFlix
ARR at churn: $42,000
Tenure: 14 months
ICP fit: 8/10 - B2B SaaS, 75 employees, solid PMF
Primary reason for churn: Switched to Amplitude due to advanced analytics needs we couldn't meet

What we did well:

- Strong relationship with eng team, they genuinely liked us
- Proactive about billing limit management, saved them $8k over tenure
- Quick response on support tickets (avg <2hr)
- Successfully onboarded them to 4 products

What we could do better:

- Saw usage decline 3 months before churn but didn't act fast enough
- Never connected with their PM team, only eng - left us blind to analytics requirements
- Should've involved product team when they asked about funnel analysis 6 months ago
- Missed that their SQL queries were getting more complex - signal they needed more

Product learnings:

- Lost to Amplitude's behavioral cohorting and advanced path analysis
- They needed cross-product funnels we don't support well yet
- Data warehouse integration wasn't mature enough for their analysts
- Tagging @max-ai team - they wanted AI insights we couldn't deliver

Process learnings:

- Health score didn't catch declining SQL query complexity (possible new signal?)
- Need a playbook for "single-department adoption" risk - eng loved us but PM didn't know we existed
- Should trigger alert when customer starts evaluating "advanced analytics" docs heavily

---

Tips for writing these

Be direct. This isn't a CYA exercise. If you missed something, own it.

Focus on prevention. Every retro should have at least one concrete "we should change X" takeaway.

Tag people. If product or process changes are needed, @ the relevant teams.

Don't make excuses. "They were never a good fit" isn't helpful. Why did we take them on? What should we have done differently?

Keep it readable. Use bullets. Be concise. Respect everyone's time.

Customer industry segments

contents/handbook/cs-and-onboarding/customer-industry-segments.md

We have thousands of customers in PostHog, many of which are in similar industries. As CSMs having an understanding of our customers' industries can help us better be an expert on PostHog works best for their specific use cases. This page serves as a resource for us to be able to collect and share industry specific vocabulary, important metrics, PostHog best practices, etc. that allow us to quickly ramp up on the industry to better engage with those customers.

Industry segment list

These segments can change as our customer data evolves, but the following serve as a starting point:

- AI & Data
- Consumer software
- Developer tools
- E-commerce
- Education
- Enterprise software
- Finance
- Healthcare
- Logistics
- Marketing

Template for industry playbook

Eventually each industry listed above will be linked to its own playbook with details its specifics. The following is a template that can be used to create the playbook:

```
### Description (general overview of what the industry is and the businesses it
consists of)
### What they care about (i.e. what is most important to their business success)
### Industry terminology
### Common software used
### Important business metrics and data
    #### Metrics
    #### Data (event taxonomy, person profiles, groups)
### PostHog products they should be using
    #### Product
        ##### Best practices
```

```
        ##### Common challenges
        ##### Cross product use cases
```

Segments in Vitally

Industry segment is a custom account trait in Vitally. You can find and edit your customer's industry on the side panel of their account page as a pinned trait. You can add a value or edit current value directly on the account page or add the industry segment as a column to any custom tables you have in Vitally.

▶ E-commerce playbook

Customer success

contents/handbook/cs-and-onboarding/customer-success.md

This is our playbook for new customer success engagement. These are customers who have been with us for a while and we are ready to establish an ongoing relationship with them.

The core job of a Customer Success Manager (CSM) is to ensure the longevity of the customer by ensuring their overall success and that they are getting the most value out of using PostHog. This may include helping the customer with onboarding, training, support, strategies, cost-saving, and more. Ultimately, the CSM serves as the customer's champion within the company and advocates on their behalf, and ensures the customer is successful.

Four principles to bear in mind:

- **Establish strong customer relationships** - customer success is built on open and honest connections with customers and empowering their growth. This will often mean having candid conversations, getting in-depth details about what drives their business and what key metrics or goals they are trying to achieve, and being proactive in helping them achieve those goals.
- **The customer comes first** - Show, not tell. Demonstrate and communicate their value to PostHog, help prioritize their needs, and under promise and over deliver. Always be transparent and honest, and deliver bad news as early as possible. Take responsibility to stay on top of their needs and inquiries, help get support prioritized, and be strategic in your approach to help them achieve their goals.
- **Provide customer value** - Beyond helping customers get optimal use out of PostHog, help with strategies, cost-saving, or other timely solutions that can benefit their business. We are always looking for ways to help customers save even if it means in the short term we don't make as much. The long term value and care matters more. Offer continual education on new features and create a cadence of communication that is helpful but not intrusive. If you're seeing something that is off, check in with them to see if they are aware of it.
- **Become the voice of the customer** - Listen with intent and advocate for your customers. You are their champion within the company and should help their voice be heard whether that's creating feedback loops to make them feel heard or get their needs prioritized, or working cross-teams to help product and engineering understand why certain features are important to the customer and their success. Advocacy only works if you're willing to go to bat for your customers.

Maximizing your chance of success

As a CSM, you'll be spending most of your time managing your book of business and investigating churn signals so that there should be **zero surprises** should a business churn. Your first initiative should be focused on establishing a relationship with your book of business and prioritizing your understanding of

their business, how they use PostHog today, and where you can add the most value to their business. It helps to approach this from a viewpoint of how you can be most helpful to your book of business as you learn what drives their success.

In order of priority, your objectives should be:

- Create a clear introduction of who you are, what a CSM can do, and what values you can provide for your customers so there's no confusion. Not everyone is familiar with what a CSM does or is clear on why they should engage with one, especially if your book of business are long time customers who may not have had a dedicated point of contact till now.
- Evaluate your assigned list for signals and churn risk and prioritize outreach accordingly. Pay extra attention to recent conversations or open support tickets that you can dive head-first into to ensure these customers are cared for. Zendesk, our support ticketing system, allows you to follow tickets and get notified when there are updates, so add yourself to open tickets for any of your assigned customers.
- Set up a Vitally Playbook notification to get alerts when new conversations occur for your book of business, allowing you to keep a pulse on when your customers write in. Use this Playbook Template as a reference. You should also set up a Vitally Open Tickets Tracker to view all of your customer's open tickets in a single view. To do this, clone this Open Tickets Tracker, remove the existing filters, then add a filter for the assigned CSM, and save to your account.
- Add yourself to all your existing customer Slack channels and invite customers who are not in Slack yet to offer them an easier way to communicate with our team for their needs. Make sure to add Pylon and relevant team members on our side. Adding customers to Slack has the added benefit of giving us an additional communication channel, as email is usually one of the worst ways to reach our ICP (though you should start with email). We've found that informing customers you'll be sending them a Slack Connect invite, then sending the invite, works significantly better than asking customers if they'd like to join us on Slack. A couple of tips to set up Shared Slack Channels with Customers

Tips on success engagement

- Highly recommend reviewing our section on getting people to talk to you. What we've found works really well when establishing an initial connection is to be candid about wanting to learn more about your customer's business, how they are currently utilizing PostHog, and to get a better understanding of where you may be able to add value for them. Most customers are pretty receptive to wanting to help, especially if it can benefit them in some way, so don't be afraid to ask directly.
- Review your existing book of business to see how many products each of your customers is currently engaged with so that in your conversations, you can better understand why they may or may not be using certain products or if they find upcoming beta features useful. Make note of how

many of them are on specific plans and if there are any opportunities you could help the customer save (always a great topic to help interest customers to engage).

- Don’t focus on a specific champion during your initial outreach. When prioritizing outreach, do look at engaged users and user types but aim to connect with multiple team members. You never know who will be responsive and become your best point of contact. Don’t focus on just reaching out to an identified champion (if one exists).
- On initial engagement calls initiated from you to learn more about the customer, it is helpful to prepare a simple agenda and re-iterate this on the call to make it easier for customers to understand what you'll be discussing, particularly if you're not familiar with their business, and what you hope to get out of the meeting, then wrap up with a summary of any action items you'll be taking to follow up with the customer. This can be a great time to gauge if the customer is interested in setting up a recurring check-in call.

Feature request tracking

`contents/handbook/cs-and-onboarding/feature-requests.md`

When working with our customers, they will occasionally ask for features which aren't in the product yet. We won't build a niche feature for a single big customer, but if we can see a request being of benefit to multiple customers, we should capture, track and feed it back to our product teams.

Urgent vs Non-urgent requests

If a customer is at risk of churn, or otherwise unhappy about the missing feature, then we should communicate this to the relevant team in their Slack channel (usually #team-xyz). Adding in the urgency, ARR and tagging the team lead is a good approach here to get some focus. Remember that you still own the customer and may need to follow up with product teams to get the right level of focus as they don't have all of the customer context that you do. Don't create false urgency where there is none - we only want to use this approach when things are *actually* urgent.

For non-urgent requests we should capture them in Vitally using the process on this page, and then share them with the teams in their Slack channels ahead of quarterly planning.

Tracking feature requests in Vitally

Current Feature Request List

We track feature requests a custom object in Vitally. You can see the current list of feature requests here. It's filterable by team, and shows the accounts and combined ARR of those accounts who have asked for the feature. There's also a Kanban board view which helps you track the progress of requests.

Adding a customer to an existing request

1. Open up the request by clicking on the title of it
2. Under Accounts near the top of the request click to Select an Account
3. If the customer has specific context or a link to a Slack discussion then add it into the text area at the bottom of the request UI. Also add in the contact information of the person asking for it, if it's not a Slack thread.

Creating a new request

If you've checked the list above and can't see an existing request then you should create a new one. You can do this in two ways:

1. When looking at the list of features, there is a Create new button in the top left of the UI.
2. When looking at an account, you can see the feature requests they are connected to in the related objects section of the UI. There's a Create new button at the top of that UI as well.

Most of the fields are self-explanatory, and the status should almost always be set to `Requested` if it's a new one, unless the team is actively working on it. Make sure you add as much context in the text area at the bottom as possible, with links to Slack/Zendesk tickets.

Basic account review

contents/handbook/cs-and-onboarding/foundation-check.md

When working with our customers, it is important to do a basic account review to get a better understanding of whether we think the customer has things set up correctly. Below is a simple checklist of things to look for, and address on your discovery call with the customer, to get a better idea of whether they are set up for success or not. It's important to note that we can only check for certain things and some things (like backend implementation) will rely on us speaking to the customer to get a better understanding of their implementation.

Events and events properties

A good starting point is verifying what events the customer is tracking, whether they have custom events set up, is autocapture enabled, and if they're collecting any event properties. These could be custom event properties or they could be autocapture attributes that have been added. Two good starting places to look are:

- **Activity**: this showcases recent events
- **Data management**: under here, you can find a number of definitions available to look at, to see if the customer may have set up custom actions or defined event and property definitions.

One of the things to look for is whether the customer has any custom actions setup. This is quite useful and is often a good sign that customers might have their setup wrong if they have not made use of actions. Some common actions to potentially look for are *renaming of events* that are more useful to the customer or bundling of events that are common such as `user signups` or `purchases`.

Additionally its worth flagging if a customer has autocapture enabled and no actions setup, that's probably worth flagging with the customer.

Reverse proxy configured

Another good place to start is to see if the customer has reverse proxy setup. There are two ways you can do this:

- If the customer is using session replay, you can to the appropriate replay video, select **Activity > Inspector > Doctor**, and search for "config", look for PostHog Config, expand the `api_host` and observe whether the URL shown is showing one of our domains i.e. us.i.posthog.com or eu.i.posthog.com, or one of their domains.
- If the customer isn't using session replay, the alternative way you can check this by adding `?__posthog_debug=true` to a URL where PostHog is being called, and pull up console logs and type `posthog.config` and look for the `api_host` property there.

If both methods are unavailable to you because the customer isn't using session replay and the hosted site isn't publicly accessible, you can simply ask the customer on the discovery call.

Persons Properties, Group Properties, and Cohorts

Next, we want to look to see if customers are making use of persons properties, check if there are signs they may be over identifying, and if they are making use of cohorts. It is beneficial to understand what sort of person properties the customer is adding, potentially look for signs of properties they might be missing base on what you understand of their business, and the kind of cohorts, if any, they are using.

If group analytics is enabled, its worth checking to see if they have group properties set and if the type of properties makes sense. Because group types are limited to five, it's important to make sure the group types are set up in a way that makes sense, and the related persons profile makes sense in the way it's associated with group properties.

Ecommerce Events

For ecommerce customers specifically, PostHog has a useful guide on ecommerce events specification that is worth checking to see if these customers were aware and have implemented custom events tracking related to the type of events we'd normally like to see (such as `sku`, `product_id`, or `category`).

SDK or library version

Make sure customers are using an up to date SDK or Library version. For this, you'll want to click on **Activity**, click on `Configure columns`, then add `Library` and `Library Version` so you can see versions of the SDK they are using. Then you can reference this against our Github repos for the up-to-date SDK versions to check if they are on the latest versions or not.

Alternatively, you can go to Metabase, look up the `Library version audit` table and see SDK versions there.

Sign up for an account when possible

If the customer's product offers a free account you can sign up for, do it and go through the workflow. This is a great way to see if events are firing properly, what events are being tracked, and get a rough idea of what might be missing so that you can make recommendations on your discovery call with the customer.

What types of dashboards does the customer have set up?

Every customer and more specifically, every team, will have a different set of goals they deeply care about. What we want to see here is if they've spent time setting up custom dashboards or insights to track specific trends, engagement, conversion metrics, or other key dashboards that indicates they're

measuring the right things beyond basic dashboards included by default. This could be things like `user sign ups`, `retention dashboards`, or `free to paid upgrades`. What you want to do is get a feel for the kind of tracking the customer has setup so that on the discovery call, you can understand if this currently aligns with their immediate goals or if there are key metrics they should be looking at but have not setup.

Are customers using data pipelines for event notifications?

This idea actually came from our own team's use of data pipeline destinations to get notified when specific events occur in Slack. It's a great additional use that could be helpful to companies that didn't consider this use case for data piplines and an easy way to try and upsell.

Query failure rate

This is a good one to check and is available in Vitally. This usually means the customer is attempting to do something and it didn't work. Great for expanding on the discovery call itself.

Getting started with customers

contents/handbook/cs-and-onboarding/getting-started-with-customers.md

As a CSM, it is your responsibility to be the expert on each of your customers, whether or not they choose to engage with you. Obviously, you'll learn more about customers that you actually talk to, but there are still plenty of ways to get to know an account, learn their use cases, and track their journey from all of the data available.

Many customers have never spoken to PostHog – some happily welcome our help, others are strongly independent. In order to be successful as a CSM, we want to understand our customers and be helpful.

Get people to talk to you also has good, helpful tactics.

Newly assigned accounts

When you're assigned a new customer account, your approach will vary depending on the existing relationship between the customer and PostHog. This guide walks you through some key steps you can take when welcoming new customers to your book.

Determine which category your customer falls into

No/low interaction with PostHog humans

These are customers who have been using PostHog but haven't had much direct contact with our team. You should conduct a more thorough assessment before your first call with them. If you haven't done your initial outreach yet, you can also use the assessment to customize your message with a specific tip from your learning.

Account and business audit

Start by gathering context about who they are and how they're using PostHog:

Understanding their business:

- What industry are they in? What products do they make?
- If you have other customers in their industry, does their usage of PostHog fit what you've seen before?
- For an even deeper dive on their business, the Sales team has a thorough account planning template that they use for cross-sell/expansion that you can take guidance from.

Reviewing their PostHog setup:

- What PostHog products are they using?

- What PostHog products are they *not* using?
- Does their setup look complete? Are they paying for products they don't use?
    - Going over the customer deployment health check guide will help you answer these questions.
- Review their product onboarding status.

Data management assessment: In their project(s), check the data management tab:

- Do they have custom events defined?
    - If not and they're using autocapture, have they defined actions?
- Do custom events have relevant properties defined?
- If they're identifying persons or groups, have they defined a meaningful set of properties on those profiles?

Answering these questions helps you identify the most important things to focus on in your initial engagements. Take a look at our basic account review page for additional things to check.

Introduce yourself

Once you've completed your audit, start reaching out. If this is an account that's being handed over from an existing contact:

- Review the Sales > CSM Handover process.
- Get introduced in their existing Slack/Teams channel or via email.
- Coordinate with the previous point of contact to ensure continuity.

If you don't have an established contact, introduce yourself to the widest blast range: org owner, org admin, users who have recently raised tickets, and users who have logged in in the last month. Even if there *seems* to be a point of contact, things probably changed – multi-thread!

Your intro message should:

- Introduce yourself as their new CSM
- Describe the value of CSM – dedicated point of contact, go-to person for help or questions, help the customer use PostHog effectively, e.g., through training or strategic guidance. Many customers misunderstand the CSM role as 'just support', so make sure to distinguish your role as a CSM apart from that.
- Value nugget: show how you can help by delivering value-add

Examples of a value nugget

Take a look at your customer's account in Vitally and Metabase to identify ways you can be helpful. Some examples include:

- Increase/decrease in events: make sure this is expected and things are implemented correctly

- Recently opened a support ticket: follow up to make sure their issue is resolved
- Concrete ways a customer can optimize their spending or improve their implementation
- Invitation to a shared Slack channel so it's easier to connect with our team.
- Lots of new users or low user engagement: offer a training session on how to use PostHog effectively
- On a legacy pricing plan: "We've moved off the legacy plan for more than a year, and I'd like to transition you to standard pricing. Happy to discuss the changes."

If there's an established Slack channel you are inheriting, do it in Slack.

Example subject lines:

You should find what you're comfortable with whilst keeping a sense of PostHog's tone of voice. Some examples include:

- Hello 👋 from your new CSM at PostHog + hook
- hi from PostHog
- Checking in from PostHog

In Vitally, you can see how other team members have reached out to customers in the past by going to an account's Active conversations tab for inspiration.

If there is no response, follow up after 2-3 business days, targeting individuals in the engineering, product, or data team. Emphasize the purpose of your reaching out - you're not trying to sell them something, you want to understand their use case and help optimize their PostHog integration.

Connect with champion

1-1 email or Slack message

Aim: start the relationship with a champion, ideally in the engineering, product, or data team

Content: Acknowledge that their time is valuable and that you will not be selling or pitching. You want to understand how to better serve the customer by understanding how they use PostHog. Would they be open for a 15-minute call? Offer to do this async as well.

Pro tip: If they're not already in Slack, don't ask; add them to Slack by sending them a direct invitation. If this is an account without an established Slack channel, you can follow our guide on shared Slack channels to set one up.

Getting-to-know-you discovery call

This is one of the most effective ways to learn what you need to know about a customer, as you can ask direct questions and spend a lot of time listening to their responses. A quick call upfront is often better than a month of back-and-forth in Slack.

Typically, this is a 15-30 minute conversation aimed at establishing rapport, understanding pain points, and beginning to formulate how you can best assist them.

Your discovery call should help you determine the level of engagement you'll have with the customer going forward. Think through the following questions:

- What is the goal you want to achieve with this customer? (Keep an eye on them vs. become more deeply embedded as a strategic partner)
    - Do they need help fixing their current setup?
    - Do they have plans/interest in implementing new PostHog products?
        - If the answer to either is yes, you can be their strategic partner and collaborate on setting up a detailed success plan.
    - Some customers may not want to engage deeply, and that is okay; still, continue to monitor their usage/spend and check in with them when appropriate.

Preparation before your call

Some things to consider before your call:

1. Understand the customer's PostHog usage:
    1. What products are they using? How are they using it? What metrics do they care about from those products?
    2. What products are they **not** using? This means products that make sense for them to use, and you want to understand why they aren't using them.
        1. For example, product analytics and web analytics are closely coupled. If the customer is using product analytics but not web analytics, understand why. Is there a reason for that? What's the objection?
2. Call out feature preview ✨
    1. Explain what feature preview is and how to enable it
    2. Recommend PostHog AI as it's usually relevant regardless of customer use case
    3. Otherwise, recommend new products that the customer likely already has (e.g., Messaging, CRM) – position it as 'You probably already have [product], this is a product we're trying to launch and would love to see how you would use it / any feedback you have. Keen to relay or rope in the engineering team directly with your feedback.'
3. Q&A on product
4. Next steps and ideal catch-up cadence.

Additional questions to consider for your call

Here are some recommended questions you could use. Please do not simply interrogate a customer with each of these questions; this is more of a question bank to use for inspiration!

- What is your role at the company, and what team are you on?
- What does your company do?
- What are your immediate and overall goals?

- Can you describe your current analytics setup and any specific tools or libraries you use alongside PostHog?
- Which user flows or features do you most want to understand better?
- Are there any areas you feel like there are blind spots or gaps in understanding?
- What problems have you encountered with analytics (e.g., ad blockers, data privacy, endpoint reliability)?
- For your team, what does success look like after implementing a new analytics solution?
- Are you more comfortable with direct SQL or do you prefer visual dashboards?
- How do you make decisions about scaling, feature adoption, and pricing flexibility?
- Who will be the main users of PostHog on your side?
- Do you have any concerns about compatibility or integration with your current stack?
- Anything weird happening in PostHog?
- What metrics do you deeply care about?
- Do you feel you are set up for success with your current PostHog setup?
- Which teams are currently active in using PostHog at your company?
- Are there opportunities for other teams to adopt PostHog?
- Would you mind making introductions to champions on other teams to learn about their use case?
- Review and discuss their current implementation and if they have any concerns.
- Would training or workshop sessions be useful for you and your team?
- How do you feel about PostHog overall?

Customer Prioritization

- Consider a separate approach for monthly and annual customers:
  - Annual plans: prioritize accounts with contract renewals in the next 3-4 months
  - Monthly plans: look for significant growth within the last quarter
- Accounts with platform packages
  - Customers on legacy "Teams" add-on ($450/month) could save $200 by switching to the "Boost" add-on if they do not require SAML SSO or managed reserve proxy. The teams add-on has now been split into:
    - Boost add-on ($250/month)
    - Scale add-on ($750/month)

Analyzing product usage

While PostHog itself is (obviously) the gold standard for understanding how customers are using our product, we also make it very easy to view this information within the account context in Vitally and in Metabase.

We use the PostHog CDP to send product events to Vitally so that we can see which specific users are most active, MAUs on an account, and how many paid products they use. We can see more specifics in the Metabase dashboard, as well.

These sources will both help you identify potential cross-sell and upsell opportunities, in the name of helping customers maximize their value in the product.

Past conversations, tickets, and Slack channels

A very valuable part of account research is also reviewing past conversations. This will give you an idea of what level of contact we've had, who the main contacts may be, what issues they've faced, and so on.

The key places to look for this information:

- **BuildBetter**: will contain recordings of customer success, sales, or onboarding calls. Once in BuildBetter, you can use the "People" data section to search for companies or individual contacts and then see the history of calls, or you can do a direct search or AI chat.
- **Pylon**: will contain the Slack channel history. You can view the "account" page, which is linked to Salesforce accounts for any of your accounts that have a Slack channel. You can filter on "Owner", which should also be mapped to the account owner from Salesforce, which lets you view all Slack or Teams interactions from accounts you own.
- **Vitally**: will contain Zendesk and email conversations under the "Active Conversations" section. This will allow you to see who the key contacts were, which support, sales, or CS individuals they have worked with in the past, and so on. It's also really helpful to see how frequently they raise tickets and what issues they have faced.

Get notifications

We use Watch Tower to monitor news about companies in our book of business and surface what matters. To get started, create an account with your PostHog email. Once in, you can create a list and select to import your book of business from either Vitally or Salesforce.

Once your book has been imported, it's important to go through and make sure the names are correct, and each entry has the correct domain. Watch Tower uses the domain to understand context about the company you're trying to monitor and both Vitally and Salesforce rarely include domain data accurately.

Finally set up how you like to be notified. Email is the default but you can also set up Slack notifications as well. Once you've ensured your list is updated correctly and notifications are set, every day Watch Tower will scan the news for info relating to any of your companies. If there's a match, you'll get a notification.

Recommended approach

The best recommendation is to find your own rhythm for how you, as an individual, prefer to learn about your customers. There's not a strict playbook. This is a compilation of the most reliable sources of knowledge to use for researching an account.

Handling customer issues

contents/handbook/cs-and-onboarding/handling-customer-issues.md

As a dedicated PostHog human for customers, you're the first point of contact for customer issues. This helps build your relationship as a technical point of contact, plus you have the most context on the customer and can help with proper escalation.

The support team and engineering teams are always available to help, but you should try to solve issues yourself before handing off to other teams. This also helps you level up your product knowledge.

Raising issues

Zendesk

We use Zendesk Support as our internal platform to manage support tickets. For specifics on how we use Zendesk, look here.

Tickets created from Slack

Customers can create tickets from Slack by adding the emoji reaction to their message. This is useful as customers can receive help even when you're asleep or on holiday. Make sure you let your customer know about this capability and it's also worth periodically reminding them about it.

If this isn't working as expected, make sure you've invited Pylon to the channel. Fill out the automated message asking for `Group` and `Severity` so the ticket is routed to the right team (customers sometimes forget so help fill it for them). Check feature ownership if you're unsure which team is responsible for a product area.

If you're investigating a ticket that your customer raised in Slack, let support know you're on it to avoid duplicate effort. You can do this by leaving an internal note directly in Zendesk.

> Tip: Customer messages from channels with Pylon also go to #support-customer-success. You can find the ticket in the channel and leave a message in the thread. This also creates an internal note in Zendesk.

Investigating issues

When investigating customer issues, it's helpful to ask for specifics – e.g. links to the insight, feature flag, or dashboard; a screenshot of the error or the specific error message.

If helpful, you can log in as the customer into their PostHog org. Clicking a link from a customer's PostHog instance will sometimes give you the option to login as the customer. Alternatively, log into US admin (EU admin), search for the org or user, and click "Log in as user". If you're not seeing this option,

ask to add you as a staff member in admin.

When investigating, use our docs, look at troubleshooting tips, search through Slack, Zendesk, GitHub, or Pylon for similar issues. If you've just joined, try to spend 30 mins to 1 hour investigating by yourself before asking for help. Onboarding is the best time to learn about PostHog products! Obviously, balance this with the urgency of the issue and use common sense.

While investigating, keep the customer in the loop by communicating progress, blockers, next steps etc.

Escalating tickets

You can escalate tickets to either the support team or the relevant engineering team. The decision depends on:

- You need help with additional context or further digging ➡️ support
- The issue requires deep technical domain knowledge ➡️ engineering

Our support team are technical engineers and can answer the majority of tickets. If in doubt, escalate to support.

If you're escalating to support, you don't need to do anything special - the ticket will stay in the support queue.

If you're escalating to engineering, in Zendesk, set the `esc.` dropdown in the left sidebar to escalated and double check the group assignee makes sense. You might need to upgrade your Zendesk role to full agent, just remember to downgrade after.

When escalating tickets, leave an internal note saying whether you're escalating this to engineering or support (and why) – so it's clear who should pick it up. Also include details about the investigation you've done and observations you've made. Even if it's confirming that you followed the customer's reproduction steps and saw the same issue, that context is incredibly valuable.

Auditing impersonations

Customers sometimes ask who from PostHog has accessed their account. You can use the following SQL query on project 2 to get a log of impersonations for a specific organization. You can get the organization ID from Vitally.

```
-- Get all user emails for an organization from persons table
WITH org_users AS (
    SELECT DISTINCT
        properties.email as user_email,
        properties.org__name as org_name
    FROM persons
```

```
    WHERE properties.organization_id = 'ORGANIZATION_ID'
        AND properties.email IS NOT NULL
)
SELECT
    e.timestamp,
    ou.org_name,
    e.properties.target_user_id as target_user_id,
    e.properties.target_user_email as target_email,
    e.event,
    e.properties.staff_user_email as staff_email,
    e.properties.mode as mode,
    e.properties.reason as reason
FROM events e
JOIN org_users ou ON e.properties.target_user_email = ou.user_email
WHERE
    e.event IN ('impersonation_started', 'impersonation_upgraded')
    AND e.timestamp >= now() - INTERVAL 30 DAY
ORDER BY e.timestamp DESC
```

Checking the health of a customer's deployment

contents/handbook/cs-and-onboarding/health-checks.md

In a world where a lot of our high-paying customers have self-served without ever speaking with a PostHog human there is scope for them to implement PostHog in a less than optimal way. This could result in people spending more than they need to, or having inaccurate reporting data available to them. Ultimately if left unchecked these things will lead to avoidable churn.

Are they paying for things they don't need?

Group analytics

Group Analytics can be a real value-add for B2B companies, allowing them to track analytics at the company or workspace level rather than an individual person. They do however need to implement group tracking in their PostHog SDK. Customers who haven't done this may end up paying for Group Analytics but not able to use it.

We have a Vitally risk indicator added to customers who are paying for Group Analytics but not using it.

To help the customer you should figure out whether they are B2B or could otherwise benefit from sending group information. If so, reach out with guidance. If not, reach out telling them that they can save by removing the Group Analytics add-on from the billing page.

Autocapture

Autocapture is a great way for users to get up and running with event capture without a huge engineering effort. Autocapture can however get very noisy very quickly, and if users aren't leveraging these events they may not be getting value out of them. You can understand a customer's Autocapture event volume from their Metabase customer usage dashboard (instructions above on how to get there). There is a breakdown of the Key event volume Last 30 days which shows the number and % of Autocapture events they are sending across all projects. If that is high (>50%) then check the Actions (by type) visualization on the same dashboard to see if they have any Autocapture actions defined. If not they are likely to not be benefitting from Autocapture events.

If they aren't benefitting from Autocapture you should reach out to let them know how best to use it. Alternatively, they can tune or turn it off by following the Autocapture configuration docs.

Session replay targeting

When Session replay is enabled it will capture all sessions by default. As every session is counted for billing purposes, customers may end up with a bunch of low value short recordings and still be paying for them.

If a customer has Session replay enabled, log in as them and look at their session replay settings. At a minimum we recommend setting the minimum duration to 2 seconds or more but there are other tuning options which they may also benefit from.

Have they implemented tracking incorrectly?

Calling identify too often

A common pattern is for users to call `posthog.identify()` on every page, or in an endless loop. Whilst this won't break their tracking (unless they use different distinct IDs in the identify call) they will end up with a drastically inflated event volume. You can diagnose this by looking at their Metabase usage dashboard in the Key event volume visualization. If either the volume of $identify or $set events is higher then 5% then something has likely gone wrong in the implementation.

You should get in touch and let them know that they only need to call `posthog.identify()` once per session.

Calling groupidentify too often

As with `identify()` above users may also end up calling `posthog.group()` more than they should. In the Key event volume visualization in Metabase if the $groupidentify count is higher than 5% they've likely set it to call once per page.

You should get in touch and let them know that they only need to call `posthog.group()` once per group per session, or when the group changes.

To see where duplicate groupidentify calls are being generated, you can use the following SQL:

```
SELECT properties.$lib AS lib, count() AS groupidentify_event_count
FROM events
WHERE event = '$groupidentify'
  AND $session_id IN (
    SELECT $session_id
    FROM events
    WHERE event = '$groupidentify'
    GROUP BY $session_id
    HAVING count() > 1
  )
  AND timestamp >= now() - INTERVAL 30 DAY
  AND timestamp < now()
GROUP BY lib
ORDER BY groupidentify_event_count DESC
```

Calling posthog.reset() before identifying the user

`Posthog.reset()` will generate a new anonymous distinct ID. If this is called before a user is identified then two anonymous unlinked user may be created. There is no easy way to proactively diagnose this however if a customer says that their tracking between web and app is off, this is a common culprit.

We have guidance on when to call `posthog.reset()` in the JavaScript library features guide.

Reverse Proxies

It is best practice for a customer to use PostHog's Managed Reverse Proxy or to configure their own for events to be sent from their own domain.

When using either PostHog's managed reverse proxy or deploying a non-managed reverse proxy, events should populate the "Library custom API host" property. Host mapping and domains can potentially be seen in Metabase. You should verify the setup with a customer.

Cookieless tracking

If a customer mentions their user/event count seems to be missing a lot of data from their website, ask them if they have implemented cookie opt-in and to share the part of their code where PostHog is initialized. Some customers may not be aware that we have specific recommendations for how to initiatlize PostHog for cookieless tracking.

For example, if they implement PostHog on their website similar to as follows:

```
posthog.init(...,
    opt_out_capturing_by_default: true
)

if (cookiePreference === 'accepted') {
    posthog.opt_in_capturing()
}
```

They will not be capturing anything for customers who visit their website and opt-out of cookies or ignore the cookie banner completely. We recommend instead they use the `cookieless_mode` parameter in their initializer as outlined in the cookieless tracking tutorial. If the customer wants to move forward with implementing cookieless mode, ensure they enable "Cookieless server hash mode" in their project settings under Project Settings > Web analytics.

Cookieless mode can help them have more accurate tracking totals because when using cookieless tracking, the PostHog SDK will generate a privacy-preserving hash, calculated on our servers.

Are feature flags resilient?

Falling back to working code

It is important that hitting the flags endpoint does not block an application from otherwise functioning correctly. If the flag fails to load or returns an unexpected value for any reason, such as `None`, `(empty string)`, or `false` you should always fall back to working code.

Server side local evaluation

Implementing Server-side local evaluation will ensure that flags continue to return values regardless of the network status of the flags endpoint. By default, PostHog will attempt to evaluate the flag locally using definitions it loads on initialization and at the `poll interval`. If this fails, PostHog then makes a server request to fetch the flag value.

As a note, server side local evaluation is billed differently than other flag requests.

Customer health tracking

contents/handbook/cs-and-onboarding/health-tracking.md

We use Vitally as a customer success platform. You can log in via Google SSO to view customer data but will need Mine or Simon to grant you admin access to let you manage your accounts. It integrates with our other systems such as PostHog, Salesforce and Zendesk to give you a complete view of what's going on with your customers.

Health scoring

Overview

Health scores are a great way to assess whether your customer is at risk of churn or in a good state and are a common pattern in Customer Success tracking. We compute an overall health score out of 10 based on the following factors and weighting. You can read more about how Vitally health scores work in their docs here.

Health score metrics are divided into two categories: Customer Engagement (25%) and Product Engagement (75%).

Customer engagement

Score Name	Measuring	Weighting
User engagement	Are they using PostHog regularly?	15%
Product experience	Are there negative experiences with the product?	5%
Company engagement	Are they engaging with PostHog humans?	5%

Product engagement

Score Name	Measuring	Weighting
Product Analytics	Event volume and users analyzing insights	33%
Session replay	Replay volume and users analyzing replays	20%
Feature flags & Experiments	Flag requests, users creating feature flags, users creating or viewing experiments	17%
Surveys & Data warehouse	Users creating and viewing surveys, volume of rows synced	5%

Customer engagement

Non-product metrics, looking holistically at: Are customers using PostHog? Do they have friction when using PostHog? Are they engaging with PostHog humans?

User engagement

This tracks whether users are logging in to PostHog. It can tell us if customers are getting value from PostHog (regardless of the products they're using). Customers that have a low active user percentage, or only have 1-3 users engaging with PostHog are at risk of churn.

Measure	Poor	Concerning	Healthy
Last seen in product	>5 days	1-5 days	≤ 1 day
Active user percentage	<20%	20-40%	≥40%
Percentage decrease in active user percentage	>20%	5-20%	≤5%
Users engaging with features	<3	3-10	≥10

Product experience

This looks at the experience of using PostHog.

Creating a lot of tickets can mean users are not satisfied with PostHog, haven't implemented PostHog correctly or aren't using the product correctly (opportunity to offer training)! Similarly, visiting docs can mean users are trying to do something and could need help.

We also look at query failure rate. Failed queries are common (users can cancel a query, there can be SQL syntax errors, etc.), however, a high failure rate means users aren't getting the data they need from PostHog. You should help investigate and provide recommendations.

Measure	Poor	Concerning	Healthy
Tickets created in last 30 days	>10	5-10	≤5
Urgent tickets that remain unresolved	>2	0-2	0
Docs visited in last 7 days	>100	20-100	≤20
Query failure rate in last 7 days	>13%	5-13%	≤5%

Company engagement

This looks at a customer's engagement with PostHog as a company. Most of PostHog's customers are happily self served so this is weighted very little in the overall healthscore.

Measure	Poor	Concerning	Healthy
Most recent meeting	>90 days	30-90 days	≤30 days
Most recent ticket	>90 days	30-90 days	≤30 days
Total product count	<3	3-6	>6

Product engagement

Across PostHog's products, we look at 2 factors – data volume & user engagement.

Data volume

This tracks *percentage decrease* in data volume over the last 30 days. We use success metrics to track billable usage over the last 30 days and compare it with the previous 30 days on a rolling basis. The percentages you see in the tables below are the *decrease* between the previous and current period.

User engagement

Data volume is a lagging indicator, by the time it drops, customers may have already decided to churn. We combine data volume with product-specific user engagement, measuring the percentage of *active users* interacting with product features over the last 14 days.

> There are products we don't include in the health score. Vitally has a limit of max 20 health metrics so we are excluding other products for now as the overall ARR from them are still very low compared to the others.

Product analytics

Measure	Poor	Concerning	Healthy
Event count last 30 days (percentage decrease)	>20%	5-20%	<= 5%
Active users analyzing insights	<20%	20-40%	≥40%

> Product analytics usage include: analyzing insights or dashboards, creating or saving insights, creating or updating dashboards

Session replay

Measure	Poor	Concerning	Healthy
Replay count last 30 days (percentage decrease)	>20%	5-20%	<= 5%
Active users watching replays	<20%	20-40%	≥40%

Feature flags & experiments

Measure	Poor	Concerning	Healthy
Decide requests last 30 days (percentage decrease)	>20%	5-20%	<= 5%
Active users creating feature flags last 30 days*	<5%	5-20%	≥20%
Active users using experiments**	<5%	5-20%	≥20%

> Feature flag usage includes: creating or updating feature flags. We look at this over 30 days instead of the usual 14 as feature flags provide value over a longer time frame. Experiments usage includes: creating experiments, viewing experiments, and launching experiments.

Surveys & data warehouse

Measure	Poor	Concerning	Healthy
Active users viewing surveys	<5%	5-20%	≥20%
Rows synced last 30 days (percentage decrease)	>20%	5-20%	<= 5%

Account indicators

Health scores are useful for tracking the long-term trends in an account, but occasionally there are more immediate point-in-time events that we should react to. These are tracked as indicators in Vitally and fall into one of two categories

- Risk indicators - show up red against the account name and indicate potential churn
- Opportunity indicators - show up green against the account name and indicate a potential opportunity for growth

Risk indicators

These are automatically applied via Vitally playbooks (see the Risk category here):

Forecasted MRR decrease

Applied if the Forecasted MRR Change is less than -10%, indicating a drop in MRR. We should look into the account to understand whether it is just a reduction in usage, or they are trending towards churn.

Increased billing page visits

Applied if there have been more than 1 visits to the billing page in the previous 7 days. Can be a good indicator that the customer needs help understanding or reducing their bill.

Query failure rate > 10%

Applied if the Query failure rate over the last 7 days (Success metric) is greater than 10%. Use Vitally to see which user was impacted and see if you can help optimize their queries or flag to our team for investigation.

Sudden decrease in event volume

Applied if the Event count last 7 days (Success metric) decreases more than 20% versus the previous 7 days. Indicates that they may have turned event tracking off.

No insights analyzed past week

Applied if `insight analyzed` was last seen greater than 7 days ago. Indicates that they may have stopped using PostHog to track analytics data.

Payment failed

Applied if there is a failed payment on their Stripe account. We should reach out to get this resolved ASAP.

Startup credit will run out this billing cycle

Applied if they are currently in the Startup plan segment but also have Forecasted MRR, meaning that they are on track to make a payment this month.

Organization owner recently removed

Applied if the Owner role has been removed from a user in the last 14 days. May be a sign that you've lost a champion.

Opportunity indicators

These are automatically applied via Vitally playbooks (see the Opportunity category here):

Forecasted MRR growth

Applied if the Forecasted MRR Change is more than 10%, indicating an increase in MRR. We should look into the account to understand whether it is likely to be deliberate or an accidental spike.

Organization owner recently added

Applied if the Owner role has been added to a user in the last 14 days. This is a good opportunity to reach out to a potential champion if you've not met them before.

How we use automation in Customer Success

contents/handbook/cs-and-onboarding/how-we-use-automation.md

How we use automation

Customer Success at PostHog means managing ~30 accounts per CSM while maintaining deep, meaningful relationships with each customer. Automation and AI tools can help surface important signals and streamline repetitive tasks, allowing CSMs to focus on strategic guidance and relationship building.

Automation should never be used as a replacement for human connection and interaction, but mainly as a tool to help a CSM be better prepared, informed, and effective.

Current automation stack

PostHog CS leverages several integrated tools to monitor account health and identify opportunities:

Core monitoring systems:

- **Vitally:** Tracks usage patterns, billing changes, health scores, and engagement metrics.
  - Opportunities and Risks are surfaced via "Indicators"
  - Health scores update regularly as a composite of multiple metrics with different weights
  - Data is synced in from Salesforce, PostHog, BuildBetter, and Stripe
- **PostHog pipelines:** Alerts for usage milestones, new product adoption, and behavioral changes. These are sent to Vitally, Salesforce, or Slack via PostHog CDP for alerts and data updates.
- **BuildBetter:** Analyzes customer calls for feature requests, pain points, and sentiment.
  - Notes from calls are automatically synced to Salesforce and Vitally
  - feature requests and painpoints are automatically added to Vitally and sent to #feature-request-feed channel
- **Zapier:** Used for numerous automations such as:
  - Renewal reminders
  - Stale Slack channel notifications
  - Billing updates, failed payments

Key automated workflows

Account monitoring triggers include:

- MRR changes exceeding certain thresholds generate investigation tasks
- inactivity periods to flag engagement reviews
- New product usage (any amount) creates cross-sell opportunity indicator
- Payment failures and low credit balances to send Slack alerts to assigned CSM
- Health score changes trigger Vitally indicators

- Annual renewal dates trigger preparation workflows in advance and a ping in Slack

Human-first automation philosophy

Every automated workflow includes deliberate human decision points. For example, when an account begins using session replay, Vitally creates an indicator suggesting outreach about their use case - but the CSM determines whether and how to engage based on the account relationship and context.

This approach ensures automation enhances rather than replaces the human elements of customer success.

Working effectively with automations

Best practices:

- Review automated tasks such as Vitally indicators, Slack alerts, and customer health scores at least twice weekly.
- Treat automated insights as starting points for investigation, not final answers!
- Set-up individual alerts in Vitally or PostHog CDP that match your own portfolio and experiences
    - As an example, setting up your accounts as a Cohort in PostHog and then setting up CDP alerts/notifications to Slack based on product usage and activity.

What remains purely human:

- Initial customer responses and relationship building
- Renewal negotiations and strategic planning
- Technical implementation guidance
- Complex problem-solving and consultative conversations

Requesting new automations

CSMs are encouraged (as are all PostHog employees) to experiment and surface new ideas frequently in Slack or team stand-up. Examples of areas where automations could be useful include, but are not limited to:

- Time savings versus implementation complexity
- Impact on customer experience
- Requirement for human judgment
- Scalability across the team

How we work

contents/handbook/cs-and-onboarding/how-we-work.md

> This page covers more of the operational detail of how our team generally works - for a broader overview of roles and responsibilities, visit the customer success team page.

Main metrics for each role

- Technical CSM: revenue retention

Book of business

Customer Success Managers

Each CSM is assigned customer accounts accumulating to ~$1.5m ARR to work with. We use the CSM Managed Segment in Vitally to track this against goals and CSMs should not assign this themselves (that's up to Dana or Charles).

Weekly Customer Success standup

In addition to the weekly sprint planning meeting on a Monday, we do an account review standup on Wednesday to discuss any at-risk accounts.

The objective of the meeting is to hold each other to account, provide direct feedback, and also support each other. It is a great place to ask for help from the team with thorny problems - you should not let your teammates fail.

How contractual bonus works - Technical CSMs

CSMs are responsible for ensuring that a larger book of existing customers - both annual and monthly - continue to use PostHog successfully. They nurture customers and are product experts - this isn't a role of just going back and forth between customers and support engineers, or collecting feedback.

> This plan will *also* almost certainly change as we scale up the size and complexity of our success machine! As above, we will always ensure folks are treated fairly when we make changes.

Variables

- Your OTE comprises an 80/20 split between base and contractual bonus.
- Bonus is paid based on revenue retention above 100%, and is *uncapped*.

    - For example, if you have 100% revenue retention and your target is 120% revenue retention, you get 0% of bonus. For 120% retention, it's 100% bonus, and for 140% retention, it's 200% bonus. This is on a sliding scale so if you hit 110% retention you get 50% bonus.
    - The Q4 2025 target is 120% half-yearly NRR. This may change in future depending on how things go.
    - To calculate retention we use the total usage over the past 2 quarters and annualize this, then compare it to the 2 quarters before that.
        - For monthly customers this is the total of their 6 invoices multiplied by 2
        - For annual customers, we look at the usage-based MRR and multiply by 2
        - For newer customers: if there's at least 2 quarters of revenue, we use quarter-on-quarter comparison for that specific customer. If it's a brand new customer with less than 1 quarter of revenue, we start measuring next quarter.
- Bonuses are paid out quarterly, and in any case after an invoice is paid
    - Bonus payments are made at the end of January, April, July, and October - at the end of each quarter, we'll monitor how many invoices actually get paid in the first two weeks of the next quarter. Fraser will send you an email that breaks down how you did.
- Your bonus is guaranteed at 100% for your first 3 months at PostHog - this gives you time to get up to speed, but also if you over-perform then you will get your additional bonus.
- If an account is added to your book:
    - If you inherit a new account that hasn't been managed by a PostHog human before, you have a 3 month grace period - if they drop or churn in that initial period, they won't be counted against you. We want to encourage you to right-size customers, rather than your deliberately letting them wastefully spend money due to some poor implementation.
    - If you inherit an account from another CSM, AE, or AM, it will count toward your NRR in that quarter, even in the first 3 months.
- How bonus is calculated:
    - In general, we compare annualized ARR over the past 2 quarters with annualized ARR 2 quarters before.
        - For Q4 2025 bonus: (Q4 + Q3 ARR) vs (Q2 + Q1 ARR)
        - For Q1 2026 bonus: (Q1 + Q4 ARR) vs (Q3 + Q2 ARR)
    - For customers on annual plans, we will look at their usage-based spending (instead of total contract amount / 12)
    - If an account is removed from your book mid-quarter, they will not be included in bonus calculation. We do this extremely rarely, even if a customer shuts down. If we have to give a customer a big refund, we'll deal with your bonus on a case by case basis depending on what happened, but usually this will still be counted.

Account allocation

- CSMs manage approximately $1.5M in ARR.
- This coverage amount will grow ~10% quarterly to match our growth targets.
- When rebalancing accounts (e.g., if accounts drop below the $20k threshold), we'll bring you up to the current quarter's target amount.

Working with engineering teams

We hire Technical CSMs. This means you are responsible for dealing with the vast majority of product queries from your customers. However, we still work closely with engineering teams!

Product requests from large customers

Sometimes an existing or potential customer may ask us to fix an issue or build new features. These can vary hugely in size and complexity. A few things to bear in mind:

- Engineers at PostHog talk to customers. It's much better to bring engineers onto calls to speak to large customer to talk to them directly than just do the call yourself and copy and paste notes back and forth. This is especially useful if a) the team was already considering building the feature at some point, b) it's an interesting new use case, or c) the customer is really unhappy for valid reasons and could churn.
- Provide as much internal context as you can. If a customer sends a one-liner in Slack, don't just copy and paste into a product team's channel - find out as much as you reasonably can first, ask clarifying questions up front etc. Otherwise the relevant team will just ask you to do this anyway.
- We already have principles for how we build for big customers - if you have a big customer with a niche use case that isn't applicable to anyone else, you should assume we won't build for them (don't be mad!)
- For any feature requests customers care deeply about, we should file and track those in Vitally.

Finally, if you are bringing engineers onto a call, brief them first - what is the call about, who will be there. And then afterwards, summarize what you talked about. This goes a long way to ensuring sales <> engineering happiness.

Complicated technical questions

You will run into questions that you don't know the answer to from time to time - this is ok! Some principles here:

- Try to solve your own problems. Deep dive the docs, ask PostHog AI, ask the rest of the sales team first - a bit of digging is a valuable opportunity for you to learn.
- Similar to the above, don't just copy and paste questions from Slack with no context. Add some commentary - 'they have asked X, their use case is generally Y, I think the answer might be Z - is that right?'. Do some of the lifting here, rather than putting all the mental load on an engineering team.

Working with customers in Slack

Most of our customers use Slack, and it's a great way for us to be responsive to them. Everyone has the permission in Slack to create a Connect channel with a customer, and you should do this as early as possible in your relationship with them.

When you've created the channel you should also add Pylon, which is used to sync Slack conversations with Zendesk so that our Support and Engineering teams can work on customer issues in a familiar context.

To add Pylon to your customer channel:

1. In the Slack desktop app, click the channel name.
2. On the Settings tab, click Add apps.
3. Type Pylon and click Add.
4. In the popup that appears in the Slack channel, select Customer Channel.
5. Add yourself as the Account Owner.
6. Click Enable.
7. Add Tim, Charles, and Abigail to the channel.

Once enabled, you can add the emoji to a Slack thread to create a new Ticket in Zendesk. Customers can also do this. Make sure that a Group and Severity are selected or the ticket won't be routed properly.

> It's your job to ensure your customer issues are resolved, make sure you follow up with Support and Engineering if you feel like the issue isn't getting the right level of attention.

Tools we use

Gmail We use Gmail for our email and the team uses many different clients from Superhuman to Spark to the default Gmail web interface. Find something that works well for you. To get your own email signature, copy the signature from someone else on the team (like Simon) and then fill in your own details.

Calendly: We use Calendly for scheduling meetings. In order to schedule a meeting between a customer and multiple members on the PostHog team, click on "Event types" in the left hand navigation, then click "+ New Event Type" button in the top right, and select "Group" from the dropdown. This will allow you to create a group meeting and add multiple team members to the event and create a link you can share with the customer.

BuildBetter: We use BuildBetter for call recording and notetaking. You will need to integrate BuildBetter with your calendar in order for it to automatically join your calls. To do so, click on settings and look for the integrations link under account (not the one under organization) and follow the steps from there.

Zoom: We use Zoom for sales calls, and if you have Calendly properly integrated, calls that are booked through the tool will default to Zoom. You can find backgrounds to use for the calls here: This is fine (and other awesome PostHog wallpapers).

Lifecycle of CSM engagement

contents/handbook/cs-and-onboarding/lifecycle-csm.md

> This page covers more of the operational details of how our team generally works - for a broader overview of roles and responsibilities, visit the customer success team page.

Introduction

When starting out as a Technical Customer Success Manager (CSM) at PostHog, you are assigned a book of business with ~30 accounts to work with. It is helpful to think of customer engagement in stages to help us identify how we should connect with customers at each stage.

Stage 1: Getting started with customers

We've written a lengthy guide on how to get started with customers, so rather than rehash some of that information here, go read that guide instead.

Stage 2: Establishing trust

Once you've gone through your entire book of business and have completed stage 1, the next stage in our journey is to develop deep trust with our champions. Trust is built over time, based on our interactions and how we manage and nurture those relationships.

Some key examples that really help with building trusts with your customers:

- **Show, not tell**: Whenever a customer has a support question, or indicates they are trying to accomplish a goal, if possible, help create the solution rather than simply sending them to our docs. It is an opportunity to create an insight or dashboard for the customer, confirm if this is what they wanted, and share with them how you were able to create such an insight or dashboard so they could do it themselves later. Be careful to balance this and not create a dependency situation where customers rely on you to create stuff for them.
- **Be timely on notifications**: We have automated alerts set up to flag unusual behavior on an account. This includes things like event spikes or decreases. Sometimes this is unexpected behavior for the customer. Being quick to draw this to their attention can be helpful for the customer to investigate and showcase your helpfulness. If there are other signals, act on it accordingly. If there's public news, such as the company just raised a new funding round, reach out to your champion on this to congratulate them earnestly.
- **Go beyond the basics**: In getting started with customers, you focused on getting to know your customers, understanding their business goals, helping with implementation, and optimizing their use. Now is a great time to focus on how you can help them optimize for success. This could be looking at how they're currently engaging with PostHog products, whether there are things they

haven't done that might be helpful to implement. For example, have they set up custom tracking funnels for high value metrics, created alerts on customer actions to track, have they tried PostHog AI, or considered implementing other features to augment the data they have (such as error tracking to figure out why conversions drop off). This is a good opportunity to cross sell but also presents an opportunity to help customers understand where they can get more value out of PostHog.

Establishing trust can take time, and your communication style and actions can play a significant role. It may be worth offering recurring calls with your champion to establish more face-to-face contact, as this can help you maintain an ongoing pulse on what's happening.

Stage 3: Getting deeply embedded with customers

At this stage, we're interested in conducting a deep dive and becoming more deeply embedded with their team to work through some of their goals. This could help establish new workflows or setups to gain deeper insights beyond what they've achieved.

Here are a couple examples that have came up previously:

- **Building a custom recommendation engine**: An ecommerce customer we were working with had implemented PostHog to track key metrics they cared about but wanted to take it a step further and use us as source of truth in customizing visitor experiences to their site. Each returning visitor would see a custom feed just for them based on prior product views, searches, or purchases. This went way beyond the scope of simply tracking events and building dashboards, and there were potential opportunities to work with the customer here on how to custom track events, push data to their backend, and more.
- **Real time alerts**: A couple customers had high value actions they wanted to track and get real time notifications on. One customer wanted to track product views, present related purchases for upsell, and get notified when a customer didn't complete a purchase. Another wanted to track downloads and figure out when visitors attempt to access a file but runs into an error. This presented opportunies to understand high value signals for our customer and how we can help them implement a custom solution using PostHog to accomplish their goals.

The goal at this stage is to help our customers succeed by getting them the key metrics they care about, and often times, requires us to connect with their team to implement custom code changes at a deeper level to accomplish this.

If your champion is in a key decision making position who can get these changes through, that's great, but if not, this is also a great opportunity to ask your champion for an introduction to the key decision maker so you can work close with them to ensure changes can be prioritized. Another method is to reach out to the team lead, such as the head of engineering or head of product, armed with what their quarterly goals are, and offer your assistance directly. You may establish another strong connection this way.

Companies have conflicting priorities but by demonstrating you understand what their core goal is, and how PostHog could solve the problem, and finding the key decision maker, you have a higher chance of convincing the team to prioritize the changes now rather than wait to add value.

New hire onboarding exercise

`contents/handbook/cs-and-onboarding/new-hire-onboarding-exercise.md`

This exercise can help you learn more about your customer's usage of PostHog while helping you ramp up on your own PostHog skills!

Tactical questions

To get started you'll need all the organization IDs for your accounts. You can get those via SQL query: `SELECT DISTINCT posthog_org_id_c, NULL as empty_column FROM salesforce.account WHERE owner_id = 'your_salesforce_id'` You can find your salesforce ID by going to your profile and copy the text in the website URL after "/User/" then export the results via CSV. (The empty column is there so we have commas as delimiter for the org ids, this allows you to directly copy and paste all the org ids into a filter input text field.)

Cohorts

1. Who are all the users in your accounts?
2. Who are all the admins / owners in your accounts? (Hint: check the current_organization_membership_level property)
3. Who are the new users in your account this week?
4. Who are the power users in your account? (Power users can be across multiple products, or you can split it by product. Define a power user as you see fit!)

Activation

1. For the new users on your accounts, how many came back to analyze an insight, watch a recording, create a feature flag, etc. within their first week?
2. What are the monthly activation rates across all your accounts for product analytics? (Hint: read this activation metric post and these insights)

Retention / Usage

1. Which of your new users have retained their usage after their first 3 months?
2. Which of your organizations have viewed /docs/ pages more than once in the past week? How many /docs/ pages views have there been across accounts for the past week?

Churn

1. Have any of your accounts churned from a specific product within the last 3 months? How many/if any across all your organizations within the last 3 months?

Strategic questions

1. Are any of your users getting stuck setting up a product?
2. What alerts / CDP destinations can you set up to help you monitor drastic changes in your account metrics in PostHog?
3. What analysis would help understand why accounts take so long to convert from first login to consistent usage?

Example answers

If you get stuck or want to verify your implementation against an example, below are existing cohorts, insights, etc. to match each question.

Cohorts:

1. Example cohort
2. Example cohort
3. Example cohort
4. Example cohort

Activation:

1. Example insight

Retention / Usage:

1. Example insight
2. Example insight

Churn:

1. Example insight

Strategic questions:

1. Example funnel - Where would you dive deep next from here?

New starter onboarding

contents/handbook/cs-and-onboarding/new-hire-onboarding.md

Your first few weeks

Welcome to the PostHog Customer Success & Onboarding team! We only hire about 1 in 400 applicants, so you've done well to make it here! Unlike a lot of companies, we don't have a super-long onboarding process and would prefer you to be up and running with your customer base as quickly as possible. Here are the things you should focus on in your first few weeks at PostHog to help you achieve that.

Ramping up is mostly self serve - we won't sit you down in a room for training for 2 weeks. If you're not sure who is supposed to make something below happen, the person responsible is almost certainly you!

Also look at the sales team's onboarding page for guidance on what *not* to do when you start. In general, there's a lot of good resources within sales to reference (as we were previously one team!)

Day 1

- Familiarize yourself with how we work at PostHog.
- Meet with Dana who will run through this plan and answer any questions you may have. In addition, come equipped to talk about any nuances around how you prefer to work (e.g. schedules, family time etc.).
- Setup relevant Sales & CS Tools
    - Integrate Gmail with Salesforce and Vitally to enable centralized communication history
- If you start on a Monday, join your first PostHog All Hands (at 4.30pm UK/8.30am PT) and be prepared to have a strong opinion on whether pineapple belongs on pizza.
- If you start on a Monday, join your first CS standup.
    - We fill in a GitHub issue every week before this meeting so we are prepared for the discussion topics. Dana will add your GitHub handle to the template.

Rest of week 1

- Confirm that you have been added as a member to the PostHog organization in GitHub. Fraser can add you if you haven't.
- Work your way through your GitHub onboarding issue that a member of the created and sent a link to.
- Ask team members in your region to be invited to some customer calls so you can gain an understanding of how we work with customers.
- Check out some BuildBetter calls and add yourself to a bunch of Slack channels - get immersed in what our customers are saying.
    - There are a few BuildBetter playlists to start with – customer training calls, PostHog knowledge calls, onboarding specialist calls, add to them as you listen!
- Learn and practise a demo of PostHog.

    - For familiarization and self-led training, follow the curriculum. You can work through this with the HogFlix Demo App project which is already populated with data. Alternatively, you can create a new project in EU PostHog instances and hook it up to your own app or HogFlix instance.
- Read all of the CS section in the Handbook as well as the Sales section, and update it as you learn more.
- Meet with Charles, the exec responsible for Customer Success.

Week 2

- During your first week, Dana will figure out your initial book of business (around 30 accounts). We will review these at the start of your second week, and make sure you understand how your targets are set.
- Shadow more live calls and listen to more BuildBetter recordings.
- Explore Vitally and Metabase – take note of any questions you have to go through during in-person onboarding.
- Once you have your book of business, try running through the onboarding exercise that Kaya designed to test your skills for working with customer accounts.
- Towards the end of the week, schedule a demo and feedback session with Dana. We might need to do a couple of iterations over the next few weeks as you take on board feedback, don't worry if that's the case!
- Get comfortable with the PostHog Docs around our main products.
- Prioritize your current book of customers, and start reaching out! You should check conversations in Vitally to see if someone else has a prior relationship as they can make a warm intro for you.

In-person onboarding

This typically happens in Week 2 or 3 and runs 3-4 days with a few existing team members, covering:

- Demo practice session with the team.
- The data we track on customers in PostHog and some hands-on exercises to get you comfortable using PostHog itself.
- Deep dive on Vitally tracking.
- No stupid questions session.

Weeks 3-4

- Focus on taking more and more ownership on calls so that team members are just there as a safety net.
- Make sure all your tooling and automation are fully set up (health indicators etc.)
- Continue to meet with your book of customers.

How do I know if I'm on track?

By the end of month 1:

- Be starting to solve technical problems for your book with occasional help
- Be leading customer calls and demos on your own
- Successfully made contact with *everyone* in your book of business
- Update this page and other relevant handbook pages with what you learned during onboarding

By the end of month 2:

- Saved your first 'we're going to churn' - it's going to happen, but you're going to save them!
- Be leading evaluations on your own

By the end of month 3:

- Be independently working with your entire book to solve tricky technical problems with minimal assistant
- On track to consistently hit your retention targets
- You've suggested and made changes to our systems that enable you to do your job better
- Think about customer health scores and add/change anything you learn here

PostHog curriculum

PostHog has a lot of products! To help you figure out how to start and continue building your knowledge, here's a recommended list of topics to work through. Do not feel as though you need to learn all the products through your first few weeks. Learning is best done working through customer use cases and requests.

Add and modify this list as you work through it! Products are added frequently, likely making this list outdated.

Fundamental

Product analytics

1. Quick primer on Product analytics
2. Creating insights:
    - Trends, Funnels
    - User Paths
        - Wildcard groups
        - Path cleaning Rules
    - Retention, Stickiness, Lifecycle
    - How to filter out test users?
3. Persons
    - What are persons and how are they created?
    - Identify()
    - identified vs anonymous events
    - Pricing

4. Groups – what is it? what is the use case? how is it charged?
5. Session replay – masking, cutting costs, filtering
6. Toolbar – heatmaps, actions

Implementation

1. How is PostHog implemented?
2. Autocapture – how do you customize autocapture? How do you leverage autocapture?
3. What are custom events? How do you set custom properties?
4. What is identify? How do you set custom person properties? How do you merge users? What is alias?
5. What are groups? How do you set group properties?
6. What are cohorts? How do you create cohorts (static and dynamic)? How are they different from groups?
7. Projects, organizations and access controls
8. More advanced use cases:
    - Cross-domain tracking
    - reverse proxy
    - cookie consent (EU)

Billing

1. How to estimate costs
2. Pre-paid credits
3. Billing Limits

Intermediate

Feature flags

1. Creating and using them in code
    - How do I ensure flags are loaded before capturing any events?
    - Can you evaluate feature flags using properties that haven't been ingested yet?
2. Locally testing feature flags using toolbar
3. Insights based on feature flags:
    - Some users have access to a beta feature. How do I filter insights for these users?
4. Local evaluation
5. Client-side bootstrapping
6. Troubleshooting

Experiments

1. Creating an experiment from PostHog UI
2. Understanding MDE, primary metrics, secondary metrics, interpreting results

3. Traffic allocation - configuring it and validating it. What are some reasons why 80/20 split may not be an 80/20 split?
4. Returning users: user sees variant A in session 1, does not convert; user sees variant B in session 2, does convert
    - Does this happen? Can the same user see different variants in different sessions? If so, how does this affect the results?
5. No-code web experiments
    - Implementation requirements
    - Landing page experiments – how to deal with flickering of content when page is first loaded?

LLM Analytics

1. Implementing with your LLM SDK
    - Privacy options
2. Generations vs traces vs spans vs sessions
3. LLM Cost Analysis
    - How do we accommodate custom LLM pricing?
4. Insight analysis

Error Tracking

1. Implementing error tracking
2. Stack traces
    - Uploading source maps
    - Releases
3. Exceptions vs issues
    - What is a fingerprint?

Other Products and Features

1. Platform add-ons (Boost/Scale/Enterprise/Teams)
2. Data pipelines
    - Sources
    - Destinations
    - Transformations
3. Surveys
    - How surveys works with feature flags/experiments
4. Workflows
5. Logs

Advanced

1. SPA (single page apps)
2. API

Alerting setup (for team leads)

We have certain automations in Vitally that your team lead needs to add you to. Please ask your team lead to add you.

- Vitally name trait playbook: create a new branch that matches assigned CSM to new team member. In this branch, add action to update account trait `CSM name` to name of the new team member. This is used to populate account owner info in tickets created by customers we own, so support knows who to reach out to.

Template for onboarding success plan

`contents/handbook/cs-and-onboarding/onboarding-success-plan.md`

Introduction

Each customer is going to be a bit unique when it comes to onboarding and implementation; especially with such a broad product surface area! There are still some best practices we can follow to collaborate with the customer and plan for their success. It really helps customers to build engagement if we can collaborate with them on a plan for their first 30 days at PostHog.

Customize the below template

- Customize the template with the goals, specific products and commitments that make sense for your customer's use case.
- Share with customer, ideally as a Slack Canvas

Template

PostHog 30-Day success plan

Customer: [Customer Name] | **CSM:** CSM or AE NAME | **Start Date:** [Date]

Our shared goal

Week 1: You're getting actionable insights from PostHog
Week 3: You've identified specific opportunities to improve your key metrics
Week 4: You're confident PostHog is driving measurable business value

---

Week 1: Quick setup & first insights

Goal: See value within 7 days

Your commitments

- [ ] **SDK Validation:** Confirm SDK is properly tracking (usually done in trial)
- [ ] **Key Stakeholders:** Attend 30-min kickoff call
- [ ] **Custom Events:** Define business-specific events you want to track
- [ ] **Primary Use Case:** Define the #1 metric you want to improve

PostHog commitments

- [ ] **Implementation Guidance:** Validate SDK setup and custom event tracking
- [ ] **Feature Flags Setup:** Configure feature flags for your use cases
- [ ] **Group Analytics Setup:** Configure B2B company-level tracking
- [ ] **Custom Dashboard:** Build initial dashboard for your key metrics
- [ ] **Training Session:** 45-min walkthrough of your specific setup
- [ ] **First Insights:** Identify 2-3 actionable findings from your data

Weekly check-in: [Day/Time] - 30 minutes

- What findings have you learned from the first few insights you set up?
- Have you come across any points of friction in the setup or areas of the product you're struggling to understand?
- What are the next actions steps for us to follow up with in the next meeting?

Week 2-3: Feature Adoption & Optimization

Goal: Expand usage across your team

Together We'll:

- **Session Replay:** Set up recordings for your key user flows
- **Feature Flags:** Deploy your first A/B test or feature rollout
- **Surveys:** Launch feedback collection (if applicable)
- **Team Training:** Onboard 3-5 additional team members
- **Advanced Analytics:** Build funnels, cohorts, or retention analysis

Bi-weekly Check-in: [Day/Time] - 30 minutes

Week 4: Value Confirmation & Next Steps

Goal: Quantify business impact and plan expansion

Success Review:

- **Measurable Impact:** Document specific improvements/insights gained
- **ROI Assessment:** Calculate value PostHog has delivered
- **Team Adoption:** Confirm regular usage across stakeholders
- **Expansion Opportunities:** Identify additional products or use cases

Business Review: [Day/Time] - 45 minutes

Post-Launch: Optimization Check-in

Goal: Maximize ROI and identify expansion opportunities

6-8 Weeks After Launch:

- **Usage Analysis:** Review which features are driving the most value
- **Advanced Features:** Explore underutilized capabilities
- **Workflow Optimization:** Streamline your team's PostHog processes
- **Expansion Planning:** Identify additional use cases or team members
- **Success Story:** Document wins for internal stakeholders

Optimization Review: [Day/Time] - 60 minutes

Success Metrics

Milestone	Target Date	Success Criteria	Status
SDK Validated	Day 2	Data flowing correctly into PostHog	⏳
Custom Events Defined	Day 3	Business-specific tracking configured	⏳
Feature Flags & Groups Setup	Day 5	B2B tracking and flags operational	⏳
Team Trained	Day 7	3+ people actively using	⏳
Actionable Insights	Day 10	2+ specific findings identified	⏳
Feature Expansion	Day 20	2+ products actively used	⏳
Business Impact	Day 28	Measurable metric improvement	⏳

Key Contacts & Communication

PostHog Team

- **PostHog Human:** [your email]
- **Support Channel:** [Dedicated Slack channel/email]
- **Documentation:** posthog.com/docs

Your Team

- **Executive Sponsor:** [Name, Role] - Strategic oversight, success validation
- **Technical Lead:** [Name, Role] - Implementation, data setup
- **Primary Users:** [Names, Roles] - Daily usage, feedback

Meeting Schedule

- **Onboarding Phase:** Training sessions and weekly progress check-ins
- **Post-Onboarding:** Monthly or bi-weekly check-ins recurring

---

What You Can Expect From Us

✅ **Rapid Response:** Same-day replies to questions/issues from your PostHog Human ✅ **Proactive Guidance:** We'll suggest optimizations based on your usage
✅ **Custom Resources:** Tailored documentation and best practices
✅ **Issue Resolution:** PostHog human will resolve issues directly and escalate internally as needed
✅ **Product Feedback:** Feedback calls or user interviews with product managers or engineers

What We Need From You

✅ **Clear Objectives:** Tell us the specific metrics you want to improve
✅ **Custom Event Planning:** Help us understand your business-specific tracking needs
✅ **Stakeholder Engagement:** Keep key people involved and responsive
✅ **Honest Feedback:** Let us know what's working and what isn't
✅ **Success Definition:** Help us understand what ROI looks like for you

---

Questions or concerns? Reach out anytime - our success is measured by your success.

This plan is our shared roadmap. We'll adjust it based on your specific needs and progress along the way.

Renewals

contents/handbook/cs-and-onboarding/renewals.md

Renewal principals

Being on a prepaid credit plan - usually annual - is a win-win solution for both PostHog and the customer. Customers get discounts on the credits they purchase and PostHog gets confirmed revenue.

When estimating renewal amount, we want to make sure we accurately determine the amount of credits the customer will need in the next 12 months (or equivalent period, e.g. if they prepaid for 6 months). This is not an opportunity to upsell - do that later by encouraging product usage.

This page walks through recommendations for approaching and handling renewals. Contract rules and how to create contracts are covered in relevant pages under our shared processes.

When to start

Start renewal conversations at least 2 months before the contract renewal date for customers you are already in frequent contact with. For customers who are quiet, start renewal discussions 3 months out to allow more time for re-engagement.

Vitally and Slack will keep you on track with automated reminders. When a customer hits the 2-month mark, they'll automatically enter the `Upcoming renewal` segment, you'll get a task assigned to you in Vitally, and Slack will send a notification.

Start by sending a message in the shared Slack channel. Things will change in a year – the person you worked with previously may not be the right person this time. Mention when the customer is set to renew and ask if they have any preferred next steps.

As you make progress in the renewal discussions, update the renewal opportunity in Salesforce.

Customers who are projected to run out of credit before renewal

You will get notified by credit bot in Slack if a customer is set to run out of credits before their renewal date. This is considered an early renewal and follow the same process. If the customer will likely run out of credits before renewal is done, make sure they have a credit card on their account so any overage bills will be paid.

Renewal discussions

Renewal conversations are best done on a call. There can be a lot of moving parts so talking through it is usually a good idea.

Before the call, review your customer's usage and start a quote in Quotehog . If you need to look at data beyond the last 6 months, you can use this PostHog dashboard and edit the variables. Check if your customer is on any legacy pricing tiers – either talk to them about moving to standard pricing, or take it into account when building a quote.

This call can be an opportunity to explore your customer's PostHog experience so far and upcoming initiatives that you can build on in the future. It's also a good idea to explain how contracts, credits, and discounts work at PostHog – our pricing philosophy and contract rules are handy pages to bring up.

When you walk through the quote, start by looking at their past usage – try to anchor to the main products they're using as there can be a lot of numbers to look at. Explain how you estimated the usage for each product to arrive at the final number. Check-in with your customer throughout to sense check you're on the right track.

After the call - share the public quote link with your customer along with any usage information you shared on the call.

Docs ownership

contents/handbook/docs-and-wizard/docs-ownership.md

Product engineering teams are responsible for writing docs and ensuring they are up-to-date. This means:

- Documenting new features when they're launched
- Reviewing and merging docs PRs generated by Inkeep when your monorepo PRs are merged
- Add doc comments to SDKs to make them easier to understand
- Clarifying documentation where needed based on support tickets
- Ensuring public betas have docs that are linked to from the feature preview menu

Read writing docs as an engineer – it's really important!

The is responsible for improving the docs. This means:

- Building tools and systems to improve baseline quality and structure
- Shipping docs content based on prioritized feedback and emerging use cases
- Reviewing and improving draft documentation created by product teams
- Improving the subjective docs experience (navigation, discovery, interactivity, etc.)
- Creating context services that power agents like the AI wizard
- Working on large scale docs projects

Ownership within the Docs & Wizard team

We've previously assigned ownership to areas of the PostHog platform and product docs to individuals, but we're presently more project orientated.

You can view what we're working on right now by:

1. Reading our goals on the page
2. Dropping in on our `#team-docs-and-wizard` Slack channel

You can share ideas / requests for new docs in the `#team-docs-and-wizard` Slack channel, or by creating an issue on the posthog.com repo.

As ever, though, PRs > issues. ;)

Sources for inspiration

There are lots of places you can go to find inspiration for what to work on during your stint, such as:

- community questions
- open issues on our project board

- feedback in #brand-mentions
- #docs-feedback
- #content-docs-ideas
- #ask-max for questions missing content
- Zendesk tickets where root cause is documentation unclear
- Inkeep chat sessions where there is a documentation gap
- Most unhelpful docs
- Most popular docs
- that annoying thing you saw that you keep meaning to go fix

FAQ

I'm really busy, can the team write docs for me?

We can help, but we can't do it all for you. We lack the context necessary to document new features. First drafts of documentation must always come from the relevant product team.

If you need help updating documentation:

- Write a draft that covers the basics, which the content team can then help review and polish.
- If multiple docs pages need updating, create an example of changes needed and then request help to complete the rest.

Bottom line: It's much easier for the content team to improve a draft than write completely new documentation, especially when documenting new features. Pull requests > Issues.

Who should review docs updates?

Tag the `docs reviewers` team on GitHub and someone will come running.

How do I add images to my docs?

If you need to add images to your docs, please upload them to Cloudinary first and then embed them into the document.

You can embed light mode and dark mode versions of the image using this code snippet:

```
<ProductScreenshot
  imageLight =
        "https://res.cloudinary.com/dmukukwp6/image/upload/add_holdout_light_ce0
        827be42.png"
  imageDark =
        "https://res.cloudinary.com/dmukukwp6/image/upload/add_holdout_dark_cc68
        7f7688.png"
  classes="rounded"
```

```
  alt="Screenshot of the form to create a new holdout"
/>
```

AI platform

contents/handbook/engineering/ai/ai-platform.md

What is the AI platform?

The PostHog AI platform is our infrastructure for building and delivering AI-powered features across all PostHog products. Instead of each team building isolated AI capabilities, we provide shared architecture, reusable components, and a consistent framework that lets everyone contribute toward our AI capabilities while maintaining quality and consistency.

Think of it like HogQL: rather than having every team write their own query engines, we built one shared system that everyone can use and extend. The AI platform follows the same philosophy — avoid reinventing AI infrastructure and prevent "death by random AI widgets."

Why we built it

Almost every team at PostHog either is building or needs to build AI features. Without a platform approach, we'd face:

- **Fragmented user experience**: Different AI interactions across products with inconsistent quality and UX patterns
- **Duplicated effort**: Multiple teams solving the same problems (authentication, error handling, rate limiting, tool calling)
- **Maintenance burden**: Each team maintaining their own AI infrastructure, models, and prompt engineering
- **Limited capabilities**: Teams constrained to simple AI features because building advanced functionality (like multi-step reasoning or agentic workflows) from scratch is too expensive

The AI platform solves these problems by providing:

1. **Shared architecture**: A single-loop agent system that any product can extend with domain-specific tools and expertise
2. **Reusable components**: Common tools (search, data access, taxonomy reading) that work across all AI features
3. **Consistent UX**: Standard patterns for AI interactions, loading states, error handling, and result presentation
4. **Platform-level improvements**: When we improve the core agent (better reasoning, faster responses, cheaper inference), all products benefit automatically

Vision: Product autonomy

The overarching goal of PostHog's AI direction is **product autonomy** — a closed loop where PostHog data automatically drives product improvements with minimal human intervention.

Here's how the loop works:

1. **Signals**: PostHog collects signals from all products and external sources — error patterns, frustration in session recordings, experiment results, survey responses, insight thresholds, support tickets, Slack threads, and more. These signals represent real problems or opportunities.
2. **Enrichment**: PostHog processes and enriches these signals, deduplicating across data sources and adding context. A vague signal like "users seem frustrated during checkout" becomes a concrete, contextualized finding.
3. **Plans**: The enriched signals are transformed into structured plans — similar to how Claude Code works, but driven by data rather than human prompts. Each plan describes what needs to happen, why, and what evidence supports it.
4. **Execution**: A sandboxed coding agent takes these plans and acts on them. Today, we're focused on automatically creating pull requests. The agent also handles instrumentation automatically — adding tracking events, feature flags, and experiments as part of the code it ships. Better instrumentation produces better signals, making the entire loop smarter over time. In the future, other artifact types will be supported — decks, growth reviews, and more.
5. **Review**: Product engineers review, iterate on, and merge (or decline) the proposed changes.
6. **Feedback**: Once a change ships, a new signal is created so the system can evaluate what happened after the PR was merged. Did the metric improve? Did new errors appear? This feeds back into step 1.
7. **Loop**: The cycle continues until the agent finds an exit condition — low actionability, non-important signals, noisy signals, de-prioritized work, etc.

```
graph LR
    Signals[Signals<br/>Internal: errors, recordings, experiments<br/>External:
support tickets, Slack] --> Enrichment[Enrichment<br/>ML & agentic
pipelines<br/>deduplicate, contextualize, prioritize]
    Enrichment --> Plans[Plans<br/>Structured, data-driven<br/>action items]
    Plans --> Execution[Execution<br/>Coding agent creates<br/>PRs and
artifacts]
    Execution --> Review[Review<br/>Human oversight,<br/>iterate or merge]
    Review --> Feedback[Feedback<br/>New signal created<br/>from shipped
```

```
changes]
    Feedback --> Signals
```

This vision connects all the individual AI products. PostHog products and external sources (support tickets, Slack) generate signals, ML and agentic pipelines enrich them into structured plans, background and local coding agents execute on those plans, and product engineers review and collaborate on the changes. The loop closes when shipped changes generate new signals that feed back into the cycle.

For how product teams can contribute to this vision, see Integration vectors for product teams.

Architecture at a glance

The AI platform has three main layers:

1. User-facing products

These are the AI features users interact with directly:

- **PostHog AI**: In-app conversational agent for interacting with PostHog
- **Deep research**: Automated investigative research for complex, open-ended problems
- **Session summaries**: Batch analysis of session recordings to find patterns
- **PostHog Code**: Agent development environment that gives each task its own isolated workspace
- **Wizard**: CLI tool for automated PostHog installation and setup
- **MCP Server**: Protocol integration for third-party AI tools like Claude Code

2. Core infrastructure

The shared components that power all products:

- **Single-loop agent**: An agent architecture that maintains full context and can dynamically load domain expertise
- **Agent modes**: Pluggable modules that give the agent specialized knowledge and tools (SQL, Analytics, CDP, etc.)
- **Core tools**: Universal features like search, data reading, and task tracking
- **MCP integration**: Exposes agent features to external tools via Model Context Protocol

3. Integration points

How everything connects together:

- Products share the same agent features through the MCP server
- Task generation systems (from Deep Research, Session Summaries, PostHog signals) feed PostHog Code
- The Wizard and PostHog Code consume MCP tools to interact with PostHog

For a detailed technical overview, see AI platform architecture.

Products overview

PostHog AI [General availability]

Your primary interface for working with PostHog. Instead of clicking through forms and menus, describe what you want in natural language. PostHog AI can create dashboards, write SQL queries, set up surveys, and answer questions about your data — all through conversation.

Best for: Quick answers, creating resources, learning PostHog, iterative exploration **Status**: Beta | **Pricing**: Paid with free tier

Learn more →

Deep research [Beta]

When you need to investigate complex, open-ended problems, Deep research digs deep. It systematically explores your data — session recordings, analytics, error logs — and produces comprehensive research reports that would take a human analyst hours to create.

Best for: Understanding why metrics changed, investigating user behavior patterns, root cause analysis **Status**: Under development | **Pricing**: Paid with free tier

Learn more →

Session summaries [Alpha]

Analyze hundreds of session recordings in minutes instead of hours. Session summaries finds patterns, clusters similar issues, and shows you what's actually happening across your user sessions — not just what you caught in the first few recordings you watched.

Best for: Understanding UX issues, debugging problems affecting multiple users, finding edge cases **Status**: Alpha | **Pricing**: Paid with free tier

Learn more →

PostHog Code [Under development]

An agent development environment that solves the messy workflow problem of engineering with coding agents. Each task gets its own isolated workspace where an agent works — you can guide the agent, review changes, and switch between workspaces, with everything related to a task in one place instead of across your terminal, editor, and GitHub.

Best for: Product engineers who work on multiple tasks simultaneously and already use agents heavily **Status**: Under development | **Pricing**: TBD

Learn more →

Wizard [General availability]

Get PostHog set up in minutes instead of hours. The Wizard detects your tech stack, generates integration code, verifies the installation, and gets you collecting data with minimal manual work.

Best for: New PostHog users, setting up new projects, quick integration **Status**: General availability | **Pricing**: Free

Learn more →

MCP server [General availability]

Bring PostHog into your development environment. The MCP server makes PostHog AI's features available to Claude Code, VS Code, and other MCP-compatible tools, so you never have to leave your editor to check analytics or create insights.

Best for: Engineers who prefer editor-based workflows, combining PostHog with other data sources **Status**: General availability | **Pricing**: Free

Learn more →

Key concepts

For a list of key concepts definitions, see the Glossary.

Getting started

For users

- **Want to try PostHog AI?** Open the chat interface in PostHog and start asking questions. See user documentation.
- **Prefer working in your editor or coding agent?** Set up the MCP server in Claude Code or VS Code.
- **Need deep investigation?** Toggle to Deep research feature in PostHog AI.

For engineers building AI features

- **Not sure where to start?** See Integration vectors for product teams for the different ways your team can contribute — MCP tools, skills, signals, and more.
- **Adding AI to your product?** Start with Team structure and collaboration to understand the process.
- **Want to add a new agent mode?** See Architecture for technical details.
- **Need implementation guidance?** Check Implementation guide for best practices and patterns.

For product managers

- **Planning an AI feature?** Read Pricing and product positioning to understand our approach.
- **Want to understand capabilities?** See Products for detailed breakdowns of each product.

What's next?

The AI platform is actively evolving. Major initiatives include:

- **Third-party context integration**: Connect PostHog AI to Slack, Zendesk, and other tools for richer context
- **PostHog Code expansion**: Moving from alpha dogfooding to broader availability
- **Deep research refinement**: Improving research strategies and denoising algorithms
- **Mode expansion**: Adding more specialized agent modes as product teams identify needs

For details on upcoming work, see Future directions.

Documentation navigation

- **Products**: Detailed information about each user-facing product
- **Architecture**: Technical deep dive on agent systems and infrastructure
- **Team Structure**: How teams collaborate on AI features
- **Implementation Guide**: Best practices, pricing, and implementation patterns

Contact

For questions about working with the AI platform:

- **Slack**: #team-posthog-ai
- **Team page**:
- **Objectives**: Current goals and initiatives

AI platform architecture

contents/handbook/engineering/ai/architecture.md

This page provides a technical deep dive into the PostHog AI platform architecture. For a high-level overview, see the AI platform overview.

AI platform architecture overview

The following diagram shows how all components of the AI platform work together:

```
graph TB
    subgraph "User Facing Products"
        PostHogAIUI[PostHog AI<br/>In-app Agent]
        DeepResearch[Deep Research]
        SessionSum[Session Summaries]
        PostHogCode[PostHog Code<br/>Desktop App]
        Wizard[Wizard<br/>CLI Tool]
        ClaudeCode[Claude Code /<br/>Other AI Tools]
    end

    subgraph "Core AI Infrastructure"
        subgraph "Single-Loop Agent"
            Agent[Agent with<br/>Full Context]
            CoreTools[Core Tools<br/>search, read_data,<br/>read_taxonomy,
<br/>todo_write]
            Modes[Agent Modes<br/>SQL, Analytics, CDP,<br/>Custom Product Modes]
        end

        MCP[MCP Server<br/>CloudFlare<br/>Model Context Protocol]
        TaskGen[Task Generation<br/>Temporal Jobs]
    end

    Agent --> CoreTools
    Agent --> Modes

    Modes --> MCP

    MCP --> PostHogAIUI
    MCP --> DeepResearch
    MCP --> SessionSum
```

```
    MCP --> PostHogCode
    MCP --> Wizard
    MCP --> ClaudeCode

    SessionSum --> TaskGen
    DeepResearch --> TaskGen

    TaskGen --> PostHogCode
```

Key integration points

1. **The agent uses dynamic modes**: The single-loop agent architecture uses dynamically loadable modes that expose PostHog capabilities.
2. **MCP provides universal access**: The MCP server makes agent features accessible to any MCP-compatible client. PostHog AI, PostHog Code, Session Summaries, Wizard, and third-party tools like Claude Code all consume the same MCP server.
3. **Task generation feeds PostHog Code**: Signals from PostHog data, PostHog AI conversations, and Deep Research investigations are processed into structured tasks that PostHog Code can execute.
4. **Shared features**: Every surface consumes the same agent features through the MCP, ensuring consistency across the platform.

Single-loop agent architecture

Mode switching

PostHog AI is based on a single-loop agent architecture, heavily inspired by Claude Code, with some PostHog unique flavour. The core insight is simple: instead of routing between multiple specialized agents that act as black boxes, we have one agent that maintains full conversation context and can dynamically load expertise as needed.

The single-loop agent has direct access to all tools, uses a todo-list pattern to track progress across long-running tasks (just like Claude Code), and provides complete visibility into every step it takes. When it needs specialized knowledge, it doesn't delegate to a sub-agent — it switches its own mode to become an expert in that domain.

How the single-loop agent works

```
sequenceDiagram
    participant User
    participant Agent as Single-Loop Agent<br/>(Full Context)
```

```
    participant Tools

    User->>Agent: "Create a funnel for signup flow"

    Agent->>Tools: Call read_taxonomy<br/>(check what events exist)
    Tools-->>Agent: Returns actual events:<br/>'user_signed_up',
'account_created'

    Agent->>Tools: Call enable_mode("Analytics")<br/>(load funnel creation
tools)
    Tools-->>Agent: Analytics mode enabled<br/>with insight creation tools

    Agent->>Tools: Create funnel with correct events:<br/>'user_signed_up' →
'account_created'
    Tools-->>Agent: Funnel created successfully

    Agent-->>User: "I've created a funnel tracking your signup flow.<br/>It
shows 45% conversion from 'user_signed_up'<br/>to 'account_created'"
```

The key differences from older architectures:

- **No hallucination**: Agent checks `read_taxonomy` before assuming event names exist
- **Full visibility**: All tool calls are visible to the agent throughout the conversation
- **Maintained context**: The agent remembers every decision it made and can build on them
- **Explainable**: The agent can justify every choice because it has complete visibility

Core tools: Always available

No matter what mode the agent is in, it always has access to a core set of tools:

The **search** tool is unified search with a `kind` discriminator. You can search documentation (`kind=docs`), search existing insights (`kind=insights`), or search other resources as we add them. This replaced having separate `search_docs` and `search_insights` tools.

The **read_data** tool lets the agent read database schema and billing information. The **read_taxonomy** tool is how the agent explores your events, entities, actions, and properties. These are crucial for avoiding hallucination problems we had before — the agent can always check what data actually exists before making assumptions.

The **enable_mode** tool is how the agent switches between different areas of expertise, which we'll discuss in detail next.

Finally, **todo_write** is the tool that lets the agent manage long-running tasks. When you ask for something complex, the agent can write out a plan, track its progress, and make sure it doesn't lose context.

Agent modes: Dynamic expertise

Here's the key innovation: instead of having specialized sub-agents, we have a single agent that can "switch gear" by switching modes. Each mode gives the agent new tools, a new system prompt with domain expertise, and example workflows (which we call "trajectories") to follow.

It works in two stages. First, a small model router analyzes the user's request and enables some default modes. Then, during the conversation, the agent can call `enable_mode("SQL")` to switch into SQL expert mode, gaining SQL-specific tools and knowledge. The agent knows which tools it had before, which new ones it gained, and can switch back or switch to a completely different mode at any time.

Each mode is defined by three things:

A **routing prompt** that explains when to activate this mode and lists the available tools. This is what the small model router and the main agent use to decide when to switch modes.

A **system prompt** that contains expert instructions for this domain. When the agent switches to CDP mode, for example, it gets a system prompt explaining how CDP destinations work, what Hog functions are, and how transformations should be structured.

Workflow trajectories that give the agent examples of how to accomplish tasks. We inject example workflows into the `todo_write` tool description. For instance, the CDP mode might include a trajectory like: "Setting up CDP destination: 1. Write HogQL transformation code, 2. Define input variables, 3. Set event/property filters, 4. Test with sample data before activating."

This architecture allows product teams to create their own modes without touching the core agent. Modes can be composed and nested. Think of it as "thousands of agents" through mode combinations, rather than a fixed set of AI products.

When do black-box sub-agents still make sense?

There are exceptions. Some processes benefit from being hidden from the main agent — usually when the logic is completely detached from the conversation context, or when you want to use strategies or optimizations that would confuse the main agent if exposed. Our agentic RAG system for insight search is a good example: it iteratively searches through insights and cherry-picks the best ones using a complex scoring system. The main agent doesn't need to see all that — it just needs the final result.

Architecture diagram

```
graph TB
    User[User Message] --> Router[Small Model Router<br/>Analyze request]
```

```
    Router --> |Enable default modes| Agent[Single-Loop Agent<br/>Full
conversation context]

    subgraph "Core Tools (Always Available)"
        Search[search<br/>docs, insights, etc.]
        ReadData[read_data<br/>schema, billing]
        ReadTax[read_taxonomy<br/>events, properties]
        TodoWrite[todo_write<br/>task tracking]
        EnableMode[enable_mode<br/>switch expertise]
    end

    Agent --> CoreTools[Core Tools]
    CoreTools --> Search
    CoreTools --> ReadData
    CoreTools --> ReadTax
    CoreTools --> TodoWrite
    CoreTools --> EnableMode

    EnableMode --> |Dynamic loading| Modes[Agent Modes]

    subgraph Modes
        SQLMode[SQL Mode<br/>+ SQL tools<br/>+ SQL system prompt<br/>+ SQL
trajectories]
        AnalyticsMode[Analytics Mode<br/>+ insight tools<br/>+ analytics
prompt<br/>+ analytics trajectories]
        CDPMode[CDP Mode<br/>+ destination tools<br/>+ CDP prompt<br/>+ CDP
trajectories]
        CustomMode[Custom Product Mode<br/>+ product tools<br/>+ product
prompt<br/>+ product trajectories]
    end

    Modes --> Agent
    Agent --> |Execute with full context| Actions[Actions<br/>Create insights,
write SQL,<br/>set up destinations, etc.]

    Actions --> Response[Response to User]
```

How PostHog AI and MCP share the same features

The problem we needed to solve: PostHog AI and the MCP server were developed by different teams, didn't offer the same tools, and had completely different architectures. Users would find features in PostHog AI that didn't exist in the MCP, and vice versa.

The solution is an abstraction layer. Agent modes expose both high-level LLM tools (like "create a funnel with these parameters") and low-level API endpoint tools (like "call POST /api/projects/{id}/insights"). Both PostHog AI and the MCP have access to the same features, just through different interfaces.

How PostHog Code and Wizard fit in

Both PostHog Code and the Wizard currently consume the MCP. This integration gives them access to all the agent modes we're building. If Claude Code (which PostHog Code uses for code generation) ever becomes a bottleneck, we could swap in PostHog's own single-loop agent since they share the same mental model. We'd need to copy over Claude Code's terminal and file system tools (bash, grep, etc.) and add them as core tools.

We could also tag modes for specific interfaces. For example, a `CodingMode(tags=["posthog-code"])` would only be exposed to the PostHog Code agent, not to PostHog AI, because it's specific to code generation workflows.

Glossary

Agent: An autonomous AI process that can reason about what to do, plan multiple steps, and take actions by calling tools. PostHog is an agent. Claude is an agent.

Single-loop architecture: An agent architecture that maintains full context throughout a conversation without delegating to black-box sub-agents. The agent can see all tools, all previous messages, and all decisions it's made.

Feature: Any Agent capability we expose to the user. Creating insights, summarizing sessions, performing a Deep research, all of these are features.

Tool: A capability the agent can call to perform actions — search docs, create insights, write SQL queries, etc.

Agent mode: A specialized configuration of an agent that gives it domain-specific tools, expert knowledge (via system prompts), and workflow examples. When PostHog AI switches to "SQL mode," it becomes an expert in writing and debugging SQL queries.

Trajectory: An example workflow showing the sequence of steps to accomplish a specific task. We use trajectories instead of the heavier "jobs-to-be-done" framework to teach agents how to use tools together effectively.

MCP (Model Context Protocol): A standard protocol for connecting AI models to external tools and data sources in a structured, secure way. Think of it like an API, but specifically designed for AI agents.

MCP Server: The component that exposes tools and data sources following the MCP specification. PostHog's MCP server makes our analytics data available to any MCP-compatible client.

MCP Client: The component that connects to MCP servers to discover and use tools. Claude Code, VS Code with AI extensions, and other tools can act as MCP clients.

Implementing AI features

contents/handbook/engineering/ai/implementation.md

This page provides implementation guidance for building AI features at PostHog. For a high-level overview, see the AI platform overview.

How PostHog AI works across surfaces

PostHog AI isn't a single product – it's a platform that works wherever customers work. Through a combination of MCP tools and skills, PostHog AI is available across any agent of the customer's choice: PostHog AI in the web, PostHog Code, Claude Code, Cursor, Codex, and others.

All of these surfaces share the same underlying capabilities. The MCP server exposes PostHog's API as atomic tools, and skills teach agents how to compose those tools into workflows. When a product team adds a new MCP tool or writes a new skill, every surface benefits automatically.

PostHog AI in the web

PostHog AI in the web is a sandboxed coding agent built on the Agents SDK (Claude Code's harness). It runs in a controlled environment with access to PostHog's full API surface and unlocks use cases that go beyond what a simple chat interface can offer:

- **Better coverage of existing products** – the agent can navigate across product boundaries, combining data from analytics, session recordings, feature flags, and more in a single workflow.
- **Advanced SQL writing and analysis** – the agent writes HogQL queries, executes them, and reasons over large result sets to answer complex analytical questions.
- **Automatic instrumentation for non-technical users** – users who aren't engineers can describe what they want to track and the agent generates instrumentation code.
- **User-created custom skills and capabilities** – customers can create their own skills to teach the agent domain-specific workflows.
- **Generative UI for complex needs** – for the most complex UI requirements, the agent can generate custom visualizations and interfaces on the fly.

PostHog Code

PostHog Code is a desktop agent that turns PostHog signals into shipped code. It watches PostHog for problems (errors, frustration patterns, user feedback) and automatically creates tasks, generates fixes, and opens pull requests with human oversight at key decision points.

Third-party agents

Engineers who prefer to work in Claude Code, Cursor, Codex, or any other MCP-compatible tool get access to the same PostHog capabilities.

Headless first, then UI

Product teams must think about AI features as **headless (UI-less) workflows**. Agents don't need UI – they compose tools and follow skills to accomplish goals. But customers do need UI, and for that we have **MCP Apps**.

The rule of thumb: **first headless, then UI for a persona.**

1. **Build the capability headless** – expose your product's API as MCP tools and write skills that teach agents how to use them. This makes the capability available across all surfaces immediately.
2. **Then build UI where it matters** – if a persona (product manager, engineer, analyst) needs a dedicated experience, build an MCP App that provides the right UI for that workflow.

This order matters because headless capabilities are reusable across every surface, while UI is specific to one. If you build UI first, you've created something that only works in one place. If you build headless first, you've created something that works everywhere, and you can always add UI later.

MCP tools vs skills

Understanding the distinction between tools and skills is essential for building effective AI features.

MCP tools are atomic capabilities – CRUD operations and simple actions. They answer "what can I do?" (list feature flags, execute SQL, create a survey, summarize a session recording). Tools should be basic primitives that agents compose into higher-level workflows.

Skills answer "how do I accomplish X?" They combine tools, domain knowledge, query patterns, and step-by-step workflows into a template that agents follow to solve a class of problems. A skill might reference multiple tools, include HogQL query examples, explain what data to verify before querying, and describe the desired outcome for the customer.

This separation matters because agents are good at composing simple tools but need guidance on *which* tools to use, in *what order*, with *what constraints*.

For implementation details:

- Adding tools to the MCP server
- Writing skills

Implementation recommendations

For engineers adding AI features

1. **Expose your product's API as MCP tools.** Every product should be accessible through the MCP server. Scaffold a YAML definition, enable the operations that make sense, and add a HogQL system table for data access. See Adding tools to the MCP server.

2. **Write skills for jobs to be done.** If your product has jobs that require domain knowledge – specific tool ordering, constraints, query patterns, or reasoning about what data to check – write a skill that teaches agents how to accomplish that job well. See Writing skills.
3. **Build UI only when a specific persona needs it.** Don't start with a UI-specific AI feature. Start headless, validate that agents can accomplish the workflow, then add UI if a persona needs a dedicated experience.

Serializer best practices

Descriptions flow through the entire pipeline:

```
Django serializer field → OpenAPI spec → Zod schema → MCP tool description
```

Product teams should type and describe their serializer fields. These descriptions are what agents read to understand tool parameters – vague or missing descriptions lead to worse agent behavior.

Tips:

- Use `help_text` on serializer fields – it becomes the OpenAPI description.
- Use `param_overrides` in YAML definitions to override generated descriptions with imperative instructions.
- Be specific about formats, constraints, and valid values.
- Avoid jargon that an LLM wouldn't understand without context.

Pricing and product positioning

How we think about pricing

With our AI pricing, we want to follow the PostHog pricing principles. Concretely, this means:

1. We offer a generous free tier
2. We charge usage-based instead of a flat subscription

The unit that matches usage the closest is token consumption. This means to fix a SQL query with AI, the user would pay very little, analysing hundreds of session recordings will cost more. Since token costs differ based on token type & model, we are passing on our own costs to our users, with a small markup, instead of having a fixed price per token.

To keep our AI pricing simple, this pricing applies to all AI features once they are in general availability, that means per-product AI features as well as Session summaries and Deep research.

So that users can learn how to use PostHog without worrying about being charged, we are keeping chats that refer to our documentation free without a limit.

How users should think about our products

PostHog AI is the main PostHog product for AI interactions. You can use it in the web for the richest experience, through PostHog Code for code-generation workflows, or through any third-party agent via MCP. The web UX is best for sharing, navigation, and linking between AI results and PostHog artifacts. PostHog AI is also trained on PostHog-specific patterns and your actual usage data, so it provides higher quality, more contextual results than a general-purpose AI.

Deep research is a feature available within PostHog AI, but also accessible through its own dedicated UI if you want to jump straight into research. Use it for open-ended investigative work where you're trying to understand a complex problem.

Session summaries is callable from PostHog AI and Deep research, and also has its own UI. Use it when you need to analyze many session recordings and extract patterns or issues.

PostHog Code is a desktop product for single-engineer use. It's separate from PostHog AI because the workflow is different – you're not asking questions, you're letting an AI agent watch PostHog for problems and automatically fix them in your codebase. Think of it as an AI assistant that lives in your development environment.

MCP is for users who prefer to work in third-party tools like Claude Code, Cursor, or Codex. You get access to PostHog's data and can combine it with other MCP servers (like Hubspot or Zendesk). The trade-off is you don't get PostHog AI's polished UX or PostHog-specific optimizations.

How to develop and test

1. **Set up the MCP stack locally.** Run `hogli dev:setup` and add the MCP stack to your local environment.
2. **Write YAML configs and skills.** Use the monorepo Claude Code skills to scaffold tool definitions and write skills (TODO: dedicated skill for this).
3. **Build skills and dump them locally.** Run `hogli build:skills` to render all skills, then unzip them into `.agents/skills/` so Claude Code can pick them up during local testing: `unzip -o products/posthog_ai/dist/skills.zip -d .agents/skills/`.
4. **Test with headless agents, not UIs.** Forget about UIs – that's for humans. Test your tools and skills by talking to Claude Code or another headless agent. If the agent can accomplish the job, the capability works.
5. **Test with PostHog Code.** Sign in to a local environment in PostHog Code and verify the end-to-end workflow.
6. **Alternatively, add the local MCP server to Claude Code.** Run `claude mcp add --transport http posthog-local http://localhost:8787/mcp` to point Claude Code at your local MCP server.

Future directions

Third-party context integration

We want to connect PostHog AI to third-party tools for additional context. Imagine PostHog AI analyzing data across PostHog, Slack messages, and Zendesk tickets to understand not just what users are doing, but what they're saying and reporting. This data could also generate signals for PostHog Code – if users are complaining about a bug in Slack and PostHog sees errors in the same area, that's a strong signal to investigate and potentially fix automatically.

Continuous instrumentation

The Wizard's future evolution involves continuous instrumentation – watching your codebase and suggesting event tracking for new features, filling gaps in existing tracking, and standardizing event patterns. This could integrate with PostHog Code to automatically handle PostHog instrumentation when generating code.

Research improvements

Deep research is being refined with better research strategies, improved denoising algorithms, and more sophisticated pattern recognition. The goal is to reduce rabbit holes and improve data interpretation accuracy.

Contact and resources

For questions about working with PostHog AI, ask in the #team-posthog-ai Slack channel.

Additional resources:

- PostHog AI team page
- PostHog AI user documentation
- PostHog AI objectives
- AI platform overview
- Adding tools to the MCP server
- Writing skills
- Products documentation
- Architecture documentation
- Team structure documentation

AI products

contents/handbook/engineering/ai/products.md

This page provides detailed information about each user-facing product in the PostHog AI platform. For a high-level overview, see the AI platform overview.

PostHog AI [Beta]

PostHog AI is our primary in-app agent, accessible through a chat interface embedded directly into the product. Think of PostHog AI as a fundamentally different way to interact with PostHog — instead of clicking buttons and filling out forms, you ask questions and make requests in natural language.

The problem we're solving

PostHog has grown incredibly powerful, but that power comes with complexity. New users face a learning curve: Which insight type should I use? How do I filter for the data I need? What's the right SQL syntax for this query? Even experienced users spend time navigating through menus and forms to accomplish what they already know they want to do.

PostHog AI eliminates this friction. You don't need to know where a feature lives or how to configure it — you just describe what you want, and PostHog AI handles the details.

Who uses PostHog AI

Everyone. PostHog AI is designed to be useful whether you're:

- A new user learning PostHog for the first time (PostHog AI explains terminology and walks you through setup)
- An engineer who knows exactly what they want and just wants to say it instead of clicking through the UI
- A product manager who wants quick answers without learning technical details
- A data analyst who needs to write complex SQL queries with help

How it works

PostHog AI is built on a single-loop agent architecture with dynamic mode switching. When you send a message, PostHog AI analyzes your request, determines which specialized "modes" it needs to activate, and dynamically loads the appropriate tools and expertise. For example, if you ask PostHog AI to "create a funnel tracking the signup flow," it might:

1. Use the `read_taxonomy` tool to check which events actually exist
2. Switch to Analytics mode to access insight creation tools
3. Switch to SQL mode if you need custom transformations
4. Switch to CDP mode if you want to set up a destination based on the funnel results

Throughout this entire process, PostHog AI maintains full context — it can see all previous messages, all decisions it's made, and all tools it's used. This is fundamentally different from older architectures we implemented where specialized sub-agents worked in isolation.

For a technical deep dive on how this works, see the Architecture page.

Key capabilities

PostHog AI can do most things you can do through the PostHog UI:

- **Search and filter**: Find insights, filter session recordings, search documentation
- **Create and modify**: Build dashboards, create insights, set up surveys
- **Write SQL**: Generate and debug HogQL queries for custom analysis
- **Learn PostHog**: Ask how features work, get recommendations on best practices, understand terminology
- **Work with data**: Read your taxonomy (events, properties, actions), check database schema, access billing information

PostHog AI is powered by Inkeep for documentation search, which means it can pull from PostHog's entire doc library to answer questions about how to use the platform.

Pricing

PostHog AI is paid platform, with a generous free tier (see Pricing).

Current status & ownership

PostHog AI is currently in beta as we migrate to the new single-loop architecture. Early results show significant improvements in reliability and capability, but we're still ironing out edge cases before moving to general availability.

The owns the architecture, performance, and UX/UI. Product teams are responsible for adding their product-specific tools and capabilities, with the PostHog AI team providing reviews and guidance (see Team Structure for details on collaboration).

Deep research [Under development]

Deep Research is PostHog AI's bigger sibling — where PostHog AI gives you quick answers, Deep Research digs deep to understand complex, open-ended problems.

The problem we're solving

Product analytics often requires real investigative work. You don't just want to know "what's my conversion rate?" — you want to understand *why* it's dropping, *which* user segments are affected, *where* in the flow they're getting stuck, and *what* patterns exist across multiple data sources. This kind of

research is time-consuming. You might spend hours jumping between dashboards, filtering recordings, cross-referencing error logs, and synthesizing findings.

Deep Research automates this investigative work. It can spend minutes or hours (depending on complexity) systematically exploring your data, following leads, and producing a comprehensive research report that would take a human analyst half a day or more.

Who uses Deep research

Deep research is designed for anyone who needs to understand complex problems:

- Founders trying to understand why growth is stalling
- Engineers debugging issues that span multiple systems
- Product managers investigating why a feature isn't performing
- Data analysts exploring patterns across customer segments

If you have a vague question that requires digging through multiple data sources to answer, Deep Research is the right tool.

How it works: Test-time diffusion

Deep research's architecture is based on Google's test-time diffusion researcher framework. Here's the high-level flow:

1. **Input**: You either start with a templated research notebook (for common research patterns) or describe your question and Deep Research generates a custom notebook structure.
2. **Parallel initialization**: Deep research simultaneously creates a draft report (outlining what it expects to find) and a research plan (what questions to investigate).
3. **Iterative research**: The agent systematically investigates each part of the research plan. It might filter session recordings, run analytics queries, check error logs, compare cohorts, and more. Each investigation adds findings to the draft report.
4. **Denoising**: As research progresses, Deep research "denoises" the draft report — removing speculative parts that turned out to be wrong, strengthening findings that are supported by data, and identifying new questions to investigate.
5. **Loop**: Research continues until the draft report is fully denoised — meaning all sections are supported by actual findings rather than speculation.
6. **Final report**: Once complete, you get a structured notebook with the findings, including embedded session recordings, charts, and data that support each conclusion.

Architecture diagram

```
graph TB

    Input[User Question] --> DraftReport[Draft report<br/>with speculative
findings]
    Input[User Question] --> ResearchPlan[Research Plan<br/>with questions to
investigate]

    ResearchPlan --> Investigate[Iterative Investigation]

    subgraph Investigation
        Investigate --> Sessions[Session Summaries]
        Investigate --> Analytics[Run Analytics<br/>Queries]
        Investigate --> Errors[Check Error<br/>Logs]
        Investigate --> Cohorts[Compare<br/>Cohorts]
    end

    Sessions --> Findings[Add Findings to<br/>Draft Report]
    Analytics --> Findings
    Errors --> Findings
    Cohorts --> Findings

    Findings --> Denoise[Denoising Process]

    subgraph Denoising
        Denoise --> Remove[Remove speculative<br/>parts that are wrong]
        Denoise --> Strengthen[Strengthen findings<br/>with data support]
        Denoise --> NewQ[Identify new<br/>questions]
    end

    NewQ --> |Add to plan| ResearchPlan

    Remove --> Check{Report fully<br/>denoised?}
    Strengthen --> Check

    Check --> |No| Investigate
    Check --> |Yes| Final[Final Notebook Report<br/>with embedded recordings,
<br/>charts, and data]

    DraftReport -.->|Continuously updated| Denoise
```

Why notebooks?

Notebooks are the perfect format for research because they combine narrative explanation with data visualization. You can see not just the conclusions ("conversion drops 40% at the payment step") but the evidence (charts showing the drop, session recordings showing users struggling, error logs showing timeouts).

We're building customizable notebook templates similar to what Granola does. You'll be able to pick a template or modify one ahead of time, so research results come back in exactly the format you need. This is especially useful for recurring research tasks where you want consistency.

Key differences from PostHog AI

While both PostHog AI and Deep research can answer questions about your data, they're optimized for different use cases:

- **PostHog AI** is fast (seconds to minutes), conversational, and best for specific questions with clear answers
- **Deep research** is thorough (minutes to hours), systematic, and best for open-ended problems that require synthesizing multiple data sources

Think of PostHog AI as your coworker who can quickly pull up data, and Deep Research as the analyst who will spend the afternoon really digging into a problem.

Access and pricing

Access Deep Research by toggling "Research" mode in PostHog AI, or via the dedicated Deep Research UI. It's a paid feature with a generous free tier (see Pricing).

Current status & ownership

Deep Research is under active development. The PostHog AI team owns Deep Research. The architecture is implemented but we're still refining the research strategies and denoising algorithms. Early results show it can find patterns and insights that human analysts miss, but it occasionally goes down rabbit holes or misinterprets data — we're working on improving these edge cases.

Session summaries [Alpha]

Session summaries solves a specific but painful problem: you have dozens or hundreds of session recordings, and you don't have time to watch them all. Instead of spending hours scanning through recordings one by one, Session Summaries analyzes them all at once and gives you a structured report of what it found.

The problem we're solving

Session recordings are incredibly valuable — they show you exactly what users are experiencing. But they're also time-consuming to review. If you have 100 recordings from users reporting checkout issues, do you really want to watch all 100? Most people watch a few, spot some patterns, and hope they caught the important stuff. This means you miss edge cases, low-frequency issues, and patterns that only emerge across many sessions.

Session summaries changes this calculus. You can analyze hundreds of recordings in minutes, with confidence that you're seeing all the significant patterns, not just the ones that happened to appear in the first few recordings you watched.

Who uses session summaries

Session summaries is designed for anyone who needs to understand patterns across multiple user sessions:

- Engineers debugging problems that only some users experience
- Product managers investigating UX issues
- Customer success teams diagnosing why users are struggling
- Researchers trying to understand how different cohorts use a feature

If you find yourself thinking "I need to watch a bunch of recordings to understand this," Session Summaries is the right tool.

How it works

You can trigger Session summaries in three ways:

1. Ask PostHog AI directly: "Summarize the last 50 sessions from company X"
2. Trigger Session summaries from the Session Replay UI or from other products
3. Let Deep research invoke it as part of a larger investigation

Here's what happens under the hood:

1. **Collection**: Session summaries retrieves all the recordings matching your criteria (time range, company, feature area, etc.)

2. **Analysis**: An AI agent "watches" a session recording (right now, analyzing the stream of metadata, and soon enough, by watching video clips), noting significant events: errors, timeouts, rage clicks, confusion indicators (rapid back-and-forth navigation), unexpected user paths, and other behavioral signals.

3. **Clustering**: Instead of giving you 50 individual summaries, Session summaries clusters similar issues together. For example, if 15 users all experience timeout errors at checkout, these get grouped into a single issue: "Timeout errors during payment processing (affects 15/50 users)."

4. **Report generation**: You get a notebook with:
    - Issue clusters ranked by frequency and severity
    - Representative video clips showing each issue
    - Context about which users/cohorts are affected
    - Patterns that might not be obvious from individual sessions

What Session summaries finds

Currently, Session Summaries is trained to identify:

- **Errors**: JavaScript errors, failed API calls, broken images
- **Timeouts**: Long loading states, hanging requests
- **Frustration signals**: Rage clicks, rapid refreshes, abandonment
- **UX issues**: Confusing flows, unexpected navigation patterns
- **Performance problems**: Slow page loads, laggy interactions

Future capabilities

We're expanding Session summaries beyond just finding problems. Future capabilities include:

- **Creative usage patterns**: "Show me where users are using the product in ways we didn't expect"
- **Workarounds**: "Find sessions where users had to work around a limitation"
- **Feature discovery**: "Which features do power users rely on that casual users don't know about?"
- **Delight moments**: "Find sessions where users had a particularly smooth experience"

The underlying technology is the same — watch many recordings, find patterns, cluster similar behaviors — but the training and prompts can be tuned for different objectives.

Access and pricing

Access Session Summaries through PostHog AI, Deep Research, or its dedicated UI entry points. It's a paid feature with a generous free tier (see Pricing).

Current status & ownership

Session summaries is in alpha. The PostHog AI team owns Session summaries. It's working well for error and frustration detection, and early users report finding issues they would have missed. We're refining the clustering algorithms (sometimes it groups issues too broadly or too narrowly) and integrating video and GIF analysis to support findings with visual confirmation.

PostHog Code [Under development]

PostHog Code is our most ambitious bet: an agent development environment that turns PostHog data into shipped code. The vision is to free product engineers from distractions so they can focus on what they love — building great features — by automating all the chores that eat up their day.

The problem we're solving

Today, product engineers spend most of their day managing random inputs: Slack messages, GitHub notifications, tickets, emails, and alerts from various monitoring tools. This work is essential but time-consuming. Experienced AI-native engineers have already evolved a workaround — they practice "structured development," creating PRDs, breaking work into tasks, and shipping incrementally. Tools like Claude Code or Cursor only work well when given clean context and well-defined tasks.

PostHog Code aims to productize that discipline, turning chaos into structured, buildable work.

Who we're building for

PostHog Code is designed for experienced product engineers who already use AI coding tools regularly. We're explicitly not targeting non-technical "vibe coders" or hobbyist users. Our initial customer profile is early-stage startups with 2-10 engineers and hundreds to low thousands of users. We'll expand to larger startups later as internal workflows and scale requirements become more complex.

How it works: From Signals to shipped code

The core insight is that PostHog collects massive amounts of data across all our products — analytics, session recordings, error tracking, surveys, experiments. All of this data can be transformed into actionable "tasks" that describe real problems to fix or opportunities to pursue.

Here's the flow:

1. **Signal generation**: Something happens in PostHog that indicates work needs to be done. This could be a recurring error pattern, frustration signals from session recordings, a survey response indicating a missing feature, or experiment results suggesting an optimization. The Signals team focuses on surfacing this data in useful ways.

2. **Task creation**: An LLM-based system receives these signals, deduplicates them across data types, and translates them into concrete tasks with appropriate context. This uses a non-deterministic approach — we use a document store and LLMs to judge how to structure tasks. A vague signal like "users seem frustrated during checkout" becomes a specific task: "Investigate and fix timeout issues in payment processing, affecting 15% of transactions from company X."

3. **Task execution**: Once a task is defined, it gets assigned to a workflow. Different tasks need different approaches — a well-defined bug fix might be a one-shot fix with human QA, while a vague feature request might need definition, breaking into chunks, gradual shipping behind a flag, and automated feedback collection.

4. **Coding**: PostHog Code uses an agent running in a cloud sandbox (though we support local execution too). The agent clones your repo, reads your codebase for context, makes changes, writes tests, and opens a pull request. Changes are automatically wrapped in feature flags when appropriate.

5. **Human oversight**: You're always in control. The desktop app shows you what PostHog Code is working on, lets you review and edit tasks, and requires your approval before shipping. This "human-in-the-loop" approach means you can trust PostHog Code to work in the background while you sleep, but nothing ships without your sign-off.

Why a desktop app?

This is a crucial design decision. We could have built PostHog Code directly into the PostHog web app, and it would work. But it wouldn't generate the adoption we need.

Desktop apps win because of bottom-up adoption. Individual engineers can choose tools that make them more productive in a permissionless, frictionless way. A desktop app feels like a personal tool — like VS Code, Cursor, or your terminal — rather than a team product that requires management buy-in. Engineers already make personal choices about vim vs VSCode, which terminal to use, which AI coding assistant to try. PostHog Code slots into that category.

The UX also matters more for tools you use all day, not just a few times a week. PostHog Code is designed to feel like something between Warp, Ghostty, and Cursor: super fast, keyboard-first with lots of shortcuts, easy to navigate with tabs and split windows. Think of it as having the directness of a CLI but with the richness of a UI when you need it.

The interface

PostHog Code is tab-based with the home tab being a task list. You navigate with arrow keys, click a task to open it in a new tab with a two-pane view: task details on the left (title, description, tags, origin, PR link) and a live log of activities on the right. When a task is in progress, it streams output to this log so you can watch the agent work. There's also a workflow builder view where you can see tasks moving through stages kanban-style.

Technical architecture

PostHog Code is built as an Electron app for speed, familiarity (React), and cross-platform ease. When a task kicks off, we have two execution options:

Cloud agent (preferred): Tasks execute in a cloud sandbox owned by the PostHog AI team. The agent runs in an isolated environment, clones the repo, does its work, and pushes to a branch. The downside is you need to grant GitHub app access. The upside is truly magical — PostHog Code can work on tasks while you sleep, and you wake up to PRs ready for review.

Local agent (more permissionless): We spin up Claude Code-like execution in the background on your local filesystem. This is the most permissionless version, closest to how developers use Claude Code today. We still give it access to the MCP and PostHog tools, and we likely need to proxy through our infrastructure to maintain control and provide a smooth experience.

We support both modes, but push for cloud execution as the optimal experience.

Architecture diagram

```
graph TB
    subgraph "PostHog Code Desktop App (Electron)"
        UI[Task List UI]
        Backend[Backend Service]
    end

    subgraph "Task Generation"
        Signals[PostHog Signals<br/>Errors, Frustration, etc.]
        DR[Deep Research]
        SS[SessionSummaries]
        TaskGen[Temporal Job<br/>Task Generation]
    end

    subgraph "Execution: Cloud Agent (Preferred)"
        CloudSDK[SDK Wrapping<br/>Coding Agent]
        Sandbox[Sandbox + API<br/>Micro VM]
    end

    subgraph "Execution: Local Agent"
        LocalRepo[Local Repo<br/>User Filesystem]
        LocalExec[Local Execution<br/>Claude Code-like]
    end

    Signals --> TaskGen
    DR --> TaskGen
    SS --> TaskGen

    TaskGen --> Backend
    Backend --> UI

    UI --> Backend
    Backend --> CloudSDK
    Backend --> LocalExec

    CloudSDK --> Sandbox
    LocalExec --> LocalRepo
```

What kinds of tasks?

PostHog Code isn't just for data-driven bug fixes. The system for shipping a fix is the same as the system for shipping any feature. A vague task needs definition, then breaking into chunks, then shipping with proper releases planned. A small, well-defined task just needs a one-shot fix and QA.

Even inspiration-driven features (not from user data) benefit from PostHog Code's workflow: add event tracking, ship behind a flag, automatically message users for feedback, set up an experiment to measure impact. PostHog Code productizes best practices for shipping features, not just fixing bugs.

Current status

Right now we're focused on dogfooding — getting the to build everything using PostHog Code itself. This lets us refine product quality and identify friction fast.

For engineers not using PostHog Code

When PostHog Code isn't the right fit (maybe you don't trust AI to ship code automatically, or your workflow is very particular), we offer "copy prompt" features throughout PostHog. In error tracking, for example, you can generate an AI prompt to fix an error and paste it into your own code editor. This bridges the gap for engineers who want AI assistance but prefer to maintain manual control.

Ownership

The dedicated owns the product. The owns the background sandboxed agents. See Team Structure for collaboration details.

Wizard: AI-powered onboarding [General availability]

The Wizard is PostHog's AI-powered installation assistant that gets you from zero to collecting data in minutes instead of hours. Instead of reading documentation, finding the right SDK, figuring out configuration, and manually integrating PostHog into your codebase, you run one command and the Wizard handles everything.

The problem we're solving

Setting up analytics is tedious. You need to pick the right SDK for your tech stack, install dependencies, configure authentication, add initialization code in the right place, set up your first events, and verify everything works. For a developer who just wants to start tracking user behavior, this feels like unnecessary friction before you even get value from the product.

Even experienced developers waste 15-30 minutes on setup. For new developers or teams trying PostHog for the first time, it can take much longer — and if anything goes wrong, they might give up entirely.

The Wizard eliminates this friction. You run a single command, answer a few questions, and the Wizard writes all the integration code for you.

Who uses Wizard

The Wizard is designed for:

- New PostHog users getting started for the first time
- Teams trying PostHog on a new project or codebase
- Developers who want to add PostHog to an existing application quickly
- Anyone who prefers automated setup over manual integration

Basically, anyone who would rather spend time using PostHog than setting it up.

How it works

The Wizard is a CLI tool that runs locally in your development environment. Here's the flow:

1. **Detection**: The Wizard scans your codebase to detect your tech stack (React, Next.js, Python, etc.), framework version, and project structure.
2. **Configuration**: It asks you a few questions — which PostHog project to connect to, whether you want autocapture enabled, any custom configuration. The questions are contextual based on what it detected.
3. **Code generation**: The Wizard writes the integration code. This includes:
    - Installing the appropriate PostHog SDK via your package manager
    - Adding initialization code in the right location for your framework
    - Setting up configuration with your project token
    - Optionally adding example event tracking code
4. **Verification**: The Wizard verifies the integration works by sending a test event to PostHog and confirming it arrives.
5. **Next steps**: It suggests what to do next — track your first custom event, set up a dashboard, or explore session recordings.

The entire experience uses Clack.cc for a polished CLI interface with clear prompts, progress indicators, and helpful error messages.

Current capabilities

Right now, the Wizard handles installation and basic setup across PostHog's supported SDKs. It's particularly good at:

- Detecting complex framework setups (like Next.js with app router vs pages router)
- Handling different package managers (npm, yarn, pnpm)
- Placing initialization code in the right location based on framework conventions
- Configuring autocapture and basic options

Future direction

The Wizard's long-term vision is much broader than one-time setup. Imagine:

- **Continuous instrumentation**: The Wizard could watch your codebase and suggest event tracking for new features. "I noticed you added a new checkout flow — want me to add tracking events?"
- **Instrumentation improvements**: "Your signup flow isn't tracking all the steps — I can add events to fill the gaps."
- **Best practices**: "You're tracking events in 5 different ways. I can standardize this for you."
- **Integration with PostHog Code**: The Wizard is already integrated into PostHog Code, handling PostHog instrumentation automatically when generating code (feature flags, experiments, custom events).

This would turn the Wizard from a one-time setup tool into an ongoing assistant that keeps your PostHog instrumentation clean and comprehensive.

Current status & ownership

The Wizard is in general availability and actively used during customer onboarding. It's currently owned by the .

MCP: PostHog for third-party tools [General availability]

The MCP (Model Context Protocol) server is PostHog's way of meeting engineers where they already are. Not everyone wants to switch to the PostHog UI to analyze data — many prefer to stay in their code editor, terminal, or favorite AI tool. The MCP server makes that possible.

The problem we're solving

Context switching is expensive. If you're deep in debugging code in VS Code and need to check PostHog analytics, opening a browser, navigating to PostHog, finding the right insight, and coming back to your editor breaks your flow. It's even worse when you're using an AI coding assistant — you want to ask "which error is affecting the most users?" or "create a funnel for the checkout flow" without leaving your development environment.

The MCP server solves this by bringing PostHog directly into the tools engineers already use. No context switching, no mental overhead.

Who uses MCP

MCP is designed for engineers who prefer working in their development environment:

- Developers using Claude Code or VS Code with AI extensions
- Engineers who want PostHog data combined with other data sources (GitHub, Zendesk, Hubspot)
- Teams with custom workflows or tooling that can consume MCP servers

- Anyone who prefers command-line or editor-based workflows over web UIs

How it works

The Model Context Protocol (MCP) is a standard for connecting AI assistants to external services. Here's what happens when you use PostHog via MCP:

1. **Connection**: Your MCP client (like Claude Code) connects to `https://mcp.posthog.com/mcp` with your PostHog API key for authentication.
2. **Tool discovery**: The client asks the MCP server what tools are available. The server returns a list of about 30 tools covering PostHog's API surface — everything from creating insights to filtering session recordings to managing feature flags.
3. **Dynamic filtering**: You can control which tools load using query parameters: `https://mcp.posthog.com/mcp?features=flags,insights,workspace`. This keeps context windows small by only loading relevant tools.
4. **Execution**: When you ask the AI assistant to do something with PostHog, it calls the appropriate MCP tools. These tools interface with PostHog's APIs (and eventually dedicated `/ai` endpoints, under development) to accomplish the task.
5. **Mode switching**: The MCP server is being aligned with our mode switching framework. This means AI agents can dynamically enable and disable different modes during a conversation, loading only the expertise they need when they need it. This solves the context window problem — currently, loading all tools takes up about 14% of Claude Code's context window, which we're reducing through dynamic tool discovery.

Key architectural decisions

The MCP server is deployed independently on CloudFlare. This gives us fast iteration, proven reliability, and excellent developer UX with quick deployments. We dogfood PostHog's customer-facing API wherever possible, which gives us good incentive to take care of it.

The MCP server also supports session state (active project ID, org ID, distinct ID), so it can fingerprint sessions and maintain context across multiple requests.

PostHog AI vs. MCP: When to use each

Both PostHog AI and MCP give you access to the same features, but they serve different workflows:

Use PostHog AI when:

- You want the best possible UX with sharing, navigation, and linking between AI results and PostHog artifacts
- You're doing exploratory analysis and want to iterate quickly
- You need Deep Research or Session Summaries features

- You want AI specifically trained on PostHog patterns and your actual usage data

Use MCP when:

- You prefer to stay in your code editor or terminal
- You're combining PostHog data with other MCP servers (GitHub, Zendesk, etc.)
- You have custom tooling that can consume MCP servers
- Your workflow is already centered around a third-party AI tool

Our goal is to make PostHog AI so good that users want to "own" their workflow in PostHog, while still supporting MCP for engineers who prefer different tools or need to combine multiple data sources.

Current Status & Ownership

MCP is in general availability. The PostHog AI team owns the MCP server, with Josh Snyder as the primary support contact. We're actively working on dynamic tool discovery to reduce context window usage and aligning the server with our mode switching framework to share features with PostHog AI.

AI platform team structure and collaboration

contents/handbook/engineering/ai/team-structure.md

This page explains how teams collaborate on AI features at PostHog. For a high-level overview, see the AI platform overview.

Who does what

The PostHog AI team

is responsible for the architecture, performance, and UX/UI of the AI platform. We build and maintain the core infrastructure – the MCP server, skills system, PostHog AI in the web, background sandboxed agents, and shared tooling (`search`, `read_data`, `read_taxonomy`, `enable_mode`). We're also proactive when we see big opportunities for PostHog or when new capabilities can be used across multiple products, like SQL generation or universal filtering.

The PostHog Code team

builds PostHog Code, an agent development environment for product engineers. Working with coding agents today is bottlenecked by messy workflows — switching between agents, branches, worktrees, and manually managing PRs across multiple applications. PostHog Code solves this by giving each task its own isolated workspace where an agent works, with everything related to a task in one place instead of scattered across your terminal, editor, and GitHub.

The PostHog Code team owns the desktop app and the task execution pipeline.

The Signals team

turns PostHog data into tasks that coding agents can work on — suggested improvements from session replays, fixes for errors from error tracking, new experiments based on product analytics data. Signals surfaces something useful, creates a task with context, and the cloud agent works on it.

Product teams

Product teams own their product's AI capabilities end-to-end. The AI platform is designed so that any team can ship MCP tools and skills independently, without needing the PostHog AI team to be involved. This means you can:

- Add MCP tools that expose your product's API to agents
- Write skills that teach agents how to accomplish jobs in your domain
- Build UI for specific personas using MCP Apps when needed

Once you ship a tool or skill, it's automatically available across every surface – PostHog AI in the web, PostHog Code, Claude Code, Cursor, and any other MCP-compatible agent.

How the teams connect

Together, these teams form the product autonomy loop:

- **Signals** surfaces useful data from PostHog, creates a task with context, and the cloud agent works on it. You review and iterate in PostHog Code.
- **PostHog AI** owns the background sandboxed agents and can start coding agent tasks during chats. These tasks are inspectable in both the web product and PostHog Code.
- **PostHog Code** is where engineers review, guide, and manage agent work across all their tasks in one place.
- **Product teams** ship their own MCP tools and skills independently. Once shipped, these are automatically available across every surface.

Integration vectors for product teams

There are multiple ways product teams can contribute to PostHog's product autonomy vision. These are listed roughly in order of effort, from easiest to most ambitious.

MCP: Expose your APIs to agents

The most obvious and lowest-effort vector. Expose your product's APIs through the MCP server so agents can interact with your features.

Effort: Low

Consumers: PostHog AI, PostHog Code, coding agents (Claude Code, Codex, etc.), Wizard, vibecoding platforms (Lovable, Replit, etc.), ChatGPT & Claude Desktop, and more.

Skills: Teach agents how to do jobs

If you've already exposed your APIs, the next step is explaining how an agent should accomplish typical jobs-to-be-done — analyzing activity in PostHog, debugging why a feature flag was turned off, implementing enterprise features, etc. Skills combine tools, domain knowledge, and step-by-step workflows into templates agents can follow.

Effort: Medium, but the impact is very high.

Consumers: PostHog AI, PostHog Code, coding agents (Claude Code, Codex, etc.), ChatGPT & Claude Desktop, and more.

Signals: Feed the autonomy loop

If your product produces actionable or near-actionable signals — an insight threshold reached, a new error-tracking issue, a frustration pattern detected — use the signals API so agents can discover these hints and act on them later. Signals are what enable the product autonomy loop. PostHog Code acts on plans generated from these signals.

Effort: Low to medium.

Consumers: PostHog Code (local development) and PostHog AI (background agents).

PostHog Code: Features for the agentic development environment

PostHog Code is an agentic development environment where coding agents work on tasks in isolated workspaces. If your product area can make those agents smarter or the engineer's workflow faster, you can build features directly into it. Think PR reviews that check session recordings for regressions, QA steps that verify instrumentation coverage, or task prioritization that weighs your product's signals. This is the highest-effort vector but also the most deeply integrated.

Effort: High.

Consumers: PostHog Code.

Automations & background agents

Run PostHog AI based on triggers from PostHog Workflows, CRON, Temporal, etc., to automate complex workflows. Example use cases: analyze an incoming support ticket based on indexed documentation and respond to the customer, or spawn a new signal like "here is a bug, fix it."

Effort: Medium to high.

Consumers: Your persona using the web browser (UI), PostHog AI, PostHog Code, coding agents (Claude Code, Codex, etc.), Wizard, vibecoding platforms (Lovable, Replit, etc.), ChatGPT & Claude Desktop, and more.

How to get started

The AI platform is self-service by design. Follow the implementation guides to add tools and skills for your product area:

1. **Add MCP tools.** Scaffold a YAML definition, enable the operations that make sense, and add a HogQL system table for data access.
2. **Write skills.** If your product has jobs that require domain knowledge – specific tool ordering, constraints, query patterns, or reasoning about what data to check – write a skill that teaches agents how to accomplish that job well.
3. **Test with headless agents.** Validate that agents can accomplish the workflow by talking to Claude Code or another MCP-compatible agent before building any UI.
4. **Tag the PostHog AI team in PRs.** We review PRs that touch the AI platform to ensure they meet our quality bar and integrate well with the rest of the system.

For the full implementation workflow, see Implementing AI features.

When to reach out

You don't need the PostHog AI team to ship tools and skills, but we're always happy to help. Reach out to us in #team-posthog-ai if:

- **You have an unusual use case** that doesn't fit the existing tool or skill patterns
- **You need something from the AI infrastructure** that isn't supported yet
- **You want design help** thinking through how your product should work with agents
- **You're unsure whether AI is the right approach** for a problem – sometimes what seems like an AI problem is better solved another way

Don't hesitate to reach out early, even if it's just a vague idea. We'd rather help you think through the approach upfront than have you discover a dead end after building.

Best practices

Start headless, then UI

Build your product's AI capabilities as headless workflows first – expose the API as MCP tools, write skills for the key jobs. This makes the capability available across all surfaces immediately. Only add dedicated UI when a specific persona needs it. See Implementing AI features for more on this approach.

Start small

Begin with simple tools and iterate based on user feedback. It's better to ship something that works reliably for one workflow than to build something ambitious that works unreliably for ten workflows.

Describe your API fields

API field descriptions flow through the entire pipeline and become what agents read to understand tool parameters. Vague or missing descriptions lead to worse agent behavior. See Adding tools to the MCP server for details.

Contact

For questions about working with the AI platform:

- **Slack**: #team-posthog-ai
- **Team page**:
- **Objectives**: Current goals and initiatives

Bug prioritization

`contents/handbook/engineering/bug-prioritization.md`

User experience degradation

When bugs are reported it's critical to properly gauge the extent and impact to be able to prioritize and respond accordingly. These are the priorities we use across the entire engineering org, along with the relevant labels to quickly identify them in GitHub.

> Please always remember to tag your issues with the relevant priority.

GitHub Label	Description
P0	Critical, breaking issue (page crash, missing functionality)
P1	Urgent, non-breaking (no crash but low usability)
P2	Semi-urgent, non-breaking, affects UX but functional
P3	Icebox, address when possible

Security issues

Security issues, due to their nature, have a different prioritization schema. This schema is also in line with our internal SOC 2 related policies (Vulnerability Management Policy). When filing security-related GitHub issues, remember to attach label `security` and the appropriate priority label. More details on filing can be found in the README of the `product-internal` repo.

> Security issue information should not be made public until a fix is live and sufficiently (ideally completely) adopted.

PostHog security issues include a priority (severity) level. This level is based on our self-calculated CVSS score for each specific vulnerability. CVSS is an industry standard vulnerability metric. You can learn more about CVSS at FIRST.org and calculate it using the FIRST.org calculator.

GitHub Label	**Priority Level**	**CVSS V3 Score Range**	**Definition**	**Examples**
security-P0	Critical	9.0 - 10.0	Vulnerabilities that cause a privilege escalation on the platform from unprivileged to admin, allows	Vulnerabilities that result in Remote Code Execution such as Vertical Authentication

GitHub Label	Priority Level	CVSS V3 Score Range	Definition	Examples
			remote code execution, financial theft, unauthorized access to/extraction of sensitive data, etc.	bypass, SSRF, XXE, SQL Injection, User authentication bypass
security-P1	High	7.0 - 8.9	Vulnerabilities that affect the security of the platform including the processes it supports.	Lateral authentication bypass, Stored XSS, some CSRF depending on impact
security-P2	Medium	4.0 - 6.9	Vulnerabilities that affect multiple users, and require little or no user interaction to trigger.	Reflective XSS, Direct object reference, URL Redirect, some CSRF depending on impact
security-P3	Low	0.1 - 3.9	Issues that affect singular users and require interaction or significant prerequisites (MitM) to trigger.	Common flaws, Debug information, Mixed Content

Working with cloud providers

contents/handbook/engineering/cloud-providers.md

AWS

How do I get access?

Create a PR to the `posthog-cloud-infra` repository to add your details in the identity center Terraform configuration with `groups = local.default_groups`.

To give someone access

1. Add the new user to the cloud infra repo (see link above)
2. Use their email address as their username
3. Add them to the "Developers" and "DevelopersRO" groups (just use `groups = local.default_groups`)
4. Add team infra as reviewer.
5. Once this is merged, tell them to use http://go/aws to log in

Elevating permissions via `#aws-access`

To access the dev AWS environment, use the `/awsaccess` slash command in the `#aws-access` Slack channel and fill out the form that appears. Make sure to set up your AWS config file as described in our docs.

A dedicated `secrets-editor` role is available for managing secrets. Use this role across all AWS environments.

EKS access via `#aws-access` is currently in development. In the near future, expect all AWS access to be managed through the `#aws-access` channel.

Permissions errors using AWS CLI

If you see something like:

```
<my-user> is not authorized to perform: <action> on resource: <resource> with an explicit deny
```

Note the "with an explicit deny" in the end which likely is due to the fact that we force Multi-Factor Authentication (MFA). Follow this guide to use a session token.

TLDR:

1. Look up your security credential MFA device name from AWS console from `https://console.aws.amazon.com/iam/home#/users/<user-name>?section=security_credentials`
2. Run `aws sts get-session-token --serial-number <arn-of-the-mfa-device> --token-code <code-from-token> --duration 129600` where `code-from-token` is the same code you'd use to login to the AWS console (e.g. from Authy app).
3. Run the following code, replacing the placeholder values with the appropriate ones:

```
export AWS_ACCESS_KEY_ID=example-access-key-as-in-previous-output
export AWS_SECRET_ACCESS_KEY=example-secret-access-key-as-in-previous-output
export AWS_SESSION_TOKEN=example-session-token-as-in-previous-output
```

4. Unset them when done (after they expire before running `get-session-token` again):

```
unset AWS_ACCESS_KEY_ID && unset AWS_SECRET_ACCESS_KEY && unset AWS_SESSION_TOKEN
```

Deploying PostHog

See the AWS self-host deployment guide.

GCP

How do I get access?

Ask in the `#team-infrastructure` Slack channel for someone to add you.

To give someone access: Navigate to PostHog project IAM and use the `+Add` button at the top to add their PostHog email address and toggle `Basic` -> `Editor` role.

Deploying PostHog

See the GCP self-host deployment guide.

DigitalOcean

How do I get access?

Ask in the `#team-infrastructure` Slack channel for someone to add you.

To give someone access: navigate to PostHog team settings page and use the `Invite Members` button to add their PostHog email address.

Edit 1-Click app info

This can be done in the vendor portal, click on `PostHog` with Approved status to edit the listing.

The code and setup files are in digitalocean/marketplace-kubernetes repository.

Deploying PostHog

See the DigitalOcean self-host deployment guide.

Consistent scripts to rule them all

`contents/handbook/engineering/conventions/scripts.md`

PostHog has 235 (at the time of writing) public repositories. Each of these repositories has a unique way to get the project up and running locally. To limit the friction of this we adapt GitHub's approach of scripts to rule them all. As they say:

> Being able to get from git clone to an up-and-running project in a development environment is imperative for fast, reliable contributions.

Not every repository will need every script. Some repositories will need scripts custom to the environment (for example, `make` files). That's all fine. The goal is to have a baseline set of scripts that we can use to get a development environment up and a known location to look for those scripts.

Standard scripts at PostHog

When starting a new project, create a `bin` directory and include the following scripts (when relevant):

- `bin/setup` - Install or upgrade dependencies (Ex. npm packages, brew packages, etc. Usually run once after cloning the repository and occasionally to upgrade packages).
- `bin/update` - Updates dependencies after a pull. This could simply call `bin/setup`.
- `bin/build` - Build the project, for projects that are compiled such as C#, Java, etc.
- `bin/start` - Start the project. For SDKs, this might start an example server.
- `bin/test` - Run tests (Ex. `npm test`, `bundle exec rspec`, etc.). Also includes linting, formatting, etc.
- `bin/fmt` - Optional: Format/lint code. This can be called by `test`.
- `bin/docs` - Optional: Generate documentation artifacts like API and SDK references.

> **Warning:** Some environments add `bin` to the `.gitignore` file by default because that's where they compile binaries to.

Example scripts are available in the PostHog/scripts repository.

Developer Experience

`contents/handbook/engineering/developer-experience.md`

The DevEx team owns the shared developer tooling and workflows that cut across all product teams: local dev, CI, builds, framework upgrades, codebase structure, type systems, migration safety, and more. If it affects how fast and safely engineers can work on code and ship it, it's probably this team's thing.

Scope

Area	What's owned
Local dev	Local stack, hogli CLI, startup time, worktrees, Docker Compose, cloud envs
CI	Pipeline speed, cost, reliability, flaky test triage, PR previews
Build & tooling	Frontend/backend build pipelines, formatters, linters
Type system	Backend/frontend type sync, OpenAPI generation, schema integrity
Upgrades	Framework/language upgrades (Django, React, TS), dependency & security updates
Architecture	Product folder structure, isolation model, legacy migration
Migrations	Safe migration tooling, migration checkers, squashing

Things you can use

Local dev

- **hogli CLI** — start the stack, run tests, format, lint, generate types. `hogli start`
- **phrocs TUI** — manage local services, restart, view logs
- **Intent system** — only start the services you need `hogli dev:setup`

CI

- **Turborepo product tests** — fast per-product CI instead of full suite
- **Hobby PR previews** — full-stack preview environment for any PR
- **Visual review** — visual regression testing with approval flow
- **PR approval agent** — auto-approve low-risk changes

Code quality

- **Auto-generated TS types** — OpenAPI from Django serializers via Orval, always in sync
- **Formatting & linting** — oxfmt, oxlint, ruff, markdownlint in CI

- **Claude Code skills** — agent guidance for hogli, migrations, DRF endpoints
- **Product isolation** — tach-enforced boundaries between products

How to work with this team

Report what's slowing you down — flaky tests, slow builds, local dev friction, tooling that doesn't work right. A lot of it is known but there might be stuff that's been missed.

Loop the team into conversations early — if your team is making decisions that touch shared tooling, CI, code architecture, or conventions, bring DevEx in. Better to be in the discussion than clean up after it. Think: new products, services, big refactors, dependency changes, CI workflow tweaks.

Shipping & releasing

contents/handbook/engineering/development-process.md

Any process is a balance between speed and control. If we have a long process that requires extensive QA and 10 approvals, we will never make mistakes because we will never release anything.

However, if we have no checks in place, we will release quickly but everything will be broken. For this reason we have some best practices for releasing things, and guidelines on how to ship.

How to decide what to build

Set milestones

To start, Product and Engineering should align on major milestones (e.g. Collaboration) which we have strong conviction will drive our success metrics for a feature.

There are two types of goals.

- **Moonshots:** These are big, scary goals where we expect to fail 50% of the time. If we fail we expect to learn something equally as valuable as if we succeed. Just scraping the goal counts as a success.
- **Roofshots:** These might also be big, but we expect to achieve them 100% of the time. These can be goals where we cannot afford to fail (e.g. Launch feature to keep us compliant with new regulation), or where we are confident in our approach and don't foresee unexpected risks or issues.

Goals should be time-bound, but since we primarily use goals for our two-weekly sprint planning we should consider them generally timebound to two weeks.

Use the following principles:

- **Clear:** Anyone with general context can read it and instantly know what specifically it means to achieve it (i.e. NOT "refactor components")
- **Finite:** There should be an obvious end to the goal and cannot go on forever (i.e. NOT "improve dashboards")
- **Assessable:** You can validate whether or not you've achieved the goal - it doesn't need to be a metric (e.g. Increased signups by 20% or Events can be ingested in any order)
- **Meaningful:** If we achieve this goal it will solve a real need for our customers (i.e. a 10x improvement in performance sounds great as a goal - but it's not meaningful if our customers are happy with the current performance)
- **Challenging:** It should too big for one person to solve on their own and require creativity or brute force to achieve in the proposed time-frame (e.g. ship correlation analysis with a killer feature no one else has)

- **Homogenous:** The goal should be all about achieving a single meaningful thing and not a collection of unconnected things (i.e. NOT 'Improve query performance and launch collaboration MVP')

Assign an owner

A single engineer should be accountable for a milestone partnering closely with other functions to ensure it's successful.

Think about other teams

Most things won't cause issues for other teams. However, if it will, don't "align teams" or write a huge RFC (unless that'd help you). Do a quick 1:1 with the most relevant person on another team.

Consider:

- The scale of the customer you're building for
- If you can get from your hacky MVP to production-ready easily. It's OK to start with basic, but be mindful of making it harder to fully roll something out in future.
- If you know what you're doing or need someone from another team's expertise to get the right architecture or overall approach. We have lots of experienced people, get their help if you would benefit from it.

If this is a big feature which will need an announcement, content, or other marketing support then it's *never* too early for the owner to let the Marketing team know. Drop a post in their Slack channel or tagging them on an issue.

Break up goals

The owner turns the ambiguous milestone into a roadmap of ambitious, meaningful, sprint-sized goals, thinking 2 - 3 sprints ahead to give other functions time. Goal principles still apply.

Iterate through the work

We used to have company-wide sprint planning sessions but as we've grown there were so many teams that it started being plan-reading and not planning.

PostHog works in two week iterations. Each team plans their work together and adds their sprint plan to a pinned issue in GitHub. If the issue for the next iteration doesn't exist when you come to comment on it then you create it.

When planning your work you should also have a retrospective for the previous iteration. Like most things at PostHog this can be a very low ceremony retro and ideally checking the team is working on the right things in the right way is a frequent thing not a once a fortnight thing.

Work in the iteration should:

- be concrete and probably achievable in 2 weeks
- have a clear owner in the team
- have a clear link to the team or company goals

As one of our values is Why not now?, during the iteration you might come across something that should be much higher priority than what was already planned. It's up to you to then decide to work on that as opposed to what was agreed in the planning session.

Evaluate success

Review impact of each major milestone and feedback into the planning process.

When we review the status of goals we classify them as follows:

- **Nailed it:** We hit the goal spectacularly. High fives all round.
- **Scraped it:** We *almost* hit the goal, but we'll need to do a little bit more next sprint to tidy up. We should adjust our workload to have fewer resources on big goals during the next sprint to comfortably get this finished.
- **Failed it:** We were nowhere near hitting the goal, but we learned some valuable lessons. We're going to go back to the drawing board. Maybe the goal wasn't right or maybe there's a different way to approach it?

What about the small stuff?

Not everything directly contributes to a company level goal. It’s important that the small stuff also gets done for us to succeed. Use the following principles:

- **Yes, and**: Be encouraging and helpful with others who are innovating. All of our biggest wins have looked like bad ideas early on.
- **Dogfooding**: Use the product yourself. When you see something that annoys you, fix it.
- **Side quests:** Smaller projects you are passionate about but may not shoot up our metrics (e.g. turbo mode).
- **Support hero:** Support hero dedicates all of their time to customers, solving the wild and wonderful issues our customers find each week.

Sizing tasks and reducing WIP

Efficient engineering organizations actively reduce Work In Progress (WIP) to avoid negative feedback loops that drive down productivity.

Hence, a PR should be optimized for two things:

1. Quality of implementation
2. The speed with which we can merge it in

PRs should ideally be sized to be doable in one day, including code review and QA. If it's not, you should break it down into smaller chunks until it fits into a day. Tasks of this size are easy to test, easy to deploy, easy to review and less likely to cause merge conflicts.

Sometimes, tasks need a few review cycles to get resolved, and PRs remain open for days. This is not ideal, but it happens. What else can you do to make sure your code gets merged quickly?

- First, start your own day by responding to review requests from colleagues, and unblocking their work. This builds goodwill and encourages them to also review your code in priority. Otherwise, if everybody jumps to implement new features before reviewing WIP, we will end up with three, different, PRs, all for the same thing.
- Test your code. Always read through your PR's changed lines, and test everything yourself, before handing it over for review. Remember that your colleagues are busy people, and you must do what you can to save their time. There's nothing more annoying than an extra 30 min review cycle that starts with *"Almost there, just it's all black now, and remove that console.log please"*.
- Help your reviewer by leaving comments that help them review trickier bits. Better yet, write these directly into the code, either as comments or by clearly labelling your variables.
- It's always good to put new features behind feature flags. It's even better to develop partial features behind feature flags. As long as it's clear what needs to be done before a flag can be lifted, you can usually get the smallest bit of any new feature out in a day this way.
- Don't be afraid to restart from scratch if the PR gets out of hand. It's a bit of time lost for you, but a lot of time saved for the reviewer if they get a clean PR to review.
- Push your code out as a draft PR early on, so everyone can see the work in progress, and comment on the validity of the general approach when needed.
- Remember that PRs can be reverted as easily as they can be merged. Don't be afraid to get stuff in early if it makes things better. Why not now?.
- Most importantly, really understand why it's paramount to reduce WIP, until you feel it in your bones.

Writing code

We're big fans of Test Driven Development (TDD). We've tried to create test infrastructure that helps you rather than annoys you. If that isn't the case, please raise an issue! Keeping tests on point is a high priority to keep developer productivity high.

Other than that, you know what to do.

Creating PRs

To make sure our issues are linked correctly to the PRs, you can tag the issue in your commit.

```
git commit -m "Closes #289 add posthog logo to website"
```

Testing code

See: How we review.

Storybook Visual Regression Tests

In our CI pipeline, we use Playwright to load our Storybook stories and take snapshots. If any changes are detected, the updated snapshots are automatically committed to the PR. This helps you quickly verify whether you've introduced unexpected changes or if the UI has been altered in the intended way.

Check the `test-runner.ts` file to see how this is configured. We use the `@storybook/test-runner` package; you can find more details in the official Storybook documentation.

Running Tests Locally

1. **Start Storybook** in one terminal:

```
pnpm storybook
# or
pnpm --filter=@posthog/storybook
```

2. **Install Playwright and run the visual tests in debug mode** in another terminal:

```
pnpm exec playwright install
pnpm --filter=@posthog/storybook test:visual:debug
```

This setup will help catch unintended UI regressions and ensure consistent visual quality.

If you wish to locally run `test-runner.ts` and output all snapshots:

```
pnpm --filter=@posthog/storybook test:visual:ci:update
```

Or if you wish to run one particular story:

```
pnpm --filter=@posthog/storybook test:visual:ci:update <path_to_story>

# example: pnpm --filter=@posthog/storybook test:visual:ci:update
        frontend/src/scenes/settings/stories/SettingsProject.stories.tsx
```

Merge conflicts with visual regression snapshots

It happens often that your PR will show conflics with our snapshots, as our CI pipeline will run `test-runner.ts` on every push, generating and pushing to your PR any significant visual changes.

Github does not allow for conflict resolution inside their website, so you must do it manually.

> The following is done on your branch in question.

1. Bring your branch up to date with `master`.

```
git fetch origin
```

2. Rebase master into your branch

```
git rebase master
```

3. Rebase your upstream into your local branch

```
git pull --rebase <your branch>
```

In your terminal, it should show you the conflicts mimicking what you see in your Github PR.

```
warning: Cannot merge binary files:
        frontend/__snapshots__/<conflicted_file_1>.png (HEAD vs. xxx (Update UI
        snapshots for `chromium` (1)))
Auto-merging frontend/__snapshots__/<conflicted_file_1>.png
CONFLICT (content): Merge conflict in
        frontend/__snapshots__/<conflicted_file_1>.png
error: could not apply xxx... Update UI snapshots for `chromium` (1)
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --
        abort".
```

If all your conflicts are only snapshots, you can simply skip it.

```
git rebase --skip
```

If all conflicts go away, then

```
git push origin --force <your branch>
```

> Why does this work? As we mentioned earlier, our CI runs `test-runner.ts` on every push, so we don't really care if these images are conflicted as they are regenerated after you push to your branch.

Deployed Preview

You can spin up a real deployed PostHog instance to test your branch by adding the `hobby-preview` label to your PR. This uses the hobby (Docker Compose) self-hosted setup under the hood.

How it works:

1. Add the `hobby-preview` label to your PR
2. CI creates a DigitalOcean droplet and deploys PostHog with your branch
3. A comment is posted to the PR with the preview URL (e.g., `https://hobby-pr-12345.posthog.dev`)
4. The droplet persists across commits so you can iterate
5. Remove the label or close the PR to clean up the droplet

When to use it:

- Testing changes in a real deployed environment
- Manual QA before merging
- Verifying Docker Compose or deployment script changes

The workflow also runs a smoke test (health check) automatically on PRs that touch deployment-related files.

Reviewing code

When we review a PR, we'll look at the following things:

- Does the PR actually solve the issue?
- Does the solution make sense?
- Will the code perform with millions of events/users/actions?
- Are there tests and do they test the right things?
- Are there any security flaws?
- Is the code in line with our coding conventions?

Things we do not care about during review:

- Syntax. If we're arguing about syntax, that means we should install a code formatter.

Merging

Merge anytime. Friday afternoon? Merge.

Our testing, reviewing and building process should be good enough that we're comfortable merging any time.

Always request a review on your pull request by a fellow team member (or leave unassigned for anyone to pick up when available). We avoid self-merge PRs unless it's an emergency fix and no one else is available (especially for posthog.com).

Once you merge a pull request, it will automatically deploy to all environments. The deployment process is documented in our charts repository. Check out the `#platform-bots` Slack channel to see how your deploy is progressing.

We're managing deployments with ArgoCD where you can also see individual resources and their status.

Deploy notification bot

After your PR is deployed to an environment, a bot automatically comments on the merged PR with the deployment status. The **dev** deployment triggers the initial comment. As **prod-us** and **prod-eu** finish deploying, the bot updates the same comment in-place rather than posting new ones.

If you don't see a comment on your PR after a deploy, give it a few minutes -- the notification runs after ArgoCD finishes syncing. If it still hasn't appeared, check the deploy workflow in PostHog/charts for failures.

Verifying your deployment

After merging, your code should deploy automatically. If you need to verify your changes are live (or troubleshoot why they're not), here's how:

1. Check state.yaml (what *should* be deployed)

The charts repository state.yaml is the source of truth for what ArgoCD is trying to deploy. Find your service (e.g., `ingestion`, `posthog`) and check the commit SHA listed.

2. Check running pods (what's *actually* deployed)

If you have cluster access, verify what's running:

```
# Find your service's pods
kubectl -n posthog get pods | grep <service-name>
```

```
# Get the image/commit running on a pod
kubectl -n posthog get pod <pod-name> -o jsonpath='{.spec.containers[0].image}'
```

3. Verify your commit is included

Use `git merge-base` to check if your commit is an ancestor of the deployed commit:

```
git fetch origin
git merge-base --is-ancestor <your-commit-sha> <deployed-commit-sha> && echo
        "Deployed" || echo "Not deployed"
```

4. Troubleshooting with ArgoCD

If `state.yaml` shows a newer commit than what's running on pods, check ArgoCD:

1. **Find the specific app** - Don't just look at the parent grouping (e.g., `ingestion`). Drill down to the specific environment app like `ingestion-events-prod-us` or `posthog-prod-eu`.
2. **Check sync status** - Is it "Synced" or "OutOfSync"? When was the last successful sync?
3. **Check if auto-sync is enabled** - Some apps may have auto-sync disabled and require manual syncing.
4. **Look at the diff** - Click "DIFF" to see what's different between desired and live state.

Common deployment issues

Symptom	Likely cause	Solution
App shows "OutOfSync"	Auto-sync disabled or sync error	Check if auto-sync is enabled; try manual sync
state.yaml updated but pods unchanged	ArgoCD hasn't synced yet	Check ArgoCD app status; may need manual intervention
Pods running old commit	Rollout stuck or image not built	Check deployment rollout status; verify CI built the image
Can't find your service in ArgoCD	Looking at wrong app grouping	Search for your specific service + environment (e.g., `ingestion-events-prod-us`)

If a deployment appears stuck, reach out in `#team-infrastructure`.

Documenting

If you build it, document it. You're in the best position to do this, and it forces you to think things through from a user perspective.

It's not the responsibility of either or teams to document features.

See our docs style guide for tips on how to write great docs.

Releasing

There are a few different ways to release code here:

- just release the code change directly
    - when you have hign confidence the change is safe
- release it behind a flag and slowly roll it out
    - when you don't need to run an AB test but want to be sure you can check the impact of the change
- release it behind a flag and roll it out on demand (we call this a closed beta)
    - when you want to slowly release this to people who know they'll likely need to give feedback
    - you know it isn't complete and you need early feedback
- release it behind a flag and run it with an AB experiment
    - you don't know what impact it will have and want to measure it
- release it behind a flag and put it in a feature preview (we call this an open beta)
    - when you want to slowly release this to people who know they'll likely need to give feedback
    - you know it isn't 100% and you need feedback
- run old and new at the same time
    - sometimes called the strangler fig https://martinfowler.com/bliki/StranglerFigApplication.html
    - run both old and new and compare the output / effect
    - you can then cut across (sometimes in stages) before removing the old code

Best practices for full releases

Opt-in betas can have rough edges, but public betas and full releases should be more polished and user friendly.

Engineers should apply the following best practices for *all* new releases:

- Ensure Marketing is aware of the launch, so a launch plan can be created.
- Ensure docs are updated to reflect the new release.
- Ensure all new features include at least one pre-made template (or equivalent) for users.

When to A/B test

There are two broad categories of things we A/B test:

- Changes intended to move a metric (eg. changing CTAs to see if it improves click-through)
- Changes that could impact large swaths of users and their behavior, to make sure there is no negative impact (eg. moving all items in the left nav into a drawer)

The former is an optimization scheme; the latter makes sure we don't break things. Just like we create tests in our codebase to make sure new code doesn't disrupt existing features, we also need to do behavioral testing to make sure our new features aren't disrupting existing user behaviors.

A/B tests make sense when:

- There is sufficient traffic to give results in 1-2 weeks
- The change isn't simply adding a new feature (eg adding a totally new feature and A/B testing if people use the feature isn't exactly informative, though you *should* be looking at metrics for features you ship to see if anyone uses them)
    - If the feature is designed to improve some other metric like retention or stickiness, then test away!
- The change impacts user behavior (eg most backend changes should have code tests - not behavioral A/B tests)

If you're not sure something should be A/B tested, run one anyway. Feature flags (which experiments run on top of) are a great kill-switch for rolling back features in case something goes sidwways. And it's always nice to know how your changes might move the numbers!

It's easy to just think "this makes more sense, let's just roll it out." Sometimes that's okay, sometimes it has unintended consequences. We obviously can't and shouldn't test everything, but running A/B tests frequently gets you comfortable with being wrong, which is a *very* handy skill to have.

A/B test metrics

Experiment design is a bit of an art. There are different types of metrics you can use in PostHog experiments. Another benefit of running experiments is forcing yourself to think through what other things your change might impact, which oftentimes doesn't happen in the regular release cycle!

Generally, a good pattern is to set up 1-2 primary metrics that you anticipate might be impacted by the A/B test, as well as 3+ secondary metrics that might also be good to keep an eye on, just in case. If you aren't sure what metrics to be testing, just ask! Lots of people are excited to help think this through (especially #team-growth and Raquel!).

Releasing a new product

We can release alphas and betas both as publicly available, or opt-in. See: Releasing new products and features.

It's always worth letting Marketing know about new betas so they can help raise awareness. The owner should tag them on an issue, or drop a message in the Marketing Slack channel.

Betas are usually announced as milestones on the public roadmap and included in the changelog by Marketing.

Product announcements

Announcements, whether for beta or final updates, are a Marketing responsibility. See: Product announcements.

In order to ensure a smooth launch the owner should tell Marketing about upcoming updates as soon as possible, or include them in an All-Hands update.

It's *never* too early to give Marketing a heads-up about something by tagging them in an issue or via the Marketing Slack channel.

Self-hosted and hobby versions

We have sunset support for our kubernetes and helm chart managed self-hosted offering. This means we no longer offer support for fixing to specific versions of PostHog. A docker image is pushed for each commit to master. Each of those versions is immediately deployed to PostHog Cloud.

The deploy-hobby script allows you to set a `POSTHOG_APP_TAG` environment variable and fix your docker-compose deployed version of PostHog. Or you can edit your docker-compose file to replace each instance of `image: posthog/posthog:$POSTHOG_APP_TAG` with a specific tag e.g. `image: posthog/posthog:9c68581779c78489cfe737cfa965b73f7fc5503c`

Feature ownership

contents/handbook/engineering/feature-ownership.md

Each feature at PostHog has an Engineering owner. This owner is responsible for maintaining the feature (keep the lights on), championing any efforts to improve it (e.g. by bringing up improvements in sprint planning), planning launches for new parts of it, and making sure it is well documented.

For shared developer tooling and infrastructure that cuts across product teams (CI, local dev, builds, migrations, etc.), see the Developer Experience page.

When a bug or feature request comes in, we tag it with the relevant label (see labels below). The owner is responsible for then prioritizing any bug/request that comes in for each feature. This does not mean working on every bug/request, an owner can make the deliberate decision that working on something is not the best thing to work on, but every request should be looked at.

Who can contribute to owned features?

Feature ownership does **not** mean that the owner is the **only** person/team who can contribute to the feature. If another team requires something from an existing feature that isn't supported, that non-owning team should build it. The owner team is responsible for reviewing PRs to make sure the code patterns and UX makes sense for the feature overall. After the change is merged, the owner team then owns it (assuming no major bugs from the initial implementation).

For example, web analytics wanted a heatmap insight type to see what times of day people were active. from web analytics opened up the necessary PRs to build this feature. It was reviewed by the , owner of all insight types, who then took responsibility for it after it was merged.

This process does four things:

- It prevents people feeling like they need to wait on another team to build out necessary functionality for them
- It ensures that features built by another team get proper review, because reviewers know they will have to own it eventually.
- It makes sure no feature is left "orphaned" with no real owner.
- It embraces our value of Why not now?.

Feature list

You can also view the list directly in GitHub and filter issues there.

Don't just copy other products

Some of the features we are building may exist in other products already. It is fine for us to be inspired by them - there's no need to reinvent the wheel when there is already a standard way our users expect things to work. However, it is not ok for us to say 'let's copy how X does it', or to ship something with the exact same look and feel as another product. This is bad for two reasons:

- We're highly unlikely to overtake everyone else if we just build the open source version of everything that is already out there.
- We may expose ourselves to legal risk/challenges from those companies, especially if they can point to a public issue where we have said 'let's copy X'.

Pricing principles

contents/handbook/engineering/feature-pricing.md

In an ideal world, Posthog's pricing enables users and organizations to:

1. Use PostHog for free if they are hobbyists or pre-PMF.
2. Experience the product before paying for it.
3. Start paying when they are ready, on their own, with few hurdles.
4. Transparently pay for the value they receive.

- e.g. Usage-based pricing on events, recordings.
- e.g. Paying per product, so they only pay for what they use.

5. Make it a no-brainer to pick PostHog over other competitors.

Our goals with these principles are to:

- Keep the engineers at PostHog as close to our customers as possible, so they can build new products or improve existing products in ways that are most impactful for them.
- Maintain low barriers to entry for our customers, so they can see value in PostHog quickly.
- Ensure transparency around the value we provide to our customers.
- Tightly couple our success with that of our customers'. The more we can help them succeed, the more we will succeed – e.g. with usage-based pricing for resources that scale.

It's important we evaluate all new features, and shifts in our pricing plans, to ensure they align with our pricing values.

In the real world

Sometimes these principles still leave room for questions – what, if anything, should be available in the free tier? What about enterprise customers?

For these types of questions, we've defined a runbook for deciding which plans, and at what limits, features should be assigned to.

We should roughly match the cheapest competitor

In general, we should roughly match the pricing of the cheapest big competitor for that product, so long as the unit economics make sense, to make it a no-brainer to use PostHog. To qualify for this, a competitor must be *making actual revenue* at significant scale - we won't match the pricing random startups or new products at existing competitors offer, since these products and GTMs aren't mature yet.

We can do this because we can upsell customers multiple of our other products. The total ACV is higher even if the per-product ACV is lower.

It's better for customers because they get all these tools that are well integrated for the cheapest possible price.

While we don't have loss leaders, we accept that we might not fully understand our cost base and make money on every product on day one. We welcome this pressure to do things more efficiently and get the costs down over time.

Every product should be priced separately

Whenever we build a product, like feature flags, or product experimentation, we should have a specific price for that product by itself. Being consistent here is less confusing than randomly combining products for example, even though it will sometimes mean more items to explain to a customer.

It means that customers who want just one product can compare each of our products to our competitors', seeing that we are cheaper everywhere, improving our self-serve top-of-funnel.

This also makes the value of each product more tangible. Usage and value are not the same thing - willigness to pay is the best indicator of the value our customers are getting from each product.

However, when one of our products has a fundamental dependency on another of our products, we should aim to bundle the cost of the dependencies in with the product's pricing so customers only pay once for using a given product.

For example, when someone calls a feature flag, we send a `$feature_flag_called` event so we can have stats. In this case, we don't charge for those events, as the events are solely related to feature flags.

Features that increase our stickiness should be free (with a reasonable limit)

A good question to ask yourself here is, "If I were to switch away from PostHog, would I feel like I am losing anything by switching?"

For example, if someone were to consider moving from PostHog to some other provider, cohorts would need to be manually recreated in the other provider, which would be tedious. However, something like Web Performance just happens and doesn't require any user involvement, so isn't sticky.

Products should work independently but shine together

Each product should be usable on its own. For example, session replay can be enabled independently of other products. But to get the most value out of it, it's best to use it together with our other products. This enables users to have rich filters using the data from the other parts of PostHog. Similarly, you can use error tracking on its own, but it's a lot more powerful if you also use session replay, enabling you to easily click through to the recording of a session where the error occurred.

Other guidelines

- We accept pricing complexity for the benefit of the users. Usage-based pricing is inherently more complex (for users and for us) than e.g. flat rates, but it ensures that users only pay for what they use, and allows us to understand true value that they're getting out of each product.
- We should always ask ourselves how newly released features should be priced, even if it's launching as a free product. A default behavior is good, but it shouldn't be used as a replacement for critically thinking about where something fits into our pricing scheme.
- Our default assumption for new features is that full usage is only available on the paid plans.
- Features that need to be experienced in order to demonstrate value should be available on the free plan but with a reasonable limit.
- Features that have the potential to grow our word-of-mouth should be free – e.g. we shouldn't (and don't) charge for extra users in an organization because the more people we get inside PostHog, the better.
- Features that are focused around extra security, permissioning, compliance, or other enterprise-style upgrades should be reserved for our enterprise pricing tier.
- We shouldn't assume that all products should be usage-based. For some products the engineering time is the most expensive part, and in these cases we should consider tiered fees, monthly flat fees, or seat-based pricing where it makes sense.
- Unless there is a very good reason not to, we should grandfather existing customers' pricing tiers if they are cheaper than the new pricing to avoid unexpected pricing changes.

Deciding on a free volume, and making changes to it

- When choosing a free volume for a new product, we should choose a value that is in line with our pricing principles: It should give customers the opportunity to experience the product before paying for it, and we should roughly match our competitors if they offer a free tier.
- Keep in mind: It's easy to increase the free tier for existing customers, but it's very painful to decrease it (since we don't want existing customers to pay more).
- If we decide to lower the free tier as part of a wider pricing change (primarily when we lower our prices), in principle we should roll out the new pricing **and** the new free tier to existing customers, because they will likely save money. An exception should be made for customers who are forecasted to pay more. In these cases we should enroll them in the new pricing, but grandfather the higher free tier.

How-to access PostHog Cloud infra

contents/handbook/engineering/how-to-access-posthog-cloud-infra.md

We've all been there. Something was just merged and now there is a bug that you are having a real hard time pinning down. You hate to do it... but you need to get on a pod or instance to troubleshoot the issue further. *SHAME*

Prerequisite

Make sure you've followed this guide to get AWS access. !!! Please follow the whole document !!!

Connect to a Kubernetes pod

After you got access to the EKS cluster and our internal network:

- `kubectl -n posthog get pods` (get names of pods, you'll want a "web" pod most likely)
- `kubectl -n posthog exec --stdin --tty <POD_NAME> -- /bin/bash` (get a shell to the running container)
- `kubectl -n posthog exec <POD_NAME> env` (run individual commands in a container)

Note: if you need a Django shell, just run the following after connecting:

```
python manage.py shell_plus
```

Connect to an EC2 instance

Please follow this guide to connect via AWS Systems Manager Agent (SSM).

How we review PRs

contents/handbook/engineering/how-we-review.md

Almost all PRs made to PostHog repositories will need a review from another engineer. We do this because, almost every time we review a PR, we find a bug, a performance issue, unnecessary code or UX that could have been confusing. Here's how we do it:

Before requesting a review

The best way to get a fast, useful review is to make your PR easy to review.

- **Keep PRs small.** If your change touches many files or mixes unrelated concerns, break it into a stack of smaller PRs. Smaller PRs get reviewed faster and reviewed better.
- **Open a draft PR.** This keeps notifications quiet and lets you iterate without pinging reviewers.
- **Add AI reviewers** (e.g. Copilot) and resolve their comments. Iterate until they're only leaving nit-level feedback.
- **Self-review your own diff.** Read through it as if you're seeing it for the first time. You'll catch obvious issues before someone else has to.
- **Write a clear description.** Explain what the change does and why. Link the issue. If there's context a reviewer needs, put it in the description — don't make them guess.
- **Add screenshots or GIFs for UI changes.** A reviewer shouldn't have to pull the branch and navigate to the right page just to see what a button looks like.
- **Make sure CI is green.** Don't ask someone to spend time reviewing code that doesn't pass checks.
- **Mark it ready for review.**

Have a flick through the code changes

What to look for:

- Does the code fit into our coding conventions?
- Is the code free of bugs?
- How will the solution perform at huge scale?
    - Are the database queries scalable (do they use the right indexes)?
    - Are the migrations safe?
- Are there tests and do they test the right things?
- Is the solution secure?
    - Is there no leakage of data between projects/organizations?
- Is the code properly instrumented for product analytics?
- Is there logging for changes potentially affecting infrastructure?
- Are analytics query changes covered by snapshot tests? Does the SQL generated make sense?

What *not* to look for:

- Syntax formatting. If we're arguing about syntax, that means we should be using a formatter or linter rule.

Run the code yourself

What to look for:

- Does the PR actually solve an issue?
  - Are we building the right thing? (We should be willing to throw away PRs or start over)
- Does the change offer a good user experience?
- Does the UI of the change fit into our design system?
- Should the code be behind a feature flag?
  - If the code is behind a feature flag, do all cases work properly? (in particular, make sure the old functionality does not break)
- Are all possible paths and inputs covered? (Try to break things!)

What *not* to look for:

- Issues unrelated to this PR. Create new, separate issues for those.

The emphasis should be on getting something out quickly. Endless review cycles sap energy and enthusiasm.

Turnaround

Aim to respond to review requests within one working day. You don't have to finish the review — even a quick "I'll look at this properly tomorrow" or "this needs someone from [@team-name] to review" unblocks the author and sets expectations. Leaving a PR in limbo for days is worse than a fast "I can't review this."

Requesting a review outside your team

Not every team has someone available to review your PR right away. Posting in #dev-stamp-exchange is a way to ask for a quick-turnaround review from someone outside your team. This is fine — but quick turnaround doesn't mean lower standards.

When this is appropriate:

- The PR is small and self-contained (think single-digit files changed)
- It doesn't require deep product or architectural context to understand
- CI is green and any automated review comments are addressed

What's still expected from the reviewer:

- Actually read the diff — don't just hit approve
- Consider using AI-assisted review tools (e.g. add Copilot as a reviewer) to catch things you might miss
- Flag anything that looks off, even if you're not deeply familiar with the area

When to push back instead of approving:

- The PR is too large or complex to review without context
- There are no tests, no description, or no visual evidence of the change working
- You're not confident the change is safe — say so. "I can't meaningfully review this, you need someone with more context" is always valid feedback

Review comment conventions

Use prefixes on your review comments so the author knows what actually needs to change before merging:

- **`blocking:`** This must be fixed before merge. Use sparingly — reserve it for bugs, security issues, or things that will break.
- **`nit:`** A minor style or naming suggestion. Take it or leave it.
- **`suggestion:`** A different approach worth considering, but the author's call.
- **`question:`** You don't understand something. Not necessarily a problem, but you'd like clarification.

If a comment doesn't have a prefix, assume it's a suggestion. This avoids the "is this a blocker or just a thought?" ambiguity that slows reviews down.

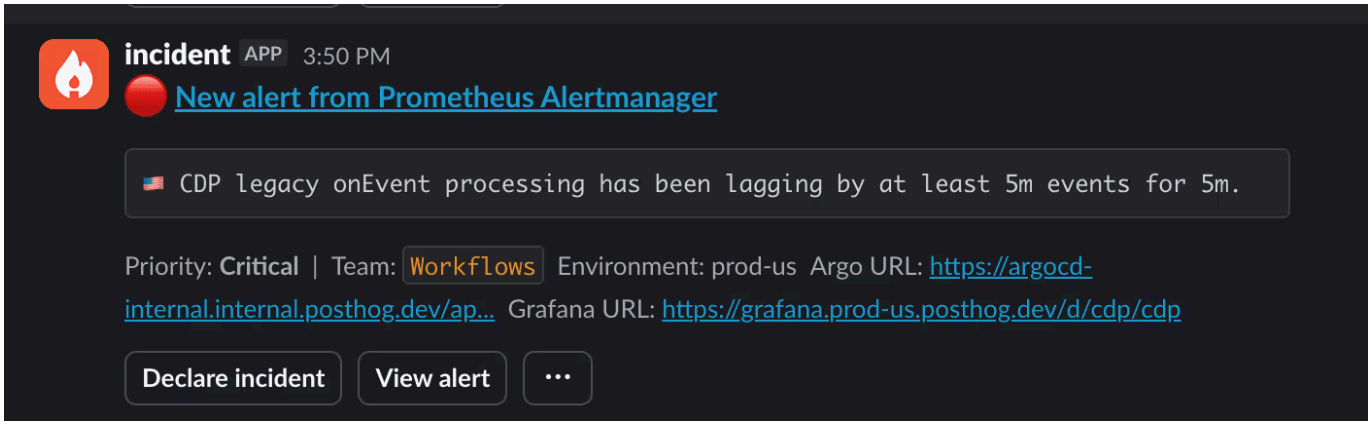
Handling an incident

`contents/handbook/engineering/operations/incidents.md`

The TL;DR

- If you get paged, acknowledge the page and look at the associated metrics - if it looks even slightly bad and not recovering - **CREATE AN INCIDENT**
- If you notice something broken with the app (not just a bug) - **CREATE AN INCIDENT**
- If you are not sure - **CREATE AN INCIDENT**
- *How?*
  - Click the `Declare incident` button on an alert or do `/inc` in any slack channel
- *What?*
  - Join the incident channel
  - Assign yourself as lead (you can always re-assign later)
  - Share whatever info you have at that time
  - Escalate by bringing in the relevant team, engineers or via incident.io using the options a the top of the channel
  - Update the statuspage if there is any noticeable impact to users

Raising an incident



Incidents are going to happen. If you'd rather watch a Loom, check out an incident drill Loom recording.

> **Anyone can declare an incident and, when in doubt, you should always raise an incident.** We'd much rather have declared an incident which turned out not to be an incident. Many incidents take too long to get called, or are missed completely because someone didn't ring the alarm when they had a suspicion something was wrong. It's *always* better to sound an alarm than not.

To declare an incident, type `/incident` anywhere in Slack. This creates a new dedicated channel for the incident and add a few stakeholders. It will trigger an alert in the #incidents channel so everyone else can be aware. Declaring an incident **doesn't** trigger any external notifications.

Once an incident is raised an automatic workflow begins that will help you summarize the issue and escalate it appropriately.

Some things that should definitely be an incident

- `us.posthog.com` (PostHog Cloud US) or `eu.posthog.com` (PostHog Cloud EU) being completely unavailable (not just for you)
- No insights can be created
- Feature flags are not being returned at all, or `/flags` is down
- Any feature is 'down' and users are unable to access their existing data through it (this can be a bug and doesn't have to be an infra incident)
- Various alerts defined as critical, such as disk space full, OOM or >5 minute ingestion lag

Things that *shouldn't* be an incident

- Insights returning incorrect data
- Events being < 5-10 minutes behind (E2E ingestion lag)
- Unable to save insights, create feature flags
- Expected disruption which happens as part of scheduled maintenance

> Planning some maintenance? Check the announcements section instead.

Security-specific guidance

Security incidents can have far-reaching consequences and should always be treated with urgency. Some examples of security-related issues that warrant raising an incident include:

- Unauthorized access to systems, data, or user accounts
- Detection of malware, ransomware, or other malicious software on company systems
- Suspicious activity on production infrastructure, such as unexpected user logins, privilege escalations, or resource consumption spikes
- Discovery of exposed credentials, sensitive data, or secrets in logs, repositories, or public forums
- Receiving a credible report of a vulnerability or exploit affecting company systems

When in doubt, err on the side of caution and raise the incident and escalate early! Better to be safe than sorry.

> **Need to make a security advisory?** We have a page for that with more detail on the process for security vulnerabilities.

Incident severity

Please refer to the following guidance when choosing the severity for your incident. If you are unsure, it's usually better to over-estimate than under-estimate!

Minor

A minor-severity incident does not usually require paging people, and can be addressed within normal working hours. It is higher priority than any bugs however, and should come before sprint work.

Examples

- Broken non-critical functionality, with no workaround. Not on the critical path for customers.
- Performance degradation. Not an outage, but our app is not performing as it should. For instance, growing (but not yet critical) ingestion lag.
- A memory leak in a database or feature. With time, this could cause a major/critical incident, but does not usually require *immediate* attention.
- A low-risk security vulnerability or non-critical misconfiguration, such as overly permissive access on a non-sensitive resource

If not dealt with, minor incidents can often become major incidents. Minor incidents are usually OK to have open for a few days, whereas anything more severe we would be trying to resolve ASAP.

Major

A major incident usually requires paging people, and should be dealt with *immediately*. They are usually opened when key or critical functionality is not working as expected.

Major incidents often become critical incidents if not resolved in a timely manner.

Examples

- Customer signup is broken
- Significantly elevated error rate
- A Denial of Service (DoS) attack or other malicious activity that affects system availability
- Discovery of exposed sensitive data (e.g., credentials, secrets) that could lead to a security breach if not remediated

Critical

An incident with very high impact on customers, and with the potential to existentially affect the company or reduce revenue.

Examples

- Posthog Cloud is completely down
- A data breach, or loss of data

- Event ingestion totally failing - we are losing events
- Discovery of an active security exploit, such as a compromised user account or system
- Detection of ransomware, malware, or unauthorized modifications to production systems

What happens during an incident?

When an incident is declared, the person who raised the incident is the incident lead. It's their responsibility to:

- Make sure the right people join the call. This includes the current on-call person (@on-call-global in Slack) and the team responsible for the alert (we have a workflow which will try to add these people automatically). Optionally, add people from Infra and the feature owner and Support. Product Marketers can assist in running communications if required.
- Take notes in the incident channel. This should include timestamps, and is a brain dump of everything that we know, and everything that we are or have tried. This will give us much more of an opportunity to learn from the incident afterwards.
- Update the status page. If the incident happens during business hours, the incident should have a watcher. If needed, ask the support team for help managing the status page so you can focus on the technical management of the incident. The status page can be updated from:
  - (recommended) the incident Slack channel using `/incident statuspage` (`/inc sp`)
  - the status page area of the incident.io dashboard (only recommended for corrections/modifications - Slack tooling provides better context)

The incident lead role is not responsible for fixing the incident, they're responsible for managing it. Sometimes that will be the same person. But if it is too much work for one person, hand over the incident lead role to someone else not actively working on the fix.

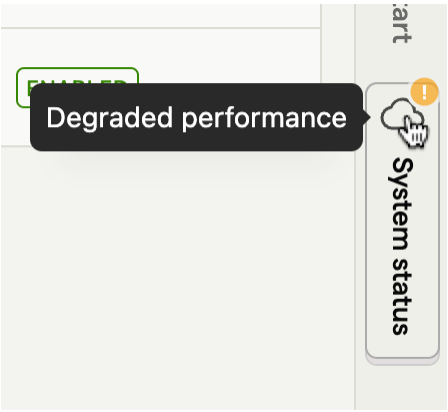
Sometimes, customer communication is required. In this case, the incident lead can ask for a comms lead to support the responding team. The best way to do this is to ask for support in the incident channel and use the `@all-marketers` group tag. Don't be shy.

You can find further production runbooks + specific strategies for debugging outages here (internal)

The PostHog status page

Our status page is the central hub for all incident communication. You can update it easily using the `/incident statuspage` (`/inc sp`) Slack command.

When updating the status page, make sure to mark the affected component appropriately (for example during an ingestion delay, setting `US Cloud` 🇺🇸 / `Event and Data Ingestion` to `Degraded Performance`). This allows PostHog's UI to gently surface incidents with a "System status" warning on the right. Only users in the affected region will see the warning:



Getting help from a comms lead

Significant incidents such as the app being partially or fully non-operational, as well as ingestion delays of 30 minutes or longer should be clearly communicated to our customers. They should get to know what is going on and what we are doing to resolve it. If the incident is minor this can usually be done by updating the status page, but it may be desirable to do additional customer communications, such as sending an email to impacted customers. When this is required, you should involve a Comms Lead and ensure the Sales team are aware.

The best way to ask for support from a Comms Lead is to post in the incident channel and use the `@all-marketers` group tag. This will alert the all relevant marketing teams.

When handling a security incident, please align with the incident responder team in the incident Slack channel about public communication of security issues. For example, it may not make sense to immediately communicate an attack publicly, as this could make the attacker aware that we are already investigating. This could it make harder for us to stop this attack for good.

When a customer is causing an incident

In the case that we need to update a *specific* customer, such as when an individual org is causing an incident, we should let them know as soon as possible. Use the following guidelines to ensure smooth communication:

- Look up the customer's org in Vitally to see if the org has an Account Exec assigned (`PostHog Default Dashboard` / righthand column, scroll down to `Key Roles`.) If so, let the AE know about the situation early.
- Ensure you are always contacting the admins of the impacted organization
- Communication is best done through Zendesk. It's usually best for the customer's assigned person (check Vitally), or the Support team, to create tickets and handle the communication for you, but don't wait if it's really urgent.
- Before sending any comms, check with the incident lead. Then, share a ticket link in the incident channel.
- If action is needed, it's better to take that action and inform the user than to ask the user to do it.

- If you're not able to take the required action, give the user deadlines for the changes they need to do and explain what will happen if they don't meet the deadline.
- Try to keep all communication on a single ticket, with all relevant parties.

In the case that we need to temporarily limit a *specific* customer's access to any functionality (e.g. temporarily prevent them from using an endpoint) as a result of certain usage resulting in an incident, we need to make sure we put an alert on their Zendesk tickets. This will make sure that anyone working on a ticket from the org will know what's happening with the org before replying (even if we've already reached out to the org, some folks at the org may not be aware, and so may open a support ticket.)

You'll just need to set the name of the org in an existing trigger in Zendesk, then reverse that change when the org's full access has been restored:

1. After Logging into Zendesk, go to the admin center
2. In the left column, expand `Objects and rules` and click on `Triggers` (under "Business rules")
3. On the Triggers page, expand `Housekeeping` and click on `Add alert for org with special-handling`
4. Under `Conditions`, the last condition is: `Organization > Organization` `Is` `PostHog`. Change `PostHog` to the name of organization who has had their access limited as a result of the incident. (Click on "PostHog" and then start typing to filter and find the org name, then click on it)
5. Scroll to the bottom of the page and click the `Save` button

Once the org has had their full access restored, repeat the steps above, but this time put `PostHog` back in the last condition, and remember to `Save` the change.

When does an incident end?

When we've identified the root cause, implemented a fix, and confirmed all customer-facing services have returned to normal. End the incident by typing `/inc close` in the incident channel. Make sure to also mark the incident as resolved on the status page.

What happens after an incident?

Once the incident is resolved, the incident lead should **step away**. Take a walk, go to the gym, have some tea, take a shower. The longer the incident took to resolve, and the more directly customer impacting it was, the more important this is. Bring another team member up to speed, hand off outstanding customer communications, and get your head clear for the post-mortem and followup actions. Anyone else heavily involved in the response should do the same.

In almost all cases, a valid incident will have a post-mortem - check out Post-mortems for more details.

On-call and escalation

`contents/handbook/engineering/operations/on-call-rotation.md`

At PostHog, every engineer is responsible for maintaining their team's services and systems. That includes:

- Tracking and visualizing key performance metrics
- Configuring automated alerting
- Documenting runbooks for on-call resolution
- First-responder to team-owned services during working hours

In addition, every engineer regardless is part of the global follow-the-sun on-call rotation.

Escalation schedules

Team schedules

Every team has 2 schedules in incident.io

- `On call: {team}`
  - This is the working-hours rotation. Each engineer should have their working hours in place here Mon-Fri with a sensible working day
  - For example 8:00-17:00 for EU based engineers is likely preferable as there will be US engineers who can take 17:00 onwards
  - Each member is responsible for ensuring this is up-to-date with PTO. You can create an override for your schedule simply assigned to "No one".
- `Support: {team}`
  - This is a weekly or bi-weekly rotation (teams can decide) that covers both who is assigned to the support hero rotation as well as the out of-hours-escalation for the extreme case

Global on-call schedule

Schedule in incident.io

> 💡 You can use @on-call-global in Slack to reach out to whoever is on call! This syncs automatically with the incident.io schedule. This group is also automatically added to all incidents.

PostHog Cloud doesn't shut down at night (*whose* night anyway?) nor on Sunday. As a 24/7 service, our goal is to be 100% operational 100% of the time. The global on-call is the last line of defense and is escalated to:

- if nobody at the `On call: {team}` level is available
- if the alert is critical but has no team assignment (for whatever reason)

This schedule has 3 week day layers:

- **Europe** (06:00 to 14:00 UTC) - (8 hours)
- **Americas East** (14:00 to 22:00 UTC) - (8 hours)
- **Americas West** (22:00 to 06:00 UTC) - (8 hours)

And 2 weekend layers:

- **Europe Weekend** (06:00 to 18:00 UTC) - (12 hours)
- **Americas Weekend** (18:00 to 06:00 UTC) - (12 hours)

Why is the on-call rotation spread across all engineers?

If you're in a product team, it's tempting to think that service alerts don't apply to you, or that when you're on call you can just hand everything off to the infrastructure team. That's not the case, because it's important that every engineer has a basic understanding of how our software is deployed, where the weak points in our systems are, and what the failure modes look like. This understanding should be all that's needed to follow the runbooks, and if you follow the causes of alerts, ultimately you'll be less likely to ship code that takes PostHog down.

Besides knowledge, being on call requires availability – including weekends. If teams had their own separate rotations, there would be more people on call in total, and each would have to stand by 24/7 as our teams aren't big enough to follow the sun. This would be more stressful because of availability constraints, while being less productive because of the rare alerts being spread across multiple people.

Before going on call

Be prepared

Because the stability of production systems is critical, on-call involves weekends too (unlike Support Hero). More likely than not, nothing will happen over the weekend – but you never know, so the important thing is to keep your laptop at hand.

Before going on call, make sure you have the **Incident.io mobile app** Android / iOS installed and configured. This way it'll be harder to miss an alert.

> TRICKY: We use Slack auth for incident.io and Slack really doesn't like you using the mobile web version. Make sure to choose `Sign in with Slack` and then use your email to login to Slack, **not** google auth as that seems to cause redirect issues for some people.

> Still having redirect issues signing up with Slack? Create a Slack password instead of using Google SSO, then log in with that password.

To get a calendar with all your on-call shifts from incident.io go to the schedules section, select `Sync calendar` at the top right and copy the link for the webcal feed. In google calendar, add a new calendar from URL and paste the link in there.

Make sure your availability is up-to-date

If you are unavailable for any of your schedules you need to act!

1. For your `On call: {team}` schedule simply click on your name in your layer, click `create an override` and then remove yourself from the list so it shows `No one`
2. For your `Support: {team}` schedule or `On call: {global}` schedules click `Request cover` at the top right. This will notify selected team members automatically to find someone to cover you (you should probably do a shout out in #ask-posthog-anything as well). You can trade whole weeks, but also just specific days. Remember not to alter the rotation's core order, as that's an easy way to accidentally shift the schedule for everyone.

Make sure you have all the access you might need

To be ready, make sure you have access to:

- PostHog Cloud admin interfaces (🇺🇸 US / 🇪🇺 EU) - post in #ask-posthog-anything to be added
- Grafana (🇺🇸 US / 🇪🇺 EU)
- ArgoCD - this is where 99% of cluster operations take place such as restarting pods, scaling things up and down etc.
- Metabase (🇺🇸 US / 🇪🇺 EU) - post in #ask-posthog-anything to be invited

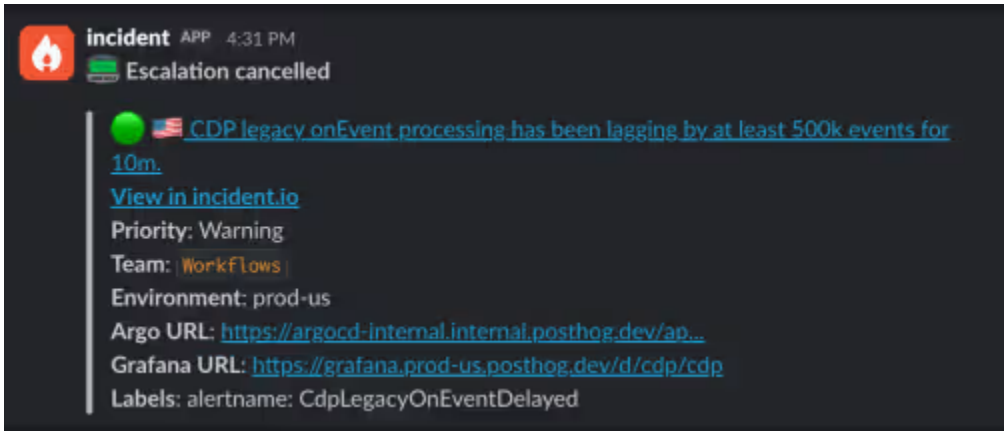
More advanced access

If you are part of a team that looks after more critical infrastructure such as infra, ingestion, workflows, error-tracking etc. then you are expected to dive deeper than the usual on-call engineer.

As well as the above access you should ensure you have access and feel comfortable working with:

- EKS over `kubectl` / `k9s`, in case you need to run Kubernetes cluster operations (such as restarting a pod) – follow this guide to get access
- Our tailnet, which gates our internal services (such as Grafana, Metabase, or runbooks) – follow this guide to join

Responding to alerts when on-call



Critical alerts will trigger per-team escalation policies which go like this:

1. If available, a member of the team associated with the alert is paged first
2. If nobody is available or nobody responds within the configured time then the `On call: global` schedule is paged

> **If at any point you get paged - always respond!** Even if you are unavailable you should respond as such (either via the app or the personal Slack notification). That way the escalation can continue to the next available person.

By default if you are being paged, especially as the global on-call, the alert is considered critical, meaning it almost definitely requires attention.

Every alert should have associated Grafana and Runbook links allowing you to quickly get more visual details of what is going on and how to respond.

When an alert fires, find if there's a runbook for it. A runbook tells you what to look at and what fixes exist. In any case, your first priority will be to understand what's going on, and the right starting point will almost always be Grafana.

Sometimes alerts are purposefully overly-sensitive and might already be fixing themselves by the time you see them. **Use your best judgement here**. If the linked graph has a spike that is clearly coming down, watch it closely and give it time for the alert to auto-resolve.

If the alert is starting to have any noticeable impact on users or you are not sure whether to raise an incident - go raise an incident. It's that simple.

If you're stumped and no resource is of help, get someone from the relevant team to shadow you while you sort the problem out. The idea is that they can help you understand the issue and where to find how to debug it. The idea is *not* for them to take over at this point, as otherwise you won't be able to learn from this incident.

Post-mortems

`contents/handbook/engineering/operations/post-mortems.md`

At PostHog, we believe that incidents are learning opportunities. Every incident, whether major or minor, provides valuable insights that help us improve our systems, processes, and response capabilities. Post-mortems are our way of capturing these lessons and ensuring we continuously improve.

Why post-mortems matter

Post-mortems serve several critical purposes:

- **Learning from failures** – Understanding what went wrong and why helps prevent similar issues
- **Process improvement** – Identifying gaps in our monitoring, alerting, or response procedures
- **Knowledge sharing** – Ensuring the entire team benefits from lessons learned
- **Documentation** – Creating a historical record of incidents and their resolutions

Post-mortem process

Incidents can be stressful and time consuming but it's equally important that we take the time to learn from them and improve our systems and processes. The longer you wait, the more details you'll forget and the less valuable the post-mortem becomes.

We use incident.io's post-mortem template which helps guide you through the process. They also have hints on what kind of content you should be focusing on in each section.

For major incidents

Major incidents require a **team call** to discuss and learn together:

1. **Write the post-mortem** – Fill out the template in the incident page (you will be prompted to do this when the incident is resolved).
2. **Fill out the Summary and DERP sections in detail** – These are the most valuable
3. **Check the timeline is accurate** –
4. **Schedule the call** – Invite `engineering@posthog.com` and any key stakeholders related to the incident
5. **Review as a group** – There may be details and other ideas people come up with in the call – you should be updating the post-mortem as you go to capture these.
6. **Share outcomes** – Post the final summary in `#incidents` (this should happen automatically when you mark the post-mortem as complete)

For minor incidents

Minor incidents can be handled more simply:

1. **Write the post-mortem** – Fill out the template
2. **Focus on the summary and DERP** – The timeline here is less important.
3. **Share the summary** – Post in `#incidents` channel for visibility (this should happen automatically when you mark the post-mortem as complete)

For false-positive incidents

Sometimes incidents are raised but turn out to be false-positives. In this case you can usually just close the incident and opt-out of the post-mortem process.

But wait! Before you do this you should consider what could have been done better to prevent the incident from happening in the first place. Clearly there was some false-alert or unclear alerting that caused the incident to be raised in the first place. It might be worth a quick post-mortem just to check that we have follow ups in place

> 💡 Remember: **The goal is not to prevent all incidents, but to learn from them and improve our systems and processes.** Every incident is an opportunity to make PostHog more reliable and our team more effective.

Public post-mortems

Some incidents require a public post-mortem. We publish these on our public post-mortems page because we believe transparency builds trust, and the wider engineering community benefits from shared lessons. A public post-mortem is needed when an incident:

- Results in permanent impact on user data (such as data loss)
- Directly disrupts customers' own services (such as SDK bugs breaking customer sites)
- Results in extended unavailability of PostHog services for customers

Process

Public post-mortems go through an internal review before being published. This isn't to hide anything – we're committed to being transparent about what happened and why. The review exists to make sure we don't accidentally expose sensitive information (such as customer data, internal credentials, or infrastructure details that could be exploited) and to ensure the post-mortem is clear and useful to readers.

1. **Write the internal post-mortem first** – Complete the normal post-mortem process above. The internal version is where you can freely discuss all details without worrying about what's safe to share publicly.
2. **Draft the public version** – Open a PR against requests-for-comments-internal using the public post-mortem template. This gives reviewers a private space to flag anything that shouldn't be public before it lands on the website.

3. **Get review** – Have the draft reviewed by relevant stakeholders. Focus on making sure the root cause, impact, and remediation are explained clearly enough to be useful, while removing anything that could compromise security or expose customer information.
4. **Publish** – Once approved, open a PR against posthog.com adding the post-mortem to `contents/handbook/company/post-mortems/` and update the list on the public post-mortems page.

Support hero

contents/handbook/engineering/operations/support-hero.md

Every week, one person in each engineering team is designated the Support Hero. If this is you this week, congratulations!

As Support Hero, your job is to investigate and resolve issues reported by customers. A single case of suspicious data or a show-stopping bug can really undermine one's confidence in a data product, so it's important that we get to the bottom of all issues.

One of the many awesome things about PostHog is that support is being dealt with by engineers and they ship fixes and improvements in real-time when you contact them. It is impossible to overstate how valuable it is for customers when they ask a question and get a shipped feature within a day.

You'll see some teams using a term of endearment for Support Hero, examples being "Infra Hero" or... "Luigi". Don't ask – we don't know.

Our Support Engineers, in the triage tickets for the , , , , , , , and teams, due to the high volume of tickets those teams get. They will resolve tickets if possible, and escalate to the engineering team responsible if they need further help.

When is my turn?

Most engineering teams run a incident.io schedule, check out the escalation schedules.

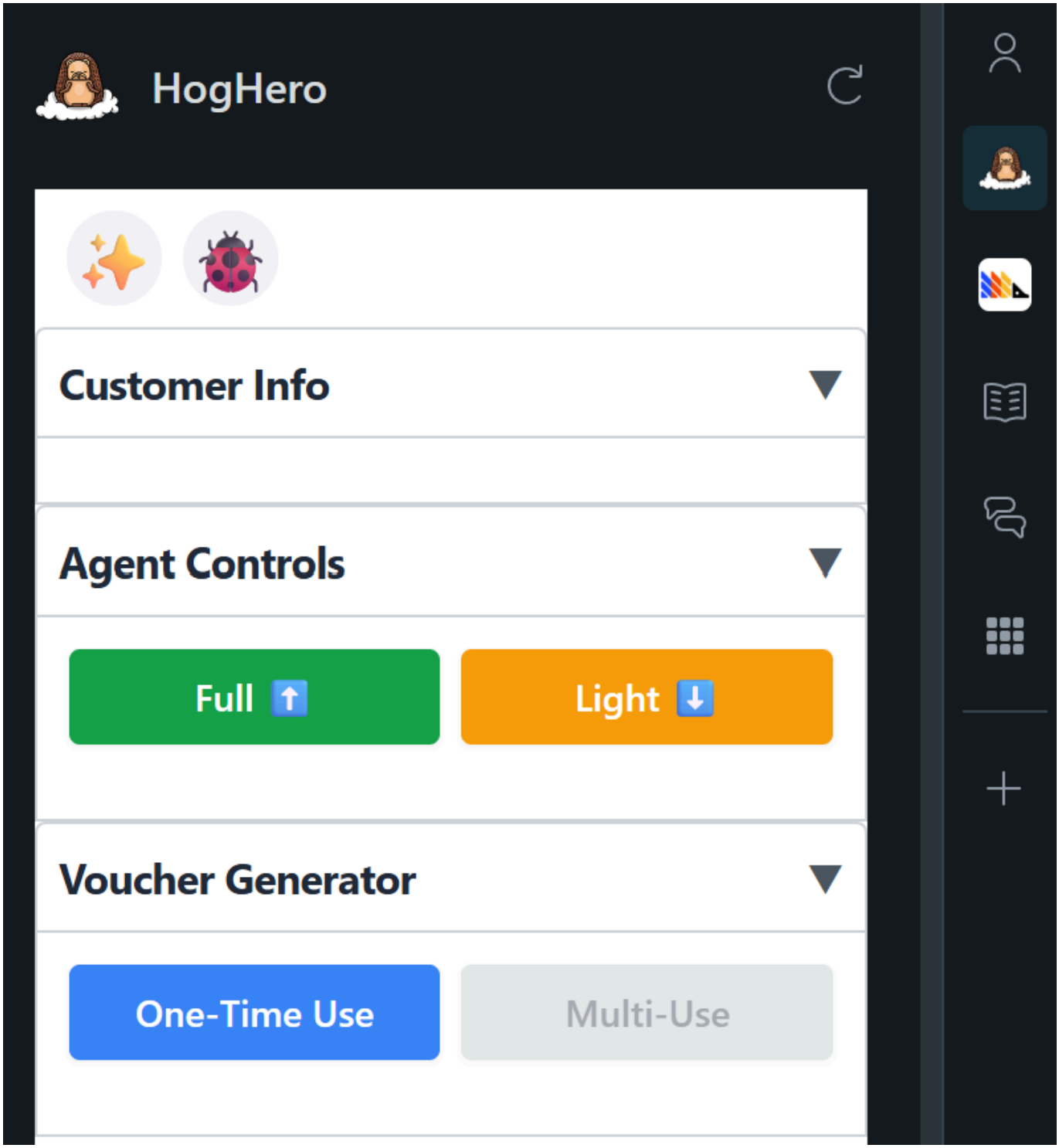
The schedules consist of contiguous blocks, but that definitely doesn't mean working 24/7 – you should just work your normal hours.

What if I'm scheduled for a week when I won't be available?

Swap with a teammate in advance! Find a volunteer by asking in Slack, then use incident.io schedule overrides. You can trade whole weeks, but also just specific days. Remember not to alter the rotation's core order, as that's an easy way to accidentally shift the schedule for everyone.

I can't assign tickets or make public replies

Everyone has access to view tickets in Zendesk however if you do not reply to tickets often you may find you currently have Light agent permissions. The HogHero app in the right sidebar should allow you to upgrade your user for your support week by clicking Full⬆️



What do I do as Support Hero?

Each engineering team has its own list of tickets in Zendesk:

- Product Analytics
- Web Analytics
- Experiments

- Feature Flags
- Replay
- Surveys
- CDP
- Infrastructure
- Auth & Billing, handled by Growth

Your job is simple: ship features and fixes, resolve ticket after ticket from your team's list, and respond to open-source PRs assigned to your team.

There are three sources of tickets:

1. In-app bug reports/feedback/support tickets sent from the Support panel (The `Help` tab in the righthand sidebar.) They include a bunch of useful links, e.g. to the admin panel and to the relevant session recording.
2. Community questions asked on PostHog.com.
3. Slack threads that have been marked with the reaction in customer support channels.

Shipping features

Some tickets ask for new features. If the feature is useful for users matching our ICP, then decide whether to just build it. Otherwise, create a feature request issue in GitHub or +1 on an existing one – you can then send a link to the user, giving them a way of tracking progress. Also make sure to let the know, since they will track feature requests for paying customers.

Sometimes a feature already exists, but a user doesn't know about it or how to use it. In this case, you should either send them a link to the relevant docs page or update the docs to make it clearer.

Fixing bugs

Others tickets report bugs or suspected bugs. Get to the bottom of each one - you never know what you'll find. If the issue decidedly affects only that one user under one-in-a-million circumstances, it might not be worth fixing. But if it's far-reaching, a proper fix is in order. And then there are "bugs" which turn out to be pure cases of confusing UX. Try to improve these too.

If not much is happening, feel free to do feature work – but in the case of a backlog in Zendesk, drop other things and roll up your sleeves. When you're Support Hero, supporting users comes first.

It might be an intense week, but you're also going to solve so many real problems, and that feels great.

Responding to external PRs

When capacity allows, the support hero serves as the first point of contact for external (open-source) PRs that affect your team's product. While we want to be good open-source citizens, **customer support always takes priority** — if you're dealing with a heavy support load, it's acceptable for PR reviews to be delayed or handled more briefly.

How external PRs are assigned

External PRs typically reach your team through one of two methods:

- **CODEOWNERS automation**: If your team has CODEOWNERS configured, PRs modifying your team's files will automatically be assigned to your team
- **Manual assignment**: For teams without CODEOWNERS set up, external PRs may be manually assigned to your team handle by other engineers who spot them

Best practices for handling external PRs

These are guidelines to aim for when you have bandwidth after handling customer support. Adapt them based on your workload:

Initial response (when possible)

- Acknowledge the PR with a thank you comment when you can
- Quick check: Are there obvious blockers like failed tests or merge conflicts? If so, politely ask the contributor to address them first
- If your support queue is overwhelming, it's okay to delay this or keep it brief

Review approach

- Be welcoming and constructive - contributors are volunteering their time
- Provide actionable feedback when you have time for a thorough review
- Consider the effort/reward tradeoff - some PRs may need more work than they're worth
- It's better to politely decline quickly than to let a PR sit without feedback for weeks

Communication tips

- Set realistic expectations based on your current workload
- Remember that external contributors can't ping us directly like teammates can
- If you know you won't get to a PR this week, a quick "Thanks for the contribution! Our team is currently focused on customer issues but we'll review this when we can" is better than silence

Common blockers to address upfront (when doing a full review)

- Ask contributors to respond to Greptile feedback before your review
- Require merge conflicts to be resolved before reviewing
- Ensure tests are passing (or understand why they're not)
- Check that the PR follows our existing code patterns and conventions

When to escalate or defer

- If the PR touches critical infrastructure or security-sensitive code
- If you're unsure about the product implications

- If your support load is too high to give it proper attention
- If a PR requires extensive back-and-forth that you don't have bandwidth for

Consider rewarding with merch

- A PR doesn't need to be merged to be reward-able
- Someone took time to care about PostHog and merch is a great way to say thank you

Managing expectations

The reality is that support hero weeks vary significantly in intensity across teams and time periods. Some weeks you might have capacity to thoroughly review several PRs; other weeks, you might barely have time to acknowledge them. That's okay. The goal is to engage with external contributions in good faith within your available bandwidth, not to maintain a perfect response rate at the expense of customer support or your well-being.

If you find yourself overwhelmed, remember:

- Customer issues come first
- A brief acknowledgment is better than nothing
- It's acceptable to hand off complex PRs to the next support hero
- Teams aren't expected to handle unlimited PRs

The key principle: We want to be responsive to our open-source community when we can, but not at the cost of our primary support responsibilities or team sustainability.

What about SDK support?

The SDK Support Hero rotation is owned by the . See the dedicated SDK support rotation page for details on how the rotation works, including how to prioritize time and handle mobile SDK issues.

Don't ask users to do work that you can do!

If folks are asking us for help, then we know the product already didn't meet their needs. Asking them to do leg-work that we could do is adding insult to injury.

For example don't ask them what version of posthog-js they're using or what their posthog config is when you can find out for yourself. Or visit their website and check the console instead of asking them if they had any errors.

If you do then have to ask them to do something, make sure you explain why you need it and what you're going to do with it.

How do I communicate?

There are two valid modes (which overlap!)

1. excited, like a labrador puppy, to discover a new way to improve the product
2. clinical and clear

Excited like a labrador puppy

The first is great for when you're talking to someone with feedback or who doesn't seem frustrated. It's important because every single support interaction is an opportunity to ship a fix or an improvement. And the excitement is how we show enough interest to properly hear the feedback.

example: "You can't do that right now, but it sounds super useful. Out of interest what does it unlock for you?"

Clinical and clear

The second is great for when the issue is tricky or the customer seems frustrated. Sometimes this goes as far as communicating in bullet points instead of paragraphs. When something isn't working the person might (quite rightly) have low tolerance for a support interaction.

example: "Ah, I see what you mean, that's not ideal! Sorry. I'll dig in to that now and let you know what I find by the end of tomorrow."

General tone

As an engineer, when answering a question, your first instinct is to give them an answer as quickly as possible. That means we often forget pleasantries, or will ignore a question until we've found the answer. So, the following guidelines:

- Always respond to a question within a reasonable timeframe during your working day. Our SLAs are explained here, but you should always try to respond to tickets quickly.
- If you're ready to look into the issue, and you think it might take a while/require a fix, just mention that and say you'll get back to them
- If you have no idea how to answer or fix their issue, @mention someone who does
- They need to know we've understood them. And have a clear picture of what their onward journey is. Are they waiting for us? How Long? Or - are we waiting for them? what for?
- Start your response with `Hey [insert name], ...` and make sure you're polite, not everyone you talk to is an engineer and as accepting of terse messages
- If they expressed frustration, acknowledging it ("Sorry for the confusion", "Apologies for the trouble" etc.) can earn goodwill quickly.
- Be sure to thank them for reporting problems, giving feedback, creating issues, PRs, etc.
- Even if you're using the support portal think about whether they'll see the message in Slack or email. A Slack message that reads like an email seems weirdly formal.
- Follow up!

- Housekeeping. Once a customer issue/question has been addressed, close the ticket in Zendesk (mark it `Solved`) to make it easy to identify outstanding conversations.
- If a user has been particularly helpful, such as raising a security or bug report, feel free to offer a small credit for the merch store.

If you have any questions about how or when to communicate with users, you can always ask the for help.

How do I prioritize?

As a business we need to ensure we are focusing support on our paying customers, as such this is the prioritization order you should apply as Support Hero. At the end of your rotation you need to ensure that any items in 1-5 are resolved or passed to the next Support Hero *as a minimum*.

1. Any requests where you are tagged by the Customer Success team in a dedicated Slack channel, as there will be some urgency needed.
2. `Open`, `escalated` Zendesk tickets for your team that have `Sales/CS Top 20`* priority.
3. `Open`, `escalated` Zendesk tickets for your team that have `High` priority.
4. `Open`, `escalated` Zendesk tickets for your team that have `Normal` priority.
5. `New` and `Open`** (non-escalated) Zendesk tickets for your team that are nearing breach or have breached SLAs
6. `Open` Zendesk tickets for your team that have low priority.

* Try to be especially responsive to any customers marked as `Sales/CS Top 20`. This set of customers is regularly reviewed by the sales team, and this priority is applied to those customers we'd like to have an especially fantastic support experience.

** Due to the way we're using Pylon, "new" tickets from high prio customer Slack channels only appear as `New` in Zendesk for a few seconds, then a webhook updates the ticket and quickly changes it to `Open`.

What if I need to confirm priority by checking a customer's MRR?

You've got a couple of options. By order of quickness:

1. Use the VIP Lookup Bot:

   In any Slack channel, type `@VIP Lookup Bot [Customer]` (without the brackets.) 'Customer' can be the organization name (case-sensitive), or their organization ID. It does work, but the results take up to 30s to load.

2. In Zendesk:

   Click the org name near the upper-left of the ticket. The left sidebar opens. There you'll see which plan they're on. If they've already paid some bills, you'll also see MRR there.

How will I know if a ticket is nearing a breach of our SLA targets?

Alerts are posted to Slack for every team which has a "group" in Zendesk. The alerts are posted to the `support-` channel for the team (or the `team-` channel for the team if the team has no `support-` channel.)

Alerts are posted for a ticket 3 hours before it breaches the next SLA. If the ticket remains untouched an hour later, another alert will be posted at 2 hours before it breaches an SLA, and again 1 hour before it breaches an SLA. The maximum number of alerts that will be posted for a single ticket is 3. (You can remove the `sla-warning` tags from a ticket if you want the alerts to be sent again for that ticket.)

How should I handle self-hosted setups?

It's fine to politely direct users to the docs for self-serve open-source support and ask them to file a GitHub issue if they believe something is broken in the docs or deployment setup. We do not otherwise provide support for self-hosted PostHog.

How should I handle organization ownership transfers?

In case a user requests for organization permissions to be altered (e.g. the only member with owner membership left the company) follow these steps:

1. The ticket should be assigned ideally to Platform features
2. Ask the user to get the current owner to log in and update ownership.
3. If the owner left and they can get access to the current owner's email, ask them do a password reset and then login as the owner and perform the action themselves.
4. If not, we should email the account owner's email to see if we get a bounce back. Also check how long it is since they logged in.
5. If accessing the current owner's email is not an option, we should have the person requesting access verify their domain ownership by providing a TXT record example for posthog verification.
6. Once verified, membership can be updated for the request.
7. Note, if they're on a paid plan we might also need to switch the contact on Stripe via a separate request to billing @posthog.com

How should I handle 2FA removal?

1. Send the following email to the account owner:

```
Subject: Confirmation Required: Removal of 2FA on your PostHog Account

Hi [name],

According to ticket #XXXX, you mentioned wanting to remove the current 2FA
method. Could you please confirm this by replying to this email?
```

```
If you haven't requested this change, please let me know immediately.

Best,
[your name]
```

2. After the user responded and confirmed the change, delete their TOTP device (EU link).

How do I use Zendesk?

We use Zendesk Support as our internal platform to manage support tickets. This ensures that we don't miss anyone, especially when their request is passed from one person to another at PostHog, or if they message us over the weekend.

Zendesk allows us to manage all our customer conversations in one place and reply through Slack or email.

Zendesk is populated with new tickets when issues are sent via the in-app Support panel (the `Help` tab in the righthand sidebar), from people outside the PostHog GitHub organization adding issues to the `posthog` and `posthog.com` repos, and new community questions. High priority customers also have Slack channels they can post support questions in. We can create Zendesk tickets from Slack questions via Pylon.

The Zendesk tickets will include links to the GitHub issue, Slack thread, or the community question so we can answer in the appropriate platform. After replying to a community question, make an `internal note` on the Zendesk ticket confirming that you've replied outside of Zendesk, and set the ticket status accordingly when submitting the internal note.

Accessing Zendesk

You can access the app via posthoghelp.zendesk.com.

The first time you sign into Zendesk, please make sure you include your name and profile picture so our users know who they are chatting with!

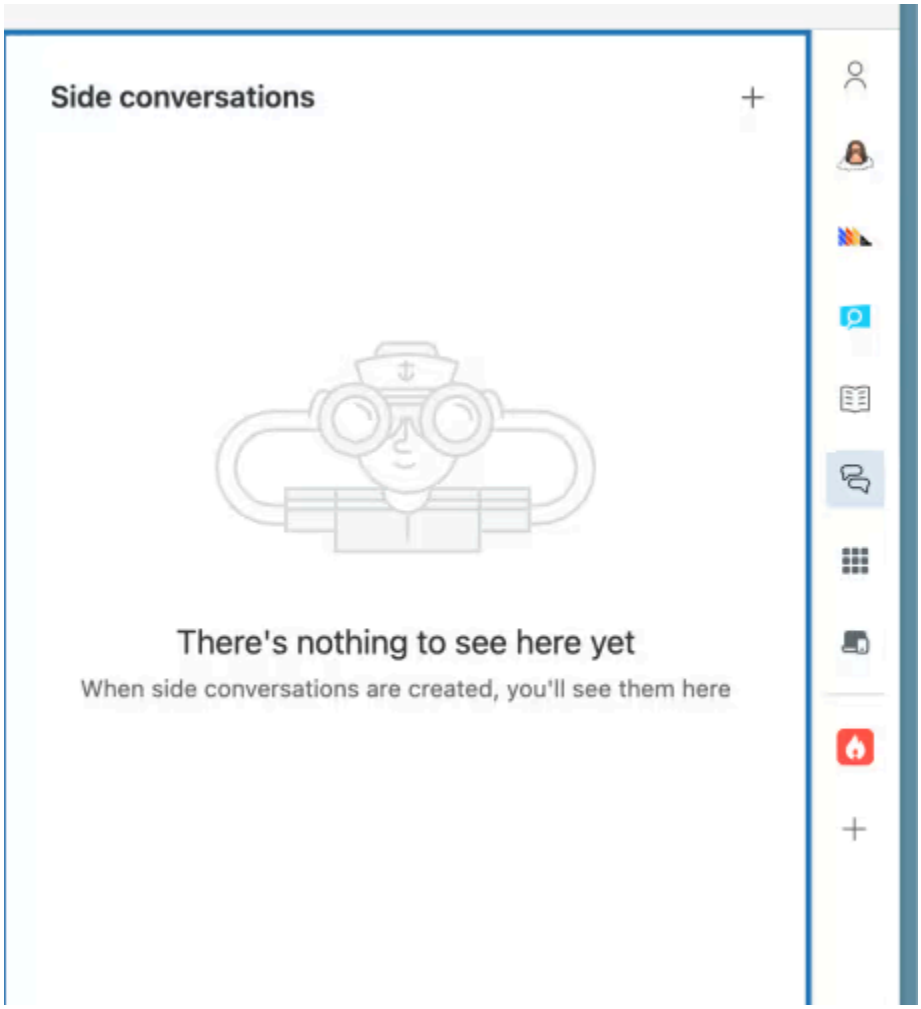
Using Zendesk

You'll spend most of your time in the Views panel, where you'll find all tickets divided into different lists depending on who they are assigned to, and whether they have been solved or not.

Tips:

- Err on the side of Solving tickets (see below) if you expect no further input from the customer, as a lot of them don't reply to confirm that the problem has been solved.

- Provide actionable information as an *Internal Note* on the Zendesk ticket (e.g. partial investigation, notes for escalating or transferring to a different team, etc.). Do not use internal notes to communicate internally about a ticket, it is far too easy to miss these notes (see below!).
- Use *side conversations* from Zendesk to Slack to communicate about a ticket internally. Slack is our primary communication tool and therefore it's much easier to have a discussion in Slack than through Zendesk internal notes. Side conversations can be started from the right hand side panel in Zendesk:



Creating tickets on behalf of users or from existing tickets

Sometimes users will contact us over Twitter, or email, asking support questions. Sometimes they will respond to old, solved ticket threads with new problems, or tickets will spiral into multiple issues. In both situations it's best to create a new ticket for the user so we can apply the correct SLAs and keep issues distinct for easy assignment.

You can ask a user to create a new ticket themselves, but it's best if we do it for them. The easiest way to do this correctly is to login to PostHog as the user, and then create a fresh ticket on their behalf using the information you have. This will ensure the correct tags, SLAs, and so on are automatically applied.

If the user raised the issue in a public forum, such as Twitter, it can be a good idea to tell them you've opened a ticket on their behalf. If the user was replying to an old, already solved ticket, you should mark the old issue to `Closed`.

Avoiding duplication of effort in Zendesk

Each team handles Zendesk queues (views) in slightly different ways. Check in with your team about whether or not to assign tickets to yourself, or keep them assigned to the team/group level. Support team folks, who work on tickets from multiple queues, often assign tickets to themselves, (and when escalating, will assign the ticket back to the team/group.)

For unassigned tickets, keep an eye out for whether someone else is already viewing a ticket (will appear in the upper-left of a ticket you're viewing, with their name, avatar and `also viewing`.) Use those as clues to avoid working on a ticket that someone is already working on (and communicate with each other when in doubt. Err on the side of making sure the ticket gets responded to within SLA/response target times.)

Also, avoid cherry-picking tickets. Pick the ticket that is closest to breaching our response targets.

Ticket Status

When responding to a ticket you should also choose an appropriate status according to the following:

- **New** - A newly created ticket, you shouldn't need to use this when responding (Note: Some tickets, such as tickets created via Slack, are changed from `New` to `Open` by automated internal notes added just after the ticket is created.)
- **Open** - The ticket is still awaiting a response/further investigation from someone in PostHog (if you've worked on the ticket and expect someone else to work on it next, make sure the other person/team knows about it by leaving an internal note on the ticket.)
- **On-Hold** - (*pauses the SLA timer*) Use this one sparingly, GitHub is better for tracking open bugs, feature requests, and technical debt, and `On-Hold` tickets are too easily overlooked. If you do need to put a ticket `On-Hold`, reply to the ticket to let the customer know. (If you've opened a bug ticket or feature request, `On-Hold` isn't needed, see `Solved` below.)
- **Pending** - (*pauses the SLA timer*) Use this for most replies to customers. Even if you think the issue is solved, the user may disagree, so `Solved` may not spark joy. When a user doesn't reply to a `Pending` ticket within 7 days, the ticket is auto-solved.
- **Solved** - (*stops the SLA timer*) The user has replied to confirm that the ticket is resolved, or you've created a bug report or feature request and shared the link with the user so they can follow it for updates.

Temp orgs for free email users

To reduce some unintended consequences of Zendesk's unavoidable use of email address domain names to associate users with organizations, we have Zendesk orgs for common free email providers.

An example of these orgs: `Gmail user – please assign to correct org`

When we get a ticket from a user with an `@gmail.com` address who has not already been manually assigned to an existing Zendesk org, that user will be assigned to the `Gmail user – ...` org (unless their PostHog org doesn't exist in Zendesk yet, in which case the correct org will be created in Zendesk.)

When you see a user assigned to a free email org on a ticket, and it is not a 'community question' ticket, please assign the user to their correct org, which is found on the `Admin` info line in the body of the ticket:

1. Click on the user's name, to the right of the org name
2. Click in the `Org.` field to change the org name
3. Click anywhere outside the field to save the change Tickets which have been set to **Pending** will auto-solve after 7 days. Customers can also respond within 20 days to a **Solved** ticket to re-open it. After 20 days, responses will create a follow-up ticket with a link to the original ticket.

Tickets that have been `Solved` will receive a CSAT survey the next day.

Content Warnings

We have a clear definition of who we do business with which means that customers who track adult or other potentially offensive content aren't automatically excluded. To avoid team members inadvertently seeing anything offensive when impersonating a customer we will automatically tag tickets from Organizations known to have this type of content with a `content_warning` tag.

This looks at the Content Warning field on the Zendesk Organization, and adds the tag if there is text in that field. If you see this tag on a ticket and want to understand more then click on the Organization name in the top left corner of the Zendesk UI and scroll down the list of fields on the left.

If you do discover any potentially offensive content in a customer account then please update this field on the Zendesk Organization so that other team members are aware of the content.

Pylon to create Zendesk tickets from Slack posts

We use Pylon to create Zendesk tickets from Slack posts. To do so, add the `:ticket:` ( ) emoji reaction to the post that you want to create a Zendesk ticket from.

Adding the `:ticket:` emoji reaction will cause Pylon to add a couple of replies in a thread under the post. The last of those replies includes options for the Zendesk ticket you're creating: Use the `Group` menu to send the ticket to the appropriate team, and the `Severity` menu to set the severity flag on the Zendesk ticket, then hit the `Submit` button.

Zendesk tickets created this way will normally be marked as high priority tickets. You can respond to them either in Zendesk or Slack, as there is a two-way sync.

Adding new teams to Zendesk.

When we've added a new team, or 🔨 split an existing team into two or more, we'll need to get them set up in Zendesk. Here's an overview of the steps:

- Create a new group in Zendesk
- Add team members to the group
- Add triggers to the `Routing for internal teams` category (Tip: Clone an existing trigger, rename it, and tweak it)
- Add views (Tip: Clone an existing view, rename it, and tweak it.)
- Add Slack notification triggers (Tip: Clone an existing trigger, yada, yada)
- Add SLA breach alerts
- Create a webhook endpoint in slack
    - Create a Slack app
    - Enable incoming webhooks
    - Create a webhook to the channel, copy the url
- Create a webhook in zendesk (Tip: Refer to existing webhooks for common settings)
    - Choose "Trigger or automation"
    - Paste the endpoint url you copied from the Slack app (Note: The built-in tool for testing webhooks in ZD has been flakey while the UI has been changing lately. Failed tests don't always mean the hook won't work. 😐)
- Create an automation in zendesk (Tip: Clone an existing automation, blah, blah, blah)
- If you've split a team, sort the tickets to the new groups as needed, then disable the triggers, automations, and views related to the old team.
- Carry on

Community questions

At the end of every page in the docs and handbook is a form where visitors can ask questions about the content of that page. (These questions also appear in the relevant category in the PostHog community.)

Community questions appear in Zendesk and tickets are closed automatically if an answer is picked as a solution on the website. Ideally, the original poster is the one who marks a response as the solution. If they don't, feel free to close the ticket in Zendesk once you've replied.

How do I answer community questions?

When a question is posted, it'll appear in Zendesk with a direct link to the question. A notification is also sent to the #community-questions channel in Slack. (You can also receive notifications about specific topics in your own small team's Slack channel. Ask the Website & Docs team for help in setting this up if you like.)

You can answer a question directly on the page where it was asked. When a reply is posted, the person who asked the question will receive an email notification. (**Important:** Don't reply to community questions directly from Zendesk.)

The first time you answer a question, you'll need to create a community account. (You'll be prompted to do this after answering a question, as posting/responding requires authentication.)

Ask in #team-website-and-docs to be upgraded to a moderator. This will also give you access to moderator controls available for each question.

Note: The PostHog.com community currently uses a separate authentication system from PostHog Cloud. There are plans to support other types of authentication so a visitor doesn't have to create a separate account for asking questions.

How do I handle a bug report or feature request?

For feature requests from low priority users, give them this link and suggest they open a feature request.

For bug reports from normal and high priority users (assuming you've confirmed it's a bug, and that there's not already an open bug report):

1. Open a bug report on our GitHub repo
2. Be sure to include a link to the insight (or other), below the repo steps
3. Include "From: `https://URL_for_Zendesk_ticket`" in the `additional info` section of the bug comment (where the URL is for the Zendesk ticket where the customer reported the bug)
4. Reply to the user to thank* them for alerting us to the bug. Let them know you've opened a bug report and provide a link to it.
5. Let them know they can follow the bug report on GitHub for updates.
6. When sending the reply, change the ticket from `Open` to `Pending`
7. In Slack, go to the team channel for the team that handles the feature that the bug report applies to (e.g. `#team-product-analytics`) and alert them with a post like "New bug report from a high priority customer: `https://github.com/PostHog/posthog/issues/nnnnnn`"

∗ consider sparking additional joy with a credit for merch

Steps for feature requests from normal and high priority users are pretty much the same, but use this form instead. If you find that there's already a matching feature request open, reply with a link to the feature request, and let them know they can upvote it by adding a "`+1`" comment.

How do I handle user requests to delete groups/organizations?

> ***WARNING:*** Do NOT click the `DELETE` button! That will delete the entire project! Just use the `Save` button after clicking the `delete` checkbox for the group.

1. Visit the Django Admin page for the project at `https://us.posthog.com/admin/posthog/team/:project_id/change/` (Make sure you use the project ID for the project where the group/org is found)
2. In the lower part of the page, find `GROUP TYPE MAPPINGS` and click on `SHOW`
3. In the righthand column, check the box for the group(s) to be deleted
4. Click the **`SAVE`** button. (**Do NOT use the `DELETE` button!**)
5. Reply to the user to confirm

Editing the API docs

contents/handbook/engineering/posthog-com/api-docs.md

The PostHog API docs are generated using drf-spectacular. It looks at the Django models and djangorestframework serializers.

> **Note:** We don't automatically add new API endpoints to the sidebar. You need to add those to src/navs/index.js

You can add a `help_text="Field that does x"` attribute to any Model or Serializer field to help users understand what a specific field is used for:

```
class Insight(models.Model):
    last_refresh: models.DateTimeField = models.DateTimeField(blank=True,
        null=True, help_text="When the cache for the result of this insight was
        last refreshed.")

class InsightSerializer(TaggedItemSerializerMixin, InsightBasicSerializer):
    filters_hash = serializers.CharField(
        read_only=True,
        help_text="A hash of the filters that generate this insight.",
    )
```

To add a description to the top of a page, add a comment to the **viewset** class:

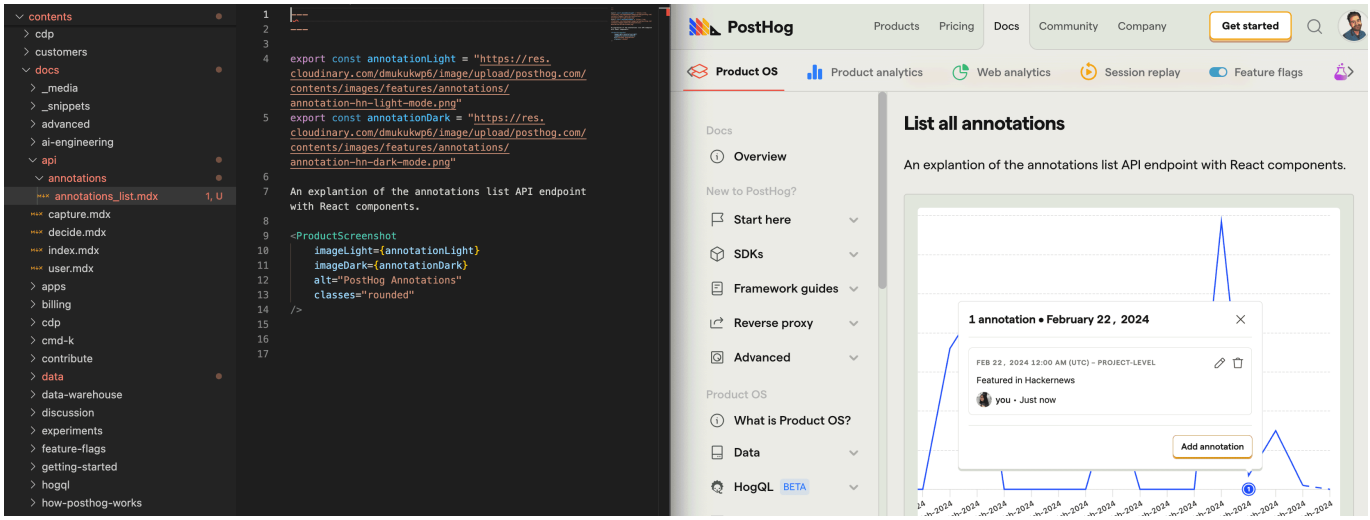
```
class InsightViewSet(TaggedItemViewSetMixin, StructuredViewSetMixin,
        viewsets.ModelViewSet):
    """
    Stores saved insights along with their entire configuration options. Saved
        insights can be stored as standalone
    reports or part of a dashboard.
    """
```

To check what any changes will roughly look like locally, you can go to http://127.0.0.1:8000/api/schema/redoc/.

To add a description to a specific endpoint, add an MDX file (named after the endpoint ID's name) to the corresponding folder its page would belong to. Then, the content in the MDX file will only appear under the specified endpoint. This is like our MDX setup, except the file name will determine which endpoint the MDX contents appear on.

For example, to add a description to the "list annotations" endpoint, you'd create a new file: `contents/docs/api/annotations/annotations_list.mdx`

Whatever you add to that file will appear under that endpoint only.



Insights serializer

The serializer for insight lives here. Each time an insight gets created we check it against these serializers, and we'll send an error to Sentry (but not the user) if it doesn't match, to ensure the API docs are up to date.

Documenting custom endpoints

If you have an `@action` endpoint or a custom endpoint (that doesn't use DRF) you can still document by providing a serializer for the request and response.

```
from drf_spectacular.utils import OpenApiResponse
from posthog.api.documentation import extend_schema
@extend_schema(
    request=FunnelSerializer,
    responses=OpenApiResponse(
        response=FunnelStepsResultsSerializer,
        description="Note, if funnel_viz_type is set the response will be
        different.",
     ),
    methods=["POST"],
    tags=["funnel"],
    operation_id="Funnels",
)
```

```
@action(methods=["GET", "POST"], detail=False)
def funnel(self, request: request.Request, *args: Any, **kwargs: Any) ->
        Response:
```

Testing API docs locally

To test or develop the API docs locally, you need to create a personal API key (see top of this page) and then export it before running gatsby, in the same terminal window:

```
export POSTHOG_PERSONAL_API_KEY=yourkey
```

Developing the website

`contents/handbook/engineering/posthog-com/developing-the-website.md`

You can contribute to the PostHog documentation, handbook, and blog in two ways:

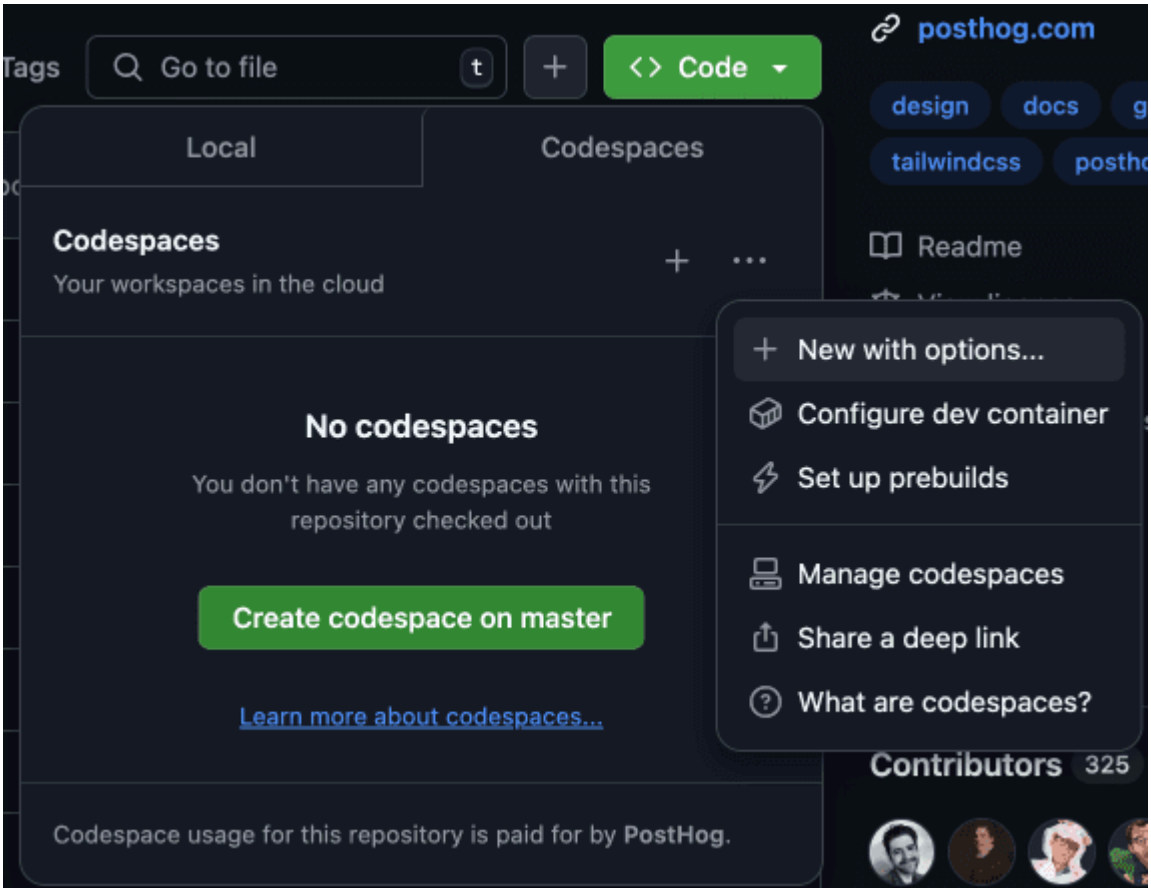
1. Create a pull request in GitHub for any page that has an **Edit this page** link on it. In this situation you must edit the page using the GitHub web editor interface. This method is suitable for text-only edits and basic file manipulation, such as renaming.
2. Run the posthog.com website in development and make changes there by creating a branch of the master codebase, committing changes to that branch and raising a pull request to merge those changes. This is the recommended method as it allows you to quickly preview your changes, as well as perform more complex changes easily.

Below, we'll explain the two options for option two.

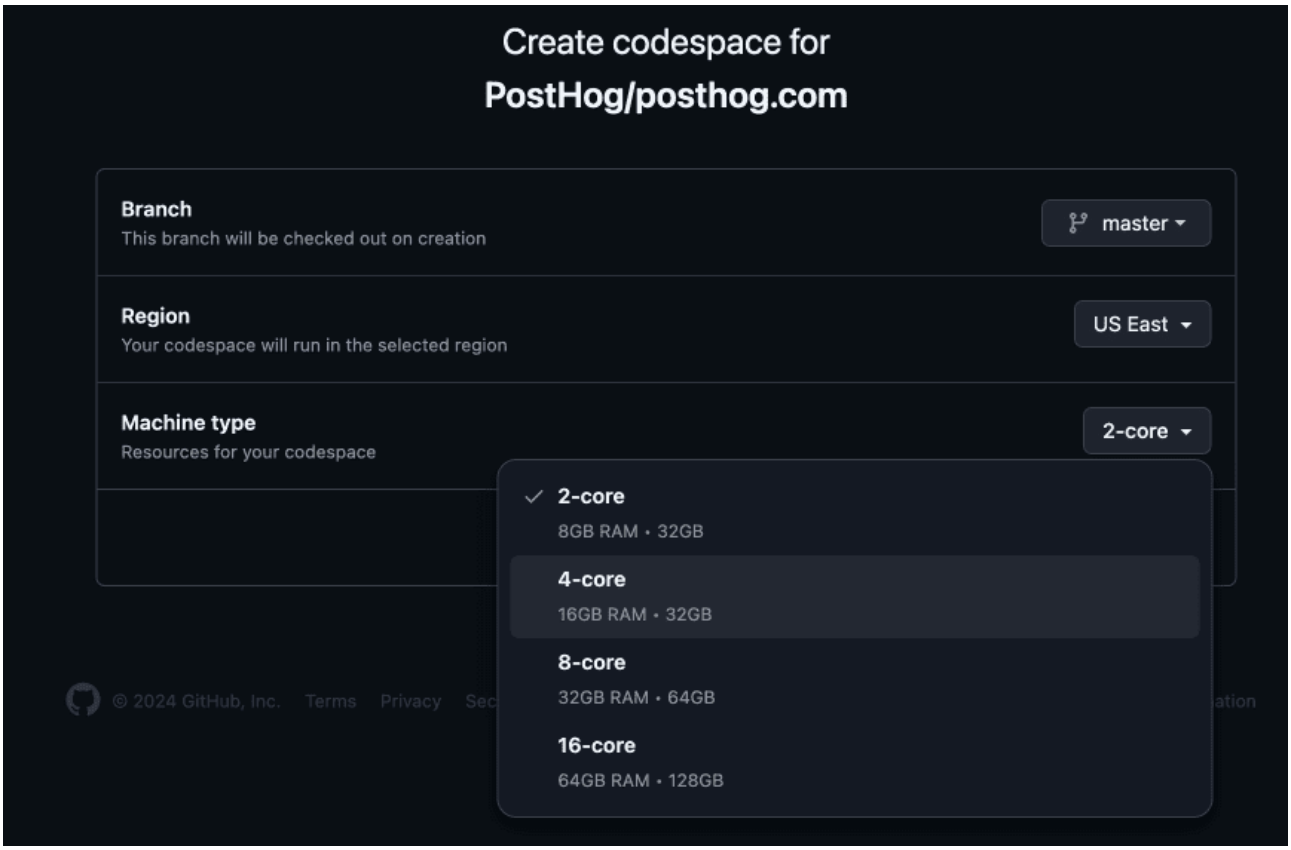
Option 1: Running with Codespaces

Creating/running the Codespace

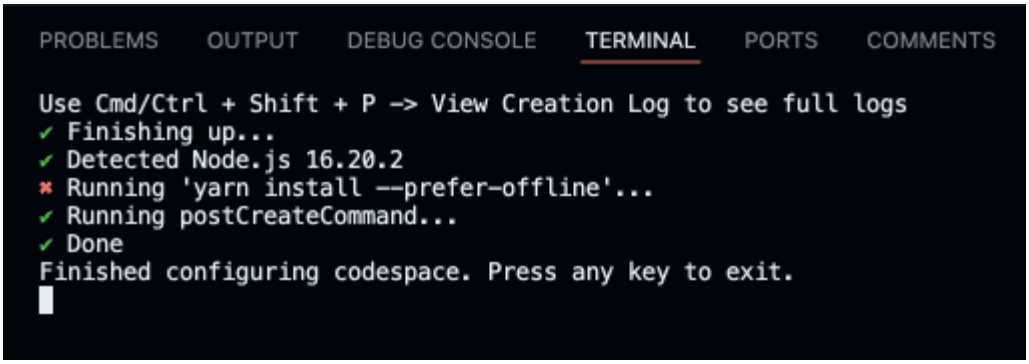
1. Open the posthog.com repository in GitHub.
2. Click the **Code** button, then the **Codespaces** tab, then under the **...** menu, choose **New with options...**



3. Under **Machine type**, choose **4-core**.

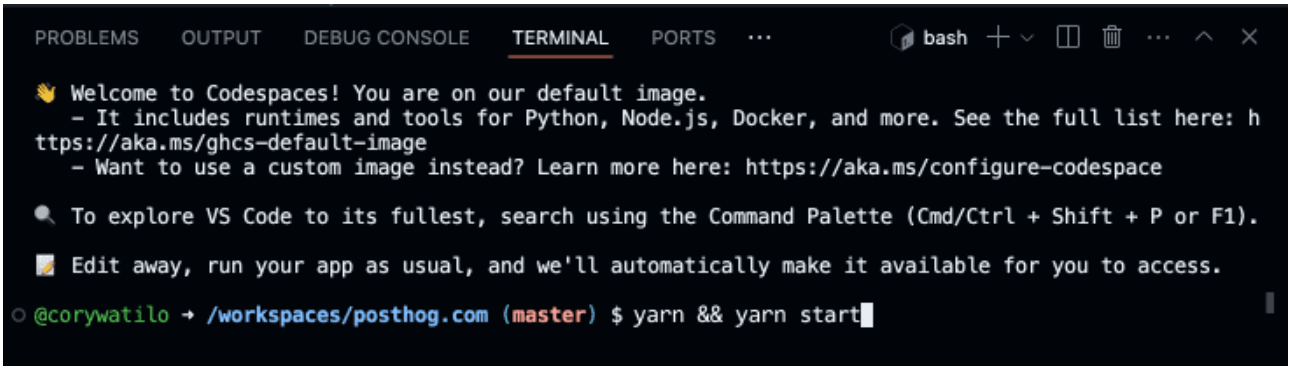


4. When the repo opens in Codespaces, it will install some things automatically.



When completed, press any key.

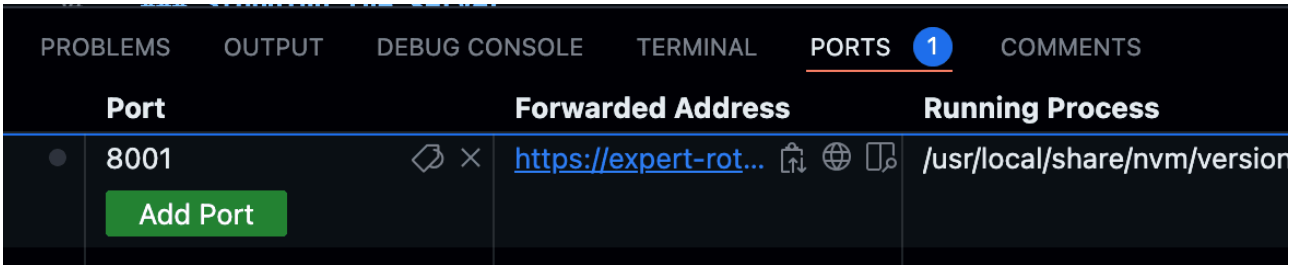
5. In terminal, type `pnpm install && pnpm start` and hit [Enter].



   - This will take a while. The last step of the process is "Building development bundle" which will take a few minutes on its own.
   - You may see a dialog that says, "Your application running on port 8001 is available." Don't be enticed by the big green button quite yet.

6. Once you see `success Writing page-data.json files...`, you can click the green **Open in browser** button which will open the site at http://localhost:8001.

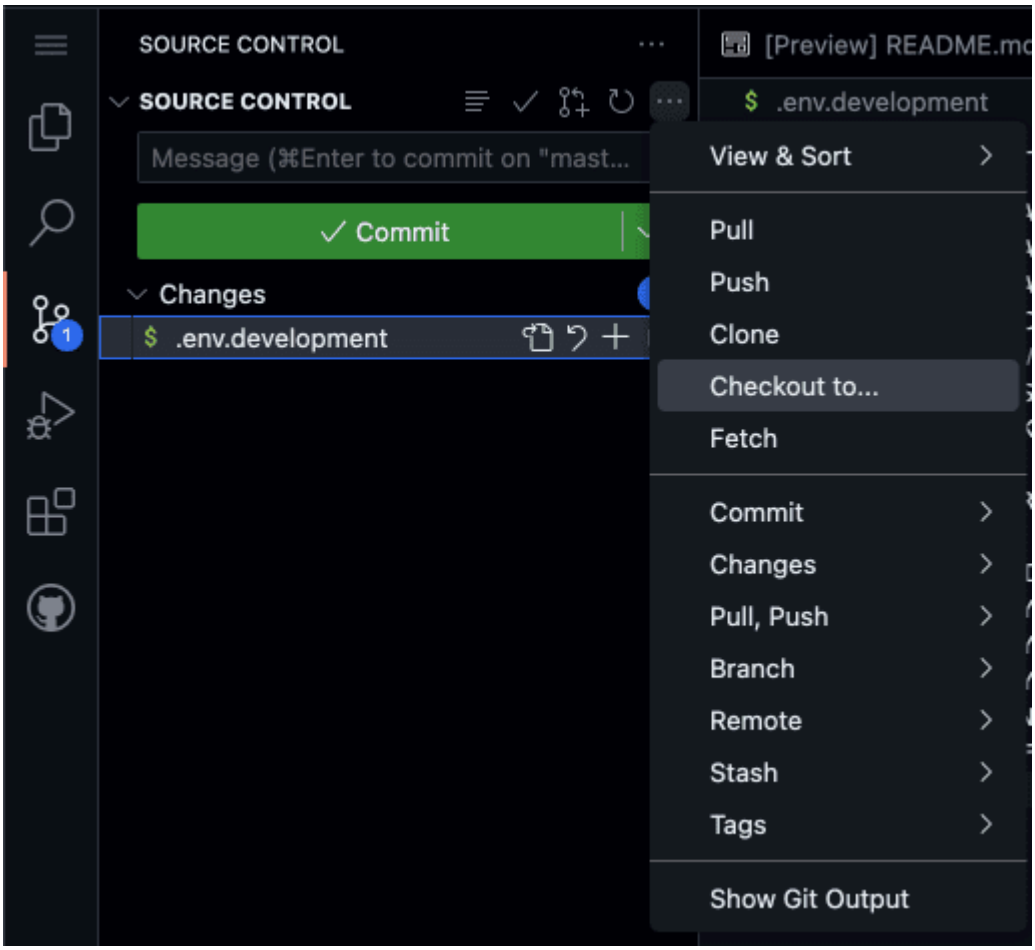
   You can also click the **Ports** tab to access the URL where you can preview the site. Cmd + click the URL seen here.



Committing and pushing changes

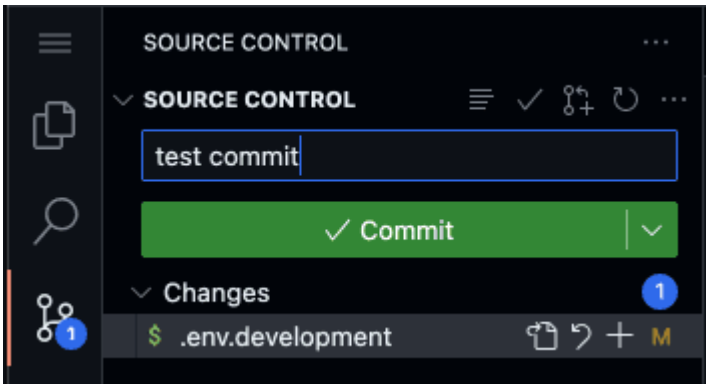
Use the built-in Git tab in VS Code to commit and push your changes.

1. From the Git source control **...** menu, choose **Checkout to...** to create a new branch.

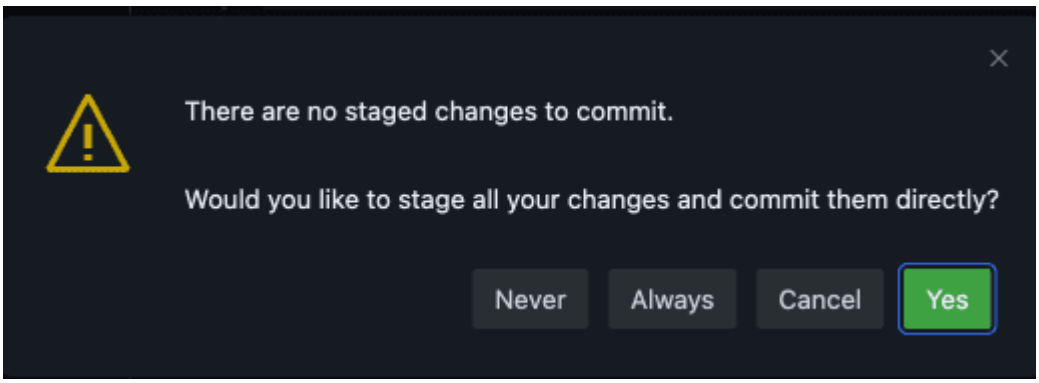


2. Type a new branch name and press enter.

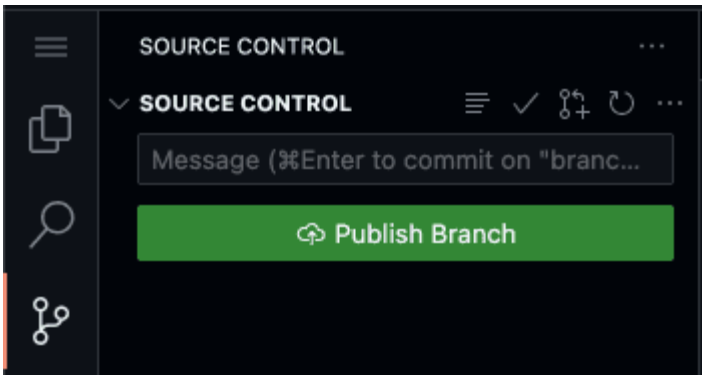
3. Now you can commit changes to your new branch. Type a commit message and use `Cmd` + `Enter` (or push the big green button).



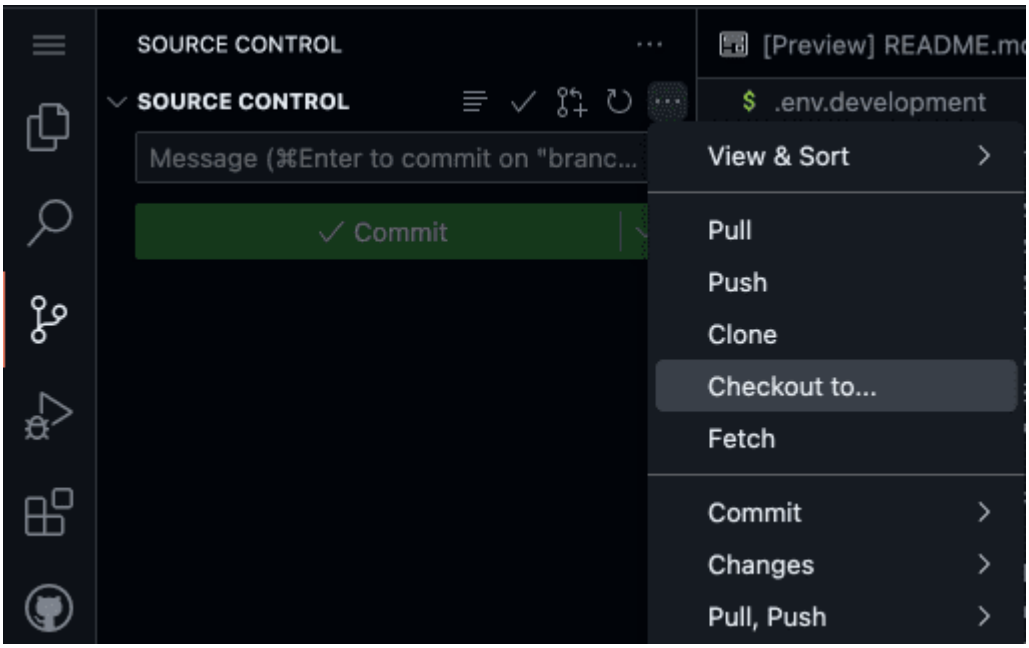
4. If you see the dialog below, choose **Always** to always stage all files you've changed. (Otherwise, you'll need to hit the + button next to each file you want to commit.)



5. Now that your changes are committed, it's time to publish them to GitHub.



> **Note:** After finish changes on your branch, be sure to switch back to `master` so you don't inadvertently make future changes to your current branch.



>

Stopping the server

1. Place your cursor into Terminal and type `Cmd+C` to stop the server.
2. In the bottom left corner of the window, click **Codspaces: [your codespace name]**, then **Stop current codespace.**

Notes

If you plan on using this codespace frequently, disable **Auto-delete codespace** in the **...** menu under the **Code > Codespaces** dropdown in the repo.

Option 2: Editing posthog.com locally

Before you begin

In order to run the PostHog website locally, you need the following installed:

- Git – version control system
- Node.js (version 22.x) – server runtime
- pnpm (version 10.x) – package manager for Node.js

- Apple Rosetta (version 2) – dynamic binary translator for Apple silicon

If you are unfamiliar with using Git from the command line (or just prefer graphical interfaces), use the GitHub Desktop app.

You may also want to familiarize yourself with these resources:

- GitHub's glossary of terms
- GitHub Desktop docs
- Visual Studio docs

Cloning the posthog.com repository

The posthog.com codebase is on GitHub at https://github.com/PostHog/posthog.com. To work on it locally, first you need to clone it to your disk:

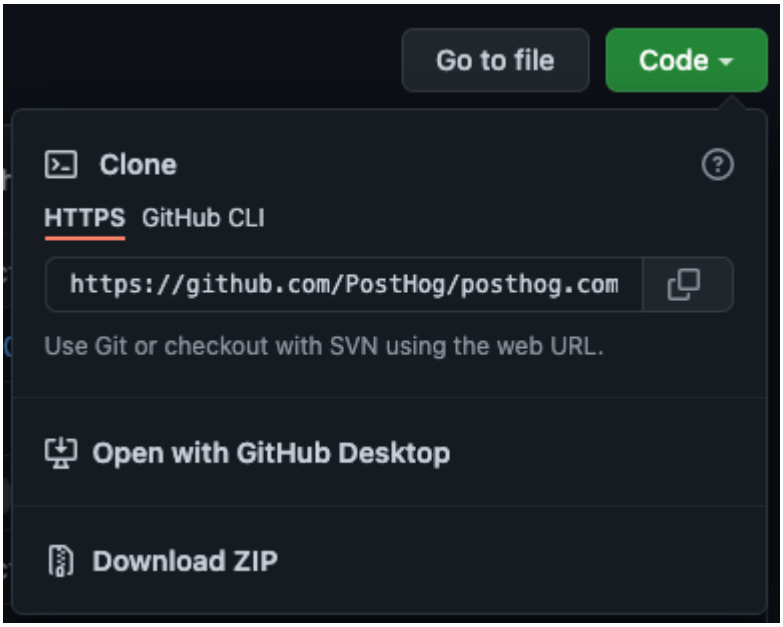
- **via the command line**

  You can clone the codebase from the command line using the following command:

  ```
  git clone git@github.com:PostHog/posthog.com.git
  ```

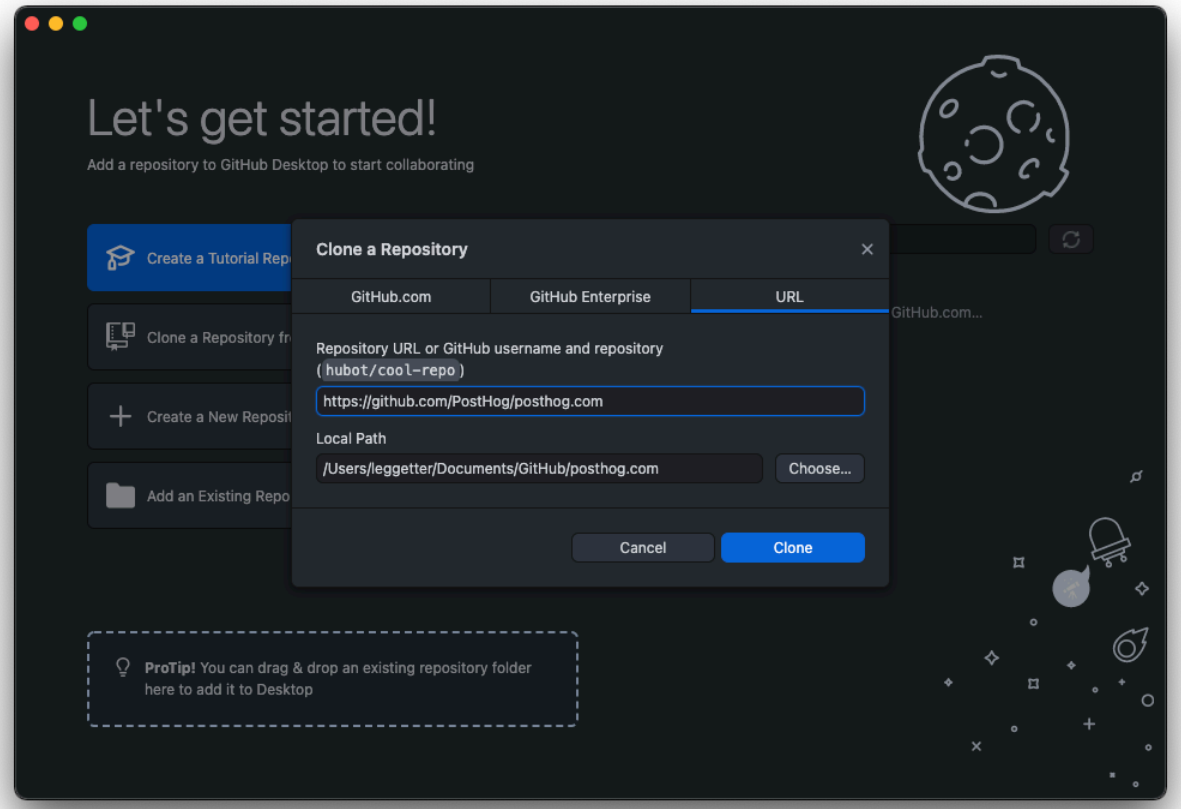
- **via GitHub Desktop**

  You can also clone the repository with GitHub Desktop installed, from the posthog.com repository page, click the **Code** button and select **Open with GitHub Desktop** from the dropdown that appears.

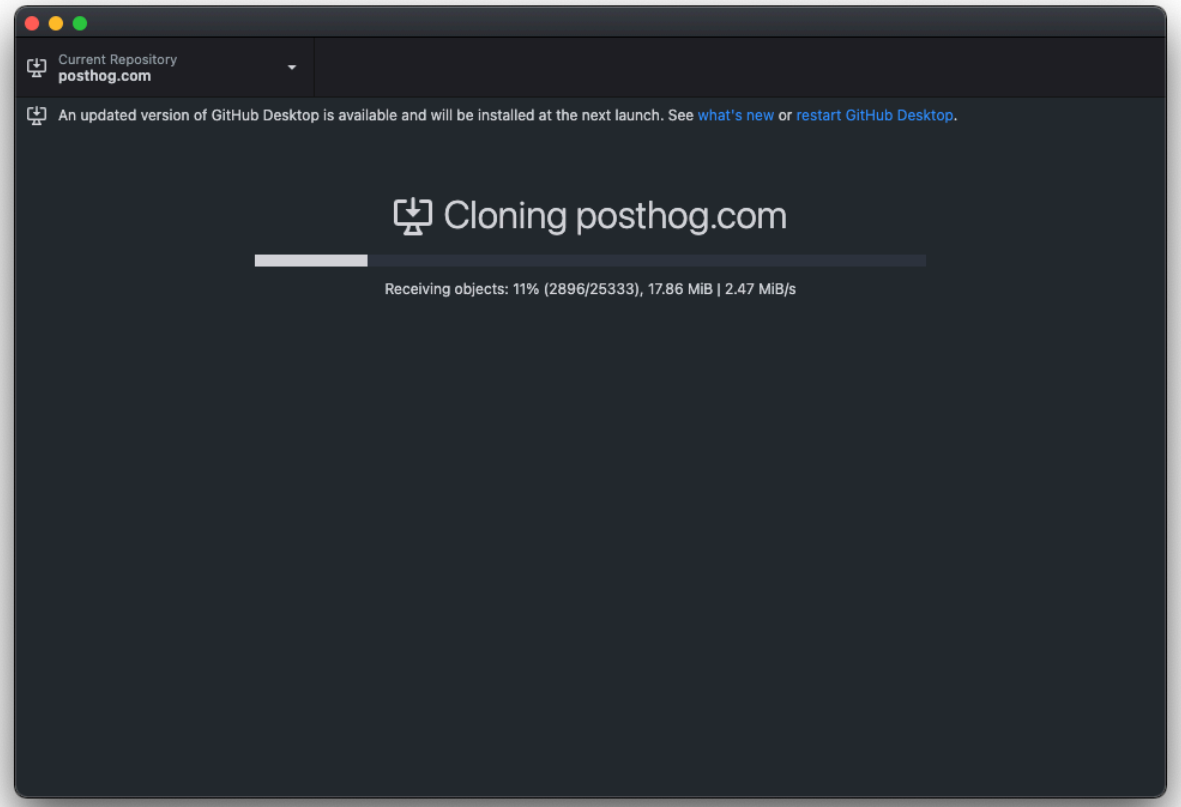
  

  You will then be prompted by the browser to confirm if you want to open the GitHub Desktop application. Select the affirmative action that has text such as **Open GitHub Desktop**.

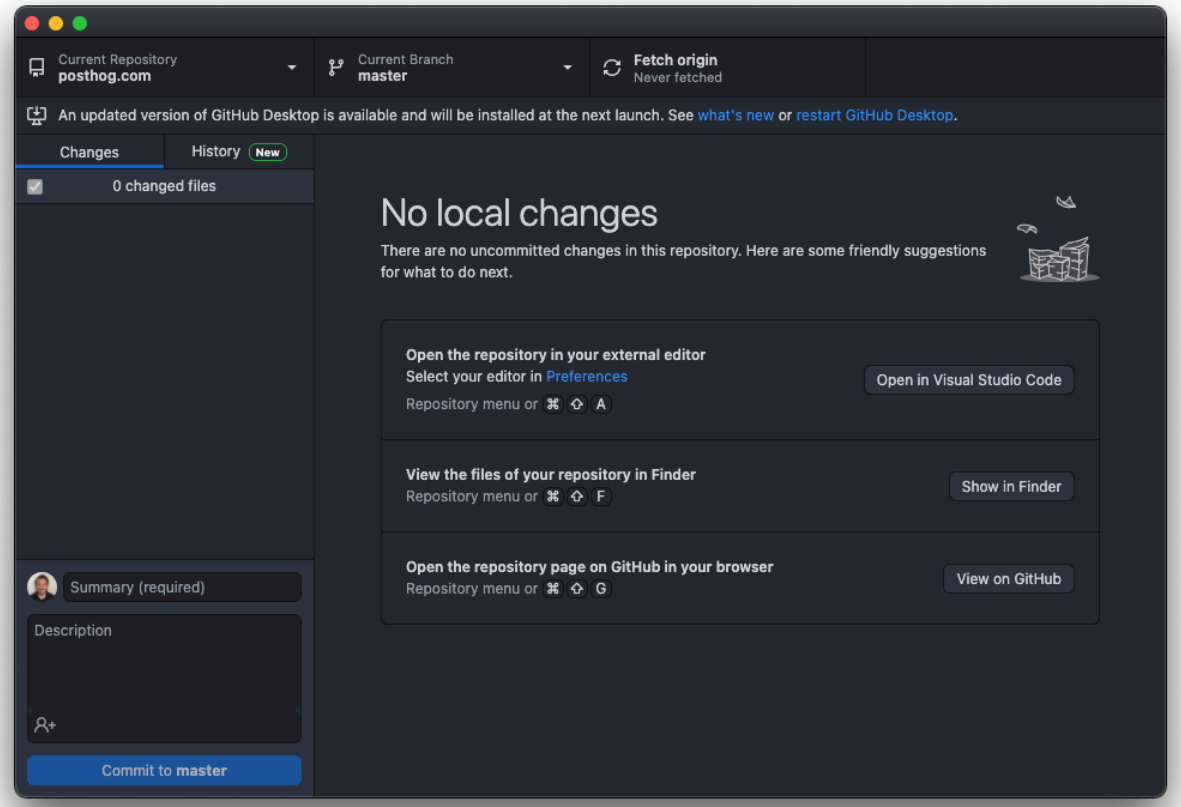
Once GitHub Desktop has opened you will be prompted to confirm the repository that is being cloned and the location on disk where you wish the code to be stored.



Click **Clone** to clone the posthog.com repository to your local disk.

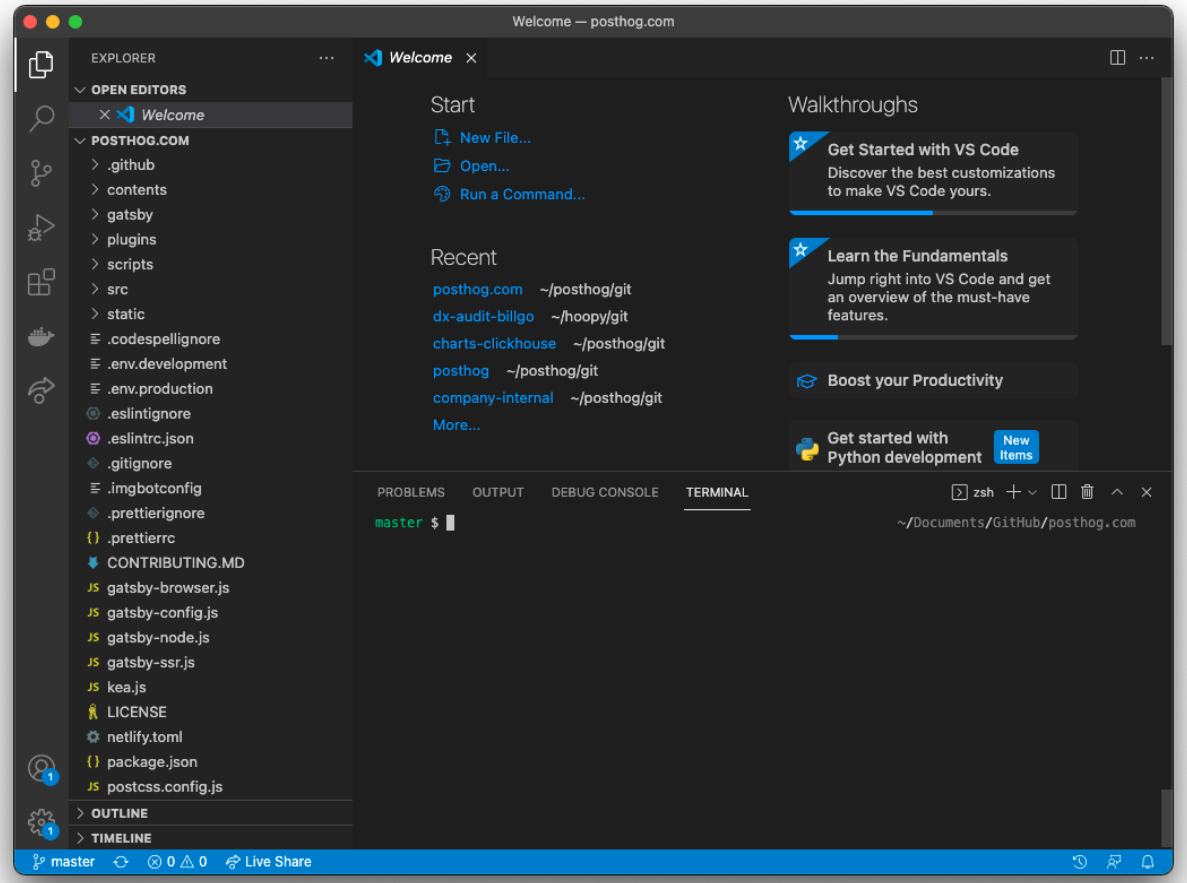


Once the clone has completed the GitHub Desktop interface will change to the following:



To view the code for the website click **Open in Visual Studio Code**. Dialogs may appear around permissions and trust as you open Visual Studio Code.

Once you have Visual Studio Code open, select the **Terminal** menu option. Within the dropdown select **New Terminal**. This will open a new terminal window within Visual Studio Code:



Don't worry! We only need to run a few commands in the command line.

Running posthog.com locally

If you're using an Apple Silicon Mac (M1+) then you'll need to run the following commands before using pnpm:

```
rm -rf ./node_modules
brew install vips
```

Type the following into the command line and press return:

```
pnpm install
```

This installs the dependency packages used by posthog.com. This may take a few minutes.

After initial setup, use the following command to start the development server:

```
pnpm install && pnpm start
```

This runs the local clone of the website, which you can use to preview changes you make before pushing them live. It takes a bit of time for some file processing and compilation to take place, but once it's completed you can access the locally running version of posthog.com via by visiting `http://localhost:8001` in your web browser.

Any time you want to preview changes you are making to the local version of the website, all you have to do is run the `pnpm start` again, wait for the command to finish running and then open `http://localhost:8001` in your web browser.

Troubleshooting

If the server fails to start, the first troubleshooting step is to clear cache. You can do this (and start the server again) by running:

```
pnpm clean && mkdir .cache && pnpm install && pnpm start
```

Minimal mode

For faster builds, you can run in minimal mode:

```
pnpm build:minimal
```

Minimal mode only builds:

- Docs pages (`/docs/*`)
- Handbook pages (`/handbook/*`)
- Blog/content posts (`/blog/*`, `/tutorials/*`, `/library/*`, `/founders/*`, `/product-engineers/*`, `/newsletter/*`, `/spotlight/*`, `/customers/*`)
- All pages in `src/pages/` (product pages, pricing, etc.)

Everything else (apps, CDP, templates, jobs, API docs, SDK references, pagination/category/tag pages) won't exist - they'll 404. Next/previous navigation links and GitHub data for roadmaps/jobs will also be absent. Sourcemap generation is disabled.

PR preview deployments (Cloudflare Pages)

Pull request previews on Cloudflare Pages use the same minimal build as above: the workflow sets `GATSBY_MINIMAL=true` (see `.github/workflows/deploy-preview.yml`). That keeps preview builds fast.

Implications for content authors:

- **Post category indexes are not built** — Routes like `/tutorials`, `/blog`, and `/posts` are not generated in previews. Opening them can fail or show a broken page (for example a blank or error screen in the site shell). This is expected.
- **Individual posts and docs still build** — Preview the change by opening the **direct URL** to the MDX page (e.g. `/tutorials/your-post-slug`, `/blog/your-post-slug`).
- **Search and listing data** — Post listings and site search rely on full production builds and indexing (e.g. Algolia). Content from a branch typically will not appear in search on the preview until it is merged to `master` and the production site is built.

Environment variables

Our website uses various APIs to pull in data from sites like GitHub (for contributors) and Ashby (our applicant tracking system). Without setting these environment variables, you may see various errors when building the site. Most of these errors are dismissible, and you can continue to edit the website.

If you need a specific environment development, ask in #posthogdotcom.

Finding the content to edit

Once you have cloned the repo, the `contents/` directory contains a few key areas:

- `docs/` = all of the documentation for PostHog's platform
- `handbook/` = the PostHog company handbook
- `blog/` = our blog posts

Inside each of these are a series of markdown files for you to edit.

Posts and blog filtering

There are two ways to filter posts by tag:

1. **Query param** — Add a `post_tags` query param to the URL, e.g., `/posts?post_tags=Comparisons`. This works on the main posts listing and allows saving/sharing filtered URLs.

2. **Static tag pages** — For SEO purposes, we generate static pages at `/{category}/{tag}`, e.g., `/blog/session-replay`. These are generated at build time in `gatsby/createPages.ts`.

Hidden from index

Some categories and tags are intentionally hidden from the main posts index view. They still appear when you filter directly to that category or tag.

Categories hidden from index: `customers`, `spotlight`, `changelog`, `comparisons`, `notes`, `repost`

Tags hidden from index: `Comparisons`

Posts can also set `hideFromIndex: true` in their frontmatter to be excluded.

These exclusions are defined in `src/components/Edition/Posts.tsx` and `src/templates/BlogPost.tsx`.

Making edits

Creating a new Git branch

When editing locally, changes should be made on a new Git branch. Branches should be given an "at a glance" informative name. For example, `posthog-website-contribution`.

- **via the command line** You can create a new Git branch from the command line by running:

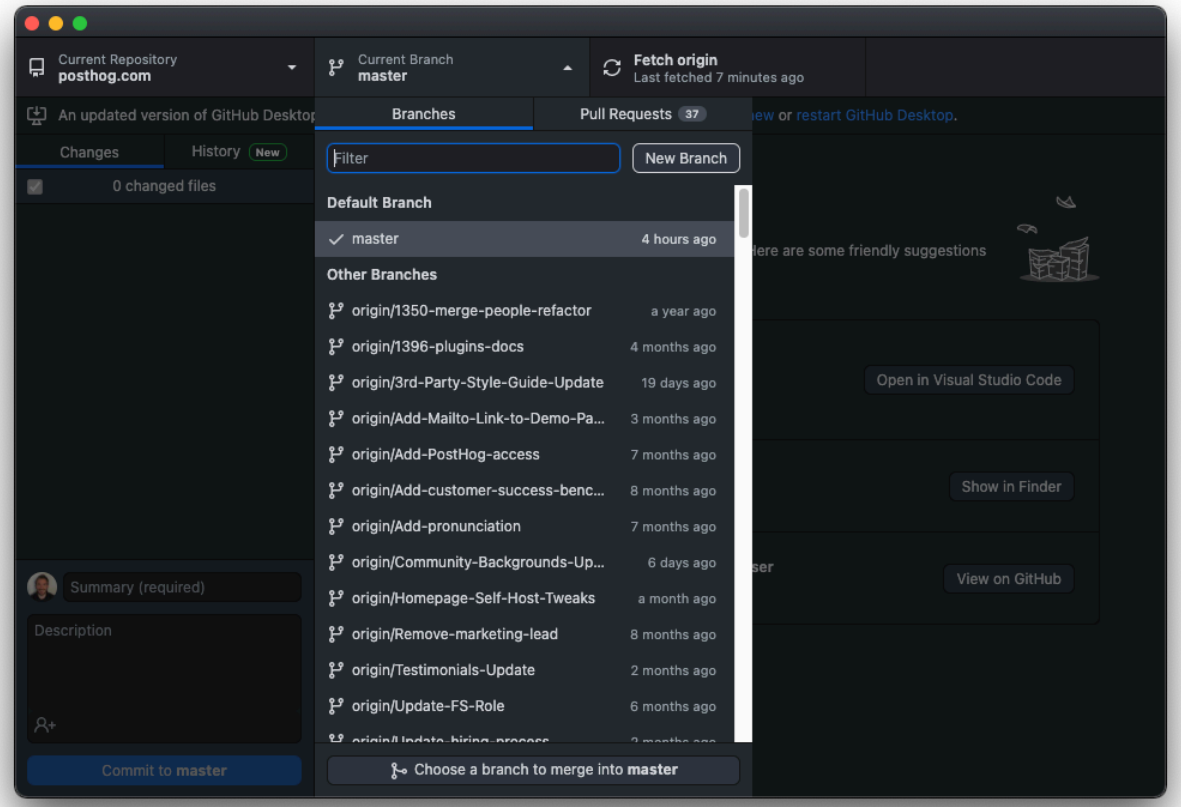
  ```
  git checkout -b [new-branch-name]
  ```

  For example:

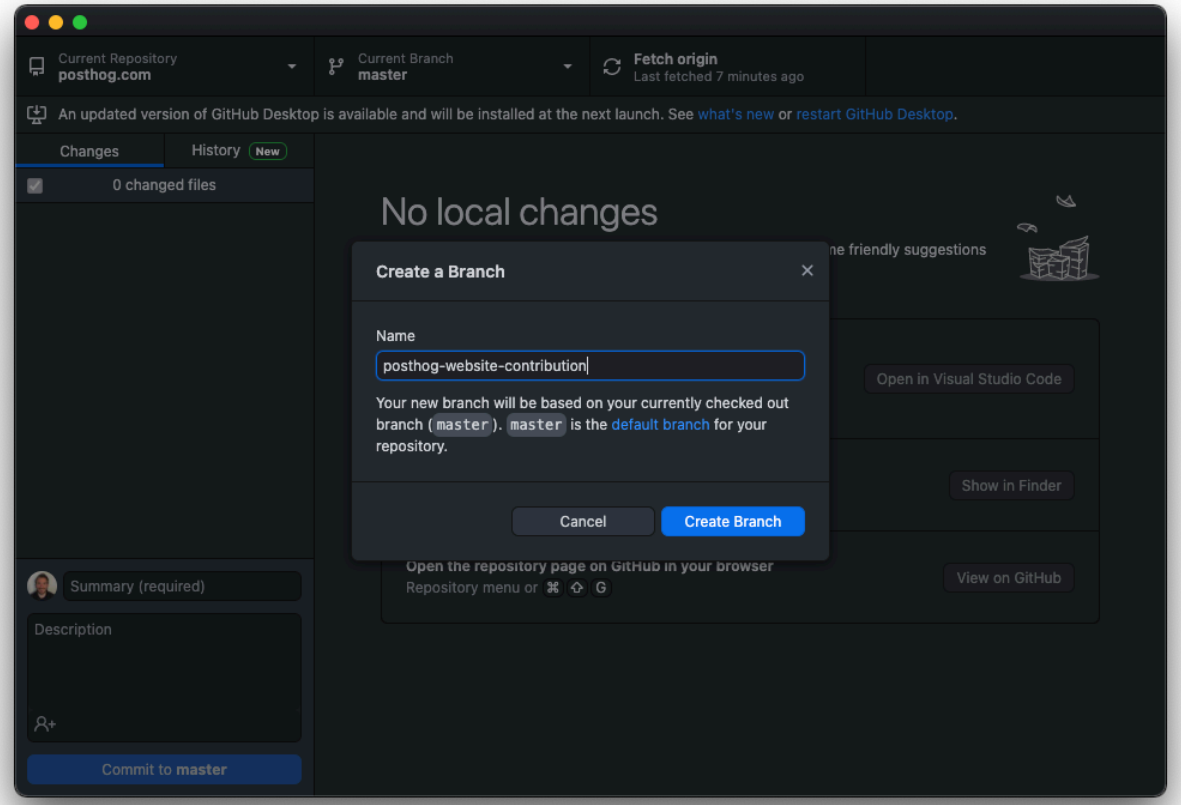
  ```
  git checkout -b posthog-website-contribution
  ```

- **via GitHub Desktop**

  You can also create a new branch in GitHub Desktop by selecting the dropdown next to the **Current Branch** name and clicking **New Branch**.



Then, in the dialog that follows, entering the new branch name.



Once you have a new branch, you can make changes.

Markdown details

Frontmatter

Most PostHog pages utilize frontmatter as a way of providing additional data to the page. Available frontmatter varies based on the template the page uses. Templates are determined based on the folder the file resides in:

Blog

Markdown files located in `/contents/blog``

```
---
date: 2021-11-16
title: The state of plugins on PostHog
rootPage: /blog
author: ["yakko-majuri"]
featuredVideo: https://www.youtube-nocookie.com/embed/TCyCryTiTbQ
```

```
featuredImage:
        https://res.cloudinary.com/dmukukwp6/image/upload/posthog.com/contents/i
        mages/blog/running-content.png
featuredImageType: full
category: Guides
tags: ["Using PostHog", "Privacy"]
seo: {
    metaTitle: Overview of PostHog Plugins
    metaDescription: Learn about the current state of plugins on PostHog and get
        valuable insights into their functionality and performance.
}
---
```

- `date`: the date the blog was posted
- `title`: the title that appears at the top of the blog post and on the blog listing page
- `rootPage`: necessary for listing all blog posts on /blog. should always be set to `/blog`
- `author`: the author(s) of the post. correlates to your handle located in /src/data/authors.json
- `featuredVideo`: the iframe src of the video that appears at the top of the post. replaces the featured image on post pages.
- `featuredImage`: the Cloudinary URL of the image that appears at the top of the post and on the blog listing page
- `featuredImageType`: `standard` | `full` - determines the width of the featured image on the blog post
- `category`: the broader category the post belongs to. one of the following:
  -
- `tags`: the more specific tag(s) the post belongs to. an array containing any number of the following:
  -
- `seo`: object containing SEO metadata:
  - `metaTitle`: String
  - `metaDescription`: String

Tutorials

Markdown files located in /contents/tutorials

```
---
date: 2022-02-14
title: How to filter out internal users
author: ['joe-martin']
featuredTutorial: false
featuredVideo: https://www.youtube-nocookie.com/embed/2bptTniYPGc
tags: ['filters', 'settings']
---
```

- `date`: the date the tutorial was posted
- `title`: the title that appears at the top of the tutorial and on the tutorial listing page
- `author`: the author(s) of the tutorial. Ccrrelates to your handle located in /src/data/authors.json
- `featuredTutorial`: determines if tutorial should be featured on the homepage
- `featuredVideo`: the iframe src of the video that appears at the top of the tutorial
- `featuredImage`: the Cloudinary URL of the image that appears at the top of the tutorial and on the tutorial listing page
- `tags`: the tag(s) the tutorial belongs to. an array containing any number of the following:
    -
- `seo`: object containing SEO metadata:
    - `metaTitle`: String
    - `metaDescription`: String

Docs & Handbook

Markdown files located in /contents/docs and /contents/handbook

```
---
title: Contribute to the website: documentation, handbook, and blog
---
```

- `title`: the title that appears at the top of the handbook / doc page
- `seo`: object containing SEO metadata:
    - `metaTitle`: String
    - `metaDescription`: String

Comparison pages

Create a table on a "PostHog vs..." page with the following components. (You can see examples of how this is used in this pull request.)

Import the components at the top of the post content (after frontmatter):

```
import { ComparisonTable } from 'components/ComparisonTable'
import { ComparisonRow } from 'components/ComparisonTable/row'
import { ComparisonHeader } from 'components/ComparisonTable/header'
```

Create a table like:

```
<ComparisonTable column1="Company name 1" column2="Company name 2">
  <ComparisonHeader category="Optional header row" />
  <ComparisonRow column1={true} column2="Text" feature="Feature name"
```

```
description="Feature description" />
</ComparisonTable>
```

In `ComparisonRow`:

- Values for `column1` and `column2` can be: `{true}` | `{false}` | `"Text string"`
- `feature` is required but `description` can be omitted (only if not using that column for the entire table)

Customers

Markdown files located in `/contents/customers`

```
---
title: How Hasura improved conversion rates by 10–20% with PostHog
customer: Hasura
logo:
        https://res.cloudinary.com/dmukukwp6/image/upload/posthog.com/contents/i
        mages/customers/hasura/logo.svg
featuredImage:
        https://res.cloudinary.com/dmukukwp6/image/upload/posthog.com/contents/i
        mages/customers/hasura/featured.jpg
industries:
    - Developer tool
users:
    - Engineering
    - UI
    - UX
    - Marketing teams
toolsUsed:
    - Funnel Analysis
    - Session Recording
    - Self-Hosting
---
```

- `title`: the title of the case study
- `customer`: the name of the customer
- `logo`: the customer logo
- `featuredImage`: the Cloudinary URL of the image that appears in the card on the customers listing page
- `industries`: a list of industries that apply to the company
- `users`: a list of user types that use the company's product
- `toolsUsed`: a list of highlighted PostHog tools used by the company

- `seo`: object containing SEO metadata:
  - `metaTitle`: String
  - `metaDescription`: String

Plain

If the file doesn't reside in one of the above folders, it uses the plain template.

```
---
title: Example Components
showTitle: false
width: lg
noindex: true
---
```

- `title`: the title that appears at the top of the page
- `showTitle`: `false` - if omitted, the title will appear at the top of the page
- `width`: `sm` | `md` | `lg` | `full` - determines the width of the page
- `noindex`: `true` | `false` - determines whether to index the page or not
- `seo`: object containing SEO metadata:
  - `metaTitle`: String
  - `metaDescription`: String

You can often refer to the source of existing pages for more examples, but if in doubt, you can always ask for help.

Adding rich media

Add images or videos to your post by uploading them to Cloudinary and including the URL in your Markdown file. Be sure to follow our best practices when adding media.

Links to/from the navigation

If you've created a new markdown file (for use in docs or handbook), you should link to it from the sidebar where appropriate.

The sidebar is generated from `src/navs/index.js`.

Redirects

Redirects are managed in `vercel.json` which is located in the root folder.

To declare a new redirect, open `vercel.json` and add an entry to the `redirects` list:

```
{ "source": "/docs/contributing/stack", "destination": "/docs/contribute/stack"
}
```

The default HTTP status code is 308 (permanent), but if the redirect should be temporary (307), it can be updated like this:

```
{ "source": "/docs/contributing/stack", "destination": "/docs/contribute/stack",
"permanent": false }
```

Committing changes

It's best to create commits that are focused on one specific area. For example, create one commit for textual changes and another for functional ones. Another example is creating a commit for changes to a section of the handbook and a different commit for updates to the documentation. This helps the pull request review process and also means specific commits can be cherry picked.

- **via the command line**

  First, stage your changes:

  ```
  git add [path-to-file]
  ```

  For example:

  ```
  git add contents/docs/contribute/updating-documentation.md
  ```

  Once all the files that have been changed are staged, you can perform the commit:

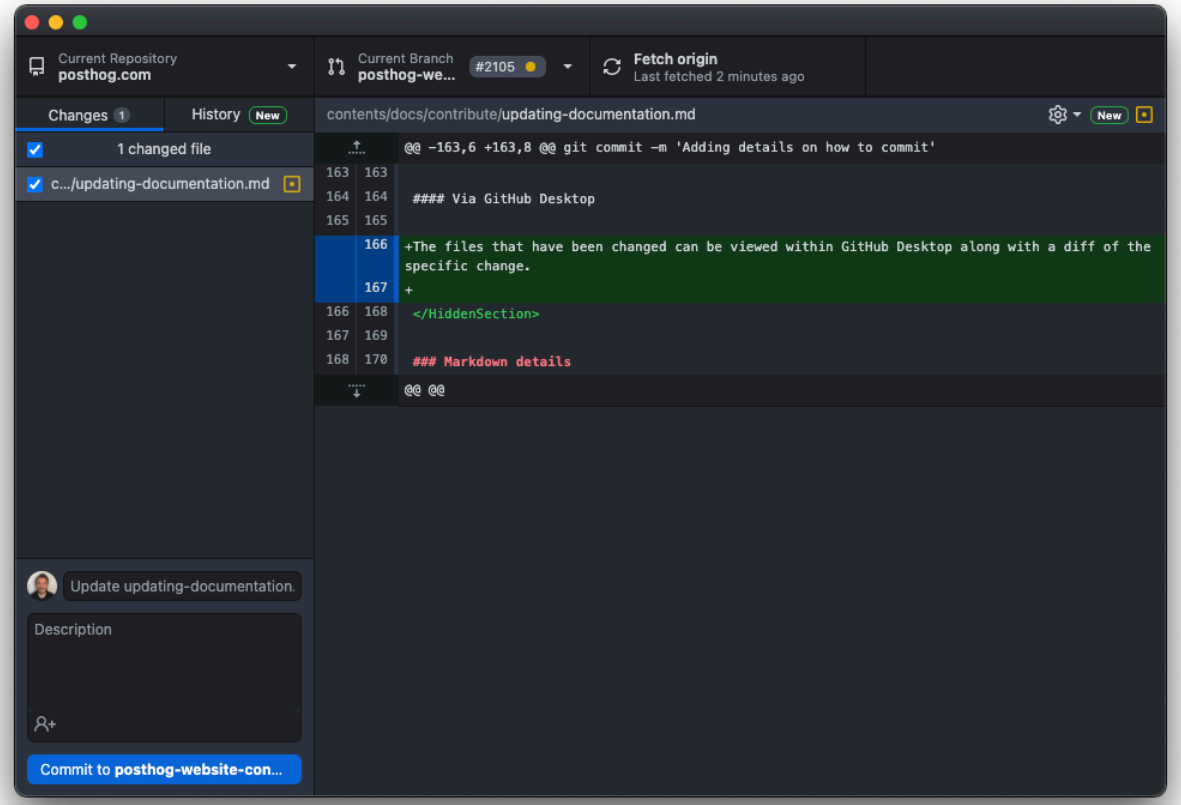
  ```
  git commit -m '[short commit message]'
  ```

  For example:

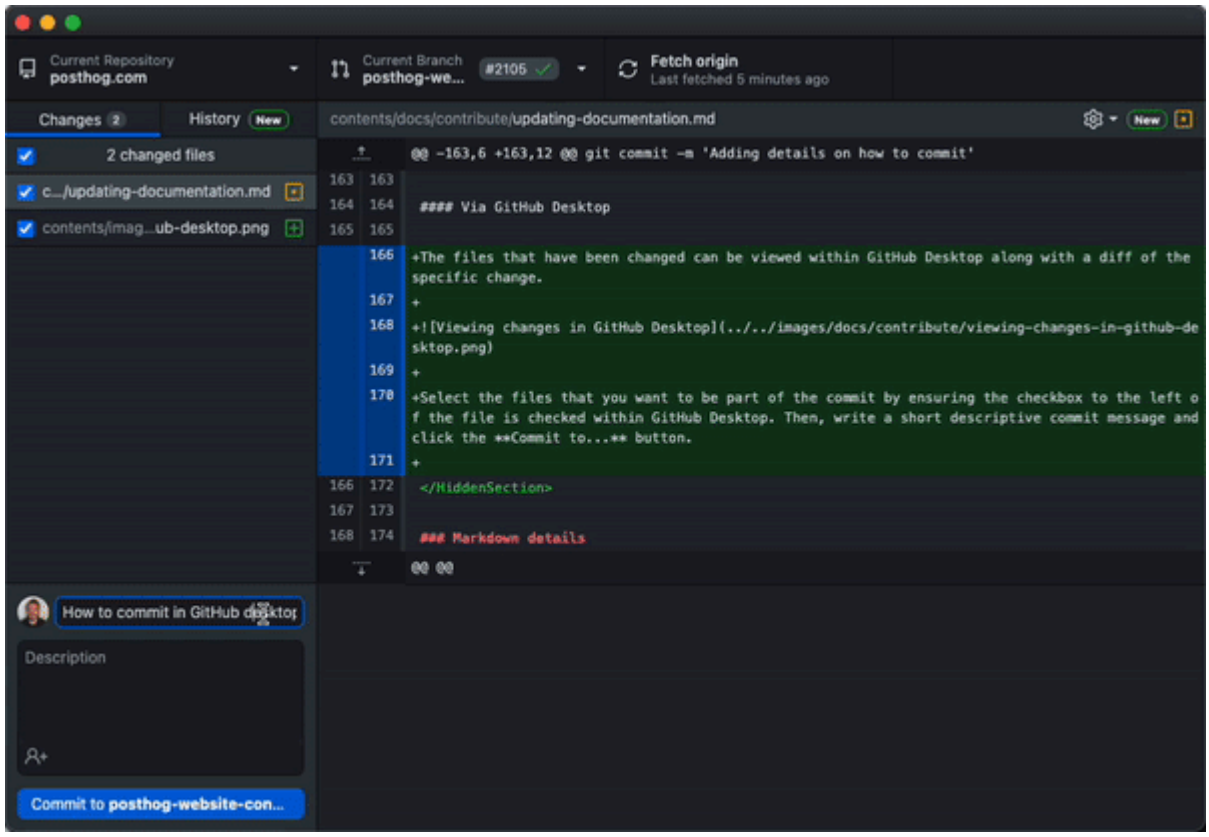
  ```
  git commit -m 'Adding details on how to commit'
  ```

- **via GitHub Desktop**

  Files that have been changed can be viewed within GitHub Desktop along with a diff of the specific change.



Select the files that you want to be part of the commit by ensuring the checkbox to the left of the file is checked within GitHub Desktop. Then, write a short descriptive commit message and click the **Commit to...** button.



Push changes to GitHub

In order to request that the changes you have made are merged into the main website branch you must first push them to GitHub.

- **via the command line**

  ```
  git push origin [branch-name]
  ```

  For example:

  ```
  git push origin posthog-website-contribution
  ```

  When this is done, the command line will show output similar to the following:

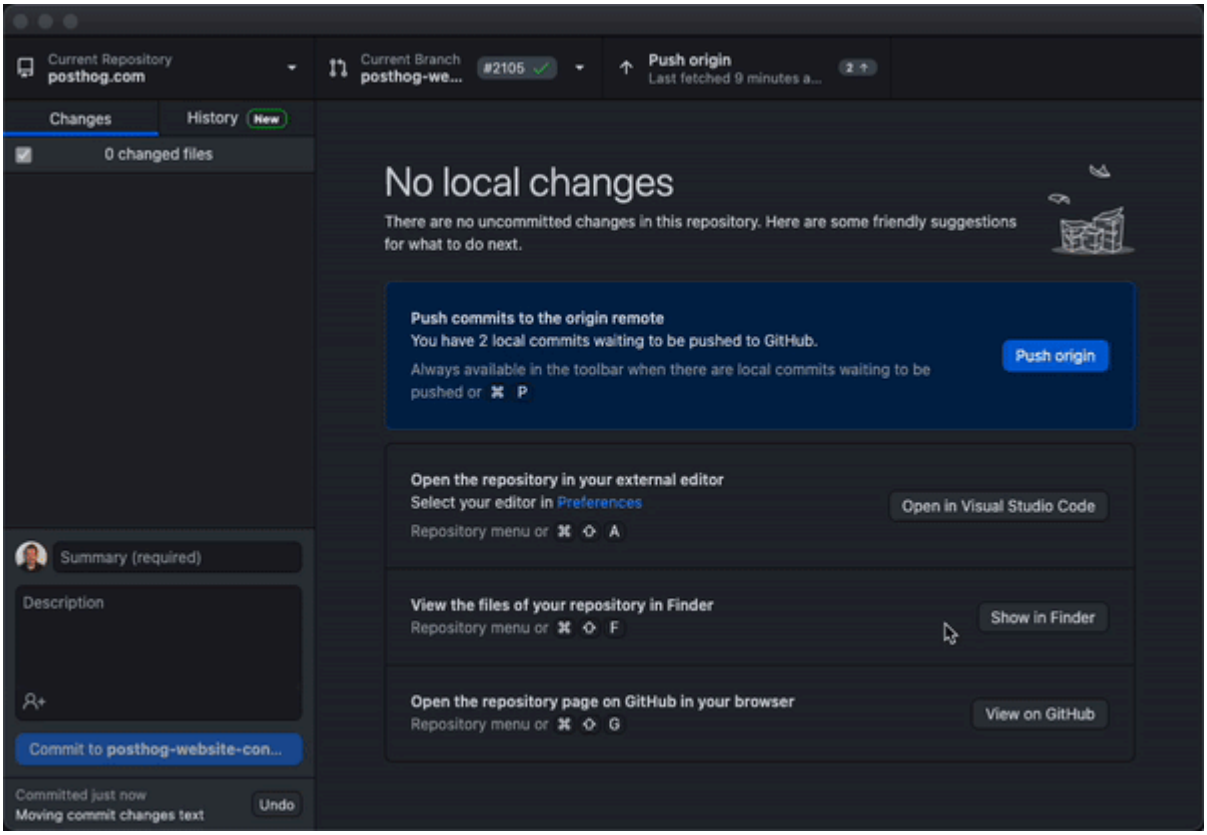
  ```
  posthog-website-contribution $ git push origin posthog-website-contribution
  Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
  remote:
  remote: Create a pull request for 'posthog-website-contribution' on GitHub
          by visiting:
  ```

```
remote:      https://github.com/PostHog/posthog.com/pull/new/posthog-
        website-contribution
remote:
To github.com:PostHog/posthog.com.git
* [new branch]        posthog-website-contribution -> posthog-website-
        contribution
```

This output tells you that you can create a pull request by visiting a link. In the case above, the link is `https://github.com/PostHog/posthog.com/pull/new/posthog-website-contribution`. Follow the link to complete your pull request.

- **via GitHub Desktop**

  Once you have committed the changes you want to push to GitHub, click the **Push origin** button.



Create a pull request

Create a pull request to request that your changes be merged into the main branch of the repository.

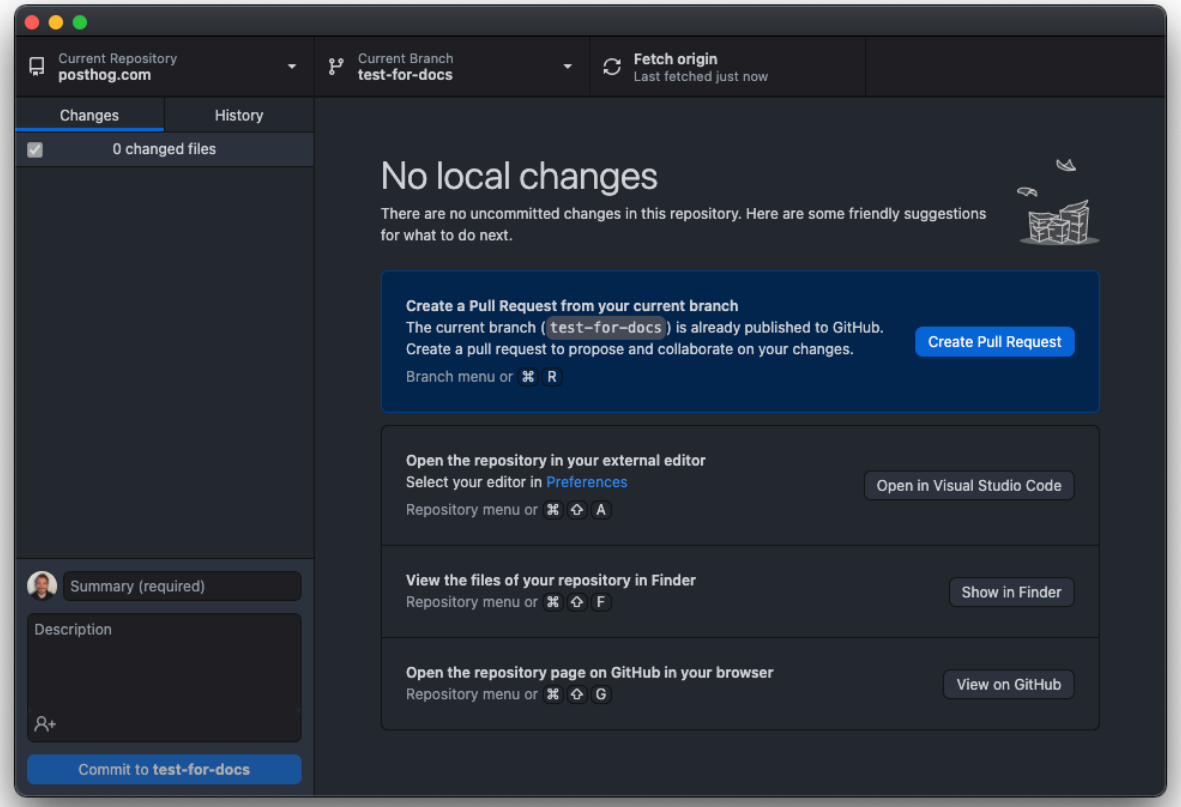
- **via the command line**

  Navigate to the link shown when you push your branch to GitHub. For example, `https://github.com/PostHog/posthog.com/pull/new/posthog-website-contribution` shown below:

```
posthog-website-contribution $ git push origin posthog-website-contribution
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'posthog-website-contribution' on GitHub
        by visiting:
remote:      https://github.com/PostHog/posthog.com/pull/new/posthog-
        website-contribution
remote:
To github.com:PostHog/posthog.com.git
* [new branch]        posthog-website-contribution -> posthog-website-
        contribution
```

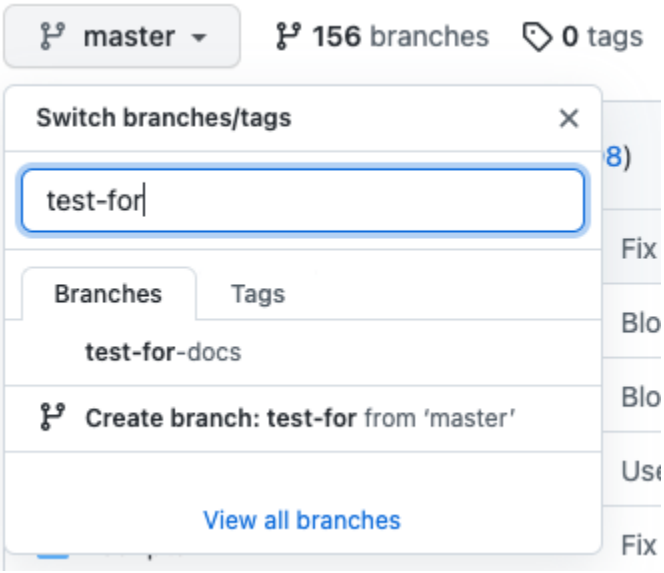
- **via GitHub Desktop**

  With the branch published, click the **Create pull request** button.



  This will open up a page on github.com in your default web browser.

If you are pushing to an existing branch, navigate to the [posthog.com repo] and switch to the new branch using the dropdown:



Then, open the **Contribute** dropdown and click the **Open pull request** button.

Make the pull request title descriptive name and complete the detail requested in the body.

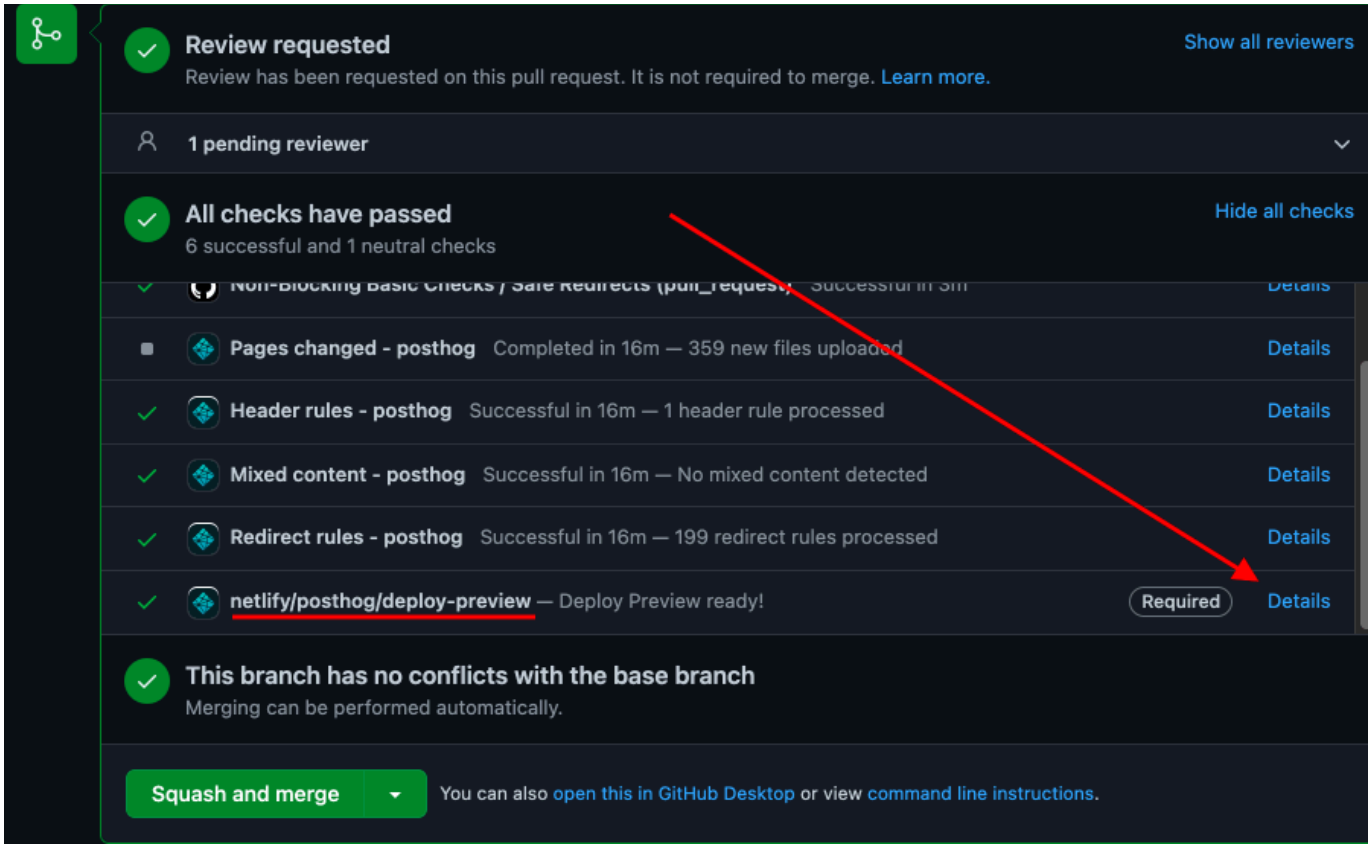
If you know who you would like to review the pull request, select them in the **Reviewers** dropdown.

Preview branch

After a series of checks are run (to ensure nothing in your pull request breaks the website), Vercel will generate a preview link available in the Vercel bot comment. This includes all of your changes, so you can preview before your pull request is merged.

An initial build can take up to 50 minutes to run. After the initial build, subsequent builds should complete in under ~15 minutes. We're limited to two concurrent builds, so if there's a backlog, this process can take longer.

Because Vercel charges per seat, we don't automatically invite all team members to our Vercel account. If your build fails, you can run `pnpm build` locally to see what's erroring out. If nothing is erroring locally, it's likely the build timed out in Vercel. The Website & Docs team monitors for failed builds, so they'll re-run it for you. If the build is urgent, give a shout in #team-website-and-docs and someone with Vercel access can trigger a rebuild for you.



> **Note:** Checks are run automatically for PostHog org members and previous contributors. First time contributors will require authorization for checks to be run by a PostHog org member.

Deployment

To get changes into production, the website deploys automatically from `master`. The build takes up to an hour, but can be delayed if other preview builds are in the queue.

Product interest tracking for onboarding

We track which products users have shown interest in by visiting product landing pages or docs. This data is stored using PostHog's `cookie_persisted_properties` feature, making it available across all posthog.com subdomains (including app.posthog.com) for onboarding personalization.

How it works

When a user visits a product-specific page (like `/product-analytics` or `/docs/session-replay`), we record that product's slug using `posthog.register()` with the property `prod_interest`. This property is configured in `cookie_persisted_properties` in `gatsby/onPreBoostrap.ts`, which means it gets stored in a cross-subdomain cookie automatically.

To read the interests, we use `posthog.get_property('prod_interest')` which returns an array of product slugs like `["product-analytics", "session-replay"]`.

We always store the most recent interests last in the array.

Code structure

The tracking is implemented in:

- `src/lib/productInterest.ts` - Core utilities using `posthog.get_property()` and `posthog.register()`
- `src/hooks/useProductInterest.ts` - React hooks for tracking
- `src/components/Products/Slides/SlidesTemplate.tsx` - Integration for product landing pages
- `src/templates/Handbook.tsx` - Integration for docs pages

Reading interests on app.posthog.com

Since this uses PostHog's built-in cookie persistence, you can read the interests on any subdomain where PostHog is initialized:

```
const interests = posthog.get_property('prod_interest') || []
// interests = ["product-analytics", "session-replay", ...]
```

Expanding usage

Everything is usually automatically handled because our website is well-structured but if you want to start tracking interest for new products you'll need to add a new entry to `PRODUCT_SLUGS` in `src/lib/productInterest.ts`

Acknowledgements

This website is based on Gatsby and is hosted with Vercel.

PostHog.com site architecture

contents/handbook/engineering/posthog-com/technical-architecture.md

PostHog.com doesn't behave like a normal website. Instead, it runs inside a desktop-style environment where every page is a draggable window. This guide explains how that system works under the hood.

Core architecture

PostHog.com runs on Gatsby with a custom windowing system built using React context providers. The entire site operates inside a desktop-like environment where traditional page navigation is replaced by window management.

At a high level, every page is wrapped in the **App Provider**, which manages global state and window logic. The **Wrapper** renders the desktop interface, and each page is displayed inside an **AppWindow** component on the **Desktop**.

Key components

- **App Provider** (`src/context/App.tsx`) – Core state management and window system
- **Wrapper** (`src/components/Wrapper/index.tsx`) – Desktop layout and window rendering
- **AppWindow** (`src/components/AppWindow/index.tsx`) – Individual window state management
- **Desktop** (`src/components/Desktop/index.tsx`) – Desktop environment with wallpapers and icons

How pages become windows

Every page in the site is wrapped using Gatsby's `wrapPageElement` API in `gatsby-browser.tsx`:

```
export const wrapPageElement = ({ element, props: { location } }) => {
    return (
        <Provider element={element} location={location}>
            <Wrapper />
        </Provider>
    )
}
```

When Gatsby loads a page, it passes:

- `element` – The page's React component
- `location` – Current route information

These get passed to the App Provider, which converts them into windows.

The App Provider system

Located at `src/context/App.tsx`, the App Provider is the core of our windowing system.

Window state management

The App Provider maintains an array of active windows in state:

```
const [windows, setWindows] = useState<AppWindow[]>([])
```

Each window object contains:

- `element` – The React component to render
- `position` – X/Y coordinates
- `size` – Width/height dimensions
- `zIndex` – Window stacking order
- `minimized` – Minimized state
- `path` – Route path
- `appSettings` – Window-specific configuration

Core functions

Key window management functions include:

- `addWindow()` – Creates and adds new windows to state
- `closeWindow()` – Removes windows from state
- `bringToFront()` – Updates z-index for window focus
- `minimizeWindow()` – Sets minimized state
- `updateWindow()` – Updates position, size, and other properties

Window routing behavior

The App Provider decides whether to create, focus, or replace a window based on navigation state:

1. **New window** – If `newWindow: true` is passed in location state, or no existing window matches the path
2. **Focus existing** – If a window with the same path already exists, bring it to the front instead of creating a duplicate
3. **Replace** – For standard navigation without `newWindow: true`, replace the content of the focused window

This prevents duplicate windows for the same route while still allowing intentional multi-window behavior.

App settings configuration

Window behavior is controlled by the `appSettings` object in `src/context/App.tsx`. Each route can have custom settings:

```
const appSettings: AppSettings = {
    '/': {
        size: {
            min: { width: 700, height: 500 },
            max: { width: 800, height: 1000 },
            fixed: false,
        },
        position: {
            center: true,
            getPositionDefaults: (size, windows, getDesktopCenterPosition) => {
                // Custom positioning logic
                // Currently only offsets the homepage window
                // so the default background is always fully visible
            },
        },
    },
    // More route configurations...
}
```

Configuration options

- **size.min/max** – Minimum and maximum window dimensions
- **size.fixed** – Whether window can be resized
- **size.autoHeight** – Auto-adjust height to content
- **position.center** – Center window on screen
- **position.getPositionDefaults** – Custom positioning function

The Wrapper component

`src/components/Wrapper/index.tsx` handles the actual desktop rendering:

```
export default function Wrapper() {
    const { windows, constraintsRef } = useApp()

    return (
        <div className="fixed inset-0 size-full flex flex-col">
            <TaskBarMenu />
```

```
            <div ref={constraintsRef} className="flex-grow relative">
                <Desktop />
                <AnimatePresence>
                    {windows.map((item) => (
                        <AppWindow item={item} key={item.key} />
                    ))}
                </AnimatePresence>
            </div>
        </div>
    )
}
```

It renders:

- Desktop background and icons
- Taskbar at the top
- All active windows with animations

It also provides drag constraints for window movement via `constraintsRef`.

Window implementation

Individual windows are implemented in `src/components/AppWindow/index.tsx` using Framer Motion for animations and drag interactions. Each window is wrapped in a Window Provider so that child components can access the current window object via the `useWindow` hook.

Key features

- **Dragging** – Windows can be dragged around the desktop
- **Resizing** – Resize handles on window borders
- **Snapping** – Windows snap to screen edges
- **Minimizing** – Windows minimize to taskbar
- **Focus management** – Click to bring windows to front
- **Chrome** – Window controls (close, minimize, maximize buttons)

Window lifecycle

1. **Creation** – New `AppWindow` object added to state
2. **Mounting** – Component mounts with entrance animation
3. **Interaction** – User can drag, resize, minimize, close
4. **Unmounting** – Exit animation before removal from state

Experience modes

The site supports two experience modes controlled by `siteSettings.experience`:

- **posthog** – Full desktop OS experience with windows
- **boring** – Traditional website navigation (used on mobile or when explicitly toggled)

During development you can manually force boring mode by setting `siteSettings.experience = 'boring'`. This is useful for debugging.

Keyboard shortcuts

Global keyboard shortcuts are handled in the App Provider:

Navigation and search

- `/` or `Cmd+K` – Open search
- `?` – Open chat

Appearance

- `,` – Display options
- `.` – Keyboard shortcuts
- `\` – Toggle theme
- `|` – Cycle wallpapers

Window control

- `Shift + Arrow keys` – Window snapping
- `Shift + W` – Close focused window
- `Shift + X` – Close all windows

SEO compatibility

Despite the desktop interface, the site maintains full SEO compatibility:

- Pages are statically generated at build time
- Each route has proper HTML structure, canonical tags, and metadata
- Search engines crawl normal static files
- Client-side windowing does not affect crawling

Development workflow

When working on the windowing system:

1. **Test window creation** – Ensure new pages create windows properly

2. **Check positioning** – Verify windows open in expected locations
3. **Test interactions** – Drag, resize, minimize, close functionality
4. **Verify animations** – Smooth entrance and exit animations
5. **Mobile compatibility** – Ensure fallback to boring mode works

Common debugging

- **Windows not appearing** – Check `appSettings` configuration for the route
- **Positioning issues** – Verify `getPositionDefaults` logic
- **Animation problems** – Check Framer Motion configurations in AppWindow
- **State sync issues** – Use React DevTools to inspect App Provider state

This architecture allows PostHog.com to feel like a desktop operating system while maintaining the benefits of a static website for performance and SEO.

Product Design, for Engineers

contents/handbook/engineering/product-design.md

We believe that everyone is a designer. Because we hire generalists, there is no expectation that every project should start by running through design *first*. It is up to you when to involve our product designers in your work.

You should start by identifying the stage and goals of your project.

v0.1 or v2?

As the feature owner, you should make a choice if you're building a very basic first iteration of something, or if you're improving the experience.

There are two paths for creating the first version of a product: v0.1 or MVP (even earlier).

v0.1

If we're attempting to reach parity on a product or feature with other competitors in the space – and there's a clear path toward how a product should work or look – there's no need to loop in a designer. You should make your best judgement, while leveraging our design system to build your feature.

MVP

If you're shipping an entirely new feature (i.e. SQL for PostHog), then you should figure out if any users even care (!), which usually means creating an MVP and releasing it behind a feature flag to some friendly users. (Pro tip: make friends by being support hero.)

During both of the above approaches, designers are happy to provide light recommendations that will improve the user experience without becoming a blocker to shipping.

v2

If you're improving an *existing* feature that is popular, you are probably creating v2. Typically when we decide to "Nail [a specific feature]", it's worth working closely with design to figure out how we can *10x* our product vs. competitors.

However you're building, please *communicate* to product design what your expectations are!

Feature Complexity

The more complex a feature is to implement, the more likely it is that involving product design will make you faster.

Your design skill

We generally hire full stack engineers, but some people think more like designers than others. This is fine - you should play to your strengths.

The less strong you are at design, the more we'd encourage you to involve a product designer.

If you're unsure about your skill level, ask a product designer for direct feedback. This is a book we'd recommend if you want to learn the mindset.

Scenarios for looping in product design

If you built something and just need some polish...

Feel free to share a link (or screenshot) of what you've built. We can provide UX or design feedback for your consideration.

If you built something and realize it needs some UX love...

Share a link (or screenshot) of what you've built. Depending on the state of the project, we can either go back to the wireframe stage to rethink some things, or figure out a phased approach to incremental improvement.

If you designed your own wireframes or mocks...

Sometimes if you have domain knowledge or have been thinking about a project for a while, it might make more sense for you to start the design process. Feel free to share with us for a second opinion, or if you think certain UIs or flows are suboptimal.

Need help brainstorming a flow? Pair with a product designer

If you'd like the help of a product designer on an MVP/v0.1-type project, a 30-60 min Zoom working session is a great way to brainstorm and sketch out ideas. Since our design team is small, we try to avoid too much "homework".

Usually during quick syncs like this, it's enough to help an engineer work through complex UX issues. Reach out to Cory if you're interested in a synchronous session like this.

Product design capacity

Sometimes product design may push back if they simply don't have capacity. It's subjective when this may happen, and it'll usually be in cases where they feel they won't be as helpful based on the above.

Read more about how product design works at PostHog - *it's very unique!*

How to do product, as an engineer

contents/handbook/engineering/product-engineering.md

Good product engineers, bad product engineers

Good product engineers:

- Ship quickly so they have a fast feedback loop
- Understand the company strategy, and prioritize based on this *and* what they believe users want
- Can easily propose ideas for what to build
- Make sure the things they've built are being used
- Follow up after they've built something to improve it if needed
- Are good at descoping things and getting products or features into people's hands quickly
- Have users that they're friendly with
- Manage to build things without lots of internal meetings
- Dive *deep* when they need to, because shipping D might also require solving A, B, and C

Bad product engineers

- Consider research something that takes two weeks rather than two hours
- Can't explain our company strategy
- Can't explain who their product is built for
- Don't know their product's competitors
- Only work on things they've been told to work on
- Don't know the names of any of their users
- Never challenge why they're being told to work on something
- Don't talk to users about what they're going to build, or what they've built
- Don't track if the things they've built are being used
- Spend 6 months on a huge feature before a user can try it
- Never remove features or complexity, often by shipping features that aren't used and leaving them
- Focus on internal alignment over company strategy and what users need
- Wait for someone else to fix an adjacent problem

How to

Validate ideas

Despite what the industry tells you, it's debatable how well you can validate ideas up front (see: the number of startups that think they'll succeed based on user interviews then find they can't get any users). Just shipping is often the best way to validate an idea. When we built PostHog, Tim and James had to

pivot 5 times – despite getting positive feedback on new ideas almost *every time*. Talking to users upfront can probably help remove totally stupid ideas fast, but for the majority of ideas "this could work", it only has a limited amount of benefit in our experience.

This gives you the best evidence (do people *actually* use it, and what do they think), but *potentially* at the highest cost as you have to build it! The challenge with this approach is making sure you de-scope the first version of the product or feature enough that users will at least try to use it, so you get enough signal that they care, without damaging our brand because the experience is so poor.

So, when you ship something:

- Consider what you are trying to learn (if anything, importantly – many things are so obvious like fixing a well-defined bug, you aren't trying to learn anything) product-wise.
- Descope it as much as possible to reduce the cost *you* incur upfront of building it.
- Judge for yourself if and how to limit brand damage (your options are one or more things like - internal use first, a feature flag rollout, messaging a couple of friendly users, not marketing it or limiting the marketing, or shooting for Minimum Lovable Product instead of Minimum Viable Product).
- Follow up... figure out if / why it is or is not being used, and iterate. If you've shipped early, it'll be crappy so *will need more work* as you figure out what users want.

Just shipping makes sense when it's very obviously in line with our company strategy (which is generally proven), and you can descope it successfully. This is almost *everything* that you may ever build here. The key is to manage the rollout carefully.

Products at PostHog generically go through three phases, and considering your phase is important when you ship new features:

1. Pre PMF at PostHog

- We only build things that *already* have successful competitors with real revenue. The implication is that "just build it" works disproportionately well because other people have already figured out that new product X has product market fit.
- The challenge is therefore figuring out if *PostHog's* users base will want the new product, as we already know the product is useful.

2. Figuring out PMF at PostHog

- This is where we are getting our very first paying customers.
- The product focus is making sure people are *delighted* with the above features. Maybe there are bugs or maybe there are too many gaps with competitors still for people to pay. Focus on how happy users are and why / why not. Keep an eye on early revenue data too - are people willing to

pay, or are they churning?

3. Post PMF at PostHog

- This is where we're scaling the number of free and paid users.
- Features at this stage either fall into:
    - (i) gaps with competitive products that we've not prioritized so far, probably based on feature requests from users, in which case the risk of them not being useful for users is pretty low or...
    - (ii) totally innovative things like new UX driven by our take on AI, or a new way to access data (like Hog or SQL), or an integrated experience that no one else can offer because they don't have all the tools in one. In these cases, it is *more important* to consider how your products are being used as you are more likely to build something that isn't useful (but at this stage, it's fine and encouraged to innovate)

There are plenty of other techniques, that you can do in parallel to get a signal on a new idea:

- **Use the public roadmap.** If you have a rough idea, create a GitHub issue and create an item on posthog.com/roadmap. This will gauge demand.
- **Ask internally for help.** There are lots of people that can help you. The CS team talk to users all the time, the support team have a strong sense of pain points, other product engineers have all talked to users, James and Tim have a broader view of how PostHog is doing, the marketing team can help you get usage or validate demand.
- **Interviews.** Our Product Managers regularly run interviews, ask to be included and give a heads up what you're trying to learn about, or just message users directly! You can even embed your calendar in our surveys app to book your own user interviews. You will need lots of existing potentially relevant customers for this to make sense, since response rates are typically low.
- **Listen to the internet.** We have a #brand-mentions channel in Slack that monitors for social mentions by customers, or get us to post a question here if you need.
- **Seek internal feedback.** We dogfood all our own products to grow our company, so ask for *internal* relevant users.

Ship things iteratively and follow up

- Use staggered rollouts: we have a product *designed* to help you do this. Depending on how risky a new feature is, start with internal users, or
- Data: check if the thing you just built is being used. Remember to add some events.
- Session replay: watch users using your thing. This can often highlight confusing UX.
- Interviews
- Support

- Listen to the internet

Iterate with users

A note on attitude first - any kind of feedback, bug report, complaint or usage is a gift from users. It's easy to get dismissive or frustrated when people don't "do what we want"! Worst case scenario is that we get ignored.

Handling users well is really important. If we do a good job responding to feedback:

1. The product improves because we do a better job at building what users want.
2. We get marketing benefits because the user will be impressed and will tell their friends.
3. We get more feedback because it teaches people that we listen and that we care.

Tone matters a lot. Whenever you are messaging a user, please consider:

- They're on the internet, so you are competing with cat videos. It better be compelling.
- They receive loads of outreach from people. If you send something generic, it will get auto filtered out by their brain.

So, how do you make yourself compelling to engage with?

The tone is your starting point. Send something informal and human. You are explicitly trying to avoid sounding like a mega corporation that treats users like numbers. You are a human, your users are human. Be friendly, light hearted and fun. Make it clear the message isn't automated if you can.

If you *must* automate messages for whatever reason, make them quirky and informal and human. "Yo I'm Manoel, my job at PostHog is making sure mobile users are happy. It looks like this includes you! I build X, Y, Z here – is there any chance we could talk about X new thing? Here's my calendar or respond and we'll find time!" sort of vibe. Don't make messages long if you want people to do something – one or two sentences.

The medium matters. The easier something is to spam, the harder it is to get hold of people. For example, email gets ignored far more than Slack or X.

Response times are very important. If you can catch someone *whilst* you're top of mind, you are likely to get 20x the response rate. That means within a minute or two of receiving a message. There is a huge drop off if you don't respond for 30+ minutes. Obviously this isn't always possible, but take opportunities if you happen to be online at the same time as someone you need feedback from. I once ran a call center – if we phoned someone who made an enquiry within 5 minutes, it was 9x more likely we'd get hold of them.

Closing the loop is the final point. If a user gives feedback or asks for something, you should ultimately respond with:

- A PR (which will likely delight them)

- A roadmap item / issue they can follow (hey this is awesome, I want to do it, this is so you can follow progress)
- A reason why we can't do their thing

Closing the loop with the above shows people we've listened and considered their points carefully, and that we respect their opinion. This means they will continue to give us feedback.

Talk to users

If you're talking to a user, there are some basic principles if you want to be a good product engineer.

Use a lot of open ended questions. Ask things like:

- Do you understand your user behavior? Why / why not?
- What do you want to know about how a feature is used?
- If I could build one thing for you right now, what would it be?

Look for evidence that users have *actually done anything* about the problems they say they have.

People want to be likable, so they'll often say they want what you're working on, even if they don't. Lots of features or products are nice-to-have versus must have. When something is a nice-to-have, people will act interested but won't get around to actually using it.

Ask things like:

- Have you tried to solve this problem before? What did you do?
- How important is this issue compared to other things you have to work on?
- Has your company spent money trying to solve this before?

Write down every interview. This helps us come back as the rest of the team, or you, consider other products or features in future.

Revenue and forecasting

`contents/handbook/engineering/revenue-and-forecasting.md`

The maintains the revenue dashboards and queries that are used to understand:

1. What our historical revenue record looks like
2. What our revenue is expected to be this month
3. What our churn, growth, expansion, and contraction look like
4. Which customers have done the above activities
5. etc

Currently, all revenue dashboards can be found in Metabase (though we hope to have them all in PostHog's own data warehouse soon 👀).

Important dashboards

(these require internal access)

- General overview: Useful to see our ARR graph and get quick links to dig into revenue for certain months.
- Revenue lifecycle: Useful to see trends over time for churn, expansion, new revenue, etc.
- Product-specific analysis (pick your product and date [month] at the top!): Kind of a combination of the above two, but for a specific product
- Revenue by customer bucket: Useful for understanding trends in contract size for financial modeling, support, etc

FAQ

How is revenue attributed to a certain month?

Revenue is attributed to a given month based on the end-date of the invoice period. For example, an invoice that has a period of 2023-01-12 to 2023-02-12 will be counted in the revenue for 2023-02.

Some invoices cover multiple months. In this case, in the `invoice_with_annual` table (which is what all our dashboards use) we take the total amount of the invoice and divide it by the number of months the invoice covers, which gives us the MRR. We then generate a row for each month with that MRR so we can count that revenue into our monthly ARR/churn/expansion/etc calculations.

When is it a "forecast" vs a real, complete number?

As soon as any given month starts, we start closing invoices. As the month goes on and customers' invoice periods end, we close more invoices. This means that as the month goes on, we get more and more confident about what our revenue will be for that month. The month's revenue can still change after the

month is over, however, due to delayed payments. This is generally not a hugely significant number, but it is something to be aware of.

How do we forecast a customer's revenue?

Our revenue is based on usage, so we do some basic math to make an educated guess about how much usage a customer will have in the current period.

- For new customers, their usage can be very spiky as they tune their implementation. We don't currently forecast their usage if it's less than 31 days since their subscription started. Instead, we just report their current usage as their forecast.
- If there are fewer than 7 days elapsed in the customer's current billing period, get the last 7 days usage (even if it's from the last period) and calculate the usage per hour. The calculate the remaining hours in the billing period, and do math to make an estimate.
- If there are more than 7 days elapsed in the customer's current billing period, do the math based on all the days in the billing period.
- If the customer has billing limits set, respect those billing limits.

How are cancelled bills handled? Are those forecasted?

As soon as someone cancels their account, their invoice is immediately closed. The revenue from that invoice immediately goes into the "completed" pile.

When are invoices updated?

A task is run nightly to sync the last 2 months of completed invoices, as well as all upcoming invoices for all customers. After the task is complete, the `invoice_with_annual` view is updated with the fresh data.

SDKs

contents/handbook/engineering/sdks/index.md

There is now a small, dedicated SDK team at PostHog (@PostHog/team-client-libraries) that helps drive direction and coordination. However, SDK development and maintenance remains a collaborative effort across the engineering organization. @PostHog/client-libraries-approvers exists in GitHub to coordinate the collaboration for those who are more interested than normal in the development of these SDKs.

How SDK work gets done

SDKs are maintained by engineers across different teams who either:

- Volunteer for the SDK support rotation to help with issues and bug fixes
- Contribute improvements when their team's product needs SDK changes (e.g., adding session replay support to a new SDK)
- Pick up SDK work during hack days or when they have spare capacity

This distributed model means SDKs get attention from engineers with diverse expertise, and ownership is shared across the company.

Want to get more deeply involved?

If you're interested in contributing to our SDKs — whether that's fixing bugs, adding features, or improving documentation — drop a message in **#team-client-libraries**. We're always happy to have more people helping out, and it's a great way to learn about different parts of the PostHog ecosystem.

You can also sign up for the SDK support rotation to get hands-on experience with SDK issues.

Slack channels

- **#team-client-libraries** — Main channel for SDK discussions and general questions
- **#support-client-libraries** — Main channel for SDK support discussions, handovers between support rotation engineers
- **#approvals-client-libraries** — Where release approval requests are posted (see Releases)

What are our SDKs?

There are too many to have a static list here. Check the libraries docs to learn more about them.

SDK support rotation

contents/handbook/engineering/sdks/support-rotation.md

The SDK Support Hero rotation is managed by the . Each week, one member of the team is designated the SDK Support Hero. The schedule is managed in incident.io.

Your primary responsibility is to make sure SDK questions get some love — across all SDKs, including mobile. During the rotation, please keep an eye on:

- Escalated SDK tickets in Zendesk
- New issues in the SDK repositories:
  - posthog-js (Web, Web Lite, React, React Native, Next, Node, AI)
  - posthog-python
  - posthog-ios
  - posthog-android
  - posthog-flutter
  - Others, see /docs/libraries

How should I prioritize my time?

Firstly, try to stay on top of new escalated Zendesk tickets and GitHub issues, and make sure that issues related to a specific team are routed to them. If there is a relevant team (e.g. the issue is related to session replay in posthog-js), you can assign the Zendesk ticket to that team, and use the team's label in GitHub. If there is no relevant team for a GitHub issue, please label with `SDK Support Hero`. Feel free to try to fix things yourself before tagging the team.

Next, please work on SDK tickets in Zendesk, and GitHub issues labelled `SDK Support Hero` (and unlabelled, but please label these!). You can use your own judgement to decide which issues to work on but please consider effort / reward / urgency / your skill set. For example, `posthog-js` usually has the most issues, but if you're a Python expert, you might want to focus on `posthog-python`.

For mobile SDK issues, prioritize accordingly — rolling out fixes on mobile apps may take weeks or even months, so faster turnaround on these is important. Make sure, however, to validate changes carefully, avoid breaking changes and think through edge cases before shipping, since our ability to correct mistakes after release is significantly constrained.

At the end of the week, please write a public handover message in `#support-client-libraries`, to let the next person know what work is in progress, let the team know how the support rotation is going in general, and to share any learnings or feedback.

Security Best Practices

contents/handbook/engineering/security.md

GitHub

SSH Keys

Connecting to GitHub requires an SSH key (unless using HTTPS). Traditional SSH keys live as text files on your filesystem, making them vulnerable to theft or misuse by malware. We explicitly prohibit the use of SSH keys stored on your filesystem.

Use Secretive or 1Password to generate and store your SSH key. We have a slight preference for Secretive because it stores your key in the macOS Secure Enclave, ensuring the key can never be exported or extracted, even by malware. Always use ECDSA or Ed25519 — don't use RSA.

Setting up with Secretive

1. Open Secretive and click the + button to create a new key.
2. Name your key "GitHub SSH" and select **Notify** in the **Protection Level** dropdown.
    - For additional protection, select **Require Authentication** instead. This will require you to use Touch ID each time the key is accessed.
3. Go to **Secretive > Integrations** in the menu bar.
4. Select your shell on the left side set the `SSH_AUTH_SOCK` environment variable as instructed. For zsh, add the following to your `~/.zshrc`:

    ```
    export
            SSH_AUTH_SOCK=~/Library/Containers/com.maxgoedjen.Secretive.SecretA
            gent/Data/socket.ssh
    ```

    Then run `source ~/.zshrc` to apply it.
5. Click on your new key in Secretive and copy the public key.
6. Go to your GitHub SSH keys settings and add a new SSH key. Paste your public key and set the key type to **Authentication Key**.
7. Test it by running:

    ```
    ssh -T git@github.com
    ```

You should see a message like "Hi username! You've successfully authenticated".

Setting up with 1Password

Follow the 1Password SSH key management guide.

Commit signing

A git commit's `Author` field is completely user controllable and can be forged. Signing your commits cryptographically proves you authored them, preventing impersonation and confusion.

You can sign commits with either Secretive or 1Password. We have a slight preference for Secretive because it stores your key in the macOS Secure Enclave, ensuring the key can never be exported or extracted, even by malware.

Setting up with Secretive

1. Open Secretive and click the + button to create a new key.
2. Name your key "Git signing key" and select **Notify** in the **Protection Level** dropdown.
3. Go to **Secretive > Integrations** in the menu bar.
4. Click **Git Signing** and select "Git signing key" from the **Secret** dropdown.
5. Copy and paste the `~/.gitconfig` and `~/.gitallowedsigners` snippets into their respective files
    - If you already have content in `~/.gitconfig`, merge the new sections into the existing file rather than replacing it.
6. Select your shell on the left side of Secretive and set the `SSH_AUTH_SOCK` environment variable as instructed. For zsh, add the following to your `~/.zshrc`:

    ```
    export
            SSH_AUTH_SOCK=~/Library/Containers/com.maxgoedjen.Secretive.SecretA
            gent/Data/socket.ssh
    ```

    Then run `source ~/.zshrc` to apply it.
7. Your `~/.gitconfig` now has a `signingkey` pointing to a file. Copy your public key to the clipboard:

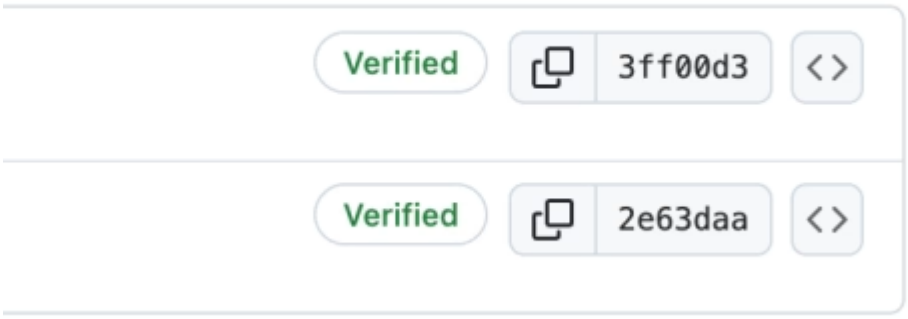
    ```
    cat <path-from-signingkey> | pbcopy
    ```

8. Go to your GitHub SSH keys settings and add a new SSH key. Paste your public key and set the key type to **Signing Key**.

9. Test it by creating an empty commit on a new branch:

```
git commit --allow-empty -m "test signing"
```

Push the branch to GitHub — you should see a green **Verified** badge on the commit.



Setting up with 1Password

Follow the 1Password git commit signing guide.

After setup

Once commit signing is configured, enable the option in your GitHub Profile to "Flag unsigned commits as unverified".

Troubleshooting

- If using iTerm/Cursor/GitHub Desktop/Sourcetree/etc., you may be endlessly prompted to "access data from other apps". You can fix this by granting the app **Full Disk Access** in **System Settings > Privacy & Security > Full Disk Access**.
- If you are prompted to complete Touch ID each time you commit, your signing key is using a **Protection Level** of **Require Authentication**. Re-follow the instructions above to generate a new signing key with a **Protection Level** of **Notify**.

GitHub Actions

Great care should be taken when writing or modifying a GitHub Actions workflow. Actions can access (and exfiltrate) secrets scoped to the repo. We scan workflows with Semgrep and CodeQL for common misconfigurations.

Authentication

Most Actions use the default `GITHUB_TOKEN`, whose permissions can be scoped via the `permissions` property. However, `GITHUB_TOKEN` cannot trigger other workflows — so commits or PRs created by an Action won't run CI, leaving PRs unmergeable without manual intervention. The workaround is a Personal Access Token (PAT) or GitHub App. We use GitHub Apps because PATs are tied to an individual user and break when that user leaves PostHog.

Scope each GitHub App to its use case and ideally a single repo. Prefer creating a new App over expanding an existing one's permissions, otherwise every Action using that App inherits permissions it doesn't need.

Send a message in #team-security if you need help setting up a new GitHub App.

External contributors

In public repos, Actions may run against PRs written by external contributors. These PRs should be reviewed thoroughly before approving workflows to run against them. Otherwise, a malicious PR could gain access to and steal all of the secrets available to the repo.

Managing secrets

AWS

Application secrets are stored in AWS Secrets Manager. To modify an app's secrets, use our secrets tool.

GitHub

Secrets used by GitHub Actions are stored in GitHub secrets. All secrets should be stored in our GitHub org rather than in an individual repo. This allows us to more easily reuse secrets across repos, and also provides a holistic view of all of our secrets. The org secret should be scoped to the specific repos that need it.

Reporting a security issue

If you believe we've been hit by a security issue, raise an incident. In the best case, it'll mean security folks look at it ASAP. In the worst case, it's a false positive and we can close the incident.

Session replay architecture

`contents/handbook/engineering/session-replay/session-replay-architecture.md`

Session recording architecture: ingestion → processing → serving

1. Capture (client-side)

PostHog-JS uses **rrweb** (record and replay the web) to:

- Serialize DOM into JSON snapshots
- Capture full snapshots (complete DOM state) + incremental snapshots (mutations/interactions)
- Track clicks, keypresses, mouse activity, console logs, network requests
- Batch events into `$snapshot_items` arrays with a `$session_id` (UUIDv7)
- Send to `/s/` (replay capture endpoint) via `$snapshot` events

Events include metadata: `$window_id`, `$session_id`, `$snapshot_source` (Web/Mobile), timestamps, distinct_id

2. Ingestion pipeline

Phase 1: Rust capture service (recordings mode)

`rust/capture/src/router.rs:235` and `rust/capture/src/v0_endpoint.rs:342`

- Separate capture service instance running with `CAPTURE_MODE=recordings`
- Receives POST to `/s/` endpoint (routed to `recording` handler)
- Validates session_id (rejects IDs >70 chars or with non-alphanumeric characters except hyphens)
- Calls `process_replay_events` to transform events into `$snapshot_items` format
- Publishes to **`session_recording_snapshot_item_events`** Kafka topic
- Or **`session_recording_snapshot_item_overflow`** if session is billing-limited (checked via Redis) OR if session id is present in Redis key @posthog/capture-overflow/replay (operational/load management)

Kafka sink (`rust/capture/src/sinks/kafka.rs`):

- Produces to primary topic: `KAFKA_SESSION_RECORDING_SNAPSHOT_ITEM_EVENTS`
- Overflow topic: `KAFKA_SESSION_RECORDING_SNAPSHOT_ITEM_OVERFLOW`

Phase 2: Blob ingestion consumer (Node.js/TypeScript)

`plugin-server/src/main/ingestion-queues/session-recording-v2/`

SessionRecordingIngester consumes from Kafka and:

1. **Parses** gzipped/JSON messages (`kafka/message-parser.ts`)
2. **Batches** by session via **SessionBatchRecorder**
3. **Buffers** events in memory per session using **SnappySessionRecorder**:

```
 public recordMessage(message: ParsedMessageData): number {
     if
...
133| {
         this.endDateTime = message.eventsRange.end
     }

     for (const [windowId, events] of
     Object.entries(message.eventsByWindowId)) {
         for (const event of events) {
             const serializedLine = JSON.stringify([windowId, event]) + '\n'
             const chunk = Buffer.from(serializedLine)
             this.uncompressedChunks.push(chunk)

             const eventTimestamp = event.timestamp
             const shouldComputeMetadata = eventPassesMetadataSwitchoverTest(
                 eventTimestamp,
                 this.metadataSwitchoverDate
             )

             if (shouldComputeMetadata) {
                 // Store segmentation event for later use in active time
     calculation
                 this.segmentationEvents.push(toSegmentationEvent(event))

                 const eventUrl = hrefFrom(event)
                 if (eventUrl) {
                     this.addUrl(eventUrl)
                 }

                 if (isClick(event)) {
                     this.clickCount += 1
                 }

                 if (isKeypress(event)) {
                     this.keypressCount += 1
                 }

                 if (isMouseActivity(event)) {
```

```
                    this.mouseActivityCount += 1
                }

                this.eventCount++
                this.size += chunk.length
            }

            rawBytesWritten += chunk.length
        }
    }

    this.messageCount += 1
    return rawBytesWritten
}
```

- Accumulates events as newline-delimited JSON: `[windowId, event]\n[windowId, event]\n...`
- Tracks metadata: click_count, keypress_count, URLs, console logs, active_milliseconds
- Compresses each session block with Snappy

4. **Flushes** periodically (max 10 seconds buffer age or 100 MB buffer size)

Persistence (`sessions/s3-session-batch-writer.ts`):

- Writes to **S3** as multipart uploads
- File structure: `{prefix}/{timestamp}-{suffix}`
- Each batch file contains multiple compressed session blocks
- Uses byte-range URLs: `s3://bucket/key?range=bytes=start-end`
- Retention-aware: writes to different S3 paths based on team retention period

Metadata written to ClickHouse via Kafka:

- Produces to **`clickhouse_session_replay_events`** topic
- Table: **`session_replay_events`** (AggregatingMergeTree, sharded)
- Stores: session_id, team_id, distinct_id, timestamps, URLs, counts (clicks/keypresses/console), block locations, retention_period_days
- Old format also used: **`session_recording_events`** (deprecated, contains raw snapshot_data)

3. Storage schema

ClickHouse tables

`session_replay_events` (primary, v2):

```
session_id, team_id, distinct_id
min_first_timestamp, max_last_timestamp
block_first_timestamps[], block_last_timestamps[], block_urls[]
first_url, all_urls[]
click_count, keypress_count, mouse_activity_count, active_milliseconds
console_log_count, console_warn_count, console_error_count
size, message_count, event_count
snapshot_source, snapshot_library
retention_period_days
```

`session_recording_events` (legacy):

- Stored raw `snapshot_data` directly in ClickHouse
- Deprecated, no usage (maybe except very old, long-lived hobby installs but unlikely and totally unsupported)

PostgreSQL

PostgreSQL writes happen when:

1. User pins to playlist → Immediate write
2. User requests persistence → Immediate write + background LTS copy task
3. Auto-trigger on save → Background LTS copy task (via post_save signal)
4. Periodic sweep → Finds recordings 24hrs-90days old without LTS path, queues background tasks

Note: Regular session recordings (not pinned/persisted) do NOT write to PostgreSQL - they only exist in ClickHouse session_replay_events table until explicitly pinned or persisted as LTS.

`posthog_sessionrecording` model:

- session_id (unique), team_id
- object_storage_path (for LTS recordings)
- full_recording_v2_path
- Metadata: duration, active_seconds, click_count, start_time, end_time, distinct_id
- Used for persisted/LTS recordings

S3 object storage

- Main storage: `session_recordings/{team_id}/{session_id}/...`
- Blob ingestion: organized by retention period + timestamp
- Files are byte-addressable compressed session blocks

4. Playback/Retrieval

API Flow (`posthog/session_recordings/session_recording_api.py`)

GET `/api/projects/:id/session_recordings/:session_id/`:

1. Loads metadata from ClickHouse `session_replay_events` or Postgres
2. Returns: duration, start_time, person info, viewed status

GET `/api/projects/:id/session_recordings/:session_id/snapshots`: Two-phase fetch:

1. **Phase 1**: Returns available sources: `["blob"]` or `["blob", "realtime"]`
   - note: "realtime" source is no longer used for blob-ingested recordings (possibly used only for Hobby)
2. **Phase 2**: Client requests `?source=blob`

Source resolution:

- **Blob (primary)**: Queries ClickHouse for block metadata
  - Gets S3 URLs with byte ranges for each session block
  - Generates pre-signed URLs (60s expiry)
  - Client fetches compressed blocks directly from S3
- **Legacy**: Recordings were originally stored in ClickHouse `session_recording_events` table (migrated away in 2024)

Query (`queries/session_replay_events.py`):

```
SessionReplayEvents().get_metadata()  # metadata
SessionReplayEvents().get_block_listing()  # S3 blob locations
```

Returns block listing:

```
[{
  "blob_key": "s3://bucket/path?range=bytes=0-1000",
  "first_timestamp": "...",
  "last_timestamp": "...",
  "first_url": "...",
  "size": 1000
}, ...]
```

Frontend playback

`frontend/src/scenes/session-recordings/player/`

1. **sessionRecordingPlayerLogic** fetches snapshot sources (only blob_v2 now, except for hobby)
2. For each snapshot source fetches snapshots
3. Decompresses Snappy blocks
4. Parses JSONL: `[windowId, event]` per line
5. Feeds to **rrweb-player** for DOM reconstruction
6. Renders in iframe with timeline controls

Metadata (`playerMetaLogic.tsx`):

- Shows person info, properties, events, console logs
- Queries events from `events` table filtered by session_id

Key optimizations

- **Compression**: Snappy for session blocks
- **Byte-range fetching**: Only fetch needed time ranges from S3
- **Pre-signed URLs**: Direct client→S3 download, no proxying
- **Buffering**: 10 second batches reduce S3 write ops
- **Sharding**: ClickHouse sharded by distinct_id
- **TTL**: Automatic expiry based on retention_period_days
- **Overflow handling**: Separate Kafka topic + limiter for billing control

Data flow summary

```
Browser (rrweb)
  → POST /s/ with $snapshot events
  → Rust Capture validates & produces to Kafka
  → Node.js Blob Ingestion buffers & compresses
  → Writes to S3 (session blocks) + ClickHouse metadata (via Kafka)
  → Frontend fetches metadata from ClickHouse
  → Frontend fetches blocks from S3 via pre-signed URLs
  → rrweb-player reconstructs & renders
```

Tech talks

contents/handbook/engineering/tech-talks.md

We encourage engineers to give tech talks on topics they're interested in/knowledgeable about.

> Recording links are only accessible by the PostHog team.

Here are our talks so far:

- "PostHog Cloud infrastructure" by James Greenhill
- "Let's Talk About PyCharm" by Marius Andra
- "Approaches to scaling" by Karl-Aksel Puulmann
- "Databases 101" by James Greenhill

How we track and manage usage

contents/handbook/engineering/usage_reports.md

Tracking and managing usage is one of the core responsibilities of the . If we do it wrong, we don't get paid.

Each organization's usage is calculated once per day and saved in a usage report. This usage report is sent to the billing service, which saves the report and sends the usage along to Stripe for the customer's subscription, if one exists.

Usage reports

Usage reports are largely generated within `posthog/posthog` - because that's where the usage happens. Every day at midnight BST a cron job runs in each instance (US and EU) to calculate usage for every single organization in the instance.

Occasionally the cron will get interrupted - when this happens the billing service won't receive or store any of the reports, and usage won't be sent to Stripe. You'll notice that usage reports have failed in two ways:

1. When looking at the Revenue dashboard in Metabase, you'll see that there are fewer reports than previous days, and one of the instance (generally US) will show 0 reports sent.
2. When looking at the Usage report insight on the Growth dashboard you'll see a big dip in an otherwise steady trend.

We don't currently have a way to automatically re-run failed usage reporting, so we have to do it manually. To do so, you'll need to follow the instructions to connect to PostHog Cloud infra. Once you do so you can run a management command to re-run the usage reports for a specific date:

```
python manage.py send_usage_report --date YYYY-MM-DD --async 1
```

where the date is the day that the usage report would have been run, so is *one day past the date where usage reports are missing*. For instance, if we had 0 usage reports on May 11, the date you'd use in the command is actually May 12 (because usage reports are reporting usage for the previous day).

It is recommended to run async using the `--async 1` option so you don't need to wait for all the billing requests to be completed synchronously. If you use this option, it'll finish with `Done!`. When using this option, it's important to go back and ensure it is completed and there are no errors / Clickhouse timeouts.

If you run the command without the async option it can take a while to run, and if it gets interrupted (eg because pods were turned over with a deploy) it'll fail again with `command terminated with exit code 137`. Simply reconnect and try again. If it's successful, you'll get a log like `21262 Reports sent!`.

Visiting customers as an engineer

contents/handbook/engineering/visiting-customers.md

As a product engineer, you're encouraged to visit customers at their offices to gather feedback and ship features or improvements on the spot.

While PostHog is fully remote - we optimize for async work, write things down, and talk to users remotely – the reality is that occasional in-person time with customers is highly valuable. Some products are hard to dogfood properly, and it can be tricky to fully grasp specific workflows or see how high-scale users actually operate.

In-person visits let you notice things that don't surface on calls: team dynamics, tools they rely on day to day and small but important friction points that get lost in remote conversations. People also tend to hold back or polish their feedback when writing it down – they might dismiss a detail as unimportant, or assume you already know something when you don't. Seeing it all unfold in real life can surface insights you'd never get otherwise. All of this makes a customer visit time well spent.

Which customer should you visit? Sometimes, when interacting with a customer on Slack, you'll notice an obvious click with someone. You'll know when this happens – they're friendly, proactive with feedback, and genuinely interested in making the product better. They might also be driving heavy usage, with multiple power users, or even adoption across different teams. If you come across a customer like this and feel curious to dig deeper into how they use PostHog, that's a strong sign they'd be a good candidate for a visit. Of course, this makes the most sense for customers whose teams are at least partly in-office – otherwise you won't get the real benefit of seeing how they work together day to day.

Here's one way to organize a great customer visit. None of this is set in stone, so feel free to adapt, and pay attention to what the customer is comfortable with.

1. Identify the biggest areas for improvement

Review the customer's Slack channel and pull together a list of the most pressing issues from the past few months. For larger customers, there may also be lots of context in BuildBetter and past recorded calls, as well as information from their sales or CS person. Focus on understanding what they're struggling with most and which upcoming features matter the most to them.

2. Lock in dates and the point of contact

Find a single point of contact to help organize the visit. Share with them the list of topics and pain points, and explain that you'd like to meet in person to get a deeper understanding. Don't underestimate this step – even if the company is very engaged on Slack, people are busy and organizing something optional like this isn't always their top priority. Having one person on their side makes it much easier to get dates confirmed, which is the main thing you need at this stage. You can sort out the details and a more precise agenda later.

As for the duration, two to three days is usually a sweet spot – enough time to spend quality time with the team without overstaying or being a distraction in their office. Use your best judgment here and agree on the timing with your point of contact. A nice option can be to tie the visit onto a small-team offsite – if the team is already traveling, extending for a couple of days can allow some extra time for this.

For large customers with an account manager assigned, it's super valuable to bring them along. They often already have a strong relationship with the customer, can ask additional questions you might not think of, and can keep things like scheduling, follow-ups, and expectation-setting smoother.

3. Book the travel

Book flights and tickets as early as possible to avoid high prices. Reach out to the People & Ops team to get a budget approved.

4. Plan the agenda

Work with your point of contact to set the agenda. Aim to get dedicated time with several people who use the product. A one-hour session is usually enough to go deep and uncover useful insights. Also offer other formats, like a company-wide training or Q&A session, and let your POC guide you on what's most valuable for their team.

5. Deep prep

A few days before traveling, take a deep dive into how their users are actually using the product. Watch session recordings, look at the kinds of records they create, and try to spot patterns. For example, in the case of experiments, check which types of metrics they use most often, how many they create per week, and whether there are obvious points of friction. Alongside this, prepare a list of questions that can help uncover deeper insights during the sessions.

At the onsite

Run the sessions! Let users show you how they use the product in real time and as naturally as possible. Give them space to talk, ask questions to dig deeper, and don't be afraid to let the conversation go off on tangents, as those often reveal the most interesting insights.

In between the sessions you'll have opportunities to code. It makes sense to prioritize small improvements you can ship and demo on the same day – this kind of quick turnaround leaves a strong impression. It's also common for team members to approach you with questions, sometimes even unrelated to your product area. Be ready for this and go out of your way to help. Solving problems in real time is one of the best parts of being there in person.

After the onsite

Revisit the transcripts of all sessions – you should record everything in Buildbetter or a similar tool. Share what you've learned with your team and discuss if any quarterly goals need to be re-prioritized based on the learnings. Summarize the features or improvements you shipped during or right after the visit and send a thank-you message to the customer's Slack that lists them clearly.

It's important for visibility to notate on the account that a customer visit took place. We can do this in Vitally, by creating a new note on the account, with the "On-site" category and copying any known people on the account to be aware of the note content. This can be helpful in many ways even if no primary person at PostHog is managing their account today. BuildBetter calls should automatically attach to the account along with any existing email conversations.

Just as important is following up in the weeks after. Customers should feel the visit was worth their time, and a big part of that is quickly actioning the items they raised, even if it's just small fixes or clear updates on progress. This builds excitement and goodwill, shows immediate impact, and shows them it was worth investing their time with you. Most importantly, you'll walk away with a much deeper understanding of how your product is really used.

Working with PostHog AI

contents/handbook/engineering/working-with-max-ai.md

PostHog AI lets users interact with PostHog's products through a chat interface and other shortcuts throughout the platform. The is responsible for building and maintaining the AI platform.

What is PostHog AI?

PostHog AI enables users to:

- Ask questions about their data in natural language
- Generate insights and reports through conversation
- Navigate PostHog's products using AI assistance
- Automate common analytics tasks

We want Max to work with *every* PostHog product, so that we can at some stage make a chat (or even complete automation with approval steps as necessary) the default experience for people using PostHog, then resorting to clicking the UX as the backup option for *most* tasks.

How to work with the PostHog AI team

Products already integrated with Max always have a supporting engineer assigned on the Max team.

This naturally distributes AI knowledge throughout the organization, while ensuring high-quality AI features that integrate properly with the platform. Implementing something new, missing features in the Max API, or seeing failures? Your supporting engineer is your go-to! Tag them directly on your PRs and questions.

If your team is integrating AI features for the first time – the PostHog AI team will do their best to assign a supporting engineer.

Just message about your plans in the #team-max-ai channel in Slack.

Product	Supporting engineer on the PostHog AI team
Product analytics	
Data warehouse	
Session replay	
CDP	

Product	Supporting engineer on the PostHog AI team
[Insert your team]	Shoot #team-max-ai a message!

Getting started

If you need AI capabilities for your product area:

1. **Reach out early**: Contact the PostHog AI team lead at #team-max-ai in Slack to discuss your requirements
2. **Define the use case**: Be specific about what AI functionality you need, or consult with us if you're trying to flesh out ideas
3. **Plan the collaboration**: Work with the PostHog AI team to determine the best approach (e.g., sending an engineer to your team for a sprint or a few sprints vs. building the feature in PostHog AI directly without your involvement, or you just being able to do it solo)
4. **Coordinate sprints**: Align on timing and resource allocation if needed. This shouldn't feel like a heavyweight process, if it is - we should change it

Best practices

- **Start small**: Begin with simple AI features and iterate based on user feedback. A lot of automation can be broken down into smaller, automatable steps!
- **Avoid death by random AI widgets - maintain consistency**: Ensure AI features follow PostHog's design patterns and user experience standards. PostHog AI team can help if you are missing a UX pattern.

Resources

- PostHog AI team page
- PostHog AI objectives
- PostHog AI documentation

Contact

For questions about working with PostHog AI, ask in the `#team-posthog-ai` Slack channel.

Writing docs (as an engineer)

`contents/handbook/engineering/writing-docs.md`

Product engineers are responsible for writing and maintaining documentation for their products. This page is a guide to help you do this.

Ownership

High-quality docs require the expertise and context of the engineers building them, which is why *you* own your product's documentation.

Docs are extra important in the age of AI. All of our docs eventually make their way into the training data of newer foundation models. The quality and accuracy of your docs directly affect how people discover your product through LLMs.

AI search is our fastest-growing channel for user signups by far. So remember to update your docs and keep them up to date.

What about the so-called Docs & Wizard team?

The can help you, but they can't write docs for everyone. They are responsible for improving the docs as a knowledge base. This means:

- Reviewing and improving docs PRs created by product teams
- Shipping docs content based on prioritized feedback and emerging use cases
- Building tools and systems to improve baseline quality and structure
- Creating context services that power agents like the AI wizard
- Working on large scale docs projects

If you want their input, hit them up in `#team-docs-and-wizard` or tag `@team-docs-wizard` in GitHub.

They've also created a comprehensive self-serve guide on how to write product docs for you to use.

When should I start writing product docs?

Three great times to write docs:

1. When you ship a new user-facing product or feature. Write docs for big product launches before they release (during early access or beta). Smaller features and updates can wait until after they are shipped.

2. When you recognize a confusion or gap in users' understanding of your product. This could be based on support tickets, sales requests, or just user feedback.

3. When you update product behavior or interfaces. Check if the docs need to be updated with new information or instructions.

Basically, if users *could* self-serve and use your product, but aren't, you should write docs to help them do so. Write the obvious docs before users start asking you obvious questions.

> **What about features behind a feature flag?** If you are releasing a product to users, even a small number of them, you should write docs for it. Include what stage the feature is at (private alpha, beta, etc. is fine). This helps ensure users can successfully use it and provides the added benefit of drumming up demand from those who discover it.

What should I write docs about?

Docs should help people:

1. Get started with your product or feature. Installing it, setting it up, and finding it in PostHog.
2. Understand what your product does, including an as complete as possible list of features and their details.
3. Make the most of your product by detailing common use cases, concepts related to your product, answering common questions, and more.

Write the docs you would want to read if you were a user.

The has created a guide on how to write product docs for you to follow. It walks through how to structure and write your product docs in detail. Start there.

Where do docs live?

Nearly all our docs live on `posthog.com/docs`. You can find the repo to add and edit docs in the contents/docs directory of posthog.com. It uses file-based routing, so the folder and file structure is the same as the URL path. You can learn more about developing the website here.

Most docs should go somewhere in your product's section. Product docs usually have sections on installation, basic set up, key features, troubleshooting, common questions, and more. Docs for platform features like SDKs, data types, and PostHog AI live in the Product OS section.

> **Don't know where a doc or feature should go?** Ask in `#team-docs-and-wizard`.

What about internal docs?

If you can make something public, you should. Being open source is a core value of PostHog. We try to avoid "internal" docs as much as possible.

If it deals with private information, like security, customer data, or competitor analysis, use one of our internal repos like `product-internal`.

You can learn more about this in the communications handbook entry.

How do I document code?

Code should be self-documenting. If it's complicated to figure out, you probably need to make it simpler. This is especially important for APIs and interfaces that other teams will interact with.

For cases where code isn't self-documenting or easy to understand, include a `README.md` file in the directory that is closest to the entry point of the code.

This `README` should:

- Describe the general flow of interacting with the functions, but stop where the code starts to become self-documenting.
- Be short. If it's long, then your interfaces should be made simpler.

For an example, see the PostHog AI `README`.

Further reading

- How to use the content writer agent
- What nobody tells developers about documentation
- Docs style guide
- PostHog style guide

All-hands topic of the day

contents/handbook/exec/all-hands-topics.md

James presents a topic of the day each week in the company all hands. The main objective of this is to repeat and reinforce key messages:

- Make sure everyone knows what our mission is, and how their work contributes
- Make sure everyone knows what our strategy is, and how their work contributes
    - This can be company strategy, but also product, GTM, pricing, hiring etc.
- Reinforce good cultural behavior/our values

An element of repetition is important because a) we are regularly adding new people to the team, and b) just hearing a message once is not enough for it to stick. 'Repetition' does not mean literally saying the same words over and over again - it's more about finding examples of things people are doing or working on, and showing how those tie back to the bigger picture of what's important at PostHog.

We generally avoid using the topic of the day to announce new things, as these should be done async. Sometimes he will go deeper on a recent announcement, e.g. why we cut pricing for X.

Important topics to revisit regularly

- PostHog's mission - to help engineers build better products
    - How we're building an enduring company
- PostHog's overall strategy
    - Every tool you need to evaluate feature success
    - Get in first
    - Be the source of truth for customer and product data
- Principles around:
    - Which products to build and why
    - Who our ICP is and how to work with them
    - How to price products, inc. what we make free
    - How we do brand and marketing
    - How we do sales
    - How to hire well
- Our values and how to maintain PostHog's culture
    - The values themselves
    - Communication
    - Giving and receiving feedback
    - How small teams work

Annual planning process

contents/handbook/exec/annual-planning.md

This is the schedule for how we run various planning processes at PostHog throughout the year, together with explanations for what each thing is and who takes part. We intentionally keep things as light as possible, but have started introducing some slightly more structured processes around longer lead things like hiring and deciding which products to build.

Besides *very* high level financial forecasting, we don't plan out any further than 12 months, because things change and we don't want to feel locked into something that doesn't make sense. All 12 month plans are rolling, and we update them every three months at least.

Changes to the plan can happen outside of this schedule - this is a rough guide, not a strict timetable.

Month	Week 1	Week 2	Week 3	Week 4
January	Monthly accounts review	12 month product plan	12 month hiring plan	Monthly accounts review
February	Board meeting			Monthly accounts review
March		Q2 goal setting		Monthly accounts review
April		12 month product plan	12 month hiring plan	Monthly accounts review
May	Board meeting	Whole company offsite		Monthly accounts review
June	H2 financial re-forecast	Q3 goal setting		Monthly accounts review
July		12 month product plan	12 month hiring plan	Monthly accounts review
August	Board meeting			Monthly accounts review
September		Q4 goal setting		Monthly accounts review
October		12 month product plan	12 month hiring plan	Monthly accounts review

Month	Week 1	Week 2	Week 3	Week 4
November	Board meeting			Monthly accounts review
December	Financial forecast	Q1 goal setting		Holidays - keep empty

What each meeting does

12 month product plan

What: We update our rolling 12 month plan, which tells us what products to build next. This then tells us who we need to hire to support the plan. The output of this plan feeds into quarterly goal setting (below).

Who:

12 month hiring plan

What: We update our rolling 12 month hiring plan, which tells us who we need to hire beyond the current quarter. The hiring plan lives in Pry.

Who:

Board meeting

What: Quarterly meeting to update the board on progress and talk through 1-2 strategic topics. Board packs are stored in Google Drive.

Who: , occasional guest presenter

Org tidy up

What: Go through all the small teams, make sure everyone is happy and in the right place, make any changes needed to support new products/general scaling.

Who:

Financial forecast

What: Review the 3 year financial forecast, add another year. Tweak based on past performance, then check it is realistic, and keeps us on track. The forecast lives in Pry.

Who: Fraser,

H2 financial reforecast

What: Midway through the year check that the 3 year forecast makes sense, tweak if necessary

Who: Fraser, Charles

Monthly accounts review

What: Review last month's management accounts against budget. November's accounts review happens in January, due to the holidays in December, as we typically get our monthly accounts around the 21st of the following month.

Who: Fraser, Charles

Pay reviews

What: We run these 3 times a year. Not in the calendar as the times shift year to year and we want flexibility as we grow.

Who:

Quarterly goal setting

What: Blitzscale pre-meeting, then individual teams meet to run their own processes.

Who: , then team leads

Whole company offsite

What: Hopefully somewhere warm.

Who: Everyone

Blitzscale responsibilities

contents/handbook/exec/responsibilities.md

This page lists which teams each Blitzscale team member is responsible for.

Person	Teams
James Hawkins	PostHog AI, Signals, LLM Analytics, Conversations, Website, Code
Tim Glaser	DevEx, Growth, People & Ops, Talent
Paul D'Ambra	Product Analytics, Analytics Platform, Web Analytics, Replay, Client Libraries, Customer Analytics, Platform UX, Query Performance (currently forming)
Ben White	Batch Exports, Infrastructure, Workflows, Ingestion, Error Tracking, ClickHouse, Logs, Feature Flags, Flags Platform, Security
Raquel Smith	Experiments, Managed Warehouse, Surveys, Platform Features, Modeling, Data Tools, Warehouse Sources
Charles Cook	Marketing, Demand Gen, Product Led Sales East, Product Led Sales West, Billing, New Business Sales, Support, Customer Success, Onboarding, Docs Wizard, Editorial, YouTube, Forward Deployed Engineering, IRL Events

Not running out of money

`contents/handbook/finance.md`

Stay calm and default alive

We don't optimize for short-run revenue growth, but we do make sure we have enough money to never feel dependent on future fundraising.

If we average 5% MoM growth, we are default alive (i.e. we'll become profitable before we run out of capital). If we average 7.5% we'll hit $100m by the end of 2026.

Maintaining a strong financial position helps us optimize for long-term revenue growth. For example, we've removed products and revenue for long-term gains.

Fundraising principles

Rule #1: Never have to fundraise – and only fundraise if all the following are true:

- It will speed us up.
- We can use the money effectively.
- The partner would improve our board.
- The increased chance of success offsets dilution.
- It reduces stress.

How do we spend it

PostHog grows by shipping, whereas most software companies grow linearly with the number of salespeople they hire.

The advantage of our approach is that it's more efficient – $1 spent on product will *forever* improve things, unlike investing $1 in cold calls. We can easily choose to be profitable if we just sit default alive and let revenue grow "automatically" based on the product we have already shipped.

The disadvantage is that scaling an engineering team is, in our opinion, harder than scaling a sales team. Since engineers' work very heavily overlaps, there is more complexity to getting this right. We may not be able to grow beyond a certain rate, no matter how much we spend.

The final disadvantage is that it's harder to predict how fast we'll grow compared to a company that grows by hiring salespeople with targets, so it takes more thought and often requires more faith!

Future

contents/handbook/future.md

TL;DR: Mid term, it's $100 million ARR by 2026, working backwards from there. Longer term, outcompete top-down competitors worth $50 billion. If we get that far, we'll have helped *tens of millions* of engineers build better products.

Will PostHog sell?

What motivates us is building an epic product and company.

We're excited by:

- Figuring out how far we can go
- Helping engineers build products - reduce the need for product and data teams
- Beating all the point solution competitors
- Having customers buy *from* us instead of us selling *to* them

We're *not* excited by:

- Selling the company for $1 billion – we think we can build a much bigger company by staying independent

$100M by 2026

We want to hit $100 million in annual revenue by the end of 2026.

We've set this since it's ambitious and keeps us accountable for some kind of financial output, but working backwards we can do it. We need around 7% monthly revenue growth to hit this.

Secondaries over selling

Everyone at PostHog has options in the company – that means they can be a shareholder. This keeps everyone focused on our long-term interests.

However, at different stages of the company's life, the value of each person's stock may be hundreds of thousands, millions, or even tens of millions of dollars. If someone doesn't have much capital but has $10 million in stock options, they'll start wanting us to sell.

As a result, we aim to let people sell some of their stock when we fundraise and once their stock has very significantly increased in value. This helps keep everyone focused on building something bigger and longer term. What we can offer will depend on what we can negotiate every time we fundraise, but this is our general philosophy.

Meetings

contents/handbook/getting-started/meetings.md

We are anti-meeting by default.

However, while we default to written and asynchronous communication, we find that having a few regular touch points for the whole team to come together on a call useful for sharing certain types of information, strengthening our culture and discussing more dynamic issues in real time.

We keep these minimal in terms of time expectation - no more than 2hrs total per week. They are usually scheduled around 8.30am PDT/4.30pm GMT to allow people across multiple timezones to attend more easily. We default to cameras on, it's nice to see real faces since we don't get many in-person moments. If you need yours off, just give the team a quick heads-up why.

You should have been invited to any relevant meetings as part of your onboarding.

Weekly schedule

- **Monday** - PostHog News all-hands meeting. Members of the team share company-wide updates about things like recruitment, product metrics and commercial performance - the doc is shared in the #general channel in Slack. We then go around and people are free to demo anything they've been working on recently. *The content of these meetings is always confidential*. All hands meetings are recorded too if you are out. Some teams also do sprint planning on a Monday.
- **Tuesday** - Meeting-free - no planned internal meetings allowed. Learn more.
- **Wednesday** - some teams do sprint planning here as well. Engineering tech talks/brown bags happen every second week, ClickHouse office hours happen on the alternate week.
- **Thursday** - Meeting-free - no planned internal meetings allowed. Learn more.
- **Friday** - extracurricular type meetings like BookHog often end up here!

The all-hands

The Monday all-hands features a few regular sections and is recorded in this document.

- **Announcements:** Revenue and churn updates, plus other major news
- **Hiring:** Updates about headcount, who is starting soon, and new hiring roles
- **Acknowledgements:** Opportunity to give kudos to your colleagues
- **Topic of the day:** Exec team talks around a particular topic
- **Q&A with James & Tim:** Ask the founders anything you want
- **Demos:** Show us what you've worked on last week

How to give a good demo

Demos are a great way to share what you've been working on and keep everyone in the loop. A little prep goes a long way toward making them useful and respectful of everyone's time. It also stops the meeting length getting out of hand!

- **Test your setup before you start.** Make sure screen sharing and your microphone work so you can dive straight in. A quick check a few minutes before the meeting avoids awkward fumbling and keeps the energy up.
- **Keep it short and purposeful.** Aim for quick, concise demos. It can help to write down some bullets to structure your demo ahead of time. Lead with why what you built is useful or interesting — that context helps people stay engaged and can be especially useful for less technical teams.
- **Don't rely on real-time.** If your demo involves triggering other services (e.g. sending emails or calling APIs), cue them up in advance or use mocks. Live demos are fun when they work but can eat into everyone's time when we're waiting for something to load.
- **Arrive with energy.** We don't expect polished sales pitches, but it's always more engaging to listen to someone who is excited to show what they've made.
- **It's okay to skip or shorten.** If something isn't working or you run out of time, feel free to skip it and move on. You can always share a Loom in #tell-posthog-anything so people can watch when it suits them.
- **Be ready when it's your turn.** Pay attention to the order of raised hands and be prepared to speak when you're up. It keeps the flow smooth and shows you're tuned in to the group.

When in doubt: a short, clear demo that explains the "so what?" beats a long one that leaves people wondering why it matters.

No meeting days (Tuesdays & Thursdays)

We try to keep these days focused on deep work. Therefore, we run no planned meetings on these days.

However, speaking ad-hoc to your teammates on this day is fine - especially:

- new people shouldn't worry about following this rule for the first couple of weeks - it's more important you get up-to-speed quickly
- if it's obvious you need a meeting

If ad-hoc meetings are regularly happening, consider improving the agenda of another regular meeting so there isn't as much context switching in people's days.

People in customer-facing roles where being on calls is a bigger part of your job don't need to stick to this as much, but please don't loop in engineers to customer calls on these days if you do by default.

Sprint planning

Each small team runs its own sprint planning meetings on whatever schedule you feel is most useful. Some teams do this on a Monday, others on a Wednesday, and sprints are usually 1-2 weeks long. We split into Small Teams for these. If a product team, your team's exec will also attend.

All sprint planning meetings are open to anyone to attend - if you are not a member of that small team, then we ask that you sit in as a non-speaking observer only.

Customer billing configurations

`contents/handbook/growth/billing/customer-billing-configurations.md`

This document outlines all possible billing configurations for customers at PostHog. The goal is to ensure the team is on the same page with the different configurations we support to ensure things move smoothly as we scale. We want to ensure they support the billing repo, dashboard, usage reports, revenue reporting, etc.

Below are the main 5 configurations we support right now. Each outlines how the Stripe accounts are setup and billing and how we account for revenue on them.

1. Free plan customers
    - We don't need to worry about these users because they aren't paying anything, even if they have a Stripe account.
2. Paid plan customers
    - Regular user on a paid plan with no credits.
    - Pay the invoice directly with no funny business
    - Can be with or without tax.
    - Every line item on the invoice is a product with a product key
    - Should be using default products/prices
    - Should only have 1 stripe account and 1 subscription
    - **Details**
        - mrr = sum(mrr products) + tax
3. Start-up plan customers
    - Startup plan where users receive credits (e.g., $50,000).
    - Credits apply to all charges until the credits run out.
    - Credit usage needs to be tracked.
    - Metadata added to Stripe account (`is_startup_plan_customer`, `credit_expires_at`, etc.).
        - There is an RFC in the works to update this metadata for better tracking.
        - We are going to revisit this process.
    - Revenue is not earned until credits are depleted or expired.
    - Can be with or without tax.
    - Should be using default products/prices.
    - Should only have 1 stripe account and 1 subscription.
    - **Details**
        - mrr = 0 while on credits
        - mrr per product = 0 while on credits
4. Enterprise customers (yearly credit purchase)
    - Enterprise customers pay an invoice for credits (before the subscription is created).
    - Once the invoice is paid, the subscription is created by CS.
    - Much of this is done via Zapier. See the docs for more info.

    - Credits apply to their usage (including the Teams addon - up to the discretion of the CS team is that's charged for).
    - Credits reduce product charges on invoices.
    - Can be with or without tax.
    - Should be using default products/prices.
    - Should only have 1 stripe account and 1 subscription.
    - **Details:**
        - mrr comes from the credits payment - yearly upfront payment (we split these)
        - mrr per product comes from the actual usage in that month (minus the credit-discount-percent on the customer)
5. Enterprise customers (amortized credit payment)
    - Similar to above, where an enterprise customer is paying for credits. This is the case where they commit but pay monthly.
    - That means they need two customers in Stripe - one for the credits and one for the usage.
    - Currently, this also means they have two customers in billing. See more below on "unsupported configurations" for how this will change.
    - The credits are charged on a recurring cadence and tracked as MRR - cadence could be monthly, quarterly, semi-annually, during the fourth phase of the moon on the second sunday of the festival of Saturnalia, etc.
    - The usage is tracked by another stripe customer with its own subscription - where credit is used against invoices.
    - Can be with or without tax.
    - **Details:**
        - the customer will have two Stripe customers, each with a subscriptions
        - the mrr comes from the credits subscription on one customer
        - the mrr per product comes from the usage subscription (paid by the credits) on the other customer (minus the credit-discount-percent on the customer)

Legacy configuration

Note: this above list is focused about the creation of new customers going forward - there are many existing configurations not covered directly in this document.

Unsupported configurations

While we don't currently fully support these, we would like to soon:

- 2 Stripe Customers, each with 1 Subscription There is another RFC outlining the current limitations in the works.

Cross sell motions

contents/handbook/growth/cross-selling/cross-sell-motions.md

Problem statement

We haven't had a specific playbook/motion/plan on how to do cross sell, until now!

CS & TAM managed accounts historically have only been *slightly* better than average when it comes to product adoption. We can change this. We have the technology. We have the power.

Described here are some of the current goals and tactics to improve effective cross-selling measures when working with customers accompanied with specific how-to guidance.

Goals

The main objective is to get existing PostHog customers to adopt more of the platform. We firmly believe that adopting more products leads to a better experience and higher satisfaction with PostHog.

For a TAM today, a quantitative goal is to move from an average 4.8 products adopted currently to 6 products adopted for AM managed accounts over the next two quarters. Accounts may be promoted to CSM coverage and continue with the adoption plan.

AEs and CSMs goals might be polar opposites. Where AEs may want wide exposure, it's also important to establish the right time for product adoption, rather than overselling something that may not apply to the initial implementation.

CSMs are not here to push new products and features; CSMs are here to ensure customers successfully use PostHog and get the most value for their business. Remember, we want to help our champions look like heroes at their companies!

Results

Cross-selling has clear expected outcomes:

1. Increase Revenue / product usage
2. Increase stickiness
3. Offer real value of the "platform" to users

Measuring success

Successful expansion strengthens customer relationships and increases account stickiness. Each additional product that delivers value makes PostHog more integral to the customer's operations. There are a number of things we can look at that deliver these results.

- Number of business value conversations

    - These could be QBRs or other one off calls. We want to increase the number of calls that are primarily discussing business value
- Number of product trials started and completed and the time it takes to adopt
- Number of in-person visits
- Product adoption quarter over quarter
- Churn rate by total number of products
- Churn rate by specific products i.e. are there products that once adopted lead to a noticeably lower (or higher) churn rate.
- Customer feedback on expanded products
- Percentage of revenue from a customer spread across products
    - Is all the revenue for a customer coming from person profiles or is is spread between recordings, events, feature flags, and exceptions

> Since we have different folks at different stages of ramp and onboarding, instead of making these metrics flat or percentage based, we are looking for an increase quarter over quarter.

Accounts that are a good fit

As a part of this process you are determining whether they are a good opportunity for cross-sell motions and if they are representative of an ideal candidate for growth. Here are some key qualities we've found to date:

- Smaller / startup size accounts that don't have existing tooling and can grow with PostHog - it is much easier to grow into using multiple products than it is to try to supplant an existing product.
- Engineer heavy - direct technical contacts that have influence in adoption
    - Product engineers
    - Technically minded leadership like CTO
    - Technically adept product manager
- Pushing the limits of PostHog - support engagement isn't "how do I do this simple thing" but "how do I tackle this complex concept"
- Heavily engaged users - we'd prefer 10 heavily engaged users over 100 low engagement accounts
- Volume of product use tied to revenue - does an increase in PostHog usage correlate to increase in revenue for them?
- Ability to ship quickly
- Being open source is a plus - it gives us insight into their implementation as well as their adoption of other tools

Optimal timing for discussions

Why Now? All good opportunities have this and are timeline driven.

Ideal moments:

- Quarterly business reviews when setting future goals
- After successful outcomes with current products

- During team growth phases (new stakeholders bring new needs)
- Credit renewal conversations (bundling opportunities)
- When customers mention relevant challenges organically
- Following positive business announcements (follow your accounts company pages on Linkedin, set Google alerts, etc)

Times to avoid or pause cross-selling:

- During active support issues or incident
- When current implementation needs attention
- Within first 30 days (unless customer-initiated)
- Following budget constraints announcements

- Complaining about price, actively seeking reduction
- Usage declining for 4+ weeks
- Key champion leaves company
- Data Engineer takes over ownership of PostHog

Optimal timeline if a customer is on an annual contract:

- **Months 1-2**: Pure adoption focus, no cross-sell
- **Months 3-6**: Prime cross-sell window
- **Months 7-9**: Reinforce value of expanded stack if adopted
- **Months 10-12**: Focus on "Why stay with PostHog" rather than expansion/cross-sell (it's too late)

Hypothetical approach

One way of approaching this that we have seen work is a research -> QBR -> recommended cross-sell

1. Account research and understanding the business
2. Some sort of engagement (QBRs?) to understand business priorities and tie them to PostHog
3. Make specific recommendations around what to adopt and how it will help with business priorities

For example, customer B2C SaaS has a business model selling subscription plans. Digging in to understand the differentiators of the plans and reviewing their custom events to ensure they are collecting the appropriate data. Come to the QBR ready to discuss the particulars of their situation. You may or may not have the info you need to make a recommendation on the call, but at the very least, you should have a direction to suggest. You could recommend customer/revenue analytics, experiments for plan adoption, and surveys for user feedback given what you know about their business model.

> This doesn't always need to be a formal QBR process. Some form of research -> discovery/interaction/recommendation is the basic flow here.

The why evolve framework for cross-selling

1. **Document Results**

    - Start every cross-sell conversation by quantifying their wins with current PostHog products
    - Example: "Your team has tracked 50M events, identified 3 major UX issues that were costing 12% conversion"
2. **Highlight Evolving Pressures**
    - Frame new needs as natural progression, not disruption
    - "As your user base grows internationally, you're facing new questions about region-specific behavior patterns"
3. **Share Hard Truths**
    - Be transparent about gaps without undermining current success
    - "Your analytics show what's happening, but your team still spends hours in user interviews trying to understand why"
4. **Emphasize Risk of No Change**
    - Show what they miss without additional products
    - "Without session replay, you're making UX decisions based on incomplete data"
5. **Describe Upside Opportunity**
    - Paint vision of complete analytics stack
    - "You'll move from guessing why users drop off to seeing exactly what frustrated them"

The new cross-sell motions playbook

You've been here awhile and just want the script. Wet get it. The following sections describe the actual approaches that fit well within PostHog for a cross-sell motion, and can be pitched in grouping by feature or by user needs.

Remember that where possible, we're providing solutions and outcomes rather than features. Today we have clear examples with the Error tracking product where customers have found success, with more direct playbooks in the works.

Common cross-sell and expansion paths: Adoption paths are good to frame products as point in time or natural progression to their implementation. For example:

- Product analytics + error tracking
- **Product analytics → Session replay:** When customers struggle to understand user behavior from metrics alone
    - "You mentioned spending hours debugging that checkout issue last week. Session replay would show you exactly what users experienced."
- **Customer facing teams → Session Replay:** When organizations need better user troubleshooting and support, not just user research into behavior.
- **Any product → Feature flags:** When teams need safer deployment strategies
    - "That rollback you had to do last month affected all users. Feature flags would have limited the impact to just 5% of traffic."
- **Feature flags → Experiments:** When customers want to test and evaluate results from feature flags; a very natural synergy here, of course
- **Experiments → surveys:** run A/B/n tests and offer surveys to back up / validate insights

- **B2B companies → Group analytics:** When tracking company-level metrics becomes critical
  - "You're currently exporting data to spreadsheets to analyze customer behavior. Group analytics provides those insights natively."
- **High event volume → Identified events:** When anonymous events limit user journey analysis
- **Growing teams → Teams add-on:** When organizations need advanced permissions and SSO

Here are some other known examples that aren't necessarily 0 to 1 linear adoption, or working backwards from what a customer is using outside of PostHog:

- Data warehouse has continued to receive special attention since the launch of PostHog's related products.
  - LLM analytics + data warehouse - enrich LLM analytics with data from other sources like Stripe or Supabase
  - Customer/Revenue analytics + data warehouse - natural fit between connecting up stripe and enriching data further
  - Read a hot topic on churn in high growth customers for specific advice
- Feature flag + mobile replay - use for sampling/roll out that is not natively supported by mobile replay
- Experiments / feature flags + error tracking - insight into errors for new/beta features, and seeing the impact of those errors on conversion rates is valuable
- Feature flags + LLM analytics - ability to granularly segment features based on cost/engagement of users. i.e. you can release higher cost models to users who have already shown a willingness to spend

Bundle "Features"

Bundling is another good way to position products by customer type and stage. The following product stacks match certain types of user needs with value.

The "early stage growth" stack

Products: Analytics + Session Replay + Surveys
Value story: "You know what users do, see how they struggle, and can ask them why"
Ideal for: B2C companies with conversion optimization focus

The "ship fast without breaking" stack

Products: Feature Flags + Error Tracking + Experiments
Value story: "Roll out safely, catch issues instantly, measure impact scientifically"
Ideal for: High-velocity teams with continuous deployment

The "revenue optimization" stack

Products: Analytics + Experiments + Revenue Analytics (via Data Warehouse)
Value story: "Track user behavior, test pricing changes, measure revenue impact" **Ideal for**: B2B businesses focused on LTV/CAC

The "vibey AI startup" stack

Products: Analytics + Flags + LLM Analytics + Error tracking **Value story**: "Tie user behavior to run cost, launching features that are both user requested and revenue generating" **Ideal for:** AI-focused startups optimizing for cost efficiency and user engagement

What to look for when cross selling

We've already seen general indicators that are worth paying attention to when it comes to success cross-selling and here is an expanded list with what to do next.

1. New PostHog product launch - did we launch a product that is a good fit for their use case? Did we add a new data pipeline source or destination?
    1. Reach with details on the new product
    2. Offer to credit them back their first month of usage so they can try it out risk free
2. Raising funding - did the customer just raise money?
    1. Congratulate the founder on the raise
    2. Lead with product that can capitalize on their opportunity to bring in more revenue / usage
        1. i.e. if they are B2C, pitching experiments to maximize conversion
3. PostHog price change - did we change pricing to make adoption more palatable?
    1. Let your main point of contact know how much they will save with the new pricing if they currently use the product
    2. If they don't suggest adoption based on the new rate and offer credits to offset learning curve
4. Revenue increase - is the customer seeing an increase in revenue?
    1. Depending on how you know about it, either congratulate them (or don't)
    2. Recommend a product that would capitalize on that revenue
        1. Error tracking to clean up issues
        2. Feature flags to launch new user features
5. New customer product launch - is the customer launching a new product that could benefit from additional PostHog goodness?
    1. Check out the product yourself (if applicable)
    2. Congratulate them on the new launch
    3. Suggest products that would help with the success of the new launch
        1. i.e. surveys for feedback, feature flags for new features
6. Competitor drops (or lacks) SDK support - does a competitor lack critical support or have they dropped support?
    1. Reach out proactively to main technical contact if there is overlap
    2. Mention our support (and lack of competitor support)

3. Send any pertinent docs
4. Follow up regularly with status updates and additional resources

7. Eng/marketing hiring - is the customer hiring more technical roles? Could we do this through LinkedIn?
    1. Prep PostHog onboarding for new user
    2. Offer call / support for getting them up to speed
    3. Suggest products that make that new hire's life easier
        1. i.e. error tracking to figure out where the gremlins are
8. New users from other business units - are we aware of / seeing people from other parts of the business asking about (or even using) PostHog?
    1. Make note of who the new users / units are
    2. Ask for a warm intro from current main point of contact
    3. Reach out 1:1 to new users to get feedback / offer help
9. Customer expanding into new geography / territory - is the customer moving into a market they weren't previously in?
    1. Ensure they are capturing the correct custom events / properties
    2. Pitch products that help with differentiating location experience
        1. i.e. feature flags for unique features based on GeoIP
10. When an owner leaves PostHog or a new owner is added - is the new owner open to other products that can help solve the problems they care about?
    1. Reach out to new owner to understand their priorities
    2. Hit any products that were previously suggested to the other owner
    3. Offer credits for adoption of the new product
11. Shift in customer business model - is the customer introducing a new type of subscription, going from on-prem to cloud, changing their fundamental offering?
    1. Dig in to understand the changes
    2. Suggest flags / experiments as a good way to get feedback / modify the experience for the new model

Alerts

What alerts would be helpful to have that would indicate good cross sell opportunities. Continue to question what would be useful to follow in order to positively influence timing.

1. Could we use our PostHog to flag when an account's revenue is increasing on their end? (not spend with PostHog, but their actual revenue)
2. Could we use signals in Vitally / PostHog to notify about new power users?
3. Could we get an alert when an account tries a new product for the first time?

Discovery through conversation

Effective discovery focuses on understanding customer challenges rather than pushing products.

Example questions to ask

The questions below are designed to spark thoughtful conversations with customers. They help uncover how teams are currently solving problems and whether there might be simpler or more effective ways to do so using PostHog.

Use these questions in preparing for calls and use them as examples for developing your own questions. Each includes the **question**, the **pain revealed**, and the **PostHog advantage**.

High-value discovery questions for upsell/cross-sell:

- "How does your team decide what to build next?"
- "What's your process for investigating customer-reported issues?"
- "How do you measure feature adoption across different customer segments?"
- "What does your deployment process look like for major changes?"
- "How do you validate that new features impact key metrics?"
- “Are there other team members on different teams you could introduce me to or that you recommend I reach out to?” (Always, always, always be asking for “referrals” in this way!)

These questions will naturally surface use cases for session replay, feature flags, experiments, and other products. We should also identify opportunities programmatically through the other data sources we have to supplement the conversation approach. It's not a recommendation to ask each and every one of these questions on a call. These are simply a guide and an example of the types of questions that will help surface opportunities.

Error tracking

Question	Pain Revealed	PostHog Advantage
When an error occurs, how easy is it for you to see exactly which user actions led up to it and how it affected the experience?	Debugging relies often relies on reproducing error	Error Tracking tied directly to replays makes root cause and impact obvious.
If you’ve built your own error tracking, how much effort goes into maintaining and correlating it with analytics?	Time wasted maintaining infra, blind spots in analysis.	Lightweight SDK that's tightly integrated with other products.
How do you decide which errors to fix first?	Prioritizing by gut feeling or frequency, not business impact.	Error Tracking + Product & Revenue Analytics can show which errors have the greatest impact.

> For more recommendations, look at the Error tracking motions

Session replay

Question	Pain Revealed	PostHog Advantage
When debugging, how often do you rely on logs or secondhand reports to reconstruct what happened?	Time lost piecing together events.	Session Replay shows exact user journey, reducing guesswork.
How do you confirm if a bug is isolated or widespread across users?	Hard to prioritize fixes without scope clarity.	Replays + analytics show impact
How do you identify user friction today?	Lacks visibility into real interactions without PM background.	Session Replay gives direct user perspective for product calls.

Feature flags

Question	Pain Revealed	PostHog Advantage
When launching a new feature, how do you manage risk of rollouts failing?	“Big bang” releases increase risk + stress.	Feature Flags enable safe, gradual rollouts & rollbacks.
How do you measure whether users actually engage with a feature once it’s enabled?	No feedback loop between rollout and usage metrics.	PostHog connects flags directly to analytics & experiments.
What’s your process for debugging an experiment if users drop off unexpectedly?	Experiments may fail without clarity on root cause.	Session Replay + Error Tracking pinpoint where the experience broke down.
How do you currently measure the business impact (e.g., revenue, retention) of an experiment?	Results limited to engagement metrics, missing real business outcomes.	Revenue Analytics + Product Analytics + Data Warehous show both engagement and business impact.

Customer/Revenue analytics

Question	Pain Revealed	PostHog Advantage
How do you measure the direct revenue impact of your features?	Work disconnected from business outcomes.	Revenue Analytics ties feature usage to revenue & LTV.
How do you weigh roadmap decisions against revenue impact today?	Guesswork in prioritization.	Revenue Analytics reveals which features drive business outcomes.

LLM analytics

Question	Pain Revealed	PostHog Advantage
When your LLM-driven features underperform, how do you pinpoint why?	No clear visibility into model errors or user friction.	LLM Analytics shows usage, performance, and cost data together.
How do you know which LLM features are helping vs hurting users?	No clear way to measure LLM impact on user behavior or business outcomes.	LLM Analytics + Session Replay shows which interactions drive value vs cause drop-offs.
How do you evaluate your LLM analytics in the context of broader product goals?	Standalone tools miss product context.	Integration ties LLM performance to actual product outcomes.

Surveys

Question	Pain Revealed	PostHog Advantage
When analyzing survey responses, how easy is it to connect them to specific user behaviors or outcomes?	Responses are siloed, making it hard to correlate feedback with analytics or events.	Surveys integrate natively with Product Analytics and Session Replay, linking responses to user journeys and metrics.
How do you target surveys to the right users without manual segmentation or guesswork?	Less targeted surveys lead to low relevance and response rates. Custom targeting requires dev time.	Display conditions use cohorts, feature flags, and events to show surveys only to specifics users, with built-in response limits.

Expansion within existing product usage - and up-selling

It's worth calling out a question again: are we selling more of the thing, a more expensive thing, or a new thing? Cross-sell and expansion opportunities can have significant overlap in product plays.

If we're planning expansion, the best way to do this is to replicate usage of existing product with *new* teams at the same company. This is a bit more straightforward conceptually, and may be harder to execute because you're likely to be starting with a new team from scratch.

You may want to consider expanding usage of the same product within the same team if there is obvious scope to do so here. This can also be difficult as it depends on the individual success and growth of their product, which you can't control.

Trial/Evaluation incentives

If we want customers to use more products, we should incentivize new product adoption. This could be in the form of credits for a specific timeframe to cover adoption and usage of the specific product. For example, if a customer wants to try out data warehouse, we offer 2-3 months of credit for any data warehouse usage as they figure out how they would use it and where it provides additional insight.

We have opportunities to get creative with how we incentivize new product adoption with users. A few ideas are:

- Bring them over at competitor pricing for X months
  - We could eat Launch Darkly's lunch
- Free trial / $0 product usage for X months
  - Related to the above suggestion for credits, this would be a more "on rails" approach
- Give them additional credits on top of their new product usage
  - If they adopt data warehouse, don't just cover their usage, give them an additional 5% for each new product adopted
- Could we offer in app notifications about good combinations of products?
  - If a user is using feature flags heavily, we should suggest experiments
- Easier migration for competitors products
- Each additional paid product adopted above 3 adds 5% discount

Error tracking cross sell

contents/handbook/growth/cross-selling/error-tracking-cross-sell.md

Identifying Error Tracking cross-sell opportunities

Our first example here is for cross-selling Error tracking, which generally has the below requirements. Feel free to copy this as a format for other bundle motions where applicable.

Product specific pre-reqs

- You understand their engineering processes and timelines (at least on a high level) and expect them to have resources available to look into Error Tracking.
- They have implemented PostHog Analytics (and ideally Session Replay) into their application(s) and are actively using those features.
- They are using one of the supported platforms for Error Tracking
- You know which teams to talk to regarding Error Tracking, you have identified the people best suited to successfully implement Error Tracking.
- They have expressed pain points that Error Tracking can help resolve.

Motion

1. Qualify if an account is suitable for this motion by understanding how they detect, prioritise, and fix errors today. If your stakeholder can't answer or isn't interested, find another stakeholder. If the answer is that don't have a tool to do this, don't link their errors to impact on key user actions within their app or that they prioritise based on error volume or another metric that is equally uncorrelated with impact on UX, this is a good opportunity for this motion.
2. Suggest that they turn on exception capture for the relevant environment, with a billing limit or free trial set so there is no cost. In exchange offer to find the impactful errors they are missing and help them move towards a UX based methodology for error prioritisation by combining errors with PostHog product analytics data.
3. Create dashboards of the new error data that correlates errors with drop-offs in conversion events (signups, checkouts, whatever is relevant here)
4. Share your analysis as a Loom or other low time commitment format for review, emphasising the uplift in conversion events if these errors were prioritised. If required present these findings back to the stakeholders.
5. If required help your stakeholder build a business case for the additional spend by linking the missed conversion events to value. For example, if the average LTV of a signed up user is 50$, multiply the dropoff in sign up events by 50 to get a rough ROI of finding and fixing these errors.
6. Pitch this value as a reason to remove the billing limit and expand usage of error tracking.

Product usage signals

Customers don't always ask for Error Tracking directly, but their usage patterns can indicate a potential need. When you review customer accounts and chat with users, look for these signals:

- Using session replay to investigate user issues, suggesting they don't have a way to detect errors automatically.
- Frequently searching through event logs or funnel drop-offs to find technical issues or user drop points.
- Setting up alerts primarily on custom events (rather than exceptions), which could indicate they lack out-of-the-box error visibility.
- Creating dashboards or insights that combine product usage with support data, showing a need to correlate bugs with user experience.
- Tagging engineering or support teams in insights to ask them to "investigate".
- Using manual workarounds to monitor application health, such as exporting incidents from PostHog to spreadsheets or other tools.
- Asking how to attribute support tickets or complaints to specific user sessions, which could be easier with automated error tracking.

Chat with users

Engage with users. Look for cues that signal gaps that Error Tracking can fill:

- "We keep seeing bugs in production but it's hard to know where they're coming from"
- "I'm not sure if we're catching all the errors our users encounter"
- "We use logs to try and track down issues, but it's pretty manual"
- "We get complaints from users, but it's hard to reproduce what happened"
- "We only find out about errors when customers report bugs"
- "Is there a way to get notified when critical errors happen in real time?"
- "We need to understand how errors are affecting our revenue or user growth"
- "It's difficult to connect the dots between bugs and their actual impact on users"

Website signals

- Visiting docs pages, such as:
  - https://posthog.com/docs/error-tracking/start-here
  - https://posthog.com/docs/error-tracking
  - https://posthog.com/docs/error-tracking/installation/react
- Visiting tutorial pages:
  - https://posthog.com/tutorials/react-error-tracking
  - https://posthog.com/tutorials/error-tracking
- Visiting the error tracking product page (https://posthog.com/error-tracking) and clicking "Get started - free"
- Reading blog posts about error tracking:
  - https://posthog.com/blog/posthog-vs-sentry

    - https://posthog.com/blog/best-sentry-alternatives
- Asking PostHog AI about error tracking
- Asking MCP about error tracking

Discovery questions

When reviewing accounts, ask:

- **Product feedback**: "How have you been using session replays and is there anything you would like this product to do differently?" This can reveal gaps that Error Tracking might fill.
- **Incomplete implementations**: Are there any products a customer started to configure but didn't finish? Understanding why can highlight pain points Error Tracking could address.
- **Product churn**: Are there any products a customer used and then stopped? Understanding why can help identify if Error Tracking could solve the underlying issues.
- **Decision-making timeline**: Are they doing annual engineering roadmaps or quarterly goals? This helps you time cross-sell conversations appropriately.
- **Competitive landscape**: Use the SDK scanner to check if they're using a competitor for error tracking, which can help you position PostHog's integrated approach.

Demonstrate the value

Once you've identified customers who'd benefit from Error Tracking, show them value in ways relevant to them.

Product Analytics

A few good starting points:

1. **Track error trends over time**: Create a trends insight for `$exception` events and create alerts when errors hit specific thresholds. You can get both historical trends and real-time notifications on high-impact exceptions to prioritize engineering work.
2. **See if errors are affecting conversion**: Combine errors with funnels to figure out if drop-off is happening because of errors – especially if errors are blocking users from getting through critical flows. You can tie this to customer lifetime value to show potential revenue loss. This is also useful for experiments - you want to make sure your variant didn't underperform because of a bug rather than the actual feature you're testing.
3. **Measure retention impact**: Track whether users who hit errors come back less frequently.

For all of these, you can layer on data like `$exception_types`, `$exception_values`, or `$exception_sources` to figure out which errors are most common and how they're impacting users.

Session Replay

Session Replay and Error Tracking work wonderfully together – probably the strongest integration we have. You can watch recordings of what users are doing in your app and get clear signals of errors they're hitting. You can search for specific events, jump straight to a given issue, and see what happened before and after – all of which provide valuable context for debugging.

When viewing a session, use the "Only show matching events" toggle to filter by exception-related events. You can use `$rageclick` to identify UI frustration that correlate with errors – this helps highlight silent frustrations users are experiencing that otherwise aren't communicated. You can also create dynamic cohorts of impacted users and take actions on them.

Other use cases

- **Feature Flags**: Roll out or revert code updates based on users who've hit specific exceptions. This lets you quickly respond to errors by targeting affected user cohorts and minimize impact if users are having a bad experience. Feature flags can act as kill switches – quickly turn off problematic features without deploying changes.
- **Data Pipeline**: Set up custom destinations to send your error tracking exceptions to other sources if the built-in alert function isn't enough.
- **AI**: Leverage PostHog AI or Claude Code to help diagnose, summarize, and prioritize issues based on impact.
- **Surveys**: Use the capture exception template to request feedback from the user when they encounter errors.
- **Error tracking integrations**: strengthen adoption of PostHog's error tracking by integrating with external issue tracking the customer is already using.

PostHog vs other error tracking

Historically, error tracking is something only engineering teams use. With PostHog, there's deliberate value for other teams. For example, marketing can figure out why conversions dipped and look at Session Replays tied to errors. This is incredibly valuable to quickly identify blockers. Other error tracking tools might give you clarity on bugs and errors, but PostHog gives you the complete picture of the user journey.

Common blockers

“This increases costs that we didn’t budget for”

We should proactively give credits so customers can trial a new product. For example:

- Free trial: give credits to cover usage of a new product for X weeks / months – make sure this is timeboxed!
- Match competitor pricing: if a competitor is significantly cheaper than PostHog, verify this and offer to bring customer over at competitor pricing for X months
- More credits: offer to give additional credits on top of new product usage

“My champion doesn’t make decisions on this product”

You should first try to build a relationship with the persona that will be users of the product. For error tracking, this will be engineers who work on areas where exceptions are critical (link to persona page).

Ask your champion how they are currently tackling the common use cases. Identify team members you want an introduction to and ask your champion for a warm connection. You can position it as a learning opportunity, asking for feedback, or a pitch (if you have a really strong understanding of the specific value-add). Help your champion with the internal pitch.

For error tracking, these questions are helpful to start the conversation:

- When an error occurs, how easy is it for you to see exactly which user actions led up to it and how it affected the experience?
    - Debugging relies often relies on reproducing error
    - Error Tracking tied directly to replays makes root cause and impact obvious.
- If you’ve built your own error tracking, how much effort goes into maintaining and correlating it with analytics?
    - Time wasted maintaining infra, blind spots in analysis.
    - Lightweight SDK that's tightly integrated with other products.
- How do you decide which errors to fix first?
    - Prioritizing by gut feeling or frequency, not business impact.
    - Error Tracking + Product & Revenue Analytics can show which errors have the greatest impact.

If you’re not sure who the persona should be, ask the product team!

“I don’t have the resource or time to implement error tracking”

Position implementation as simple, especially if you’re asking your customer to try out a product for the first time. This is where you shine as a technical success person. Help your customer cut through the cognitive load of figuring out implementation.

Error tracking can be implemented with one click, or two lines of code*(*depending on the SDK). Hyperlink to the project settings to enable exception autocapture or share the snippet addition for the SDK they’re using. Follow up with a rough plan that is tied with their needs, such as:

1. Enable exception autocapture – see events flow through
2. Assess the errors, issue groupings – decide if you want to customise default properties so you’re getting higher quality signals
3. Work with errors - update the status, view stacktraces, watch session replays and assign to teammates
4. Set up alerts

You can also help create dashboards to help your customer understand the value of the product.

Action items

- What are common Error Tracking dashboards PostHog and current Error Tracking customers are using? How can we help users get started with similar dashboards as easily as possible?
- A good starting point for Error Tracking are customers already using Analytics and Session Replay. What other combination of products does Error Tracking work well with?
- What is a high level story that shows the value of using Error Tracking in PostHog compared to other solutions the customer is using already? How does it help them to be able to correlate data from Error Tracking with e.g. Analytics and Session Replay?
    - e.g. as an eCommerce customer, being able to correlate exceptions related to shopping carts with the Analytics data about the value of that shopping cart would allow customers to prioritize fixing bugs based on lost revenue.
    - e.g. as a b2c company, prioritize errors happening as part of the signup funnel
- What metrics do we track to measure success of this initiative?
    - Percentage of CSM managed accounts using Error Tracking each quarter
    - New Error Tracking MRR for CSM managed accounts in X quarters

Technical recommendations

- Error tracking for the web is significantly less useful without proper sourcemaps. You can see under "Symbol sets" in the configuration menu if the required files are being uploaded correctly.
- Encourage customers to set up roles so that issues can be assigned internally to the right people.
- Use the SDK Doctor to make sure they're on the latest SDK versions.

How we upsell and cross-sell

contents/handbook/growth/cross-selling/how-we-upsell-and-cross-sell.md

How to cross-sell and upsell additional PostHog products

Cross-selling is a primary focus across all growth oriented teams. In fact, "cross-sell" is mentioned here as many times as **success**... applying to customer-facing roles including AEs, CSMs, and TAMs.

Success at PostHog comes from identifying genuine customer needs and demonstrating how additional related products solve real problems. The never ending objective is helping customers extract more value from PostHog, which naturally leads to increased product adoption.

Equally important is cross-selling exposure inherent from teams such as product and marketing. If the product, brand, onboarding, and what we are telling customers day to day is inconsistent, we're going to have a bad time.

What does cross-selling mean?

These descriptions below will describe the overall who/what/why and we will evolve specific motions people have found useful to cover the how/when. For a baseline, we will use these general definitions with specific context following for PostHog:

- Cross-sell: selling additional products that are useful and complementary to existing customer adoption
- Expansion: customer is or will be needing more of already adopted products, in the form of more volume or different business groups and teams in a similar pattern
- Upsell (upgrades): offering advanced functionality to an already adopted product, or they're a candidate for an add-on, both of which have higher costs

There are many ways a customer can signal or be primed for growth. All forms we will lay out here which may be in the form of usage, by voicing it directly, or you introducing the right products at the right time.

Wait! What should the relationship look like before you attempt cross-sell?

The relationship is first.

Leading with a cross sell motion is a bit like coming out the gate with offers related to contract billing terms and credits. While there are some limited circumstances where this makes sense, we should almost always start by focusing on the relationship.

The best way to build that relationship is to help the customer. That could be leaning in on a support ticket, offering recommendations around getting more our of PostHog, reducing spend, or even helping clarify docs.

Customers genuinely like PostHog, so engage them like a friendly acquaintance. Being hyper responsive to requests is a great way to build up good will. Another way is to own problems and follow them through to conclusion. Even if support is taking the lead, stay engaged and tie up any loose ends.

Here is a general checklist that should be met before putting a plan to cross-sell. Specific product motions may have additional pre-requsites.

- You have an active relationship with the customer – there are regular touchpoints and they are responsive to your outreach.
- You understand their product and PostHog implementation. You know which technologies they are using, and how PostHog fits into their setup.
- There are no major open issues with their PostHog implementation. They are happy with their current setup and aren't voicing major frustrations.
- There is no active risk to their renewal, and you aren't already negotiating that renewal.
- Clear path to talk to the right people
    - Ask your current champion who the interesting likely people would be to talk to
    - Be prepared to identify the right ICPs within the customer team
    - Identify teams that are responsible for critical paths/functions within their codebase, some examples across products are
        - Billing teams
        - Authentication & authorization teams
        - Data API teams (e.g. REST or GraphQL teams, that see a high volume of queries)
        - Management API teams (who have to deal with orchestration failures across projects)
        - Support tooling teams
        - Job titles that would be interesting are e.g. Platform Engineers, Backend Engineers (especially if they are on one of the teams mentioned above), anybody owning reliability or infrastructure

Growth best practices

Do:

- Focus on solving documented customer problems
- Provide trial access for evaluation
- Share relevant case studies and documentation
- Set clear success criteria before expansion
- Optimize their implementation before introducing new things
- Follow up on product experiments, even small ones

Generally, this is all to provide them with a solution that will make their life better (and make them look better!). It's a win-win.

Don't:

- Recommend products without clear use cases (it's okay to give awareness or suggest trying out something new)
- Create urgency where none exists
- Introduce expansion topics during crisis moments
- Overwhelm customers with too many products at once

We are friends, now what?

As you build a relationship with the account, learning about who they are, how they make money, and what they care about should naturally happen. Even so, you may need to dig in further, especially if their business is complex. Doing everything you can on your end to understand their business before asking business questions is another way to establish your expertise and build that good will.

There's a balance as any time you put additional burden on your champion or a stakeholder, you are less likely to help them achieve a positive outcome for us or for them. This is common as additional products require additional work to implement.

Then, opportunities!

Identifying growth opportunities

We use a combination of proactive outreach, insights, and automated alerts in tools such as Vitally to identify cross-sell opportunities. Below are some examples and we will go in more detail on specific motions.

You can use these signals which are documented on health-tracking alongside regular customer interactions to prioritize outreach.

The best opportunities connect products to customer outcomes using their terminology and context.

Example signals of cross-sell opportunities

- Web Analytics Opp (Marketing): Triggers when companies with marketing roles, >50 employees, and no visits to the web analytics page are identified
- B2B Group Analytics Opp: Triggers when group count is 0, group analytics plan is null, and the company type is B2B
- Replay Upsell Opp: Triggers for companies with customer success, sales, product, or customer service roles as PostHog users but no session replay usage
- FF Opp (High Engineer %): Triggers when a company has a high % of engineers but isn't using feature flags
- FF Opp (No Experiments): Triggers for companies who have users in product, marketing, leadership, or engineering roles haven't viewed any experiments

Strong expansion signals

- Consistent usage approaching billing limits
- Multiple departments accessing PostHog
- Questions about problems that other PostHog products solve
- 20%+ month-over-month event volume growth
- Custom implementations replicating native PostHog features
- Actively using PostHog competitors identified by using BuiltWith, Wappalyzer, or internal SDK Scanner.

How to run a cross-sell process

You made it here! You have the relationship, and you have the hunch (clear signals) a customer is good for cross-sell. Let's put it in to standard practice by following and building upon cross-selling motions

Here's a taste of what follows:

- First you need to find out who cares about the problem that our other products solve - is it the existing team or the new team?
    - Use a tool like The Org to help you identify new people.
    - Make sure you are asking for introductions to other teams during the regularly scheduled checkin calls - ‘who else would benefit from this?’, 'are there other teams with similar pain points?'. They will know better than any outside tool to gap organizational relationships.
    - In-person visits can help accelerate this
- Then you need to find out what are they using now to solve the problem (if anything) - surface this during the check in calls that you already have scheduled as part of onboarding if it's the existing team. If you're talking a new team, you'll effectively run this as a new sales process.
- Your approach will depend on the product that makes sense here:
    - If it's already a mature product we have shipped, you should aim to show how the product complements what they *already* are using in PostHog - don't just arbitrarily sell in a product for the sake of it. For example, you can say ‘other customers that look like you are doing X, this is what we’re seeing’.
    - If it's something in beta or coming soon, you should start giving them sneak peeks of what's on our roadmap. You can also schedule a feedback session with the relevant product engineer if they’re a great fit - customers *love* this. Again, consider playing the founder card for something *really* new and big.
- Understanding the blockers to using other products - these could be:
    - Privacy/compliance concerns (e.g. viewing session recordings) - we have a lot of documentation on this
    - Already doing it in house/with something else - demonstrated cool ways in which the products integrate and save their team time
    - May be too far down the line with their own data warehouse - it is hard to do a replacement at this stage, so instead talk about how you can enrich their data in PostHog with what's already in their data warehouse

- Not ready to invest the time and resources to implement more tools - tie this to the pain of *not* having an additional solution in place and emphasize time to value is extremely quick with PostHog e.g. with autocapture, session replay, and (soon) no-code experiments.

> Pro tip - if a customer isn't using a PostHog product and there is no obvious reason why they shouldn't, ask them directly why they're not using it!

Tracking cross sells

`contents/handbook/growth/cross-selling/tracking-cross-sells.md`

Cross-sell opportunity tracking

TAMs create Salesforce opportunities for cross-sell deals they're actively working. This is how we measure whether TAMs are driving multi-product adoption vs. benefiting from organic growth, and how we learn which motions actually work.

This is **measurement only** - it doesn't change how commission works.

What counts as a cross-sell opportunity?

All four must be true:

- Customer is already paying for at least one PostHog product
- TAM is targeting adoption of a **different** product
- Expected MRR on the new product is **≥$100/month**
- TAM is running an actual sales motion (not just hoping they adopt it)

If a customer spontaneously starts using and paying for a product, you don't need to retroactively create an opp. This is for intentional motions only.

How to create a cross-sell opportunity

Use the existing Salesforce record type: **`New revenue – existing customer`**

Opportunity stages

Stage	What it means
Discovery	Identified use case, talking to stakeholders about the problem
Demo	Showing them the product, connecting it to their specific needs
Trial	Customer is actively testing the product (see trial guidelines below)
Closed Won	Customer is paying ≥$100/month on the product
Closed Lost	Didn't convert - document why

Not every deal needs every stage. If a customer already knows the product and just needs help getting started, skip to Trial. The stages exist for tracking, not bureaucracy.

When closing an opp (won or lost), do it manually even though Vitally may auto-close goals when a revenue threshold is met. Consciously closing the opp shows you're on top of the account and creates a clean intent-to-outcome link in our data.

Trial guidelines

If you're giving a customer extended time or capacity to try a product before paying, use one of these approaches:

Option A: Billing limit

- Set a billing limit on the product (e.g., $500/month cap)
- Customer uses it, hits the limit, decides if they want to pay for more
- Time-box it: 30 days is reasonable, 60 days max

Option B: Trial

- Add credits to their account to cover the trial period
- Document the amount and expiration
- Clear expectation: "We're giving you X weeks to try this for free"

Either way

- Set a clear end date
- Schedule the follow-up conversation **before** the trial starts
- Document success criteria: what does "this worked" look like?

What this is NOT

- **Not a quota change.** Commission still works the same way.
- **Not required for organic adoption.** Only for deals you're intentionally driving.
- **Not for tiny expansions.** <$100/month expected MRR doesn't need an opp unless it's part of a trial/POC leading to real adoption.

What we're measuring

Cross-sell metrics are tracked in the sales growth review. After each quarter, we should be able to answer:

1. How many cross-sell oops did TAMs create?
2. What was the win rate?
3. Which products had the highest/lowest conversion?

4. What was the average deal size?
5. What was the average cycle time (discovery → closed)?
6. What reasons are we seeing for Closed Lost?

Growth reviews

contents/handbook/growth/growth-engineering/growth-sessions.md

Now that PostHog has found product market fit, we hold regular growth reviews where we plan work to accelerate our growth and drive revenue. These sessions are only worth running *after* you've found product market fit because until then you need to focus on building a solution users feel they really need. Once you've found product market fit, growth sessions can help you optimize.

We've established a successful pattern for running these meetings every four weeks and actions we've taken from them have led to some significant increases in monthly revenue growth.

Attendees

We find it's important to bring a mixture of technical people, and those with wide context of what the business is working on. Regular attendees include...

- Raquel (Manages lots of engineering teams)
- Tim (Co-CEO)
- James (Co-CEO)
- Charles (Sales/Marketing/Ops Exec)

Running a growth review

Before the meeting

- Before the meeting, everyone creates a list of hypotheses. These can be problems which limit growth, or opportunities to accelerate. They should all be related to growth engineering (e.g. not simple bug fixes), and they should be focused on improving our long term monthly growth rate.

Examples:

```
* "New, more expensive pricing has hurt signups"

* "We're not solving bugs quickly and this is hurting word of mouth growth"

* "We are losing B2C customers to open source"

* "We should charge for products separately so we can upsell on features rather
than volume. This would let us charge sooner than our current 1m events/month
limit, and would let us keep prices low for people that don't use the full
breadth of the platform"
```

During the meeting

- When the meeting starts, allocate 10 minutes for people to read through everything and to add any additional hypotheses.
- After reading, watch session recordings of users going through key flows of the product. Allocate 20 minutes for this. This may generate some further hypotheses. For example, users converting to paying customers, users activating, users inviting their colleagues.
- During the meeting, you should work through each hypothesis to determine if it is correct and decide how to respond. Answer each hypothesis using available data and write up any actions, such as experiments you want to run, with clearly allocated owners.

After the meeting

- Four weeks later, re-run the entire meeting. You may end up carrying some hypotheses from session to session if they're blocked or lower priority.

Growth session template

```
## Agenda

* 10 mins to read through / add or edit hypotheses
* 20 mins to review session recordings
* 4 hours to work through hypotheses and to create actions

## Hypothesis

Why aren't we growing faster?

*

## Actions

*
```

Relevant docs

- Growth review recurring doc (Internal only)

Per-product activation

contents/handbook/growth/growth-engineering/per-product-activation.md

Because PostHog offers so many products, and people sign up with all sorts of different needs, we track activation separately for each product.

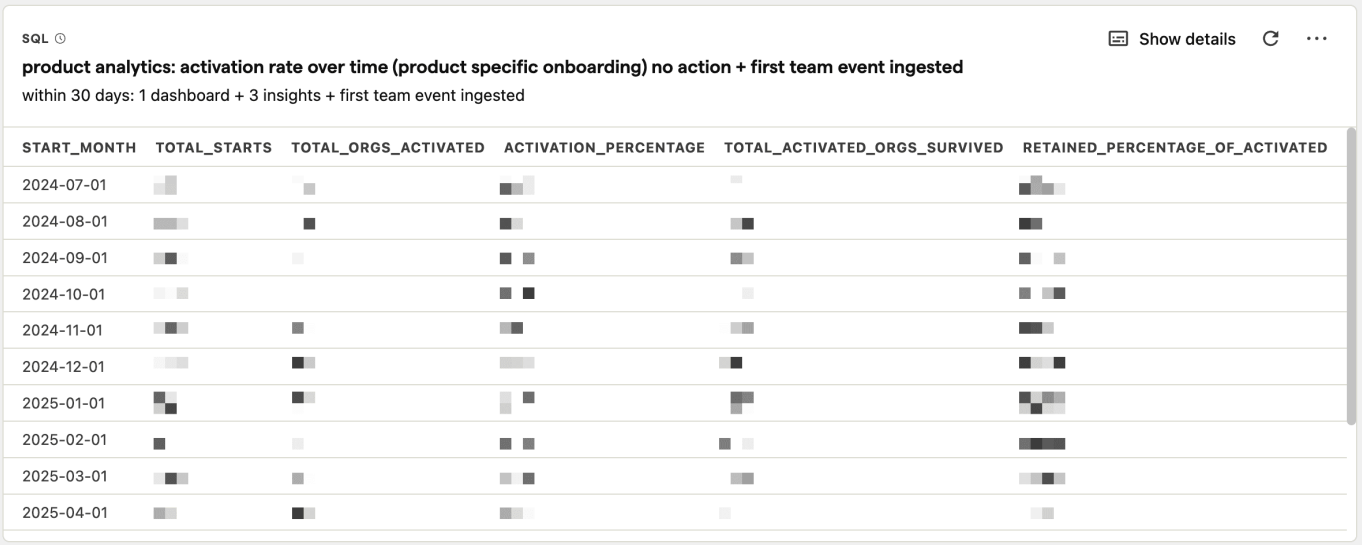
Every product should have activation criteria - these are used to determine if a user has activated for a specific product yet. If they haven't, and they've showed intent for that product, we can nudge them in the right direction. These are also used to understand what retention looks like for the product, and to figure out what PostHog can do to offer a better experience!

How we track activation, and how to set up an activation query for a new product

This is the basic structure of our activation queries:

1. An organization triggered a 'product intent' -> This is the 'upfunnel' metric
2. An organization met the 'activation criteria', usually one, multiple, or a set of qualifying events in a given time period (e.g. 14 days) -> This is the 'downfunnel' metric
3. An organization triggered an event correlating with product usage 3 months after they showed product intent -> This is the retention / survived metric

Here is an example structure:



You can find all per-product activation queries on this dashboard.

Picking the right activation criteria

The ideal activation metric strikes a balance: enough companies should reach activation (so it's not too restrictive), while those who activate should have high retention (so it's not too easy). To find a couple of potential definitions, you want to look at product usage and think about what behavior *could* correlate with successful activation (aka the "aha-moment"). This could be things such as

1. Has done a key event once (such as launched an experiment)
2. Has done a key event multiple times (such as analyzed 2 insights)
3. Has done a combination of key events (such as watched 5 recordings, and set a recording filter)

To pick the best activation definion, it's recommended to write the activation queries for multiple potential activation definitions (~5-10), and compare the activation and retention numbers. This leads to a much higher confidence in the activation metric than just picking your best guess.

Which definition is the best indicator for long term retention? You want to pick a definition that gets a sizable number of organizations to activate, but also to retain. But be careful: If you pick a activation definition where only 1% activate, and 100% of those 1% retain, your activation metric is too narrow!

Note on the retention / survived definition: For this, it's recommended you pick whatever tells you they are an active user. It can be the same as your activation definition, or something a bit simpler, as long as it is closely related to the user actually using the product (e.g. in replay, activation is currently defined as analysing 5 recordings AND setting a filter, usage is simply defined as having analysed one or more recordings).

If you haven't already, make sure you also track product intents for your product. It's worth noting that adding new product intents will impact your activation rates (e.g. an existing user intent might be stronger or weaker than an onboarding intent). If you are comparing activation rates historically, it might be worth filtering for intents that rarely change, such as "onboarding product selected".

Read this blog post for a deep dive into how we first came up with our activation definions.

Structure of the SQL query

Our activation SQL queries consist of two parts: A materialised view to count the eligible events, a SQL query on top of the materialised view to count the conversion percentages. We use materialised views to make these queries more performant.

We store the activation logic in SQL queries and not in code to make it easier to see our activation definitions, to experiment with new definitions, and to drill down to understand why a certain bucket might not perform so well.

The following activation logic is stored in the materialised views:

1. Count only the first product intent per organization (since product usage intents can be triggered multiple times by the same org), as well as filter out cross sell product intents

2. Check if an organization meets the activation definition within 30 days of showing product intent
3. Check if an organization meets the retetion / survived definition within 3 months of showing product intent

Here is an example materialised view query. To write your own, we recommend copying the query and change the product & event filtering criteria as needed.

The following logic is stored in the SQL query:

1. Check if a organization is both activated AND retained to be counted in retention / survived
2. Calculate the conversion percentages from product intent -> activation -> retention / survived

To write your own, we also recommend copying one of the existing queries. All our activation queries follow the same structure, which we should also follow for new products. Once you've found a good definion of activation for your product, please do add the final activation query to this dashboard.

Tracking activation in the code

We use SQL queries to analyze activation. In addition, we track product intents and activation in the code. We do this so that in the future we could act on this, e.g. someone showed intent, but they didn't activate? Show them in an-app banner or send them an email.

To add a new product to this, you can add the activation criteria in the product intent model.

This code is run every time an intent is updated. For example, if the activation criteria is "save 4 insights", and we send a product intent every time someone clicks "new insight", we'll also check at that time if they have 4 insights saved, and if so mark them as activated.

Why does this matter?

Tracking activation is important, because it tells us how many companies start using our products successfully each month, and how many retain. Measuring it month over month allows us to see trends, and whether improvements to the product actually made a difference.

If the activation metrics look good, it gives us the peace of mind to focus on new feature development. But if they trend downwards, it's probably a good time to look into our onboarding and "first time user" funnels to see in which areas our UX can be improved.

Product intents

contents/handbook/growth/growth-engineering/product-intents.md

Because PostHog offers so many products, and people sign up with all sorts of different needs, we track activation separately for each product.

> To learn more about what activation is and how we measure it, check out this blog post.

To make sure we're measuring activation properly, we need to know when someone is interested in using a product. If we put everyone who signs up into the activation funnel for each product - without even knowing if they are actually interested in it - then we'd end up with a super large top of funnel, murky metrics, and a dismal activation rate for all products.

So instead of just putting all people into each activation funnel, we try to identify people who are interested in using a product. In other words, we identify people who show *intent* to use a product, and we record these people (and the moment the intent happened) as events. These types of events are called *product intents*. The product intents mark the top of the funnel for each product's activation funnel.

Product intents are:

- Flexible, so they can happen anywhere and any time
- Stored in the database, so we can use them in the product if we want
- Convertible, so we know if someone who has shown intent for a product has successfully activated

When should I start capturing product intents?

As soon as your product is in any sort of public beta you should start tracking product intents. This is *not* because you should be hyper focusing on your product's activation numbers at this stage - instead it is so that we can start collecting data for later on when we want to determine a good activation metric.

So, collecting product intents should be a precursor to any sort of public release.

What makes a good product intent?

People click around in the UI a fair amount, so generally you want to find something sufficiently deep, or something that happens multiple times, before saying someone has shown intent. Here are a few examples:

- In onboarding, we ask people what products they are interested in. This is a very direct way of indicating intent! If your product has an onboarding flow, we automatically collect a product intent for it.
- For data warehouse, when someone actually clicks to set up a source, we consider that an intent. It's a couple pages deep, so people who've gotten there are less likely to just be clicking around.

- If someone views docs for your product multiple times (you could keep a counter in localstorage), that could be sent as a product intent.

Is the onboarding product intent good enough for my product?

Nope. Lots of people join PostHog with a single product in mind - and then later realize that we offer other products they also want to use. Each product should have product intents being recorded somewhere past onboarding, so they aren't missing out on data about these types of post-signup customers.

How do I use these product intent things?

Generally we've made the plumbing such that recording these product intents is quite easy.

1. Figure out where you think the product intent event should happen.
2. When someone clicks that button / views that page / does that thing, then simply call `addProductIntent` in the `teamLogic`. That fires off an API request that records the product intent in the database and sends the event for you. You don't need to send the event yourself - it's all handled.
    - You must include the *context* of the intent with this API call, this is so that you can understand what is driving product intent in the analytics. You can store this in the `intent_context` — and you can find existing intent contexts in the product intents utility.

It's worth noting that adding new product intents will impact your activation rates (e.g. an existing user intent might be stronger or weaker than an onboarding intent). If you are comparing activation rates historically, it might be worth filtering for intents that rarely change, such as "onboarding product selected".

Cross Sells

As well as understanding what actions users take when trying out a product, it's also useful to encourage users to try out other products that would be helpful for them. If you are using product analytics for example, session replay is a really helpful way to understand *why* a metric is what it is. If you are creating an onboarding funnel to understand your conversion, running an experiment to improve that conversion would be helpful.

We track cross-sells within the product using the same product intent framework. There is a helper for this in the `teamsLogic` called `addProductIntentForCrossSell`, which you can use to track cross sells. You can find these in analytics using the usual event for product intent (`user showed product intent`) and filtering by `type=cross_sell`.

Why does this matter?

It's important that we understand if people who are trying to use our product are actually successful in doing so. This is a likely imperfect, but better-than-nothing, way to do that. If people aren't having the success we'd expect for a mature product (ie no large feature gaps with competitors), then we should probably look into why - and this gives us a cohort of people to examine, talk to, and track.

Giving credits to customers

contents/handbook/growth/revops/credits.md

Sometimes we might want to offer a customer one time credits to cover an upcoming invoice, for example when accommodating a trial for a new product or offering compensation for a recent incident. Here's how to do that.

- Go to Billing Admin → Credits
- Click Add Credit at the top right.
- Click Add credit at the top right.
    - Select the customer from dropdown or search
    - Enter the total credit amount.
    - choose from the following options for reason:
        - First Big Bill
        - Unwanted Spike
        - Trial Accommodation
        - Promo Credit
        - Incident Credit
        - Bug Credit
        - Other
    - Add any internal notes for context in Notes section.
    - Include a link to a related Slack message, Zendesk ticket, or internal discussion in reference link field
- Click Save and the credit will automatically be added to the customer's balance in Stripe and applied to their next invoice.

Things to keep in mind

- Credits only apply to upcoming invoices.
- If you're trying to adjust a completed invoice, this should be handled as a refund instead.
- Always include enough context in your notes or reference link so others understand why the credit was given.

Lead assignments during time off

`contents/handbook/growth/revops/lead-assignment-ooo.md`

We want to make sure we don't assign new leads to people who are out of office both in Salesforce and in Default (our contact form submission routing system)

In Salesforce → lead assignment tracker

- There's a checkbox called Is Active
- When someone goes on leave, uncheck this box to make them inactive so they don't receive any new leads

When they return:

- Recheck the box so they start receiving leads again.
- Use the Manual Lead Adjustment column to rebalance their totals if needed (more on that below).

In Default app → routing → queues

- Find the queues the person is part of (US East, US West, EMEA, Asia based on location, or All and Max Availability Queues for everyone)
- Under each queue find Status column
- Toggle their status to Inactive so Default to stop routing new leads to them
- When they return, Toggle them back to Active in relevant queues

Important: Even if the person marks themselves as "Out of Office" in their Default personal settings, that does not stop lead assignments. We still need to manually toggle them off in the queues.

When to take these actions

- If taking <= 5 days off: Turn them off the day they leave and turn back on the day they return.
- If taking > 5 days off: Turn them off 2 days before their scheduled time off to avoid assigning new leads right before they leave. Turn them back on the day they return, so they have fresh leads to work on after returning.

Calibration after time off

When someone's inactive, others in the queue keep receiving leads so their totals rise. We need to use the calibration fields to make sure all leads don't go to the person who comes back from OOO based on pure total lead count.

In Default Queues you'll see Total Assignments and Calibration columns

- Let's say the person was out two weeks, and others got +10 more leads per person during this time.

- To rebalance, add +10 under the Calibration column of the OOO person before activating them in the queue
- This prevents the system from dumping 10 leads on them the moment they're reactivated.

In Salesforce: do the same using the Manual Leads Adjustment column in Lead Assignment Tracker.

Lifecycle analysis

contents/handbook/growth/revops/lifecycle-analysis.md

Understanding how our revenue moves through different lifecycle stages helps us identify the specific drivers behind our growth, not just the net change in revenue. We use lifecycle analysis to see how much growth comes from new customers, expansions, contractions, and churn. We analyze this at both total revenue and per product levels to understand each component of our business.

Customer lifecycle stages

new

Customers in their first month of paying us.

reactivated

Customers who previously churned but have returned with monthly spend > 0. This signals customers who may be using our services on and off.

flat

Existing customers whose monthly spend remained exactly the same as the previous month. This represents our stable, predictable revenue base.

growing

Existing customers whose spend increased compared to the previous month. This shows how successful we are with our upsell, cross sell, and product expansion efforts.

shrinking

Existing customers whose monthly spend decreased but didn't reach zero. This could be due to usage based fluctuations but also an early warning indicator of customer dissatisfaction or competitive pressure. We pay close attention to this amount and do deeper analysis to understand the reasons behind.

How we calculate lifecycle components

New revenue: total monthly revenue from customers in their first month

```
SUM(CASE WHEN growth_lifecycle_stage = 'NEW' THEN mrr ELSE 0 END)
```

Reactivated revenue: total monthly revenue from customers who returned after churning

```
SUM(CASE WHEN growth_lifecycle_stage = 'REACTIVATED' THEN mrr ELSE 0 END)
```

Retained revenue: baseline revenue that continuing customers maintained

```
SUM(CASE
    WHEN growth_lifecycle_stage = 'FLAT' THEN mrr
    WHEN growth_lifecycle_stage IN ('GROWING', 'SHRINKING') THEN prev_month_mrr
    ELSE 0 END)
```

Expansion revenue: Additional revenue gained from existing customers through increased usage

```
SUM(CASE WHEN growth_lifecycle_stage = 'GROWING' THEN mrr - prev_month_mrr ELSE
        0 END)
```

Contraction revenue: Revenue lost from existing customers due to reduced usage (negative value)

```
SUM(CASE WHEN growth_lifecycle_stage = 'SHRINKING' THEN mrr - prev_month_mrr
        ELSE 0 END)
```

Churned revenue: Revenue lost from customers who went to $0 (negative value)

```
-SUM(prev_mrr) WHERE prev_mrr > 0 AND curr_mrr = 0
```

Rate calculations

Baseline revenue

This the total monthly revenue at the end of previous month which is the denominator for our rate calculations:

Key rates

New rate: How much new revenue we acquired relative to our at baseline revenue

```
new_rate = (new_revenue / baseline_revenue) × 100
```

Example: 10% new rate means we acquired new revenue equal to 10% of our at baseline revenue

Expansion rate: Growth from existing customers as a percentage of base

```
expansion_rate = (expansion_revenue / baseline_revenue) × 100
```

Example: 5% expansion rate means existing customers increased their spend by 5% on average

Contraction rate: Revenue decrease from existing customers due to lower usage

```
contraction_rate = (contraction_revenue / baseline_revenue) × 100
```

Example: -3% contraction rate means we lost 3% of revenue from reduced customer usage

Churn rate: Percentage of at baseline revenue that was completely lost

```
churn_rate = (churned_revenue / baseline_revenue) × 100
```

Example: 2% churn rate means 2% of our baseline revenue base churned completely

Notes on data

- Analysis covers last 18 months of historical data plus 2 months forward
- Churned customers are identified by transition from revenue > 0 to revenue = 0
- Reactivation requires at least one month gap with revenue = 0

Overview

contents/handbook/growth/revops/overview.md

How RevOps works

RevOps at PostHog is the Product Manager for Sales, Marketing, and Executive teams. Just as PMs help engineering teams build better products by connecting user needs with technical solutions, RevOps helps go to market teams make better decisions by connecting different parts of the business together.

We do this by combining data from marketing, sales, product usage, and customer success to show what's working and what isn't. While individual teams deeply understand their specific areas, we provide insights about how different parts of the business affect each other and help teams see these connections to drive revenue growth for PostHog.

RevOps values

1. Make data simple
2. Build for self service
3. Automate relentlessly

1. Make data simple

PostHog has data everywhere: product usage, sales pipelines, support tickets, revenue numbers. With this wealth of data comes complexity. We turn this scattered data into clear insights teams can use.

This means:

- Creating reliable, unified views that combine data from multiple sources
- Building clear abstractions that hide unnecessary complexity
- Maintaining source of truth definitions for key metrics
- Finding useful patterns in customer behavior
- Showing how each team's work affects our customers

Creating views to show total monthly MRR, per product MRR, and per product usage, and filters out the anomalies to make sure make sure analyses are accurate and consistent across teams was one of the early steps we took in this direction. Unifying data from our billing system, Salesforce, and Vitally to have full context on biggest gainers and losers queries to show full context on which customers' spend changed the most was another one to simplify access to this info and quickly take action when needed.

2. Build for self service

Teams should get the information they need without waiting for RevOps. Like engineers ship without PM approval, go to market teams should be able to analyze and act on data without asking us.

This means:

- Making all our data and processes visible by default
- Helping teams answer their own questions

For example, we built a self-managing lead pool where leads automatically move if they haven't been touched in 7 days. Instead of leads getting stuck with specific AEs, any sales team member can now pick up and run with these potential opportunities. This keeps leads fresh and moving while giving everyone on the team a chance to work with promising accounts, no RevOps intervention needed.

3. Automate relentlessly

Manual work wastes time and doesn't scale. If someone has to do something twice, we automate it. We rely on teams to tell us what's not working because they see the problems first.

If a team at PostHog struggles with revenue operations, we've probably:

- Not automated enough tasks
- Tried to automate something we shouldn't
- Made data too hard to access
- Miss important customer data

For example we built an automated workflow that identifies product qualified leads in real-time. When a company hits key milestones (like having 5+ active users and using multiple products) and matches our ICP they're automatically flagged as a new lead in Salesforce with their usage data so the sales team can now instantly see which customers can benefit from outreach and why instead of having to piece this information together themselves.

RevOps vision

Things we want to be brilliant at

Standardize key metrics: Own and maintain clear, consistent definitions for our most important business metrics including:

- How we recognize revenue (annual vs monthly plans, upfront vs usage based payments)
- Revenue retention calculations (what counts as expansion vs new business)
- Customer definitions (who's an active customer, who's usage qualified)
- How we forecast revenue (how do we predict future revenue based on current usage patterns, conversion rates, and expansion signals)

This ensures everyone across the company uses the same language and measures success the same way.

Connect the dots: Help teams understand how their work impacts others, things like:

- Track how specific marketing campaigns drive upsell and cross sell

- Measure what corraletes with strong retention rates
- Monitor which product features lead to customers expanding their usage

Rapid insights: Build self service tools that help teams quickly answer their own questions:

- Dashboards to easily track real time changes in key metrics
- Alerts when important customers change their usage patterns
- Easy ways to analyze customer behavior without needing SQL

Things we want to do next

Revenue attribution: Understand how customers move from free to paid, including what features they use, how long it takes, and what influences faster conversions. When a customer upgrades or buys more, know exactly why: was it reading docs? using a specific feature? talking to support?

Predictive analytics: Build on our work around identifying expansion signals to get ahead of customer behavior, find customers likely to buy more before they ask, and identify unhappy customers before they leave.

Things we don't want to spend time on

Being the "data police": We don't want to spend time enforcing data formats or policing how teams use tools. We focus on making it easy to do the right thing, not enforcing rules

Running reports for people: If someone regularly needs data, we should teach them how to get it themselves.

Clean up projects: If we're constantly cleaning up data problems, we've built the wrong systems, and should fix the source problems instead.

Responsibilities

What RevOps owns

Revenue insights:

- Reporting company wide metrics: revenue, retention, expansion, churn
- Help sales, marketing, and exec teams understand what drives revenue
- Identify patterns in customer behavior
- Build shared understanding of revenue and retention reporting across teams

Sales tech stack including:

- Salesforce administration and optimization
- Enrichment and intelligence tools (e.g. Clay, Clearbit, Sales Navigator)
- Contract management systems (Pandadoc) SalesOps section in handbook has more information.

What RevOps supports but doesn't own

Revenue reporting and forecasting: RevOps provides recommendations and improvements but does not own implementation and maintenance. This is currently owned by the .

Marketing operations: Marketing owns their campaigns and analytics, we help connect marketing data with revenue outcomes.

Product operations: Product teams own their metrics and experimentation, but we help track how they impact overall revenue.

What RevOps doesn't do

- Financial accounting (though we work closely with Finance)
- Individual sales deal management
- Billing/invoicing platforms, and data infrastructure for revenue reporting owned by the

Retention metrics

contents/handbook/growth/revops/retention-metrics.md

We use Net Dollar Retention (NDR) and Gross Dollar Retention (GDR) to track how well we're retaining and growing customer revenue over time. We use adjusted revenue to calculate these retention metrics for a more accurate picture of our business. This way, we get clearer signals about retention by removing the noise from spikes, trials, and organizational shifts.

How we calculate

We use a rolling time period approach that compares customer revenue from a base month to the current month:

- For each calendar month, we look backward to identify the "base month" (3, 6, or 12 months prior)
- We identify customers who were active in that base month with minimum $5K annual revenue
- We compare what those same customers were paying then versus what they're paying now

NDR (Net Dollar Retention)

NDR shows total revenue retention including expansions, contractions, and churn.

Formula: Sum(current_month_mrr) / Sum(base_month_mrr)

If NDR > 100%: We're growing revenue from existing customers (expansions outpace contractions/churn) If NDR = 100%: We're maintaining the same revenue from existing customers If NDR < 100%: We're losing revenue from existing customers

GDR (Gross Dollar Retention)

GDR shows how much of our base revenue we're retaining, without counting expansions.

Formula: Sum(MIN(current_month_mrr, base_month_mrr)) / Sum(base_month_mrr)

GDR caps each customer's current revenue at their base month amount so it only measures downgrades and churn.

Why $5K+ ARR customers?

- these customers represent the majority of our revenue
- they tend to have more established implementations
- they have invested enough time and resources to properly implement our product
- focusing on these customers provides clearer retention signals by filtering out noise from small, transient accounts

Why rolling retention?

- Allows us to continuously monitor monthly trends without waiting for cohorts to mature
- Includes all customers in each calculation, giving us larger sample sizes and more reliable metrics
- More commonly used in reporting and easily understood by those who want a simple, current retention number

Cohort based retention for lifecycle insights

We also track cohort based GDR and NDR as well as cohort based usage retention to:

- Understand how retention evolves throughout different stages of customer lifecycle
- See if newer cohorts perform better or worse than older ones
- Track how significant business or product changes may impact specific cohorts
- Recognize seasonal or other time based patterns in customer behavior

Revenue adjustments

contents/handbook/growth/revops/revenue-adjustments.md

Raw revenue numbers can sometimes be misleading due to various factors that don't reflect the true health of our business. Our adjusted revenue methodology helps us account for these factors to get a clearer picture of our growth.

Why we adjust revenue

1. Create a more accurate representation of our business performance
2. Identify growth patterns by removing short term noise due to the nature of usage based revenue
3. Easily spot any anomalies in growth patterns
4. Standardize our reporting methodology

Where we use adjusted revenue

We continue to report unadjusted revenue for our top line reporting and overall growth metrics. We use adjusted revenue for retention metrics (NDR/GDR) and other business lifecycle analysis to get a clearer picture of our growth. This way we maintain standard financial reporting (unadjusted revenue) while getting a better understanding of our performance (via adjusted revenue).

Adjustments

We make the following primary adjustments to our revenue data:

1. Trial adjustments

Revenue from customers who are testing our platform with the intention of potentially moving to self hosted or another solution.

Why: Including trial revenue in our regular metrics could create an artificially inflated view of sustainable ARR, especially if we know the customer is likely to leave.

How:

- We flag accounts as "trials" when we know they're evaluating our service (typically identified by our sales team)
- We remove the revenue from these accounts when calculating retention and other business metrics

2. Revenue spike adjustments

One time significant increases in customer spending that don't represent sustainable revenue growth. This could be due to bot attacks, implementation errors, or sudden unexpected increase in volume on the customer side.

Why: These spikes can significantly distort our monthly growth metrics and don't represent sustainable revenue we can count on going forward.

How: We define a spike when all these conditions are met:

- Customer's current month revenue is more than twice the average of the previous two months
- The increase is at least $1000 in a given month
- The following month's revenue decreases by 50% or more from the spike month

3. Annual plan adjustments

Accounting for the full spending potential of customers on annual plans who receive discounted credits.

Why: This gives us insight into the actual usage value customers are receiving, which is often higher than what they pay due to annual discount incentives.

How: Calculate what customers would have paid without the annual plan discount (annual_mrr_value = total_mrr / (1 - discount))

4. Account consolidations

Reconciling multiple accounts that belong to the same organization.

Why: Organizations sometimes have multiple accounts that should be viewed as a single customer for accurate revenue analysis. This way we can make sure we're tracking true customer retention rather than treating internal movements as churn

How:

- identify and consolidate accounts when customers move from EU to US instances
- combine revenue and usage from different teams under the same organization that may exist as separate PostHog accounts

5. One time credit / refund adjustments

Revenue credits that temporarily drop a customer’s billed MRR to zero (e.g., one time promo credit, incident credit etc.)

Why: If we count one time large credits as churn in our retention math we’ll understate our true net revenue trends. Excluding these prevents misleading dips and spikes in monthly growth patterns.

How: check for following criteria:

- credit is issued for a promo, outage, or a billing correction AND
- total credit amount ≥ 0.5 * prior month revenue AND
- next month revenue is >= previous 2 month avg revenue

if all three are satisfied we override that month's revenue to equal prior month's revenue.

Account allocation and handover

contents/handbook/growth/sales/account-allocation.md

We have different roles within the team who manage customers at various stages in their lifecycle. Customers will typically sign up and start paying for PostHog themselves, or land as customers via a Technical Account Executive. Once customers hit $20k a year in spend with us they should have a dedicated Technical Account Manager or Customer Success Manager.

TAM vs CSM

Technical Account Managers (Sales Team) and Customer Success Managers (Customer Success Team) are the primary owner of customers spending $20k a year and above; and we aim to have full coverage of those customers across the two teams and roles. When deciding whether a customer should be with a TAM or CSM we factor in to account their usage of our primary products.

Primary products are the set of billable main products which we believe that all engineers should be using, not including add-ons or platform features. Our current set of primary products are:

- Session replay
- Feature flags
- Error tracking

We track whether a customer is paying for each product in Vitally using the `Paying for <Product Name>` trait.

- Customers already paying for all of the primary products are considered expanded to the max and should go to a Customer Success Manager. They should be pretty sticky as a customer so the main focus here is retention.
- Otherwise, there is room to grow and a Technical Account Manager should be focused on getting them using the three primary products.

> This allocation may vary depending on team capacity - there may be some accounts who only have 1 or 2 paid products allocated to a CSM rather than a TAM where there is more capacity in the CSM team for example.

Quarterly book planning

At the start of each quarter, TAMs should prepare their book of business with the following constraints in mind:

Target book size: 10-15 accounts with a combined ~$1.5m ARR. This gives TAMs enough focus to actually move the needle on expansion and credit pre-purchases.

Maximum book size: 15 accounts. New leads or handoffs from CS/Onboarding/TAEs will push a TAM above this throughout the quarter, but if you're starting a quarter at 18 accounts, you need to find a way to get to 15 or fewer.

Accounts to remove from your book

Before the quarter starts, review each account and remove those that meet any of the following criteria:

- **Churned or dropped below $20k ARR** – unless you have a documented, specific plan to get them above $20k this quarter
- **On YC Plan** – accounts 20*k*–50k on the YC plan will go to the YC role on the CS team. Accounts above $50k should be a candidate for TAM referral if there is growth opportunity.
- **Fully expanded and committed** – if the account has all 3 core products adopted (Session Replay, Feature Flags, Error Tracking), has a discount agreement in place, and has no viable levers for net new revenue, they should go to a CSM
- **No viable expansion levers** – if there's genuinely no path to growth, it shouldn't be consuming TAM bandwidth. You need to document what you've tried here so that we know all avenues for growth have been exhausted.

What is NOT a valid reason to hand off

Low engagement or an account being "difficult to work with" is not a reason to pass them off. That's literally your job. Specifically:

- Account doesn't respond to your outreach
- Champion left and you haven't re-established relationships
- Low user activity or poor health score
- You don't like working with them / they don't like you

If an account is struggling on these dimensions, that's a signal you need to invest more effort – not hand them off. You should only hand off accounts that are in a **good state**.

Doing the allocation

It's Simon's job, with input from Charles and Team Leads, to review the list of $20K accounts without an owner, as well as accounts which need to be handed over from TAE and TAMs. We use the criteria above to figure out which team should own a customer, and then use Vitally data to understand which region they are primarily based in. Looking at the user list in Vitally will show you where the most users are so make a judgement call on where the TAM or CSM should be based to best support and engage with the customer. Once this has been decided the New Owner trait is populated with one of the following:

- US TAM
- US CSM
- EU TAM

- EU CSM

And then it is down to the Team Leads to figure out which team member is taking on the customer.

Quarterly allocation process

At the start of each quarter, Simon (with input from Charles and Team Leads) reviews:

1. **$20K accounts without an owner** – accounts that need to be assigned
2. **Accounts flagged for handover** from TAEs, TAMs, and CSMs
3. **TAM books exceeding 15 accounts** – identifying accounts that should move to CSM or another TAM
4. **CSM accounts with expansion potential** – identifying accounts that should move to a TAM

Once Simon determines whether an account belongs with a TAM or CSM (and which region), the `New Owner` trait is populated, and Team Leads assign the specific team member.

Mid-quarter changes

Account removals should only happen at the end of the quarter so that quota can be calculated correctly. However, accounts can be **added** to your book at any time if you're confident there's growth potential.

If you're assigned an account with a previous owner, work with them on a proper handover. If the customer isn't in a healthy state (usage and engagement-wise), push back and ask the previous owner to get them to a good state first.

> New accounts with no previous owner come with a 3 month grace period – if they churn in that initial period, they won't count against your quota. Don't ask for the `AM Managed` segment to be added until you're confident there's growth potential.

Top 40 account management

Our highest-spend customers (~Top 40 by ARR) get special consideration for ownership decisions. Simon and Charles regularly review these accounts to:

- **Minimize ownership changes** – frequent handoffs create whiplash for customers and damage relationships
- **Ensure continuity** – the bar for changing ownership on a Top 40 account is higher than for the rest of the book
- **Make judgment calls** – sometimes a TAM should keep a "fully expanded" account if the relationship is strong and there's long-term strategic value

For Top 40 accounts, ownership changes (TAM→CSM or CSM→TAM) are decided directly by Simon and Charles, not through the standard Team Lead allocation process.

Handing over customers

To help the new owner of a customer hit the ground running, we should make sure that the customer is in a good state and that a warm introduction happens. Typical handoffs between roles are:

Transition	Typical timing	Condition
TAE → TAM	When onboarded, typically 3 months after initial credit purchase OR 12 months after initial credit pre-purchase if the account is retained by the TAE	Customer onboarded to 1-2 primary products
TAE → CSM	When onboarded, typically 3 months after initial credit purchase OR 12 months after initial credit pre-purchase if the account is retained by the TAE	Customer onboarded to 3+ primary products
TAM → CSM	After expansion completes	All 3 core products adopted, discount agreement in place, no remaining expansion levers
CSM → TAM	When expansion opportunity identified	Customer not fully expanded and has clear growth potential

> For accounts who will be landing at $100k+ a year or have high expansion potential after the initial deal, we should involve a TAM early in the process to ensure a smooth transition. See the section further down this page on how this works.

For handover to take place there should be an Account Plan (saved as a note on the account in Vitally) and the customer should have been onboarded properly to the products they are currently paying for.

> All open invoices should also have been paid before handing over. It makes sense to use existing relationships to chase payments, rather than the new owner's first action needing to be chasing payments/suspending access for non-payment.

> For TAE accounts being handed over, set the New Owner to `Ready to move` in Vitally and then flag this with Simon directly. There's no need to wait for the end of the quarter to do this. He will review the plan and current state of the customer and then work with TAM or CSM leads to assign a new owner.

Account Plan

Every account being handed over should have an up-to-date Account Plan saved as a note in Vitally. The existing owner should ensure that this is current and schedule a handover call to walk through it with the new owner. Feel free to push back and ask for it as the new owner if this doesn't happen! Ask your team lead or Simon for help with this if you're not getting the information you need from the previous owner.

Product Onboarding

Before handing over a customer, the existing owner needs to ensure that the customer is onboarded properly to the products they are paying for. We should first ensure that they are only paying for what they need to as detailed in the health checks section of the handbook and then ensure the following steps have been completed, depending on the products they are paying for:

> This is an initial pass at what good onboarding looks like for each product. We will refine this and add it to Vitally as a checklist to work through with the customer.

General principles

- They are aware of how to get support both via Slack and in-app and where in-app is more appropriate.
- They have the correct owners and admins set up in their PostHog organization.
- We have the correct finance contact details in Stripe.

Product analytics

- They have set up tracking, implementing posthog.identify() and posthog.group() correctly where appropriate.
- They are aware of the difference between anonymous and identified events.
- Event capture is tuned and automatic events have been turned off where not wanted.
- We have completed training for the core user base, so that they are aware of concepts such as Actions, Cohorts etc.
- They have set some insights and dashboards aligned with their use case for PostHog.

Session replay

- They have set up tracking using posthog-js.
- They are aware of the different recording controls and how to use them.
- They have implemented privacy controls where necessary.
- We have completed training for the core user base so that they know how to find specific recordings, as well as navigate from other products to session replays (e.g. from a funnel)

Feature flags

- They understand how to integrate feature flags into their workflow.
- Feature flag calls are implemented correctly so as not to artificially inflate the bill.
- They understand the current targeting mechanisms which are available.
- We've conducted training on how to set up Feature Flags and Experiments.

Data warehouse

- They have connected up the sources they need to.

- They are aware of the difference between incremental and full sync and the impact on billing.
- We've conducted training on using SQL in PostHog, creating views and joining on person data.

Error Tracking

- They have set up tracking using posthog-js.

Account handover checklist

Every account handover should include a 15-30 minute call between the outgoing and incoming owner. This checklist helps you prep for that call and make sure nothing falls through the cracks.

When to use this

- When a TAE-led customer is being handed over to a TAM after the initial contract is signed
- When a TAM is taking over an account from another TAM or TAE mid-lifecycle
- As a prep guide for the 15-30 minute handover call between outgoing and incoming owner

Before the handover call

The incoming TAM should prepare by reviewing the following in Vitally and SFDC before the call, so the handover conversation can focus on context that *isn't* in the data.

Self-serve research (do this first)

- [ ] **Vitally account overview** – MRR, ARR, health score, segments, paid products, usage traits
- [ ] **Billing & contract details** – annual plan dates, credit balances, discounts, renewal date, billing limits
- [ ] **Product adoption** – which products are they paying for? What's underutilized?
- [ ] **Usage metrics** – active users, project count, Feature Flag requests, Session Replay volume, insight/dashboard engagement
- [ ] **Support history** – recent Zendesk tickets, tags, severity, resolution status
- [ ] **Conversations & notes** – read all Vitally notes, meeting summaries, and conversation history
- [ ] **SFDC opportunity** – deal value, stage, next steps, close date
- [ ] **Admin emails & user list** – identify who's active, who has admin access, what domains are in play
- [ ] **The customer's product** – sign up or browse their website. Understand what they do and how they make money

Prepare questions based on gaps in the data. The handover call should focus on things you *can't* learn from Vitally.

Handover call agenda

This isn't an exhaustive list and not every item needs to be covered every time. Use your judgment based on what's relevant to the account.

1. Relationships & people

This is the most valuable part of the handover – relationship context doesn't live in any tool.

- [ ] **Who is the champion?** Name, role, communication style, what motivates them
- [ ] **Who is the economic decision-maker?** Who signs off on renewals and expansion?
- [ ] **Who are the power users?** Engineers, PMs, analysts – who lives in PostHog daily?
- [ ] **Org structure?** Parent/subsidiary dynamics, relevant teams, reporting lines
- [ ] **Any recent people changes?** Champions who left, new hires, reorgs
- [ ] **General vibe?** Easy to work with? High-maintenance? Responsive or hard to reach?
- [ ] **Preferred communication style?** Slack-first? Email? Regular calls or async?
- [ ] **Has the customer been told about the handover?** If not, agree on how to introduce the new TAM

2. Commercial context

- [ ] **Open proposals or negotiations** – anything in-flight that needs immediate follow-up?
- [ ] **Renewal strategy** – what's the plan? Any risks?
- [ ] **Discount/credit context** – why were discounts given? What was promised?
- [ ] **Budget & procurement** – annual budget cycle, procurement process, finance contacts
- [ ] **Expansion potential** – realistic growth ceiling? New teams, new brands, new products?

3. Technical & product state

- [ ] **Implementation maturity** – basic tracking or advanced setup?
- [ ] **Known technical issues** – open bugs, workarounds, or frustrations?
- [ ] **Integration landscape** – what else are they using? Any competitors still in play?
- [ ] **Product gaps** – feature requests or limitations that are blockers? Note any features with committed delivery timelines from our product teams
- [ ] **Onboarding completeness** – per the onboarding checklist, which products are properly onboarded?

4. Risks & opportunities

- [ ] **Top risks** – what keeps you up at night? Champion risk, competitor risk, budget risk?
- [ ] **Top opportunities** – lowest-hanging fruit for expansion or deeper adoption?
- [ ] **Unfinished business** – anything you wanted to do but didn't get to?
- [ ] **Anything I should avoid?** Sensitive topics, past friction, internal politics?

After the handover call

Immediate actions (within 1 week)

- [ ] **Update Vitally** – ensure New Owner trait is set, update account plan note with handover context
- [ ] **Save an account plan** – create or update the account plan as a Vitally note, incorporating handover insights
- [ ] **Introduce yourself to the customer** – warm intro (ideally the TAE introduces you) or cold intro via Slack/email
- [ ] **Follow up on any open items** – pick up in-flight proposals, unresolved tickets, or pending conversations

Tips for a good handover

- **Focus the call on what's not in the data.** You can read Vitally yourself – use the call for relationship context, political dynamics, and unwritten history.
- **Ask "what would you do next if you were keeping this account?"** This often surfaces the most actionable insight.
- **Move fast on your intro.** The longer the gap between handover and first contact, the more momentum you lose.
- **Keep the previous owner in the loop** for the first few weeks if there are open commercial conversations. In some cases they can also serve as a secondary support point for timezone coverage or as an escalation contact.

Unassign yourself in Vitally

Once the handover is complete, the outgoing owner should unassign themselves from the account in Vitally. This ensures the new owner is the sole point of contact and avoids confusion about who is responsible for the account.

High potential customers

For TAE-led customers who will be landing at $100k+ a year or have high expansion potential into new product areas, we should introduce a TAM earlier on than normal.

The prime time for this is when the technical win is confirmed - the TAM should be introduced to the customer by the TAE in an evaluation or POC wrap-up call when we know that the customer has selected PostHog.

The introduction is purely for relationship building and continuity purposes so that the TAM can hit the ground running with the customer after the initial credit pre-purchase is signed. It's still on the TAE to work with the customer on the deal, and as such only the TAE will be recognized on the initial deal for commission purposes. After the initial deal is closed won the TAM will take over the account in their book of business.

The TAE and TAM should use their overlapping time to work with the customer on a documented onboarding plan per the above guidance.

Account planning

contents/handbook/growth/sales/account-planning.md

This account planning framework is designed to help you quickly get up to speed on accounts and consistently think through a set of questions that deepen the partnership with your accounts. It encourages a proactive approach to identifying expansion and cross-sell opportunities, driving growth and customer success. This template was initially developed for managing a book of business primarily consisting of smaller startups. It will likely need modification to be useful for larger, more enterprise, accounts.

Here are some times where it may help:

- When onboarding a new account.
- As part of a regular account review cadence (e.g., quarterly).
- When a significant change occurs within an account (e.g., new funding, leadership change, new strategic initiative, product launch).
- To proactively identify and strategize on expansion or cross-sell opportunities.
- Any other time you want to. Or not. No one is checking your work.

Account planning template

I. Business info:

- Name:
- Description:
- Website:
- HQ location:
- Business type: B2B SaaS, E-commerce, Marketplace, Developer Tools, Fintech, Healthcare, etc.
- Goal: Identify typical key metrics and challenges for this company type/stage (e.g., MRR, CAC, LTV for SaaS; GMV, AOV for E-commerce).

A. Business stage:

Businesses have different goals and constraints based on funding and stage. A venture-backed early-stage company may be very cash-conscious and focused on product-market fit, while a more established enterprise may be more focused on scaling, efficiency, and locking in multi-year discounts.

- Current stage: (Seed, Series A/B/C+, Public, Bootstrapped, Acquired by X)
- Funding:
- Investors:

B. How they make money:

- Key product(s) or services they offer: List their main offerings that their customers use.
- Pricing, if available:

C. Vibe-based matrix:

Refer to the vibe-based sales matrix (internal only) - where does this account fit?

- Potential ways to cross-sell:
- Main risks:

II. Product impressions:

You should always try out a customer's product where possible, as it can give you useful context. It helps you identify best practices we can recommend, understand their user experience firsthand, and spot potential cross-sell or value-add opportunities for PostHog.

- Personal impression/user experience notes: Your observations as a potential user – what was intuitive, confusing, delightful? Any obvious pain points or areas for improvement? How do they handle onboarding, core workflows, etc.?
- Potential areas where PostHog could provide insight for their product development/UX: "Their checkout flow has 5 steps; Session Replay could help them identify drop-off points." "They just launched a new mobile app; Product Analytics is crucial for tracking adoption."
- Their product roadmap: What do you see on their roadmap that PostHog can enable for them?

III. Hiring roles / goals:

Review their careers page, LinkedIn job postings, and any announcements about team growth. Hiring trends can tell us what the business will be focused on in the next 12-24 months, what skills they're prioritizing, and what type of growth the business is forecasting.

- Roles currently hiring for:
- Goals associated with these new roles: (Document Product and growth related goals. This tells us what the business will be seeking)
- Skills associated with these new roles: (Are there specific technical skills required? This can educate us about the customer's tech stack.)
- How to position PostHog as an enabler for these roles/goals:

IV. Business objectives :

Collect as much context as you can about the customer and their goals. Look for opportunities to align PostHog to those goals.

- What are they trying to achieve with PostHog? / What larger business goals does PostHog support? Increase feature adoption by X%, reduce onboarding drop-off by Y%, identify top 3 user paths to conversion, validate hypotheses for new product Z.
- Do they feel the value from PostHog aligns with their expectations and investment? Yes/No/Partially. Are they happy with the value received?
- What obstacles are they facing?
    - In using PostHog effectively: Technical hurdles in instrumentation, knowledge gaps in using advanced features, specific feature limitations for their use case, not enough time/resources allocated.
    - In their overall goal achievement (where PostHog might help but isn't fully leveraged): Lack of engineering resources to implement new tracking, siloed data preventing holistic views, difficulty translating data into actionable insights.
- Are there upcoming constraints? Budget freezes, code freezes, headcount restrictions, technical migrations, major product re-platforming, seasonality impacting their business.
- What do their future needs look like (6-18 months)? Where does PostHog fit into their roadmap? Scaling analytics capabilities, new product launches requiring instrumentation, desire for A/B testing at scale, moving towards a more data-driven culture, interest in predictive analytics.
- How healthy do they think the relationship is? Have they enjoyed their interactions with us (Sales, Support, CS)?

V. Stakeholders and power users:

- For key contacts:
    - Name & role / title:
    - Priorities & Goals:
    - Attending any trade shows/events where PostHog might be?
    - Preferred Communication Channel / Cadence:

VI. Current PostHog products in use:

This can be easily checked in Vitally. Thinking through this in a structured fashion may be helpful when taking on a new account.

- Asses the usage or maturity level for each product: Basic (simple insights), Intermediate (custom events, funnels), Advanced (correlation analysis, complex flags)

A. Underutilized products & cross-sell mapping:

In Vitally, you can often easily identify which products are underutilized or not adopted. It may be beneficial to map these out in a more general sense.

- Product 1 (e.g., surveys):
- Potential use case: "Gather qualitative feedback on new feature X." "Run NPS surveys directly in-app."

- Next step to introduce/drive adoption: "Share case study on Surveys." "Offer to help set up their first survey."
- Relevant case study / content:
- Product 2 (e.g., A/B testing):
- Potential use case: "Test different onboarding flows." "Optimize CTA button placement."
- Next step to introduce/drive adoption: "Discuss their experimentation roadmap." "Show demo of A/B testing setup."
- Relevant case study / content:

B. Optimization opportunities (for existing products):

- "Not using custom events enough; could help them track X more granularly." "Could benefit from more complex funnels to understand Y." "Haven't explored correlation analysis for Z." "Feature flags could be used for targeted rollouts to segment A."

VII. Context / suggestions from others at PostHog:

Check Active Conversations in Vitally, support ticketing, Slack channels, CRM history.

VIII. Open requests and feedback

Has the customer submitted any feature requests or other relevant feedback?

IX. Risks:

- Aware of any risks to this account's renewal or growth? Champion leaving, key user turnover, budget cuts, low adoption/engagement, unresolved issues, evolving needs, upcoming renewal date with low perceived value, negative sentiment.
- Mitigation strategy:

Accounts overview

contents/handbook/growth/sales/accounts-overview.md

This is a high level overview of where leads and customer accounts go at different stages of their interactions with us. We use various criteria to figure out where the best place is for a customer to go. You find further details in this section of the handbook.

As we grow, this will keep changing!

```
flowchart TB
    A@{ label: "<b>New business leads<br></b>- Booked a demo (organic, paid ads)
<br>- Emailed sales@<br>- Used &gt;50% startup credits + invoice &gt;$5k<br>-
'Cool company' in ocean.io<br>- Using PostHog, $0 spend, trigger for hiring
increase, web/social increase, fundraise" } --> C["TECHNICAL ACCOUNT EXECUTIVE"]
    B["<b>Expansion leads</b><br>-MRR $500-1,667, &gt; 50 employees, &gt; 7
users, ICP country, paying &gt; 3 months<br>- High ICP score + Scale plan<br>-
Off startup plan in next 2 months + last invoice &gt;$1500<br>- &gt;$1k MRR +
&gt;50% change"] --> D["TECHNICAL ACCOUNT MANAGER"]
    n1["<b>Manual leads</b><br>- Anyone can create - use your discretion"] --> C
    n2@{ label: "<span style=\"color:\"><b>Onboarding leads</b><br>-
F</span>irst bill of $100 + business email<br>- Not otherwise a lead" } -->
n8["ONBOARDING SPECIALIST"]
    C --> n4["$20k+ potential? (TAE decides)"]
    n4 -- No --> n3["SELF SERVE"]
    n4 -- Yes --> n5["Close then nurture for 12 months"]
    n5 --> n7["Using replay + flags + error tracking AND expanded to full
potential? (Simon decides)"]
    n7 -- No --> D
    n7 -- Yes --> n6["CUSTOMER SUCCESS MANAGER"]
    n8 --> n9["$20k+ potential? (Onboarding/BDR decides)"]
    n9 -- Yes --> C
    n9 -- No --> n3
    n3 -- "Organic growth to<br>$2k+ MRR" --> n14["Account review<br>(Simon
decides)"]
    n14 -- "TAM criteria met" --> D
    n14 -- "Not yet" --> n3

    D --> n10["Expanded to full potential? (TAM proposes, Simon signs off)"]
    n10 -- Yes --> n6
    n11["BDR"] --> n9
```

```
    n3 -- "Some become..." --> B
    n12["<b>Manual leads</b><br>- Anyone can create - use your discretion"] -->
D
    n6 -- If drops <$20k --> n3

    A@{ shape: rounded}
    B@{ shape: rounded}
    n1@{ shape: rounded}
    n2@{ shape: rounded}
    n8@{ shape: rect}
    n4@{ shape: diam}
    n3@{ shape: rect}
    n5@{ shape: rounded}
    n7@{ shape: diam}
    n6@{ shape: rect}
    n9@{ shape: diam}
    n10@{ shape: diam}
    n11@{ shape: rect}
    n12@{ shape: rounded}
```

Automations

contents/handbook/growth/sales/automations.md

Automations

As Vitally connects all of our Product, Stripe, Zendesk and HubSpot data together it's the best place to trigger automations via Playbooks. These Playbooks can call a webhook after Accounts or Users meet certain criteria. This allows us to call out to Zapier to use their inbuilt actions to update Zendesk, HubSpot, Slack and more. We use Zapier extensively throughout the company for automation. There is a shared Zapier login in 1Password.

Connecting everything together

Vitally requires a consistent `external_id` to be present to link everything together. For Accounts, we use the `posthog_organization_id` and for Users it's their `email`.

Vitally Segmentation

Vitally uses Playbooks to put Accounts and Users into Segments, which are useful for reporting as well as targeting of Playbooks. We have the following Segmentation Playbooks defined:

1. Segment Name: $60K ARR
    - Playbook Link
    - Criteria
        - Account not in Startups Segment
        - ARR >= $60K
2. Segment Name: $20K ARR
    - Playbook Link
    - Criteria
        - Account not in Startups Segment
        - ARR >= $20K AND ARR < $60K
3. Segment Name: Startup Plan
    - Playbook Link
    - Used to track companies either on PostHog for Startups or the YC Program
    - Criteria
        - Stripe Account Balance < 0 (e.g. they have credit remaining)
        - Stripe Credit Expires at > 0 days from today (e.g. it hasn't expired yet)
        - Stripe Is Startup Plan Metadata is not false or null (e.g. they haven't been marked as being on a paid annual plan)
4. Segment Name: Annual Plan

    - Playbook Link
    - Criteria
        - Stripe Subscription interval is yearly
        - OR
        - Stripe Account Balance < 0 (e.g. they have credit remaining)
        - Stripe Credit Expires at > 0 days from today (e.g. it hasn't expired yet)
        - Stripe Is Startup Plan Metadata is false or null (e.g. they haven't been marked as being on the startup plan)
5. Segment Name: Active Trial
    - Playbook Link
    - Criteria
        - Free Trial Until is greater than 0 days from now (comes from the Billing Postgres connection)
6. Segment Name: First payment forecasted this month
    - Playbook Link
    - Criteria
        - Lifetime Value (LTV) is 0 (e.g. they have never paid us)
        - Stripe current period end is greater than 0 days from now
        - Forecasted MRR > $1

Vitally -> Zendesk/HubSpot Automation

As Vitally has Subscription/Segment information it's the best place to drive Zendesk tagging and other activities.

Ensuring that Vitally Accounts have corresponding Zendesk Organization and HubSpot Companies associated with them

The New Orgs to Zendesk and HubSpot via Zapier playbook triggers on Accounts where there is *no associated Zendesk ID* but there *is a Stripe Customer email*, so that we can look up the contact and company information in HubSpot. When these criteria are matched the playbook sends the following traits to a webhook which triggers the Vitally Webhook to New Zendesk Org and HubSpot Zap:

- orgName - PostHog Organization Name
- orgID - PostHog Organization ID
- customerID - Billing Customer ID
- email - Stripe Email
- siteURL - the URL of the PostHog Cloud they're on (useful to have in Zendesk)

The Zap then:

1. Tries to find a HubSpot contact matching the email
2. If successful, looks up the associated HubSpot Company

3. Sets the posthog_organization_id property in HubSpot so that Vitally will be able to link to the Company
4. Creates a Zendesk Organization with the following properties:
    - `name` - Billing Customer ID - PostHog Organiation Name (e.g. 12345 - PostHog) which guarantees uniqueness
    - `external_id` - PostHog Organization ID
    - `ph_external_id_org` - PostHog Organization ID custom field (because `external_id` cannot be used in triggers and automations)
    - `domain` - The domain name from the HubSpot Company Object (which might not exist - see below)

There are some scenarios (e.g. gmail signups) where HubSpot doesn't have an associated company record and as such there won't be a domain to supply to Zendesk. In this case the automation completes but also adds the Email and Zendesk org information to the Zendesk Orgs Without a Domain table. For each row there are two buttons:

- Add Domain to Org - Will fire a Zap to extract the company name from the email and set it on the Zendesk Organization. Use this one if it's clearly the right company domain.
- Add User to Org - Will fire a Zap which adds that individual user to the Zendesk Organization. Use this if it's a webmail provider (e.g. gmail) as we don't everyone with a @gmail.com email creating tickets for this Organization, but do want this user to get the right level of support for their Organization.

Tagging Zendesk Organizations based on Segment and Subscription information

As Vitally is the best source of truth for Active Subscription / Payment information which informs our Zendesk ticket prioritization, there are a number of Vitally playbooks which will trigger the webhook associated with the Vitally Webhook to Zendesk Tags Zap, passing along the following traits:

- zendesk_id - The Zendesk Organization ID (internal, not external_id)
- playbook name - The tags to set on the Zendesk Organization

The Zap then updates the specific Zendesk Organization with the requested tags.

1. Zendesk Tag: `priority_customer`
    - Playbook Link
    - Criteria
        - Account is PostHog
        - Account not in Startup Plan Segment
        - ARR >= $20K
        - Zendesk Org ID is set
2. Zendesk Tag: `paying_customer`
    - Playbook Link
    - Criteria
        - Account not in Startup Plan Segment

        - ARR > 0 and < $20K
        - Zendesk Org ID is set
3. Zendesk Tag: `non_paying`
    - Playbook Link
    - Criteria
        - Account is not PostHog
        - Account not in Startup Plan or Active Trial Segments
        - ARR = 0
        - Zendesk Org ID is set
4. Zendesk Tag: `churned`
    - Playbook Link
    - Criteria
        - Account is not PostHog
        - Account not in Active Trial Segment
        - Stripe Subscription Status is Cancelled
        - Zendesk Org ID is set
5. Zendesk Tag: `startup_plan`
    - Playbook Link
    - Criteria
        - Account in Startup Plan Segment
        - Zendesk Org ID is set
6. Zendesk Tag: `trial`
    - Playbook Link
    - Criteria
        - Account in Active Trial Segment
        - Zendesk Org ID is set

Setting the correct organization in Zendesk for new tickets

Zendesk uses the requester's email domain to associate the ticket and the requester with an organization in Zendesk. To mitigate the problems this causes, we have a Zap named Set user's correct organization ID which:

- Checks each new ticket for a value in the custom ticket field `organization_id`. (This ID is added to each support ticket submitted via our Help sidebar `Email an engineer` form, and community questions.)
- Looks for a Zendesk org with an `external_id` which matches the value of the custom ticket field `organization_id`
- If no match is found, the Zap stops and does nothing
- If a match is found, the Zap sets the user's Zendesk organization accordingly, and then sets an `org-checked` tag to prevent the Zap from running repeatedly on the same ticket.

[Deprecated] HubSpot and Zendesk tagging

The Zaps in this folder have been mostly turned off in favour of the Vitally automations above, however there are some which are still enabled as we need to figure out how to handle them via Vitally.

Billing trial activated event -> HubSpot and Zendesk

This needs to be moved to Vitally

This Zap is triggered when a trial is activated in the Billing UI (triggered by the Billing trial activated action).

1. Looks up the associated email in Clearbit
2. Continues only if there is an associated Company in the Clearbit payload
3. Calls the Update tags on Zendesk org Sub Zap to create/update a Zendesk org with the Name and Domain from Clearbit and Organization ID as the Zendesk External ID (so that it will link the Zendesk org to the Vitally Account)
4. Finds the Company by name in HubSpot
5. Sets the Organization ID and Trial end date in HubSpot.
6. Creates an engagement (Task) in HubSpot for Simon reminding him of the trial end date.

HubSpot Automation

There are three zaps in this folder which create follow-on deals when any hands-on pipeline deal is closed:

1. HubSpot Inbound Hands-on Deal Closed to Renewal Deal (Zapier automation details)
2. HubSpot PQL Hands-on Deal Closed to Renewal Deal (Zapier automation details)
3. HubSpot Renewal Deal Closed to Renewal Deal (Zapier automation details)

They're triggered by a deal closing in the respective pipeline. It figures out the new deal close date based on the term in the existing deal (1,2,3 years) and then creates a new deal in the renewal pipeline, with amounts and ownership copied over too.

Sales Pipeline Events to PostHog

This folder contains Zaps which ensure we are tracking pipeline updates as PostHog events, so that we can model our sales pipeline as a funnel.

Calendly Event Scheduled to PostHog

This Zap is triggered when a new event is created via Calendly, this:

1. Looks up the PostHog Distinct ID via the email address of the person

2. Captures a `calendly.event_scheduled` event in PostHog with either the Distinct ID above or email address as the Distinct ID if there wasn't a match.

HubSpot Deal Stage Changes to PostHog

This Zap is triggered when a deal stage is updated in HubSpot, this:

1. Transforms the HubSpot ID of the Pipeline and Stage to the names via lookup tables and only carries on if matches are found
2. Gets the Deal Contact and Owner information
3. Captures a `<pipeline-name> <stage-name>` event in PostHog with the Contact email as the Distinct ID

Annual Plan Automation

To ensure consistency in the setup of annual plans we have Zapier Automation to take care of all of the Stripe-related object setup.

Load Contract Details to Annual Plan Table

Once an Order Form is closed in PandaDoc, This Zap will add a new row to the Annual Plan Table with the following information set:

1. Order Form ID
2. Customer Email
3. Customer Address
4. Company Domain
5. Contract Start Date
6. Contract Term (months)
7. Credit Amount
8. Discount
9. Price

Create or Update Stripe Customer

If the Customer has an existing record in Stripe (e.g. they are already subscribed to PostHog) then copy their Customer ID (starts `cus_`) from Stripe to the *Stripe Customer ID* column. If they don't have an existing Customer in Stripe then click the *Create Stripe Customer* button in the table to trigger a Zap to create one. The Zap also automatically adds the ID to the table.

Create Invoice

Once you click the Create Invoice button this Zap will create a Stripe Invoice in draft format. The following table fields need to be populated for this to work so check them before clicking the button:

1. Start Date

2. Term (months)
3. Credit Amount
4. Price

Once it's completed it'll populate the table with the Invoice ID and Link. Review this in Stripe, and when you are ready send the Invoice to the customer.

Note: You need to send the invoice to the customer before you apply the credit below. If you apply the credit whilst the Invoice is in a draft state it'll just pay the invoice with the credit, which defeats the purpose

Apply Stripe Credit / Zendesk Tags

Here you can click *Apply Credit* to trigger a Zap which applies the Stripe Credit and Zendesk tags using the corresponding Sub Zaps. It will apply the `priority_customer` tag if the price is above $20k, and `paying_customer` otherwise.

Schedule Subscription

If the customer doesn't already have a running monthly subscription this Zap will create one with the desired configuration of paid products. Select the products you want to include and then click the Schedule Subscription button. It'll create a Subscription which is either Scheduled if the Start Date is in the future, or live if it is in the past.

Remember to update the Subscription in the Billing Admin Portal

Note: It has the current default Stripe Price IDs hardcoded in the Zap so if we update those we need to remember to update them in this Zap too.

YC Program

This process is documented in the YC Onboarding section of the handbook.

PostHog for Startups

Work in progress

Sub-Zaps

These are used in a few different places to ensure we do things in a consistent manner. It also ensures repetitive tasks are easy to update if needed.

[Deprecated] Update tags on Zendesk org

Mostly deprecated as we use Vitally for this now

This Zap ensures that a Zendesk org is created and tagged correctly

1. Accepts the following inputs:
    1. Company name (required)
    2. Domain (required)
    3. Tags
    4. Organization ID
    5. Instance
    6. Startup plan or Trial ends at
2. Formats tags and startup/trial ends at in case of missing data
3. Formats startup/trial end in YYYY-MM-DD
4. Creates or Updates an organization with the information above

Apply Stripe Credit

This Zap applies credit and associated metadata to a Stripe Customer object

1. Accepts the following inputs:
    1. Duration (e.g. 1 year or 6 months)
    2. Stripe Customer ID
    3. Amount (dollars)
    4. Description (optional)
    5. Credit start date
    6. Is startup credit
2. Calculates the credit end date from the Start Date + Duration
3. Converts Dollars to Cents (for Stripe)
4. Adds the credit balance via the Stripe API
5. Updates the following metadata on the Customer Object:
    1. `credit_expires_at`
    2. `is_startup_plan_customer`

Billing

contents/handbook/growth/sales/billing.md

Managing billing

> This section explains how PostHog's billing system works. Most billing operations described below are handled exclusively by the and are not self serve. Sales should coordinate with the billing team for any billing modifications, pricing changes, or technical billing tasks rather than attempting to implement these directly.

All PostHog instances talk to a common external **Billing Service**. This service is the single point for managing billing across PostHog Cloud US, PostHog Cloud EU (and ,formerly, self-hosted customers).

The Billing Service is the source of truth for product information, what plans are offered on those products (eg a free vs a paid plan on Session Replay), and feature entitlements on those plans. Our payment provider Stripe is the source of truth for customer information, invoices, and payments. The billing service communicates with Stripe to pull all the relevant information together before responding to customer requests.

Credit-based Plan Automation

To ensure consistency in the setup of credit-based plans we have Zapier Automation to take care of all of the Stripe-related object setup.

Loading contract details

Once an Order Form is closed in PandaDoc, Zapier will add a new row to the Credit-based Plan Table with the PandaDoc ID of the document. The table will have the following information automatically filled in: PandaDoc Order Form, Company Name, Customer Email, Credit Amount, Discount, Price, Start Date, Term, PostHog Org ID.

Upfront Payment Setup

Step 1: Update Zapier table with existing Stripe ID

If this is a new contract for an existing customer, you will need to add their existing Stripe Customer ID manually to the table. You can find this information in Vitally under Traits. If this is a brand new customer, click “Create Stripe Customer” button to assign them a new ID.

Step 2: Create invoice

- Go to the Credit-based Plan Table and click “Create Invoice - Upfront”. This will:
  - Create a draft Invoice object against the Stripe Customer Object.

    - Add the ID of the Invoice to the table (for easy review later on). The due date of the invoice will be the Contract Start Date + 30 days which are our standard payment terms. You might need to manually change this if we have different terms with the customer.

Step 3: Verify invoice details and send

- Click the invoice link in the table to open it in Stripe, or use the Invoice ID to locate the invoice in Stripe.
- Ensure all details are correct, particularly the Customer's Billing/Shipping addresses and Tax ID on the Customer object.
- If the customer has an existing credit balance in Stripe (common for renewals), remove the credit before sending the invoice. Otherwise, Stripe will automatically apply the credit balance to the invoice. After the invoice is sent, you can reapply the credit.
- Send the invoice to the customer and wait for the payment to be completed. Ensure that the customer is aware that payment is via Bank Transfer only (no checks).

Do not proceed to the next steps until invoice is finalized. Any credits added to an account gets automatically applied to outstanding invoices. If you add credits before payment is completed, the credits will settle any existing debts, and customer will not be able to make a payment.

> For customers using Bill.com for payment, when they submit the invoice to the Bill platform it strips out the Stripe virtual account details. You'll need to ask them to follow the instructions in this help article to set the correct bank details for us in the Bill.com platform. The account details is provided in the invoice sent over. They'll need to make sure they use the original contact information and not your email if you're set as signer on the contract so we can process payments in the right account. In case they don't do this, we have a default customer account on Stripe which the money will go to. If this happens, mark their invoice as paid manually and then generate a new one against our default customer account to use the funds.

Step 4: Apply credits

- Make sure that the payment is fully processed to avoid any automatic deductions.
- **If customer wishes to begin using credits immediately:** return to the Zapier table after you've verified payment completion and click the "Apply Credit" button.
- **If customer wishes to begin using credits in the next billing cycle:** ask the RevOps team to apply the credits at the end of the current billing cycle.

Step 5: Schedule subscription

- If the client has an existing subscription, no further action is needed.
- If this is a brand new account:
    - Select checkboxes for all the products the client intends to use as part of their subscription.
    - Click the "Schedule Subscription" button. Using the data from the table row where the button was clicked, this will:

- Consolidate all of the Price IDs into a query string which the Stripe API accepts.
- Create a Subscription Schedule (as it may start in the future) containing all of the prices. We calculate the number of iterations based on the term of the contract. An iteration in this case is 1 year, the maximum allowed by Stripe.
- Add the ID of the Subscription Schedule to the table

Failed/late payments

We define late payments as follows:

1. For credit-based customers, that have not made payment on an invoice and their due date has passed. The first invoice is usually 30 days from the contract start date (Net 30) although can differ based on other contractual terms. This rule applies to all payment terms, including and not limited to annual and quarterly, regardless whether there are still credits available or not.
2. For pay-as-you-go usage-based customers, we will attempt 4 automated payments using the card we have on file. Each failed payment sends an alert to the #sales-alerts Slack channel. After 4 failed payments we will stop attempting to take further payments.

In either of the above scenarios the account owner as defined in Vitally needs to take action to ensure that payment is made. If there is no owner in Vitally, Simon will handle this process. If you are an AE, remember this also has impact on your commission, as we don't pay out until the customer has paid their invoice.

You can find a list of failed and overdue payments in PostHog

Step 1 - On the day their payment becomes late

As the account owner you will be assigned a risk indicator in Vitally, as well as being tagged in an alert in #sales-alerts. For unmanaged accounts with a failed payment of $1500 or more Simon and Dana are tagged instead.

You should reach out to any known contacts, as well as any finance email addresses we have in Stripe asking for payment to be made immediately. For credit-based customers, you can download the Invoice PDF from the Stripe invoice page, and for monthly customers you can get the payment link from the Stripe invoice page. To get a payment update link, click on the subscription, then click actions in the top right corner and choose share payment update link. Make it easy for them to make payment by including these details in your email.

> Make it clear in this outreach that if we don't receive payment in the next 7 calendar days, their user access will be suspended. If they come back to you with genuine reasons why they need more time, use your discretion with the next steps.

Step 2 - 1 day before suspending user access

Reach out to all active users on the account, and let them know that access will be suspended tomorrow due to the failed payment. This often creates urgency and will get any late payment resolved.

Step 3 - Suspending user access

To prevent users from being able to log in you need to go to the Django admin panel for their organization, then set the "Active" field to "No", with the reason selected from the dropdown: "Access revoked due to an unpaid balance." Then, hit save.

After completing this, email or Slack all users in the organization letting them know that access has been suspended and what they can do to rectify the situation. Also make it clear that if this isn't resolved within the next 7 days we will revert them back to the Free tier and they be subjected to the usage limits of that tier (e.g. they are likely to lose tracking data).

> If they do pay after this point make sure to re-enable user access by reversing the above in Django admin.

Step 4 - 1 day before cancelling their subscription

Reach out to all contacts letting them know that due to the failed payment we will be terminating their subscription tomorrow.

> Make it clear in this outreach that once the subscription is terminated they will be subject to the free tier usage limits and we won't store any data above that limit.

Step 5 - Cancelling their subscription

You can cancel their subscription in Stripe - navigate to their Stripe customer page, and then click the `...` next to their active subscription to find the Cancel option.

At this point they will be notified about this automatically via the billing service.

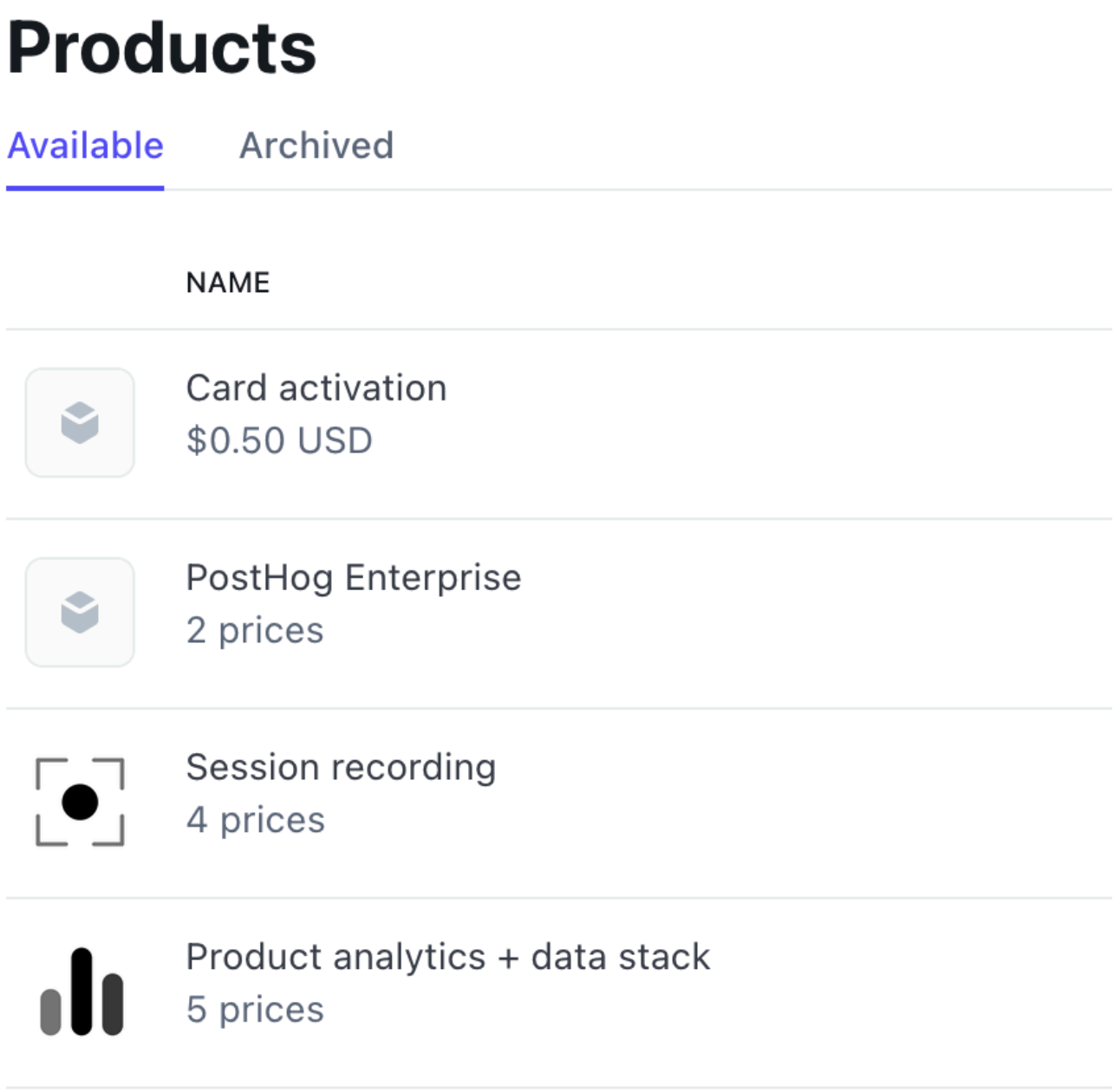
Repeated failed payments

After three consecutive missed payment periods, the customer must provide advance payment covering three months of service based on their typical usage before account access is restored. If the customer disagrees or fails to make the advance payment, the account may be reverted to the Free Tier.

Stripe Products & Prices

> ⚠️ Product and price modifications are restricted and handled exclusively by the . These changes are only made in rare cases and require billing team approval and implementation. Do not attempt to modify products or prices directly - contact the billing team for any pricing-related requests.

Each of our billable Products has an entry in Stripe with each Product having multiple Prices. We use a billing config file to determine what is shown in the UI and how billing should behave. We use very limited metadata on some of these prices to allow the Billing Service to appropriately load and offer products to the instances:



Custom metadata

On Stripe Products

- `posthog_product_key`: `posthog_analytics` | `session_replay` | ... -> This allows PostHog to find and map the relevant products. **Important:** There should never be more than 1 Stripe product with the same `posthog_product_key`. The list of keys is defined in the main billing config.

On Stripe Product Prices The following keys are used to manage Startup prices:

- `plan` - Any Startup plan prices must have the `plan` metadata set to `startup` to have their subscription automatically moved to the default (paid) prices. If not, when their subscription ends they will instead be switched to the free plans for all products.
- `valid_days` -> The number of days a price is valid for, before automatically switching to another plan (the `default` plan unless `move_to_price_id` is set). Useful to create pricing that is only valid for a specific period, e.g. for the startup plans. Note: if more than one price with `valid_days` is added to a subscription, the validity period will be the *shortest* of the two, before resetting all plans to the default ones
- `move_to_price_id` -> Can be used to specify if the customer needs to be moved to a specific pricing, rather than the default one, at the end of the subscription period.

Working with pricing

Each Product has multiple prices that can be used in a subscription. Which price is default depends on the billing config file. The `default` price in Stripe does not affect the actual default price for a product. This is instead defined in the billing config. In general, if coming from the UI, a customer will subscribe to certain prices depending on the config. There are special prices named `Free` which can be used to give a product for free. These can be added manually and are typically used for Enterprisey customers who pay a flat fee up-front and $0 for the actual usage (which we still want to track but not charge for).

Types of billing plans we support

We generally support the following types of billing plans:

- Standard metered
  - This includes usage-based and metered, even if it has custom price tiers or is a special program like the Startup program.
- Metered, but with discount coupon
- Flat first tier, metered after
- Up-front payment, $0 first tier, metered after
- Flat up-front, no metering (renegotiate contract if they go over)

If at all possible, it's best to stay with these types of billing plans because we already support them, and adding extra stuff will increase complexity. If you do need to add a different type of billing plan, chat with the before agreeing to anything with a customer to make sure it's possible!

Coupons and Discounts

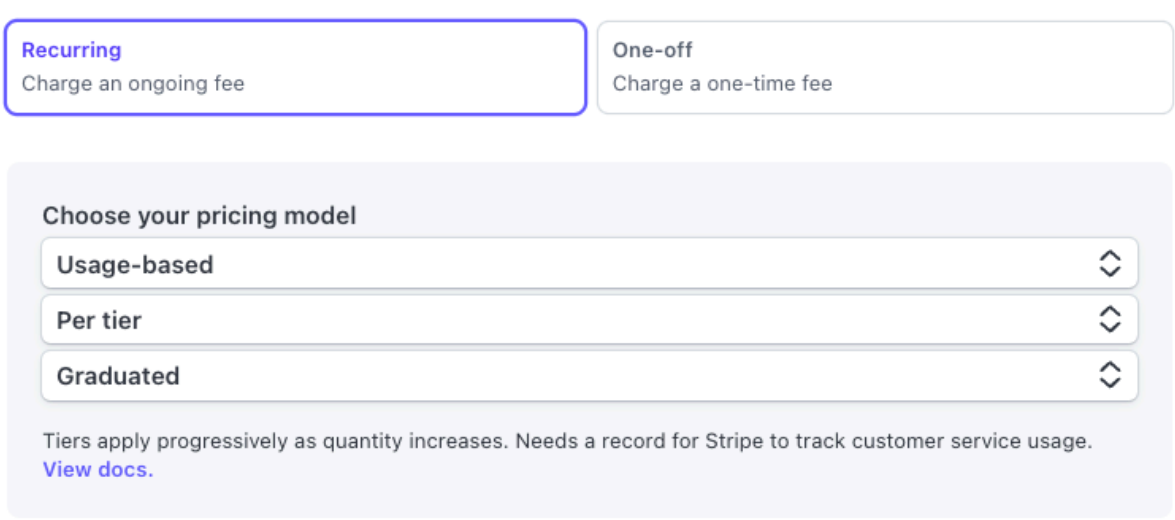
As much as possible the existing prices should be used in combination with `Coupons` to offer custom deals to customers. Coupons are applied to the *Customer* in Stripe, not to the customer's subscription.

1. Visit the customer in the Stripe dashboard.
2. Select Actions -> Apply Coupon.
3. Select the coupon to apply.

4. The UI should soon reflect the change. If you need it to reflect immediately, use the "Sync selected customers with Stripe" action in Django Admin.

When calculating usage limits, discounts are taken into consideration *before* the limit is calculated. This means that if the customer sets a billing limit of $200 and has a 20% discount, they will get charged $200 for _$250 worth of volume_.

Creating new or bespoke prices

1. Go to the appropriate product in question (**do not create your own Product**)
2. Click "Add another price"
3. **Important**: For metered products (e.g. Product Analytics, Session Replay), set up the price as follows:
    - Select `Recurring`, `Usage-based`, `Per tier`, and `Graduated`.
    - 

    - Under Advanced, set the "Metered usage charge method" to `Most recent usage value during period`. This is crucial as the Billing Service will send the correct number of units (events, recordings, etc) every day, so any errors that caused excess usage to be reported can self-heal with the next reporting cycle.
    -

4. Expand the `additional options` and add a straightforward Price Description like `Custom – {date of creation}`
5. Add the tiers as you see fit
    - If the custom prices are for a product and addons (eg. Product analytics and Group analytics) the tier volumes need to be exactly the same between the two products/prices. If tier 3 for Product analytics is up to 15M and tier 3 for Group analytics is for 16M, you'll get errors from the billing service).

- If you are making a custom price for just one product (ie. someone is getting special pricing for Product Analytics but will get the normal pricing for Group Analytics), make sure the tiers match up between the main product and the addons.

6. Add custom metadata if needed.

Plans

> ⚠️ Plan modifications are handled exclusively by the . Do not attempt to modify plans directly, contact the billing team for any plan related requests.

You can find a list of available plans in the billing repo. These are found inside `costants/plans`, divided by folder. Each plan can have a list of features, and a price. Features are used to infer which features are available in the product, for a customer on that plan. You can manually change the plan for a customer by updating the `plans_map` in the billing admin panel.

Paid features for employee side projects

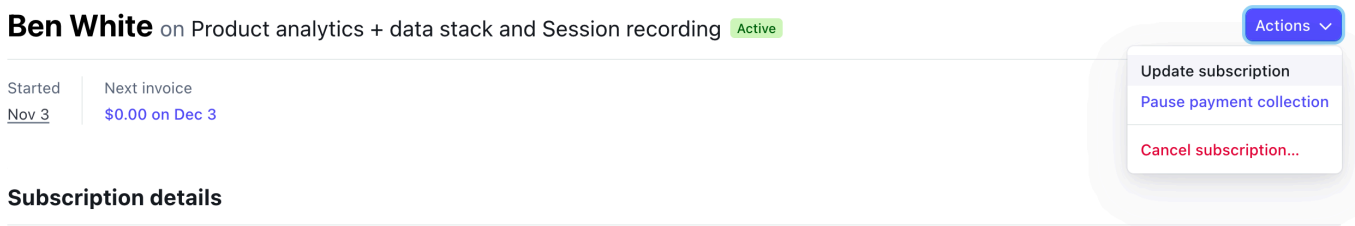
Employees can get access to paid features (like Boost) on personal or side projects. Ask in #team-billing with your organization ID and someone can set this up. There are two approaches for platform add-ons:

1. **Special billing-only plan**: Add a plan like `boost-addon-20250602` to the customer's `plans_map` in the billing admin. These plans exist only in the billing system and grant features without a Stripe subscription.

2. **Long trial**: Create a trial that does not auto-convert with a long `expires_at` date. This works well for temporary access or when you want a clear end date.

Updating subscriptions

Stripe subscriptions can be modified relatively freely for example if moving to a custom pricing plan.



1. Look up the customer on [Stripe dashboard][stripe_dashboard] using their email address or Stripe ID (this can be found in the Billing Service admin under `Customers`).
2. Click on the customer's current subscription.
3. Click on *Update subscription*.
4. Remove the old item from the pricing table and add the new item.
    - Enterprise: Use existing enterprise prices or create new ones.
    - Startup plan: Use existing Startup plan prices.
5. Click on *Update subscription*. Do not schedule the update for a later time. There will be unintended side effects if the changes are not applied immediately.

6. Do not prorate the subscription.
7. The changes should be reflected for the user within a few minutes.

> **NOTE:** Removing a metered product price (events, recordings) and adding a new price will likely reset the usage. This is fine as the Billing Service will update it during the next sync.

Self-hosted differences

Self-hosted billing is no longer supported except for legacy customers who were using the paid kubernetes deployment.

Billing for data pipelines

For information about data pipeline pricing and billing, please visit our pricing page.

Communication templates for new feature adoption (TAMs)

`contents/handbook/growth/sales/communications-templates-feature-adoption.md`

Purpose

Marketing drives awareness at scale. TAMs help customers get value in their own projects. This page gives a simple plan to turn new launches into real outcomes for specific customers.

See also: How we work.

How this differs from marketing comms

- **Audience:** Named accounts and real people, not a marketing list
- **Goal:** First use of new feature and proof of value, not clicks or reach
- **Tone:** Consultative, specific to their setup
- **Channel:** Slack if shared

Cadence

Before new feature launch: internal heads-up
TAMs get notified early about a launch. Understand why it matters, what it does, and who benefits. Write a few short notes for target accounts.

Launch day
Marketing sends the announcement.

After the marketing launch: personal nudge
Send a short note right after the marketing announcement to build on the awareness. The TAM message should tie the feature to the customer's stated goal and give one clear action they can do now in their own PostHog project.

> Example
> "Last quarter you set a goal to reduce activation drop-off. We just shipped **[new feature]**. You can turn it on in your project and try it on your onboarding flow. I recorded a 30-second Loom in a demo project: {loom_link}. If helpful, here is the direct link: {project_link}."

A week or two after launch: data-triggered follow-up
Look at usage in their project. Follow up based on what happened.

- **Adoption detected**
  "Looks like **[new feature]** is on in your project. Is it moving {goal metric}? If you have notes, I will pass them to the product team."
- **No adoption**
  "Was thinking about your goal to achieve **[goal metric]** and how **[new feature]** might help with that. So I wanted to send a nudge in case it fell off the list. You can switch it on here: {project_link}."

Next QBR after launch
Bring the feature as a solution to the overall strategy, not a simple list of new features to go through. "Given your target to improve {goal}, we can be more relevant with these improvements: **[selected new items]**."

Realistic examples

Experiments → no-code experiments
"You said you want to lift **{goal}**. No-code experiments lets PMs ship A/B tests without engineering. Turn it on here: {project_link}. Start with **{page_or_flow}** where we saw friction. Check **{metric}** in this view: {report_link}. Short Loom with the steps: {loom_link}."
See: no-code web experiments and getting started with experiments.

Feature flags → quick holdout
"You're planning to roll out **{feature_or_copy_change}** to improve **{goal}**. Keep a **{holdout_percent}%** holdout on the flag so you can see real lift before full rollout. Flag settings: {project_link}. First results show here: {report_link}."
See: feature flags and holdout testing tutorial.

Session replay paired with an insight
"We saw a drop-off at **{step_or_page}**, which blocks **{goal}**. Create this funnel **{event_sequence}** and open replays linked to step **{step_number}**. Start here: {report_link}. This pairs the number with the clips so you can see why."
See: session replay.

Workflows
"Follow-ups after **{event}** are manual today, so **{goal}** slips. Turn on a workflow that triggers on **{event_or_property}** and sends **{message_or_action}**. Enable here: {project_link}. First run appears here: {report_link}. Adjust, then expand."
See: workflows – start here.

LLM analytics
"You're aiming to increase **{success_rate_metric}** for **{ai_feature}** and keep **{cost_metric}** in check. Turn on LLM analytics: {project_link}. Watch prompts, responses, success rate, and cost per **{n}** prompts. First view to check: {report_link}."
See: LLM analytics – start here.

Heatmaps
“People hesitate on **{page}**, which hurts **{goal}**. Open heatmaps: {project_link}. Compare **{version_or_date_range}** to see what changed. First view: {report_link}. Use this to pick the next tweak.”
See: heatmaps.

Potential measurement

- **Feature adoption rate** in the targeted accounts
- **Time to first use** from the launch-day note
- **Built-in product surveys**
  Some areas collect feedback in product. Watch for those notifications from your accounts.

Potential user segmentation for message adjustment in the future

Power users and beta candidates
“You are a heavy user of {area}. **[New thing]** is ready. You can turn it on here: {project_link}. If you want a head start, I can share a tiny checklist.”

Flag users without experiments
“You already ship behind flags. Add a small holdout on the next release so you can prove lift before rollout. Here is the page in your project: {project_link}.”

Single-product users with nearby needs
“You use {current module} to hit {goal}. {Adjacent module} helps with the next step. Short Loom with setup: {loom_link}. Direct link in your project: {project_link}.”

Adoption laggards on the core path
“Checking in on **[new feature]**. You can enable it here: {project_link}. If it is not a focus right now, all good.”

High-traffic, low-conversion areas
“{page or step} has volume and a drop-off. Try **[new feature]** here first. I can share a minimal setup so you see a signal this week.”

Automation ideas

- We should use Vitally and data from their PostHog instance to get automated account recommendations for which accounts would benefit from which new features the most.

Communication templates for funding and exits

contents/handbook/growth/sales/communications-templates-fundraising.md

Purpose

When a PostHog customer raises a new round of funding, gets acquired, or goes public, it's a major moment for their team. Our goal is to celebrate the milestone in a way that's *genuine, brief, and human*, not transactional or opportunistic. We avoid product pitches, feature plugs, or follow-up asks in the initial message.

1. Communication principles

Keep it human, not salesy The first message should feel like a note from one person to another—not a brand announcement. Congratulate them sincerely, acknowledge their achievement, and stop there.

Example: "Massive congrats on the Series C! I imagine this is a huge moment for you and the team — hope you're all taking time to celebrate."

Timing matters

- **Day-of:** Congratulatory message (no CTA).
- **~3–4 weeks later:** Follow-up that references specifics from the company's press release, interviews, or roadmap to open a thoughtful, relevant conversation.

Personal > Personalized We avoid template-like phrasing. If we can't find something personal to say about the customer or their journey, it's better to say less.

Channels

- Slack (preferred, if shared).
- Email (if Slack unavailable).
- Optionally LinkedIn comment/like from company handle (but no DM from AE unless relationship exists).

2. Follow-up framework (3–4 weeks later)

Use their own public statements as the hook. Reference their goals, product direction, or challenges expressed in press coverage or announcements, and connect them meaningfully to where PostHog can help.

Structure:

1. Open by referencing their recent announcement and one or two specifics (quote, metric, or goal).
2. Briefly connect that to how PostHog can support those goals.

3. Ask a question or offer to share something relevant (not a demo or pitch deck). *Example:* "Hey — congrats again on the Series B! I read about Heidi's plans to scale your AI work globally and tackle latency head-on. We've been working on similar challenges at PostHog around speed and reliability for teams deploying AI at scale. Let's talk about what's worked for other customers in keeping things fast and cost-efficient as usage grows — when's a good time to connect?"

3. Key takeaways

- **Don't sell in the moment.** Celebrate authentically.
- **Follow up with relevance.** Use their words, not ours, to frame the value.
- **Keep the tone human, concise, and professional.**
- **When in doubt:** err on the side of sincerity over specificity.

Communication templates for incidents

contents/handbook/growth/sales/communications-templates-incidents.md

When things go wrong, our priority is simple: **keep customers informed, quickly and clearly.**

This section covers how we communicate during service disruptions, from small hiccups to major outages. We aim to be **transparent, human, and proactive** — sharing what we know (and what we don't) in plain English.

> For the engineering incident response process, see Handling an incident.

PostHog customers rely on us to power their products, so we provide honest, timely updates through the right channels — usually Slack or email, and occasionally SMS for high-touch accounts.

Core principles

1. Transparency > Perfection

Share what we know, when we know it, clearly and without "status-speak."

2. Human-centric

Messages come from people, not "The PostHog Team." Show empathy and ownership ("I know this might interrupt your work; here's what we're doing.")

3. Consistency

Use a consistent structure and timing so customers know what to expect.

4. Proactive by default

Reach out before customers ask, even if it's just to say, "We're aware and investigating."

Severity levels

Level	Description	Examples	Channels	Cadence
SEV 0 – Emergency	Existential service failure; all or most customers impacted with no workaround. CMOC sends immediately via broadcast. Account owners do **not** gate comms.	Full platform outage, data loss, security breach with active customer impact.	Pylon broadcast → Email → DM/SMS	Immediate broadcast, then every 15–30 min; postmortem within 24h

Level	Description	Examples	Channels	Cadence
SEV 1 – Critical	Major outage or data loss; widespread impact.	API unavailable, ingestion halted, login failures.	Slack → Email → (DM or SMS if needed)	Every 30–60 min; postmortem within 48h
SEV 2 – Major	Partial degradation or downtime; workaround available.	Replay or query delays >30 min, flag evaluation slow.	Slack or Email	Every 1–2 hrs
SEV 3 – Minor	Limited impact or slow recovery.	Billing sync delays, isolated org issues.	Slack	Start and close
SEV 4 – Informational / Planned	Maintenance or recovered incidents.	DB upgrade, scaling events.	Email or Slack broadcast	Before + after window

Templates

Emergency (SEV 0)

> **This overrides the standard workflow.** CMOC sends directly to all affected customer channels via Pylon broadcast without waiting for account owners. Account owners follow up individually once online.

Initial broadcast (Pylon):

We're investigating a major incident affecting [feature/service]. [Symptom — e.g., "Event ingestion is fully stopped" or "The PostHog app is unreachable."]

Our engineering team is actively working on a fix. We'll post updates here every 15–30 minutes until this is resolved.

Update template:

Update on [feature/service]: [Status — e.g., "Root cause identified. Fix is being deployed." or "Still investigating. No ETA yet, but narrowing it down."]

Next update in ~[15/30] minutes.

Resolution template:

[Feature/service] is back online as of [time UTC].

Root cause: [one-line summary]. Duration: [start–end]. Impact: [brief description of what customers experienced].

We're monitoring closely. A full postmortem will follow within 24 hours.

If you experienced data gaps or have concerns about impact to your project, reply here and your account owner will follow up directly.

Critical

Subject: PostHog Outage – We're investigating

Hey [Name/Team],

We're investigating a major outage affecting [feature]. You may see [symptom]. Engineers are on it — updates every 30 minutes until resolved.

We know this may disrupt your work — thanks for your patience while we get things back online.

— [Your Name], PostHog

Follow-Up (Resolution): Good news — the issue is resolved. Root cause: [summary]. Duration: [start–end]. Impact: [brief effect].

We're monitoring and will share a full write-up within 48 hours.

Major

Subject: Performance issues in [Feature]

Hey [Name],

We're seeing performance issues in [component]. You might notice [impact]. We're mitigating and will update within the hour.

Thanks for your patience! — [Your Name], PostHog

Minor

Subject: Slower performance in \[area\]

FYI — This shouldn't block you, but we're monitoring closely. I'll update once it's stable.

Planned maintenance

Subject: Maintenance – [Service/Region]

Heads up — maintenance on [system] from [time window]. No downtime expected, but queries or replays may be briefly delayed. We'll confirm once complete.

Tone and voice

Principle	Example	Avoid
Direct	“Event ingestion is paused.”	“We are experiencing an issue affecting a subset of users.”
Empathetic	“I know this blocks work; it’s our top priority.”	“We apologize for the inconvenience.”
Plain English	“Dashboards might not update.”	“You may experience degraded query latency.”
Ownership	“We identified a config issue on our side.”	“A third-party dependency caused an issue.”

Coordination within GTM

Engineering manages detection and resolution (see engineering incident handbook). GTM ensures clear, consistent customer updates, without duplication or coverage gaps.

Goals

- Keep a single source of truth for comms, managed by the CMOC.
- Maintain global coverage so customers always hear from us.
- Enable fast, clear handoffs between teams.

Roles & responsibilities

Role	Responsibility
Communications Manager On-Call (CMOC)	Activated for any incident requiring GTM notification. Drafts all comms using handbook templates. Coordinates with engineering for context and keeps a central log of who’s been notified. Manages regional handoffs if incidents span time zones or owners are offline.
AM/AE/CSM	Sends comms to their accounts using CMOC drafts. If offline (PTO, off-hours, or time zone), CMOC assigns a regional backup.
Regional Backup (Americas / EMEA / APAC)	Covers accounts when owners are offline. Takes handoff from CMOC, sends comms, and ensures follow-up continuity.
Engineering Incident Lead	Owns technical response and provides updates to CMOC for accurate messaging.

> All coordination between CMOC and Account Owners should happen in #group-cs-sales-support transparently so that everyone who manages customers is in the loop.

Workflow

> **SEV 0 override:** For SEV 0 incidents, the CMOC skips steps 4–5 and sends the initial message directly via Pylon broadcast to all affected customer channels. Account owners are notified in #group-cs-sales-support simultaneously, and take over individual follow-up threads once online. The CMOC continues to own broadcast updates until the incident is resolved or downgraded.

1. **Incident declared** (Engineering).
2. **CMOC activated**, notified of impact.
3. **Assess customer impact**, this insight (or this Google Sheet as a backup) will help you understand which customers are using which components in which cloud environment.
4. **CMOC drafts the initial message**, shares with the Account Owners in #group-cs-sales-support
5. **AM/AE/CSM sends to accounts**; backup sends if primary is offline.
6. **Updates** drafted by CMOC (30–60 min for SEV1, 1–2 hrs for SEV2).
7. **Regional handoffs** coordinated by CMOC.
8. **Resolution**: CMOC drafts closure; AM/AE/CSM (or backup) sends.
9. **Post-incident**: CMOC archives thread; GTM logs feedback and follow-ups.
10. **Postmortem**: Engineering writes technical summary; GTM adds comms learnings.

Example Slack workflow (Critical)

1. **Incident created**: #inc-2025-11-05-posthog-feature-flags-error.
2. **SRE posts summary; CMOC coordinates comms.**
3. **CMOC drafts message** and shares with the **Account Owner** (the person responsible for the affected accounts).
4. **Account Owner sends the message** to their customers. Example outbound: “We’re investigating an outage affecting event ingestion. Updates every 30 minutes.”
5. **During:** “Root cause identified (Redis queue saturation). Fix in progress.”
6. **Resolution:** “Resolved at 11:42 UTC. Write-up soon.”

Using Pylon for broadcasts

It's best that communications are shared directly from the account owner; however, if speed is of the essence, i.e. for a SEV 1 or security issue, and some folks are not yet working or on PTO, the CMOC can use Pylon to send a broadcast to all customer Slack channels en masse:

1. Log into app.usepylon.com with your PostHog Slack account.
2. Go to the *Broadcasts* link on the left-hand side of the navigation.
3. Click *Create Broadcast* in the top right-hand corner of the UI.

4. Enter the message you want to send, ensuring the formatting looks correct in the preview on the right-hand side.
5. Ensure the Send as option is set to PostHog, not your own user (unless you want to handle 450+ potential separate threads)
6. Click *Next* in the top right-hand corner of the UI.
7. Select your audience. You can use the filters to select all channels not owned by specific people, e.g., those who are currently online and communicating 1:1 with customers.
8. Make sure you click *Add to Audience* to add the selected channels to the broadcast.
9. Click *Next* in the top right-hand corner of the UI.
10. Set the engagement notification channel to be #support-customer-success
11. Check that you're happy with the message and audience and click *Send Now* in the top right of the UI.
12. Ask everyone online to monitor #support-customer-success for replies and respond where necessary.

Communications templates

contents/handbook/growth/sales/communications-templates.md

We've put together suggested communications templates that TAMs can draw on for various situations like startup plan roll off, incidents, churn risk increase, or new feature rollouts.

These templates are not meant to be restrictive, but a general idea of how we communicate with customers.

Available templates

- **Communication templates for incidents** - How to communicate with customers during service disruptions, from minor issues to critical outages
- **Communication templates for funding and exits** - How to celebrate customer milestones like funding rounds, acquisitions, or IPOs in a genuine, human way
- **Communication templates for new feature adoption** - How TAMs can drive adoption of new features with targeted, consultative outreach to specific customers

Contract rules

contents/handbook/growth/sales/contract-rules.md

We are transparent about how we contract with customers, including what discounts are available. It's better for them, and better for us. We are allergic to the phrase 'let me talk to my manager and see what we can do' - we follow a principled approach.

Discounts

We don't offer discounts to customers paying monthly, irrespective of commitment.

Although our standard monthly pricing has volume discounts built in, it's common practice when negotiating software contracts for the customer (and their procurement team) to ask for a discount. We follow the 4 discount levers framework, being transparent about what drives our discounting:

The 4 discount levers & why they matter to us

Our general principle is that discounts are earned, not given. Each lever represents real benefit to both parties:

1. **Volume:** The amount of credits purchased - *Larger deals have economies of scale. Our cost to serve a $500k customer is not 10x a $50k customer, so we can share those savings*
2. **Timing of Cash:** When we receive payment - *Money today is worth more than money tomorrow. Cash in hand lets us invest in product, hire engineers, and grow the business faster*
3. **Length of Commitment:** Contract duration - *Longer commitments reduce our customer acquisition costs. We spend less on people doing renewals and can invest more in product development*
4. **Ability to Forecast:** Mutual agreement to timing - *When both parties commit to specific dates (contract close, renewal timing), it helps us plan resources and helps customers secure budget*

Every discount reflects a value exchange that provides a sound basis for our pricing. We don't offer unilateral concessions - better pricing comes from moving on one or more of these levers. This framework gives both sides a clear frame of reference for what drives pricing decisions.

How our discounts work

In our consumption-based pricing model, the first way for a customer to reduce spend is to ensure that they are only sending data to us which is valuable to them. There is different guidance here depending on which product(s) they are looking at.

Beyond optimization, we offer discounts based on four levers:

1. Volume discount (based on credit purchase amount - Customers must qualify for this discount before receiving discounts 2 through 4)

- **$25-59k:** 20% base discount
- **$60-99k:** 25% base discount
- **$100-249k:** 30% base discount
- **$250-499k:** 35% base discount
- **$500-999k:** 40% base discount
- **$1M+:** Contact us for custom pricing

2. Length of commitment discount (additive)

- **1-year commitment:** No additional discount
- **2-year commitment:** +3% additional discount
- **3-year commitment:** +5% additional discount (doesn't stack)
- **4-years or more:** Contact us for custom pricing

3. Timing of cash discount (additive)

- **Net 30 (our standard):** No additional discount
- **Multi-year deals:** +2.5% per additional year paid upfront (i.e. +2.5% for 2 years, +5% for 3 years)
- **Extended payment terms:** -2.5% for every 15 days beyond Net 30 (e.g., Net 60 = -5% from total discount)

> Note: We require upfront payment for all discounted contracts. Quarterly or split payment terms are not available as they impact our cash flow and add administrative overhead. If the full projected amount exceeds budget, customers can purchase fewer credits upfront at the corresponding discount tier and then add more later.

4. Ability to forecast - mutual commitment to timing (additive)

For monthly-to-annual conversions & net new agreements: +5% additional, non-recurring discount

- Available when both parties commit to specific, mutually agreed date for contract signature (this is not an end-of-quarter discount)
- To include the 5% discount on the order form, we require written confirmation by the customer's designated signatory of the customer's intent to sign an order form by a specific, mutually agreed upon date - *this needs to come from the person who will actually sign the order form*
- This is about creating predictability for both sides, not artificial deadlines
- This is a one-time discount, which will be offered once during a monthly-to-annual conversion or net new agreement cycle

For renewals: +5% additional discount

- Early renewal commitment (60+ days before expiration)

If timelines change: We will handle these on a case by case basis, but the default is to withdraw the additional discount if the customer does not sign an order form by the time that was originally agreed.

> You shouldn't offer discounts above the levels outlined here. If you go outside of these rules without clearing it with (TAEs), (TAMs or CSMs), or (as backup), you should assume by default that the deal will not count toward your quota.

Why we require up-front payment for credit purchases

We've found that split payment terms create friction for both teams – customers chasing internal approvals, us chasing invoices, nobody focused on delivering value. When customers pay quarterly or monthly, they often consume credits faster than they pay for them, effectively turning us into a line of credit. We are vendors, not lenders. Our focus is on building the best product, not managing accounts receivable. Up-front payments keep everyone focused on customer success and let us invest cash immediately into building features and support. If a customer needs payment flexibility, we're happy to adjust the credit amount and discount, per guidelines above, to fit their budget while maintaining up-front payment.

Credit for case studies

We don't offer additional discounts in exchange for a case study, as paying for case studies can devalue them. We should be working to get our customers to a state of happiness such that they are willing to tell everyone how great PostHog is without needing to pay for it.

Self-serve discounts

We also offer a way for customers to receive discounts on their usage without talking to sales or being on an Enterprise plan. In PostHog, if a customer meets certain criteria, they will see a banner on their billing page with a call-to-action (CTA) to "buy credits". The form they fill out will be auto-populated with an estimated number of credits based on their last 3 months of usage, but they can adjust this value as needed. They will have the option to pay with the card on file or to receive an invoice. Credits will be applied to their account once the invoice is paid.

Requirements for self-serve discounts:

- 3 or more paid invoices
- Average of $280 or more across the last three invoices
- No open invoices
- Not currently on the up plan, a legacy plan, or having existing credits

Additional notes on self-serve discounts:

- For credit purchases below $25,000, the discount is 10% off. Credit purchases of $25,000 and above follow the standard volume discount structure above.
- Instead of providing all credits upfront, we apply 1/12 of the credits each month for the next 12 months. These credits do not expire for 1 year after they've been applied.

- If a customer uses all credits in a month, they will be billed for extra usage at the standard rate.

Non-profit discounts

We offer additional discounts to non-profits:

- For credit purchases below $25k: 15% discount (instead of standard 10% self-serve or no discount)
- For credit purchases $25k - $100k: An additional 5% on top of standard volume discount
- For credit purchases above $100k: Standard volume discounts apply (no additional non-profit discount)

We use tax law in the country of origin to determine what is a not for profit entity. If a customer can provide proof they fit their country's definition, the discount is applicable subject to the guidance above.

When evaluating a discount, it's important to review our margin calculations to ensure we remain margin positive, especially for larger accounts.

To set up the non-profit discount in Stripe, follow these instructions.

> Non-profit discounts only provide an additional 5% on top of standard volume discounts, and only for credit purchases between $25,000 and $100,000.

Legacy discounts

You might see some customers with a 30% discount on their monthly Stripe subscription. These were added when the only way we billed for PostHog was through event pricing. This was originally designed to offset the cost versus competitors who had unbundled Group Analytics or Data Pipelines. These customers will typically be on a higher per-event price plan, so we should look to get them migrated to standard pricing as soon as possible.

Startup plan discounts

For customers on our startup plan, we offer two months free credit when signing a prepaid deal. This encourages startups to use their credits to understand usage, and then commit to a longer term plan with PostHog. This offer is available until the first billing date after the credits expire. If a customer has used up their credits before the expiration date, they still have until the original expiration date to decide and claim the offer. The amount of free credits is determined by how much they purchase on a prepaid plan. By default, we work with customers on prepaid plans that will cover their usage for the next 12 months.

> Important clarification: operationally this is implemented as free credits applied before the contract start date, not as extra credits inside the contract term unless a specific dollar amount for the free credits is explicitly included under Special Terms.

You should follow the same inbound sales process and work with the customer on understanding and optimizing their usage. Then follow these additional steps take to present the prepaid plan + free credits option(s):

1. Review the customer's average monthly cost
2. Estimate the prepaid equivalent for 12 months of coverage (e.g. [monthly cost x 12])
3. Inform them they can take advantage of this offer, which allows them to:
    - purchase credits equivalent to ~12 months of usage (expiring 12 months after the contract start date), and
    - receive ~2 months of additional usage for free, applied before the contract start date.
4. If the customer wants to purchase fewer credits than the option above, then they will receive an additional 1/6 of the amount they wish to purchase for free.

All free credits associated with startup plan roll-offs are one-time only, and should be denoted in the special terms of the contract as "An additional credit in the amount of XXXXX (offered to customers in exchange for rolling off the Startup plan) to be applied to Customer's account upon signature with the same expiration date."

For contracting purposes, these free credits should either be applied before the contract term or included in the 12 month credit amount. If they are being applied before the contract term, adjust the contract date to start 2 months later and the one-time credits can be applied to cover the 2 invoices before the contract start date. In this case, the credits do not need to be called out in the contract, and the opportunity owner can add these credits as a one time credit in billing admin.

Margin negative deals

In exceptional circumstances, we may explore providing additional discounts which eat into our operating margin for the following cases:

1. They are a strategic logo we'd like to land as a brand-new customer.
2. We are taking their business from a competitor.
3. We are preventing them from churning to a competitor.

For the avoidance of doubt, these types of deals are **very rare** (~1 per year), and not offered to customers with standard usage volumes.

If you believe you have a customer who falls into one of these categories and would like to provide additional credit/discount then in the first instance run through the opportunity details including margin calculation with your manager, who will then clear it with Simon (TAMs/CSMs) or Charles (TAEs).

Additional credit purchase

As it's often difficult to right-size the credit needed for a longer term plan as a standard we offer to honor the discount provided in the original purchase for any additional credit purchased in the first half of a contract term (e.g. 6 months for an annual plan). Within the first 6 months given our billing usage reports we should be able to predict whether the customer is going to run out of credit or not. There are also alerts set up in `#sales-alerts` to help notify account owners about this.

Price guarantees & lock-ins

We do not offer price guarantees for the following reasons:

1. We regularly *lower* prices, which would result in higher costs for customers who've locked in a price
2. We occasionally split or restructure products (e.g. Data Pipelines unbundled), which makes guarantees administratively complex
3. Customers are in full control of their usage and can thus adjust their spending patterns as needed

This request most often comes from procurement teams unfamiliar with our pricing philosophy. Address it proactively in commercial discussions, but if there is push back, reference the above points. As an example:

> "We've dropped Events pricing [X]% over [timeframe]. A price guarantee would have cost you more. We're committed to matching the cheapest at every scale—if we're not, tell us. Our prepaid credits for usage based pricing gives budget control without betting against our commitment to low prices."

Multi-year credit allocation

Paid up-front

We will allocate all the credit purchased to the Stripe account when the contract is signed. As above, they can purchase additional credit in the first half of the contract term and take advantage of the same discount as specified in the original contract.

Paid yearly

We will allocate the credit for that year to the Stripe account when the contract is signed, and then again when subsequent annual invoices are raised.

If a customer wishes to use subsequent year's credit early they must agree to pay the invoice for that year early before the credit is transferred.

The additional credit purchase applies to each year separately, e.g. they can purchase additional credits at the same discount level in the first 6 months of each year.

You can see a signed multi-year contract set up in this way by navigating to Documents -> Examples (folder) inside of PandaDoc.

Uptime SLA

Customers only get an uptime SLA if:

1. They have subscribed to the Enterprise add-on; or

2. You agree it with them as a special term as part of their contract if they are spending $100k+ ARR *post* discount (i.e. $ spend, not credit usage).

An uptime SLA are not available to customers outside of these cases. You should certainly not agree to an SLA for customers on regular monthly contracts, and even for annual contracts it is not a given - it's one of multiple pieces you may have in play as you negotiate terms (much like a case study).

More details on how exactly the uptime SLA works can be found in our terms.

Payment method

For customers paying monthly, we only accept credit card payments, which will be taken automatically via Stripe at the end of their monthly billing period.

For customers purchasing credits upfront, we only take bank transfers because:

- For large payment amounts, the fees we incur are higher for credit card payments.
- Our Sales Ops automations are set up to handle bank transfer payments.

You should confirm ahead of the customer signing the order form that they are happy and set up to pay by bank transfer. If they are absolutely unable to accommodate bank transfer we can accept credit card payments under the following conditions:

- We have a card on file which we can immediately charge for the full invoice amount.
- They pay immediately on the contract start date (i.e. no Net 30)

If your customer must pay via credit card, you absolutely *need* to let Mine (Simon as backup) know ahead of the order form being signed as there is a lot of manual work needed up front to make this work.

> We absolutely do not allow payment by check. This is made clear on order forms.

Contract buyouts

> **Are you a potential customer who wants to speak to us about a contract buyout?**
> Get in touch with the Sales team via your shared Slack channel, or reach out directly.

Sometimes customers will be locked into a contract with a competitor, but want to switch to PostHog when their contract is up. In this case, we are willing to let them use PostHog for free for up to 6 months. This is beneficial to PostHog as well, as we can get them set up and using PostHog sooner, capitalizing on the momentum of their interest today, and giving them more time to get comfortable with the platform.

Some rules:

- They need to share a copy of their current contract/pricing/bank statement as proof.
- They sign up to an annual contract worth $20k+/year, paid up front. Their PostHog contract starts when their current one expires.

- Their usage in the overlap period needs to be proportionate to the contract they've signed, ie. if they sign a $50k contract and have 6 months to run, they get $25k of PostHog credit for free.
- The competitor they're using has to be 'real', ie. not some random side project. As a general rule, anyone we have written a comparison article about counts.
- We have final discretion on deciding who gets the deal.
- We can still provide a standard free trial period of 2-4 weeks before they sign the contract, as they will likely need to figure out whether PostHog is right for them before committing.

> Normal commission rules apply here - commission is paid in the quarter in which the customer pays their invoice.

New business renewal credits

If a customer is currently *not* a paying user of PostHog, but is a user of one of our competitors, about to renew, and is shopping for a better deal, we are willing to significantly undercut the quoted renewal price. This is because those customers are not that likely to move over to us anyway, and quoting them a lower price works out in our favour either way:

1. If the competitor matches our much lower offer, and the customer accepts, we've reduced their revenue by a significant amount
2. If the customer accepts, we've gained net new revenue we otherwise would have missed out on, and we have the opportunity to sell more.

In order for this to not mess up later renewals, the way we do this is by giving them credit for the first year in order to reach a total discount of 40%. For example, if the quote from the competitor is $50k, and the total cost for our product (including other discounts) is $40k, we will give them additional credits worth $10k, in order to undercut the total quote by 40%.

In order to qualify for this, the customer needs to send us the full quote document from the competitor.

Credit over/under usage for contracts

When they don't have enough credit to cover their term

We have CreditBot alerts set up in #sales-alerts when a customer is going to run out of credit before their contract term ends, with the estimated runway remaining. The Vitally account owner (AE or CSM) will be tagged in this message. It's best to be proactive here so that the customer is right-sized well before the credit runs out:

- If they will run out of credit or wish to buy more within the **first 6 months** of the contract term, they can still take advantage of their initial discount. You'll need to have them sign a new order form which adds the additional credit, and it should expire on the date of the original order form.

    - Example: Their original order form was signed on 1st January with a 12-month term. Their expansion order form could be signed on 1st June with a 7-month term. Make sure the end date lines up with the end date of the original contract to avoid any issues with the billing server and ARR calculation.
- If they will run out of credit with **less than 2 months** remaining on their initial term, as long as they sign a renewal order form to start at the end of the original contract term we will cover their usage for free until the renewal date, assuming the renewal order form is signed before they run out of credit and their new contract amount is equal to or greater than the current contract amount.
- If they fall **in between** the two cases above (running out of credit with <6 months and >2 months to go) then we need them to sign a new 12 month (or longer) order form lined up with their monthly billing date. This makes ARR calculation slightly trickier as there are two overlapping contracts in play at the same time.
    - Example: Their original order form was signed on 1st January with a 12-month term and they run out of credits in September. We need a new 12-month order form in place with a Contract Start Date of September 1st.

For any of the above scenarios you should use our discounting principles which apply to the credit purchase amount.

> In scenario one above, if their expansion credit purchase takes them into a higher volume discount tier, we should include this discount tier for them in the expansion contract. We won't issue a refund for the difference in discount when the expansion order form discount tier is greater than the discount tier of the original order form.

When they will end the contract term with credit remaining

We can roll up to half the amount of credit from the original order form to a new contract term, provided that the customer signs a renewal contract of equal or higher spend than the original contract.

When a customer doesn't renew their credit purchase

When a customer chooses not to renew a prepaid credit contract we automatically remove any remaining credits on the expiry date. Their account will then roll onto our standard monthly plan and they'll be charged for usage. It's the customer's responsibility to stop sending us events or cancel their subscription and downgrade to the free tier if they don't want to keep paying.

Varying contractual terms

When we vary terms

If a customer wants to vary either our standard template DPA, BAA, or MSA terms, it is a substantial effort for our legal team to review these suggested changes (also known as "redlines").

At a minimum, we will only do this for contracts above $20k a year, and we should expect even higher amounts of committed revenue if they are asking for big changes (e.g. changing significant provisions, adding Service Level Agreements, etc.). A customer needs to either be spending this amount at present, or agree to commit to this spend via an annual contract, in order to initiate legal review of suggested changes. We evaluate all requested changes proportionally against their annual committed spend with PostHog. A customers annual committed spend needs to be defined before proceeding to a negotiation over legal terms, otherwise there is no frame of reference for the negotiation.

We also sometimes receive unsolicited requests to vary our terms. In these instances the legal team will redirect the customer to work with their PostHog contact person for this, as we will only review redlines for a managed customer or opportunity where the potential annual revenue is understood.

> See the guidance below if the customer asks to use their own contract instead of ours

How customers should suggest requested terms

The customer should redline the current .docx version of the document in question. You can find the latest versions of the templates in the Team Internal Info tab in the #team-sales Slack channel (do not save versions locally).

> We don't accept redlines on our standard terms of service and if a customer has proposed this you should share the correct templates with them before involving legal.

Once they have returned the redlines to you first check to ensure that they have used the template which you provided, and then share the document for review in the #legal channel. There will usually be a few rounds of back and forth as we converge on an agreement. You will continue to represent PostHog's position to your customer throughout the negotiation. Please work with #legal on the appropriate responses and speak clearly to our customers.

What customers should expect

PostHog evaluates legal risk assumed against annual revenue received. In other words, contractual terms will be varied in proportion to the customer's committed annual spend with PostHog.

To illustrate with examples:

- A customer committing to spend just $20k USD annually should not expect significant deviations from PostHog's standard terms. Minor, clarifying edits will be acceptable. We will not spend our time going back and forth for this amount. We may respond to significant changes with a polite, "no," rather than negotiating, to communicate clearly.
- A customer committing to spend $80k USD annually would be able to request slightly more sigificant deviations from PostHog's standard terms, and we will evaluate the suggested terms through the lens of legal risk assumed against annual revenue received. This will be a negotiation, and we will represent PostHog's position clearly as we go along.

- A customer committing to spend $160k USD annually (or more) would be able request even more sigificant deviations from PostHog's standard terms, and we will evaluate the suggested terms through the lens of legal risk assumed against annual revenue received. This will be a negotiation, and we will represent PostHog's position clearly as we go along.

At any potential level of annual spend, PostHog will not proceed under unreasonable legal terms. Certain suggested terms may be non-starters for PostHog.

Varying terms for trials and proofs of concept (POCs) for prospective customers

We don't vary PostHog's standard terms for trials and proofs of concepts (POCs) for prospective customers.

All prospective customers are welcome to try PostHog for free and under our standard terms (including our standard DPA and BAA, if applicable).

We don't negotiate terms for trials and POCs for three reasons:

1. Unlike many of our competitors, an annual subscription is not required to access PostHog, so a negotiated agreement is not necessary to use our services. Our product-led motion is designed to support customers trying PostHog.
2. When evaluating custom legal terms, PostHog evaluates legal risk assumed against annual revenue received.
3. Because prospective customers are paying us $0 for a free, sales-led trial or POC, there is no frame of reference for us to evaluate any potential custom terms. Spending our time and legal resources negotiating these terms is premature when a prospective customer doesn't know that they want to proceed with PostHog at all, much more at a qualifying level of annual usage.

Once the trial concludes, and per our guidance on varying terms, we will be happy to evaluate custom legal terms for an otherwise qualified PostHog customer.

Using non-PostHog contracts

If a customer requests to use a non-PostHog drafted contract for documents such as a DPA, MSA, Order Form, or BAA, we generally decline, except in special circumstances (see 'When we vary terms for customers'). We avoid doing this as it adds too much risk for us, and also because reviewing and negotiating non-standard terms introduces significant operational inefficiencies and doesn't scale well as we continue to grow. We typically do not even consider using customer paper unless the deal is over $200k annually or involves an extremely blue-chip company. It is best to manage this expectation early and just avoid entertaining the idea with customers as soon as possible.

We are somewhat more flexible when it comes to NDAs. That said, since we contract through our U.S. entity, we require customer NDAs to be governed by U.S. law. This is necessary to maintain consistency and ensure we're not taking on legal or operational risk in jurisdictions where we don't operate or fully

understand the legal landscape. This is mainly about ensuring we can review and manage agreements efficiently with our limited legal resources.

Managing contracts

contents/handbook/growth/sales/contracts.md

Creating and managing contracts

For customers who want to sign up for an annual (or longer) plan there is some additional paperwork needed to capture their contractual commitment to a minimum term, and likely custom pricing as well. At a minimum, they should sign an Order Form which references our standard terms and privacy notice. In addition, they may want a custom Master Services Agreement (MSA) or Data Processing Agreement (DPA).

What about monthly customers?

Anyone on a monthly plan simply agrees to our terms and privacy policy when they sign up.

QuoteHog pricing calculator

While we offer transparent pricing available to all, you can use QuoteHog for customers who need a "formal quote," or who have very high volumes, or otherwise have bespoke needs.

Sign into QuoteHog via your PostHog Google account/SSO. Upon login, you will see a list of existing quotes, sorted by the created date. You can view a previously created quote or create a new quote using the "New Quote" button at the top right.

The quoting interface is intuitive and, of course, uses the same pricing we display publicly. Feel free to involve a customer in creating a quote if the opportunity presents itself and you think it would build trust.

> Be sure to always click the "Save" button after making changes to a quote. QuoteHog does not autosave.

Quotes can be shared externally or embedded in an external source. Clicking the Dot Menu from a Quote and click "Share". If someone asks for a PDF version of a quote, you can view the external version and print it to PDF.

QuoteHog also provides Stripe reported usage and spend for existing customers. To do this, you need to first connect QuoteHog to Salesforce from the profile page. As you build a quote, click "Add customer info" and search for your customer account. This also allows you to link the quote to an existing Salesforce opportunity.

When building a quote for an annual plan conversion or renewal, consider:

1. How is usage trending? Looking at the past 6 month's of usage (usage history tab in QuoteHog):

- If usage is trending up, calculate the growth rate and project expected volume for a year.

- If usage is stable, project based on the latest month's volume or the average or the maximum.

> Note: QuoteHog's input expects monthly volume, so after estimating annual volume, don't forget to convert it to monthly volume.

2. Is there opportunity to adopt additional products? How does that affect future usage?

You can create quotes with multiple options: e.g. one based on current usage, one with a higher tier to account for growth potential.

The legacy pricing calculator is available here.

Order form

An order form is a lightweight document that captures the customer details, credit amount, discount, term, and signatures from both PostHog and the customer. They are either governed by our standard terms or a custom MSA (see below).

You will likely need to use QuoteHog to get the correct credit amount to be included in the order form.

Creating an order form

We use PandaDoc to handle document generation, routing and signature. Ask or for access if you don't have it.

1. The order form template to use is titled `[Client.Company] PostHog Cloud Order Form – <MMM YYYY>`
2. When looking at the template, click the link to **Use this template** in the top bar.
3. In the Add recipients box which pops up:
    1. Replace `<MM YYYY>` with the month and year the contract starts (e.g. March 2023)
    2. Add the Client email, first and last name
    3. Add the PostHog Signer email - normally the team member who is responsible for the customer (AE or CSM).
    4. Click continue
4. In the pricing table, set the total amount of credit in the Amount box next to **PostHog Cloud Credit**
5. Remove the Enterprise Plan line item if not needed.
6. At the bottom of the pricing table, set the **Discount %** just above the Total
7. On the right of the screen there is a sidebar, select the **Variables** tab and populate them as follows:

- **Client Address Information** - Needs to be their legal correspondence address (check with your customer contact)
- **Client.Company** - The legal company name
- **Contract.Discount** - The discount % (appears in the Additional credit purchase section)
- **Startup credits** - If the customer qualifies for the 2 free months when rolling off the startup plan, add up their total and discount as normal, and then add a note about the free credits in this format: "An additional credit in the amount of XXXXX (offered to customers in exchange for rolling off the Startup plan) to be applied to Customer's account upon signature with the same expiration date." For example, if a customer is signing a standard $20k annual contract to get the 20% discount, the total will be $25k, 20% discount of $5k, total cost to the customer would be $20k. In the notes, you would write: "An additional credit in the amount of USD $4,166.67 (offered to customers in exchange for rolling off the Startup plan) to be applied to Customer's account upon signature with the same expiration date."
- **Contract.EffectiveDate**
    - Set the start date of the contract in the format DD MMM YYYY (e.g., 01 Feb 2023). For a new customer, this would be the date they choose to start their subscription. For an existing customer, we have two options:
        - **Immediate Activation:** If the customer wishes to start using the credits immediately, set the start date to the beginning of their current billing period. This backdating ensures that the credits are applied correctly to the current billing cycle.
        - **Next Billing Cycle:** If the customer prefers to begin their annual plan at the start of their next billing cycle, set the start date accordingly. This option aligns the contract start date with the upcoming billing period.
        - For example, let's say it's October 15 and you're setting up an annual plan. You have a pay-as-you-go subscription that started on September 1, and the next billing date is November 1.
            - If a customer wants to start using credits immediately for the October cycle, your contract start date should be October 1.
            - If a customer wants to start using credits starting the next billing cycle, your contract start date should be November 1.
            - If you set the start date correctly, our Zapier automation flow will create the invoices with correct dates so our revenue calculations are not affected from the transition.
    - **Note:** Pay-as-you-go products are charged after the end of the period, while flat-rate subscriptions are charged at the beginning of the period. As a result the first two payments on a monthly schedule may occur within the same billing period as part of

the transition. Make sure to send a note to the customer to ensure they're fully informed!

- **Startup credits** - If the customer qualifies for the 2 free months set the start date of the contract for 2 months in the future, to account for the two free months ahead of the contract.

- **Contract.Term** - The term in months of the contract (12 months by default)

8. If they are paying monthly change:
   - Payment Terms to `12 equal monthly payments from Contract start date`.
   - Payment Method to `Credit Card`.
9. If an MSA is being used rather than the standard terms you will need to replace the following text:

   > PostHog Cloud License Terms appearing at: https://www.posthog.com/terms and Privacy Policy appearing at: https://posthog.com/privacy (collectively the "**Agreement**")

   with either

   > PostHog Cloud License Terms entered into by and between the Parties on or about the date hereof and Privacy Policy appearing at: https://posthog.com/privacy (collectively the "**Agreement**").

   or, if the Customer insists on including the exact date of the MSA to remove ambiguity,

   > PostHog Cloud License Terms entered into by and between the Parties on or about [INSERT DATE OF EXECUTION OF MSA] and Privacy Policy appearing at: https://posthog.com/privacy (collectively the "**Agreement**").

10. You should link the order form to the opportunity record in Salesforce using the `Contract Link` field in the "Opportunity Closure Details" so that we have a reference to the completed paperwork from our CRM.

Routing an order form for review and signature

1. When viewing the order form, check the recipients tab in the sidebar. The Client and PostHog roles should be filled in.
2. A signing order should also be set, with the Client signing first (so they can review it before we sign).
3. Ensure Document forwarding and Signature forwarding are set to on so that our Contact can re-assign the document if needed.
4. Click Send at the top of the document and add a message explaining the context of the order form.
5. Once the Client and then PostHog have signed it you should get an email to confirm completion.
6. Don't forget to link to an opportunity in Salesforce and mark the associated opportunity as Closed Won.

7. Zapier will automatically add a record in the Annual Plan Table with the PandaDoc Order Form ID.
8. Celebrate!

Manual upload of signed order form

We prefer to keep all signatures in PandaDoc, but sometimes clients may prefer to sign a PDF copy. One way to minimize this is to send contracts for initial review via PandaDoc when possible. It is ok to have multiple drafts in PandaDoc as long as we have the final signed copy in there as well. When a client signs an order form outside of PandaDoc, please follow these steps to complete the process:

1. If you have previously created a draft, find the document from the Documents page in PandaDoc (note: you cannot change the status from Home or inside a document).
    - Select "Change Status" from the three-dot menu on the right.
    - Upload the signed PDF of the document.
    - Mark the status as completed.
    - Check Audit Trail to make sure the signed version is uploaded correctly.
    - Link to an opportunity in Salesforce and close the associated opportunity as Closed Won.
2. If no draft exists, upload the signed document directly ad a new document in PandaDoc.
    - Mark the status as completed.
    - Link to an opportunity in Salesforce and close the associated opportunity as Closed Won.

Once you the signed form in PandaDoc is marked as complete and the Salesforce opportunity status is set to Closed Won, the RevOps team will get a notification and handle setting up the subscription and invoicing. See the Billing page for steps on how the billing setup works for more information.

Master Services Agreement (MSA)

Occasionally, customers will want to sign an MSA instead of referencing our terms in an order form.

1. Download a copy of the PostHog Cloud MSA as a Word Document (legal teams prefer this format) and share it with your Customer contact.
2. They may want to propose changes (also known as 'redlines'). Work with Hector or Fraser to get these agreed.
3. Create a new document in PandaDoc, you can choose to either import from Google Drive or upload from your local machine. This should be the clean, non-redlined document as agreed by both parties.
4. Change the name to be `PostHog Cloud MSA – CUSTOMER LEGAL NAME`.
5. Add the Client and your name and PostHog email as roles.
6. Add a Signature, Name and Title field for both PostHog and the Customer.
7. Check the signing order (Client, then PostHog normally).
8. Send for signature - so long as any proposed changes have been reviewed and approved by Hector or Fraser, you are free to sign on behalf of PostHog

Sometimes large customers will ask for changes to our MSA. We have a list of the kinds of changes we will/won't consider in a private repo in the company-internal sales contract changes directory that you can generally agree to without the Ops team reviewing. However, if you are ever in doubt, ask in `#legal` in Slack

Business Associate Agreement (BAA)

We offer HIPAA Compliance on PostHog Cloud and as such health companies will require us to sign a Business Associate Agreement with them. As this means we take on increased financial risk in case of a breach we ask them as a minimum to subscribe to one of the platform packages which is a guaranteed monthly payment. A maximum of one BAA per organization will be signed. Under most circumstances, it should be the company that owns the org/pays us.

1. Ask the customer to subscribe to the platform add-on (as well as any other paid plans they wish to use). You can verify this in Vitally by ensuring that they are in the `Teams Plan` segment.
2. Create a new document from the PandaDoc template.
3. All you need to do it set the `Client.Company` variable and then send it to them for review and signature.
4. It has been pre-signed by Fraser and will automatically add today's date as the date of signature for PostHog.
5. You'll get a notification when everybody has signed it - we have automation in place to ensure that the `HIPAA BAA Signed Date` property on the customer's Salesforce Account record is updated.

> We only provide our default BAA for platform add-on subscribers - customization requires >$20k annual spend. The BAA only remains active for as long as the customer is subscribed to a platform add-on - if they unsubscribe, we send them a message that their BAA will become inactive at the end of the month in which they cancelled. A customer who is on a platform add-on trial (with a credit card in PostHog) is eligible to sign a default BAA, but you should make it clear to them that the default BAA will be voided if/when the platform add-on subscription lapses. If the lead is not sure whether they will need a custom BAA and their usage wouldn't put them at $20K, then it is worth pushing them to get legal feedback by sending them our BAA before moving forward, else you risk spending a lot of time on an evaluation that ends up at $450/month.

Non-disclosure Agreement (NDA)

In some cases, prospective or current customers require a mutual Non-disclosure Agreement (MNDA) in place before conversastion or product activity can proceed. Terms already specify Confidentiality and if there is still a situation where a documented agreement is requested this can be easily accommodated.

- Access PandaDoc and Create a New Document
- Use the current PostHog - NDA template
- Add your desired contact as a recipient and follow the usual PandaDoc process

- When document is complete, it will be stored in the Document library and can also be attached to the Salesforce account for future reference

Trust center approvals

Requests that originate from the Trust Center automatically get sent an NDA in the request from SafeBase to PandaDoc. Once the document is fully signed, access will automatically be granted.

Managing our CRM

contents/handbook/growth/sales/crm.md

Overview

We use Salesforce as our customer relationship management ('CRM') platform. If you need access, you can ask for an invite.

As a first step, make sure you connect your Gmail account under your Salesforce settings. Go to Settings → Connected Accounts → Gmail and connect it. This ensures all your customer emails sync automatically with Salesforce. Next, make sure your Gmail account is connected in Vitally. This is essential so that we capture the full customer context and avoid duplicate or conflicting outreach.

As a general principle, we try to make sure as much customer communication as possible is captured in Salesforce rather than in individual email inboxes so that we make sure our users are getting a great experience (and not confusing or duplicate messages from different team members!). You should use the channel that suits the user, not us. Just make sure you keep Salesforce up to date with your interactions. We've found Slack messages usually get better response rates than email.

For existing customers, you'll sometimes send emails directly from Vitally. To ensure these also make it to Salesforce, first look up your *Email to Salesforce Address* from the personal settings page in Salesforce, and then add it to your Vitally gmail settings.

All Slack messages sync up with the corresponding account in Salesforce. We use Pylon for this sync, so make sure Pylon is added to the customer Slack channel integrations and the channel is linked to the Salesforce account properly for the sync to work smoothly.

You are most likely to use the following regularly:

- *Tasks* A task represents a potential sales follow up or engagement. Every new inbound inquiry (via form or email) is now created as a task on an account and contact.
- *Opportunities* - An opportunity is a qualified lead that has been assessed and is considered a potential sales deal (with an estimated revenue and an expected close date). This is where we manage our customers through their buying cycle.
- *Contacts* - Contacts are individuals who use PostHog or contacts we interact with. You can create contacts manually or convert a Lead to a Contact after evaluating the lead and deciding to continue working with them.
- *Accounts* - You will also create an account record to associate with any contact. You can associate multiple contacts with a single account. If you enter the company's domain name, we have data enrichment in place to pull in additional data on the organization.

Salesforce offers a ton of resources if you want to dig deeper.

Managing our CRM

People currently come into Salesforce through one of the following ways:

- Email inquiries: messages sent to sales@posthog.com
- Website forms: when they complete a contact or demo request form on our website
- Product sign ups: All signups are saved as a contact record in Salesforce
- Manual Entry: When a team member manually adds a contact, such as meeting someone interested in PostHog at an event

New PostHog signups

When a `user signed up` (Cloud signup) event is ingested into PostHog we use the Salesforce App to sync contact data into Salesforce. We also populate the following Salesforce properties if they are set in the PostHog event:

- selected_deployment_type - usually `cloud` or `hosted_clickhouse`
- product_signup_ts - the time they signed up/purchased a license
- is_organization_first_user - whether they have created a new organization or joined an existing one
- role_at_organization - the role they self-selected when signing up (used in Lead scoring)

Completed contact form

We have a contact us form on posthog.com where we ask users can get in touch with us. The sales@ alias gets an email notification and a notification is also sent to #sales-leads in Slack when one of these forms is submitted.

These submissions are processed through the Default app and routed into Salesforce as tasks. Tasks are then automatically assigned to the right team member based on account ownership and territory (see below).

If the submission is clearly a support or billing request, you don't need to reach out manually:

- On the task, select the disqualification reason **Billing Support Request** or **Support Request**.
- This automatically creates a Zendesk ticket for the correct team.
- No manual outreach is needed; automation handles it.

Zendesk Integration

If you add "sf-lead" tag to a ticket in Zendesk, a new lead will be automatically created in Salesforce. This helps streamline the process of converting support questions or tickets into potential sales opportunities directly from Zendesk.

If you see "Zendesk" as the lead source, please review the ticket under the Zendesk widget in Salesforce which allows you to view the full context within salesforce. It will also appear in sales_form_message field for quick review of last request before the Zendesk ticket is converted to a lead.

Forwarding sales opportunities

If you are not in the sales team but are engaged with a client and identify a sales opportunity, forward the email chain to sales@posthog.com. A new lead will be automatically created in salesforce and assigned to the appropriate AE based on existing criteria. This way we can smoothly hand off potential opportunities and track things properly!

Important: The email must be forwarded (not replied to), and sales@posthog.com must be in the "To:" field—not CC or BCC—for the automation to work correctly.

Task assignment logic

When a new task is created, we first check whether the associated account already has an owner:

- If the account has an owner, task is automatically assigned to that person.
- If the account is unowned, account and task are assigned to a salesperson via round robin within their territory.

This ensures we avoid double assignments and maintain clear ownership.

Territories

- U.S. West
- U.S. East
- Europe & Africa
- Asia & Middle East
- Australia & New Zealand (ANZ)
- Territory 1 (U.S. state unknown)
- Territory 3 (Unknown, but otherwise qualified)

Each territory runs its own round robin assignment for new, unowned accounts.

Stale task reassignment

If a task is assigned to a someone but remains untouched for 10 days, it will be automatically reassigned once via round robin. If it remains untouched after reassignment, it will be automatically disqualified with the **Stale – Autoclosed** reason.

Converting tasks to opportunities

If a task represents a qualified opportunity:

- Open the task and check the box labeled “Create new opportunity.”
- Choose the appropriate Opportunity record type:
  - New Revenue for brand-new customers.
  - New Revenue – Existing Customer for upsells, cross-sells, or expansions.
  - Existing – Convert to Annual for pay as you go customers moving to an annual plan.
  - Renewal for existing annual customers renewing their plan.
- This automatically creates and links the opportunity to the task.
- You can then click the opportunity link to add deal details (value, close date, etc.)

Use the following criteria (loosely based on traditional BANT qualification) to determine when a task should be converted to an opportunity:

- You've had at least one call with the customer to establish a relationship.
- There's a clearly identified problem that PostHog can solve.
- They have acknowledged that the problem is important enough to work on solving now.
- You've had a rough pricing discussion, and confirmed that their budget is in the same ballpark.
- The person you're in touch with is the decision maker, or has committed to introducing you to the decision maker.
- You have an agreed next step which moves things forwards such as:
  - Signing up for a PostHog organization
  - Implementing PostHog
  - Getting an MNDA in place
  - Accepting a Slack Connect invite and asking questions in a Slack channel
  - Accepting a call invite with their boss/buying committee

All of the above criteria should be met before creating an opportunity. By doing so you drastically increase the odds of bringing them onboard as a successful customer.

If you aren't able to confidently say that you have covered the above, you should keep them as a Lead in the Nurturing stage.

Task disqualification reasons (reference)

When you disqualify a task, choose the picklist reason that best matches the situation. Salesforce groups reasons into categories; full definitions are on the field in Salesforce. This table is a quick map so the team uses the same buckets.

Category	What it means	Reasons (picklist)
Auto-Dispositioned	System closed the task or sent auto emails without hands-on sales triage.	Below Threshold – Auto; Stale – Autoclosed
Re-route	Send the conversation to another PostHog team.	Support Request; Billing Support Request; Existing Customer Inquiry; Event Request; Partnership Request

Category	What it means	Reasons (picklist)
Not a Lead	Not a commercial sales opportunity for this path.	Spam; Duplicate Lead; Non-Commercial; Startup Plan / YC; Self-Hosted Requirement; Business Closed; Feedback; BAA / DPA Request
Unreachable	You cannot reach them or they stopped responding; split by whether they are worth revisiting.	No Response – Pass; No Response – Prospect; Invalid Contact Info
Fit	You could assess fit; outcome is ICP, product, technical sponsor, or competitive situation.	Below Sales Assist Threshold – Pass; Below Sales Assist Threshold – Prospect; Not a Good Fit; No Technical Resource; No Product Fit; Using Competitor / Unsolicited RFP
Timing / Economics	Fit may be fine later; budget, timing, or capacity block a deal now.	No Budget; No Current Need; Resource Constraints
Other	None of the above; use sparingly.	Other

Splits and follow-ups (pick **Pass** vs **Prospect** carefully so reporting and nurture stay accurate):

- **No Response – Pass** — No response and no meaningful signal (e.g. low lead score, no usage); terminal for active pipeline.
- **No Response – Prospect** — They showed qualifying signals but went dark; create a **follow-up task** with a date (revisit in roughly 3–6 months).
- **Below Sales Assist Threshold – Pass** — TAE judged under ~$20K potential with no signals worth revisiting.
- **Below Sales Assist Threshold – Prospect** — Same economic band but signals worth another pass (ICP, growth, usage); create a **follow-up task** (e.g. BDR or named list). If nothing happens within ~90 days, revisit whether this split is useful.
- **Using Competitor / Unsolicited RFP** — Locked in or chose a competitor; set a reminder to check in in about **9 months** (see new sales playbook).
- **Other** — Requires a free-text comment when selected; if a large share of disqualifications land here, propose a new reason.

Manual entry

If you meet a potential customer elsewhere (e.g., events, introductions, referrals):

- Create the Account and Contact manually.
- Assign the correct Lead Source from drop down.
- Create a Lead Task for any action item or sales follow up.

Support and billing routing

For support or billing questions submitted via the sales channel, disqualify with **Support Request** or **Billing Support Request** as in Completed contact form (Zendesk ticket automation). If you still see legacy lead records from older flows, the same reasons apply; ticket creation may use this Zapier path for some automations.

Below Threshold – Auto (Customer.io)

When you should route someone to self-serve onboarding instead of hands-on sales, mark the task **Below Threshold – Auto**. That triggers the automated onboarding flow in customer.io, which guides them without manual outreach. Manual TAE judgment under the sales-assist threshold uses **Below Sales Assist Threshold – Pass** or **Below Sales Assist Threshold – Prospect** (see splits above), not this auto reason.

Spam

These mostly come into the sales inbox rather than the contact form. Whilst there is a `Spam` disqualification reason in Salesforce we can also prevent users from emailing the group again by banning them in the Sales Google Group. If you do ban someone bear in mind they won't be able to email our sales email until the ban is lifted so only use this for genuine spam (e.g. people trying to sell us competitor user lists).

Lead qualification criteria

- Do they match our ideal customer profile?
- Do they have a need that PostHog can help with?
- Have they shown interest in our product/service?
- Are they looking to make a purchasing decision within a reasonable timeframe?

Best practices

- Make sure all new leads are contacted within 24 hours.
- Keep all lead information up-to-date and accurate in Salesforce.
- Periodically review lead statuses and update them as needed.

Handling time off (PTO)

By default, when you are out leads will still be routed to you, and as we have no expectation of you being available whilst on PTO leads may be missed and not followed up on. To mitigate this we need to temporarily remove you from lead round robin:

- If you are out for 1 consecutive day or less:
    - Ensure your calendar is up to date with your time off, so that Default doesn't schedule meetings for when you are out.

- If you are out for longer than 1 consecutive day:
    - Ensure your calendar is up to date with your time off, so that Default doesn't schedule meetings for when you are out.
    - Let Mine or Simon know 2 working days before you leave that you are out and need to be taken out of the round robin temporarily
        - Mine or Simon will then set you to inactive on the Leads assignment tracker in Salesforce.
        - They will also set a reminder to re-add you the day before you are back.

Opportunities

Opportunities track potential deals in Salesforce. Managing opportunities effectively is important for tracking progress, forecasting revenue, and ensuring accurate reporting. In our sales process, for each lead conversion we create an Opportunity. Correctly identifying the appropriate opportunity record type is important to optimize our processes.

Opportunity record types

New Revenue: Select this type when engaging with a customer who has never paid us before. This includes new customers and startup customers transitioning to a paid plan for the first time.

New Revenue – Existing Customer: Choose this type for additional credits to a customer who is already paying us. This includes upsells, cross sells, or expansion within the same account.

Existing - Convert to Annual: Choose this when discussing an annual contract with a pay-as-you-go customer.

Renewal: Choose this type when an existing customer is renewing their contract or subscription for our products or services. We automatically create a renewal opportunity if an 'Annual Plan' type opportunity is Closed (more on these later).

Opportunity types

Annual Plan: Select this type when the customer agrees to pay for a year-long+ subscription to our services.

Monthly Plan: Choose this type when the customer opts for a month-to-month subscription to our services. Amount field still reflects ARR here.

How to create an opportunity

Convert a task

If you're working a lead and want to create an opportunity from a task, simply check the Create New Opp checkbox and select the appropriate Opportunity Record Type from the dropdown.

This ensures the Lead Source is correctly carried over to the new Opportunity, and the task and opportunity remain linked for full visibility.

Creating an opportunity from scratch

You can also create an opportunity directly from scratch, but make sure to connect it to a Contact and an Account so all relevant data is linked properly. To do so:

- Go to the Opportunities tab by clicking on the App Launcher (the grid icon) and searching for "Opportunities."
- Click the "New" button and select the correct record type
- Fill in Opportunity Details:
- Opportunity Name
- Close Date: Choose the estimated date when the opportunity is expected to close.
- Term (Months): Default is 12, update for multi year deals.
- Total Credit Amount: Total value of the contract before discounts.
- Discount (%): Percent discount applied to the total.
- ARR Discounted: Automatically calculated annualized revenue after discount.
- Contract Start Date: Date the contract begins.
- Contract End Date: Automatically calculated based on Start Date + Term.
- Stage: Select the current stage of the opportunity in the sales process.
- Type: If you know whether they're interested in paying on a monthly or an annual basis (if blank this will be Monthly by default)
- Connect to an Account: In the "Account Name" field, search for and select the account associated with the opportunity. If the account does not exist, create a new account first.
- Connect to a Contact: link any specific contact you're in touch with regarding this opportunity by adding them to the "Contact Roles" list.

Opportunity stages

Stages will differ depending on the chosen Opportunity Record Type. The following stages are for the New and Existing Business Record Types:

1. Problem Agreement - Buyer explicitly acknowledges they have a meaningful problem that can be qualified (e.g. "What happens if you don't solve this problem?") Exit criteria:

- **Identified & implicated pain with specific, quantifiable metrics (time/money/risk)**
- Answer to "What happens if you do nothing?" documented with real consequence
- Buyer explicitly said "This is a problem we need to solve" (not just "interesting")

2. Solution Agreement - Buyer confirms our solution is best suited to solve their problem. Can be as simple as "We think PostHog will work for us" Exit criteria:

- **Active product usage OR completed POC/trial**

- **Clear, documentable decision made for PostHog (with or without comparing alternatives)**
- Economic Buyer identified (name + title)
- Champion identified (name)

3. Priority Agreement - A senior decision-maker acknowledges the problem as a priority and validates our solution. Exit criteria:

- **Budget confirmed (amount range OR "yes, funded")**
- **Decision process mapped (who approves, what steps, timeline)**
- Economic Buyer said this is a priority (exact quote documented)
- Champion tested (evidence they're advocating internally)
- Compelling event known (deadline: budget cycle, launch, renewal, etc.)

4. Commercial Agreement - Mutual agreement is reached on price and all contractual terms. Exit criteria:

- **Price agreed in writing (email/quote with amount + terms)**
- **All commercial terms agreed (payment terms, contract length, prepaid amount)**
- **Paper process mapped (legal, security, procurement steps + owners + timeline)**

5. Vendor Approval - Buyer completes internal processes (legal, security, procurement) and contract is executed. Exit criteria:

- **Contract signed**

6. Closed Won (100%) - They have signed the contract and are officially a PostHog customer.
7. Closed Lost (0%) - At some point in the pipeline they decided not to use us. The Loss Reason field is required for any opportunity to be marked as Closed lost.

Bolded exit criteria indicate the minimum standard for the opportunity to advance stages (for typically smaller, more transational deals). More detail is available on the stages and the exit criteria for each state in this spreadsheet

Forecast categories

Commit: PostHog is integrated and the buyer has stated an intent to purchase within the Close Date quarter.

Best case: PostHog is or is being implemented, volume justifies an annual commitment, and the buyer has stated interest in purchasing with the Close Date quarter.

Pipeline: Buyer is actively evaluating PostHog or intends to evaluate PostHog within the Close Date quarter and early volume/discussion indicates an annual contract could be justified.

Omitted: Not used. You can omit from Forecast by moving the Opportunity to a new quarter or marking it as Closed - Lost.

Forecast categories should be re-evaluated on an ongoing basis. While it is not ideal for Opportunities to move to an earlier category, we should do so if this reflects reality, especially as quarter end approaches. In addition, we should think about what we can do differently in future to make the forecast more accurate.

Renewal pipeline

When an opportunity with Annual Plan type is Closed Won, a Salesforce flow will create an opportunity associated with the contact and account from the original opportunity. The following fields will also be set:

- **Amount** - Copied over from the original opportunity
- **ARR up for renewal** - Copied over from the original amount; so that we can track expansion/churn
- **Close date** - 4 weeks in the future (may need adjusting if the opportunity record isn't closed on the contract start date)

The renewal pipeline stages are:

1. Qualification (10%) - They have just became a PostHog customer and we're helping them getting set up.
2. Meeting booked (20%) - They have reached a steady state where we consider them self-sufficient with PostHog.
3. Product Evaluation (50%) - This step becomes relevant if decision makers have changed in organization or if new teams within the company are considering using us.
4. Commercial & Legal Review (80%) - We are now working with them on contractual items such as custom pricing, MSAs etc.
5. Closed Won (100%) - They have signed the contract.
6. Closed Lost (0%) - At some point in the pipeline they decided not to renew. We should make a note as to the reasons why and optionally set a reminder task to follow up with them if we have improvements that could change their mind on our roadmap.

Opportunity notes

The "Opportunity Notes" section is to track key actions and next steps to manage an opportunity and avoid missed follow-ups. It has the following fields:

- Next Steps: Add actions or tasks required to move the opportunity forward. Be clear and concise to ensure anyone reviewing the opportunity understands what needs to happen next.
- For the New Business Sales Team, the Next Step should have three specific elements: 1) a timestamp -- when was this change made, 2) the owner at the customer for the next step -- who do we expect to take the action? 3) a binary outcome - (what will we/you get) related to the stage, with the next step date reflecting when the outcome is expected.
- Next Step Date: Enter the date by which the next step should be completed. This helps in maintaining timelines and keeping follow-ups on track.

Opportunity closure details

This section is to add additional information for opportunities that are won or lost to capture context and details to setup customer account correctly:

- Loss Reason: A required field for any opportunity marked as "Closed - Lost." Pick the most appropriate option from the dropdown to help identify patterns.
- Additional Loss Context: Optional field to add further insights into the loss. It's great to include specific customer feedback if available.
- Contract Start Date: Especially important for correct account setup and tracking renewals. This date must match the start date of the customer's current billing period for which they intend to apply credits. Setting this correctly ensures that any purchased or applied credits can be used immediately for the intended billing cycle.
    - Example: if a customer's billing period runs from September 21 to October 21, and they purchase credits on October 15, the contract start date must be September 21 for credits to be applied to their current billing period. If instead the start date is set to a later date, credits would only apply to the next billing period, meaning the customer won't be able to use them right away. see more info on under contracts
- Products: Select the products discussed/planned to be used as part of the opportunity. Make sure to include all addons so RevOps can ensure the customer's subscription is set up correctly.
- Contract Link: Link to the contract in PandaDoc for easy access and reference.

Self-serve opportunities

If you feel like a customer doesn't fit a hands-on flow, then you mark the lead or opportunity as self serve. There are two ways to do this:

1. Self serve - no interaction

Use this checkbox when you decide to move a new lead to the automated self serve flow without any personal interaction or discussion. You can use this checkbox when a lead does not meet the necessary qualifications for direct engagement and the automated self serve emails would be sufficient for successful onboarding.

How to use:

- Go to the lead record in Salesforce.
- Click the checkbox labeled "Self Serve - No Interaction" under the Lead Details section.
- Once marked, the automated self serve email flow will be triggered, no need to do anything else.

2. Self serve - post-engagement

Use this checkbox if you have engaged with the lead in some form, such as a demo or discussion, but you believe they can proceed without further involvement.

How to use:

- Go to the opportunity record in Salesforce.
- Click the checkbox labeled "Self Serve - Post Engagement" under the Opportunity Information section.

Important notes:

- There are no automated email flows attached to this checkbox. Once you have spoken with a customer at least once, all future communications should come directly from you.
- Separately, these customers will still receive the standard onboarding emails from the app regardless of their self serve status in salesforce.

Points to consider when marking leads as self serve:

- Usage Volume: If their usage volume is around 5 million monthly events and 100,000 recordings, they should be hands-on.
- Annual Commitment: If they want an annual commitment, keep them in the hands-on pipeline.
- Guided Evaluation Help: If they need help with a guided evaluation and their potential value is high enough, create a Slack Connect channel to assist them during the evaluation and keep them in the hands-on pipeline.
- None of the Above: If none of the above apply, move them to self-serve.

When moving someone to self-serve we should set them up for success by using the Post Demo route to self-serve. This encourages them to sign up to PostHog Cloud and provides some helpful getting started pointers. If there were any follow-up questions from initial meeting we should answer those in this email as well.

> If you move an opportunity to self-serve then it won't be included in your quota retirement/commission calculation (as you aren't working on it).

All done - now what?

This is just the beginning of what will hopefully be an awesome relationship with a new customer!

We are just getting started here, but a few things that you should do:

- If they are a large/target customer, they should already have a Slack Connect channel in our company workspace.
- Check in with them regularly and ensure they aren't blocked by support/other issues

How to respond to frequently asked questions

contents/handbook/growth/sales/customer-faqs.md

Here's how to respond to common customer requests. These usually arrive in the form of new contact form submissions but may also be asked by existing customers too.

Can you increase my rate limits?

Here's how we'd break down use cases:

- if the use case is exporting all the data so they can do further transformation or activation in other tools -> use batch exports
- if the use case is ultimately going to be accessing our API programmatically with a pre-defined query -> use endpoints product
- if the use case is essentially wrapping PostHog and allowing the customer to query whatever they want (in other words, if they want a different UI for querying PostHog data) -> use `/query` API endpoint

```
flowchart TD
    A{Is the API hitting <code style='padding: 4px; border-radius:
8px;'>/query</code> rate limits?}
    B{Does the use case fit endpoints?<br />(i.e. B2B2C user-facing analytics,
data-powered APIs, internal home-grown dashboards)}
    C["Explain the use case in #team-data-modeling <br />(we're keen to talk to
beta users!)"]
    D{Should they use batch exports instead?}
    E[Redirect them to start paying for batch exports.]
    F[1. Assume we're not increasing rate limits.<br />2. Reach out to #team-
clickhouse with query details.]
    G["Go to the relevant team for that API<br />(Feature flags, Surveys, ..)"]

    A -->|Yes| B
    A -->|No| G
    B -->|Yes| C
    B -->|No| D
    D -->|Yes| E
    D -->|No| F
```

See RFC #438 for more context.

Do you have plans to add more hosting options outside of the US and EU?

Right now, no. The vast majority of our customers are happy to host on one or the other, with EU being the preferred domain for GDPR compliance. This is not a "never", just not in the near future.

Do you have a dummy account we can mess around with?

No, the best way to trial PostHog is to start sending your own data into it. When a trial is filled with dummy data, which isn't relevant to the specific team, the overall engagement and success of the trial is lower.

Does PostHog follow the MEDDPICC sales methodology?

Yes! But like everything we do here, it's not what you would expect. At PostHog, MEDDPICC means "Make every deal a delightful PostHog implementation - Charles Cook"

New customer onboarding

contents/handbook/growth/sales/customer-onboarding.md

Sales-led

Day -1 - Session: Initial demo

Our moat is that we have a fully-integrated tool that allows customers to go across Analytics, Recordings and Experimentation easily. We want new customers to see the value of this as quickly as possible when evaluating us against other solutions.

For high-touch prospective customers the following process will get them onboarded quickly so that they can experience the value we provide using their own product data.

The process should run for 2 weeks by default, but can be extended if we think it's worth the additional effort.

The aim at the end of the evaluation is to have them:

1. Sending in auto or custom captured event data
2. Enabled session recordings
3. Created a trend chart tracking User Acquisition
4. Created a funnel tracking Activation
5. Added the above to a dashboard

At the end of the demo call

If at the end of a demo call we think a customer qualifies for high-touch onboarding we should outline our suggested evaluation approach. If they aren't quite ready to kick the evaluation off then we should follow up with a templated email reminding them of the process, then check in with them after they've had some time to regroup.

High touch criteria

As a small team we have limited bandwidth to run customer evaluations so we need to focus on potential customers who:

1. Are likely to contract above $20k with us. (Ideally we qualify this by giving indicative pricing in the demo)
2. Are likely to enter into an annual contract. (This is quite a high-effort process for people just going month to month)
3. Are ready to get hands-on with PostHog and will make a decision in weeks, not months.

Expectations of the customer

We'll need them to be able to demo their product to us, as well as attend two or more zoom calls where we scope out the data and help them get set up.

Ideally we will also have them in Slack Connect channel so that we can provide responsive support and expose them to the wider PostHog team.

Some customers may wish to use MS Teams rather than Slack - we can sync our Slack with Teams via Pylon to do this. First you will need an MS Teams licence - ask Simon for one. Then, follow the instructions on the link here to get set up. Before adding the customer into the channel, remember to test it on both sides to ensure the integration is working correctly.

Day 0 - Session: Kick off

At the start of the evaluation, we want to review their product to understand and advise on the best approach to tracking, as well as address any privacy concerns associated with session recordings.

By the end of the call we should have a plan for event capture/opt-out capture and an agreed timeline to get that set up.

Prerequisites

The customer should come prepared to demo their product to us, where we can help figure out the key tracking events needed for the evaluation to be successful.

If they don't already know about AARRR we should share our AARRR blog post and Tracking Plan and ask them to review it before the call.

Structure

1. Review goals and structure of this session
2. Review key concepts:
    - Acquisition
    - Activation
    - User Identification
    - Cohorts
    - Groups
    - Privacy / opt out capture
3. Have customer demo their app to you, focusing on where the above information is captured
    - During the demo agree where Acquisition/Activation/Identification take place
    - Get the CSS selectors and pages of any items to opt-out of capture
    - Agree any additional properties that need to be captured
4. Recap and agree the tracking and other implementation details

5. Agree the timeline to have tracking implemented and set up the following call (ideally 3 days after capture is implemented)

Deliverables

1. A partially filled-in tracking plan detailing Activation and Acquisition
2. A code snippet showing them how to implement tracking for their product (including Identification and Groups if applicable)
3. Elements and pages to add opt-out-capture to

Day 3 - Session: Using PostHog

The aim of this call is to get the customer familiar with navigating PostHog as well as:

- Defining Actions
- Defining Cohorts
- Creating Insights
- Creating Dashboards
- Finding Recordings

As much as possible the customer should be sharing their screen and driving the session, by teaching them to fish they become comfortable and self-sufficient with PostHog.

Prerequisites

Tracking should be set up in line with what was shared after the previous call.

Structure

1. Review goals/agenda
2. Have them share screen and guide them through:
    1. Live events
    2. Creating actions
    3. (Optionally if using Autocapture) the toolbar
    4. Creating cohorts
    5. Creating their Acquisiton trend insight
    6. Creating their Activation funnel
    7. Adding insights to a dashboard
    8. Navigate from dashboard to insight to recordings
3. Note any inconsistencies or missing tracking information and plan to follow-up to help get that set up 4
4. Show them the billing page and their projected usage (pricing discussion)

Deliverables

1. Updated tracking guidance based on issues discovered in the guided demo
2. Updated pricing quote based on volume

Next Steps

Every trial should have an end date by which time we expect the customer to make a decision on whether PostHog is right for them. If they need more time we first need to understand what they've not seen so we can proactively help them see everything they need to do make a decision (within reason).

If they do become a customer (yay!) then we should agree a regular check in call cadence with them from the start (it's much harder to do after they are in the steady state).

Customer Success-led

1-hr onboarding call

Customers with a platform add-on (and up) are entitled to a one hour kickoff/implementation call. This could include a high-level discussion of how PostHog fits into their stack, troubleshooting issues they've hit so far, or walking them through as they code it up, for simple setups. In practice we only need to worry about this for product-led / customers who haven't talked to sales before. [TO DO] Include a team calendly link in the welcome emails for Teams purchases or set up a separate campaign to email from a CSM.

Ongoing Training

Enterprise customers will also receive 1-2 hours of bespoke training per quarter according to their needs. This can be delivered in a few formats depending on where the customer is in their PostHog journey:

1. A deep-dive on a specific topic of their choosing.
2. Question and Answer session with their CSM.
3. An intro/set-up session for a PostHog product they've not used yet.

Sales to CSM "handoff"

Customer lifecycle handoff/ownership is described in the sales and customer success overview.

In-person sessions with customers

contents/handbook/growth/sales/customer-onsites.md

In-person visits work best for accounts paying or likely to pay $50k+/year where you've identified specific opportunities for expansion, are navigating renewal conversations, or need to overcome significant technical or organizational hurdles. They're a high-effort, high-reward play - use them strategically.

This outline of what to consider and how to plan an in-person visit is designed to provide ideas, inspiration and avoid common pitfalls. It is not a framework nor a definitive guide - your intuition and experience should ultimately guide the who, what and how of in-person meetings.

Account manager driven vs customer driven visits

Account manager driven

Sometimes you'll be the driver for the visit happening, whether by pitching a specific outcome, or offering to 'drop in' when you're around. When this is the case, it's important that you have a tight, well defined schedule and some goals (whether shared with customer or not) for the visit. Don't leave your champion(s) to do the work of planning and scheduling the sessions - you want them to be able to join the session, have a positive and productive experience, and continue evangelising us without adding to their to-do list.

Customer driven

Sometimes the purpose of an onsite is defined clearly by the customer, for example overcoming a specific technical challenge, or building an analytical framework for a new product. In this situation, definitely schedule significant time to address their primary goal, but do not underestimate the opportunity to create less formal contexts with smaller audiences, especially your champions.

Formal Sessions

This section will cover the length, audience, purpose and content of sessions that you could include as part of your onsite - pick and choose like a menu. At a minimum you'll want a broader session with a big audience and a narrow session with the key stakeholder(s) and champions to progress a relationship with an account.

PostHog user analytics jam

Length: An hour, possibly longer with a small focused audience

Audience: Users of PostHog, the more the better, armed with laptops and logged in

Purpose: To level-up how users engage with our platform, spark new ideas and inspiration for customer teams, and expand usage and impact by enabling the audience on our tools.

Content: Set an explicit goal that you think is achievable within the session for you to work towards with the users. Invite ideas and suggestions to get the jam going, but make sure you have some solid ideas in your back pocket to get things started. Building a compound score (i.e Customer effort score, time to value, onboarding friction) can be a good one if the customer has no ideas. Do not start demoing, but do show users useful shortcuts (PostHogAI, Actions, Cohorts, Workflows, Realtime destinations etc) if relevant. Conclude by summarising progress, and suggesting some follow-up and continuation tasks to take it to the next level.

Check-in and planning with champions

Length: 30 minutes to an hour

Audience: The champions of PostHog at your account

Purpose: To level set about PostHog's reputation and role within the customer, and unearth any opportunities or risks, while building stronger relationships with champions. Also a chance to test the water for any cross-sell proposals, and ensure champions are aware of all of our products.

Content: Discovery and planning with the champions, potentially over a meal or in another less formal context - certainly not in front of their teams or boss, if relevant. Give the champions time to air any frustrations, and ask direct questions about renewal, their roadmap, any current or future needs in their team or beyond. This is a good time to find out where you really are with a customer and what organizational challenges you may need to navigate, as well as expand their perception of what we can do. Some good questions to ask:

-"Do you intend to renew with PostHog? If not, why not?" -"What tools are other teams using alongside PostHog data to get the full picture of user behavior?" -"What goals and focus areas are on the table for the next year? How does PostHog fit into those?"

New user demo on your own data

Length: 40 minutes with time for questions

Audience: Customer employees who do not yet use PostHog, or are very new.

Purpose: To increase the number of PostHog users at the customer, and expand laterally into teams that may otherwise not use us.

Content: A demo on the customer's data. Tailor this and create insights or show features that'll be a good jumping off point for the audience to go further on their own. Make sure to note who your audience is and check-in with any that don't start logging in within a week or two.

Executive summary

Length: 30 minutes

Audience: Senior folks - csuite or VPs

Purpose: To improve the perception of our value to senior people, while discovering our position and estimation at the decision-making layer of the customer.

Content: A punchy, direct delivery of information focused on value that connects PostHog to the key goals and objectives of our customer.

Example: Connect PostHog usage to a metric the exec cares about: "Your team shipped 12 features last quarter. Using PostHog's feature flags and analytics together, the product team can now measure impact within 24 hours instead of waiting for your monthly business review. This means faster iteration and less risk of shipping things that don't move the needle on [their key metric]."

Manufacturing Informal Access

Why this matters

The formal sessions are theater - everyone is in meeting mode, and the group is usually to large for real candor. Deeper information comes out over lunch, walking between buildings, or after a drink: you need to create contexts where people forget you're "the vendor" and just talk to you like a colleague.

Making the invitation

Bad: "I'd love to take you to dinner to discuss PostHog's roadmap" Good: "I'm grabbing dinner at [specific place] after our session - you're welcome to join if you're around" Notice: You're doing it anyway, specific location, low pressure, no stated agenda.

Common mistakes to avoid

- Don't treat this as a sales pitch - if you're showing up just to close a deal, you're not expanding your relationship and understanding of the customer.
- Don't over-schedule - leaving buffer time for organic conversations is often more valuable than cramming in another session
- Don't show up unprepared on their data - spend time before the visit building something useful in their PostHog instance that you can show them

Follow-up

Make sure you take advantage of good will and being front of mind with the customer after the visit to followup on any outstanding goals, move any paused commercial conversations forward, or ask for access to any teams or key people you weren't able to reach while in person.

Running product training sessions

contents/handbook/growth/sales/customer-training.md

> **Purpose** - Get new PostHog customers self-sufficient and primed to explore more of PostHog. **Format** - Two live group sessions for up to 30 people across product, marketing, and engineering. **Total TAM time** - ~2.5–3.5 hours live delivery, plus ~one hour prep per session.

This document tells you what to cover in a training session and why it matters. It's not a script or the only way to approach training. Consider this a good baseline but you will want to always run it in your own voice, in whatever order fits the customer's needs. Product training is a separate and optional activity you can run with an account if you believe it will increase usage and help them derive more value from PostHog.

PostHog AI and MCP should be woven into every demo and practical. Don't teach them as standalone features. Use them as the way you build things in front of the room. The goal is for everyone to leave thinking AI-assisted analytics is the normal way to work.

Pre-session work

Do this before Session 1. The prep is what separates a useful session from a product tour that is easily forgotten.

Understand what will make the session valuable

Gather answers to the following questions (you may know the answers but it's still worth asking the customer directly):

- "What are the three questions about your users you can't answer today?"
- "What tools are you replacing or supplementing with PostHog?" (GA4, Amplitude, LaunchDarkly, etc.)
- "Who on your team do you want using PostHog daily? What are their roles?"
- "How heavily have your teams adopted AI tools and workflows?" (This sets up the MCP conversation later.)

Decide what to show

- If the customer already has a PostHog instance, you may want to use it for the training session. It is easier for new users to understand how PostHog works if they're familiar with the data they're looking at. Make sure you have permission from an Owner or Admin before Session 1.

- Review their account in Metabase to get a better understanding of existing usage. Are events named consistently? Are they identifying users? Is autocapture on? Note the gaps – you'll address them in Session 1.
- If the customer's instance has not been well instrumented (poor event taxonomy, instrumentation gaps, or limited product adoption), it may be better to use a demo instance with data that closely resembles what the customer might have. If it's really bad, consider waiting to do training until you've helped them improve it.

Prep a dashboard

- Build a dashboard from their actual data (or something closely resembling it). Three to five insights that answer their key user questions above. You'll refine it live, but showing up with something already built saves 15 minutes of dead air and proves you did the homework.

Prep a session summary

- Generating a session summary can take a few minutes. Instead of doing it live during training, create one ahead of time that you can show.

Confirm attendees have access

- Confirm via admin panel that attendees can sign up by themselves (Authentication Domains have been configured) or that they've been invited.
- Consider sending attendees a welcome email if they're new to PostHog. Use this as an opportunity to tell them what PostHog is and how they can access it.

If possible, schedule both sessions in the same week. Monday/Thursday or Tuesday/Friday. A long gap between sessions kills momentum.

Session 1: PostHog fundamentals

Audience - Everyone. Product, engineering, marketing, data, leadership.

Duration - 90 minutes (65 min content, 15 min Q&A, 10 min buffer).

Deliverable - A working dashboard in their project. Every attendee knows how to build an insight and watch a replay.

Why it matters

This is the only session that's guaranteed. If they never show up for Session 2, this has to be enough to make PostHog stick. Every topic here maps to the most-visited pages in our docs.

Topics to cover

Intro to PostHog (5 min)

- We want to jump into the product as quickly as possible – keep this brief. It's not a sales pitch but rather an overview of the platform and why it can help product engineers build more successful products.
- Avoid talking through the individual products at this point. It's better to show them.
- Make sure everyone is able to sign into PostHog.

The data model (10 min)

Events

- Start on the Activity page and explain that Events are the backbone of how PostHog works. Differentiate between autocapture and custom events. Open an event to show that it has properties.
- Quick note on frontend vs. backend capture. Backend is more reliable. Frontend captures richer interaction data. Both have a place.
- Ask PostHog AI "What are the most common events in this project?". This will orient the room and let them see PostHog AI can access and understand their events.

Persons and properties

- Show the difference between identified and anonymous users. Explain how `identify()` stitches anonymous and known users together.
- Show them a person profile and what lives there. Point out what person properties are and give examples of useful ones in the customer's context.

Cohorts

- Build one live using PostHog AI: "users who signed up in the last seven days" or whatever matches their product. Give real examples of other useful cohorts they may want to explore (e.g. power users, early adopters, likely to churn, etc.)
- Explain that cohorts can be automatically updated and are reusable across different parts of PostHog. Create cohorts to learn from the behaviors of specific groups of users.

Pause for questions. Allow for some awkward silence.

Building insights (15 min)

The core of the session. Don't teach insight types in the abstract. Build them around a real question from the pre-session prep.

Trends

- "How many users signed up this week vs. last week?"
- Show total count, unique users, breakdown by property, and aggregation of property values.
- Flip through visualizations: line, bar, number. Same data tells different stories depending on the display.

Funnels

- "Where are users dropping off in our onboarding flow?"
- Build a three- to four-step funnel from their actual events.
- Show conversion rate, the drop-off step, how to click into the users who dropped off, and tie it back to cohorts.
- If the data supports it, show correlation analysis. ("Users with property X convert 2x better.")

For smaller audiences (~10 people), encourage attendees to build an insight themselves by prompting PostHog AI or clicking through the UI. Try: "Show me a funnel from page_view to sign_up to first_project_created in the last 30 days." For larger groups, this gets chaotic – demo it yourself and save the hands-on exercise for Session Replay.

Session Replay (15 min)

Connect Session Replay to the funnel you built. Show the numbers, then show the human behind the numbers.

- Mention the filters: by event, by person property, by error, feature flag, rage clicks, console logs. Plant the seed that PostHog products all work well together.
- Show how to create and save a playlist.

Ask the audience to build a Session Replay filter using PostHog AI or the UI.

- End this portion of the training by explaining AI session summaries. Show the real example from your prep work.

Dashboards (10 min)

- Take the insights you built and save them to the starter kit dashboard.
- Show sharing, date range controls, and pinning.
- Mention dashboard templates for teams that want something pre-built.

- Show subscriptions: schedule a weekly dashboard email to their team. (Single best way to keep PostHog in people's inboxes without any TAM effort.)

Quick overview of what else exists (5 min)

Don't demo any of these. Name them so the room knows what's available and that they're tied to events.

- Feature Flags and Experiments, including no-code web experiments (product + eng)
  - Change your app and see the impact on Product Analytics data
- Web Analytics dashboard (marketing)
  - Understand who is visiting your site, where they're coming from, whether they're converting, and if they become active users
- Surveys (product + marketing)
  - Collect qualitative feedback by triggering in-app surveys based on user actions
- LLM Analytics (teams building AI features)
  - Understand how people are interacting with your LLM-based features
- Error Tracking and Logs (engineering)
  - Capture errors as events so that you can see how exceptions are influencing user behavior
- Data Warehouse and SQL editor (data / power users)
  - Query other data, such as prod dbs or Stripe transactions, alongside your Product Analytics data
- MCP server (engineering teams using AI coding tools)
- CDP & Workflows

Q&A (15 min)

Open floor. If nobody asks any questions, mention some of the below examples as commonly asked questions. This may make people feel more comfortable.

- "How do I filter out internal users?" – Point to the tutorial. Top-10 docs page for a reason.
- "What's the difference between Web Analytics and Product Analytics?" – Web Analytics is the pre-built dashboard for high-level metrics. Product Analytics enables custom insights for deeper questions.
- "Can I share this with people who don't have PostHog access?" – Yes. Subscriptions, embeds, PNG exports.
- "Is it possible to group related events and analyze them as one?"
- "Do I need to involve engineering every time I want to track a new event?"

After Session 1

- Drop a screenshot and a link of the dashboard into the shared Slack channel within 24 hours. Tag the team lead.

- Send a follow-up with links to the three or four most relevant docs or tutorials based on what came up in Q&A.
- Send the MCP setup and share a Loom video of accessing PostHog data from Claude or ChatGPT.

Session 2: Pick your track

Duration - 60 minutes (40 min content, 15 min Q&A, five min buffer).

Offered as two tracks. The customer picks one, or runs both if they have the headcount. Schedule it two to four days after Session 1.

Session 2 is optional. Hype it up, but don't treat it as a dealbreaker. If a customer only does Session 1, they're still in solid shape.

Track A: Product + engineering

Audience - PMs, engineers, data scientists. Anyone who ships features.

Deliverable - A live feature flag targeting a real user segment, plus a draft experiment with a defined hypothesis.

Feature Flags (15 min)

The gateway to Experiments. Nail this first.

- Create a flag together targeting a real segment (beta users, a specific country, a percentage rollout).
- Walk through the lifecycle: create, roll out to X%, check analytics, roll to 100% or kill it.
- Briefly cover multivariate flags, payloads, early access feature management.
- For engineering-heavy rooms: mention local evaluation and bootstrapping. These are top docs pages because engineers want flags that resolve fast on the client.

Experiments (10 min)

Start from a hypothesis, not a feature. Ask the room: "What's something you're debating shipping right now?"

- Set up a draft experiment: hypothesis, primary metric (a funnel or trend), control and test variant.
- Walk through how to read results. When to call it. Use a real experiment with good data.
- Mention no-code web experiments for quick wins that don't need eng work.

LLM Analytics (10 min)

Big for any team building AI features. Clustering in particular can provide insights that are otherwise hard to come by.

- Show what gets captured automatically: conversations, token usage, cost per model, latency, error rates.
- Walk through a generation: input, output, tokens, cost.
- Show traces for multi-step LLM workflows.
- Connect it to Session Replay: "here's the replay of a user interacting with your AI feature, alongside the trace."

If they're not building AI features, skip this. Spend the time on Feature Flags and Experiments instead.

MCP

- Make sure everyone who wants to use the MCP server has either set it up already or knows where to find the docs.
- Ask for a volunteer to try using MCP to create a feature flag. "Create a feature flag called 'new-checkout-flow' with 20% rollout targeting users in the US." For teams already using AI coding tools, this will probably be the single biggest takeaway from the entire training.

Bonus topics if time allows

- **Group analytics** - Account-level analysis for B2B products (setting up groups, group properties, group-based flags). This is the most important bonus topic for B2B customers – if the customer sells to businesses rather than consumers, prioritize this over the others.
- **Error Tracking** - Auto-captured errors, stack traces, connecting errors to replays
- **Data Warehouse** - SQL Editor and bringing in data from other sources

Track B: Marketing + growth

Audience - Marketing, growth, content, demand gen. Anyone who cares about acquisition and conversion.

Deliverable - A Web Analytics dashboard configured for their site, plus a live survey draft targeting a real user segment.

Web Analytics (15 min)

- Walk through the pre-built dashboard: visitors, bounce rate, top pages, traffic sources, devices.

- Marketing analytics: UTM tracking, channel attribution, entry/exit pages.
- Web vitals: page load performance. (Matters for SEO, matters for ad spend efficiency.)
- Clarify the relationship with Product Analytics: Web Analytics is aggregate and pre-built, Product Analytics is custom and user-level. Same data, different lenses.

Advanced funnels for marketing (10 min)

Build on what they learned in Session 1, applied to marketing use cases.

- Funnel from landing page visit to signup (or whatever their conversion event is).
- Break down by UTM source to show which channels convert best.
- Correlation analysis: what properties predict conversion?
- Time-to-convert: how long from first visit to signup?
- PostHog AI moment – "why are users dropping off between step 2 and 3?"

Surveys (7 min)

- Create a popover survey targeting a real page or user segment.
- Targeting options: URL, user properties, event triggers, device type.
- Question types: open-ended, rating, NPS, multiple choice.
- Mention response limits, scheduling, custom appearance.
- PostHog AI moment – summarize responses using AI.

Session Replay for marketing (5 min)

- Filter replays to users from a specific traffic source or landing page.
- Show rage clicks and dead clicks on key conversion pages.
- "Watch three replays of users who hit your pricing page but didn't sign up" is a strong closer for marketing audiences.

Workflows (3 min)

- Reach out to users at the right time based on their behavior.
- Show templates.

After Session 2

- Follow up on any unanswered questions. Share docs to anything that piqued their interest.
- Reach out to anyone that was invited but didn't join. Share some of the most interesting learning / Q&A topics.

Engagement tips

For the "too busy" crowd

Offer a 15-minute micro session. If someone reschedules twice, don't push. Offer to screenshare and build one thing while they watch. Low commitment, high value. Most people who do a micro session rebook the full one.

For $80k+ customers who are in-office – pitch a half-day onsite. Frame it as "we'll sit with your team and build your analytics stack together." Informal one-on-one time at someone's desk is worth 3x a scheduled Zoom.

Gather feedback with Surveys

Set up a PostHog survey targeting training participants after each session. This does two things: it collects real feedback on the training, and it shows attendees a live example of Surveys in action on themselves. Good dog-fooding moment.

Retention, Expansion & Cross-sell

contents/handbook/growth/sales/expansion-and-retention.md

As a Technical Account Manager, you'll spend as much time managing your existing book of business as you will closing product-led leads. Your first priority is retaining them - this is counter balanced to an upwards Land, Expand, Retain motion. We have to work twice as hard if we're trying to close new deals and make up for lost customers. You'll typically be assigned a bunch of customers who are paying monthly - this means they could turn off PostHog at any time.

Once you're confident that a customer isn't going anywhere, then you want to think about how you can expand their usage. Usually (but not always) this is after they've signed an prepaid credit contract.

In order of priority, your objectives should be following all points of "REREE":

- Retention - establish multiple strong lines of communication
- Expansion - cross-sell additional products
- Retention - secure a discounted, credit-based commitment (maybe, but not always - hard to do on just a single product!)
- Expansion - expand usage of the same product into new teams
- Expansion - expand usage of the same product in the same team

The reason why we put cross-sell so high up the list is that we have seen that by *far* the happiest and best-retained PostHog users, including from a revenue retention perspective, are those who have adopted 2+ products. It makes sense - it's relatively straightforward to replace PostHog if you're just using product analytics, but it's much tougher if you're using analytics + experiments + session replay.

Retention

Your objectives are to:

1. Get people to talk to you
2. Get a longer term commitment (maybe!)

1. Get people to talk to you

We have a handy guide to this in the getting people to talk to you playbook.

2. Get a longer term commitment (maybe!)

Once you've established contact, you basically want to get them into the same flow as if they were a new customer (and give them the same level of attention). You will be doing a combo of discovery and commercial evaluation, as the customer will want to figure out whether a prepaid credit contract with PostHog makes sense vs. what they've already got.

> Do not push for discounted, credit-based plan no matter what - consider what actually makes sense here! Some customers are very highly likely to stick with PostHog even if they are paying monthly, e.g. if they have many users regularly logging in, lots of product activity, multi-product adoption etc. Do not turn up to a new customer and the first thing they hear from you be 'would you like to pre-purchase credits?'

You'll also go through the same contracting process with them. We usually find that convincing a customer happily paying monthly to switch to prepaid credits is quite difficult, especially if they are a fast-growing startup (who tend to value flexibility over pure cost saving). This means that the discounts may not be as effective. If you're finding this is the case, you can get them on an prepaid credit plan but paying monthly or quarterly and halve the discount you offer.

Steady state retention

These are customers that are happily using PostHog long term, and are neither a churn risk nor likely to have expansion potential. Managing this group is much more automated and taken care of by CSMs, who do things like tracking usage and setting up alerts in Vitally to trigger outreach from us when a customer changes their usage behavior (either up or down).

An important part of retention here is also to ensure support issues are fixed in a timely manner. We deliberately don't want to invest a huge amount in hands-on customer success here, because that can often paper over cracks in the product experience or quality of our customer support, so staying hands-off here is an intentional strategy. In the future, we will build out this playbook a lot more.

Expansion & cross-sell

> Note: AEs and CSMs also do expansion at PostHog therefore this is not a Product-Led Sales TAM only approach. This is because we all are constantly on a sales footing with customers - for the most part, we don't do steady state account management with an arbitrary 10% uplift at renewal team.

An overview of how to drive expansion with a customer can be found in the cross-selling pages.

Principles for visiting customers

If you offer to do a meeting in person with a customer, they'll then feel obliged to introduce you to other people to make good use of your time. Trying to get them to adopt more products can be a good trigger, but generally you should be matching the cadence for in-person meetings with the size of contract (ie. more regular for Very Large, less regular for Large). If necessary you can request a budget for travel and accommodation in Brex.

Generally speaking you should be trying to regularly see customers in your book of business who are $100k+ annually, or could get there. Occasionally you can pull in James/Tim if they are traveling to SF/NY especially, or if the customer is in London.

If you regularly visit customers, you can (and should) take some sweet merch. You can self-serve this using a discount code pinned in our team Slack channel to get 100% off your order.

Make sure to log notes in Vitally when customer visits take place. This can be done by creating a new note with the "On-site" category and describing any key details and takeaways.

Expansion strategies

contents/handbook/growth/sales/expansion-strategies.md

The Expansion and Retention page lays out the REREE priority order for managing your book: retain, expand (cross-sell), retain (commit), expand (new teams), expand (same team). The cross-sell motions page tells you *what* to sell. The use-case-selling playbooks tell you *how to frame it*. This page covers the layer underneath: **how you structurally grow an account.**

Not every account grows the same way. A 30-person startup with one engineering team is a completely different expansion motion than a 500-person company with four business units. You need to pick the right approach for the account you're working with, and sometimes run multiple strategies in parallel.

These four strategies are not mutually exclusive. Most accounts will involve a combination over time. But being deliberate about which one you're running right now, on this account, this quarter, makes it much easier to focus your effort and measure progress.

Overview

Strategy	Core idea	Best for
Go deeper	Layer more products onto the team already using PostHog	Accounts with 1-2 products adopted, strong engagement, same team
Build champions	Grow usage and advocacy from individual power users up	Accounts where you lack executive access, or where adoption is engineer-driven
Expand into new teams	Replicate PostHog usage in a different team or business unit	Larger orgs with multiple engineering teams, product lines, or workloads
Move upward	Engage leadership to drive org-wide adoption or commitment	Accounts with strong bottom-up usage ready for credit purchase or org-wide rollout

Strategy 1: Go deeper on the existing team

The team already uses Product Analytics. You help them adopt Session Replay, then Experiments, then Error Tracking. Same people, same workload, more products.

When to use it

- Account has strong engagement but low product count (1-2 paid products)
- Your champion is receptive and has the ability to try new things without heavy approvals
- The team's workflows have natural gaps that other PostHog products fill

How to execute

1. Review their current product adoption against the use-case-selling framework. Identify which use case they're closest to completing and what product fills the next gap.
2. Tie the recommendation to something they've already told you. "You mentioned spending time reproducing bugs from user reports — Session Replay shows you exactly what happened" is better than "you should try Session Replay."
3. Offer a trial incentive if needed. 2-3 months of credited usage for a new product removes the risk for them. See trial/evaluation incentives.
4. Follow up with hands-on help. Don't just suggest the product — help them set it up, build their first dashboard or workflow, and show value in week one. A product training session can accelerate adoption if the team is large enough to justify it.

Signals that it's working

- New product shows up in their billing within 30 days
- Champion starts referencing the new product in conversations unprompted
- Usage is sustained (not a one-time spike)

Common mistakes

- Pitching products the team has no use for just to hit a product count target. If they don't need Surveys, don't push Surveys.
- Suggesting too many products at once. Pick the one with the highest likelihood of adoption and land that first.
- Not following up after the initial setup. Products adopted without guidance often get abandoned.

> Each additional product above 1 adds 0.2x to the quota multiplier (from a 0.7x base). Going from 1 to 3 paid products moves the multiplier from 0.7x to 1.1x on the same ARR. This is the most direct way to improve your quota math.

Strategy 2: Build champions from the bottom up

You identify 2-3 power users inside the account who are getting serious value from PostHog, and you invest in making them successful. They become your internal advocates, and their enthusiasm pulls in more users and more products organically.

When to use it

- You don't have (or can't get) executive access
- The account is engineer-driven and decisions happen bottoms-up
- There are individual users who are clearly engaged but haven't been given direct attention
- Early-stage companies where the "champion" might be a founding engineer or product-minded CTO

How to execute

1. **Identify power users.** Check who's logging in most frequently, who's creating dashboards and insights, who's asking questions in your Slack channel. These are your champions, whether they know it yet or not. If you're struggling to make initial contact, the getting people to talk to you playbook has specific tactics.
2. **Invest in them directly.** Share tips specific to what they're building. Point them at features they haven't found yet. Help them build something impressive they can show their team. The goal is to make them look like heroes internally.
3. **Equip them to sell internally.** When your champion wants to bring in Session Replay for their team, give them the ammunition: a short summary of what it does, rough cost estimate, and how to pitch it to their manager. The cross-sell motions page has product-specific discovery questions and value stories you can adapt for this.
4. **Ask for introductions.** Once you've built trust, ask your champion to introduce you to other people in the org. "Are there other teams that might find this useful?" is a low-pressure way to open the door to multi-team expansion.

Signals that it's working

- Your champion starts CC'ing or introducing colleagues
- New users from the account start showing up in PostHog
- Your champion brings problems to you proactively rather than waiting for you to reach out

Common mistakes

- Over-relying on a single champion. If your one contact leaves, you lose the account. Always work toward having at least 2-3 relationships.
- Treating champion building as a substitute for commercial conversations. Champions are great, but at some point someone needs to talk about contracts and commitments. Don't avoid that conversation forever.
- Ignoring quiet power users. The person creating 20 dashboards a week but never responding to your messages is still a champion — they're just not talking to you yet.

Strategy 3: Expand into new teams

Engineering Team A uses PostHog for product analytics. You get introduced to Engineering Team B (different product line, different business unit, different workload) and replicate the adoption. Same org, net new usage.

When to use it

- The org has multiple engineering teams, product lines, or business units
- Your existing team's usage is mature and there's limited room to grow with them alone
- You've identified (or your champion has mentioned) other teams with relevant use cases

- The account is at a stage where workload expansion is the primary growth lever (typically $60k+ ARR)

How to execute

1. **Map the org.** During discovery with your existing contacts, ask: "How many products or apps does your company maintain?" and "Which teams have their own engineering org?" Each product/app is a potential new workload. Your account plan should explicitly document known workloads and which teams own them.
2. **Get a warm introduction.** Cold outreach to a new team inside an existing account almost never works. Ask your champion to introduce you, or use in-person visits (people feel obligated to introduce you to others when you're physically there).
3. **Treat the new team like a new customer.** They have different needs, different stakeholders, different technical contexts. Don't assume that what worked for Team A will work for Team B. Run fresh discovery and consider offering a training session to get the new team up to speed.
4. **Start with the use case that fits, not the product the other team uses.** Team A might use Product Analytics heavily, but Team B might need Error Tracking first. Let the use-case-selling framework guide the conversation.

Signals that it's working

- New projects created in the PostHog org for the new team's workload
- Event volume from new sources (different SDKs, different domains, different app identifiers)
- New admin users from a different team or department

Common mistakes

- Assuming the new team has the same priorities as the existing one. They probably don't.
- Trying to expand into new teams when the existing team's implementation is shaky. If Team A is unhappy or poorly set up, they'll warn Team B off.
- Not involving your champion in the introduction. Going around your existing contacts to reach new teams damages trust.

> New team adoption is often the biggest single expansion lever in larger accounts. A new workload can mean an entirely new use case stack, which adds both ARR and product multiplier simultaneously.

Strategy 4: Move upward through stakeholders

You've built strong usage and advocacy at the IC and team lead level. Now you engage a VP Engineering, CTO, or Head of Product to drive an org-wide commitment: annual contract, standardization on PostHog, top-down mandate to adopt across teams.

When to use it

- Strong bottom-up adoption already exists (multiple users, multiple products, sustained usage)
- The account is large enough that an org-wide deal is meaningful ($60k+ ARR potential)
- You have evidence of value you can present to leadership (usage data, time saved, problems solved)
- There's a commercial event on the horizon (contract renewal, budget cycle, new fiscal year)

How to execute

1. **Build the business case before you ask for the meeting.** Pull together usage data, product adoption, number of active users, and any concrete outcomes your champions have shared. Leadership doesn't care that Session Replay is cool. They care that it reduced bug reproduction time by 50% and saved 10 engineering hours a week.
2. **Get introduced, don't cold-call.** Ask your champion to set up the meeting. "Would it make sense to loop in [VP] so we can talk about how PostHog fits into the broader engineering org?" Your champion's internal credibility is what opens the door.
3. **Frame the conversation around their priorities, not yours.** Leadership cares about consolidation (fewer vendors, fewer contracts), cost predictability (annual plan vs. monthly surprises), and organizational efficiency (one platform for all teams vs. five point solutions). Lead with those.
4. **Have a specific commercial proposal ready.** Don't go in with "we should do an annual deal." Go in with "based on your current usage of $X/month across these teams, here's what an annual commitment would look like, including the discount and what that saves you." See contract rules for discount structures, and remember that even after an annual deal is signed, additional usage beyond the annual run rate still counts toward your quota.
5. **Use the meeting to also open multi-team expansion.** "Are there other teams that should be using PostHog but aren't?" is a natural question when you're talking to someone with org-wide visibility.

Signals that it's working

- Leadership agrees to a meeting and brings relevant people
- Conversations shift from "should we keep using PostHog" to "how do we roll this out more broadly"
- Procurement or finance gets involved (this is a good sign, even if it slows things down)

Common mistakes

- Going over your champion's head without their knowledge. This destroys trust and usually backfires.
- Trying to go upward before you have bottom-up proof. If leadership asks "do our teams actually use this?" and the answer is weak, you've wasted the meeting and it's very hard to get a second one.
- Treating the executive meeting as a product demo. Executives don't want a tour of features. They want to understand business impact and cost.

- Moving upward too early in the relationship. If you're still establishing trust with the IC team, forcing an executive conversation feels pushy and premature.

Choosing the right strategy

There's no formula here, but some patterns hold:

Account situation	Start with
Small team, 1-2 products, strong engagement	Go deeper
Low executive access, engineer-driven org	Build champions
Large org, multiple teams or products	Expand into new teams
Strong bottom-up usage, approaching renewal or budget cycle	Move upward
New account, first 90 days	Go deeper (always start here)

For most accounts under $40k ARR with a single team, **go deeper** is the right default. You're adding products to the team that's already bought in.

For accounts over $60k ARR with multiple teams, **expand into new teams** is usually where the biggest growth lives. You can only go so deep with one team before you hit a ceiling.

Build champions and **move upward** are not standalone strategies — they're how you enable the other two. You build champions so they can pull you into new teams. You move upward so leadership can mandate adoption across the org. They're force multipliers, not end goals.

The best TAMs are running 2-3 of these in parallel on their largest accounts. One team is going deeper on products. A champion in that team is introducing you to another team. And you're building toward an executive conversation that ties it all together into an annual commitment.

Getting people to talk to you

contents/handbook/growth/sales/getting-people-to-talk-to-you.md

Product engineers, our ICP, are very self-serve and happy to implement PostHog themselves and read the docs without ever interacting with someone unless they have support queries.

Why is it helpful for someone to talk to you?

The reasons have to be *genuinely helpful* ones - just 'having a point of contact' is not enough. Reasons include:

- You can save them money:
  - They've implemented PostHog in a silly way and are consuming stuff they don't need
  - They can pre-commit and get a discount on credit
- You can help them get more out of PostHog for the same amount of money, e.g. if they're ingesting loads of events but not using features to their fullest
- You can train their team on how to use PostHog, so they don't have to
- You can make them aware of upcoming or new products that are specifically useful for their use case
- You can be a shortcut to premium support, if they are in your book of business

If you go down the 'saving money' route, bear in mind two things:

- Prepaid credit never works as an opener - 'save money by fixing implementation' >>> 'save money by committing to credit at a discount'
- Buying a bunch of credits at a nice discount is much nicer to hear than 'please commit to a scary annual plan' - they can commit for a year, 6 months, whatever so long as they buy >$20k up front

How to get people to talk to you

This is usually the most difficult bit! Sometimes customers will proactively reach out to us because they see their bill rocketing, but we have many customers who have happily self-served to a very high level of spend without feeling any need to talk to us. In particular, engineers have no interest in jumping on a call with you 99% of the time.

- Offer to optimize their usage/reduce their billing - if they are pointlessly tracking a bunch of junk, tell them! Otherwise they'll just find out themselves and churn anyway.
- Tell them about new or upcoming features or products that they may not be aware of which you know could be a great fit for them (and let them try them out for free).
- Use multiple channels - email is usually the worst way to reach our ICP. Slack, in-app Surveys or even Telegram are all usually better. But try email first anyway.

- Ping everyone in the account individually (don't do group, no one will reply) - start with the most active users first. New users are also good.
- Loom videos sharing your observations about their usage/account provide a personalized and human touch which can go a long way to building lasting relationships. Ask Simon for an invitation to our company account if you don't have access.
- Adding the contact on LinkedIn and sending a very human video or audio message can work really well - even for technical people (use the LinkedIn mobile app).
- Figure out what the non-technical people in their team need and then go out and talk to them - get someone who isn't an engineer to talk to us given engineers don't want to.
- If they submit a support request, jump in and respond yourself to try and build a relationship.
- Ask the wider team for help - we have to get creative here! You'd be surprised how often somebody knows someone...

> Before you do any of this stuff, get to know your customer as well as you possibly can. Don't do clickbaity things or trick people into talking to you - it'll just annoy them. And definitely don't just offer a generic checkin 'to see how things are going'!

Ideally you want to get multiple people into a shared Slack channel, as we've found this enables the best communication and allows us to provide them with great support. Just adding a bunch people to the Slack channel is also a legit tactic - forgiveness, not permission.

Your first message where we've never had contact

Despite the organization using PostHog, they may not recognize you/PostHog, or may not even be the correct person to talk to about PostHog, which means your message needs to be well crafted.

When crafting a message, consider the following:

1. Your initial outreach isn't about you, it is about them. Lead with customer-centered comms. Avoid leading with being attached to their account or telling them how you are there to help them. Tim has some great thoughts on this subject.
2. Avoid fluff. "I'm just reaching out to", "I just wanted to" etc. are empty phrases that take longer to get to the point. Before you hit send, reread and see if there is anything you can cut out.
3. Lead with value within the first sentence. If it takes a paragraph to get there, you won't get responses.
4. Ask yourself, if I got this email to the sales@ email box, would I engage it? Would I even give it a second look?

Some examples of good emails that have worked:

> Hello [name], It looks like your Product Analytics usage has increased over the past month and I wanted to ensure that the increase was expected. Here are some tools you can use to ensure you are collecting the correct events and getting valuable insights from them. We have a whole host of tutorials and guides to help you get the most out of PostHog. If you have any questions, don't hesitate to ask.

> [First], Wanted to reach out direct since I noticed the [Company] team ramp up usage in PostHog recently. We'll typically reach out to help with optimizing event capture and make recommendations with regards to instrumentation + querying in PostHog. Up for a chat? Here's my calendar, feel free to grab a time that works best for you. Cheers,

Asking for introductions

If you feel like you have done a good job with a customer, and have genuinely been helpful, it's ok to ask for a favor back. You can be specific and ask for a direct introduction to a person you want to talk to, or try go a bit more broad and ask the person if they know anyone who would benefit from some help with PostHog. Either way, a warm introduction from a colleague is always going to be better than reaching out on your own. Something like "Hey Leon, our session last week seemed to have landed well. I'm glad you found it useful. I was wondering if you could help me out. Your team is growing really quickly, and there's a bunch of new folks starting to use PostHog. I imagine not all of them are super comfortable with the platform yet and could use a helping hand. Could you introduce me to Simon, Charles and Scott?"

Just been handed an account?

Sometimes you'll get a customer in your book who was previously working with someone else on the PostHog team. A pre-existing relationship can help, but it's not guaranteed they'll want to talk to you.

We've found a message like this in Slack/email works well after the intro:

> Thanks [PostHog team mate] Hey [customer] :blob-wave: Excited to be working with you! As I take over, it would be a big help if we could schedule a quick 15–20 minutes intro call [link to your Calendly]. Just a chance for me to learn more and figure out how I can best support you going forward. Let me know if you'd be open to that.

We've found most people will respond to this.

Have you been ghosted?

If you've had a conversation with someone, there was interest on their side and then they suddenly go dark, using the John Barrows Ghosting Sequence can revivify them.

1. After 2 weeks of valuable follow-up and you've not heard back, reply-all to the latest email thread. Change the subject to: "Still interested?"

And put in the body:

> `[Name]`
>
> Still looking at options like PostHog to solve `[business problem they previously acknowledged]`?
>
> Let me know either way.

That last line is very important because it gives them a safe option to say "no". About half will respond. 2. If there's no response again after another week, change the subject again to "Did I lose you?"

Leave the body empty. This will pick up about 80% of people who go dark. If not, close out the opportunity 3 days after this final message.

LinkedIn Sales Nav

To get notified about new hires and other changes to the accounts you manage, you can set up lists of accounts to track in LinkedIn Sales Nav.

1. Search for an account you want and click on their profile.
2. Click the star icon on the left, and then choose a list to add them to.
3. Optional, tailor the notifications you get in LinkedIn

You will now be notified any time a senior hire joins your account, which will be helpful for tracking folks to reach out to and give advanced signals around potential data science hires.

Historical import

`contents/handbook/growth/sales/historical-import.md`

Historical import

Since our system does not experience a huge variance in incoming traffic aside from the occasional instrumentation bug, it's important to give the pipeline team a heads up in advance, since we may rate limit the requests. Additionally, we need to clarify a few commercial and technical points before giving the green light.

1. Make sure they have their product questions answered first, ie, they are not relying on historical import data to validate their use case. It's ok for this to be a contingency of them using the product/paying us, but we should be pretty sure that they are committed so we can avoid asking pipeline team to spend (sometimes considerable) effort managing an import only to have a user decide we're not a good fit.
2. Customer should answer the following:
    - When is this scheduled for (strong preference for weekdays with the most EU-timezone overlap)
    - Regarding the actual request(s), what can we expect around:
        - batching
        - distinct_id variance, specifically, max events per distinct_id for the top few users (eg `select count(*), distinct_id from events group by distinct_id order by count(*) desc limit 100` or similar)
        - event types (some require more consideration to process than others, eg $create_alias , $identify etc)
    - Will you have apps enabled
    - What will be the peak rate and total duration of calls
    - What will the ramp-up profile look like

If the count of events for a given distinct_id is too high, we may relax the constraint that events for a single distinct_id are always sent to the same kafka partition, which means these events might not be processed in the correct order. This can be problematic for merging events, where order of ingestion matters (eg an alias event arriving before an identify event on which it depends). This will need to be communicated to the customer.

Load testing

If a customer mentions load testing, get answers to the above and then alert the pipeline team asap, so that accommodations can be made, as this may require scaling up to handle properly. If a customer plans to send a large volume of single capture requests all at once, rather than ramp up to a peak over some time period, that is not a load test but more like a denial of service (DoS).

How to do discovery

contents/handbook/growth/sales/how-to-do-discovery.md

Discovery

The discovery mindset

Discovery isn't walking through every PostHog feature. It's having real conversations with customers to figure out if PostHog will be a good fit for them. Learn their problems, see how they solve things today, and find the people who'll get excited enough to bring us in.

This is meant to be a guide, not a rule-set. Each person has their own unique style. The goal here is to surface the right insights by providing a framework for how to go about *asking the right questions* vs. a talk-track for how to run discussions with customers.

Core principles:

- **Curiosity over pitching** - Be genuinely interested in their challenges
- **Find the pain** - People buy solutions to problems, not features
- **Identify champions** - Someone needs to sell internally when you're not there

> Discovery isn't one-sided questioning - it's give and take. You learn something, you show something, you ask questions, repeat. The goal is understanding what customers are trying to accomplish so we can focus on relevant features rather than discussing everything PostHog can do.

Why discovery matters

PostHog is a broad product suite with common combinations depending on the use case. Discovery can help us provide customers with a better experience by understanding their specific needs so we can:

- **Conduct a demo that actually matters** - No one wants to sit through features they'll never use
- **Draw connections between their problems and our products** - There are 10+ products (and counting), we want to help them find the right combination
- **Skip the irrelevant stuff and get to the good bits** - Customers' time is valuable, and generic sales calls aren't
- **Reduce time & effort needed for them to make a confident decision** - By understanding requirements upfront, we can address concerns early and focus on what matters most to stakeholders

Timeline

We don't need to cram every question into the first call. Discovery is always happening and we have many customers who stay with us long term. Use each touchpoint to learn something new.

Other channels

Beyond the 1st call, there are other spaces where we frequently communicate with customers:

- Zoom / Phone
- Email
- Shared Slack channel
- DMs in Slack / LinkedIn / X
- Text / SMS

> Most customers are preferential towards Slack while others like email/Zoom. Slack is central to communications at PostHog and tends to be a great place to offer real-time support and ask questions.

Before your 1st call

Prep work

Discovery includes preparation. Before speaking with any new customer interested in engaging further with PostHog, it's helpful to gather some basic knowledge to help with demoing relevant features and determining if it's a good fit.

Examples:

- Learn more about our ICP
- Cross-reference Vitally, PostHog, Slack and Salesforce for any prior engagement or current activity/status
- Learn more about who you're speaking with via LinkedIn/X
- Visit the company's website, learn about their product, who they are marketing/selling to, language they are using. Familiarize yourself with what may be important to them
- Use ChatGPT, Perplexity, Claude, etc. to help research the company, their industry, macroeconomic factors and potential use cases for PostHog

Asking questions

Discovery is about understanding the real problem through natural conversation. The goal is to be genuinely curious about their situation, not to interrogate them.

Question principles:

- Use "what" and "how" to signal curiosity rather than judgment

- Start questions with "tell me...", "explain to me...", or "describe to me..." to avoid yes/no answers
- Focus on understanding their current state and challenges
- Ask about consequences and impact naturally as the conversation flows

Understanding customer goals

Instead of asking about room for more PostHog products, ask about what the customer is trying to accomplish. Questions like "what's coming up in your roadmap over the next few months?" get better intel without feeling like an upsell and it's just generally a much more natural conversation.

When you understand their goals, you can frame PostHog around outcomes instead of features. For example, if you learn they're launching a jobs board and their GTM leans on niche SEO, you can shape the demo around using web analytics to nail that launch. You're telling a story where PostHog helps them succeed, not just showing what buttons do.

Goal-discovery questions:

- "What's coming up in your product roadmap over the next few months?"
- "What's the biggest thing your team is trying to accomplish this quarter?"
- "What does success look like for your team right now?"
- "What's keeping you up at night with work?" (only if you have genuine rapport)

When to use these:

- **In prep** — research their roadmap, recent launches, and blog posts beforehand, then confirm on the call
- **During discovery** — weave into the opening conversation naturally
- **At the end of a call** — as a lightweight follow-up: "before we wrap, anything big coming up I should know about?"
- **In follow-up comms** — as part of a warm intro or handoff (e.g. TAM introducing to the engineering team)

Important: This only works if you're genuinely curious. It's not a checklist item for every call — forced interest is gross and salesy. But when the connection is there, it's a much better place to frame what we offer from.

What makes PostHog different

The demo

Demoing PostHog is an important part of our sales process and how we first introduce PostHog to customers. It brings immediate value to a call, is consistent with other messaging and builds credibility with technical folks.

A demo can also be a great format where questions bubble naturally.

Principles:

- Leverage PostHog's technical credibility by showing vs. telling
- Uses the demo as a conversation starter rather than a monologue
- Be adaptable

Examples:

- Demo of Product Analytics: *Showing a funnel analysis*
  - Questions: "Is there a conversion flow you're currently struggling to understand?"
- Demo of Session Replay: *Showing user session with errors*
  - Questions: "How do you know if users are struggling with your product?"
- Demo of Web Analytics: *Showing UTM sources breakdown*
  - Questions: "How are you currently attributing conversions across channels?"
- Demo of Autocapture: *Showing retroactive insight creation*
  - Questions: "How much dev time do you currently spend on instrumentation?"
- Demo of Data pipelines: *Showing how to create a destination*
  - Questions: "Is there anywhere you'd ideally like to send data back out to?"
- Demo of Error Tracking: *Showing error dashboard*
  - Questions: "How are you prioritizing bugs to fix first?"
- Demo of Data warehouse: *Showing available sources*
  - Questions: "Are there other data sources that would be valuable to query alongside PostHog data?"
- Demo of Experiments: *Showing Experiment dashboard*
  - Questions: "How are you currently cross-referencing test results with other user-behavior data?"
- Demo of LLM Observability: *Showing LLM Dashboard*
  - Questions: "How are you gathering data for your AI/LLM products?"

Other questions you could ask while demoing:

- "Who else would find this valuable?"
- "How does this compare to how you're handling this today?"
- "Of what was covered, what did you find most valuable?"

Qualifying

A key component of discovery is qualifying customers to ensure they are a good fit and whether they're speaking with the right people at PostHog. You can find more about how we qualify at PostHog in the new sales qualification guide.

Qualifiers:

- They fit the ICP
- Path to $2k/mo or $20k/year in spend

- Clear problem that PostHog can solve
- Have technical resources to assist with instrumentation of PostHog
- Budget and timeline in place (or keen on moving quickly)

Disqualifiers:

- **No path to >=20*Kannualspend * * – *Ourteamlooksaftercustomerswhoarepayingorcouldpay*[> =20k+/yr](https://posthog.com/handbook/growth/sales/new-sales#leads-below-the-sales-assist-threshold-less-than-20k-arr)
- **Not in ICP** - You can gather great product insights when chatting with people in other roles, but we tend to work best with our ICP
- **Can't meet technical requirements** - To be successful, the customer needs to (at minimum), be able to implement PostHog via the JS/SDKs
- **Support request** - Be helpful, but if it's better suited for support, you can send them thru the available Support channels
- **Startup program** - For companies who qualify, we have a special program designed for startups interested in PostHog
- **Needs to vary our terms** - Either to start an evaluation/PoC of PostHog or generally but without a path to >=$20k in annual spend
- **No engineering resources** - There is some coding required for a tool like PostHog and customers will need some engineering support to be successful
- **Strict compliance constraints** - A customer may ask for a very niche security or privacy certification that we don't have
- **Need self-hosted** - PostHog's self-hosted open-source deployment is made for hobbyists and since we're a small team, we can only provide limited support for it

Identifying your champion

Champions aren't just customers you're friendly with - they're people who will actively sell PostHog internally. While you won't always find a champion, working with one when possible can streamline deals and provide us with valuable feedback along the way.

Examples

Questions to identify champions:

- "Who else is affected by this problem?" (Look for advocacy in their response)
- "How do you typically evaluate new tools at [Company]?" (Champions know the process)
- "What would need to happen for this to get approved?" (Champions understand internal politics)
- "Who would be most excited about solving this?" (Champions will often name themselves)

Characteristics to listen for:

- Using "we" and "us" language (ownership)
- Asking for detailed technical / instrumentation questions

- Mentioning budget or approval processes prior to you asking
- Referencing internal stakeholders by name
- Expressing personal frustration with their current state
- Have a vision for what future state needs to look like

Follow up questions for champions:

- "What's your role in making this decision?"
- "How have you handled similar evaluations in the past?"
- "What concerns might others have about changing tools?"
- "Besides you, who else would we need to win over?"
- "What questions should I be asking that I haven't asked yet?"
- "If you were me, how would you go about positioning PostHog?"
- "What's the best way to position this to [insert stakeholder here]"
- "How can I help you build your internal case for PostHog?"

> While you can start identifying potential champions early in the process, building the relationship is an ongoing effort.

Discovery call structure

> Give yourself enough time to demo - it can make all the difference!

1. Opening & understanding the situation (~5-7mins)

Goal: Get rapport, learn about their setup, and uncover any frustrations.

Potential questions to flow between:

- "What prompted you to reach out to PostHog?"
- "What are you using for analytics today and how's it working?"
- "What's your experience with tools like PostHog?"
- "Who on your team uses this data and for what?"
- "What decisions are you trying to make that you can't make today?"
- "How does your team typically evaluate new tools?"
- "Is there anyone else who should be part of these conversations?"

2. PostHog demo (~15-20min)

Goal: Show PostHog, focus on relevant features, establish technical credibility, get feedback and ask questions.

Reference the demo section above for how you can incorporate discovery into your demo and learn more about how we do sales in the initial demo playbook.

3. Closing and next steps (~3-10min)

Goal: Establish timeline, confirm mutual fit and next steps.

- If it's not a fit, that's okay! We want to ensure we're not wasting anyone's time. Get alignment from the customer and there may be an opportunity in the future
- If there is a clear opportunity, offer up some actionable next steps (free trial, invite to Slack, generating a quote, help reduce spend, scheduling a call etc.)
- Set expectation that you'll follow up via email or Slack
- Gain an understanding of their timeline
- Route the customer to the next best channel if they are better handled by a separate team (Support, Startup program etc.)

We like to keep things conversational - if you're genuinely curious about their situation, this should all come naturally!

Summary

Discovery can help with addressing gaps in your knowledge about a customer and makes efficient use of both your time and theirs. By understanding their actual needs, challenges, and decision-making process upfront, you can:

- Keep conversations focused
- Avoid wasting time on irrelevant features or solutions
- Build trust through genuine understanding
- Identify the right internal champions
- Move deals forward more efficiently

Helpful docs for more learning:

- Overview
- Account planning
- Outbound sales
- Inbound sales

How we work

contents/handbook/growth/sales/how-we-work.md

> This page covers more of the operational detail of how our team generally works - for a broader overview of roles and responsibilities, visit the overview page.

Roles

We have three types of roles:

- Technical Account Executives - closing new business from inbound and outbound leads and expanding their usage of PostHog in the next 12 months
- Technical Account Managers - expansion from existing customers, closing new business from product-led leads
- COMING SOON: Business Development Reps, aka BDRs - generating leads for team new business

Technical Account Executives

TAEs work with:

- People who email sales@ directly
- People who book a demo via contact sales
- Other triggers we see in product, supplemented by data from Clay

As we start to generate cold outbound leads, these will be routed to TAEs to work with as well. Customers move off of a TAE to a TAM or CSM 3 months after closing on a prepaid contract (usually annual) - you have to ensure they are well set up, not just contract signed!

TAE Territory Review

In addition to the weekly sprint planning meeting on a Monday, we do a weekly territory review standup on Wednesday. A Technical AE is picked at random, and we spend 30min going through:

1. Brief, mid-week announcements (if any)
2. For one random Technical AE as chosen by the wheel of names - SFDC Hygiene check — is the deal value, stage, and close date accurate? Are the next steps up to date? No story time here, just data.
3. Biweekly, we review all larger ($50k+) opportunities across all Technical AE. For each opportunity, the person reports and discusses:

- Opportunity value and close date - why this value? when do we think it will close?
- Progress towards exit criteria of the current stage
- Concerns and questions about the opportunity

4. On alternate weeks from the larger deal review, we run wheel of names again (excluding the Technical AE selected for the hygiene check), and the selected Technical AE reports and discusses the opportunities in their pipeline, including:

- Starting with later stage opportunities, discuss opportunity value and close date - is the value solid? and what confidence do we have in the close?
- Progress towards exit criteria of the current stage
- Concerns and questions about the opportunity

> The objective of the meeting is to hold each other to account, provide direct feedback, and also support each other. It is a great place to ask for help from the team with thorny problems - you should not let your teammates fail.

Technical Account Managers

Each TAM is assigned up to 15 existing customer accounts to work with. Additionally, you will manage inbound leads as they are assigned to you in your territory. Overall, the hard cap on existing book + new leads is 25 accounts, so staying extremely focused is important.

We use the "AM Managed" Segment in Vitally to show that an account is part of somebody's book of business and therefore included in individual and team quota calculations. AMs should not assign this themselves (that's up to Simon or Charles), but can add themselves as the Account Executive in Vitally to make it easier to track things you're working on.

For Product-led leads we will only add them to your book for quota purposes if you have a solid plan in place for conversion to prepaid credit or cross-product adoption. Account Owners can use the "Leads" Segment in Vitally to separately track these from the main managed book.

At the end of each quarter we will review your accounts and look to hand off some to bring your focus account list back down to 10. Simon and Charles will also review everyone's accounts each month proactively to make sure that the balance of accounts across the team makes sense.

TAM Territory Review

In addition to the weekly sprint planning meeting on a Monday, we do a weekly territory review standup on Wednesday. A Technical AM is picked at random and runs through the following for each customer in their book of business in Vitally:

1. Rate your relationship with them (no connection yet/made contact/answering their questions in Slack/trusted advisor)
2. What's your next step with that customer (annual plan, cross-sell etc).
3. Are they a churn risk and why?

> The objective of the meeting is to hold each other to account, provide direct feedback, and also support each other. It is a great place to ask for help from the team with thorny problems - you should not let your teammates fail.

Handing off customers to Technical CSMs

We want to ensure the expansion potential of a customer has been thoroughly exhausted before moving to a Technical CSM for more steady-state retention. When you want to move a customer off your book you should talk it through with Simon. Here are the things we will be looking at:

1. Have you tried multiple times to make contact with all of the active users in an account?
    - An *Active User* is someone who has been seen in Vitally in the past month
    - When you reach out, demonstrate how you can help that person out, be specific to their role/usage of PostHog.
2. Are they using all PostHog products?
    - If they have been customers for a while they may not be aware of new products like Surveys and Data Warehouse. Look at their usage and see if there are any obvious cross-sell opportunities.
    - Could they benefit from some of the advanced capabilities and training/support available in Teams/Enterprise?
3. Is there an opportunity to cross-sell to a different team?
    - Have a look at what they are tracking with PostHog. If it's an app then maybe get in touch with the marketing team to talk about Web Analytics or No Code Testing
    - Are they a multi-product company? Find out if there are other teams who aren't using us who could benefit from PostHog today, and then use your current users as an internal reference.

If the answer to any of the above questions is 'no' then it's likely that there is more work to be done with a customer, but we will use a common sense approach here.

> A customer being negative/difficult to work with isn't a reason to remove them from your book. It's your job to turn them around to being a happy customer (AKA be their favorite).

How commission works - Technical Account Executives

General principles

- When thinking about commission, we want to particularly incentivize:
    - Landing new customers
    - Quickly expanding them into new products using the relationship you've developed in onboarding them as customers.
- We aim for a 50/50 split between base/commission when calculating OTE by default.

> This plan will almost certainly change as we scale up the size and complexity of our sales machine! This is completely normal - we will ensure everyone is always treated fairly, but you need to be comfortable with this. For now we are generally trying to optimize for something straightforward here so it’s easy for PostHog (and you) to calculate commission. Fraser runs this process, so if you have any questions, ask him in the first instance.

Variables

- Your quota is set for the year and then divided by 4 - this means you don't have to cram deals into the end of a quarter.
- Commission is *uncapped* and paid out on a sliding scale based on the % of your quota you hit. Hit 100% quota, get 100% of commission. 0% for 0%. And 200% for 200%.
- Quota is based on $ amount sold, not credits/product usage, so you can't in theory sell a $500k deal with an 80% discount and claim the full $500k to your quota, for example. Ways to hit quota:
    - The invoice payment amount for any pre-purchased credit deals in the first 12 months after they become a paying customer.
        - If the purchase is a renewal of an earlier credit purchase (i.e. at the end of the first year) then you'll get recognised on the difference between the initial purchase and renewal purchase.
- ARR from monthly customers for the first *12 months* after you sign them up as a monthly customer as long as you are the primary account owner.
- For multiyear contracts, we will true the quota ARR up to the year 1 equivalent amount as you'll have given a deeper discount but there is more committed revenue for PostHog which is a good thing.
    - The way we work this out is by taking the annual credit purchased by the customer and applying the standard 1 year discount to it.
- Your quota will depend on your OTE
- Commission is paid out quarterly, and is subject to clawbacks if the invoices remain unpaid.
    - We want you to secure upfront payment - which helps PostHog and helps you.
        - If you close an annual contract with monthly payments, you will still get recognized for the full commission amount, but the actual payout of your commission will be quarterly.
    - We want you to ensure the customer has paid, and we don't want AEs to throw invoice chasing to a finance person
        - This means you should make friends with the finance person on the customer's side, and ensure all payment paperwork is in order to allow for the customer to pay.
    - For monthly customers, commission is only paid after all 3 invoices have been paid
        - Commission is still paid out quarterly even if the customer pays monthly
    - Overdue invoices from the current quarter will be excluded from commission payouts, with the cutoff being the 14th of the calendar month following the quarter (January, April, July, and October)
    - Invoices that are issued in the final period of the current quarter, but are due at a date beyond the 14th of the calendar month following the quarter (January, April, July, and October), will be paid on the good faith assumption that the customer will pay on time and you will assist in securing timely payment. If the invoice becomes overdue in a future quarter, it will be subject to a clawback in that quarter.
    - If we have to give a customer a big refund, we'll deal with your commission on a case by case basis or via clawback.

    - Commission payments are made at the end of January, April, July, and October. Fraser will send you an email that breaks down your commission and explains how you did.
- In your first 3 months, you'll be paid 100% OTE fixed. You can find more info on how quotas work in your ramp period [in the new hire FAQ]

Performance expectations for Technical Account Executives

There are [cultural] and [role-based expectations] for TAEs at PostHog. We also now have enough data to define minimum performance exceptions for TAEs relative to the annual commmission targets.

After your ramp period, you should expect to have a performance conversation with your lead and Charles if:

- You are under 80% of your annual quota, *and*
- You have finished two consecutive quarters under 70% of your quarterly target

These standards are likely to change as the TAE role evolves. Any changes will be reflected in the handbook. We will always consider any relevant context when having these conversations with you - quota does not exist in a vacuum!

How commission works - Technical Account Managers

General principles

- When thinking about commission, we want to particularly incentivize:
    - Cross-selling new products - all-in-one is how we will beat the competition.
- We aim for a 50/50 split between base/commission when calculating OTE by default.

> This plan will almost certainly change as we scale up the size and complexity of our sales machine! This is completely normal - we will ensure everyone is always treated fairly, but you need to be comfortable with this. For now we are generally trying to optimize for something straightforward here so it's easy for PostHog (and you) to calculate commission. Fraser runs this process, so if you have any questions, ask him in the first instance.

Variables

- Your quota is set as *the additional $ on a usage basis you are expected to add to your book of business* - ie. any new product usage counts. This is different from TAEs, because here we care about the invoiced usage *not* the actual $ amount.
    - For example, if you start a quarter with $700k in ARR and are set a target to grow this by $200k ARR, your commission is based on your attainment towards the $200k figure based on amounts invoiced.
    - We measure the change in annualised quarterly ARR. Take Q1's usage ARR x4, compare it to Q2's usage ARR x4 - the difference in these numbers is your attainment towards quota.

        - Where a customer is new and has <3 periods in the previous quarter, we will use the number of periods to calculate the ARR. eg if it has one month it is that month multiplied by 12, if it has two invoices, then its the total of those two months multiplied by 6.
        - When a customer has churned eg they have no revenue in the final period of the month and/or they have cancelled their subscription, then that quarter will count as $0 ARR.
- This means you can hit quota by a combo of bringing in new business and expanding existing. Because your target is based on invoiced usage, this means that even if you have an annual customer in your book, you can still expand their usage and get recognized for that.
    - It also means that you are less likely to totally neglect existing customers because if they reduce usage, it hurts your overall ARR figure.
- We apply a multiplier to each invoice in the calculation based on how many of our primary products they are paying for, to incentivise cross-sell.
    - Primary products are: Product Analytics, Session Replay, Feature Flags, Surveys, Error Tracking, LLM Analytics, Data Warehouse, CDP Destinations, Workflows, Logs, and PostHog AI.
    - We start off at a base of 0.7x for customers with only 1 paid product, as it represents a bigger churn risk.
    - We then apply an additional 0.2x for each paid product above 1 (ie, 3 paid products = 1.1x)
    - A product is counted as paid if the invoice amount for that product is greater than $200
- Commission is *uncapped* and paid out based on the % of your quota you hit, on a sliding scale. Hit 100% commission, get 100% of commission. 0% for 0%. And 200% for 200%. Ways to hit quota:
    - Increase ARR for your monthly customers
    - For customers already on annual plans, additional usage ARR *beyond* their annual run rate - for example, if you have a customer on a $120k annual contract, but they are being invoiced $20k/mo for their usage, you will get recognized on the additional $10k/mo
    - Your quota will depend on your OTE
- Commission is paid out quarterly, and in any case after an invoice is paid
    - We don't want TAMs to throw invoice chasing to a finance person - you should make friends with the finance person on the customer's side too
    - For monthly customers, commission is only paid after the first 2 invoices have been paid (ie. you don't get commission due to a random spike)
        - To clarify, this means the first 2 invoices the customer has ever paid, ie. you still get commission from 'your' month 1 if you inherit a paying monthly customer
        - Commission is still paid out quarterly even if the customer pays monthly
    - If we have to give a customer a big refund, we'll deal with your commission on a case by case basis - in the future we may introduce a more formal clawback
    - Commission payments are made at the end of January, April, July, and October - at the end of each quarter, we'll monitor how many invoices actually get paid in the first two weeks of the next quarter. Fraser will send you an email that breaks down your commission into the above 4 buckets and how you did.

- In your first 3 months you are expected to retain your existing book and have closed at least one deal (either totally new or converting an existing customer to annual) - you'll be paid 100% OTE fixed. You can find more info on how quotas work in your ramp period in the new hire FAQ

> Your quota and assigned customers are likely to change slightly from quarter to quarter. In any case, your quota will be amended appropriately (up or down) to account for any movement. We will also be flexible in making changes mid-quarter if it's obviously the sensible thing to do. If you inherit a new account, you have a 3 month grace period - if they churn in that initial period, they won't be counted against your quota.
>
> If you have a customer you converted from monthly to annual under the old, non-usage-based commission plan, you won't *also* get recognized for additional usage beyond their annual run rate in the first year - no double dipping!

If you believe there is a justifiable reason to vary these rules, then in the first instance talk them through with your team lead. Simon (Charles as backup) will be the decider here.

You can see how we are tracking on the TAM Quota Tracker dashboard.

TAM book of business rules

1. Only accounts with the `AM Managed` segment in Vitally will be counted towards your quota. Simon adds this manually after reviewing with you and your team lead.
2. All accounts in the `AM Managed` segment need an account plan in Vitally, which is updated and reviewed with your manager regularly.
3. If you are assigned an account with no previous owner, you have up to 3 months to figure out whether they should be in your book or not. Don't ask for the `AM Managed` segment to be added until you're happy that there is growth potential there.
4. If you are assigned an account with a previous owner, work with them on the handover process. If the customer isn't in a healthy state usage and engagement-wise, feel free to push back and ask for the previous owner's help in getting them to a good state before taking ownership. If you really can't resolve this, then talk first to your team lead. If you can't resolve it, Simon will be the tie breaker. It may be that we need you to work on the account regardless but will treat it as a lead with the same rules as point 3 above.
5. Accounts which you've previously been paid quota on need to stay in your `AM Managed` book until they are handed over as per 3 above, or until they churn/fall below $20K ARR. In this case, we will keep them in the `AM Managed` segment for quota calculation purposes and then remove them after the quarterly calculations are complete.
6. Nominally, you should have 15 accounts/around $1.5m in ARR in your `AM Managed` book. There is some wiggle room here, but if you find yourself with 25+ accounts, it's unlikely that you'll be able to give them the level of focus we expect from a TAM, so you should be prepared to hand some over to another team member.
7. You can have accounts added to your book at any time, if you are comfortable that there is growth potential there. Removal of accounts should only happen at the end of the quarter so that quota can be calculated.

8. If you actively work to reduce a customer's spend with us by optimizing their usage, we may exclude that usage drop from quota calculation. We will review this on a case by case basis but at the very minimum you'll need documented evidence of the work you did to optimize their usage before it dropped. This should first be reviewed with your team lead who will then ask for approval from Simon. To make the process easier, drop the details of your optimizations as a note on the customer record in Vitally.

> We have a bunch of accounts where they are declining for reasons that have nothing to do with a TAM's actions. We also have a bunch where they are growing in the same way. These even each other out in the bigger picture of hundreds of accounts, if anything in favor of the latter. If they fit the criteria for having a TAM assigned, you should be prepared to continue to manage both types of customers in your book, as churn prevention is a key part of the TAM role too.

How commission works - BDRs

General principles

- When thinking about commission, we want to particularly incentivize:
  - Generating high quality leads
  - Getting people in who fit our ICP, ie. are easier for us to sell to
- We aim for a 70/30 split between base/commission when calculating OTE by default.

> This plan will almost certainly change as we scale up the size and complexity of our sales machine! This is completely normal - we will ensure everyone is always treated fairly, but you need to be comfortable with this. For now we are generally trying to optimize for something straightforward here so it's easy for PostHog (and you) to calculate commission. Fraser runs this process, so if you have any questions, ask him in the first instance.

Variables

- Quota is based on the number of sales qualified opportunities you generate - basically when an account moves into the Opportunity stage in SFDC
- Your quota is set for the year and then divided by 4 - this means you don't have to cram meetings into the end of a quarter.
- Commission is *uncapped* and paid out on a sliding scale based on the % of your quota you hit. Hit 100% quota, get 100% of commission. 0% for 0%. And 200% for 200%.
- Commission is paid out quarterly.
- There is no guaranteed commission during ramp, as the ramp period for BDRs is much shorter than for TAMs/TAEs.

Team lead quota

From your first full quarter as a team lead in Sales, you will move to a 60% base 40% commission split in reflection of your new player/coach role. This will be based on your team's quota attainment although you will still have your own individual quota target.

> Your individual quota will be lower than others in the team as you'll be spending more time on managing the team, but we still want you to demonstrate the sales individual contributor skills to your team. You should aim for 80% team management, 20% IC work, and the quota will reflect that.

To calculate the team quota, we combine the quota of all team members with proration applied if they are still ramping:

- For fully ramped team members we add 100% of their quota to the team quota.
- For team members who begin the quarter still in their first three months in the role we add 50% of their quota to the team quota.

Example: With a flat quota of $250,000 and 3 fully ramped people, and 1 ramping, the team quota would be 875, 000((250,000 * 3) + $125,000)

> If someone leaves the team, we may recalculate the team quota depending on how their accounts and opportunities are reallocated to others in the team. If someone joins the team, we don't change the team target, and don't count their contribution towards the existing target, to keep it simple.

Travel to see customers

You are likely to need to travel a lot more than the typical PostHog team member in order to meet customers. Please make sure that you follow our company travel policy and act in PostHog's best interests. We trust you to do the right thing here and won't pre-approve your travel plans, but we do keep track of what people are spending and the Ops team will follow up with you if it looks like you are wasting money here. We are not a giant company that pays for fancy flights, accommodation, and meals so please be sensible.

Working with engineering teams

We hire Technical AEs. This means you are responsible for dealing with the vast majority of product queries from your customers. However, we still work closely with engineering teams!

Product requests from large customers

Sometimes an existing or potential customer may ask us to fix an issue or build new features. These can vary hugely in size and complexity. A few things to bear in mind:

- Engineers at PostHog talk to customers. It's much better to bring engineers onto calls to speak to a large customer to talk to them directly than just do the call yourself and copy and paste notes back and forth. This is especially useful if a) the team was already considering building the feature at some point, b) it's an interesting new use case, or c) the customer is really unhappy for valid reasons and could churn.
- Provide as much internal context as you can. If a customer sends a one-liner in Slack, don't just copy and paste into a product team's channel - find out as much as you reasonably can first, ask clarifying questions up front etc. Otherwise the relevant team will just ask you to do this anyway.
- We already have principles for how we build for big customers - if you have a big customer with a niche use case that isn't applicable to anyone else, you should assume we won't build for them (don't be mad!)

Finally, if you are bringing engineers onto a call, brief them first - what is the call about, who will be there. And then afterwards, summarize what you talked about. This goes a long way to ensuring sales <> engineering happiness.

Complicated technical questions

You will run into questions that you don't know the answer to from time to time - this is ok! Some principles here:

- Try to solve your own problems. Deep dive the docs, ask PostHog AI, ask the rest of the sales team first - a bit of digging is a valuable opportunity for you to learn.
- Similar to the above, don't just copy and paste questions from Slack with no context. Add some commentary - 'they have asked X, their use case is generally Y, I think the answer might be Z - is that right?'. Do some of the lifting here, rather than putting all the mental load on an engineering team.
- If you open a ticket in Zendesk and know which team the ticket needs to go to, make sure to select "escalated" on the ticket so that it will bypass support and go straight to that team.

Working with customers in Slack

Most of our customers use Slack, and it's a great way for us to be responsive to them. Everyone has the permission in Slack to create a Connect channel with a customer, and you should do this as early as possible in your relationship with them.

When you've created the channel you should also add Pylon, which is used to sync Slack conversations with Zendesk so that our Support and Engineering teams can work on customer issues in a familiar context.

To add Pylon to your customer channel:

1. In the Slack desktop app, click the channel name.
2. On the Settings tab, click Add apps.
3. Type Pylon and click Add.

4. In the popup that appears in the Slack channel, select Customer Channel.
5. Add yourself as the Account Owner.
6. Click Enable.
7. Add Tim, Simon, Charles, and Abigail to the channel.

Once enabled, you can add the emoji to a Slack thread to create a new Ticket in Zendesk. Customers can also do this. Make sure that a Group and Severity are selected or the ticket won't be routed properly.

> It's your job to ensure your customer issues are resolved, make sure you follow up with Support and Engineering if you feel like the issue isn't getting the right level of attention.

Lead routing & scoring

contents/handbook/growth/sales/lead-scoring.md

Lead routing

Generally speaking, companies already using PostHog and spending money will be routed to the product-led sales team. Leads where the customer is earlier in their lifecycle with us, e.g. using PostHog but not spending money, will go to the new business sales team.

> We frequently tweak these rules and experiment with different signals to see which work best. Generally you should be aiming for a 20% conversion rate from these types of leads.

They follow the normal territory assignment rules in Salesforce, and are routed either to Technical Account Executives or Technical Account Managers depending on the type.

Product-led sales team

1. Customers with MRR between $500-1,667, employee count > 50, user count > 7, based in ICP country, and has been paying for at least 3 months
2. Customers who have high ICP score and subscribe to the Scale plan
3. Customers with MRR >$1K and >50% forecasted spend increase this month
4. Unmanaged customers with >$20K ARR who raise a support ticket

New business sales team

1. Completed the book a demo form (organic inbound, paid ads campaign, or outbound)
2. Onboarding specialist referral
3. First signup from a company with 500+ employees who have ingested at least 1 event and invited at least 1 person
4. Customers who have used 50% or more of their startup credits and had a last invoice greater than $5000
5. Customers set to roll off startup plan in the next ~100 days with last invoice between 2*k*–5k
6. Customers who are set to roll off the startup plan in the next two months and had a last invoice greater than $1500
7. AE named lists

Ben experiments to find more winners:

1. Emailed sales@

BDR team

Campaigns are all tracked in Lemlist - these change week-to-week.

Manual (Lorena):

1. Filled contact sales but then went silent, never talked to an AE
2. Tried PostHog but did not convert - signed up but went inactive, never paid, never talked to an AE (in PostHog Data Warehouse)
3. Engineers + Engineering Managers who follow us on LinkedIn but are not customers
4. Event attendees

Not started yet:

1. Closed lost opportunities (new biz *and* renewals) 5+ months old where reason was 'unresponsive'
2. Churned accounts that churned 5+ months ago
3. Companies with recent fundraising activity - good opportunities, but very noisy

Automated (Abhischek):

1. Warmbound - $100-499 MRR at some point in the account's history
2. Job switchers
3. High spenders in Stripe network with <$500 PostHog MRR that doesn't trigger an TAE/TAM lead
4. (Coming soon) Requests for Trust Center access that require an NDA

Anyone at PostHog can also manually flag an account as a high potential lead. This includes new or low spend accounts with strong net new potential or existing paying customers with credible expansion potential. To create a lead, go to the customer's Vitally record and add a Segment for `AM referral` (product-led sales) or `AE referral` (new business).

Demo booking

Customers that want to book a demo and show strong ICP fit signals are automatically get shown a booking link for a demo with a TAE. Those <20 are for the TAE to manually review and schedule. Default is our contact form submission routing system for managing this.

- We have an AI qualifier step to classify submissions as sales/support/spam. If 'support' or 'spam', it'll skip round robin - 'support' will auto create Zendesk tickets, 'spam' are dropped.
- Accounts also have to match 2/3 requirements for revenue, title, and/or industry to see the instant scheduler.
- If an account has had a lead disqualified within the last month, we no longer show the scheduler.

Lead scoring

We calculate lead scores in Salesforce to help us prioritize our inbound book of business. Put simply, the higher the score the higher value a potential contract with a customer **should** be. We use Clearbit to enhance our contact information as it is created and then compute a score out of 70 in Salesforce based on the following parameters:

- *Employee count* - larger companies are more likely to have a bigger customer base and more usage data to capture. They are also more likely to need an Enterprise plan.
- *Ability to pay* - indicates whether a company is likely to pay for a product like PostHog to solve their problems. This is computed from the estimated company revenue.
- *Role* - from experience we sell best to people in an engineering, product or leadership role.
- *Country* - from experience we know that certain countries have a higher inclination to pay for software so we weight those.

> Note that we also calculate an ICP score in Salesforce. This is more marketing aligned and designed to show us whether we are capturing who we are building for as inbound leads.

Metric	Value	Score
Employee Count	1-10	0
	11-1000	10
	1000+	20
Ability to pay	Estimated Revenue 0*m*–1m	0
	Estimated Revenue 1*m*–10m	5
	Estimated Revenue 10*m*–100m	10
	Estimated Revenue $100m+	20
Role	engineering	10
	product	10
	leadership/founder	10
	marketing	5
	other	0
Sub-role	data_science_engineer	10
	project_engineer	10
	software_engineer	10
	web_engineer	10
	founder/ceo	10
	other	0

Metric	Value	Score
Country	Austria, Canada, France, Germany, Japan, Norway, Sweden, UK, USA	10
	Australia, Belgium, Estonia, Finland, Georgia, Guernsey, Netherlands, New Zealand, Poland, Portugal, Singapore	5
	Other	0

New starter onboarding

`contents/handbook/growth/sales/new-hire-onboarding.md`

Your first few weeks

Welcome to the PostHog Sales team! We only hire about 1 in 400 applicants, so you've done well to make it here! Unlike a lot of companies, we don't have a super-long onboarding process and would prefer you to be up and running with your customer base as quickly as possible. Here are the things you should focus on in your first few weeks at PostHog to help you achieve that.

Ramping up is mostly self serve - we won't sit you down in a room for training for 2 weeks. If you're not sure who is supposed to make something below happen, the person responsible is almost certainly you!

How to fail

But first...

Sales at PostHog isn't like most other software companies! These are some of the things that you *shouldn't* do:

- Wait til you're ready to talk to customers. Jump in sooner than you feel comfortable - it is by far the fastest way to learn. A great, low-risk way to practise is chat to inbound leads who you think aren't going to be high paying customers - the cost of doing badly is very low!
- Lazily forward customer questions to engineering teams without any context. That's super annoying. Instead:
    - Try to solve the problem yourself, read the Docs etc.
    - Try asking in #ask-max Slack channel
    - Ask the rest of the Sales team
    - *Then* forward to the relevant engineering team *and add context* - are they a huge oppo, evaluating or already paying, technical/non-technical etc.? Help them help you
        - Bonus points for: 'I think [this] is the answer, am I on the right track?'
- Execute your previous company's playbook. We're trying to do things differently from 90% of the industry. But please do tell us what has *and* hasn't worked at previous places.
- Keep information to yourself - share openly and frequently the things you are learning, what you've got right or wrong. We don't do lone wolfing here. PostHog is a huge product, so it's ok to ask dumb questions - so long as you've tried to figure it out yourself first!
- Use sales BS language - if you don't know the answer that's fine! Don't promise features. Don't use vague, non-specific language. Talk to our customers like real human beings, not 'prospects'. And don't be discouraged if they say 'can we talk to someone more technical'!
- Being slow to reply to customers - even if it's just to acknowledge their message. Make sure you have notifications turned on for *all* messages in your customer Slack channels (not just for mentions).

Technical Account Executive ramp

Day 1

- Meet with Ben who will run through this plan and answer any questions you may have. In addition, come equipped to talk about any nuances around how you prefer to work (e.g. schedules, family time etc.).
- Setup relevant Sales & CS Tools
- If you start on a Monday, join your first PostHog All Hands (at 4.30pm UK/8.30am PT) and be prepared to have a strong opinion on whether pineapple belongs on pizza.
- If you start on a Monday, join your first Sales standup.
    - We fill in a GitHub issue every week before this meeting so we are prepared for the discussion topics. Simon will add your GitHub handle to the template.

Rest of week 1

- Ask team members in your region to be invited to some customer calls so you can gain an understanding of how we work with customers.
- Check out some Buildbetter calls and add yourself to a bunch of Slack channels - get immersed in what our customers are saying.
- Learn and practise a demo of PostHog.
- Read all of the Sales section in the Handbook, and update it as you learn more.
- Meet with Charles, the exec responsible for Sales.
- Meet with Simon, Sales Lead
- PostHog integration exercise - by the end of week 1:
    - Find/build a blank app which doesn't yet have PostHog integrated. You should be able to vibe code something simple with React using Cursor or Lovable.dev
    - Once you've got your app up and running get PostHog deployed and capturing events and replays. Default config is fine.
    - Implement a custom event
    - Implement user identification
    - Record a loom showing what you've done and share it in our team channel

Week 2

- During your first week, Ben will go through the sales process with you and answer any questions you may have about the playbook.
- Shadow more live calls and listen to more Buildbetter recordings
- Towards the end of the week, schedule a demo and feedback session with Ben. We might need to do a couple of iterations over the next few weeks as you take on board feedback, don't worry if that's the case!
- Get comfortable with the PostHog Docs around our main products.

- We'll start routing new Salesforce Leads to you at the end of week 1. Start to review these and reach out, using a shared booking link with someone else from your region so they can back you up in the first few weeks. This is a great option to practise and fail.
- Make sure you're comfortable with the Shared Processes section of the Handbook
- Once you start getting leads / accounts, ping Simon to be added to the TAM quota tracker.

In-person onboarding

Ideally, this will happen in Week 3 or 4, and will be with a few existing team members (depending on where we do it) and will be 3-4 days covering:

- Demo practice session with the team.
- The data we track on customers in PostHog and some hands-on exercises to get you comfortable using PostHog itself.
- Deep dive on Vitally tracking.
- No stupid questions session.

Weeks 3-4

- Focus on taking more and more ownership on calls so that team members are just there as a safety net.
- Continue to meet with customers and reaching out to new leads.
- Add your Calendly link to your Slack profile so customers can book in with you directly (this works!)

How do I know if I'm on track?

By the end of month 1:

- Be leading customer calls and demos on your own
- Have evaluations in flight (with support from the team if needed)
- Have closed your first prepaid credit deal of any size

By the end of month 2:

- Be leading evaluations on your own
- Seeing strong conversion from your outreach to new leads
- Have closed multiple contracts by this point through the whole process

By the end of month 3:

- You've built out a strong pipeline and plan, looking 1-2 quarters ahead
- On track to hit 100% quota by the end of month 6

Technical Account Manager ramp

Day 1

- Meet with Landon who will run through this plan and answer any questions you may have. In addition, come equipped to talk about any nuances around how you prefer to work (e.g. schedules, family time etc.).
- Setup relevant Sales & CS Tools
- If you start on a Monday, join your first PostHog All Hands (at 4.30pm UK/8.30am PT) and be prepared to have a strong opinion on whether pineapple belongs on pizza.
- If you start on a Monday, join your first Sales standup.
    - We fill in a GitHub issue every week before this meeting so we are prepared for the discussion topics. Simon will add your GitHub handle to the template.

Rest of week 1

- Ask team members in your region to be invited to some customer calls so you can gain an understanding of how we work with customers.
- Check out some Buildbetter calls and add yourself to a bunch of Slack channels - get immersed in what our customers are saying.
- Learn and practise a demo of PostHog.
- Read all of the Sales section in the Handbook, and update it as you learn more.
- Meet with Charles, the exec responsible for Sales.
- Meet with Simon, Sales Lead
- PostHog integration exercise - by the end of week 1:
    - Find/build a blank app which doesn't yet have PostHog integrated. You should be able to vibe code something simple with React using Cursor or Lovable.dev
    - Once you've got your app up and running get PostHog deployed and capturing events and replays. Default config is fine.
    - Implement a custom event
    - Implement user identification
    - Record a loom showing what you've done and share it in our team channel

Week 2

- During your first week, Simon will figure out your initial book of business (10 accounts). We will review these at the start of your second week, and make sure you understand how your targets are set.
- Shadow more live calls and listen to more Buildbetter recordings.
- Towards the end of the week, schedule a demo and feedback session with Landon. We might need to do a couple of iterations over the next few weeks as you take on board feedback, don't worry if that's the case!
- Prioritize your current book of customers, and start reaching out!
- Get comfortable with the PostHog Docs around our main products.

- We'll start routing new Salesforce Leads to you at the end of week 1. Start to review these and reach out, using a shared booking link with someone else from your region so they can back you up in the first few weeks. This is a great option to practise and fail.

In-person onboarding

Ideally, this will happen in Week 3 or 4, and will be with a few existing team members (depending on where we do it) and will be 3-4 days covering:

- Demo practice session with the team.
- The data we track on customers in PostHog and some hands-on exercises to get you comfortable using PostHog itself.
- Deep dive on Vitally tracking.
- No stupid questions session.

Weeks 3-4

- Focus on taking more and more ownership on calls so that team members are just there as a safety net.
- Continue to meet with your book of customers and inbound leads.
- Add your Calendly link to your Slack profile so customers can book in with you directly (this works!)

How do I know if I'm on track?

By the end of month 1:

- Be leading customer calls and demos on your own
- Have evaluations in flight (with support from the team if needed)
- Successfully made contact with *everyone* in your book of business

By the end of month 2:

- Have closed your first prepaid credit deal of any size (net new or conversion to)
- Be leading evaluations on your own
- Have identified some opportunities to add to your book from self-serve signups who aren't paying yet

By the end of month 3:

- Have closed multiple contracts by this point (either new or expansion/renewal) through the whole process
- Be driving multiple opportunities for cross sell through your accounts, ie. at least one customer is actively using a product they weren't before you started (at any scale)

By the end of month 4:

- On track to hit 100% quota by the end of month 6

Getting your equipment setup right

In addition to following the guidance in the spending money section of the Handbook, there are a few things you should do to make sure you're set up to give high quality demos that look professional:

- Buy a webcam. These are cheap, but significantly better than those built into your Macbook. Logitech is perfectly fine here. This should cost up to $100.
- Buy a microphone. The mic on your Airpods will not do it. Rode do good quality, affordable mics - nice thing about these is that you can plug in wired headphones so you can still hear yourself talk. Again, up to $100 should be fine.
    - If you prefer to use headphones with a built-in boom mic instead, that's also ok.
- Take calls at your desk. Don't dial in from your garden because it's a nice day and then spend the call apologizing for the lousy WiFi.
- You don't have to go all ring light, but think about your background a bit. You don't have to construct some elaborate bookshelf situation behind you, but consider using one of our nice wallpapers.

Alerting setup (for team leads)

We have certain automations in Vitally that your team lead needs to add you to. Please ask your team lead to add you.

- Vitally name trait playbook : create a new branch that matches assigned AE to new team member. In this branch, add action to update account trait `AE name` to name of the new team member. This is used to populate account owner info in tickets created by customers we own, so support knows who to reach out to.

New hire frequently asked questions

How does my quota work during my ramp period?

Your first three months of commission are paid at 100% fixed OTE. This will be calculated based on the date you start. If you start before the 15th of a month, you will get 100% fixed OTE for that month and two of the subsequent months. For example, if you start on Jan 13th, you will get 100% fixed OTE for Jan, Feb & Mar. If you start *on* Jan 17th, you would get two months of 100% fixed OTE for Q1 and one month of 100% fixed OTE for Q2 in addition to two months of your quota'ed commission.

How does support work at PostHog?

- Generally, you're expected to be able to be the first line of support for customers at PostHog. You should be able to answer *most* yourself - that's why we hire *Technical* AEs and AMs after all! #ask-max in Slack can often help too.
- If you can't solve a customer's problem (it happens) then follow our standard support process.

Can I login as a customer?

- Visit the /admin/ endpoint on the cloud they are on. You can then search for them via email and log in. Be careful clicking around here as you can accidentally delete a person/organization! You need to get their permission first unless it's an emergency, i.e. to resolve an incident.

Are there any influential folks in our space I should read/listen to?

- Join the `#newsletters` channel in Slack for updates from a curated collection of influential folks in our industry.

New business sales

contents/handbook/growth/sales/new-sales.md

We build PostHog for product engineers. While many non-technical folks use PostHog successfully every day, our sales process is built with technical folks in mind. Once implemented, a customer may use PostHog for all manner of things (and we hope they do!).

Three other general principles to bear in mind:

- Aim to **drive the sales process** as much as possible. Most of our customers in our core segment don't regularly buy software like PostHog, so you will need to guide them through the process - for fast growing startups, this may be their first time. Even for much larger customers, this still may be the case. Remember that we've sold software hundreds of times - it's ok to guide the customer here. Inbound leads =/= the customer wants to drive.
- **Discovery is an ongoing process** - you want to be finding out useful information at each stage, not just going through a checklist at the beginning. Asking 'why' can be a really powerful tool, and you should be curious throughout, not just selling.
- At the end of each conversation with a customer, you should aim to **be really specific and prescriptive about next steps**. If you haven't identified a next step, the opportunity is Closed - Lost. Waiting for a customer to come back to you is not a valid next step - instead you should be saying things like "at this point, usually the next best step is for us to do X".

Maximizing your chance of success

Selling software, especially to larger companies, can be a complex process with lots of stakeholders involved. When moving your deal along you should aim to know as much about the following as possible given where you are in the process (inspired by MEDDPICC):

- Pain - Do they have a problem?
  - Is it painful enough right now that they are willing to adopt a new solution to solve it?
  - Will PostHog solve this problem for them?
- Timeline - When do they want to have a contract signed/solution in place?
- Pricing - We should know the rough size of the opportunity, and that that is in line with expectations and budget of the customer.
- Champion - Are you working with a champion who is going to sell internally?
  - Are they the buyer (see below) or do they know who the buyer is?
- Decision - How are they evaluating us?
  - Is it competitive? Who?
  - What is their criteria for success?
- Economic Buyer - Sometimes your Champion has to convince upper management to spend money.
  - Is that person aware of PostHog and on board with signing a contract if an evaluation is successful?

- Paper process - After a successful evaluation what happens next?
  - Do they need a Custom DPA/BAA/MSA?
  - Is there a security review needed?
  - Who signs the Order Form?

These are presented in the most likely order that you will be able to discover them, although that is not a hard and fast rule.

They are also available as Opportunity fields in Salesforce and as such you should keep them up to date when you learn more.

> Always follow the lead to opportunity conversion guidelines when creating opportunities in Salesforce

Sales process

This is an overview for what you should actually be doing with a customer at each stage of the sales process. For details on how to manage this in our CRM, visit our Salesforce docs. The steps are:

1. You get a lead
2. You qualify
3. First call (30 minutes) - Discovery & initial demo
4. Second call (60 minutes) - Technical deep dive (if needed)
5. Product evaluation
6. Security & legal review (only if asked - skip otherwise)
7. Commercial evaluation
8. Closed - Won or Lost

1. You get a lead

We're constantly experimenting with the best lead types, documented here. Info on *how* leads are assigned can be found here.

2. You qualify

Once you have been assigned a lead, you'll want to qualify them before scheduling a call. Things to consider:

- What is their Lead Score?
- Which products are they looking to use? What's their use case?
- How large is their company? Revenue? Have they raised funding? (ie. will they pay >$20k for PostHog?)
- What is the role of the person who filled out the form?
- Are engineers already involved in the sales process, or will they need to be brought in?

Most companies add friction here by making customers jump on a call first to qualify them. We don't do this when we are confident that product engineers are, or will become, involved in the sale. We may ask for a 2nd call with the engineers involved if we're confident that PostHog can help and want to make sure they agree.

It's also totally fine to ask a customer questions over email in advance of the demo to make sure you're making the best use of their time - just be specific. A few clarifying questions is fine, a 30 question survey is not.

> **Examples of good discovery questions**
>
> - What is the problem? What is this problem affecting?
> - What metric is impacted as a result of this? What metric would be improved as a result of PostHog?
> - How important is this problem to the wider team?
> - What attempts have been made to fix this so far? Why has no attempt been made to fix this?
> - Why fix this now? Why fix this ahead of the other important things happening at your company?
> - Are you looking at {product} as a point solution that would slot into the rest of your stack, or are you looking to consolidate multiple tools and have a single source of truth?
> - What does the rest of your stack look like? What other tools or data would you want PostHog data to connect to?
> - Who owns that data stack? Do you have a data team or data engineers?
> - Who will be the consumers of PostHog data? How are they currently answering their questions, and how easy is it for them to do so with existing tooling?

If you're pretty sure that they should be qualified out of our sales process, you should still be helpful over email - some customers just use the form to get in touch and don't want to actually have a demo (e.g. they have a billing question or are asking about compliance things like HIPAA.) There is a Claude skill that can help draft genuinely helpful responses to folks like these - it will put together helpful resources from our docs and blog posts that can help customers get started, even if they don't qualify for a call with a salesperson.

Requests for Proposals (RFPs)

There are two types of RFPs:

- We have context on what they're trying to accomplish and where we have qualified their specific needs ahead of time. These are *solicited* RFPs, and we generally reply to these.
- We just get an RFP randomly without any context. These are *unsolicited* RFPs, and we generally don't reply to these.

If it's an unsolicited RFP where we haven't had any prior contact or usage from the company then it is **highly likely** that you will burn a lot of time for nothing and you are free to decline. If you find the unsolicited RFP otherwise compelling and want to proceed, the suggested approach here is to see if anyone from the company has recently signed up to PostHog. If so, then make contact with them to see if they are aware of the RFP and can provide more information on PostHog's inclusion.

If you can't identify anyone who has recently signed up to PostHog, then ask the person who sent you the RFP for a call to gather more context before making a decision on whether to fill it in. If they aren't willing to get on a call then it's likely that we are not their vendor of choice, and they are using us to make up the numbers in a tender process. As such, we shouldn't spend time on this kind of activity. If you choose to spend time with these, timebox your effort to ensure you are not devoting a week to a 500 question RFP where we have very slim chances of success. Your time is your most valuable asset.

If it's a solicited RFP, you're free to proceed so long as the opportunity is qualified as a whole and you carefully balance the level of effort required in the RFP against the opportunity for you & PostHog. Again, a 500 question RFP may not be worth it if they plan on spending <$20k for PostHog (a 50 question RFP may not even be worth it in this instance)! Use your best judgement, and it is generally still wise to timebox your effort.

Bigger Opportunities at Bigger Companies

When you're working a deal north of $100k at a larger company, the playbook shifts. Generally, expect to challenge their stated evaluation criteria early, as well as sell to multiple people and functions within the organization. You need to dig past the surface-level requirements they may list and get to the real decision drivers. Question the "why" and "how" behind their stated criteria, because committee-driven procurement processes can hide the actual priorities behind "just so" rubrics that can obscure the real reasons they will buy (or disqualify).

On the relationship side, you need a strategy for engaging their leadership and developing champions at multiple levels within the account. If a key leadership stakeholder goes dark, escalate to PostHog leadership to help re-engage. If needed, don't be afraid to translate PostHog titles to something they would understand (e.g. Generic Exec Person = COO). For deals this size, on-site presence can also matter — you should attempt to build relationships in person, not just over Zoom.

Lastly, take a prescriptive and consultative approach to their evaluation process. The larger the opportunity, the more proactive you need to be about controlling the process. Ask for help from your lead, your team, and in Slack. These opportunities take a team effort.

Startups

If they're eligible for the Startup Plan, route them to the application form and disqualify them as it's not an immediate opportunity (but we sincerely hope they grow into loyal PostHog customers). If their usage will burn through their credits quickly, you should feel free to switch their lead status to `Nurture` and keep close tabs on them. Per our usual approach to sales, we want to make sure they're successful in this "high-use" scenario and are building with us for the long-term.

You can also redirect them to use the In-app support modal if they have a product-related question - this will then be routed to the right team, as well as showing them CTAs to upgrade for high priority support.

Leads below the sales assist threshold (less than $20K ARR)

We often get requests for demos from leads or existing customers who are below our sales assist threshold, and who don't have a defined use case for PostHog. It usually comes in the form of "show me all the features" or "I need someone to demo to me." These can be large time sinks because they are non-technical, don't have a clear idea of what they want, and are unlikely to ever grow into a sales-assist level customer.

We also want to be helpful to our current or potential customers, regardless of spend. Time permitting, we can offer a demo if they are willing to give us the information we need to put something together:

- What tech stack are you on?
- What features / products are you interested in?
- What questions do you have?

This makes the demo actually valuable and can be an opportunity for you to learn more and get some demo practice. You'll also find that 90% of these requesters never respond because they are either unable or unwilling to engage with the questions, which allows you to avoid the biggest time sinks.

If you realize that they will be too small (<$20k) to go through our sales-led process and you are unable to get this information from them, you should route to self-serve.

3. First call (30 minutes) - Discovery & initial demo

Your goals on this call depend on who shows up. You should know who's coming ahead of time and be prepared to change your approach based on the actual attendees.

The ideal outcome is getting engineers to be hands-on with PostHog as quickly as possible.

Path A: Engineers are present on the first call

When you have engineers on the call from a qualified company (ICP fit or otherwise highly qualified), your goal is to get them using PostHog immediately.

Structure:

1. **Intro & Qualification** (5-10 min)
    - Friendly banter
    - Focused discovery on their use case
    - Articulate PostHog's vision and how it relates to their needs
    - Show PostHog in a technical demo as soon as possible
2. **Technical Demo** (15 min)

- Highly tailored to their use case
- Light on pitching, heavy on showing docs and GitHub
- Use our Demo Project linked to Hogflix
- Start with their biggest problem first, stay there until they're happy

3. **Call Close** (5-10 min)

    - **Part 1:** Confirm they agree PostHog solves their problem; scope what success looks like for their trial; answer questions, particularly around pricing: Secure BANT
    - **Part 2:** Ask for trial signup or PostHog org conversion; confirm next steps on the call; ensure that any "give" by PostHog receives a "get" from the customer (typically feedback on the trial at this phase)
    - Answer questions and objections

Success looks like:

- They commit to using PostHog in a reasonable timeframe
- You have a plan to get their feedback on the product as soon as they use it

Path B: No/minimal engineers present (non-technical stakeholders)

When engineers aren't on the call, your goal is to earn a second call with their engineering team, while also being helpful to the non-technical stakeholders in discussing PostHog.

Structure:

1. **Intro** (5 min)

    - Friendly banter
    - Get BANT info upfront (Budget, Authority, Need, Timeline)
    - Articulate PostHog's vision
    - Scope their use case

2. **Qualify or Disqualify** (10 min) - we do this politely and constructively. The customer's time is valuable and we know best who succeeds with PostHog, so we're driving the sale.

    - **Qualify if:** They have technical capability to succeed, engineers will be involved in the sales process, and there's product/solution fit
    - **Disqualify if:** They're non-technical with no engineer involvement, or there's no product/solution fit

3. **Demo** (10 min)

    - **If qualifying:** Show enough to validate required functionality and earn the next call with engineers
    - **If disqualifying:** Show what's needed to validate the lack of fit

4. **Call Close** (5 min)
    - **If qualifying:** Schedule the next call, ensure engineer attendance, set initial scope for technical demo
    - **If disqualifying:** Direct them to better resources, politely thank them for their time, ask them to reach back out when engineers are involved

Success looks like:

- **If qualifying:** You have a call with their engineers on the calendar (Step 5 below) and they understand why it will be helpful
- **If disqualifying:** They understand why PostHog isn't a fit right now and appreciate your helpful transparency

> **Important:** If you can't get a second call scheduled, be skeptical of the opportunity. Keep the task in nurture status until it's on the calendar - only convert to an opportunity after the call is confirmed.

General demo tips

We have various slide templates - ask someone on the Sales team for an invite to our Pitch account. Use the deck as scaffolding, pulling out relevant slides. **Do not spend the demo presenting a deck with an engineering team** - most people at PostHog spend 90% of the demo call actually in product or talking to the customer about their needs. But sometimes, there is a legitimate need for a deck.

Before you demo, make sure there is enough data to properly showcase our features. If needed, you can use Hogbot to generate more synthetic data. This is built by the sales team for the sales team, so if you see anything you want to improve, don't hesitate to submit a PR!

You should give a relevant and pointed demo - don't just throw everything in, as the customer will get overwhelmed. If you don't show what's important first, people on the call will become distracted.

For example, a customer may say "we need to see how our customers are using our platform". In this case, a good approach is to go straight to Session Replay, then tie Replay into Analytics, then go from there.

> Start with what their biggest problem/request is, stay there until they are happy, then move on to point two. We don't want to fall into the trap of doing the same demo for each customer regardless of what they say at the beginning.

Make sure you cover:

- Who is there and what their roles are - in particular, are they the decision-makers?
- Why do they *need* PostHog?
- Demo specific product features according to what they asked for

    - Consider complementary products, e.g. analytics + replay; analytics + warehouse; surveys + flags
    - Don't promise things we can't/don't want to deliver
- How pricing works with an indication of their potential spend if you have enough information
- Next steps - this is really important, don't just end the demo with no clear follow-up action

4. Second call - Path B (60 minutes) - Technical deep dive

This call happens when engineers weren't on the first call. Your goal is to qualify the opportunity through the engineers and get them hands-on with PostHog.

Structure:

1. **Intro** (5 min)
    - Friendly banter
    - Confirm everyone's roles and responsibilities
    - Set context from the previous call
    - Reiterate PostHog's vision
2. **Discovery** (15 min)
    - Confirm use case(s) relative to the engineer's understanding
    - Dig into the engineer's role in shipping and their workflow
    - Confirm BANT, particularly timeline as it relates to the PostHog implementation
    - Understand their technical stack and how PostHog fits in
3. **Technical Demo** (30 min)
    - Given the likely mixed audience, you can take a broader view of PostHog and how it supports technical and non-technical users alike.
    - Even so, cater to the engineer's role in the project and the power of PostHog for product/engineering teams
    - Show relevant documentation and GitHub integrations
    - Check for engagement from the buyer persona - note any disengagement
    - Start with what their biggest problem is, stay there until they're happy, then move on
4. **Call Close** (10 min)
    - Ask for trial signup or PostHog org conversion - be specific in asking for clarity here.
    - Ensure that any "give" by PostHog receives a "get" from the customer
    - Answer questions and objections as they arise, particularly around pricing.
    - Be specific about next steps

Success looks like:

- You've met the engineers and understand their role

- You've qualified the customer's use case and involvement in the project
- You know when the engineers hands will be on keyboards trying PostHog
- You know how/when the opportunity will convert

BANT

By the end of either the 1st or 2nd call with a customer, you should have a defined idea about:

1. **Budget** - Calculate and share a rough ballpark figure based on which products they'll use and their expected usage. Articulate the process by which a sales-led trial will help them refine the estimate.
2. **Need** - Is PostHog a good fit? Be politely honest if we're not, to avoid wasting everyone's time.
3. **Authority** - Who will make the decision at the customer organization? Who holds the budget?
4. **Timeline** - When does the trial start? When are they looking to make a decision/have a contract in place?

> It's really easy to convince yourself that you've got a well-qualified opportunity after a demo goes well. Everybody has been laughing and having fun so they must love PostHog right? You need to be more objective than that - ask the AI in the call recording to rate you on BANT qualification to see whether you actually got all of the information you need to confirm that a real opportunity exists here. If you are missing any qualification information, don't be afraid to go back and ask your champion for additional context here. It'll save you wasting a whole bunch of time helping a customer in an evaluation where they aren't serious about buying PostHog, and the inevitable **Closed - Lost** which comes as a result of that.

5. Product evaluation

Once qualified, and if you think they are a good prospect for our sales-led process, your first priority is to try and get them into trial of PostHog with a shared Slack channel as quickly as possible. If you close them, a shared Slack channel will also be their primary channel for support. Add the Pylon app to the channel and it will automate the support bot and channel description. React with a to customer messages or tag `@support` to create a ticket in a thread. Generally it's better to seek forgiveness than ask permission for adding people to a Slack channel - use your judgement.

Some customers may wish to use MS Teams rather than Slack - we can sync our Slack with Teams via Pylon to do this. First you will need an MS Teams licence - ask Simon for one. Then, set up a Slack channel. Then, follow the instructions here to get set up. Before adding the customer into the channel, remember to test it on both sides to ensure the integration is working correctly.

You should then follow up with a standard email/Slack message that:

- Summarizes what they're after and how Posthog is a great/bad fit
- Lays out next steps on both sides
- Shares a proposed timetable for the evaluation and onboarding process
- Includes any useful links (e.g. Docs page, competitor comparisons, relevant case studies)

Probably as a separate message, you should set out the criteria for the product evaluation to be considered a success - the evaluation will almost certainly fail if you just leave the customer to noodle around trying PostHog.

If the customer isn't super clear on how to articulate the success criteria then use the following as inspiration:

- Product/web analytics + session replay: get tracking set up, turn on replay, privacy controls, figure out user ID, get set up insights/dashboards.
- Feature flags + experiments: snippet, FF in the code, person ID and properties for targeting, deploy flag, run the experiment.
- Surveys: deploy a survey, view and analyze the results
- Data warehouse: set up the warehouse, sync at least 1 data source or pull additional person data in to enrich an insight

> Don't be over-reliant on support during the evaluation. As the AE, you should be highly focused on customers during their evaluation to maximize your chance of success. We deliberately hire people we know customers will love working with, so now is your time to shine.

1. Guide them on how to set up tracking depending on their app paying attention to common points of friction such as:
    - Anonymous vs Identified events.
    - Tracking pageviews in single page apps.
    - Deploying a reverse proxy.
2. Guide them on creating insights either based on:
    - Metrics they've shared that they need to see or;
    - Things we know companies often want to track (e.g. the AARRR framework).
3. Once you have a week's worth of data in, calculate pricing based on their actual usage and proactively share this.
4. A week before the trial period ends have a wrap-up call to ensure that they have seen everything they need to see, and identify any last remaining areas you can help them with, and next steps after the trial ends.

In an ideal world this involves multiple calls per week during the trial period so that you can build a trusted relationship with the customer, but don't force that if they prefer to use Slack/Email.

> If non-technical people such as Product Managers, Marketing, etc. are involved we know from prior experience that the PostHog UI, while powerful, can be overwhelming, especially if they have used similar tools in the past. You should be prepared to run multiple remote or in-person sessions with these people to ensure that they get what they need out of the evaluation.

We usually set up the following trials depending on likely contract size:

- $20-60k - 2 weeks
- $60k+ - 4 weeks

5. Security & legal review

Most customers don't need this beyond sharing our existing documentation. This step often occurs in parallel with product evaluation. Usually only bigger companies ask for this.

> You do not need an NDA to share PostHog internal policies - by default most of these should be publicly available in the Handbook anyway, though some are only stored in Drata. If a customer asks you to sign their NDA, you can sign, but have our counsel review it first. As a starting point it must be governed by US law, and mutual.

If the customer requires a vendor questionnaire or security questionnaire then it's best for the AE involved to try and fill it out. If a company reaches out initially with this request, it's often best to try and understand if the customer has an intention to pay or at least grow into a paying customer before investing a lot of time filling it out. If there are any questions that are unclear post the specific question in #team-people-and-ops channel. It is easy to get driven into filling out security questionnaires for accounts that would come in below the sales assist threshold. If the lead is pushing security review without having had any commercial discussions, be transparent up front and let them know that we only do security review for accounts at $20k annual spend or greater. We are happy to work with them to understand their usage, and at that point, further entertain security discussions or point them towards a self serve path.

Some customers may need payment details up front as part of their vendor onboarding process. Stripe allows you to generate these ahead of them signing the contract — you can see how to do it in the billing guide for applying credits.

If you need help with anything data privacy or MSA-related, ping Fraser for help.

6. Commercial evaluation

The Contracts page has full guidance on the nuts and bolts of how to put together a commercial proposal - we use PandaDoc.

Don't be the AE who gets to this point and suddenly realizes you have no idea who the buyer is! You should already know this, their budget, their purchasing process etc. already as part of your discovery - if you're finding out now, hopefully it's not too late...

By this point, you may have run into some additional objections. These are the most common, and how to handle:

- Gap in the product - introduce the customer to the relevant product engineer to build together (but first agree with product team if it's a reasonable ask). We have found this approach works exceptionally well for our newer products.
- Pricing issue - understand their budget; our discounts section had the different levers you can pull to get a customer to the right price point. You can also help them tune their usage to lower costs. We don't buy customers out of existing contracts, and we don't do deals where year 1 is super cheap then we ratchet up the price in year 2.

- Performance (e.g. slow dashboards) - for very large customers, usually get Tim involved, or he can loop in the right engineer to help.
- Confidence in PostHog - often this Handbook page is enough. For Very Large companies who need to be sold a bit more on the company vision, you can get James H involved.
- Unsure how much credit they need - suggesting the customer pay monthly for one or two months can help here, especially when there is not a technical driver that can do the mental math to figure out volumes. It's also a good expectation to set at the end of the trial that they will roll onto monthly, which can be pitched as a way to de-risk for the customer if there are still loose ends or a deal is dragging.

> Ahead of the contract being signed, you'll also need to understand the customer's invoicing process. Companies will typically have a Finance or AP team who should be the billing contact in Stripe. Make sure you are also aware of any special invoicing requirements (e.g. a Purchase Order number) well ahead of the invoice being generated. Follow our contract rules here - e.g. no payment by check, ever.

7. Closed - won

Hooray! This is defined as when the contract is signed by *everyone*. 'They're about to sign' - NOT CLOSED. 'I've sent a DocuSign' - NOT CLOSED EITHER.

If an opp moves forward with PostHog on a month-to-month basis, but is below $20k annual spend, change the type to "Monthly Contract" and mark it as closed - won in Salesforce.

Once the contract is signed, it lives in PandaDoc. Next step - get them set up with billing.

Now it's time to set up an onboarding plan. We will templatize this, but for now you should send them something in the first week that includes:

- How to manage billing/credits
- Set up regular calls/checkins
    - $60k+ - every 1-2 weeks for the first 6 months, then every month
    - $20-60k - every month for the first 3 months, then quarterly
- Schedule training for the champion and/or additional people as needed - the more people you get successfully using PostHog, the more likely they are to retain

Here is minimum checklist of things that we find customers should know how to do:

- Actions
- Cohorts
- Data taxonomy
- Notebooks
- Activity log
- Internal and test user filtering

Post-onboarding, you'll want to change gears to start thinking about retention, expansion and/or cross-sell.

Simon and Charles review accounts every month to see if/when it makes sense to reassign accounts once they've closed.

7. Closed - lost

Oh no! It's ok - the most important thing here is that we learn. You should capture the reason in the Salesforce opportunity - this could be:

- Product/feature gap
- Performance concerns
- Security/privacy concerns
- Pricing
- They chose to stick with current setup
- Champion left
- Business restructured/disappeared
- Don’t know (disappeared)

Add detailed comments as well, including what, if anything, we could have done differently (even if not realistic - e.g. build an entirely new product).

For certain categories, you should create followup tasks:

- If they went with a competitor, create a reminder to check in with them in 9 months’ time.
- If it was a feature gap, contact them when that thing is built using the sub-categories above.
- If it was a security/privacy concern, contact them when we get the relevant certification etc.
- If they chose to stick with current, check in again every 6 months.

Share info about closed-lost people internally where it will help us learn - this may be with the sales team, relevant product team, or the company as a whole in Slack. The important thing is not to blame each other for losses, it's to find opportunities to do better next time!

Outbound sales

contents/handbook/growth/sales/outbound-sales.md

Woah woah woah, we're doing outbound?

Yes! But do not be afraid:

- We are not doing this to 'go enterprise' - for now we're trying to reach more of our ICP.
- Our investors did not ask us to do this - we came up with it ourselves.

So why are we doing it now? I thought our inbound pipeline was good?

Outbound sales is a thing we will need to get really good at as we continue to scale PostHog, as 100% inbound eventually dries up. We are not going to be the first company in history to build a huge Saas business with zero outbound, and most companies like us start thinking about outbound around our ARR. Even the largest, most beloved devtool products of all time do this - they just do it in a smart way.

We want to start doing outbound now because, if we wait til inbound slows down, we'll panic and make bad decisions, trash the brand, and copy and paste what other boring companies have done in a short-sighted way that doesn't work for our audience.

Outbound is helpful because it is a good way to generate more leads in a semi-predictable way - and there are lots of cool ways to do it in 2025 using GTM engineering, agents etc. We should view outbound as a type of hyper-focused *marketing* that generates sales opportunities.

Let's get on the same page - what *is* outbound?

'Outbound' means a few different things. This is how we think about it in relation to customers:

1. Using PostHog and spending a lot of money
2. Using PostHog and spending a little money
3. Using PostHog with good engagement/high ICP, but not spending any money
4. Person signed up at some point, but not really using, usually just kicking the tires
5. Not signed up, but has heard of PostHog
6. Not signed up, never heard of PostHog

> None of these people are currently talking to us - that's why they are under the umbrella of 'outbound'.

We'll call 1-3 'warm' outbound and 4-6 'cold' outbound.

What we're doing today

Our model is:

- TAMs do warm outbound to group 1-2
- BDRs do warmish outound to groups 3-5
- TAEs do cold outbound to a top 10 list in groups 5-6

Our focus today is on inbound leads, getting much better at warm outbound (we have a huge number of leads that we could be converting better), and experimenting with colder outbound. is leading our experiments today with the .

Check out the leads page for more detail on lead types, where they go, and the specific outbound campaigns we're running. These are changing very frequently as we figure out what does and doesn't work.

How TAEs talk to outbounded prospects

Remember, *we* contacted *them* — be transparent about our process and who we build for. How well we do discovery in our initial conversations will dictate how well (or poorly) we position PostHog.

If they're interested, we'll show them how to try PostHog and help them along the way; if they're not a fit, we'll say so honestly. We need to earn the right for each step and not assume their interest.

So, what does that mean for a first conversation? We:

1. Do research & get context
2. We are human & transparent when we meet them
3. Explore their role & current state/stack
4. Qualify or disqualify
5. With explicit permission, give a brief PostHog pitch
6. Ask the hard question
7. Provide a relevant next step & schedule it on the call
8. Action the task
9. Rinse, lather, and repeat

Goal: help them decide if PostHog solves a real problem, not close in one call.

In order:

1. We do research & get context

Do basic account research:

- Prompt your LLM of choice for facts (especially with MCP access). Ask:
    - What's their tech stack? (Job postings, BuiltWith/Wappalyzer, 1Password)

    - Recent company news (funding, launches, hiring)
    - Their role + tenure (LinkedIn, their website)
- Why did they agree to this meeting? (Read 's notes in the New Business Slack)
- What problem or pain did flag?

Use this to form a call hypothesis.

2. We are human & transparent when we meet them

We contacted *them*. This call only makes sense if we can solve a real problem for them. Start with:

```
*"Hey [name], thanks for making the time. I know this was a cold outreach from
[Dmytro/our team], so I really appreciate you giving me 30 minutes."*

*"Before we dive in, I'm curious – what made you decide to take this call?*
```

Often this is enough. If they're vague or skeptical, get specific with your pre-prepared hypothesis:

```
*"[Dmytro mentioned/I saw] that [specific trigger – e.g. you're growing
team/launching new product/scaling analytics]. We work with companies at your
stage who struggle with [specific pain – e.g. fragmented analytics tools/poor
data quality/lack of actionable insights]. I wanted to understand if that's
actually a problem you're facing."*

*"If it is, I can share how other companies like yours have solved it. If it's
not a problem, I'll tell you honestly (or you can tell me) and we'll keep this
short. Will that work for you?"*
```

If they answer clearly, set a simple agenda:

```
*"Got it. Here's what I was thinking for today: I'd love to understand how
you're handling [your role/the use case behind the trigger] now, what's working
and what's not, and then share how other companies like yours have approached
it. If it seems relevant, we can go deeper. If not, I'll tell you honestly and
we'll keep this short. Sound fair?"*
```

3. Explore their role & current state/stack - find the pain

As says, companies don't buy software; humans do. Start with their role/team.

```
*"Tell me more about your role and team."*
```

Then move to the trigger/use-case:

```
*“How are you thinking about [use-case/trigger] in that role/team? What do you need to understand about your product/users/customers?"*
```

Other prompts:

```
*"What are you using for [use-case/trigger] today? And how'd you end up with that setup?”*

*"What do you love about it? What drives you crazy?"*
```

You’re digging for pain, urgency, and priority in this part of the conversation. Drill in as needed:

- What's the pain and is it urgent/quantified? - *"You mentioned [pain]. Help me understand the impact. What's that costing you - in staff time, in missed opportunities, in money?"*
- Is it a priority, and do they have a sense of timeline? - *"How much is [frustration] actually getting in the way? Is it blocking you or just annoying?" "Is there a timeline or trigger that makes solving this more urgent?"*
- What does the decision process look like? - *"Hypothetically, if you did decide to switch tools, how does that work at [company]? Who gets involved?" "Who controls the budget for this kind of thing?" "Have you ever bought a tool like this before at [company]? What was that process like?"*

4. Qualify or disqualify

Run a quick mental evaluation of their answers on the call. Assess four factors:

- Specific pain identified
- Line of sight to impact (time/money)
- Timeline (next 6 months)
- Authority or direct line to buyer

If unclear, ask directly, e.g. timeline:

```
*Why is this a problem you're trying to solve this / next quarter?*
```

Their answer tells you if this is a priority.

If you have fewer than four, disqualify politely.

```
*"Based on our conversation, and being completely honest, I don't think we're the right fit because [reason]. My recommendation: [alternative]. If [use-case/trigger] changes, do please reach out."*
```

If you have all four, ask permission to pitch.

Disqualify outbound tasks that won't convert.

> **Bonus: end early** if they're disqualified or disinterested. If highly qualified and eager, skip the pitch and go straight to a next step.

5. With explicit permission, give a brief PostHog pitch

Open with what you heard and ask for permission to pitch:

```
*"Based on what you shared - [their pain] - let me tell you how PostHog works and you tell me if it's relevant. Does that work for you?"*
```

Pivot to a tailored elevator pitch (below is generic):

```
*"PostHog makes dev tools that help product engineers build successful products. These include many discrete tools that help with user behavior and analytics, product engineering, communication and data - all in one platform."*

*"Companies switch for three reasons: (1) tired of fragmented tools, (2) want engineers and product teams to have direct access to data, (3) our transparent, usage-based pricing."*
```

6. Ask the hard question

Ask:

```
*"Does that sound like it solves the problem you described?"*
```

If they're uncertain, emphasize the free trial:

```
*"Knowing that we offer folks like you a free trial period to evaluate PostHog for yourself, does it sound like PostHog solves the problem you described?"*
```

Wait. Embrace the pause. And, get their answer. If we don't solve a problem for them, this isn't worth continuing.

7. Provide a relevant next step & schedule it on the call

If qualified and interested, propose a next step and book it on the call:

```
*"What makes sense as a next step? Demo? Trial? Talk to your team?"* *"Okay, I'll [take action]. Let's reconnect on [book specific date/time now]."*
```

If hesitant or marginal, ask:

```
*"Here's what I'm hearing: [summary]. Not sure if we're a fit yet. What would help you figure that out?"*
```

If they disqualify themselves post-pitch, disqualify:

```
*"Based on our conversation, and being completely honest, I don't think we're the right fit because [reason]. My recommendation: [alternative]. If [use-case/trigger] changes, do please reach out."*
```

8. Action the task in PostHog's Salesforce

This is internal hygiene. Track tasks to reflect the opportunity:

- If qualified + next step, create an opportunity in `Problem Agreement` and use stage exit criteria
- If marginal/no next step, switch task from `In progress` to `Nurturing` and progress them toward an opportunity
- If not qualified, disqualify with reason and share feedback with in Slack

9. Rinse, lather, and repeat

You should always aim to get them into a shared Slack channel or establish a regular communication cadence with them (call/email). Nothing will happen if we aren't talking.

Where else you take a qualified outbound sales opportunity is dependent on the specifics of your conversation.

Your process may resemble later stages of the new business sales process.

Otherwise, you can:

- Book a technical demo with the person's team

- Ask for an introduction to the best contact at the company
- Record a Loom of specific features to show how PostHog works
- Ship them documentation and a code sample to demonstrate how PostHog can be configured
- #domoreweird in a delightful way
- Schedule a kickoff to get their trial started
- Ship them merch

What won't change: qualify each step, solve a real problem, and don't assume interest just because a task became an opportunity. Stay focused on their pain and you'll earn the right to keep moving.

How our outbound data pipelines work

So far we run three automated pipelines that enrich accounts, surface timely signals, and qualify targets. manages these.

Salesforce enrichment (weekly)

Every week, we pull all Salesforce Accounts and enrich them via the Harmonic API with company info like funding history, headcount, and traction metrics. Our SSoT of Accounts (Single Source of Truth) is the Salesforce Accounts table.

Before enriching, we filter out personal email domains (Gmail, Yahoo, etc.) and normalize website domains so matching is consistent.

Job switchers → Clay (daily)

A daily query (Clickhouse + Customer.io) detects job-change signals — someone who was at a company using PostHog just moved to a new role. Only changed or new records are sent to a Clay webhook so we stay within Clay's submission limits.

Why this matters: when someone who already knows PostHog changes jobs, that's a timely outreach moment. They're evaluating tools at their new company and already have context on what we do.

Product-led outbound → Clay (daily)

First, a daily Warmbound query pulls a base set of target accounts filtered by revenue band (MRR 100–499), company size (50+ employees), and company type.

Then a second qualification step filters those accounts against product signals. Only accounts that pass both steps *and* have changed since the last sync are sent to Clay.

An account passes the second step if it shows buying intent through signals like:

- Using two or more PostHog products
- High event volume
- Two or more new team members in the last 30 days

- Adopting new products they weren't using before

This focuses outbound on accounts that are already engaged with the product (i.e warmbound), not just random companies that match a firmographic profile.

Where data lives and flows

```
graph LR
    SF[Salesforce Accounts - SSoT]
    H[Harmonic API]
    PG[Postgres]
    CH[ClickHouse]
    Q[Qualification]
    CL[Clay]
    LL[Lemlist]

    H -- "enrichment (weekly)" --> SF
    SF -- "accounts" --> Q
    PG -- "billing data" --> CH
    CH -- "usage & signals" --> Q
    Q -- "qualified targets (daily)" --> CL
    CL -- "selected accounts" --> LL
```

System	Role	How often updated
Salesforce	Account records, opportunity tracking, enriched fields	Weekly (enrichment), real-time (sales activity)
Harmonic	Company enrichment data (funding, headcount, traction)	Weekly via enrichment pipeline
ClickHouse	Product usage data, job-change signals, ICP scoring	Daily via pipeline queries
Postgres	Organization and billing data	Continuous
Clay	Outbound qualification and personalization	Daily via webhook syncs
Lemlist	Email sequencing and outreach delivery	Via Clay

Appendix: full GTM data flow

This is the broader picture of how data moves across all our GTM systems, not just the outbound pipelines above.

```
graph TD
    subgraph PostHog
        PH[New signup in PostHog]
        CDP[CDP Destinations - Events, Person Info, Org Info]
        CF[Contact Form]
        BILLING[New customer in billing]
    end

    subgraph Enrichment
        HARMONIC[Harmonic]
        CLEARBIT[Clearbit]
        CLAY[Clay]
    end

    PH -- "PH destination" --> SF
    PH -- "new org" --> BILLING
    CDP --> DEFAULT[Default app]
    CF --> DEFAULT
    DEFAULT --> SF

    SF[Salesforce - Contacts + Accounts]

    SF <--> HARMONIC
    SF <--> CLEARBIT
    SF <--> CLAY
    BILLING --> SF

    SF --> VITALLY[Vitally]
    CIO[Customer.io] --> VITALLY
    ZENDESK[Zendesk] --> VITALLY
    PYLON[Pylon] --> VITALLY
    GMAIL[Gmail] --> VITALLY
    STRIPE[Stripe] --> VITALLY
```

Overview

contents/handbook/growth/sales/overview.md

Our primary focus is on making our paying customers successful, not forcing sales through. This mostly means an inbound sales model, but we are also running some outbound sales experiments.

While this means working with a smaller number of users than typical B2B SaaS companies, we know that the people we talk to are mostly already pre-qualified and genuinely interested in potentially using PostHog.

The Sales team act as genuine partners with our users. We should feel as motivated to help and delight users as if we were on their team. In practical terms, this means:

- No BS sales-y talk - we are direct, open and honest with customers. We share as much as possible publicly, rather than hiding it behind a mandatory demo call. We are honest when we don't know the answer, or if we're not sure that PostHog is the right solution for a customer.
- Speed - we are weirdly responsive. If a customer is in a rush, we do our best to work at their pace. We are clear about expectations, and do not promise what we cannot deliver to close a deal.
- Engineers helping engineers - there is nothing more frustrating than talking to a salesperson who can't give you all the answers. We keep 'let me find out from the team' to an absolute minimum.
- Being power users of PostHog is a must - otherwise we won't be credible. PostHog is a big and growing platform, so this is a challenge to stay on top of!
- We prioritize getting people set up on multiple products as early as is feasible, as this makes PostHog far more valuable to them and increases our chances of retaining them.
- We don't do margin negative deals in order to win - this doesn't set us up for a successful long term relationship with a paying customer if we're ultimately losing money to land them. Yes, this includes fancy companies whose logos would make us look good.

Sales team vision

Things we want to be great at

- **Technically capable:** We deliver genuinely useful insights about things those customers care about (can be purely PostHog-related, but also general advice). Our ICP are ‘self-servers', so ideally we teach them how to do something, rather than doing it for them. A great support experience is part of this.
- **Speed:** We want to be highly reactive, low process, and reliant on other teams as little as possible to ship things. We want to get stuff wrong quickly, then iterate.

- **Cross sell:** PostHog gets much more powerful as customers adopt multiple products that all share the underlying data. And they stick around longer. It's a win win.
- **Warm outbound to product leads:** We get hundreds of ICP signups to PostHog every week, and we want to make sure we're laser focused on ensuring they have the best possible experience with PostHog by proactively reaching out to them based on certain triggers. Some people call this 'warm outbound'.

Things we're interested in trying out

- **Cold outbound:** Most companies do this really badly. We're interested in seeing how we could do this the PostHog way.

Things we don't want to spend time on

- **Events:** These *may* be a good way for us to reach more of our ICP in future, done in the right way, e.g. by giving talks. However events are not a scalable/automatable channel, and are slightly in the zone of 'outbound sales'.
- **Winning the deal at all costs:** We have overall ARR targets at PostHog, but these are not exclusively achieved by the Sales & CS team - the vast majority of our paying customers come in without ever talking to us. This means that revenue isn't the CS team's responsibility alone, so we don't have to close deals where we get a short term bump to revenue in exchange for long term pain/churn (e.g. forcing a non-ICP deal to close with extremely discounted pricing).

How to work with different types of customer

We look after customers who are paying or could pay $20k+/yr. This means sometimes we will work with existing smaller accounts if we see potential to grow them into larger ones.

> We've written an internal playbook for how to manage different types of customers - this goes into a lot more detail about company style, how they work, likely PostHog adopters, how to communicate etc.

'Enterprise' customers

As we get bigger, we're getting more inbound demand from larger organizations which have a very different buying process from our smaller customers. If we want to reach our ambitious revenue goals, we'll need to get good at selling to this segment of customer. However, we need to do this without compromising our focus on building a great product for our ICP.

To prevent us going down the wrong path with deals like these, we follow 4 simple principles:

- **We don't contract deliverables**. Otherwise a single customer could have too much of an impact on team morale and priorities.

- **We will build things for a big customer, as long as we are confident they won't be the only user of that thing.**
- **Customers need to try PostHog before they ask us to change things**. We love feedback from customers. We don't love big requirement documents from people that haven't used our product before.
- **We don't care about losing deals**. If we have to walk away from a deal because we'd have to compromise on these principles, we will. We can do this because we have a really strong growth engine with our ICP customers.

We'd typically define a deal as a large deal if it has most of the following:

- The customer puts us through a lengthy procurement process (3+ months)
- The customer wants us to build new features
- There are multiple stakeholders on the customer side, some or all of whom are not engineers
- The deal is larger than $250k/year

Who the Sales team are

Our small team page is maintained on the Sales & CS team page. In addition to people who share PostHog's culture, we also value:

- People who have very high empathy with product teams and their needs
- People who are happy to choose their own objectives if it meets a business goal
- Low ego, and a willingness to turn around even the most disgruntled and unreasonable customer
- Hands-on people not motivated by managing a team
- We would want to buy PostHog from them - this is more important than cool logos

Product enablement

contents/handbook/growth/sales/product-enablement.md

Overview

PostHog has a broad and growing set of products, and folks in GTM roles need to develop and maintain deep product knowledge for each individually, as well as understand how they work together. This deep understanding helps us drive initial adoption of PostHog as well as cross-sell and expansion. Without a structured enablement process, we face several challenges:

- **Onboarding gaps**: New joiners lack a clear path to learn our products systematically
- **Keeping current**: With continuous product development, it's difficult for team members to stay up to date on changes and new features
- **Uneven expertise**: Knowledge levels vary across the team, leading to inconsistent customer experiences
- **Missed opportunities**: Without comprehensive product knowledge, we may miss adoption and expansion opportunities with customers

> Rather than hiring an external sales enablement person (who would need significant time to ramp up on PostHog and wouldn't speak with customers regularly enough), we're leveraging internal expertise to build and maintain our enablement program.

How it works

Subject-Matter Experts (SMEs)

Each product area has a designated SME from the Sales, CS, or Onboarding teams. The SME is responsible for:

- **Coordinating content creation** for their product area (not necessarily creating everything themselves)
- **Keeping content current** as products evolve
- **Facilitating knowledge sharing** through regular updates
- **Being a bridge between GTM and Product** to ensure a two-way feedback loop exists

> **Important**: SMEs are enablers, not gatekeepers. The goal is to level up the entire team, not to create dependencies on specific individuals.

Content areas

For each product, SMEs should develop and maintain content covering:

- **Sales messaging** for customers not currently using the product

- **Demo best practices** showing how to effectively demonstrate their product
- **AI features** how to use PostHog AI for maximum effect within their product
- **Implementation considerations** to help customers avoid common issues
- **Real-world use cases** demonstrating practical applications
- **Competitive positioning** relative to alternative solutions
- **Customer stories** highlighting successful implementations
- **Product updates** covering new features and changes

Content should be primarily recorded (Loom, Gong) or visual (Pitch) to support our async, distributed team.

New hire onboarding

New joiners to Sales, CS, and Onboarding teams go through PostHog GTM Academy, which incorporates product training content from SMEs in a structured learning path. This ensures consistent foundational knowledge across the team.

Staying current

It's on the SME to schedule a regular (nominally monthly, but this may vary by product - use your judgement here) update call with the GTM team and someone from their product team to cover:

- Share recent developments and new features
- Demonstrate new capabilities
- Hold a "no stupid questions" session for team members to ask anything about the product

> We're a global team. Try to schedule the meeting to get as many folks live as possible (8-10 AM Pacific time is an ideal slot here), but also ensure it is recorded.

Content storage

- **Handbook**: This page lists SMEs and links to training materials
- **Video content**: Stored in Loom
- **Written/visual content**: Stored in Pitch or Google Drive
- **Last updated dates**: Displayed alongside each product area's materials

All content should include a "last updated" date so team members know they're working with current information.

> As we develop content, we can link directly to it from this page.

Product areas and SMEs

Product Area	SME	Last Content Update
Product analytics	Ben Smith	-
Web/Customer/Revenue analytics	Jon	-
Session replay	Dana	-
Feature flags	Sachin	-
Experiments	Sachin	-
Error tracking	Christophe	-
Surveys/Product tours	Leon	-
Data pipelines (batch and realtime)	Ryan	-
Data warehouse	Ryan	-
LLM Analytics	Leo	-
Workflows	Phil	-
PostHog Code	Landon	-
Logs	Sean	-

For SMEs

Getting started as an SME

If you've volunteered to be an SME for a product area:

> First of all, thank you!

1. **Connect with the product team** - Consider joining sprint planning calls to stay informed
2. **Audit existing content** - Review what training materials already exist
3. **Identify gaps** - Determine what content needs to be created or updated
4. **Recruit help** - Enlist others (team members, product team, etc.) to help create content
5. **Establish a cadence** - Plan regular content reviews and updates

Best practices

- **Don't work alone**: Work with product teams and other GTM team members to develop comprehensive materials

- **Stay connected**: Join product team sprint planning calls if helpful and time zones allow
- **Gather feedback**: Channel top feature requests to product teams (but don't act as a gatekeeper)
- **Keep it fresh**: Review and update content at least quarterly

What this is NOT

This enablement program is **not**:

- **A replacement for individual expertise**: Everyone should use SME content to develop their own product knowledge
- **An expert escalation path**: SMEs aren't brought into customer conversations as specialists; the goal is to enable everyone to be effective independently
- **A one-person show**: SMEs coordinate content creation but shouldn't develop everything themselves
- **Cross-sell playbooks**: That's a separate initiative
- **Customer-facing content**: This is internal enablement; customer-specific demos should be created by account owners as needed

New products

When a new product is approaching customer availability:

1. Identify an SME early in the development process (bonus points if you self identify with a handbook PR)
2. SME coordinates with the product team to understand capabilities and use cases
3. SME develops initial training content before general availability
4. SME delivers a new product training session to the wider GTM team
5. Product area is added to the table above

Switching SMEs

If you don't want to be an SME for a product area anymore, or want to switch with someone else to stay fresh, first identify someone else who is willing to step in and make the change as a PR to this page.

Product-led Sales

contents/handbook/growth/sales/product-led-sales.md

A large proportion of our paying customer base sign up to a paid plan without ever talking to the Sales team. We don't want to force these customers through a sales process if they don't need it, but we also know that having a human to help them through the process on hand is likely to maximize the chances of retaining them as a paying customer long term. Longer term, we know that customers who have worked with a member of the PostHog sales team retain better, and are much more likely to expand their usage through cross-sell.

Product-led lead generation

Product-led leads can be generated in different ways - see this page for more info.

> Some product-led leads might have already chatted with someone on our team. Before reaching out, take a quick look in Vitally to see if there's any prior activity, and check in with the AE or team member who was involved to get the full picture if needed. Every lead has a "Vitally account URL" field in Salesforce which links directly to their Vitally profile for easy review.

Working with the customer

Just as with the inbound sales process, it's on you to decide how you qualify the lead. If you think they have potential to end up paying more than $20k a year then you should reach out to introduce yourself and offer help. As they have likely done a lot of research themselves, they may not need a demo so a 30-minute discovery is probably more appropriate here.

> Getting people already happily using PostHog to talk to you can be challenging - here are a few things you might want to try.

If it's a viable opportunity then you should convert the lead to an opportunity and then follow the New sales process. Bear in mind that you can join it at any point depending on where the customer is at in their buying journey (e.g. you might skip product evaluation if they are ready to buy). If they are eligible for a shared Slack channel and they do not already have one, set one up.

Even if after speaking with them you think they may not end up at $20k+, you should educate them on how to get help, as well as the value of adding our Scale, Boost, and Enterprise plans.

Startup plan roll off

Customers who are rolling off the startup plan present a unique opportunity, as they are already using PostHog and may well be spending >$20k annually. Just like any other customer, we want to help them reduce spend and get the most out of their existing usage, while also educating them on the savings

involved with an discounted, credit-based plan.

For customers that may have started implementation late, or ran into issues during their startup period, at our discretion, we can extend the life of their credits by 3 months. To do so, visit that customer's billing admin page (linked in Vitally), scroll to the bottom, and click the `extend startup plan` button.

Getting recognized on the deal

As they have already shown intent by signing up/subscribing, you will need to demonstrate that you have actively worked on the opportunity to include it in your book of business. We will use a common sense approach here but sending a couple of emails and 1 call won't be classed as 'actively working'. We want to ensure there is concrete sales activity going on with this customer. Simon will make the call here, escalating if needed.

Professional services

contents/handbook/growth/sales/professional-services.md

Some potential customers either expect to pay for professional services to help them get set up. There are others who don't ask for this, but where we can tell it would be helpful for them.

> For now this is only a service we offer to potential customers by default, so this will mainly be of interest to the .

Who we can offer professional services to

A good candidate for this probably has some combination of the following:

- ARR in the $100k+ region, either today or in the next 12 months (actual spend, not credit)
- Large-ish company with complex needs
- Have explicitly asked about paid implementation services and training
- The team adopting PostHog work in person at a single location
- They are not set up with PostHog yet, ie. we would be helping them implement PostHog in their codebase
- They are able and willing to give our team access to their codebase
- We are helping them get set up with session replay, feature flags, LLM analytics, and/or error tracking

If you are working with someone where this might be applicable, ping Charles and Simon first, as we can offer to send a forward deployed engineer to work with them to help get set up. **Please don't just offer this to anyone without checking in**, as we don't have unlimited capacity.

For ongoing training, this is something that we are solving for separately, but is not within the scope of professional services at the moment.

What work is included

Typically, we will send a forward deployed engineer to work with a customer for a week in person. What we charge depends heavily on the nature and scope of the implementation, but in any case starting at $10k. Simon will work with you to figure out the relevant scope of work and contractual terms.

We don't offer this for free, because it is a valuable service that customers expect to pay for. We also don't offer it as a freebie negotiation tactic, because that devalues it for all other customers.

The specific checklist of what will be implemented depends on the customer, but the following sections detail the broad topics we can cover.

Implementation scoping

This should be conducted ahead of time to ensure that we deliver services according to the customer's needs. We will document a plan for:

- PostHog SDK implementation.
- User identification.
- Privacy controls.
- Sampling.
- Autocapture tuning.
- Custom event instrumentation.
- Identifying key insights/dashboards to be created.

At the end of this session we should have a good plan and understanding of any onsite work which needs to take place, as well as who in the customer our engineer will be working with, and what level of access to customer systems they require.

Technical implementation

Here we make sure that PostHog is correctly integrated into the codebase using one or more of our SDKs. We should also:

- Implement any user identification or privacy controls that are required.
- Ensure they are only tracking the events which they need to (including implementing custom events where necessary).
- Integrate the first set of feature flags into their application.
- Set up the relevant dashboards and insights as documented in the scoping phase.
- Ensure that everything is set up according to our best practice guidance.

Getting started training

Once data is integrated, we should provide an intro to PostHog session to the customer to teach them the basics of how to use PostHog. This will be tailored to their needs but should provide a baseline understanding of how to navigate the UI, where to find events, create insights, filter replays, etc.

> Whilst Sales and CS folks also provide ongoing training to customers in their book of business, it's important to ensure they have a basic understanding of PostHog, especially if they are brand new.

Migration

Whilst it's crucial to get live data flowing in to PostHog, the customer may also want to bring over historic data to PostHog from their previous tools. This will normally be product analytics data, and we have both managed and manual processes for this depending on the incumbent tool.

> Longer term it's expected that forward deployed engineers will own the managed migration tools and also build out that capability.

Once data is migrated, we may also need to implement dashboards and insights from the previous tool. There's no automated way to do this currently, so we will need a login to the previous tool to understand and recreate the visualizations they need to move over.

If the customer is replacing a feature flagging tool and has existing feature flags in place, we will need to migrate the flags in the codebase and ensure that the flags and targeting are set up correctly in PostHog.

Data connectivity

This will normally involve the setup of realtime and batch export destinations for other tools in the customer's stack. We'll need API keys for the relevant tools as well as an agreed criteria for choosing which events will go to which destination.

This may also involve getting data into PostHog without using our SDKs, for example, by using the Webhooks or Data Warehouse sources.

SQL Query implementation

Some customers aren't able to write SQL themselves and don't want to rely on PostHog AI. We can scope and write the SQL queries that they need, as well as creating the relevant views and person joins so that all of their data is connected.

Statement of work

Before any onsite work we will need to document a Statement of Work (SOW) which outlines the scope of work and the agreed terms of service. We should incorporate what we learn in the scoping phase into this to ensure we have all the customer's needs covered and allocate the right amount of time to the project.

Refunds

contents/handbook/growth/sales/refunds.md

We know things happen and sometimes you might need to issue a refund. Here's how we handle common scenarios:

Learning curve

Just got off of the startup plan/new client accidentally used us a lot.

We issue refunds or credits in this category if this is the first bill >$1 and/or they meet eligibility criteria as explained below.

Unexpected stardom

Side project sudden volume spike

We issue refunds or credits in this category if this is the first bill >$1 and/or usage spiked by >200% compared to their average usage over the past three months, and the company doesn't have any revenue, or is a hobby project.

Under attack

Bot spike/abusive user drove traffic which in turn increased PostHog usage

We flag accounts with unusual activity spikes for review, and refund or issue credits to cover the overage amount once the issue has been resolved. The issued amount covers any amount exceeding the average usage of the three months preceding the spike.

Wrong setup

New feature trial with incorrect configuration

We issue refunds or credits in this category if the customer was charged for features they didn't intend to use due to default settings or configuration errors, and this is the first occurrence of unintended usage charges.

Eligibility criteria

Customer must meet the following criteria to get a refund:

- The request is made within 30 days of the billing date.
- The customer provides a reasonable explanation for the request, fitting into one of the scenarios.
- The account does not show signs of fraudulent activity or abuse of PostHog services.

- In cases of volume spikes, the unusual usage has ceased and there is evidence that the customer has taken measures (like implementing a billing limit or managing event volume).

Repeat incidents

For first incident response, we follow standard policy above and provide guidance for preventing future incidents (e.g. ask them to implement billing limits)

Subsequent incidents:

- First, check if the customer has acted on PostHog's earlier recommendations.
- If they have not yet fixed the issue, refunds are conditional. Give them a window to implement the fix, and offer a partial refund (up to 50%) while they address it.
- If they have made good faith fixes but the issue still occurred, then we issue a full or partial refund depending on severity.
- Always warn that repeated incidents may not be refunded again.
- For third incident and beyond, refunds may be declined unless there are extraordinary circumstances (e.g. a PostHog bug).

Request channels and processing

Refund requests can come through different channels:

In-app ticket

- Support team reviews the request, issues refund or credits based on the eligibility and criteria outlined above, and responds to customer.

Contact sales form or email to sales@posthog.com Account Executives can direct these to the Support team using the ticket emoji in the #website-contact-sales Slack channel to auto-create a Zendesk ticket

Large account requests

- For large accounts managed by an AE, AE may lead the customer conversation and can loop in Support or RevOps team as needed to process credits or refund in Stripe.

Processing credits or refunds

How to calculate overage amount

Review customer usage

Before doing a refund, review customer's usage. Some useful sources:

- If you have access to Vitally, find the customer's Metabase dashboard link under the 'Usage Dashboard Link' trait. The 'Event counts by type last X days' insight is particularly useful here - you can change the lookback period to see a longer time range.
- This dashboard for an overview with usage reports and invoice history.
- This dashboard to help identify issues for customers with many projects
- You can also make a copy of this PostHog insight and use organization id to review account usage. Note that the org usage report can run 2-3 times per day, so numbers may be duplicated/inflated.

What's "normal" vs "weird" usage:

- Normal: Gradual increase, weekly patterns
- Suspicious: Sudden 10x jumps, severe spikes
- Bot attack: Lots of similar events

Identify baseline usage

- Calculate the average usage over the past three months to establish a baseline.
- Cross check the amount calculated with the last two invoices paid by this client to make sure they're consistent.

Calculate overage

- Subtract the baseline usage from the spike amount to find the total overage. Example: If average monthly usage was 1 million events per month and a spike resulted in 3 million events, the overage amount would be 2 million events.

For event-specific overages (optional):

If you want more precision when a single event type is inflated, use the 'Event counts by type last X days' insight in the Metabase dashboard:

1. Change the lookback days to find the baseline period before the spike
2. Identify the inflated event type and compare its spike volume to the baseline
3. The difference is your overage amount for that event type

Calculate the amount to refund/credit

- Use the pricing calculator to calculate the total price for baseline and overage volumes. The difference between the two will be the refund amount.
- Alternatively, you can use QuoteHog - go to the usage history tab, build a price option from a specific month's usage, and subtract the inflated volume to see what the bill would have been without the spike.
- Don't just put in the overage amount in the calculator - doing this would give you the wrong amount because of our tiered pricing structure. Calculating the difference between regular usage and usage with overage is the accurate way to calculate actual amount.

- Add a note in the Zendesk ticket with a breakdown of calculations, the baseline average, and the overage. This transparency can be helpful if the customer has questions.

Refund or credit?

- Issue credits if the customer's period hasn't ended yet and the invoice isn't finalized. It is much easier and better for users and us to avoid payment if we can!
- If invoice is finalized and this is a first time request, issue a refund via a credit note (do not use the refund button, this is important for correct revenue attribution).
- If the customer has overdue invoices and needs changes on that, we need to apply credit notes. Escalate such cases to RevOps.

How to issue refunds or credits

Prerequisites

You need Support specialist level access to Stripe, ask Simon for access.

Issuing credits

1. Go to billing admin
2. Next to 'Credits', click on 'Add'
3. In the 'Customer' field, use the drop-down menu to find your customer
4. In the 'Amount' field, set an amount of credits you wish to issue for this customer
5. In the 'Reason' field, select a reason which best describes why you're issuing the credits
6. Add an optional note in the 'Notes' field
7. Include an optional link in the 'Reference link' field, e.g. Zendesk ticket, Slack message link, etc.
8. Click 'Save and view'
9. Confirm that the credits were successfully added to the customer's balance in Stripe under 'Customer invoice balance'

Issuing a refund

Refunds are now initiated through Billing Admin and finalized in Stripe via a credit note.

There are two ways to reach the Add Refund screen.

Option A:

1. Navigate to Billing Admin → Customers.
2. Find the right Customer (search by organization ID or customer ID).
3. Once in the Customer view, scroll down to *Related invoices* section. Find the right one (you can identify it by its id, dates or amount).
4. Click on "Start refund"

Option B:

1. Navigate to Billing Admin → Invoices.
2. Find the right invoice (search by invoice ID, organization ID, etc).
3. Click and open the invoice view.
4. Once in the view, click on the top right button "Start refund"

Once you do that (through any of the two options), you'll land on the "Add refund" screen. From there, you can continue with the refund:

1. Allocate refund amounts per product. Refunds must be issued per product. Enter the refund amount for each affected product. You may need to do more math here: for an event spike refund may span Product Analytics, Person Profiles, and Group Analytics. Billing Admin does not automatically split refunds across products, you must do the math and allocate amounts manually. As you enter per product amounts, the total refund amount updates automatically.
2. Select refund reasons: Choose a Stripe refund reason (required) and select an internal reason (used for internal reporting and analysis)
3. Add any relevant notes or context (e.g. Zendesk ticket, Slack link, short explanation)
4. Once you review everything and all looks good save the refund in Billing Admin. This will issue a Stripe credit note, which is processed as a refund to the customer's default payment method. Stripe automatically sends a notification email to the customer.

Fixed fee product refunds

For fixed-fee subscriptions (e.g. Boost plan), Stripe's default proration behavior can cause double crediting.

Example: A customer subscribes to a fixed fee add on by accident and requests a refund. After we issue a credit note, they cancel their subscription. When this happens, Stripe automatically creates a prorated "unused time" line item on the next upcoming invoice. This results in the customer being credited twice:

- once via the manual credit note
- again via the prorated unused time credit

To prevent overcrediting, we need to manually delete the pending invoice item that Stripe creates after the subscription cancellation.

Steps:

1. Find customer profile in Stripe (you can search by organization id)
2. Locate the proration adjustment under Pending Invoice Items.
3. Manually delete the line item.
4. Add a note in Zendesk documenting that the proration line was removed to avoid double crediting.

Spotting suspicious stuff - watch out for:

- Multiple accounts that seem connected - easiest way to spot this is to look up user profile in Vitally and check connected accounts. If one email is connected to multiple accounts it is good to check previous requests and refunds on all related accounts as well.
- High refunds notice in Stripe (This will appear as a yellow box notification next to the customer's name on the customer page in Stripe)
- Usage that doesn't make sense for customer size, or business details don't add up

When to escalate to RevOps

- Something seems off
- They've asked for multiple refunds lately
- The case doesn't match the simple cases above
- It's a big customer (spending $1,667+ monthly)
- Need to create or modify an invoice for the correction. Support team should not create or modify invoices. Invoicing responsibilities will be handled by RevOps to maintain accuracy.

Tag in Zendesk and share what you checked, what you think we should do, and any other relevant context. RevOps will review usage trends and customer lifecycle (e.g. new client, high-value account) to figure out next steps.

Our approach

We'd rather fix unexpected usage issues than have customers pay one massive invoice and then reduce spending or leave us. The goal is to maintain a fair, transparent relationship that works for everyone in the long term.

Trial periods and usage spikes

- We're generous with trial periods for actively engaged new ICP customers
- Tag accounts as "trial" in the billing system. If they're already paying when actively testing the product, inform Mine/RevOps for proper tagging and revenue recognition
- If you spot accidental usage spikes, proactively reach out and work with customers to reduce their spend to fit their budget and needs
- When you're unsure about handling a specific case, ping Mine/RevOps directly for review

Risk mitigation and churn prevention

`contents/handbook/growth/sales/risk-mitigation-and-churn-prevention.md`

If you're actively thinking about churn prevention in response to a customer churn threat or major red flag, it's already way too late.

Churn prevention is best done from early, and often, risk mitigation practices.

We should default to flagging "at risk" accounts using the "Churn Risk" segment in Vitally well before the customer has told you they are exploring alternatives. If you have the slightest inkling that something may look off or something has you feeling a bit uncomfortable, flag it. This could be anything from not taking action on a recommendation you gave them for too long, down-trending volume with no apparent seasonality cause, only one or two core users of the platform, or no Slack activity for an extended period. To name a few.

There are a few risk-mitigation strategies you'll want to incorporate that serve as early detection and proactive mitigation, as well as a process for what to do when an account is actively at risk.

Risk mitigation (proactive)

Risk mitigation is about building habits that surface problems before they become emergencies. If you're doing this well, you'll rarely need the reactive playbook below.

Quarterly account planning

Every AM Managed account should have an Account Plan note created in Vitally once per quarter. You should also review this weekly with your manager. This forces you to step back and evaluate the account holistically rather than just reacting to whatever's in front of you.

Title format: `Q[X] Account Plan – [Company Name]`

Use the Account Plan template in Vitally, which auto-populates key fields from the account record. The template covers:

Account overview

- ARR, business description, website, HQ location
- Business type (B2B SaaS, E-commerce, Marketplace, Developer Tools, Fintech, Healthcare, etc.)
- Key metrics relevant to their business model
- Business stage and funding

Business objectives

- What they're trying to achieve with PostHog (specific goals, not vague "analytics")

- How PostHog connects to their larger business objectives
- Whether value aligns with their expectations
- Obstacles they're facing, both in using PostHog and in their broader goals
- Upcoming constraints (budget freezes, code freezes, migrations, seasonality)
- Future needs over 6-18 months

Stakeholders and users

- Admin emails and total user count
- Percentage of active users (30 days)
- Key contacts with their priorities, goals, and preferred communication
- Multithreading status: do we have two-way dialogue with technical stakeholders, budget holders, and end-users?
- Record of multithreading attempts and progress

Current usage and cross-sell

- Products adopted (Product Analytics, Feature Flags, Group Analytics, Error Tracking, Session Replay, Surveys, Data Warehouse)
- Usage notes on how they're actually using each product
- Cross-sell opportunities with specific use cases, next steps, and relevant content to share
- Optimization opportunities for existing products (underused features, configuration improvements)

Risks

- Document each risk with: the challenge, what's at stake, plan of action, and next key date

Action items

- Multithreading: who, why, progress, anyone at PostHog who can help
- Finding new opportunity: what are you doing to find new revenue?
- Mitigating risk: what are you doing to uncover and mitigate risk?

If you can't fill out most of this template, that's a signal you need to dig deeper into the account. An incomplete account plan usually means incomplete understanding of the customer.

Early warning signals

These aren't emergencies yet, but they should make you pay closer attention. Many of these are also tracked automatically as risk indicators in Vitally.

Signal	Why it matters
Recommendation not actioned for 2+ weeks	They're not engaging with your guidance

Signal	Why it matters
Down-trending event volume (no seasonality)	Usage decline is a leading indicator of churn
Only 1-2 active users	Single point of failure, low organizational buy-in
No Slack activity for 14+ days	Relationship is going cold
Billing page visits without context	They're evaluating costs, possibly shopping around
Champion changed roles or left	Your internal advocate is gone
Support tickets spiking	Something is broken or frustrating
Single product usage	Low switching cost, easy to replace
Data exported to external warehouse only	PostHog becomes a pipe, not a destination

When you notice any of these, don't wait. Reach out, dig in, and address it before it compounds.

Drive adoption of behavioral products

Not all PostHog products carry the same switching cost. Customers who primarily use PostHog as a data pipe to an external warehouse are structurally riskier. If their analysts query Snowflake or BigQuery, PostHog becomes invisible and replaceable.

Behavioral products create stickiness because they're used directly in PostHog's UI and embed into day-to-day workflows:

Product	Why it's sticky
Surveys	Active feedback collection tied to user segments. Hard to replicate externally.
Cohorts	Saved user segments used across insights, feature flags, and experiments. Accumulated investment.
Workflows	Automated actions triggered by product events. Operational dependency.
Feature flags & experiments	Engineering teams build release processes around them. Deep integration.
Session replay	Qualitative context that doesn't exist in a data warehouse. Unique value.

When you see a customer heavily reliant on data pipelines and external analytics, proactively introduce behavioral products. Frame it as expanding what they can do, not replacing their warehouse workflow. The goal is to make PostHog the place where decisions get made, not just where data passes through.

Practical moves:

- Show them how cohorts can power targeted surveys or experiments
- Demo how session replay answers "why" questions that SQL can't
- Introduce workflows for automated follow-ups based on product events
- Help them build dashboards they actually use inside PostHog

If they're doing all their analysis in Looker or Mode, ask why. Sometimes it's habit, sometimes it's a gap we can fill. See communication templates for new feature adoption for outreach examples.

Implementation health

A lot of customers self-serve without ever talking to a PostHog human. This means they can implement PostHog in ways that cause problems down the road: inflated bills, inaccurate data, or features that don't work as expected. Left unchecked, these issues lead to avoidable churn.

Proactively check for common implementation issues, especially for newer accounts or accounts that haven't had a technical review. See checking the health of a customer's deployment for the full checklist.

Billing waste:

- **Group Analytics enabled but not implemented.** We have a Vitally risk indicator for this. If they're B2B and could benefit, help them implement it. If not, tell them to remove the add-on.
- **Autocapture noise.** If >50% of events are autocapture and they haven't defined any autocapture actions, they're likely paying for events they don't use. Help them tune or disable it.
- **Session replay capturing everything.** Default settings capture all sessions. At minimum, recommend setting minimum duration to 2+ seconds to filter out low-value recordings.

Tracking issues that erode trust:

- **Calling identify() on every page.** Inflates event volume dramatically. They only need to call it once per session.
- **Calling group() on every page.** Same problem. Once per group per session is enough, or when the group changes.
- **Calling posthog.reset() before identify.** Creates unlinked anonymous users. Common culprit when web-to-app tracking seems broken. See guidance in the JavaScript library features guide.
- **No reverse proxy.** Best practice is to use PostHog's managed reverse proxy or configure their own. Events from their own domain improve reliability and ad-blocker resistance.

Feature flag resilience:

- **No fallback code.** If flags fail to load, the app should still work. Check that they're falling back to working code when flags return unexpected values.
- **No local evaluation (server-side).** Server-side local evaluation ensures flags work regardless of network status. Important for reliability.

When you find implementation issues, don't just tell them what's wrong. Help them fix it. A customer who had a billing problem you solved is more loyal than one who never had a problem at all.

De-risking common churn scenarios

Most churn follows predictable patterns. See common churn reasons for the full list. Here's how to de-risk the scenarios we have some control over:

Churn scenario	De-risking strategy
Champion leaves	Multi-thread relationships across teams. The more users actively in PostHog, the less one departure matters.
Champion isn't the decision maker	Identify and build relationships with actual decision makers. Your champion can help with introductions.
Customer builds internally or switches to competitor	Drive multi-product adoption. Harder to replace five products than one.
Poor customer experience	Stay on top of open issues proactively. Circle back before they have to follow up. Rebuild trust through responsiveness.
Customer can't extract value	Offer workshops, training, or hands-on help building specific insights. Don't wait for them to ask.
Missing critical feature	Loop in the relevant PM and engineering team. Be transparent about what we can and can't do. Make sure the request is also tracked in Vitally
PostHog isn't trusted as source of truth	Dig into data discrepancies. Often an implementation issue. If they're exporting everything to another tool, they're one step from leaving.
Privacy/compliance concerns	Help them understand data controls, masking, privacy controls, cookieless tracking, and data deletion options. Often they assume they can't use features when they actually can.

For scenarios outside our control (acquisition, company shuts down, not ICP fit), document what happened and share learnings with the team. There's usually something we can learn even when we couldn't have changed the outcome.

Churn prevention (reactive)

When an account is actively at risk (they've told you they're evaluating alternatives, usage has cratered, or you've lost a champion) you need to move fast and follow a clear process.

When to flag an account as at risk

Add the account to the **Churn Risk** segment in Vitally if any of the following are true:

- Customer explicitly mentions evaluating alternatives or considering churning
- Usage has dropped 30%+ with no seasonal explanation
- Key champion has left and no replacement relationship exists
- Payment has failed and they're unresponsive
- They've asked for a full data export
- Health score has been "Poor" for 4+ consecutive weeks
- Contract is up for renewal in <90 days with negative engagement signals
- They're not using PostHog as source of truth (all analysis happens externally)

When in doubt, flag it. It's easier to remove a flag than to explain why we didn't see a churn coming.

Internal process

1. Add churn risk segment in Vitally

When you flag an account as at risk, add a note in vitally with:

- Account name and ARR
- What triggered the risk flag
- What you know about the situation
- What help you need (if any)
- What you are doing to mitigate the churn

The churn risk bot should automatically post this in the #customer-churn slack channel. This keeps the team informed and surfaces accounts that might need additional support or visibility.

2. Weekly at-risk account review

We hold a weekly team meeting to review all accounts in the Churn Risk segment in Vitally. Come prepared to:

- Give a 60-second status update on each at-risk account you own
- Share what you've tried and what's working or not
- Ask for help or ideas from the team

The objective is accountability and support. If an account has been at-risk for 4+ weeks with no improvement, we need to either escalate for additional support or accept the loss and document learnings.

3. Escalation

Escalation means getting support at a higher level, not handing off the account. You remain the owner and primary contact. Use your best judgment on when to pull in additional resources:

- **Engineering involvement:** If there's a technical issue, bug, or feature gap that's driving the churn risk, loop in the relevant engineering team directly. Tag them in Slack, share context, and ask for their help.
- **Product involvement:** If the customer needs a feature we don't have or is struggling with product limitations, bring in the relevant PM. They may want to join a call to understand the use case.
- **Leadership involvement:** For strategic accounts or situations that need executive attention (pricing negotiations, product commitments, relationship rescue at the exec level), loop in your team lead. For accounts >$50k ARR at serious risk, involve Charles or Simon to get their perspective and support.

The goal of escalation is to get the right people involved to help you save the account, not to pass the problem to someone else. You're still driving the relationship and the recovery plan.

Recovery playbook

Once flagged, your job is to diagnose and act:

Diagnose the root cause. Is this price, product, relationship, implementation, or business change? You can't fix what you don't understand. Use the churn scenarios above as a checklist.

Get on a call. Don't try to save accounts over email. Get face-to-face (or video) time to have a real conversation.

Listen more than pitch. Understand their perspective fully before proposing solutions.

Be honest about gaps. If we can't do something they need, say so. Credibility matters more than closing a save. This aligns with our sales principles: we don't care about losing deals if we'd have to compromise on our principles.

Create a recovery plan. Document specific actions with dates and owners. Share it with the customer so they know you're taking this seriously.

Follow up relentlessly. A save isn't done until the risk is resolved and usage is stable. Check in weekly until you're confident.

When churn happens

Not every at-risk account can be saved. When a customer churns, write a retro and share it in #customer-churn as soon as possible while the details are fresh. See learn from churn for the template and guidance.

Summary

Activity	Cadence
Account Plan note in Vitally	Quarterly
Implementation health check	At onboarding + annually
Early warning signal monitoring	Ongoing
Behavioral product adoption push	Ongoing (especially for warehouse-heavy accounts)
#customer-churn posts	As needed when flagging risk
At-risk account review meeting	Weekly
Recovery calls with at-risk accounts	Within 48 hours of flagging
Churn retros in #customer-churn	Within 1 week of churn

The best churn prevention is never needing to prevent churn. Build the habits, check implementation health, drive behavioral product adoption, and catch problems early.

Running trials

contents/handbook/growth/sales/running-trials.md

Running trials at PostHog

A trial creates space for you and the customer to validate technical fit, agree on success criteria, and close the deal. This guide covers when to offer trials and how to run them effectively.

When to offer Trials

Offer a trial when:

- Qualified for sales-assist (>$20K ARR potential)
- Clear decision makers and timeline identified
- Customer wants technical validation
- Customer is ready to invest time in an evaluation of PostHog

Skip the trial when:

- Below sales-assist threshold (route to self-serve)
- No clear timeline or decision process
- They are price shopping with no technical criteria
- They are already committed to buying

Trial length: 2 or 4 Weeks?

We default to 2 weeks (20–60K ARR) unless there's a compelling reason for 4 weeks.

Extend to 4 weeks when:

- Deal value >$60K ARR
- Customer has a complex technical environment requiring extended implementation
- Multiple stakeholder validation required
- They are evaluating PostHog alongside other products

Trial "must-haves" and "should-haves"

Every trial *must* include:

1. **SDK installed and events firing** - Can't trial without data. This is a non-negotiable, and should occur as a "Day 0" item requiring completion before proceeding to any of the recommended timeline steps below.

2. **Documented success criteria** - Define what "winning" looks like for them. What do they need to see for the trial to be considered a success. Leaving them to their own devices and hoping they envision their success with PostHog in the way that we do is not likely to work.
3. **Documented timeline** - Define the "when" for each of the success criteria in #2.

Every trial *should* include:

1. **Kickoff call** - Use this time to align on the "must-haves": success criteria, timeline. An understanding of "how" they will evaluate is just as critical as what you can be doing throughout the trial to help check off the success criteria. Using this time to collaboratively build the success criteria ensures alignment and mutual understanding.
2. **Shared Slack channel** - Set up before kickoff if possible, so that there's a more "live" way to communicate that comes with better and more accessible support. See Shared Slack Channels with Customers for additional guidance.
3. **Onboarding success plan** - Use the 30-day onboarding success plan template as a starting point, then iterate where appropriate. Adapt for trial length, and share with the customer as a Slack canvas in the shared Slack channel.

Suggested timeline

> Note: Per above, the trial shouldn't progress past this point until the SDK is installed and event data is being sent to PostHog.

Days 1-3: Kickoff & Review

- 30-min kickoff with key stakeholders
- Review settings and configuration aligned with trial goals (Identified events, Session Replay controls, Group analytics etc.)
- Custom events (real KPIs) and properties instrumented (not just pageviews)
    - This is crucial. We want to be able to tie back event data in PostHog to actionable business insights stakeholders care about.
- Build initial dashboards together with customers

Days 4-7: Initial validation

- Additional insights created (trends, funnels, paths)
- Complimentary products introduced (session replay, error tracking, LLM analytics, data warehouse etc.)
- Start to gather initial feedback from test group
- Start to validate success criteria

Business process happens in parallel

As customers are technically validating PostHog, you should also start to work with them on initiating procurement, reviewing legal requirements and aligning on price.

- **Ask:** "What does the procurement process look like?"
- **Validate:** Build and agree on a quote together in QuoteHog based on expected volumes. Does pricing make sense and work within their budget?
- **Identify:** Who approves? What legal/security requirements exist?
- **Timeline:** Confirm we're still on track to close by trial end. "How long does procurement typically take?"

Days 7-14: Close

- Feedback session with stakeholders
- Confirm their success criteria has been met
- Address any remaining concerns
- Validate procurement is in motion
- Mutually aim to get an order form signed by trial end

Support approach

Different customers may need and/or request different levels of support during their trial. We should match the customer's energy accordingly:

Hands-on (larger deals, less technical teams):

- Weekly check-ins
- Proactive dashboard building
- Regular training sessions
- Frequent Slack engagement

Self-serve (technical teams, smaller deals):

- We're available when needed
- Very quick response times
- Async support via Slack

Most trials fall somewhere in-between. It's up to us to read the room and adapt.

Monitoring engagement

It's important to have visibility into a customer's usage and engagement in order to validate whether or not they will be successful with PostHog.

These signals are not guaranteed to always indicate success. Some teams are chattier than others, some teams like to keep comms over email, others enjoy regular Zoom meetings - see above for notes on Hands-on vs. Self-serve approaches.

> Pro tip: You can always use session replay to also check their activity and learn how they are using PostHog! Leverage PostHog AI to help you analyze multiple user sessions. Look for things like:

- How well users are onboarding
- Points of friction, confusion, or frustration
- Product usage & discovery - are they doing the things they said they wanted to?

High engagement signals:

- Active Slack channel (questions, wins, feedback)
- High insight creation volume
- Extensive event instrumentation
- Regular meetings scheduled

Low engagement signals:

- Quiet Slack channel
- Limited insight creation
- No logins for 3-5+ days

If engagement drops, be proactive:

- Build dashboards tied to their KPIs and share with stakeholders
- Send Loom videos showing interesting insights from their data or record your own demos
- Ask directly: "Is this still a priority?"

Your time is valuable. If timing isn't right, it's okay to pause the trial and reconnect with them at a later date.

Extensions

It's common that a customer needs more time to validate PostHog. People get sick, take vacations, priorities change. There are a number of reasons why a customer may need an extension and we're happy to be amenable to them while getting a good understanding of the path to trial end, win or lose.

When considering granting extra time (7-30 days), we should ask for something in return. Examples include:

- Introduction to key stakeholders
- Commitment to start procurement process in parallel
- Verbal confirmation they're moving forward
- Weekly check-in calls until trial end

Always understand: Why do they need more time? What specifically needs to be accomplished?

Wrapping up the trial

Whether it's a 14 or 30-day trial, you should already be getting clear signals that the customer will choose PostHog halfway through. Schedule a feedback session to confirm the technical win and understand what else needs to happen before we make things official.

In your feedback session, be sure to:

- Confirm they've achieved what they need to in order to buy PostHog
- Address any remaining concerns
- Discuss next steps and validate timeline
- Confirm that procurement is in motion

Don't wait for the trial to end to start closing conversations. If success criteria are met early, no need to wait. You can always start moving towards the required closing steps (order form, amount, where to send invoices, etc.) at the customer's pace.

Sales & CS Tools

contents/handbook/growth/sales/sales-and-cs-tools.md

Sales, CS & Onboarding Tools

Here are the common tools the Sales, CS, and Onboarding teams use daily.

Tools through Google and Single Sign-On (SSO)

- Metabase US and EU
- PostHog US and EU instances. Login to both as this is needed for admin access
  - PostHog App + Website reference within PostHog US instance
- Pylon - use Slack SSO
- QuoteHog
- Zendesk
- Zoom
- *Note: Add yourself to group emails sent to sales@posthog.com or cs@posthog.com by joining the corresponding Google Group (sales@ or cs@). It's important you don't mark these emails as spam as Google will unsubscribe you from these group emails.*

Tools by invitation

Mine or Simon can help you out for access or invites for the following tools:

- Gong - ask Simon or your team lead to invite you
  - Download the Gong Meeting Manager extension to trigger a user consent page when joining calls
  - Connect your Calendly link (if using one) with the Gong consent page through this guide on Slack
- LinkedIn Sales Navigator - ask Mine for an invite if needed
- PostHog Billing - ask #team-billing
- Salesforce - ask Mine to invite you if not done already
- Stripe - ask Simon or Dana to invite you, then sign up using your PostHog email
- PandaDoc - ask Simon or Dana to invite you
- Pitch - ask Simon or Dana to invite you
- Vitally - use Google SSO, then ask Simon or Dana to upgrade your role so you can create traits and see success metrics

Tools that you may find useful and not required

- BuildBetter for historic meeting recordings
- Calendly.com for shared meeting booking links - ask Simon or Dana to invite you
- Clay for account and contact enrichment

- Granola (app) for AI note taking in meetings
- In Your Face (app) don't miss meeting notifications
- Loom for short videos (Simon can invite you to the company account)
- Scratchpad for AI agents and a friendlier SFDC UI (sign up as part of your software budget)
- spark (app) AI powered inbox
- Superhuman (app) AI powered inbox
- Wappalyzer for identifying tech stack on customer sites (also as a Chrome/Firefox extension). Credentials in 1Password
- Alfred / Raycast for automated workflows.
- An IDE, like VS Code, Zed, or Cursor. It will come in handy for coding and development tasks.
- Your next favorite tool!

Useful Slack channels

- #team-customer-success - the Customer Success team.
- #team-new-business-sales - the New Business sales team.
- #team-product-led-sales - the Product-led sales team.
- #team-onboarding - the Onboarding team.
- #cs-sales-support - cross-team discussion for everyone who owns customers.
- #sales - for general sales topics.
- #incidents - for notifications of incidents which may impact customers. Make sure you set this to alert you for every message so that you know when something is up.
- #sales-alerts - automated notifications from Vitally.
- #sales-leads - automated notifications for new sales leads.
- #legal - for requests from our legal folks.
- #closed-won - yay!
- #closed-lost - boo!
- #customer-churn - discussion of potential and actual customer churn.
- #changelog - product launches.
- #today-i-learned - where we share our learnings.

Sales operations

contents/handbook/growth/sales/sales-operations.md

Overview

This page outlines how we manage customers, differentiating those who make contact via booking a meeting with us (hands-on) versus those who sign up and get going themselves (self-serve).

If you are looking for guidance on how to manage customers in HubSpot specifically, visit our CRM page.

Hands-on Process

1. Customer will either:
    1. Fill in the contact form on the contact page, which captures what they are interested in as well as metrics such as MAUs, event count etc.
    2. Email us directly at sales@
2. We'll do some ICP scoring and either route them to self-serve or email them introducing ourselves and answering any questions they've shared as well as offering up a call/demo to discuss their needs further.
3. On the initial call we'll spend some time understanding what they want and then optionally give a demo if that's what they are there for.
4. Ensure call notes go into HubSpot against the contact/company/deal so that they are shared amongst the wider team
5. If they are ready to get started with PostHog, we should either:
    1. For lower volume customers we should send them a getting started templated email which providers pointers on how to get set up as well as where to get help if they get stuck.
    2. For higher volume customers we can create a Slack Connect channel in our Company Slack, this allows us to provide more focused support to ensure that they are successful.
6. As a priority we should get them sending data in from production (even just a small part of their app) so that they can see the value of PostHog quickly (decreasing time to revenue) see how we do this in the Onboarding section below.

Self-serve Process

For customers that sign up themselves, and begin using the product, we provide a number of self-serve resources, including:

1. Docs
2. Tutorials
3. Pre-recorded demo
4. Community page

Additionally, all users can contact us for support/bugs/feedback using the ? icon in the top right of the PostHog app. This is routed to the appropriate team in Zendesk.

Ensuring customers see value quickly

Most potential customers will show up because they want to replace an existing analytics product, or start doing product analytics from scratch. In either case, we should show them the power of PostHog as quickly as possible. To that end, getting live **production** data through our pipeline and available for analysis should be our top priority.

1. Help them get set up with tracking their production site/app using one of our client or server libraries.
    1. JavaScript / Autocapture is the easiest - also make sure to turn on Session Replay.
2. If people aren't sure what they want to track, AARRR is a great framework to use and will give people a good taster on the types on insight they can see. We have a number of supporting resources:
    1. A blog post on getting started with the framework.
    2. A sample AARRR tracking plan which we can give to customers to fill in. It shows how we do things at PostHog and may help inspire people who don't know how to get started.
3. Encourage them to create dashboards for them to show off PostHog in the wider organization.
4. Keep on top of any support requests / blockers they may have.

Free trials?

Generally speaking we don't need to do anything around free trials as our free tier has a generous 1m events, 1m feature flag calls, and 5k sessions. If a customer is going to go over this limit pretty quickly then we can agree to give them 2 weeks of free usage - this can be done in the billing service. See the billing page for more info (and the latest on this).

Figuring out the best solution for a customer

Assuming PostHog is the best solution for a customer, you should look at their level of scale and if they have any specific security needs to determine the most appropriate plan for them.

In general, PostHog Cloud is the best option for customers. It is much more scalable than self-hosted instances, doesn't require devops time to configure, monitor, and run, and is also the only way to use all of PostHog's paid features. In certain cases, the open source / free product may be the best choice, if customers are very technical, and also have a strong data control requirement.

What about Open Source?

Open Source will be appealing to customers who want to self-host, but are happy with 1 project only and community-based support.

By contrast, paid has premium features around collaboration - such as user permissions so people can't delete everything, multiple projects to keep data tidy, basically functionality to keep things running smoothly when you have lots of logins.

Okay, they're using PostHog. Now what?

Congratulations, this is the best part! Now we focus on making customers successful at unlocking insights into their product.

Read about how we do this in the dedicated handbook section, Ensuring Customer Delight at PostHog.

How we measure revenue

We typically use two top-level metrics when looking at revenue: MRR (monthly recurring revenue) and NRR (net revenue retention).

The easiest way to see these is on the go/revenue dashboard. These queries were built by Tim

FAQs

Can I give a customer a discount?

Again, no need - we already have usage-based pricing which is *heavily* discounted at higher volumes, and we only bill month-to-month, so customers don't need to feel locked in to a longer term contract. If it's high volume (B2C) we can do this on an ad-hoc basis.

How do I work with a customer who wants to sign an MSA?

This occasionally happens when we are dealing with very large companies, who may prefer to sign an MSA due to their internal procurement processes or to have the security of a locked-in contract from a pricing perspective. We have a contract version of our standard terms and conditions that we can use for this - ask Charles.

We'd only really look to do this with people spending $20k+ per year - we don't do it below this value because of the legal effort required.

How do I find out a customer's usage?

The tool we use for this currently is Pocus, which combines revenue, PostHog, HubSpot data all in one place. You can search for the org/user/domain using cmd + k and the popover should give a deep dive into usage across products, revenue, engagement, etc.

Can a customer transfer from self-hosted (e.g. Open Source) to Cloud?

Yes! See migration tools repo for events and the migrate meta repo for everything else.

Can a customer transfer from Cloud to Self-hosted?

Yes! Raise a support ticket in the app under Data Management.

What if the customer knows their user volumes but has no idea about number of events?

A good approach is to point them to our Downsampler app and set it to say only capture 1% of users. If they then go to their billing page, they can see the events count. Multiplying this by 100 will indicate their actual likely volume, without creating a ton of risk that they spend too much money.

We also did a study on PostHog Cloud and most companies were within the range of 50-100 per user per month.

What privacy features does PostHog offer?

- Self-hosting so no data needs to go to a 3rd party
- You can block Auto Capture on certain elements
- You can use PostHog without cookies
- You can mask IPs
- We make it trivial to delete a user's data if requested to do so

What apps are available?

We have the full list here. We also accept apps built by the community, which we audit first before adding to the list.

Procuring and selling via AWS

contents/handbook/growth/sales/selling-via-aws.md

PostHog is now available on AWS Marketplace for SaaS products. The way we've chosen to list our product is that it is not available as a public offering and instead is "listed" but only available via AWS "Private Offers", which means we create custom order forms for each customer through AWS that they accept via their portal.

AWS Marketplace lets vendors use their own terms and MSA. For now, PostHog team members set the price as a lump sum credit purchase for **annual pre-payment only**. Down the road, if we change our listing to public on the marketplace, we could set up usage-based billing through AWS (but that's future state).

Why this matters

1. **Our ICP lives in AWS** - Product engineers already have AWS access and budget. Adding PostHog to their AWS bill just makes sense since we're part of their product infrastructure stack
2. **Procurement bypass** - Organizations have bigger, more flexible AWS bills. Way easier to add a line item there than set up a whole new vendor
3. **Customer kickbacks** - Buyers get ~3% of purchase price back as AWS credits (sweet deal for them)
4. **TAM incentives** - AWS TAMs get SPIF'd for marketplace sales (we should apply to ISV Accelerate to fully capitalize on this)

Current requirements

For now, we're keeping it simple:

- **Annual contracts only** (upfront payment)
- **Minimum $100k deal size** (this is flexible, but let's start here)

Using Clazar for private offers

Since AWS Marketplace can be a pain to navigate, we're using Clazar to manage this. Clazar ties private offers directly to Salesforce (something AWS doesn't do natively). Future state: would be nice if QuoteHog could create these directly too!

Initial setup

Before you start:

- Make sure you have access to Clazar

- Have signed order form from customer with "AWS Marketplace" selected as the billing method
- Have the customer's AWS account ID ready

Creating a private offer via Clazar

Option 1: Direct from Salesforce (recommended)

1. **Open the opportunity** in Salesforce
2. **Navigate to the AWS Private Offers widget** (should be on the opportunity page)
3. **Click "Create Private Offer"** - Clazar pre-fills most fields from the opportunity
4. **Fill in the required fields:**
    - **Buyer AWS Account ID** (critical - double check this and it should be their management/root account ID)
    - **Offer Name** - Something clear like "PostHog Annual - [Company Name] - MM/YYYY"
    - **Contract Duration** - 12 months (we're annual only right now)
    - **Expiration Date** - Expiry on the offer itself. Usually 30 days out is fine.
5. **Configure pricing:**
    - Set as **upfront payment** (non-FPS offer)
    - Enter the negotiated price
    - Currency: USD (can do EUR, GBP, JPY if needed)
6. **Choose EULA type:**
    - Use **Standard Contract for AWS Marketplace** unless legal says otherwise
    - If custom EULA needed, upload the PDF (max 5 docs)
7. **Review and submit** - Takes ~45 minutes to generate in AWS (yes, really takes that long sometimes)

Option 2: Via Clazar platform

If you need more control or Salesforce isn't cooperating:

1. **Log into Clazar** at app.clazar.io
2. **Navigate to Private Offers** in the main menu
3. **Click "Create New Private Offer"**
4. **Fill in buyer details:**
    - Company name
    - AWS Account ID (must be exact)
5. **Configure offer details:**
    - Friendly offer name
    - Expiration date (when offer becomes void)
    - Contract duration
    - Start date (first service day if net new, day of renewal otherwise)
    - Offer type: Choose "Contract" with upfront payment
6. **Set dimensions and pricing:**
    - Add your product dimensions

    - Set prices for each dimension
    - For annual deals, configure as single upfront payment
7. **Legal terms:**
    - Select EULA type (Standard Contract or Custom)
    - Upload any additional documents if needed
8. **Review the status tab** - Should be green when ready
9. **Submit the offer**

After creating the offer

1. **Wait for generation** (~45 minutes)
2. **Clazar sends notifications** when the offer is live
3. **Share with customer:**
    - Send them the private offer URL
    - Include the Offer ID for reference
    - Remind them to log into the AWS account specified
4. **Track in Salesforce** - Status syncs automatically via Clazar

Common issues & solutions

Customer can't see the offer:

- They're not logged into the right AWS account
- Offer expired (check expiration date)
- Wrong AWS account ID used (most common issue)

Offer needs changes after creation:

- Can't edit accepted offers
- Create an Agreement-Based Offer (ABO) for modifications
- Customer accepts ABO, which cancels the previous agreement

Payment not showing up:

- AWS disbursements take time (check the disbursement schedule)
- Verify the offer was actually accepted in AWS

Pro tips

- **Double-check AWS Account IDs** - This is where most mistakes happen
- **Set realistic expiration dates** - 30 days is standard, but adjust based on deal timeline
- **Keep offers simple** - Complex payment schedules = more room for error
- **Document everything in Salesforce** - Let Clazar sync do the heavy lifting

Shared Slack Channels with Customers

contents/handbook/growth/sales/slack-channels.md

We offer shared Slack channels to customers and prospective customers in several circumstances:

- Prospective, new customers can have a shared Slack channel for the duration of a trial period, and keep a shared Slack channel after the trial if they qualify with at least $20k in annual, committed spend OR a subscribe to a support package which includes a shared Slack.
- Product-led customers can earn a shared Slack channel by growing beyond $20k in annualized spend.
- Existing customers can earn a shared Slack channel by committing to $20k in annual spend OR growing beyond $20k in annualized spend.

We use shared Slack channels to provide timely support and to build relationships with those at our customers shipping things with PostHog.

Shared Slack channels allow many folks at PostHog to support our customers. And, a shared Slack channel must be configured correctly in order for this support to work.

Setting up a Shared Slack Channel via Slack Connect

We use Slack Connect to share Slack channels with our customers.

To get a shared Slack channel going, follow these steps:

1. Create a new Slack channel - the expected syntax for the name is `posthog-[customername]`.
2. When determining the `[customername]`, make sure to make it searchable (avoid acronyms, if possible).
3. Obviously, invite the relevant customer folks! Be sure that you're inviting them to the *channel* you've created and not our Slack workspace.
4. Invite certain leaders who want to help monitor the channel, including: Tim, Charles, Abigail, Simon, your team lead and anyone else internal who may be connected to the customer. PostHog folks will sometimes join the channel if they're interested in the customer or the use-case
5. Invite Pylon to ensure those from PostHog and the customer can create support tickets from Slack threads - Use the a slash command in Slack to invite Pylon `/invite @pylon` . Pylon will join and prompt you with some questions. Note that this is a `customer channel`, and select yourself as the channel owner. If your name is not available in the dropdown, you can login to Pylon and add yourself as the owner from the Pylon UI. You can check to see if the connection is established in the "Account Mapping" section in Pylon.
6. Set your preferences to "Get notifications for all messages" in the channel -- this will ensure you don't miss a message and allow for speedy support.

7. Ensure that the Slack channel name is recorded on the relevant Salesforce Account record in the `Slack Channel` field (Pylon should sync this automatically) -- If the `[customername]` in Slack is different from the Account record name in Salesforce, Pylon will not automatically match the two.
8. Grab the Admin Panel link (from Vitally under PostHog Default Dashboard) and in the channel add this as a new link. Name it Org Link and add a new folder called Support. This is helpful to our Support Team for quickly accessing the customer's account when questions are posted in Slack.
9. Add your role and title to the channel description (e.g. Technical CSM: FirstName LastName). This will help team members identify who's the main point of contact for this customer.

If you have any questions as you go, ping your colleagues for support in your team channel.

Editing a pylon integration with Slack

If you accidentally set the wrong channel for a feed or mess up some other pylon settings, you need to log into the Pylon admin to change it. You can SSO via Slack, just put in your PostHog email address. Once logged in, click on accounts in the left rail, find the account you need to change, click on it, and on the right rail, you will see Slack integration settings.

Using MS Teams via Pylon

Some customers may wish to use MS Teams rather than Slack - we can sync our Slack with Teams via Pylon to do this. First you will need an MS Teams licence - ask Simon, or Dana for one. Then, set up a Slack channel according to the instructions above. Then, follow these steps:

1. In Teams go to "See all your teams" and then "Create team". When naming the team the expected syntax is `[CustomerName-PostHog]. Make sure to set the team type to Public and name the first channel "Shared" then finish creating the team.
2. Now, go to the team you created and go to the Apps tab. Click "Get more apps" and open the Posthog Team app that's under "Built for your org". Select the Shared channel from your newly created team.
3. On this page in Pylon you should see your new team listed, using the search dropdown to connect it to the Pylon account associated with the customer.
4. Before adding the customer into the Teams, remember to test it on both sides to ensure the integration is working correctly. After you test it, invite the relevant customer folks by adding them as members to the team!

Onboard Your Customer to Slack Support

Welcome them to the channel when they join!

Set context for the channel's purpose and timing (if applicable). Let them know that they may hear from anyone at PostHog who is monitoring the channel, and also don't miss the opportunity to train them how to open a ticket with the Pylon app.

A message like this one does wonders to help them understand how to open a ticket if you're not online to help yourself:

> We also have an app here that will open a ZenDesk ticket if I'm sleeping. You only have to add the emoji to the thread and it will open a ZenDesk ticket automatically, and capture the back and forth in the specific Slack thread that received that emoji. You can also `@support` in a thread to open a ticket as well. It's a good habit to get into in order to make sure our distributed team can help.

The New sales playbook has more on ensuring that the customer is set up for success.

TAM excellence

contents/handbook/growth/sales/tam-excellence.md

A question we often get asked is: "What makes an excellent TAM?" This touches almost every stage of the role, from candidates (what would I have to do to succeed at this potential role?) to new hires (where should I be spending my time?) to folks in their first year (how do I know I'm doing well?) to old-timers (how do I keep up with the rest of the team?)

An excellent TAM:

General Principle	Specific Examples
Distinguish yourself from other vendors with personalized outreach	Record personalized videos, submit PRs for a customer's software, send personalized food or merch, sign up for their product, invite them to events, or make donations in their name
Dig into a customer, find what will be valuable to them, and surface it in a way that deepens the relationship	Get the content right — saving money, expert recommendations, ideas for success beyond PostHog — and deliver it in a way that earns attention
Take ownership of a customer problem and see it through to resolution	Even when a full fix isn't possible, owning the issue and driving it to conclusion builds trust — people notice when you see things through
Build relationships for the long term so that today's work pays off a year from now	Don't write off a quiet or unresponsive customer — the relationship being built now is for next year's expansion, not this quarter's
Balance cross-sell with non-sales value so customers feel helped, not sold to	Avoid making every interaction a revenue conversation — give them value that doesn't require opening the wallet
Show sincere interest in the customer's business and back it up with real knowledge	Congratulate them on product launches, give feedback on their product, leave them reviews — and actually know their product well enough to do so
Have the technical depth to get hands dirty and help with technical questions	You shouldn't be implementing for customers as a rule, but being capable of it demonstrates you understand enough to be genuinely useful
Don't accept "we're good" as a final answer — keep engaging to find where you can help	"Talk to us in 6 months" usually means "don't upsell me" — respond with concrete suggestions on cost reduction or real pain points instead
Regularly share learnings — both wins and failures — publicly with the team	When you learn something (especially from a mistake), sharing it helps others avoid the same issues and raises the whole team's effectiveness

General Principle	Specific Examples
Develop a sense for when an account is at risk and act proactively	Be close enough to your accounts that you can detect when something feels off, and address it before it becomes a problem
Balance directness and transparency with knowing when to give a customer space	Understanding what the right balance looks like for each individual customer is the art of the role
Spot gaps in the sales process and proactively fix them rather than complain	An excellent TAM doesn't sit on process frustrations — they take ownership and make improvements
Identify the customer's key business goals and align PostHog directly to those outcomes	If a customer wants to increase conversions or grow premium plans, show specifically how PostHog helps reach that goal — make it essential, not just a nice-to-have
Enable and upskill your customer champion so they look good within their org	Do the hard work for your champion, then let them take all the credit
Be a relentless advocate for customer interests internally	TAMs are closest to customers — engineers need to hear from you about how customers are actually feeling
Continuously grow your PostHog expertise to keep pace with the product	PostHog is constantly changing — never shy away from selling a new product because of unfamiliarity; learn it

Team lead responsibilities

contents/handbook/growth/sales/team-leads.md

General principles

As team lead in a customer-facing role you'll be responsible for making sure that your team is exceeding expectations when it comes to their specific role at PostHog. This generally means that:

1. They have a solid plan for any managed customers in their book of business or deals in their pipeline
2. They are proactively building relationships with their customers, even those who are hard to engage with
3. They are flagging any potential churn as soon as they become aware of it
4. You are proactively helping them when they are struggling with what to do next on a customer or deal
5. You are providing continuous feedback to them, especially when their performance is below expectations

Team-specific responsibilities

Product-led Sales

Technical Account Managers (TAMs) own a book of business of nominally around 15 customers with an ARR of $1.5m, and also look to bring new customers into that book via product-led leads.

- Once a TAM has hit 15 managed customers or ARR of $1.5m, we should stop generating new product-led leads for them to allow them to focus on and grow their existing book. Flag this to Simon (Mine as backup) when this needs to happen.
- If a TAM's book of business is too big then your first port of call should be to balance customers across your team, failing that ask for help from your peers in other teams (Simon as backup).
- TAMs will want to bring new leads into their book of business so that they count towards their quota. It's your job to make sure that they have a solid relationship with the customer and a plan in place for growing them too. Once you're happy, then let Simon know who will review and add them to the quota tracker.
- With big customers over $100k in ARR you should be prepared to lean in and help the TAM with that customer more directly, owning different levels of the relationship.
- Take the lead in driving churn risk and cross-sell team calls, ensuring that we stick to planning and next steps rather than storytelling.
- Ensure that TAMs are on top of any credit renewals well before they expire.
- Check the TAM quota tracker to see if there are any discrepancies and encourage your team to do so regularly as well.

- Coming up to the end of the quarter, work with your team to identify any customers that are ready to be owned by a Customer Success Manager. Review these with Simon ahead of quarter end so that we can make for a clean transition of customers.
    - Set the New Owner trait in vitally to `EU CSM` or `US CSM` based on geography.

Customer Success

Customer Success Managers (CSMs) own a book of business of around 30 customers with an ARR of $1.5m and focus mainly on keeping them as customers (retention).

- Once a CSM's book is full, work to balance customers across the team ideally with a reasonable timezone overlap with the customer.
- Take the lead in driving churn risk team calls, ensuring that we stick to planning and next steps rather than storytelling.
- Ensure that CSMs are on top of any credit renewals well before they expire.

Onboarding

The Onboarding team operates at scale, supporting hundreds of customers whose MRR falls below the TAM/CSM threshold. As a result, the number of customers in the program, as well as the ARR represented, can fluctuate from month to month. The team is currently focused on customers whose first bill is forecasted at $500+ MRR. Its north star metric is maintaining 90% logo retention through customers' first three months.

- Make sure the Onboarding Specialists prioritize work effectively, engage with customers ahead of renewal, and prevent accounts from falling through the cracks.
- Ensure the Onboarding Specialists adapt to customer needs, continue providing value, and maintain a high-quality bar.
- Lead the team's continuous improvement efforts across internal processes and the overall onboarding program.
- Ensure high-spend accounts are properly qualified for Sales handoff and that the handoff process runs smoothly.
- Monitor team coverage and customer volume to identify and flag hiring needs proactively.

Other things we collectively need to stay on top of

Tracking new products

When we know that a new product is being launched, we need to ensure that Vitally tracking is in place for that product before it is launched. This involves:

1. Updating the Postgres integration to ensure that we are tracking the following traits for the product:
    - `product_name_ltv` added to the `completed` CTE - the lifetime value of the product

    - `product_name_forecasted_mrr` added to the `forecasted` CTE - the MRR forecasted for the product
    - `product_name_last_month` added to the `last_month` CTE - the last month payment for the product
    - `product_name_billing_limit` added to the main query - the current billing limit for the product
    - also add the traits from the 3 CTEs above into the main query
2. Once the traits are in place, create a success metric to capture the product's data usage if applicable.
    - Sum of a property on events -> `billing usage report` -> `product_count_in_period` over the last 30 days
3. If there is a specific engagement to track how people use the product in PostHog add it to the Vitally engagement events Action
    - If you do this, also ensure there is a corresponding success metric for this (Total event count over 14 days)

Incident comms

We need to ensure that teams are able to proactively follow our incident comms process. We're not quite ready for a full on-call rotation yet, but Simon or Dana take the lead as Communications Manager On-Call (CMOC) when an incident is declared in EU working hours, and Landon or Tyler take the lead when it's during the US working day.

Trials

`contents/handbook/growth/sales/trials.md`

Prerequisites

- Customer needs to have an Organization set up on the EU or US Cloud
- You need access to the billing admin (billing.posthog.com). Ask Raquel or Simon for access.

Process for giving a customer a free trial:

1. Log in to Billing with your Google SSO login.
2. Click the `Trials` link on the left sidebar.
3. Click the `Add Trial` button (top right).
4. Fill out the trial form

- Customer: search for customer by Organization Name or ID
- Status: set to `Active`
- Target: set to whatever is needed (paid, teams, enterprise)
- Type: set to `Standard`
- Expiration date: set it to whatever is needed (2 weeks, 4 weeks for larger ($100k+) customers, etc)
- Check `Silence notifications` if you don't want them to get trial notifications

5. Click `Save`
6. The next time that Customer visits PostHog, their `AvailableFeatures` will be updated to reflect the standard premium features (they might have to refresh their page to properly sync the new billing information).
7. Once this date passes their `AvailableFeatures` will be reset to the free plan unless they have subscribed within this time.

Additional steps for existing customers with paid subscriptions For customers with existing paid subscriptions we need to complete additional steps to make sure they are billed correctly.

Important: Ask Mine to update Stripe and billing admin so she can make sure revenue numbers are unaffected and customer isn't billed while on trial.

1. Follow the steps above to create a trial.
2. Remove Stripe Subscription ID in the Billing Admin (keep the Stripe Customer ID).
3. Set all products in the product map to a free status.
4. Cancel subscription in stripe: Ensure the subscription is canceled in Stripe so they are not billed during the trial.
5. Create new subscription before trial ends and update Billing Admin so customer experience isn't affected when transitioning back to a paid plan.

If they need a shared Slack channel as part of the trial, follow these instructions.

Consider framing a collaborative method for progressing in the trial period with timed objectives. If it's new and depending on the level of engagement, we can use a detailed success plan.

Turning knowledge into agent skills

contents/handbook/growth/sales/turning-knowledge-into-agent-skills.md

Our documentation is a critical piece of PostHog's **context flywheel** – a system that connects our codebase to our docs, which then feeds into our AI agents, the Wizard, and PostHog Code. When documentation is outdated, the agents that help customers integrate PostHog become outdated too.

This means your knowledge directly powers our AI tools. When you write down what you know, it doesn't just help humans – it helps robots help customers faster.

Why your contributions matter

The team has automated writing documentation from PR merges using InKeep, which indexes our codebase and docs to create first-pass drafts. But there's knowledge that only comes from working directly with customers:

- Common integration patterns and gotchas
- Real-world use cases and configurations
- Cross-selling playbooks and discovery questions
- Troubleshooting steps for edge cases

This knowledge lives in your head. When you write it down in the handbook, it can be transformed into **skills** – portable packages of context that the AI Wizard can use to help customers.

How to contribute

1. Write it in the handbook first

The handbook is the appropriate place to document playbooks, processes, and tribal knowledge. We have Markdown rendering for both the documentation and handbook, so content can flow between them.

Good things to document:

- How you help customers solve specific problems
- Discovery questions that uncover customer needs
- Common configurations you walk customers through
- Troubleshooting steps for issues you see repeatedly

2. Make it actionable

The context mill transforms handbook content into skills for the AI Wizard. To make your content skill-ready:

- **Be specific**: Include actual steps, not just concepts

- **Show examples**: Real code snippets and configurations help
- **Explain the "why"**: Context about when to use something helps the AI apply it correctly
- **Use clear structure**: Headers, bullet points, and numbered steps work better than walls of text

For example, the Logs skill is just a root prompt with some reference material – it's that simple.

3. Think about automation

When you write something down, ask: "Could an agent do this automatically?"

For example, if you write a playbook for gathering a customer dossier, that could become an automated web search agent task. If you document how to identify cross-sell opportunities, that becomes a skill the Wizard can use.

What gets turned into skills?

Skills are defined using a YAML specification that allows for different variants based on app detection and context. The team currently has over 60 skills the Wizard can use.

Your handbook contributions can become skills that:

- Help customers integrate specific products
- Run migration analyses from competitors (Amplitude, Sentry, etc.)
- Diagnose common issues
- Generate tracking plans based on customer needs

The Wizard and how it helps customers

The AI Wizard is a one-line `npx` command that runs an agent to integrate PostHog:

```
npx -y @posthog/wizard@latest
```

Here's what it does automatically:

- Instruments multiple products like Product Analytics, Web Analytics, Session Replay, Error Tracking, and others
- Installs the right SDKs for their stack
- Scans their codebase to understand their product
- Creates 10-15 custom events based on product flows it identifies
- Writes both client-side and server-side code for full-stack implementations
- Creates an insight and dashboard in PostHog

This dramatically reduces manual integration work – what might take 3-5 hours happens in minutes. And it produces customized code tailored to each customer's setup.

The Wizard is agentic software that runs on our docs. When you write something down, the Wizard can execute it as a skill – it's like turning documentation into executable code.

So ask yourself: if an agent can read, analyze, and understand a user's codebase, what else could it discover or build to help the user get value from PostHog faster?

Those answers can become Wizard skills – and new ways of creating customers.

How this helps you help customers

Close the gap in customer conversations

The Wizard and skills architecture lets you close the gap between customer-facing hypotheticals and technical diagnostics without needing engineering present. If a customer asks "how would I track X?", you can point them to the Wizard or a specific skill.

Portable diagnostics

If customers are hesitant to run an agent on their codebase, they can receive the open-source skills package directly. This gives them 80-95% of the Wizard's functionality to run locally with their own tools.

Better onboarding

For new customers, the Wizard provides an excellent launchpad – 10-15 best-practice events that help them overcome the initial difficulty of deciding what to track. The best approach is to:

1. Run the Wizard first
2. Review what it did together with the customer
3. Come up with a plan and tweaks

The Wizard helps you get past the blank page. It's much easier to iterate from there!

Getting started

1. **Identify something you repeat**: What do you explain to customers often?
2. **Write it down**: Create a handbook page or add to an existing one
3. **Make it specific**: Include actual steps, code, or configurations
4. **Let the team know**: Share in Slack that you've added something that might make a good skill

The written-down knowledge also enables automation beyond the Wizard – like having agents gather customer dossiers based on your specifications or analyze competitor implementations before a call.

Your knowledge is valuable. Write it down, and it becomes executable.

User event streams

contents/handbook/growth/sales/user-event-streams.md

Using PostHog's data pipelines (CDP), you can create a real-time feed of customer activity directly in Slack. This lets you monitor how users in your book of business are engaging with PostHog without constantly checking dashboards or running queries.

This is valuable for getting a pulse on account health and engagement patterns. You'll see who's actively using the product, which features they're exploring, and when they might be hitting friction points. It's not meant to replace proper data analysis, but it gives you the "vibes" and can help you time your outreach more effectively. For example, if you notice someone reading a lot of feature flag docs and then creating several flags, you know they're actively working on something and might appreciate a quick check-in.

A word of caution: don't be a creep about this. Use it to inform when and how you reach out, not to surveil every click. If you notice someone's activity suggests they need help, check in naturally without revealing you're monitoring their every move.

How to set up your event stream

1. Get your account organization IDs

Query PostHog's data warehouse to pull Salesforce data and create a CSV of all PostHog organization IDs for accounts you own. This gives you the list of orgs to monitor.

2. Create a CDP destination

Set up a new data pipelines destination using a webhook endpoint. This is where you'll send the filtered events.

3. Filter for your accounts

Configure the destination to filter for all events where the organization ID matches your CSV list of org IDs. This ensures you only see activity from your accounts.

4. Select relevant events

Add filters for events that represent meaningful user actions across product areas:

Product analytics & insights:

- insight created
- cohort created
- action created

- annotation created
- dashboard created

Session replay:

- recording analyzed
- viewed recordings from experiment

Feature flags & experiments:

- feature flag created
- experiment created
- experiment launched
- experiment completed
- experiment viewed

Surveys:

- survey launched

AI & Max:

- chat with ai
- AI generation (LLM)
- ai_hog_function_prompted
- ai_hog_function_accepted
- sql-editor-accepted-suggestion
- LLMa events for seeing user prompts

Data pipelines:

- batch export enabled
- batch import created
- export succeeded

Error tracking:

- error_tracking_issue_created

Engagement & product intent:

- person viewed
- user showed product intent
- Pageview (where url contains "docs")
- toolbar mode triggered

Billing & account health:

- billing product activated
- billing limits updated
- Hit billing limit
- billing addon removed
- billing subscription paid
- billing subscription cancelled
- billing invoice payment failed
- annual plan credit purchase
- billing trial activated
- autocapture_opt_out team setting updated
- session_recording_opt_in team setting updated
- autocapture_exceptions_opt_in team setting updated

Team growth:

- team member invited

5. Customize the payload

Modify the webhook payload to include:

- User email
- Event name
- Current URL
- Any other properties valuable for your real-time stream (e.g., organization name, event properties)
- You can link right to replays (docs), just know they may not be available immediately

6. Route to Slack

Send the data to a Slack App endpoint, Zapier, or Relay.app to transform and redirect the events to your personal channel like `your-name-alerts` or `your-name-user-event-stream`.

What you'll get

A real-time feed of user activity that helps you:

- Identify active power users and champions
- Spot when accounts are exploring new features (cross-sell opportunity)
- Notice declining engagement patterns early
- Time your outreach when users are actively working on something
- Get a general sense of account health without diving into dashboards

Remember: this is supplementary context, not a replacement for proper account analysis and data review.

Matching PostHog to a business type

contents/handbook/growth/sales/utilization-by-business-type.md

This guide provides detailed instructions on how to achieve key business metrics using PostHog. Each business type has specific metrics that matter most, and this guide shows you exactly how to set up PostHog to track and optimize for those metrics.

B2B SaaS

Common business problems & personas

B2B SaaS companies often grapple with a core set of challenges that directly impact growth and sustainability:

Key business problems

- **High churn rates** – Customers discontinuing subscriptions, leading to revenue loss and reduced customer lifetime value.
- **Low trial-to-paid conversion** – Users not converting from free or trial plans to paid subscriptions.
- **Poor feature adoption** – Users not utilizing key features that drive product value and stickiness.
- **Long sales cycles** – Extended time from initial lead engagement to customer conversion.
- **Low customer satisfaction** – Reflected in poor Net Promoter Scores (NPS) and negative customer feedback.
- **Inefficient onboarding** – Users dropping off or struggling during initial setup and product adoption.
- **Expansion revenue challenges** – Difficulty identifying opportunities or successfully upselling and cross-selling existing customers.
- **High support ticket volume** – An elevated number of support requests, often indicating underlying product issues or user friction.

Primary personas & their pain points

Product managers

- **Pain points:** Inability to identify which features drive retention, difficulty prioritizing roadmap items, lack of data on user behavior and product usage.
- **PostHog solutions:** Comprehensive feature usage tracking, granular cohort analysis, session recordings for in-depth UX insights, robust A/B testing for feature optimization and validation.

Customer success managers

- **Pain points:** Reactive churn management, challenges in proactively identifying at-risk customers, limited visibility into overall customer health.
- **PostHog solutions:** Data-driven churn prediction models, customizable customer health scoring, proactive engagement tracking, automated alerts for at-risk accounts based on behavioral signals.

Sales teams

- **Pain points:** Extended sales cycles, inefficient lead qualification processes, difficulty understanding specific prospect needs and product fit.
- **PostHog solutions:** Product usage-based lead scoring, detailed prospect behavior tracking, optimization of conversion funnels to accelerate deal velocity.

Marketing teams

- **Pain points:** High customer acquisition costs (CAC), inaccurate campaign attribution, difficulty measuring the true return on investment (ROI) of marketing efforts.
- **PostHog solutions:** Advanced UTM tracking, comprehensive conversion funnel analysis, cohort analysis by acquisition source, customizable campaign performance dashboards.

Executives

- **Pain points:** Lack of holistic visibility into business health, challenges in making data-driven strategic decisions, cumbersome stakeholder reporting.
- **PostHog solutions:** Intuitive executive dashboards, real-time key metric tracking, automated reporting, and actionable business intelligence insights.

Key metrics & PostHog

MRR/ARR (monthly/annual recurring revenue)

Importance: Measures the predictable revenue a SaaS business generates monthly or annually. It's crucial for forecasting, valuation, and understanding the company's financial health and growth trajectory.

PostHog approach: Track subscription events (subscription_created, subscription_upgraded, etc.) with properties like plan_tier, amount, and currency. PostHog helps analyze conversion funnels (e.g., trial_started to subscription_created), visualize revenue retention with cohort analysis on dashboards, and set up alerts for significant MRR changes. For non-technical users, autocapture on pricing pages and CTAs can power no-code funnels and session recordings to optimize conversion flows and pricing interactions.

CAC (customer acquisition cost)

Importance: The average cost to acquire a new customer. Understanding CAC is vital for marketing efficiency, profitability, and ensuring sustainable growth.

PostHog approach: Track marketing touchpoints (ad_clicked, demo_scheduled) and lead generation form submissions with properties like source, campaign, and UTM parameters. Integrate marketing spend data into PostHog for a unified view. Use funnel analysis to identify efficient acquisition channels and dashboards to visualize CAC trends by channel. Autocapture can track landing page visits and form submissions, enabling non-technical users to analyze lead quality by traffic source and optimize landing page UX with session recordings.

LTV (lifetime value)

Importance: The total revenue a business expects to generate from a single customer relationship over their lifetime. A high LTV indicates strong customer relationships and product value, enabling higher CACs and more aggressive growth strategies.

PostHog approach: Track all revenue-generating activities (subscription_payment, addon_purchase, upgrade) with customer segment and acquisition properties. Conduct cohort analysis for revenue retention and correlation analysis to identify high-value behaviors. PostHog's predictive analytics can forecast LTV. For non-technical users, autocapture can track feature usage and upgrade page visits to understand engagement patterns that correlate with high LTV, allowing for dashboards showing feature adoption by segment and alerts for potential churn signals impacting LTV.

Churn rate

Importance: The rate at which customers cancel their subscriptions or cease to use a service. High churn is detrimental to growth and directly impacts MRR/ARR and LTV, highlighting product-market fit or customer experience issues.

PostHog approach: Monitor engagement and usage patterns (feature_used, login, session_started) with properties like user_activity_level and feature_adoption. Use session recordings to understand behavior of churned users and correlation analysis to pinpoint churn indicators. Set up automated churn prediction models and alerts for at-risk users. Non-technical users can leverage autocapture to track declines in activity, analyze pages churned users stop visiting, and use session recordings to review churned user journeys.

NPS (net promoter score)

Importance: A widely used metric to gauge customer loyalty and satisfaction, indicating a customer's willingness to recommend a product or service. High NPS often correlates with retention and expansion revenue.

PostHog approach: Implement in-app NPS surveys using PostHog's survey feature. Track nps_survey_submitted events with user segment and usage properties. Analyze correlations between NPS and product usage patterns. Non-technical users can easily create surveys, configure triggers, and track completion rates. Dashboards can show NPS trends by segment, and session recordings can analyze user interactions with survey prompts to optimize feedback collection.

Feature adoption

Importance: Measures the extent to which users discover, use, and continue to use specific product features. High feature adoption indicates that users are deriving value, which is crucial for retention, upsell opportunities, and validating product development efforts.

PostHog approach: Track granular feature usage (feature_accessed, feature_completed) with feature name and user segment properties. Use funnel analysis for onboarding flows and session recordings to identify friction. Implement feature flags for controlled rollouts and A/B testing for optimization. Non-technical users can use autocapture for feature page visits and button clicks, analyze user journeys to feature discovery, and create dashboards for adoption rates. Alerts can be set for changes in feature usage.

B2C SaaS

Common business problems & personas

Key business problems

- **High user churn** – Consumers canceling subscriptions after initial excitement
- **Low activation rates** – Users not completing key onboarding steps
- **Poor user engagement** – Users not returning to use the product regularly
- **High customer acquisition costs** – Expensive to acquire individual consumers
- **Low viral coefficient** – Users not referring friends and family
- **Poor mobile experience** – Mobile users having difficulty with the product
- **Seasonal usage patterns** – Inconsistent usage throughout the year
- **Difficulty scaling support** – High volume of individual user support requests

Primary personas & their pain points

Product Managers

- **Pain points:** High user churn, low activation rates, poor user engagement, difficulty understanding consumer behavior
- **PostHog solutions:** User behavior analysis, activation funnel optimization, engagement tracking, consumer journey mapping

Growth Teams

- **Pain Points:** High CAC, low viral coefficient, poor user acquisition, difficulty scaling growth
- **PostHog Solutions:** CAC analysis, viral coefficient tracking, user acquisition optimization, growth loop identification

Customer Success Teams

- **Pain Points:** High support volume, poor user satisfaction, difficulty scaling support, low user retention
- **PostHog Solutions:** Support ticket analysis, user satisfaction tracking, automated support optimization, retention analytics

Marketing Teams

- **Pain Points:** Poor campaign attribution, high CAC, ineffective user acquisition, seasonal usage challenges
- **PostHog Solutions:** Campaign attribution tracking, CAC optimization, user acquisition analysis, seasonal trend identification

Mobile Teams

- **Pain Points:** Poor mobile experience, low mobile engagement, mobile-specific bugs, app store optimization
- **PostHog Solutions:** Mobile experience analysis, mobile engagement tracking, mobile bug monitoring, app store performance analytics

Key metrics & PostHog

User Activation Rate

Importance: Measures the percentage of new users who complete key onboarding steps and experience the product's core value. High activation is crucial for retention and indicates a successful onboarding experience.

PostHog approach: Track activation events (`account_created`, `onboarding_completed`) with properties like activation step and acquisition source. Use funnel analysis to optimize time-to-value, and cohort analysis to track activation rates on dashboards. Session recordings can help identify activation friction points, and alerts can be set for activation rate drops. Non-technical users can use autocapture for onboarding page visits and tutorial interactions to create no-code funnels and analyze user behavior.

Daily/Monthly Active Users (DAU/MAU)

Importance: Measures user engagement and product stickiness by tracking the number of unique users who interact with the product on a daily or monthly basis. A high DAU/MAU ratio indicates strong, consistent user value.

PostHog approach: Track user activity events like `session_started` and `feature_used` with properties such as user segment and session duration. Create dashboards for real-time DAU/MAU tracking and trend analysis. Calculate stickiness (DAU/MAU ratio) and use cohort analysis to track engagement over time. Alerts can be configured for significant engagement drops. Autocapture can track page visits and feature interactions, enabling non-technical users to analyze engagement patterns and identify popular features.

Customer Lifetime Value (CLV)

Importance: Represents the total revenue a business can expect from a single customer account throughout their relationship. CLV is a key indicator of long-term profitability and customer loyalty.

PostHog approach: Track all revenue events (`subscription_started`, `purchase_made`) with properties like purchase amount and acquisition source. Use cohort analysis to analyze CLV by acquisition month and correlation analysis to identify high-value behaviors. PostHog's predictive analytics can be used for CLV forecasting. For non-technical users, autocapture on purchase pages and upgrade buttons helps track the user journey to purchase and identify which features drive upgrades, with session recordings providing insights into purchase behavior.

Viral Coefficient

Importance: Measures the number of new users an existing user generates, indicating the effectiveness of viral loops and word-of-mouth growth. A coefficient greater than one signifies exponential growth.

PostHog approach: Track viral events like `referral_sent` and `invitation_accepted` with properties such as referral type and conversion rate. Use funnel analysis to optimize referral flows and A/B test referral incentives and messaging. Dashboards can show viral coefficient trends. Non-technical users can use autocapture to track share button clicks and referral page visits, using session recordings to understand and optimize referral behavior.

User Retention Rate

Importance: The percentage of users who continue to use the product over a given period. It's a critical metric for sustainable growth, reflecting long-term product value and user satisfaction.

PostHog approach: Track retention events like `user_returned` and `session_started`. Create retention dashboards with cohort analysis by acquisition source to track trends over time. Use session recordings to understand the behavior of retained users and correlation analysis to identify key retention-driving features. Set up automated alerts for retention drops. Autocapture allows non-technical users to track user return patterns and feature usage that correlates with retention.

Mobile App Performance

Importance: Measures the responsiveness, stability, and overall user experience of a mobile application. Good performance is essential for user satisfaction and retention on mobile devices.

PostHog approach: Track mobile-specific events like `app_opened` and `app_crashed` with properties such as app version and device type. Use PostHog's real user monitoring for performance and Core Web Vitals tracking. Create mobile performance dashboards, set up crash monitoring with alerts, and use session recordings to identify mobile-specific UX issues. Non-technical users can leverage autocapture to track mobile interactions and compare mobile vs. desktop usage patterns.

E-commerce

Common business problems & personas

Key business problems

- **High cart abandonment rates** – Customers adding items but not completing purchases
- **Low conversion rates** – Visitors not converting to customers
- **Poor product discovery** – Customers unable to find products they want
- **High return rates** – Products being returned frequently
- **Seasonal inventory issues** – Over/under stocking during peak periods
- **Poor mobile experience** – Mobile users having difficulty shopping
- **Low customer lifetime value** – Customers making only one purchase
- **Ineffective marketing attribution** – Difficulty tracking which campaigns drive sales

Primary personas & their pain points

E-commerce Managers

- **Pain points:** Can't identify why customers abandon carts, struggle to optimize product pages, lack visibility into customer journey
- **PostHog solutions:** Cart abandonment funnels, session recordings for UX insights, conversion rate optimization, customer journey mapping

Marketing Teams

- **Pain Points:** Poor campaign attribution, difficulty measuring marketing ROI, ineffective retargeting campaigns
- **PostHog Solutions:** UTM tracking, conversion funnel analysis, cohort analysis by traffic source, retargeting audience creation

Product Teams

- **Pain Points:** Poor product page performance, low product discovery, ineffective search functionality
- **PostHog Solutions:** Product page heatmaps, search behavior tracking, product recommendation optimization, A/B testing for product pages

Customer Service Teams

- **Pain Points:** High support ticket volume, difficulty understanding customer issues, poor customer satisfaction
- **PostHog Solutions:** Customer journey analysis, session recordings for issue identification, customer satisfaction tracking, support ticket correlation

Inventory Managers

- **Pain Points:** Poor demand forecasting, seasonal inventory issues, over/under stocking
- **PostHog Solutions:** Product performance tracking, demand pattern analysis, seasonal trend identification, inventory optimization insights

Key metrics & PostHog

GMV (Gross Merchandise Value)

Importance: Represents the total value of all goods sold over a specific period. GMV is the primary measure of an e-commerce platform's scale and is essential for understanding top-line growth and market share.

PostHog approach: Track all purchase events (`product_viewed`, `add_to_cart`, `purchase_completed`) with properties like product category, price, and quantity. Connect to your e-commerce platform for comprehensive data. Create dashboards for real-time GMV tracking and product performance analysis by category. Use cohort analysis to track customer value over time and set up alerts for unusual GMV patterns. For non-technical users, autocapture on product pages and "add to cart" buttons can track the conversion journey and identify popular products, with session recordings helping to optimize product pages.

AOV (Average Order Value)

Importance: The average amount spent each time a customer places an order. Increasing AOV is a key strategy for maximizing revenue without increasing the number of customers, directly impacting profitability.

PostHog approach: Track cart and purchase events (`cart_updated`, `purchase_completed`) with properties like cart value and discount applied. Use funnel analysis to optimize the cart and identify abandonment points. A/B test pricing and product recommendations to find effective upselling strategies. Use correlation analysis to identify behaviors of customers with high AOV. Non-technical users can use autocapture to track interactions on the cart page, analyze abandonment patterns, and use session recordings to optimize the checkout flow.

Conversion Rate

Importance: The percentage of visitors who complete a purchase. This is a critical metric for gauging the effectiveness of the entire customer journey, from landing page to checkout, and is a primary indicator of site performance and user experience.

PostHog approach: Track all steps in the conversion funnel (`page_viewed`, `product_viewed`, `add_to_cart`, `checkout_started`, `purchase_completed`) with properties like traffic source and device type. Create comprehensive conversion funnels to identify drop-off points and use session recordings to understand checkout friction. A/B test checkout flows and product pages to optimize the user path. Non-technical users can use autocapture to track all funnel page visits and interactions, creating funnels and using session recordings to optimize conversion paths with no code.

Cart Abandonment

Importance: The rate at which users add items to their cart but leave without completing the purchase. A high cart abandonment rate often indicates friction in the checkout process, unexpected costs, or a poor user experience.

PostHog approach: Track cart interactions like `add_to_cart` and `remove_from_cart`. Use session recordings to understand the behavior of users who abandon their carts, and implement exit-intent surveys to gather direct feedback on abandonment reasons. Create funnels that specifically track the checkout process to pinpoint exact drop-off points. This data can inform cart abandonment recovery strategies. Non-technical users can use autocapture to track all cart page interactions and build abandonment funnels to analyze user behavior.

Customer Lifetime Value

Importance: The total revenue a business can expect from a single customer throughout their relationship. CLV is vital for making strategic decisions about marketing spend, customer acquisition, and retention efforts, ensuring long-term profitability.

PostHog approach: Track all customer interactions, including `purchase_completed`, `return_requested`, and `support_contacted`, with properties like purchase history and acquisition source. Create cohort analyses by acquisition month to understand how customer value evolves. Use correlation analysis to identify behaviors of high-value customers and PostHog's predictive analytics for CLV forecasting. Non-technical users can use autocapture on account and order history pages to track engagement patterns and use session recordings to understand high-value customer behavior.

Marketplace

Common business problems & personas

Key business problems

- **Supply-demand imbalance** – Too many buyers/sellers on one side of the marketplace
- **Low trust and safety** – Users concerned about fraud or poor quality
- **Poor matching algorithms** – Buyers and sellers not connecting effectively
- **High customer acquisition costs** – Expensive to acquire both buyers and sellers
- **Network effects challenges** – Difficulty achieving critical mass
- **Payment and escrow issues** – Complex payment flows and trust concerns
- **Quality control problems** – Inconsistent service/product quality
- **Geographic expansion challenges** – Difficulty scaling to new markets

Primary personas & their pain points

Marketplace Operations Managers

- **Pain points:** Can't balance supply and demand, struggle with quality control, lack visibility into marketplace health
- **PostHog solutions:** Supply-demand analytics, quality metrics tracking, marketplace health dashboards, operational insights

Trust & Safety Teams

- **Pain Points:** High fraud rates, poor user verification, difficulty identifying bad actors
- **PostHog Solutions:** Fraud detection patterns, user behavior analysis, trust score tracking, automated risk alerts

Product Managers

- **Pain points:** Poor matching algorithms, low user engagement, ineffective search and discovery
- **PostHog solutions:** User behavior analysis, search optimization, matching algorithm improvement, engagement tracking

Growth Teams

- **Pain Points:** High CAC for both sides, poor network effects, slow marketplace growth
- **PostHog Solutions:** Network effects measurement, growth loop optimization, viral coefficient tracking, user acquisition analysis

Customer Success Teams

- **Pain Points:** High support volume, poor user satisfaction, difficulty resolving disputes
- **PostHog Solutions:** User journey analysis, satisfaction tracking, dispute resolution insights, support optimization

Key metrics & PostHog

GMV (Gross Merchandise Value)

Importance: Represents the total value of all transactions between buyers and sellers on the platform over a specific period. It is the primary indicator of a marketplace's scale, liquidity, and overall health, reflecting its ability to facilitate transactions and generate value for its users.

PostHog approach: Track marketplace transaction events like `listing_viewed`, `booking_requested`, and `transaction_completed` with properties such as category, price, seller_id, and buyer_id. Integrate with payment processors for comprehensive data. Use PostHog to create real-time GMV dashboards with breakdowns by category, set up seller and buyer performance tracking, conduct cohort analysis to monitor marketplace growth, and create alerts for unusual transaction patterns. Non-technical users can use autocapture to track listing views and booking requests, creating funnels to analyze the path to a completed transaction and using session recordings to optimize the user journey.

Take Rate

Importance: The percentage of GMV that the marketplace captures as revenue (commission or fees). It is a crucial metric for understanding the marketplace's business model effectiveness and profitability. Optimizing the take rate is key to sustainable growth.

PostHog approach: Track commission events like `commission_earned` from `transaction_completed` events, with properties for transaction amount, commission percentage, and category. Analyze revenue and profitability by category on dashboards. This allows for identifying opportunities to optimize the take rate, for example by analyzing its drivers with correlation analysis and setting up alerts for significant changes. Non-technical users can build dashboards to monitor take rate trends across different product categories or seller tiers, helping to inform pricing strategy without writing any code.

Supply/Demand Balance

Importance: Measures the equilibrium between the number of sellers (supply) and buyers (demand) on the platform. A balanced marketplace ensures a good user experience for both sides, preventing situations like too few products for buyers or too few customers for sellers, which can lead to churn.

PostHog approach: Track supply-side events (`listing_created`, `service_offered`) and demand-side events (`search_performed`, `booking_requested`). Use properties like category, location, and search terms to analyze supply-demand gaps on dashboards. Funnel analysis can reveal booking conversion rates, while alerts can notify of imbalances, helping to identify and act on new market opportunities. Non-technical users can create dashboards that visualize searches with no results, providing a simple way to spot unmet demand and guide supply-side growth efforts.

Network Effects

Importance: Measures how the value of the platform increases for users as more people use it. Strong network effects create a powerful competitive advantage (a "moat") and are the engine of sustainable, viral growth for marketplaces. It's what makes a marketplace more valuable as it scales.

PostHog approach: Track network interaction events like `user_referred`, `invitation_accepted`, and `cross_side_activity` (e.g., a user being both a buyer and seller). Use properties to distinguish user types. Dashboards can visualize network growth and viral coefficients. Cohort analysis is key to measuring how network effects develop over time for different user groups, and alerts can highlight opportunities for growth. Non-technical users can use autocapture on referral pages and share buttons to analyze the effectiveness of viral loops and optimize the user flow with session recordings.

Trust & Safety Metrics

Importance: Trust is the currency of a marketplace. These metrics, such as user ratings, review rates, fraud reports, and dispute rates, measure the level of safety and reliability on the platform. High trust is essential for encouraging transactions, retaining users, and building a strong brand reputation.

PostHog approach: Track trust-related events like `review_submitted`, `dispute_filed`, and `fraud_detected`, enriched with properties on user reputation and transaction history. Dashboards can monitor trust scores and fraud rates. Session recordings are invaluable for investigating suspicious user behavior and understanding how trust is built (or broken) in user flows. Set up alerts for fraud signals and use correlation analysis to identify key indicators of trust. Non-technical users can create surveys to collect user feedback on trust and use session recordings to review the user journey for those who file disputes.

Developer Tools

Common business problems & personas

Key business problems

- **Low developer adoption** – Developers not integrating or using the tool
- **High API error rates** – Poor API performance and reliability
- **Poor documentation engagement** – Developers struggling to understand the product
- **Low community engagement** – Lack of developer community growth
- **High support ticket volume** – Developers needing extensive support
- **Poor onboarding experience** – Developers dropping off during setup
- **Low feature adoption** – Developers not using advanced features
- **Difficulty measuring developer success** – Hard to track developer outcomes

Primary personas & their pain points

Developer Relations Teams

- **Pain points:** Low developer adoption, poor community engagement, difficulty measuring developer success
- **PostHog solutions:** Developer adoption tracking, community engagement analytics, developer success metrics, community health monitoring

Product Engineers

- **Pain points:** High API error rates, poor performance, difficulty debugging issues
- **PostHog solutions:** API performance monitoring, error tracking, real user monitoring, performance optimization insights

Technical Documentation Teams

- **Pain points:** Poor documentation engagement, developers struggling to find answers, high support volume
- **PostHog solutions:** Documentation usage analytics, search behavior tracking, content performance analysis, support ticket correlation

Developer Success Teams

- **Pain points:** High support ticket volume, poor onboarding experience, low feature adoption
- **PostHog solutions:** Support ticket analysis, onboarding funnel optimization, feature adoption tracking, developer journey mapping

Growth Teams

- **Pain points:** Low developer acquisition, poor retention, difficulty measuring developer LTV
- **PostHog solutions:** Developer acquisition tracking, retention analysis, developer LTV measurement, growth loop optimization

Key metrics & PostHog

Developer Adoption

Importance: Measures the rate at which developers start using a tool, from initial signup to making their first API call. It's the most critical top-of-funnel metric for developer tools, as it indicates the health of the onboarding process and the tool's initial appeal. High adoption is a leading indicator of future growth and product-market fit.

PostHog approach: Track key developer touchpoints like `account_created`, `sdk_installed`, and `api_call_made` with properties for tech stack and company size. Create adoption funnels to analyze the journey from first contact to active use, identifying drop-off points. Use cohort analysis to track developer retention over time and map the developer journey to understand common paths to success. Alerts can signal developer churn risk. Non-technical users, like DevRel teams, can build these funnels and dashboards without code to monitor adoption trends and measure the impact of their initiatives.

API Usage

Importance: Tracks the frequency, volume, and patterns of API calls made by developers. This metric is vital for understanding which features are most valuable, how developers are integrating the product, and the overall health and performance of the API. It directly reflects product engagement and stickiness for a developer-focused product.

PostHog approach: Instrument all API endpoints to track events like `api_request` and `api_error`, with properties for the specific endpoint, response time, and error type. Create API performance dashboards to monitor usage, latency, and error rates in real-time. Set up alerts for performance degradation or spikes in errors. Use correlation analysis to understand which usage patterns are associated with retention or expansion. Non-technical users can use dashboards to see which endpoints are most popular and identify which customers are experiencing the most errors.

Documentation Engagement

Importance: For developer tools, documentation is the product. This metric measures how developers interact with documentation, including page views, search queries, and time spent on pages. High engagement indicates that the documentation is useful and helps developers solve problems, which is critical for adoption and reducing support load.

PostHog approach: Track documentation interactions like `docs_page_viewed`, `code_sample_copied`, and `tutorial_completed`, with properties for the page, search terms, and user segment. Use session recordings to see where developers get stuck or confused. Analyze search patterns to identify content gaps and create dashboards to monitor documentation effectiveness. Non-technical users, like technical writers, can use these insights to prioritize content updates and improve the developer experience without needing to write code.

Community Growth

Importance: Measures the health and vibrancy of the developer community around a product (e.g., on GitHub, Slack, Discord). A growing, active community provides social proof, drives word-of-mouth adoption, offers scalable support, and is a rich source of product feedback. It acts as a moat and a powerful growth engine.

PostHog approach: Track community interactions from various platforms by sending events like `forum_post_created`, `github_issue_opened`, or `community_event_attended`. Use properties to segment by contribution level and topic. Create dashboards to monitor community engagement and growth trends. Use cohort analysis to track member retention and identify "power users" who can become community champions. Non-technical users, like community managers, can easily track these metrics to demonstrate the value of their programs.

Support Ticket Volume

Importance: The number of support tickets created by developers. While some tickets are expected, a high volume, especially on recurring themes, points to friction in the product, confusing documentation, or a poor onboarding experience. Analyzing this data is key to improving the product and reducing operational costs.

PostHog approach: Integrate your support system (e.g., Zendesk, Jira) with PostHog to track `support_ticket_created` and `support_ticket_resolved` events. Enrich these events with properties like ticket type, priority, and resolution time. Use correlation analysis to link support tickets to specific in-product behaviors or documentation pages, identifying the root cause of developer friction. Dashboards can help monitor support trends and efficiency. This allows non-technical team members to identify which product areas are generating the most support load.

Fintech

Common business problems & personas

Key business problems

- **High fraud rates** – Sophisticated fraud attempts and false positives
- **Poor compliance tracking** – Difficulty meeting regulatory requirements
- **Low transaction success rates** – Payment failures and processing issues
- **High customer acquisition costs** – Expensive to acquire and verify customers
- **Poor user trust** – Users concerned about security and data privacy
- **Complex onboarding flows** – Lengthy KYC/AML processes causing drop-offs
- **Low feature adoption** – Users not utilizing advanced financial features
- **Regulatory reporting challenges** – Difficulty generating required reports

Primary personas & their pain points

Risk & Compliance Teams

- **Pain points:** High fraud rates, poor compliance tracking, regulatory reporting challenges
- **PostHog solutions:** Fraud detection patterns, compliance monitoring, regulatory reporting automation, risk assessment analytics

Product Managers

- **Pain points:** Poor user trust, complex onboarding flows, low feature adoption
- **PostHog solutions:** User trust analysis, onboarding optimization, feature adoption tracking, user experience improvement

Engineering Teams

- **Pain points:** High transaction failure rates, poor API performance, security concerns
- **PostHog solutions:** Transaction monitoring, API performance tracking, security event monitoring, error rate optimization

Customer Success Teams

- **Pain points:** High support volume, poor customer satisfaction, complex issue resolution
- **PostHog solutions:** Support ticket analysis, customer satisfaction tracking, issue resolution insights, customer journey optimization

Growth Teams

- **Pain points:** High CAC, poor conversion rates, difficulty measuring customer LTV
- **PostHog solutions:** CAC analysis, conversion funnel optimization, customer LTV measurement, growth loop identification

Key metrics & PostHog

Transaction Volume

Importance: Measures the total number or value of transactions processed by the platform. This is a fundamental indicator of a fintech product's adoption, usage, and overall scale. It directly impacts revenue and is a key signal of market traction and business health.

PostHog approach: Track all financial transaction events like `transaction_initiated`, `transaction_completed`, and `transaction_failed` with detailed properties such as transaction type, amount, currency, and user segment. Use dashboards for real-time monitoring of transaction volume and success rates. Correlation analysis can help understand what user behaviors lead to more transactions, and alerts can be set for unusual spikes or dips in activity. Non-technical users can build funnels to analyze the transaction flow and identify drop-off points without writing any code.

Fraud Rate

Importance: The percentage of transactions that are fraudulent. In fintech, managing fraud is critical for financial stability, maintaining user trust, and meeting regulatory obligations. A low fraud rate is essential for long-term viability and building a reputable platform.

PostHog approach: Track fraud and risk-related events such as `fraud_detected`, `risk_assessment_failed`, or `verification_completed`. Enrich this data with properties like risk factors, fraud type, and user behavior patterns. Session recordings are invaluable for investigating suspicious user behavior to understand fraud vectors. Create dashboards to monitor fraud rates in real-time and set up alerts for emerging fraud patterns. Non-technical risk teams can use session recordings to review suspicious sessions flagged by alerts.

Compliance Metrics

Importance: Measures adherence to financial regulations like KYC (Know Your Customer) and AML (Anti-Money Laundering). For fintech companies, compliance is not optional; it's a license to operate. Tracking these metrics is crucial for avoiding fines, legal penalties, and reputational damage.

PostHog approach: Track all compliance-related events, such as `kyc_started`, `kyc_completed`, and `aml_check_failed`. Use properties to log the compliance type, status, and user segment. This creates a detailed audit trail for regulatory purposes. Dashboards can provide a real-time view of compliance status and help monitor the efficiency of these critical flows. Alerts can be configured to flag compliance failures, allowing teams to act quickly. Non-technical compliance officers can use funnels to analyze and optimize the KYC process.

Customer Acquisition Cost

Importance: The total cost to acquire a new, verified customer. Fintech often has high acquisition costs due to marketing, compliance, and verification expenses. Understanding and optimizing CAC is crucial for ensuring profitability and scaling the business sustainably.

PostHog approach: Track the entire acquisition funnel, from `ad_clicked` and `account_opened` to `verification_completed` and `first_transaction`. Enrich these events with properties like acquisition source, campaign, and verification costs. Use funnel analysis to identify drop-off points in the onboarding and KYC process. A/B testing can be used to optimize landing pages and onboarding flows to reduce CAC. Non-technical marketers can use dashboards to compare the CAC and LTV across different channels.

Regulatory Reporting

Importance: This tracks the ability of the company to generate accurate and timely reports for regulatory bodies. Efficient and reliable reporting processes are essential for demonstrating compliance and avoiding penalties. While PostHog doesn't generate the reports, it can monitor the internal processes that do.

PostHog approach: Track internal events related to the reporting process, such as `report_generated`, `audit_trail_requested`, and `compliance_check_completed`. Use properties to specify the report type and its status. This provides visibility into the operational health of the reporting systems. Dashboards can be used to monitor the success and timeliness of report generation, and alerts can be set up to flag any failures or delays in the process, ensuring the compliance team is aware of any issues.

Healthcare/Medtech

Common business problems & personas

Key business problems

- **Poor patient outcomes** – Patients not achieving desired health results
- **Low user adoption** – Healthcare providers not using the system effectively
- **Compliance violations** – Difficulty meeting HIPAA and other regulatory requirements
- **Poor clinical workflow efficiency** – Inefficient processes causing delays
- **Data accuracy issues** – Incorrect or incomplete patient data
- **High training costs** – Expensive to train healthcare staff on new systems
- **Poor integration** – Systems not working well with existing healthcare infrastructure
- **Security concerns** – Patient data privacy and security risks

Primary personas & their pain points

Clinical Teams

- **Pain points:** Poor clinical workflow efficiency, data accuracy issues, difficulty tracking patient outcomes
- **PostHog solutions:** Workflow optimization, data quality monitoring, patient outcome tracking, clinical efficiency analytics

Compliance Officers

- **Pain points:** Compliance violations, poor audit trails, difficulty meeting regulatory requirements
- **PostHog solutions:** Compliance monitoring, audit trail automation, regulatory reporting, data access tracking

IT/Engineering Teams

- **Pain points:** Poor system integration, security concerns, performance issues
- **PostHog solutions:** Integration monitoring, security event tracking, performance optimization, system health monitoring

Training Teams

- **Pain points:** High training costs, poor user adoption, difficulty measuring training effectiveness
- **PostHog solutions:** User adoption tracking, training effectiveness measurement, onboarding optimization, learning analytics

Product Managers

- **Pain points:** Poor user experience, low feature adoption, difficulty measuring clinical impact
- **PostHog solutions:** User experience analysis, feature adoption tracking, clinical impact measurement, product optimization

Key metrics & PostHog

Patient Outcomes

Importance: This is the core metric for any healthcare product, measuring the actual health impact on patients. Demonstrating positive patient outcomes is crucial for clinical validation, provider adoption, regulatory approval, and building patient trust. It is the ultimate measure of product value and efficacy.

PostHog approach: Track key events in the patient journey, such as `treatment_plan_started`, `outcome_measured`, and `follow_up_completed`. Use properties to segment by treatment type, patient demographics, and specific outcome metrics. Cohort analysis can track how outcomes trend over time for different patient groups. Dashboards can visualize progress towards clinical goals, and correlation analysis can help identify which product features are linked to better outcomes. Non-technical users, like clinicians, can use dashboards to monitor patient progress without writing code.

Compliance Metrics

Importance: Healthcare is a highly regulated industry (e.g., HIPAA in the US). Compliance metrics track adherence to these regulations, particularly around data privacy and security. Failure to comply can result in severe penalties, loss of trust, and legal action, making it a foundational requirement for any MedTech product.

PostHog approach: Track all compliance-related events, such as `hipaa_audit_trail_accessed`, `data_access_logged`, and `patient_consent_obtained`. Properties should include the user role, type of data accessed, and audit results to create an immutable log. Dashboards can provide a real-time view of compliance activities, and alerts can be set up for any unauthorized access attempts or compliance failures. Non-technical compliance officers can use these dashboards to monitor activity and generate reports.

User Adoption

Importance: Measures how effectively healthcare providers (doctors, nurses, etc.) are integrating a new tool into their daily work. Low adoption by clinicians can undermine the intended benefits of a technology, regardless of its potential. High adoption is key to realizing efficiency gains and improving patient care at scale.

PostHog approach: Track user interactions such as `feature_used`, `workflow_completed`, and `training_module_completed`. Segment by user role (e.g., doctor, nurse) using properties. Adoption funnels can show where users drop off during onboarding. Session recordings are invaluable for understanding how clinicians use the product in a real-world context. Alerts can flag low adoption in specific departments. Non-technical training teams can analyze session recordings to improve their training materials.

Clinical Workflow Efficiency

Importance: Measures the time and effort required for clinicians to complete tasks using the product. In the high-pressure healthcare environment, time is a critical resource. Improving workflow efficiency can reduce clinician burnout, lower operational costs, and allow more time for direct patient care.

PostHog approach: Track workflow events from start to finish: `workflow_started`, `step_completed`, `workflow_completed`. Use properties to capture the duration of each step and the user role. Funnel analysis is perfect for identifying bottlenecks where users get stuck or take too long. Dashboards can monitor average completion times for key workflows. Non-technical managers can use these funnels to identify areas for process improvement without needing technical assistance.

Data Accuracy

Importance: In healthcare, critical decisions are made based on patient data. Inaccurate or incomplete data can lead to misdiagnosis, incorrect treatment, and serious patient harm. This metric tracks the integrity and reliability of the data within the system, which is fundamental to patient safety.

PostHog approach: Track data entry and validation events like `data_entered`, `data_validated`, and `error_detected`. Use properties to specify the data type, validation method, and error type. Create dashboards to monitor data quality trends and error rates. Correlation analysis can help identify if specific user roles or workflow steps are associated with higher error rates. Alerts can notify teams of spikes in data entry errors, allowing for swift investigation.

Content/Media

Common business problems & personas

Key business problems

- **Low content engagement** – Users not consuming or interacting with content
- **Poor content discovery** – Users unable to find relevant content
- **Low subscription conversion** – Free users not converting to paid subscribers
- **Poor ad performance** – Low click-through rates and ad revenue
- **High content production costs** – Expensive to create quality content
- **Poor user retention** – Users not returning to consume more content
- **Ineffective content recommendations** – Poor personalization algorithms
- **Seasonal content challenges** – Difficulty maintaining engagement year-round

Primary personas & their pain points

Content Teams

- **Pain points:** Low content engagement, poor content discovery, high production costs
- **PostHog solutions:** Content performance analytics, engagement tracking, content discovery optimization, ROI measurement

Product Managers

- **Pain points:** Poor user retention, ineffective recommendations, low subscription conversion
- **PostHog solutions:** Retention analysis, recommendation algorithm optimization, conversion funnel analysis, user journey mapping

Marketing Teams

- **Pain points:** Poor ad performance, low subscription conversion, ineffective content marketing
- **PostHog solutions:** Ad performance tracking, conversion optimization, content marketing ROI, audience segmentation

Editorial Teams

- **Pain points:** Poor content performance, difficulty understanding audience preferences, seasonal engagement challenges

- **PostHog solutions:** Content performance analytics, audience preference analysis, seasonal trend identification, editorial optimization

Revenue Teams

- **Pain points:** Low subscription revenue, poor ad performance, difficulty monetizing content
- **PostHog solutions:** Revenue analytics, subscription optimization, ad performance tracking, monetization strategy insights

Key metrics & PostHog

Engagement Rate

Importance: Measures how actively users are interacting with content beyond just viewing it (e.g., likes, shares, comments, time spent). It's a key indicator of content quality and audience resonance. High engagement suggests that the content is valuable, which is crucial for building a loyal audience and driving retention.

PostHog approach: Track engagement events like `content_viewed`, `time_spent_on_page`, `video_played_to_75%`, and `article_shared`. Use properties to segment by content type and user segment. A custom "engagement score" can be created using formulas in PostHog to weigh different interactions. Cohort analysis can track how engagement evolves for different user groups. Non-technical editors can use dashboards to see which articles are most engaging to inform their content strategy.

Content Performance

Importance: Provides a holistic view of how individual pieces of content contribute to business goals, from views to conversions. Understanding what content performs well is essential for optimizing content strategy, allocating production resources effectively, and maximizing the ROI of content creation.

PostHog approach: Track the content lifecycle with events like `content_published`, `content_viewed`, and `content_shared`, enriched with properties like category, author, and format. Use correlation analysis to identify the attributes of successful content (e.g., "how-to" articles over 1500 words drive the most shares). Dashboards can rank content by performance, and alerts can notify teams when a piece of content starts trending. Non-technical content teams can use these insights to double down on what works.

User Retention

Importance: Measures the percentage of users who return to the platform over time. For media companies, retention is the lifeblood of the business, as it's far more cost-effective than acquisition. High retention indicates that users find ongoing value in the content, which is key for long-term growth and subscription revenue.

PostHog approach: Track retention by monitoring `user_returned` or `session_started` events. Use PostHog's retention cohorts to analyze how retention differs by acquisition source or first content consumed. Correlation analysis can identify behaviors (e.g., subscribing to a newsletter) that are leading indicators of retention. Churn prediction models can help proactively identify at-risk users. Non-technical marketers can use cohorts to understand the long-term value of users from different campaigns.

Ad Revenue

Importance: For ad-supported media companies, this metric directly measures financial performance. Optimizing ad revenue involves balancing user experience with monetization, making it crucial to track metrics like impressions, click-through rates (CTR), and revenue per user.

PostHog approach: Track ad-related events like `ad_impression`, `ad_click`, and `ad_revenue_generated`. Use properties to segment by ad type, placement, and user segment. A/B test different ad placements and formats to see what generates the most revenue without harming engagement. Dashboards can monitor ad performance in real-time, and alerts can flag underperforming ad units. Non-technical revenue teams can use these dashboards to track progress against revenue goals.

Subscription Metrics

Importance: For subscription-based media companies, metrics like conversion rate, subscriber LTV, and churn are the ultimate measure of business health. They track the ability to convert casual readers into paying subscribers and retain them, directly reflecting the perceived value of the premium offering.

PostHog approach: Track the entire subscription funnel with events like `paywall_hit`, `subscription_started`, `subscription_renewed`, and `subscription_cancelled`. Use funnel analysis to identify drop-off points in the conversion process and properties like plan type to segment subscribers. Cohort analysis is essential for tracking subscriber LTV and churn over time. Non-technical product managers can use funnels to optimize the checkout flow and A/B test different paywall strategies.

Who we do business with

contents/handbook/growth/sales/who-we-do-business-with.md

We firmly adhere to laws in countries where we do business, and welcome everyone abiding by those legal restrictions to be our customers - paid or free, in all but a few very exceptional circumstances:

- The customer is engaging in illegal or unlawful behavior.
- The customer is encouraging violence or discriminating against legally protected groups.

In these cases, we may choose not to do business with the customer.

Sanctioned countries and companies

US laws mean we may also be prohibited from working with certain companies, due to ongoing US sanctions. In this case we do not have discretion - we are banned from working with these companies entirely.

If you need to check if a particular company appears on a US sanctions list, you can use the US Treasury's Sanction Search. In particular, you should be mindful of companies that sign up which are based in the following territories:

- Balkans
- Belarus
- Burundi
- Central African Republic
- Crimea
- Democratic Republic of the Congo
- Iraq
- Libya
- Lebanon
- Myanmar (formerly Burma)
- Russia
- Sudan
- South Sudan
- Somalia
- Ukraine
- Venezuela
- Yemen
- Zimbabwe

US sanctions mean that we are not allowed to offer services at all to *any* companies based in:

- Cuba

- Iran
- North Korea
- Syria

Update for June 2024 US sanctions against Russia

In June 2024, the US Treasury's Office of Foreign Asset Control issued updated sanctions against Russia which prohibit the sale or supply of services to individuals or organizations in Russia. The sanctions take effect on September 10, 2024 and continue indefinitely.

We must comply with these sanctions, so in August 2024 we contacted impacted individuals to let them know we would make the following changes on **September 9th, 2024**:

- We no longer accept any payments from individuals or organizations based in Russia
- We block access to PostHog for all individuals in Russia, based on their IP
- We terminated paid accounts with all customers located in Russia

There are some exemptions to the sanctions including any service to any entity located in the Russian Federation that is owned or controlled, directly or indirectly, by a U.S. person.

> If a customer believes they've been incorrectly impacted by our response to these sanctions, or have further questions about them, ask them to contact sales@posthog.com so we can investigate.

Checking whether we can do business with a customer

If you work in Sales, CS & Onboarding, or Support and are not sure if we are able to work with a customer you are dealing with, ask in #legal and one of the team will be able to let you know either way. For the most part, these edge cases are to do with customers attempting to work around sanctions in their country, though other edge cases can also occur.

Customers who track adult or other potentially offensive content aren't automatically excluded - we have content warnings set up in Zendesk for them. If you are working with their account more regularly as part of the Sales or CS & Onboarding teams, we also recommend that you avoid logging in as them, and that you provide any training using demo data.

Why buy PostHog

contents/handbook/growth/sales/why-buy-posthog.md

AKA our Value Proposition, these are some of the things we've found useful to share when chatting to customers about why PostHog is different and better than our competitors. As a company, the primary user persona we are building for are Product Engineers, so we focus on them first. We then provide messaging for the other roles we may encounter in an inbound sales cycle, and still want to be successful when selling to them.

Product Engineers

One-liner

> We help you debug and ship your product faster.

Summary

By integrating PostHog into your app, you'll be able to track and diagnose errors, roll out and test new features and gain a better understanding of your user behavior. With that greater understanding, you'll then be able to take action on it and respond to your user needs quickly and effectively. Getting all of these capabilities through one SDK means you reduce the overhead of maintaining your app and can focus on shipping your product.

Use cases

- Automated error tracking for front and back end, coupled with other capabilities like product analytics and session replay lets you understand where the biggest issues are in your app, see them happening in real time and then diagnose and fix them.
- Target new features at a segment of your user base, see them experiencing them in real time and get feedback via surveys on what's working and what's not.
- Test out new features by splitting old and new experiences between users - PostHog's statistical model will help you understand which variant of a feature to choose and then safely roll that out to all of your users.
- Understand and debug how your users consume AI in your product and monitor performance and cost when using different models.
- Respond to churn by triggering a survey when a subscription is canceled to understand what went wrong for them and how you can improve your product.

Product Managers

One-liner

> Self-serve analytics without needing to ask your engineers or data team for help.

Summary

After your engineers integrate the PostHog SDK, you'll be able to self-serve analytics without asking your data team for insights. We automatically track user interactions with your app and then let you tag key events for use in analytics. You'll also be able to navigate from the data to individual user interactions to see how users interact with your app and make informed product decisions, and then finally use behavioral triggers to send feedback surveys and more all without engineering effort.

Use cases

- Create trends, funnels and other insights without asking your engineers to instrument events. We automatically track pageviews, clicks, rageclicks etc and then make it easy to visualize these with insights Product Managers will be familiar with.
- Easily uncover user friction by following the drop-offs in a funnel to replays to understand what the user experiences. Surface any errors to your engineering team via issue assignment to get your user problems solved quickly.
- Enrich your product data with revenue and other data to gain a deeper understanding of what drives revenue growth in your product. It's only a few clicks to integrate most data sources and then you'll be able to enrich your user data with additional metrics without a data team. We do the heavy lifting for you.
- Ask questions of your product - We create the insights for you, all you need to do is ask PostHog AI questions about your product.
- Respond to churn by triggering a survey when a subscription is canceled to understand what went wrong for them.
- Create event-driven workflows to automatically reach out to customers who hit a certain point in your product journey.

Marketing

One-liner

> A familiar analytics experience with all of the integrations you need to decide where to focus your marketing efforts.

Summary

By deploying our simple JavaScript snippet on your website you'll capture all of the data you need to measure channel performance, and then visualize that data in a familiar format without any additional report writing. Optionally hook up Stripe or other revenue sources to measure revenue attribution.

Use cases

- Replace Google Analytics to get a view on your marketing data which is familiar to experienced marketers. Recent updates to GA4 have not sat well with that persona so folks are looking for something more familiar.
- Define conversion funnels to understand which content drives your users to sign up to your product.
- View aggregated page engagement with heatmaps and scroll depth tracking understanding what's popular in your content.
- Easily connect revenue data with a few clicks to get a deeper understanding of which marketing efforts drive the most revenue.
- Ad platform connection provides pre-built insights to help you understand your campaign performance and associated costs.

Data Engineers

One-liner

> A complete developer platform which fits into your existing data stack.

Summary

Using PostHog's CDP lets you aggregate data from multiple technologies and platforms. It takes a few clicks to set up exports of that data to your data warehouse, and your product and engineering teams can self-serve their own analytics from within PostHog.

Use cases

- Aggregate your user and error data from web and mobile apps, backend systems, ad platforms and others into your data warehouse via our simple to set up batch exports. Avoid needing to set up ETL jobs from disparate sources and figuring out APIs.
- Let your engineers and product team self-serve analytics and error tracking from within a familiar platform.

General talking points for all roles

- By having all of the products you need in one place, you reduce the burden of navigating and paying for different tools on all of your teams.
- We only build products which we know people use today already (e.g. have product market fit) but provide them in an integrated and cost-effective manner.
- Analytics is an after-the-fact passive activity, but your events in PostHog can be so much more than that. You can use PostHog events to react to customer behaviors, without investing engineering time to make those workflows.

- Our usage-based pricing means that you'll only pay for what you use and have full control of those costs, unlike opaque software contracts, where the prices go up every year with zero innovation attached.
- Plus, the way we do sales is different.

Per-product sales enablement

The product marketing team has created sales enablement materials, covering some product information and general objection handling for specific products. These exist as Google Docs as they are living documents, but are listed below.

- Managed data warehouse (DuckDB)

If there are additional products you'd like to see this sort of material for, let the team know in the **#team-marketing** Slack channel.

AI/LLM Observability

contents/handbook/growth/use-case-selling/ai-llm-observability.md

What is the job to be done?

"Help me understand how my AI features perform, what they cost, and how users interact with them."

- Track model performance: latency, cost per query, token usage, error rates across providers and models
- Evaluate AI output quality and detect regressions after prompt or model changes
- Understand how users actually interact with AI-generated output (not just whether the model responded)
- A/B test prompts, models, and parameters against real user behavior metrics, not just model-level metrics
- Monitor cost attribution by user, organization, or feature so you know where your OpenAI bill is going
- Catch model failures, hallucinations, and timeouts alongside traditional application errors

This is the fastest growing segment of our customer base. AI-native companies are adopting PostHog at a high rate, but often only for LLM Observability or only for Product Analytics. The cross-sell opportunity is significant because AI products have unique observability needs that span multiple PostHog products.

The buyer persona is distinct: AI engineers care about model-level metrics (latency, cost, token usage, accuracy) first, user-level analytics second. Leading with the AI story opens the door to everything else.

What PostHog products are relevant?

- **LLM Observability (core)** — Model performance, cost tracking, latency monitoring. Trace individual LLM calls with inputs, outputs, token counts, latency, and cost. Aggregate views by model, provider, feature, user, or organization.
- **AI Evals** — Score and evaluate AI outputs, proactively surface quality issues and user struggles. Run automated evaluations after prompt or model changes to catch regressions that look "fine" from an error rate perspective but degrade user experience. This is a bridge product: its primary home is AI/LLM Obs, but it unlocks value in Product Intelligence (surface where users struggle based on output quality) and Release Engineering (catch quality regressions after prompt/model changes).
- **Product Analytics** — User behavior around AI features. How do users interact with AI-generated output? Do they accept suggestions, reject them, regenerate? Which users/organizations drive the most AI usage (and cost)? Funnels and retention for AI-powered flows.
- **Experiments** — A/B test prompts, models, and parameters against real user behavior metrics. "Does GPT-4o produce better outcomes than Claude for this use case?" measured not by model benchmarks but by user conversion, retention, and satisfaction.

- **Error Tracking** — Catch model failures, hallucinations (if detectable), timeouts, rate limit errors. Traditional error tracking applied to the AI layer. When combined with LLM Observability, you get both the exception and the model-level context.
- **Session Replay** — See the user experience of AI features. Watch how users interact with AI-generated content: do they read it? Copy it? Regenerate? Leave? This qualitative layer is especially valuable because AI feature UX is hard to quantify with events alone.
- **PostHog AI** — Query model performance data in natural language. "Which prompts have the highest latency and cost?" or "Show me the error rate by model this week." Useful for AI engineers who want fast answers about their own AI infrastructure. (Example prompts)

Adoption path and expansion path

Entry point

Usually **LLM Observability** or **Product Analytics**. Two common patterns:

1. **Model-first:** AI engineer wants to understand model performance: latency, cost, token usage. They start with LLM Observability for tracing and cost attribution, then realize they need to understand how users interact with the output (Product Analytics), whether the output is actually good (AI Evals), and how to test improvements (Experiments).
2. **Product-first:** AI product team is building a product with AI features and starts with Product Analytics to track user behavior. They realize they need model-level metrics alongside user metrics, which pulls in LLM Observability. From there, they want to evaluate quality (AI Evals) and test prompt/model changes (Experiments).

Primary expansion path

LLM Observability → + AI Evals → + Product Analytics (user behavior) → + Experiments (prompt/model testing) → + Error Tracking → + Session Replay

The logic of each step:

- LLM Observability → AI Evals: They can see model performance metrics (latency, cost, tokens). They need to know if the output is actually *good*. Evals score quality and detect regressions after changes.
- AI Evals → Product Analytics: They know the model is performing well technically. But are users actually getting value from the AI features? Product Analytics tracks how users interact with AI output: acceptance rates, regeneration rates, downstream conversion.
- Product Analytics → Experiments: They've identified differences in AI feature performance. Now they want to test improvements: different prompts, different models, different parameters. Experiments lets them A/B test with real user behavior as the success metric, not just model benchmarks.

- Experiments → Error Tracking: They're iterating on AI features. Error Tracking catches model failures, rate limit errors, and timeouts. Combined with LLM Observability, they get the full picture: exception + model context.
- Error Tracking → Session Replay: They're catching errors and measuring metrics. Session Replay shows them *how users experience* AI features, especially in ambiguous cases where the model didn't error but the output wasn't helpful.

Alternate expansion paths

Starting from Product Analytics: An AI product team already using PostHog for product analytics. They add LLM Observability to get model-level metrics alongside their user behavior data. From there, AI Evals and Experiments are natural adds.

Starting from Error Tracking: Team catching model failures with Error Tracking. They realize traditional error tracking misses quality regressions (model responds but with worse output). AI Evals fills this gap, pulling in LLM Observability for the full model-level context.

Business impact of solving the problem

AI-native companies are the fastest growing customer segment. Getting in early with LLM Observability means PostHog becomes the default platform as these companies scale. AI-native startups that adopt PostHog at seed stage often grow into significant accounts.

The cross-sell opportunity is uniquely strong. AI products sit at the intersection of multiple PostHog use cases: model observability (AI/LLM Obs), user behavior analytics (Product Intelligence), release management for prompt/model changes (Release Engineering), and error tracking for model failures (Observability). One AI customer can reasonably adopt products from 4+ use cases.

No one else has this combination. Langfuse and Helicone do LLM tracing. Amplitude does product analytics. Sentry does error tracking. No one connects model performance → output quality → user behavior → business outcomes in one platform. That's PostHog's pitch.

AI Evals is the bridge product. For any account building AI features, AI Evals connects AI/LLM Observability to Product Intelligence (are users struggling based on output quality?) and Release Engineering (did a prompt change cause a quality regression?). It's a natural entry point into multiple use cases from a single product.

Personas to target

Persona	Role Examples	What They Care About	How They Evaluate
AI Engineer	ML Engineer, AI Engineer, Applied AI	Model performance, cost optimization, latency, quality	"Can I see cost per query by model, trace individual calls, and detect quality regressions?"

Persona	Role Examples	What They Care About	How They Evaluate
AI Product Manager	AI PM, Product Lead (AI features)	User experience of AI features, adoption rates, business impact	"Can I see how users interact with our AI features and whether they drive retention?"
AI Founder	Founder, CTO at AI-native startup	All of the above. Cost control. Speed. Not paying for 5 tools.	"How fast can I set this up and how much does it replace?"
AI Product Engineer	Full-stack engineer building AI features	Instrumentation, debugging, prompt iteration cycle time	"How easy is it to instrument? Can I see trace-level detail for debugging?"

Signals in Vitally & PostHog

Vitally indicators this use case is relevant

Signal	Where to Find It	What It Means
LLM Observability is active	Product usage data	AI/LLM Obs use case is live. Full expansion path available.
Company tags include "AI" or "LLM" or "ML"	Company info / tags	AI-native or AI-building company. This use case is likely relevant even if they haven't adopted LLM Observability yet.
High Product Analytics usage + AI company	Product usage + company type	They're using analytics but haven't connected model-level metrics. LLM Observability is the add.
Customer mentions Langfuse, Helicone, or "LLM costs" in notes	Vitally notes / conversations	Direct signal. They're thinking about AI observability and may be using a competitor or building it in-house.

PostHog usage signals

Signal	How to Check	What It Means
LLM-related custom events (e.g., `llm_generation`, `ai_response`)	Event property explorer	They're tracking AI events in Product Analytics. LLM Observability would give them model-level detail.
High LLM Observability trace volume	Product usage metrics	Active AI instrumentation. Ripe for AI Evals and Experiments.
Experiments on AI-related features	Experiments list	They're already A/B testing AI features. Validate they're using LLM Obs for model-level measurement.

Signal	How to Check	What It Means
Error Tracking exceptions from AI/model code	Error tracking events	Model failures are happening. LLM Observability gives context beyond the stack trace.

Command of the Message

Discovery questions

- What AI/LLM features are you building? How central are they to your product?
- How do you track model performance today? Cost, latency, token usage — where does that data live?
- When you change a prompt or switch models, how do you know the output quality held up?
- Can you tell which users or organizations are driving the most AI cost?
- How do you decide whether to use GPT-4o vs. Claude vs. a smaller model for a given feature? Is that data-driven or gut feel?
- When your AI feature produces a bad output, how do you find out? User complaint? Manual testing?
- Do you A/B test different prompts or models? How do you measure success — model benchmarks or actual user behavior?
- How do users interact with AI-generated output in your product? Do you track acceptance rates, regeneration, or downstream actions?

Negative consequences (of not solving this)

- AI costs grow unchecked because there's no visibility into cost per query by user, feature, or model
- Prompt or model changes degrade output quality but nobody notices until users complain
- Model-level metrics (latency, tokens) are tracked separately from user behavior, so the team can't answer "are users actually getting value from the AI feature?"
- A/B testing prompts or models relies on model benchmarks, not real user outcomes, leading to optimizations that don't translate to business results
- AI failures (timeouts, rate limits, hallucinations) are caught ad hoc instead of systematically

Desired state

- One platform for model performance, output quality, user behavior, and business outcomes
- Cost attribution by user, organization, and feature so the team knows where the AI budget goes
- Automated quality evaluation after every prompt or model change
- A/B testing of prompts and models measured against real user behavior, not just model benchmarks
- AI errors caught proactively alongside traditional application errors
- The full picture: model trace → output quality score → user interaction → business outcome, all connected

Positive outcomes

- AI costs decrease through visibility and optimization (knowing which models/prompts to use where)
- Quality regressions caught before they reach users at scale
- Faster AI iteration cycle: change a prompt, evaluate quality, measure user impact, all in one tool
- Better model selection decisions based on user outcomes, not just model benchmarks
- Consolidation of AI observability (Langfuse/Helicone) with product analytics and error tracking into one platform

Success metrics

Customer-facing:

- AI feature usage and adoption rates improve (users get more value from AI output)
- AI costs per unit of value decrease (better model selection, prompt optimization)
- Quality regression detection time decreases (evals catch issues faster)
- Prompt/model experiment velocity increases

TAM-facing:

- Customer expands from LLM Obs-only to multi-product (Product Analytics, Experiments, Error Tracking)
- AI Evals adoption grows (quality evaluation is active, not just tracing)
- Experiment count on AI features increases
- Non-AI products start being adopted (Session Replay, Feature Flags) as the broader platform becomes familiar

Competitive positioning

Our positioning

- **Model performance + user behavior in one platform.** Langfuse traces your LLM calls. PostHog traces your LLM calls AND shows you how users interact with the output AND lets you A/B test improvements AND catches errors. No one else connects the full stack.
- **Real user outcomes, not just model metrics.** Other AI observability tools optimize for model performance (latency, cost, perplexity). PostHog lets you optimize for user outcomes: did the user accept the suggestion? Did they convert? Did they come back?
- **Experiments on prompts/models measured by business metrics.** A/B test GPT-4o vs. Claude measured not by BLEU score but by user conversion and retention. This is what actually matters for AI product decisions.
- **AI observability + traditional observability.** Error Tracking catches model failures. Session Replay shows the user experience. Product Analytics measures business impact. It's one platform, not AI observability siloed from everything else.

Competitor quick reference

Competitor	What They Do	Our Advantage	Their Advantage
Langfuse	Open-source LLM tracing, prompt management, evals	Broader platform (product analytics, experiments, replay, error tracking); user behavior metrics; not just model metrics	More mature LLM-specific features; open-source community; purpose-built prompt management
Helicone	LLM request logging, cost tracking, caching	Broader platform; user behavior connection; experiments; not a single-purpose tool	Simpler to set up for basic LLM logging; built-in caching/rate limiting features
Braintrust	LLM evals, logging, prompt playground	Broader platform; user behavior metrics; production monitoring not just offline evals	More mature eval framework; better prompt playground and iteration workflow
Datadog LLM Monitoring	LLM tracing as part of broader APM	Product analytics integration; user behavior; better pricing for AI-native startups	Full APM stack; enterprise-grade; part of existing Datadog deployment for bigger companies

Honest assessment: Our strongest position is with AI-native startups and teams building AI features inside existing products. The pitch is "one platform for everything" instead of Langfuse + Amplitude + Sentry + a flag tool. We're weaker against teams that want the deepest possible LLM-specific tooling (Langfuse's prompt management and eval framework are more mature). We're also weaker against enterprise teams already embedded in Datadog. Our sweet spot is AI teams that want model performance connected to user outcomes in one place, without managing 4 vendors.

Pain points & known limitations

Pain Point	Impact	Workaround / Solution
LLM Observability feature set is newer than Langfuse	Teams expecting Langfuse-level prompt management and eval detail may find gaps	Be honest about maturity. Position the breadth of the platform (analytics, experiments, replay) as the differentiator. Langfuse is great for pure LLM tracing; PostHog is better when you also need to understand user behavior and business impact.
AI Evals may not support all evaluation frameworks	Teams with custom eval pipelines may want more flexibility	Check current eval capabilities. For custom frameworks, PostHog's API and data warehouse can integrate with existing eval pipelines.
Session Replay for AI chat interfaces	Chat-based AI products generate a lot of replay	Configure sampling rules. Focus replay viewing on sessions with error events or low AI quality scores.

Pain Point	Impact	Workaround / Solution
can be noisy	data per session	

Getting a customer started

What does an evaluation look like?

- **Scope:** Instrument their primary AI feature with LLM Observability tracing. Set up cost attribution by model and feature. If they have a prompt change planned, set up AI Evals before and after.
- **Timeline:** 1 to 3 days to start capturing LLM traces. 1 to 2 weeks for meaningful cost and performance data. Eval comparisons depend on the change cycle.
- **Success criteria:** Can you see cost per query by model? Can you trace individual LLM calls with inputs and outputs? Can you detect a quality regression after a prompt change?
- **PostHog investment:** LLM Observability free tier covers 100K events/month. Product Analytics free tier covers 1M events. Experiments are included with Feature Flags.
- **Key requirement:** They need to instrument their LLM calls using the PostHog SDK or API. See the AI Engineering docs for integration guides by framework.

Onboarding checklist

- [ ] Instrument primary AI feature with LLM Observability tracing
- [ ] Verify traces are capturing: model, inputs, outputs, token counts, latency, cost
- [ ] Set up a cost attribution dashboard: cost by model, by feature, by user/organization
- [ ] Configure AI Evals for output quality scoring on the primary AI feature
- [ ] Build a "Model Health" dashboard: latency, cost, error rates, quality scores
- [ ] Enable Error Tracking for model failures, timeouts, and rate limit errors
- [ ] Set up Product Analytics tracking for AI feature user interactions (accept, reject, regenerate, downstream actions)
- [ ] Enable Session Replay to watch how users interact with AI output
- [ ] Plan first Experiment on an AI feature: different prompt, different model, or different parameter

Cross-sell pathways from this use case

If Using...	They Might Need...	Why	Conversation Starter
LLM Observability only	AI Evals	They can see model metrics but don't know if the output is actually *good*	"You can see your model's latency and cost. But do you know if the quality held up after your last prompt change?"

If Using...	They Might Need...	Why	Conversation Starter
LLM Obs + AI Evals	Product Analytics	They know model performance and quality. They don't know how users interact with the output.	"Your model is fast and the quality is high. But are users actually accepting the suggestions and converting?"
LLM Obs + Product Analytics	Experiments	They see model metrics and user behavior. They want to improve.	"You can see GPT-4o costs more but users seem to prefer it. Want to run a proper A/B test to quantify the difference?"
AI feature releasing changes	Release Engineering (Feature Flags)	They're changing prompts/models and want controlled rollout	"When you change your prompt, do you ship to everyone at once? Feature flags let you roll out to 5% first and measure before going wide."
AI features in PostHog	Product Intelligence (for the product team)	AI team is in PostHog. The broader product team should be too.	"Your AI team uses PostHog for model metrics. Has the product team seen what they can do with funnels and retention for non-AI features?"
Error Tracking for AI errors	Observability (full stack)	They're catching AI errors but not traditional application errors	"You're tracking model failures. Are you also catching the non-AI exceptions? Error Tracking works for your entire stack."

Internal resources

- **LLM Observability docs:** AI Engineering
- **Product Analytics docs:** Product Analytics · Funnels · Retention
- **Experiments docs:** Experiments
- **Error Tracking docs:** Error Tracking
- **Session Replay docs:** Session Replay
- **PostHog AI docs:** Enable PostHog AI · Example prompts
- **Competitive battlecard:** *To be added: Langfuse / Helicone competitive positioning*

Appendix: Company archetype considerations

Archetype + Stage	Framing	Key Products	Buyer
AI Native — Early	"You need to understand your model costs, catch quality regressions, and see how users interact with your AI features, all without hiring a data team or buying 4 tools." Speed and simplicity. One platform.	LLM Observability, AI Evals, Product Analytics, PostHog AI	Founder, AI engineer, founding PM
AI Native — Scaled	"You're scaling AI features across your product. You need cost attribution by team/feature, automated quality evaluation, prompt/model experimentation, and the ability to connect model performance to business outcomes."	LLM Observability, AI Evals, Product Analytics, Experiments, Error Tracking, Session Replay	Head of AI/ML, AI PM, VP Eng
Cloud Native — Any (building AI features)	"You're adding AI features to an existing product. PostHog already tracks your users. Now connect model performance to user behavior so you can optimize the AI experience alongside everything else." The pitch here is extending their existing PostHog usage, not adopting a new tool.	LLM Observability, AI Evals (added to existing PostHog stack)	Engineering team building the AI feature, PM who owns the AI feature

Customer Experience

contents/handbook/growth/use-case-selling/customer-experience.md

What is the job to be done?

"When a customer runs into an issue, we're able to quickly understand exactly what happened, identify the problem, and verify a fix, without bouncing between multiple tools or wasting engineering time trying to reproduce it."

- Build a repeatable debugging workflow where support, product, and engineering share the same context
- Give support teams the ability to see what actually happened, not just what the user reported
- Connect technical debugging (errors, logs) to user behavior (replay, analytics) and satisfaction signals (NPS, CSAT)
- Trace AI-powered workflows end to end when things go wrong

Most companies don't have a customer experience system. They have tickets in one place, errors in another, logs somewhere else, analytics owned by product, and engineers manually trying to reproduce bugs. The goal of this use case is to help a company build a unified debugging workflow where support, product, and engineering share the same context.

What PostHog products are relevant?

- **Product Analytics (core)** — Understand what a user was trying to do before something broke. Identify patterns in drop-offs, error frequency, and issue clustering across users or accounts.
- **Session Replay** — See exactly what the user did, not what they think they did. Capture console logs and network calls alongside the visual recording. The single most impactful product for support and debugging workflows.
- **Error Tracking** — Capture frontend and backend exceptions tied to users and releases. See whether other users have been experiencing the same issue. Structured, queryable error data instead of ad hoc log searches.
- **Group Analytics** + **Person Profiles** — Give support and CS a clean, holistic view of a user or account. See all events, replays, errors, and properties for a specific person or organization.
- **Logging** *beta* — Inspect structured backend logs connected to the same user session. When replay and error tracking show what happened on the frontend, logs show what happened on the server.
- **LLM Observability** — See prompts, outputs, latency, and token usage for AI-powered workflows. When an AI feature misbehaves, trace it back to the specific generation.
- **Surveys** — Capture frustration signals (NPS, CSAT) and tie them directly to broken flows. When someone leaves a low score, you can click through to their session and see what went wrong.
- **Experiments** — Validate that fixes actually improved the experience. After resolving a class of issues, measure whether user satisfaction and completion rates improved.

Adoption path and expansion path

Entry point

Usually **Session Replay** or **Product Analytics**. Common entry scenarios:

1. **"We can't reproduce bugs":** Support needs to see what happened instead of relying on screenshots and user descriptions. Session Replay is the direct answer.
2. **"Something is breaking but we don't know why":** Product notices drop-offs or support volume spikes and needs visibility into what's causing them. Product Analytics surfaces the pattern, Session Replay provides the detail.

Primary expansion path

Product Analytics → + Session Replay → + Error Tracking → + Logs / LLM Observability → + Surveys

The logic of each step:

- Product Analytics → Session Replay: They know *what* happened (drop-offs, error rates). They need to see *why*. Session Replay provides the qualitative context behind the quantitative signal.
- Session Replay → Error Tracking: Seeing something break visually isn't enough. They want structured, queryable errors tied to users and releases. Error Tracking makes debugging systematic instead of ad hoc.
- Error Tracking → Logs / LLM Observability: Now they want to see what happened server-side or inside AI workflows. Logs provide backend context. LLM Observability traces AI-specific issues (hallucinations, prompt regressions, latency spikes).
- Logs / LLM Observability → Surveys: After stabilizing debugging, they want a simple way to detect frustration and measure whether reliability improvements are being felt by users. Surveys close the feedback loop.

This expansion happens naturally because each step removes a layer of uncertainty.

Alternate expansion paths

Starting from Session Replay as a replacement for another session recording tool. They adopt Session Replay to replace Hotjar, FullStory, or LogRocket. Expand by introducing autocapture (Product Analytics), Error Tracking for structured bug data, and Group Analytics for account-level views.

Business impact of solving the problem

Engineering time savings. If bug reproduction drops from 2 hours to 30-60 minutes, teams get fewer context switches, fewer escalations, and more roadmap velocity. Even modest improvements here can easily justify the cost of the entire PostHog contract.

Escalation reduction. When support can view replay, check errors, and inspect logs, they resolve more issues without pulling in engineering. That means the roadmap doesn't stall and customer response times improve.

Revenue protection. When enterprise customers report issues, speed and clarity matter. Being able to say "here's exactly what happened and here's the fix" builds trust. Slow, unclear debugging erodes it.

AI risk mitigation. For AI-powered products, LLM Observability catches the things that would otherwise go unnoticed: hallucinations that are hard to trace, prompt regressions, and latency spikes. Without it, product credibility degrades quietly.

Personas to target

Persona	Role Examples	What They Care About	How They Evaluate
Support Leader	Head of Support, Support Ops	Faster resolution, fewer escalations	MTTR, escalation rate
Engineering Lead	EM, Staff Eng	Reproducible bugs, fewer interruptions	Debugging time, context switches
Product Manager	PM, Product Lead	Understanding friction, user-reported issues	Drop-off rates, issue frequency
AI Lead	Head of AI, Applied AI Eng	Model reliability, output quality	Output quality, latency, trace coverage
CS Leader	VP CS, Head of CS	Customer trust, proactive issue resolution	NPS trends tied to product issues

Signals in Vitally & PostHog

Vitally indicators this use case is relevant

Signal	Where to Find It	What It Means
Users with a support title	User list in Vitally	They're already bringing support folks into PostHog. CX workflow is emerging organically.
High session replay spend / volume	Product spend breakdown, usage metrics	They're investing heavily in replay. This use case helps them get more value from that spend by connecting replay to errors, logs, and surveys.

Signal	Where to Find It	What It Means
High support ticket volume	`vitally.custom.supportTickets`	They're dealing with a lot of customer issues. PostHog can help them debug faster.
Multiple user roles in PostHog (eng + support + product)	User list, admin emails	Cross-functional usage signals that CX workflows are already forming.

PostHog usage signals

Signal	How to Check	What It Means
Session Replay filtered by error events	Replay usage patterns	They're connecting replay to debugging. The CX workflow is clicking.
Person profile lookups increasing	Product Analytics usage	Support or CS is investigating individual users. Group Analytics could formalize this.
Error Tracking adoption alongside replay	Product spend data	They're building the debugging stack. Logs and surveys are natural next steps.
Console log / network tab usage in replays	Replay engagement metrics	They're using replay for technical debugging, not just UX review. Strong CX signal.

Health score implications

- **Event volume:** Should stay relatively similar (this use case doesn't fundamentally change event instrumentation)
- **User engagement:** More users spending more time in PostHog (support, CS, and product teams joining engineering)
- **Product count:** Should drive adoption of Error Tracking, Group Analytics, Logs, Surveys, and more

Command of the Message

Discovery questions

- How do you currently investigate a reported issue? Walk me through the workflow.
- How long does it take to reproduce a bug reported by a customer?
- How many tools do you open to debug one ticket?
- Can support see backend errors or do they escalate everything to engineering?
- Can you trace an AI output back to its prompt and context?
- When someone leaves a low NPS score, can you see what went wrong in their session?
- How do you confirm that a fix actually worked for the users who were affected?

Negative consequences (of not solving this)

- Engineering time wasted on reproduction instead of shipping
- Constant escalations and interruptions from support to engineering
- Enterprise deals slowed or lost due to reliability concerns and slow issue resolution
- AI features degrading silently with no visibility into output quality
- Customer frustration that shows up only at churn, not when it's actionable

Desired state

- Support shares one link (replay + errors + logs) and engineering has full context in seconds
- Engineers see replay + errors + logs without switching tools or asking "can you try that again?"
- AI output is traceable end to end: prompt, context, output, user reaction
- Fixes are validated against real user behavior, not just "it works on my machine"
- Frustration signals (low NPS, rage clicks) are visible immediately and tied to specific sessions
- Debugging becomes fast, predictable, and systematized

Positive outcomes

- 30-70% reduction in debugging time (reproduction to resolution)
- Fewer escalations from support to engineering
- More roadmap velocity (engineering spends time building, not debugging)
- Higher customer trust through faster, more transparent issue resolution
- Clear signal when users are frustrated, tied to exactly what went wrong

Success metrics

Customer-facing:

- CSAT/NPS improvement tied to faster issue resolution
- Mean time to resolution (MTTR) decrease
- Reduction in support-to-engineering escalation rate

TAM-facing:

- More active users in PostHog (support, CS, product teams joining engineering)
- Multi-product adoption growth (Session Replay + Error Tracking + Logs + Surveys)
- Session Replay usage increasing as debugging workflows mature

Competitive positioning

Our positioning

- **Unified visibility stack.** Behavior, replay, errors, logs, AI observability, and surveys tied to the same user. Click from an NPS score to a session replay to an error to a log line. No other platform connects all of these.

- **Developer-first tooling.** Built for teams that want control, not black-box dashboards. HogQL, API access, and transparent data model.
- **Consolidation play.** Replace multiple tools (Hotjar + Sentry + separate logging + survey tool) and cut integration overhead. One SDK, one data model, one platform.

Where we are strongest: We win when teams want behavioral and technical context in one place, engineering and product collaborate closely, AI is part of the product, and speed and simplicity matter more than enterprise ceremony.

Where we are weaker: We're not the right fit when deep distributed tracing or advanced APM is required, enterprise ITSM workflows (ServiceNow, Jira Service Management) dominate the support stack, or security policies prohibit session replay. In those cases, we complement rather than replace.

Competitor quick reference

Competitor	What They Do	Our Advantage	Their Advantage
FullStory	Session replay + digital experience analytics	Error tracking, logs, AI observability, experiments all in one platform; developer-first; better pricing	More mature DXP features; enterprise CX tooling; dedicated support workflow integrations
LogRocket	Session replay + error tracking + performance monitoring	Broader product suite (analytics, flags, experiments, surveys); AI observability; consolidation story	Purpose-built for debugging workflows; tighter Jira/Zendesk integrations out of the box
Hotjar	Session replay + heatmaps + surveys	Full analytics platform; error tracking; feature flags; engineering-grade tooling	Simpler UX for non-technical users; lower barrier to entry for marketing/UX teams
Sentry	Error tracking + performance monitoring + session replay	Deeper product analytics; session replay tied to behavior data; AI observability; surveys	More mature error tracking; broader language/framework support; larger install base
Datadog	Full observability: APM, logs, metrics, errors, RUM	Product analytics integration; session replay depth; significantly cheaper	Complete observability stack (APM, traces, metrics); enterprise-grade; massive ecosystem

Honest assessment: Our strongest position is against teams already using PostHog for analytics or feature flags who are paying separately for a replay/debugging tool. The consolidation pitch is concrete and saves money. We're weaker against teams with deeply embedded ITSM workflows (ServiceNow, PagerDuty integrations) or teams that need enterprise-grade distributed tracing. Our sweet spot is product-led companies where engineering, product, and support are closely aligned and want one platform for the full debugging loop.

Pain points & known limitations

Pain Point	Impact	Workaround / Solution
No native ticketing system integration	Support teams using Zendesk/Intercom can't auto-link replays to tickets	Share replay URLs manually in tickets. Data Pipelines can push events to external tools. Webhook integrations available for some platforms.
Logging is beta	Teams expecting production-grade centralized logging may find gaps	Set expectations on maturity. For teams with existing logging (ELK, Papertrail), PostHog logging complements rather than replaces initially.
Session replay privacy controls require configuration	Sensitive data in replays may block adoption for regulated industries	PostHog has extensive privacy controls including masking, blocking, and network payload filtering. Requires upfront configuration.
No APM or distributed tracing	Can't replace backend performance monitoring for complex microservice architectures	Be honest about the roadmap. Position PostHog as the user-facing debugging layer. Backend APM stays in their existing tool (Datadog, New Relic) for now.
Mobile replay limitations	Mobile session replay is newer and less mature than web	Check mobile replay docs for current platform support. Set expectations on feature parity with web replay.

Exceptions / edge cases:

- **Healthcare/regulated with strict PHI requirements:** Session replay may require significant masking configuration or may not be feasible. Recommend focusing on Error Tracking + Logs + Analytics without replay, or ensure their compliance team reviews PostHog's privacy controls and HIPAA BAA (available with Boost package).
- **Large enterprise with ServiceNow-centric workflows:** If their entire support operation routes through ServiceNow with complex escalation rules, PostHog is a complement (providing the debugging context), not a replacement for their ITSM platform.

Getting a customer started

What does an evaluation look like?

- **Scope:** Enable Session Replay on their primary application. Connect Error Tracking. Set up Person Profiles so support can look up individual users.
- **Timeline:** 1-2 days to start capturing replays and errors. 1 week to have enough data for support to start using it in real ticket workflows.

- **Success criteria:** Can support find a user's session when a bug is reported? Can they see errors tied to that session? Can they share a replay link with engineering that includes full context? Can they do this without escalating?
- **PostHog investment:** Session Replay free tier covers 5K recordings/month. Error Tracking free tier covers 100K exceptions/month. Product Analytics free tier covers 1M events/month.
- **Key requirement:** They need the PostHog SDK integrated with user identification so replays and errors are tied to specific users. If they're already using PostHog, this may just require enabling replay and error tracking.

Onboarding checklist

- [ ] Enable Session Replay with user identification configured
- [ ] Enable Error Tracking in the SDK configuration
- [ ] Set up Person Profiles so support can search for individual users
- [ ] Configure privacy controls for any sensitive fields (forms, PII)
- [ ] Walk support through finding a user's session and errors (training session)
- [ ] Build a "Customer Health" dashboard: error trends by account, replay volume, NPS scores
- [ ] Set up alerts for error spikes or new error types
- [ ] If applicable, enable Logging (beta) for backend context alongside replays
- [ ] If applicable, connect Surveys (NPS/CSAT) and tie responses to session data

Objection handling

Objection	Response
"We already have a session replay tool (Hotjar/FullStory/LogRocket)"	PostHog connects replay to errors, logs, analytics, and surveys in one platform. With separate tools, your support team still has to switch between 3-4 tabs to debug one issue. Consolidating also saves on vendor costs.
"Our support team isn't technical enough for PostHog"	The replay viewer is visual and intuitive. Support doesn't need to write queries. They search for a user, watch the session, and share the link. We can do a training session to get them comfortable.
"We need this integrated with Zendesk/Intercom"	You can paste replay links directly into tickets today. For automated workflows, Data Pipelines can push events to external tools via webhooks.
"Session replay has privacy concerns"	PostHog has extensive privacy controls: input masking, DOM element blocking, network payload filtering, and more. We can configure these during onboarding. HIPAA BAA is available with the Boost package.
"We're not sure this justifies adding another tool"	If you're already on PostHog for analytics or flags, this isn't another tool. It's enabling more of the platform you already pay for. If

Objection	Response
	you're not on PostHog yet, the free tiers let you evaluate without financial risk.

Cross-sell pathways from this use case

If Using...	They Might Need...	Why	Conversation Starter
Session Replay only	Error Tracking	They're watching replays to find bugs. Structured error data makes this systematic instead of manual.	"You're watching sessions to find bugs. What if errors were automatically captured and grouped so you could see which ones affect the most users?"
Session Replay + Error Tracking	Logging	They have frontend context but need backend visibility when debugging server-side issues.	"You can see the user's session and the error. But what was happening on the server at the same time?"
Session Replay + Error Tracking	Product Intelligence (for the product team)	Support and engineering are in PostHog for debugging. The product team would benefit from the same analytics for feature development.	"Your support team is using PostHog to debug issues. Has your product team seen what they can do with funnels and retention in the same platform?"
Replay + Errors + Analytics	Surveys (NPS/CSAT)	They're debugging reactively. Surveys let them detect frustration proactively and tie it to specific sessions.	"You're great at debugging reported issues. But how do you find the frustrated users who never file a ticket?"
Replay + Errors (debugging AI features)	LLM Observability	Traditional debugging misses AI-specific issues: prompt quality, hallucinations, latency.	"You're catching errors in your AI features. But are you seeing when the model gives a bad answer that isn't technically an error?"
Replay + Errors (engineering in PostHog)	Release Engineering (Feature Flags)	Engineering is in PostHog for debugging. Feature flags for safe releases is a natural add.	"You're tracking bugs after releases. What if you could gate features behind flags and roll back without a deploy?"
Group Analytics + Person Profiles	Data Infrastructure (Data Warehouse)	They want to combine PostHog user/account data with CRM or billing data for a complete customer view.	"You're looking at users in PostHog. What if you could see their Stripe revenue and HubSpot status alongside their product behavior?"

Internal resources

- **Session Replay docs:** Session Replay
- **Error Tracking docs:** Error Tracking
- **Product Analytics docs:** Product Analytics
- **Person Profiles docs:** Persons
- **Group Analytics docs:** Group Analytics
- **Surveys docs:** Surveys
- **LLM Observability docs:** AI Engineering
- **Privacy controls:** Session Replay Privacy
- **PostHog AI docs:** Enable PostHog AI · Example prompts
- **Competitive battlecard:** *To be added: FullStory / LogRocket / Hotjar competitive positioning*

Appendix: Company archetype considerations

Archetype + Stage	Framing	Key Products	Buyer
AI Native — Early	"Your AI features will break in ways that aren't exceptions. PostHog lets support see the user's session, engineering sees the error, and you can trace the LLM call that caused it. All in one place, free tier included."	Session Replay, Error Tracking, LLM Observability	CTO, founding engineer
AI Native — Scaled	"Support escalates AI issues to engineering because they can't see what the model did. PostHog gives support replay + LLM traces so they can triage without pulling engineers off the roadmap." Bridge to AI/LLM Observability and Product Intelligence.	Session Replay, Error Tracking, LLM Observability, Logging, Surveys	VP Eng, Head of Support, AI Lead
Cloud Native — Early	"Stop asking users to send screenshots. Session Replay shows you exactly what happened. Error Tracking catches it automatically. Support and engineering share the same context."	Session Replay, Error Tracking, Person Profiles	CTO, Head of Support, founding engineer
Cloud Native — Scaled	"Your support team escalates everything because they can't see errors or logs. PostHog gives them replay + errors + backend logs so they can resolve more issues without pulling in engineering." Consolidation pitch: replace FullStory/LogRocket + Sentry with one platform.	Session Replay, Error Tracking, Logging, Group Analytics, Surveys	VP Eng, Head of Support, VP CS

Archetype + Stage	Framing	Key Products	Buyer
Cloud Native — Enterprise	"Multiple teams, multiple products, and debugging context spread across 5 tools. PostHog gives support, engineering, and product a shared view: replay, errors, logs, and satisfaction data tied to the same user and account. Fewer escalations, faster resolution, better customer trust."	Full CX stack + Enterprise package (RBAC, SSO, dedicated support)	VP Eng, VP CS, Director of Support, CTO

Data Infrastructure

`contents/handbook/growth/use-case-selling/data-infrastructure.md`

What is the job to be done?

"Help me unify product data with business data and get it where it needs to go."

- Bring external data *into* PostHog from Stripe, HubSpot, Salesforce, databases, and other sources
- Combine PostHog event data with revenue data, CRM data, or other business data for unified analysis
- Query across product events and business data without building and maintaining custom ETL pipelines
- Export PostHog data to existing warehouses (Snowflake, BigQuery, Redshift) so it's part of the company's data stack
- Feed enriched data to downstream tools: BI platforms, ad platforms, CRMs, marketing tools

This is the "stickiness" use case. Once PostHog is part of a company's data infrastructure, receiving data from Stripe, HubSpot, and databases AND feeding data out to their BI layer, it becomes very hard to rip out. This also makes their product data more valuable as it is enriched with additional business context. Data infrastructure customers also tend to have the highest retention rates.

However, this is also the hardest use case to sell into. Data teams are skeptical of analytics tools playing in the data engineering space. Product maturity matters a lot here.

What PostHog products are relevant?

- **Data Warehouse (core)** — Bring external data into PostHog. Connect Stripe, HubSpot, Salesforce, Postgres, MySQL, Snowflake, BigQuery, and many more sources. Query across PostHog events and external data using HogQL. Build unified dashboards that combine product behavior with revenue, CRM, and business data.
- **Data Pipelines / Batch Exports (core)** — Send PostHog data out to external destinations. Batch exports to S3, Snowflake, BigQuery, Postgres, Redshift, Databricks, Azure Blob. Realtime destinations to Slack, HubSpot, Salesforce, ad platforms, and more. Transformations to clean, enrich, or filter data before it lands.
- **Product Analytics** — The query engine for unified data. Once external data is in the Data Warehouse, Product Analytics becomes the interface for querying across all of it. HogQL gives SQL access to everything. Dashboards combine product events with business metrics.

Adoption path and expansion path

Entry point

Usually **Data Warehouse** or **Batch Exports**. Two common patterns:

1. **Data out (Batch Exports first):** Data team wants to export PostHog event data to their existing warehouse (Snowflake, BigQuery, Redshift) so it can be queried alongside other business data in their BI tool. This is the "PostHog as a data source" entry point. They're not replacing their warehouse. They're adding PostHog data to it. Ideally, we want PostHog to be the hub of their data, but this is typically an indicator that they're beginning to think of their data holistically.
2. **Data in (Data Warehouse first):** Data team (or product/other team) wants to bring external data *into* PostHog to enrich product analytics. "Show me retention by Stripe plan" or "Which HubSpot leads are actually active in the product?" requires combining PostHog events with external data. This is the strongest entry point because it keeps teams inside PostHog for analysis.

Primary expansion path

Data Warehouse (bring external data IN) → + Product Analytics (query unified data) → + Batch Exports (send PostHog data OUT)

The logic of each step:

- Data Warehouse → Product Analytics: They've connected external data (Stripe, HubSpot, databases). Now they use Product Analytics (and HogQL) as the query interface for unified data. Dashboards combine product events with revenue data, CRM data, and more. PostHog becomes the single analytics interface.
- Product Analytics → Batch Exports: They're doing all their analysis in PostHog, but other teams or BI tools still need access to PostHog event data. Batch exports feed their warehouse so the rest of the org can benefit too.

Alternate expansion paths

Starting from Realtime Destinations: They want to push PostHog events to downstream tools in real-time. Conversion events to ad platforms (Meta, Google Ads). User activity to CRM (HubSpot, Salesforce). Alerts to Slack. This pulls in Data Pipelines and naturally leads to "if we can push data out, can we pull data in?" which is the Data Warehouse.

Starting from Product Analytics (HogQL power users): Advanced analytics users writing HogQL queries hit the ceiling of PostHog-only data. They want to join against their Stripe data, their CRM data, or their database. Data Warehouse is the answer.

Business impact of solving the problem

This is the highest-stickiness use case. When PostHog is both receiving data from Stripe/HubSpot/databases and feeding data out to Snowflake/BigQuery/BI tools, ripping it out means rebuilding multiple data pipelines. This creates deep infrastructure-level lock-in that goes beyond any single user or team.

Data infrastructure customers have the highest retention rates. Accounts with active batch exports and warehouse connections churn at significantly lower rates than analytics-only accounts. The integration depth creates switching costs that product satisfaction alone doesn't.

However, this is the hardest use case to sell into. Data teams are skeptical. They've built their stack around tools like Fivetran, dbt, Snowflake, and Looker. They see PostHog as an analytics tool, not a data infrastructure tool. Credibility with data engineers requires demonstrating real technical capability, not just talking about consolidation.

The "lightweight warehouse" pitch resonates with early-stage companies. Teams that don't yet have a Snowflake/BigQuery setup find PostHog's Data Warehouse attractive because it gives them warehouse capabilities (join external data, run SQL) without a separate warehouse vendor. For these teams, PostHog isn't replacing their warehouse. It *is* their warehouse.

Personas to target

Persona	Role Examples	What They Care About	How They Evaluate
Data Engineer	Data Eng, Analytics Eng, Data Platform	Pipeline reliability, query performance, schema flexibility, not maintaining custom ETL	"Is the sync reliable? Can I run complex joins? What's the query latency on large datasets?"
Data Team Lead	Head of Data, Director of Analytics, Data Lead	Tool consolidation, cost, team productivity, data governance	"Does this reduce our pipeline maintenance burden? What's the cost vs. Fivetran?"
Product Ops / BizOps	Product Ops, RevOps, BizOps	Unified view of product and business data, self-serve dashboards	"Can I see product usage next to Stripe revenue and HubSpot pipeline without asking the data team?"
Founder (early stage)	CTO, technical founder, first data hire	Not building a data warehouse yet. Wants unified analytics without a complex stack.	"Can I query my Stripe data alongside PostHog events without setting up Snowflake?"

Signals in Vitally & PostHog

Vitally indicators this use case is relevant

Signal	Where to Find It	What It Means
Active batch exports	`active_batch_exports` in Vitally traits	They're already exporting data. The Data Warehouse (bringing data in) is the natural next step, and they're likely not thinking of PostHog being their data warehouse.
Active external data schemas	`active_external_data_schemas` in Vitally traits	They've connected external data sources. They're using PostHog as a data platform, not just analytics.
High rows synced (30 day)	`rowsSyncedLast30DaysIfSendingData`	Significant data movement. Data Infrastructure is an active use case.
Customer mentions Fivetran, Snowflake, or "data warehouse" in notes	Vitally notes / conversations	Data team is involved. This use case may be relevant.
HogQL usage is high	Usage metrics	Power users writing SQL. They're likely to want to query across external data too, or have more complex analytics needs/capabilities.

PostHog usage signals

Signal	How to Check	What It Means
Batch exports configured and running	Pipeline configuration	They're exporting data. Explore whether bringing data in (Data Warehouse) would add value. What are they doing with the PostHog data in the warehouse?
External data sources connected (Stripe, HubSpot, etc.)	Data Warehouse source list	Active Data Infrastructure use case. Look for expansion: more sources, more query complexity.
HogQL queries joining external data	Saved insights with warehouse tables	They're doing unified analysis. This is the power use case. Encourage more connections.
High realtime destination volume	Pipeline metrics	They're pushing events to downstream tools. Explore whether they need more destinations or more complex transformations.

Signal	How to Check	What It Means
		They may also be solving in point solutions when they could simplify in PostHog.

Command of the Message

Discovery questions

- Where does your product data live today? How do you combine it with business data (revenue, CRM, etc.)?
- Do you have a data warehouse (Snowflake, BigQuery, Redshift)? How does PostHog data get there?
- When someone asks "what's our retention by Stripe plan?" or "which HubSpot leads are active in the product?", how long does it take to answer?
- How many custom ETL pipelines are you maintaining? How much engineering time goes into keeping them running?
- What tools do you use for data pipelines today? (Fivetran? Airbyte? Custom scripts?)
- Do you push PostHog data to any downstream tools? (BI, CRM, ad platforms)
- If you could query your Stripe/HubSpot/database data alongside PostHog events in one place, what questions would you ask first?

Negative consequences (of not solving this)

- Product data and business data live in separate silos. Answering cross-domain questions requires custom ETL, manual exports or switching between multiple tools/dashboards
- Just as bad as difficult to answer questions - teams give different answers to the same question
- Data engineers spend time maintaining pipelines between PostHog and the warehouse instead of doing analysis
- "Which cohort of users drives the most revenue?" requires stitching data from PostHog, Stripe, and the CRM, which takes days instead of minutes
- PostHog events aren't in the warehouse, so BI dashboards that need product behavior data are stale or incomplete
- Conversion events don't flow back to ad platforms, so marketing can't optimize campaigns against real product data

Desired state

- External data automatically flows into PostHog for enriched product analytics
- Product events, revenue data, CRM data, and database data queryable in one place
- Anyone can build a dashboard that combines product behavior with business outcomes
- No custom ETL pipelines to maintain between PostHog and the rest of the data stack
- PostHog data automatically flows to the warehouse for BI and downstream analysis

Positive outcomes

- Deeper product analytics: join against revenue, CRM, and business data for richer insights
- Faster time-to-answer for cross-domain questions (retention by plan, revenue by feature, engagement by lead score)
- Reduced engineering time maintaining data pipelines
- External data available in PostHog for teams that prefer PostHog's analytics interface
- Account becomes deeply embedded in PostHog's ecosystem (high switching cost, low churn risk)
- PostHog data available in BI tools for teams that prefer Looker/Mode/Metabase

Success metrics

Customer-facing:

- Cross-domain queries (product + business data) that previously took days now take minutes
- Pipeline maintenance burden decreases (fewer custom ETL jobs)
- More teams can self-serve analytics because the data is unified
- Every team is working from the same data

TAM-facing:

- External data sources connected grows (more data flowing in)
- HogQL queries increasingly reference warehouse tables (unified analysis happening)
- Account retention strengthens (infrastructure-level stickiness)
- Batch export volume grows (more data flowing out)

Competitive positioning

Our positioning

- **Query across everything with HogQL.** Join PostHog events with Stripe revenue data, HubSpot contacts, or your Postgres database in a single SQL query. No separate BI tool required for many use cases.
- **Built into the analytics platform.** The Data Warehouse isn't a separate product. It's integrated with Product Analytics, dashboards, cohorts, and every other PostHog feature. External data becomes first-class data.
- **Lightweight warehouse for early-stage teams.** Teams without Snowflake/BigQuery get warehouse capabilities as part of PostHog. No separate vendor, no separate setup.
- **Bidirectional data flow.** Data Warehouse brings external data *into* PostHog. Batch Exports and Pipelines push PostHog data *out*. Two-way integration with the customer's data stack, not just one direction.

Competitor quick reference

Competitor	What They Do	Our Advantage	Their Advantage
Snowflake / BigQuery	Cloud data warehouse	We have analytics built on top; no BI tool needed for product questions; simpler for teams that just need PostHog + business data	Real data warehouse: unlimited scale, advanced SQL, mature ecosystem, governance
Fivetran	Managed data pipelines (sources to warehouse)	We're the analytics platform AND the pipe; data stays in PostHog for analytics; simpler for early-stage teams	Far more source connectors; more mature data governance; enterprise-grade reliability
Census / Hightouch	Reverse ETL (warehouse to business tools)	We push data from PostHog directly, no warehouse intermediate step needed; simpler architecture	More destination integrations; audience management features; built for marketing/ops teams
Segment	CDP (collect events, route to destinations)	We're the analytics platform AND the pipe; no separate CDP needed	More destination integrations; more mature event collection; established in enterprise CDP workflows

Honest assessment: We are not trying to replace Snowflake or BigQuery. For teams with a mature data stack (Fivetran + Snowflake + dbt + Looker), PostHog's Data Warehouse is a complement, not a replacement. Batch Exports feed PostHog data into their stack; Data Warehouse brings their data into PostHog for product-specific analysis. The full replacement pitch only works for early-stage teams that don't have a warehouse yet and want PostHog to serve double duty. Early stage teams may also have experience with the complexity of layering in data systems, so may be more open to centralizing tooling, and Batch Exports always allow teams to not feel vendor lock-in. Be calibrated about which accounts can realistically adopt this as infrastructure vs. a convenience feature.

Pain points & known limitations

Pain Point	Impact	Workaround / Solution
Data Warehouse query performance at very large scale	Teams with billions of rows in external sources may hit performance limits	PostHog's Data Warehouse is optimized for product analytics query patterns, not general-purpose warehousing. For very large datasets, batch exports to Snowflake/BigQuery may be more appropriate.
Source connector coverage doesn't match Fivetran	Some niche data sources may not be supported	Check available sources. For unsupported sources, the API and S3/GCS import paths can bridge the gap.

Pain Point	Impact	Workaround / Solution
Data engineering teams may not trust PostHog as a warehouse	Credibility gap: "you're an analytics tool, not a data platform"	Don't oversell. Position as a complement to their existing stack (batch exports out, key sources in) rather than a full replacement. Demonstrate HogQL query capability with their actual data to build credibility.
Batch export latency may not meet real-time requirements	Teams needing sub-minute data freshness in their warehouse	Batch exports are periodic (hourly default). For real-time needs, use Realtime Destinations instead. Set expectations on latency during evaluation.

Getting a customer started

What does an evaluation look like?

- **Scope:** Connect one external data source (usually Stripe or their primary database) to PostHog Data Warehouse. Set up one batch export to their warehouse if they have one. Build a dashboard that joins PostHog events with external data.
- **Timeline:** 1 to 3 days to connect sources and exports. 1 week to build meaningful unified dashboards.
- **Success criteria:** Can you query across PostHog events and external data in HogQL? Can you build a dashboard showing "retention by Stripe plan" or "engagement by CRM stage"? Is the batch export reliably delivering data to their warehouse?
- **PostHog investment:** Data Warehouse free tier covers 1M rows/month + free historical syncs. Batch Exports free tier covers 1M rows/month.
- **Key requirement:** They need API credentials for the external data sources they want to connect. For batch exports, they need write access to their warehouse.

Onboarding checklist

- [ ] Connect primary external data source to Data Warehouse (usually Stripe or primary database)
- [ ] Verify data is syncing correctly and queryable via HogQL
- [ ] Build a unified query: join PostHog events with external data (e.g., "retention by Stripe plan")
- [ ] Set up batch export to their warehouse (Snowflake, BigQuery, Redshift, S3)
- [ ] Verify batch export is running reliably with expected data freshness
- [ ] Configure at least one Realtime Destination if they need event data in downstream tools (Slack alerts, CRM sync, ad platform conversions)
- [ ] Build a "Unified Analytics" dashboard combining product events with business data
- [ ] Introduce the data team to HogQL if they're SQL-comfortable (HogQL docs)
- [ ] Identify additional data sources to connect (CRM, other databases, ad platforms)

Cross-sell pathways from this use case

If Using...	They Might Need...	Why	Conversation Starter
Batch Exports only	Data Warehouse (bring data in)	They're pushing PostHog data out. Bringing business data in would let them do unified analysis in PostHog directly.	"You're exporting PostHog data to Snowflake. What if you could bring your Stripe data *into* PostHog and skip the context-switch?"
Data Warehouse (Stripe connected)	Revenue Analytics	They've connected Stripe data. Revenue Analytics gives them pre-built MRR, LTV, and churn dashboards.	"You've got Stripe connected. Have you seen Revenue Analytics? It gives you MRR, churn, and LTV dashboards out of the box."
Data Pipelines to CRM	Growth & Marketing	They're pushing data to HubSpot/Salesforce. The growth team could use more of the marketing analytics stack.	"You're syncing data to your CRM. Has the marketing team seen Web Analytics and Marketing Analytics for attribution?"
Data Warehouse + Product Analytics	Product Intelligence (for the product team)	They're doing unified data analysis. The product team should be using the full analytics suite.	"Your data team is doing advanced queries. Are your PMs using funnels, retention, and session replay for product decisions?"
Data team in PostHog	Any use case for other teams	Data team is in PostHog and advocates for it. Expand to product, engineering, or growth.	"Your data team loves PostHog. Which other teams could benefit? Product? Engineering? Growth?"

Internal resources

- **Data Warehouse docs:** Data Warehouse · Sources · HogQL
- **Data Pipelines docs:** CDP overview · Batch exports · Realtime destinations · Transformations
- **Revenue Analytics docs:** Getting started · Dashboard
- **External data source guides:** Stripe · HubSpot · Salesforce · Postgres
- **Batch export guides:** S3 · Snowflake · BigQuery

Appendix: Company archetype considerations

Archetype + Stage	Framing	Key Products	Buyer
AI Native — Early	"You don't need a data warehouse yet. PostHog connects to Stripe and your database, so you can query everything in one place without setting up Snowflake." Lightweight warehouse pitch.	Data Warehouse, Product Analytics (HogQL)	CTO, founding engineer, first data hire
AI Native — Scaled	"You're scaling and your data team is building a proper stack. PostHog batch exports feed your warehouse, and Data Warehouse brings key business data in for product analytics." Complement, not replace.	Data Warehouse, Batch Exports, Pipelines	Data team lead, analytics engineer
Cloud Native — Early	"Same as AI Native early. PostHog as the lightweight warehouse for teams that don't want a separate data stack yet."	Data Warehouse, Product Analytics (HogQL)	CTO, first data hire
Cloud Native — Scaled	"Your data stack is mature. PostHog fits in as both a data source (batch exports to Snowflake) and an analytics destination (Data Warehouse pulls in Stripe/HubSpot). No custom ETL needed."	Batch Exports, Data Warehouse, Pipelines	Data engineering team, analytics engineering team
Cloud Native — Enterprise	"Multiple teams, multiple data sources, complex pipeline requirements. PostHog integrates bidirectionally with your existing stack and gives product/growth teams self-serve analytics over unified data." Governance, reliability, and scale matter here.	Full Data Infrastructure stack + Enterprise package	Head of Data, Director of Analytics, Data Platform Lead

Appendix: PostHog data maturity

Stage	Primary Tool	Data Sources	Who Owns	PostHog Position
1	Point solutions (GA, prod DB)	Scattered	Nobody	Not yet adopted
2	PostHog	Product events	Prod/Eng	Primary analytics
3	PostHog + Data Pipelines	Product + Business	Cross-functional	Hub for analytics
4	PostHog + Data Pipelines + Warehouse	Everything	Cross-functional	Source of truth

Stage	Primary Tool	Data Sources	Who Owns	PostHog Position
5	PostHog + Batch Exports + External warehouse	Everything	Data Team	Source + destination

Growth & Marketing

`contents/handbook/growth/use-case-selling/growth-and-marketing.md`

What is the job to be done?

"Help me understand what drives acquisition, conversion, and revenue, and automate actions based on user behavior."

- Know which channels and campaigns actually drive signups and revenue, not just clicks
- Build and optimize conversion funnels from first touch through activation and monetization
- Attribute revenue back to marketing spend and understand true ROAS
- Automate engagement based on user behavior: onboarding nudges, re-engagement, lifecycle campaigns
- Get conversion and behavior data into ad platforms, CRMs, and marketing tools without building custom pipelines
- Run experiments on landing pages, pricing, onboarding flows, and activation sequences
- Collect on-site feedback (exit intent, NPS, CSAT) and tie it directly to user behavior
- Let non-technical marketing users ask questions about their data without waiting for an analyst

> **Guidance:** This is probably the most underserved use case in our current motion. We have the products — Web Analytics, Marketing Analytics, Workflows, Product Tours, Pipelines, Revenue Analytics, Surveys — but we rarely lead with this story. Marketing teams are spending $10k+/month on Segment, Mixpanel, GA4, and various CDPs to do what PostHog can do in one place. Don't sell individual products here. Sell the consolidation of their marketing data stack.

What PostHog products are relevant?

- **Web Analytics (core)** — Traffic, referrers, UTM tracking, page performance, bounce rates. The replacement for GA4 that doesn't require a PhD to configure. First-party data collection that actually works with ad blockers. (Dashboard overview)
- **Marketing Analytics** *beta* — Ad campaign attribution, channel performance, ROAS tracking. Connect ad spend to actual product signups and revenue events. Multi-touch attribution across channels.
- **Product Analytics** — Conversion funnels, retention curves, cohort analysis, activation metrics. The layer that connects "they visited the site" to "they became a paying customer." Lifecycle analysis to understand where users are in the journey.
- **Workflows** — Automated engagement sequences triggered by user behavior. Lifecycle emails, re-engagement campaigns, onboarding drips, churn prevention. Act on what analytics reveals instead of just reporting on it. (Email drip campaign guide · Configure channels)

- **Product Tours** *alpha* — In-app guided onboarding, activation nudges, feature adoption prompts, conversion nudges. The in-product complement to Workflows' out-of-product engagement. (Creating tours)
- **Data Pipelines** — Push conversion events and user data to ad platforms (Google, Meta, LinkedIn), CRMs (HubSpot, Salesforce), and data warehouses. Close the loop on campaign optimization by feeding real conversion data back to where it's used. (Realtime destinations · Batch exports)
- **Revenue Analytics** — Track revenue by cohort, plan, feature, and channel. Understand LTV, MRR, expansion revenue, and churn at a user and account level. (Dashboard)
- **Surveys** — On-site feedback, exit-intent surveys, NPS, CSAT, post-purchase surveys. Capture qualitative signal at key moments in the funnel and tie responses to user behavior data.
- **Experiments** — A/B test landing pages, pricing pages, onboarding flows, checkout experiences, and activation sequences against real conversion and revenue metrics. This is a key stickiness driver: once a growth team is running experiments, they need engineering to implement the variants, which pulls engineering into PostHog and creates a cross-team dependency.
- **Feature Flags** — The implementation layer for experiments. Growth/CRO defines the test; engineering implements it via feature flags. Also used for targeted rollouts to specific user segments, geo-targeting, and progressive delivery of growth initiatives. Feature Flags are the bridge product that connects the growth team's use case to the engineering team's workflow, and opens the door to the Release Engineering use case.
- **PostHog AI** — Natural language querying for non-technical marketing users. "Which campaign drove the most signups last month?" without needing HogQL or analyst support. (Example prompts)

Adoption path and expansion path

Entry point

Usually **Web Analytics**, **Product Analytics**, or **Experiments**. Three common patterns:

1. **Marketing-first:** Marketing team wants to replace GA4 or understand channel attribution. They start with Web Analytics for traffic and referrer data, then quickly want to connect that to downstream conversion events (Product Analytics) and campaign spend (Marketing Analytics).
2. **Growth-first:** A growth engineer or product-led growth team is already using PostHog for product analytics — building funnels, tracking activation, measuring retention. They want to connect the top of funnel (how users found us) to the bottom (did they convert and retain). Web Analytics and Marketing Analytics extend their existing setup upstream.
3. **CRO / Experimentation-first:** A Growth PM or CRO specialist wants to run A/B tests on signup flows, pricing pages, or onboarding sequences. They come in through Experiments, which requires Feature Flags, and Feature Flags require engineering to implement. This is a natural multithreading play: the growth team defines the experiment, engineering implements the flag, and now both teams are in PostHog.

Primary expansion path

Web Analytics → Marketing Analytics → Product Analytics (funnels/retention) → Experiments + Feature Flags → Data Pipelines (to CRM/ad platforms) → Workflows / Product Tours → Revenue Analytics → Surveys

The logic of each step:

- Web Analytics → Marketing Analytics: They can see traffic and referrers but want to connect that to actual ad spend and ROAS.
- Marketing Analytics → Product Analytics: They know which channels bring users, now they need to know which channels bring users *who convert and retain*.
- Product Analytics → Experiments + Feature Flags: They've identified drop-off points and want to test fixes. Experiments require Feature Flags, which require engineering to implement. **This is the key multithreading moment:** the growth team defines the hypothesis, engineering implements the flag, and both are now active in PostHog.
- Experiments → Data Pipelines: They've validated what works, now they need to feed real conversion events back to ad platforms and push user data to their CRM.
- Pipelines → Workflows / Product Tours: They've been reporting on drop-offs and now want to actually *do something* about them. Automated re-engagement when a user goes cold. In-app nudges to complete onboarding.
- Workflows → Revenue Analytics: Engagement is automated, now they want to measure the revenue impact. LTV by channel, revenue attribution to specific campaigns and workflows.
- Revenue Analytics → Surveys: They're measuring revenue impact and want to add a qualitative layer. "Why did you cancel?" exit surveys. Post-purchase NPS.

Alternate expansion paths

Starting from Product Analytics (growth engineering): A growth team already deep in PostHog funnels and experiments. They expand upstream into Web Analytics and Marketing Analytics for channel attribution, and downstream into Workflows and Product Tours for activation automation.

Starting from Surveys: A product or CX team is running NPS or CSAT surveys. They want to connect low scores to actual behavior (what happened right before someone gave a 3/10?), which pulls in Product Analytics and Session Replay. The growth team then sees the survey infrastructure and wants to use it for exit-intent and post-signup feedback.

Starting from Experiments (CRO / Growth PM entry — the engineering bridge): A CRO specialist or Growth PM wants to A/B test their signup flow. They come in through Experiments, which creates a Feature Flag under the hood. The flag needs to be implemented in code, so engineering gets pulled into PostHog. This is high-value for three reasons: (1) **it makes the account sticky** — once feature flags are in the codebase, they're not easy to rip out; (2) **it creates a multithreading opportunity** — you now have both the growth team and engineering as active users; and (3) **it's a bridge to Release Engineering** — once engineering is using flags for experiments, they often realize they can use the same infrastructure for progressive rollouts and kill switches.

Business impact of solving the problem

The buyer is different from other use cases. Growth and Marketing targets growth engineers, marketing leads, demand gen managers, CRO specialists, and GTM engineers. In most organizations, these are separate from the product analytics buyer (PM) and the engineering buyer (EM/platform). They often have their own budget and their own stack. Winning this buyer opens a parallel revenue stream within the same account.

Marketing stack consolidation is a real, quantifiable cost savings. Companies routinely spend $10k+/month across GA4, Segment, Mixpanel, Amplitude, CDPs, and various point solutions. The consolidation argument is concrete: fewer vendor contracts, fewer integrations to maintain, one source of truth for conversion data.

This use case gives newer products a reason to exist. Workflows, Product Tours, Marketing Analytics, and Revenue Analytics are all relatively new PostHog products with lower attach rates. Without a use case frame, they're standalone features looking for a buyer. Within Growth and Marketing, each one has a clear role and a natural "next step" in the conversation.

Growth and Marketing creates demand for other use cases. Once a marketing team is in PostHog and sees the depth of product analytics, they pull in the product team (Product Intelligence). Once the growth team is running experiments, engineering gets involved (Release Engineering). This use case is a wedge into broader platform adoption.

Experiments and Feature Flags are the stickiness and multithreading lever. When a CRO or Growth PM starts running A/B tests, feature flags get embedded in the codebase. That's a fundamentally different level of integration than a marketing team viewing dashboards. Flags are in production code, maintained by engineers, and not easy to remove. More importantly, it gives TAMs a natural path to multithread: you now have a growth/marketing champion *and* an engineering champion using the same platform.

Personas to target

Persona	Role Examples	What They Care About	How They Evaluate
Growth Engineer	Growth Eng, PLG Engineer, GTM Engineer	Conversion funnels, activation metrics, experiment velocity, pipeline reliability	"Can I build a full-funnel view from ad click to paid conversion in one tool?"
Marketing Lead	Head of Marketing, VP Demand Gen, Marketing Ops	Channel attribution, ROAS, campaign performance, cost per acquisition	"Can I see which campaigns actually drive revenue, not just clicks?"
CRO / Growth PM	Growth PM, CRO Specialist, Head of	Conversion rate optimization, experiment velocity, activation rates.	"Can I run experiments on our signup flow and measure

Persona	Role Examples	What They Care About	How They Evaluate
	Growth	Needs engineering to implement experiments, making this persona the key multithreading catalyst.	revenue impact? How fast can engineering implement a test?"
Founding Growth	Founder, first growth hire at early-stage startup	All of the above. Wearing all hats. Speed, simplicity, not paying for 5 tools	"How fast can I set this up and how many tools does it replace?"
Marketing Analyst	Marketing Analyst, Data Analyst (Marketing)	Data accuracy, attribution modeling, cohort analysis, reporting	"Can I trust this data? Can I build reports without engineering help?"

Signals in Vitally & PostHog

Vitally indicators this use case is relevant

Signal	Where to Find It	What It Means
Web Analytics is active but no other products adopted	Product usage data	They came in through the marketing door — there's a full expansion path waiting
Customer mentions GA4, Segment, or CDP in notes	Vitally notes / conversations	They have marketing stack pain and may be open to consolidation
Multiple marketing/growth team members invited	User list in Vitally	The growth team is in PostHog, not just engineering — this use case is live
Low Pipelines / Workflows usage despite high analytics usage	Product spend breakdown	They're analyzing but not acting — Workflows and Pipelines are natural next steps
Experiments or Feature Flags usage initiated by growth/marketing team (not engineering)	Product usage data + user roles	The CRO/Growth PM persona is active — this is the engineering bridge moment

PostHog usage signals

Signal	How to Check	What It Means
UTM parameters appearing in event properties	Event property explorer	They're tracking acquisition sources — Marketing Analytics is a natural add

Signal	How to Check	What It Means
Funnels built around signup/checkout/activation	Saved insights	Growth team is active and measuring conversion — ripe for Experiments and Workflows
Experiments created but low flag evaluation volume	Experiments list + flag usage	Growth team is trying to experiment but engineering hasn't implemented the flags yet — TAM opportunity to facilitate the handoff
Feature flags being used primarily for experiments (not releases)	Flag list + experiment linkage	Growth-driven flag usage — explore whether they'd also use flags for progressive rollouts (Release Engineering cross-sell)
Web Analytics pageview volume growing	Product usage metrics	Marketing is driving more traffic — they'll want attribution and ROAS soon
Batch exports configured to ad platforms or CRM	Pipeline configuration	They're already trying to close the data loop — deeper Pipelines usage is the play

Health score implications

- **Event volume:** Growing web analytics and pageview volume means marketing is scaling. Flat or declining volume may mean they've stalled or are sending traffic data elsewhere.
- **User engagement:** Watch for non-engineering users actively building dashboards and insights. If only engineers use PostHog, the marketing team hasn't been onboarded — that's both a risk and an opportunity.
- **Product count:** Growth and Marketing touches the most products of any use case. Low product count with this persona is a sign there's major expansion headroom. If they're using analytics but not Experiments + Feature Flags, that's the next natural move.

Command of the Message

Discovery questions (current state)

- How do you track which channels and campaigns drive signups today? Can you tie that all the way through to revenue?
- What does your current marketing/growth tool stack look like? (GA4? Segment? CDP? How many vendors?)
- When you run a paid campaign, how do you measure whether it actually worked? How long does it take to get that answer?
- Do you send conversion events back to your ad platforms? How is that pipeline built and maintained?
- How do you currently onboard new users? Is it automated or manual? What triggers the onboarding flow?

- When someone drops off your signup or checkout funnel, can you see why? Do you have any automated re-engagement?
- How does your growth team decide what to experiment on? How do you measure experiment results?
- Are you running A/B tests on your signup flow, pricing page, or onboarding today? What tool are you using? Who implements the variants, the growth team or engineering?
- When an experiment wins, how do you roll it out to 100% of users? Is that process smooth or does it require a separate deploy?
- Can your marketing team answer their own questions about performance, or do they depend on engineering/data for every query?
- How do you attribute revenue to specific campaigns, channels, or touchpoints?
- What's your biggest frustration with your current analytics or attribution setup?

Negative consequences (of not solving this)

- Marketing spend is optimized against proxy metrics (clicks, impressions) instead of actual conversions and revenue
- Attribution is broken or incomplete — nobody trusts the numbers, so decisions are gut-driven
- Conversion data doesn't flow back to ad platforms, so campaign optimization is flying blind
- Growth team builds custom ETL pipelines to move data between tools — fragile, expensive to maintain
- Onboarding and re-engagement are manual or time-based instead of behavior-driven
- Marketing and product teams use different tools with different numbers, leading to misalignment
- Growth team can't run experiments because they depend on engineering for every test, or they use a separate tool that isn't connected to their analytics
- New users drop off and nobody acts on it because there's no automation layer

Desired state

- One platform that tracks the full journey from ad click to paid conversion to revenue
- Marketing team can self-serve answers about channel performance, ROAS, and conversion without waiting for analysts
- Conversion events automatically flow to ad platforms and CRMs — no custom pipelines to maintain
- Onboarding, re-engagement, and lifecycle campaigns fire automatically based on real user behavior
- In-app nudges guide users through activation at exactly the right moment
- Every experiment is measured against real business metrics, and the same feature flags used for experiments can be reused for progressive rollouts
- Growth and engineering collaborate through a shared platform: growth defines the hypothesis, engineering implements the flag, both see the results
- Revenue is attributable to specific channels, campaigns, and user cohorts

Positive outcomes

- 20-40% reduction in marketing tool spend through consolidation (GA4 + Segment + CDP + point solutions → PostHog)
- Higher ROAS from feeding real conversion data back to ad platforms
- Faster experiment velocity — growth team runs more tests because the tooling is integrated
- Experiments + Feature Flags create a shared workflow between growth and engineering, reducing silos and making the account harder to churn
- Increased activation and retention from behavior-driven onboarding (Workflows + Product Tours)
- Marketing and product aligned on the same data, same source of truth
- Non-technical marketing users can query data in natural language via PostHog AI

Success metrics

Customer-facing:

- Conversion rate from visitor → signup → activation → paid improves measurably
- ROAS on paid channels increases after feeding real conversion events back to ad platforms
- Time-to-first-value for new users decreases (measured via activation funnel)
- Marketing stack vendor count decreases (consolidation)
- Growth team experiment velocity increases (more experiments shipped per quarter)

TAM-facing:

- Customer expands from Web Analytics-only (or Product Analytics-only) to multi-product
- Non-engineering users (marketing, growth) are active in PostHog
- Engineering users are active alongside growth users (multithreaded account)
- Feature Flags are embedded in the codebase (stickiness indicator)
- Experiments velocity increases (more experiments created per quarter)
- Pipeline volume grows (more data flowing out to ad platforms and CRMs)
- Workflow and Product Tour usage grows (automation is active, not just analytics)

Competitive positioning

Our positioning

- **Full-funnel in one platform.** No other tool connects web traffic → channel attribution → conversion funnels → user behavior → revenue → automated engagement in a single product. GA4 stops at the website. Segment stops at the pipe. Amplitude stops at the dashboard. PostHog goes from first click to lifetime revenue *and* lets you act on it.
- **First-party data collection that works.** PostHog's first-party tracking isn't blocked by ad blockers the way GA4 and third-party pixels are. More accurate data, better attribution, higher match rates when syncing conversions to ad platforms.

- **Analytics + automation in the same tool.** Most analytics platforms show you the drop-off. PostHog lets you fix it with Workflows (re-engage users) and Product Tours (guide users in-app). The insight-to-action loop is closed.
- **Marketing stack consolidation = real cost savings.** Replace GA4 + Segment + CDP + survey tool + experimentation tool with one platform.
- **PostHog AI lowers the adoption bar for non-technical users.** Marketing users who will never learn SQL or HogQL can ask questions in natural language.

Competitor quick reference

Competitor	What They Do	Our Advantage	Their Advantage
GA4	Web analytics, basic attribution, Google Ads integration	Full-funnel beyond the website; first-party data; product analytics depth	Deepest Google Ads integration; free tier is very generous; universal adoption
Segment	CDP — collects events and routes them to destinations	We're the analytics platform *and* the pipe; no need for a separate CDP layer	More destination integrations; more mature data governance
Amplitude	Product analytics with some marketing analytics features	Broader product coverage (flags, replay, surveys, workflows); better pricing	More mature marketing-specific features (audiences, campaign impact)
Mixpanel	Product analytics focused on funnels and retention	Broader platform (web analytics, flags, replay, workflows); no sampling	Deeper mobile analytics; some marketing teams prefer the UX
HubSpot Marketing Hub	Marketing automation, email, CRM, basic analytics	Engineering-grade analytics; deeper funnel analysis; experiments	Native CRM integration; better email deliverability; non-technical UX
Heap	Auto-capture product analytics	We also auto-capture, plus flags, experiments, replay, surveys, workflows	Retroactive analytics (virtual events) is a strong pitch for non-technical teams

Honest assessment: Our strongest position is against teams using 3+ tools to do what PostHog does in one. The consolidation pitch is genuine. We're weaker against teams deeply embedded in the Google ecosystem (GA4 + Google Ads + Looker) where switching cost is high. We're also weaker against HubSpot where marketing automation is the primary need. Our sweet spot is technical growth teams and PLG companies where the growth engineer is the buyer.

Pain points & known limitations

Pain Point	Impact	Workaround / Solution
Marketing Analytics is beta — feature set is still maturing	Some customers may expect parity with GA4 or dedicated attribution tools	Set expectations during onboarding. Position as "growing fast" and highlight the advantage of attribution data living alongside product analytics.
Workflows is new — not as feature-rich as mature marketing automation	Teams expecting advanced email sequencing, lead scoring, or complex branching may find gaps	Position as behavior-driven automation, not a full HubSpot replacement. For heavy email automation, PostHog complements an existing tool via Data Pipelines.
Product Tours is alpha — limited customization	Teams with complex onboarding needs may hit walls	Position as the integrated option. For advanced tooltip/modal UX, keep a dedicated tool and use PostHog for analytics + experimentation.
Pipeline destination coverage may not match Segment's breadth	Some niche destinations may not be supported	Check available destinations before promising. Data Warehouse + Batch Exports covers the most common needs. Webhook destination can bridge gaps.
Non-technical marketing users may find the UI intimidating	Adoption risk: marketing team tries PostHog, finds it too "engineering-y," and reverts to GA4	Lead with PostHog AI for querying. Build pre-configured dashboards during onboarding. Web Analytics UI is intentionally simpler — start them there.

Exceptions / edge cases:

- **Enterprise demand gen teams with complex lead scoring and email nurture:** If the primary need is marketing automation, PostHog is not the right primary tool. Recommend keeping HubSpot/Marketo and using PostHog for analytics, attribution, and experimentation. Data Pipelines bridges the two.
- **Teams deeply embedded in the Google ecosystem:** If they run Google Ads, use GA4, and report in Looker, switching cost is very high. Position PostHog as a complement for product analytics and conversion funnel depth, not a full GA4 replacement. Over time, as Marketing Analytics matures, the replacement conversation becomes easier.

Getting a customer started

What does an evaluation look like?

- **Scope:** Instrument their primary acquisition funnel: landing page → signup → activation event → first conversion/payment. Add UTM tracking and connect web analytics. If they have paid campaigns, set up Marketing Analytics.

- **Timeline:** 2 to 4 weeks to see meaningful data. Channel attribution and funnel insights start showing value within the first week if traffic is decent. Experiments need enough traffic for statistical significance, so timeline varies.
- **Success criteria:** Can you answer: "Which channel drives the most *activated* users (not just signups)?" Can you see the full funnel from first visit to conversion? Can you tell which campaigns are worth the spend?
- **PostHog investment:** Web Analytics and Product Analytics free tiers cover a substantial evaluation. Marketing Analytics (beta) is included. Surveys and Experiments have generous free tiers.
- **Key requirement:** They need to instrument key conversion events (signup, activation, purchase/upgrade) with proper UTM parameters. If they want Pipelines, they need API credentials for their ad platforms or CRM. See the performance marketing tutorial for a step-by-step walkthrough.

Onboarding checklist

- [ ] Install PostHog snippet on marketing site and product (web analytics installation + autocapture)
- [ ] Verify UTM parameters are being captured on key landing pages
- [ ] Define and instrument core conversion events: signup, activation, first purchase/upgrade
- [ ] Build the primary conversion funnel in Product Analytics
- [ ] Set up Web Analytics dashboard with traffic, referrers, and top pages
- [ ] Connect Marketing Analytics to ad platforms (if running paid campaigns)
- [ ] Configure at least one Data Pipeline destination (CRM or ad platform conversion sync)
- [ ] Build a pre-configured "Growth Dashboard" for the marketing team
- [ ] Introduce PostHog AI to non-technical marketing users (example prompts)
- [ ] Set up one Workflow or Product Tour for a key drop-off point

Objection handling

Objection	Response
"We already use GA4 and it's free."	GA4 is great for basic web traffic. But can it show you which channels drive users who *activate and pay*, not just visit? Can it send real conversion events back to your ad platforms? PostHog starts free too, and it goes all the way to revenue. (Web Analytics · Funnels)
"We need Segment for our data pipelines."	What destinations are you sending to? PostHog has built-in Data Pipelines for the most common ones. You may not need a separate CDP layer if PostHog is already collecting the events. Let's look at your current destinations and see what's covered.

Objection	Response
"Our marketing team isn't technical enough for PostHog."	That's exactly why we built PostHog AI — your marketing team can ask questions in plain English. Web Analytics is also designed to be simple and familiar. We'll set up dashboards during onboarding so they have value from day one.
"Marketing Analytics is beta — can we trust it?"	Fair concern. The core data infrastructure is built on the same battle-tested PostHog platform that handles billions of events. The beta label means we're still adding features, not that the data is unreliable. And your feedback directly shapes the roadmap.
"We'd need to rip out our whole marketing stack to use PostHog."	You don't have to rip out anything on day one. Start by adding PostHog alongside your existing tools. Once you see the value of having attribution, funnels, and automation in one place, the consolidation happens naturally. Data Pipelines keeps your existing tools fed.
"Workflows seems basic compared to HubSpot/Braze."	It is newer. The trade-off is that PostHog Workflows is triggered by real product behavior data, not just email opens and form fills. If you need complex email nurture sequences, keep your email tool and use PostHog for behavior-driven automation. They complement each other via Data Pipelines.
"Our growth team wants to experiment but engineering is too busy to implement flags."	That's actually a common starting point. The first experiment is the hardest because engineering needs to set up the Feature Flag SDK. But once the SDK is in place, subsequent experiments are much faster. Most teams find that after the first 2 to 3 experiments, the loop is smooth. And engineering now has flag infrastructure they can use for their own releases too.

Cross-sell pathways from this use case

If Using...	They Might Need...	Why	Conversation Starter
Web Analytics + Marketing Analytics	Product Analytics (funnels, retention)	They can see traffic and channels but need to connect it to actual user behavior and conversion	"You know which channels bring traffic — but do you know which channels bring users who *retain*?"
Product Analytics (funnels)	Experiments + Feature Flags	They've identified drop-off points and want to test fixes	"You've found the drop-off. Want to test whether a new flow actually improves conversion?"
Product Analytics + Experiments	Workflows + Product Tours	They know what works from experiments and want to operationalize it	"You proved the new onboarding works in an experiment. Now let's roll it out as a Product Tour for everyone."

If Using...	They Might Need...	Why	Conversation Starter
Experiments + Feature Flags (growth-driven)	Release Engineering (for the eng team)	Engineering is already implementing flags for experiments — they can use those same flags for progressive rollouts	"Your engineering team is already using feature flags for growth experiments. Have they considered using the same infrastructure for all their releases?"
Web Analytics + Product Analytics	Data Pipelines	They're analyzing conversion but not feeding it back to ad platforms or CRM	"You're measuring real conversions — are you sending those back to Meta and Google so their algorithms can optimize?"
Funnels + Workflows	Revenue Analytics	They're driving and automating conversion but need to measure the revenue impact	"You've automated re-engagement. Now let's see which cohorts and channels drive the most LTV."
Any Growth & Marketing products	Session Replay	They see a funnel drop-off but don't know *why*	"Your checkout funnel drops 40% at step 3. Want to watch what users are actually doing at that step?"
Growth & Marketing stack established	Product Intelligence (for the product team)	Marketing/growth is in PostHog — the product team should be too	"Your growth team already uses PostHog for funnels and experiments. Has the product team seen what they can do with cohorts and retention analysis?"

Internal resources

- **Web Analytics docs:** Getting started · Dashboard
- **Marketing Analytics docs:** Marketing Analytics
- **Product Analytics docs:** Funnels · Retention · Lifecycle · Cohorts
- **Workflows docs:** Getting started · Email drip campaigns · Configure channels
- **Product Tours docs:** Getting started · Creating tours
- **Data Pipelines docs:** CDP overview · Realtime destinations · Batch exports · HubSpot destination
- **Revenue Analytics docs:** Getting started · Dashboard (MRR/ARR)
- **Surveys docs:** Creating surveys
- **Experiments docs:** Experiments · Exposures
- **Feature Flags docs:** Getting started
- **PostHog AI docs:** Enable PostHog AI · Example prompts
- **UTM tracking:** UTM segmentation
- **Tutorial:** How to track performance marketing

- **Competitive battlecard:** *To be added: GA4 / Segment / CDP competitive positioning*
- **Product team:** *To be added: Slack channels for Web Analytics, Marketing Analytics, Workflows, Product Tours, Pipelines, Revenue Analytics teams*

Appendix: Company archetype considerations

Archetype + Stage	Framing	Key Products	Buyer
AI Native — Early	"You need to get users to your AI product, get them activated, and understand what channels work, all without hiring a data team." Speed matters. Experiments are high-value early.	Web Analytics, Product Analytics (funnels), Experiments, Feature Flags, PostHog AI	Founder, first growth hire, GTM engineer
AI Native — Scaled	"You're scaling acquisition and need to optimize spend, automate onboarding, and connect marketing data to product engagement."	Web Analytics, Marketing Analytics, Product Analytics, Experiments, Feature Flags, Pipelines, Workflows, Revenue Analytics	Head of Growth, Growth Engineering Lead
Cloud Native — Early	"You're investing in growth for the first time and want to build it right. One tool for attribution, funnels, experiments, and engagement."	Web Analytics, Product Analytics, Experiments, Feature Flags, Surveys	Founder, first PM, growth engineer
Cloud Native — Scaled	"Your marketing stack is fragmented and expensive. Consolidate attribution, conversion analytics, engagement automation, and experimentation into one platform." Experiments + Feature Flags are the multithreading lever.	Web Analytics, Marketing Analytics, Product Analytics, Experiments, Feature Flags, Pipelines, Workflows, Product Tours, Revenue Analytics	VP Growth, Head of Growth, CRO, Marketing Ops
Cloud Native — Enterprise	"Multiple teams, multiple products, multiple markets, and none of them agree on the numbers. PostHog gives you a single source of truth for acquisition, conversion, and revenue across all properties."	Full stack. Pipelines and Revenue Analytics are especially important.	VP Marketing, CMO, Head of Growth, Marketing Ops

Observability

contents/handbook/growth/use-case-selling/observability.md

What is the job to be done?

"Help me know when things break, understand why, and fix them fast."

- Catch exceptions and regressions before users report them
- See the user's actual experience when an error occurred, not just a stack trace
- Understand the *business impact* of incidents (which users were affected, what revenue was at risk), not just the technical impact
- Centralize log collection and search alongside error data
- Triage incidents faster with natural language queries

This is where our roadmap is heading and where significant market opportunity exists. The long-term vision is a full observability stack that competes with Datadog and Sentry on their home turf, but with the massive advantage that our observability data is connected to product analytics data. No other vendor can tell you "this API endpoint is slow AND here's the business impact in terms of user drop-off and revenue loss."

Separating this from Release Engineering is important because the buyer is often different (SRE/platform team vs. product engineering), the competitive landscape is different (Datadog/Sentry vs. LaunchDarkly), and the expansion path is different.

What PostHog products are relevant?

- **Error Tracking (core)** — Track exceptions, get alerts, resolve issues. Automatic grouping of similar errors, stack traces, affected user counts. The starting point for most Observability adoption.
- **Session Replay** — User impact of errors, visual reproduction. When an error fires, click through to the user's session and see exactly what happened. This is the killer differentiation vs. Sentry: you don't just see the stack trace, you see the user's actual experience.
- **Product Analytics** — Error correlation with user behavior and business impact. Answer "how many users hit this error?" and "did this error cause drop-off in our conversion funnel?" Connect technical incidents to business outcomes.
- **PostHog AI** — Natural language incident triage. "What errors spiked in the last hour and which users were affected?" without writing a query. Faster mean time to understanding during incidents. (Example prompts)
- **Logging** *beta* — Centralized log collection and search. Logs are table stakes for any observability stack. Having logs alongside errors, replays, and analytics means the full debugging context lives in one place.

- *Roadmap: APM, API tracing* — Not shipped yet. When these arrive, the observability story becomes complete: errors + logs + traces + replay + analytics in one platform.

Adoption path and expansion path

Entry point

Usually **Error Tracking**. Team wants to catch exceptions and regressions. Common entry scenarios:

1. **Sentry replacement:** They're paying for Sentry and want to consolidate into PostHog (which they're already using for analytics or flags). Error Tracking is the direct replacement.
2. **First observability tool:** Early-stage company that hasn't invested in error tracking yet. PostHog's free tier (100K exceptions/month) lets them start without a new vendor relationship.
3. **Session Replay → Error Tracking:** They're already using Session Replay for debugging and discover that errors surfaced in replays could be tracked systematically with Error Tracking.

Primary expansion path

Error Tracking → + Session Replay (error context) → + Logging → + Product Analytics (impact analysis)

The logic of each step:

- Error Tracking → Session Replay: They can see the error and the stack trace. But they can't see what the user was doing when it happened. Session Replay lets them click from an error event directly to the user's session and watch the full context. This is the single most differentiated feature in our Observability story.
- Session Replay → Logging: They're seeing errors and user sessions. They need the backend context: what was the server doing when this error fired? Centralized logs complete the debugging picture.
- Logging → Product Analytics: They're debugging individual errors. Now they want to understand the aggregate impact: how many users are hitting this error? Is it correlated with a specific funnel step? Did this bug cause a revenue drop? Product Analytics connects technical incidents to business outcomes.

Future expansion (roadmap dependent)

As APM and tracing ship, the path extends: Logging → APM → Tracing, completing the full observability stack. Position this honestly: name the vision, be transparent about what's available today vs. what's coming.

Business impact of solving the problem

Observability data connected to product analytics is a moat. Every other observability tool (Datadog, Sentry, New Relic) can tell you "this endpoint threw an error." Only PostHog can tell you "this error affected 500 users, 30 of whom were in the middle of checkout, resulting in an estimated $15k in lost revenue this week." That's a fundamentally different conversation with engineering leadership.

Session Replay as error context is a killer feature. Sentry shows you a stack trace. PostHog shows you the user's actual experience. For frontend and full-stack debugging, this is dramatically faster for reproduction and resolution.

Consolidation play for accounts already using PostHog. If they're already on PostHog for analytics or flags, adding Error Tracking and Logging means one fewer vendor (Sentry, Datadog) to manage. The consolidation saves money and reduces context-switching.

This use case has the highest growth ceiling. The observability market is enormous (Datadog alone is $25B+). Our story gets stronger with every product we ship in this space.

Personas to target

Persona	Role Examples	What They Care About	How They Evaluate
SRE / Platform Engineer	SRE, Platform Eng, Infrastructure Eng	Reliability, alerting, mean time to resolution, not getting paged at 3am	"Will this catch issues before users report them? How fast can I triage?"
Backend Engineer	Backend Eng, API Engineer, Server-side Eng	Stack traces, log correlation, reproducing bugs efficiently	"Can I see what happened on the server when this error fired?"
Product Engineer	Full-stack Eng, Frontend Eng	User-facing bugs, reproduction, understanding the user impact of errors	"Can I see the user's session when this error happened?"
Engineering Manager	EM, VP Eng, Director of Eng	Team velocity, incident metrics (MTTR, error rates), cost of observability tooling	"How does this reduce our incident response time? What does it cost vs. Sentry/Datadog?"
Founder (early stage)	CTO, first engineer	Catching bugs before users complain, not paying Datadog prices	"Does this work out of the box and is it affordable?"

Signals in Vitally & PostHog

Vitally indicators this use case is relevant

Signal	Where to Find It	What It Means
Error Tracking is active but low product count	Product spend breakdown	They've started with errors. Full Observability expansion path available.
Customer mentions Sentry or Datadog in notes	Vitally notes / conversations	Competitive displacement opportunity. Consolidation pitch.
High Session Replay usage with error-related viewing patterns	Product usage data	They're using replay for debugging already. Error Tracking formalizes this.
Engineering-heavy user base, no PM users	User list in Vitally	Engineering-first account. Observability and Release Engineering are the primary use cases.

PostHog usage signals

Signal	How to Check	What It Means
Error tracking exceptions growing week over week	Product usage metrics	They're instrumenting more of their stack. Good adoption signal.
Session Replay filtered by error events	Replay usage patterns	They're connecting replay to error debugging. The integration is clicking.
High error volume but no alerting configured	Error tracking settings	They're collecting errors but not acting on them. Help them set up alerts.
Product Analytics queries referencing error events	Saved insights	They're starting to connect errors to business impact. Encourage this.

Command of the Message

Discovery questions

- When something breaks in production, how long does it take your team to find out? From users? From monitoring?
- How do you currently track and prioritize errors? Do you have a tool for this, or is it ad hoc?
- When you see an error, how do you reproduce it? How long does reproduction typically take?
- Can you tell which users were affected by a specific error? Do you know the business impact?
- What does your current observability stack look like? (Sentry? Datadog? New Relic? How many tools?)
- How much are you spending on observability tooling today?

- When an incident happens, how many tools does your team switch between to understand what happened?
- Do you have centralized logging? Where do your logs live today?

Negative consequences (of not solving this)

- Errors are discovered through user complaints, not proactive monitoring
- Stack traces exist but reproduction is guesswork because there's no user session context
- Engineering can quantify "how many errors" but not "how many users affected" or "what revenue was lost"
- Multiple observability tools (Sentry for errors, Datadog for APM, separate logging) with no connection between them
- Incident triage is slow because context is spread across 3+ tools
- Observability costs are high and growing (Datadog pricing, Sentry pricing) without clear ROI

Desired state

- Errors are caught proactively with alerts before users report them
- Every error links to the user's actual session, so reproduction takes seconds, not hours
- Error impact is measured in business terms: affected users, affected revenue, affected funnels
- Errors, logs, and user sessions live in one platform alongside product analytics
- Incident triage is faster because all context is in one place

Positive outcomes

- Reduced MTTR (mean time to resolution) from instant session replay access on errors
- Fewer user-reported bugs (proactive error detection + alerting)
- Better incident prioritization based on business impact, not just error count
- Observability cost reduction through consolidation (replace Sentry + separate logging)
- Engineering leadership gets business-impact reporting on reliability, not just technical metrics

Success metrics

Customer-facing:

- Mean time to resolution for user-facing bugs decreases
- Percentage of errors caught proactively (via alerting) vs. user-reported increases
- Error rate trends downward as bugs are prioritized by impact and fixed systematically

TAM-facing:

- Error Tracking exception volume grows (instrumenting more of their stack)
- Session Replay usage increases with error-filtered viewing (integration is working)
- Logging adoption starts (filling out the observability stack)
- Product Analytics queries reference error data (connecting to business impact)

Competitive positioning

Our positioning

- **Errors + replay in one platform.** See the stack trace and the user's actual session. No other error tracking tool offers this depth of user context. Sentry shows you the error. PostHog shows you the experience.
- **Business impact, not just error count.** Connect errors to user behavior and revenue with Product Analytics. "This error caused 200 users to abandon checkout" is a different conversation than "this error fired 500 times."
- **Consolidation for existing PostHog users.** If they're already using PostHog for analytics or flags, adding Error Tracking means one fewer vendor. Same data platform, one less tool to manage.
- **Logging completes the picture.** Errors, user sessions, and backend logs in one place. No switching between Sentry, Papertrail, and Amplitude to understand an incident.

Competitor quick reference

Competitor	What They Do	Our Advantage	Their Advantage
Sentry	Error tracking, performance monitoring, session replay	Deeper product analytics integration; business impact context; flag/experiment connection; better pricing	More mature error tracking features; broader language support; larger install base; dedicated performance monitoring
Datadog	Full observability: APM, logs, metrics, errors	Product analytics integration; session replay depth; much cheaper	Complete observability stack (APM, traces, metrics); enterprise-grade; massive ecosystem
New Relic	Full observability: APM, logs, errors, distributed tracing	Product analytics integration; session replay; simpler pricing	Complete observability stack; mature enterprise features

Honest assessment: Our Observability story is credible but incomplete. Error Tracking + Session Replay + Logging is a meaningful starting point, and the connection to product analytics is genuinely differentiated. But we don't have APM or tracing yet. We can't position PostHog as a full Datadog replacement today. The honest pitch is: "For error tracking, we're better than Sentry because of the user context. For full observability, we're building toward it, and in the meantime, the product analytics connection gives you something no other observability tool offers." Be transparent about what's available today vs. what's on the roadmap.

Pain points & known limitations

Pain Point	Impact	Workaround / Solution
No APM or tracing yet	Can't replace Datadog for teams that need full backend observability	Be honest about the roadmap. Position PostHog as complementary for now: errors + replay + analytics in PostHog, APM in their existing tool. The consolidation play gets stronger as we ship more.
Logging is beta	Teams expecting production-grade centralized logging may find gaps	Set expectations on maturity. For teams with existing logging (ELK, Papertrail), PostHog logging complements rather than replaces initially.
Error Tracking language/framework support may lag Sentry	Sentry supports a very wide range of languages and frameworks	Check Error Tracking docs for current support. For unsupported frameworks, generic exception capture via the API may work.
No built-in on-call/incident management	Teams wanting PagerDuty-style incident workflows won't find it here	PostHog alerts can trigger webhooks to PagerDuty, Slack, etc. Error Tracking is about detection and context, not incident management workflows.

Getting a customer started

What does an evaluation look like?

- **Scope:** Enable Error Tracking on their primary application. Connect Session Replay to error events. Set up alerts for critical error spikes.
- **Timeline:** 1 to 3 days to start capturing errors. 1 week to have meaningful error data and session replay context.
- **Success criteria:** Can you see errors grouped by type with affected user counts? Can you click from an error to the user's session replay? Can you get alerted when a new error type spikes?
- **PostHog investment:** Error Tracking free tier covers 100K exceptions/month. Session Replay free tier covers 5K recordings.
- **Key requirement:** They need to integrate the PostHog SDK or connect their existing error capture to PostHog's Error Tracking. If they're already using PostHog's SDK, Error Tracking may just need to be enabled.

Onboarding checklist

- [ ] Enable Error Tracking in the PostHog SDK configuration
- [ ] Verify errors are being captured and grouped correctly
- [ ] Connect Session Replay to error events (verify you can click from error → replay)
- [ ] Set up alerts for critical error types or spike detection

- [ ] Build an "Error Health" dashboard: error trends, top errors by affected users, error rate by release
- [ ] Review the top 5 errors with the team, using session replay context to prioritize fixes
- [ ] If applicable, enable Logging (beta) for backend log context
- [ ] Create a Product Analytics query that correlates errors with funnel drop-off or business metrics

Cross-sell pathways from this use case

If Using...	They Might Need...	Why	Conversation Starter
Error Tracking only	Session Replay	They see stack traces but can't reproduce the user experience	"You can see the error. Want to see exactly what the user was doing when it happened?"
Error Tracking + Session Replay	Logging	They have frontend error context but need backend logs	"You can see the user's session. But what was happening on the server at the same time?"
Error Tracking + analytics correlation	Product Intelligence (for the product team)	They're connecting errors to user impact. The product team would benefit from the same analytics.	"You're measuring error impact on users. Has your product team seen what they can do with funnels and retention in the same platform?"
Error Tracking (engineering in PostHog)	Release Engineering (same engineering team)	Engineering is in PostHog for errors. Feature flags for safe releases is a natural add.	"You're tracking errors after releases. What if you could gate features behind flags and roll back without a deploy?"
Error Tracking for AI features	AI/LLM Observability	Traditional error tracking misses AI quality regressions	"You're catching exceptions, but are you catching when your model starts giving worse answers? That's a different kind of 'error.'"

Internal resources

- **Error Tracking docs:** Error Tracking
- **Session Replay docs:** Session Replay
- **Product Analytics docs:** Product Analytics
- **PostHog AI docs:** Enable PostHog AI · Example prompts
- **Competitive battlecard:** *To be added: Sentry / Datadog competitive positioning*

Appendix: Company archetype considerations

Archetype + Stage	Framing	Key Products	Buyer
AI Native — Early	"You're shipping fast and breaking things. PostHog catches errors and shows you the user's experience when they hit a bug. No Sentry bill required." Error Tracking + Session Replay is the sweet spot.	Error Tracking, Session Replay	CTO, founding engineer
AI Native — Scaled	"Your AI features have failure modes that traditional error tracking misses: hallucinations, slow responses, quality regressions. PostHog catches the technical errors AND lets you evaluate output quality." Bridge to AI/LLM Observability.	Error Tracking, Session Replay, Logging, AI Evals	VP Eng, Platform Lead, SRE
Cloud Native — Early	"Stop finding bugs from user complaints. Error Tracking catches exceptions automatically, and Session Replay lets you see exactly what happened. 100K exceptions/month free."	Error Tracking, Session Replay	CTO, founding engineer
Cloud Native — Scaled	"Your team is juggling Sentry, Papertrail, and Datadog. PostHog consolidates error tracking, logging, and user context into the platform you already use for analytics." Consolidation pitch.	Error Tracking, Session Replay, Logging, Product Analytics	VP Eng, SRE Lead, Platform team
Cloud Native — Enterprise	"Multiple teams, multiple services, and incident context spread across 5 tools. PostHog gives you errors + logs + user sessions + business impact in one platform. No more switching between Sentry, Datadog, and Amplitude during an incident."	Full Observability stack + Enterprise package	VP Eng, Director of SRE, Platform leadership

Product Intelligence

contents/handbook/growth/use-case-selling/product-intelligence.md

What is the job to be done?

"Help me understand what users do, why they do it, and what to build next."

- Understand how users navigate your product and where they get stuck
- Identify which features drive retention and which are ignored
- Get qualitative context for quantitative patterns (the "why" behind the numbers)
- Validate product hypotheses with experiments before committing engineering resources
- Collect direct user feedback at key moments in the product experience
- Track business outcomes (revenue, expansion) tied to product usage
- Act on insights by guiding users through onboarding and feature adoption directly inside the product

This is our bread and butter. Most accounts start here. The risk is they *stay* here as a single product analytics customer and never expand. The opportunity is that Product Intelligence naturally creates demand for the other use cases once teams start acting on what they learn.

What PostHog products are relevant?

- **Product Analytics (core)** — Funnels, retention, cohorts, lifecycle analysis, trends, user paths. The quantitative foundation for understanding what users do.
- **Session Replay** — The qualitative layer. Watch what users actually do when the numbers say they're dropping off. Bridges the gap between "40% drop at step 3" and "oh, the button doesn't render on mobile Safari."
- **Surveys** — Direct feedback loop at key moments. NPS after onboarding, CSAT after support, "why did you cancel?" on churn. Ties qualitative signal to quantitative behavior data.
- **Revenue Analytics** — Business outcome tracking. Connect product usage to MRR, expansion revenue, LTV, and churn. Lets PMs prove that the feature they shipped actually moved the revenue needle. (Dashboard)
- **Experiments** — Validate hypotheses with statistical rigor before committing to a full build. A/B test changes against real metrics, not gut feel. Requires Feature Flags for implementation.
- **Workflows** — Onboarding sequences, activation nudges, lifecycle engagement. The action layer: once you've identified a drop-off in analytics, Workflows lets you do something about it automatically. (Email drip campaigns)
- **Product Tours** *alpha* — In-app guided onboarding, feature adoption prompts. The in-product complement to Workflows' out-of-product engagement. Guide users through the right path at exactly the right moment. (Creating tours)

- **AI Evals** — For products with AI features: proactively surface where users are struggling based on AI output quality. This is product intelligence driven by AI observability. A bridge product to the AI/LLM Observability use case.
- **PostHog AI** — Natural language querying and insight discovery. Lowers the bar for non-technical product stakeholders to self-serve. A PM asks "why did retention drop last week?" instead of building a custom query. (Example prompts)

Adoption path and expansion path

Entry point

Usually **Product Analytics**. Customer starts tracking events, builds dashboards, creates their first funnel. Then they hit the ceiling of quantitative data alone: "I can see *that* users drop off, but not *why*."

Primary expansion path

Product Analytics → + Session Replay → + Surveys → + Experiments → + Revenue Analytics → + Workflows / Product Tours

The logic of each step:

- Product Analytics → Session Replay: They know *what* is happening (40% drop at step 3). They need to see *why*. Session Replay gives them the qualitative context that numbers can't.
- Session Replay → Surveys: They're watching replays and forming hypotheses about why users struggle. Surveys let them ask users directly at the moment of friction, then tie responses back to behavior data.
- Surveys → Experiments: They've identified the problem through analytics, replay, and feedback. Now they want to test a fix. Experiments require Feature Flags, which gets engineering involved (multithreading moment).
- Experiments → Revenue Analytics: They've validated changes with experiments and want to prove business impact. Revenue Analytics connects product usage to MRR, expansion, and churn.
- Revenue Analytics → Workflows / Product Tours: They've identified what drives value. Now they want to guide users toward those high-value behaviors automatically, through in-app tours and behavior-triggered engagement sequences.

Alternate expansion paths

B2B accounts with Group Analytics: B2B SaaS companies almost always need company-level analytics alongside user-level. If they're B2B and not using Group Analytics, that's a significant upsell opportunity. Group Analytics lets them answer "which *companies* are most engaged" not just "which users."

Starting from Session Replay: Some accounts come in through Session Replay first (debugging, QA, customer support use cases). They realize they need Product Analytics to quantify what they're seeing qualitatively. The expansion path reverses: Replay → Analytics → Surveys → Experiments.

Product teams that ship AI features: If the product has AI components, AI Evals can proactively surface where users are struggling based on output quality. This bridges Product Intelligence into AI/LLM Observability.

Business impact of solving the problem

This is the use case with the largest existing install base. Most PostHog accounts start with Product Analytics. The expansion opportunity isn't convincing them to adopt PostHog. It's convincing them to go beyond a single product and use the full Product Intelligence stack.

The Workflows and Product Tours close-the-loop story is powerful. You identify a drop-off point (analytics), you understand why users leave (session replay, surveys), and now you can actually fix it by guiding users through the right path (product tours) or re-engaging them when they disengage (workflows). That's a complete insight-to-action cycle that no competitor offers in one platform.

Product Intelligence creates demand for other use cases. Once the product team is deep in PostHog, they pull in the growth team (Growth & Marketing use case) for acquisition and activation. Once they're running experiments, engineering gets involved in rollouts (Release Engineering). This is the gateway use case.

Personas to target

Persona	Role Examples	What They Care About	How They Evaluate
Product Manager	PM, Senior PM, Head of Product	Feature adoption, retention, user journeys, proving impact to leadership	"Can I see which features drive retention and prove ROI to my VP?"
Product Engineer	Full-stack eng on a product team	Fast instrumentation, reliable data, not maintaining a data pipeline	"How fast can I instrument this and how reliable is the data?"
UX Researcher	UX Researcher, Design Lead	User behavior patterns, qualitative + quantitative, session-level detail	"Can I watch real user sessions filtered by the cohort I'm studying?"
Designer	Product Designer, UX Designer	How users interact with new designs, A/B testing UI changes	"Can I see the before/after impact of my design changes?"
Founder (early stage)	Founder, CTO at seed/Series A	All of the above. Finding product-market fit. Speed.	"Does this help me figure out what to build next?"

Signals in Vitally & PostHog

Vitally indicators this use case is relevant

Signal	Where to Find It	What It Means
Product Analytics is the only paid product	Product spend breakdown	Classic single-product account. Full expansion path available.
High insight/dashboard creation per active user	Engagement metrics	Product team is actively using PostHog for analysis. They're ready for deeper tools.
Session Replay is free-tier only or not used	Product usage data	They're doing quantitative analysis without qualitative context. Session Replay is the obvious next step.
B2B company without Group Analytics	Company type + product spend	Major upsell opportunity. B2B companies need company-level analytics.
Multiple PM or design roles in the user list	User list in Vitally	Product team is in PostHog, not just engineering. This use case is live.

PostHog usage signals

Signal	How to Check	What It Means
Funnels and retention insights being created regularly	Saved insights	Product team is actively measuring conversion and retention. Ripe for Experiments.
Session Replay enabled but low viewing rate	Replay settings vs. replay views	They've turned it on but aren't using it. Needs onboarding or a nudge to connect it to their analytics workflow.
No experiments running despite active analytics	Experiments list	They're identifying problems but not testing solutions. Experiments is the next conversation.
Dashboards shared across multiple users	Dashboard sharing settings	They're collaborating on insights. Good health signal and potential for team expansion.
High event volume, low survey usage	Product usage metrics	They have the traffic to run surveys but haven't started. Low-hanging cross-sell.

Command of the Message

Discovery questions

- How does your product team decide what to build next? What data informs that decision?
- When you see a drop-off in a funnel, how do you figure out *why* users are leaving?
- How do you measure whether a new feature is successful after launch?
- Do you collect direct user feedback inside the product? How is that connected to your analytics?
- When you have a hypothesis about user behavior, how do you validate it? Do you run experiments?
- How do you prove to leadership that a product investment drove business results (revenue, retention)?
- How many tools does your product team use to understand users? (Analytics, replay, surveys, experiments — how many vendors?)
- Can your PMs answer their own questions, or do they depend on data/engineering for every query?

Negative consequences (of not solving this)

- Product decisions are based on gut feel or incomplete data because the team can't connect behavior to outcomes
- PMs can see *that* users drop off but not *why*, leading to guesswork about what to fix
- Experiments are rare or nonexistent because the tooling is disconnected from analytics, so every test requires a separate setup
- User feedback (surveys) lives in a separate tool, disconnected from behavior data, so you can't answer "what happened right before this user gave us a 3/10?"
- Product team can't prove business impact, making it hard to justify investment or prioritize
- Insights are identified but never acted on because there's no automation layer to guide users or re-engage them

Desired state

- One platform for the full cycle: measure behavior → watch sessions → collect feedback → test changes → measure revenue impact → act on insights
- PMs can self-serve answers without waiting for a data team
- Every product change is measured against real retention and revenue metrics
- Onboarding and feature adoption are guided automatically based on user behavior
- Product team and engineering share the same platform, reducing tool fragmentation

Positive outcomes

- Faster product decisions: cycle time from "we see a problem" to "we've validated a fix" drops significantly
- Higher retention from catching and addressing drop-off points systematically
- Better resource allocation: experiments prove what works before engineering commits to a full build
- Product team can demonstrate revenue impact to leadership, strengthening their influence

- Tool consolidation: replace separate analytics + replay + survey + experimentation vendors with one platform

Success metrics

Customer-facing:

- Feature adoption rates improve for targeted features
- Retention curves flatten or improve for key cohorts
- Experiment velocity increases (more hypotheses tested per quarter)
- Time from insight to action decreases

TAM-facing:

- Customer expands from Product Analytics-only to multi-product (Session Replay, Surveys, Experiments)
- Multiple product team members (PMs, designers) are active, not just engineers
- Experiment usage grows (indicates the product team is using PostHog for decisions, not just reporting)
- Workflow or Product Tour usage starts (they're closing the insight-to-action loop)

Competitive positioning

Our positioning

- **Quantitative + qualitative in one platform.** Product Analytics and Session Replay together. No switching between Amplitude and Hotjar. Filter replays by funnel drop-off, cohort, or event.
- **Insight to action, not just insight.** Most analytics tools stop at the dashboard. PostHog lets you act on what you find with Workflows, Product Tours, and Experiments.
- **Experiments built into the analytics workflow.** See a drop-off in a funnel, right-click to create an experiment, measure the result in the same tool. No separate experimentation platform.
- **PostHog AI makes analytics accessible.** PMs who aren't comfortable with SQL can ask questions in plain English.

Competitor quick reference

Competitor	What They Do	Our Advantage	Their Advantage
Amplitude	Product analytics, cohorts, experiments	Broader platform (replay, flags, surveys, workflows); better pricing; open source	More mature ML features (predictions, audiences); larger enterprise install base
Mixpanel	Product analytics, funnels, retention	Broader platform; no sampling; replay + surveys + flags included	Some teams prefer the UX; strong mobile analytics

Competitor	What They Do	Our Advantage	Their Advantage
Hotjar	Session replay + basic surveys	Engineering-grade analytics alongside replay; experiments; flags	Simpler UX for non-technical users; purpose-built for UX research
Heap	Auto-capture product analytics, session replay	Also auto-capture, plus flags, experiments, surveys, workflows	Retroactive analytics (virtual events) is a strong pitch
Pendo	Product analytics + in-app guides	Deeper analytics; experiments; open source; better pricing	More mature in-app guides; stronger enterprise PM workflow features

Honest assessment: Our strongest position is the breadth of the platform. No competitor offers analytics + replay + surveys + experiments + workflows + product tours in one tool. We're weaker against Amplitude in very large enterprises where their ML features and enterprise sales motion are more mature. We're weaker against Hotjar/Pendo for non-technical product teams who want a simpler, more opinionated UX. Our sweet spot is technical product teams at companies with engineers who value depth, flexibility, and not paying for 5 separate tools.

Pain points & known limitations

Pain Point	Impact	Workaround / Solution
Product Tours is alpha, limited customization	Teams with complex in-app onboarding needs may hit walls	Position as the integrated option. For advanced tooltip/modal UX, keep a dedicated tool (Appcues, Pendo) and use PostHog for analytics + experimentation.
Workflows is new, less mature than dedicated engagement tools	Teams expecting Braze-level email sequencing will find gaps	Position as behavior-driven automation, not a full lifecycle marketing replacement. Complement with existing tools via Data Pipelines.
No built-in heatmaps	Some UX teams expect heatmaps as part of the qualitative toolkit	Session Replay provides more context than heatmaps (full session vs. aggregated click positions). Toolbar provides some click-map functionality.
Learning curve for non-technical PMs	PMs used to Amplitude's guided UX may find PostHog's flexibility overwhelming initially	Lead with PostHog AI for querying. Build pre-configured dashboards during onboarding. Start with simple funnels and retention, not HogQL.

Getting a customer started

What does an evaluation look like?

- **Scope:** Instrument their core product flow: signup → key activation event → retention-defining action → conversion/upgrade. Build the primary funnel and retention analysis. Enable Session Replay on key flows.
- **Timeline:** 1 to 2 weeks to see value from analytics and replay. Experiments need enough traffic for statistical significance, so timeline varies.
- **Success criteria:** Can you answer: "Where do users drop off in our core flow, and why?" Can you see the full retention curve by cohort? Can you watch a replay of a user who dropped off?
- **PostHog investment:** Product Analytics free tier covers 1M events. Session Replay free tier covers 5K recordings. Surveys, Experiments have generous free tiers.

Onboarding checklist

- [ ] Install PostHog SDK with autocapture enabled (getting started)
- [ ] Define and instrument core conversion events (signup, activation, key feature usage, upgrade)
- [ ] Build the primary conversion funnel
- [ ] Set up retention analysis for key activation events
- [ ] Enable Session Replay on production (configure sampling if needed)
- [ ] Watch 5+ replays filtered by funnel drop-off to get qualitative context
- [ ] Build a "Product Health" dashboard for the product team (funnels, retention, trends)
- [ ] Introduce PostHog AI to PMs for self-serve querying
- [ ] Set up one Survey at a key friction point (post-signup NPS, feature feedback)
- [ ] Plan first Experiment targeting a known drop-off point

Cross-sell pathways from this use case

If Using...	They Might Need...	Why	Conversation Starter
Product Analytics only	Session Replay	They see the numbers but not the *why*	"You can see 40% drop off at step 3. Want to watch what's actually happening?"
Product Analytics + Session Replay	Surveys	They're forming hypotheses from replays and want direct user input	"You're watching sessions and seeing confusion. Want to ask users directly what's tripping them up?"
Product Analytics + Surveys	Experiments	They've identified problems and want to validate fixes	"You know the problem. Let's test whether your proposed fix actually works before building it fully."

If Using...	They Might Need...	Why	Conversation Starter
Experiments running	Revenue Analytics	They're testing changes but measuring proxy metrics, not revenue	"Your experiment improved conversion by 15%. But did it actually increase MRR?"
Analytics + Experiments mature	Workflows + Product Tours	They know what works and want to operationalize it	"You proved the new onboarding flow works. Now let's guide every new user through it automatically."
Product team in PostHog	Growth & Marketing (for the growth team)	Product team is in PostHog. Growth team should be too.	"Your PMs are using PostHog for product decisions. Has the growth team seen what they can do with funnels and experiments for conversion optimization?"
B2B account, no Group Analytics	Group Analytics add-on	B2B companies need company-level analytics	"You're tracking individual users. But do you know which *companies* are most engaged and which are at risk?"
Product team using flags for experiments	Release Engineering (for the eng team)	Engineering is implementing flags for experiments. They can use them for releases too.	"Your engineers are already deploying feature flags for experiments. Have they considered using the same infrastructure for all their releases?"

Internal resources

- **Product Analytics docs:** Funnels · Retention · Lifecycle · Cohorts · User paths · SQL
- **Session Replay docs:** Session Replay
- **Surveys docs:** Creating surveys
- **Experiments docs:** Experiments · Exposures
- **Revenue Analytics docs:** Getting started · Dashboard
- **Workflows docs:** Getting started · Email drip campaigns
- **Product Tours docs:** Getting started · Creating tours
- **Feature Flags docs:** Getting started
- **PostHog AI docs:** Enable PostHog AI · Example prompts
- **Group Analytics docs:** Group Analytics

Appendix: Company archetype considerations

Archetype + Stage	Framing	Key Products	Buyer
AI Native — Early	Product Intelligence looks different here. There's no UX researcher. A GTM engineer or founding PM is looking at funnels, activation rates, and conversion. Frame it as "understand what makes users stick" not "deep behavioral research."	Product Analytics (funnels, retention), Session Replay, Experiments, PostHog AI	Founder, founding PM, GTM engineer
AI Native — Scaled	Starting to formalize the product function. May have a dedicated PM. AI Evals becomes relevant as a bridge: evaluating AI output quality is product intelligence for AI products.	Product Analytics, Session Replay, Surveys, Experiments, AI Evals, Revenue Analytics	PM, Head of Product, AI Product Lead
Cloud Native — Early	First real analytics investment. They need to find product-market fit. Speed matters. Don't overwhelm with features. Start with funnels and retention, add replay and surveys as they mature.	Product Analytics, Session Replay, PostHog AI	Founder, first PM, product engineer
Cloud Native — Scaled	Dedicated product team with PMs, designers, maybe UX researchers. They want depth: cohort analysis, retention by feature, experiment velocity. Workflows and Product Tours become relevant for operationalizing insights.	Full Product Intelligence stack. Group Analytics if B2B.	Head of Product, VP Product, UX Research Lead
Cloud Native — Enterprise	Multiple product teams, multiple workloads. The play is expanding PostHog from one team to many. Standardization and governance matter. RBAC (Enterprise package) becomes relevant.	Full stack + Group Analytics + Enterprise package	VP Product, CPO, product ops

Release Engineering

contents/handbook/growth/use-case-selling/release-engineering.md

What is the job to be done?

"Help me ship faster without breaking things, control who sees what, and validate that changes actually work."

- Safely roll out features to specific users or groups before a full release
- Instantly kill a bad deploy without a rollback or hotfix
- Measure the actual impact of a release on key metrics, not just "it didn't crash"
- Reproduce user-reported bugs from the user's actual perspective during a rollout
- Run A/B tests tied to releases so every ship is a learning opportunity
- Detect quality regressions in AI features after prompt or model changes

What PostHog products are relevant?

- **Feature Flags (core)** — Controlled rollouts, percentage-based releases, targeted delivery to specific users/groups, kill switches. The foundation of safe shipping. Engineering teams use flags to decouple deployment from release: code ships to production but features are gated behind flags. (Getting started · Multivariate flags)
- **Experiments** — A/B testing tied directly to releases. "We shipped a new checkout flow behind a flag. Did it actually improve conversion, or just look better in the demo?" Experiments are billed with Feature Flags, so customers with flags already have access. (Creating experiments)
- **Session Replay** — Reproduce bugs from the user's actual perspective during rollout. When a user reports "the new feature is broken," you don't need to guess. Filter replays by feature flag variant and watch exactly what happened. Also useful for rollout validation: watch how real users interact with the new feature before expanding the rollout.
- **AI Evals** — For products with AI features: detect quality regressions after prompt or model changes. Traditional error tracking won't catch a model that starts producing lower-quality output. Evals compare output quality before and after a change, catching regressions that look "fine" from an error rate perspective but degrade user experience.

Adoption path and expansion path

Entry point

Usually **Feature Flags**. Engineering team wants controlled rollouts. Common entry scenarios:

1. **Progressive rollout:** Team wants to ship a risky change to 5% of users, monitor, then expand gradually. Feature flags give them the gate; they quickly want metrics to know when it's safe to expand (Experiments).

2. **Kill switch:** After a bad deploy that took hours to roll back, engineering wants instant off-switches for new features. Feature flags are the answer.
3. **Growth team bridge:** The growth team wants to run an A/B test on the signup flow. Experiments requires Feature Flags, which requires engineering to implement. Engineering gets pulled into PostHog through the growth team's request. (See the Growth & Marketing playbook for this entry path.)

Primary expansion path

Feature Flags → + Experiments → + Session Replay (for debugging and rollout validation)

The logic of each step:

- Feature Flags → Experiments: They're rolling out features behind flags but only monitoring for crashes, not measuring business impact. Experiments lets them answer "did this change actually improve the metric we care about?" Since Experiments is billed with Feature Flags, the barrier is adoption, not cost.
- Experiments → Session Replay: They're measuring impact quantitatively but can't debug issues qualitatively. When an experiment shows the control is outperforming the variant, they need to see *why*. Filter replays by flag variant, watch what's going wrong.

Alternate expansion paths

Starting from Experiments (growth-driven): The growth team wants to A/B test, which requires engineering to implement flags. Engineering discovers they can use the same flag infrastructure for all their releases. This is the reverse entry: growth team is the catalyst, engineering becomes the power user. The growth team stays in Growth & Marketing; engineering lands in Release Engineering.

AI product teams: After a prompt or model change, engineering wants to verify quality hasn't regressed. AI Evals catches regressions that traditional error tracking misses. This bridges into AI/LLM Observability.

Business impact of solving the problem

This is a different buyer than Product Intelligence. Release Engineering targets engineering managers, platform teams, and individual developers. In most organizations, these are separate from the product analytics buyer (PM). Selling to engineering unlocks a parallel revenue stream from the same account. Two budget holders, two champions, much stickier account.

Feature Flags in the codebase are sticky. Once feature flags are integrated into the release workflow and embedded in production code, they're very hard to rip out. This isn't a dashboard someone stops logging into. It's infrastructure that engineering depends on for every deploy. This makes Release Engineering accounts among the most defensible in our book.

The tight integration between flags and experiments is genuinely differentiated. LaunchDarkly has flags but weak experimentation. Standalone experimentation tools (Statsig, Eppo) have experiments but aren't integrated with the broader analytics platform. PostHog connects flags → experiments → product analytics → session replay in one tool.

Experiments + Feature Flags create the multithreading bridge. When growth wants to experiment and engineering implements the flags, both teams are in PostHog. This is one of the best ways to get multithreaded in an account if you aren't already.

Personas to target

Persona	Role Examples	What They Care About	How They Evaluate
Engineering Manager	EM, VP Eng, Director of Eng	Release velocity, incident rate, rollback time, team productivity	"Will this make my team ship faster with fewer incidents?"
Platform Engineer	Platform Eng, DevEx, Infrastructure	Developer experience, flag management at scale, API reliability	"How does this scale to thousands of flags? What's the API latency?"
Individual Developer	Senior Eng, Staff Eng, Product Engineer	Fast to implement, doesn't slow down CI/CD, good SDK quality	"How many lines of code to add a flag? Does the SDK suck?"
Founding Engineer	CTO, first engineers at early-stage startup	Speed, simplicity, not paying for LaunchDarkly's enterprise pricing	"How fast can I set this up and how much does it cost?"

Signals in Vitally & PostHog

Vitally indicators this use case is relevant

Signal	Where to Find It	What It Means
Feature Flags is the primary or only paid product	Product spend breakdown	Engineering-first account. Full Release Engineering expansion path available.
High flag evaluation volume, low experiment count	Product usage data	They're using flags for rollouts but not measuring impact. Experiments is the next conversation.
Customer mentions LaunchDarkly in notes	Vitally notes / conversations	Competitive displacement opportunity. They may be paying LaunchDarkly prices for flags alone.
Engineering-only users (no PMs or marketing)	User list in Vitally	Engineering-first adoption. Release Engineering is the primary use case. Product Intelligence is the cross-sell.

PostHog usage signals

Signal	How to Check	What It Means
Feature flags created frequently but no experiments	Flag list vs. experiments list	They're using flags for rollouts but not measuring impact. Low-hanging Experiments adoption.
Flags with high evaluation volume	Flag usage metrics	Flags are in production, integrated into the codebase. High stickiness.
Session Replay enabled but not filtered by flag variant	Replay usage	They're recording sessions but not connecting them to rollout debugging. Onboarding opportunity.
Multiple flags per user/team	Flag list + creators	Multiple engineers are using flags. Good health signal and potential for team-wide adoption.

Command of the Message

Discovery questions

- How do you currently roll out new features? All at once, or gradually?
- When a deploy goes wrong, how long does it take to roll back? What's that process look like?
- After you ship a feature, how do you know it's working? What metrics do you check?
- Do you run A/B tests on product changes? How is that connected to your release process?
- When a user reports a bug in a new feature, how do you reproduce it?
- How many deploys per day/week does your team ship? What slows that down?
- Are you using a feature flag tool today? What do you like and dislike about it?
- How does your growth team run experiments? Does engineering implement those, or is it separate?

Negative consequences (of not solving this)

- Risky deploys require full rollbacks, costing hours of engineering time and user trust
- No way to gradually roll out to a subset of users, so every release is all-or-nothing
- Features ship without measuring impact, so the team doesn't know if changes actually helped
- Bug reproduction is guesswork because there's no way to see the user's actual experience during a rollout
- Engineering and growth/product teams use separate tools, so experiment results don't connect to release decisions
- High LaunchDarkly costs for feature flagging alone, without experiments or analytics integration

Desired state

- Every feature ships behind a flag with gradual rollout and instant kill switch capability
- Every release is measured against real business metrics, not just error rates

- When a user reports a bug in a new feature, engineers can watch their exact session filtered by flag variant
- Growth team experiments and engineering rollouts use the same infrastructure
- Flag, experiment, and analytics data live in one platform, so the full picture is visible without switching tools

Positive outcomes

- Faster release cycles: engineers ship with confidence because they can roll back instantly
- Fewer incidents: gradual rollouts catch issues at 5% instead of 100%
- Better product decisions: every release is also a measurement opportunity
- Reduced tooling cost: replace LaunchDarkly + separate experimentation tool with one platform
- Multithreaded account: growth and engineering share the same platform for experiments and rollouts

Success metrics

Customer-facing:

- Release velocity increases (more deploys per week)
- Mean time to recovery from bad deploys decreases
- Percentage of releases measured with experiments increases
- Bug reproduction time decreases (engineers can watch filtered replays)

TAM-facing:

- Feature Flag evaluation volume grows (flags are being used more broadly)
- Experiment count increases (moving from "just flags" to "flags + measurement")
- Session Replay adoption grows alongside flag usage (debugging workflow)
- Non-engineering users (growth, PM) start creating experiments (multithreading indicator)

Competitive positioning

Our positioning

- **Flags + experiments + analytics in one platform.** The only tool where you can create a flag, run an experiment, measure the result in Product Analytics, and watch user sessions filtered by variant. No stitching together LaunchDarkly + Statsig + a replay tool.
- **Experiments included with Feature Flags.** Experiments are billed as part of Feature Flags. Customers using flags already have experimentation. The barrier is awareness and adoption, not cost.
- **Session Replay filtered by flag variant.** When an experiment shows the control winning, filter replays by the losing variant and watch what went wrong. No other flag tool offers this.
- **Better pricing than LaunchDarkly.** LaunchDarkly is expensive and charges separately for experimentation. PostHog bundles it and prices on requests, not seats.

Competitor quick reference

Competitor	What They Do	Our Advantage	Their Advantage
LaunchDarkly	Feature flags, targeting, enterprise flag management	Experiments included; analytics integration; session replay; far better pricing	More mature enterprise flag management; larger feature set for complex targeting rules; bigger enterprise install base
Statsig	Feature flags + experimentation + analytics	Broader platform (replay, surveys, workflows); open source	Purpose-built for experimentation; strong warehouse-native story; more advanced statistical methods
Eppo	Warehouse-native experimentation	Broader platform; doesn't require a data warehouse; integrated replay	Warehouse-native means they use your existing data; more advanced statistical methodology
Split.io	Feature flags + experimentation	Broader platform; better pricing; integrated analytics	More mature enterprise integrations

Honest assessment: Our strongest position is against teams paying LaunchDarkly prices for flags alone and not getting experiments included. The "flags + experiments + analytics in one platform" pitch is genuine and saves money. We're weaker against teams that need very complex flag management at enterprise scale (LaunchDarkly's core strength) or teams that want warehouse-native experimentation (Eppo's pitch). Our sweet spot is engineering teams that want the full loop: flag a feature, measure its impact, debug issues with replay, all in one tool.

Pain points & known limitations

Pain Point	Impact	Workaround / Solution
Flag management UX is simpler than LaunchDarkly's	Enterprise teams with hundreds of flags may want more organizational features	PostHog flags work well at scale. For very complex targeting, review the multivariate flags and payloads documentation.
No built-in flag approval workflows	Some enterprise teams want PR-style review before a flag goes live	Use existing code review processes (flags are in code). PostHog audit logs track changes.
Statistical methodology is Bayesian	Teams preferring frequentist methods may push back	Bayesian is faster to reach conclusions and easier to interpret. For teams that insist on frequentist, this is a real limitation.

Getting a customer started

What does an evaluation look like?

- **Scope:** Implement feature flags on one upcoming release. Ship behind a flag with gradual rollout. Optionally set up an experiment to measure impact.
- **Timeline:** 1 to 2 days to implement first flag. 1 to 2 weeks to see experiment results (depends on traffic).
- **Success criteria:** Can you gate a feature behind a flag and roll it out gradually? Can you instantly kill a flag if something goes wrong? Can you measure the impact of the change with an experiment?
- **PostHog investment:** Feature Flags free tier covers 1M requests. Experiments are included.
- **Key requirement:** Engineering needs to integrate the PostHog SDK into their codebase. This is the implementation step. Once the SDK is in, adding new flags is trivial.

Onboarding checklist

- [ ] Install PostHog SDK in the application (Feature Flags getting started)
- [ ] Create first feature flag for an upcoming release
- [ ] Set up gradual rollout (start at 5-10%, monitor, expand)
- [ ] Test kill switch: turn flag off and verify the feature is immediately disabled
- [ ] Set up first Experiment tied to a flagged feature, measuring a real business metric
- [ ] Enable Session Replay and filter replays by flag variant to debug an issue
- [ ] Review experiment results and use them to make a ship/no-ship decision
- [ ] Plan second experiment to establish the workflow as a team habit

Cross-sell pathways from this use case

If Using...	They Might Need...	Why	Conversation Starter
Feature Flags only	Experiments	They're gating features but not measuring impact	"You're rolling out features safely. But do you know if they're actually *working*? Experiments are included with your flags."
Feature Flags + Experiments	Session Replay	They're measuring impact but can't debug qualitative issues	"Your experiment shows the control winning. Want to watch what users in the losing variant are actually experiencing?"
Feature Flags (engineering-driven)	Product Intelligence (for the product team)	Engineering is in PostHog. Product team should be too.	"Your engineers use PostHog for releases. Has your product team seen the analytics? They could track feature adoption and retention without a separate tool."

If Using...	They Might Need...	Why	Conversation Starter
Feature Flags (for growth experiments)	Growth & Marketing (for the growth team)	Growth team initiated the experiments, engineering implemented the flags. Expand the growth side.	"Your growth team started the experiments. Have they explored Web Analytics and Marketing Analytics for attribution?"
Feature Flags + Experiments	Error Tracking / Observability	They're catching issues via experiments but want proactive error detection	"You're catching regressions through experiments. Error Tracking would catch exceptions before they show up in your metrics."
AI product releasing prompt/model changes	AI/LLM Observability	They need to detect quality regressions that error tracking won't catch	"After your last prompt change, did output quality hold up? AI Evals would tell you automatically."

Internal resources

- **Feature Flags docs:** Getting started · Feature Flags · Multivariate flags · Payloads
- **Experiments docs:** Experiments · Creating experiments · Exposures
- **Session Replay docs:** Session Replay
- **Competitive battlecard:** *To be added: LaunchDarkly competitive positioning*

Appendix: Company archetype considerations

Archetype + Stage	Framing	Key Products	Buyer
AI Native — Early	"Ship fast, break nothing. Feature flags let you deploy AI features to a subset of users and measure quality before going wide." AI Evals is especially relevant here.	Feature Flags, Experiments, AI Evals	CTO, founding engineer
AI Native — Scaled	"Your engineering team is growing and releases are getting riskier. Feature flags give everyone a safety net, and experiments make sure every change is measured."	Feature Flags, Experiments, Session Replay	VP Eng, Platform Lead
Cloud Native — Early	"Stop doing all-or-nothing deploys. Ship behind a flag, measure the impact, roll back in one click if something breaks." Speed and simplicity matter.	Feature Flags, Experiments	CTO, founding engineer

Archetype + Stage	Framing	Key Products	Buyer
Cloud Native — Scaled	"Multiple teams shipping to the same product. Feature flags give each team independent release control. Experiments ensure changes are measured, not just shipped."	Feature Flags, Experiments, Session Replay	VP Eng, EM, Platform team
Cloud Native — Enterprise	"Standardize your release process across teams and BUs. Feature flags + experiments give you a consistent framework for safe, measured releases at scale." Governance (audit logs, RBAC) matters here.	Feature Flags, Experiments, Session Replay + Enterprise package	VP Eng, Director of Platform, DevEx Lead

Use-case selling

contents/handbook/growth/use-case-selling/use-case-selling.md

We sell products. Customers buy solutions.

When we pitch "add Surveys," it sounds like we're trying to increase their bill. When we pitch "here's how to close the loop on why users drop off," it sounds like we're solving their problem. Same product. Different framing. Very different conversion rate.

Use cases are how we sell. Products are how we bill. A use case is a discrete problem a team is trying to solve, supported by a combination of PostHog products. Billing, metering, and packaging don't change. What changes is how we talk about it, how we organize around it, and how we measure adoption.

Each use case has a full playbook with discovery questions, competitive positioning, expansion paths, objection handling, and onboarding checklists.

The seven use cases

Use case	Job to be done	Core buyer
Product Intelligence	"Help me understand what users do, why they do it, and what to build next."	PMs, designers, product engineers, founders
Release Engineering	"Help me ship faster without breaking things."	Engineering managers, platform teams, developers
Observability	"Help me know when things break, understand why, and fix them fast."	SREs, platform engineers, DevOps
Growth & Marketing	"Help me understand what drives acquisition, conversion, and revenue."	Growth engineers, marketing leads, CRO, GTM engineers
AI/LLM Observability	"Help me understand how my AI features perform, what they cost, and how users interact with them."	AI/ML engineers, AI PMs, AI founders
Data Infrastructure	"Help me unify product data with business data and get it where it needs to go."	Data engineers, analytics engineers, product ops
Customer Experience	"Help me quickly understand what happened, identify the problem, and verify a fix."	Support leaders, engineering leads, CS leaders

Product coverage matrix

Product	Primary use case	Secondary use cases
Product Analytics	Product Intelligence	Growth & Marketing, AI/LLM Obs, Customer Experience
Session Replay	Product Intelligence	Release Engineering, Observability, AI/LLM Obs, Customer Experience
Feature Flags	Release Engineering	
Experiments	Release Engineering	Product Intelligence, AI/LLM Obs, Growth & Marketing, Customer Experience
Error Tracking	Observability	AI/LLM Obs, Customer Experience
Surveys	Product Intelligence	Growth & Marketing, Customer Experience
Web Analytics	Growth & Marketing	
Marketing Analytics *beta*	Growth & Marketing	
Revenue Analytics	Growth & Marketing	Product Intelligence
Workflows	Growth & Marketing	Product Intelligence
Product Tours *beta*	Growth & Marketing	Product Intelligence
LLM Observability	AI/LLM Obs	Customer Experience
AI Evals	AI/LLM Obs	Product Intelligence, Release Engineering
Data Warehouse	Data Infrastructure	
Data Pipelines / Batch Exports	Data Infrastructure	Growth & Marketing
PostHog AI	Horizontal (all)	

Product	Primary use case	Secondary use cases
Logging *beta*	Observability	Customer Experience

Playbook structure

Every use case playbook follows the same sections, so TAMs know where to find what they need:

1. Job to be done
2. Relevant PostHog products (with doc links)
3. Adoption and expansion paths
4. Business impact
5. Personas to target
6. Signals in Vitally & PostHog
7. Command of the Message (discovery, negative consequences, desired state, outcomes, metrics)
8. Competitive positioning
9. Pain points & known limitations
10. Getting a customer started (evaluation scope, onboarding checklist)
11. Objection handling
12. Cross-sell pathways to other use cases
13. Internal resources
14. Company archetype considerations

How you can help

contents/handbook/help.md

People who work at PostHog have come from all sorts of different backgrounds – large companies, small companies, agencies, single-founder startups, and so on.

Their modes of working necessarily differ from each other and from PostHog, but we expect you to work in some fairly specific ways.

Being the transparent company that we are, we want to let you know about those expectations, because you can only meet expectations when you are aware of them!

Generally, our values are a great place to start, as is as the handbook page on culture, but here are a few specific ways to apply those values, and reinforce our culture as we grow:

Getting yourself up and running quickly

If you're new, the default goal is to be able to get work done autonomously.

This will require:

- You know how to ship things in our environment and with your equipment
- You get to know your team and have a sense of who can review your work
- You get what the company, your customers, and your small team care about and needs to get done, so you can prioritize appropriately

Everything else is a means to this end! We often do onboarding in person to accelerate all the above. This usually takes around a week.

Ask for help, but only after you've tried first

If you've just joined us, there's a lot you probably don't know. That's okay! However, we *do* expect that you try to help yourself. Here's a framework to use as a guide:

- If it's one of our own products that we build and sell:
    - We expect you to read our docs.
    - We expect you to search through GitHub or Slack.
    - If it's in beta, it might be buggy or missing features. Definitely request/report them, but also figure out how to do your job with plan B and plan C if it won't be fixed quickly - this is what we generally call grit.
    - For engineers: try to figure something out by yourself for at least 1 hour, but don't remain totally stuck on something for more than 2 hours before asking for help. This time window should increase over time as you run into more questions that likely no one has the answer

to, at which point it's time to dig in and figure it out.

- If it's an external product:
  - We expect you to read the docs
  - If it takes you more than 10 mins to figure it out, then ask someone internally.

You can also try self-serving an answer in our #ask-max Slack channel. It's trained on our handbook and documentation, so it's capable of answering both questions about internal processes and procedures, as well as product-related questions.

If you don't get the context you're looking for, try #ask-posthog-anything where team members are willing to point you in the right direction. Take a moment to explain how you've tried to help yourself and linking to resources. That saves others valuable time searching the docs again, or typing up a suggestion to do just that.

Don't expect perfection

PostHog is a startup. As solid as our stack / product / CI / dev experience is for a company of our size (super solid, tbh), it might not be the extremely-well-oiled machine you had at BigCo. If something doesn't Just Work, follow the framework above to get help.

We're all human - you shouldn't expect perfection for adhering to our culture, either. But you should help others learn how to stick to our culture, especially new joiners. We're all prone to occasional lapses, and it takes everyone on the team nudging each other in the right direction to keep us all on track. If you notice something happening all the time, take it upon yourself to make it better - see the next section!.

Make it better

If you run into something you found that is confusing or needs fixing, we expect you to update the docs or handbook at minimum, and if you're keen then definitely improve the experience yourself. For example, CI is *everyone's* job. If it sucks, fix it.

That being said, there is often a *reason* why things are the way they are. That reason might be "because no one wanted to fix it," but it also might be "because it broke yesterday and we're on it" or "we've carefully considered this before and decided to make it this way."

We encourage you to step on toes, but don't be a bull in a china shop. Context is oftentimes your best friend – gather it up and keep it close.

Don't wait for someone else

We expect you to be proactive about answering questions in your domain, even very early on after you are hired – e.g. after the first week. Look in the code. Read the docs. Find the answer.

Being wrong is way better than being silent – if you are wrong, someone will correct you. If you are silent, you're not doing your job.

Similarly, if you need something to get done, you are responsible for making *sure* it gets done. This is not your team lead's job or some other team's job - if you need it, you own it. *Most* of the time this means doing it yourself (see section on helping yourself above); other times it means getting the right people together to understand the urgency and do it with you. But at the end of the day, the responsibility rests on you.

Have an opinion

You definitely don't need to have opinions on everything, but you should absolutely have opinions on your area of expertise.

If you don't have opinions on your area, you are realistically then just waiting for someone to tell you what to do, which is very much at odds with our autonomous way of working.

Opinions can take a bit to form, and that's okay – you don't need to have them on day one. But we expect you to start forming them rather early on, even if it's just on little things.

Look around corners

We expect you to be thinking through not only the one change you're making right now, but also how that change plays out down the road. What might happen with this code / process / thing in 6 months? Where will that leave my change today?

We do have more senior people on the team (both in industry experience and in their tenure at PostHog), but they shouldn't be the only ones looking ahead – you should be the primary one looking ahead for *your* changes.

Don't assign issues to people

You can list and categorize issues. If you want someone to see an issue, @mention them and/or Slack them the link.

Don't yolo merge

Do not "yolo merge" – i.e.: force a change to our website or platform without someone else checking it. This should *only* happen in emergencies, *even* for simple changes. It is *so* frequent that we find issues. If you have *any* doubt, get someone else to look at it first.

PRs > issues > Slack

Bias for action. If you can just pick up the work, do so. We want a culture of individual contribution *not* of delegation.

It is fine (and encouraged) to pick-up side quests, or to deviate from your goals if you think you should. Especially if something is a quick fix, do it yourself as part of our value that *You're the driver*.

If you aren't able to make a change yourself, create an issue in GitHub. Avoid simply relaying to-dos in Slack as a means of getting someone to pick up a task. It's hard to track and easy to forget.

Do things as publicly as possible by default

For discussions, public repos are the best place. Then private ones, then Slack public channels, then Slack private channels or DMs. This is part of our *"Make it public"* value, and helps with general context setting for the wider team, which means everyone can work more autonomously.

There are only a few exceptions to what we can't share publicly, for example if you are discussing security concerns, specific customers (for privacy reasons), revenue, or growth numbers (since these cause signalling issues with investors or competitors).

Internally, *everything* can be shared apart from people issues – such as HR / personal (i.e. recruitment or health data).

Be proactive with community questions

Don't *only* help the community when you're the person on support hero in your small team. No matter what your goals may be, if you can quickly ship fixes to real life user problems, then you are going to build goodwill, word-of-mouth growth, and a better product all in one swoop.

You can find these in posthog.com/questions.

And if you don't work here...

Apply for a job at PostHog!

How we get users

contents/handbook/how-we-get-users.md

Over 100,000 users have signed up to PostHog.

Most companies build their product with a particular user in mind. We build *everything* around our ideal customer profile.

So when it comes to marketing and sales, we are optimizing for developer experience.

Why we're like this

We've met a lot of successful founders in our space who are full of regret, despite leading companies with well over $100m in annual revenue! The one regret they *all* had in common was letting go of their growth engine (people recommending their product to each other) and getting focused on sales. They all wound up exiting. That's why they told us this stuff.

The way this pans out? As they got bigger, they gradually shifted from building for users toward building for buyers, hoping to optimize for revenue growth. Over time, that killed their word-of-mouth growth, which caused them to have to work harder for each sale. So they got more salesy, and so on. They became companies that focused on making a bunch of money *by* building a product, instead of being companies focused on building a great product.

We won't make this mistake.

For us, marketing is creating useful content

Our is small. Way, *way* smaller than our competitors'. Winning on volume of content is out of the question. So we'd better win on quality.

This constraint has worked out pretty well for us. Distribution is pretty easy when the thing you're working on is good enough to generate word-of-mouth growth – and this helps build an enduring developer brand.

We even hire full-stack developers into our marketing team to make sure we can cover the full depth needed in a lot of our tutorials, docs, and posts.

Things you won't find our marketing team doing: removing information from our website to increase conversion, focusing on paid ads, or letting colleagues ship content they aren't proud of.

We happily spend lots of money on our website

Most companies call it their "marketing website". You already know it's going to be crappy.

We treat our website as a product. With real investment. When we were just a couple of people, we realized that our website *is* our sales team – since our users would want to self-serve as much as possible.

When we started out, we also realized that all our competitors had crappy marketing websites.

And, as with so much that we do, we get an *increasing return* on quality. If we do things noticeably better than everyone else, then we're remarkable. That results in word-of-mouth growth.

We make it extremely easy for you to buy PostHog

Most sales teams do a bunch of low-quality cold outbound that harm their company's reputation and 99.999% of the time is ignored. And then once they've got you on a call, they pepper you with MEDDPICC questions before actually letting you see the product. Who knows, maybe 3 meetings later they'll share some pricing too!

We do things a little bit differently. Customers *buy* from us, we don't *sell* to them. It means we can instead invest our money in shipping more (and better) products, at lower prices than our competitors, to provide a sustainable advantage.

Fun fact: the total spend we have on marketing and sales per customer we acquire pays itself off within 3 months of them signing up for a paid plan. "Best in class" is considered to be one year...!

How we make money

contents/handbook/how-we-make-money.md

We make money from those that have it and like our products. We don't make money from those that don't.

How we do sales is based on the best experience for our Ideal Customer Profile

I cannot think of any harder group than developers to convince, via a cold-call or email, to buy software. We should focus on inbound.

All the other rules here are based on what we felt would be the best experience for an engineering customer, whilst allowing us to grow revenue in the long run.

Don't let pricing get in the way

Before a user has decided to buy the product, we should let them try it for free. Not only does this mean they can immediately self-serve without having to get budget internally, it also reduces the need for a large sales team to convince them otherwise. When someone is looking for a solution, they are ready to install it – but only if we can get out of the way commercially.

Once a user likes the product, we don't want to create a big decision around continuing to expand their usage with us. (For example, if we suddenly charged a large recurring price per month.) Instead, we charge a tiny fraction of a cent for each extra event they send.

Charge based on what people use, and give users control

Some users want to start with just a little usage of one product. Others replace five products with us. We should price to reflect this. We believe it's better to have a little extra pricing complexity to provide a much better value option, than an "all-in-one" price.

We charge by product *and* by usage of those products that people need.

Beyond which products we use, we look for other ways to give users control, such as spending limits on session recordings.

These principles mean that they will spend less than they otherwise would have, *but* it means they'll stick around. We don't want users to churn if they are unhappy with what they're spending; we want them to better manage how they use the platform.

Match the cheapest for each individual product

We can make it up by selling other products to the customer over time. This way, it's always a no-brainer to pick PostHog, we get as much word-of-mouth growth as possible, *and* our single product competitors can't compete since they have nowhere to go.

Principles for dealing with big customers

The most important thing here is to remain focused on building the best product, not on what a single big customer needs.

- **We don’t care about losing deals.** If we have to walk away from a deal because we'd have to compromise on these principles, we will. We can do this because we have a really strong growth engine with our ICP customers.
- **We don't contract deliverables,** and we especially don't contract to provide deliverables by a certain date. This is because, on principle, a single customer is forcing us to build something.
- **We will build things for a big customer, as long as we are confident they won’t be the only user of that thing.** Re-shuffling the roadmap a bit could make sense - but adding new things that others wouldn't use, doesn't.
- **Customers need to try PostHog before they expect us to change things.** We love feedback from customers. We don't love big requirement documents from people that haven't used our product before.

Enduringly low prices

contents/handbook/low-prices.md

We want our customers to spend their money on their engineering team, not on buying ten software products.

Here is the list of advantages we have and why they matter.

We can sell multiple products to the same people

So, do you want to buy ten products for $1k each or all ten for $5k? Or, better yet, each one separately?

We can pull this off because we're focused on getting in first – we don't follow the whims of whatever an enterprise may have.

No sales needed

Our competitors spend more on sales and marketing than product development. Nearly all our sales are self-serve *and* 70% of our customers find us through word-of-mouth growth.

Multiple products, one dataset

We aim to be the source of truth for customer and product data. The products we build all work from this same dataset, instead of ten different vendors all paying to store the same data as each other – each with their own platform teams.

A technical audience who need *docs*, not technical support

Many of our products are traditionally sold to non-technical people. They need more help setting up SDKs or snippets. We work with engineers who simply need good docs. Writing and maintaining those (and an open-source codebase that people can inspect or even fix bugs in themselves) saves thousands of support questions each year.

Using open-source technology

We've often had second-mover advantages.

One of these is that we could use the latest open-source technologies, like ClickHouse. Many of our competitors have had to build their own databases. You can guess which is more efficient for storing tens of billions of events and serving millions of analytics queries. Better them than us!

How we make users happy

contents/handbook/making-users-happy.md

User happiness is fundamentally important. How do we achieve this?

Building products that people want

First, someone internally will suggest an idea. Sometimes this will come from James and Tim, but it has, just as frequently, come from anyone else on the team.

If it requires a new team to build it – which it usually will – we'll start by hiring an ex-founder who is technical. We'll onboard them into the existing team that has the most overlap. This helps get them used to working with our codebase, as well as with the culture we look for from each team.

That person builds the MVP, and the only goal is to figure out if anyone will use it. With some products, the MVP may have more scope if we feel especially confident. Once the new product is in a non-embarrassing state (that won't harm our brand), we add pricing to it and put it on our website. This drives more demand. At this stage, the goal is to get the product to product-market fit in PostHog's platform, which means working with customers until we have five delighted, paying customers.

Once all this is done – which we'd expect to take a few months – we can start to innovate. This usually means some kind of platform play, such as extending the product to enhance everything else we're working on, or shipping another new product that would work well with it.

Engineers talk to users and provide support

You should be *as close as possible* to your users, feeling whatever they feel, so you have as much information as possible to make the product great.

For established products with a lot of usage questions (how do I create an insight that does X, for example), Customer Success helps with support.

Before a new product is even made, we'll add it to our public roadmap. Once it ships, we'll use our own tools to get customer interviews, feedback, and data, and we'll always aim to "close the loop" with users - coming back with: a pull request, a GitHub issue they can follow in the open, or an explanation of why we can't make a feature they've asked for.

This means the product improves, users are impressed and recommend us to others, *and* we show users that we listen, encouraging them to keep going through this loop with us, faster and faster.

Campaigns and coupons

`contents/handbook/marketing/campaigns-and-coupons.md`

We run promotional campaigns with partners (e.g., newsletters, influencers) that offer exclusive benefits to their audiences via coupon codes.

How it works

1. **Campaign setup**: Campaigns are created in Billing Admin with a strategy defining what benefits are granted (e.g., free addons, increased limits, credits)
2. **Code distribution**: Coupon codes are exported as CSV and shared with the partner for distribution to their audience
3. **Redemption**: Users visit `/coupons/{campaign-slug}` to redeem their code (requires paid PostHog subscription)
4. **Expiration**: Benefits can automatically expire after the campaign period (e.g., 12 months)

Onboarding flow integration

When new users sign up via a campaign link (e.g., `posthog.com/signup?next=/coupons/lenny`), they're shown the coupon redemption page early in onboarding:

1. User signs up with `?next=/coupons/lenny` query param
2. After signup, they're redirected to `/onboarding/coupons/lenny` instead of directly to `/coupons/lenny`
3. They can claim the coupon or skip and continue to product setup
4. After claiming/skipping, they proceed to the normal onboarding flow (use-case selection or products page)

This ensures new users see the coupon offer before diving into product configuration.

Note: Existing (already onboarded) users bypass this and go directly to `/coupons/:campaign`.

Example: Lenny's Newsletter

When launched, our partnership with Lenny's Newsletter offered their annual subscribers:

- Free Scale addon
- 2x free tier limits on all usage-based products
- Valid for 12 months from redemption
- Only for organizations with no paid invoices before December 1st 2025

Redemption page: `/coupons/lenny`

Creating a new campaign

For technical implementation details, see the internal billing docs.

Co-marketing

contents/handbook/marketing/co-marketing.md

PostHog complements a lot of other software companies. Since we're active in the startup ecosystem and built around integrations, co-marketing opportunities come up naturally.

Who takes the lead with co-marketing?

Sales, engineering, or support will sometimes tag marketing into customer Slack channels where someone mentions co-marketing. There's no obligation to say yes just because a partner is enthusiastic. If you're unsure whether something is worth pursuing, ask in the **#team-marketing** Slack channel.

If it does seem promising, a product marketer will take the lead and loop in events or other teams as needed.

What this article doesn't cover:

- Influencers and newsletters
- Rev-share partners and individual consultants
- Sponsorships

Integrations and CDP destinations

We have a CDP with 50+ destinations and a data warehouse that connects to tools like databases, Stripe, and HubSpot. Any time we ship an integration, there's a baseline level of co-marketing we should do:

- Make sure docs are solid on both sides
- Add it to the changelog (and changelog email if it's really good)
- A simple social media post like this one

If you have a new integration which deserves marketing support, the best way to get it is to ask in the **#team-marketing** Slack channel. The team will discuss and specific product marketer will take responsibility for running co-marketing.

When to level up: Most integrations stop here. But if there's genuine opportunity, then it's worth doing more:

- A practical tutorial like this one
- A video walkthrough if the workflow is complex
- A blog article or other co-authored content

Typically whenever we are pursuing work like this with a partner, we work with them to reciprocate to their audience through their channels.

In some cases where there's big partnership potential, the partnership is of a strategic significance, and the ICP is the same, feel free to explore a joint in-person event that will gather the community of both partners and deliver value from both sides.

> Like all PostHog marketing, co-marketing should be useful to the reader. A super simple way to signal compatibility without being promotional is to casually reference partner companies in docs and editorial. However, we avoid case studies where there isn't an interesting story, guest posts, and other marketing 'fluff' content.

For example, PostHog docs might say "If you're routing LLMs (e.g. via OpenRouter)..." while OpenRouter docs say "Track downstream behavior in PostHog..."

We do this out of goodwill anyway in blog posts like this. It helps readers and costs us nothing.

Enterprise integrations

We haven't done much co-marketing with enterprises like Slack or HubSpot because big companies typically move slowly, and we haven't prioritized it.

If you find yourself working with an enterprise integration partner that's actually responsive and interested in co-marketing, go for it. Just don't let their timeline block other work.

PostHog customers

If a customer is a logo we'd proudly show on the site, represents who we build for, and is getting real value from PostHog, then a case study usually makes sense.

Examples:

- PostHog + Supabase
- PostHog + Mintlify
- PostHog + Lovable

Social media co-marketing for case studies naturally follows since most companies are excited to have their story featured. It's usually worth raising an art request for these opportunities.

We also will typically thank customers who participate in case studies and collab content by sending them a merch voucher. We're nice like that.

Startup and ecosystem partnerships

We already run a strong startup program. Accepted companies get $50K in PostHog credits plus access to partner benefits. This is one of the best types of co-marketing because it's a simple value exchange: we help their users, they help ours.

However, we are very selective about which teams we partner with here because these partnerships usually offer outsized benefits to them. As a rule, we want to have no more than three such partners at once - and it's one-in, one-out.

Examples:

- Easier incidents with Incident.io ($1,500 off a teams plan)
- Better SDKs with Speakeasy (50% off for 6 months)
- Better search with Chroma ($5,000 of credit for their search and retrieval service)

If we're signing anything with legal commitments, that needs to go via #legal. If it's an informal exchange of perks, you can usually just coordinate directly with the partner company.

If you're giving PostHog credits, check with the to explore the options. See the campaigns and coupons handbook entry for more detail.

When a new startup perk goes live:

- Add the offer to the PostHog for startups page
- Create a new Startup landing page for this partner
- Announce it on social media like this
- Add the offer to relevant emails and onboarding flows (the startup program has dedicated flows)

As a rule we don't commit to reporting sign-up performance to startup partners, as it just adds overhead and they should have their own methods of tracking. We also don't typically agree to rev share deals as part of this program, as it's a long tail activity.

> More questions about startup program partnerships? Head to the project Slack channel.

Other ecosystem partners

Co-marketing goes both ways! PostHog's startup program is promoted through partner channels like Stripe Atlas perks and the Fin Startup Pack.

We maintain a spreadsheet with most of our current partnerships. This also includes partnerships with VCs and PE firms. For the most part we do no co-marketing with these partners, though this may change. This spreadsheet doesn't list all VC partnerships via GetProven, as these are best tracked directly through that tool.

Co-sponsored events

Events are a great place to co-market and vary from intimate gatherings to large scale meetups. These are higher effort and don't usually sit under product marketing alone. Tag Daniel early – he's the best judge of what events and co-sponsorships will actually land.

Examples:

- We buy AI YC pitch event with Chroma, Mintlify (and others)
- MCP Builder breakfast with Fiberplane
- Building with (and for) AI event with Vercel and Profound

Virtual events can also work for co-marketing, we just try to avoid boring ones. For example, we participated in ElevenLab's Worldwide Hackathon, which was rad.

Co-branded merch

Merch collaborations are cool, but should be rare. They require real work from the design team and need a clear purpose beyond "we're partners now."

A good example is the limited edition t-shirts we did with Supabase, which was way more fun than a press release.

If you think a merch collab makes sense for a co-sponsored event, use the art request template. Please give thought to distribution and lead-times and add this to the request.

What if the partner wants more?

Not every co-marketing play warrants maximum effort. If someone's pushing for more but the product overlap is thin, it's okay to suggest starting smaller. A changelog mention and social post can always expand into more later if there's real traction.

Customer case studies

contents/handbook/marketing/customer-case-studies.md

What makes a good case study?

Case studies should make our users look smart, our products look useful, and PostHog look like a company people actually want to talk to.

Things we don't care about:

- if they pay us or not (most customers don't)
- if they use every tool in the box (they might be a power user of only one product)
- if they have a recognizable brand (big logos are nice, but more frequent, smaller stories often beat enterprise red tape)

Things we do care about:

- that PostHog has helped them achieve meaningful results
- that they represent who we build for
- that someone else might benefit from reading their story

> Case studies are typically owned by the . They live in `/contents/customers/` and appear on posthog.com/customers. If you have a suggestion for who we should interview, let us know in the marketing channel.

Creating a case study

1. Identify the right customer

Start by asking the PM for that product. PMs do lots of user interviews and can suggest warm leads. You can also post in company Slack channels, but give some context for what you're looking for.

2. Make contact

Got a lead? Before reaching out, search for the company in Vitally. If they already have an assigned Account Executive or CSM, give that person a heads-up — they might already be working on something with the customer or have extra context on what to ask them about.

If there's no one assigned in Vitally, you're clear to go ahead and reach out directly.

Some customers have a dedicated Slack channel. If they do, that's usually the fastest way to coordinate. Otherwise, send an email.

3. Lay the groundwork

Someone agreed to chat? Hooray! Make a GitHub issue to draft some questions, tag any relevant sales/CS people, and note if you'll need artwork later.

4. Schedule the interview

Who you talk to for interviews doesn't really matter. Speak to engineers, founders, PMs, or anyone who seems keen to chat. If you're unsure who to interview, email a few people at the company and see who bites.

> We use Calendly for scheduling external meetings, such as demos or product feedback calls. If you need an account, ask Charles to invite you to the PostHog team account.

How to be a good interviewer:

1. Do some preliminary fact-finding (don't waste time asking general info about the interviewee's company and role)
2. Come prepared with good, open-ended questions
3. Relax and have a nice chat (30 minutes is plenty)

5. After the call

Trust your gut — if it feels like a good story, it probably is. Worst case, it's still user feedback to pass on to other small teams.

If it is worth turning into a case study, draft a PR right away while it's still fresh. Ask at least one teammate to review it to catch any grammar mistakes (or really bad jokes).

Best practices:

- **Be specific** - Use real numbers and measurable outcomes where possible
- **Use quotes** - Let the customer's voice come through
- **Keep it concise** - Aim for between 700-1400 words including quotes

6. Review and approval

PR looking good? Tag the customer in Github for review. You're not asking for copy edits – just a quick fact check. Legal and PR teams will sometimes want to be looped in for approval as well. They might also request using Google Docs instead of GitHub.

Do what you need to do. The goal is to get the rubber stamp.

> If your draft might include anything private such as screenshots of customer dashboards, keep it in an internal repo like requests-for-comments-internal just to be safe.

7. Publish!

Most people are excited to be featured and will sign off quickly. If you need artwork to go with the case study, use the art or brand request template

Once the case study is merged and live on the website, the last step is to send a merch credit to the participants as thank you.

That's it - you did it!

Marketing Events

contents/handbook/marketing/events.md

> Want PostHog to be involved in your event? See how we do Community events. Want to start a co-working group for builders? Check out our Community incubator program and submit the form. If you'd like to add an IRL event to the events page, contact or .

We did 45 events in real life (IRL) in 2025 and we're just getting started. While we're 100% remote and set up to work asynchronously–we've found real benefits in getting together with users in real life. All our public events are showcased on the events page.

Events have to be focused on and valuable to our ICP. We prefer not to be a small fish in a big pond, hence we mostly pass on big conferences. And we prefer pull over push, so we gravitate towards content and formats that educate and activate while avoiding booths, badge-scanning, buying attendee lists, paying to speak, and webinars.

The event formats we prefer (and organize ourselves) fall into one of these:

- Hands-on gatherings that enable our users to build better products for their customers
- Experiences that allow engineers and founders to walk away with unique product engineering insights
- Getting product engineers together to identify problems and build solutions for users
- AFK time that we ourselves enjoy like hiking, gaming, cycling, cooking classes, etc.

All plans come together – from conception through to final delivery – on our event management tool which centralizes owners, logstics and feedback in one place.

Community incubator

We connect builders around the world by helping them start IRL micro-communities that gather for recurring co-working sessions. As we know from our own sprints, offsites, and hackathons, we can build a whole lot when we gather in person with other people who have a bias for action.

We have already seen how this format makes a higher impact on communities because of the velocity built over weeks and months of communal work, collaboration, and creativity.

Taj leading his builder group in Philadelphia

Geographies

The pilot program started in tech hubs mostly in North America, UK, and EU. We now have communities in Austin, Philadelphia, Singapore, New York, Barcelona, and Lahore. At this time, we're open to groups starting in any city with a population of more than half of a million people.

Co-working structure

The focus is on weekly, bi-weekly, or monthly gatherings with small groups of ~10 people. Gatherings typically take place during weekday evenings or weekends and go for about 3 hours.

Discuss → Build → Demo

We suggest starting with a roundtable discussion of the latest dev news and trends, then each builder can set their own goals for the session. Allow at least 2 hours for building, and then close out the session with demos to show off what you built. Outside of co-working, the group is encouraged to get together to connect for an AFK activity such as a walk, bike ride, hike, or local sightseeing.

Venues

The ideal venues for the community incubator are free-to-use spaces conducive to a group comfortably working (accessible, quiet, Wi-Fi-enabled.) They are typically held in tech or VC offices that have an available room, libraries, and co-working spaces. If you have a venue and want to host a builder group, reach out to Daniel directly.

Community events

Community events are in real life (IRL) manifestations of our mission organized by enthusiastic partners and customers. They usually originate when someone has identified an interesting topic or problem set for an event and want to help people move faster, smarter, and more together.

These are some of the event formats we're most actively pursuing:

- **Builder Breakfasts:** bringing together 35 engineers for unconference-style discussions of hyperspecific software problem sets they are encountering. Breakfast served.
- **AI Talks with Demos:** technical meetups for 75 people with live demos, where startups and scaleups demonstrate how they're deploying AI native tools to solve various problems.
- **Founder Fireside** deep dives with one of our co-founders (and) for 100 founders and product engineers.

What community events are **not** for:

- Forcing PostHog or any other product into conversations with people
- Watching or planning things rather than doing them
- Just networking for the sake of chit chat

Formulating a purpose and structure

All impactful event follow the principles of user-driven development which stem from the user problem or requests. Who is the ideal attendee profile for your event? They might be your customers, fellow founders, local engineers or any other collection(s) of people. Talk to them first to validate if the event is worth your time.

Put real effort into this first step. Defining the "what, why and how" of an event beforehand will pay off on event day. Let our shared values guide you. Don’t submit your event for support until your answer to “Would I attend this?” is a clear “YES!”

Getting support

Financial support: We are happy to support the growing ecosystem of PostHog users and product engineers more broadly through financial sponsorship. We do this often for events that align with everything outlined on this page. Budgetary support typically falls in the range from $500 to $3,500. When we support monetarily, it almost always involves some added level of engagement.

Speakers: Want a speaker in our ecosystem (team PostHog, customers, partners)? We'll try our best. When considering speakers for your events try to avoid:

- Corporate speak aficionados spewing tedious enterprise marketing nonsense
- People LARPing (live action role-playing) as executives
- Loudest person pretending to know more than they do

Content: If your speaker(s) are unsure of what to talk about, consider going back to the purpose of the event. Otherwise, we have plenty of material for your inspiration.

Merch: We use the store merch processes to handle distribution of PostHog-branded merch. We tend to be generous with merch for community events. Outline what you had in mind in the issue.

Co-promotion: Most of the time the help requested is in the form of promotion. As a general rule, we don't promote events we aren't supporting or co-hosting ourselves. We decide when to repost community events on our social media channels and email on a case by case basis.

Venue and catering: Identify the vendors and costs and include them in the GitHub issue. If the event will not be possible without monetary support, make that clear. We may support the cost of venue, food, or beverages but require the paper napkin math.

Feedback: You'll learn more by doing than planning so don't worry about having every detail complete before submitting for feedback from our team.

Branding it

Our brand is a reflection of us and how we're experienced by others, including events.

Words: Naming products is hard. Same goes for naming events and writing their descriptions. As a prerequisite, read our primer on writing for developers. Try your best to come up with event names that communicate the 'what?' and will attract the 'who?' And then again ask yourself, "would I attend this?"

Pictures: Every event is improved with a flyer or poster that showcases the essence of the experience. We keep a comprehensive list of brand assets and guidelines on the brand assets page. Share your assets and we'll give feedback. Depending on the scale and timing of the event, our team may be able to help with branding as well.

Event recaps

Community events are better when organizers share what happened, what you learned, and any follow-up actions. We value feedback and expect the same from event organizers. In addition to what you learned and feedback from attendees, we ask that you share any photos, videos, quotes, data points with our team.

Sponsoring external events

We often get invited to sponsor events - these range in size, location, and audience. We rarely say yes. For these to be a worthwhile endeavor, the sponsorship should be a win-win primarily for the end user and secondarily for us. Hence, it's important that the audience, content, format, and ethos to all align. Even if we don't sponsor financially, we encourage team members to speak at events and we can support with merch. Ask in the #team-irl-events channel.

Speaking at events

If you're interested in attending or speaking at a developer conference, consider submitting a CFP (Call for Papers) to one of these events taking place in 2026. If you don't see an event you're interested in, please add it directly in the reference sheet.

For first-time yappers, reference the speaker's guide. If you need inspiration for a talk, pretty much any practice we use for actual production code is fair game. This includes integrations and implementations with other products. And at this point people are interested in not just what we build but how we did it.

Sponsoring student organizations

Sometimes students at varying universities ask us if we are interested in sponsoring their career fairs, hackathons, or other student-led initiatives. We don't currently participate in these. Although we don't use specific years of experience as a qualifier for hiring, we rarely hire students straight out of school. If there is a custom partnership you have in mind or it involves an existing employee's alma-mater, ask in the #team-irl-events channel.

Exporting a blog post image from Figma

`contents/handbook/marketing/exporting-blog-post-image.md`

Overview

Blog post images are created in Figma. The image appears at the top of each blog post, above the headline. It's also used as the Open Graph image.

Dimensions

Open Graph images are `1200x630`, so we stick with those dimensions to keep this simple. (This is approximately double the size they'll be displayed at, making them look nice and crisp on HiDPI screens.)

Export an image from Figma

1. Custom blog art lives in Figma: Art board → Blog
2. Make sure artwork fills the entire frame.
3. Ensure frame doesn't have a border.
4. Rename the frame of the image to closely match the blog post title in a slug format. (Ex: `writing-for-developers`, where we remove capital letters and punctuation, and replace spaces with hyphens. This will become the filename that is uploaded to the server.) It's best to omit articles (a, and, the).
5. Export the image as a PNG (at 1x).
6. Save the image and add it to the issue.

The image should be uploaded by the person creating the blog post.

Adding to a blog post

1. Upload the file to `/contents/images/blog`.
2. Make sure the filename matches the reference to the image in the .md file.

Incident comms

contents/handbook/marketing/incident-comms.md

> These guidelines are for marketers who support engineering during incidents. For engineers, we have additional guidance on how to declare and handle an incident. For GTM workflows and templates, see the communication templates for incidents.

Incidents happen. Each one is different and not all incidents require comms, but when they do we need to have clear processes in mind.

For this reason we've kept our guidelines as flexible as possible and focused on providing high-level guidance and responsibilities. In the event that an incident occurs we trust each others' judgement on when to adhere or deviate from these guidelines.

Appointing a Comms Lead

During and following an incident, Product Marketing Managers (PMMs) generally assume responsibility for handling customer communication at a broad level.

- If an incident is focused on a particular product and that team has a PMM focused on it, that PMM typically takes responsibility and becomes the Comms Lead.
- If this is unclear or there's no dedicated PMM, then ownership should be decided by the available PMMs and a single Comms Lead should be clearly designated in the incident channel.

The role of the Comms Lead typically involves planning how we will respond at a high level by:

- Creating a simple comms plan (who we talk to, what we say, and when).
- Taking ownership of any large-scale communication to users.
- Coordinating with Support, Sales, and Success so we don't duplicate or contradict each other.

> **Oh no --- all the PMMs are on holiday or asleep!** If this happens, the incident lead may appoint a Comms Lead from the Content Team or another team. If the incident lead fails to appoint a Comms Lead, Team Blitzscale should appoint someone to lead Comms.

Guidelines for Comms Leads

These are principles to keep in mind during any incident:

- **Identify the per-product impact.** This helps scope the customer impact. Always clarify the impact on feature flags, experiments, and workflows especially. It's always worth asking how it impacts each product and if any data is lost, or merely delayed.

- **Don't rush external comms.** It's better to be slower and correct than fast and wrong. The status page and support tickets usually cover the early phase while details are changing quickly.
- **Default to transparency, not overcommunication.** We shouldn't send comms unless there's a definite impact and a clear story to tell. If we do send external comms, target **owners and admins in impacted orgs** where possible, rather than being too noisy.
- **Use the status page as the primary public channel.** The status page should be the main place we direct users to during an incident. Extra channels (emails, social posts) are the exception, not the rule. If a post-mortem is created, this supersedes the status page.
- **Aim not to send broad customer comms until an incident is resolved or a post-mortem is published.** Major or critical incidents will often have a public post-mortem – this should usually be the backbone of any wider comms. Don't communicate before resolution unless there is a strong need.

> **When handling a security incident:** align with the incident lead in the incident Slack channel about public communication of security issues before proceeding. E.g. it could make sense to hold back communication of an attack publicly, as this could make the attacker aware that we are investigating already. This could it make harder for us to stop this attack for good. However, in some cases of data breach and security incidents, like the download of malicious packages, it is better to notify users immediately, in case the incident lead has identified that **users can take action to prevent the malicious packages from spreading further.**

What does the Comms Lead do?

At a high level, the Comms Lead is responsible for *how* we talk about the incident, not for fixing it. In practice, that usually means:

- **Join the incident channel immediately and make your role clear.**
- **Stay in the loop without adding noise.** Read the summaries and updates, follow the thread, and avoid asking for updates just to "check in". Only jump in when you need information for comms, or have a specific ask.
- **Make sure the status page is accurate and up to date.** Check-in periodically to ensure the status is updated at least once every six hours, that the current impact is accurately described, and that the incident is closed when needed. The Incident Lead is responsible for these updates.
- **Decide whether we actually need outbound comms.** If we do, you should put together a plan for doing so (below).
- **Draft, coordinate, and send any messages.** When we *do* decide to communicate:
    - Create a comms plan to coordinate the response. This should be an issue in an internal repo. Here's an example comms plan for a critical incident.

    - Share comms drafts in the incident channel and on the comms plan for quick fact-checks from the incident lead or engineers.
    - Keep messages in plain English, impact-first, and avoid status-speak. Use existing communication templates for incidents as a reference.
    - By default, communicate through email rather than in social posts. Social posts can exacerbate an issue.
    - Direct users to the status page or post-mortem (if available) as the source of truth.

    > **When do we need to notify users immediately?** For security incidents, like the download of malicious packages, in case the incident lead has identified that users can take action to reduce their risk, we should notify users immediately with clear steps how to act on their side. Product downtime that doesn't involve security breaches/attacks should be addressed after the incident is closed and we have the context needed to inform users.

- **Support the post-mortem process.** For major/critical incidents you may need to help shape and review the post-mortem with the incident lead and approvers (Tim and/or Ben, and Charles). Once published, use the post-mortem as the primary reference for any follow-up comms (emails, service messages, etc.), rather than rewriting multiple different explanations.

    > **After a data breach/security incident** the comms lead should contribute to the post-mortem by transparently addressing the impact, what went well, and what could have gone better.

- **Keep Sales and Support teams notified of impact.** Often these teams are dealing with the brunt of the customer response and your goal should be support them by giving them the information they need to respond effectively.

- **Handover to another Comms Lead, if needed.** Most comms can be handled quickly, but in the event of a long-running issue you should develop a plan to handover or continue monitoring the incident status.

These steps are a starting point, not a script. In practice, the Comms Lead's job is to keep communication **accurate, calm, and useful** --- and to reduce noise, not add to it.

What does the Comms Lead not do?

The Comms Lead is typically not responsible for:

- **Updating the Status Page.** This should fall to the Incident Lead.
- **Updating VIP customers.** This usually is best handled by the Sales team.
- **Providing technical support to users.** They can leave that to the Support team.
- **Making technical decisions about the incident.** The Incident Lead will handle this.

Overview

`contents/handbook/marketing/index.md`

How marketing works

Marketing at PostHog is a collaborative effort across several teams. It is also specifically a multi-disciplinary creative team in it's own right. If that's confusing, you can check Who can help me? for more on who to talk to.

Marketing values

1. Be opinionated
2. Pull, don't push
3. No sneaky shit

1. Be opinionated

PostHog was created because we believed that product analytics was broken, and we had a vision of how it could be much better. We're more than just product analytics now, but the principles are the same.

We need to reflect this vision in our marketing and content, and not dilute it with boring corporate-speak. When we write content, we take a firm stance on what we believe is right. We would rather have 50% of people love us and 50% hate us than 80% mildly agree with us.

We communicate clearly, directly, and honestly.

It's ok to have a sense of humor. We are more likely to die because we are forgettable, not because we made a lame joke once. We have a very distinctive and weird company culture, and we should share that with customers instead of putting on a fake corporate persona when we talk to them. PostHog should not look like a generic software company.

(Sometimes we use terminology like 'value propositions' because that is the standard marketing term for a well-understood concept. That's allowed.)

2. Pull, don't push

We focus on word of mouth by default.

We believe customers will judge us first and foremost on our product (i.e. our app, our website, and our docs). We won't set ourselves up for long-term success if we push customers into using us.

If a customer doesn't choose PostHog, that means either:

1. The product isn't good enough

2. The product isn't the right solution for them
3. We didn't communicate the product and its benefits well enough

We don't believe companies will be long-term customers of a competitor because they did a better job of spamming them with generic marketing. We know this because we frequently have customers switching from a competitor to us – they are not afraid to do this.

Tackling (1) is the responsibility of everyone at PostHog. The job of marketing teams is to avoid spending time advertising to people in group (2), and making sure we do a great job avoiding (3).

This means:

- Making sure our comms are extremely high quality
- Sharing our messages in the right places, where relevant users can see them
- Spending enough time and/or money in those places so that our messages get through

3. No sneaky shit

Our ideal customers are technical and acutely aware of the tedious, clickbaity, hyperbolic marketing tactics that software companies use to try and entice them. Stop. It's patronizing to them and the marketing people creating the content.

For these reasons, we:

- Don't use any analytics except PostHog. No Google Analytics, Facebook Pixel etc. Customer trust is more important than making our marketing team's lives easier.
- Don't make claims about our product that are not 100% genuine and verifiable. And we don't make promises for future functionality either beyond what's already in GitHub.
- Don't unfairly criticize or make false claims about our competitors. We will compare ourselves to them to help customers make a decision, and occasionally they will be a better solution for what a customer needs. And it's ok to have a sense of humor about this.
- Don't bombard customers with 'deals', pop-ups, and other dark patterns. These devalue our product in the long term.
- Don't pretend our customers are different from us – i.e. more gullible, more susceptible to marketing. We are an engineering-led team building products for other engineers. If you wouldn't like it, assume our customers wouldn't either.
- Don't do cold email marketing to acquire new customers. When was the last time you read the 8th email a company sent you and thought 'ok yes, I now want to use this product'?

Marketing vision

Beyond PostHog's company mission and strategy, we have some marketing-specific areas we want to focus on.

Things we want to be brilliant at

- **Word of mouth mindset:** We want to build a hugely successful company driven primarily by word of mouth, rather than paid ads or PR. This means being known for quality in all things we do.
- **Helping our ideal customers be successful:** Through our docs, tutorials, newsletter, emails, video, and beyond, we help our ideal customers be more successful, both generally in their goals as founders and engineers, and as users of PostHog.
- **Launches:** Our team ships a lot of products and features. We need launches to break through the noise and get noticed. This helps create the momentum products need to succeed.
- **World-class documentation:** We work with product teams to maintain up-to-date, high quality docs. We work with Website & Docs to ensure users can discover them. Doing this enables users to "self-serve," discover what they need, and get the most out of PostHog.
- **Supporting YC founders:** Lots of PostHog's DNA comes from Y Combinator. Their companies and founders are our ideal customers. We've done a great job being valuable to them (50%+ of batches using us) and want to continue to do so.
- **Merch:** We make the coolest tech company merch. Let's keep it this way.

Things we want to be good at

- **Events:** We have been involved in some events, but we are still figuring out "the PostHog way" to do them. We don't just want to be a name on the sponsor list. We want to create superfans.
- **Social media:** Specifically Twitter, where we've seen good traction posting on James' personal account and the PostHog brand account. We have been posting more on LinkedIn to promote the newsletter. We don't use any other social media channels.
- **Paid ads:** We run a lot of paid ads on Google and others. It is fuel for everything else we are doing. We want to be good at this, but do it in a way unique to PostHog. We're not throwing everything at the wall and seeing what sticks. We have a minimum brand bar we need to hit.
- **Graphics:** We're not the most visually focused team, but creating visuals and animations is a great way to communicate complex ideas. They also make for excellent content. Create a basic version and get the design pros (Cory, Lottie) to help.
- **Developer influencers:** We sponsor creators like Theo and Fireship to drive awareness and signups to PostHog. Many of the influencers we sponsor don't work out, but the ones that work drive great results.

- **Billboards:** Billboards are a way to get our brand in front of a lot of people.
- **Doing sales without salespeople:** Rather than care a lot about "capturing every lead" or "marketing qualified leads," we'd rather work with sales to create content that helps potential customers, ideally without a salesperson.

Things we might want to be good at but haven't tested yet

- **Broader partnerships:** PostHog is a complement to a bunch of types of companies, from vibe coding tools to infrastructure platforms. Our data warehouse and CDP are built to enable integrations. How can we leverage this?
- **Video essays:** Video essay style content is a natural extension of what we are doing in our newsletter. When done well, it is what "great video content" looks like.

Things we don't want to spend time on

- **Optimizing marketing spend:** We're more concerned about growing fast than being the most efficient marketing team. Go fast, run experiments, look for upside.
- **Big, highly coordinated marketing campaigns:** We can do them, but our reactive, short turnaround campaigns have been far more successful.
- **PR:** If we do word of mouth well, our community will be far more valuable/credible than an appearance in TechCrunch.
- **Being cool and interesting people in online communities:** There are a bunch of communities we *could* be more active in like Reddit and Discord, but we'd prefer to focus on our own community first.
- **Conferences.** We're not a natural fit for conferences and being a small fish in a big pond isn't really our style.
- **Short-form video.** We tried it, but it didn't work. Our audience *might* be there, but we're not flashy or dedicated enough to reach them.

Influencers

contents/handbook/marketing/influencers.md

We work with creators and influencers to make content about PostHog and sponsor placements to drive awareness and sign ups.

We're open to inbound proposals, if you're interested in collaboration with us, you can send emails directly to , we're always open to seeing thoughtful proposals.

Some of the influencers we sponsor include:

- Theo. He is a great partner for us. His audience is ideal, he is doing YouTube right, and he's a PostHog user.
- Fireship
- Chris Raroque

Sourcing and evaluating influencers

- You can find new influencers by looking at the creators engineers share or mention internally, searching for ones who have made relevant videos, looking at the recommendations of ones we've already sponsored, inbound, and Passionfroot.
- Their audience needs to be relevant to us, ideally targeting our ICP, but broadly engineers and founders.
- Even within this group, there are some categories to avoid like job interview prep, career growth, low-level engineering, and heavy computer science focus. Try to find web or mobile developers, product engineers, startup founders, and indiehackers instead.
- The channel is growing, gaining in subscriber growth rate and views. Use Social Blade to see this.
- They should have engaged audiences. Strong Twitter or Discord communities are good signs as well as view to like/comment ratios:
    - **Weak**: 0.001 comment/view, 0.02 like/view
    - **Average**: 0.001–0.002 comment/view, 0.02–0.03 like/view
    - **Good**: 0.002–0.005 comment/view, 0.03–0.05 like/view
    - **Excellent:** 0.005+ comment/view, 0.05+ like/view
- Above 5k views per video. Anything below this is just not worth your time. This number will likely grow over time. Larger influencers, although they charge a lot more, are often more efficient, so we don't have an upper limit for size.

- Both short video creators and podcasters **haven't** worked well for us in terms of conversion and signups. We're open to trying this again in the future though.

Negotiating with influencers

- Make sure you know what type of slot it is: pre-roll, mid-roll, end-roll, integration.
- Ask for examples of other ad slots they've run to judge the quality of ad read.
- For influencers we've never worked with, you should negotiate quite aggressively. The number they give is usually pulled out of a hat and is often 2-3x higher than it should be.
- Use our model for how much a placement should cost to get to a better number and work towards that. Feel free to make changes to the model if you think it's not accurate.
- Make sure the link is in the top 3 lines of the description.
- We pay invoices net 30 (30 days after they've sent them to us).

What should the placement actually look like?

- Make a judgement call on whether to use a unique link set up with Dub (like `https://go.posthog.com/sponsored`) to point to specific UTMs unique to each video or an influencer-specific link using `posthog.com` redirects (like `posthog.com/theo`) to the same UTMs across videos. These links should have `utm_source` and `utm_campaign` set to the influencer and campaign name.
- Make sure they tell their audience to "mention them on sign up" or "say that they heard about PostHog from them" so we can track the attribution.
- We generally let influencers decide on what the ad read is like so that it best fits their audience. We can provide guidance on what to talk about though. These points are helpful to share:
    - PostHog is a developer platform that helps people build successful products. We provide a suite of dev tools to help them do this.
    - This means product analytics, web analytics, session replay, error tracking, experimentation, feature flags, LLM analytics, surveys, and more.
    - We also have a CDP for sending data to 50+ destinations and a data warehouse product that lets you connect to external sources like your database, Stripe or Hubspot to query with SQL (or no code insights) alongside your product data.
    - The goal of these tools is to help founders and engineers debug their product, understand their customers, analyze usage, and ship a more successful product faster.
    - We have a generous free tier for every one of our products. You can sign up and get started with all of them for free right away. 90% of users use PostHog for free.

    - Setup is as simple as installing one of our SDKs or pasting a snippet into your site header, we then autocapture data like pageviews, button clicks, and sessions. We also have SDKs for all the popular backend languages like Python, Node, Go, etc.

- There are a handful of video assets for them to use here. Feel free to add more, but also suggest the website and in-app as sources.
- Ideally, a placement should relate to ongoing marketing efforts, like a new product launch, product push, or pricing change.

Measuring impact

Some metrics we look at for individual videos include:

- CPM (cost per thousand views)
- Unique sessions from the custom link (or clicks if you're using a Dub link)
- Sign ups, either from converting from sessions or who mentioned the influencer on sign up

We track these on our marketing budget and spending spreadsheet. We also have an influencer marketing performance dashboard in PostHog that can help you get an overall view of different influencer's performance.

Who can help me?

contents/handbook/marketing/ownership.md

If you have a general marketing question, go to `#team-marketing` in Slack.

If you need help with the website, go to `#posthogdotcom`.

Here's a quick guide to who to ask if you want help with a specific marketing activity.

- ▶ I need a product marketer, but I don't know which
- ▶ I'm interested in running, attending, or speaking at an event
- ▶ I want to launch my product out of beta
- ▶ I need help with documentation
- ▶ Someone wants to write a guest post / is requesting a backlink etc.
- ▶ Someone wants to partner with us
- ▶ Someone wants us to sponsor them
- ▶ I want to create a video, or have a video idea we should try...
- ▶ A customer is interested in doing a case study with us
- ▶ A customer has an issue with merch
- ▶ I have a question / problem / suggestion for the website
- ▶ Hey, can we run some paid ads for my product?
- ▶ I need a new hedgehog design, illustration, or other art asset.
- ▶ I need a font, logo, etc.
- ▶ A journalist has contacted me

Paid ads

contents/handbook/marketing/paid.md

The paid ads team exists to do two things only:

- Generate high quality new business leads for the sales team
- Convert high-intent, high-ICP-scoring engineers searching for PostHog or products we offer into signups that drive MRR growth

We don't do paid ads for general awareness of the PostHog brand - our website, content, and word of mouth are much better (and cheaper) ways to do this.

> For now paid ads sits within the , but will become its own thing once we have more people. This page is for paid ads for PostHog in general. If you're looking for paid ads for our newsletter, see the newsletter ads guide.

Channels

We currently run ads on:

- Google Search - *conversion*
- Bing (for DuckDuckGo) - *conversion*
- LinkedIn - *conversion* + *leads*
- Reddit - *conversion* + *leads*
- YouTube - *conversion*

We have previously tried and no longer use X, Product Hunt, Carbon Ads, and Google Display, as they did not drive high quality user signups or leads. We usually focus campaigns on users in the US, Canada, UK, Germany and France, as these tend to lead to the most high quality signups and leads.

We work with Hey to manage these channels - they set up the campaigns and ensure that spend is paced properly. We have a shared internal Slack channel, and has 2 check-in calls with them each month.

In addition to Hey, we also have a monthly call with Google Partners who provide feedback on performance and competitive analysis on a per product basis as requested.

Mission 1 - Generating new business leads

We have four ways we target new leads:

- ABM on LinkedIn
- ABM on Reddit
- Light prospecting on Reddit

- Light prospecting on LinkedIn

We use a variety of creative campaigns here which we don't list in the Handbook, as they keep changing over time. Some principles though:

- We are open to using gated content if it is fun and/or weird and/or actually useful
- Gated content must be freely available elsewhere

The full flow of how this works can be found here.

Tracking conversion & conversion optimization

Using 3rd party trackers or pixels like Google Tag Manager is against our brand and values, so we use a combination of PostHog, BigQuery, Clay, Clearbit, & Census.

PostHog sends back anonymized (click ID) conversion data to each ad platform with conversion values based on ICP score to improve lead quality via target ROAS bidding. Our goal is to use our ads program as a powerhouse for the sales team and a key tool for onboarding users that will improve both MRR and CAC:LTV ratio.

In order to keep our privacy policy front of mind we've built a bespoke conversion tracking system that uses the following flow:

PostHog > Clearbit > BigQuery > Census > Ad Platforms.

You can learn more about this flow here here.

We take privacy seriously, and follow these principles:

- If it creates third-party cookies for us, don't do it
- All testing must align with our privacy policy
- Always verify what data is collected and how it is used
- Don't collect or share any user PII contained within PostHog, obviously (Including IP addresses)
- Limit data collection only to what is *absolutely* required
- Always be transparent with users about what we're collecting, if any
- All ClickIDs are considered safe to send back to each ad platform

Mission 2 - Converting people to signup

We do this through search ads on Google and Bing, and you can find the master sheet of ad copy here.

We change up campaigns frequently, but generally run campaigns for:

- Brand
- Individual products
- Competitors

We generally turn these on and off depending on performance and spend, and review copy every 4 weeks. The flow is writes copy, reviews. We try both fun and straightforward copy. Even if the fun stuff doesn't convert super well, we keep it if it's doing ok as it helps with our brand - we know people screenshot and share it sometimes.

We aim for as much product coverage as possible unless there are compelling reasons to not do them (e.g. it's just very expensive). We prioritize ads for those products closest to our ICP.

> It is typically only worth running paid ads for individual products once they are generally available, with pricing, and where we feel the feature set is broadly on parity with the main competitors.

Landing pages

We use custom presentations that match the style of our website as our landing pages. We have an internal guide on creating presentations.

In addition, PostHog now allows us to copy a URL to share a set of open windows in a specified layout allowing further customization for ad landing pages.

How we work

Brand guidelines and creative

By default, all paid ads visual creative should be based on stuff that already exists in some form on one of:

- Our website
- Product for Engineers
- Hoggies library
- Merch
- Events
- Video
- Billboards

We take anything we've ever created there, and then repurpose/reformat/reconfigure it as an ad. This minimizes approvals - because these assets have previously been through a round of approval with design, we can use them knowing we don't need to get approval again. The only check required is then between and on the concept and/or copy. This means we are doing less creative work, but the upside is that we can move faster (and have a lot of Lego bricks to play with). works with on this.

For the copy itself, we also use the search ads copy where we can as a starting point, so we're not repeating work.

If we have a particular campaign in mind that *really does* require a new, one-off asset, then we request it from Lottie in the usual way.

Budget

maintains the media plan, which can be found here.

Growth review

runs a monthly growth review with Charles where we look at the main performance metrics for the month prior. Here are the main sheet and commentary. For completeness, this also covers the organic funnel, though the main focus is still paid.

Product Positioning

contents/handbook/marketing/positioning.md

How do we name things?

Here's a typical flow:

- engineer builds cool thing
- engineer gives it a name
- design thinks it should be called something else
- um

So, how do we name things:

- engineer builds cool thing
- *sometimes* James or Tim realize it's happening and get the positioning right first time around
- but if they don't / or don't spot it...
- engineer gives it a name
- design iterates the name (and adds it to the all hands so we can get everyone else to realize this has happened)
- everyone - reinforce the name if people are calling things the wrong thing

This has a downside - it's messier from a user perspective, but the upside is that design / "execs" aren't a blocker to getting work out the door. In practice, we rarely push hard on marketing a new thing to users anyway (usually we soft launch stuff) so we think the downside is pretty minimal.

Picking a good name

By default, everything should be positioned as something a *user* is familiar with, not what is necessarily the most technically accurate description.

For example, when we build new products, we often name them based on what the major competitors are calling themselves.

This means users get it way faster, so we grow more quickly, and it encourages us to build the basic features that a given product needs versus trying to innovate *before* we hit product market fit with a new product in our platform.

What positioning actually means

Positioning is more than just picking a name. It's about understanding how users will encounter, understand, and use what we're building.

It also means being clear about **what problem we're solving** and **who it's for**. Are we building this for someone debugging an issue right now, or for someone planning next quarter's roadmap? The same feature might be positioned differently depending on the context.

We also think about how new capabilities fit into the **broader PostHog story**. Every new product should reinforce our core positioning: one platform that gives engineers everything they need to build successful products.

Positioning is dynamic

The reality is that positioning changes as products mature. Early on, we might position something narrowly to get feedback from a specific user segment. As it grows and we understand usage patterns, we can broaden or refine that positioning.

We're comfortable with this iterative approach because it means we're not overthinking positioning before we know what users actually want, and how the product fits into the broader market.

Product announcements

contents/handbook/marketing/product-announcements.md

> **Have something you want to announce?** Let the Marketing team know! If it's an iterative update, you can also demo it in the all-hands, or post in the #tell-posthog-anything Slack channel.

Product marketers take responsibility for coordinating and publicizing news about PostHog, including product announcements. We also help with incident and maintenance announcements, if needed.

Types of announcement

We classify announcements using the general guidelines below, with full discretion for doing something different.

Minor announcements

Minor announcements involve changes which have no noticeable impact on the experience of most users. They can involve small visual changes, such as UI tweaks, but are more often small bug fixes or back-end changes. They do not require action from users and pose no known risk.

We may typically support minor announcements by:

- Including them in the weekly changelog update.
- Writing a short Twitter and/or LinkedIn post.

An example of a minor announcement is the UUID format change.

Medium announcements

Medium announcements involve changes which have a noticeable impact on the experience of some users, but not the majority. They are likely to involve visual or functional changes, such as adding a chart type, but do not introduce wholly new features. They do not require action from users and pose no known risk.

We may typically support medium announcements by:

- Including them in the weekly changelog update and related emails.
- Creating an in-app changelog notification.
- Writing a Twitter and LinkedIn post.

An example of a medium announcement includes the launch of the NPS app.

Major announcements

Major announcements involve changes which have a noticeable impact on the experience of most users, or require specific action from affected users. They may introduce new features, require product downtime, or include opt-in betas for upcoming work.

We might do anything and everything for a major announcement.

Examples of major announcements include the surveys beta or the analytics pricing change.

New product announcements

New product launches are major announcements. They have their own GitHub template: Launch Plan. Product marketers should always create a launch plan for new product announcements.

For new product announcements we generally apply the following best practices:

- Ensure the product has a product page added to the website.
- Ensure the product team has implemented intent and activation signals for the product.
- Ensure the product has at least one customer story created for it within 3 weeks of launch. example
- Ensure we publish best practice content for the product and link to it from docs. example
- Ensure the product has at least one tutorial created for it at launch. example
- Ensure launch activities (such as changelog) link clearly to the docs.
- Ensure the product is added to email and in-app onboarding flows.
- Ensure the product is added to the pricing page (this is typically owned by the product team's PM and the)
- Submit an art request for any creative assets needed for the email campaign, blog post, social media posts etc...

Comms should also be aware of the engineering best practices for product launches, so we can be sure that features launch well.

If the product is moving from free beta to paid general availability (GA) you might also want to choose a reward for beta users. Examples of this include giving PostHog AI beta users 30 extra days of unlimited free usage, or giving Workflows beta users a discount code for merch.

PR announcements

We do not typically do public relations for anything other than company-level news. We have separate processes and guides for managing press announcements.

Maintenance communications

Occasionally, we have to conduct scheduled maintenance. When this happens, it's important that we tell users about it in advance if they would experience any disruption.

> If you're aware of any upcoming maintenance which would cause disruption, please inform the Support, Marketing, and Customer Success teams as soon as possible. Marketing will ensure that users are notified as the work is planned and completed. Customer Success may wish to inform specific users at the time.

Typically, Product Marketers take responsibility for informing users about maintenance work beforehand by telling users who will be impacted through email and other channels.

When informing users about maintenance, it is important to answer all of the following points:

- When will the maintenance occur?
- How long will it take?
- Who will be impacted?
- Will any data be lost?
- Do users need to take any sort of action?
- How will feature flags and experiments be impacted?
- What will the impact be? Will insights, etc., still function?
- Why is the maintenance being done, and what benefit will there be for users?

We typically notify users of upcoming maintenance by email, so the Marketing team will need a way to target the correct users before they can update them. For smaller maintenance updates which will not cause any user updates, engineering teams can also update our status page.

Incidents communications

When an incident is declared the Brand team should join the incident channel as observers, and monitor to make sure that customer comms are handled correctly.

Guide for doing PostHog talks and demos IRL

contents/handbook/marketing/speaker-guide.md

You volunteered or have been asked to speak at a dev meetup, give a demo at a conference, or present PostHog to a virtual or in-person audience. Maybe you said yes before you thought too hard about it. That's fine — good talks happen this way. This guide is for preparing and delivering your talk.

> For examples from other speaker, reference slides from previous talks. **Have any questions?** Ask in #team-irl-events or ping whoever put you up to this.

1. Know your room before you write a word

Before you build anything, answer three questions:

Who's in the room? A meetup for early-stage founders is different from a Clickhouse conference. Find out: their persona. Are they product engineers or founders? What sized team do they work at? What stack? What company stage? Also, how many people will be in the room? Ask the organizer — they want your talk to land too.

What format are you filling? Confirm the exact setup:

- Length (20 minutes including Q&A? 45 minutes? Lightning talk?)
- Slides, live demo, or both?
- Will you have reliable Wi-Fi, or should you run demos locally?
- Microphone or projecting your voice?
- Is it recorded?

What else is on the agenda? If you're one of four speakers, you don't want to cover the same ground as the person before you. Get the full lineup.

2. How we talk about PostHog (and how we don't)

No talk should ever be a blatant product or company pitch. Whatever your audience, they didn’t come to this event to receive a pitch (anyone can visit PostHog.com themselves.)

The PostHog voice in talks:

- **Let the work speak.** Lead with what you built, what broke, what you learned. PostHog appears naturally because you work here and we love to dogfood.
- **Share real lessons.** The interesting part of any content is the thing that went wrong, the assumption you had to throw out, the number that surprised you, the unpopular opinion.
- **Have opinions.** "Here's what I think about this" is more useful than "there are tradeoffs on both sides." Take a position, avoid hedging.

- **Be inclusive with language.** Avoid jargon that excludes people who aren't already PostHog users. Assume the audience is smart, not that they're familiar.

If you finish writing your talk and the word "PostHog" only comes up three times, that's probably good.

3. Build the talk around one true thing

Kelsey Hightower — a best in class technical speaker — doesn't use slides as a crutch. He treats his talk like a live demonstration of a belief. Every word moves toward a single point.

Pick your one true thing → build evidence for it → show it working live

That's the whole structure. You don't need five points. One claim and the proof. Good examples of what a "one true thing" sounds like:

- *What we learned from pivoting 5 times before reaching PMF*
- *Lessons from a year of building AI agents in production*
- *Why we stopped writing unit tests for our data pipeline (and what we do instead)*
- *How we cut our onboarding drop-off by 60% without a redesign*

Notice what these have in common: they're useful to the audience whether or not PostHog exists.

4. Your demo is the talk

Software demos should tell a story, not show features. The biggest mistake we can make demoing PostHog at events is simply narrating the UI instead of showing a problem being solved.

Bad: "So here's the dashboard. You can see we have charts. This one is a trend. This one is a funnel..."

Good: "We shipped a new onboarding flow last Tuesday. By Wednesday I was looking at this drop-off and thinking something was wrong. Here's what I found." Then show *that*.

Pick one real scenario — something that happened at PostHog related to your work, or something a real user told you. Build the entire demo around it.

Demo setup checklist:

- [ ] Use a demo project, not a live account with customer data
- [ ] Pre-load data that looks real — sparse data makes features look broken
- [ ] Disable notifications on your laptop
- [ ] Silence your phone
- [ ] Bookmark your demo URL — don't type it live
- [ ] Know what happens if Wi-Fi dies (screenshots as backup)
- [ ] Zoom your browser to 125–150% so the back row can read it
- [ ] Test the projector before anyone arrives

If you're pre-recording your demo, #team-youtube, has created this helpful guide.

5. Building your slides

A few principles for building out slides:

- Before the slides, start on paper or in a notes app and build out your talk outline
- PostHog talks use our standard slide template in Figma. Here's a guide on how to use it.
- Code on slides: use a large font (24pt minimum), a dark background, and only show the lines that matter. If you're pasting a full file, you've already lost.
- One idea per slide. If you're writing full sentences, you're writing speaker notes, not a slide. If applicable, allow memes to replace text.
- If a slide doesn't support the one true thing you identified in step 3, cut it.
- Speaking of speaker notes, you will save yourself time and head space if you always have notes.

For feedback on design or help with navigating the PostHog brand assets (Hoggies included), stop by #team-marketing

6. Practice out loud. Twice minimum.

Reading your talk in your head doesn't count. Your mouth is slower than your brain. The VM Brasseur public speaking guide has a useful rule: practice until the words feel *boring* to you. If they still feel fresh and interesting when you say them, you haven't done it enough.

Two run-throughs, out loud, at speaking pace, with your actual demo running.

The "cut" rule: If you stumble on a section more than twice in practice, that section is probably bad. Rehearsal reveals structural problems — stumbling usually means the logic isn't clear, not that you need to practice more. Stop, figure out *why* it's hard to say, and fix the content.

7. The first 60 seconds are everything

Open with something that makes the room lean in:

- An uncomfortable question: *"How many of you are flying blind on what users do after signup?"*
- A specific number: *"We had 847 session replays from one user in a single session. Here's why."*
- A short story: *"A customer emailed us asking why their funnel had 0% conversion. The button said 'Sumbit'."*

Don't introduce yourself first. The host does that. You start with the thing. Then you can re-introduce yourself to set the context of why you're the person qualified to speak on this subject.

8. Prepare to not know something

We always want to encourage Q&A after our talks as it builds conversation and connection. Someone will ask a question you can't answer. Don't bullshit. The right response: *"I don't know — but here's how I'd find out, and I'll follow up with you."* Then actually follow up.

If you receive a question that you believe is off-topic or unfitting for the setting, you can let the asked know this and express an interest in moving on to the next one.

9. After the talk

- **Express an willingness to keep the conversation going** - by letting the audience know that you (and any other team members in the room) are sticking around to chat more.
- **Write down the questions you couldn't answer** — do this right away so you don't forget and can focus on interacting with attendees the remainder of the event.
- **Tell the marketing team** — a 2-line Slack in #team-irl-events with the event recap, approximate audience size, and any interesting take-aways.
- **Share your slides** — Share them on social, QR code, email, path of least resistance. Don't make people hunt.

Examples from previous talks:

- Feb 2026 - - Lisbon SaaS Founders Presentation
- Feb 2026 - - 10 learning from launching an agentic AI product at scale
- Nov 2025 - - How to build a cult
- Oct 2025 - - Code that fixes itself
- Oct 2024 - - Parsing at the speed of light

Dashboard templates

contents/handbook/marketing/templates.md

Dashboard templates simultaneously showcase the use cases of PostHog and make it easier for users to get started. You can find a full list of them on the templates page.

This is "internal" documentation to show PostHog staff how to add new global templates.

Let us know on this GitHub issue if you'd like to see templates that are private for your team.

Creating a new dashboard template

1. Create your dashboard with all the insights you want on it. Be sure to add descriptions to both.
2. Open the dashboard dropdown, click "Save as template."
3. Add variables as objects with the format below. Reference them in your template by adding the ID in curly brackets, like `{SIGNUPS}`, to replace the placeholder event.

```
"variables": [
  {
    "id": "SIGNUPS",
    "name": "Signups",
    "type": "event",
    "default": {},
    "required": true,
    "description": "The event you use to define a user signing up"
  }
],
```

4. Once done, click "Create new template." Test that it works in the team project.
5. Create a dashboard image in Figma in the Hoggies file. Make the size of image small (like 396x208). Export and upload to Cloudinary.
6. With the URL, go to templates tab under dashboards, click the three dots to the far right of your template, and click "Edit." Add the URL to the `image_url` field and press **Update template**.
7. For the website, copy the same hedgehog as a small square thumbnail image (400x400) with a transparent background. Export and upload to Cloudinary.
8. While you are in Figma, create a 1920x1080 feature image with a couple of the insights. Export and upload to Cloudinary.

9. In the `posthog.com/contents/templates` folder, copy another `.mdx` file from another template, and modify for your new template. Add the thumbnail and feature images you uploaded to Cloudinary.
10. Open a pull request.
11. Once merged, click the three dots on the far right again, and click "Make visible to everyone."
12. To add to EU Cloud, click the three dots to edit the template and copy the JSON. Go to the PostHog EU Cloud instance, create a new blank dashboard, click "Save as template", paste the JSON (minus `deleted`, `created_at`, `created_by`, `team_id`, and `scope`), and "Create new template." Add `image_url`, edit, and test if needed. Finally, make visible to everyone.

Removing a dashboard template

If you ever need to remove a dashboard template, you need to:

1. Open the templates list
2. Click on the three dots to the right of the template you want to remove and then click **Make visible to this team only**. This is a required step before you can delete it.
3. Click on the three dots again and then click **Delete Dashboard**.

Be sure of what you're doing as this is a non-reversible action.

Overview

`contents/handbook/marketing/video.md`

The YouTube team's mission is to:

- Increase awareness of PostHog, especially among people in our ideal customer profile, through content that is genuinely entertaining
- Help developers and PostHog users be more successful through, through content that is genuinely informative

Our model is that of content creators, not a marketing department, as it is for the rest of content. This means that we start from a place of producing great video first that stands alone, irrespective of its connection to PostHog and our products. We have found over the years that if we get this right, the marketing benefits naturally follow, but if we start from a marketing-first perspective, people are never as interested.

As a result, our focus is on awareness, not converting people to sign up or revenue. Other parts of our marketing and product are a much better fit for that.

Who is our audience?

It should be the same as who we are building for, but specifically:

- **Product engineers:** Software engineers who want to improve their product skills, understand users, and build successful new products.
- **Founders:** Technical and non-technical founders seeking advice on how to run a successful startup.

Who is our competition?

Our competition is *not* our competitors' video content (the bar is too low), it is the other stuff that our audience is generally watching and enjoying. Think popular YouTube creators, podcasts, even viral content on TikTok and Instagram.

Entertaining vs. informative

Videos exist on a spectrum from 100% entertaining to 100% informative. Some are in the middle and do a bit of both. Generally, when we are making a video, we should try to aggressively pursue one path only, as doing both is *extremely difficult* - the most likely outcome is that we fail at both and end up with something that is ok.

What does success look like in 2026?

- We have a successful YouTube channel. Success means momentum. Momentum means a predictable publishing schedule of videos that fit our audience and brand, thousands of views per video, and a strong understanding of what works.
- Video that extends the reach of the brand. We have a consistent and well-oiled system for producing video that extends the reach of PostHog products, features, and the brand.
- We are a benchmark. People love what we do. They want to copy what we do. We are proud of our work. People on the internet cite our video as an example how to do video right.

What we're working on

Going from most informative to most entertaining:

- **Core brand assets:** Demo videos and short feature trailers that showcase cool things you can do in PostHog.
- **Launch videos:** The goal is to have a creative format that's repeatable and can be shot at (reasonably) short notice.
    - These can also easily be repurposed as ads
- **Startup stories:** Regular YouTube output focused on telling the stories of cool companies that our audience is interested in.
- **Brand-centric hero videos:** We have a unique culture and brand, and we want to share that through weird content that no one else would invest in because it's too hard - who else would do an action figures video for April Fools'?
    - This is where we have the greatest freedom to #do-more-weird - we just need to be wary of the pitfalls of corporate try hard

How to work with the video team

- Know what type of video you want to make. Try to fit it into one of the above categories (and remember, we're up for doing a "one-off" film that is totally different to what we've done before).
    - Got a random idea? Share in the content and video ideas channel.
    - Got a specific request? Use the video request form and be thorough.
- Know your product, even if it's early. What does it do? What are the important features? What are some real world use cases? What's coming next? Why should someone pick it over a competitor? What's interesting about it? Many videos will rely on these details.
- Accept that the team may say no to your cool idea. Like all small teams, Team YouTube have the final say in what is worked on.

Working with the website team

contents/handbook/marketing/working-with-website.md

The website is owned by and . For general questions or quick updates, the best place to start is the `#posthogdotcom` Slack channel.

For most pieces of work, like blog posts and copy updates, you can ship without needing a review from the website team. However, for larger pieces of work — a new product page, a significant copy overhaul, a new landing page — there's a more structured process to follow.

> **Why can't I vibecode?** You can, but vibecoded stuff tends to be harder for the website team to review and has a tendency to not work well with some existing systems.

Requesting large website changes

1. Draft the content in a Google Doc

Start with words, not designs. Write out the full copy, structure, and any specific requirements. This gives the website team something concrete to work from, and keeps early-stage feedback focused on what matters: the message.

2. Submit it to the website team as a GitHub issue

Open an issue in the posthog.com repo using the Website Request template and link your Google Doc. Include:

- A brief description of what you're trying to achieve and why
- A link to your Google Doc
- Any relevant context or references
- Your timeline and deadline, if there is one

3. The website team builds from that and opens a PR

Once the issue is picked up, the website team will build the page. They'll open a PR and tag you when it's ready for review so you can give feedback on changes.

4. Review and give feedback from the PR

Review the PR, leave comments, and iterate from there. This is the right moment to give design and layout feedback — not before, when things are still just ideas.

Why this process

This approach is designed to stop time being spent on designs that don't get used. It also keeps the dynamic clear: the PMM team hands off, the website team builds, and everyone can feedback.

We don't want to overbake or complicate this process. This is as simple as it can be and as complex as it needs to be.

Chrome extension billing case study: Wildfire Systems

contents/handbook/onboarding/chrome-extension-billing-case-study-wildfire.md

Summary

Wildfire Systems implemented PostHog in a Chrome Extension environment. Due to how extensions handle session and identity persistence, they experienced unusually high event volume and feature flag calls, which led to inflated billing.

This document explains the technical causes, the customer's solution, and how to identify similar cases using Metabase.

Technical root cause

Issue	Explanation
PostHog re-initialized on every extension wake	Chrome extensions create a new runtime context when switching from idle to active. Each context re-initialized PostHog without access to prior storage.
A new `distinct_id` was created each time	Since local storage is isolated per context, the PostHog SDK could not persist the ID. This triggered a new anonymous ID on each wake cycle.
`identify()` was called repeatedly	Each new ID triggered a comparison to the persisted UUID. Since they always differed, `identify()` was called each time.
`identify()` triggered `reloadFeatureFlags()`	Every call to `identify()` refreshed feature flags.
`/flags` requests were billed, even when quota-limited	PostHog counted these requests toward the usage quota, even if the response returned no flags.
Added budget mid-cycle had no effect	When the team increased their billing limit, it did not retroactively unlock flags. Only new requests after the monthly reset were allowed.

Fix implemented by the customer

The Wildfire team applied the correct approach:

1. **Persisted a shared UUID via `chrome.storage.local`** This ID was generated once, then reused across all extension contexts.
2. **Bootstrapped PostHog with the UUID** On every initialization, the `distinct_id` was passed via `bootstrap`.

3. **Avoided calling `identify()` unnecessarily** The team checked if the existing `distinct_id` matched the UUID before calling `identify()`.
4. **Minimized `/flags` requests** Bootstrapped feature flag values were passed during init, reducing the need for real-time flag fetches.
5. **Used PostHog dashboard to monitor** The "My PostHog Billable Usage" dashboard showed real-time data to verify that fixes worked.

How to spot this in metabase

If a customer is using a Chrome Extension without proper initialization, you will often see the following patterns in the usage dashboard:

1. Extremely high `identify` event counts

- `identify` makes up more than 70 to 90 percent of all events
- Often accompanied by minimal actual user activity events (clicks, views, etc)

2. `/flags` usage is abnormally high

- Feature flags represent a significant portion of the total volume or cost
- Check the forecasted bill by product to confirm this

3. Total event volume appears inflated without a matching frontend footprint

- Session volume may be high without corresponding actions
- Repeated initialize-then-identify patterns from ephemeral clients can drive this

4. Usage patterns appear to "pulse" or reset regularly

- Graphs show sharp daily spikes at regular intervals
- Indicates extension wake cycles creating new sessions and IDs

5. No batch exports, minimal standard library usage

- Chrome extensions often do not use session replay, heatmaps, or full web libraries
- You may see a custom library version or just raw SDK usage

6. High `$set`, `$identify`, `$groupidentify` volume with few custom events

- Suggests backend or SDK-driven implementations without user interaction data

When you see these signs together, it is a good idea to ask: "Are you using PostHog in a browser extension product or other ephemeral context?"

If confirmed, you can share bootstrapping and identity persistence best practices.

Recommendations for extension developers

Task	Details
Persist ID manually	Use `chrome.storage.local` to persist UUID
Bootstrap identity	Pass the UUID during `posthog.init()` with `bootstrap.distinctID`
Avoid repeated `identify()`	Only call `identify()` if the ID has changed
Reduce `/flags` usage	Use `bootstrap.featureFlags` and disable polling if needed
Monitor proactively	Use the "My PostHog Billable Usage" dashboard
Educate early	Customers should be aware that extensions require manual handling

Metabase account analysis playbook

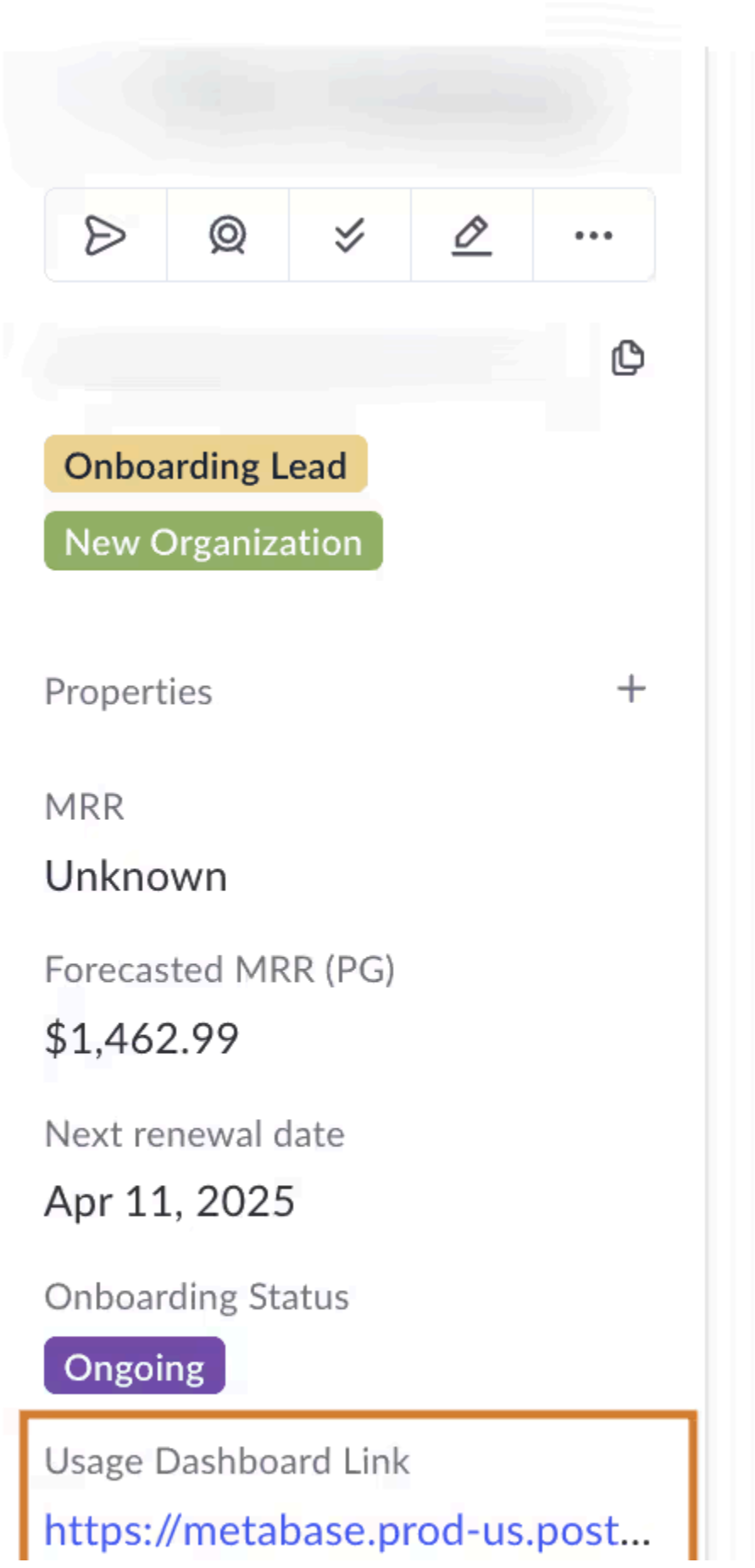
contents/handbook/onboarding/metabase-account-analysis.md

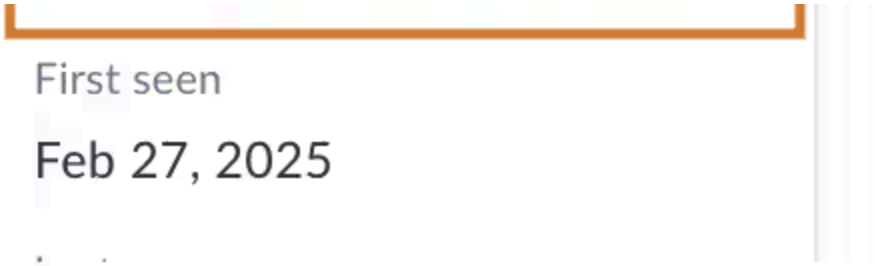
Summary

Metabase dashboards mirror a customer's PostHog usage so we can diagnose billing, implementation quality, and quick-win optimizations. For audio and video learners, check out these:

- Metabase overview recording
- Billing deep dive walkthrough

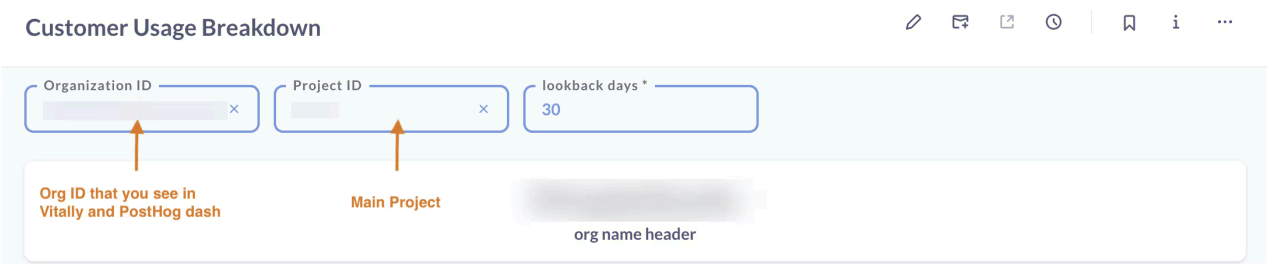
While checking the account in Vitally, you can access a dedicated Metabase dashboard for this account directly from the sidebar:


Onboarding Lead
New Organization
Properties
MRR
Unknown
Forecasted MRR (PG)
$1,462.99
Next renewal date
Apr 11, 2025
Onboarding Status
Ongoing
Usage Dashboard Link
https://metabase.prod-us.post...



(Note that you may need to configure your properties first to see that by clicking on the "+" button next to properties)

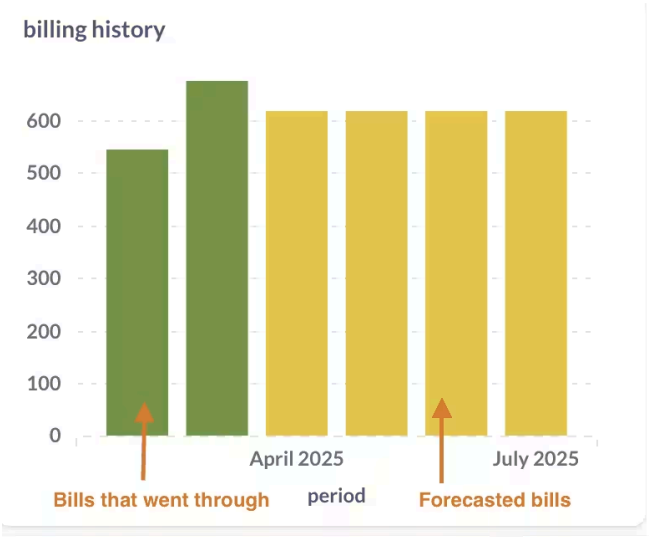
There are two Metabase instances, US and EU, that correspond with the PostHog instance that the customer is on. There might be some visual differences, but accessible data should be roughly the same:

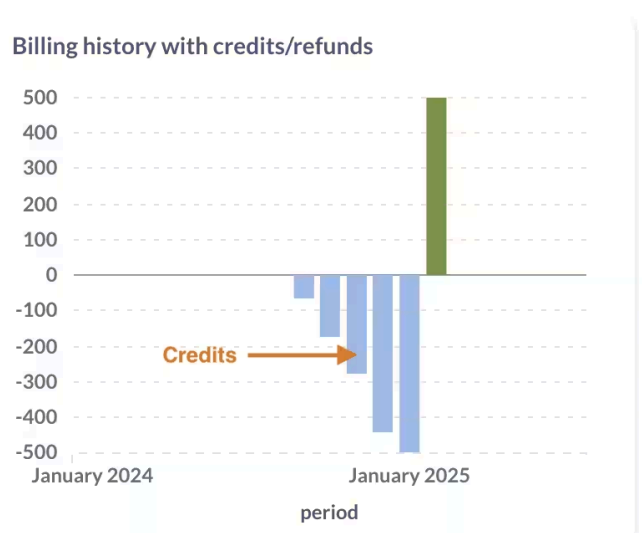


What to pay attention to

1. Billing history

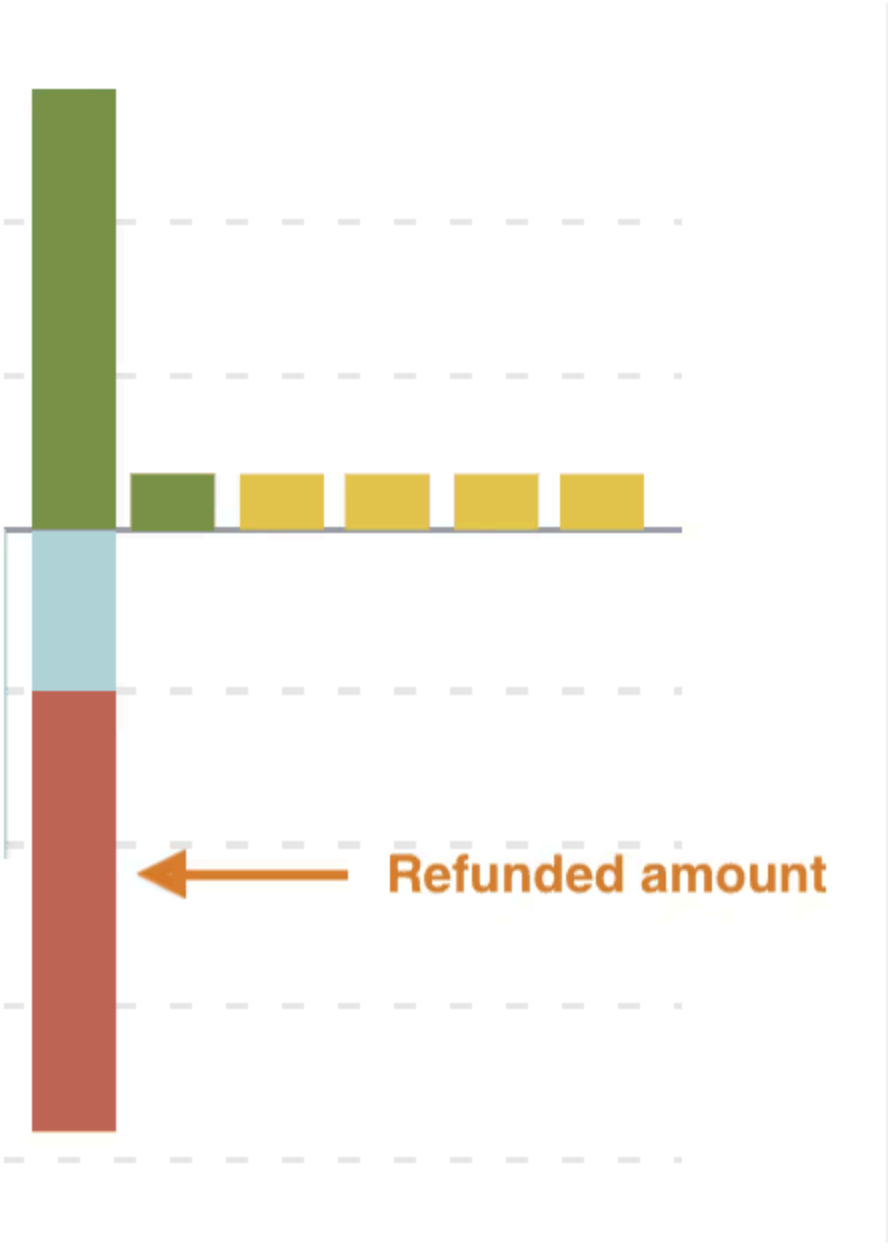
Here you can see an overview of the billing: past bills that went through, bills that were covered with the usage of credits, refunds, and future forecast:



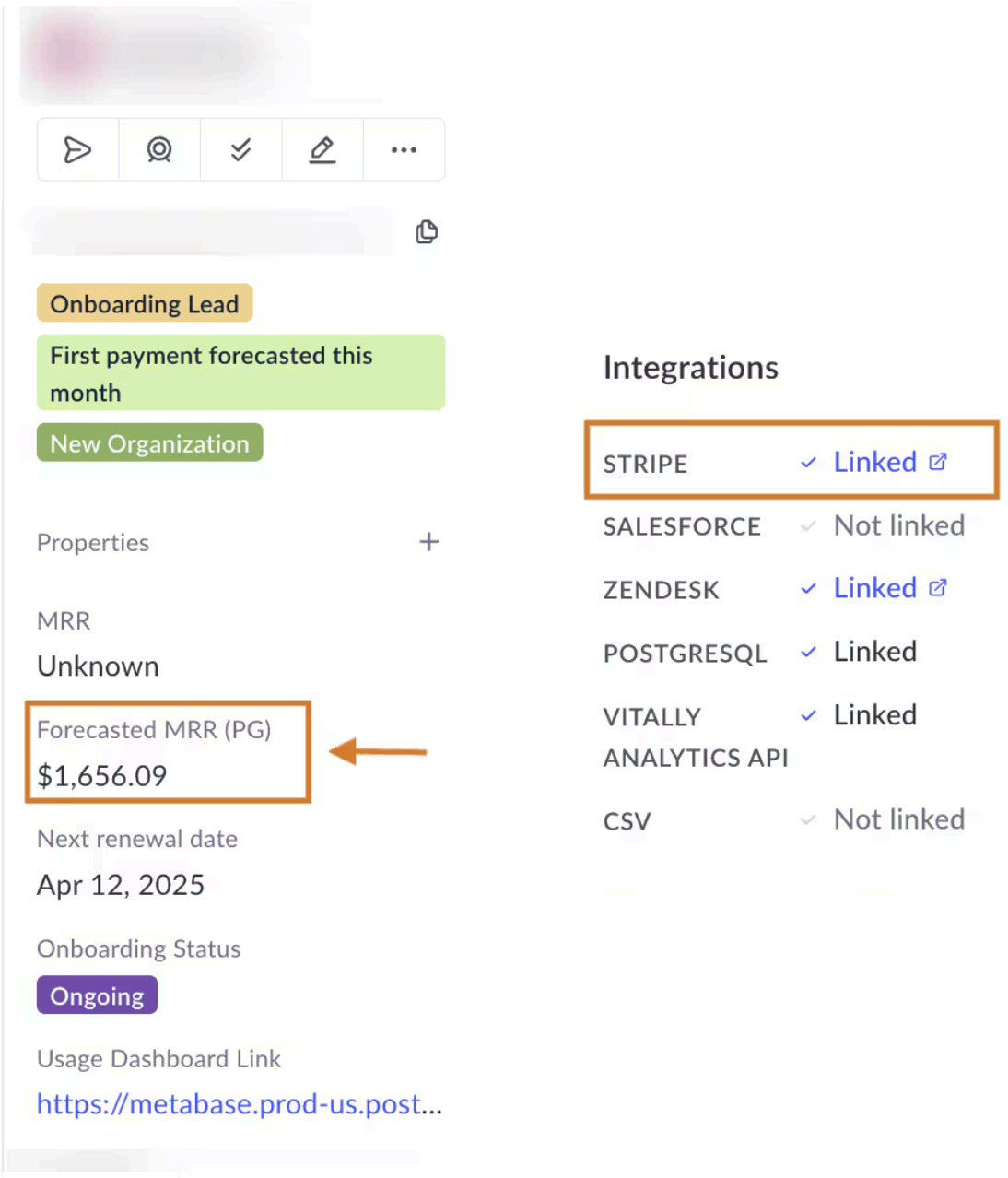


Looking at credits can be especially relevant for Startup or YC Plan customers (that usually use credits or might run out of credits and also display a forecasted MRR). If we ever issued one-off credits (e.g. for spikes), their usage will also be similarly displayed here.

Refunds are also visible with a red bar:

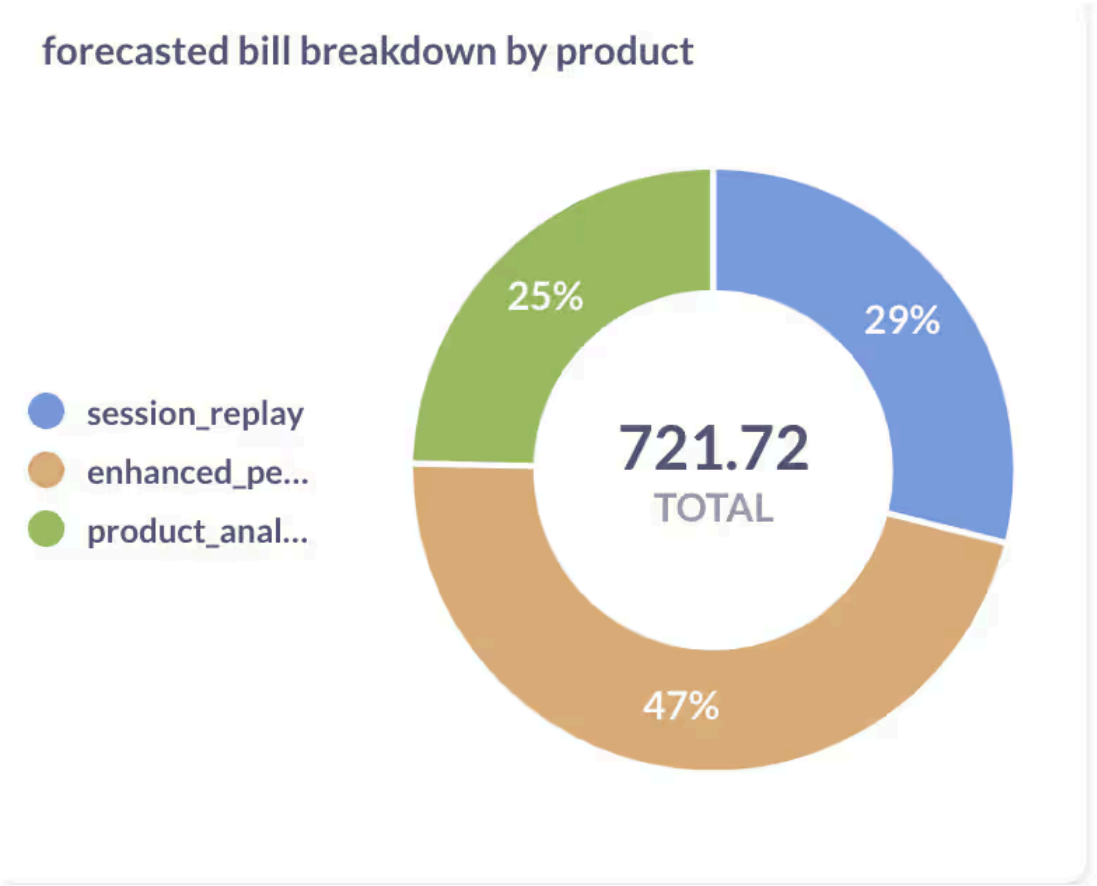


The forecast corresponds with the forecasted MRR that you can see in Vitally's sidebar, while if you're interested in how much has been incurred so far, it's something you can check directly in Stripe's invoice:



2. Forecasted bill breakdown by product (all projects)

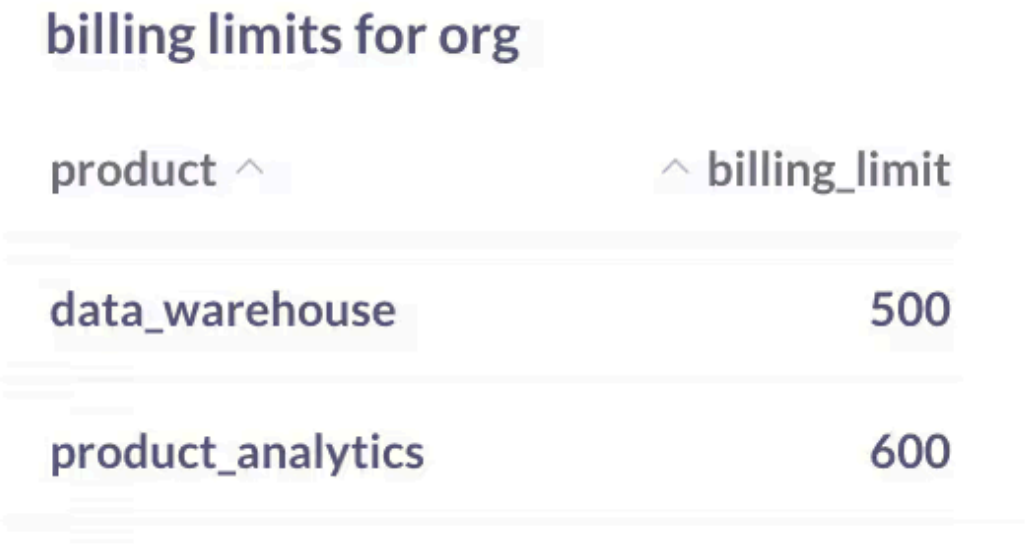
This is where you can quickly see what constitutes the majority of the bill and what can be a lever to reduce customers' spending. It's the best place to start the conversation around the value they take from PostHog.



Check the highest % to see if we can share some recommendations on how to reduce the spend, see if it's not caused by improper implementation, and pay attention to whether the user is not billed for the add-ons that they don't use in practice (e.g., Groups, data pipeline).

3. Billing limits

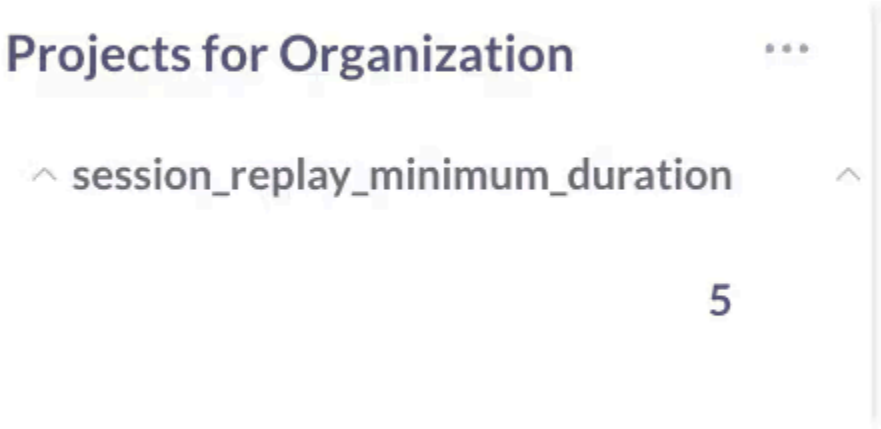
The default limit for the data warehouse is $500, and $150 for PostHog AI, so that's something you may see quite often. Seeing other billing limits added might be an indication that someone could benefit from a more long-term solution and a cost-cutting strategy, as once the limit is hit, the data is not ingested anymore and is lost forever. Billing limits are just a temporary patch, but not a solid solution.



4. Projects for the organization

This is where you can see whether any Session Replay controls have been implemented (minimum duration, sampling, feature flags). Any URL/event triggers won't be visible here.

If controls are missing but usage is high, recommend applying them before scaling replay usage. Most users should at least have minimum session duration enabled as most < 2 second recordings are not valuable but still racks up usage and billing.



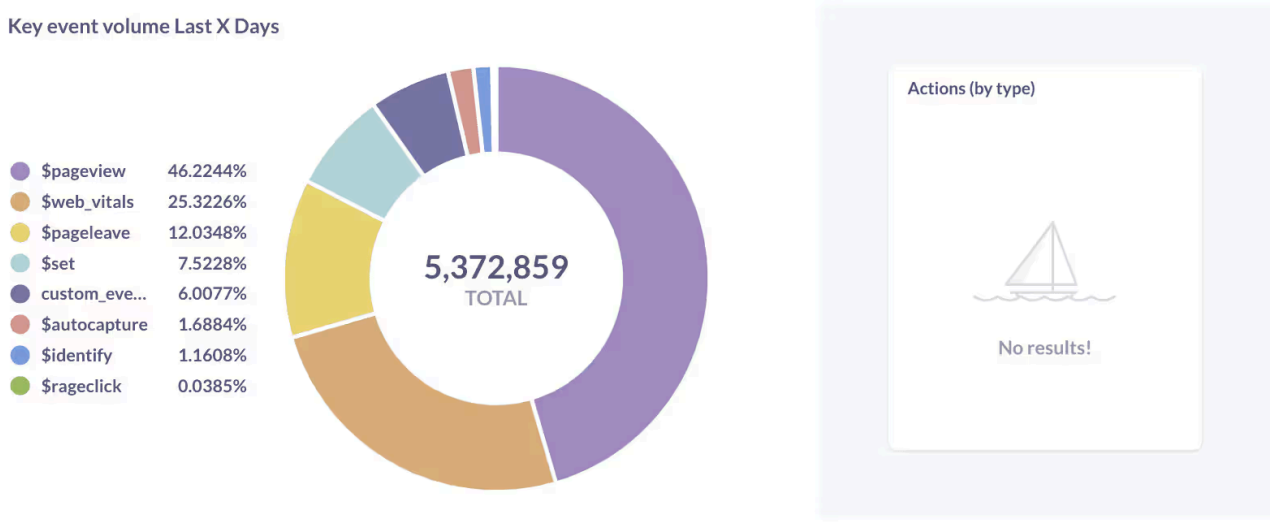
Session replay controls must be added for each project separately.

5. Org membership permission level

We use `all_admins_owners` to send our outreach emails on the onboarding team. You can copy all of them for your first email, and if the list gets too long, you can compare it with the list of active users in Vitally to see who might see your email. After a while, if you haven't heard back, next time you can experiment with emailing recently active users from Vitally as well.

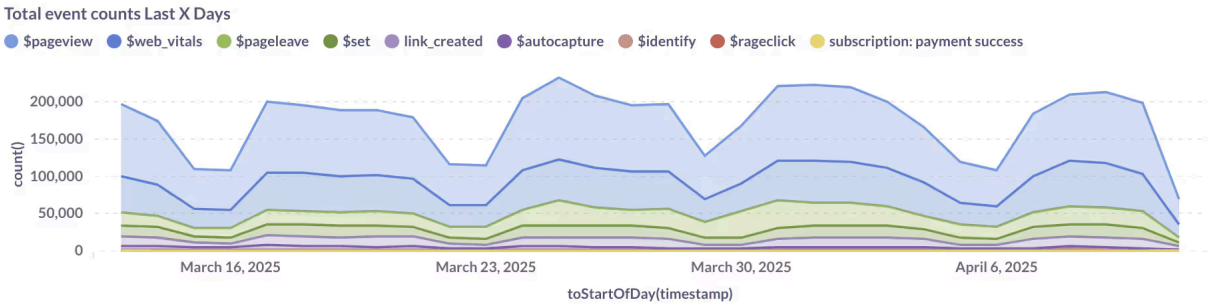
6. Key event volume (all projects)

The heart of our analysis. You can see the % of the most used event types. You can see whether they're using Autocapture, custom events, or when they have an unusual spike in $pageleave events. If you see a high ratio of autocapture events, but 0 Actions in the "Actions (by type)" graph, you can assume that they may not take enough value from it.



7. Total event counts (all projects)

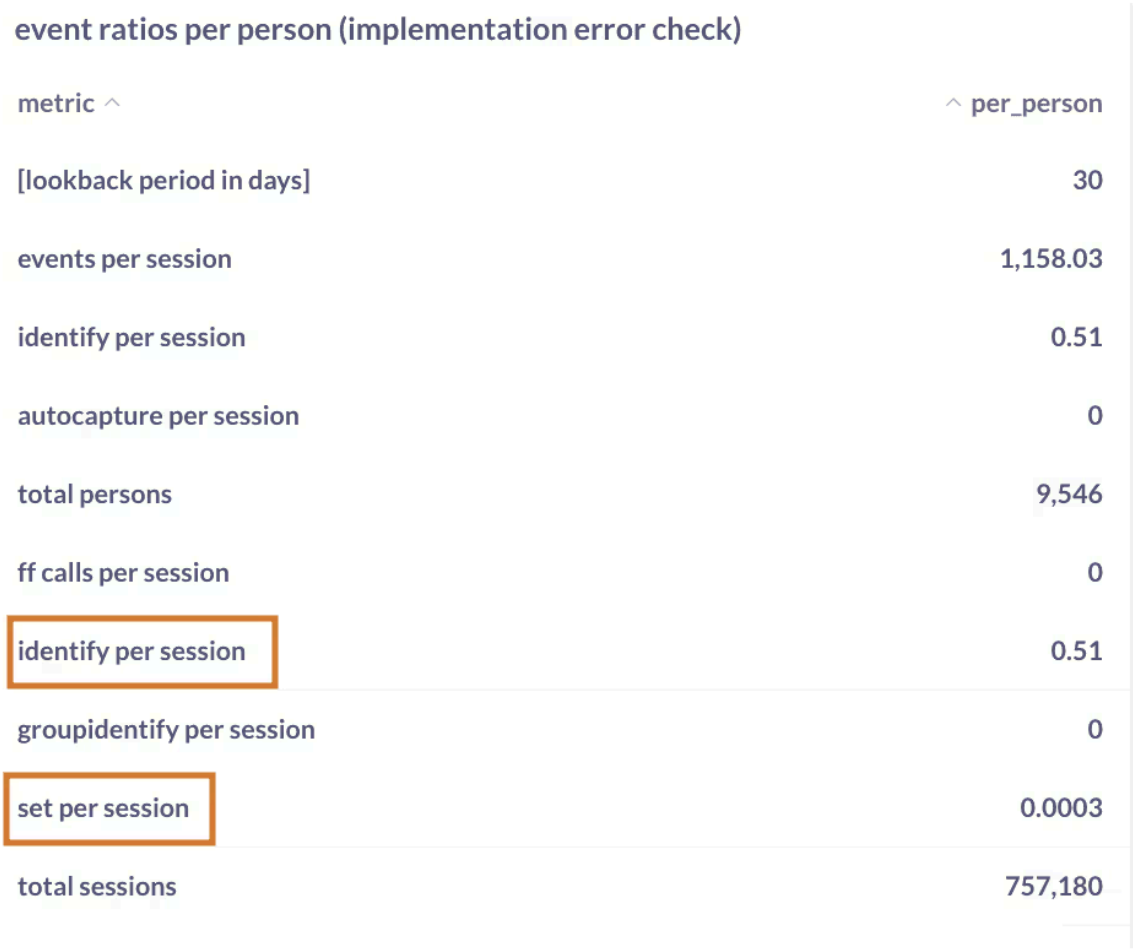
A supplementary chart to the "Key event volume". Both should be reviewed together. The area chart shows the most used events, and potential unexpected spikes. Events marked by the $ sign are the default PostHog events.



This graph corresponds with "Billable usage insight" insight within the "My PostHog billable usage" dashboard template. It's a good idea to show it to users, so that they can keep an eye on their usage and decide if everything they see there is needed for their tracking.

8. Event ratios per person or session (implementation error check)

If you see an unnatural spike in $set or $identify events in the two previous charts, here you can see whether their implementation is correct. Usually, 1-3 calls per session are alright, and things may get tricky if it's more than 4. If they are using group analytics, pay attention to the `groupidentify` calls per session as well. Feature flag calls per session can also help indicate further troubleshooting may be needed if it's too high:

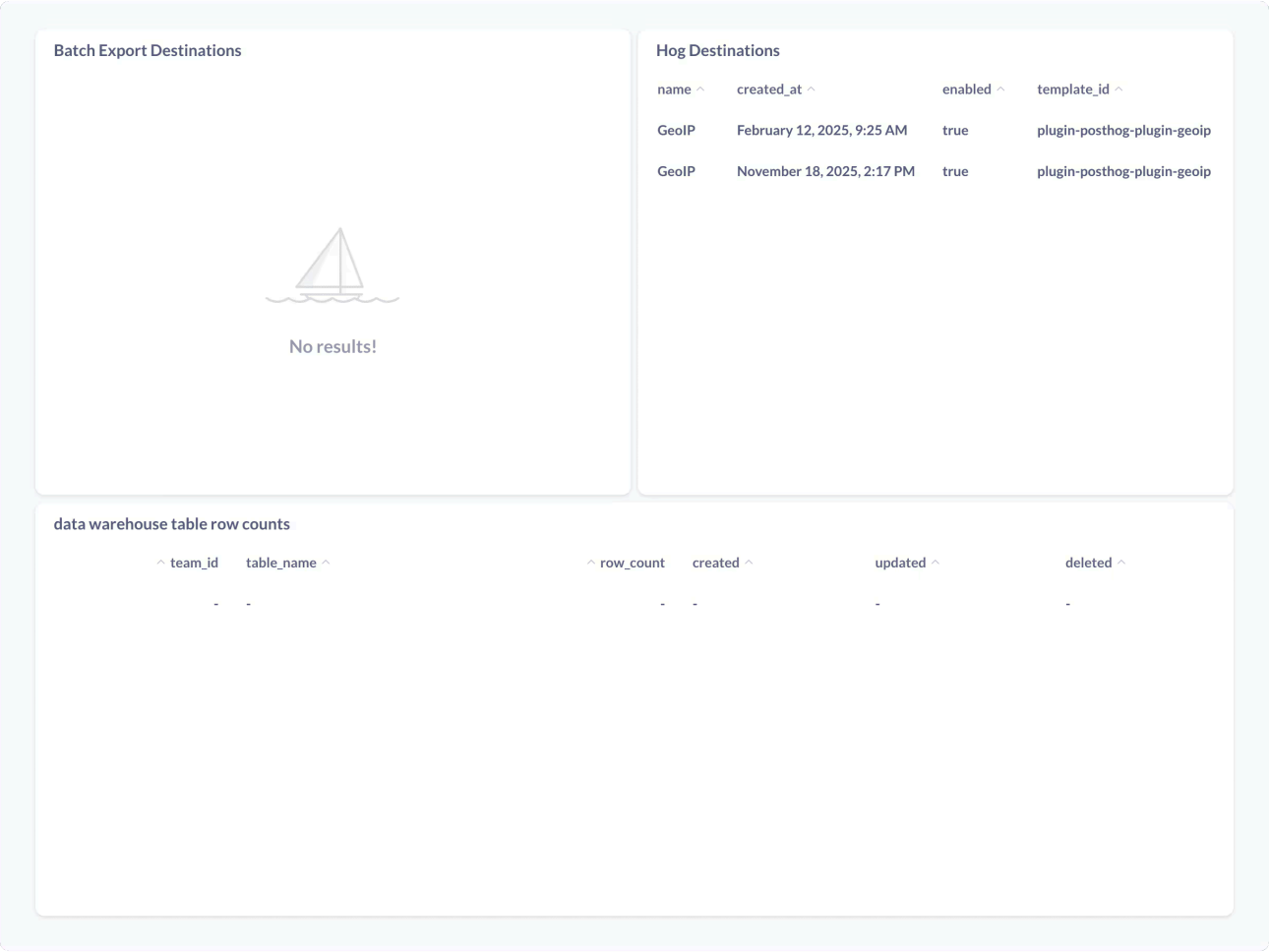


event ratios per person (implementation error check)

metric	per_person
[lookback period in days]	30
events per session	1,158.03
identify per session	0.51
autocapture per session	0
total persons	9,546
ff calls per session	0
identify per session	0.51
groupidentify per session	0
set per session	0.0003
total sessions	757,180

9. Hog Destinations

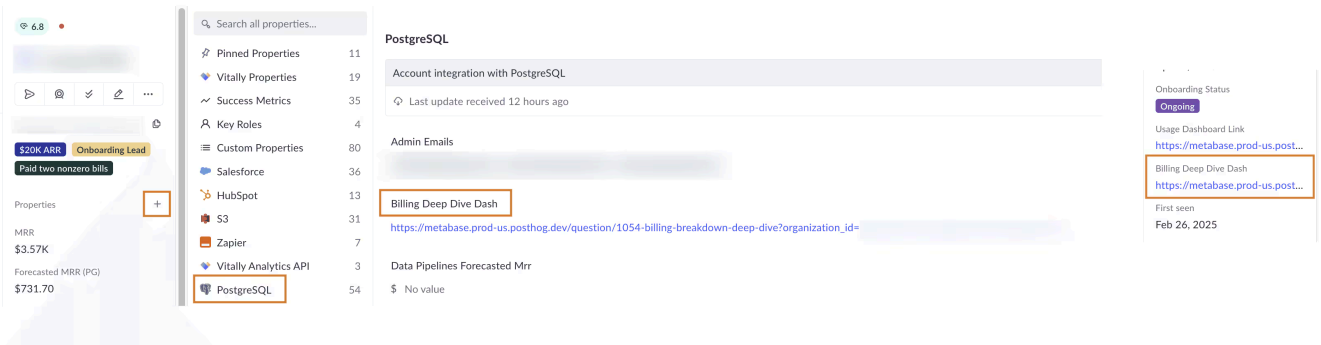
If they pay for data pipelines but have no active destinations, flag the mismatch and suggest enabling or removing the add-on. Newer accounts pay by usage (rather than the add-on), so keep an eye out for that as well.



You should also check whether they are using batch exports or data warehouse syncs.

Billing Deep Dive dashboard

The link is already available in our Daily view in Vitally, but make sure you also have it handy in the Account's sidebar as well. Go to Properties > PostgreSQL> Billing Deep Dive Dash, and click on a pin icon:

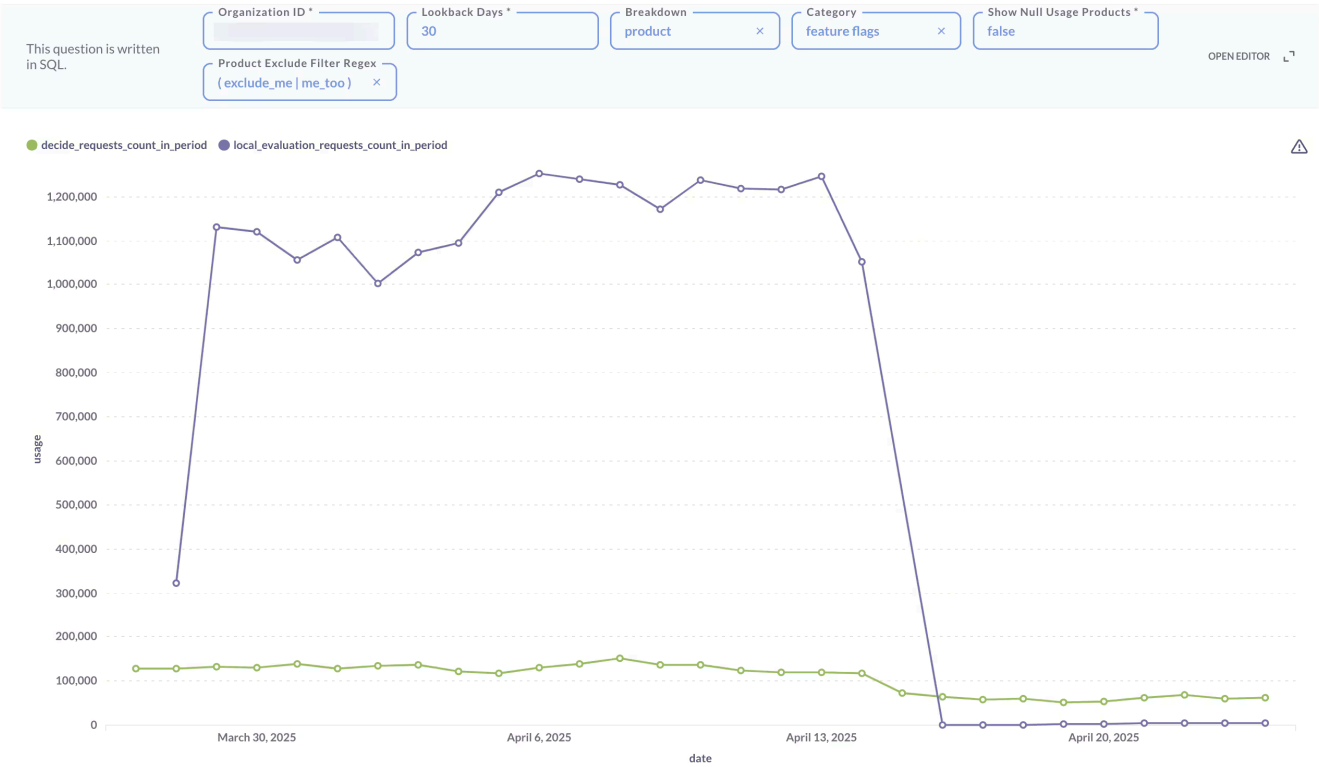


This dashboard is extremely helpful to dive deeper into the usage of Feature Flags and Session Replay, which is not that clear and easily accessible in the default Metabase dashboard. It gives you more insight into mobile vs. standard web reply, or decide vs. local evaluation requests for feature flags.

It's really handy when you want to investigate a spike in usage (e.g., due to an error in the implementation) and for how long it lasted. Some users struggling with their config may ask you about it specifically.

Pick the appropriate feature from the Category dropdown, update the filter, and adjust the period.

Here, for example, you can see an error in the implementation of the Feature Flags local evaluation, how it compares to Feature Flags in the front-end, and when the problem has been resolved:



The breakdown by "product x team_id" helps understand the usage per project (by project ID). It's a very popular feature request that the Billing team is working on so that it can also be accessible within PostHog.

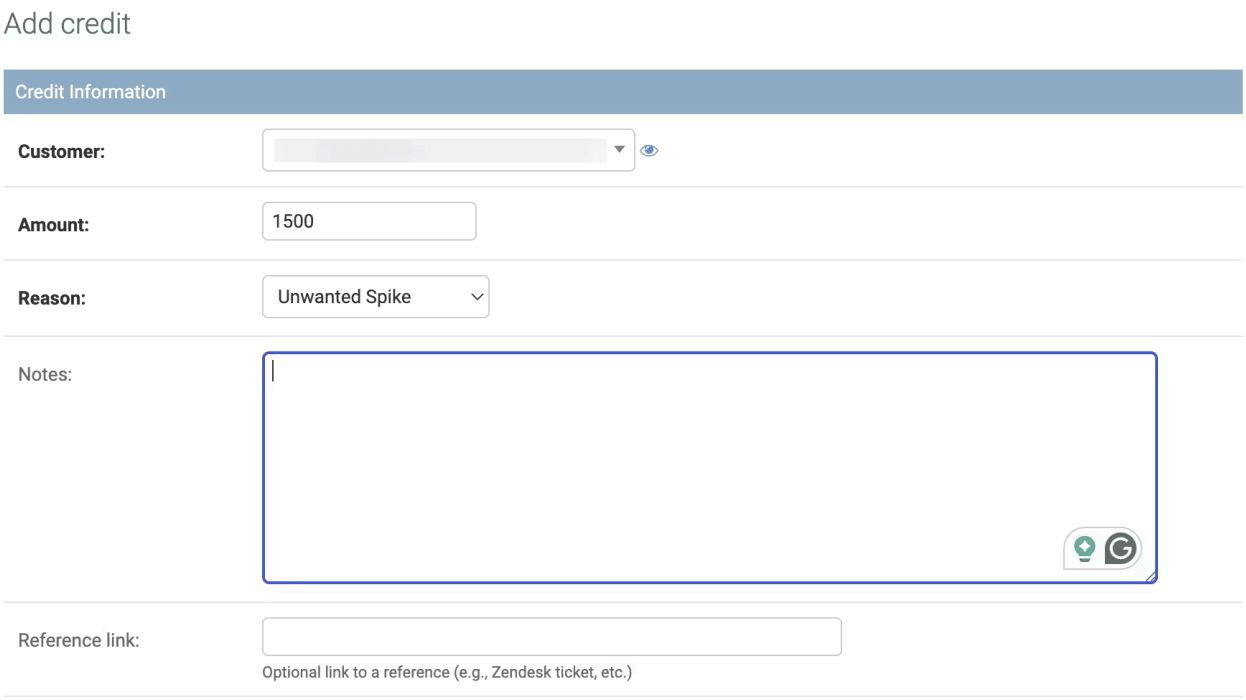
Operational billing actions

Re-authenticate the billing admin

Go to billing.posthog.com with your PostHog Gmail, then billing.posthog.com/admin should load again after login.

Adding credits

Watch this video: How to add credits (Loom)



In the billing admin portal, click “Add” next to Credits, search by Organization ID, set amount and reason, and leave an internal note.

Credits now fund the Stripe balance; legacy “credits expire” fields may still appear. Notify Billing if credits do not stick.

After adding the credits, return to Stripe to ensure the changes were applied correctly.

Rectifying a failed invoice

Watch this video Issuing a credit note (Loom)

Sometimes, you may notice that a customer has deliberately not paid their invoice due to an unexpected spike in product usage or because the bill exceeded their planned budget. In some cases, they haven’t reached out to us for help before the bill was renewed.

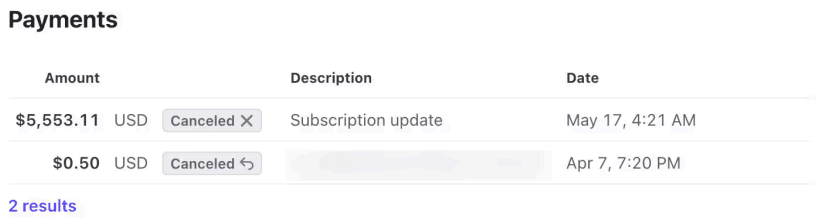
In these situations, we're here to help them sort out their usage and billing. However, it's no longer possible to simply add credits—we now need to adjust the already-issued invoice directly in Stripe.

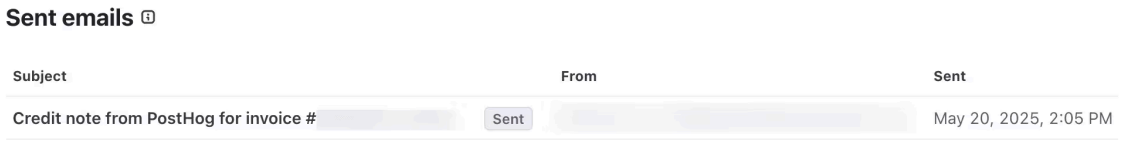
This can be done using credit notes, which allow you to either compensate the full amount or offer a pro-rated relief. Please ensure you have the appropriate Stripe permissions; otherwise, you may not be able to access this option:

- Access the invoice in Stripe.
- Look for the “Issue a credit note” option.

- Select a reason and add an internal note explaining the context.
- Check the items to credit. If you're compensating the full amount, all items should be selected. If you're addressing a specific issue—such as an unexpected spike in usage—select only the relevant line items to discount that specific amount.

After saving the changes, you should see on the main invoice page that the invoice is marked as "Canceled", and that the credit note has been sent to the user.





As a best practice, always engage with the user first to understand their specific situation. This allows them to confirm whether an unexpected event occurred on their end, rather than us proactively rectifying failed invoices. The reason for this is that the customer may churn, and in that case, the invoice might be considered uncollectible rather than refunded.

Lastly, always add a note in Stripe to explain why credits are being issued.

New hire onboarding

contents/handbook/onboarding/new-hire-onboarding.md

Your first few weeks

Welcome to the PostHog's Onboarding team! We only hire about 1 in 400 applicants, so you've done well to make it here! Unlike a lot of companies, we don't have a super-long onboarding process and would prefer you to be up and running with your customer base as quickly as possible.

Here are the things you should focus on in your first few weeks at PostHog to help you achieve that.

Ramping up is mostly self-serve - we won't sit you down in a room for training for 2 weeks. If you're not sure who is supposed to make something below happen, the person responsible is almost certainly you!

Also look at the sales team's onboarding page for guidance on what *not* to do when you start. In general, there's a lot of good resources within sales to reference (as we were previously one team!)

Day 1

- Familiarize yourself with how we work at PostHog.
- Meet with Magda, who will run through this plan and answer any questions you may have. In addition, come equipped to talk about any nuances around how you prefer to work (e.g., schedules, family time, etc.).
- Setup relevant tools and check out tools specific for the Onboarding team.
    - Integrate Gmail with Salesforce and Vitally to enable centralized communication history.
- If you start on a Monday, join your first PostHog All Hands (at 4.30 pm UK/8.30 am PT) and be prepared to have a strong opinion on whether pineapple belongs on pizza.
- If you start on a Monday, join your first Onboarding standup.
    - We fill in a GitHub issue every week before this meeting, so we are prepared for the discussion topics. Magda will add your GitHub handle to the template.
    - Book time with Magda to go over the nuts and bolts of the role - which leads get onboarding, what signals we're looking for, and how to reach out.

Rest of week 1

- Confirm that you have been added as a member to the PostHog organization in GitHub. Fraser can add you if you haven't.
- Work your way through your GitHub onboarding issue that a member of the should have created and sent a link to.
- Ask team members in your region to be invited to some customer calls so you can gain an understanding of how we work with customers.

- Check out some BuildBetter and Gong calls and add yourself to a bunch of Slack channels - get immersed in what our customers are saying.
    - There are a few BuildBetter playlists to start with – customer training calls, PostHog knowledge calls, onboarding specialist calls, add to them as you listen!
- Learn and practice a demo of PostHog.
    - For familiarization and self-led training, follow the product curriculum. You work through this with the HogFlix Demo 3000 project, which is already populated with data. Alternatively, you can create a new project in either the US or EU PostHog instances and hook it up to your own app or HogFlix instance.
- Read all of the Onboarding section in the Handbook as well as the Sales and Customer Success section, and update it as you learn more.
- Meet with Charles, the exec responsible for customer-facing teams, and schedule monthly 1:1s with Simon.

Week 2

- Shadow more live calls and listen to more BuildBetter recordings.
- Explore Vitally and Metabase – take note of any questions you have to go through during in-person onboarding.
- Try running through the onboarding exercise that Kaya designed to test your skills for working with customer accounts.
- Towards the end of the week, schedule a demo and feedback session with Magda. We might need to do a couple of iterations over the next few weeks as you take on board feedback, don't worry if that's the case!
- Get comfortable with the PostHog Docs around our main products.
- Book time with Magda for a deep dive on PostHog billing, sales process/routing. This will help you understand how your role fits in the broader context of the customer-facing teams.
- We'll start routing new leads to you at the end of week 2. Start to review these and reach out, using a shared booking link with someone else from your region so they can back you up in the first few weeks. This is a great option to practice and fail.

In-person onboarding

Ideally, this will happen in Week 2-3, with the company of a few colleagues (depending on where we do it and who's around). It will be 3-4 days covering (among others):

- Demo practice session with the team.
- The data we track on customers in PostHog and some hands-on exercises to get you comfortable using PostHog itself.
- Deep dive on Vitally and Metabase.
- Toolkit and internal processes.
- No stupid questions session.

Detailed training plan available in the new hire onboarding checklist. This is a checklist for the Manager, you don't have to read it beforehand.

Weeks 3-4

- You're already reaching out to our customer base.
- Focus on taking more and more ownership on calls so that team members are just there as a safety net.
- Continue to meet with customers - very quickly, you should be doing these solo. The customers you are working with will mostly just be getting started, so you'll see a lot of very familiar patterns emerge.
- Make sure all your tooling is fully set up.
- Set up a call with Daniel to get a "soft intro" to Vitally playbooks, segments, and our internal metrics. It's not a deep-dive - just getting familiar.
- Keep working on your product knowledge. You can find a couple of exercises here.

How do I know if I'm on track?

By the end of month 1:

- You continue learning from the received feedback.
- You have a good grasp of our internal processes and workflows.
- You're the driver, you take initiative, prioritize your time well, and work independently.
- You lead customer calls on your own.
- You should consistently craft a minimum of 10 outreach messages per day, without compromising on quality, style, or accuracy.
- You don't let accounts fall through the cracks before their renewal.
- You pass well-qualified opportunities to Sales via the `Onboarding referral` segment.

By the end of month 2:

- You're a PostHog power user - most questions you raise can only be answered by product engineers rather than the support team.
- You should show steady growth in both call volume and outreach activity, with a minimum of 15 outreach emails sent per day.
- You fully participate in team discussions and contribute to team projects.

By the end of month 3:

- You've implemented process and system-level changes to make your job better/more effective.
- You raise the bar and surface ideas contributing to the growth of the team.
- Customers are happy after interacting with you, and you meaningfully contribute to their success.

General expectations

Our customers are always central to our work. There's time and space to work on fun projects, but that should never happen at the expense of our customers. Below are some non-exhaustive tips to help you stay on track during and after the probation period.

Core responsibilities

- Make sure you're on top of our customer base in Vitally. Don't let renewals fall through the cracks.
- It's not a race - quality over quantity, always. Spend time reviewing accounts, be accurate, and genuinely helpful in what you email to customers.
- Listen to customers and respond to their needs. It's a conversation, not a monologue with a set agenda.
- Make sure your Calendly schedule stays available for customers. You're the master of your own calendar, and we trust you, but be reasonable.
- Do the Sales handoff where appropriate and always provide context on the customer.
- Maintain the hygiene - follow the process in Vitally, add notes, or tasks when necessary. Remember that we share the workspace, so keep the order.
- You're the driver, so take initiative. Keep in mind improvements, optimization, process, or content updates as you go. Don't be afraid to voice your ideas.

Communication and ownership

- Make sure you read the Handbook page on Communication. Always answer pings, even if it's just to say "I'll look into it later". Emoji reactions can be helpful to indicate that you acknowledged or completed the task.
- Read the Handbook page on Feedback) Share and ask for feedback, take it with grace. Remember that it should be kind and factual, not based on assumptions.
- Publish your Sprint planning update on time. It should be done on Monday at the latest, so that others have time to read it before the Sprint planning call.
- Be yourself and stay human. Don't send AI-generated emails to customers.
- Set an autoresponder in your inbox whenever you take a longer time off, and direct your customers to `onboarding@posthog.com` so that we can pick up your conversations.
- Share your knowledge! Share what you learnt, or found out - the whole team will benefit from it.
- Don't be late for calls. It's rude, the same as eating on them.
- Use the Time Off tool to communicate your absence in advance.
- Keep your promises. If you promised to follow up with a customer or colleague, or work on a project, do it.
- Stick to deadlines.

Staying up to date

Everything at PostHog changes really fast. That's how to keep up as a start:

- Follow #tell-posthog-anything and #demo-posthog-anything daily for general announcements.

- Follow #cs-sales-support channel daily - it's crucial for all folks in customer-facing roles.
- Stay on top of the #incidents channel to know what's going on and be able to inform customers.
- From time to time, it's worth taking a look at #sales, #customer-success, #team-new-business-sales, #team-product-led-sales to get inspired by new ideas that other customer-facing folks implement, and what's going on in general
- #changelog channel to stay up to date with newly released products and features.
- #today-i-learnt set up by Daniel, great for sharing and learning new things.
- Attend All-hands calls, and if you skip them for any reason, catch up with the recording. It lets you stay on top of the new features we release and our direction.
- Set a Slack matcher for specific keywords (e.g, "onboarding referral" if you want to see leads we pass to Sales).
- Slack also has a cool AI Recap feature - it might be helpful.
- Don't let your product knowledge get stale - check docs, use the product.

Onboarding conversations playbook

contents/handbook/onboarding/onboarding-conversations-playbook.md

Our customers are busy, self-serve by default, and allergic to anything that feels like a time sink. We deliver the most value when we can talk directly, so it's worth being intentional and trying creative ways to earn that conversation.

That said, we've repeatedly seen customers implement our recommendations even when they never reply. That's why we don't gate value behind a meeting - we provide it regardless.

Check out the Getting people to talk to you page in the Sales Handbook and our learnings below. As you experiment, add more and share what worked!

Our guiding principles

- **Stay human**. Be yourself, stay casual, open, and friendly. Aim for "talking to a friend," not a script.
- **Be genuinely helpful**. Reduce complexity. Offer simple next steps that save the customer time and effort.
- **Be prescriptive**. Don't just explain options - recommend the best path for this customer, and say why. You're the expert. Here's a good example.
- **Be generous**. If a refund or credit is clearly the right call, make it happen. Use your good judgment.

Outreach

Your first message is your best chance to earn attention. It should feel like practical help from a real person - not a pitch. Lead with a specific observation, a clear benefit, and an easy next step.

Captivating subject lines

Avoid generic subjects ("Checking in", "Following up"). Instead, experiment with short, specific lines and anchor them to a specific outcome.

Use the following product signals:

- **Billing / pricing signal** - e.g. first bill coming up, increased number of billing page visits
    - "Your first PostHog bill is coming up - quick way to save $"
    - "PostHog bill coming up - quick way to reduce it"
    - "Noticed a spike in costs - 2 ways to bring it down"
    - "Cost check: a small tweak that can reduce event volume"

- **Docs signal** - increased visits to the docs pages
    - “Noticed docs activity - need help with [topic]?”
- **Event spike / instrumentation signal**
    - “Too many events? Let’s fix it.”
    - “I think you’re tracking more than you need”
    - “Event spike yesterday - need help figuring it out?”
- **General value offer (audit / review)**
    - “Quick data audit from the Onboarding team”
    - “Tracking review: 3 improvements I’d make”
    - “Want me to sanity-check your {events / funnels / flags}?”
    - “Data audit: 3 tracking gaps I’d fix first”
    - “I recorded a 2-min walkthrough for your setup”
    - “A recommended dashboard for [their use case]”
    - “Worth a look before your next release” - “Are you trying to do [goal]? (I can help)”

Content

Keep it short. Don’t overwhelm the reader. It’s tempting to include every tip and best practice, but concise emails get read and replied to. Share the headline observation and the next step; save the deep dive for the call (or a follow-up).

Set expectations early. If you want consistent engagement throughout onboarding, be explicit about what the program includes and why it’s worth their time. When customers know what to expect and how to use our time, they’re more likely to participate. Setting clear boundaries also helps - what you can help with, and for how long we’re around.

Use prior context to be proactive. Before you hit send, take a minute to scan prior threads. If a customer spoke with Sales during an evaluation, check what came up and reference it (e.g., “I saw you covered X with [Name]”) so your email feels connected. And look for other loose ends too, e.g., an old support ticket, or a question from months ago. Following up with a real solution feels personal, and proactive delight gets noticed.

Checking in

This is where we can have a real impact on product adoption and usage expansion. Think of it as a value-driven "soft cross-sell".

Don’t just repeat yourself. Avoid rehashing the same observations from your first message. If your earlier advice still hasn’t been implemented, send a small, friendly nudge. Otherwise, bring something new:

- Look at what they’re actively using right now.
- Infer what they might be trying to measure or achieve as a business.

Mainly, help them get to an “aha” moment, and/or suggest one or two features they’d benefit from, but may not have discovered or had time to try. PostHog features become more powerful when used together (e.g., funnels/error tracking + session replay + PostHog AI). Share a specific guide, an example, or a Loom video, so the customer doesn’t have to poke around to figure it out. You can take some inspiration from Use Case Selling handbook pages.

Lastly, if the customer is trending toward growth (usage, team expansion, increasing volume), it’s okay to mention pre-paid credits and the option of dedicated human support early. Framing it as “when you’re ready” gives them time to consider it and makes a future Sales handoff smoother.

No response?

Review the list of users on the account: who’s active in PostHog, what roles they have, and who is most likely to own outcomes (implementation, analytics, product, engineering) vs. commercial topics (billing/procurement). Choose a small set of the most relevant people (3-4 total) and avoid repeatedly emailing everyone.

Tailor the email to their likely concerns:

- Engineers: how to implement/reduce noise
- PMs/analytics: insights, funnels, retention, experiments
- Finance/procurement: cost control

A small, human touch can help here! Use what’s publicly obvious or clearly relevant (their product category, their website messaging, their goals). If you genuinely relate (e.g., you’re learning a language and they build a language app), one sentence can be enough to build rapport. That’s also a great tip for the first outreach.

Preparing for the call

Start from a health check

Use Vitally and Metabase to understand the customer’s current setup. For easier access, you can pin the "Engagement Metric Dashboard" custom trait in Vitally, where you can take a closer look at power users in the organization, the usage of AI or error tracking, and more.

You can supplement Metabase analysis with the HogSpy extension to audit the implementation of identify, flags, and experiments.

Then zoom out to learn about their business, their product, and the rest of their stack. The better your context, the faster you’ll get to relevant recommendations.

Lead with their KPIs

Use the customer's KPIs (usually captured in the booking form) to drive your prep. Ask yourself: what would "success" look like for them? Come prepared with 2-3 concrete use cases tied to those KPIs (e.g., a specific insight type, dashboard, funnel, experiment, etc.). This Handbook page can be a good source of inspiration.

Map the stack and spot opportunities

Check Wappalyzer (login details in 1Password). It's not always perfectly accurate, but it's usually good enough to understand the tools they rely on. Use it to identify integrations, suggest Sources/Destinations where it makes sense (e.g., HubSpot),

It might be a great moment to position PostHog as the place where multiple tools can connect under one hood.

Customers respond well when we're proactive, especially when we show them a path they hadn't considered. PostHog is most powerful when features compound, so part of prep is identifying the next adoption step that unlocks more value. You can take some inspiration from Use Case Selling handbook pages as well.

Use AI to broaden your angles

AI can help you sanity-check assumptions and surface ideas you might miss. Customer-facing teams at PostHog use PostHog AI, Claude (with PostHog + Vitally MCPs), Cursor, or Antigravity. Use it to generate questions, identify likely "aha" moments, and draft call checklists, then apply human judgment to keep it relevant.

You can also run PostHog AI on the customer instance (visible only to us, no cost incurred) to do the account audit. Prompt below.

▶ PostHog AI prompt

On the call

- **Start with a quick discovery (3–5 minutes)**. What they shared in the booking form may not reflect today's priorities or the goals of everyone on the call. Confirm what outcome they want by the end of the session.
- **Have the relevant docs ready**. If you can anticipate the topic of the session, keep the key docs open so you can screen-share them quickly.
- **Show, don't tell. Build things live**. If you discuss funnels, dashboards, cohorts, or flags, create one. Save it so the customer can revisit it later. **-Connect features**. Show how features compound and check this Handbook page for inspiration:
    - Funnels → drop-off → jump into Session Replay to understand it better and create a cohort
    - Error tracking → watch related replays
    - Experiments → measurable impact → rolling out the winning variant

- **If you don't know something, don't guess**. Open the docs or use PostHog AI during the call. It builds trust and teaches them how to self-serve.
- **Check the event schema (if relevant)**. If their KPIs require certain milestones, verify they're capturing the right events/properties. E.g.:
  - Walk through their signup/purchase flow and compare it to events captured.
  - Use PostHog AI to watch Session Replays and suggest missing milestone events.
- **Spot unused events**. Show what's used vs. unused and where volume can be reduced. This is an easy way to explain optimization opportunities and cost control:
  - Activity → Event counts → last 30 days
  - Open an event → check if it's used in any saved insights/queries
- **Introduce our beta features** (if relevant). Encourage customers to use them and share feedback. It can positively impact adoption before the feature becomes a paid product.
- **If growth signals are strong, plant the seed early**. If the account is on a positive trajectory, introduce the idea of prepaid credits coming with a discount and the option of a dedicated PostHog human.

Email Follow-up

- Send it the same day. Use the momentum!
- Include the public Gong recording link.
- Loop in everybody. If some folks couldn't attend, include them anyway so they can catch up async.
- Summarize the call and send resources. Include some extra resources if you feel it would be beneficial as well. For example, our YouTube playlist is great!
- If relevant, give them one quick win. Encourage a small task they can do immediately after the call to lock in value and reinforce learning.
- Share any feedback or feature requests with the relevant product team. Their responsiveness can help you deliver some customer happiness! It's always great to be able to send a GitHub link to follow in your email.

Onboarding Data

contents/handbook/onboarding/onboarding-data.md

Data architecture overview

Data used by Onboarding Specialists comes from three main sources:

Billing Postgres *admin panel view*

- Customer account, subscription, and invoice type data
- Usage reports and consumption metrics
- Revenue amortization calculations
- Billing forecast / spike calculations

Production Postgres *admin panel view (US)*

- Organizations and projects configuration
- User accounts and permissions
- Product settings and feature flags
- Warehouse tables, pipeline source/destination info

ClickHouse

- Event and person data for all teams (projects)
- person to distinct_id mappings

Query capabilities

- **Metabase** queries production databases directly but cannot combine Postgres and ClickHouse in a single query
- **PostHog analytics** limited to Team 2 data, but can query across databases in a single query
- **Cross-organization analysis** requires Metabase for customer event analysis, including:
    - Library usage breakdowns
    - Event volume metrics
    - Implementation diagnostics

Vitally integration

We sync customer data between Vitally and PostHog bi-directionally

Data sync pipeline

To Vitally:

- Custom traits sync nightly from billing Postgres via SQL queries in Vitally
- Product engagement events sent through data pipelines using this action
- Billing spike detection from `billing_spike` table, defined in this PostHog function

From Vitally:

- All Vitally traits accessible as `traits.\`vitally.custom.traitNameFromVitally\`` in PostHog queries, eg see the `onboarding_accounts_timestamp_check` view)
- JSON storage format (requires cleaning for arrays/complex fields)
- Data syncs via data warehouse connection

Known limitations

- Conversations table lacks organization/user mapping
    - Messages table implementation status unclear

Onboarding pipeline tracking

Pipeline stages

We track customers through defined onboarding stages with automated timestamp capture:

1. **Onboarding segment entry** - Customer enters onboarding criteria
2. **Outreach sent** - Initial contact via email (manual update)
3. **Customer engagement** - Response received (manual update)
4. **Nurture phase** - Post-intro call follow-up (manual update)
5. **Completion/churn** - Final outcome tracking

Each stage transition is managed through Vitally playbooks with automatic timestamp updates.

Key data tables

For onboarding analysis, these tables provide essential data:

Table	Purpose	Key fields
`invoice_with_annual`	Billing data with revenue amortization	Revenue (mrr), billing period, type (annual, completed, upcoming, etc)
`vitally_accounts`	Customer properties and traits	All Vitally custom traits, health scores, usage

Table	Purpose	Key fields
`posthog_organization`	Org-level configurations	Settings, feature access, creation date
`posthog_project`	Project/team settings	Project configuration, team members
`billing_spike`	Usage anomaly detection	Spike timestamps, magnitude, affected metrics

Onboarding program

contents/handbook/onboarding/onboarding-program.md

Getting started with any new tool can be overwhelming, and PostHog is no exception. We want to make sure you're configured correctly, using the right features for your needs, and seeing real value.

That's why we offer personalized onboarding. Whether you need help with initial setup, want to optimize your billing, or are looking to align PostHog with your business goals, we're here to help.

What to expect

Our onboarding program spans **8 weeks** and includes:

- **Account review and optimization tips** - We'll review your current setup and share recommendations to help you get the most value while minimizing costs.
- **First call (30 minutes)** - Let's get hands-on! We'll walk through optimization practices together and answer any technical questions about integrating PostHog into your stack.
- **Second call (30 minutes, optional but encouraged)** - Ready to go deeper? This follow-up call focuses on using PostHog to achieve your specific business goals and KPIs. Bring your team — the more, the merrier!
- **Final check-in** - We'll do one last review to make sure everything is working smoothly, share additional resources, and point you to ongoing support options.

Timeline

Here's a typical timeline, though we're flexible and can adjust to your schedule:

- **Week 1**: Initial account review and outreach
- **Weeks 2-3**: Ideally, a first call scheduled, or we can continue working async!
- **Week 5**: We check in to make sure you're all set before your PostHog bill comes up
- **Weeks 4-6**: Second call (if requested), and space for unanswered questions
- **Week 8**: Onboarding graduation!

Your success is our success

Here's the thing: most teams struggle with knowing *what* to track, not just *how* to track it. The first call gets you set up correctly. The second call is where the magic happens. We'll help you decide on the metrics that actually drive your business decisions and create a roadmap for using PostHog strategically.

This is where customers see the biggest ROI, and it's completely free. We highly encourage you to take advantage of it.

Onboarding team

contents/handbook/onboarding/onboarding-team.md

How we work

First and foremost, we're account-agnostic, which makes us different from other GTM teams. This means that we don't have our book of customers, and our focus is on being fast, responsive, and available to a huge number of customers. This is precisely why we use, e.g., a Team Link for customers to book the call - they can choose a person closest to their time zone, and both the experience and value provided remain the same across the team.

Day-to-day, we collaborate closely with Account Executives and Account Managers, especially when a customer would benefit from a dedicated PostHog human, and the Support team on solving issues.

Onboarding sessions are a mine of information about our users and their needs, which makes us a fantastic liaison for the Product teams. We share product feedback whenever it surfaces.

Since the Onboarding team is still a relatively new addition to a wider GTM team, we're a highly collaborative and creative bunch who are not afraid to try new ideas, iterate, and build the foundation for the future Onboarding endeavors.

What does this team do?

The core job of an Onboarding Specialist (OS) is to ensure a successful start of the user journey with PostHog. That means making sure that our customers get the most value out of using PostHog, they are aware of best practices, their setup is solid, and they don't pay for something they don't need. Ultimately, we serve as the customer's sparring partner in achieving their goals, so we need to understand their needs, their business, and where they're coming from.

The north star metric for the Onboarding team is 3-month logo retention at 90% from the first $100+ forecasted bill, which can be tracked in the onboarding team retention dashboard.

We also care about net dollar retention for this segment, but we treat it as an auxiliary metric.

Which customers get onboarding?

The segment consists of customers who self-serve PostHog and generate a forecasted bill of over $500. In practice, because billing is metered and in arrears, and we don't know what people will pay when they sign up (or when they first exceed a $100 forecast), so *most* accounts > $500 forecast are routed to us. We also handle a couple of other segments:

- YC program participants at the roll-off of the plan.
- Startup customers rolling off, who have generated a first bill in the 500–1500 range.
- Startup plan customers with high credit usage (> ~$1500).

- Hype startups we want to work with (despite being below $ thresholds), or longer-standing customers that have paid in this range and need billing or setup assistance.

Which customers are out of scope

Since we primarily focus on customers who've signed up and have a forecasted bill, in most circumstances, we're not the right choice to talk to customers who've:

- Not signed up/generated a bill, but have contacted sales.
- Are early-stage startups on the startup plan with no billing/low credit usage (<$500/mo).
- Customers who paid over 3 bills

Merch store consultation

Customers who normally fall outside our scope still have a chance to get help! They can buy an Onboarding consultation via our merch store.

After making the purchase, the customer gets a link to book a meeting, and they can contact our Billing team if they can't find an appropriate time slot. The billing team handles issuing credits/refunds accordingly. However, since it's a paid service, we should prioritize these and try to make space in our calendars, if possible.

A few things to keep in mind:

- The booking link has no expiry, so there's no need to follow up with the customer if they haven't booked the call instantly - they can do so at the most convenient time.
- If someone did book the meeting but didn't show up at a consultation, issue them credits for the missed meeting, and follow up with the customer to offer a meeting at another time.
- If they had a call with us and then need more help, we'll offset the onboarding call cost against a professional services package.
- If the customer has a CSM, AE, or AM assigned, their dedicated PostHog human should run the call.
- If the customer already belongs to the Onboarding bucket, prioritize meeting with them, and add credits to their account, as they would get the same service anyway.

Internally, when someone purchases a call, we get notified in our Slack channel. Check who completed the purchase, look them up in Vitally for more context, and check whether they booked a call. Change the status in Vitally to `Paid Call purchased` for tracking purposes, and add a note if needed.

Tooling

Check out the list of shared tools.

The team-specific tools for this team are:

- Onboarding hub in Vitally, and main view with Onboarding accounts.
- Shared Calendly link - make sure to add buffer to your schedule to avoid having calls back-to-back.

- Github project board.
- Onboarding Google Drive with all relevant documents.
- Alfred workflows.

How to succeed

How to deal with complex technical issues

Our role is pretty hybrid and lives at the intersection of other teams. As much as we love solving our own problems, escalations may happen. Here's a brief guide on how to handle them:

- Do your homework – check our docs, ask PostHog AI, and search Slack and Zendesk for similar questions. You can also check GitHub to see whether we have a bug or enhancement logged. If that doesn't bring you closer to a solution, ask in the team Slack channel.
- Don't be afraid to admit when you don't know something. Note it down and circle back once you've found the answer! Honesty goes a long way.
- Consider sharing a Loom recording in your reply to the user – It might be more efficient than a written instruction.
- If the issue requires in-depth troubleshooting, you can direct the user to create a ticket from the app, or you can do so on their behalf. Just remember to let them know before you do, so they're not surprised when they see it in the UI!
- Before escalating the issue to Support, gather as much information and context as possible so your handover is informative and thorough. You can also share a recording of the call with the team, highlighting the relevant timestamp.
- If a support issue lands in your inbox, forwarding it to supportreply@posthog.com should do the trick. Make sure to double-check in Zendesk that the ticket is not marked as Solved.
- Ideally, after the meeting with the user, they should know how to seek further help. That includes using PostHog AI, consulting the docs, and reaching out to our Support team.

How to deepen your knowledge

- Go through Sales docs, especially Contract Rules, Creating Contracts, and others from the SalesOps section. There will be some related conversations that you'll need to handle yourself, so come prepared.
- Add yourself to some AEs' Slack channels to see what kinds of questions are being asked and how they're solved.
- Check recordings in the Technical product troubleshooting and Sales & CS Trainings BuildBetter folders.
- Go through Product Homework and Analytics Exercise.
- Go through the PostHog curriculum.
- Check out Troubleshooting tips and attend/watch Product AMAs that are scheduled periodically.
- Take CoachHog for a spin!

Onboarding process and tracking

contents/handbook/onboarding/onboarding-tracking.md

The onboarding team operates a high volume, high velocity sales pipeline with all pay-as-you-go (or YC) accounts that are forecasted to spend > $500 and are not otherwise engaged by Sales/CSM. As such, Onboarding is a linear flow moving from initial outreach to confirming the product is configured properly, ending with customers who are happy paying multiple bills. We aim to keep engagements to ~8 weeks, or 2 full billing periods, but in practice, there is some spillover depending on responsiveness.

Principles

Our onboarding program was created to offer necessary help, increase the value our customers get from PostHog, and assist them in achieving their business goals.

The program is guided by a few key principles:

- Help with initial configuration and billing, offer advice on usage.
- Assist customers with any technical questions they have around fitting PostHog into their stack.
- Act as strategic partners in achieving business goals.
- Adapt to varied levels of engagement while ensuring value for everyone.
- Encourage time spent in PostHog, trying things out. Adoption can be fun!
- Share best practices to leverage PostHog tailored to specific use cases.

Internal process

Vitaly views

Daily view (link)

Sort your view by the “Next Renewal Date” column to reach out to users in a timely manner. Since our role is focused on proactively providing users with value and setting them up for success, we’ve found it’s best to contact them ~14 days before their bill renews. This gives them enough time to see our email, schedule a call, and implement potential improvements in their setup.

Keep an eye on “Onboarding Pipeline,” which indicates whether the account is New or Onboarding has been initiated.

In the view, you have other useful columns like OS Priority, OS Last Messaged, forecasted MRR, or who’s assigned to the account. All these help in prioritizing your work.

Maintaining good hygiene and attention to detail is key here. Keep labels up to date and make sure not to miss accounts that were recently added to the segment—they might appear at the top of the list among accounts you’ve already worked through.

Remember to add a short summary from meetings in a Note, and if you need to follow up at some point, create a Task with a due date.

Kanban view (link)

A supplementary view that's great for getting a general overview of progress.

Onboarding program - logic and sequence

There are two paths for customers to progress through the onboarding process: those who engage with us in some way, and those who show little or no engagement.

User engagement is tracked behind the scenes with the time stamp in Vitally, thanks to which we can query relevant data.

For day-to-day operations, these are the statuses we use to track users in the **Onboarding Pipeline** property:

- `1. New Account` - Where new customers land when they enter the `Onboarding Lead` segment. During the outreach, we audit the account and share observations on current usage and optimization tips. We point out any configuration issues affecting their bill or ability to use the product properly. Our main objective is oriented towards trimming unnecessary spending, and communicating our position that we are the cheapest for every product. This step can also involve refunding/adjusting bills for misconfigurations, per our policy.
- `2. Onboarding Initiated` - Assigned as soon as we send out the initial outreach email.
- `3. Onboarded` - The onboarding program has been completed. The status chances automatically after the Graduation email is sent, or we change it manually for some reason.
- `Sales Handoff` - Assigned to customers that we hand over to sales. This can happen at any stage throughout the process.
- `Paid call purchased` - Assigned when someone buys our consultation via the merch store. The status is applied automatically via this Workflow.

The last two are not numbered, as they happen "outside" of the regular pipeline.

> Note: You may need to add this property to your views in Vitally. It's found under `Custom Traits`.

For higher-spend accounts ($500+), we have `Check-in` **Onboarding Status** property that's triggered between 15-21 day in the Onboarding Journey. It serves us as a visual helper and reminder to circle back to the account, see if our advice was followed, record a Loom video, or share some extra resources. It's a great opportunity to re-engage customers and show them some other PostHog's capabilities that they may not know about.

The complete Onboarding Journey looks as follows:

Weeks	Actions
Week 1	First outreach to a New Account
Week 2	
Week 3	Extra check-in for $500+ accounts
Week 4	
Week 5	2nd outreach (automated) - all accounts
Week 6	
Week 7	
Week 8	Graduation (automated) - all accounts

The last two stages of the Onboarding Journey are automated with Vitally playbooks. Second outreach prompts users to surface unanswered questions and book a session with us, and the Graduation email is a nice way to conclude the journey and point out other avenues where users can get help. It's also where we ask for feedback about our Onboarding.

Account analysis for outreach and meetings

- Take a look at the Metabase primer and follow the tips included there.
- Check and get familiar with the Account health check and the Onboarding conversations pages.
- Use our docs, and link to relevant information.
- Check the Matching PostHog to a business type page to understand your customers better in general.
- Use Wappalyzer (browser or extension) to understand the customer's tech stack better. Credentials available in 1password.

How this is organized in Vitally via Playbooks

General playbooks

- Onboarding Pipeline Traits track the stage.
- Boost plan lead for onboarding specialist
- 100-500 Onboarding Segment Logic - first payment due >100 (updated 9/2/2025)
- [Onboarding Pipeline] 1. New Non-startup Account (Onboarding lead first payment due
- [Onboarding Pipeline] 1. New Startup Account (Startup lead for onboarding specialist)

Setting timestamps for each stage

- [Onboarding Pipeline] 2. Onboarding Initiated -> set timestamp

- [Onboarding Pipeline] 3. Engaged (Email/Call) -> set timestamp
- [Onboarding Pipeline] 4. (Nurture) -> set timestamp
- [Onboarding Pipeline] 6a. Onboarded — No Engagement -> set timestamp
- [Onboarding Pipeline] 6b. Onboarded — Engaged -> set timestamp
- [Onboarding Pipeline] 6c. Sales Handoff -> set timestamp
- [Onboarding Pipeline] 6d. Churned -> set timestamp

Automations

- [Pipeline Automation — Stage 1 -> 2] Has been contacted by Onboarding Specialist - Accounts we reach out to — any with a convo started by Magda or Dan get set to `2. Onboarding Initiated`
- [Pipeline Automation — Stage 2 - 3] Move status from Onboarding Initiated to Call booked
- [Pipeline Automation — Stage X - 5] Update status to Awaiting Final Outreach - This status is assigned ~10 days before the next renewal date (after having gone through any other step in the pipeline)
- sales lead - first payment due — all 2000+

Other

- Post-onboarding satisfaction survey trigger

Segments

Going forward, we only have one main segment: `Onboarding Lead`. We'll be retiring `Onboarding – engaged` as soon as we have worked through all the legacy accounts.

We also use the `Onboarding Lead 100–500MRR` auxiliary segment to provide us with more information about the account and help us prioritize the work.

`Onboarding Completed` segment corresponds with `3.Onboarded` trait and it serves as a visual indicator for other teams that the Onboarding has been completed.

Alerts and revenue tracking

We occasionally shift our attention to help customers who may need more urgent assistance. For these, we have a few types of alerts (tasks) in Vitally, where Magda is a failsafe if the account doesn't have an assigned OS.

- **Failed payments alert** - This is more of a safety net, as users are informed when it happens. It's a good moment to reach out and offer help in figuring out their volume/billing.
- **Upcoming large invoice alert** - It lets us prioritize the customer to touch base and make sure the bill doesn't come as a surprise.
- **Event/Feature Flag/Replay/Error tracking spike indicator for OS** - unusually high usage that may point to a misconfiguration and require our assistance.

To help our Revenue team get the forecasting right, we now have a `Payment Risk Assessment` field in the Vitally dashboard, where we can manually mark when we see that the customer is unlikely to pay their invoice.

Specific cases

How to stop automation

There might be some specific situations where you're actively engaged with the customer, and you don't want the email automation to fire off (e.g., the second outreach). You can easily spot when the email automation is scheduled by checking "Onboarding Pipeline Stage Times" widget in the Vitally dashboard.

To stop the automation, you can flip the account to the `Onboarding Completed` status. This change will block the second outreach, but will still fire the graduation email.

Failed payments

We may get a Failed Payment alert on a customer we still haven't engaged with. In this case, since we're unsure whether the customer is going to stay with us, we don't have to do a deep account analysis. It's enough to remind them to settle the invoice, offer help, and briefly point out obvious spikes and main drivers of the bill.

It's enough to reach out just once, as the finance team monitors the payments and handles the account deactivation.

Currently, we exclude some subject line keywords, like "payment", "outstanding", and "fail" in the "Set onboarding initiated" playbook in order to avoid the status change from `New Account` to `Onboarding initiated`. In other words, when you reach out regarding a failed payment, the account should stay as `New Account` and resurface in Vitally's queue before the next renewal, if the customer settles the payment. When that happens, do our regular outreach with a deep account analysis and enroll them in the program.

Sales handover

contents/handbook/onboarding/sales-handover.md

Initial qualification

Direct handover - skipping onboarding

If you see that a customer is spending **more than $1,000 monthly**, evaluate whether their usage looks stable and legitimate, and make sure that MRR doesn't come from an unwanted event spike or misconfiguration issue.

If that's the case, you can pass the account to Sales even **without speaking with the customer first**, as long as you've confirmed that the high spend is intentional. The goal is to react quickly to healthy, high-spend accounts—but avoid passing through problematic ones.

Before you hand off, also consider month-over-month growth. A flat $1.2k account is a very different lead from a $1k account that doubled organically last month. Growth rate matters to the Sales person deciding whether to prioritise the lead, so call it out.

Be courteous and **leave a note in Vitally** with context on the account before handing off. Include what you spotted in Metabase, any relevant billing patterns, and your read on why the spend is legitimate. The Sales person receiving the lead should be able to pick it up without having to dig.

Handover during onboarding - engaged customers

While talking with customers or analyzing the account, do some discovery to understand the reason behind their high spend and assess whether there's potential for stable spend or usage moving forward. If they're happy continuing with PostHog, you can mention our discounted pre-paid plan, which helps them save ~20%. However, if they prefer paying monthly, they are more than welcome to do so!

We typically hand the account over to Sales when a customer is interested in the annual plan, requires additional contractual or legal support, or we notice potential ourselves.

Make sure to include in Vitally our point of contact, i.e., the person you've been in touch with, while handing over the account to Sales.

Unresponsive customers during onboarding

Historically, there's still a good chance that they'll talk with Sales after passing them on! AEs have been successful in reaching out and securing long-term commitments. **You don't have to wait for the customer to complete the onboarding program - you can pass them earlier on, if they didn't respond to our initial message**, and if you see that the account matches criteria and the handover makes sense.

If there are any pending config issues that you raised before but the customer didn't respond to, just provide relevant context to the fellow AE/TAM in Vitally - sometimes it might be a good conversation starter!

Lead creation

If you come across an account with growth potential or stable high-level spend (especially if that high spend has occurred over the past two - three months and there are no pending issues to resolve), that might benefit from an annual plan or general sales engagement, you can add them to the `Onboarding referral` segment in Vitally. Within a few minutes, this will automatically create a Salesforce lead and assign it using round-robin logic.

After a few minutes, your lead will appear in the `#sales-leads` Slack channel, tagged as "Onboarding referral". As a good practice, leave a note in Vitally for the Account Executive with some relevant context on the customer. You can also ping an assigned AE on Slack, if any further follow up is needed.

The automation flips the Vitally's Onboarding pipeline trait to `Sales Handoff` so that we have not only data on the leads passed, but also a visual indication.

Proactively looking out for opportunities

- If you see an account with a promising, positive growth trajectory, but they may achieve the $ threshold after finishing the onboarding program, **set a task in Vitally assigned to yourself** in order to circle back after some time and see if they're eligible for being passed on to Sales.
- If you reach out to a high spender (+$1000), add a Vitally task to see if the first bill came through. If it did, we could pass them on to Sales faster, without having to circle back to the account before the second bill comes up. We currently have a playbook running that automates task creation.

Confusion about previous Sales engagement

Some pointers on what to pay attention to in Vitally while checking for prior Sales engagement:

- Pin `temporary owner` trait in your Vitally sidebar - the trait is set when a Salesforce lead task exists (otherwise it's "null")
- Segments (e.g. TAM/CSM Candidate, $20k MRR, Active Trial, Active Self-serve Trial, Annual Plan, etc.)
- Slack channel (following the naming convention `#posthog-[company name]`)
- Key Roles (is someone assigned to the account?)
- Trial Status widget in the Onboarding dashboard
- Active Conversations and Meetings (any trace of a booked call or an ongoing conversation)
- Notes

If Sales are already engaged, there's no need to create an `Onboarding Referral`.

If the account has engaged with the Sales team at some point and it's unclear where the conversation stands, ping your fellow AE to make sure you're not overlapping efforts.

If it's clear there's a duplication issue and we shouldn't be involved, ping Mine to double-check the logic.

What to do when Sales is involved?

If an account is in the `Onboarding Lead` segment, but there are recent Active Conversations in Vitally from a TAE/TAM (or scheduled meetings), and TAE/TAM confirms they are already actively engaged with the account, add a Vitally note saying: "Removing from Onboarding Lead segment — Sales already engaged." Then remove the account from the segment and delete both the pipeline trait and the timestamp.

Benefits

contents/handbook/people/benefits.md

Outside of our generous pay and equity, we also offer several other exceptional benefits to our team. We want to provide exceptional benefits when it comes to things that help you do your job better, and in line with the market for well-funded startups for everything else.

If you have any ideas for how we can improve our benefits offering, then please let us know!

Time off

Everyone in the team has unlimited, permissionless time off.

We also offer parental leave for new parents.

Equipment and co-working

As we are fully remote, we provide all equipment you need to have an ergonomic setup at home to be as productive as possible. We provide all team members with a company card for this purpose.

If you ever need change of scenery, co-working or working from a cafe or WeWork All Access are available, just follow our expense policy e.g. we trust you to do the right thing.

> Please message Kendal to get added to our company WeWork account.

Meeting up

We do regular team offsites - recent trips have included Mexico, Aruba, Iceland, and Portugal! Small Teams also have their own offsites at least once a year.

We also encourage people and teams to meet up in person *in addition* to the offsites. If you are working on a problem that is better worked on in person, then you should do this. Our expense policy is about trusting you to make the best decisions. Travelling can be distracting so we expect you to exercise judgement when doing this.

For any work-related travel, we also use Project Wren for carbon offsetting.

Free merch

People like our merch. If you want more, here's how to get it!

As always, we expect you to use this with restraint and with your own good judgement. The merch store should not become your sole source of clothing for your wardrobe, nor where you go any time a friend has a birthday. But sure, go ahead and buy your mom (or yourself) a hat or a hoodie!

Support open-source projects

Everyone gets a monthly open-source sponsorship budget to spend as they see fit to support open source projects of their choice.

We'll be your first investor

We'll be your first investor and biggest cheerleader, if you spend two years at PostHog and leave to start a new company. We're looking for entrepreneurs and a strong Why not now?!

Learning and development

We currently offer a Training budget and free books - you can find more on the relevant pages.

Country specific benefits

> With everyone being distributed across the world, we do our best to provide the same benefits to everyone, but they vary slightly by country depending on the services that are available and local regulations.

US

401k contribution

In the US, our 401k plan is managed by Vestwell and we match up to 4%.

Health care

In the US, you'll enroll in benefits through BambooHR and manage your coverage through UnitedHealthcare for medical and Guardian for dental and vision. PostHog pays 100% of the premium of the Platinum plan for team members, and 75% for dependents.

We offer the option to opt in to a Flexible Savings Account (FSA), which is a tax-advantaged account that allows you to contribute pre-tax dollars up to $3,400 per year to be used on out-of-pocket medical expenses. The FSA is a "use it or lose it" benefit, so any dollars that are not spent by the end of the year return to the company.

There is also the option to choose a lower tier, high deductible health plan (HDHP), which will qualify you for a Health Savings Account (HSA) that has further tax benefits beyond what the FSA provides. At the end of the year, any unused money rolls over and the contribution limit resets.

UK

Pension

In the UK, we use Royal London. Team members contribute 5% and PostHog contributes 4%, but you can opt out if you like. You can also transfer out of the plan as frequently as you want, in case you would rather manage your own private pension.

Private health insurance

In the UK, we use Aviva for private healthcare (£100 excess per policy year) and Medicash as our cash plan for dental and vision. Children are included for free. Both of these are taxable benefits which will affect your Personal Allowance each tax year, and you can opt out at any time with 1 month notice.

Nursery

In the UK, we offer the workplace nursery scheme. This enables you to pay for your children's nursery using your pre-tax salary, saving you up to 45% in nursery fees.

If you are interested in this, first check with your nursery that they are part of the scheme, then message Kendal to get this set up.

Cyclescheme

In the UK we offer Cyclescheme to save money on new cycling gear. To get started, activate your Cyclescheme account via the Workplace Extras registration form.

Other countries

Pensions

In countries where you are employed under Deel's EOR service, we make pension contributions in line with legal requirements.

> Unfortunately, we are currently legally unable to provide pensions to contractors.

Private health insurance

We offer private health insurance in countries where it is considered market to do so. For Ireland, Spain, Netherlands, Portugal & Canada the health insurer varies depending on market and offering via the Deel platform and can be subject to change. Please login to Deel to find the policy relevant to your market or reach out to the Ops team if you have any questions.

Career progression

contents/handbook/people/career-progression.md

Helping the company win helps you win

The best way to progress your career at PostHog is to understand your team's *and* PostHog's objectives, then:

- Ship fast towards these
- Give and receive direct feedback to help yourself and others do the same
- Fix problems when you see them - early objectives are often wrong

You are what you do. Getting promoted in a company that is struggling, is very hard. However, if the company is succeeding, it'll be easy to justify, *and* to afford, pay rises where people are performing very well.

Give a shit about your work, your team, and our users

These three are the inputs that lead to the output of career progression. If you focus only on yourself, no value is provided and you won't progress. If you only focus on your team, you won't build the right thing for our users. Having a consistently caring attitude will in the long term lead to progression - if you do this, PostHog will progress you.

When we IPO, you will literally walk into any job anywhere

While being able to talk about all the cool stuff you built will help you in your future career, being an early employee that took us from very early to public is a huge and an *exceptionally* rare career achievement. That's how you leap multiple positions into an exec role, or whatever else you want to do.

Ways we help you progress

Hire and maintain a team of excellent people, all working transparently, that you can learn from

We are disciplined with maintaining a high bar. And since everyone works so transparently, you can learn from watching what everyone is doing - from how board meetings work, to why we picked a company strategy, down to why our frontend is the way it is.

Give you loads of autonomy

We don't limit you, and will push for much more than you may think is possible. It will feel hard, but rewarding. You will get used to not asking for approval.

Give you lots of interesting problems to work on

PostHog has a wide variety of challenges - from data, to entire new products and features, to design and UX tradeoffs. On the go-to-market side, we're wildly different - you'll learn about self serve, bottom up adoption, handling a community, and how giving things away for free leads to us making money.

We have small teams - we can move people around as we grow to provide variety and to let people switch up their focus if things get stale.

Lightweight management

You have someone to talk to, but without being micromanaged. Their priority is to support you, and we give them resources to make them a better manager. They will also do a regular career check-in with you as part of your 1-1s to ensure you're on the right track.

Build a huge open source portfolio

Better than a fancy title - you can show future employers or investors what you built and the problems you solved.

Your team around you see your everyday work more than a manager - get direct feedback from them

Great people + direct feedback = learning.

Ways we do not help you progress

A checklist of things / a formal career progression framework

This is self-interested by its nature, so creates the wrong incentives. The benefits of frameworks only start to outweigh their drawbacks when you need to start coordinating 100s of people.

Fancy titles

We don't have a wide range of titles - we want people to be as equal as possible in order to enable autonomy versus micromanagement. Your open source work speaks for itself.

Getting a manager to progress you

This gives too much power to managers. No one else can really do this for you - your motivation to progress has to be intrinsic to be sustainable.

Feedback

`contents/handbook/people/feedback.md`

Feedback at PostHog

Sharing and receiving feedback openly is *really* important to us at PostHog. Part of creating a highly autonomous culture where people feel empowered is maintaining the most transparent and open flow of information that we can.

This includes giving feedback to each other, so we know we are working on the right things, in the right way. While giving feedback to a team member can feel awkward, especially if it is not positive or if you are talking to someone with more experience than you, we believe that it is an important part of not letting others fail.

'Open and honest' doesn't mean 'being an asshole' – we expect feedback to be direct, but shared with good intentions and in the spirit of genuinely helping that person and PostHog as a whole to improve. Please make sure your feedback is constructive and based on observations, not *emotions*. If possible, share examples to help the feedback receiver understand the context of the feedback.

Full team feedback sessions

We run full team 360-degree feedback session as part of every offsite. Some teams will do them during their own small team offsite, while others choose to do them as part of the whole company offsite. The session gives everyone the opportunity to give and receive feedback to everyone else. If your team works closely with another or is very small, you may combine with another team (but keep attendees to <8 if you can).

Ground rules

- Everybody participates! You should have a think and write up your notes in advance – don't try and wing it on the day.
- Preparation includes reading our handbook about how to be a good feedback giver and receiver.
- As a guide the mix of positive and constructive feedback will vary. You should spend more time talking over the constructive even if you have a long list of positive things to share – this is an opportunity to help each other to grow.
- Everyone is expected to give feedback to everyone, even if they don’t work together directly. It may be very short feedback, which is ok!

- That being said, avoid piling on and repeating feedback others have given unless you have a different perspective or can add more context. It is ok to say "+1 to what X said about Y" and move on. Do not spend 2 min repeating the same point that has already been made by someone else.
- Everyone is responsible for noting down and actioning their own feedback (ie. the people team won't do this for you).
- What is discussed is for the benefit of those present and does not need to be shared with others who were not present. It is ok to follow up with anyone on feedback you received or gave after the session.
- Use a notebook, or worst case keep notes on your phone. You will be much less present if you use your laptop, so we generally discourage these.

How to give good feedback

We know that giving feedback can sometimes be difficult, so here are a few tips on how to give good feedback:

- If something went wrong, focus on what has actually happened, not on whose fault it is. Assigning blame is not productive.
- Be as specific as you can with your feedback. An example can be helpful to give the recipient context.
- Sometimes a question can be more useful if you feel you lack the full context. For example 'I've noticed that you sometimes do X. Can you explain to me what your thought process is when you are doing that?'
- If your feedback is about behavior, focus on the behavior itself and its impact on you, rather than attacking the person's character. For example 'When you do X, it makes me feel Y. Would you be willing to do Z instead?'
- Remember that positive feedback is really important – we should reinforce and affirm the things we want that person to keep doing!

We expect everyone to support each other by giving lots of feedback – it's not ok to stay quiet if you have something constructive to share.

How to receive feedback well

If someone is making the effort to give you feedback, you should reciprocate by receiving that feedback well. Being a good feedback receiver means that people will be more inclined to give you feedback in the future, which will help you to grow!

Here are a few tips to help you do this:

- Assume positive intent on the part of the feedback giver.
- Try not to hear attack - listen for what is behind the words.
- It can be useful to paraphrase the feedback to ensure you have understood it correctly, or ask questions to clarify.
- You do not have to accept all feedback! However, it's probably worth taking time to reflect on it, rather than reacting in the moment. There is a difference between acknowledging feedback and disagreeing with it.

README sessions

At small team offsites we may also run README sessions in addition to 360 feedback sessions. Typically we find it useful to run these README sessions as early as possible during the offsite and before 360 feedback, as they are a great way to get to know your team.

README sessions are an opportunity for you to help others understand more about your background, communication style, and interests. You can share as much or as little as you feel is appropriate. Some things which you may wish to consider include:

- What you're good at
- What you're bad at
- What you like
- What you don't like
- How best to work with/help you

It's OK to ask short, clarifying questions when someone has finished, but sessions shouldn't become Q&As.

Team surveys

We run team surveys every 6 months using the *Pulse Surveys by Deel* Slack app. These are set up to run automatically, including reminder messages in Slack, so you don't need to chase people manually. Charles and Coua have admin access to the surveys in Slack.

The questions are based on the ones used by Culture Amp and cover categories such as Company Confidence, Culture, Growth etc. on a 1 ('strongly disagree') to 5 ('strongly agree') scale. The benchmark used is against Culture Amp's 'new tech' companies with less than 200 people. We then take the average score out of 5 and multiple it by 20 to get a % number. A bit rough, but close enough so we can compare with the benchmark.

Only the People & Ops and Exec teams have access to the full list of responses, which are not anonymous.

We follow a template to report a summary of the results in an Issue. You can view the latest survey results here - just copy the formatting ever.

Current list of questions

- I understand PostHog's goals and can see how my work contributes to them. (1 to 5)
- At PostHog, we have open and honest two-way communication. (1 to 5)
- I receive appropriate recognition for my work at PostHog. (1 to 5)
- I believe that my total compensation (salary + equity + benefits) is fair, relative to similar roles at other companies. (1 to 5)
- The leaders at PostHog keep people informed about what is happening. (1 to 5)
- If you were to leave PostHog, what would be the reason? (Free text field)
- PostHog is in a position to really succeed over the next three years. (1 to 5)
- What motivates you right now? (Free text field)
- My manager and team around me genuinely care about my wellbeing. (1 to 5)
- I feel like I am learning and growing at PostHog. (1 to 5)
- Generally, I believe my workload is reasonable for my role and I am able to arrange time out from work when I need it. (1 to 5)
- The support I am receiving and processes we have in place allow me to do my best possible work. (1 to 5)
- I would recommend PostHog as a great place to work. (1 to 5)
- I see myself still working at PostHog in two years' time. (1 to 5)

Finance

contents/handbook/people/finance.md

Finance mission

We exist to make your life easier. You should spend time shipping great products, instead of wrestling with restrictive financial controls. We want to keep things distraction-free for you, and remove admin obstacles from your path. If you are spending your afternoon arguing with an expense system, we have failed.

We aim to build a system that works for you, and not the other way around. The deal is that we take on the messy, admin heavy-lifting behind the scenes. But we ask that you don't skip the small steps (like uploading a receipt) because when you bypass the easy steps today, it snowballs into a painful cleanup job later.

When we design processes, our first question is "how can we make this disappear for the team?" or "will this ensure fewer Slack pings for us?" We don't use restrictive approval flows. We operate on high trust and give you the context to make good spending decisions rather than block you with red tape. We consider your time very carefully, 15 minutes distraction per person is days of productivity lost across the company. Sometimes we do have to consider the lesser of two evils, eg. asking everybody to do a small task now, to unlock less distractions later.

We also give you financial insights to make PostHog even better - we don't gatekeep the numbers. We want you to have visibility into monthly, quarterly, annual financial performance and SaaS metrics. We benchmark ourselves against public companies and peers in the industry so we know where we're headed!

Finance principles

This is how we think about financing PostHog as a business:

- We're efficient because it allows us to build more products.
- We want to *always* be default alive.
    - Losing control means becoming inefficient, because we would need to raise more money from VCs, who would push us for more aggressive growth to meet *their* goals. This means building fewer products and spending more on shorter-term bets, like large sales teams.
    - Being default alive means that profitability is always an option we can take, even if it is not a goal right now.
- Going from inefficient to efficient is *really* hard, so we always want to default to being efficient. When we think about our products, this means:
    - Hiring efficiency always matters.
        - New products start with just 1 or 2 people - we don't spin up a whole team on day 1.

        - Products at scale should stay efficient as they'll be able to ship faster without worrying about coordination costs.
    - COGS only matter at scale.
        - New products shouldn't have to worry about this - they should optimize for speed.
        - Products at scale have to be profitable - if they aren't, we won't stay default alive.
- We trust the team to spend money sensibly in the best interests of PostHog, and not to waste money.

Finance runbooks

These can be found in the `company-internal` repo within the finance directory.

Grievances and Disciplinary Process

contents/handbook/people/grievances.md

While these issues are hopefully an extremely rare occurrence, it's important for us to have a clear process around how we do this stuff in order to ensure everyone is treated fairly and transparently.

A couple of notes before we get started:

- Any outcome of these processes will *only* be shared with the people involved, not the wider team. Likewise, we ask people involved to maintain confidentiality in these cases. This is to give the best chance of a team member being able to fix their behavior and to work together with the rest of the company in future. Any misconduct issue may be deeply personal and sensitive compared to, for example, a performance issue.
- We will do our best to balance being thorough with coming to a speedy resolution for everyone involved. We'd expect a process to take ~4 weeks.
- Our default assumption is that we can resolve disciplinary issues and grievances internally. However, if an issue or grievance is particularly serious or difficult to resolve, we may need to bring in external help.
- This isn't a court of law - we're just trying to establish what is most likely to have happened based on the information we have.

These policies are deliberately short and simple, and use the Acas template as a model. If you have any detailed questions about how they work in practice, please ask Charles.

Disciplinary process

In cases of minor misconduct which cannot be resolved informally, we may issue a verbal warning.

In cases of serious misconduct, or multiple instances of minor misconduct, we may issue a written warning, and then a final written warning. If these do not resolve the issue, we may move to dismissal with or without severance, depending on the circumstances.

We may omit any of the stages of procedure listed above as circumstances require - for example, if the misconduct is exceptionally serious.

Serious misconduct includes things such as:

- Discrimination, bullying, or harassment
- Theft or fraud
- Physical violence
- Deliberate and/or serious damage to property
- Drug or alcohol abuse at work
- Causing loss, damage or injury through serious negligence

- Intentional breach of confidentiality
- A material breach of your employment contract

If you are a person being accused of misconduct, you will be advised in writing prior to any relevant meeting with you of your alleged misconduct, and will be given a reasonable opportunity to respond prior to a formal meeting. Meetings are usually held with Fraser. If you are in the UK (or other jurisdictions where the right to bring other people with you is a legal requirement), you are entitled to bring a colleague or trade union representative to these meetings. If this is the case, please let us know who you are bringing in advance.

We will send round written notes afterwards, which will be kept confidential.

Grievance process

> All proceedings are confidential, and you will never be punished for bringing a grievance (unless it's obviously malicious), even if no action is taken.

Victims of harassment or bullying should disengage from the situation immediately and seek support. You can speak to Fraser about your grievance and he can help you. If he is not available, talk to Carol (US timezones) or Tara (Europe timezones).

Most grievances otherwise can usually be resolved informally between you and the person involved - if it is informal and you're unsure what to do, talk to your manager. If it is about your manager, talk to *their* manager or ask Fraser. If the matter cannot be resolved informally, you should put the details of your grievance in writing and send it to Fraser (or if the matter concerns him, please send it to James or Tim). There is no particular format to follow, and you can start at this step if needed.

To make sure we can investigate your grievance properly:

- Try and raise your grievance as soon as possible - it's easier to figure out what happened that way.
- Give specific examples of the behavior that you felt was misconduct. Try to avoid sweeping statements.
- Avoid including hearsay or other people's comments in your grievance.
- While this process is confidential, our default assumption is that grievances are not made anonymously as this makes it harder for us to investigate or to report back to those raising the complaint.
- Please be understanding with those dealing with your grievance. We take these issues very, very seriously, and likely any action we take is difficult in one way or another.

Fraser will hold a meeting(s) to discuss further. If you are in the UK (or other certain jurisdictions where the right to bring other people with you is a legal requirement - in which case we require you to confirm the other attendees in advance of the meeting), you are entitled to bring a colleague or trade union representative to these meetings, and we will send round written notes afterwards, which will be kept

confidential to those in the meeting and those the complaint is being made about. The number/type of meetings held is flexible depending on the nature of the grievance. You are not obliged to attend a meeting with the person you have a grievance against if you don't want to.

If, following investigation, your grievance is not upheld, then we will support everyone in rebuilding their working relationship to the extent it is possible. We may consider making arrangements to avoid the affected parties working together closely.

Whistleblowing

Whistleblowing is where you observe illegal or dangerous behavior, and is different from raising a grievance as it may not affect you directly. In this case, please email Fraser and Hector. This includes things like criminal offences, someone's health and safety being in danger, or damage to the environment. You can also whistleblow about someone trying to cover up information about any of these issues. We will broadly follow the same process outlined above for grievances.

If your concern is a personal one, it will usually not be covered by whistleblowing. In these cases, you should raise a grievance.

Appeals

If you disagree with the outcome of the above processes, you have the right to appeal if you can demonstrate why you believe a particular aspect of the investigation has materially affected the outcome. Appeals must be submitted within 2 weeks of receiving the outcome.

If an appeal is submitted, we'll arrange a final meeting within a reasonable time period. Any decision made here will be final and there is no further right of appeal. We will aim for the meeting to be held by a member of the Exec team who wasn't involved in the process previously.

Design Hiring

`contents/handbook/people/hiring-process/design-hiring.md`

Design hiring at PostHog

Our design team is small and we don't hire into this team very often. Please check our careers page for our open roles.

What we are looking for in Design hires

Beyond the specific skills listed in the job description, we always generally look for:

- Strong eye for design
- Experience working with Figma
- Ability to ship iteratively
- Communication skills
    - Do they have writing errors in their cover letter? What does their online presence look like?
    - Moreso than other companies, all of our communication is written and public for the world to see. Good written communication is key.

Design hiring process

1. Culture interview

This is our standard first round culture interview with the People & Ops team.

2. Technical interview and portfolio review

The technical interview round is a 2-part interview, lasting up to 90 minutes in total.

The first half of the interview will be with Cory and 1 or 2 team members, and it will focus on your Product and Design thinking. You can expect questions around your typical design process and how you prioritize.

The second part of the interview will be a portfolio interview, where you will meet a few other members of the team. You will present a deep dive into your portfolio, covering the end-to-end process from strategy to design to impact.

3. Design SuperDay

The final stage of our interview process is the PostHog SuperDay. This is a paid full day of work with us, which we can flexibly arrange around your schedule.

A Design SuperDay usually looks like this (*there is a degree of flexibility due to time zone differences):*

- Kick-off session with Cory(Lead Designer)
- Meet the founders - Tim and James
- Time to focus on the task - we can provide support via your personal Slack channel
- On days when we have company wide meetings, we will invite you along to that and give you a chance to introduce yourself. On days without company-wide meetings, we will arrange for you to meet a few members of our team for a casual lunch/coffee break

> In line with our values and culture, you might get short replies like "step on toes" or "bias for action".

Developer Relations Hiring

`contents/handbook/people/hiring-process/devrel-hiring.md`

Developer Relations hiring at PostHog

Developer Relations are relatively new at PostHog. However, the team will be growing as the company grows and as we increase our engagement with developer communities.

Currently, we are hiring for the following Engineering roles:

- Developer Educator

What we are looking for in DevRel hires

Beyond the specific skills listed in the job description, we generally look for:

- DevRel at PostHog is undertaken with an ethos of empathy and collaboration. Any DevRel hire must clearly demonstrate this.
- Has built something from scratch, ideally with minimal outside help
  - You may have been the founder of a startup, or built an impressive side project. You may have also worked on a project at work where you were the only developer.
- Communication skills
  - Are there any writing errors in their cover letter? What does their online presence look like?
  - More so than other companies, all of our communication is written and public for the world to see. Good written communication is key.
- Community focus
  - Our DevRel team works very closely with our customers - they do community support, demos, and help with installation and integration. All potential hires need to be driven by delivering the best possible experience for their customers.

DevRel hiring process

1. Culture interview

The culture interview usually lasts around 30 minutes and will be with someone from our . This round is loosely structured into 4 different sections:

1. PostHog - mission, vision, team, way of working etc. If it was cold outreach, we provide a little more context up front.
2. Candidate background and mindset.
3. Talk about the hiring process and check if the candidate has seen our compensation calculator so we know we're roughly aligned.
4. Answer any open questions.

We are looking for proactivity, directness, good communication skills, an awareness of the impact of the candidate's work, and evidence of iteration or a growth mindset.

2. Technical Interview

Most developer relations roles will go into detail about the candidates experience and thoughts on:

- API standards
- API usage best practices
- SDKs and libraries
- Documentation
- Tutorials
- Video content
- Data analysis and manipulation
- General programming skills

DevRel SuperDay

The final stage of our interview process is the PostHog SuperDay. This is a paid full day of work with us, which we can flexibly arrange around your schedule.

We will share the task with you at the start of the day. The task is representative of the work someone in this role at PostHog is doing, and it is always the same for each candidate, so we can make clear comparisons.Day

This gives you the chance to learn how we work, and for us to see your quality and speed of work, as well as the way you communicate. It is a very demanding day of work, but we all want you to succeed!

An DevRel SuperDay usually looks like this (*there is a degree of flexibility due to time zone differences)*:

- Kick-off and ideation session where we'll define the specific deliverables for the Super Day. These will most likely include:
  - GitHub repo with code and README (we'll create and setup a private repo)
  - Written or video tutorial
- Time to focus on the task, we can provide support via your personal Slack channel
- On days when we have company-wide meetings, we will invite you along to that and give you a chance to introduce yourself. On days without company wide meetings, we will arrange for you to meet a few members of our team for a casual lunch/coffee break
- Depending on the time zone, we might arrange a wrap up session at the end of the day

You can expect to hear back from us within two working days of your SuperDay. We will also make the payment for your SuperDay as soon as possible.

If we decide to make you an offer, we will most likely arrange a call to discuss feedback and next steps.

If we don't make an offer, we will give you as much constructive feedback as possible.

Engineering Hiring

contents/handbook/people/hiring-process/engineering-hiring.md

Engineering hiring at PostHog

Engineers make up around 60% of our team, and we are almost always hiring for Engineering roles. This page provides internal documentation on our engineering hiring process, including roles and best practices for interviewers. You can find all open roles at PostHog on our careers page, in case you want to refer someone.

What we are looking for in engineering hires

Beyond the specific skills listed in the job description, we generally look for:

- Experience with relevant technologies (Python or similar, React or similar, something to do with big data is a bonus)
  - We don't care how many years of professional experience someone has, but depending on our current team structure we may be looking for more or less experienced people for a role - if that's the case, we will be explicit in the job spec.
- Has built something from scratch, ideally with minimal outside help
  - They may have been the founder of a startup, or built an impressive side project. They may have also worked on a project at work where they were the only developer.
- Communication skills
  - More so than other companies, all of our communication is written and public for the world to see. Good written communication is key.
- User-centric
  - Our engineering team work very closely with our users - they do customer support, demos, and help with implementation. All potential engineers need to be excited by the prospect of getting to work directly with users.

Engineering hiring process

Hiring is a team effort, and we need everyone to contribute to make the best new hires. Talent Partners handle all scheduling throughout the interview process, and support both interviewers, candidates and hiring leads.

You can find more information regarding the hiring process in the handbook, or reach out to @talent-folks in Slack.

Culture screen

The culture screen is handled by the Talent team. Normally this is a 20-30 min call where they make an initial assessment for the candidate's fit for the job, culture, communication style, and sort all logistics.

Technical screen

The technical interview is an hour-long technical interview with one of our engineers. This might be architecture design or diving more into past technical experiences in more of a workshop style. No whiteboarding or brain teasers. We share our guide to preparing for the technical screen with candidates so they know what mindset to bring.

Sometimes when you get part of the way into a technical interview it becomes clear that the person is not a fit. Because rejecting candidates needs to be done in a specific way, please continue the interview as usual and do not reject on the call. It's okay to end the interview a bit early - interviews often don't take the entire time, and it's okay to give this caveat ahead of every technical interview.

You should use the technical exercise guide when evaluating candidates at this stage.

> You may be shadowed by another PostHog team member – a shadow is someone who listens in, but doesn't participate. This is something we do regularly among technical interviewers, as a way of improving the hiring process. During high season, we may ask some of you to record these interviews for training purposes to help us onboard and train new interviewers faster. The candidate will, of course, have the chance to opt out by either letting their recruiter know in advance or letting you know at the start of the interview - you should always ask the candidate for their permission before recording and this will never affect the outcome of the interview - there are many reasons why someone may opt out from being recorded.

Culture & motivation chat

One of our co-founders or execs – Tim or James, depending on scheduling – will meet with the candidate for a short 15 min chat to dive deeper into culture and motivation.

Engineering SuperDay

The final stage of our interview process is the PostHog SuperDay. This is a paid full day of work, which we can flexibly arrange around the candidate's schedule. We share our guide to preparing for the engineering SuperDay with candidates so they know what to expect.

For full-stack roles, the task involves building a small web service (both backend and frontend) over a full day. The task is designed to be *too much* work for one person to complete in a day, in order to get a sense of their ability to prioritize (and ship!).

Each engineering SuperDay will have a SuperDay buddy, this person will conduct the interview halfway through the day, will be available in Slack throughout the day to answer any questions, they will also be giving feedback on the SuperDay output.

An engineering SuperDay usually looks like this (*there is a degree of flexibility due to time zone differences)*:

- An invitation to a personal Slack channel for the SuperDay, which we'll use throughout the day

- This will include:- the talent team, cofounders, exec, hiring lead, & the SuperDay buddy
- Time to focus on the task
- An interview with the SuperDay buddy
- A chat with James, Tim, or an exec, whoever they didn't meet with in the previous stage
- Wrapping up – at the end of the work day, they'll send us what they've built, along with a summary

Usually the Superday buddy will review the output, but they can ask other engineers for input when needed, and we'll get back to the candidate with our final decision ASAP (always within a few days).

Overall, candidates should spend at least 80% of their time and energy on the task and less than 20% on meeting people, as we base our decision on their output of the day. However, we encourage everyone to use the Slack channel as much as needed for any questions or problems.

How to become an interviewer at PostHog

As PostHog grows and our hiring goals get bigger and bigger, to achieve that we will need more people taking interviews and assessing people in those interviews. As we scale, it's important that we maintain a calibration across interviewers by onboarding each new interviewer to the interviewing process carefully.

If you wish to get involved in interviewing, you can request so by contacting the talent team using the @talent-folks handle in Slack in the #team-talent channel. Please note, that if you are in your first 90 days at PostHog, you should not be focussing on interviewing and focus on ramping up and onboarding successfully. Even shadowing interviews can be distracting, so consider even leaving these until after your first 90 days.

Once you have let the talent team know, you'll work closely with a Talent Partner to get you up to speed. This will involve them scheduling you to shadow at least two live technical screens with two of our most experienced interviewers. They will also share with you the relevant watching and reading materials that should be consumed before conducting your first interview. Your very first interview will be shadowed by one of our experienced interviewers and they will give you feedback, on both their assessment of the candidate interviewed and then how you did. Based on how this goes, either you can get another interview shadowed by another engineer for more feedback, or you can go out on your own.

Your first interview alone

Your first interview alone might feel daunting, and it should. So it is best to prepare as much as possible, read all the available materials on the candidate ahead of schedule. Prepare how you will manage your time based off the interviews you've shadowed and the feedback from our shadowed interview. Block out time at the end of the interview to ensure you have time to write up your notes and reflect on the candidate and provide the feedback. Eventually you will get into the habit of being able to rely on AI notetaker notes (or your own notes) to be able to come back to leave the feedback but on your first go, it's important to have all the information fresh and give yourself plenty of time to go through this process. You want to feel excited about a candidate once you've finished up with them, but amongst this is a new experience so it's important to give yourself the ability to feel excited and go into detail on how you rate them. You want to be able to feel beyond reasonable doubt that you are making the right decision.

You should try to have as extensive notes as possible, this is good hygiene for interviewers at later stages to dig into areas of uncertainty. Always make sure to make any flags or doubts you have very clear.

Your first 5-10 interviews

Once you have conducted your first 5-10 interviews you will want to reflect on how you are getting on. The best way to do this is to review how your candidates have got on. If you've rejected more than 6 or 7, then perhaps you might be being too harsh. You can ask other interviewers or a talent partner to assess your notes and see if they would have been more lenient. The technical screen has about a 50% pass rate, so keep that in mind.

For the candidates that you put through, it's worth keeping a close eye on how they perform at stage 3 & stage 4 and seeing how they have done. Did those flags that you have ultimately lead to them failing, should you have just said no? Where flags that you had actually not a problem, if so, how can you dig in to them better next time to understand them better.

You should try and keep your approach relatively consistent for the first 5-10 interviews so you can then introduce changes afterwards and see if they yield better or worse results. If anything is very obviously not working, change it immediately.

How to keep improving as an interviewer

- Regularly shadow other interviewers, have other interviewers shadow you. Give each other feedback
- Speak with Talent Partners, ask for their feedback
- Ask exec's why they gave specific scores about candidates you interviewed
- Keep track of how the candidates you have assessed get on in the later stages

Preparing for the engineering SuperDay

contents/handbook/people/hiring-process/engineering-superday.md

If you've been invited to a PostHog SuperDay, here's what to expect and how to set yourself up for success.

What the SuperDay looks like

The SuperDay is a paid full day of work ($1,000 USD). You'll receive a project task at the start of your day and submit your work at the end. The project is the main focus -- you'll build something from scratch, tailored to the role you're applying for. Expect to spend the majority of your day on it.

Scheduled throughout the day, you'll also have:

- **A debugging session (45 min)** -- a separate pairing session where you'll work through bugs and features in an existing codebase with an interviewer. This is a different codebase from your project.
- **A check-in call** -- your SuperDay buddy (the same engineer available to help you in Slack) will review your project progress partway through the day, ask about your decisions, and offer feedback.
- **A short chat with a co-founder or exec** -- a brief conversation about culture and motivation.

You'll also have access to a dedicated Slack channel with the team throughout the day. Use it -- share progress, ask questions, surface blockers.

The project

What we're evaluating

Shipping and execution

The scope of the task is deliberately broad, so you have room to make prioritization choices. You won't finish everything -- that's expected. We want to see how you decide what matters most.

Strong candidates ship a working core feature early, then layer on improvements. They make deliberate choices about what to build and what to skip, and they can explain why. A functional product that solves the core problem well beats a half-finished product that tries to do everything.

Technical depth

We care about the quality of what you build. This means thoughtful architecture decisions, clean code, sensible error handling, and attention to edge cases in the data you're working with.

The best candidates go beyond surface-level implementation. They notice patterns and anomalies in the data. They think critically about whether their solution is actually correct.

Product sense

PostHog engineers are product engineers. We want to see you think about the person using what you're building. Is the interface intuitive? Does the output actually help someone make a decision? Would you be proud to demo this to a customer?

Think about the *utility* of what you're building.

Problem-solving and creativity

The strongest SuperDay submissions show candidates who thought deeply about the problem. They adapted when something wasn't working and found ways to make the tool more useful beyond the basic requirements. For example, if the core task asks you to visualize data, a strong candidate might notice something interesting in the data itself -- an unexpected pattern, a segment that behaves differently -- and surface that insight in the product. That kind of curiosity matters more than adding extra UI polish.

We notice when someone asks "what would actually help a user here?" and lets that guide what they build next.

How to prepare for the project

- **Use tools you know well.** You can use whatever technologies you're comfortable with. Pick tools you're productive in so you can move fast.
- **Think about data.** You'll be working with a dataset. Before jumping into code, spend time understanding what the data looks like, what stories it tells, and where it might have quirks. The best candidates treat the data as a first-class part of the problem.
- **Get comfortable explaining your decisions.** During the check-in call, we'll ask about the choices you made. Why this architecture? Why this feature first? What would you do differently with more time? Reflect on your decisions as you go.

During the day

- **Communicate proactively.** Share progress updates and blockers in Slack. Ask clarifying questions early if something is ambiguous. Don't wait until you're stuck for hours.
- **Commit regularly.** We'll review your git history. Frequent, meaningful commits show us how you work, how you think, and how you build incrementally.
- **Prioritize ruthlessly.** Build the core feature first and get it working end to end. Then improve it. Then add more. Resist the urge to gold-plate any single piece before the whole thing works.
- **Take feedback seriously.** At the check-in, your interviewer will offer suggestions. We pay attention to whether those suggestions show up in your final submission. This is a signal of how you'd work with teammates day to day.

The debugging session

What to expect

You'll join a live coding environment with an interviewer and work through a series of problems in an unfamiliar codebase. The problems range from fixing bugs to improving performance to implementing a small feature. You won't know the codebase in advance -- that's the point.

You're allowed to use Google for reference, but we ask that you don't use AI tools – we want to see how *you* think about debugging. Treat the interviewer like a colleague -- you can ask them questions, think out loud, and discuss approaches.

How to prepare

- **Practice reading other people's code.** Pick an open-source project you've never seen, find a bug report, and try to trace the issue through the codebase. Building a mental model of an unfamiliar system is the core skill here.
- **Brush up on debugging fundamentals.** Understand how to trace request flows through a web application. Be comfortable reading error messages, stack traces, and logs. Know how to isolate problems systematically rather than guessing.

During the session

- **Read the README first.** Take a few minutes to read the documentation and understand the system before touching anything. Candidates who jump straight into code without understanding the architecture tend to struggle.
- **Form a hypothesis before changing things.** Resist the urge to start editing code immediately. Think about what might be going wrong, then go looking for evidence.
- **Narrate your thinking.** Talk through your thought process, even when you're uncertain. Saying "I think the problem might be here because..." goes a long way. The interviewer can only follow reasoning they can hear.
- **Verify your fixes.** After making a change, confirm it works and check that you haven't broken something else.
- **Ask questions.** The interviewer is there to help. Treat them like a colleague you're pairing with.

What not to worry about

- **Perfection.** We don't expect you to finish everything. We care about the quality of what you ship, the decisions you make, and how you work under constraints.
- **Specific technologies.** Use whatever stack you're strongest in for the project. For the debugging session, you'll work with what's already there -- we're evaluating your problem-solving ability.
- **Getting stuck.** Everyone gets stuck. What matters is what you do next. Ask questions. Try a different approach. Talk through what you're thinking.

A note on AI tools

You can use AI tools during the project portion of the day -- we know this is how many engineers work. But you need to *understand* what you've built. During the check-in, we'll ask you to walk through your architecture, explain your decisions, and reason about your code. If you can't defend and explain your solution, it won't matter how polished it looks.

For the debugging session, we ask that you don't use AI tools beyond basic autocomplete. We want to see how you reason about code.

What comes next

After the SuperDay, everyone involved will leave their feedback. We aim to get back to you with a decision within 48 hours. You can read more about the full interview process here.

If you've made it this far, good luck -- we're rooting for you.

Preparing for the technical screen

contents/handbook/people/hiring-process/engineering-tech-screen.md

If you've been invited to a technical screen at PostHog, here's what to expect and how to show up prepared.

What the technical screen looks like

The technical screen is a 60 minute architecture and design discussion with one of our engineers. You'll work through an open-ended problem together, and there are many reasonable approaches. We're primarily interested in *how* you think.

What we're evaluating

The session is intended to discover where your knowledge is wide and where your knowledge is deep. The best candidates tell us when they're communicating their direct experience, when they're talking about work they were close to but not part of, and when they've not done something but know that's how to solve that type of problem.

System design instincts

We want to see well-developed intuition for how systems work in practice – choosing the right tool for the job, understanding where complexity is warranted, and reasoning about what happens as requirements change. This is about technical depth and breadth, not just scale.

For example (not from what we'll discuss): how would you design a notification system that needs to reach millions of users without overwhelming downstream services? If you're building a deployment pipeline, where do you put the guardrails so a bad deploy doesn't take down production?

Strong candidates reach for these concepts naturally as part of their design.

The "why" behind your decisions

We want to hear *why* you'd choose a given technology. Saying "I'd use Postgres" is fine. Saying "I'd use Postgres here because the access patterns are relational and consistency matters more than write throughput for this part of the system" is much better.

Every design decision involves tradeoffs. We want to hear you articulate them – even when there isn't a clear winner. Knowing when *not* to use a technology is just as valuable as knowing when to reach for it. Showing that you understand the costs of your choices matters a lot.

Problem-solving approach

The strongest candidates slow down before they speed up. They ask clarifying questions. They scope the problem. They decompose it into pieces they can reason about individually.

We're looking at your process: do you clarify requirements before committing to an approach? Can you break a big problem into smaller ones? When you hit a fork in the road, how do you decide which way to go?

If you find yourself wanting to immediately start listing technologies, pause. Take a breath. Ask a question instead.

Product sense

PostHog engineers ship product, work directly with customers, make product decisions, and own outcomes end to end. In the technical screen, this shows up as thinking about the *user* of whatever you're designing.

Who is using this? What do they actually need? If you're designing an alerting system, do you think about what happens when someone gets paged at 3am for a non-critical issue? If a design decision trades off developer convenience for a better user experience, which do you lean toward and why?

You should be someone who thinks about your users, not just your systems.

Autonomy and independent thinking

PostHog is a company of small teams with high autonomy. We need people who can identify problems, figure out solutions, and drive them forward on their own.

In the interview, this shows up as taking ownership of the problem. Drive the conversation. Propose ideas. Change direction when something doesn't work. Treat the interviewer as a collaborator.

How to prepare

The best preparation is reflection. More concretely, here's what we recommend:

- **Think about systems you've built or worked on.** What went well? What would you change? What broke, and why? The ability to reflect honestly on past work is one of the strongest signals we see.
- **Practice thinking out loud.** Walk through your thought process, even when you're uncertain – especially when you're uncertain. The interviewer can only evaluate reasoning they can hear.
- **Get comfortable with ambiguity.** The problem will be open-ended on purpose. There is no single correct answer. We want to see how you navigate uncertainty.
- **Brush up on fundamentals, not trivia.** You should understand how the building blocks of modern systems work and when to reach for them. You don't need to know the exact configuration flags for any particular technology.

What not to worry about

- **Specific language or framework expertise.** We care about your ability to reason about systems, regardless of which stack you've used.
- **Getting the "right" answer.** There isn't one. A thoughtful wrong turn that you recover from tells us a lot.
- **Sounding polished.** Genuine thinking beats a practiced presentation. It's okay to say "I'm not sure, but here's how I'd figure it out."

A note on the format

We want this to feel like a working session. The interviewer is there to collaborate with you, ask follow-up questions, and sometimes push back on your ideas. If something isn't clear, ask. If you want to change direction, say so. The best interviews feel like a conversation between two engineers solving a problem together.

If you pass the technical screen, you'll meet one of our co-founders or execs for a short culture and motivation chat, followed by a PostHog SuperDay. You can read more about the full interview process.

Good luck – we're rooting for you.

Leadership Hiring

contents/handbook/people/hiring-process/exec-hiring.md

Leadership hiring at PostHog

We deliberately keep our structure flat and we don't believe in having a lot of fancy titles early on. However, as we grow, we will hire people into more senior-type positions.

With our senior leadership hiring, more so than normal, we are aiming for speed, and as always, quality. If a candidate is amazing but doesn't fit with a specific role need we have *right now*, we still aim to treat the hiring process with the same urgency as if posthog.com has gone down.

Hiring process

Preparation

Before we kick-off the hiring process for a role, we make sure to have everything we need for the role prepared:

- James or Tim to write job description, Blitzscale team review
- Post the role - share in our networks (we may not publicize this in all our usual channels as these types of roles can attract a very high volume of candidates who are not relevant)
- Ask investors for referrals
- Agree on salary benchmark and equity level - this usually doesn't fit in our compensation calculator
- Decide on interview process - this might be bespoke (see below)
- The People team will build a market map and share it with the leadership team, outreach ideally coming from the founders

Interview process

In order to ensure speed, we aim to finish the process within 5 working days (assuming the candidate has availability). This is a rough guide that can be adapted

Day 1: Candidate meets Coua - *30-45 minutes*

- Culture
- Important information: time frame, salary expectations (base/equity/bonus/other), visa, other open processes
- Answer open Qs

Day 2: Candidate meets James and/or Tim - *45-60 minutes*

- History, mission, vision
- Role responsibilities

- Role outlook (team, development etc)

Day 3: Technical Interview with James/Tim + respective team - *60 minutes*

- Background and experience
- Technical deep-dive
- Scenario-based questions

Day 3: Meet rest of the team - Charles - *30-45 minutes*

- Strategy and long term outlook
- Culture fit

Day 4: SuperDay (*optional*) or meet the team (standup or informal lunch)

Day 4: Wrap up call with James and/or Tim - *30 minutes*

- Answer any open questions, potentially talk about offer details already
- Coua to follow up via email

Day 5: Offer out

- Coua to send official offer and comp sheet with James/Tim/Charles in CC
- James/Tim/Charles to drop a quick message how excited they are
- ☝️

> Depending on the role, we might also schedule a call with one of our investors.

We take exceptional people when they come along - and we really mean that! Don't see a specific role listed? That doesn't mean we won't have a spot for you.

In cases where a candidate reaches out without us having a role posted, we follow the same process as above, and work through all open tasks we would usually prepare for on day 1 and 2.

Interview technique - principles to follow

`contents/handbook/people/hiring-process/how-to-interview.md`

Reminder: hiring the right team is the most leveraged activity we can do. Whatever you do, focus on getting the strongest signal from a candidate in an interview. Do *not* focus on scalability / efficiency.

Focus on themes

Many well-intentioned interviewers will create a long list of questions that they'll follow rigorously. This is likely to lead to shallow answers.

You're trying to understand how a human being operates, so go deep. It'll be more interesting for both of you, and will give a stronger signal.

Prepare the themes in advance you'll want to ask about. For a cultural interview, it might be something like:

- scrappiness
- low ego
- ambition
- able to write code
- optimist

For each of the above themes, consider some good questions in advance. Use these as starting off points.

Ask permission to get what you need

Candidates' expectations of interviews vary wildly.

Less experienced candidates, or those in less competitive markets, often expect an intense questioning.

The reverse is true for more experienced candidates - who will want to understand if the company is performing well, aligned, and many more things to help them pick the right place to work.

At the start of the interview, "name it". Say to the candidate something like "hey, I need to go deep on how you work to do the best assessment of a good fit here, is it ok if I focus this interview primarily on that for the first 20 minutes? Then I'll leave 10 minutes at the end for questions. If we overrun, I can book more time with you." Other times, you'll need to explain the opportunity more - for example, if the candidate came through cold outreach. Have a clear idea before you start, and explain it to the candidate up front.

Work as a team

Focus your questions on the areas you're stronger at. If you're great at scrappiness, you're probably best suited to spotting it in others.

It's more important to validate a few things well, and to get others to dig deeper in other areas, than it is for you to do a shallow interview across everything. If you miss something, just create a clear ask of your next colleague to cover the area you missed, or to dig deeper on an area you felt uncomfortable with.

Don't bias yourself

When you do your final write up on a candidate, do this ahead of reading the feedback from others.

Humans are evolved to stick to their tribes - if you know that your colleagues believe X, you're much more likely to believe X. Reading others' feedback means you are less likely to say no to someone because of minor concerns or to push for a candidate with hidden talent. Both things we need to do.

Bonus points - writing your own independent decision in front of your peers, forces you to clarify your feedback properly.

Figure out why you're not excited

You will be asked to give a score out of 4 for each interview (where 4 is the highest). If you don't give a 4/4, please articulate as clearly as you can why, even if it's a minor concern. This helps (i) subsequent interviewers to dig further into a concern to validate/invalidate it and (ii) it may cause other people to spot/mention the same issue - which can stop us moving forward with someone that won't be a good fit.

Some of the hardest decisions are when lots of people are *fairly* lukewarm on a candidate. This is particularly likely when a candidate has relevant experience but is a poor cultural fit.

Beware of how you're feeling through the interview, and adapt as you go.

If a technical interview makes you feel worried someone isn't fast, energetic, or intelligent enough, or whatever else - do some digging on those themes.

Write out mild concerns

Imagine any perceived issue being magnified 10x when the candidate starts. Mention in your feedback to others if you had a mild concern about something. Sometimes you'll find that everyone shares this concern, which means we shouldn't hire.

Get specific

Going into detail helps you figure out the difference between someone that *sounds* good and someone that *is* good at their job.

How did they solve the impressive sounding technical problem? Why did they solve it like that? Did they drive the project or were they a passenger? Who actually wrote the code?

In one interview, assessing organization skill, I've even found out how a candidate used to organize her fridge. "What's something you've done that is so organized, that it was weird".

Keep it on track

Some candidates, due to nerves, will go down rabbit holes. The ability to sum up information concisely, under pressure, usually isn't something that appears in our job descriptions.

Therefore, if a candidate goes way off track, it's in their interest for you to politely interrupt "hey I think I've got what I need here on this question, I'm going to move on so we cover everything - is that ok?"

Focus on slope

... and be very wary of getting seduced by companies rather than people. Candidates who've worked at places with strong product market fit will have had an easier time achieving results. Some of our best people have come from a string of very average startups.

As the interviewer, you should feel a little nervous

A short interview has a *huge* impact on our company - either hiring the right or wrong person. There are lots of hard-to-reverse consequences of not getting it right.

Bring energy into the interview. Be engaged. You are part of our brand.

Marketing Hiring

`contents/handbook/people/hiring-process/marketing-hiring.md`

Marketing hiring at PostHog

Our is small and we don't hire into this team very often. Please check our careers page for our open roles.

What we are looking for in marketing hires

Beyond the specific skills listed in the job description, we always generally look for:

- Communication skills
  - Moreso than other companies, all of our communication is written and public for the world to see. Good written communication is key.
  - Are they opinionated? Do they avoid generic marketing-speak?
- T-shaped people
  - We generally look for people who are generalists with a spike in one particular area, vs. specialists
  - We avoid people who are interested in building and managing a large team

Marketing hiring process

1. Culture interview

This is our standard culture interview with the People & Ops team. We will at this stage also ask for work samples or portfolios, to get a better feeling for the work a candidate has done in the past.

2. Technical interview and portfolio review

The technical interview round usually lasts 45-60 minutes and usually involves two of our team members. They will ask questions around background and previous experience, as well as some scenario-based questions. At the end, they will leave time to answer any open questions.

If relevant, we'll go through a candidate's portfolio.

3. Marketing SuperDay

The final stage of our interview process is the PostHog SuperDay. This is a paid full day of work, which we can flexibly arrange around your schedule.

The task will usually be actual marketing work, involving creating a piece of content or talking to customers, though we don't actually publish the work. We usually give a fairly open-ended task, where it is up to you to decide how you want to prioritize and tackle it.

A Marketing SuperDay usually looks like this (*there is a degree of flexibility due to time zone differences):*

- Kick-off session
- Meet the founders - Tim and James
- Time to focus on the task, we can provide support via your personal Slack channel
- Informal session with a few team members
- Meet a few members of our team for a quick chat

Overall, you should spend at least 80% of your time and energy on the task and less than 20% on meeting people, as we will base our decision on your output of the day. However, we encourage everyone to use the Slack channel as much as needed for any questions or problems.

> In line with our values and culture, you might get short replies like "step on toes" or "bias for action".

Operations Hiring

`contents/handbook/people/hiring-process/operations-hiring.md`

People & Ops hiring at PostHog

People & Ops at PostHog covers legal, finance, people and culture.

This is our smallest team, and we don't hire very often. That means that each new hire has a disproportionately high impact compared with other, larger teams. Please check our careers page for our open roles.

What we are looking for in Operations hires

Outside of the skills listed in the job description, we are generally looking for:

- Warmth and positive energy - the kind of person that other people *want* to ask for help
- Proactive and organised, with a strong attention to detail
- Ability to prioritize
- You are not afraid to step on toes - you have a bias to action and lots of initiative
- Top-notch communication skills, setting an example to the rest of the team
- Willingness to dive in and learn technical concepts and engage with tools like GitHub

Operations hiring process

Culture interview

This is our usual first round interview with a member of the People & Ops team.

Technical Interview

The technical interview usually lasts between 45-60 minutes and you will probably meet a member of our as well as Charles. For this round, you can expect questions about your background, together with scenario-based questions.

Operations SuperDay

The final stage of our interview process is what we call a PostHog SuperDay. This is a paid full day of work, which we can flexibly arrange around your schedule.

We will share the task with you at the start of the day. The task is representative of the work someone in this role at PostHog is doing, and it is always the same for each candidate, so we can make clear comparisons. It will typically involve doing actual PostHog work, e.g. sourcing candidates or planning an offsite.

An Operations SuperDay usually looks like this (*there is a degree of flexibility due to time zone differences):*

- Kick-off session
- Meet the founders
- Time to focus on the task, we can provide support via your personal Slack channel
- Informal session with a team member
- Meet a few members of our team for a quick chat

Overall, you should spend at least 80% of your time and energy on the task and less than 20% on meeting people, as we will base our decision on your output of the day. However, we encourage everyone to use the Slack channel as much as needed for any questions or problems.

> In line with our values and culture, you might get short replies like "step on toes" or "bias for action".

Sales and Customer Success Hiring

contents/handbook/people/hiring-process/sales-cs-hiring.md

Sales and Customer Success hiring at PostHog

Our Sales and Customer Success teams look after customers paying $20k a year or more for PostHog, as well as new customers who *may* end up in that bucket. The job of the teams is to land and expand usage of PostHog in these customers. We have three roles on the Sales and CS team:

- Technical Account Executives focused on closing new business from inbound and outbound leads
- Technical Account Managers focused on expansion from existing customers and closing new business from product-led leads.
- Customer Success Managers focused on the retention of customers using all of our products already.
- Onboarding Specialists focused on ensuring newer and smaller customers are set up for success with PostHog.

We've proven that the way we do sales works at a small scale, we are now growing the team in line with increased top-of-funnel growth for PostHog. Please check our careers page for our open roles.

What we are looking for in Sales and CS hires

Outside of the skills listed in the job description, we are generally looking for:

- Technical aptitude - our team members are the primary person responsible for the customer relationship, and that includes solving technical problems.
- Great interpersonal skills - we need people that our customers are excited to work with.
- A genuine passion for helping customers be successful.
- Prioritization skills - you'll be working with a book of business at different states and will have to prioritize your work accordingly.
- People who are motivated by revenue growth.

How we evaluate candidates

We need to be particularly sensitive to culture at this stage. We can handle someone underperforming much better than someone who is a poor culture fit due to the impact on the broader team. We don't want to end up taking cold leads from BDRs so we can run MEDPICC from our car phone while promising 50% discounts if they sign before the next full moon. We want someone who is comfortable carrying a sales conversation while also possessing the technical chops to talk to engineers and get their hands dirty.

We want someone who can own technical problems and even if they don't have the answer, understand enough of the context to provide that to engineers. We want someone who sees themselves as the first line of defense for our engineers, because engineering time is valuable; it's a win when they can solve a problem without additional engineering lean in.

A great litmus test for a candidate is if they are comfortable instrumenting PostHog and can speak to how they actually implement it on a site. That's typically a good indicator that they've got the right technical prowess.

We want someone who is in it to develop customers for the long run, we don't want someone who is here for quick churn and burn to pump up quota attainment. Building a relationship with a product engineer requires actually knowing PostHog, not just knowing *about* PostHog.

Ultimately, we want someone who we'd want to buy from.

Sales hiring process

Culture interview

This is our usual first-round interview with a member of the People & Ops team.

Technical interview and demo

The small team interview with the relevant team lead usually lasts 45 minutes. The majority of this session will be a demo role-play so that we can assess how you talk about your current product to a prospective customer who knows nothing about it. You can assume that the customer is a prospective buyer but otherwise knows nothing about your product, and you should approach the demo as if they were a real prospective customer. After a short introduction, we will jump into the role play where you should aim to present and demo your product for 20 minutes with 10 minutes of questions and answers at the end. After this, we will go over some technical interview questions and allow you to ask anything that's on your mind.

Culture and motivation interview

In this 30-minute interview, you'll meet with Simon, who will be trying to answer "Are they a good cultural fit for the Sales team at PostHog?".

Sales SuperDay

The final stage of our interview process is what we call a PostHog SuperDay. This is a paid full day of work, which we can flexibly arrange around your schedule.

We will share the task with you at the start of the day. The task is representative of the work someone in this role at PostHog is doing, and it is always the same for each candidate, so we can make clear comparisons. It will typically involve doing actual PostHog work, e.g. prioritizing customers, doing a demo, etc.

A Sales and CS SuperDay usually looks like this (*there is a degree of flexibility due to time zone differences):*

- Kick-off session
- Meet with Tim, who will be trying to answer "Would I buy from this person?"
- Meet with Charles, who will be doing a culture and vibe check.
- Time to focus on the task, we can provide support via your personal Slack channel (use the channel, don't slide into people's DMs)
- PostHog demo role-play with the team lead and Simon
- Meet a few members of our team for a quick chat

Overall, you should spend at least 80% of your time and energy on the task and less than 20% on meeting people, as we will base our decision on your output of the day. However, we encourage everyone to use the Slack channel as much as needed for any questions or problems.

> In line with our values and culture, you might get short replies like "step on toes" or "bias for action".

CS and Onboarding hiring process

Culture interview

This is our usual first-round interview with a member of the People & Ops team.

Small Team interview

The small team interview with the relevant team lead usually lasts 45 minutes. For this round, we will use scenario-based questions to assess your technical and customer skills, as well as your knowledge of PostHog. As part of this, we will ask you to give a quick pitch of PostHog (not a full demo).

Culture and motivation interview

In this 30-minute interview, you'll meet with Simon, who will be trying to answer "Are they a good cultural fit for the Sales team at PostHog?".

CS and Onboarding SuperDay

The final stage of our interview process is what we call a PostHog SuperDay. This is a paid full day of work, which we can flexibly arrange around your schedule.

We will share the task with you at the start of the day. The task is representative of the work someone in this role at PostHog is doing, and it is always the same for each candidate, so we can make clear comparisons. It will typically involve doing actual PostHog work, e.g. prioritizing customers, doing a demo, etc.

A CS and Onboarding SuperDay usually looks like this (*there is a degree of flexibility due to time zone differences):*

- Kick-off session
- Meet with Tim, who will be trying to answer "Would I buy from this person?"
- Meet with Charles, who will be doing a culture and vibe check.
- Time to focus on the task, we can provide support via your personal Slack channel (use the channel, don't slide into people's DMs)
- Demo role-play with the team lead and Simon
- Meet a few members of our team for a quick chat

Overall, you should spend at least 80% of your time and energy on the task and less than 20% on meeting people, as we will base our decision on your output of the day. However, we encourage everyone to use the Slack channel as much as needed for any questions or problems.

> In line with our values and culture, you might get short replies like "step on toes" or "bias for action".

Hogpatch operations

`contents/handbook/people/hogpatch-operations.md`

Hogpatch is our San Francisco coworking space, shared with a handpicked group of YC founders who are active PostHog users. Teammates in the Bay Area (or visiting) are welcome to drop in regularly to work, meet users, gather feedback, or join events we host in the space. Here's a quick guide to how it all operates.

, our Office Manager, is on site **Mondays to Thursdays from 9am–5pm**. For any issues while she is not in, please reach out to the `#project-hogpatch` channel or DM her directly.

Building access

PostHog teammates don't need an invite to use Hogpatch — the space is open for you whenever you're in town. If you know your SF travel dates in advance, it's helpful to post in the #sf-bay-area channel so we can make sure the space is ready for you.

- If you are visiting for the first time, ring the **black intercom doorbell** on the front door.
- will be notified and will remotely unlock the door for you.
- and also share intercom access so can open the door for you 24/7.
- There's no check-in or reservations needed, it's a very relaxed setup.

Parking

- We have a garage in the building that allows up to **4–5 cars max**.
- This is for **internal employees only**, and spots are limited.
- Please ask if you'd like a **fob**. If you're driving in as a one-off, let Judy know beforehand so she can let you in/out on the day you visit. Always remember to close the garage door when leaving the premises. If you're worried you have forgotten, Judy can double check this via CCTV.

Housekeeping

- If you notice anything missing or that needs replenishing, reach out to on the `#project-hogpatch` channel.
- A **cleaner** is scheduled to come in **once a week** to tidy up and water plants. This will be updated when we open to YC founders/frequent visitors.
- A **gardener** comes in **every two weeks** to maintain the plants around the space.

FYIs

- **Please turn off the lights** in the building if you're the last to leave for the night. The phone booths have lights that will automatically turn off.
- Remember to **lock your laptop** when you step away from your screen.
- If you see **unfamiliar faces** (that don't look like YC founders), raise any concerns with Judy.

Hogpatch - Our SF Home for YC Founders

contents/handbook/people/hogpatch.md

Hogpatch is our dedicated coworking space in San Francisco for YC founders in the current batch - a lofty, airy, light-filled warehouse just around the corner from YC. It's invite-only and open 24/7, giving you a reliable place to work whenever you need it. We've set it up because we know how valuable a quiet space is when you're bouncing between office hours, customer calls, and late-night sprints.

Hogpatch is a free space to help your batch experience a little easier. There's no catch! We're not here to sell you PostHog. We hope that this becomes a dependable home base in the Dogpatch neighborhood for when you need one.

What's Inside

- **24/7 coworking space:** bring your cofounder and get heads-down in deep product work or last-minute demo day prep.
- **10 Gbps fiber internet:** the fastest wifi in the neighborhood to help you build at lightning speed.
- **Private desks and high-res monitors:** space to work comfortably and focus without distraction.
- **Comfortable phone booths:** quiet spots for user feedback or investor meetings (and perfect for quick calls).
- **Limited edition merch:** free, exclusive PostHog gear you won't find anywhere else.
- **Events space:** occasionally used for founder-focused gatherings, but most of the time open as extra room to spread out.
- **Snacks, coffee, and extra comforts:** endless caffeine on tap.

Getting Started

Access

Hogpatch is invite-only, and isn't open to the whole batch. We keep numbers low so it stays calm, focused, and actually useful. When space opens up, we handpick a few YC companies to join. If we want to tap you on the shoulder with an invite, you'll hear from us directly.

- We do maintain a waitlist, but you can't apply - we handpick founders when there's room to welcome more.
- Each invite lasts 3 months. If you join at the start of your batch, that'll usually cover you through demo day. If you join later, you're still welcome to keep using Hogpatch after your batch ends.

Passes

Once you've been invited, Judy (our office manager) will send you and your cofounders a digital wallet pass. The pass gives you 24/7 self-service access to the space, ideal for late-night sprints or weekend hacking sessions. You can come in anytime, day or night. There's no check-in or reservations needed.

Location

Hogpatch is just 100 yards from YC's office, with the nearest Muni stop at 20th Street. You'll find us behind a discreet door off 3rd Street - the exact location is listed on your digital wallet pass. Scan your pass on the front door QR reader to get in, or buzz the intercom for help.

Support

You'll bump into our product engineers from time to time. They're in the space because they enjoy chatting with founders, and they're happy to give product feedback or help set up your dashboards ahead of demo day.

, our office manager, keeps things running smoothly 9am–5pm, Mon–Fri. Outside those hours, treat the space like it's your own. If anything urgent comes up, Judy's your go-to. Her details are in your welcome email.

Workspots & hangouts

- **Desks & phone booths** - no booking system, no hassle. Just grab a spot when you arrive.
- **Visitors:** you're welcome to bring in customers, investors, or anyone else you need to meet with, but the space is dedicated to working so we ask you not to bring friends or un-invited YC founders. The space is yours to use - just don't host a house party here.
- **Events:** every few weeks we'll host something in the space. You're welcome to stick around or join in, but we'll always do our best not to disrupt your focus.

Are you trying to sell me PostHog?

Not at all. Hogpatch is a perk for select YC founders who already know us through the Bookface deal. We know you're focused on building your company, not listening to pitches - so think of this space as a convenient homebase whilst you're hopping around the Dogpatch area, not a sales funnel. We went through YC too, and wanted to create a space that takes off some of the stress whilst you're going through a batch.

Offboarding

contents/handbook/people/offboarding.md

Offboarding team members is a sensitive time. The aim of this policy is to create transparency around how this process works. This offboarding policy *does not* apply to regular contractors who are doing short term work for us.

Voluntary departure

In this case, the team member chooses to leave PostHog.

We ask for 30 days of notice by default (unless locally a different maximum or minimum limit applies), and for team members to work during that notice period. This is so we have some time to find someone to hire and to enable a handover. Please assume by default we will expect team members to work all of this period.

> If you are a current team member and you are thinking about resigning from PostHog, we encourage you to speak with your manager or the to discuss your reasons for wanting to leave. While we don't want to persuade anyone who is unhappy to stay, you may find that the best solution involves changing things here at PostHog, rather than going somewhere else. If resignation is the only solution after you have discussed your concerns, please send an email communicating your intention to resign to people@posthog.com. We will then start a discussion around what is needed for the handover.

Involuntary departure

In this case, we are letting the team member go.

This is generally for performance reasons or because the company's needs have changed and the role can no longer be justified. If the decision is down to performance issues, we will have already communicated feedback to the individual and given them time to take the feedback on board. However, performance issues sadly can't always be resolved, which means we might ultimately need to end someone's employment.

Tim and James are responsible for making any final decision to let someone go.

We use the following general process for managing people whose performance isn't up to the right standard. We modify this slightly depending on the specific nature of the role and how long they have been at PostHog, so the process isn't identical in every case:

- Typically, a manager or member of the exec team will raise that a team member isn't meeting our performance expectations. They discuss what the issues are and a plan to improve. The relevant exec team member follows up with the person's manager.

- The person's manager has a meeting with the team member to let them know explicitly that their performance is not meeting expectations, and that if it continues then they will not be able to continue at PostHog. This meeting may include a member of the exec team, depending on role.
    - If the person is a manager, we usually collect feedback from their team beforehand.
- We outline to the team member exactly what good performance looks like. We collaborate with them to come up with a plan (e.g. specific things to ship), and a timeline for improvement. Usually this is a few weeks for someone new to PostHog, but may be longer if the person has been at PostHog for a while. We schedule a time for a follow up conversation at the end of this period.
    - If the person doesn't accept the feedback at the time and/or we don't feel like there is a realistic path to them improving, we may follow up to let them go sooner than this.
- At the follow up meeting, we either confirm that performance has improved and they are on the right track, or that we have decided to let them go right away.

In cases where a team member's role can no longer be justified, we usually make a decision as an exec team and then let the team member know straight away - unfortunately it is not feasible to let someone know that we are thinking of getting rid of their role.

In either case, we will usually ask the team member to stop working immediately. Final pay and severance are calculated as below.

> If a team member wants to resign but is deliberately trying to get let go so that they receive 4 months' severance, we may treat this as a material breach of your employment contract, which is gross misconduct. In such cases, team members are not eligible for any severance beyond the statutory minimum where they live.

Communicating departures

In the case of voluntary departure, we will ask the team member if they wish to share what they're up to next with the team. If you have resigned, please speak to the relevant team Blitzscale member to agree on who will communicate you are leaving. Please don't announce your resignation until the relevant member of the Blitzscale team has given the go ahead as they may need to prepare accordingly for the impact of your resignation.

In the case of involuntary departure, we will aim to be as transparent as possible about the reasons behind the departure, while respecting the individual's privacy.

Please be aware that PostHog cannot always provide context around why people are leaving when they do.

References and employment verification

At PostHog, we keep things simple: we don’t give personal or professional references for former team members. When someone asks about a past or current employee, the only information we share is their dates of employment.

Expectations for current employees: If someone contacts you directly about a current or former employee:

- Don't answer the request yourself — just forward it to the People & Ops team.
- Don't share opinions or details about someone's performance or why they left.
- Don't speak (or appear to speak) on behalf of PostHog.

This helps us keep things consistent and protects everyone's privacy.

The offboarding process

For team leads

If a team lead has resigned, the Blitzscale team should figure out who will take on the team lead responsibilities and have that prepared to let the team know just before the resignation is announced or as part of the announcement.

For involuntary leavers, we will schedule a call. During the call, someone on the ops team needs to complete the offboarding checklist.

We will then send over an email covering the following points with the team member:

1. Final pay
2. Share options vested
3. Company property
4. Business expenses
5. Personal email to the company (optional)

Final pay

Final pay will be determined based on length of service and the reasons for leaving:

- If the offboarding is voluntary, they will be paid up until their last day. We will look at the amount of holiday taken in the last 12 months and will pay any "unused" vacation pay assuming they would have taken 25 days (since we offer unlimited vacation periods).
- If the offboarding is involuntary and due to performance reasons or a change in business needs, they will receive 4 months of pay. This includes any notice period we're legally required to provide. To qualify, they must have been at PostHog for at least 3 months (6 months for sales roles). For our teammates in Germany who've been with us for at least 6 months, we'll follow local laws and standard market practices for notice and severance. If they have been with PostHog for less time, they will receive 1 month of pay and, if a US team member, we will also cover healthcare costs through the end of the next calendar month.
- If the offboarding is involuntary and for gross misconduct, including breach of contract, they may be paid the statutory minimum required only, and receive no notice. This is at our discretion depending on the circumstances.

> We ask departing team members to sign a post-termination certificate, separation agreement or release in order to receive payments beyond their final day of work. If we do not receive this, then we will only pay in line with statutory and contractual requirements.

Please note that if there are local laws which are applicable, we will pay the greater of the above or the legally required minimum.

Share options vested

If a team member has been allocated share options, we will confirm how many have vested and the process by which they may wish to exercise them. We have a team-friendly post-departure exercise window of 10 years, and most team members who leave will be deemed a 'good leaver' unless they have been terminated due to gross misconduct.

Offboarding checklist

This is maintained as an issue template in GitHub. The People team will create a new offboarding Issue for each leaver.

Onboarding

contents/handbook/people/onboarding.md

Welcome to PostHog!

Giving a new joiner a great onboarding experience is super important to us. We want new joiners to feel they've made the right decision to join us, and that they are excited and committed to what we're doing as a company.

> Want to introduce a new joiner to the People team for onboarding, but don't know who on the team does what? Just introduce them to people@posthog.com and a member of the team will jump in and take it from there!

Our team is spread across the world, and so are our new joiners. In order to ensure the best possible onboarding experience, we aim for the new joiner to meet up with someone from their team in their first week. Depending on the new joiner's location, they might fly out to one of our team members, or the other way around. So the onboarding experience will look a little bit different, depending on where the new joiner is based and which team they will be joining.

Onboarding checklist

This is maintained as an issue template in GitHub. The People team will create a new onboarding issue for each new joiner.

Onboarding email

We send an introductory email to all new hires to welcome them to the team and ease them in the some of the essential actions we need them to take. This needs communicating openly, as users may not be able to access the `company-internal` repo yet. So, we send them an email.

> Once you've joined PostHog, we will **not** use email for communicating with each other. For example, James or Tim will never ask you to do something critical over email only – they'll always confirm it over Slack, and so will everyone else. Be extremely cautious of direct emails from James, Tim, or other people of PostHog.

The onboarding email is sent by the People team directly. We want to strike a balance between sending attractive, personalized emails and avoiding creating process or using overpowered tools, such as Customer.io or Mailchimp. So, we landed on a simple email with the necessary links.

This doc is a suggested template with important actions specified, though we recommend personalizing it to the individual. We've linked to these as docs and direct images to make the formatting easier for you, but here is an accompanying image for use in emails.

Onboarding buddy

Every new joiner at PostHog has an onboarding buddy. If possible, a new joiner will meet their onboarding buddy in person during their first week. In case in-person onboarding isn't an option, we will make alternative arrangements. The onboarding buddy is usually a member of the team a new joiner is joining - ideally the team lead - and they can help with any questions that pop up and with socializing during the first couple of weeks at PostHog. Of course, everyone is available to help, but it's nice to have a dedicated person to help.

Guidance for onboarding buddies

- Once we have decided which team a new joiner will join, the People & Ops team will reach out to the team to find an onboarding buddy. *Please make sure that you don't have any leave booked in the week before and the two weeks after the new starter joins*
- We will intro the new joiner and the onboarding buddy via email - please say hi and decide together where and when the in person onboarding will happen.

> If any travel is needed for the in-person onboarding, please check our Spending Money page and book your travel accordingly. *You don't need to let us team know, just use your Brex/Revolut card.*

- Please make sure you spend at least 3 days together, working through the first week onboarding list and spending time working on any role-specific tasks that are outlined in the new joiner's personal onboarding issue.

    - Make sure to add the details of the in-person onboarding to the In-person Onboarding Calendar so that other PostHog team members can join, if possible. Simply create an event in your calendar and then invite the in-person onboarding calendar as a guest.
- You will remain the new joiner's main point of contact for the first few weeks, so please continue to check in with them at least once a week for the first month or so.

In-person onboarding

Except under special circumstances, new joiners meet with members of their team in-person to go through the onboarding process. Upon acceptance of an offer, your Team Lead will notify the People & Operations team who will help you coordinate travel if necessary. We encourage team leads to consider the Hedgehouse as a location for in-person onboarding. Regardless of location, everyone should have their own bedroom.

In these cases, the process is:

- Preemptively create the new team member a Google account
- Issue them a Brex card to their work email with a sufficiently high temporary balance to cover travel costs
- Have the new team member book travel as usual

While there is no fixed budget for onboardings they should be relatively less expensive than a small team offsite, which is $2,000 per person. Some considerations to reduce the cost:

- Avoid intercontinental travel or choose a location that limits it to the minimum number of people possible
- Consider doing more casual social activities that are less expensive: dinners, drinks etc
- You can request budget for the team lead +1 more team member as an onboarding budget, any other team members joining can use their working together budget (be mindful that onboardings are distracting so the more team members you have join, the less productive the team will be that week, you also have offsites for the team to all get together).
    - Create a Slack channel for the onboarding (`#onboarding-[who]-[team]-[month]-[year]-[where]`) add everyone who's going
- The new team member already has their own onboarding budget to book their flights and accommodation, so do not include them in the budget.
- See if there are any other onboardings at the same time you could pair up with

Aim to keep things sensible and cheap. As always, use your best judgement when spending money. Request a budget in Brex in USD for any onboardings you are doing. There will of course be some exceptions to this, please just include the reasoning in your brex budget request, and ensure to list who the budget request is for.

> You should by default avoid combining in-person onboarding with small team offsites as they serve different purposes. The focus of onboarding is generally on making the new team member successful, but offsites feature things like hackathons and 360 feedback which aren't usually

> helpful for this and detract from useful onboarding time. However, it may occasionally make sense to combine the two - just use your judgement.

It is important that you make the most out of the sync time with the new joiner on your team. You should not spend the whole week sitting next to them doing your usual work. Having something planned each day is sufficient, some ideas include:

- An intro to PostHogs values & strategy
- A history of your team, your current quarterly goals and a product demo of the features you own (product teams only)
- Interview feedback session between the team lead and new joiner
- Deep dive into a specific feature where you walk through the code (product teams only)
- Mock demo (sales team only)
- "No stupid questions" where the new joiner is expected to come with questions they had since starting

Your first week

Your first week can definitely be a bit overwhelming at any new company, so here's what you can (roughly) expect!

- You will meet (either in person or virtually) your team lead to discuss goals and aims over the next 30/60/90 days and beyond
- You will get all your equipment set up and get access to all the accounts you need
- You will receive your new hire kit (which includes No Rules Rules which we encourage everyone to read as it gives you a great insight into how we work as a company)
- You should try and set up a few calls with a range of people to introduce yourself
- You should try and speak to some actual users of your product. Your manager or PM will help you set these up, and this can be a great source of things to work on in your first week.
- You should dive straight in, fix a typo in the handbook, ship a tiny bug fix, anything to get you going!

Engineering

We hire engineers on a regular basis, running in-person onboarding practically every time. Over the years, we've learned a lot about doing this efficiently and there's much to gain from sharing the knowledge between teams.

Based on this ongoing learning process, here are our **five rules for onboarding an engineer**:

1. Ship something together on day one – even if tiny! It feels great to hit the ground running, with a development environment all ready to go.
2. Run 1:1 learning sessions with the new teammate every day. Give them all the context they need to succeed. By the end of the onboarding, each team member present should've run at least one such session.

▶ Looking for learning session ideas?

3. Do at least one brainstorming session on a topic important for the team, writing down actionable conclusions. Use the time together to discuss issues and involve the new joiner in decisions. 4. Pair whenever possible. You're all sitting next to each other, so pick work that can benefit from in-person collaboration. 5. Have fun, because life isn't all work! Do some sightseeing, go out for dinner, or find a fun activity – just hang out together any way you like.

Tools we use

We use a number of different tools to organise our work and communicate at PostHog. Below is a summary list of the most important ones - this list is not intended to be exhaustive

Everyone

- Google Suite - Gmail, Google Apps such as Docs, Sheets, Slides
- GitHub - most comms and product work
- Slack - we have an internal workspace and a users Slack as well
- Brex (US, RoW) or Revolut (UK, EU) - company cards and expenses tracking
- Shopify - powers our merch store
- Time off by Deel (Slack App) - holiday tracking
- Bamboo HR - payroll and benefits (US)
- Deel - contractor & EOR payroll & HRIS

Engineering

- AWS
- Incident.io
- Heroku
- Grafana

Design

- Figma

Ops, People & CS

- Salesforce - customer CRM
- Zendesk - our support platform
- Mosaic - financial modelling
- Carta - cap table management
- Fondo - US accounting
- Deel - international payroll and contracts management
- Ashby - recruitment
- Micromerch - merch inventory management, YC onboarding merch, and merch drop-shipping for small events

Signatories

Charles, James and Tim at this time are the only people able to sign legal paperwork on behalf of the company.

How we work

Now it's time to dive into some of the more practical stuff - these are the most important pages:

1. Communication - we have a distinctive style. If PostHog is your first all-remote company, this page is especially helpful.
2. Team structure - we are structured in Small Teams. These pages will help you get the lay of the land, and who does what.
3. Management - we have a relatively unusual approach to management, and it is possible that you will not be familiar with our approach.

Working in GitHub

We use GitHub for *everything*, including non-engineering task management. This might take some getting used to if you are non-technical. If that is the case, we have a detailed guide on how to set up a local version of Posthog.com so that you can make changes to the docs, handbook and website and a blog about why we use GitHub as our CMS to help you out.

Our most active repositories (aka 'repos') are:

- PostHog - main app
- PostHog.com - website
- Product Internal - product-related issues that need to be kept internal, e.g. security issues, customer-specific issues (private)
- Company Internal - company-facing issues, e.g. internal processes, hiring planning (private)

When you have a new Issue or want to submit a Pull Request, you do that in the relevant repo.

We use GitHub Projects to track the status of Issues in an easily viewable way. When you create an Issue, you can assign it to a Project - think of a Project as a way of organising and filtering Issues in a particular view. This makes it easy for Small Teams to easily track what they are working on without having to jump between different repos. Some Issues may be assigned to multiple Projects if they involve the work of more than one team.

You can also assign an Issue to a specific person, and tag it with a relevant label - use these to help people filter more easily.

Each Small Team has its own Project for tracking their Issues - full list here. Most teams run two week sprints - as part of onboarding, you will be invited to the relevant planning meetings.

Support hero training

Employees are occasionally called upon to act as support heroes, or need to deal with support tickets that are escalated to them. This most often applies to engineers, but can include any employee regardless of their team. For this reason, we need everyone to have a broad idea of our support processes and know how we deal with customers.

All new hires should schedule a 30 minute session with the support engineer closest to their timezone within their first three weeks at PostHog.

In this call the support engineer will be able to answer any questions, as well as demonstrate how we deal with support at PostHog. In particular, the support engineer should cover:

- [ ] What the role of a support hero is and how they can expect to receive tickets/escalations
- [ ] An overview of where tickets come from and how to differentiate between paying/free users
- [ ] How to create tickets from Slack threads and reassign tickets to other teams
- [ ] Advice on how to communicate with customers and prioritize tickets
- [ ] How and when to mark tickets as 'On Hold' or 'Pending'
- [ ] What our SLAs are and what ticket severity indicates
- [ ] HogHero - how to deal with bug reports and feature requests, and how to merch customers (including macros)
- [ ] How to avoid duplication of effort in Zendesk
- [ ] Which views should be used in Zendesk
- [ ] How to use side conversations in Zendesk

It can be especially helpful for new hires if support engineers demonstrate how to solve a few simple tickets from start to finish, through shadowing.

30/60/90 day check-ins

Managers are responsible for helping their new members navigate the first 3 months probationary period. There is a strong importance on 1) providing feedback to the new team member, and 2) communicating with execs about unresolved performance issues, so that there is enough time for action. Managers are, again, not responsible for hiring or firing, nor communicating these possibilities directly to teammates - this is handled by the exec team, and is frankly a very rare situation - the vast majority of people we hire do pass their probationary period!

As part of the onboarding checklist, the Ops team will schedule reminders for a new team member's manager at the 30, 60 & 90 day mark to serve as a reminder that these checkpoints have arrived and to make sure everything is on track for the probationary period to be passed.

- 30-day checkin
    - Manager to provide initial feedback to the team member - especially if there is constructive feedback that needs to be given to ensure the person passes probation.

    - It's also a good time to reinforce the positive work that has been done by somebody on the right track.
- 60-day checkin
    - Ops team will check in with the manager to see if things are on track.
    - Manager to provide another round of feedback to the team member.
        - If things are going well, the manager might want to give an indication of this as it can ease any fears the team member may have.
    - Exec team will get involved as necessary to provide an additional layer of feedback to the team member.
- 90-day checkin
    - We've made it! Congratulate the new team member on passing their probationary period.
    - Give any extra feedback as necessary, though things *should* feel like they're humming at this point.

Feedback is a really important part of the onboarding process and as a manager it's a good idea to ensure the new team member receives feedback from their peers - either from you collecting it or them receiving it directly from their peers. It won't always be possible or necessary to do a 360 feedback session within the first 3 months, so it's up to you as a manager how best to approach that. As a manager you can also have blind spots on performance, so checking in with their peers can be helpful and can be done during your normal 1-1s.

These check-ins are designed to ensure every new starter is set up for success. Every manager will deal with these slightly differently, but it will hopefully be clear to everybody by around the 60 day mark how things are going and what needs to be worked on, if anything. It is important for a manager to ensure that they do not wait for one of these check-ins to communicate with an exec team member that there could be issues with the team member passing probation. They should let them know immediately, so that a fair and reasonable plan can be put in action ASAP.

If you have any issues or any feedback on how to improve a specific intro just post in the #team-people-and-ops slack channel and tag the relevant people

Finding answers at PostHog

Need help finding something? We have a strong culture of self-sourcing answers - it helps you get unblocked faster and builds your intuition for where things live. Start with #ask-max, our AI that's read every handbook and documentation page. It'll point you to relevant docs instantly and is available 24/7.

Of course, if you're stuck or need context beyond what's documented, just ask in the relevant channel - people are always happy to help. The goal isn't to make you figure everything out alone, it's to give you the fastest path to answers.

Slack Channels

Below are a list of Slack channels you may find helpful:

Work-related channels

- `#ask-max` - Max has access to all of our documentation and our handbook, and is a great place to start with many questions
- `#content-docs-ideas` - for suggesting ideas for the newsletter, tutorials, and docs to be written by the content and docs team
- `#brand-mentions`
- `#do-more-weird`
- `#newsletters`
- `#team-blitzscale`
- `#dev`
- `#general`
- `#alerts`
- `#industry-news`
- `#changelog` - keep up with all the cool things we're shipping across the team

Social channels

We encourage you to join and create channels focused around different types of hobbies and interests. We explicitly don't allow channels based on categories that we legally (and rightly!) can't discriminate against in the hiring process, such as gender, sex, political affiliation, religion, and age.

- `#food`
- `#kids`
- `#no-context-posthog`
- `#random`
- `#whereintheworld`
- `#devel-random`
- `#books-and-films`
- `#climbing`
- `#coffee-snobs`
- `#dad-jokes`
- `#fitness`
- `#hoglife`
- `#rockets`
- `#stonks`
- `#cycling`
- `#listening-to`
- `#design-inspiration`

Location specific channels

- `#london`
- `#germany`

- `#sf-bay-area` etc.

Overview

contents/handbook/people/overview.md

The Ops team's primary goal is to make PostHog an incredible place to work by removing distractions from other small teams. We keep PostHog running smoothly without implementing lots of unnecessary processes.

We are also responsible for growing the team by adding in world class talent to new and existing small teams. We want to do this while retaining our world-class team by making PostHog the most transparent company in the world, and the best place for people to work in general.

Ops provide all the tools, literally and metaphorically, needed for our team to come in and do their best work. Practically, this looks like:

- A small ops team covering a wide range of disciplines that can react to any incoming items to allow the rest of the business to focus on what they do best - building products.
- Nailing the basics of working at a start-up for our team members. Making sure things like payroll, onboarding, offboarding etc. all work smoothly and are on autopilot as much as possible.
- Thinking slightly further ahead than the “here and now" to predict when we may need to make changes like hiring new team members, changing our spending patterns to manage cash, managing our comp structure, implementing a new tool etc.
- Setting a very high bar for bringing people on board. We are always looking for the best people in their field, or people on their way to becoming exceptional at their jobs. This is so important to us, it's enshrined in our company values.
- Manage compliance projects like SOC 2 or HIPAA to help us land larger customers. These projects require input from some small team members, but Ops will make sure everybody knows who and what is needed.
- Partnering with the Exec team to work on people initiatives to build a diverse and inclusive culture at PostHog. We want to put a strong sense of belonging at the heart of everything we do.
- Running any disciplinary or grievance process that may occasionally arise.

These are some things that Ops is *not* responsible for which you might see at other companies

- Resolving performance issues and creating things like performance management plans. This is the Exec team's responsibility, working with the relevant managers.
- Booking travel or accommodation for when you travel. You have the ability to do this yourself and we trust you will spend money carefully. The exception to this is our company offsite where we book accommodation.
- Setting compensation - Exec team do this.
- Centralized customer support - this sits with the Customer Success team generally, with individual small teams having a support hero on rotation.
- Company wide goal setting - the Exec team ensure this happens, and small teams run their own planning

- Training - this is self serve

Ops team values

- Take ownership
- Be supremely reliable
- Act with care & compassion

Take ownership

When something falls on the Ops team, we make it very clear we are the owners of that specific thing. We communicate clearly with other teams when we require their input and we make it as easy as possible for them to help us achieve the desired outcome. We are quick to triage things that don't have a clear owner and we get them into great shape before we expect others to have to interact with it. This could be anything from a compliance matter to how our merch process works. We say 'here's how this could get done' rather than 'that's not my job'.

Be supremely reliable

If we say will take care of something, we take care of it - no exceptions. We are often trusted with big and small things, and we take all of them seriously. This also means we will keep you in the loop if something can't happen as we originally intended. We are trusted with a lot of sensitive information, from our team members' personal details to specific company info that we need to protect. When people trust the Ops team with something, they need to know it's getting done properly.

Act with care & compassion

The Ops team has got your back. We treat everybody with respect - caring deeply about the success of the business means caring deeply about the success of every team member, irrespective of things like seniority. We want to be sure we will be proud of how we handled any situation. This doesn't just apply to our team members but to anybody interviewing, the customers we deal with, and anybody else we interact with externally such as suppliers.

Philosophy club

contents/handbook/people/philosophy-club.md

Philosophy club runs once a month, and organizes it. We spend each 30min session discussing one philosophical question in the Socratic tradition. A short text is shared in advance as light pre-reading from the Stanford Encyclopedia of Philosophy. These can be a bit dry, so feel free to use alternatives - the Crash Course videos are quite good.

If you're interested in joining, ask to add you to the recurring event in `#team-people-and-ops`.

Structure

Each session is 30min:

- 5min — framing: one person summarizes the pre-read in plain language
- 10min — clarification: we define the terms in the question
- 10min — probing: share examples from real life
- 10min — synthesis: each person states if their belief has altered or not

The only rule is no observers - if you join a session, you should expect to take part.

Topics

See below for the full roadmap, with a link to each pre-read. This is a 1 year commitment if you want to do the whole thing, but each question stands alone, and you can attend as many sessions as you want!

1. What counts as knowledge?
2. Can we trust our own reasoning?
3. If behavior is shaped by causes, what does accountability mean?
4. Is success about happiness, achievement, or meaning?
5. Are habits more important than intentions?
6. Should decisions prioritize results even when methods feel wrong?
7. Is fairness equality, merit, or need?
8. When is challenging authority ethically required?
9. Is trust a rational calculation or a leap without evidence?
10. Does technology shape our behaviour or simply reflect it?
11. Should work be intrinsically meaningful or primarily instrumental?
12. Do we discover purpose, or create it through choices?

Product Manager ramp up plan

`contents/handbook/people/ramp-up/product-manager.md`

This is a rough guide to ramping up as a product manager in PostHog.

Timeline

Day 1

Outcome: Get started

- Get set up and follow your onboarding checklist
- Do your first analysis in PostHog (e.g. Who are our biggest customers? what's the most used feature?)
- Set up time to meet everyone in your team and understand their current strategy, motivations and risks
- Read up about the OKRs of your team
- Attend the Company All Hands

Week 1

Outcome: Get stuck into execution

- Join your teams' standups
- Learn about their current projects
- Arrange and host 2+ calls (through customer success initially) and get feedback from customers about ongoing projects
- Use PostHog to gather data to support with executing existing projects
- Share an interesting finding from PostHog in the demo section of the company all hands

Month 1

Outcome: Your teams are hitting their goals faster

- Finding opportunities to reduce scope and increase impact of big projects
- Giving the team the context they need to design and build really amazing solutions to customer problems
- Enabling the team move faster by finding and removing bottlenecks

Quarter 1

Outcome: Hit your goals and set the strategy for the next quarter

- Pull out the stops to get your team across the line with their existing goals

- Work with leadership and your team to define ambitious goals for next quarter
- Use data and customer context to rationalize priorities

Specialism

As well as your day job with specific teams, its important we have PMs having company level impact across the following specialisms too.

Analytics

- You're performing analyics on how customers use our products outside of your team's scope
- This analysis defines how the company priorities what to build across product
- You push the limits of what's possible in PostHog to help us build more advanced tools and you work with SQL to get answers to the most complex queries

Customer Research

- You're always talking with our customers, and you intimately know how their frictions with our products
- You're giving customer insights to every team (in and outside of product) to help them prioritize better
- You're the first to hear about new opportunities and problems that our customers need solved and you capture this information

Growth

- You work closely with growth engineering and leadership to understand how we can accelerate activation and revenue growth
- You know inside-out our funnels for growth and provide the context for to make quick changes to validate hypothesis and grow faster

Side gigs

contents/handbook/people/side-gigs.md

PostHog looks for passion in the people it hires. This often correlates with people who have side projects as a hobby. For example, we view pre-existing open source work as a strong qualifier that you're good enough at programming that it's fun to do rather than frustrating and hard!

These side gigs may sometimes earn you money. Sometimes, you may one day want your side gig to become your main gig.

We have deliberately called them "side gigs", as we are ok with you earning money on the side. We are not ok with this being your main focus and PostHog being just a paycheck. Quite simply, we are too small for PostHog not to be your main motivation. For this reason, we also currently don't offer part time work as an option at PostHog.

Managing time

The key distinction to something being a side gig, and thus it being appropriate, is its impact on your work and the amount of time involved.

A few hours a month on a paid side gig is acceptable. In any case, side gigs should by default be something you work on in your personal time, and they should not impact the work you do at PostHog.

In a few cases, you may want your side gig to become your full time work one day. That is ok - please just let us know, so we can create a plan. We know the key to motivated people is to help you achieve your long term goals, and to align this with what PostHog needs, whether or not you eventually achieve them with us.

Above everything else, if you are going above and beyond for PostHog and you're still able to look after yourself properly, side gigs (whether paid or unpaid) are totally fine. We don't think that's possible beyond a certain level of time/energy commitment to them, but we are very happy for you to spend a little time on them each week.

Intellectual property

Just to reassure you, PostHog won't try to claim ownership of any intellectual property (IP) you create in your personal time, e.g. if you are contributing to another open source project as a hobby. However, you need to be *really* careful that you do not introduce any of PostHog's non-open source IP into any project that you work on - this can cause serious legal headaches. As a rule, anything from PostHog that is explicitly MIT-licensed is fine to use, anything else is not.

Ideas that start at PostHog

If an idea, project, or product comes out of your work at PostHog (for example during hackathons, offsites, team projects, or internal experiments), it should be treated as PostHog work by default.

When thinking about whether something is truly a personal side gig or PostHog work, it's important to consider where the idea originated, who it was built for, how it's been shared or used internally, and whether PostHog data, tools, equipment, or infrastructure are involved.

If you'd like to take an idea that started at PostHog and develop it as a personal or external side project, please get explicit sign-off first so we can avoid any confusion around ownership, data use, or future plans. Without that sign-off, these projects should live within PostHog repos and follow PostHog processes.

If you are ever worried about this, please talk to Fraser and he can help you figure out the best solution here, especially if what you are working on directly competes with something PostHog has built or is on our roadmap.

Getting signoff

We ask you to please just check in with the relevant Exec team member for your team (ie. James, Tim, or Charles) to get their confirmation before going ahead with any side gigs.

If your side gig existed before you joined PostHog, this will usually be covered as part of the onboarding process.

Spending money

`contents/handbook/people/spending-money.md`

There are many occasions when you will need to spend company money. PostHog is a lean organization - the less we spend, the more time we have to make sure the company takes off. However, it is more important you are productive, healthy, and happy.

Guiding principles

We have a context-based expense policy inspired by the book 'No Rules Rules'. We're empowering team members to make good decisions while maintaining transparency and accountability.

Ask yourself:

- Can you clearly explain why this expense is in PostHog's best interest?
- Would you be comfortable explaining this expense to the entire team publicly?

If the answer is **yes**, it is likely the expense is in the best interest of the company and supports your productivity.

If not, think again.

The goal here is to empower you. However, when in doubt, ask your team lead for context, not permission.

- Still in doubt? Ask Janani in #team-people-and-ops.

How it works

- All company expenses (offsites, software/tool subscriptions, merch, etc.) will have common company-wide budgets.
- You'll be assigned a single `User Limit` of $5,000 per month in Brex from which you can spend money on individual subscriptions, coworking/collaboration, equipment (except laptops and Mac Studio Monitors - ping `#team-people-and-ops` for these), training, etc. If you need an increase in the limit, request it on Brex.

Transparency & accountability

- All expenses are visible company-wide
- If we have questions about an expense or need documentation, we'll reach out for clarification
- Team leads get monthly reports showing all direct report expenses

Logistics

UK employees

Use your Revolut if the expense is over £75 *and* has UK VAT on it. If not, use your Brex.

The company can claim back VAT on these larger purchases - the more money we claim back, the more money we have to #DoMoreWeird

Please make sure that the invoice is addressed to Hiberly Ltd and our registered address (this can be found in the Important Company Details sheet).

Receipts

All expenses over $75 or £75 must have itemized receipts attached and memo updated within 14 days of the charge, as this is what our auditors require.

- Brex: upload the itemized receipt using the Brex app, text, email or Slack, and update the memo using the Brex app, text, email or Slack.
- Revolut: email your itemized receipts to ukinvoices@posthog.com along with a memo in the body of the email

In extreme cases, expenses with no receipts above $75 or £75 may be deducted from your pay if we can't verify the business purpose - this is mainly for repeat offenders where someone's clearly ignoring the policy.

We need an *itemized receipt or invoices* because Brex auto-verifies these using the information on the file. For example, this means the full booking confirmation email for flights/hotels, detailed bill from the restaurant for a team dinner etc. Please do not upload cropped images that show just the amount or just the credit card machine confirmation slip - without context, the receipt is pointless

> Template for a thorough memo
>
> - What: [item/service purchased]
> - Why: [business reason - how this helps PostHog/your work]
> - Context: [relevant details - who attended, what project, etc.]

Reviewing expenses

Finance reviews:

- All expenses over $1000 (Brex)
- Random sampling of expenses under $1000 (Brex)
- All Revolut expenses

Team Leads

You'll get monthly expense reports for your direct reports. You have context Finance doesn't, which will help justify spending decisions to the auditors. How much you dig into these is up to you - the goal is catching patterns, not micromanaging.

Why documentation matters

Missing receipts and memos create extra work during audit season - auditors will need to dig into those expenses, which means hours of back-and-forth with you about charges from several months ago! A quick receipt upload and memo update now saves everyone time later.

We're legally prohibited from paying personal expenses on behalf of team member, and are at risk of penalties/fines if this happens.

> The flexible, trust-based policy we have only works when everyone maintains proper documentation. If team members consistently fail to upload receipts and add memos, we'll either need to implement rigid pre-approvals for all spending, or treat repeated violations as performance management issues. Which we really don't want to do!

Budget structure on Brex

We're not going to police your spending with hard limits but we will continue to use budgets and limits on Brex for audits, and because categorizing this stuff properly is essential for things like Board reporting, tax compliance, and seeing how we're doing against our budget.

- Offsite budgets - managed by Kendal:
  - Whole company offsite
  - Mixed team offsites
  - Small team offsites
  - Onboardings
- Company software budget
  - All subscriptions/tools used at a company level - (Janani should be default admin for all of these, so she can handle receipts etc.
- User Limit of $5,000 per month, per user
  - This covers all co-working/collaboration costs, training, individual software subscription, etc.
  - For subscriptions and other recurring monthly expenses, we recommend using the virtual card under your User Limit to ensure it gets categorized to the correct budget by default

Joining an offsite? *Only use the offsite budget*, not your User Limit - it helps the People & Ops track travel spend accurately against budgets. Let Kendal know in #team-people-and-ops.

Frequently asked questions 🤨

- Can I use my personal card for a work-related expense?
  - No, you **must use** your Brex or Revolut for all work-related expenses.
  - The company earns points which we use towards billboard campaigns. The Finance team also doesn't have the bandwidth to process lots of reimbursement requests.
- What if I accidentally used my personal card for a work-related expense?
  - Claim a reimbursement with an itemized receipt on Brex within 90 days of incurring the expense, with a memo (context on what was the expense for and why your Brex wasn't used).
  - We cannot process any reimbursements without a receipt.
- Can I use my Brex/Revolut for a personal expense?
  - Obviously not.
- What happens if I forgot to assign the right spend limit before using Brex?
  - Go into your Brex account and re-assign the charge to the correct limit.
- Can I bulk upload receipts to Brex?
  - Yes, you can! Here are some helpful articles:
    - Attaching receipts to any expense
    - Attaching receipts for multiple expenses
- What do I do if I have a new tool/enterprise software subscription for the team to use?
  - Add Janani as the Billing Admin to manage payments.
- If I'm asked for a billing email for bill payments, what do I use?
  - Use finance@posthog.com
- What if I'm driving for work-related purposes?
  - You can claim a mileage reimbursement through Brex. **Do not** separately expense fuel.
- What if I accidentally used the company card for a personal expense?
  - Login to Brex > find the charge > click on 'Repay' > Repay to the bank account details provided in our banking runbook
  - For Revolut charges, ping Janani in #team-people-and-ops
- How do I get access to WeWork?
  - We have a company All Access account - ask Kendal in #team-people-and-ops.
- What if I've received a bill I need Finance to pay?
  - Submit the bill using the 'Bill Pay' feature on Brex, with context on what it is for, and which budget the bill should be coming out of, once you have verified it.
  - Finance processes all bill payments on Wednesdays.
- How do I upgrade my flight ticket to premium or business using my personal points/card?
  - Book an economy ticket using your Brex, then upgrade afterwards using your personal points/card.
  - If you can only book premium travel using personal miles or points/card then request a reimbursement for the cost of the economy fare in Brex, along with a screenshot of the cost of the economy fare at the time of booking.

How we handle inappropriate spending

Expenses that could be construed as personal will be flagged as non-business expenses by auditors, as they will be considered a taxable benefit that PostHog has provided to you.

Examples of inappropriate spend include:

- Personal expenses during business travel (for example: gym sessions/fitness classes, groceries)
- Entertainment subscriptions for personal use (for example: Spotify, Netflix, Amazon Prime, etc.)
- Expenses that benefit you personally rather than the business
- Anything you wouldn't feel comfortable explaining publicly

If the inappropriate spend was due to a misunderstanding, e.g. you genuinely thought an expense was in PostHog's best interest, but we disagree, we'll provide clarification and context. If you knowingly and deliberately spent money in ways that are not in PostHog's best interest, or tried to intentionally circumvent the guidelines, we will probably treat this as serious misconduct.

Expense guidelines

Equipment

Laptop & monitor

Talk to Tara who handles most Macbook and Apple Studio Display purchases - ping her on #team-people-and-ops.

- Having equipment purchases centralized helps ensure accurate accounting and tracking of fixed assets for the audit.
- We expect you to ship the Macbook and Apple Studio Display back when you leave PostHog.
- Apple Studio Displays are only for Product Engineers (high density screen) and Sales/CS/Onboarding teams (built-in high quality webcam and microphone). For all other teams that feel they could benefit from an enhanced monitor, there are some really great competitors to the Studio Display at a fraction of the price - like the Clarity Pro 27" or another solid option is this LG screen You can purchase these using your personal limit.
    - If you order a Studio Display during your probation period but end up leaving PostHog, we can recover the cost of the display and you won't need to return it.
- Laptop guidelines
    - For engineering roles (product, platform, & support), we recommend a Macbook Pro 14-inch M5 Pro, with the 18-core CPU, 20-core GPU upgrade and 64GB of RAM.
    - For sales & CS roles, we buy the Macbook Pro 14-inch M5, with 10-core GPU, 16-core and the 32GB RAM upgrade.
    - All other roles, we issue Macbook Pros. Wherever possible we will redistribute engineering models (Macbook Pros) no longer in use to allow you to have a more powerful machine for running PostHog Code.

- Apple offers multiple screen sizes. The larger screen sizes (15 inches +), are disproportionately more expensive. If you are realistically going to do most of your work at home, it is more rational to pick a smaller laptop size, and to get a large monitor.
- We only purchase laptops with an English keyboard configuration (US, International or British is fine) - this enables us to easily pass your laptop on to someone else if you upgrade or leave.
- In the unlikely case that you need to purchase your own laptop:
  - Check if Amazon has sales before purchasing through Apple
  - US only: use your Brex since we earn cashback
  - UK only: use your Revolut since we claim back the VAT
  - Do not get AppleCare since it doesn't have great value for money
- You can request a new laptop in `#team-people-and-ops` if it is over 4 years old or significantly impacting your productivity. We do ask that you do some diligence to make sure it's not a setup issue though - i.e. other applications aren't hogging the memory, etc.
- Part of `#team-client-libraries` and need to purchase a phone for testing? Talk to Tara in `#team-people-and-ops`.

Yubikey (for specific roles only)

Passkeys are the preferred way of securing accounts. In some cases Passkeys aren't supported by the service provider.

If you find yourself in a team requiring access to these kinds of tools where a Yubikey is required then you should purchase them as recommended on the on the MFA page using your Brex card. If you aren't sure if you need one then you probably don't and should instead be using Passkeys

Other equipment

- Keyboard/mouse/laptop stand: Check Amazon and Apple for discounts. Refurbished items usually work just fine. Nextstand make great value laptop stands that are portable.
- Desk & chair - again, refurbished is a great way to get high quality for less. If you live in the UK, Office Resale offer a range of like-new refurbished designer furniture.

As a guide, here's what we'd consider reasonable spend:

- Desk - up to $500
- Chair - up to $500
- Keyboard - up to $250

Software

We are *strongly opposed* to introducing new software that is designed for collaboration by default. There needs to be a very significant upside to introducing a new piece of software to outweigh its cost.

The cost of introducing new collaborative software is that it creates another place where todo items/comments/communication can exist. This creates a disproportionate amount of complexity.

Individual software is down to your personal preference, and we encourage you to share cool software.

- There are some tools used by team members individually - if they become more widely adopted, it makes sense to have a company account. Talk to Tara on #team-people-and-ops

You can ask for access to team/company tools by submitted a request in Slack. Find the Zluri app in Slack. type: /accessrequest and press enter. You'll get a pop up that allows you to search for the app you'd like access to. Add any specific information about license level/type if necessary. The request will then be sent to the team member who owns/manages access to the plaftorm. Once they have provided access to the platform, they'll confirm via the Zluri task and you'll also receive confirmation. You should then receive an email invite, or be able to login via SSO depending on the tool. If you do not see a tool in the app that you believe is centrally managed, drop a line in the [#team-people-and-ops] channel

- Some tools that you might find useful to know about:
  - Loom: you'll be added as a *Creator Lite* which allows you to record 25 videos/mo at up to 5 minutes in length. If you need a full Creator account (unlimited videos, advanced features), specify in your Zluri access request
  - Zoom: we use Google Meet by default, but you can use Zoom for free (up to 40-minute calls). Should you need longer meetings, please specify in your Zluri access request (But does anyone *really* need longer meetings?)
  - Superhuman: everyone has their own favorite email app, but Superhuman users will make sure you don't forget theirs. We have a team plan to keep those folks happy with inbox zero and inner peace.
  - Granola: It's absolutely okay to use AI note-takers so you can stay engaged in meetings without writing everything down. Feel free to choose your own but please be aware of who the sub-processors are to ensure they do not use a competitor for analytics.
- IDEs: Visual Studio, VIM and PyCharm are the most popular within our team. IDEs range widely in cost; best in class IDE suites can cost up to $700, which is not a great value proposition for most engineers.

Travel

- We travel in economy by default and do not pay for business class
  - If you're unsure of your travel plans and believe you may have to cancel, it may be worth spending a bit extra to book flex tickets that allow a full refund to your Brex
  - It may be worth occasionally upgrading to Premium Economy if you're travelling a lot for work and the cost is not unreasonably high, particularly if you're working the next day
- Consider signing up for programs like Global Entry if you are regularly traveling to countries that offer it, using your Brex; this saves you time, particularly when traveling to the US.
- When traveling internationally, use your Brex to expense a reasonable eSIM. PostHog does not cover roaming charges for your phone.

- When using your Brex internationally, use the local currency since Brex generally offers a better exchange rate.
- PostHog has international insurance for our work trips, so do not buy travel insurance when traveling on behalf of PostHog.
- It's fine to book your outbound / return flights for a different day to when you are required to be there as long as the flight is a similar price or less.
- Any other costs outside of the days you are required to be at an event are of course *not* covered.

> If you find yourself needing to do extra travel outside of the regular things listed above, e.g. you've been asked to take a last minute trip to work on an emergency project, we may pay for a nicer seat here, especially if you are traveling at very short notice or long haul. Ask on #team-people-and-ops if you think this may apply to you. This is intended for genuine one-offs, not where you've decided you'd like to come along to an extra offsite!

- We strongly encourage team members to try and work together in person when practical. This isn't limited to just working with people in your team, but we expect that you have a reasonable reason you need to work together.
- If you're in the same place as other team members, even if you aren't directly working together, PostHog will cover the cost of a dinner or a fun activity
- When visiting customers (or potential customers), we should look for opportunities to connect with them over a meal. These don't need to be extravagant, but they should be appropriate to the size and expectations of the customer. If you would be comfortable justifying the spend publicly in All Hands, you're probably fine.

Sponsorships

If you believe an open-source project is fundamentally important to the success of PostHog then we should set up a recurring sponsorship. In this case, see the open-source sponsorship Marketing initiative.

Talent

contents/handbook/people/talent.md

Talent Principles

The talent team is uniquely placed at PostHog to have an outsized influence on the people that join the company in comparison to the rest of the business. This is why we ask our talent partners to think like owners.

PostHog's business model requires us to build and automate more products, to do that we need more engineers. Once we have more products, we require commercial team members to market those products and to look after the customers that sign up. We also need to support those customers and support the growth of the business. This means that the people that join PostHog directly impact our growth, hence why we invest so heavily in finding and retaining great people.

At some companies the Talent team are seen as a support function, at PostHog we view them as a growth function, so we need our talent partners to be thinking about the growth of the business as a whole, not just as a headcount.

This means taking ownership of understanding what products we need to build, the types of people we need to build those products, understanding how our commercial team make our customers successful and what types of people fit into those commercial roles. Talent partners should also understand how our funnels are working and what is needed to solve any problems within them.

Talent partners at PostHog do not just interview candidates and put them through, they own the process from beginning to end and should use all their skills to make that process successful.

Removing distractions from the rest of the team

As a talent partner, part of the role is to ensure we are removing distractions from the rest of the business where we can. Practically, what this looks like in talent is that other team members really only need to concentrate on assessment and shouldn't need to concern themselves with other areas of the process.

Throughout the recruitment process there are different ways that talent partners can remove distractions.

Sourcing Talent

Our default recruitment strategy is 100% inbound. We are not an outbound-first recruiting style organization and don't want to become one. That said, there are times when sourcing is the right tool for acquiring new candidates. This section explains when to reach for it, how to do it well, and how to know if it's working.

When To Source

We should source when inbound alone isn't generating enough qualified candidates at the top of the funnel for a specific role. This can happen for many reasons, including:

- The role requires a niche skill set, and the addressable talent pool is small.
- When headcount for a particular role is very high.
- A role has been open for 4+ weeks and we're not seeing enough quality applications.

How We Source

Be targeted, not generic. We don't do high-volume spray-and-pray outreach. We'd rather send 15 highly personalized messages than 150 templated auto-messages. Sourcing at PostHog should feel like a curated referral from someone who's done their homework, not a LinkedIn InMail blast.

Practically, this looks like:

- **Research before you reach out.** Look at what they've built, shipped, written, or contributed to. If you can't find a specific reason to reach out to them beyond "their title matches", don't reach out.
- **Lead with what they'd work on, not who we are.** Most sourced messages open with "we are an amazing company..."; this comes across generically and doesn't always grab attention. Make sure they know it's a personalized message. Open with the problem they'd solve, or the product they'd help build. Link to the specific small team page they'd work with, a recent GitHub issue, or a shipped feature that's relevant to their background.
- **Be transparent about who we are and how we work.** Link to the handbook. Link to the compensation calculator. Link to relevant blog posts. PostHog's transparency is one of our biggest differentiators in recruiting: use it!
- **Use the right channel.** LinkedIn is usually our default, but for engineers, consider reaching out via GitHub (if they're active contributors), Twitter/X (if they post about relevant topics), or relevant community forums (HN, specific open source communities). Match the channel to where the person actually is. This can also occasionally be via email.

What A Good Sourcing Message Looks Like

Here's the kind of thing that *usually* works:

> Hey (name), I was taking a look at your work on (specific project/contribution). We're building (specific thing) at PostHog and it really seems like the kind of problem that needs someone with your background in (specific skill). If you haven't heard of us at PostHog, we're fully remote, pay transparently, and our entire company handbook is public; you can read exactly how the team you'd join operates before you even start day 1: (link to small team page). If this seems even 1% interesting to you, let me know, and I can set up a call for you with one of our talent partners handling the role!

And here's what doesn't usually work:

> Hi (name), I came across your profile and was impressed! PostHog is a fast-growing product analytics company backed by Y Combinator. We're looking for world-class talent to join our world-class team. Would you be open to a conversation?

The first message is specific and gives the candidate real information. The second is generic and could be about any company hiring any role.

Sourced Candidates In the Process

Sourced candidates follow a slightly different path at the start of the process. See Managing sourced candidates for the exact mechanics.

The key difference to keep in mind: sourced candidates didn't come to us; we went to them.

Some context related to this, to end this sourcing section:

- **You'll need to invest more in educating sourced candidates about PostHog.** Inbound candidates have usually read the handbook and tried the product on their own. Sourced candidates haven't always done this (but ideally should before their call). This can look like sending more content-heavy emails (linking to the handbook, blog posts, team pages, etc.) and generally doing more "educating" throughout the early stages of the process, to bring sourced candidates up to the same level of context that inbound candidates arrive with naturally.
- **Expect lower conversion at later stages.** Sourced candidates haven't self-selected the same way inbound candidates have, so their drop-off rates will naturally be higher.
- **Track it separately.** We should always know what percentage of our hires came from sourcing vs. inbound vs. referrals.

Top of the funnel

Talent partners should be speaking to hiring managers before a job goes live to understand the types of candidate that they should be looking for at the top of the funnel, this part should continue to be honed as you learn more from feedback at the various stages of the process. We want to avoid passing through inappropriate candidates, that waste time. It is a talent partner's responsibility to screen applications at the top of the funnel, here we are looking for signal that people can be effective at PostHog. We look for things like:-

- have they worked at companies that have scaled like PostHog wants to
- have they been a founder before
- have they led on projects
- have they written a personalized cover letter explaining how they fit the role
- have they displayed high levels of ownership
- are they weird?

Once you've screened the application and moved them forward you will have a culture screening call. We also want to ensure that we are putting relevant and motivated candidates through to the next round. At the screening stage it is important to make sure your notes are well organised and clear, not just for the next stage. These notes will be reviewed at SuperDay and will be taken into consideration if we are going to hire this person so make sure to have these in good order.

Post-screen, pre-SuperDay

Talent partners are responsible for moving candidates through the next two stages, we rely on automation from Ashby to allow candidates to book directly in calendars. Talent partners are responsible for the candidate experience so if a candidate or an interviewer can't make that time, then you need to step in and resolve the issue. It is important that interviewers know that they need to maintain an up to date calendar, however sometimes last minute changes do occur.

SuperDay

Talent partners are responsible for scheduling these, you can read more in the hiring process SuperDay section. The people involved should be focused on assessing the candidate so talent partners need to be alert for helping with any logistics to make sure the SuperDay runs smoothly.

Speed v quality

At PostHog, we have two major forces playing against each other when it comes to hiring. We want to move quickly, when we want to hire somebody, usually they should have started last week. The reason for that is we always want to hire for quality, and this takes time. This makes life as a talent partner a constant balance between these two things. The way we try and balance these things is try and move as quickly as possible with the things in our control. We aim to review applications ASAP, usually within 48 hours of application. Then we want to make sure that candidates can book their first round call with us within 2 business days. This keeps the momentum going from a candidate deciding to apply, to speaking to somebody. Our aim is to get back to every candidate by the end of the day after their interview, this is difficult, so whilst it's an aim we don't always hit it. We should be pushing interviewers to get feedback to us ASAP. The longer we leave canddiates without a decision the slower we will move.

Speed is a team effort, we should always keep things moving for each other. This means that if we see something that needs arranged and you can do it now, do it. No need to wait for the person who was last in contact to come online, just give them a heads up that it's done. This shared ownership mentality of speed is what will help us succeed.

When it comes to quality, we are always looking to be impressed. We use a rating system out of 4 and it is rare that we would hire somebody without receiving a 4 from somebody in the process. We know that exceptional people are usually spiky, they don't have an evenly distributed skill set so people can have reservations about a certain area and they can still be exceptional. Talent partners need to be aware of

when to push and pull when it comes to hiring decisions. There will be circumstances when there are hard decisions over whether we should hire somebody or not and talent partners should be prepared to offer opinions. This could be vouching for a candidate that is similar to other successful team members who we might be hesitating on and conversely stepping in when it looks like we might be making a hiring that isn't appropriate.

When a hiring process is moving slowly, or we seem to be rejecting lots of candidates at SuperDay, it is up to the talent partner to realize and own this. They should review what is going on, understand the problem and aim to fix this. They should get in front of this as soon as possible.

Maintaining quality is also about ensuring that our interviewers are assessing candidates in a consistent way, so spotting when there are inconsistencies are coming up in feedback and doing something about that is important.

Evaluating success

A talent partner will be judged on how many excellent candidates they can get into the business and how those candidates manage to impact the business' performance. We would much rather we had consistently great people coming in and moving the business forward, than if we had lots of people joining but it's not helping us grow. We want talent partners to be able to just assess a candidate on a screening call but be able to figure out how we continue to scale the growth of PostHog, with great people at the heart of it.

Time off

contents/handbook/people/time-off.md

We offer our team unlimited time off, but with an expectation that you take *at least 25 days off a year*, including national holidays. This is to make sure that people can take time off flexibly while not feeling guilty about being on vacation.

The reason for this policy is that it's critical for PostHog that we hire people we can trust to be responsible with their time off - enough that they can recharge, but not so much that it means we don't get any work done. The People & Ops team will look into holiday usage occasionally to encourage people who haven't taken the minimum time off to do so. The 25 days is a minimum, not a guide.

As general guidance, we don't care about a few days here and there. If you are taking significantly more vacation time than most - for example, 40 days - we would be very surprised if you aren't causing a strain on the rest of your team as a result.

Permissionless time off

We care about your results, not how long you work.

You do not need to get approval for time off from your manager. Instead, we expect everyone to coordinate with their team to make sure that we're still able to move forwards in your absence. You should avoid things like:

- Having an entire Small Team off - this means we can't provide support to customers
- Having the only X people who can do some totally critical task at PostHog off - if this is unavoidable, try to make sure one of you can at least check in if something goes horribly wrong

How to book time off in PTO by Deel

Before you start, make sure that:

- You have authorized the PTO by Deel app in Slack to connect to your Google Calendar
- You have subscribed to the team time off calendar

If you don't do this, your holiday won't show up in the team time off calendar.

To book a day off:

- Book it on the PTO by Deel app in Slack. There are various types of time off you can select. It will be automatically approved and added to the team time off calendar (with the exception of longer term leave.) It will also be added to your manager's personal calendar.
- Do not book directly on deel.com as it does not sync with Slack, and the team will not know you are out. Yes, it is confusing that Deel have two separate systems here.

- There are four types of time off you can select
  - PTO - this will be the majority of the time for vacation or any other time off that doesn't fit below
  - Out Sick - this is when you're sick and will take time off unexpectedly
  - Parental Leave
  - Medical Leave - this is planned time off for you personally, on medical grounds - if it is a family member who requires support meaning you will be off, this is PTO.
- Block out your own personal GCal to show that you are out. This is because PTO by Deel *only* books in an all day event in your calendar to show that you are out. If you don't do this, automated meetings such as interviews or demos might still get booked into your cal.
- Set an out of office message on your email and have it point to someone else on the team, or hey@posthog.com. PTO by Deel will automatically set your Slack status to out of office and will autorespond to Slack messages.

> Please manually book in public holidays you plan to take off as well. We have team members working in countries all over the world, so it is not practical for us to book these all in on your behalf. Some people also prefer to work on certain days even if they're considered a public holiday in the country they are living in or visiting. In the Time Off by Deel app, you can use the Bulk Add by Region feature to quickly identify and add the public holidays you want off.

The same rules as above apply regardless of the holiday length and type. Sick leave and any other types of time off should also be booked in the same way.

How to cancel time off

If you decide to cancel your holiday, drop a message in #team-people-and-ops and a member of the team will cancel the holiday for you, as only admins can delete holidays.

Flexible working

We operate on a trust basis and we don't count hours or days worked. We trust everyone to manage their own time.

Whether you have an appointment with your doctor, school run with your kids, or you want to finish an hour early to meet friends or family - we don't mind and you don't need to tell us. Please just add it to your calendar and, if you are doing anything that could require you to be immediately available (ie support hero / or any customer-facing role), please make sure you have cover.

When you should have time off

You are sick

If you are sick, you don't need to work and you will be paid - the upper limit for paid sick leave for your country will be specified in your contract. This is assuming you need a day or two off, then just take them.

Please let your manager know if you need to take off due to illness as soon as you are able to and add it to PTO by Deel. You shouldn't pre-emptively book a bunch of days off sick, as you can't know how long you will actually be sick for and you may trigger the need for a doctor's note (see below). Just book the day or two off that you are sick then add more if you still feel unwell.

For extended periods of illness (5+ work days), or if you are going over the limit in your country/state, please speak to Fraser so we can work out a plan. In most countries, we will need a doctor's note from you.

If you have a medical condition you know will take you away from work regularly, please let Fraser know so we can work out accommodations with you and your manager.

Bereavements / Child loss

We do not define “closeness” and we won't ask about your relationship to the person or what they meant to you. Please just let us know up front how much time you would like to take.

Our bereavement policy also covers pregnancy and child loss for both parents, with no questions asked. Please take at least 2 weeks of paid leave.

If you need extended time for physical or mental health reasons, we will treat it as extended sick leave - just chat to Fraser.

Jury duty / voting / childcare disasters, aka 'life stuff'

There are lots of situations where life needs to come first. Please let it - just be communicative with your team and fit your work around it as you need. We trust you will do the right thing here.

If you are summonsed for jury duty, please let Fraser know right away - we can often get an exception granted if we have enough notice.

Parental leave

Parental leave is exceptional as it needs to be significantly longer than a typical vacation. Anyone at PostHog, regardless of gender, is able to take parental leave, and regardless of whether you've become a parent through childbirth or adoption.

This table explains the amount of paid time off, depending on how long you've been at PostHog

Time at posthog	maternity leave	paternity leave
under 6 months	3 weeks	2 - 3 weeks
6 - 12 months	12 weeks	2 - 3 weeks
over 12 months	up to 24 weeks	6 weeks

> Parental leave at PostHog is designed to be more generous than your local jurisdiction's legal requirements. This means that in most cases you will receive the PostHog policy, if you live in a country with more generous parental leave, then you will receive that. This PostHog policy is not designed to be *in addition* to your specific state/country policy.

We only pay the enhanced parental leave in one continuous block.

Parental leave isn't supposed to be combined with our unlimited PTO policy here - we aren't prescriptive and will trust your judgement. If you need a longer break after childbirth or a staggered return reach out to Fraser or your manager. But please note that we usually won't allow you do a combination of parental leave plus a long holiday in addition to that to extend your time off.

Please communicate parental leave to Fraser as soon as you feel comfortable doing so, and in any case at least 4 months before it will begin. They will let the People & Ops team know, who will follow up on any logistical arrangements around salary etc. and any statutory paperwork that needs doing.

Maternity leave

The above is in reference to Paid Time Off (PTO). Maternity leave can be extended using unpaid time off, please work with your team to find a reasonable solution for both your family and your team, and let Fraser know the total amount of time you expect to take off as soon as possible.

> For quota-carrying Sales roles taking 12 weeks or longer, your OTE will be calculated by averaging your sales quota attainment for the prior two full quarters (capped at 100% OTE).

Paternity leave

We do not offer unpaid leave for Paternity leave.

Birthday and anniversaries

We celebrate all the big and little milestones at PostHog, including birthdays and work anniversaries. We celebrate each team member as a reminder of how much we appreciate them. Kendal is currently responsible for organizing these.

Birthdays

We have partnered with Wellbox to send all team members a personalized giftset for their birthday.

These are the steps for making an order:

1. Log into our Wellbox account (details in 1Password)
2. Select the birthday gift to send
3. Fill out delivery information
4. All set!

The birthday gift usually arrives on the day of or 1-3 days prior to the birthday. Shipping fees: UK shipping is free while all other countries will have shipping fees.

Anniversaries

On your first anniversary with PostHog, you will receive a giftcard from Giftogram or Prezzee (if you are based in the UK) which can be used on a wide selection of brands. On your second anniversary you'll be gifted a customized Lego minifig in a display case, and on your third anniversary, you'll receive a personalized gift from Wellbox.

1st year anniversary

For the first year anniversary, we give 50*forUSgiftcards*/55 for all other countries gift cards to cover service fees:

1. Login into Giftogram by using your gmail credentials
2. Two ways to create a new Giftogram, on the tool bar above where it says “Create and Send" or you can click on the right hand side on the blue button “Send a Giftogram".
3. Walk through the following steps:

- Select the appropriate campaign: US Campaign= US team members, GCode Campaign= EU+ALL team members, and CA Campaign= Canada team members
- Select a card design of your choice (easiest to just use the anniversary theme)
- Next screen, select “individual”, email as a delivery method, and add value (see above for amount) and continue to the next step
- Enter the individual’s PostHog email address. You can add multiple email addresses if there is one then one anniversary. The amount will add itself on the right hand side as you add more individuals. Then, continue to the next step
- Delivery message; select PostHog team as the sender and select the drop down “1st year anniversary” as the pre-populated message or you can create your own personal message
- Last step, schedule the delivery date and you’re done!

2nd year anniversary

The second year anniversary gets you a customized Lego figurine:

1. Log into Fab-brick (login credentials are shared in People & Ops 1Password vault)
2. Select the third tab “MiniFig Creator” and design your mini fig to look like the individual you’re celebrating!
3. Make sure to include a display case and the three tier brick option
4. After you’ve completed your design, check out. There should already be a Brex card on file. Please make sure you add the individual’s correct mailing address.

3rd year anniversary

The third year anniversary is a pack of gifts provided via Wellbox.

1. Select the 3rd Anniversary gift in our profile
2. Fill out delivery info
3. You're all set!

The gift will usually arrive on the day of or 1-3 days prior to the anniversary date. Shipping fees: UK shipping is free while all other countries will have shipping fees.

4th year anniversary

On your 4th anniversary at PostHog as a big thank you for sticking with us, we give you a choice of 3 gifts:

1. Sage Barista Touch coffee machine
2. Apple 27-inch 5K Retina Studio Display with standard glass and tilt-adjustable stand
3. Rimowa luggage set (large trunk, cabin bag, packing cube, toiletries bag)

On the run up to your anniversary, our Ops team will send you a link to the gift options questionnaire and order your 4 year anniversary gift once we receive your completed form. Thank you for making PostHog great!

Training

contents/handbook/people/training.md

The better you are at your job, the better PostHog is overall!

Books

Everyone at PostHog is eligible to buy books to help you in your job.

The reason we think books can be more helpful than just Googling stuff, is that the level of quality has to be higher for them to get published.

You may buy a couple of books a month without asking for permission. As a general rule, spending up to $50/month on books is fine and requires no extra permission. You can use your books budget towards audiobooks and podcasts as well, if you prefer.

Books do not have to be tied directly to your area, and they only need be loosely relevant to your work. For example, biographies of leaders can help a manager to learn, and can in fact be more valuable than a tactical book on management. Likewise, if you're an engineer, a book on design can also be particularly valuable for you to read. Additionally, we host a monthly book club called BookHog, and the budget can be used for those books as well.

Training budget

We have an annual training budget for every team member, regardless of seniority. The budget can be used for relevant courses, training, formal qualifications, or attending conferences. You do not need approval to spend your budget, but you might want to speak to your manager first, in case they have some useful feedback or pointers to a better idea.

We strongly encourage all non-technical team members to take some kind of entry level programming course - it's part of our 'you're the driver' culture that everyone can at least understand very basic concepts around software development. Codecademy is a good place to start and they cover many of the technologies we use, such as Python and React.

The training budget is $1000 per calendar year, but this *isn't* a hard limit - if you want to spend in excess of this, request an increase to your budget in Brex and it should usually be granted.

If possible, please share your learnings with the team afterwards!

Conferences

You can use your training budget for time spent talking at conferences and user groups, including coaching others. It is expected that you would spend up to half a day a month on these activities. Like the training budget, this isn't a hard limit. If you think you need more than that talk to your manager in the first instance.

Product metrics

contents/handbook/product/metrics.md

We track a short list of metrics in each per-product growth review. The idea of a standardised list of metrics is that each product team has roughly the same metrics they care about, and we can compare metrics across products and across time, such as revenue growth, to see how we compare.

Our growth review metrics strike a balance between depth, efficiency and "measuring what matters". We want to make sure our metrics alert us of potential negative (or positive!) developments, giving us enough signal to dive deeper into lower-level metrics.

If as a new product manager or team lead you want to look at a wider range of metrics, please do! These can either be incorporated in the growth reviews, or in ad-hoc metrics reviews you or your team are doing.

Metrics we use in growth reviews

Revenue

These queries are written and owned by the . They are standardised across products, and match the combined PostHog revenue queries. If you are intending a change, please chat to the billing team first.

- Link to combined revenue dashboard
- Link to per-product revenue dashboard

Note that currently, refunds are not removed from per-product revenue, which is something to note in a growth review if there is a sizable refund that month.

Metric	Notes
Monthly recurring revenue (MRR)	
Annual recurring revenue (ARR)	
MoM growth rate	For more mature products, we want this to be over 9%, for newer products between 15-20% on average
New revenue growth rate	
Revenue expansion rate	

Metric	Notes
Revenue contraction rate	
Revenue churn rate	
Revenue retention rate	
Total paying customers count	
Paying customers growth rate	
Quarterly net revenue retention (NRR)	Instead of a rolling metric, we use the quarterly values and report on it once a quarter. The rolling metric is available on the dashboard too, as it can be helpful for debugging
Annualised NRR	Same as above
Revenue share	For revenue products like data pipelines or product analytics, it makes sense to calculate CDP/batch exports/anonymous-only share of revenue, to understand individual product contribution better

Usage

Product usage metrics are defined by the PM or small team lead. When setting up metrics for a new product, it's recommended to start with a longer list and trim it once user behavior is better understood. We recommend adding all relevant product metrics to one dashboard that is accessible, kept up to date and reviewed by the whole small team. For better discoverability, some of us use the appendix ™ to mark the primary usage dashboard.

This dashboard can also include NPS & support metrics (see below).

- Link to session replay usage metrics dashboard (example)

Metric	Notes
Unique monthly users - count	As defined by a key product action we also use in the activation definition, such as "flag created" or "recording analyzed"
Unique monthly users, growth rate	

Metric	Notes
Unique monthly organizations - count	Same definition as unique monthly users
Unique monthly organizations, growth rate	
Activation	Guide how to define activation for a new product; Dashboard that contains all per-product activation queries
Usage retention (1, 3, 6-month)	Report on it once a quarter. Retention changes slowly, it will be easier to see changes zoomed out

NPS

We have a NPS survey set up for each product. They need regular updates due to some survey limitations. If you want to set up a new NPS survey, speak to the Surveys PM (), he can help you set one up and keep it updated.

We track a 4-week NPS score, but we don't have the volume of responses we need to get reliable results. This is why we include the open-ended feedback in the growth reviews, as this is usually more actionable.

- Link to session replay NPS survey (example)
- Link to session replay 4 week NPS score insight (example)
- Link to session replay list of feedback insight (example)

Metric	Notes
NPS score - last 4 weeks	Include constructive feedback as a comment in the spreadsheet for context

Support

Similar to our revenue metrics, we are reusing queries the support team has set up, broken down by product. If you need to make a change or want to understand how SLA reporting works in detail, speak to the support team.

- Link to overall support dashboard
- Link to per-product ticket count insight
- Link to per-product SLA insight

Metric	Notes
Created tickets	
Escalated tickets	

Metric	Notes
Escalated tickets - SLA	The insight also tracks non-escalated tickets SLA, which is useful to be aware of, but we don't need to report on it in every growth review
Ratio no. of users vs. no. of tickets	Formula dividing no. of tickets / unique monthly users

Metrics outside of growth reviews

If there are any other metrics you want to track to understand how well your product is doing, or which areas need improvement, go for it! Just make sure you are not tracking too many metrics, causing you to lose sight of what matters.

Tips for increasing metrics awareness in a small product team

If you are a PM at PostHog, you will be more successful if your whole team is aware and keeps track of your per-product metrics, instead of just you summarising growth review insights once a month. Here are some tips we found are working well:

- Speak to your team what they care about and are interested in tracking, and add those metrics to your dashboard
- Link the revenue & usage dashboards in your team's Slack channel, so they are easy to find for your team
- If your team has shipped a new feature, encourage them to set up an event, and track the new feature's usage on the usage dashboard. For example you could have a "feature usage" section on the dashboard that tracks multiple features
- Some teams do a "metrics Monday" where they review the usage dashboard together in the Monday standup, looking for trends and anomalies that may help you make "mid month adjustments". These sessions are usually led by an engineer, not the PM
- You will likely have to try a bunch of things to find what sticks with your team. Ultimately, you want to make sure you and your team are looking at the same metrics, and everyone in your team knows how to find the relevant dashboards. It's a team effort!

Per-product cost & margin analysis

contents/handbook/product/per-product-cost-margin-analysis.md

Understanding your product's infrastructure costs helps you make pricing decisions, contextualize growth reviews, and catch problems early. This guide covers how to build per-product cost allocations.

Why this matters

As products scale, margins matter. A product with healthy margins can afford aggressive pricing; one with tight margins needs to be more careful. You can't know which you're in without understanding costs.

Cost visibility also helps you:

- Spot inefficiencies (why is this component 3x more expensive than expected?)
- Measure engineering wins (did that optimization actually save money?)
- Make pricing decisions (can we afford a price cut?)
- Catch cost spikes before they become problems

When to do this

This makes sense for:

- Products with product market fit
- Products with meaningful revenue
- Products with non-trivial infrastructure (storage, compute-heavy)
- Products where you're considering pricing changes

For early-stage products, don't bother. Ship first, optimize later.

The process

1. Map your architecture

Before talking to infra, sketch out what your product actually uses.

Write path: How does data get in?

- What services process incoming data?
- What queues buffer it?
- Where does it get stored?

Read path: How does data get back to users?

- What services handle queries?

- What databases get hit?

Storage: Where does data live?

- S3 buckets
- ClickHouse tables
- Redis/caches
- Postgres

You don't need to be perfect. The infra team will validate. But a rough diagram saves time.

2. Understand the cost buckets

Infrastructure costs fall into two types:

Type	What it means	Examples	How to allocate
Direct	Tagged specifically for your product	Product-specific k8s nodepools, dedicated S3 buckets	100% to your product
Shared	Used by multiple products	Load balancers, reverse proxies, shared caches	Proportional (e.g., 20% of traffic = 20% of cost)

Direct costs are easy. Shared costs require estimating your product's share of usage.

3. Work with infra on tagging

Reach out to #team-infrastructure to kick off the process – they can help you estimate your product's traffic share and navigate the tooling.

We use a FinOps tool for cost allocation. The infrastructure team sets up allocation tags that group AWS resources by product/function.

What you bring:

- Your architecture diagram
- List of services that should be allocated to your product
- An engineer who can validate the list is complete

What infra does:

- Verifies which resources are already tagged
- Identifies gaps (resources that should be allocated but aren't)
- Sets up allocation rules
- Creates reports you can pull

Expect iteration. First allocations are rarely complete. Common gaps:

- Inter-AZ data transfer (network costs between availability zones)
- Shared infrastructure not proportionally allocated
- Read-path resources missing (easy to focus only on write path)

ClickHouse costs (separate process)

If your product queries ClickHouse, you'll need to work with #team-clickhouse to get query cost attribution. This is separate from FinOps tagging.

ClickHouse costs are attributed by analyzing `query_log` to see which queries belong to your product. The ClickHouse team can help set up a query or dashboard to track this. Note that query attribution may require code changes to tag queries with a product identifier — this isn't just a dashboard exercise.

For some products (like Session Replay), ClickHouse query costs are a small percentage of total – queries are lightweight (list/fetch metadata). For analytics-heavy products, ClickHouse costs will be a much larger share.

We run ClickHouse in multiple regions (e.g., US and EU), make sure you account for costs in each.

4. Interpret the tags

The FinOps tool organizes costs using allocation tags. Here's how to think about them:

Product-specific tags (direct costs)

- These capture resources dedicated to your product
- Example: a "session replay" tag might include capture services, message queues, and S3 storage
- Use these as-is – 100% goes to your product

Shared infrastructure tags (need a proxy)

- These capture resources used by everyone
- Example: an "ingress" tag might include load balancers and reverse proxies for all products
- You need to estimate your product's share (e.g., "my product is ~20% of traffic, so allocate 20% of ingress costs")

Network transfer tags

- Inter-AZ transfer costs show up separately from compute
- Filter these tags to your product's services to get direct network costs

When pulling reports, make sure you're not double-counting. If you select multiple tags, check whether they overlap.

5. Build the cost model

Once you have the cost data, build a simple model for unit economics.

Get the totals:

- Total cost from FinOps tool (direct + allocated share of shared)
- Total volume from billing tables (recordings, events, whatever your unit is)

Calculate unit economics:

```
cost_per_unit = total_cost / total_volume
revenue_per_unit = total_revenue / total_volume
margin = (revenue_per_unit - cost_per_unit) / revenue_per_unit
```

Break down by component (optional but useful):

```
cost_per_unit = compute_cost/volume + (storage_rate × avg_size ×
retention_period)
```

This helps you understand what drives costs. For storage-heavy products, storage will be significant portion of costs. For compute-heavy products, compute dominates.

Test your assumptions. Key inputs like traffic share, retention period, and storage rates are estimates. Check how sensitive your margin is to these — if your traffic proxy is 20% ± 5%, what's the range? If effective retention is 60 days vs 90 days, how much does storage cost swing? If the answer changes materially, document the range rather than a point estimate.

6. Document your assumptions

Every cost model has assumptions. Write them down so future-you (or someone else) understands what's in vs out.

Common things to document:

- What's included in "direct" costs
- What proxy you used for shared costs (and why)
- How you calculated volume (billable units? ingested units?)
- Retention assumptions for storage costs
- What's explicitly NOT included

7. Set up monitoring

Once your allocation is stable, set up alerts:

- Total product cost: alert if week-over-week change > 15%
- Individual components: alert if WoW change > 25%

You want to catch:

- Runaway cost increases
- Broken tagging (cost dropped 50%? probably a bug)
- Optimization wins (did that change save money?)

8. Add to growth reviews

Include margin metrics in your growth reviews:

Metric	Notes
Total cost	From FinOps tool
Cost per unit	Total cost / volume
Gross margin	(Revenue - Cost) / Revenue
Cost trend	MoM change

Healthy products have stable or improving margins as they scale. If margins are declining, investigate.

What to expect

Timeline: 2-4 weeks to get a reliable allocation, depending on how well-tagged your resources are.

Accuracy: First pass is typically 80-90% complete. Shared resources and edge cases take time. Directionally correct is fine; perfect is not required.

Keeping it current

Cost models rot. The two main causes:

- **Architecture changes.** When your product adds new services, storage backends, or processing pipelines, the cost allocation needs updating. Build this into your launch process — when shipping a new component, close the loop with infra and your FinOps team to get it tagged before you lose track of it.
- **Tagging breakage.** Warehouse syncs, allocation rules, and report configurations can break silently. If your cost dashboard suddenly drops or flatlines, check the data pipeline before assuming costs actually changed. Set up a staleness check (e.g., alert if latest data is more than 3 days old).

Review your cost model quarterly at minimum, or whenever you ship significant infrastructure changes.

Common mistakes

- **Ignoring shared infrastructure.** Your product uses load balancers, proxies, and other shared resources. If you only count dedicated resources, you're understating costs.
- **Forgetting network costs.** Inter-AZ data transfer is easy to miss. It can be 5-10% of total costs.
- **Expecting precision.** This is cost allocation, not accounting. You're trying to understand rough margins and trends, not get to the penny.
- - **Double-counting within shared infrastructure tags.** An allocation rule may already bundle multiple resource types together. For example, an ingress tag might include both proxy compute and load balancers. Verify what's inside each tag before adding components separately — otherwise you'll overcount shared costs.
- **Not involving your FinOps vendor.** If you use a FinOps tool, loop in their support team to validate allocation rules. They can confirm what's bundled inside a tag faster than you can reverse-engineer it from reports.

Worked example

See the Session Replay unit economics RFC for a complete example of this process applied to a real product.

Contacts

- **Infrastructure / cost tooling:** #team-infrastructure
- **ClickHouse cost attribution:** #team-clickhouse
- **Billing data:** Billing team

Links

- Per-product growth reviews

Per-product growth reviews

contents/handbook/product/per-product-growth-reviews.md

For products that have product-market fit and are generating revenue, we are doing monthly **per-product growth reviews**. We recommend to do the growth reviews at the start of the month, to review the previous month. Most growth reviews happen asynchronously with the PM reviewing key metrics, analysing anomalies and sharing an overview with the team in Slack.

Objectives

The objective of the growth review is to review key product metrics and understand changes that have occurred over the preceding four weeks. By reviewing metrics on a schedule, we can spot issues faster than when reviewing them only sporadically.

Looking at the same metrics regularly will increase our understanding how they relate to each other, whether metric changes are expected or exceptions, and will make efforts to improve them more successful.

The growth reviews should focus on analysing anomalies instead of expected metric behavior, especially as teams become more familiar with their data.

Outside of the regular monthly reviews, it's the job of the Product Manager to regularly monitor these metrics, becoming an expert in their nuances. Should a metric deviate from the norm, they are responsible for presenting a well-researched explanation during the review.

Contents

Recurring analysis

During these reviews, we assess both input and output metrics. Input metrics, serving as leading indicators, significantly impact output metrics like revenue and retention.

Here are some examples:

- Input Metrics: Things customers care about and factors in our control, like onboarding, key product actions (such as `recording analyzed`), and performance
- Output Metrics: MRR, retention (revenue & usage), NPS score

As mentioned before, we aim to analyse the same set of metrics month over month, so we can see trends and anomalies. However, there can be cases where we decide to change a metric if it's a better indicator of long-term success, particularly for product activation and key product actions.

We've found that the best way to review what is a quite long list of metrics is to combine all numbers (revenue as well as usage) in one spreadsheet with a new column for each month, and only open individual graphs where required. Below is a screenshot that shows a part of our growth review document. View the internal growth review spreadsheet for internal users.

Date of growth review: 08/05/24			
Session Replay: Revenue	**Trend**	**April (as of 06/05/24)**	**March (as of 31/03/24)**
Monthly recurring revenue (MRR)	↗		
Annual recurring revenue (ARR)	↗		
MoM growth rate	↗		
New revenue growth rate	↘		
Revenue expansion rate	↗		
Revenue contraction rate	↗		
Revenue churn rate	↗		
Revenue retention rate	↗		
Total paying customers count	↗		
Paying customers growth rate	↗		
Average MRR per customer	↗		
Average ARR per customer	↗		
Quarterly NRR	-		
Annualized NRR	-		
Logo revenue retention (1 month)	↗		
Logo revenue retention (3 months)	↗		
Logo revenue retention (6 months)	↗		

Monthly focus areas

To make growth reviews more actionable, each of the three growth reviews per quarter should have a slightly different angle:

First growth review of the quarter (1 week in):

- Review the product / research goal planned as part of the team's quarterly planning
  - If we have answered this, have we answered the biggest unknown for the product?
  - Are there any other topics / research we could work on as a secondary priority?

Second growth review of the quarter (5 weeks in):

- Has our research yielded anything useful so far? What do we need to focus on understanding before planning in a few weeks?
- From the things we've shipped so far this quarter, did we learn anything?
- The outcome should be a list of open questions we should answer before the next quarterly planning

Third growth review of the quarter (9 weeks in):

- What impact did the things we have shipped this quarter have on our metrics, and overall success of the product?
- Looking back over the quarter so far, was growth healthy or are there issues we should address in the next quarter?
- This session will be the least actionable, since it is happening around the same time as the quarterly planning, so it's a good time to do an overall health check

Deep dives

In each growth review, we usually do a couple of deep dives. Topics can be proposed in a preceding growth review, by team members, or it is simply something the Product Manager finds worthwhile. Here are a couple of examples:

- Why was churn so high last month? Can we identify any reasons?
- Where in the onboarding funnel do new users struggle?
- Can we find leading indicators that predict long-term product usage? (e.g. Facebook's 7 friends in 10 days)
- Are there any difference between high ICP and non ICP customers in how they use the product?
- Are our 10 biggest customers happy users of the product?

In-sync or async?

While monthly metrics reviews are important, they are not always actionable. Or, sometimes a metric might be suboptimal, but we decide not to focus on it, because we have more important topics to work on. Since PostHog's culture leans towards no meetings by default, we are not meeting every month to review the metrics in-sync. For in-sync growth reviews, the following guidelines apply:

- 1 quarterly in-sync growth review for existing products & PMs
- 3 monthly in-sync growth reviews in a row for a new product or new PM
- In-sync growth review any time a PM spots an issue in the metrics they would like to discuss
- Additionally, the team lead and the responsible exec can also ask for a in-sync growth review

In-sync growth reviews are usually joined by the PM, the team lead and the responsible exec. Team members usually don't join the growth reviews, but a summary and the full analysis is accessible to everyone and shared via Slack.

Structure of the in-sync growth reviews

During the meeting

- Metrics walkthrough
    - Led by PM
    - Participants note questions/comments they have

- Review of questions/comments that were made before or during the metrics walkthrough
- Walkthrough + discussion of deep dives
- If required: Ad-hoc analysis of questions that came up
- Agree on to dos for the next meeting

Before the meeting

- PM prepares growth review
- PM shares summary + links to analysis as well as the growth review document with the participants as well as the small team, so everyone has the chance to review the document and add notes before the meeting

After the meeting

- PM shares summary of meeting discussion + outcome with the small team
- PM makes sure all to dos are completed by the next growth reviews

Structure of the async growth reviews

Very similar to the above, except that the metrics walkthrough doesn't happen in a meeting. The PM shares a summary of the full growth review they prepared (key metrics, deep dives, anomalies and follow-ups) in the team's Slack with the team lead and responsible exec tagged. The whole team is encouraged to read up on the full notes & numbers that are linked, and ask follow-up questions.

Useful material (internal links)

- Main growth review document (Session Replay example)
- Metrics overview spreadsheet
- PostHog notebook with relevant usage graphs (Session Replay example)
- Metabase dashboard for per-product revenue

Prioritizing work for mature products

contents/handbook/product/prioritizing-work-for-mature-products.md

There's no golden rule or perfect framework for prioritization. Here are a few things we should keep in mind when building mature products (products with substantial revenue and usage).

- **Always prioritize our ICP (Ideal Customer Profile)**
  - As products mature, we start to get more and more users who are not strictly within our ICP. Building for these users is great - but only if we've satisfied the core needs of our company-wide ICP first.
  - Should different products have different ICPs?
    - The very first ICP should always be the same. However, some products will fill their needs sooner than others.
- **Prevent churn before capturing growth**
  - Not all churn is preventable - some customers' businesses don't make it. These customers are usually in our smallest revenue bucket and shouldn't really impact prioritization - unless the question is about how to make them more successful (which is a good question for sure, but not always relevant).
  - Preventable churn usually comes from:
    - Lots of bugs or particularly bad bugs
    - Missing features that competitors offer
    - Pricing
    - Not proving value often enough (eg. lack of use because it doesn't fit into their workflow)
  - Churn should be roughy equivalent across products - if yours is high, figure out why and fix it. Growth is less effective when you have a leaky bucket.
- **Use metrics as a guide, not the end-all-be-all**
  - Trust your instincts and your convictions. If you *just know* that something is the right thing to build, then build it.
  - Use metrics to help point you in the direction of what general area to focus on if you don't have conviction elsewhere.
- **Reprioritize enterprise feature requests when it makes sense**
  - Larger enterprises will sometimes make esoteric requests. Other times their requests are totally valid - but we're busy with other things.
  - If the following are true, you should prioritize their needs immediately:
    - The customer represents a large churn risk
    - The feature has been requested by multiple customers and will be widely used
    - The lack of the feature prevents them from using the core product
- **Don't put off the hard things**
  - If something seems important but difficult, and another thing seems easy, don't just do the easy things. Teams that do the difficult things will pull ahead. (assuming doing the difficult thing is possible - usually we want to see some evidence that it is)

What we look for in product managers

contents/handbook/product/product-manager-hiring.md

> This page outlines what makes a great product manager at PostHog: The traits, skills, and mindset we look for when hiring and developing PMs.
>
> For how the role works day to day, see What product managers do at PostHog.

User closeness

We expect every PM at PostHog to be **obsessed** with users. Not just to enjoy talking to them, but to *crave* it. Great PMs feel uneasy when they or their team go too long without a real user conversation.

This obsession can show up in different ways: maybe in the past they've been a founder, a product engineer, a user researcher, or worked in another deeply user-facing role. The path doesn't matter, but the curiosity does.

Talking to users is **table stakes** at PostHog. It's a skill anyone who cares can learn quickly and keep refining over time, independently of their role and background.

A red flag is someone who's worked in user-facing roles for years but shows little genuine curiosity or interest in understanding users.

Metrics ownership

Owning metrics requires two distinct capabilities:

Technical capability

PMs at PostHog are expected to do their own data analysis.
They must be fluent in **SQL** and comfortable investigating metrics directly in our data warehouse.
Additional experience in **data modeling**, **analytics engineering**, or **building dashboards** is a plus.

Depth of experience

Beyond technical skill, we look for PMs who have lived with metrics over time.
Not just "churn was high, I ran five interviews, we fixed churn."

We want PMs who have owned a product for months or years, stayed close to metrics like retention, churn, or revenue, and have gone deep into diagnosing and improving them.

Ideal candidates can share examples such as:

- Defining or refining an activation metric and validating it
- Breaking down retention by customer segment and acting on findings
- Investigating churn and iterating multiple times until a successful fix was found
- Building a churn model or cross-sell analysis to guide prioritization

This experience is much harder to teach than talking to users, so we actively screen for it.

Product sense

Finally, PMs need **strong product sense:** The ability to recognize what makes a product feel powerful, intuitive, and cohesive.

This doesn't mean micromanaging every design detail. It means having the judgment to know when the product experience is drifting away from what "feels right" and stepping in at the right level of detail.

Examples of how this shows up day to day:

- **Spotting when a flow adds friction:** Noticing when a process feels slower, more complex, or less clear than it should.
- **Recognizing when defaults feel off:** Identifying when the starting experience leads users in the wrong direction or creates unnecessary confusion.
- **Calling out incoherence:** Seeing when patterns, language, or tone diverge from the rest of the product and risk breaking the overall consistency.
- **Identifying one-way door decisions:** Understanding when a design, naming, or architectural choice would be hard to reverse and needs another round of iteration before shipping.
- **Balancing speed vs. craft:** Knowing when quick progress is fine and when quality and polish are essential for long-term product perception.
- **Ensuring cohesiveness:** Making sure the product not only works, but feels coherent, purposeful, and aligned with the company's principles.

Strong product sense means keeping a holistic view. Understanding not just what works, but what feels right to users and to PostHog's product philosophy.

Hands-on with code (optional but valuable)

It's not a requirement that PMs at PostHog know how to code, but it helps.
PMs who can navigate a codebase, make small changes, or who have built small side projects often find it easier to empathize with engineers on their team and also with our target users (developers).

We also find that PMs who occasionally ship a small PR in their product:

- Build stronger relationships with engineers
- Better understand technical trade-offs

- Make more grounded product decisions

They don't need to be an engineer, but curiosity about how things work and the willingness to dive in and experiment is a strong advantage.

Culture fit

PostHog PMs need to combine **strong opinions with deep trust in their team**.
We want PMs who:

- Have conviction and clear opinions about the areas they are (co-)responsible for: Metrics, pricing, users, positioning.
- Can confidently act as the informed captain when needed, owning decisions in their domain.
- At the same time, know when to disagree and commit and support a team lead or another informed captain's decision even if they personally would have done something differently.
- Trust the team on all the decisions that are not theirs to make.

For example:

- A PM may strongly influence pricing, positioning, or launch readiness.
- But they should not overrule roadmap or technical decisions made by engineers or the team lead.

Their job is to lead with context, to make a compelling case grounded in data and user insight.
If the team decides differently, the PM assumes good intent and trusts that choice.

Ultimately, at PostHog:

- The team lead owns the product.
- The PM ensures the product is positioned and priced for success, and keeps feeding user and metric context into the team.

We value PMs who show conviction where it counts, humility where it doesn't, and trust in their team above all else.

What product managers do at PostHog

contents/handbook/product/product-manager-role.md

> This page explains what product managers do at PostHog: How the role works, what PMs are responsible for, and how they collaborate with their teams.
>
> If you're looking for what we value in PMs and how we hire them, see What we look for in product managers.

The role at a glance

At PostHog, product managers (PMs) exist to bring clarity and context to their teams.
They do not own roadmaps or dictate what to build next. Instead, they ensure the team deeply understands its users and its product's performance, so that the right decisions happen naturally.

A PM **owns** the following three areas for their product and team:

Systems for user discovery & product thinking

- Ensure a system is in place for the PM and engineers to hold discovery calls with users regularly.
- Have a process for storing customer feedback so that it's retrievable if we ever decide to use it (but not putting it "in the backlog.")
- Organize interviews, lead metrics reviews, or host Replay watch parties.

Team metrics

- Always know how the product is performing, both in usage and revenue.
- Be aware of key trends, areas of concern, and emerging opportunities.
- When metrics move in the wrong direction, create a plan or hypothesis for where to dig deeper next.

> We are following a defined format for growth reviews to make sure it's easy to compare performance across products. See Per-product growth reviews for more information.

Pricing

- Regularly review whether pricing aligns with the value users get from the product.
- Identify when pricing or packaging creates friction for adoption, retention, or expansion.
- If we decide to make a pricing change, work on a new pricing model taking into account our pricing principles, and involve the billing and marketing teams at the right time to get the changes live.
- Track the impact of pricing changes on revenue, conversion, and churn to understand what worked and what didn't.

A PM **shares ownership** of the following areas with their product team.
This doesn't mean it's a lesser priority for a PM to own projects here. It simply means product engineers are equally expected to contribute.

User understanding

- Understand who users are, what problems they face, and how they feel about the product. This context should be sourced through any relevant channel: user interviews, surveys, PostHog, support tickets, GitHub issues, BuildBetter recordings, or other data sources.
- Refine the product's target persona(s) as the product matures, including their needs / jobs to be done, and track this information on the team page.

Monitoring of the competitive landscape & industry trends

- Research best-in-class competitors and identify the biggest gaps between our offering and theirs.
- Facilitate input from Sales, Support, and users, who are often acutely aware of the features we're missing — especially if they're about to cause someone to churn.
- Keep an eye on new product launches and industry shifts to ensure we're not missing emerging opportunities — see *The Innovator's Dilemma* for reference.

A PM **supports** the following areas for their product and team.
In consultation with the team, they might own projects in these areas:

- Launching a new product in beta or in general availability
- Deciding on what we should be building next
- Providing feedback on UX for features

Everything else in the PM role builds on these foundations.

The product lifecycle and the PM's focus

The PM's work evolves with the product.
While the principles stay constant — context and clarity — the emphasis shifts by stage of the product.

> **Note on the table below:** Especially in the early stages (1 & 2), a team usually doesn't have a PM yet — product engineers own all product decisions.
> Review this table with the context from "The role at a glance" (own/share/support) and "Deciding when to add a product manager to a team."

Stage	Goal	Key PM questions	Typical PM work	Example projects
1. Idea → Beta	Get a first version of the	- Who are we building for?	- Lead or synthesize early user research	—

Stage	Goal	Key PM questions	Typical PM work	Example projects
	product into users' hands	- What is the 80/20 of features? - What needs to happen to get this into beta as quickly as possible?	- Define MVP scope and launch criteria - Shape initial positioning → who the product is for and why it matters - Coordinate beta launch activities and internal comms (beta program, website copy, docs)	
2. Beta → General Availability	Launch a product that users want to pay for	- What's missing in the product to start charging for it? - How do we charge for it?	- Lead or synthesize early user research - Decide pricing model - Coordinate launch activities and internal comms (beta program, website copy, docs)	PostHog AI pricing, renaming "Max AI" to "PostHog AI" for clearer positioning

Stage	Goal	Key PM questions	Typical PM work	Example projects
3. General Availability → Growth Stage	Strengthen and expand adoption	- Are users truly getting value? - Where are they dropping off or getting stuck? - What drives retention and revenue growth?	- Regularly review usage data, churn patterns, and feedback loops - Run interviews and other user research to understand user sentiment and evolving needs - Research “best in class” competitors to highlight gaps - Identify the biggest opportunities for improvement or expansion - Frame problems clearly so the team can decide autonomously what to build next - Collaborate on pricing adjustments or repositioning as understanding deepens	User interviews on error tracking, Surveys pricing change, Research into needs of data engineers after re-positioning of Data Warehouse / Data Stack
4. High Revenue / Mature Product	Sustain growth and manage complexity	- What’s limiting growth now? - What new segments, features, or pricing models allow us to sustain a 9% MoM revenue growth rate?	- Track advanced revenue and usage metrics (activation, retention, revenue retention) - Share context and opportunities in identified problem areas with the team - Run “conviction sprints” to refresh understanding of ICP and persona(s) as they relate to the product - Ensure product strategy stays grounded in real user	—

Stage	Goal	Key PM questions	Typical PM work	Example projects
			and business outcomes	

Deciding when to add a product manager to a team

Because most engineers at PostHog have strong product skills, many early-stage products don't need a PM right away (stages 1 & 2).

At this stage, it's typically the team lead's job to decide:

1. What to build
2. When to release the first version
3. How to charge for it

...with feedback and guidance from the Blitzscale team.

There are a few exceptions:
When a team is working on an infrastructure-heavy product, it's more important that the team lead has strong engineering and infra skills than product skills.
In that case, it can make sense to add a PM early who can focus on product direction, positioning, and user research while the lead focuses on technical execution.

In most cases, though, we add a PM **after a product has launched and started generating revenue (stage 3)**.
Once a product is live, product opportunities multiply, and it becomes the PM's job to surface what matters most, connect user feedback with metrics, and ensure the team's effort goes where it has the biggest impact.

Ultimately, it's up to the Blitzscale team to decide when adding a PM will help the team ship faster and focus more effectively. When that becomes true, it's time to bring in a PM.

Collaboration and decision-making with the team

PostHog is an **asynchronous and autonomous organization**.
PMs do not "own" the roadmap or feature priorities. The closest thing we have to a "product owner" at PostHog is the team lead.

Instead:

- Engineers choose what to work on next based on shared context.

- The PM's job is to provide that context: articulate problems, surface data, and make trade-offs visible.
- When major calls (like positioning, pricing, or launch scope) need to be made, the PM and team lead collaborate closely to select an informed captain: One person who makes the decisions and sees them through. The other can provide feedback when requested, but focuses elsewhere.
- A PM typically chooses their own projects, but the team and team lead can "assign" them tasks when needed.

Product management at PostHog

`contents/handbook/product/product-team.md`

PostHog has a product-minded engineering organization. Engineers own sprint planning and spec'ing out solutions.

So, what is the role of product managers at PostHog? PMs set context across multiple products for how products are being used, what the competitive landscape is like, what users are feeling about PostHog, and how they're using things.

Among other things, they

1. run growth reviews for products that have product-market-fit
2. organize user interviews
3. coach product engineers on "how to do product"

For a more in-depth look at the product role at PostHog, see What product managers do at PostHog.

How PMs work

Small team membership

Each PM belongs to a small number of our small engineering teams, so that all teams have a strong sense that the PM is there to support them equally. This also ensures that the PM has the time to dive deep into issues that require it. PMs join small team standups and planning whenever it makes sense, but they are not required to attend *all* team meetings. This is up to the PM to decide when it makes sense to join these, and when their time is better spent elsewhere.

Here is a overview that shows which of our PMs currently works with which team:

<table>
<tr><td></td></tr>
<tr><td></td></tr>
</table>

-
-
-
-
- Cross-product growth reviews

-
-
-
-

-
-
-

Product teams with no PM currently

-
-
-
-
-

Product goals

Product managers primarily support their teams in reaching their goals. The top two priorities of each PM are to run a growth review at the beginning of every month for each of their products, and to organize regular user interviews. (Our rule of thumb is 1 interview per week per PM).

The quarterly per-product planning typically highlight the biggest blind spots a team or product has (e.g. what metrics or parts of the product do we think have potential, but we don't have enough context yet). Teams are encouraged to include their "biggest unknown" as a research goal for the PM to own as part of their quarterly goals. Findings should be shared asynchronously via a GitHub PR in the product-internal repo, and in the growth reviews or team standups where applicable.

To keep track of their projects across teams, PMs should track their personal quarterly goals transparently somewhere, for example in the public PostHog Meta repo.

As the PM team, we are usually also pursuing a couple of side projects each quarter with the goal of leveling up how we do Product at PostHog.

In Q2 2026, we are working on the following themes:

- Ensure our metrics definitions are mature enough for our scale (e.g. inconsistencies in activation & product intent) -> Annika
    - Investigate ways to make updates to activation definitions sustainable & consistent -> Mike
- Figure out information flows between product, growth and marketing, so that we get the right exposure for new and mature products -> Abe
- Make sure our products & tracking are set up for a agent-first world
    - Tracking human vs agent actions, and defining success for each -> every PM
    - Make sure each product has use cases and a “reason for being” in an agent-first world -> every product team
- Think about ideas how to fix the “accountability gap” in growth reviews -> Cory
    - From product teams, expect more accountability
    - From PMs, get growth reviews done early in the month

“Business-as-usual” themes:

- Make sure we standardise best practices & share knowledge as a group of 5 (soon to be 7) without central leadership, and without adding bureaucracy & friction
- Use our PM brownbags to share how we use AI to be more effective

Releasing new products and features

contents/handbook/product/releasing-new-products-and-features.md

import { CalloutBox } from 'components/Docs/CalloutBox'

This guide walks you through the full lifecycle of releasing new products and features at PostHog, from initial planning to general availability.

For complete step-by-step checklists when creating a new product, use the new product RFC template.

Overview of the product lifecycle

New products at PostHog go through four phases:

1. **Setting up** - Initial planning and alpha development behind a feature flag
2. **Alpha** - Slowly adding customers you've spoken with to the feature flag
3. **Beta** - Opening up to all users who want to opt-in
4. **General availability (GA)** - Full launch with pricing and marketing

PostHog includes a variety of early access features in the feature previews section of a users' settings page, as well as a roadmap of feature previews which are coming soon.

Items in the feature previews section can be toggled on or off if users want to try a feature out. Items in the coming soon section enable users to register their interest so that we can contact them with updates. Both sections work only at the user-level and not at the org or project level.

Please refer to the RFC for what the actual steps are. Duplicating them here would cause them to go out-of-sync extremely quickly. We'll simply explain the rationale behind each of the stages.

Phase 1: Setting up a product

Adding items to the coming soon menu early offers several advantages. It enables us to gauge interest in a new feature via sign-ups, equips our marketing teams with news they can promote to users, and ensures that betas can have sample users ready from the moment they launch.

Coming soon features can either be large or small, so use your judgement about what is of interest to users, but it should be something that you expect to work on in the next 3-6 months.

Phase 2: Alpha

During alpha, you're testing with a small group of customers you've specifically invited. It's fine to have bugs and your testers know that's the case. You're also actively working on fixing all known bugs before we can move this on to an opt-in scenario.

Phase 3: Beta

Beta is when you open up the product to all users who want to opt-in. Betas do not need to have been in `concept` stage first.

Once you are ready to move an item from the coming soon roadmap to a beta which users can interact with, update the stage from `concept` to `beta` (or `alpha`). This triggers an automatic notification to all subscribed users letting them know that the beta is available. Users who registered interest during the Concept stage can then opt in to enable the feature.

Make sure your early access feature flag includes a `product_key` on the payload field to give people access to the product in their sidebar. Check the new product RFC for more details.

Beta requirements

A beta doesn't need to be perfect, but it should provide value to the user and have base elements of functionality. It doesn't need to be feature complete, but it should provide more than a mocked up front end. We aim not to leave items in beta unless they are in active development. All betas should be clearly documented.

Betas do not need to be performant for high-volume users and can have big bugs, but should be clearly marked as such in the UI.

Betas should include a title, description, feedback button, payload with `product_key` and link to basic docs

All betas should follow the best practices below in order to provide a minimum amount of information and usability for customers.

- Betas in the feature preview menu should include a title and short description
- Betas in the feature preview menu should include a 'Give feedback' button
- Betas in the feature preview menu should have documentation (marked as beta) linked to them
- Betas should have a feature owner
- Betas should have a `product_key`

Product teams are responsible for writing documentation, but the can help, if needed. Titles, descriptions, and links can be added using the early access menu.

It's helpful to let the Marketing teams know when new betas are added. They'll then add the beta to the changelog, organize any marketing announcements, plan a full announcement for full release, create an email onboarding flow to help you collect user feedback, and anything else you need. You can let them know via the Marketing Slack channel.

Collecting beta feedback

Teams are encouraged to collect feedback from users in current betas so that they can build better products and we have some automations in place to facilitate this.

After a week in any new beta, users will trigger an automatic email from the `beta-feedback@posthog.com` Google Group. This email will ask them, essentially, for any suggested changes to the beta. By default, all team leads and exec team members are in this Google Group and will get daily digests of responses. Others are invited to add themselves to the group, or change their notification settings.

Regardless, emails to this Google Group will sync to the PostHog Feedback Slack channel for general awareness. Team leads are encouraged to respond to beta feedback emails.

Teams can collect additional feedback if needed and the is able to help with creating feedback emails or funnels.

Phase 4: Launching to general availability

Once a beta is mature enough, you may want to launch it into general availability (GA).

If you're planning to launch your product in a specific quarter, you MUST let the Marketing team know at the start of the quarter.

Smaller features which don't require major announcements should be announced internally via the Tell PostHog Anything channel so other teams are aware.

You can set the feature flag to release to 100% of users BEFORE the Marketing launch, you don't need to wait for it.

See product announcements for marketing requirements during launch.

> **How do I work with marketing and billing teams?** The short version here is to try and give other teams as much notice as possible when starting a launch cycle. Marketing and billing teams typically ask for two weeks of notice before a major launch, as a minimum. It's the responsibility of the team lead to ensure these teams are aware of upcoming launches.

Who's responsible?

The Team Lead is typically responsible for:

- Creating and managing the RFC
- Keeping Marketing and Billing teams informed about product progress
- Ensuring timely communication (at least 2-3 weeks notice before a major launch)

Team members can be assigned specific tasks within the RFC checklist.

Related resources

- Deciding which products we build
- Small teams and launching products
- Product announcements
- Per-product activation
- Writing ClickHouse queries for new products

User feedback

contents/handbook/product/user-feedback.md

> 😍 Want to share feedback? File a GitHub issue or reach out directly. **We're always happy to hear from you!**

We actively seek (outbound) input in everything we work on. In addition to having multiple channels to continuously receive inbound feedback, we generally do active outbound feedback requests for:

- General product and experience feedback. Continuous effort to gather general feedback on the product and their holistic experience with PostHog are led by the PMs supporting the team.
- Usability tests. We generally run these for new big features the Engineering team is working on. Run by the engineer building that feature.

Feedback call process

Recruiting users

Ways to invite users for an interview:

- PostHog Surveys. We even have a template for user interviews.
    - We recommend to create a cohort first (with a static copy), and use that as a display condition.
    - We have more and more surveys running, therefore it's best if a survey wait period is applied. We recommend 14 days.
    - If your cohort is large (>1,000 users), it's best to not roll it out to 100%, as you might get overwhelmed by the amount of interviews in a short period of time. Start with 20-30% and increase if you need more interviews.
    - PMs use cal.com for scheduling interviews, but you can choose a different tool as well.
- If a customer has requested a feature through Slack, then message them directly.
- Email customers who have subscribed to the feature on the roadmap.
- Email a slice of users, e.g. top users of your product, or conversely, people who churned.

Email writing inspiration

When crafting user outreach, just put yourself in the shoes of the person about to receive the message. How can you help each other by getting on that quick call?

Here's an example of an email from a real project, crafted by to learn about the problems of top users of the PostHog AI beta:

> Subject: Quick chat about your PostHog AI experience?
>
> Hey $FIRST_NAME, Michael from PostHog engineering here!

> I'm focused on improving PostHog's AI features, and I saw you've been using our AI assistant. I want to make PostHog AI 10x better for you – and it'll be a gamechanger to hear about your personal experience with it.
> What do you say about a 30min chat about your product-building workflow, this week or in a couple of weeks? I promise in return you'll get a better tool for your job, plus $40 of PostHog merch. :)
>
> Feel free to pick any time that suits you in my calendar at $CAL_DOT_COM_LINK, or send me your own calendar! I'm excited to hear from you.
>
> $GMAIL_SIGNATURE

Scheduling

- Add all feedback calls to the User interviews calendar. If the invite was created from your own calendar, you can simply add "User interviews" as an invitee.

During the call

- We recommend recording interviews using BuildBetter. Please, always ask and make sure the user is comfortable with being recorded before doing so.
    - Once the user confirms they are okay with this, you can easily trigger the recording by clicking on `Join a call` on the BuildBetter homepage and copy-pasting the call URL.
    - If you do not have access to BuildBetter yet, drop a message in the `#pm` Slack channel.
    - If you have any feedback, bug to report or feature request for BuildBetter, we have a shared Slack channel with them, feel free to directly message their team there.
- Do a quick round of intros if you haven't met previously.
- If this is the first interview with the user, ask them for context about their company, their role, if they're technical.

After the call

1. If you used BuildBetter, the tool will automatically generate a summary for you under the recording. We recommend checking this, and adding any additional thoughts, because the AI can sometimes pick up things incorrectly. You can also generate a doc using the platform, where you can give very specific prompts for the outline of the summary.
    1. We also want to keep recordings easily identifiable, therefore please rename the recording to `[topic of the interview] user interview with [first name of the user]`, e.g. Web analytics user interview with Joe.
2. In case recording wasn't possible, add the notes to the Google Doc.
3. Share a short summary of the user interview in the `#posthog-feedback` Slack channel.
4. If the user reported specific bugs or requested specific features, open the relevant issues in GitHub. Be sure to link to their person profile in case our engineers needs more context when scoping/building.
5. Generate the reward for the user (see below).

    1. Most of the time, the reward will be a gift card for the PostHog merch store. If it's the case, create the gift card in Shopify.
6. Follow-up with the user. Send any applicable rewards, links to any opened GitHub issues, and answers to any outstanding questions.

Rewards

We strongly value our users' time. As such, we usually send a small gift of appreciation. We have the following general *guidelines*, but just use your best judgement.

- As a thank you for a call, we send users a gift card to the PostHog merch store with around $40 of value (great for a swanky T-shirt plus stickers).
- If the user wasn't up for a call, but nevertheless replied with a bunch of useful feedback async, it's good vibes to send them smaller gift card, with $20 of value.
- When merch isn't an option (e.g. user has received some already), we can offer the user an equivalent-value gift card with Open Collective.

Instructions on how to create gift cards can be found in the merch store customer section.

Repositories of information

We keep a log of user feedback in the following places:

- **BuildBetter.** Starting 2025, we keep track of all user interviews (recordings & notes) in BuildBetter.
- **Feedback notes.** Feedback notes are mainly kept in this Google doc.
- **Old recordings.** All older recordings are kept in this folder in the Product shared drive.

Wherever feedback happens, share it

Any PostHog team member may receive feedback at any time, whether doing sales, customer support, on forums outside of PostHog or even friends & family. If you receive feedback for PostHog, it's important to **share it with the rest of the team.** To do so, just add it to the #posthog-feedback channel.

> To ensure feedback durability and visibility, the #posthog-feedback channel should not be used as the primary source of *storage*. Please add the feedback to the main Google doc.

We strongly recommend that everyone joins at least one user call per month. Regardless of your role, you will always benefit from staying in the loop with our users and their pain.

In-Person Customer Visits

`contents/handbook/product/visiting-customers.md`

Right now, PMs conduct a lot of remote interviews with customers about their specific products to bring context to their teams. As the number of PostHog products grows, and as customers increasingly use multiple products together, small teams risk developing a siloed view of how our customers actually use PostHog.

This matters because:

- We lose context on how customers use multiple products together to complete their “job to be done”.
- Teams optimize locally (their feature, their metric) and can accidentally degrade the cross-product experience.
- We miss expansion opportunities because we don’t see how multiple PostHog products are used in the wild.
- We miss important context on how to make a whole organization successful with PostHog, not just a single user or use case.

One partial solution is to **go meet customers in person.**

The following is a guide to share what has worked, and what hasn't for others who might want to try this out.

Setting up meetings

1. **Create an open flexible script.**
   Develop a small, flexible set of questions related to your product area, but keep them broader than your typical product interview. Leave space to understand organizational dynamics, team workflows, and feedback across PostHog’s full product suite.

   Feel free to use interview time to watch customers use PostHog directly, and ask them questions about why they take specific approaches as they navigate around the product.

2. **Pick a metro hub, and research potential customers.**
   Choose a city with a good number of PostHog customers. Aim to identify ~12–15 customers across different sizes and maturity levels in that region. 15 may sound like a lot but you'll likely only be able to talk to 30% of these customers in the end.

   You should do deep research on each account - take notes of which products, and features our customers are using Vitally (and of course PostHog). Who at those companies is using specific features and have a look a few session recordings. You should refer back to these notes before meeting the customer in person so you are informed.

3. **Coordinate with Sales and Customer Success.**
   Post in the relevant Sales and Customer Success Slack channels about your plans. Tag the account owners for the customers you'd like to visit. Ask if it's a good time to reach out, whether they have additional context, or if there are other relevant customers or prospects to reach out to as well.

   > Hey @[sales/cs members] I'm visiting [City] in 2 weeks and would love to meet some of our customers in person. I was thinking of reaching out to [customer a], [customer b], [customer c] Is now a good time to chat with them? Any other company who uses [product] a good company to go visit?

4. **Join customer Slack connections.**
   Introduce yourself in each shared channel. A good intro might look like:

   > 👋 Hey everyone! I'm [Your Name] from the PostHog product team — I'm visiting [City] next week and would love to meet some of our customers in person. If you're up for a chat over coffee or lunch to share feedback on how you use PostHog please let me know

5. **Have your Sales owner tag relevant people** within the customer org in that Slack thread. When the account owner directly tags people, the response rate increases significantly. Here are some examples of what worked:

   > @[relevant customer member] any ideas on who from [customer] may be around and interested in this? We find this kind of thing to be pretty mutually beneficial for us to learn about your needs, but also to help shape our offering as well

   > @[relevant customer member], [Your Name] from the product team is visiting [City] next week. They can visit and help your team better get better set up with [products]

   > cc @[relevant customer member]

6. **Schedule meetings.**
   Send calendar invites to everyone who responds positively. (We have typically found about 30% of outreach resulted in a meeting.)

7. **Remind participants.**
   Post a friendly reminder in the thread a day before each meeting.

Conducting meetings

8. **Be flexible.**
   Some meetings will last 30 minutes, others 2 hours. Lunch-time slots often work well, you can grab food together, build rapport, and then dig in. If other posthog team members are free and local in the area and want to come along, feel free to bring them too.

9. **Bring merch.**
   Small things like hats or shirts go a long way. Drop a message in #merch and Kendal can help you place an order.

10. **Structure the time.**
    An effective structure we found was:

    - 20 min casual lunch or intro conversation
    - 30 min “enablement session” — A place and time for as many of the customers's employees to come ask any PostHog-related questions they’ve been holding onto
    - 20 min focused discussion — Your prepared questions and reflections. If you have a enablement session you can often sneak these questions in while working with customers.

After the meetings

11. **Follow up.**
    Thank them for their time, and if the customers had questions you could not immediately solve in your in person meetings, message and tag employees at PostHog who could help.

12. **Reflect and Share.**
    You should walk away with a much clearer view of PostHog from your customer’s perspective — not just how they use your product, but why they use PostHog, what types of questions or jobs they are trying to complete with PostHog, and how they use *PostHog as a whole*.

    Share this in the **posthog-feedback channel** or somewhere similar. Feel comfortable sharing a longform write up and tag people and teams that are relevant. Here's an abbreviated example:

    > I had the opportunity to meet multiple customers in person over the last few weeks. I went in with the intention of focusing on data pipelines and messaging. However, I was open to receiving feedback on our entire product suite. Two interesting customers were [customer a] [customer b] [customer a] does xyz, and currently uses product analytics, session replay, data pipelines, and feature flags. I was able to talk to [customer employee name], who is the head of engineering, as well as another employee who ran a business unit. They were the power users. They use product analytics in two main ways:
    >
    > 1. Business review. They had a high-level dashboard that was setup earlier and the leadership team would view it every single week, tracking changes in things like MAUs and other business critical product KPIs.
    > 2. Ad hoc product insights. If the GM of one of the product lines had a question that popped up in his mind, he would go to PostHog to try to answer it first. They used insights and dashboards and were pretty comfortable with breakdowns and some of the more advanced features. However, interestingly, they did not use sql queries and they were not aware that you could even use sql either. Data pipelines was used to inform colleagues of high-value customer interactions in the product. This was

> done via a Slack destination. This is a very specific job to be done that I've seen in a number of other companies as well. They were particularly interested in organization-level views, specifically:
> - Organizations who have completed key events
> - Organizations who have not completed key events The primary complaint and frustration that they had with PostHog product analytics was that it was very difficult to search for people who have not done things or organizations where things have not occurred. I was surprised that both [customer a] and [customer b] did not know you can use sql insights with product analytics @product-analytics-folks One other thing to note was both customers also did not know where they could find a list of all events, and definitions (I showed them the data management tab). One customer commented it would be good if AI added a description automatically of what each event actually signified, and if there were easy ways to delete old events. Data governance came up a lot even with this mid sized companies

How we got here

contents/handbook/story.md

Things that influenced us

Books

- No Rules Rules (Erin Meyer / Reed Hastings)
- Principles (Ray Dalio)

Other companies

- Atlassian – multi product, inbound, dev centric company, totally dominant in their categories, scaled way beyond $1bn
- AWS – multi product, pricing model, UX (!)
- Pager Duty – pricing model, product led growth
- Hubspot – built a $25bn company despite competing with Salesforce
- GitLab – while we run *very* differently and have a different business model, we were inspired by their transparency
- Sentry – branding and bottom-up approach
- Algolia – developers doing marketing
- GitHub – enterprise go-to-market is 200 developers forcing their company to buy us

Handbook

Like many things at PostHog, this handbook has scrappy origins.

Tim and James were planning on launching on Hacker News, and wanted to look as mature as possible. We felt that few people would want to use a flaky new startup's product seriously. So we asked ourselves: how do we signal that we're mature?

We looked around at some big, boring companies and realized they all had huge footer sections on their websites with lots of links! How do we produce a lot of content to add to our footer when the product, at that time, was so simple? The answer: we should write up how we want to work.

Once we started writing the handbook, we realized it would transform our company. Every team member, and even strangers on the internet, could suggest changes. If you're doing something in public, you're going to think it through better. Ultimately, it made us treat the company as our product.

It's a classic example of getting information by doing, rather than by planning too carefully.

Timeline

January 2020: The start

PostHog was founded by James and Tim on January 23, 2020.

We started working together on a startup in August 2019. Our first idea was to help engineers manage technical debt. It didn't work out, but we realized the power of treating growth as an engineering problem. We also knew that many engineers struggle to understand the impact they have on the people who use what they build.

There are plenty of product analytics tools out there, but all the alternatives are SaaS-based. While they're powerful, they can be frustrating for developers. From our perspective, these tools can be problematic because:

- We don't want to send all our user data to third parties
- We want full underlying data access
- They don't give you choice and control over pricing

February 2020: Launch

We got into Y Combinator's W20 batch, and, just a couple of weeks after starting, realized that we needed to build PostHog.

We launched on Hacker News with our MVP, just four weeks after we started writing code. PostHog was our sixth idea – we had been pivoting almost once a month for half a year. Boy were we relieved!

The response was overwhelmingly positive. We had over 300 deployments in a couple of days. Two weeks later, we'd gone past 1,500 stars on GitHub.

Since then, we've realized we weren't just onto a cool side project, we were onto what could be a huge company. It turned out there were a lot of developers who liked us who wanted a better choice, built for them.

April 2020: $3m seed round

After we finished Y Combinator, we raised a $3.025m seed round. This was from Y Combinator's Continuity Fund, 1984 Ventures.

As we started raising, we started hiring. We brought on board Marius, Eric and James G.

May 2020: First 1,000 users

We kept shipping, people kept coming!

October 2020: Billions of events supported

This was a major update – PostHog started providing ClickHouse support. Whilst we launched based on PostgreSQL, as it was the easiest option to ship quickly, this enabled us to scale to billions of events.

November 2020: Building a platform

We realized that our users, whether startups, scale ups or enterprises, have simple needs across a broad range of use cases in understanding user behavior.

PostHog now supported product analytics, feature flags, and session replays.

December 2020: $9m Series A

We kept growing organically and took the opportunity to raise a 9*MSeriesA*, *toppingourfundingupto*[12M](https://posthog.com/blog/posthog-announces-9-million-dollar-series-A) led by GV (formerly Google Ventures).

Our focus remained firmly product, engineering and design oriented, so we increased our team in those areas.

We now had employees in ten countries, and it still felt like day one.

Everyone takes a mandatory two weeks off over Christmas to relax.

June 2021: $15m Series B

We raised a $15m Series B a little ahead of schedule, led by existing investor Y Combinator.

We're now focused on achieving strong product-market fit with our target segment in 2021.

Our team had grown to 25 people in 10 countries.

September 2021: Product-market fit achieved for PostHog Scale

We achieved product-market fit for our open-source product and PostHog Scale.

Our revenue quickly rose as a result. Now we needed to optimize it.

We were 30 people in 12 countries.

January 2022: Sales comes from our team, not our founders

We hired two Customer Success experts dealing with all inbound requests. We hired two more engineers, since most questions customers have are technical.

December 2022: 6x revenue growth

We had a fantastic year. While the tech market crashed, we grew 6x and reached millions in revenue, with a sub-two-month CAC payback period. We set $10m ARR as our next goal, with a gross margin of 70% – both of which should mean we've got all the metrics needed for the next fundraise.

We optimized revenue growth by implementing a product-led CRM for our customer success team, adding to our content team size, and creating a two-person growth engineering team. These teams all make a big difference!

We deepened all of our product areas significantly – we frequently win deals as a standalone session recording, feature flagging or experimentation tool. Session recording usage started to match product analytics usage.

Our infrastructure is far more stable and scalable – much more of it runs as code. We can now offer EU- or US-based hosting for our customers' data.

We're now 38 people in lots of countries. We're not adding lots of headcount over the next 12 months, though. We're staying lean and letting revenue continue to rise rapidly.

February 2023: Focus on mass adoption

We're doing well at monetizing high-growth startups due to our optimization work, averaging over 15% MoM for the last six months.

We've decided to double down on mass adoption of the platform in high potential startups instead of focusing on enterprise. Simply, this will better help us increase the number of successful products in the world. As a result, we've removed support for paid self-hosted deployment and are doubling down on our open source and cloud projects. We have released a free tier of PostHog.

We went from "product analytics with some extra stuff thrown in" to "Product OS" and started charging for session replay separately.

In the product, we're working on making the experience slicker, and we have plans for a standalone quality CDP in Q2.

March 2023: Decided to ship a warehouse

For a long time, we were happy competing with lots of $1-2 billion companies, each providing point solutions. We felt our market was just the sum of all of theirs.

But we kept seeing companies streaming their PostHog data to a warehouse - such as BigQuery. We even lost our then-largest customer for this reason - where their source of truth became their warehouse instead of PostHog.

So we decided we would ship our own warehouse, enabling us to remain the source of truth for customer and product data. This would let us offer a better integrated service to our customers, and meant we could work on a bigger challenge.

August 2023: Growth continues

We've doubled revenue so far this year without any increase in headcount. We've hit 15.7% MoM for the last 12 months. Our CAC payback is now just five days. Our numbers are exceptional. We even discounted several of our products. We've added ten extra roles and will be profitable in around a year.

We have user surveys and the data warehouse in private beta. Other products are being positioned as first-class products of their own (AKA "The Great Unbundling"). This means we can make it clearer for new folk to get what we do, give more ownership (which means more speed) to our own teams, and we can compete on commercials more effectively.

Our infrastructure has become pleasingly stable. The biggest challenge is scaling our data pipeline, and making sure we give as much responsibility as we can to each small team owning each product for their own pipelines, where rational to do so.

October 2023: Winning the internet

We are often mentioned as an alternative to product analytics tools.

We are winning the internet when we get more of this for our *other* products. We don't have to win everything, but we need to get into the comparison each time. This is *starting* to happen, but to *win* the Internet, we need to see this happening daily.

Multiple products are early like the warehouse, ETL, surveys (used a lot but not paid), feature flags and experimentation (first revenue), CDP (pipelines being rebuilt, webhooks next), notebooks, and web analytics.

There is a lot of supporting work needing to be done. This includes:

- Helping teams with their per-product onboarding and growth experiments, infra, ingestion, dev tooling, sales, support, and marketing.
- Promoting each product in its own right (i.e. through what we cover in marketing).
- Keep nurturing the content / community growth, i.e. newsletter, and the `/posts` concept.

January 2024: Well, that was good

That was quite the year.

We wound up quadrupling our revenue, but only increasing our net headcount by three people in 2023. Last year, we validated that we could get multiple products to product-market fit (like feature flags and experimentation).

We built more integrations between our products like HogQL, notebooks, CDP, and the data warehouse.

Now, we are doubling down. We shipped a lot in Q4 2023, but every product could be improved a lot. We're caring about the craft of our products:

- Major missing features vs competitors
- Scalability/stability
- Developer UX
- Talking to users and incorporating their feedback
- Nailing support for your product - fixing things

Products are not limited to engineers working on the app. It includes what customer success, marketing, and ops are working on. Everything can be considered a product.

Each team should be aiming to feel proud of what they've built by the end of the quarter.

April 2024: We're now the default for startups

54% of the first Y Combinator batch this year adopted PostHog.

Tim and James turned up to talk at batch events and we were surprised at the number of groupies wearing PostHog merch – our merch is really cool now, we've gone way beyond the logo-on-a-black-t-shirt standard.

As far as we can tell, we're in the top three products used by YC companies.

July 2024: Price cuts ftw

We cut pricing by up to 80% for our two most popular products, including for our existing customer base. This was popular with users and led to faster growth.

We've started doing growth reviews for almost every product we have. We run through each product's metrics (revenue/usage/support/performance) and feedback / reasons for any churn that has happened, so we can truly treat each small team like a startup. This session is designed so the engineering team leads may choose to reprioritize work, or not.

October 2024: 100,000 customers, and speeding up – more products and more people

We hit 100,000 customers either paying or free, and over a quarter of a million users. We've started hiring a lot faster as growth has continued this year. We're now 65 ish people with ~9 products.

We've added some people in sales, but it is strictly (i) sales assist, talking to people that have asked to speak to us, and (ii) cross sell to existing customers.

We do *not* do outbound, so we can remain efficient and either hire more engineers or cut our pricing for our customers so more of them recommend us!

We've hired a sales engineer super early (Mine, she's awesome) and we're really working on the culture in this team proactively.

Strategy-wise, we're just leaning into our basic three principles, which we're seeing more and more evidence are working well:

1. **All the tools in one** – We want to go wider still. We think we can provide *every* piece of SaaS that startups use, starting with those closest to customer data. We want to expand to a customer support product, the marketing and sales stack of tools too.

2. **Get in first** – Don't go upmarket. We're closing enterprises regularly, but we're not trying that hard here. We're trying to stay away from complex migrations for users who use many products already.

3. **Be the source of truth** – Our own data warehouse is now available and very popular.

Revenue is in the low $10s of millions of ARR. We're very strongly default alive and will struggle to not end up profitable next year. Every time we get close to being profitable, we start speeding up hiring.

Revenue growth is fast enough and we're getting so many unprompted offers for investment (that we aren't taking) that money isn't really a meaningful constraint anymore. Whilst we have a great grip on each product's individual performance, our understanding of cross sell is a little weak, so we're working on that now.

Our marketing is getting weirder. It's more and more fun. We've commissioned a puppet, coming in January. Watch this space. Our newsletter, Product for Engineers, now has 20,000 subscribers and it's growing fast.

We're realizing that the more ambitious we are, the easier it gets – customers get excited, investors get excited, employees get excited. We can now see a real path to being a $100bn+ company and changing how software teams work industry-wide.

Brand

`contents/handbook/strategy/brand.md`

Brand matters to us, greatly. It's one of the four major reasons people get recommended PostHog so directly helps us grow. Everyone else is largely terrible at it so it's a massive opportunity to build a long term advantage as a company, and frankly it's fun. It's every interaction we have with our users and comes from how the company itself is designed. It's more than hedgehogs:

- How our pricing works
- The way we word our emails
- The vibe on sales calls
- *You get it by now*

The harsh truth of cat videos

When it comes to attention on the internet, you are competing with cat videos and TikTok, *not* B2B SaaS competitors. Be realistic - if it's not actually funny (and it's "corporate try hard") then it's not good enough. At one point we realized we were getting cutesy - "ooh a hedgehog". That's not interesting enough for people outside PostHog, even if we think it's cool.

It is thus encouraged to be rogue / sarcastic / meme-y / unhinged / weird.

Our competitors are (i) more defensive and self-interested in their approach (focused on optimizing revenue growth), and (ii) more boring. Let's keep it that way. If we have fun, we'll stick it out longer and will win in the long term.

Brand first

We should always optimize to not piss users off unless they're being totally, extremely unreasonable, in which case figure out how to be the bigger person. Even when that costs us revenue.

For example, we should refund customers when they screw up their tracking and get a shock bill.

Pavlovian merch response

Give it out to people who say nice things about us. That'll create an army of developer warriors fighting for PostHog on the internet!

Breaking bad news

Sometimes you may need to tell customers something they don't want to hear - e.g. "we don't have X planned in our roadmap". Instead of a vague "I'll share this feedback" type response, be specific and give context like "Hey we don't have that planned because we're focused on X, Y, Z at the moment. If you want to suggest it to the wider team, you can do so by X".

Karma

Be helpful to other companies. We are here to increase the number of successful companies in the world – especially those with high potential that are putting in the work, like YC current batch ones. For example, if a YC company reaches out, take them seriously and buy their product (if it's genuinely valuable and safe to do so) or give direct feedback if not.

Hacker News premortem

Hacker News is a very intensely logical and critical place – in a good and bad way. If you are doing something think, "How would this go down on Hacker News?" If the answer is "poorly" then change it. This rule of thumb applies to everything, not just stuff literally getting posted there.

Brand assets

We keep a comprehensive list of brand assets and guidelines for their use on the dedicated brand assets page.

Strong team

contents/handbook/strong-team.md

You're the driver is one of our values. 90% of a startup's problems are solved by just having the right group of people in ~~the building~~ Slack.

Personality traits that cause people to be successful here

Genuine builders. Some people do jobs for the money. Those that have truly found their passion are *far* stronger.

Easy to work with. People who are low ego, flexible, energetic, and upbeat will raise those around them. We often, but don't exclusively, hire those with more experience since it's easier for them to contribute meaningfully. Things can and do get *very* hard here – whether it's scaling, shipping complex products, handling a stream of support requests, or trying to ship something that touches multiple teams. We need those who won't get disheartened, and will collaborate, iterate, and ship their way out of anything. We proactively reward those that do these things, not those that self-promote.

Will join us on the journey. Some people are inspirational to work with – they lift others up. We have a *huge* opportunity at PostHog, and it often feels like we've caught lightning in a bottle. Anyone joining the company at this stage could make this the last job we all ever need. We want people that will push to get this done, for each other's sake. We don't hire mercenaries. We need to feel people here are producing the best work of their lives.

Drivers not passengers. Proactive people that can fully own projects and get them done (or make sure they get help) are what we need. For many of our roles, while it isn't a common job title, internally we have the concept of product engineers – people who can take high-level requirements, decide what to build, do so with customers, and keep iterating.

Great (and terrible) reasons to join us

Let's start with **why you *should* join:**

- You want to ship an epic product with incredible people.
- You want impact and autonomy, and work well with uncertainty.

Why you *should not* join:

- Getting our brand on your resumé. If you join for self-promotion, you (ironically) won't do well! Apply to a bigger company who can give you a clearer career ladder.
- Getting a pay rise. We pay generously, but you'll need to *love* building to be happy here. You'll need to be here a long time to get the real upside from options.

- Mainly wanting to lead others. Reluctant managers are often the best. We don't pay more if you manage others. We want people to lead by example by doing an exceptional job of individual work.

A small group of stronger people and compensation

When we raised our Series A, one of the first things we did was to make sure we didn't lose our existing team (at least for pay reasons!) *before* we added more people to it. This is still true today – we proactively review everyone's pay three to four times a year and increase it if people have leveled up.

When it comes to churn due to pay, fairness is just as important as the absolute level. We do this in line with a transparent pay system that we even make public. We aim to pay generously and fairly between people.

For options, we offer the most generous terms possible as it feels like the right thing to do. We think this makes it as likely as possible people can see huge upside if we are successful (making it easier to raise *and* more realistic that people will actually get money from them). That motivates everybody.

One of the hardest parts about building a high performance team, is letting people go when they aren't performing. We are decisive and do this faster than many others would. We offer four months severance when we let people go for performance reasons to give people more time to move on – and so it's easier for us to make a change if we need to.

While we will give direct feedback, if we don't see this being responded to quickly ahead of letting someone go, we will part ways, so people can find a job they are better suited to, and so we can find a team member better suited to the job. The end result is that everyone on the team is contributing meaningfully.

Growing the team beyond hiring

We hire insanely talented people to build products ourselves, but sometimes acquihires or acquisitions help us move faster by adding engineering capacity and expertise. These situations are rare so we're often reactive with these - but we've set clear principles to make sure we handle them consistently.

Acquihiring

This is an efficient way to onboard great engineers without all the complexity of an acquisition. For us, an acquihire means closing down your old company as you and your team join PostHog purely as talent, which we will match up with our product teams where it makes sense.

- **Everyone goes through our standard interview process.** There are no exceptions, even if you join as a group. will organize each interview stage with your team members individually so everyone goes through the process in the same timeframe.
- We do not pay for acquihires - we just hire the people. Sometimes we'll pay a premium if it makes hiring multiple people easier.

    - For YC founders, we may sometimes pay a premium. This is treated like additional compensation that vests over the standard PostHog equity schedule (not a lump sum upfront).
    - For engineers, we pay our normal salary with the possibility of a discretionary bonus after probation.
- If you quit PostHog to start your own company, we won't acquire or acquihire you back (though former employees-turned-founders are welcome to apply and join again under normal hiring processes).
- We never acquire companies bigger than one small team.

Acquiring products

IP inside PostHog

By default, we are not interested in acquiring IP. If we can build something ourselves, we will. The only exception is IP with deep technical value we don't believe we can replicate internally.

If we want a product inside PostHog but lack the domain expertise, we might acquire the team behind it - with the expectation your team rebuilds the product natively into our platform and migrates users. This would be the case where, without domain knowledge, projects might take us an unreasonable amount of time to ship or get deprioritized. Even then, we will only do this if the price is right. We generally won't pay a premium as the value comes from the team's expertise, not the legacy product.

IP outside PostHog

We do not acquire products that sit completely outside our platform (e.g. an IDE). Our strategy changes often, and owning something disconnected would create pressure to keep it alive.

The exception would be if the product technically lives outside the platform but directly enhances PostHog's value (e.g. a new way to use PostHog data inside a terminal), where we may consider an acquihire or paying a premium.

Existing customers

We generally do not want to convert any existing customers you have into PostHog customers directly. They may be different from our ICP and put pressure on the team to build something different than what we would otherwise plan to offer.

Your customers are of course welcome to sign up for PostHog and use our existing products and new ones once they are launched, but we don't make promises to these customers about features or support for their existing workflows.

Acquisitions for marketing

We are not interested in acquiring companies just for their audience or marketing. That's a distraction, and we're confident in the strength of our own marketing team. If we want more marketing, we'll invest in it directly.

Customer support

contents/handbook/support/customer-support.md

You can build a good company by focusing on getting lots of customers. To build a *great* company, you must delight your existing customers. This means that the journey doesn't simply end once we sign up a user - even more important is to ensure that PostHog is consistently delivering value for them.

How we ensure amazing customer support

It's easy for customers to reach us

We have a few different routes for users to contact us. As an open source company, our bias is towards increasing the bandwidth of communication with our users and making it easy for them to reach us through a clearly defined, simple set of channels.

These are the ways in which customers can currently reach us:

- **Support ticket** - Customers can create a support ticket directly within the PostHog app, under the help menu. This offers both users and PostHog engineers the best possible experience as Zendesk is automatically populated with a bunch of helpful context that makes troubleshooting easier. When in doubt, customers should be directed here.
- **Community questions** - users can also search previously answered questions that have been asked anywhere on posthog.com in our Docs. This is a great way to help us improve our Docs for simpler use-case type questions, but more complex questions should be re-routed via a support ticket.
- **Dedicated Slack channels** - For higher-paying (or potential higher-paying) customers, we offer a dedicated channel on our main company Slack.

Sometimes, people reach out to us with support issues on Twitter/X. Regardless of whether someone reaches out to your personal account or to the company account the broad approach should be as follows:

1. Check first if they already have a ticket in Zendesk (either in-app or via /questions). There is nothing more annoying for a user than being asked to create a support ticket if they already have. If you don't have Zendesk access, ask someone in CS.
2. If no tickets exist, explain that we can't provide support over social media and ask them to create a support ticket within the app - this is *much* better than trying to solve their problem over Twitter as Zendesk pulls in a bunch of contextual information and is easier to collaborate in. Do this from the **PostHog** Twitter account - otherwise you will get personally contacted every time this user wants help.
3. If yes, say that we can see their ticket and reassure them that all tickets are triaged and responded to. Let CS know that you have done this. Again, use the PostHog Twitter account.

Your objective should be to get the conversation into Zendesk ASAP, because it's easier to help the person there and to avoid setting a precedent that complaining visibly on social media results in an expedited response. An exception to this rule is if you are engaging with someone who has provided general feedback about PostHog - feel free to use your personal account if someone has a feature request or similar. If a user engage in a way which causes you *any* distress, you can skip all of the above and just highlight it in Slack for CS to deal with.

> Sometimes users ask about the progress of certain issues that are important to them on GitHub. We don't consider GitHub to be a proper 'support' channel, but it is a useful place to gauge the popularity of feature requests or the prevalence of issues.

Support is done by actual engineers

All support at PostHog is done by actual, full-time engineers. We have two types of engineers:

- Support engineers, who are focused solely on support across multiple products and sit in the
- Product engineers, who are focused on products and take on support responsibilities in a support hero rotation

What do Support Engineers do?

Right now, support engineers provide the first level of support for the following teams:

- Product analytics
- Web analytics
- Session replay
- Feature flags
- Experiments
- Surveys
- Data warehouse
- Batch exports
- Sales & CS (Customer Success)

Support engineers respond to and solve as many tickets as they can for these products, or escalate tickets to the appropriate product engineer if needed. For all other products, the engineers on those teams are directly responsible for support. The support runbook is maintained on the Support Hero page.

When we hire new support engineers they will usually spend the first few weeks focused just on product and web analytics tickets, until they've started to build more familiarity with the platform as a whole.

What do Support Heroes do?

One person on each product team takes on the **Support Hero** role each week. This is a rotating responsibility, where the person involved spends a significant chunk of their time responding to support queries across Slack, email and Zendesk, and sharing that feedback with the team and/or building

features and fixes in response. We find each stint as Support Hero throws up a lot of really valuable feedback.

Response targets, SLAs, and CSAT surveys

Response Targets

We have a high volume of tickets and we're a small team, so we're not able to respond to all issues equally. For this reason we prioritize tickets according to the customer's plan. We set a response target for each plan so that we can be sure that tickets are being handled effectively.

Note that tickets are automatically prioritized in Zendesk and users are updated with information about response targets to set appropriate expectations. In all cases, tickets are routed to the appropriate team and that team is responsible for meeting the response target.

The response times listed below are targets for an initial response, and it's possible we will respond faster. These targets are listed in calendar hours Monday - Friday. Please note that we do not offer any level of weekend customer support.

Plan level	Target response time
Free	Community support only
Pay-as-you-go	72 hours
Boost	48 hours
Scale	24 hours
Enterprise	8 hours

Within Zendesk, we will further prioritize tickets based on their selected severity. If you come across a ticket that doesn't have the severity set appropriately according to our severity level guidelines, then you should update the ticket with the appropriate severity level.

As a general rule, we aim to prioritize customers who pay for support, or who are otherwise considered a priority customer, to ensure they get the best possible support experience.

> ***NOTE:*** If a user has recently upgraded to the Enterprise plan, their tickets may not automatically be tagged as Enterprise in the PostHog Priority field in Zendesk. If this happens, manually set the Priority field to Enterprise to ensure they get in the proper queue.

Follow-up / next reply response targets

Our follow-up response targets and next reply targets are the same as the initial response targets. We believe that customers should receive regular updates on the status of their query - even if the update is that we're working on it and there's nothing meaningful to report at present.

Escalated ticket response targets

When support engineers need to escalate issues to other engineering teams for deeper investigation, the investigations can take longer but we should still check in with the customer to let them know! For escalated tickets, our response targets are the same as for all other tickets.

> ***NOTE:*** The targets are for a reply to the user. If the escalation turns out to be a bug or feature request, the reported issue doesn't have to be solved by the response target date, we just need to reply to the user. That reply may be to let them know it won't be fixed right away, but that we have opened a bug report or feature request. If we've opened a feature request or a bug report, you can refer the user to the GitHub issue for updates, and `Solve` the ticket. If you're replying with info that should resolve the issue, leave it in a `Pending` state (will be auto-solved in 7 days if the user doesn't reply.) If the user replied to confirm the issue is resolved, `Solve` the ticket. Use `On-Hold` sparingly, e.g. if you intend to get back to the user soon (more than a week, less than a month.)

CSAT Surveys

We send out CSAT surveys after a ticket has been closed for at least 3 days using this Automation. The emails contain a link to https://survey.posthog.com/ with their `distinct_id`, `ticketId`, and the assigned team as query parameters, which are being used alongside their satisfaction rating to capture a `survey sent` event. The code for the survey website is in the PostHog-csat repo and the responses can be viewed in this dashboard.

As an incentive, we offer to feed one hedgehog for every survey sent. is the current holder of the hedgehog feeding rights, and takes care of this by making a quarterly donation to the Suffolk Prickles Hedgehog Rescue Charity.

Guidelines for doing support at PostHog

Dealing with difficult or abusive users

We very occasionally receive messages from people who are abusive, or who we suspect may have a mental illness. These can come via the app, or Community Questions. We do not expect support engineers to deal with abuse of any kind, ever.

If this happens, notify , or . They will either take this on, or advise you on how to reply.

Dealing with legal requests from users

We very rarely receive messages from people wishing to make a legal claim against PostHog, such as cease and desist letters. These can come via the app, or Community Questions. Do not respond to these requests. Instead, notify or immediately. They will either take this on, or advise you on how to reply.

Dealing with billing issues

Issues related to billing are handled exclusively by our billing engineers. Billing support is currently lead by . Most tickets get routed directly to the , however some issues require technical investigation before the billing issue can be resolved. In such cases, add as a follower to the support ticket from the outset, and leave an internal note briefly explaining what will eventually be required. Complete whatever technical investigation is required and then let the customer know you are handing them over to the .

Users asking for demos, consultations or partnerships

We often receive requests for demos, consultations or other sales-related requests. Most of the time these can be escalated to the Sales team if they arrive via Zendesk. If they arrive directly via email you can forward them to sales@posthog.com.

We also often get requests for partnerships, backlinks, or messages trying to sell us baby Yahama pianos. Sometimes, people want to invest in PostHog. Most of these are obviously spam and can be ignored, but if you think an opportunity may be genuine then you can forward it to so he can take over.

Users asking for their data to be deleted

Most of the time users can self-serve deletion requests and should be encouraged to do so in order to save time and ensure they take responsibility for deleting their own data. Users can delete their environment, project, and organization in the appropriate 'Danger Zone' section of their settings page if they have the correct permissions. Admins can remove members from their organization in the Members page.

If a user refuses to delete their own data, you must first confirm they have the permissions to do this by checking their email address matches that an organization admin. As an extra layer of security, you should also ask them to confirm their address by emailing you directly from it (e.g. not through Zendesk.) Only then should you delete any data on their behalf.

If a user asks for us to delete all of their *personal* data in compliance with GDPR, you should confirm their identity as described above and delete the user from PostHog. Finally, you should notify so he can delete customer data from our email marketing systems, and so he can coordinate further data deletion across our systems.

Targeted deletion requests

Occasionally users will mistakenly share sensitive data which should not have been shared via event/person properties. As such they wish to be more targeted in their deletion by removing only certain properties or events instead of an entire project.

> Before taking any deletion action, they should ensure that they are no longer sending the sensitive data to us either by redacting information client-side or setting up a CDP transformation. If they don't do this first they will continue to send us the sensitive data even after deletion is actioned.

Due to the the nature of how our infrastructure works, events and properties cannot be amended once they are stored in Clickhouse. As such, the only way to remove sensitive data is to delete the person profile associated with the events where the sensitive data has been captured. This can be achieved in the app or via the API. As per our deletion docs, the person profile will be removed immediately but the events will take some time (days or even weeks) to be removed.

> If they aren't using person profiles, they won't be able to use this method and as such will need to revert back to deleting the entire project containing the sensitive data.

For customers spending $20K and above a year our Clickhouse team may be able to craft a more targeted event deletion/property amendment query. There are no guarantees here and it is very time consuming hence why we will only explore this for high-paying customers. If you have a customer in this situation and the above methods won't work for them; escalate a support ticket to the Clickhouse team with as much detail as possible on the event and property names where the data is leaked so that they can create a query to process the deletion. To expedite this you should ask the customer for a SQL query which correctly identifies the events or properties to be deleted, or help them in crafting that. Also verify that the numbers returned by this query match what the customer expects to see. Once started this can also take some time (days or weeks) so you should set those expectations with the customer.

> If they need to remove data immediately, the only way to do this is the delete the project. There are no other alternatives.

Handling sales leads

If a support ticket should be handled by one of the sales/onboarding teams, use the `Create a lead` macro in Zendesk to respond to the customer. The macro adds the `sf-lead` tag to the ticket, which will automatically create a new lead in Salesforce. This automation is documented in the Sales area of the handbook.

Community

Support =/= community - we consider them to be separate things.

Tutorials

We want to help teams of all sizes learn how to ask the right product analytics questions to grow their product. To help, we create content in the form of tutorials, blog posts, and videos.

We've also created a bunch of useful templates that cover many of the most popular PostHog use cases.

Support team incident response

`contents/handbook/support/support-incident-response.md`

When things break, we need to make sure users know what's happening and feel supported through it. This page covers how the support team handles incidents - what we do, when we do it, and how we stay aligned with engineering, marketing, and sales.

Raising an incident

Anyone can and should raise an incident if they suspect there is one. This includes support team members. When in doubt, always raise an incident - it's much better to declare something that turns out not to be an incident than to miss a real one.

Declaring an incident doesn't trigger any external notifications. It just creates an incident channel and alerts the right people internally.

If you're seeing multiple tickets about the same issue, or if something seems seriously broken, type `/incident` in any Slack channel to declare one. See the full guide on raising an incident for more details.

Once you've raised the incident, you should raise your hand to watch it from a support perspective, or actively hand it over to someone else on the team.

Your role during an incident

- If you suspect an incident, raise one - type `/incident` in any Slack channel
- When an incident is declared, a workflow posts to #team-support asking for someone to raise their hand
- Whoever raises their hand owns watching that incident from a support perspective:
  - Monitor the incident channel for updates, and ensure the status page has clear messaging
  - Respond to new and existing tickets related to the incident, creating a macro as needed
  - Keep the team updated in #team-support with anything relevant
  - Hand over to the next timezone if the incident runs long
- Don't duplicate comms - coordinate with the Comms Lead and TAMs/CSMs as needed

When an incident is declared

When an incident gets declared, our incident.io workflow automatically posts to #team-support. This post asks for someone from support to raise their hand and take ownership of watching the incident. All members of the support team are responsible for making sure that an incident has a Support Watcher assigned during business hours.

Support team members aren't automatically added to incident channels. You can keep an eye on #incidents for an overview of what's currently open. When you raise your hand in #team-support to watch an incident, join the incident channel using the link in the workflow post.

When you join the incident channel, you'll be automatically assigned the Support Watcher role in incident.io. This makes it clear and visible to both the support team and the incident team who is managing the incident from a support perspective.

If nobody from support joins the incident channel, the incident lead will get a nudge reminding them to assign the Support Watcher role, along with a note that support only watches incidents during business hours.

If you're online and available during your normal working hours, raise your hand on that thread. This is informal - it's just whoever can do it. If nobody responds after a few minutes and you're around, go ahead and volunteer even if you're in the middle of something else.

We don't have on-call support coverage. You're only expected to raise your hand for incidents during your normal working hours. If an incident is declared outside of working hours, support tickets will either need to wait until support working hours resume, or be handled by the @on-call-global person from engineering.

Once you've raised your hand and joined the incident channel, you'll be assigned the Support Watcher role. You own:

- Following the incident channel and keeping up with status updates
- Ensuring the status page has clear, customer-friendly messaging about the impact
- Looking through existing tickets in the queue to see if any were opened because of this incident
- Creating a macro (if appropriate) for responding to customers about the incident
- Checking for new tickets coming into the support queue during the incident
- Passing along relevant updates or highlights to the rest of the support team in #team-support
- Understanding the user impact so the team can respond to tickets accurately

Your job isn't to fix the incident. Your job is to be the bridge between the incident response and the support team, and to make sure users opening tickets get accurate information.

Using the status page

The status page is our source of truth during incidents. The incident lead is typically responsible for keeping it updated, but as the support team member watching the incident, you should make sure the messaging is clear and customer-friendly.

Review the status page messaging when it's updated to ensure:

- The impact is described in terms customers will understand (not just "elevated errors" or technical jargon)

- The affected components are marked correctly
- The messaging isn't ambiguous - users should understand what's broken and how it affects them

If the status page update is too generic or unclear, work with the incident lead to improve it. You can update it yourself using `/incident statuspage` (`/inc sp`) in the incident channel.

Good status page messaging:

> Feature flags are being returned but with 30-60 second delays instead of the usual <1 second response time. All other PostHog features are operating normally.

Unclear status page messaging:

> Elevated errors in the feature flags service.

Always link to the status page in ticket responses. Users should be able to check it themselves for updates rather than having to ask us every hour.

Creating a macro for the incident

If the incident is likely to generate multiple support tickets (most incidents do), create a macro so the whole team can respond consistently. To create the macro:

1. Clone the Incident information macro as your starting template
2. Look at the incident number in incident.io (e.g., INC-123)
3. Add the tag incident/[number] to the macro (e.g., incident/123). This ensures all tickets using this macro are automatically tagged with the incident number for tracking.

The macro should include:

- A brief description of what we currently understand about the incident
- The specific impact to users (what's broken, what's working, what's degraded)
- A link to the incident on our status page

Keep it simple and factual.

If there's a Comms Lead assigned:

- For major or security incidents, share the macro draft with them before using it. We want to make sure we're saying consistent things across all channels. The Comms Lead should review the macro to ensure the messaging aligns with broader customer communications.
- For minor incidents, share the macro with them after you've created it so they're aware of what messaging we're using.

If there's no Comms Lead assigned: Use your best judgement to create a clear, factual macro. If you think the incident warrants coordination with Marketing, mention it in the incident channel - but don't let that block you from responding to customers.

Update the macro if the situation changes significantly. If you do update it, let the Comms Lead know (if there is one). Delete the macro once the incident is resolved.

Let the team know in #team-support when you've created the macro so they know to use it.

Handling incoming tickets during an incident

If you're the person who raised your hand to watch the incident, you're also responsible for keeping an eye on the support queue during the incident. Check through the queue for any existing tickets which might have been raised before the incident was declared, and then continue to monitor for new tickets being raised related to this incident.

Sort your tickets by newest so you can easily spot new tickets coming in. This makes it much easier to catch incident-related issues as they arrive.

Don't send generic "we're working on it" messages. Use the macro if you created one, or link to the status page, explain what we know about the impact, and give them a real timeline if we have one. If we don't have a timeline, say that too.

Example response:

> Hey - yes, we're seeing this too. There's an incident affecting feature flag requests right now. You can follow updates on our status page, but the short version is that flags are returning but with higher latency than normal. The team is working on it and we'll update the status page as we know more. I'll be sure to let you know when it's resolved.

Important: Always attach incident-related tickets to the open incident using the incident.io app in the right-hand sidebar in Zendesk. This helps us track the user impact and keeps everything organized. Anyone on the support team responding to incident-related tickets should do this, not just the person watching the incident.

If you're seeing the same issue across multiple tickets, drop a note in the incident channel. Sometimes support spots patterns before monitoring does. Also share this in #team-support so the rest of the team knows what to expect.

Creating proactive tickets

Sometimes we need to reach out to users proactively during an incident - for example, if a specific org caused the incident or was significantly affected. The engineering team may ask us in #team-support to create tickets for affected customers.

Before creating any proactive tickets, check with the incident lead and coordinate with the Comms Lead to ensure we're not duplicating their communications.

Keeping the team updated

As the person watching the incident, keep the rest of the support team informed in #team-support. Share:

- Major updates about what's happening
- Changes to the user impact
- Patterns you're seeing in tickets
- When the incident is resolved

You don't need to copy every single update from the incident channel. Just share the things that would help someone else on the team respond to a ticket about this incident accurately.

Working with the Comms Lead

For an incident that requires external comms, Marketing will appoint a Comms Lead. They own external communication strategy. We don't.

As the support team member watching the incident, you should coordinate with the Comms Lead:

- Coordinate on macro creation (see 'Creating a macro for the incident' above)
- If you're seeing patterns in support tickets that might inform their messaging
- If you need help with a particularly complex or sensitive customer communication
- When you update the macro - let them know what changed

If you think external comms are required but there isn't a Comms lead assigned, you can request one by asking in #team-marketing or using the @all-marketers tag in Slack.

Coordinating with TAMs and CSMs

Enterprise customers often have dedicated TAMs (Technical Account Managers) or CSMs (Customer Success Managers) from the Sales/CS team. When these customers reach out about an incident - either through their Slack channels or via tickets - we need to coordinate our response.

For minor incidents, we can usually just respond ourselves. Keep it straightforward and use the macro if you created one.

For major incidents, Sales/CS teams may want to handle communication with their customers directly. Check #cs-sales-support to see if they're coordinating a response plan. If you're unsure whether to respond to a particular customer:

- Check #cs-sales-support to see if there's discussion about the incident
- Create a side conversation from the Zendesk ticket into #cs-sales-support and ask if they'd prefer to handle the communication with this customer

- If nobody responds and the customer is waiting, respond yourself - it's better than leaving them hanging

Remember that TAMs and CSMs work in specific timezones. If an enterprise customer reaches out when their TAM/CSM is offline or on holiday, don't leave them waiting. Respond to their question. You can loop in their TAM/CSM as a heads-up, but the customer should get an answer from someone.

Handing over across timezones

If an incident is still ongoing when you're about to log off for the day, hand over to someone who's still working or coming online. Try to hand over to someone who has the most working hours ahead of them - this avoids multiple handovers.

Post in #team-support via the original workflow thread with:

- Current status of the incident (what's broken, what's being done about it)
- Roughly how many support tickets we've seen related to it
- Any key information the next person needs to know
- Anything you told users in tickets that the team should be consistent about

If you're US West Coast based and logging off for the day, write detailed handover notes given that nobody in EU will be online yet. This way they can pick it up smoothly when they start their day.

If you're picking up an incident from someone in a previous timezone, read their notes, scan the incident channel for updates since those notes were written, and jump in. Raise your hand on the original #team-support workflow post if you haven't already.

After an incident resolves

Once the customer-facing impact of the incident is resolved:

- Find all incident-related tickets by filtering for the relevant `incident/xxx` tag in Zendesk (these are automatically added when you attach tickets to the incident via the incident.io app in Zendesk)
- Go back through these tickets and update users that the incident is resolved
- Check if any docs or help content needs updating based on what happened - if you can ship a quick docs fix or FAQ update, do it now while it's fresh
- Delete the incident macro you created

For major incidents, there will be a post-mortem. Read it. If you have feedback from the support side - things we could have done better, information we were missing, communication that didn't work, patterns you saw in tickets - add it to the post-mortem document or share it in the incident channel. Your perspective on the user impact and customer communication is valuable.

Technical support subject matter experts (SMEs)

contents/handbook/support/support-smes.md

Why we have SMEs

As we add more products to PostHog, it becomes increasingly difficult for individual support engineers to effectively work across every product. SMEs help us maintain deep expertise across our products and ensure every ticket gets answered by someone who really knows their stuff.

By allowing SMEs to own groups of PostHog products, we build the knowledge needed to delight users with better and faster answers, and develop close relationships with product teams so we can advocate for fixes and features that actually matter to users.

Product ownership

Product groups

The various PostHog products have been split into the following product groups:

- Analytics (analytics platform, customer analytics, product analytics, web analytics, growth)
- Unclassified (tickets in the 'Support' group)
- Flags (experiments, feature flags, surveys)
- Data (batch exports, data stack, ingestion, workflows)
- Replay (replay)
- Observability + AI & client libraries (error tracking, PostHog AI, LLM analytics, SDK/Implementation, mobile)

A note on these groupings: These product groups are based on current ticket volumes. As products grow or new ones launch, we'll split or reorganize them. This structure will evolve with our needs.

SME ownership

All technical support engineers, regardless of SME ownership, work on:

- **Analytics products** - work for this product group is shared as it represents the highest proportion of our tickets.
- **Unclassified tickets** - where possible, these tickets should be updated with the correct product group.

Beyond that, we have SMEs who own specific product groups. For each product group, we select one person from EU and one from NA to maintain timezone coverage:

Flags

- EU: , NA:

Data

- EU: , NA:

Replay

- EU: , NA:

Observability + AI & client libraries

- EU: , NA:

Analytics

- EU:

What SMEs actually do

Being an SME means you're the go-to person for your product group. This breaks down into three key aspects:

Own the customer perspective

- Maintain oversight of all tickets in your product group
- Spot patterns and common themes
- Understand what bugs are frustrating users most
- Know what features users are asking for

Partner with engineering teams

- Build good relationships with the engineering teams who own your products
- Consider attending their standups occasionally (you don't need to go to every one)
- Keep an eye on their Slack channels to know what's recently shipped and what's being worked on
- Understand what's on their roadmap
- Bring customer context to help with their quarterly planning - what bugs are most prominent, what features users want most

Improve support

- Look to improve the support experience within your product group
- Identify ways we can proactively help customers and prevent tickets (e.g. docs, in-app prompts, automatic checks)
- Look for ways to streamline support operations for your product group (i.e. identify most common investigation steps that could be surfaced onto tickets automatically)

How to work as an SME

Your Zendesk views

SMEs each have a dedicated view in Zendesk that includes:

- Tickets created from Slack channels
- Tickets submitted via the help sidebar
- Community questions

These views contain tickets from your specific product groups (see groupings above) **and** all shared product groups (analytics and unclassified tickets). If there are any unclassified tickets that appear in your view (tickets in the 'Support' group), then where possible please assign these to the correct product. Let know if there are certain types of tickets which regularly appear in the 'Support' group.

Important: These views show tickets assigned to other team members too, giving you full context of your products. Jump in if you know something off the top of your head or see someone stuck.

Your daily workflow

Start your day with your SME views. Build your knowledge. Get really good at your products. Once you're on top of your SME queue, move to the Technical support shared view which has all tickets the technical support team is responsible for.

But here's the key: you're not locked into only your SME products. The goal is expertise, not silos. If you're caught up and the shared queue needs attention, dive in. If you're swamped and someone else can help with your SME queue, ask for it.

Coverage and coordination

You and your SME counterpart in the other timezone should work together to:

- Share knowledge, patterns, and themes you're seeing in tickets
- Coordinate on holiday planning - can you stagger time off to maintain coverage?
- Call out when your SME queue is especially busy and you need help
- Communicate in #team-support about coverage gaps or when you need backup

As we grow, we'll need less manual coordination. For now, always consider coverage and communicate proactively.

Support team overview

contents/handbook/support/support-team.md

The support team exists to help our users succeed with PostHog, and we do that differently than most support teams.

We're not a ticket-routing operation. We genuinely care about making our users' experience exceptional, which we do by being a deeply technical team that takes pride in solving problems ourselves. We write code, ship fixes, update docs, and build internal tooling to deliver that experience. We move fast, stay humble, and believe that great support is about empowering users, not just answering questions.

What makes us great

We communicate clearly and don't hide behind jargon. We're relentlessly curious - pulling at every thread when investigating an issue, and seeing the bigger picture beyond the immediate problem. We're always looking for ways to improve, whether that's our processes, our docs, or our own skills. We're thorough without being slow, thoughtful without overthinking, and we genuinely care about getting things right for our users.

Our values

Take ownership

Own your work from start to finish. Be proactive and self-driven. Don't wait to be told what to do. When you see a problem, jump in and solve it. Be resourceful, curious, and hands-on. If something needs doing, figure it out and make it happen. Taking ownership means being accountable for outcomes, not just tasks.

Delight users

Go beyond solving problems. Create moments that make users' days better. Be genuinely caring, reassuringly human, and empathetic in every interaction. Surprise users with your thoughtfulness and responsiveness. Bring positivity and warmth to technical conversations. When users walk away from an interaction with you, they should feel helped, valued, and hopefully a little bit delighted.

Stay humble

Check your ego at the door. Take feedback as a gift and be open to learning from anyone, regardless of their experience level or role. Share knowledge freely with the team and communicate with transparency and honesty. We get better together by staying curious, admitting what we don't know, and helping each other grow. No one has all the answers, and that's okay.

Ship fixes

Be deeply technical and hands-on. Don't just log bugs or pass tickets along - write the fix yourself. Raise PRs for docs improvements, patch code, and solve problems end-to-end. We're engineers who happen to do support, not support agents who escalate to engineers. If you can fix it, ship it. That's what makes PostHog support special.

What we do

We help users through in-app support (which routes to Zendesk), community questions, and Slack channels for enterprise customers. But we don't stop at answering questions:

- **Ship code:** We write and merge small bug fixes and improvements ourselves.
- **Improve docs:** We contribute fixes, clarify sections, and add missing information.
- **Build internal tools:** We create tools like HogHero for internal efficiency, SDK Doctor for proactive customer help, and automations to streamline our work.
- **Share product feedback:** We surface patterns and pain points we see from users.
- **Answer community questions:** We respond to questions in our community forums.

We provide support Monday through Friday, 9am GMT to 5pm PST. We focus on being consistently excellent during our coverage hours, with clear expectations set for users.

What we don't do

- **Route tickets:** We solve problems ourselves rather than passing them along.
- **Hide behind processes:** We care more about outcomes than following rigid procedures.
- **Work in silos:** We're integrated into product development and actively contribute to company discussions.
- **Accept "that's not my job":** If we can help, we do. If we can't, we figure out who can and make the connection.

Our long-term vision

Support should be a competitive advantage for PostHog. Users choose us partly because they know they'll get exceptional support. They stay partly because they feel valued and helped. They recommend us partly because they've had great experiences with our team.

As PostHog grows, we're scaling thoughtfully. We prioritize keeping the team technical, staying true to our values, and maintaining our user-first culture. We value attitude and aptitude over experience - we need people who can jump into the unknown and figure things out. We want to contribute code and build tools, while keeping quality and user-centricity at the core of everything we do.

The benchmark we're aiming for: other companies should measure themselves against PostHog support - not because we answer tickets quickly, but because we genuinely help users succeed.

Support zero weeks

contents/handbook/support/support-zero.md

Support isn't just about tickets! Well... it's a lot about tickets - but we don't judge the success of support engineers solely by how many tickets they solve. Instead, we like to free up support engineers to spend some time working on other tasks which help users. These tasks can include working on their quarterly goals, building new support features, contributing small PRs for bug fixes, or whatever else they think will help us move faster.

Why are support zero weeks useful?

The goal of zero weeks is to make non-ticket time more efficient and effective, and get more of our quarterly work done as a result.

At times we can really struggle to pull ourselves away from tickets and focus on the bigger picture. Having a block of dedicated non-ticket time allows us to spend time shipping things that will help us become better as a support team, and allow us to better help our customers.

How do support zero weeks work?

Each support team member is given an allocation of *2 support zero weeks in each quarter* (i.e. 10 working days). These are weeks that each team member can book.

Team members are encouraged to consider taking the same zero weeks as someone else working on the same quarterly goal (so it can be done hackathon-style, you can consider using your meetup budget, etc)

Before the quarter starts

- [ ] During quarterly goal planning we scope out goals that we think are achievable in our zero time each quarter.
- [ ] Let know if you have any preference or restrictions on when you can take your zero weeks.
- [ ] will check the PTO calendar, consider time zone constraints, and propose a schedule of zero weeks for the quarter. For each support team member, we will aim to schedule one zero week in the first half of the quarter and the other in the second half of the quarter.
- [ ] The weeks will be shared in the support zero weeks calendar (so we don't forget who is doing what).
- [ ] Each team member should create a meta GitHub issue for their quarterly goal work.

Before your zero week

- [ ] Hand over all of your in-progress tickets (including any in a pending state that you believe are ongoing / going to come back). Do this as a message into #team-support.
- [ ] Set yourself as unavailable in Zendesk using the Out of Office app.
- [ ] Set a status in Slack so it's clear to the rest of the team that you're on a zero week

During your zero week

- [ ] Try your best to avoid the ticket queue (i.e. generally don't pick up new tickets or respond on tickets you were previously working on, except in exceptional circumstances).
- [ ] Do reassign tickets back to the main ticket group if they accidentally come back assigned to you directly for some reason.
- [ ] Do respond to any questions the team ask in #team-support about tickets you were previously working on.
- [ ] Do consider that you may get pulled back onto tickets on a particular day if absolutely necessary (to be avoided as much as possible).
- [ ] Make sure to keep notes and design choices publicly available on your meta GitHub issue.

> ⚠️ Team members who are working on tickets need to be aware of ticket queue and highlight in #team-support if the workload is getting too high

After your zero week

- [ ] Set yourself as available in Zendesk using the Out of Office app.
- [ ] Ask in #team-support if there are any tickets that the team would like to return to you that you were previously handling.
- [ ] Ask in #team-support for any feedback that the team has for you based on any of your tickets they have handled during your zero week.
- [ ] Consider if this is a good stage in your goal to seek feedback from the team (likely via your meta GitHub issue or a different RFC). Please do consider if it's worth the team's time at this stage.
- [ ] Get stuck into tickets!

What does this mean for side quests outside of quarterly goal work?

- Do them!
- The purpose of zero weeks is to give space and focus for quarterly goal work, not restrict what you can do on a day-to-day basis.
- Where you have small things you'd like to do (docs updates, small PRs/bug fixes, setting up example apps, etc), you can absolutely do these alongside your ticket work.
- Please just bear in mind that tickets are generally our highest priority (especially `Sales/CS Top 20` and enterprise customers).
- If there are larger pieces of work that you'd like to do, we can chat about updating your quarterly goals.

Troubleshooting tips

contents/handbook/support/troubleshooting-tips.md

A collection of tips & tricks on helping to troubleshoot customer issues.

General

- Add `distinct_id` as a column to see how the `distinct_id` changes (use SQL expression column). This is useful when troubleshooting identify / person profile related issues.
- To breakdown events by anonymous vs identified, use this SQL snippet: `IF(person.properties.$creator_event_uuid IS NOT NULL, 'Identified', 'Anonymous') AS user_type`
- Debug mode: append `?__posthog_debug=true` to a site that has posthog running, e.g. https://app.mywebsite.com/login?__posthog_debug=true. This can show lots of useful information like logs and config.
- To check a customer's PostHog configuration:
  - In session replay, enable doctor and look for `posthog config` event.
  - Open the customer's site in debug mode (see above)
- To check if a customer is using a reverse proxy, look at their api_host configuration. If it shows `us.i.posthog` or `eu.i.posthog` – then they are not using a reverse proxy.

Feature flags

- Check team activity to see if users have made any changes to the flag. Take note of the timestamp of any changes and see if it explains any discrepancies.

Funnels

Common funnel troubleshooting steps

- Ensure you understand the conversion goal the user is tracking, clarifying this often helps, even if the user knows they're doing the right thing
  - Are they reasoning about it right?
  - Have they chosen the correct events?
  - Are their events sent when they think/expect them to be sent?
  - Are they filtering the correct events for that flow?
- If it's a mix between frontend and backend events, they must ensure identification is done right
- For reports about unexpected drop-offs, look at each event in the funnel separately to understand if they've dropped off on their own (using a Trends insight helps).
- **Attribution type (example ticket)**: Users often report that their experiment funnels show a lot of `false` or `none` values for the feature flag breakdown. This is commonly the cause because they have "First touchpoint" selected as attribution, but they want "Last touchpoint." This is only

relevant when they're using funnel analysis outside of the experiment, since the experiment already only takes into account users who had the expected variant by the end of the funnel.

- **Search for events by the user's IP address** (which you can find in the event properties for the Web SDK). This works if the whole flow happens on the frontend, sometimes you can find that the same IP address has multiple distinct_id, which means that user may have multiple identities which funnels count separately.
- **Create funnels for each interval in their funnel**: Something that would have given us more helpful information sooner would have been creating a funnel for step 1 to step 2 and another funnel for step 2 to step 3. This was what ultimately confirmed there was a user identification problem.
- **Funnels and user paths use different queries (example ticket)**: If a user reports seeing different numbers in funnels and user paths, that's expected, these use different queries in the backend and measure different outcomes.
- **Look for identification splits**: If they're identifying users in different environments (backend vs frontend) different libraries (Web vs Segment), or even with multiple IDs (logged out vs logged in), or even cross-subdomain without proper persistence (cookies) or different implementation configuration, all of these could cause a desync between identities which can break a funnel.

Connecting frontend and backend identities

To connect frontend and backend identities, you only need to use the same `distinct_id` in both frontend and backend events. How you sync these depends on your system but here are some ways:

Recommended: Set the `distinct_id` based on a known user ID: If you have a stable internal user ID, set `posthog.identify('your-user-id')` on the frontend, and use that same ID in backend events. This ensures alignment across both environments

- Alias: identify with id 1 -> alias with id 2 as alias and id 1 as distinct_id
- Wrong: identify with id 1 -> identify with id 2 -> alias

Use a signed token or cookie: Store the `distinct_id` in a cookie or session token shared between frontend and backend, especially if you're using server-side rendering or middleware that handles both sides.

Pass the ID from frontend to backend: When a user logs in or performs a tracked action, capture their `distinct_id` in the frontend (e.g., using `posthog.get_distinct_id()`), then include it in API requests or session headers so your backend can reuse it when sending events. Careful, you're relying on PostHog's distinct_id here, which may not be an expected value.

A potential pitfall is `posthog.reset()`.

Values

contents/handbook/values.md

These are the principles for the behavior we care about.

You're the driver

We hire people that are really great at their jobs, and get out of their way. There are no deadlines, very minimal coordination and you won't have us breathing down your neck.

In return, we ask for extraordinarily high ownership. To succeed you need to be intrinsically motivated.

Great people at PostHog can take very high level direction, and ship quickly to find out as quickly as possible if our plans can survive contact with customers!

Being the driver means getting stuff done *yourself*. We've had non technical people create hardware products, coding in C++, we've got designers that will write Tailwind and React - rather than just create the file in Figma. Our salespeople answer technical questions without engineers as backup - and if they don't know the answer, they educate themselves more deeply for next time. We like people to go full stack instead to reduce the number of dependencies.

Building a company isn't a solo sport. We're Ted Lasso (although I've not watched to the end, I hope they win) not Wolf of Wall Street. We expect you to take high ownership of the *company* and *your team* being successful. This means when you see something wrong, you fix it or give direct feedback - it's not ok to watch your colleagues fail.

Make it public

We default to transparency with everything we work on. That means we make a lot of things public: our code, our handbook, our roadmap, how we pay (or even let go of) people, what our strategy is, and who we have raised money from.

Internally, a culture of transparency looks like managers telling you to raise feedback directly with the person it concerns instead of solving problems for you, it means changing teams around in public Slack channels, it means detailed financial information, live updates on fundraising and board slide access.

Being transparent externally helps us achieve our mission - we write about what we're working on so the world can take advantage of the lessons we're learning, and so they know how to work with us better. Knowing that thousands of people will read our handbook pages forces clearer thinking. And, for free, we can build trust in a way other vendors just choose not to.

> There are a few things that we are internally transparent about, but that should not be shared publicly. Anything related to our company financials is strictly confidential and should not be shared externally, including our current revenue numbers, ACV, burn rate, etc. Anything in a public press release is fine to share!

Do more weird

So much about how we work is different.

Weirdness can just be the absurd lengths we are willing to go to. It can mean redesigning an already world-class website, for the 5th time. It can mean shipping *literally* every product that relates to customer data, with teams of just one to five people competing with $200bn+ companies, successfully.

We aren't weird for the sake of it. We want the company perfectly optimized for our strategy. We have small teams when very few others do, because we are going to build 50+ products. We post billboards of our founders' faces because no one else is brave enough thus it stands out. Even the little things - like having *pricing* on our pricing page!

We've even written a guide on how you can do more weird.

Why not now?

Why not now? means getting things done *proactively*, *today*. You do not need consensus to do things – focus your energy on shipping what's most valuable for our customers and the company, then take ownership of making it happen, not on getting buy in from others. You certainly shouldn't wait until next quarter if your new idea makes more sense to work on than your previous goal.

We have learned the clearest lessons at PostHog by doing things, not from hypothesizing about them. If we're debating doing something, just trying it is the best way to learn. Doing more planning is rarely the right way to figure out if something will work, doing the thing is the answer by default here.

Sometimes this approach might mean you ship something that others don't agree with. You will need to be willing to throw away work sometimes, because the upside – not needing to get lots of approval to do stuff and being able to take more bets – means we all move so much faster that mistakes are a lot less costly.

Why not now? doesn't just mean shipping huge product features. It may mean diving into a small customer support issue quickly to delight them – this is one of the main reasons people recommend us to others.

Optimistic by default

We have a lot of control over our direction, and we've been very well served by shooting for the best case scenario every time we make a decision. You'll hear us say things like "play offense, not defense", "how do we 10x this", "how do we win in 10 years' time". Aiming for the best possible upside and sometimes missing is much better than never trying.

This is especially true when we think of new ideas - any big new thing can sound pretty silly at first, almost by definition. You'll hear PostHog war cries like "we haven't built our defining feature yet, maybe this". It never is, but that's *exactly* the point. What we've already done is less important than what we do next. If we make new ideas painful to share with others, they'll eventually stop coming.

At a simple level, we want to be surrounded by people that are enthusiastic, passionate and happy. PostHog is a group of people working together with a shared goal. A positive, encouraging atmosphere simply means everyone is going to have a lot more fun and will be able to stick around for the full adventure here.

Put more grandiosely, PostHog is wildly ambitious, and with that, a level of optimism is *required*. You cannot change the world without first *believing* you can change the world. People not believing is probably a bigger deal than people not being able to.

Deciding which products we build

`contents/handbook/which-products.md`

Providing all the tools in one is a core part of our strategy.

Shipping them in the right order is key to a fast return on investment from every new product.

How we pick new products

Until products are built and launched, it's hard to predict which ones will do well. Because of this, we want to be working on a mix of new products at any given time. Some we're very sure will do well, others might be more of a bet with a potentially big outcome. This guidance is therefore less prescriptive that it could otherwise be.

Products we know will work well if we ship them:

- Products engineers use at all company stages
  - Think error tracking or feature flags. The persona doesn't change as the company gets bigger.
  - Especially true if it works for a 2 person startup, because that means we get in first
- Products that already have a $1bn competitor in the market (e.g. a company with around $100M in revenue)
- Products that are very easy to integrate for our existing customers.
  - For example, users can enable the product in PostHog without needing to make a code change, or products that built on top of data that people are already collecting in PostHog
- Products that *you* are excited to build.
  - People pursuing their interests get more done, go much further, and execute to a better standard.
- Products that our customers are asking for

Products we're less excited about building:

- Products where the ICP quickly changes to someone outside the product team, especially teams far removed from engineering
  - For example, a CRM. We'd be more excited about building a customer support tool, as support often is a task that involves engineering.

How new products get built

Sometimes the Blitzscale team will decide a new product needs to be built. They'll find someone internally to run it, ideally someone who's been at PostHog for at least 6 months (we tried getting new people to ship new products, but they often struggled to ship quickly).

Other times you might have an idea for a great product we should build. In that case, use the New Product RFC template. You might choose to hack together a prototype of the product to demo and show off, which you should do! Blitzscale only needs to get involved if you want to start working on this product full time. At that point, we are choosing whether to invest a pretty serious amount of money into launching it, so we want to get that right.

For a complete walkthrough of the product lifecycle, see releasing new products and features.

Next products on deck

From our roadmap, here's what we're currently working on:

- Endpoints - `#team-data-modeling`
- Logs - `#project-logs`
- Product autonomy - `#team-array`
- Customer Analytics `#team-customer-analytics`
    - Revenue analytics now included in customer analytics
- Workflows `#team-workflows`

And these are the products we think we'll focus on next:

- 100x the toolbar - likely `#team-array`
- Metrics
- APM
- BI over any database (not just those synced to our data warehouse)
- Support -> PRs
- AI answers and docs

How to pick which feature within an existing product to build

In the early days, you'll be shipping the main few features that your category of product has as standard. In product analytics, this would be something like (1) capturing events, (2) trends, (3) funnels, (4) retention, and (5) person views.

Once this is done, you'll get a stream of feature requests and bug reports from users. You can't go too wrong if you listen to these and, by default, prioritize those that help us get in first, first. For example, with our data warehouse, we picked multi-tenant architecture because we wanted startups to be able to get started for free or very little initial cost - even though a single tenant approach would have given us an MVP faster. Sometimes, if sales are asking, you may choose to prioritize a feature for a big customer earlier, but you should never do this when you wouldn't have shipped it at some stage anyway. However, be cognizant of how often you do this, and whether now is the right time to be shifting your persona focus.

Later on, you can then *innovate* several ways:

- unpeel your product - you start with the software, then offer API access, then offer better API access, then infrastructure (if you are feeling brave) - *by default, start with this* reminder: charge for API access appropriately, speak to Annika for help figuring this out. Doing this increases our luck surface area (it means your users will find new use cases).
- features more specific to our ICP (make it more engineering-y, more customization, more power)
- integrate it with our other products (either feature them *in* the product you just built, or feature your product in *theirs*)

Who we build for

contents/handbook/who-we-build-for.md

We define who we build for as ICP (ie, the company) and the Persona (ie the actual person using the product).

Our current ICP

AKA our ideal customer profile.

We build for the people building products at **high-growth startups**.

Marketing and customer success should primarily focus on this ICP, but should also develop ***high-potential* customers** – customers that are likely to later become high-growth customers (e.g. PostHog itself during YC). We should be in maintenance mode for **hobbyists**, such as engineers building side projects. We want to be the first tool that technical founders add to their product.

	High-growth startup
Description	Startups that have product-market fit and are quickly scaling up with new customers, hiring, and adding more revenue.
Criteria	- 15-500 employees - $100k+/month in revenue *or* very large number of consumer users - Raised from leading investors - Not yet IPO'ed
Why they matter?	- Able to efficiently monetize them - Very quick sales cycle - Act as key opinion leaders for earlier-stage startups/slower moving companies - Strong opinions on what they need - helping us build a better product
Examples	PostHog anytime from their Series B to IPO, Supabase, ElevenLabs

Our current Persona

Persona is the job title or role of the person actually using a product in PostHog. Each team will focus more or less on different members of the product team. This is detailed on their team pages.

As companies get bigger, the type of person that uses a product changes. As an example:

- We initially built product analytics for engineers at startups.

- As those companies get a little bit bigger, they'll hire Product Managers who will mostly use product analytics. PMs have more complicated requirements for what a product analytics tool needs to do.
- Even bigger companies often have specialized "analytics engineers." These people are the most demanding.

Each product should start with a single persona, usually an early person (preferably engineer) at a startup. Teams should make sure to build a really good product with PMF for that single persona. As the product and user-base matures, new personas will emerge as users. You only serve that new persona if you've found PMF and satisfied requirements for the initial persona.

You still need to keep your initial personas happy too, which is tricky, but important as that initial persona is how we get in first.

How do you know if you have PMF and satisfied requirements? Look at churn. If the initial persona is churning from your product, you still have work to do to retain that persona before moving onto others. If instead the product has been handed off to another persona in the org, and *they* are churning, that's an indication that you may need to start supporting the needs of this next persona.

We've not always been successful at building products for personas other than engineers. We're now at a stage where we need to be in order to continue growing.

Churn?

If a team does not currently support a persona, and that persona churns off of using that product, we are okay with that, as long as that doesn't cause the customer to churn off of PostHog entirely. We should try to support those personas to gracefully move off of PostHog. For example: we are okay with sales people churning off to a CRM, and we'll provide exports to export PostHog data to those systems.

Why does PostHog exist? Our mission and strategy

contents/handbook/why-does-posthog-exist.md

Our mission

Equip every developer to build successful products.

Why is that our mission?

Since the beginning, we've believed that engineers should be way more involved in making product decisions than they've been historically. In order to help them do that, we've built a collection of tools for engineers.

Similar tools to the ones we've built have existed for a long time, but they were always built with other users in mind. By building things like product analytics, session replays, feature flags and a data warehouse for engineers first, we give engineers the ability to make product decisions themselves. This massively increases the speed at which engineers can make good decisions.

The other way PostHog helps engineers is by combining *all* the tools they need into one product. This avoids a ton of work integrating and linking up various products, both when integrating and ongoingly.

We try to help engineers from the very beginning, when their product is just being built. We do that by having generous free tiers, and no need to talk to sales to get started.

Our strategy

1. Be the source of truth for all product context

Building a successful product is hard; doing so when you don't understand your customers is even harder. It's wild that no one has already provided a complete record of everything engineers need to ship products. This has happened because the entire industry has focused on integration instead of consolidation.

Traditionally, as companies scale, their data warehouse becomes the source of truth, and non-warehouse native tools (like product analytics) become less relevant as engineers lose trust in the data they collect, simply because they are misused and divorced from the source of truth. Every company winds up with a huge mess of data spaghetti, with their business logic still spread across dozens or hundreds of tools.

We provide developer infrastructure - by providing *every* tool engineers need in one place, we can:

- Enhance the utility of all the tools when used together by engineering teams

- Increase trust in data by eliminating complex data stacks that engineers have to navigate
- Automate everything better than anyone else can, by using AI across this wider context
- Continue to provide all the tools engineering teams need as they grow

2. Provide every tool engineers need to build successful products

We aim to offer every tool engineering teams need to debug, understand, and improve their products. From session replay for debugging to feature flags for safe deployments, we help engineers ship better code faster.

We can then get our AI to work across all of them together, whilst making every individual tool cheaper than the rest of the market - since we provide so many we can charge less. This means engineers get better tools at a fraction of the cost of piecing together solutions from multiple vendors.

3. Get in first

Since developers exist first in a startup, by getting in with them early, we are naturally upstream of every other tool they might have considered using. Although anyone can pick up our products (and lots of mature companies certainly do), this means we can best deliver developer infrastructure to early stage companies, and so should focus there by default.

Once we land a customer, we then let them pull *us* upmarket as they grow. But not before. We don't want to hire a big enterprise sales team and go upmarket before our existing customers are there. This keeps us efficient and able to stay focused on building tools that engineers actually want to use.

4. Automate the iteration process

Because we have all the context on both users and the product, we can automate large chunks of the cycle of shipping -> observing -> iterating.

Secret master plan

- Ship every tool and all the data that engineering teams need to understand their product and users
- Use that to speed up the cycle of shipping -> observing -> iterating
- Eventually, automate the entire cycle

We're a wide company with small teams

contents/handbook/wide-company.md

Part of our strategy is to provide all the tools in one for evaluating feature success.

Speed

This means we need to ship a *lot* of products into one platform. We can see a need for at least 20. That's a lot of engineering work.

After we'd started hiring, we asked ourselves a question – how could we structure the company to optimize for speed above everything else?

I happened to go to an excellent talk by Jeff Lawson, the CEO of Twilio. It made me realize I should be asking, "Who ships more per person, a startup or an enterprise?" Clearly the former. So we structured PostHog like a series of startups.

Small teams

We decided that we should split PostHog into a series of small teams, each working like its own startup, fully owning at least one of our products.

As with any startup, the principles that govern these small teams are:

- Can decide what to build within their own products
- Can ship without outside interference as far as possible
    - No product management by default
    - No product design by default
- Should work directly with its own users (until it has hit product-market fit within PostHog's platform)
- Should be small

Minimal hierarchy

We deliberately keep the number of levels and people managers at PostHog to the absolute minimum we can get away with. This maximizes team member autononomy *and* increases shipping speed, as you don't need to run things past a manager or wait to get something signed off the vast majority of the time.

This means that, if you need something or need to flag an issue, you are strongly encouraged to communicate *directly* with the person or team working on the thing you care about. We want to avoid people going up and down the org chart via managers as much as possible. 90% of the time, this

approach means you'll get what you need faster. 10% of the time, this might cause a tiny bit of confusion if what you are asking for doesn't beautifully align with that team's objectives. We believe that trade-off is ok - we'll figure it out.

We have a tiny exec team - this is what they are responsible for:

- Set the overall direction and strategy for PostHog
- Decide which products to build
- Make key people decisions (e.g. who to hire, pay, disciplinary issues)
- Ensure complicated cross-team initiatives run smoothly (e.g. pricing)

For *everything else*, you and/or your small team should be able to decide this or talk directly to the teams involved. This includes deciding which feature to build next within a particular product. We trust you to bring in the right people as you feel appropriate, relative to the scale of what you're doing.

PostHog is *not* a good place for managers who are territorial and prefer for all communication to go through them for 'efficiency'. Over time, doing this would undermine autonomy and cause our best people to quit!

Titles based on what you do

Companies give out titles to people that primarily show how senior they are.

This means titles, as adopted by the wider world, imply that *seniority* is more important than what people do. We do not believe that seniority should determine how decisions get made - people should own decisions in their area of the business. We trust every employee to fully own their area of the business.

When you are prompted to put your title somewhere like LinkedIn, please just say as clearly as you can what you are focused on. Please do not focus on how senior you are. Feel free to be weird with it.

In other words, instead of your title being "Senior Engineer at PostHog" (which is not a title that exists at PostHog anyway), it's actually "Product Analytics Engineer at PostHog."

Goal setting

When you build a startup from scratch, you are in an existential crisis. One day you might be building a gym, the next day a software product for accountants. The problem changes. At PostHog, we give each small team a product to build. (James and Tim focus on *which* products we should build, as they often need sequencing.)

Once we had product-market fit, and we had reached 15 people or so, we realized we needed to set some kind of goals. We started by using OKRs as they're pretty standard.

However, one of our engineers one day told me, "I realized I needed to change my objective. Then I started rewriting my OKRs into the handbook. I realized I was spending time stressing about the wording of it, which was going to have zero impact on what I knew I had to build." That seemed silly, so instead we make a point of calling them just "goals". We intentionally don't sweat the wording.

Another best practice we choose to ignore is "goals should be output driven". It sounds great in principle, but what is going to happen after a product team, which is nearly every team here, sets an output driven goal like "improve activation by 20%"?

Either the team will decide on some things it should build, or they won't manage to figure out what to build to do this. In either case, if a team knows what it should achieve, it should then figure out which things it needs to ship, and write those things down instead. It's clearer, and clearer is faster.

And if that list turns out not to be helping our metrics? Switch the goal to a new thing.

Building a world-class engineering environment

contents/handbook/world-class-engineering.md

We know we've got to be quick to build all the tools in one. So we better have a world-class engineering environment that lets us build everything. How do we do that?

No product management by default

Engineers decide what to build. If you need help, our product managers (we have four today) will give you coaching.

If an engineer at PostHog believes they should work on X, they can build X. We'd prefer you ship ten things quickly (and make a couple of mistakes) than plan too much. You will tend to gather more information by *doing* rather than *planning*.

There are *some* exceptions - for example, where we need to work on architecture, but we leave it down to you to decide when you should plan more or just get started.

Transparency is fuel for autonomy

In nearly any company, having each engineer decide what to work on would fail. Why? They simply would lack enough context over what the company is aiming for, or what everyone else is up to.

PostHog is exceptionally transparent. You're reading our public handbook after all.

It starts with hiring

Finally, we hire people we think will flourish in an autonomous environment. We often hire people with broader rather than narrower skill sets, who are more flexible. They've often started (and often failed) their own startups. They're low ego and flexible. They're builders at heart who love innovating and working like this.

One of the things we've learned is the very strongest engineers are usually those who want autonomy the most, and so freedom is a great way to attract and retain world-class talent. Now that we're lucky enough to have people like this already here, people see PostHog as a destination company, accelerating further our access to some of the best people in the world at what they do.

A high percentage of our employees are engineers

If we want to ship a lot, we need to figure out how we can have most of capital go into engineering.

We have zero outbound sales, and a hyperefficient go-to-market, largely driven by self-serve. Since we focus on engineers, we have less customer support and set-up handholding than all our competitors.

80% of the company are shipping product.

Deep work

When you're doing engineering, you're in the business of building up large, abstracted models in your head of how the code works. That takes time and requires focus. Doing a ton of meetings is a great way to screw this up.

We therefore have meeting-free days every Tuesday and Thursday. We encourage you to call it out if things are going into your calendar on these days. Since we also are all remote, these usually give you lengths of uninterrupted time to get your work done.

The only exceptions to this rule are for customer success and recruitment, who may need to have external meetings with users or candidates on these days in order to do their jobs.